DRAFT

THE ULTIMATE DEVELOPER GUIDE

C++ MASTERY

Kapsamlı Rehber, Eğitim ve Referans Kitabı

Modern C++, Automotive, Gömülü Sistemler, Çift Dilli Anlatım ve Detaylı Örnekler

- Memory & Pointers
- OOP & Design
- Modern C++
- STL & Templates
- Concurrency
- Under the Hood
- Automotive
- AUTOSAR & SDV
- Network

Bilingual EN/TR | 18 Modules | AUTOSAR & SDV | 55K+ Lines

Murat Mecit KAHRAMANLI

Nisan 2026

C++ Mastery

Örnekler ile Açıklanmış Kapsamlı C++ Eğitim Kılavuzu

18 Modül · 119 Kaynak Dosya · ~55.000 Satır · C++17/C++20/C++23

Modül	Konu	Standart
1	Temeller ve Hafıza	C++17
2	Nesne Yönelimli Programlama	C++17
3	Güvenlik ve Performans	C++17/C++23
4	Şablonlar ve STL	C++17
5	Sertifikasyon Hazırlığı	C++17
6	Endüstri Standartları	C++17
7	OS Derinlikleri	C++17
8	Üst Düzey Kurallar	C++17
9	İleri Mekanikler	C++20
10	Tasarım Kalıpları	C++20
11	Gömülü Sistemler	C++20
12	Eşyordamlar	C++20
13	Ağ Programlama	C++20
14	Otomotiv Ağları	C++20
15	Donanım Arayüzleri	C++20
16	Otomotiv Yazılım Mimarisi	C++20
17	Yazılım Tanımlı Araç	C++20
18	Sürücü Geliştirme	C++20

Prepared by **Murat Mecit KAHRAMANLI**

Nisan 2026

Telif Hakkı ve Yasal Bilgiler

Telif Hakkı

© 2026 Murat Mecit KAHRAMANLI. Tüm hakları saklıdır.

Lisans

Bu kitaptaki metin içeriği, açıklamalar ve düzen **Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)** lisansı altında lisanslanmıştır. Kaynak kod örnekleri ise **Apache License 2.0** altında sunulmaktadır.

Bu Ne Anlama Gelir?

Kitap içeriği (CC BY-NC-SA 4.0): Atıf vererek paylaşabilir, uyarlayabilirsiniz; ancak ticari amaçla kullanamazsınız ve türev çalışmalarınızı aynı lisansla paylaşmalısınız.

Kaynak kod örnekleri (Apache 2.0): Kopyalayabilir, değiştirebilir ve kendi projelerinizde (ticari dahil) serbestçe kullanabilirsiniz; ancak orijinal telif hakkı bildirimini korumalı, yapılan değişiklikleri belirtmeli ve lisansın bir kopyasını eklemelisiniz.

Sorumluluk Reddi

Bu kitap eğitim amaçlı hazırlanmıştır. Yazar, içerikteki bilgilerin doğruluğu veya eksiksizliği konusunda garanti vermez. Kod örnekleri öğretim amacıyla basitleştirilmiş olup, üretim ortamında doğrudan kullanılması önerilmez. Yazar, bu kitabın kullanımından doğabilecek herhangi bir zarardan sorumlu tutulamaz.

Atıf ve Kaynaklar

C++ standart referansları için cppreference.com, ISO/IEC 14882 ve C++ Core Guidelines kaynak olarak kullanılmıştır. Tüm ticari markalar ilgili sahiplerine aittir.

İlk Baskı: Nisan 2026

Sürüm: 1.0 (Draft v2)

GitHub: github.com/nimbustan/cpp_mastery

Web: nimbustan.github.io/cpp_mastery

İletişim: nimbustan@github

İçindekiler

Modül 1: Basics — Temeller ve Hafıza (C++17) 7

→ 01. Variables And Memory 7
→ 02. Pointers And References 10
→ 03. Functions And Overloading 12
→ 04. Operators And Expressions 16
→ 05. Control Flow Deepdive 20
→ 06. Strings And Arrays 25
→ 07. Enums Type Conversions 28
→ 08. File Io And Streams 34
→ 09. Scope Namespace Recursion 40
→ 10. Filesystem And Binary Io 46

Modül 2: OOP — Nesne Yönelimli Programlama (C++17) 53

→ 01. Classes Objects 53
→ 02. Inheritance Polymorphism 64
→ 03. Rule Of Three Five 70
→ 04. Multiple Inheritance 76
→ 05. Encapsulation Abstraction 78
→ 06. Static And Friends 81
→ 07. Nested And Local Classes 87
→ 08. Rtti Typeid 93

Modül 3: Modern C++ — Güvenlik ve Performans (C++17/C++23) 99

→ 01. Smart Pointers 99
→ 02. Lambdas 104
→ 03. Move Semantics Rvalue 113
→ 04. Modern Types 119
→ 05. Structured Bindings Optional 122
→ 06. Cpp23 Features 129

Modül 4: Templates & STL — Şablonlar ve STL (C++17) 134

→ 01. Templates Stl 134
→ 02. Functors Callbacks 140
→ 03. Stl Containers Deepdive 142
→ 04. Iterators And Adaptors 148
→ 05. Algorithms Masterclass 155
→ 06. Associative Containers 163
→ 07. String Operations 171
→ 08. Ranges And Views 177
→ 09. Json And Serialization 185

Modül 5: Certification Prep — Sertifikasyon Hazırlığı (C++17) 199

→ 01. Undefined Behavior 199
→ 02. Object Slicing Virtual 201
→ 03. Type Casting 203
→ 04. Exceptions Handling 206
→ 05. Const Correctness 209
→ 06. Interview Gotchas 215
→ 07. Error Handling Advanced 221

■ **Modül 6: Industry Standards — Endüstri Standartları (C++17)** 229
→ 01. Concurrency Threads 229
→ 02. Cmake Basics 234
→ 03. Unit Testing Gtest 235
→ 04. Valgrind Profiling 241
→ 05. Async Futures 246
→ 06. Debugging Techniques 252
→ 07. Git For Cpp Devs 260
→ 08. Cmake Deepdive 266
→ 09. Mutex Lock Deepdive 273
→ 10. Advanced Logging 281
→ 11. Gtest Advanced 289
■ **Modül 7: Under the Hood — OS Derinlikleri (C++17)** 299
→ 01. Vtable Memory Layout 299
→ 02. Sigsegv And Raii 301
→ 03. Fork Vs Thread Pool 303
→ 04. Memory Model And Cache 306
→ 05. Compilation Pipeline 312
→ 06. Abi And Name Mangling 317
→ 07. Inline Assembly 322
→ 08. Simd Intrinsics 329
→ 09. Cpp20 Modules 337
→ 10. Linking Deepdive 348
→ 11. Low Level Optimization 357
■ **Modül 8: C++ Core Guidelines — Üst Düzey Kurallar (C++17)** 363
→ 01. Philosophy And Interfaces 363
→ 02. Resource Management Rule Of Zero 366
→ 03. Error Handling 369
→ 04. Concurrency And Performance 372
→ 05. Smart Pointer Guidelines 374
→ 06. Class Design Guidelines 380
→ 07. Api Design 385
→ 08. Maintainable Cpp 391
→ 09. Clean Code Reference 398
■ **Modül 9: Advanced Mechanics — İleri Mekanikler (C++20)** 403
→ 01. Bit Manipulation And Alignment 403
→ 02. Operator Overloading 405
→ 03. Advanced Pointers 410
→ 04. Constant Evaluation 412
→ 05. Type Traits And Concepts 414
→ 06. Sfinae And Tag Dispatch 416
→ 07. Crtp Pattern 422
→ 08. Template Metaprogramming 428

■ **Modül 10: Design Patterns — Tasarım Kalıpları** (C++20) 436
→ 01. Solid Principles 436
→ 02. Creational Patterns 441
→ 03. Behavioral Patterns 447
→ 04. Structural Patterns 453
→ 05. Testable Design 459
→ 06. Pattern Decision Guide 464
→ 07. Embedded Automotive Patterns 474
■ **Modül 11: Embedded & Real-Time — Gömülü Sistemler** (C++20) 489
→ 01. Volatile Hardware 489
→ 02. Bitfields Protocols 493
→ 03. Embedded Constraints 497
→ 04. Rtos Concurrency 503
→ 05. Advanced Parallelism 509
■ **Modül 12: Coroutines — Eşyordamlar** (C++20) 515
→ 01. Coroutine Fundamentals 515
→ 02. Async Coroutine Patterns 527
■ **Modül 13: Networking — Ağ Programlama** (C++20) 542
→ 01. Socket Fundamentals 542
→ 02. Nonblocking Io Epoll 552
■ **Modül 14: Automotive Networks — Otomotiv Ağları** (C++20) 570
→ 01. Can Fd And Xl 570
→ 02. Lin Bus 585
→ 03. Flexray 597
→ 04. Automotive Ethernet 608
→ 05. Inter Mcu Communication 621
→ 06. Uds Diagnostics 633
■ **Modül 15: HW Interfaces — Donanım Arayüzleri** (C++20) 647
→ 01. Gpio And Registers 647
→ 02. Adc Dac 658
→ 03. Serial Protocols Deep 667
→ 04. Dma And Interrupts 681
→ 05. Memory Types 693
■ **Modül 16: AUTOSAR — Otomotiv Yazılım Mimarisi** (C++20) 703
→ 01. Classic Platform 703
→ 02. Adaptive Platform 717
→ 03. Functional Safety 730
→ 04. Automotive Cybersecurity 741
→ 05. Misra Cert Coding Standard 753
■ **Modül 17: SDV — Yazılım Tanımlı Araç** (C++20) 764
→ 01. Sdv Architecture 764
→ 02. Ota Updates 774
→ 03. Middleware Someip Dds 786
■ **Modül 18: Driver Dev — Sürücü Geliştirme** (C++20) 798
→ 01. Driver Hal Bsp 798
→ 02. Zero Cost Abstraction 809
→ 03. Peripheral Driver 818
→ 04. Mini Ecu Project 831
■ **Ek: Derleme ve Referans Kılavuzu** (Derleme · Araçlar · Referanslar) 845

Giriş: main.cpp

Konu: C++ Mastery — Automotive & Embedded C++ Curriculum / Otomotiv ve Gömülü Sistem Odaklı

18 modules, 118 source files covering C++17/C++20/C++23. Bilingual (EN/TR) annotations with cppreference.com depth notes throughout. Designed for engineers targeting the Automotive & Embedded Systems domain. Over 55,000 lines of annotated, production-grade C++ examples.

18 modül, 118 kaynak dosya, C++17/C++20/C++23 kapsamında. Otomotiv ve Gömülü Sistem alanını hedefleyen mühendisler için iki dilli (EN/TR) açıklamalar ve cppreference.com derinlik notları ile donatılmıştır. 55.000+ satırlık açıklamalı, üretim kalitesinde C++ örnekleri içerir.

Kaynak Kod / Source Code

```cpp
#include <iostream>

int main() {
  std::cout << "\n";
  std::cout << "  ╔═══════════════════════════════════════════╗\n";
  std::cout << "  ║            C + +   M A S T E R Y          ╕\n";
  std::cout << "  ║   Automotive & Embedded Curriculum (2026)  ╕\n";
  std::cout << "  ╚═══════════════════════════════════════════╝\n";
  std::cout << "\n";
  std::cout << "  Modules: 18 | Files: 118 | C++17 / C++20 / C++23\n";
  std::cout << "  Domain : Automotive & Embedded Systems\n";
  std::cout << "  Lines  : 55,000+\n\n";

  std::cout << "  [MODULE MAP]\n";
  std::cout << "  ──────────────────────────────────────────\n";
  std::cout << "   1. Basics            — Temeller ve Hafiza\n";
  std::cout << "   2. OOP               — Nesne Yonelimli\n";
  std::cout << "   3. Modern C++        — Guvenlik & Performans\n";
  std::cout << "   4. Advanced STL      — Ileri Kutuphane\n";
  std::cout << "   5. Certification     — Sertifikasyon\n";
  std::cout << "   6. Industry          — Endustri Uretimi\n";
  std::cout << "   7. Under the Hood    — OS Derinlikleri\n";
  std::cout << "   8. Core Guidelines   — Ust Duzey Kurallar\n";
  std::cout << "   9. Advanced Mech.    — Ileri Mekanikler\n";
  std::cout << "  10. Design Patterns   — Tasarim Kaliplari\n";
  std::cout << "  11. Embedded & RTOS   — Gomulu Sistemler\n";
  std::cout << "  12. Coroutines        — Esyordamlar\n";
  std::cout << "  13. Networking        — Ag Programlama\n";
  std::cout << "  14. Auto Networks     — Otomotiv Aglari\n";
  std::cout << "  15. HW Interfaces     — Donanim Arayuzleri\n";
  std::cout << "  16. AUTOSAR           — Otomotiv Mimarisi\n";
  std::cout << "  17. SDV               — Yazilim Tanimli Arac\n";
  std::cout << "  18. Driver Dev        — Surucu Gelistirme\n";
  std::cout << "  ──────────────────────────────────────────\n";
  std::cout << "\n";
  std::cout << "  README.md  -> Full Table of Contents\n";
  std::cout << "  Happy Coding! / Iyi Calismalar!\n\n";
  return 0;
}
```

Modül 1: Basics — Temeller ve Hafıza

Temeller ve Hafıza

10 kaynak dosya · Standart: C++17

C++17

► 01. Variables And Memory

module_01_basics/01_variables_and_memory.cpp

Konu: Variables, Data Types, and Memory Model — Değişkenler, Veri Tipleri ve Bellek Modeli

Teori / Theory

THEORY: Primitive Data Types & Memory

Every variable in C++ is a named region of memory. When you write `int age = 30;`, the compiler reserves 4 bytes on the Stack, names that region "age", and writes the binary representation of 30 into it. The Stack is a LIFO (Last-In, First-Out) memory region that grows/shrinks automatically as functions are called/returned. Local variables live here — they are born when their scope `{` begins, and die when `}` ends. The Heap is a separate region used for dynamic allocation (`new`/`delete`) where objects survive beyond function scope. Understanding WHERE a variable lives (Stack vs Heap) is the foundation of all C++ memory management.
C++ defines several primitive data types, each consuming a specific amount of memory. The `sizeof()` operator reveals the exact byte count at compile-time. Choosing the correct type directly affects memory usage, performance, and correctness — a `float` where a `double` is needed causes silent precision loss; an `int` where a `long long` is needed causes overflow.

TEORİ: İlkel Veri Tipleri ve Bellek

C++'ta her değişken, bellekte isimlendirilmiş bir bölgedir. `int age = 30;` yazdığınızda, derleyici Stack'te (Yığın) 4 baytlık bir alan ayırır, o bölgeye "age" adını verir ve 30'un ikili (binary) temsilini oraya yazar. Stack, fonksiyonlar çağrıldıkça/döndükçe otomatik olarak büyüyen/küçülen bir LIFO (Son Giren İlk Çıkar) bellek bölgesidir. Yerel değişkenler burada yaşar — kapsamları `{` ile başladığında doğar, `}` ile bittiğinde ölür. Heap (Öbek) ise `new`/`delete` ile dinamik tahsis yapılan ayrı bir bölgedir; buradaki nesneler fonksiyon ötesinde yaşayabilir. Bir değişkenin NEREDE yaşadığını (Stack vs Heap) anlamak, tüm C++ bellek yönetiminin temelidir.
C++ birden fazla ilkel veri tipi tanımlar ve her biri bellekte belirli miktarda yer kaplar. `sizeof()` operatörü derleme zamanında tam bayt sayısını gösterir. Doğru tipi seçmek, bellek kullanımını, performansı ve doğruluğu doğrudan etkiler — `double` gereken yerde `float` kullanmak sessiz hassasiyet kaybına, `long long` gereken yerde `int` kullanmak taşmaya (overflow) neden olur.

THEORY: Stack vs Heap — Where Do Variables Live?

The Stack is fast (allocation is just moving a pointer), automatically managed, and limited in size (typically 1-8 MB). The Heap is slower (OS must find free blocks), manually managed (you must `delete` what you `new`), but virtually unlimited. Rule of thumb:
1. Use Stack for small, short-lived variables (int, float, small arrays).
2. Use Heap for large data or data that must outlive the current scope.
3. In Modern C++, prefer smart pointers (unique_ptr, shared_ptr) over raw `new`/`delete`.

TEORİ: Stack vs Heap — Değişkenler Nerede Yaşar?

Stack hızlıdır (tahsis sadece bir pointer kaydırmaktır), otomatik yönetilir ve boyutu sınırlıdır (genellikle 1-8 MB). Heap daha yavaştır (işletim sistemi boş blok aramalıdır), manuel yönetilir (`new` ile aldığınızı `delete` ile geri vermelisiniz) ama neredeyse sınırsız büyüklüktedir. Temel kural:
1. Küçük ve kısa ömürlü değişkenler için Stack kullanın (int, float, küçük diziler).
2. Büyük veriler veya mevcut kapsamın ötesinde yaşaması gereken veriler için Heap kullanın.
3. Modern C++'ta ham `new`/`delete` yerine akıllı işaretçileri (unique_ptr, shared_ptr) tercih edin.

CPPREF Depth: Undefined Behavior (UB) & Strict Aliasing

cppreference.com explicitly warns: "Reading an uninitialized block of memory is Undefined Behavior (UB)." Also, casting a `float*` to an `int*` violates the "Strict Aliasing Rule" in C++, allowing the optimizer to silently break your firmware.

CPPREF Derinlik: Tanımsız Davranış (UB) ve Sıkı Örtüşme

cppreference uyarıyor: İlk değer (0 veya başka bir sayı) atanmamış bir değişkeni ekrana basmak "Tanımsız Davranış (UB)" yaratır, firmware rastgele çöker. Ayrıca bir `float` pointer'ını zorla `int` pointer'ına çevirmek C++'ın "Strict Aliasing (Sıkı Örtüşme)" kuralını çiğner, derleyici optimizasyon yaparken kodu bozar!

Kaynak Kod / Source Code

```cpp
// Fixed width integer types (Sabit genişlikli tam sayı tipleri için)
#include <cstdint>
#include <iostream>

int main() {
  std::cout << "--- C++ Primitive Data Types & Memory ---" << std::endl;
  std::cout << "--- C++ İlkel Veri Tipleri ve Bellek ---\n" << std::endl;

  // 1. Integer Types (Tam Sayı Tipleri)
  // ==========================================================================
  // EN: The C++ standard does NOT guarantee the size of `int` — it only guarantees minimum
  // ranges. On most 64-bit systems `int` is 4 bytes, but on some embedded platforms it could
  // be 2 bytes. For portable, safety-critical code (MISRA, AUTOSAR), ALWAYS use fixed-width
  // types from <cstdint>: int8_t, int16_t, int32_t, int64_t.
  //
  // TR: C++ standardı, `int`'in boyutunu GARANTİ ETMEZ — sadece minimum aralıkları garanti
  // eder. Çoğu 64-bit sistemde `int` 4 bayt olsa da, bazı gömülü platformlarda 2 bayt
  // olabilir. Taşınabilir ve güvenlik kritik kodlarda (MISRA, AUTOSAR) DAİMA <cstdint>'ten
  // sabit genişlikli tipler kullanın: int8_t, int16_t, int32_t, int64_t.

  int age = 30; // 4 bytes on most systems (Çoğu sistemde 4 bayt, ama garanti değil!)
  std::cout << "int takes: " << sizeof(int) << " bytes (bayt). Example: " << age << std::endl;

  // [CPPREF DEPTH: Uninitialized Variables / CPPREF DERİNLİK: Başlatılmamış Değişkenler]
  // EN: DANGER (UB)! Reading an uninitialized local variable is one of the most common bugs in
  // C++. The variable holds whatever garbage data was left at that RAM address from a previous
  // function call. The program may work in debug mode but crash in release mode because the
  // optimizer assumes no UB and makes aggressive optimizations based on that assumption.
  //
  // TR: TEHLİKE (UB)! Başlatılmamış yerel değişken okumak, C++'taki en yaygın hatalardan
  // biridir. Değişken, o RAM adresinde önceki fonksiyon çağrısından kalan çöp verileri tutar.
  // Program debug modunda çalışabilir ama release modunda çökebilir çünkü optimize edici UB
  // olmadığını varsayarak agresif optimizasyonlar yapar.
  int uninitializedAge;
  // EN: Suppress warning. Reading it is UB!
  // TR: Uyarıyı bastırır. Okumak tanımsız davranıştır (UB)!
  (void)uninitializedAge;
```

```cpp
    // EN: Doing this causes Undefined Behavior.
    // TR: Bunu yapmak Tanımsız Davranış'a (UB) neden olur. std::cout << uninitializedAge;

    // Fixed width (Sabit genişlikli)
    int16_t smallNumber = 32767; // Guaranteed 2 bytes (Garantili 2 bayt, 16 bits)
    // Guaranteed 4 bytes (Garantili 4 bayt, 32 bits)
    int32_t standardNumber = 2147483647;

    std::cout << "int16_t takes: " << sizeof(int16_t) << " bytes. Value: " << smallNumber <<
        std::endl;
    std::cout << "int32_t takes: " << sizeof(int32_t) << " bytes. Value: " << standardNumber << "\n"
        << std::endl;

    // 2. Floating Point Types (Ondalıklı Sayı Tipleri)
    // ==========================================================================================
    // EN: Floating-point numbers follow the IEEE 754 standard. `float` uses 32 bits (23-bit
    // mantissa = ~7 decimal digits of precision). `double` uses 64 bits (52-bit mantissa = ~15
    // decimal digits). NEVER compare floats with `==` because of rounding errors! Use an epsilon:
    // `std::abs(a - b) < 1e-9`. In financial software, avoid floats entirely — use integer cents.
    //
    // TR: Kayan noktalı sayılar IEEE 754 standardını takip eder. `float` 32 bit kullanır (23-bit
    // mantis = ~7 ondalık basamak hassasiyet). `double` 64 bit kullanır (52-bit mantis = ~15
    // ondalık basamak). Yuvarlama hataları nedeniyle float'ları ASLA `==` ile karşılaştırmayın!
    // Epsilon kullanın: `std::abs(a - b) < 1e-9`. Finansal yazılımda float'dan tamamen kaçının
    // — tam sayı olarak kuruş/cent birimiyle çalışın.

    float piFloat = 3.14159f;         // 'f' suffix = float literal (f eki float literal belirtir)
    double piDouble = 3.14159265359;  // Default: double (Varsayılan: double, daha yüksek hassasiyet)

    std::cout << "float takes: " << sizeof(float) << " bytes. Value: " << piFloat << std::endl;
    std::cout << "double takes: " << sizeof(double) << " bytes. Value: " << piDouble << "\n" <<
        std::endl;

    // 3. Characters and Booleans (Karakterler ve Mantıksal Tipler)
    // ==========================================================================================
    // EN: `char` is actually a 1-byte INTEGER (0-255 or -128 to 127). It stores an ASCII code,
    // not a "letter". 'A' == 65 in memory. This is why you can do math with chars:
    // `char lower = 'A' + 32;` gives 'a'. `bool` is also an integer: `true` == 1, `false` == 0.
    // In conditions, ANY non-zero value is truthy: `if (42)` is true, `if (0)` is false.
    //
    // TR: `char` aslında 1 baytlık bir TAM SAYIDIR (0-255 veya -128 ile 127). Bir "harf" değil,
    // ASCII kodu saklar. Bellekte 'A' == 65'tir. Bu yüzden char ile matematik yapabilirsiniz:
    // `char lower = 'A' + 32;` sonucu 'a' verir. `bool` da bir tam sayıdır: `true` == 1,
    // `false` == 0. Koşullarda sıfır olmayan HER değer doğrudur: `if (42)` true, `if (0)` false.

    char grade = 'A';      // Stores ASCII 65, not the letter (ASCII 65 saklar, harf değil)
    bool isCppFun = true;  // Stored as integer 1 (Bellekte tam sayı 1 olarak saklanır)

    std::cout << "char takes: " << sizeof(char) << " byte. Value: " << grade << std::endl;
    std::cout << "bool takes: " << sizeof(bool) << " byte. Value: " << isCppFun << "\n" << std::endl;

    // 4. Constants (Sabitler)
    // ==========================================================================================
    // EN: `const` makes a variable read-only AFTER initialization — the value is set at runtime
    // and cannot be changed afterward. `constexpr` is stronger: the value MUST be computable at
    // COMPILE-TIME. The compiler embeds it directly into the binary as a literal constant — no
    // memory is allocated, no runtime cost. Prefer `constexpr` over `#define` macros: it is
    // type-safe, scoped, and debuggable. In Modern C++ (C++17+), `constexpr` can even be used
    // with `if constexpr` for compile-time branching.
    //
    // TR: `const` bir değişkeni başlatıldıktan SONRA salt okunur yapar — değer çalışma zamanında
    // belirlenir ve sonra değiştirilemez. `constexpr` daha güçlüdür: değer DERLEME ZAMANINDA
    // hesaplanabilir OLMALIDIR. Derleyici değeri doğrudan binary'ye sabit olarak gömer — bellek
    // ayrılmaz, çalışma zamanı maliyeti yoktur. `#define` makroları yerine `constexpr` tercih
    // edin: tip güvenlidir, kapsamı vardır ve debug edilebilir. Modern C++'ta (C++17+)
    // `constexpr` ile `if constexpr` kullanarak derleme zamanlı dallanma bile yapılabilir.

    const int MAX_USERS = 100;  // Runtime constant (Çalışma zamanı sabiti)

```

```
    // EN: constexpr came with C++11. The compiler evaluates the expression at compile-time and
    // embeds the result directly. No CPU cycles are wasted at runtime calculating this.
    // TR: constexpr C++11 ile geldi. Derleyici ifadeyi derleme zamanında hesaplar ve sonucu
    // doğrudan gömer. Çalışma zamanında bu hesaplama için CPU döngüsü harcanmaz.
    constexpr int SECONDS_IN_MIN = 60;
    constexpr int MINS_IN_HOUR = 60;
    // Compiler hardcodes this as 3600!
    constexpr int SECONDS_IN_HOUR = SECONDS_IN_MIN * MINS_IN_HOUR;
                                        // (Derleyici bunu 3600 olarak yazar!)

    std::cout << "Constants:" << std::endl;
    std::cout << "MAX_USERS: " << MAX_USERS << std::endl;
    std::cout << "SECONDS_IN_HOUR (constexpr): " << SECONDS_IN_HOUR << "\n" << std::endl;

    return 0;
}
```

► 02. Pointers And References

module_01_basics/02_pointers_and_references.cpp

Konu: Basics: Pointers, References, and Memory Addresses — Temeller: İşaretçiler, Referanslar

Teori / Theory

THEORY: Why Do We Need Pointers (*) and References (&)?

When you pass a variable to a function by default in C++, it makes a FULL COPY of that variable (Pass by Value). If you pass a 1GB Image file to a function, the computer freezes while duplicating 1GB in RAM! Pointers (*) and References (&) solve this by passing the "Memory Address". You just give the function the GPS coordinates; it goes there and modifies the original.

TEORİ: Neden İşaretçilere ve Referanslara İhtiyacımız Var?

C++'ta bir değişkene parametre olarak müdahale ettiğinizde varsayılan olarak o değişkenin "Birebir Kopyası" çıkarılır (Değerle Geçiş - Pass by Value). Eğer 1 GB'lık yüksek çözünürlüklü bir resmi bir "Filtre" fonksiyonuna yollarsanız, sistem o 1 GB'ı RAM'de tamamen kopyalar ve bilgisayar donar! Çözüm: İşaretçiler (*) ve Referanslar (&). Bu ikili, nesneyi kopyalamak yerine RAM'deki "GPS Koordinatını (Adresini)" fonksiyona yollar.

THEORY: Pointer (*) vs Reference (&)

1. POINTER (*): Inherited from C. Can be NULL/nullptr. Flexible, can be REASSIGNED to point to another object during runtime. Can be dangerous.
2. REFERENCE (&): Introduced in C++. CAN NEVER be NULL (Must be initialized when declared). Safer and cleaner syntax. Once initialized, it gets LOCKED and cannot point to anything else.

TEORİ: İşaretçi vs Referans

1. POINTER (*): C dilinden kalmadır. NULL/nullptr olabilir. Esnektir, çalışırken başka bir nesneyi gösterecek şekilde DEĞİŞTİRİLEBİLİR. Tehlikeli olabilir.
2. REFERENCE (&): C++ ile gelmiştir. ASLA NULL olamaz (Tanımlandığı an bir hedefi olmalıdır). Daha güvenli ve temiz bir sözdizimi vardır. Oluşturulduktan sonra BAŞKA bir şeyi gösteremez, tamamen kilitlenir.

CPPREF Depth: Dangling References & Pointer Decay

Returning a reference or pointer to a LOCAL variable from a function is a catastrophic Undefined Behavior (Dangling Pointer). Because local variables die on the Stack when the function ends. Also, arrays naturally "decay" into a pointer to their first element.

CPPREF Derinlik: Bağlantısız Referanslar ve İşaretçi Azalması

Bir fonksiyondan "Yerel (Local)" bir değişkenin Referansını veya Pointer'ını `return` ile geri döndürmek felakettir (Dangling Pointer - Boşlukta Sallanan). Çünkü fonksiyon bitince (}) yerel obje Stack'ten (Yığın) silinir. Siz silinmiş bir RAM arazisini göstermiş olursunuz. (SIGSEGV - Hack!)

Kaynak Kod / Source Code

```cpp
#include <iostream>

// 1. PASS BY VALUE (KOPYA YÖNTEMİ) - Bad practice for large data / Büyük veriler için kötü
// pratik
void swapByValue(int a, int b) {
  int temp = a;
  a = b;
  b = temp;
  // EN: The original values in main() WILL NOT CHANGE because 'a' and 'b' are just copies.
  // TR: Main fonksiyonundaki asli değerler DEĞİŞMEYECEKTİR çünkü a ve b sadece 'kopya'lardı.
}

// 2. PASS BY POINTER (İŞARETÇİ YÖNTEMİ) - C style (Tehlikeli ve kalabalık sözdizimi)
void swapByPointer(int *ptrA, int *ptrB) {
  if (ptrA == nullptr || ptrB == nullptr)
    return; // Safety check is mandatory! / Güvenlik kontrolü şarttır!

  // * operator (Dereference): Access the VALUE at the address! / Adresin İÇİNDEKİ değere ulaş!
  int temp = *ptrA;
  *ptrA = *ptrB;
  *ptrB = temp;
}

// 3. PASS BY REFERENCE (REFERANS YÖNTEMİ) - The heart of Modern C++! / Modern C++'ın kalbi!
void swapByReference(int &refA, int &refB) {
  // EN: No null check needed. Cleaner syntax.
  // TR: Null kontrolüne gerek yoktur. Daha temizdir.
  int temp = refA;
  refA = refB;
  refB = temp;
}

// [CPPREF DEPTH: UB (Undefined Behavior) Example / CPPREF DERİNLİK: UB (Tanımsız Davranış) Örneği]
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wreturn-local-addr"
int *generateDanglingPointer_DO_NOT_DO_THIS() {
  int localBomb = 100;
  // EN: WARNING! Returning address of stack memory!
  // TR: Asla yapma! Stack'teki gecici degiskenin adresi dondurulmez.
  return &localBomb; // NOLINT — intentional UB demo
}
#pragma GCC diagnostic pop

int main() {
  std::cout << "=== MODULE 1: POINTERS & REFERENCES ===\n" << std::endl;

  int num1 = 10;
  int num2 = 99;

  std::cout << "[RAM COORDINATES / ADRESLER]" << std::endl;
  std::cout << "Address of num1 (&): " << &num1 << " | Value (Değer): " << num1 << std::endl;

```

```
    // 1. Test: COPY (Çalışmayacak / Won't work!)
    swapByValue(num1, num2);
    std::cout << "\n[TEST] After swapByValue (Copy): num1 = " << num1 << ", num2 = " << num2 <<
        std::endl;

    // 2. Test: POINTER (Çalışacak ama Çirkin / Will work but ugly!) Passing addresses instead of
    // values / "Değerleri değil, adresleri gönderiyorum"
    swapByPointer(&num1, &num2);
    std::cout << "[TEST] After swapByPointer (Pointer): num1 = " << num1 << ", num2 = " << num2 <<
        std::endl;

    // 3. Test: REFERENCE (Geri Çeviriyoruz, Zarif C++ Yolu / Reverting using elegant C++ way)
    // Syntactically passed like values, but actually passed by
    swapByReference( num1, num2);
                      // reference! / Tıpkı değer yollar gibi gönderiyoruz!
    std::cout << "[TEST] After swapByReference (Reference): num1 = " << num1 << ", num2 = " << num2
        << std::endl;

    // [Interview Question / Mülakat Sorusu]: Const Reference
    // EN: "How do I pass a massive object without COPYING it, but also ensuring the function
    // CANNOT MODIFY it?"
    // TR: "Eğer fonksiyona devasa bir nesneyi yollarken KOPYALANMASINI İSTEMİYORSAM ama aynı
    // zamanda değiştirmesini de İSTEMİYORSAM ne yapmalıyım?" Cevap: `const std::string& hugeData`
    // ! No copy (RAM saved) & Read-Only (Değiştirilemez/Salt Okunur).

    return 0;
}
```

► 03. Functions And Overloading

module_01_basics/03_functions_and_overloading.cpp

Konu: Basics: Functions, Overloading & Default Parameters — Temeller: Fonksiyonlar, Aşırı

Teori / Theory

THEORY: Function Anatomy

A function in C++ has 4 parts:
1. Return type — what the function gives back (int, double, void, bool, ...)
2. Name — identifier used to call it
3. Parameters — input data (can be empty)
4. Body — the code block `{ }` that executes
Declaration (prototype) tells the compiler "this function exists": int add(int a, int b); Definition provides the actual body: int add(int a, int b) { return a + b; }

TEORİ: Fonksiyon Anatomisi

C++'ta bir fonksiyonun 4 parçası vardır:
1. Dönüş tipi — fonksiyonun geri verdiği şey (int, double, void, bool, ...)
2. İsim — çağırmak için kullanılan tanımlayıcı
3. Parametreler — giriş verileri (boş olabilir)
4. Gövde — çalışan kod bloğu `{ }`
Bildirim (prototip) derleyiciye "bu fonksiyon var" der: int topla(int a, int b); Tanımlama asıl gövdeyi sağlar: int topla(int a, int b) { return a + b; }

THEORY: Why Function Overloading is Possible

In C, you CANNOT have two functions with the same name — the compiler only checks the name. In C++, the compiler appends parameter types to the function name behind the scenes (called "Name Mangling"): print(int) -> _Z5printi print(float) -> _Z5printf Thus you can have many functions with the same name as long as parameters differ!

TEORİ: Aşırı Yükleme Neden Mümkündür

C dilinde aynı isimde iki fonksiyon YAZAMAZSINIZ — derleyici sadece isme bakar. C++'ta derleyici fonksiyon isimlerine parametre tiplerini ekler ("Name Mangling"): yazdir(int) -> _Z6yazdiri yazdir(float) -> _Z6yazdirf Parametreler farklı olduğu sürece aynı isimde istediğiniz kadar fonksiyon olabilir!

CPPREF Depth: Overload Resolution Rules

How does the compiler decide which overloaded function to call?
1. Exact Match — passing exactly an `int` to `print(int)`.
2. Promotion — passing `char` promotes to `int` safely.
3. Std Conversion — passing `float` converts to `double`. If ambiguous, the compiler throws a Hard Error (build fails).

CPPREF Derinlik: Aşırı Yükleme Çözüm Kuralları

Derleyici hangi fonksiyonu çağıracağına şu rütbe sistemiyle karar verir:
1. Birebir Eşleşme — `int` parametreye tam `int` göndermek.
2. Yükseltme — `char` gönderilirse güvenle `int`'e yükseltilir.
3. Standart Çevirim — `float` gönderilirse `double`'a dönüştürülür. İki fonksiyon eşit uyuyorsa derleyici "Belirsizlik (Ambiguity)" hatası fırlatır.

→ Ref: https://en.cppreference.com/w/cpp/language/overload_resolution
→ Ref: https://en.cppreference.com/w/cpp/language/default_arguments
→ Ref: https://en.cppreference.com/w/cpp/language/inline

Kaynak Kod / Source Code

```cpp
#include <cmath>
#include <iostream>
#include <string>

// ================================================================================
// 1. FUNCTION DECLARATIONS (PROTOTYPES) / FONKSİYON BİLDİRİMLERİ (PROTOTİPLER)
// ================================================================================
// EN: Prototypes tell the compiler about the function signature BEFORE main(). The actual body
// (definition) can come after main(). This is how large projects are organized with header files.
// TR: Prototipler fonksiyon imzasını main()'den ÖNCE derleyiciye bildirir. Asıl gövde (tanım)
// main()'den sonra gelebilir. Büyük projelerde header dosyaları böyle çalışır.
double celsiusToFahrenheit(double celsius);
double fahrenheitToCelsius(double fahrenheit);

// ================================================================================
// 2. FUNCTION OVERLOADING / FONKSİYON AŞIRI YÜKLEMESİ
// ================================================================================
// EN: Same name, different parameter types — compiler picks the correct one via Name Mangling.
// TR: Aynı isim, farklı parametre tipleri — derleyici Name Mangling ile doğru olanı seçer.
void displaySensor(int rawValue) {
  std::cout << "  [INT]    Raw sensor: " << rawValue << " counts" << std::endl;
}

void displaySensor(double voltage) {
  std::cout << "  [DOUBLE] Sensor voltage: " << voltage << " V" << std::endl;
}

void displaySensor(const std::string &name) {
  std::cout << "  [STRING] Sensor name: " << name << std::endl;
}
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════════
// 3. DEFAULT PARAMETERS / VARSAYILAN PARAMETRELER
// ═══════════════════════════════════════════════════════════════════════════════
// EN: Parameters with default values can be omitted at call site. Defaults must be specified
// from RIGHT to LEFT — you cannot skip a middle parameter.
// TR: Varsayılan değerli parametreler çağrı yerinde atlanabilir. Varsayılanlar SAĞDAN SOLA
// belirtilmelidir — ortadaki bir parametreyi atlayamazsınız.
void logDTC(const std::string &code, int severity = 1,
            const std::string &module = "UNKNOWN") {
  std::cout << "  DTC: " << code << " | Severity: " << severity
            << " | Module: " << module << std::endl;
}

// ═══════════════════════════════════════════════════════════════════════════════
// 4. INLINE FUNCTIONS / SATIR İÇİ FONKSİYONLAR
// ═══════════════════════════════════════════════════════════════════════════════
// EN: `inline` suggests the compiler to replace the function call with the function body
// directly at the call site, avoiding the overhead of a function call (push/pop stack frame).
// The compiler may ignore the hint if the function is too complex. In modern C++, the compiler
// often inlines small functions automatically (even without the keyword).
// TR: `inline` derleyiciye fonksiyon çağrısını, çağrı noktasında doğrudan fonksiyon gövdesiyle
// değiştirmesini önerir (stack frame push/pop maliyetinden kaçınır). Fonksiyon çok karmaşıksa
// derleyici bu öneriyi görmezden gelebilir. Modern C++'ta derleyici küçük fonksiyonları zaten
// otomatik inline eder.
inline double clampRPM(double rpm, double minRpm = 800.0, double maxRpm = 7000.0) {
  if (rpm < minRpm) return minRpm;
  if (rpm > maxRpm) return maxRpm;
  return rpm;
}

// ═══════════════════════════════════════════════════════════════════════════════
// 5. CONSTEXPR FUNCTIONS / DERLEME ZAMANI FONKSİYONLARI
// ═══════════════════════════════════════════════════════════════════════════════
// EN: `constexpr` functions CAN be evaluated at compile time if called with compile-time
// constants. If called with runtime values, they behave like normal functions. This gives
// ZERO runtime cost for constant expressions.
// TR: `constexpr` fonksiyonlar derleme zamanı sabitleriyle çağrılırsa derleme zamanında
// hesaplanabilir. Çalışma zamanı değerleriyle çağrılırsa normal fonksiyon gibi davranır.
// Sabit ifadeler için SIFIR çalışma zamanı maliyeti sağlar.
constexpr int calculateBaudDivisor(int clockHz, int baudRate) {
  return clockHz / (16 * baudRate);
}

// ═══════════════════════════════════════════════════════════════════════════════
// 6. RETURN TYPE EXAMPLES / DÖNÜŞ TİPİ ÖRNEKLERİ
// ═══════════════════════════════════════════════════════════════════════════════
// EN: Functions can return different types: int, double, bool, void (nothing), string, etc.
// TR: Fonksiyonlar farklı tipler döndürebilir: int, double, bool, void (hiçbir şey), string vb.

// Returns bool — used for condition checks / bool döndürür — koşul kontrolü için
bool isOverheating(double tempC) { return tempC > 110.0; }

// Returns void — performs action, returns nothing / void — eylem yapar, hiçbir şey döndürmez
void printDivider() {
  std::cout << "  ----------------------------------------" << std::endl;
}

// Returns int — calculation result / int döndürür — hesaplama sonucu
int absoluteDifference(int a, int b) { return std::abs(a - b); }

// ═══════════════════════════════════════════════════════════════════════════
//                                  MAIN
// ═══════════════════════════════════════════════════════════════════════════
int main() {
  std::cout << "=== MODULE 1: FUNCTIONS & OVERLOADING ===\n" << std::endl;

  // ═══════════════════════════════════════════════════════════════════════════
  // 1. FUNCTION DECLARATIONS — calling prototyped functions / Prototipli fonksiyonları çağırma
  // ═══════════════════════════════════════════════════════════════════════════
  std::cout << "--- 1. Function Declaration & Definition ---" << std::endl;
  double boilingF = celsiusToFahrenheit(100.0);
  double bodyC = fahrenheitToCelsius(98.6);
  std::cout << "  100 °C = " << boilingF << " °F" << std::endl;
  std::cout << "  98.6 °F = " << bodyC << " °C\n" << std::endl;

```

```cpp
    // ═══════════════════════════════════════════════════════════════════════════
    // 2. FUNCTION OVERLOADING / FONKSİYON AŞIRI YÜKLEMESİ
    // ═══════════════════════════════════════════════════════════════════════════
    std::cout << "--- 2. Function Overloading ---" << std::endl;

    // EN: Compiler selects the correct overload based on argument type.
    // TR: Derleyici argüman tipine göre doğru aşırı yüklemeyi seçer.
    displaySensor(4095);              // int version / int sürümü
    displaySensor(3.28);              // double version / double sürümü
    displaySensor("Knock Sensor B2"); // string version / string sürümü
    std::cout << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════
    // 3. RETURN TYPE VARIETY / DÖNÜŞ TİPİ ÇEŞİTLİLİĞİ
    // ═══════════════════════════════════════════════════════════════════════════
    std::cout << "--- 3. Return Types ---" << std::endl;

    double engineTemp = 115.0;
    std::cout << "  Engine temp: " << engineTemp << " °C" << std::endl;
    std::cout << "  Overheating? " << (isOverheating(engineTemp) ? "YES" : "NO") << std::endl;

    int rpm1 = 3500, rpm2 = 4200;
    std::cout << "  RPM difference: |" << rpm1 << " - " << rpm2 << "| = "
              << absoluteDifference(rpm1, rpm2) << std::endl;

    printDivider();
    std::cout << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════
    // 4. DEFAULT PARAMETERS / VARSAYILAN PARAMETRELER
    // ═══════════════════════════════════════════════════════════════════════════
    std::cout << "--- 4. Default Parameters ---" << std::endl;

    // EN: Call with varying number of arguments — defaults fill the rest.
    // TR: Farklı sayıda argümanla çağır — geri kalanı varsayılanlar doldurur.
    logDTC("P0300");                  // severity=1, module="UNKNOWN"
    logDTC("P0171", 2);              // module="UNKNOWN"
    logDTC("P0420", 3, "Powertrain"); // All specified / Hepsi belirtilmiş
    std::cout << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════
    // 5. INLINE FUNCTION / SATIR İÇİ FONKSİYON
    // ═══════════════════════════════════════════════════════════════════════════
    std::cout << "--- 5. Inline Function ---" << std::endl;

    // EN: clampRPM is likely inlined by the compiler — no function call overhead.
    // TR: clampRPM derleyici tarafından büyük olasılıkla inline edilir — fonksiyon çağrı maliyeti yok.
    std::cout << "  clampRPM(500)  = " << clampRPM(500) << " (min clamped)" << std::endl;
    std::cout << "  clampRPM(3500) = " << clampRPM(3500) << " (unchanged)" << std::endl;
    std::cout << "  clampRPM(9000) = " << clampRPM(9000) << " (max clamped)" << std::endl;
    std::cout << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════
    // 6. CONSTEXPR FUNCTION / DERLEME ZAMANI FONKSİYONU
    // ═══════════════════════════════════════════════════════════════════════════
    std::cout << "--- 6. constexpr Function ---" << std::endl;

    // EN: This is evaluated at COMPILE TIME — zero runtime cost. The result is baked into the
    // binary. No function call happens at runtime.
    // TR: Bu DERLEME ZAMANINDA hesaplanır — sıfır çalışma zamanı maliyeti. Sonuç binary'ye gömülür.
    constexpr int divisor115200 = calculateBaudDivisor(16000000, 115200);
    constexpr int divisor9600 = calculateBaudDivisor(16000000, 9600);

    std::cout << "  UART 16MHz / 115200 baud -> divisor = " << divisor115200 << std::endl;
    std::cout << "  UART 16MHz / 9600 baud   -> divisor = " << divisor9600 << std::endl;

    // EN: Can also be called with runtime values — behaves like a normal function then.
    // TR: Çalışma zamanı değerleriyle de çağrılabilir — o zaman normal fonksiyon gibi davranır.
    int runtimeClock = 8000000;
    int runtimeBaud = 19200;
    int runtimeDiv = calculateBaudDivisor(runtimeClock, runtimeBaud);
    std::cout << "  UART 8MHz / 19200 baud   -> divisor = " << runtimeDiv << " (runtime)"
              << std::endl;
    std::cout << std::endl;

    return 0;
}
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════════
// FUNCTION DEFINITIONS (after main) / FONKSİYON TANIMLARI (main'den sonra)
// ═══════════════════════════════════════════════════════════════════════════════
// EN: These were declared (prototyped) BEFORE main, defined here AFTER main.
// This is the standard pattern: declaration in .h (header), definition in .cpp (source).
// TR: Bunlar main'den ÖNCE bildirildi (prototip), burada main'den SONRA tanımlandı.
// Standart kalıp: bildirim .h (başlık) dosyasında, tanım .cpp (kaynak) dosyasında.

double celsiusToFahrenheit(double celsius) {
  return (celsius * 9.0 / 5.0) + 32.0;
}

double fahrenheitToCelsius(double fahrenheit) {
  return (fahrenheit - 32.0) * 5.0 / 9.0;
}
```

► 04. Operators And Expressions

module_01_basics/04_operators_and_expressions.cpp

Konu: Basics: Operators, Expressions & Precedence — Temeller: Operatörler, İfadeler ve

Teori / Theory

THEORY: Operator Categories in C++

C++ provides a rich set of operators grouped as:
1. Arithmetic (+, -, *, /, %)
2. Comparison (==, !=, <, >, <=, >=)
3. Logical (&&, ||, !)
4. Bitwise (&, |, ^, ~, <<, >>)
5. Assignment (=, +=, -=, *=, /=, %=, <<=, >>=, &=, |=, ^=)
6. Ternary (condition ? trueVal : falseVal)
7. Increment (++x pre, x++ post, --x, x--)

TEORİ: C++ Operatör Kategorileri

C++ zengin bir operatör setine sahiptir:
1. Aritmetik (+, -, *, /, %)
2. Karşılaştırma (==, !=, <, >, <=, >=)
3. Mantıksal (&&, ||, !)
4. Bitsel (&, |, ^, ~, <<, >>)
5. Atama (=, +=, -=, *=, /=, %=, <<=, >>=, &=, |=, ^=)
6. Üçlü (Ternary) (koşul ? doğru : yanlış)
7. Artırma/Azaltma (++x ön, x++ son, --x, x--)

CPPREF Depth: Operator Precedence & Associativity

Operators follow a strict "Precedence Table" (highest -> lowest):
1. `()` (Parentheses) — Highest
2. `* / %` — Left-to-Right (L→R)
3. `+ -` — L→R
4. `<< >>` — L→R
5. `< <= > >=` — L→R
6. `== !=` — L→R
7. `&` (Bitwise AND) — L→R
8. `^` (XOR) — L→R
9. `|` (Bitwise OR) — L→R
10. `&&` — L→R
11. `||` — L→R
12. `?:` (Ternary) — R→L
13. `= += -= ...` — R→L (Assignment is Right-to-Left!)

CPPREF Derinlik: Operatör Önceliği ve İlişkilendirme Yönü

Operatörler katı bir "Öncelik Tablosu"na uyar (yüksek → düşük):
1. `()` (Parantez) — En yüksek
2. `* / %` — Soldan Sağa (L→R)
3. `+ -` — L→R
4. `<< >>` — L→R
5. `< <= > >=` — L→R
6. `== !=` — L→R
7. `&` (Bitsel AND) — L→R
8. `^` (XOR) — L→R
9. `|` (Bitsel OR) — L→R
10. `&&` — L→R
11. `||` — L→R
12. `?:` (Üçlü Ternary) — R→L
13. `= += -= ...` — R→L (Atama Sağdan Soladır!)

→ Ref: https://en.cppreference.com/w/cpp/language/operator_precedence

Kaynak Kod / Source Code

```cpp
#include <bitset>
#include <iostream>

int main() {
  std::cout << "=== MODULE 1: OPERATORS & EXPRESSIONS ===\n" << std::endl;

  // ==========================================================================
  // 1. ARITHMETIC OPERATORS / ARİTMETİK OPERATÖRLER
  // ==========================================================================
  std::cout << "--- 1. Arithmetic Operators ---" << std::endl;

  int rpm = 3500;                       // Engine RPM
  int idleRpm = 800;                    // Idle RPM threshold
  int rpmAboveIdle = rpm - idleRpm;     // 2700
  int doubleRpm = rpm * 2;             // 7000
  int avgRpm = (rpm + idleRpm) / 2;    // 2150
  int remainder = rpm % 1000;           // 500 (modulo)

  std::cout << "RPM: " << rpm << std::endl;
  std::cout << "RPM above idle: " << rpmAboveIdle << std::endl;
  std::cout << "Double RPM: " << doubleRpm << std::endl;
  std::cout << "Average with idle: " << avgRpm << std::endl;
  std::cout << "RPM % 1000 (modulo): " << remainder << "\n" << std::endl;

```

```cpp
    // EN: Integer division truncates! 7 / 2 = 3 (not 3.5)
    // TR: Tam sayı bölmesi kesir kısmını atar! 7 / 2 = 3 (3.5 değil)
    std::cout << "Integer division: 7 / 2 = " << (7 / 2) << std::endl;
    std::cout << "Double  division: 7.0 / 2 = " << (7.0 / 2) << "\n" << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // 2. COMPARISON OPERATORS / KARŞILAŞTIRMA OPERATÖRLERİ
    // ═══════════════════════════════════════════════════════════════════════════════
    std::cout << "--- 2. Comparison Operators ---" << std::endl;

    int coolantTemp = 95;          // °C
    int overheatThreshold = 110;   // °C limit

    // EN: These all return bool (true=1 / false=0).
    // TR: Hepsi bool döndürür (true=1 / false=0).
    std::cout << "coolantTemp == 95  : " << (coolantTemp == 95) << std::endl;
    std::cout << "coolantTemp != 100 : " << (coolantTemp != 100) << std::endl;
    std::cout << "coolantTemp <  110 : " << (coolantTemp < overheatThreshold)
              << std::endl;
    std::cout << "coolantTemp >= 90  : " << (coolantTemp >= 90) << "\n"
              << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // 3. LOGICAL OPERATORS / MANTIKSAL OPERATÖRLER
    // ═══════════════════════════════════════════════════════════════════════════════
    std::cout << "--- 3. Logical Operators ---" << std::endl;

    bool engineOn = true;
    bool seatbeltOn = false;

    // EN: && = AND, || = OR, ! = NOT
    // TR: && = VE, || = VEYA, ! = DEĞİL
    std::cout << "Engine ON && Seatbelt ON : " << (engineOn && seatbeltOn)
              << std::endl;
    std::cout << "Engine ON || Seatbelt ON : " << (engineOn || seatbeltOn)
              << std::endl;
    std::cout << "!seatbeltOn (NOT)        : " << (!seatbeltOn) << std::endl;

    // EN: Short-circuit evaluation: if left side of && is false, right side is NEVER evaluated.
    // This saves CPU cycles in embedded systems.
    // TR: Kısa devre değerlendirmesi: && ifadesinin sol tarafı false ise sağ taraf ASLA
    // değerlendirilmez. Gömülü sistemlerde CPU döngüsü kazandırır.
    if (engineOn && coolantTemp > 80) {
      std::cout << "[ECU] Engine running and coolant warm — normal.\n"
                << std::endl;
    }

    // ═══════════════════════════════════════════════════════════════════════════════
    // 4. INCREMENT / DECREMENT (ARTIRMA / AZALTMA)
    // ═══════════════════════════════════════════════════════════════════════════════
    std::cout << "--- 4. Increment & Decrement ---" << std::endl;

    int fuelLevel = 50; // % fuel remaining

    // EN: Pre-increment (++x): Increments FIRST, then uses the new value.
    // TR: Ön artırma (++x): ÖNCE artırır, sonra kullanır.
    std::cout << "Pre-increment ++fuelLevel: " << ++fuelLevel << std::endl; // 51

    // EN: Post-increment (x++): Uses current value FIRST, then increments.
    // TR: Son artırma (x++): ÖNCE mevcut değeri kullanır, sonra artırır.
    std::cout << "Post-increment fuelLevel++: " << fuelLevel++ << std::endl; // 51
    std::cout << "After post-increment     : " << fuelLevel << std::endl;   // 52

    // Pre-decrement and Post-decrement
    std::cout << "Pre-decrement --fuelLevel: " << --fuelLevel << std::endl; // 51
    std::cout << "Post-decrement fuelLevel--:" << fuelLevel-- << std::endl; // 51
    std::cout << "After post-decrement     : " << fuelLevel << "\n"
              << std::endl; // 50

```

```
94    // ==========================================================================
95    // 5. COMPOUND ASSIGNMENT / BİLEŞİK ATAMA OPERATÖRLERİ
96    // ==========================================================================
97    std::cout << "--- 5. Compound Assignment ---" << std::endl;
98
99    int throttle = 30; // Throttle position %
100   std::cout << "Initial throttle: " << throttle << " %" << std::endl;
101
102   throttle += 20; // throttle = throttle + 20
103   std::cout << "After += 20     : " << throttle << " %" << std::endl;
104
105   throttle -= 10; // throttle = throttle - 10
106   std::cout << "After -= 10     : " << throttle << " %" << std::endl;
107
108   throttle *= 2; // throttle = throttle * 2
109   std::cout << "After *= 2      : " << throttle << " %" << std::endl;
110
111   throttle /= 4; // throttle = throttle / 4
112   std::cout << "After /= 4      : " << throttle << " %" << std::endl;
113
114   throttle %= 7; // throttle = throttle % 7
115   std::cout << "After %= 7      : " << throttle << " %\n" << std::endl;
116
117   // ==========================================================================
118   // 6. TERNARY OPERATOR / ÜÇLÜ OPERATÖR
119   // ==========================================================================
120   std::cout << "--- 6. Ternary Operator ---" << std::endl;
121
122   int batteryVoltage = 12;
123   // EN: condition ? value_if_true : value_if_false
124   // TR: koşul ? doğruysa_değer : yanlışsa_değer
125   std::string batteryStatus =
126       (batteryVoltage >= 12) ? "NORMAL" : "LOW VOLTAGE";
127   std::cout << "Battery (" << batteryVoltage
128             << "V): " << batteryStatus << std::endl;
129
130   // EN: Nested ternary (use sparingly — hurts readability)
131   // TR: İç içe ternary (dikkatli kullanın — okunabilirliği bozar)
132   int speedKmh = 135;
133   std::string zone = (speedKmh > 130) ? "DANGER"
134                    : (speedKmh > 90) ? "HIGHWAY"
135                                      : "CITY";
136   std::cout << "Speed " << speedKmh << " km/h -> Zone: " << zone << "\n"
137             << std::endl;
138
139   // ==========================================================================
140   // 7. BITWISE OPERATORS (Preview) / BİTSEL OPERATÖRLER (Ön İzleme)
141   // ==========================================================================
142   std::cout << "--- 7. Bitwise Operators (Preview) ---" << std::endl;
143
144   // EN: Covered in depth in Module 9. Quick demo here.
145   // std::bitset<8> prints values in binary representation.
146   // TR: Detaylı anlatım Modül 9'da. Burada kısa önizleme.
147   // std::bitset<8> değerleri ikili (binary) formatta yazdırır.
148   unsigned char statusReg = 0b00001010; // Binary literal (C++14)
149
150   std::cout << "OR  (0b1010 | 0b0101): " << std::bitset<8>(0b1010 | 0b0101) << std::endl;
151   std::cout << "AND (0b1010 & 0b1100): " << std::bitset<8>(0b1010 & 0b1100) << std::endl;
152   std::cout << "XOR (0b1010 ^ 0b1100): " << std::bitset<8>(0b1010 ^ 0b1100) << std::endl;
153   std::cout << "Left Shift (1 << 3)  : " << std::bitset<8>(1 << 3) << std::endl;
154   std::cout << "Right Shift (16 >> 2): " << std::bitset<8>(16 >> 2) << std::endl;
155   std::cout << "NOT (~statusReg(~0b00001010)): "
156             << std::bitset<8>(static_cast<unsigned char>(~statusReg))
157             << "\n" << std::endl;
158
159   // ==========================================================================
160   // 8. OPERATOR PRECEDENCE DEMO / OPERATÖR ÖNCELİK GÖSTERİMİ
161   // ==========================================================================
162   std::cout << "--- 8. Precedence Demo ---" << std::endl;
163
```

```
164     // EN: Without parentheses, * binds tighter than +.
165     // TR: Parantez olmadan * operatörü + operatöründen önce çalışır.
166     int resultA = 2 + 3 * 4;     // 14 (not 20!)
167     int resultB = (2 + 3) * 4;   // 20 (forced with parentheses)
168
169     std::cout << "2 + 3 * 4     = " << resultA << " (mul first!)" << std::endl;
170     std::cout << "(2 + 3) * 4   = " << resultB << " (forced add first)"
171               << std::endl;
172
173     // EN: Assignment is RIGHT-TO-LEFT (R→L).
174     // TR: Atama operatörü SAĞDAN SOLA (R→L) çalışır.
175     int a = 0;
176     int b = 0;
177     int c = 0;
178     a = b = c = 42; // c=42, b=c(42), a=b(42)
179     std::cout << "a = b = c = 42 -> a=" << a << " b=" << b << " c=" << c
180               << std::endl;
181
182     return 0;
183   }
```

► 05. Control Flow Deepdive

module_01_basics/05_control_flow_deepdive.cpp

Konu: Basics: Control Flow Deep Dive — Temeller: Akış Kontrolü Derinlemesine

Teori / Theory

THEORY: Decision Structures

Every program needs decision-making. C++ provides:
1. if / else if / else: General-purpose branching.
2. switch / case: Optimized jump-table for integral types (int, char, enum).
3. Ternary (?:): Inline conditional (covered in 04_operators).
In automotive ECU firmware, decision structures control everything: engine management, climate, ABS activation, lighting logic.

TEORİ: Karar Yapıları

Her program karar vermeye ihtiyaç duyar. C++ şunları sunar:
1. if / else if / else: Genel amaçlı dallanma.
2. switch / case: Tam sayı tipleri için optimize edilmiş atlama tablosu.
3. Ternary (?:): Satır içi koşul (04_operators dosyasında anlatıldı).
Otomotiv ECU firmware'inde karar yapıları her şeyi kontrol eder: motor yönetimi, klima, ABS aktivasyonu, aydınlatma mantığı.

THEORY: Loop Structures

Loops repeat a block of code:
1. for: When the iteration count is KNOWN.
2. while: When it depends on a CONDITION (unknown iterations).
3. do-while: Same as while, but executes AT LEAST ONCE.
4. Range-based for (C++11): Safe iteration over containers.
5. break: Exit the loop immediately.
6. continue: Skip this iteration, go to next.

TEORİ: Döngü Yapıları

Döngüler bir kod bloğunu tekrarlar:
1. for: İterasyon sayısı BİLİNDİĞİNDE.
2. while: Bir KOŞULA bağlı olduğunda (belirsiz iterasyon sayısı).
3. do-while: while ile aynı ama EN AZ BİR KEZ çalışır.
4. Range-based for (C++11): Konteynerler üzerinde güvenli iterasyon.
5. break: Döngüden anında çık.
6. continue: Bu iterasyonu atla, sonrakine geç.

CPPREF Depth: Structured Bindings & Init-Statements (C++17)

C++17 allows `if (auto x = compute(); x > threshold)` — the variable is scoped ONLY inside the if-block. This reduces variable leakage. Also, `switch` supports init-statements: `switch (auto code = getDTC(); code)`.

CPPREF Derinlik: Yapısal Bağlamalar ve Başlatma İfadeleri (C++17)

C++17 ile `if (auto x = hesapla(); x > eşik)` yazabilirsiniz. Değişken SADECE if bloğunun içinde geçerlidir. Bu değişken sızıntısını azaltır. `switch` de başlatma ifadesini destekler.

→ Ref: https://en.cppreference.com/w/cpp/language/if
→ Ref: https://en.cppreference.com/w/cpp/language/switch

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <string>

int main() {
  std::cout << "=== MODULE 1: CONTROL FLOW DEEP DIVE ===\n" << std::endl;

  // ================================================================================
  // 1. IF / ELSE IF / ELSE
  // ================================================================================
  std::cout << "--- 1. if / else if / else ---" << std::endl;

  int coolantTempC = 105;

  // EN: Multi-branch decision for coolant temperature monitoring.
  // TR: Soğutma suyu sıcaklığı izleme için çok dallı karar yapısı.
  if (coolantTempC > 120) {
    std::cout << "[ALARM] CRITICAL OVERHEAT: " << coolantTempC << " °C!"
              << std::endl;
  } else if (coolantTempC > 100) {
    std::cout << "[WARNING] Coolant high: " << coolantTempC << " °C"
              << std::endl;
  } else if (coolantTempC > 80) {
    std::cout << "[NORMAL] Coolant normal: " << coolantTempC << " °C"
              << std::endl;
  } else {
    std::cout << "[INFO] Engine cold: " << coolantTempC << " °C" << std::endl;
  }

  // C++17 if with init-statement / C++17 başlatma ifadeli if
  // EN: C++17 allows declaring a variable in the if-statement itself.
  //     The variable lives in if + all else if + else blocks, but NOT outside.
  //     You can also reassign it inside any branch (unless declared const).
  //     Equivalent to: { double v = 13.8; if (v > 14.0) {...} else if (...) {...} else {...} }
  // TR: C++17 ile değişken doğrudan if ifadesinde tanımlanabilir.
  //     Değişken if + tüm else if + else bloklarında yaşar, dışarıda ÖLÜR.
  //     const olmadığı sürece herhangi bir dalda yeni değer atanabilir.
  //     Eşdeğeri: { double v = 13.8; if (v > 14.0) {...} else if (...) {...} else {...} }
  if (double voltage = 13.8; voltage > 14.0) {
```

```
39        std::cout << "[ECU] Overcharge detected: " << voltage << " V" << std::endl;
40      } else if (voltage > 12.0) {
41        // EN: 'voltage' is still accessible here — same scope as if-block.
42        // TR: 'voltage' burada da erişilebilir — if bloğuyla aynı scope.
43        std::cout << "[ECU] Battery voltage OK: " << voltage << " V" << std::endl;
44      } else {
45        // EN: Even in else, the variable is alive. We can reassign it.
46        // TR: else dalında da değişken yaşıyor. Yeni değer atayabiliriz.
47        voltage = 0.0;
48        std::cout << "[ECU] LOW voltage! Reset to: " << voltage << " V"
49                  << std::endl;
50      }
51      // EN: 'voltage' is now OUT OF SCOPE — cannot be used here.
52      // TR: 'voltage' artık SCOPE DIŞI — burada kullanılamaz.
53      std::cout << std::endl;
54
55      // ═══════════════════════════════════════════════════════════════════════════
56      // 2. SWITCH / CASE
57      // ═══════════════════════════════════════════════════════════════════════════
58      std::cout << "--- 2. switch / case ---" << std::endl;
59
60      // EN: DTC (Diagnostic Trouble Code) severity levels: 1=Info, 2=Warning, 3=Critical. switch
61      // creates an O(1) jump table.
62      // TR: DTC (Teşhis Arıza Kodu) önem seviyesi. switch O(1) atlama tablosu oluşturur.
63      int dtcSeverity = 2;
64
65      switch (dtcSeverity) {
66      case 1:
67        std::cout << "[DTC] Severity 1: Informational" << std::endl;
68        break; // EN: Without break, execution "falls through" to next case!
69               // TR: break olmadan bir sonraki case'e düşer!
70      case 2:
71        std::cout << "[DTC] Severity 2: Warning — schedule maintenance"
72                  << std::endl;
73        break;
74      case 3:
75        std::cout << "[DTC] Severity 3: CRITICAL — stop vehicle!" << std::endl;
76        break;
77      default:
78        std::cout << "[DTC] Unknown severity level" << std::endl;
79        break;
80      }
81
82      // EN: Intentional fall-through demo (grouping cases).
83      // TR: Kasıtlı fall-through gösterimi (case'leri gruplama).
84      char gearPosition = 'D';
85      std::cout << "Gear: " << gearPosition << " -> ";
86      switch (gearPosition) {
87      case 'D':
88      case 'd':
89        std::cout << "Drive mode" << std::endl;
90        break;
91      case 'R':
92      case 'r':
93        std::cout << "Reverse mode" << std::endl;
94        break;
95      case 'N':
96      case 'n':
97        std::cout << "Neutral" << std::endl;
98        break;
99      case 'P':
100     case 'p':
101       std::cout << "Park" << std::endl;
102       break;
103     default:
104       std::cout << "Invalid gear!" << std::endl;
105       break;
106     }
107     std::cout << std::endl;
```

```cpp

    // ==========================================================================
    // 3. CLASSIC FOR LOOP / KLASİK FOR DÖNGÜSÜ
    // ==========================================================================
    std::cout << "--- 3. Classic for loop ---" << std::endl;

    // EN: Simulate scanning 6 CAN bus message slots.
    // TR: 6 CAN bus mesaj slotunu taramak.
    for (int slot = 0; slot < 6; ++slot) {
      std::cout << "Scanning CAN slot #" << slot << std::endl;
    }
    std::cout << std::endl;

    // ==========================================================================
    // 4. WHILE LOOP / WHILE DÖNGÜSÜ
    // ==========================================================================
    std::cout << "--- 4. while loop ---" << std::endl;

    // EN: Simulate coolant warm-up until threshold.
    // TR: Soğutma suyu ısınmasını eşiğe kadar simüle et.
    int warmUpTemp = 20;
    while (warmUpTemp < 80) {
      warmUpTemp += 15; // Simulating heating
      std::cout << "Warming up... " << warmUpTemp << " °C" << std::endl;
    }
    std::cout << "[ECU] Operating temperature reached!\n" << std::endl;

    // ==========================================================================
    // 5. DO-WHILE LOOP / DO-WHILE DÖNGÜSÜ
    // ==========================================================================
    std::cout << "--- 5. do-while loop ---" << std::endl;

    // EN: Executes AT LEAST ONCE, even if condition is already false. Use case: Sensor self-test
    // that must run at least once on boot.
    // TR: Koşul baştan yanlış olsa bile EN AZ BİR KEZ çalışır. Kullanım: Açılışta en az bir kez
    // çalışması gereken sensör öz testi.
    int sensorRetry = 0;
    do {
      std::cout << "Sensor self-test attempt #" << (sensorRetry + 1)
                << std::endl;
      sensorRetry++;
    } while (sensorRetry < 3);
    std::cout << "[ECU] Self-test complete.\n" << std::endl;

    // ==========================================================================
    // 6. BREAK & CONTINUE
    // ==========================================================================
    std::cout << "--- 6. break & continue ---" << std::endl;

    // EN: Scan 10 ECU addresses. Skip address 5 (reserved), stop at 8 (limit).
    // TR: 10 ECU adresini tara. 5'i atla (ayrılmış), 8'de dur (sınır).
    for (int addr = 1; addr <= 10; ++addr) {
      if (addr == 5) {
        std::cout << "  Address " << addr << " -> RESERVED (skip)" << std::endl;
        continue; // Skip this iteration / Bu iterasyonu atla
      }
      if (addr == 8) {
        std::cout << "  Address " << addr << " -> LIMIT reached (break)"
                  << std::endl;
        break; // Exit the for loop entirely / Döngüden tamamen çık
      }
      std::cout << "  Address " << addr << " -> OK" << std::endl;
    }
    std::cout << std::endl;
```

```

  // ==========================================================================
  // 7. NESTED LOOPS / İÇ İÇE DÖNGÜLER
  // ==========================================================================
  std::cout << "--- 7. Nested Loops ---" << std::endl;

  // EN: 4 cylinders × 3 sensors each = 12 readings.
  // TR: 4 silindir × 3 sensör = 12 okuma.
  for (int cyl = 1; cyl <= 4; ++cyl) {
    for (int sensor = 1; sensor <= 3; ++sensor) {
      std::cout << "  Cyl" << cyl << "-Sensor" << sensor << " ";
    }
    std::cout << std::endl;
  }
  std::cout << std::endl;

  // ==========================================================================
  // 8. RANGE-BASED FOR (C++11) — revisited
  // ==========================================================================
  std::cout << "--- 8. Range-based for (C++11) ---" << std::endl;

  // EN: Iterating over DTC codes stored in an array.
  // TR: Dizide saklanan DTC kodlarını dolaşma.
  std::string dtcCodes[] = {"P0300", "P0171", "P0420", "P0442"};

  for (const auto &dtc : dtcCodes) {
    std::cout << "  Active DTC: " << dtc << std::endl;
  }
  std::cout << std::endl;

  // ==========================================================================
  // 9. INFINITE LOOP PATTERN (common in embedded) / SONSUZ DÖNGÜ KALIP
  // ==========================================================================
  std::cout << "--- 9. Infinite Loop Pattern ---" << std::endl;
  // EN: In real embedded firmware, `while(true)` or `for(;;)` is the main super-loop. Here we
  // simulate with a counter.
  // TR: Gerçek gömülü firmware'de `while(true)` veya `for(;;)` ana süper-döngüdür. Burada bir
  // sayaçla simüle ediyoruz.
  int tick = 0;
  while (true) {
    std::cout << "  ECU tick #" << tick << std::endl;
    ++tick;
    if (tick >= 5) {
      std::cout << "  [DEMO] Exiting super-loop after 5 ticks.\n" << std::endl;
      break;
    }
  }

  return 0;
}
```

► 06. Strings And Arrays

module_01_basics/06_strings_and_arrays.cpp

Konu: Basics: Strings and Arrays — Temeller: Dizeler (Stringler) ve Diziler

Teori / Theory

THEORY: C-Strings vs std::string

C++ inherited NULL-terminated character arrays from C (`char str[]`). These are dangerous: no bounds checking, manual memory management, easy buffer overflows. Modern C++ uses `std::string` from `<string>`:
1. Automatic memory management (grows/shrinks as needed).
2. Safe concatenation with `+` operator.
3. Rich API: `.length()`, `.substr()`, `.find()`, `.compare()`, etc.

TEORİ: C-Stringleri vs std::string

C++ dilinden NULL sonlandırmalı karakter dizilerini miras aldı (`char str[]`). Bunlar tehlikelidir: sınır kontrolü yok, manuel bellek yönetimi, kolay buffer overflow. Modern C++ `<string>` kütüphanesindeki `std::string` kullanır:
1. Otomatik bellek yönetimi (ihtiyaca göre büyür/küçülür).
2. `+` operatörü ile güvenli birleştirme.
3. Zengin API: `.length()`, `.substr()`, `.find()`, `.compare()`, vb.

THEORY: C-Style Arrays vs std::array

C-style arrays (`int arr[5]`) decay to pointers when passed to functions, losing size information. `std::array<int, 5>` (C++11) wraps a fixed-size array with:
1. Bounds checking via `.at()`.
2. Knows its own size via `.size()`.
3. For dynamic sizes, use `std::vector`.

TEORİ: C Dizileri vs std::array

C-tarzı diziler (`int arr[5]`) fonksiyonlara geçildiğinde pointer'a dönüşür ve boyut bilgisini kaybeder. `std::array<int, 5>` (C++11) sabit boyutlu diziyi şu özelliklerle sarar:
1. `.at()` ile sınır kontrolü.
2. `.size()` ile kendi boyutunu bilme.
3. Dinamik boyut için `std::vector` kullanın.

CPPREF Depth: Small String Optimization (SSO)

Most `std::string` implementations apply SSO: strings shorter than ~15-22 characters are stored INSIDE the object (on the stack), avoiding heap allocation entirely. Longer strings trigger a `new` allocation. This is crucial for embedded systems where heap usage is expensive.

CPPREF Derinlik: Küçük String Optimizasyonu (SSO)

Çoğu `std::string` implementasyonu SSO uygular: ~15-22 karakterden kısa string'ler objenin İÇİNDE (stack'te) saklanır, heap tahsisinden tamamen kaçınılır. Uzun string'ler `new` ile heap'te yer açar. Heap kullanımının pahalı olduğu gömülü sistemlerde bu kritiktir.

→ Ref: https://en.cppreference.com/w/cpp/string/basic_string
→ Ref: https://en.cppreference.com/w/cpp/container/array

Kaynak Kod / Source Code

```cpp
#include <array>
#include <iostream>
#include <string>

int main() {
  std::cout << "=== MODULE 1: STRINGS & ARRAYS ===\n" << std::endl;

  // ═══════════════════════════════════════════════════════════════════════════
  // 1. C-STYLE STRING (Legacy — avoid in new code)
  // ═══════════════════════════════════════════════════════════════════════════
  std::cout << "--- 1. C-Style String (Legacy) ---" << std::endl;

  // EN: NULL-terminated character array. Last char is '\0' automatically.
  // TR: NULL sonlandırmalı karakter dizisi. Son karakter otomatik '\0' eklenir.
  char vinCstr[] = "WBAPH5C55BA271234";
  std::cout << "VIN (C-String): " << vinCstr << std::endl;
  std::cout << "First char    : " << vinCstr[0] << std::endl;
  std::cout << "Array size    : " << sizeof(vinCstr)
            << " (17 chars + 1 null)\n" << std::endl;

  // ═══════════════════════════════════════════════════════════════════════════
  // 2. std::string (Modern C++)
  // ═══════════════════════════════════════════════════════════════════════════
  std::cout << "--- 2. std::string (Modern C++) ---" << std::endl;

  std::string ecuName = "BCM_BodyControl";
  std::string version = "v2.4.1";

  // Concatenation / Birleştirme
  std::string fullId = ecuName + " [" + version + "]";
  std::cout << "Full ID       : " << fullId << std::endl;
  std::cout << "Length        : " << fullId.length() << std::endl;
  std::cout << "Char at [4]   : " << fullId[4] << std::endl;
  std::cout << "Char at .at(4): " << fullId.at(4) << " (bounds-checked)"
            << std::endl;

  // Substring / Alt dize
  std::string prefix = fullId.substr(0, 3); // "BCM"
  std::cout << "Prefix (0,3)  : " << prefix << std::endl;

  // Find / Arama
  size_t pos = fullId.find("Body");
  if (pos != std::string::npos) {
    std::cout << "Found 'Body' at index: " << pos << std::endl;
  }

  // Compare / Karşılaştırma
  std::string other = "BCM_BodyControl [v2.4.1]";
  std::cout << "Equal to other? : " << (fullId == other ? "Yes" : "No")
            << std::endl;

  // Append / Ekleme
  fullId.append(" — Active");
  std::cout << "After append  : " << fullId << std::endl;

  // Empty check / Boşluk kontrolü
  std::string emptyStr;
  std::cout << "emptyStr.empty(): " << emptyStr.empty() << "\n" << std::endl;

  // ═══════════════════════════════════════════════════════════════════════════
  // 3. C-STYLE ARRAY
  // ═══════════════════════════════════════════════════════════════════════════
  std::cout << "--- 3. C-Style Array ---" << std::endl;

```

```
65    // EN: Fixed-size array of sensor voltage readings (mV).
66    // TR: Sabit boyutlu sensör voltaj okumaları dizisi (mV).
67    int sensorMv[5] = {3300, 2800, 3100, 2950, 3050};
68
69    // Classic for iteration
70    for (int i = 0; i < 5; ++i) {
71      std::cout << "  Sensor[" << i << "] = " << sensorMv[i] << " mV"
72                << std::endl;
73    }
74    std::cout << "Array sizeof: " << sizeof(sensorMv)
75              << " bytes (" << sizeof(sensorMv) / sizeof(sensorMv[0])
76              << " elements)\n" << std::endl;
77
78    // ==========================================================================
79    // 4. std::array<T, N> (C++11) — Modern Fixed-Size Array
80    // ==========================================================================
81    std::cout << "--- 4. std::array (Modern C++11) ---" << std::endl;
82
83    // EN: Knows its own size, supports bounds-checked access with .at().
84    // TR: Kendi boyutunu bilir, .at() ile sınır kontrolü sağlar.
85    std::array<double, 4> egtTemps = {650.5, 720.3, 695.0, 710.8};
86
87    std::cout << "EGT array size: " << egtTemps.size() << std::endl;
88    for (size_t i = 0; i < egtTemps.size(); ++i) {
89      std::cout << "  EGT_Cyl" << (i + 1) << ": " << egtTemps[i] << " °C"
90                << std::endl;
91    }
92
93    // Safe access with .at() — throws std::out_of_range if bad index.
94    // TR: .at() ile güvenli erişim — kötü indeks verilirse exception fırlatır.
95    std::cout << "egtTemps.at(2): " << egtTemps.at(2) << " °C" << std::endl;
96    std::cout << "Front: " << egtTemps.front()
97              << " | Back: " << egtTemps.back() << "\n" << std::endl;
98
99    // ==========================================================================
100   // 5. MULTIDIMENSIONAL ARRAY / ÇOK BOYUTLU DİZİ
101   // ==========================================================================
102   std::cout << "--- 5. Multidimensional Array ---" << std::endl;
103
104   // EN: 3 ECUs × 4 sensor channels each (voltage mV readings).
105   // TR: 3 ECU × 4 sensör kanalı (mV okuma değerleri).
106   int sensorGrid[3][4] = {
107       {3300, 2900, 3100, 3050}, // ECU 0: BCM
108       {3200, 3000, 2800, 3150}, // ECU 1: Powertrain
109       {3100, 3250, 2950, 3000}  // ECU 2: HVAC
110   };
111
112   const char *ecuNames[] = {"BCM", "Powertrain", "HVAC"};
113   for (int ecu = 0; ecu < 3; ++ecu) {
114     std::cout << "  " << ecuNames[ecu] << ": ";
115     for (int ch = 0; ch < 4; ++ch) {
116       std::cout << sensorGrid[ecu][ch] << " mV";
117       if (ch < 3) std::cout << ", ";
118     }
119     std::cout << std::endl;
120   }
121   std::cout << std::endl;
122
123   // ==========================================================================
124   // 6. STRING ITERATION / STRING ÜZERİNDE İTERASYON
125   // ==========================================================================
126   std::cout << "--- 6. String Iteration ---" << std::endl;
127
128   std::string vin = "WBAPH5C55BA271234";
129   std::cout << "VIN chars: ";
130   for (const char &ch : vin) {
131     std::cout << ch << ' ';
132   }
133   std::cout << "\n" << std::endl;
```

```
134
135     // EN: Count digits in VIN.
136     // TR: VIN içindeki rakamları say.
137     int digitCount = 0;
138     for (const char &ch : vin) {
139       if (ch >= '0' && ch <= '9') {
140         ++digitCount;
141       }
142     }
143     std::cout << "Digits in VIN: " << digitCount << std::endl;
144
145     return 0;
146   }
```

► 07. Enums Type Conversions

module_01_basics/07_enums_type_conversions.cpp

Konu: Basics: Enumerations, auto, typedef/using & Type Conversions — Temeller:

Teori / Theory

THEORY: enum vs enum class

Classic C-style `enum` leaks its members into the enclosing scope and implicitly converts to `int`. This causes name collisions and bugs. Modern `enum class` (C++11) provides:
1. Scoped names: `Color::Red` (not just `Red`).
2. No implicit conversion to int — you must use `static_cast`.
3. Underlying type can be specified: `enum class Gear : uint8_t`.

TEORİ: enum ve enum class Karşılaştırması

Klasik C-tarzı `enum` üyelerini kapsam dışına sızdırır ve `int`'e dolaylı olarak dönüşür. Bu isim çakışmasına ve hatalara neden olur. Modern `enum class` (C++11):
1. Kapsamlı isimler: `Color::Red` (sadece `Red` değil).
2. int'e dolaylı dönüşüm YOK — `static_cast` kullanmalısınız.
3. Alt tür belirlenebilir: `enum class Gear : uint8_t`.

THEORY: Type Conversions

C++ performs two kinds of conversions:
1. Implicit (automatic): The compiler silently widens/narrows types (e.g., int → double). Narrowing conversions can lose data!
2. Explicit (casts): a. `static_cast<T>()`: Compile-time type conversion (SAFEST). b. `(T)x` or `T(x)`: C-style cast (DANGEROUS, no checks). c. `reinterpret_cast`, `const_cast`, `dynamic_cast` for advanced cases.

TEORİ: Tür Dönüşümleri

C++ iki tür dönüşüm yapar:
1. Dolaylı (otomatik): Derleyici sessizce tipleri genişletir/daraltır. Daraltma dönüşümleri veri kaybına yol açabilir!
2. Açık (cast): a. `static_cast<T>()`: Derleme zamanı dönüşümü (EN GÜVENLİ). b. `(T)x` veya `T(x)`: C-tarzı cast (TEHLİKELİ, kontrol yok). c. `reinterpret_cast`, `const_cast`, `dynamic_cast` ileri seviye durumlar için.

CPPREF Depth: auto Type Deduction

`auto` (C++11) tells the compiler to deduce the type from the initializer. It reduces boilerplate but should be used carefully:
1. Good: `auto it = vec.begin();` (avoids the long iterator type).
2. Bad: `auto x = 42;` (is it int? unsigned? long? — clarity lost).
3. `decltype(expr)`: Deduces the type of an expression without evaluating it.

CPPREF Derinlik: auto Tip Çıkarımı

`auto` (C++11) derleyiciye başlatıcıdan tipi çıkarmasını söyler. Standart kodu azaltır ama dikkatli kullanılmalıdır:
1. İyi: `auto it = vec.begin();` (uzun iterator tipinden kaçınır).
2. Kötü: `auto x = 42;` (int mi? unsigned mı? long mu? — netlik kaybolur).
3. `decltype(expr)`: Bir ifadenin tipini, onu çalıştırmadan çıkarır.

→ Ref: https://en.cppreference.com/w/cpp/language/enum
→ Ref: https://en.cppreference.com/w/cpp/language/static_cast
→ Ref: https://en.cppreference.com/w/cpp/language/auto

Kaynak Kod / Source Code

```cpp
#include <cstdint> // uint8_t
#include <iostream>
#include <string>

// ═══════════════════════════════════════════════════════════════════════════
// 1. CLASSIC enum (C-Style — avoid in new code)
// ═══════════════════════════════════════════════════════════════════════════
// EN: Classic enum members LEAK into the surrounding scope. After this declaration,
//     RED, GREEN, BLUE become global names — as if you wrote `const int RED = 0;`.
//     Problem: If another enum also defines RED, you get a COMPILE ERROR (name collision).
//     Also, classic enum implicitly converts to int, so `int x = RED;` compiles silently.
//     For these reasons, prefer `enum class` in modern C++.
// TR: Klasik enum üyeleri KAPSAM DIŞINA SIZAR. Bu bildirimden sonra RED, GREEN, BLUE
//     global isimler olur — sanki `const int RED = 0;` yazmışsınız gibi.
//     Sorun: Başka bir enum da RED tanımlarsa, DERLEME HATASI alırsınız (isim çakışması).
//     Ayrıca klasik enum int'e dolaylı dönüşür, yani `int x = RED;` sessizce derlenir.
//     Bu nedenlerle modern C++'ta `enum class` tercih edin.
enum LegacyIndicatorColor { RED, GREEN, BLUE };

// ═══════════════════════════════════════════════════════════════════════════
// 2. MODERN enum class (C++11 — strongly typed)
// ═══════════════════════════════════════════════════════════════════════════
// EN: Scoped and type-safe. Must use GearPosition::Park syntax.
//     `: uint8_t` specifies the UNDERLYING TYPE — each enumerator is stored as a
//     single byte (0-255). Without this, the compiler picks the default (usually int).
//     In embedded/automotive code, specifying the underlying type is essential for
//     memory layout control and protocol serialization (e.g., CAN frame bytes).
// TR: Kapsamlı ve tip güvenli. GearPosition::Park sözdizimi zorunlu.
//     `: uint8_t` ALT TÜR belirtir — her numaralandırıcı tek bayt (0-255) olarak saklanır.
//     Bu olmadan derleyici varsayılanı seçer (genellikle int).
//     Gömülü/otomotiv kodda, bellek düzeni kontrolü ve protokol serileştirmesi
//     (örn. CAN çerçeve baytları) için alt türü belirtmek gereklidir.
enum class GearPosition : uint8_t {
  Park = 0,
  Reverse = 1,
  Neutral = 2,
  Drive = 3,
  Sport = 4
};
```

```cpp

// EN: ECU operating states for a Body Control Module.
// TR: Gövde Kontrol Modülü (BCM) için ECU çalışma durumları.
enum class ECUState { Off, Booting, Running, Sleep, Error };

// ═══════════════════════════════════════════════════════════════════════════════
// 3. typedef & using (Type Aliases)
// ═══════════════════════════════════════════════════════════════════════════════
// EN: Type aliases give meaningful names to primitive types. Instead of writing
//     `unsigned long` everywhere, you write `SensorId` — the code becomes self-documenting.
//     `typedef` is the old C-style syntax. It works but reads "backwards".
// TR: Tür takma adları, ilkel türlere anlamlı isimler verir. Her yere
//     `unsigned long` yazmak yerine `SensorId` yazarsınız — kod kendi kendini belgeler.
//     `typedef` eski C tarzı sözdizimidir. Çalışır ama "tersinden" okunur.
typedef unsigned long SensorId;

// EN: `using` (C++11) is the modern replacement — reads left-to-right like assignment.
//     `using MilliVolts = int;` means "MilliVolts IS an int".
//     It's also the ONLY syntax that supports template aliases:
//         template<typename T> using Vec = std::vector<T>;  // typedef cannot do this!
//     Prefer `using` in all new code.
// TR: `using` (C++11) modern karşılıktır — atama gibi soldan sağa okunur.
//     `using MilliVolts = int;` demek "MilliVolts bir int'TIR" demektir.
//     Ayrıca şablon takma adlarını DESTEKLEYEn TEK sözdizimidir:
//         template<typename T> using Vec = std::vector<T>;  // typedef bunu yapamaz!
//     Tüm yeni kodda `using` tercih edin.
using MilliVolts = int;
using TemperatureC = double;

int main() {
  std::cout << "=== MODULE 1: ENUMS, AUTO & TYPE CONVERSIONS ===\n"
            << std::endl;

  // ═══════════════════════════════════════════════════════════════════════════
  // 1. CLASSIC ENUM DEMO
  // ═══════════════════════════════════════════════════════════════════════════
  std::cout << "--- 1. Classic enum (Legacy) ---" << std::endl;

  // EN: We can write just `GREEN` — no prefix needed. This is the "leaking" behavior:
  //     GREEN is visible in the ENTIRE scope as if it were a plain integer constant.
  //     If another enum had Green, you'd get a name collision error.
  // TR: Sadece `GREEN` yazabiliriz — önek gerekmez. Bu "sızma" davranışıdır:
  //     GREEN, düz bir tamsayı sabiti gibi TÜM KAPSAMDA görünürdür.
  //     Başka bir enum'da Green olsaydı, isim çakışması hatası alırdınız.
  LegacyIndicatorColor indicator = GREEN;

  // EN: Classic enum SILENTLY converts to int — no cast needed!
  //     This compiles without warning: the compiler treats GREEN as just `1`.
  //     This is dangerous because you could accidentally compare colors with gear positions
  //     and the compiler would not catch the logical error.
  // TR: Klasik enum SESSİZCE int'e dönüşür — cast gerekmez!
  //     Bu uyarısız derlenir: derleyici GREEN'i sadece `1` olarak işler.
  //     Bu tehlikelidir çünkü yanlışlıkla renkleri vites pozisyonlarıyla
  //     karşılaştırabilirsiniz ve derleyici mantıksal hatayı yakalaMAZ.
  int colorCode = indicator;
  std::cout << "Indicator color code (int): " << colorCode << "\n"
            << std::endl;

  // ═══════════════════════════════════════════════════════════════════════════
  // 2. MODERN ENUM CLASS DEMO
  // ═══════════════════════════════════════════════════════════════════════════
  std::cout << "--- 2. enum class (Modern C++11) ---" << std::endl;

  // EN: enum class members are SCOPED — you must write GearPosition::Drive,
  //     not just "Drive". This prevents name collisions across different enums.
  // TR: enum class üyeleri KAPSAMLIDIR — sadece "Drive" değil, GearPosition::Drive
  //     yazmalısınız. Bu, farklı enum'lar arasındaki isim çakışmasını önler.
  GearPosition gear = GearPosition::Drive;

```

```
    // EN: Unlike classic enum, enum class does NOT implicitly convert to int.
    //     The line below would cause a COMPILE ERROR:
    //         int gearNum = gear;   // ERROR: cannot convert 'GearPosition' to 'int'
    //     You MUST use static_cast<> to explicitly convert. This is by design —
    //     it prevents accidental mixing of unrelated enum types with integers.
    // TR: Klasik enum'dan farklı olarak, enum class int'e DOLAYLI dönüşmez.
    //     Aşağıdaki satır DERLEME HATASI verir:
    //         int gearNum = gear;   // HATA: 'GearPosition' → 'int' dönüştürülemez
    //     Açıkça dönüştürmek için static_cast<> KULLANMALISINIZ. Bu tasarım gereğidir —
    //     ilgisiz enum türlerinin tamsayılarla yanlışlıkla karışmasını önler.
    int gearNum = static_cast<int>(gear);
    std::cout << "Gear position: " << gearNum << " (Drive)" << std::endl;

    // EN: Underlying type is uint8_t (we specified it in the declaration).
    //     static_cast to uint8_t gives the raw byte value.
    // TR: Alt tür uint8_t'dir (bildirimde belirttik).
    //     static_cast ile uint8_t'ye dönüştürmek ham bayt değerini verir.
    auto sportRaw = static_cast<uint8_t>(GearPosition::Sport);
    std::cout << "Sport raw (uint8_t): " << static_cast<int>(sportRaw) << std::endl;

    // EN: enum class is ideal for switch statements — the compiler can warn
    //     if you forget to handle a case (with -Wswitch).
    // TR: enum class switch ifadeleri için idealdir — bir durumu ele almayı
    //     unutursanız derleyici uyarı verebilir (-Wswitch ile).
    ECUState bcmState = ECUState::Running;
    switch (bcmState) {
    case ECUState::Off:
      std::cout << "BCM: OFF" << std::endl;
      break;
    case ECUState::Booting:
      std::cout << "BCM: Booting..." << std::endl;
      break;
    case ECUState::Running:
      std::cout << "BCM: Running normally" << std::endl;
      break;
    case ECUState::Sleep:
      std::cout << "BCM: Sleep mode" << std::endl;
      break;
    case ECUState::Error:
      std::cout << "BCM: ERROR!" << std::endl;
      break;
    }
    std::cout << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════
    // 3. typedef / using DEMO
    // ═══════════════════════════════════════════════════════════════════════════
    std::cout << "--- 3. typedef & using aliases ---" << std::endl;

    // EN: Instead of `unsigned long egtId = 1001UL;`, we write `SensorId egtId = 1001UL;`.
    //     The compiled code is IDENTICAL — type aliases don't create new types, just new names.
    //     But the readability improvement is significant: you immediately know this
    //     variable holds a sensor identifier, not just "some unsigned long".
    // TR: `unsigned long egtId = 1001UL;` yerine `SensorId egtId = 1001UL;` yazıyoruz.
    //     Derlenen kod BİREBİR AYNIDIR — tür takma adları yeni tür oluşturmaz, sadece yeni isim.
    //     Ama okunabilirlik artışı önemlidir: bu değişkenin sadece "herhangi bir unsigned long"
    //     değil, bir sensör kimliği tuttuğunu hemen anlarsınız.
    SensorId egtId = 1001UL;
    MilliVolts reading = 3300;
    TemperatureC ambientTemp = 22.5;

    std::cout << "Sensor ID  : " << egtId << std::endl;
    std::cout << "Reading    : " << reading << " mV" << std::endl;
    std::cout << "Ambient    : " << ambientTemp << " °C\n" << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════
    // 4. auto TYPE DEDUCTION (C++11)
    // ═══════════════════════════════════════════════════════════════════════════
    std::cout << "--- 4. auto keyword (C++11) ---" << std::endl;

```

```cpp
    // EN: `auto` tells the compiler: "look at the right-hand side and figure out the type".
    //     13.8 is a double literal       → voltage becomes `double`.
    //     5 is an int literal             → ecuCount becomes `int`.
    //     std::string("...") is a string  → vinStr becomes `std::string`.
    //     auto does NOT mean "untyped" — the type is fixed at compile time, just inferred.
    // TR: `auto` derleyiciye der ki: "sağ taraftaki değere bak ve türü kendin bul".
    //     13.8 bir double literal'dır     → voltage `double` olur.
    //     5 bir int literal'dır           → ecuCount `int` olur.
    //     std::string("...") bir string   → vinStr `std::string` olur.
    //     auto "tipsiz" demek DEĞİLDİR — tür derleme zamanında sabittir, sadece çıkarılır.
    auto voltage = 13.8;
    auto ecuCount = 5;
    auto vinStr = std::string("WBAPH5C55BA271234");

    // EN: CAUTION: `auto x = "hello";` deduces `const char*`, NOT `std::string`!
    //     If you want std::string, write: `auto s = std::string("hello");`
    //     or use the string literal suffix: `auto s = "hello"s;` (C++14, needs `using namespace std::literals;`)
    // TR: DİKKAT: `auto x = "hello";` türü `const char*` çıkarır, `std::string` DEĞİL!
    //     std::string istiyorsanız: `auto s = std::string("hello");` yazın
    //     veya string literal son eki kullanın: `auto s = "hello"s;` (C++14, `using namespace std::literals;` gerektirir)

    std::cout << "voltage (auto = double): " << voltage << std::endl;
    std::cout << "ecuCount (auto = int)  : " << ecuCount << std::endl;
    std::cout << "vinStr (auto = string) : " << vinStr << "\n" << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // 5. IMPLICIT CONVERSIONS / DOLAYLI DÖNÜŞÜMLER
    // ═══════════════════════════════════════════════════════════════════════════════
    std::cout << "--- 5. Implicit Conversions ---" << std::endl;

    // EN: WIDENING (safe): int → double. The compiler automatically promotes the
    //     smaller type to the larger one. 3500 becomes 3500.0 with no data loss.
    //     This happens because double can represent all int values (up to 2^53).
    //     Widening conversions: bool→int, int→long, int→double, float→double.
    // TR: GENİŞLETME (güvenli): int → double. Derleyici küçük türü otomatik olarak
    //     büyük türe yükseltir. 3500, veri kaybı olmadan 3500.0 olur.
    //     Bu, double'ın tüm int değerlerini temsil edebilmesinden kaynaklanır (2^53'e kadar).
    //     Genişletme dönüşümleri: bool→int, int→long, int→double, float→double.
    int rpm = 3500;
    double rpmDouble = rpm;
    std::cout << "int 3500 -> double: " << rpmDouble << std::endl;

    // EN: NARROWING (DANGEROUS!): double → int. The fractional part is TRUNCATED
    //     (chopped off, NOT rounded). 2.75 becomes 2, not 3.
    //     The compiler may warn with -Wconversion. In C++11 brace initialization,
    //     narrowing is a COMPILE ERROR: `int x{2.75};` → error!
    //     Always use static_cast to make narrowing explicit and intentional.
    // TR: DARALTMA (TEHLİKELİ!): double → int. Kesirli kısım kesilir.
    //     (kesilir, yuvarlanMAZ). 2.75, 3 değil 2 olur.
    //     Derleyici -Wconversion ile uyarı verebilir. C++11 küme parantezi başlatmada
    //     daraltma DERLEME HATASIDIR: `int x{2.75};` → hata!
    //     Daraltmayı açık ve bilinçli yapmak için her zaman static_cast kullanın.
    double precisePressure = 2.75;
    int roughPressure = static_cast<int>(precisePressure); // 2 (not 2.75!)
    std::cout << "double 2.75 -> int: " << roughPressure
              << " (fractional part LOST!)" << std::endl;

    // EN: bool → int: `true` becomes 1, `false` becomes 0. This is guaranteed by
    //     the C++ standard. The reverse also works: any non-zero int becomes `true`,
    //     zero becomes `false`. This is used heavily in conditions:
    //         if (absFlag) { ... }  // works because non-zero = true
    // TR: bool → int: `true` 1 olur, `false` 0 olur. Bu C++ standardı tarafından
    //     garanti edilir. Tersi de çalışır: sıfır olmayan herhangi bir int `true` olur,
    //     sıfır `false` olur. Bu, koşullarda yoğun olarak kullanılır:
    //         if (absFlag) { ... }  // çalışır çünkü sıfır olmayan = true
    bool absActive = true;
    int absFlag = absActive;
    std::cout << "bool true -> int: " << absFlag << "\n" << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // 6. EXPLICIT CONVERSIONS (static_cast) / AÇIK DÖNÜŞÜMLER
    // ═══════════════════════════════════════════════════════════════════════════════
    std::cout << "--- 6. static_cast (Explicit) ---" << std::endl;

```

```cpp
    // EN: `static_cast<T>(expr)` is the SAFEST C++ cast. The compiler checks at
    //     compile time whether the conversion is valid. If it's not (e.g., casting
    //     a pointer to an unrelated class), you get a COMPILE ERROR — not silent UB.
    //     Use static_cast whenever you intentionally convert between numeric types.
    // TR: `static_cast<T>(expr)` EN GÜVENLİ C++ cast'idir. Derleyici derleme
    //     zamanında dönüşümün geçerli olup olmadığını kontrol eder. Geçerli değilse
    //     (örn. ilgisiz bir sınıfa pointer cast'i), sessiz UB değil DERLEME HATASI alırsınız.
    //     Sayısal türler arasında bilinçli dönüşüm yapacağınızda static_cast kullanın.
    int totalDistance = 15000; // meters
    int numSegments = 7;

    // EN: CLASSIC TRAP — Integer division truncates! When BOTH operands are int,
    //     C++ performs INTEGER division: 15000/7 = 2142 (not 2142.857...).
    //     The fractional part is silently discarded. This is one of the most
    //     common bugs in C/C++ code.
    // TR: KLASİK TUZAK — Tamsayı bölmesi kesir kısmını atar! İKİ operand da int
    //     olduğunda, C++ TAMSAYI bölmesi yapar: 15000/7 = 2142 (2142.857... değil).
    //     Kesirli kısım sessizce atılır. Bu, C/C++ kodundaki en yaygın hatalardan biridir.
    std::cout << "Integer div: " << totalDistance / numSegments << std::endl;

    // EN: SOLUTION — Cast at least ONE operand to double before division.
    //     This forces floating-point division and preserves the fractional part.
    //     `static_cast<double>(15000) / 7` → `15000.0 / 7` → `2142.857...`
    // TR: ÇÖZÜM — Bölmeden önce EN AZ BİR operandı double'a cast edin.
    //     Bu, kayan noktalı bölmeyi zorlar ve kesirli kısmı korur.
    //     `static_cast<double>(15000) / 7` → `15000.0 / 7` → `2142.857...`
    double avgSegment =
        static_cast<double>(totalDistance) / static_cast<double>(numSegments);
    std::cout << "static_cast div: " << avgSegment << " m/segment" << std::endl;

    // EN: Casting enum class to its underlying type. GearPosition::Sport has value 4.
    //     We cast to uint8_t (the underlying type we declared), then to int for printing.
    //     Why double cast? Because std::cout treats uint8_t as a character (ASCII 4),
    //     not as the number 4. The outer static_cast<int> converts it to a printable number.
    // TR: enum class'ı temel türüne dönüştürme. GearPosition::Sport değeri 4'tür.
    //     Önce uint8_t'ye (bildirdiğimiz alt tür), sonra yazdırma için int'e cast ediyoruz.
    //     Neden çift cast? Çünkü std::cout, uint8_t'yi sayı 4 olarak değil, karakter
    //     (ASCII 4) olarak işler. Dıştaki static_cast<int> onu yazdırılabilir sayıya çevirir.
    auto gearValue = static_cast<uint8_t>(GearPosition::Sport);
    std::cout << "GearPosition::Sport -> uint8_t: "
              << static_cast<int>(gearValue) << "\n" << std::endl;

    // ==========================================================================
    // 7. C-STYLE CAST (Avoid!) / C-TARZI CAST (Kaçının!)
    // ==========================================================================
    std::cout << "--- 7. C-Style Cast (Dangerous Legacy) ---" << std::endl;

    // EN: C-style cast `(int)x` is inherited from C. It looks simple, but it's the
    //     MOST DANGEROUS cast because the compiler tries MULTIPLE cast types in order:
    //         1. const_cast       (removes const — can cause UB if you write to it)
    //         2. static_cast      (normal type conversion)
    //         3. reinterpret_cast (raw bit reinterpretation — almost always UB)
    //     You have NO CONTROL over which one the compiler picks. It just picks the
    //     first one that compiles — even if the result is undefined behavior.
    //     In modern C++, ALWAYS use the named casts (static_cast, etc.) instead.
    //     Named casts are searchable with grep/IDE and make your INTENT explicit.
    // TR: C-tarzı cast `(int)x` C dilinden miras kalmıştır. Basit görünür ama
    //     EN TEHLİKELİ cast'tir çünkü derleyici SIRALI OLARAK birden fazla cast dener:
    //         1. const_cast       (const'ı kaldırır — yazarsanız UB olabilir)
    //         2. static_cast      (normal tür dönüşümü)
    //         3. reinterpret_cast (ham bit yeniden yorumlama — neredeyse her zaman UB)
    //     Derleyicinin hangisini seçtiği üzerinde HİÇBİR KONTROLÜNÜZ YOKTUR. Derlenen
    //     ilkini seçer — sonuç tanımsız davranış olsa bile.
    //     Modern C++'ta HER ZAMAN isimli cast'leri (static_cast, vb.) kullanın.
    //     İsimli cast'ler grep/IDE ile aranabilir ve NİYETİNİZİ açıkça belirtir.
    double sensorVoltage = 4.85;
    int rawADC = (int)sensorVoltage; // C-style: works but unsafe
    std::cout << "C-style (int)4.85 = " << rawADC
              << " (Compiles, but avoid in modern C++!)" << std::endl;

    return 0;
}
```

► 08. File Io And Streams

module_01_basics/08_file_io_and_streams.cpp

Konu: Basics: File I/O and Stream Operations — Temeller: Dosya G/Ç ve Akış İşlemleri

Teori / Theory

THEORY: C++ I/O Streams

C++ I/O is built on a stream hierarchy:
1. `std::cin` → Standard Input (keyboard).
2. `std::cout` → Standard Output (console).
3. `std::cerr` → Standard Error (unbuffered, for errors).
4. `std::clog` → Standard Log (buffered).
For File I/O, `<fstream>` provides:
1. `std::ofstream` → Write to file (Output File Stream).
2. `std::ifstream` → Read from file (Input File Stream).
3. `std::fstream` → Read AND Write.

TEORİ: C++ G/Ç Akışları

C++ G/Ç bir akış hiyerarşisi üzerine kurulmuştur:
1. `std::cin` → Standart Girdi (klavye).
2. `std::cout` → Standart Çıktı (konsol).
3. `std::cerr` → Standart Hata (tamponlanmamış, hatalar için).
4. `std::clog` → Standart Günlük (tamponlanmış).
Dosya G/Ç için `<fstream>` şunları sağlar:
1. `std::ofstream` → Dosyaya yaz (Çıktı Dosya Akışı).
2. `std::ifstream` → Dosyadan oku (Girdi Dosya Akışı).
3. `std::fstream` → Hem Oku hem de Yaz.

CPPREF Depth: RAII and File Handles

`std::ofstream` and `std::ifstream` use RAII: the file is automatically closed when the object goes out of scope (destructor runs). You do NOT need to call `.close()` explicitly if the object is about to be destroyed. Calling `.close()` is only needed if you want to reopen or reuse the stream.
The `is_open()` check is critical — opening a file can FAIL silently (bad path, permissions), and writing to a failed stream is Undefined Behavior.

CPPREF Derinlik: RAII ve Dosya Tanıtıcıları

`std::ofstream` ve `std::ifstream` RAII kullanır: nesne kapsam dışına çıktığında dosya otomatik kapatılır (yıkıcı çalışır). Nesne yok edilecekse `.close()` çağırmanıza GEREK YOKTUR. `.is_open()` kontrolü kritiktir — dosya açma sessizce başarısız olabilir.

→ Ref: https://en.cppreference.com/w/cpp/io/basic_ofstream
→ Ref: https://en.cppreference.com/w/cpp/io/basic_ifstream

Kaynak Kod / Source Code

```cpp
#include <fstream>
#include <iostream>
#include <string>

int main() {
  std::cout << "=== MODULE 1: FILE I/O & STREAMS ===\n" << std::endl;

  // ================================================================================
  // 1. CONSOLE OUTPUT FORMATTING / KONSOL ÇIKTI BİÇİMLENDİRME
  // ================================================================================
  std::cout << "--- 1. Console Output ---" << std::endl;

  // EN: BUFFERING — std::cout collects characters in an internal buffer and writes
  //     them to the screen in chunks (efficient). `std::endl` does TWO things:
  //         1. Inserts a newline character '\n'.
  //         2. FLUSHES the buffer — forces all pending data to the screen immediately.
  //     Flushing on every line is SLOW (each flush = a system call to the OS).
  //     Plain '\n' only inserts the newline WITHOUT flushing — the buffer is written
  //     when it's full or when the program ends. In performance-critical or embedded
  //     code, prefer '\n' and flush only when you truly need immediate output.
  // TR: TAMPONLAMA — std::cout karakterleri dahili bir tamponda toplar ve ekrana
  //     parçalar halinde yazar (verimli). `std::endl` İKİ şey yapar:
  //         1. '\n' yeni satır karakteri ekler.
  //         2. Tamponu TEMİZLER — bekleyen tüm veriyi hemen ekrana yazar.
  //     Her satırda temizleme YAVAŞTIR (her temizleme = OS'a sistem çağrısı).
  //     Düz '\n' SADECE yeni satır ekler, temizleme YAPMAZ — tampon dolduğunda
  //     veya program bittiğinde yazılır. Performans kritik/gömülü kodda '\n'
  //     tercih edin, yalnızca anında çıktı gerektiğinde flush yapın.
  std::cout << "Line 1 with endl" << std::endl; // Newline + flush (slow)
  std::cout << "Line 2 with \\n\n";              // Newline only, no flush (fast)

  // EN: std::cerr is UNBUFFERED — every character is written immediately, even
  //     if the program crashes right after. This is why errors go to cerr:
  //     if the program segfaults, buffered cout data might be LOST, but cerr
  //     output is guaranteed to have been written. cerr also writes to stderr
  //     (file descriptor 2), which can be redirected separately from stdout.
  //     Example: `./program > output.txt 2> errors.txt`
  // TR: std::cerr TAMPONLANMAMIŞTIR — her karakter hemen yazılır, program hemen
  //     ardından çökse bile. Hataların cerr'e gitmesinin nedeni budur:
  //     program segfault yaparsa, tamponlanmış cout verisi KAYBOLABİLİR, ama
  //     cerr çıktısının yazıldığı GARANTİDİR. cerr ayrıca stderr'e (dosya
  //     tanımlayıcı 2) yazar, stdout'tan ayrı yönlendirilebilir.
  //     Örnek: `./program > output.txt 2> errors.txt`
  std::cerr << "[ERROR] Simulated ECU fault (stderr)\n" << std::endl;

  // ================================================================================
  // 2. CONSOLE INPUT (std::cin) / KONSOL GİRDİSİ
  // ================================================================================
  std::cout << "--- 2. Console Input (std::cin) ---" << std::endl;

  // EN: std::cin reads from stdin (keyboard). The >> operator reads ONE TOKEN
  //     at a time, separated by whitespace (space, tab, newline).
  //     TRAP: If you type "John Doe", `cin >> name` reads only "John"!
  //     The " Doe" part stays in the buffer and pollutes the NEXT read.
  //
  //     Pattern for single value:
  //         int userRpm;
  //         std::cout << "Enter RPM: ";
  //         std::cin >> userRpm;
  //
  //     Pattern for full line (including spaces):
  //         std::string fullLine;
  //         std::getline(std::cin, fullLine);
  //
  //     COMMON BUG: Mixing >> and getline. After `cin >> x`, the '\n' remains
  //     in the buffer. The next `getline()` reads an EMPTY string!
  //     Fix: call `std::cin.ignore()` between >> and getline to discard '\n'.
  //
  // TR: std::cin stdin'den (klavye) okur. >> operatörü boşlukla (boşluk, tab,
  //     satır sonu) ayrılmış seferde TEK TOKEN okur.
  //     TUZAK: "John Doe" yazarsanız, `cin >> name` sadece "John" okur!
  //     " Doe" kısmı tamponda kalır ve SONRAKI okumayı kirletir.
  //
  //     Tek değer kalıbı:
```

```
  //         int userRpm;
  //         std::cout << "Enter RPM: ";
  //         std::cin >> userRpm;
  //
  //     Tam satır kalıbı (boşluk dahil):
  //         std::string fullLine;
  //         std::getline(std::cin, fullLine);
  //
  //     YAYGIN HATA: >> ile getline karıştırma. `cin >> x`'ten sonra '\n'
  //     tamponda kalır. Sonraki `getline()` BOŞ string okur!
  //     Çözüm: >> ile getline arasında `std::cin.ignore()` çağırarak '\n' atın.
  //
  std::cout << "[SKIPPED] Interactive cin disabled for automated build.\n"
            << std::endl;

  // ==========================================================================
  // 3. WRITE TO FILE (std::ofstream) / DOSYAYA YAZMA
  // ==========================================================================
  std::cout << "--- 3. Write to File (std::ofstream) ---" << std::endl;

  // EN: std::ofstream (Output File Stream) opens a file for WRITING.
  //     DEFAULT BEHAVIOR: If the file exists, it is ERASED and overwritten.
  //     If the file doesn't exist, it is CREATED. This is equivalent to
  //     opening with `std::ios::out | std::ios::trunc` (truncate).
  //
  //     The `{ }` braces create a SCOPE BLOCK. We put the ofstream inside it
  //     so that when we reach the closing `}`, the ofstream destructor runs
  //     and AUTOMATICALLY closes the file. This is RAII (Resource Acquisition
  //     Is Initialization) — the most important C++ idiom for resource safety.
  //     No need to call `.close()` manually — the destructor handles it.
  //
  // TR: std::ofstream (Çıkış Dosya Akışı) bir dosyayı YAZMA için açar.
  //     VARSAYILAN DAVRANIŞ: Dosya varsa SİLİNİR ve üzerine yazılır.
  //     Dosya yoksa OLUŞTURULUR. Bu, `std::ios::out | std::ios::trunc`
  //     (kesme) ile açmaya eşdeğerdir.
  //
  //     `{ }` süslü parantezler bir KAPSAM BLOĞU oluşturur. ofstream'i içine
  //     koyarız — kapanış `}` satırına ulaştığımızda ofstream yıkıcısı çalışır
  //     ve dosyayı OTOMATİK KAPATIR. Bu RAII'dir (Kaynak Edinimi Başlatmadır)
  //     — kaynak güvenliği için en önemli C++ deyimidir.
  //     `.close()` çağırmaya gerek yok — yıkıcı halleder.
  const std::string filename = "/tmp/ecu_dtc_log.txt";
  {
    std::ofstream outFile(filename);

    // EN: ALWAYS check is_open() after creating a file stream! Opening can
    //     FAIL SILENTLY for many reasons: bad path, no write permission,
    //     disk full, directory doesn't exist. If you write to a failed stream,
    //     the data simply VANISHES — no crash, no error, just silent data loss.
    // TR: Dosya akışı oluşturduktan sonra HER ZAMAN is_open() kontrol edin!
    //     Açma birçok nedenle SESSİZCE BAŞARISız olabilir: geçersiz yol, yazma
    //     izni yok, disk dolu, dizin mevcut değil. Başarısız akışa yazarsanız
    //     veri sessizce KAYBOLUR — çökme yok, hata yok, sadece sessiz veri kaybı.
    if (!outFile.is_open()) {
      std::cerr << "[ERROR] Cannot open file for writing: " << filename
                << std::endl;
      return 1;
    }

    // EN: The << operator works exactly like std::cout — because both inherit
    //     from std::ostream. Anything you can print to console, you can write
    //     to a file with the same syntax.
    // TR: << operatörü aynen std::cout gibi çalışır — çünkü ikisi de
    //     std::ostream'den türer. Konsola yazdırabildiğiniz her şeyi aynı
    //     sözdizimi ile dosyaya yazabilirsiniz.
    outFile << "=== ECU DTC LOG ===" << std::endl;
    outFile << "DTC: P0300 — Random/Multiple Cylinder Misfire Detected"
            << std::endl;
    outFile << "DTC: P0171 — System Too Lean (Bank 1)" << std::endl;
    outFile << "DTC: P0420 — Catalyst Efficiency Below Threshold" << std::endl;
    outFile << "DTC: P0442 — EVAP System Leak Detected (Small)" << std::endl;
    outFile << "Total DTCs: 4" << std::endl;

    std::cout << "Written 4 DTCs to: " << filename << std::endl;

```

```
    // EN: When this closing brace `}` is reached, `outFile` goes out of scope.
    //     Its destructor runs → calls close() → flushes buffer → releases file handle.
    //     This is RAII in action: you CANNOT forget to close the file.
    // TR: Bu kapanış süslü parantezine `}` ulaşıldığında `outFile` kapsam dışına çıkar.
    //     Yıkıcısı çalışır → close() çağırır → tamponu temizler → dosya tanıtıcısını serbest bırakır.
    //     Bu RAII'nin uygulamasıdır: dosyayı kapatmayı UNUTAMAZSINIZ.
  }

  // ═══════════════════════════════════════════════════════════════════════════════
  // 4. READ FROM FILE (std::ifstream) / DOSYADAN OKUMA
  // ═══════════════════════════════════════════════════════════════════════════════
  std::cout << "\n--- 4. Read from File (std::ifstream) ---" << std::endl;

  {
    // EN: std::ifstream (Input File Stream) opens a file for READING.
    //     If the file doesn't exist, is_open() returns false — no exception thrown.
    // TR: std::ifstream (Girdi Dosya Akışı) bir dosyayı OKUMA için açar.
    //     Dosya yoksa is_open() false döner — istisna fırlatılmaz.
    std::ifstream inFile(filename);

    if (!inFile.is_open()) {
      std::cerr << "[ERROR] Cannot open file for reading: " << filename
                << std::endl;
      return 1;
    }

    // EN: std::getline(stream, string) reads characters until it finds '\n'.
    //     It stores everything EXCEPT the '\n' into `line`, then discards '\n'.
    //     When the stream reaches EOF (End Of File), getline returns false
    //     and the while loop stops.
    //
    //     WHY getline INSTEAD OF >>?
    //     `inFile >> word` reads ONE WORD (stops at whitespace).
    //     `getline(inFile, line)` reads the ENTIRE LINE (including spaces).
    //     For log files, config files, CSV — you almost always want getline.
    //
    // TR: std::getline(akış, string) '\n' bulana kadar karakterleri okur.
    //     '\n' HARİÇ her şeyi `line`'a kaydeder, sonra '\n'i atar.
    //     Akış EOF'a (Dosya Sonu) ulaşınca getline false döner ve while döngüsü durur.
    //
    //     NEDEN >> YERİNE getline?
    //     `inFile >> word` TEK KELİME okur (boşlukta durur).
    //     `getline(inFile, line)` TÜM SATIRI okur (boşluklar dahil).
    //     Log, config, CSV dosyaları için neredeyse her zaman getline istersiniz.
    std::string line;
    int lineNum = 1;
    while (std::getline(inFile, line)) {
      std::cout << "  [L" << lineNum << "] " << line << std::endl;
      ++lineNum;
    }

    std::cout << "Read " << (lineNum - 1) << " lines from: " << filename
              << std::endl;
  }

  // ═══════════════════════════════════════════════════════════════════════════════
  // 5. APPEND TO FILE / DOSYAYA EKLEME
  // ═══════════════════════════════════════════════════════════════════════════════
  std::cout << "\n--- 5. Append to File ---" << std::endl;

  {
```

```
    // EN: The second parameter controls the OPEN MODE. Important modes:
    //     std::ios::out   → Write (default for ofstream). ERASES existing content.
    //     std::ios::app   → Append. Writes are always added to the END of the file.
    //                       Existing content is PRESERVED. File is created if absent.
    //     std::ios::ate   → Open and seek to End, but you CAN seek back (unlike app).
    //     std::ios::in    → Read (default for ifstream).
    //     std::ios::binary→ Binary mode (no text translation of \n ↔ \r\n).
    //     Modes can be combined with `|`: `std::ios::out | std::ios::binary`
    //
    //     KEY DIFFERENCE — app vs ate:
    //     app: Every write goes to EOF. Even if you seekp() backward, the write
    //          still appends. This is SAFE for concurrent logging.
    //     ate: Opens at EOF, but you CAN seekp() to any position and overwrite.
    //
    // TR: İkinci parametre AÇMA MODUNU kontrol eder. Önemli modlar:
    //     std::ios::out   → Yaz (ofstream varsayılanı). Mevcut içeriği SİLER.
    //     std::ios::app   → Ekle. Yazılar her zaman dosya SONUNA eklenir.
    //                       Mevcut içerik KORUNUR. Dosya yoksa oluşturulur.
    //     std::ios::ate   → Aç ve Sona git, ama geri seek YAPABİLİRSİNİZ (app'ten farkı).
    //     std::ios::in    → Oku (ifstream varsayılanı).
    //     std::ios::binary→ İkili mod ('\n' ↔ '\r\n' metin çevrimi yok).
    //     Modlar `|` ile birleştirilebilir: `std::ios::out | std::ios::binary`
    //
    //
    //     KRİTİK FARK — app vs ate:
    //     app: Her yazma EOF'a gider. seekp() geriye gitseler bile yazma yine ekler.
    //          Eşzamanlı loglama için GÜVENLİDİR.
    //     ate: EOF'ta açılır ama seekp() ile herhangi bir konuma GİDİP üzerine yazabilirsiniz.
    std::ofstream appendFile(filename, std::ios::app);

    if (!appendFile.is_open()) {
      std::cerr << "[ERROR] Cannot open file for appending." << std::endl;
      return 1;
    }

    appendFile << "DTC: P0128 — Coolant Temperature Below Thermostat Range"
               << std::endl;
    appendFile << "Total DTCs: 5 (updated)" << std::endl;

    std::cout << "Appended 1 more DTC to: " << filename << std::endl;
  }

  // ═══════════════════════════════════════════════════════════════════════════════
  // 6. RE-READ TO VERIFY APPEND / EKLEME DOĞRULAMA İÇİN TEKRAR OKU
  // ═══════════════════════════════════════════════════════════════════════════════
  std::cout << "\n--- 6. Verify Append ---" << std::endl;

  // EN: Re-read the entire file to confirm that Section 5's append worked.
  //     The file should now contain the original 4 DTCs PLUS the 1 we appended.
  //     This pattern (write → close → re-read) is common in testing/validation.
  // TR: Bölüm 5'teki eklemenin çalıştığını doğrulamak için dosyanın tamamını tekrar oku.
  //     Dosya artık orijinal 4 DTC ARTI eklediğimiz 1 DTC'yi içermelidir.
  //     Bu kalıp (yaz → kapat → tekrar oku) test/doğrulamada yaygındır.
  {
    std::ifstream verifyFile(filename);
    if (verifyFile.is_open()) {
      std::string line;
      while (std::getline(verifyFile, line)) {
        std::cout << "  " << line << std::endl;
      }
    }
  }

  // ═══════════════════════════════════════════════════════════════════════════════
  // 7. FILE STATUS CHECKS / DOSYA DURUM KONTROLLERİ
  // ═══════════════════════════════════════════════════════════════════════════════
  std::cout << "\n--- 7. File Status Checks ---" << std::endl;

```

```
279     // EN: Every stream has a set of STATE FLAGS you can query:
280     //     good()    → True if NO errors at all. The stream is ready for I/O.
281     //     fail()    → True if a logical error occurred (wrong type read, open failed).
282     //                 Also true when badbit is set.
283     //     bad()     → True if an IRRECOVERABLE I/O error occurred (disk failure).
284     //     eof()     → True if the stream reached End Of File.
285     //     is_open() → True if the underlying file was successfully associated.
286     //
287     //     A stream used in a boolean context (e.g., `if (inFile)`) checks !fail().
288     //     This is why `while (std::getline(inFile, line))` stops at EOF — getline
289     //     sets eofbit on the stream, which makes fail() return true.
290     //
291     // TR: Her akışın sorgulayabileceğiniz DURUM BAYRAKLARI vardır:
292     //     good()    → Hiç hata yoksa true. Akış G/Ç'ye hazır.
293     //     fail()    → Mantıksal hata oluştuysa true (yanlış tip okundu, açma başarısız).
294     //                 badbit ayarlıysa da true döner.
295     //     bad()     → KURTARILAMAZ G/Ç hatası oluştuysa true (disk arızası).
296     //     eof()     → Akış Dosya Sonuna ulaştığında true.
297     //     is_open() → Alttaki dosya başarıyla ilişkilendirildiyse true.
298     //
299     //     Boolean bağlamda bir akış (ör. `if (inFile)`) !fail() kontrol eder.
300     //     `while (std::getline(inFile, line))` EOF'ta durmasının nedeni budur —
301     //     getline akışta eofbit ayarlar, bu fail()'in true dönmesini sağlar.
302
303     // EN: Test with a file that does NOT exist:
304     // TR: VAR OLMAYAN bir dosya ile test:
305     std::ifstream badFile("/tmp/non_existent_file_xyz.txt");
306     std::cout << "Bad file is_open(): " << badFile.is_open() << std::endl;
307     std::cout << "Bad file good()   : " << badFile.good() << std::endl;
308     std::cout << "Bad file fail()   : " << badFile.fail() << std::endl;
309
310     // EN: Test with a file that DOES exist (we created it in Section 3):
311     // TR: VAR OLAN bir dosya ile test (Bölüm 3'te oluşturduk):
312     std::ifstream goodFile(filename);
313     std::cout << "Good file is_open(): " << goodFile.is_open() << std::endl;
314     std::cout << "Good file good()   : " << goodFile.good() << std::endl;
315
316     return 0;
317 }
```

► 09. Scope Namespace Recursion

module_01_basics/09_scope_namespace_recursion.cpp

Konu: Basics: Scope, Lifetime, Namespace & Recursion — Temeller: Kapsam, Ömür, İsim Alanı ve

Teori / Theory

THEORY: Scope & Lifetime

Variables live in "scopes" — regions delimited by `{ }`. A variable declared inside a block is INVISIBLE outside. When the closing `}` is reached, the variable is DESTROYED (stack unwinding). Key scopes:
1. Global scope: Declared outside all functions. Lives for the whole program.
2. Function/Local scope: Inside a function or block `{ }`.
3. `static` local: Initialized once, persists across calls (like a global but scoped to the function). Useful for caching sensor calibration data.

TEORİ: Kapsam ve Ömür

Değişkenler "kapsam" (scope) içinde yaşar — `{ }` ile sınırlanır. Bir blok içinde tanımlanan değişken dışarıda GÖRÜNMEZdir. Kapanış `}` e ulaşıldığında yok edilir. Anahtar kapsamlar:
1. Global kapsam: Tüm fonksiyonların dışında. Program boyunca yaşar.
2. Fonksiyon/Yerel kapsam: Fonksiyon veya blok `{ }` içinde.
3. `static` yerel: Bir kez başlatılır, çağrılar arasında kalıcıdır.

THEORY: Namespaces

Namespaces prevent name collisions in large codebases. Without them, two libraries defining `init()` would cause a linker error. Use `::` to access members. `using namespace std;` pollutes the global scope — avoid it.

TEORİ: İsim Alanları

İsim alanları büyük kod tabanlarında isim çakışmalarını önler. `using namespace std;` global kapsamı kirletir — kaçının.

CPPREF Depth: Recursion & Stack Overflow

Recursion means a function calls ITSELF. Every call pushes a new frame onto the call stack. Without a proper "base case", the stack overflows (SIGSEGV). Stack size is typically 1-8 MB. For deeply recursive algorithms, prefer iterative solutions or tail-call optimization hints.

CPPREF Derinlik: Özyineleme ve Yığın Taşması

Özyineleme bir fonksiyonun KENDİSİNİ çağırması demektir. Her çağrı çağrı yığınına yeni bir çerçeve ekler. Uygun bir "temel durum" (base case) olmadan yığın taşar (SIGSEGV). Yığın boyutu genellikle 1-8 MB'tır.

→ Ref: https://en.cppreference.com/w/cpp/language/scope
→ Ref: https://en.cppreference.com/w/cpp/language/namespace

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <string>

// ═══════════════════════════════════════════════════════════════════════════════
// 1. GLOBAL SCOPE VARIABLE / GLOBAL DEĞİŞKEN
// ═══════════════════════════════════════════════════════════════════════════════
// EN: Global variables are declared OUTSIDE all functions. They live in the DATA segment
//     of the process memory (not on the stack). Their lifetime = entire program duration.
//     They are ZERO-INITIALIZED by default (unlike local variables which are GARBAGE).
//
//     DANGER: In multi-file projects, the initialization ORDER of globals across
//     different .cpp files is UNDEFINED ("Static Initialization Order Fiasco").
//     File A's global might use File B's global before B is initialized → UB.
//     Solution: Use function-local statics ("Meyers Singleton" idiom) instead.
//
//     `const` globals are fine because they are immutable and typically optimized
//     into read-only memory by the compiler.
//
// TR: Global değişkenler tüm fonksiyonların DIŞINDA tanımlanır. Proses belleğinin DATA
//     segmentinde yaşarlar (yığında değil). Ömürleri = tüm program süresi.
//     Varsayılan olarak SIFIR BAŞLATILIRLAR (yerel değişkenler ÇÖP olan değil).
//
//     TEHLİKE: Çok dosyalı projelerde, farklı .cpp dosyalarındaki globallerin
//     başlatma SIRASI TANIMSIZDIR ("Statik Başlatma Sırası Felaketi").
//     A dosyasının global'i, B başlatılmadan B'nin global'ini kullanabilir → UB.
//     Çözüm: Bunun yerine fonksiyon-yerel static kullanın ("Meyers Singleton" deyimi).
//
//     `const` globaller güvenlidir çünkü değiştirilemezler ve genellikle derleyici
//     tarafından salt okunur belleğe optimize edilirler.
const std::string FIRMWARE_VERSION = "v3.2.1";


// ═══════════════════════════════════════════════════════════════════════════════
// 2. NAMESPACES / İSİM ALANLARI
// ═══════════════════════════════════════════════════════════════════════════════
// EN: Namespaces group related code under a unique name to prevent collisions.
//     Without namespaces, if Powertrain and BodyControl both define `init()`,
//     the linker would see TWO definitions of the same function → linker error.
//     With namespaces, they become `Powertrain::init()` and `BodyControl::init()`
//     — two completely separate functions.
//
//     The `::` operator is called the SCOPE RESOLUTION OPERATOR. It tells the
//     compiler exactly which namespace's function you mean.
//
//     WHY `using namespace std;` IS DANGEROUS:
//     `std` contains thousands of names (min, max, count, distance, move, swap...).
//     `using namespace std;` dumps ALL of them into your scope. If you define
//     your own `count` variable, it silently collides with `std::count`.
//     In header files, this is ESPECIALLY catastrophic — every file that #includes
//     your header inherits the pollution. ALWAYS use `std::` prefix instead.
//
// TR: İsim alanları ilişkili kodu çakışmayı önlemek için benzersiz bir isim altında gruplar.
//     İsim alanları olmadan, Powertrain ve BodyControl ikisi de `init()` tanımlarsa,
//     bağlayıcı aynı fonksiyonun İKİ tanımını görür → bağlayıcı hatası.
//     İsim alanlarıyla `Powertrain::init()` ve `BodyControl::init()` olurlar
//     — iki tamamen ayrı fonksiyon.
//
//     `::` operatörüne KAPSAM ÇÖZÜMLEME OPERATÖRÜ denir. Derleyiciye hangi
//     isim alanının fonksiyonunu kastettiğinizi söyler.
//
//     NEDEN `using namespace std;` TEHLİKELİDİR:
//     `std` binlerce isim içerir (min, max, count, distance, move, swap...).
//     `using namespace std;` HEPSİNİ kapsamınıza döker. Kendi `count`
//     değişkeninizi tanımlarsanız `std::count` ile sessizce çakışır.
//     Başlık dosyalarında bu ÖZELLİKLE FELAKETTİR — başlığınızı #include
//     eden her dosya kirliliği miras alır. HER ZAMAN `std::` öneki kullanın.
namespace Powertrain {
void init() {
  std::cout << "  [Powertrain] Engine management initialized." << std::endl;
}
int getStatus() { return 1; } // 1 = OK
} // namespace Powertrain
```

```cpp

namespace BodyControl {
void init() {
  std::cout << "  [BodyControl] Door/window/lighting initialized." << std::endl;
}
int getStatus() { return 1; } // 1 = OK
} // namespace BodyControl

// EN: Nested namespace — C++17 shorthand syntax. Before C++17 you had to write:
//         namespace Vehicle { namespace Diagnostics { ... } }
//     C++17 allows the compact `Vehicle::Diagnostics` form.
//     Nested namespaces are common in large projects: `company::product::module`.
// TR: İç içe isim alanı — C++17 kısa sözdizimi. C++17 öncesi şöyle yazmanız gerekirdi:
//         namespace Vehicle { namespace Diagnostics { ... } }
//     C++17 kompakt `Vehicle::Diagnostics` biçimine izin verir.
//     İç içe isim alanları büyük projelerde yaygındır: `sirket::urun::modul`.
namespace Vehicle::Diagnostics {
void scanAll() {
  std::cout << "  [Vehicle::Diagnostics] Full OBD-II scan complete."
            << std::endl;
}
} // namespace Vehicle::Diagnostics

// ═══════════════════════════════════════════════════════════════════════════════
// 3. STATIC LOCAL VARIABLE / STATİK YEREL DEĞİŞKEN
// ═══════════════════════════════════════════════════════════════════════════════
// EN: A function-local `static` variable combines two properties:
//     1. LOCAL SCOPE: Only visible inside this function (encapsulated).
//     2. STATIC LIFETIME: Initialized the FIRST time the function is called,
//        then PERSISTS in memory until the program ends (not destroyed on return).
//     It lives in the DATA segment (like a global), but its NAME is only
//     visible inside the function (unlike a global).
//
//     C++11 GUARANTEE: Initialization of function-local statics is THREAD-SAFE.
//     If two threads call readSensor() simultaneously for the first time, only
//     ONE thread initializes readCount — the other waits. ("Magic statics").
//
//     Use cases: call counters, lazy-initialized caches, Meyers Singleton.
//
// TR: Fonksiyon-yerel `static` değişken iki özelliği birleştirir:
//     1. YEREL KAPSAM: Sadece bu fonksiyonun içinde görünür (kapsüllenmiş).
//     2. STATİK ÖMÜR: Fonksiyon İLK çağrıldığında başlatılır, sonra program
//        bitene kadar bellekte KALIR (dönüşte yok edilmez).
//     DATA segmentinde yaşar (global gibi), ama İSMİ sadece fonksiyon
//     içinde görünür (globalden farklı olarak).
//
//     C++11 GARANTİSİ: Fonksiyon-yerel static'lerin başlatılması İŞ PARÇACIĞI
//     GÜVENLİDİR. İki iş parçacığı aynı anda ilk kez readSensor() çağırırsa,
//     sadece BİRİ readCount'u başlatır — diğeri bekler. ("Sihirli static'ler").
//
//     Kullanım alanları: çağrı sayaçları, tembel başlatılan önbellekler, Meyers Singleton.
void readSensor() {
  // EN: `static int readCount = 0;` — this line executes ONLY ONCE (first call).
  //     On subsequent calls, it is SKIPPED. readCount retains its value from
  //     the previous call. Without `static`, readCount would be 0 every time.
  // TR: `static int readCount = 0;` — bu satır SADECE BİR KEZ çalışır (ilk çağrı).
  //     Sonraki çağrılarda ATLANIR. readCount önceki çağrıdaki değerini korur.
  //     `static` olmadan readCount her seferinde 0 olurdu.
  static int readCount = 0;
  ++readCount;
  std::cout << "  Sensor read #" << readCount << " (static counter)"
            << std::endl;
}
```

```

// ═══════════════════════════════════════════════════════════════════════════════
// 4. RECURSION / ÖZYİNELEME
// ═══════════════════════════════════════════════════════════════════════════════
// EN: Factorial — the "hello world" of recursion.
//     factorial(5) → 5 × factorial(4) → 5 × 4 × factorial(3) → ... → 5×4×3×2×1 = 120
//
//     Every recursive function MUST have:
//     1. BASE CASE: The condition that STOPS recursion (here: n <= 1 → return 1).
//        Without it, the function calls itself forever → stack overflow (SIGSEGV).
//     2. RECURSIVE CASE: The call that makes the problem SMALLER (here: n-1).
//        Each call MUST move toward the base case, otherwise → infinite recursion.
//
//     HOW IT WORKS IN MEMORY:
//     Each call pushes a new STACK FRAME containing: parameters, local variables,
//     and the return address. factorial(5) creates 5 frames on the stack.
//     When the base case returns, frames are POPPED one by one (unwinding),
//     multiplying the results back up the chain.
//
// TR: Faktöriyel — özyinelemenin "merhaba dünya"sı.
//     factorial(5) → 5 × factorial(4) → 5 × 4 × factorial(3) → ... → 5×4×3×2×1 = 120
//
//     Her özyinelemeli fonksiyonun OLMASI GEREKENLER:
//     1. TEMEL DURUM: Özyinelemeyi DURDURAN koşul (burada: n <= 1 → return 1).
//        Olmadan fonksiyon sonsuza dek kendini çağırır → yığın taşması (SIGSEGV).
//     2. ÖZYİNELEMELİ DURUM: Problemi KÜÇÜLTEN çağrı (burada: n-1).
//        Her çağrı temel duruma YAKLAŞMALIDIR, yoksa → sonsuz özyineleme.
//
//     BELLEKTE NASIL ÇALIŞIR:
//     Her çağrı yeni bir YIĞIN ÇERÇEVESİ ekler: parametreler, yerel değişkenler
//     ve dönüş adresi. factorial(5) yığında 5 çerçeve oluşturur.
//     Temel durum döndüğünde, çerçeveler birer birer ÇIKARILIR (geri sarma),
//     sonuçları zincirde yukarı çarparak birleştirir.
int factorial(int n) {
  if (n <= 1) return 1;       // Base case / Temel durum
  return n * factorial(n - 1); // Recursive case / Özyinelemeli durum
}

// EN: Fibonacci — demonstrates the DANGER of naive recursion.
//     Sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55...
//     Each number = sum of the two before it: fib(n) = fib(n-1) + fib(n-2).
//
//     PROBLEM: This implementation has O(2^n) time complexity!
//     fibonacci(40) makes over 200 MILLION calls. fibonacci(6) alone creates
//     this call tree:  fib(6) → fib(5) + fib(4)
//                      fib(5) → fib(4) + fib(3)    ← fib(4) computed AGAIN!
//     The same sub-problems are recalculated exponentially many times.
//
//     SOLUTIONS (covered in later modules):
//     1. Memoization: Cache results in a map/array → O(n) time, O(n) space.
//     2. Iterative: Use a simple loop with two variables → O(n) time, O(1) space.
//     3. Dynamic Programming: Bottom-up table filling.
//
// TR: Fibonacci — naif özyinelemenin TEHLİKESİNİ gösterir.
//     Dizi: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55...
//     Her sayı = önceki ikisinin toplamı: fib(n) = fib(n-1) + fib(n-2).
//
//     PROBLEM: Bu uygulama O(2^n) zaman karmaşıklığına sahiptir!
//     fibonacci(40) 200 MİLYONDAN fazla çağrı yapar. fibonacci(6) bile
//     şu çağrı ağacını oluşturur: fib(6) → fib(5) + fib(4)
//                                 fib(5) → fib(4) + fib(3) ← fib(4) TEKRAR hesaplanıyor!
//     Aynı alt problemler üssel olarak defalarca yeniden hesaplanır.
//
//     ÇÖZÜMLER (sonraki modüllerde):
//     1. Memoization: Sonuçları map/dizide önbelleğe al → O(n) zaman, O(n) alan.
//     2. İteratif: İki değişkenli basit döngü → O(n) zaman, O(1) alan.
//     3. Dinamik Programlama: Aşağıdan yukarıya tablo doldurma.
int fibonacci(int n) {
  if (n <= 0) return 0;  // Base case 1
  if (n == 1) return 1;  // Base case 2
  return fibonacci(n - 1) + fibonacci(n - 2);  // Two recursive calls!
}
```

```cpp

// EN: TAIL RECURSION example — the recursive call is the LAST operation.
//     Some compilers can optimize this into a loop (no extra stack frames).
//     This eliminates stack overflow risk for deep recursion.
//     GCC with -O2 can perform Tail Call Optimization (TCO) on this pattern.
// TR: KUYRUK ÖZYİNELEME örneği — özyinelemeli çağrı SON işlemdir.
//     Bazı derleyiciler bunu döngüye optimize edebilir (ekstra yığın çerçevesi yok).
//     Bu, derin özyinelemede yığın taşması riskini ortadan kaldırır.
//     GCC -O2 ile bu kalıpta Kuyruk Çağrı Optimizasyonu (TCO) yapabilir.
void shutdownCountdown(int n) {
  if (n < 0) return;  // Base case: countdown finished
  std::cout << "  Shutdown in: " << n << "..." << std::endl;
  shutdownCountdown(n - 1);  // Tail call: nothing happens after this returns
}

int main() {
  std::cout << "=== MODULE 1: SCOPE, NAMESPACE & RECURSION ===\n" << std::endl;

  // ═══════════════════════════════════════════════════════════════════════════
  // 1. SCOPE DEMONSTRATION / KAPSAM GÖSTERİMİ
  // ═══════════════════════════════════════════════════════════════════════════
  std::cout << "--- 1. Scope & Lifetime ---" << std::endl;

  // EN: Global variables are accessible from ANY function in this file.
  //     FIRMWARE_VERSION was declared at file scope (line ~65) and lives
  //     until the program terminates. It's `const`, so it cannot be modified.
  // TR: Global değişkenler bu dosyadaki HERHANGİ bir fonksiyondan erişilebilir.
  //     FIRMWARE_VERSION dosya kapsamında (~65. satır) tanımlandı ve program
  //     sonlanana kadar yaşar. `const` olduğu için değiştirilemez.
  std::cout << "Global firmware version: " << FIRMWARE_VERSION << std::endl;

  {
    // EN: The `{ }` braces create a NEW SCOPE (block scope). Any variable
    //     declared here is BORN at this point and DIES at the closing `}`.
    //     The compiler DEALLOCATES the stack space automatically.
    //     This is useful for limiting variable lifetime and freeing resources early.
    // TR: `{ }` süslü parantezler YENİ BİR KAPSAM (blok kapsamı) oluşturur.
    //     Burada tanımlanan her değişken bu noktada DOĞAR ve kapanış `}` da ÖLÜR.
    //     Derleyici yığın alanını otomatik olarak serbest bırakır.
    //     Değişken ömrünü sınırlamak ve kaynakları erken serbest bırakmak için kullanışlıdır.
    int localRpm = 3500;
    std::cout << "Inside block: localRpm = " << localRpm << std::endl;
  }
  // EN: localRpm is DESTROYED here. Trying to use it would cause:
  //         error: 'localRpm' was not declared in this scope
  //     This is NOT a runtime error — it's a COMPILE-TIME error. The compiler
  //     enforces scope rules and prevents you from accessing dead variables.
  // TR: localRpm burada YOK EDİLDİ. Kullanmaya çalışmak şu hatayı verir:
  //         error: 'localRpm' was not declared in this scope
  //     Bu çalışma zamanı hatası DEĞİLDİR — DERLEME ZAMANI hatasıdır. Derleyici
  //     kapsam kurallarını uygular ve ölü değişkenlere erişmenizi engeller.

  // EN: VARIABLE SHADOWING — when an inner scope declares a variable with the
  //     same name as an outer scope variable, the inner one "shadows" (hides) it.
  //     The outer variable still exists but is UNREACHABLE by that name.
  //     Our -Wshadow flag warns about this, so we use different names here.
  //     In real code, shadowing is a common source of subtle bugs.
  // TR: DEĞİŞKEN GÖLGELEME — iç kapsam, dış kapsamdaki bir değişkenle aynı
  //     isimde değişken tanımladığında, içteki dıştakini "gölgeler" (gizler).
  //     Dış değişken hâlâ var ama o isimle ERİŞİLEMEZ.
  //     -Wshadow bayrağımız bunu uyarır, bu yüzden burada farklı isimler kullanıyoruz.
  //     Gerçek kodda, gölgeleme ince hataların yaygın bir kaynağıdır.
  int outerTemp = 25;
  std::cout << "Outer temp: " << outerTemp << std::endl;
  {
    int innerTemp = 90;
    std::cout << "Inner temp (different scope): " << innerTemp << std::endl;
  }
  std::cout << "Outer temp unchanged: " << outerTemp << "\n" << std::endl;
```

```cpp

    // ═══════════════════════════════════════════════════════════════════════════════
    // 2. NAMESPACE DEMO / İSİM ALANI GÖSTERİMİ
    // ═══════════════════════════════════════════════════════════════════════════════
    std::cout << "--- 2. Namespaces ---" << std::endl;

    // EN: Both Powertrain and BodyControl define init(), but there's NO collision
    //     because the full names are Powertrain::init() and BodyControl::init().
    //     Think of namespaces like folders on a filesystem: you can have
    //     /Powertrain/init and /BodyControl/init without conflict.
    // TR: Hem Powertrain hem BodyControl init() tanımlıyor, ama ÇAKIŞMA YOK
    //     çünkü tam isimleri Powertrain::init() ve BodyControl::init().
    //     İsim alanlarını dosya sistemindeki klasörler gibi düşünün:
    //     /Powertrain/init ve /BodyControl/init çakışmadan var olabilir.
    Powertrain::init();
    BodyControl::init();

    // EN: Nested namespace accessed with double :: — Vehicle::Diagnostics::scanAll().
    //     In real automotive software, you might see: oem::vehicle::diag::readDTC().
    // TR: İç içe isim alanına çift :: ile erişilir — Vehicle::Diagnostics::scanAll().
    //     Gerçek otomotiv yazılımında görebilirsiniz: oem::vehicle::diag::readDTC().
    Vehicle::Diagnostics::scanAll();

    std::cout << "Powertrain status: " << Powertrain::getStatus() << std::endl;
    std::cout << "BodyControl status: " << BodyControl::getStatus() << "\n"
              << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // 3. STATIC LOCAL VARIABLE / STATİK YEREL DEĞİŞKEN
    // ═══════════════════════════════════════════════════════════════════════════════
    std::cout << "--- 3. Static Local Variable ---" << std::endl;

    // EN: Calling readSensor() 3 times. Watch the counter INCREMENT each time:
    //     Call 1 → readCount becomes 1 (initialized from 0)
    //     Call 2 → readCount becomes 2 (NOT re-initialized, just incremented)
    //     Call 3 → readCount becomes 3
    //     If readCount were a normal (non-static) local, it would be 1 every call.
    // TR: readSensor()'u 3 kez çağırıyoruz. Sayacın her seferinde ARTTIĞINI izleyin:
    //     Çağrı 1 → readCount 1 olur (0'dan başlatıldı)
    //     Çağrı 2 → readCount 2 olur (yeniden başlatılMAZ, sadece artırılır)
    //     Çağrı 3 → readCount 3 olur
    //     readCount normal (statik olmayan) yerel olsaydı, her çağrıda 1 olurdu.
    readSensor(); // #1
    readSensor(); // #2
    readSensor(); // #3
    std::cout << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // 4. RECURSION DEMOS / ÖZYİNELEME GÖSTERİMLERİ
    // ═══════════════════════════════════════════════════════════════════════════════
    std::cout << "--- 4. Recursion ---" << std::endl;

    // EN: FACTORIAL — each call multiplies n by the result of factorial(n-1).
    //     Call stack for factorial(5):
    //         factorial(5) waits for factorial(4)
    //         factorial(4) waits for factorial(3)
    //         factorial(3) waits for factorial(2)
    //         factorial(2) waits for factorial(1)
    //         factorial(1) → returns 1 (BASE CASE HIT!)
    //     Then unwinding: 1 → 2×1=2 → 3×2=6 → 4×6=24 → 5×24=120
    // TR: FAKTÖRİYEL — her çağrı n'i factorial(n-1) sonucuyla çarpar.
    //     factorial(5) için çağrı yığını:
    //         factorial(5) factorial(4)'ü bekler
    //         factorial(4) factorial(3)'ü bekler
    //         ... → factorial(1) → 1 döner (TEMEL DURUM!)
    //     Sonra geri sarma: 1 → 2×1=2 → 3×2=6 → 4×6=24 → 5×24=120
    std::cout << "factorial(5) = " << factorial(5) << std::endl;   // 120
    std::cout << "factorial(10) = " << factorial(10) << std::endl; // 3628800

```

```
    // EN: FIBONACCI — demonstrates exponential blowup of naive recursion.
    //     fibonacci(10) = 55, but it takes 177 function calls to compute.
    //     fibonacci(30) would take over 2 MILLION calls. In production code,
    //     NEVER use this naive approach — use iteration or memoization.
    // TR: FIBONACCI — naif özyinelemenin üssel patlamasını gösterir.
    //     fibonacci(10) = 55, ama hesaplamak 177 fonksiyon çağrısı sürer.
    //     fibonacci(30) 2 MİLYONDAN fazla çağrı yapar. Üretim kodunda
    //     bu naif yaklaşımı ASLA kullanmayın — iterasyon veya memoization kullanın.
    std::cout << "fibonacci(6) = " << fibonacci(6) << std::endl;   // 8
    std::cout << "fibonacci(10) = " << fibonacci(10) << std::endl; // 55

    // EN: COUNTDOWN — practical recursion: each call prints n, then calls (n-1).
    //     This is a TAIL RECURSIVE pattern because the recursive call is the
    //     absolute last thing the function does (no computation after it returns).
    // TR: GERİ SAYIM — pratik özyineleme: her çağrı n'i yazdırır, sonra (n-1) çağırır.
    //     Bu KUYRUK ÖZYİNELEME kalıbıdır çünkü özyinelemeli çağrı fonksiyonun
    //     yaptığı mutlak SON şeydir (döndükten sonra hesaplama yok).
    std::cout << "\nECU shutdown sequence:" << std::endl;
    shutdownCountdown(3);

    return 0;
}
```

► 10. Filesystem And Binary Io

module_01_basics/10_filesystem_and_binary_io.cpp

Konu: std::filesystem & Binary I/O — Dosya Sistemi ve İkili G/Ç

Teori / Theory

THEORY: std::filesystem (C++17)

C++17 introduced <filesystem> — a complete, portable, cross-platform API for file and directory operations.
No more platform-specific code!
Key types:

```
┌──────────────────────┬──────────────────────────────────────┐
│ Type                 │ Purpose                              │
├──────────────────────┼──────────────────────────────────────┤
│ std::filesystem::path  │ File/dir path object (operator/, concat)  │
│ directory_entry        │ A single directory item (file, dir, link) │
│ directory_iterator     │ Iterate flat directory                    │
│ recursive_dir_iterator │ Iterate all subdirectories recursively    │
│ file_status            │ Type (regular, directory, symlink, ...)   │
│ space_info             │ Disk capacity, free, available            │
└──────────────────────┴──────────────────────────────────────┘
```

Key functions:
- exists(), is_regular_file(), is_directory(), is_symlink()
- create_directory(), create_directories() (mkdir -p)
- copy(), copy_file(), rename(), remove(), remove_all()
- file_size(), last_write_time(), permissions()
- current_path(), temp_directory_path(), absolute()
- path::filename(), stem(), extension(), parent_path()
- path::replace_extension(), path operator/

TEORİ: std::filesystem (C++17)

C++17 <filesystem> ile tam, taşınabilir, platformlar arası dosya ve dizin işlemleri API'si geldi. Artık platforma özel koda gerek yok!
Temel tipler:

Tip	Amaç
std::filesystem::path	Dosya/dizin yolu nesnesi (operator/ ile)
directory_entry	Tek dizin öğesi (dosya, dizin, sembolik)
directory_iterator	Düz dizin taraması
recursive_dir_iterator	Alt dizinleri dahil özyinelemeli tarama
file_status	Tür bilgisi (regular, directory, symlink)
space_info	Disk kapasitesi, boş alan, kullanılabilir

Temel fonksiyonlar:
- exists(), is_regular_file(), is_directory(), is_symlink()
- create_directory(), create_directories() (mkdir -p)
- copy(), copy_file(), rename(), remove(), remove_all()
- file_size(), last_write_time(), permissions()
- current_path(), temp_directory_path(), absolute()
- path::filename(), stem(), extension(), parent_path()
- path::replace_extension(), path operator/

THEORY: Binary I/O

Binary I/O writes raw bytes (no text formatting):
Text I/O vs Binary I/O:

Aspect	Text I/O	Binary I/O
Mode	ios::in/out	ios::binary
Functions	operator<< / >>	write() / read()
Data	Human-readable text	Raw bytes (like in memory)
Newlines	Platform-converted	Preserved exactly
Size	Variable	Fixed (sizeof struct)
Speed	Slower (formatting)	Faster (direct copy)
Use case	Config, logs, CSV	Images, databases, firmware

Binary I/O pattern: ofstream out("data.bin", ios::binary); out.write(reinterpret_cast<const char*>(&data), sizeof(data));
ifstream in("data.bin", ios::binary); in.read(reinterpret_cast<char*>(&data), sizeof(data));
WARNING: Binary I/O of structs is NOT portable:
- Different machines may have different endianness
- Struct padding may differ between compilers
- Use __attribute__((packed)) or #pragma pack for cross-platform

TEORİ: İkili G/Ç

İkili G/Ç ham baytları yazar (metin biçimlendirmesi yok):
Metin G/Ç vs İkili G/Ç:

```
┌──────────────┬──────────────────────┬──────────────────────────┐
│ Özellik      │ Metin G/Ç            │ İkili G/Ç                │
├──────────────┼──────────────────────┼──────────────────────────┤
│ Mod          │ ios::in/out          │ ios::binary              │
│ Fonksiyonlar │ operator<< / >>      │ write() / read()         │
│ Veri         │ Okunabilir metin     │ Ham baytlar (bellekteki gibi)│
│ Satır sonu   │ Platforma göre dönüşür│ Aynen korunur           │
│ Boyut        │ Değişken             │ Sabit (sizeof struct)    │
│ Hız          │ Yavaş (biçimlendirme) │ Hızlı (doğrudan kopyalama) │
│ Kullanım     │ Yapılandırma, log, CSV│ Görüntü, veritabanı, FW  │
└──────────────┴──────────────────────┴──────────────────────────┘
```

İkili G/Ç deseni: ofstream out("data.bin", ios::binary); out.write(reinterpret_cast<const char*>(&data), sizeof(data));
ifstream in("data.bin", ios::binary); in.read(reinterpret_cast<char*>(&data), sizeof(data));
DİKKAT: Struct'ın ikili G/Ç ile taşınabilirliği sorunlu:
- Farklı makineler farklı endianness kullanabilir
- Struct padding derleyiciler arasında farklılık gösterebilir
- Platformlar arası için __attribute__((packed)) veya #pragma pack kullanın

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <fstream>
#include <filesystem>
#include <string>
#include <vector>
#include <iomanip>
#include <cstring>
#include <cstdint>

namespace fs = std::filesystem;

// ═══════════════════════════════════════════════════════════════════════
// PART 1: std::filesystem::path Operations
// ═══════════════════════════════════════════════════════════════════════

void demo_path_operations() {
    std::cout << "--- Demo 1: std::filesystem::path ---\n";

    // EN: Constructing paths with operator/
    // TR: operator/ ile yol oluşturma
    fs::path base = "/home/ecu";
    fs::path config = base / "config" / "sensors.json";

    std::cout << "    Full path:    " << config << "\n";
    std::cout << "    Parent:       " << config.parent_path() << "\n";
    std::cout << "    Filename:     " << config.filename() << "\n";
    std::cout << "    Stem:         " << config.stem() << "\n";
    std::cout << "    Extension:    " << config.extension() << "\n";

    // EN: Replace extension
    // TR: Uzantıyı değiştir
    fs::path backup = config;
    backup.replace_extension(".bak");
    std::cout << "    Backup:       " << backup << "\n";

    // EN: Relative path construction
    // TR: Göreli yol oluşturma
    fs::path rel = fs::path("src") / "module_01" / "main.cpp";
    std::cout << "    Relative:     " << rel << "\n";
    std::cout << "    Is absolute:  " << std::boolalpha << rel.is_absolute() << "\n\n";
}
```

```

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: Directory Operations
// ═══════════════════════════════════════════════════════════════════════════

void demo_directory_operations() {
    std::cout << "--- Demo 2: Directory Operations ---\n";

    // EN: Create a temporary test directory structure
    // TR: Geçici test dizin yapısı oluştur
    fs::path test_dir = fs::temp_directory_path() / "cpp_mastery_test";

    // EN: Clean up any previous run
    // TR: Önceki çalıştırmanın kalıntılarını temizle
    if (fs::exists(test_dir)) {
        fs::remove_all(test_dir);
    }

    // EN: create_directories = mkdir -p (creates parent dirs too)
    // TR: create_directories = mkdir -p (üst dizinleri de oluşturur)
    fs::create_directories(test_dir / "sensors" / "temperature");
    fs::create_directories(test_dir / "sensors" / "pressure");
    fs::create_directories(test_dir / "logs");

    // EN: Create test files
    // TR: Test dosyaları oluştur — ofstream ile dosyaya metin yazılır
    std::ofstream(test_dir / "readme.txt") << "ECU Sensor Data\n";
    std::ofstream(test_dir / "sensors" / "temp_cal.dat") << "25.0 1.0 0.5\n";
    std::ofstream(test_dir / "logs" / "dtc.log") << "P0100: MAF sensor\n";

    // EN: Check existence and type
    // TR: Varlığı ve türü kontrol et
    std::cout << "    test_dir exists: " << fs::exists(test_dir) << "\n";
    std::cout << "    is_directory:    " << fs::is_directory(test_dir) << "\n";
    std::cout << "    readme exists:   " << fs::is_regular_file(test_dir / "readme.txt") << "\n";

    // EN: File size
    // TR: Dosya boyutu — file_size() ile byte cinsinden döndürülür
    auto readme_size = fs::file_size(test_dir / "readme.txt");
    std::cout << "    readme.txt size: " << readme_size << " bytes\n";

    // EN: Directory iteration (flat)
    // TR: Dizin iterasyonu (düz)
    std::cout << "    Contents of test_dir:\n";
    for (const auto& entry : fs::directory_iterator(test_dir)) {
        std::cout << "      " << (entry.is_directory() ? "[DIR] " : "[FILE]")
                  << " " << entry.path().filename().string() << "\n";
    }

    // EN: Recursive iteration
    // TR: Özyinelemeli iterasyon
    std::cout << "    Recursive listing:\n";
    for (const auto& entry : fs::recursive_directory_iterator(test_dir)) {
        auto rel = fs::relative(entry.path(), test_dir);
        std::cout << "      " << (entry.is_directory() ? "[DIR] " : "      ")
                  << rel.string() << "\n";
    }

    // EN: Copy file
    // TR: Dosyayı kopyala — copy_file() atomik kopyalama yapar
    fs::copy_file(test_dir / "readme.txt", test_dir / "readme_backup.txt");
    std::cout << "    Copied readme.txt → readme_backup.txt\n";

    // EN: Rename (move)
    // TR: Yeniden adlandır (taşı) — aynı dosya sistemi içinde atomik
    fs::rename(test_dir / "readme_backup.txt", test_dir / "readme_v2.txt");
    std::cout << "    Renamed readme_backup.txt → readme_v2.txt\n";

    // EN: Disk space
    // TR: Disk alanı — space() ile toplam kapasite ve boş alan sorgulanır
    auto space = fs::space(test_dir);
    std::cout << "    Disk: " << (space.capacity / (1024*1024*1024)) << " GB total, "
              << (space.available / (1024*1024*1024)) << " GB free\n";

```

```
    // EN: Cleanup
    // TR: Temizlik — remove_all() dizini ve içindekileri siler
    fs::remove_all(test_dir);
    std::cout << "    Cleaned up test directory\n\n";
}

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: Binary I/O
// ═══════════════════════════════════════════════════════════════════════════

// EN: Packed struct for binary serialization
// TR: İkili serileştirme için paketlenmiş struct
#pragma pack(push, 1)
struct SensorRecord {
    uint32_t sensor_id;
    float temperature;
    float pressure;
    uint64_t timestamp_ms;
};
#pragma pack(pop)

void demo_binary_io() {
    std::cout << "--- Demo 3: Binary I/O ---\n";

    fs::path bin_file = fs::temp_directory_path() / "sensors.bin";

    // EN: Write binary records
    // TR: İkili kayıtları yaz
    std::vector<SensorRecord> records = {
        {1, 25.3f, 1013.25f, 1000},
        {2, 30.1f, 1015.50f, 2000},
        {3, 22.8f, 1010.00f, 3000},
        {4, 28.5f, 1012.75f, 4000},
    };

    {
        std::ofstream out(bin_file, std::ios::binary);
        // EN: Write count first, then records
        // TR: Önce sayıyı yaz, sonra kayıtları
        uint32_t count = static_cast<uint32_t>(records.size());
        out.write(reinterpret_cast<const char*>(&count), sizeof(count));
        out.write(reinterpret_cast<const char*>(records.data()),
                  static_cast<std::streamsize>(records.size() * sizeof(SensorRecord)));
    }

    auto file_size = fs::file_size(bin_file);
    std::cout << "    Wrote " << records.size() << " records to binary file\n";
    std::cout << "    File size: " << file_size << " bytes"
              << " (header=" << sizeof(uint32_t)
              << " + " << records.size() << " × " << sizeof(SensorRecord)
              << " = " << sizeof(uint32_t) + records.size() * sizeof(SensorRecord)
              << ")\n";
    std::cout << "    SensorRecord size: " << sizeof(SensorRecord) << " bytes (packed)\n";

    // EN: Read binary records back
    // TR: İkili kayıtları geri oku
    std::vector<SensorRecord> loaded;
    {
        std::ifstream in(bin_file, std::ios::binary);
        uint32_t count = 0;
        in.read(reinterpret_cast<char*>(&count), sizeof(count));
        loaded.resize(count);
        in.read(reinterpret_cast<char*>(loaded.data()),
                static_cast<std::streamsize>(count * sizeof(SensorRecord)));
    }

    std::cout << "    Read back " << loaded.size() << " records:\n";
    for (const auto& r : loaded) {
        std::cout << "      Sensor " << r.sensor_id
                  << ": T=" << r.temperature << "C"
                  << ", P=" << r.pressure << "hPa"
                  << ", ts=" << r.timestamp_ms << "ms\n";
    }
```

```

    // EN: Verify data matches
    // TR: Veri eşleşmesini doğrula — yazdığımız ve okuduğumuz verilerin aynı olduğunu kontrol et
    bool match = (records.size() == loaded.size());
    for (size_t i = 0; match && i < records.size(); ++i) {
        match = (records[i].sensor_id == loaded[i].sensor_id) &&
                (records[i].temperature == loaded[i].temperature);
    }
    std::cout << "    Data integrity check: " << (match ? "PASS" : "FAIL") << "\n";

    fs::remove(bin_file);
    std::cout << "\n";
}

// ═══════════════════════════════════════════════════════════════════════════
// PART 4: Error Handling with filesystem
// ═══════════════════════════════════════════════════════════════════════════

void demo_filesystem_error_handling() {
    std::cout << "--- Demo 4: Filesystem Error Handling ---\n";

    // EN: Method 1: exceptions (default)
    // TR: Yöntem 1: istisnalar (varsayılan)
    try {
        auto sz = fs::file_size("/nonexistent/path/file.txt");
        std::cout << "    Size: " << sz << "\n";  // never reached — TR: buraya ulaşılamaz
    } catch (const fs::filesystem_error& e) {
        std::cout << "    Exception: " << e.what() << "\n";
        std::cout << "    Path1: " << e.path1() << "\n";
        std::cout << "    Code: " << e.code().message() << "\n";
    }

    // EN: Method 2: error_code (no exceptions — preferred for embedded)
    // TR: Yöntem 2: error_code (istisna yok — gömülü için tercih edilen)
    std::error_code ec;
    auto sz = fs::file_size("/nonexistent/path/file.txt", ec);
    if (ec) {
        std::cout << "    error_code: " << ec.message() << "\n";
    } else {
        std::cout << "    Size: " << sz << "\n";
    }

    bool exists = fs::exists("/tmp", ec);
    std::cout << "    /tmp exists: " << std::boolalpha << exists
              << " (no throw)\n\n";
}

// ═══════════════════════════════════════════════════════════════════════════
// PART 5: Practical Patterns
// ═══════════════════════════════════════════════════════════════════════════

void demo_practical_patterns() {
    std::cout << "--- Demo 5: Practical Patterns ---\n";

    // EN: Pattern 1: Find all files with extension
    // TR: Kalıp 1: Belirli uzantılı tüm dosyaları bul
    std::cout << "    .cpp files in module_01:\n";
    fs::path module_dir = "module_01_basics";
    if (fs::exists(module_dir)) {
        for (const auto& entry : fs::directory_iterator(module_dir)) {
            if (entry.is_regular_file() && entry.path().extension() == ".cpp") {
                std::cout << "      " << entry.path().filename().string()
                          << " (" << entry.file_size() << " bytes)\n";
            }
        }
    }

    // EN: Pattern 2: Ensure output directory exists before writing
    // TR: Kalıp 2: Yazmadan önce çıktı dizininin var olduğundan emin ol
    fs::path output = fs::temp_directory_path() / "ecu_output" / "diagnostics";
    fs::create_directories(output);  // Safe to call even if exists
    std::ofstream(output / "report.txt") << "DTC Report: No faults\n";
    std::cout << "    Created output dir + report: " << output.string() << "\n";
    fs::remove_all(output.parent_path());

```

```
264     // EN: Pattern 3: Temporary file with scope guard
265     // TR: Kalıp 3: Kapsam korumalısı ile geçici dosya
266     auto temp = fs::temp_directory_path() / "ecu_temp_XXXXXX.dat";
267     {
268         std::ofstream f(temp, std::ios::binary);
269         uint32_t magic = 0xDEADBEEF;
270         f.write(reinterpret_cast<const char*>(&magic), sizeof(magic));
271     }
272     std::cout << "    Temp file: " << temp.filename().string()
273               << " (" << fs::file_size(temp) << " bytes)\n";
274     fs::remove(temp);
275     std::cout << "    Temp file cleaned up\n\n";
276 }
277
278 // ============================================================================
279 // MAIN
280 // ============================================================================
281 int main() {
282     std::cout << "============================================\n";
283     std::cout << "   Module 01 - Filesystem & Binary I/O\n";
284     std::cout << "============================================\n\n";
285
286     demo_path_operations();
287     demo_directory_operations();
288     demo_binary_io();
289     demo_filesystem_error_handling();
290     demo_practical_patterns();
291
292     // ─── Reference Table ──────────────────────────────────────────────
293     std::cout << "--- Reference: Key std::filesystem Functions ---\n";
294     std::cout << "   ┌──────────────────────────┬──────────────────────────────┐\n";
295     std::cout << "   │ Function                 │ Purpose                      │\n";
296     std::cout << "   ├──────────────────────────┼──────────────────────────────┤\n";
297     std::cout << "   │ exists(p)                │ Check if path exists         │\n";
298     std::cout << "   │ is_regular_file(p)       │ Is it a regular file?        │\n";
299     std::cout << "   │ is_directory(p)          │ Is it a directory?           │\n";
300     std::cout << "   │ file_size(p)             │ Size in bytes                │\n";
301     std::cout << "   │ create_directories(p)    │ mkdir -p equivalent          │\n";
302     std::cout << "   │ copy_file(src, dst)      │ Copy a file                  │\n";
303     std::cout << "   │ rename(old, new)         │ Move/rename file or dir      │\n";
304     std::cout << "   │ remove_all(p)            │ rm -rf equivalent            │\n";
305     std::cout << "   │ directory_iterator(p)    │ Iterate directory (flat)     │\n";
306     std::cout << "   │ recursive_directory_iter(p) │ Iterate all subdirs       │\n";
307     std::cout << "   │ relative(p, base)        │ Compute relative path        │\n";
308     std::cout << "   │ current_path()           │ Get/set working directory    │\n";
309     std::cout << "   │ temp_directory_path()    │ Platform temp directory      │\n";
310     std::cout << "   │ space(p)                 │ Disk space info              │\n";
311     std::cout << "   └──────────────────────────┴──────────────────────────────┘\n\n";
312
313     std::cout << "============================================\n";
314     std::cout << "   End of Filesystem & Binary I/O\n";
315     std::cout << "============================================\n";
316
317     return 0;
318 }
```

Modül 2: OOP — Nesne Yönelimli Programlama

Nesne Yönelimli Programlama

8 kaynak dosya · Standart: C++17

C++17

► 01. Classes Objects

module_02_oop/01_classes_objects.cpp

Konu: Module 02 - Classes & Objects: C++ OOP'nin temel yapı taşları — Fundamental building blocks of C++ OOP

Teori / Theory

THEORY: Class vs Object

A CLASS is a blueprint/template that defines data (member variables) and behavior (member functions). An OBJECT is a concrete instance of that class living in memory with its own copy of member variables. Think: Class = Cookie cutter, Object = Cookie

TEORİ: Sınıf vs Nesne

SINIF (class), veri (üye değişkenler) ve davranışı (üye fonksiyonlar) tanımlayan bir şablondur. NESNE (object) ise o sınıfın bellekte yaşayan, kendi üye değişken kopyasına sahip somut bir örneğidir. Düşün: Sınıf = Kurabiye kalıbı, Nesne = Kurabiye

THEORY: Access Specifiers

C++ has 3 access levels:
• private: Only accessible inside the class itself (DEFAULT for class)
• protected: Accessible inside class + derived classes (inheritance)
• public: Accessible from anywhere Rule of thumb: Data → private, Interface → public

TEORİ: Erişim Belirleyiciler

C++'da 3 erişim seviyesi var:
• private: Sadece sınıf içinden erişilebilir (class için VARSAYILAN)
• protected: Sınıf içi + türetilmiş sınıflardan erişilebilir
• public: Her yerden erişilebilir Kural: Veri → private, Arayüz → public

THEORY: Constructor & Destructor

Constructor = special function called automatically when object is created.
- Same name as class, no return type
- Can be overloaded (multiple constructors with different parameters)
- Member Initializer List (MIL) is preferred over body assignment because MIL directly constructs members, body assigns AFTER default construction Destructor = called automatically when object goes out of scope.
- Name: ~ClassName(), no parameters, no return type, CANNOT be overloaded
- LIFO order: last constructed = first destroyed

TEORİ: Yapıcı & Yıkıcı

Constructor = nesne oluşturulunca otomatik çağrılan özel fonksiyon.
- Sınıf adıyla aynı, dönüş tipi yok
- Overload edilebilir (farklı parametrelerle birden fazla constructor)
- Member Initializer List (MIL), gövde atamadan tercih edilir çünkü MIL üyeleri doğrudan oluşturur, gövde varsayılan oluşturma SONRASI atar Destructor = nesne scope'dan çıkınca otomatik çağrılır.
- İsim: ~SınıfAdı(), parametre yok, dönüş tipi yok, overload edilemez
- LIFO sırası: son oluşturulan = ilk yıkılan

THEORY: struct vs class

In C++, struct and class are almost identical. The ONLY difference:
• struct: members are PUBLIC by default
• class: members are PRIVATE by default Convention: struct for POD (Plain Old Data) with no invariants, class for objects with encapsulation and behavior.

TEORİ: struct ve class Farkı

C++'da struct ve class neredeyse aynıdır. TEK FARK:
• struct: üyeler varsayılan olarak PUBLIC
• class: üyeler varsayılan olarak PRIVATE Gelenek: struct → basit veri (POD), class → kapsülleme + davranış

THEORY: this Pointer

Every non-static member function receives a hidden pointer called `this` that points to the object on which the function was called. this->member is implicit; you write it explicitly when: (1) Parameter name shadows member name (2) Returning *this for method chaining (3) Passing the current object to another function

TEORİ: this İşaretçisi

Her non-static üye fonksiyon, fonksiyonun çağrıldığı nesneyi gösteren gizli bir `this` işaretçisi alır. this->member örtüktür; şu durumlarda açıkça yazılır: (1) Parametre adı üye adını gölgelediğinde (2) Method chaining için *this döndürürken (3) Mevcut nesneyi başka fonksiyona geçirirken

THEORY: const Member Functions

A member function marked `const` promises NOT to modify any member variable.
- The `this` pointer becomes `const ClassName*` inside that function
- const objects can ONLY call const member functions
- Getters should ALWAYS be const
- A const function calling a non-const function = COMPILE ERROR

TEORİ: const Üye Fonksiyonlar

`const` ile işaretlenen üye fonksiyon, hiçbir üye değişkeni DEĞİŞTİRMEYECEĞİNİ garanti eder.
- `this` işaretçisi o fonksiyon içinde `const ClassName*` olur
- const nesneler SADECE const üye fonksiyonları çağırabilir
- Getter'lar HER ZAMAN const olmalı
- const fonksiyondan non-const fonksiyon çağırmak = DERLEME HATASI

→ Ref: https://en.cppreference.com/w/cpp/language/classes
→ Ref: https://en.cppreference.com/w/cpp/language/constructor
→ Ref: https://en.cppreference.com/w/cpp/language/destructor

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <string>
#include <cstdint>

// ────────────────────────────────────────────────────────────────────────
// GLOBAL: Automotive-themed class
// ────────────────────────────────────────────────────────────────────────

/**
 * @class TemperatureSensor
 * @brief EN: Represents an automotive temperature sensor with encapsulation
 *        TR: Kapsülleme ile otomotiv sıcaklık sensörünü temsil eder
 *
 * EN: This class demonstrates:
 *     - private data members (encapsulation)
 *     - constructor with Member Initializer List
 *     - destructor with cleanup message
 *     - const getter functions
 *     - setter with input validation
 *     - behavior method that operates on internal state
 *     - `this` pointer usage
 *
 * TR: Bu sınıf şunları gösterir:
 *     - private veri üyeleri (kapsülleme)
 *     - Member Initializer List ile constructor
 *     - temizleme mesajlı destructor
 *     - const getter fonksiyonları
 *     - girdi doğrulamalı setter
 *     - iç durum üzerinde çalışan davranış metodu
 *     - `this` işaretçisi kullanımı
 */
class TemperatureSensor {
private:
    // EN: Data is PRIVATE → cannot be accessed from outside the class
    // TR: Veri PRIVATE → sınıf dışından erişilemez
    std::string sensorId_;      // EN: Unique sensor identifier / TR: Benzersiz sensör kimliği
    double currentTempC_;       // EN: Current temperature in Celsius / TR: Mevcut sıcaklık (Celsius)
    double minThreshold_;       // EN: Minimum threshold (alert if below) / TR: Min eşik (altında uyarı)
    double maxThreshold_;       // EN: Maximum threshold (alert if above) / TR: Maks eşik (üstünde uyarı)
    int readingCount_;          // EN: How many readings taken / TR: Kaç okuma yapıldı

public:
    // ── CONSTRUCTOR ──────────────────────────────────────────────────

```

```cpp
    /**
     * EN: Parameterized constructor with Member Initializer List (MIL)
     *     MIL directly constructs each member → more efficient than body assignment
     *     Body assignment: first default-constructs, THEN assigns (2 steps for non-trivial types)
     *
     * TR: Member Initializer List (MIL) ile parametreli constructor
     *     MIL her üyeyi doğrudan oluşturur → gövde atamasından daha verimli
     *     Gövde ataması: önce varsayılan oluşturma, SONRA atama (basit olmayan tipler için 2 adım)
     */
    TemperatureSensor(const std::string& id, double minTh, double maxTh)
        : sensorId_(id)           // EN: Direct construction (1 step) / TR: Doğrudan oluşturma (1 adım)
        , currentTempC_(0.0)      // EN: Initialize to zero / TR: Sıfıra başlat
        , minThreshold_(minTh)    // EN: Set minimum threshold / TR: Min eşiği ayarla
        , maxThreshold_(maxTh)    // EN: Set maximum threshold / TR: Maks eşiği ayarla
        , readingCount_(0)        // EN: No readings yet / TR: Henüz okuma yok
    {
        // EN: Constructor body — runs AFTER MIL completes
        //     Use body for logic that can't go in MIL (validation, logging, etc.)
        // TR: Constructor gövdesi — MIL tamamlandıktan SONRA çalışır
        //     MIL'e koyulamayan mantık için gövdeyi kullan (doğrulama, loglama vb.)
        std::cout << "[CONSTRUCTOR] Sensor '" << sensorId_ << "' created"
                  << " [range: " << minThreshold_ << "C - " << maxThreshold_ << "C]\n";
    }

    // ── DESTRUCTOR ──────────────────────────────────────────────────

    /**
     * EN: Destructor — called automatically when object goes out of scope
     *     Perfect place for cleanup: releasing resources, logging, etc.
     *     Cannot be overloaded, cannot take parameters, no return type.
     *     Destruction order is LIFO (Last In, First Out) — reverse of construction.
     *
     * TR: Destructor — nesne scope'dan çıkınca otomatik çağrılır
     *     Temizlik için ideal: kaynak serbest bırakma, loglama vb.
     *     Overload edilemez, parametre alamaz, dönüş tipi yok.
     *     Yıkım sırası LIFO (Son Giren, İlk Çıkar) — oluşturma sırasının tersi.
     */
    ~TemperatureSensor() {
        std::cout << "[DESTRUCTOR] Sensor '" << sensorId_ << "' destroyed"
                  << " (total readings: " << readingCount_ << ")\n";
    }

    // ── CONST GETTERS ───────────────────────────────────────────────

    /**
     * EN: Getter marked `const` → promises not to modify any member
     *     The `const` after the parameter list makes `this` pointer const:
     *       TemperatureSensor* this  →  const TemperatureSensor* this
     *     Rule: ALL getters should be const. If you forget const,
     *     the function CANNOT be called on a const object.
     *
     * TR: `const` ile işaretli getter → hiçbir üyeyi değiştirmeyeceğini garanti eder
     *     Parametre listesinden sonraki `const`, this işaretçisini const yapar:
     *       TemperatureSensor* this  →  const TemperatureSensor* this
     *     Kural: TÜM getter'lar const olmalı. const unutulursa,
     *     fonksiyon const nesnede ÇAĞRILAMAZ.
     */
    std::string getId() const { return sensorId_; }
    double getTemp() const { return currentTempC_; }
    int getReadingCount() const { return readingCount_; }

    // ── SETTER WITH VALIDATION ──────────────────────────────────────

    /**
     * EN: Setter demonstrates encapsulation: external code can't set temp directly,
     *     must go through this function which validates input.
     *     Without encapsulation: object.temp = -9999; (no protection!)
     *     With encapsulation:    object.updateTemp(-9999); → rejected with message
     *
     * TR: Setter kapsüllemeyi gösterir: dışarıdan temp'e doğrudan erişilemez,
     *     bu fonksiyon aracılığıyla girdi doğrulanır.
     *     Kapsülleme olmadan: object.temp = -9999; (korumasız!)
     *     Kapsülleme ile:     object.updateTemp(-9999); → mesajla reddedilir
     */
```

```
119     void updateTemperature(double newTempC) {
120         // EN: Validate — real sensors have physical limits
121         // TR: Doğrula — gerçek sensörlerin fiziksel limitleri var
122         if (newTempC < -273.15) {
123             // EN: Absolute zero is -273.15°C, nothing can be colder
124             // TR: Mutlak sıfır -273.15°C, daha soğuk olamaz
125             std::cout << "  [WARN] Invalid temperature " << newTempC
126                       << "C (below absolute zero!) — ignored\n";
127             return;  // EN: Reject invalid input / TR: Geçersiz girdiyi reddet
128         }
129         currentTempC_ = newTempC;
130         ++readingCount_;
131     }
132
133     // ── BEHAVIOR METHOD ──────────────────────────────────────────────
134
135     /**
136      * EN: This method operates on internal state to produce a result.
137      *     It checks thresholds and reports status. This is const because
138      *     it only reads member variables, never modifies them.
139      *
140      * TR: Bu metod iç durum üzerinde çalışarak sonuç üretir. Eşikleri
141      *     kontrol edip durum bildirir. Sadece üye değişkenleri okuduğu,
142      *     değiştirmediği için const.
143      */
144     void reportStatus() const {
145         std::cout << "  Sensor [" << sensorId_ << "]: " << currentTempC_ << "C";
146
147         // EN: Compare against thresholds
148         // TR: Eşiklerle karşılaştır
149         if (currentTempC_ < minThreshold_) {
150             std::cout << " ⚠ BELOW minimum (" << minThreshold_ << "C)";
151         } else if (currentTempC_ > maxThreshold_) {
152             std::cout << " ⚠ ABOVE maximum (" << maxThreshold_ << "C)";
153         } else {
154             std::cout << " ✓ Normal range";
155         }
156         std::cout << "  [readings: " << readingCount_ << "]\n";
157     }
158
159     // ── THIS POINTER DEMO ────────────────────────────────────────────
160
161     /**
162      * EN: Demonstrates explicit `this` pointer usage.
163      *     Case: parameter name 'id' shadows member — `this->` resolves ambiguity.
164      *     Without `this->`, 'id' refers to the parameter, not the member.
165      *
166      * TR: Açık `this` işaretçi kullanımını gösterir.
167      *     Durum: parametre adı 'sensorId_' üyeyi gölgeler — `this->` belirsizliği çözer.
168      *     `this->` olmadan 'sensorId_' parametreye, üyeye değil, atıfta bulunur.
169      */
170     void reassignId(const std::string& sensorId_) {
171         // EN: Without `this->`, both sides refer to the parameter (self-assignment, USELESS)
172         // TR: `this->` olmadan iki taraf da parametreye atıfta bulunur (kendi kendine atama,
    FAYDASIZ)
173         // sensorId_ = sensorId_;  // EN: BUG! Both are the parameter / TR: HATA! İkisi de parametre
174
175         // EN: With `this->`, left side is the MEMBER, right side is the PARAMETER
176         // TR: `this->` ile sol taraf ÜYE, sağ taraf PARAMETRE
177         this->sensorId_ = sensorId_;
178         std::cout << "  [INFO] Sensor ID reassigned to: " << this->sensorId_ << "\n";
179     }
180
181     /**
182      * EN: Returns reference to *this for method chaining pattern.
183      *     Usage: sensor.setMin(10).setMax(50);  ← each call returns the same object
184      *
185      * TR: Method chaining kalıbı için *this referansı döndürür.
186      *     Kullanım: sensor.setMin(10).setMax(50);  ← her çağrı aynı nesneyi döndürür
187      */
188     TemperatureSensor& setMinThreshold(double val) {
189         minThreshold_ = val;
190         return *this;  // EN: Return the current object / TR: Mevcut nesneyi döndür
191     }
192
```

```cpp
    TemperatureSensor& setMaxThreshold(double val) {
        maxThreshold_ = val;
        return *this;
    }
};

// ────────────────────────────────────────────────────────────────────────
// GLOBAL: struct vs class demonstration
// ────────────────────────────────────────────────────────────────────────

/**
 * @struct CANFrame
 * @brief EN: POD struct — all members public by default, no invariants
 *        TR: POD struct — tüm üyeler varsayılan olarak public, değişmez yok
 *
 * EN: Use struct for simple data containers with no encapsulation needs.
 *     This represents a CAN bus frame — just raw data, no validation logic.
 *     struct members are PUBLIC by default (vs class: PRIVATE by default).
 *
 * TR: Kapsülleme ihtiyacı olmayan basit veri taşıyıcıları için struct kullan.
 *     Bu bir CAN bus çerçevesini temsil eder — sadece ham veri, doğrulama yok.
 *     struct üyeleri varsayılan PUBLIC (class'ta: varsayılan PRIVATE).
 */
struct CANFrame {
    // EN: No need to write "public:" — struct is public by default
    // TR: "public:" yazmaya gerek yok — struct varsayılan public
    uint32_t id;          // EN: CAN frame ID / TR: CAN çerçeve ID'si
    uint8_t  length;      // EN: Data Length Code (0-8) / TR: Veri Uzunluk Kodu (0-8)
    uint8_t  data[8];     // EN: Payload bytes / TR: Yük baytları
    bool     isExtended;  // EN: Standard (11-bit) or Extended (29-bit) / TR: Standart veya Genişletilmiş
};

// ────────────────────────────────────────────────────────────────────────
// GLOBAL: Constructor overloading demonstration
// ────────────────────────────────────────────────────────────────────────

/**
 * @class ECUModule
 * @brief EN: Demonstrates constructor overloading (default + parameterized)
 *        TR: Constructor overloading'i gösterir (varsayılan + parametreli)
 *
 * EN: A class can have MULTIPLE constructors with different parameter lists.
 *     The compiler selects the correct one based on arguments at the call site.
 *     If you define ANY constructor, the compiler does NOT auto-generate a default one.
 *     To get it back: ECUModule() = default;
 *
 * TR: Bir sınıfın farklı parametre listelerine sahip BİRDEN FAZLA constructor'ı olabilir.
 *     Derleyici çağrı yerindeki argümanlara göre doğru olanı seçer.
 *     HERHANGİ bir constructor tanımlarsan, derleyici varsayılan olanı otomatik OLUŞTURMAZ.
 *     Geri almak için: ECUModule() = default;
 */
class ECUModule {
private:
    std::string name_;
    std::string firmwareVersion_;
    bool isActive_;

public:
    // EN: Default constructor — creates module with placeholder values
    //     This exists so we can create an ECUModule without arguments: ECUModule ecu;
    // TR: Varsayılan constructor — yer tutucu değerlerle modül oluşturur
    //     Bu, argümansız ECUModule oluşturabilmemiz için var: ECUModule ecu;
    ECUModule()
        : name_("UNKNOWN")
        , firmwareVersion_("0.0.0")
        , isActive_(false)
    {
        std::cout << "[CONSTRUCTOR] ECU default created: " << name_ << "\n";
    }
```

```cpp

    // EN: Parameterized constructor — user specifies name and firmware version
    // TR: Parametreli constructor — kullanıcı ad ve firmware sürümünü belirtir
    ECUModule(const std::string& name, const std::string& fw)
        : name_(name)
        , firmwareVersion_(fw)
        , isActive_(true)
    {
        std::cout << "[CONSTRUCTOR] ECU created: " << name_ << " (fw: " << firmwareVersion_ << ")\n";
    }

    // EN: Destructor
    // TR: Yıkıcı
    ~ECUModule() {
        std::cout << "[DESTRUCTOR] ECU '" << name_ << "' destroyed\n";
    }

    // EN: Getters — all const / TR: Getter'lar — hepsi const
    std::string getName() const { return name_; }
    std::string getFirmware() const { return firmwareVersion_; }
    bool isActive() const { return isActive_; }

    // EN: Display info / TR: Bilgi göster
    void printInfo() const {
        std::cout << "  ECU [" << name_ << "] fw=" << firmwareVersion_
                  << " active=" << (isActive_ ? "yes" : "no") << "\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// MAIN — Demonstrations
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 02 - Classes & Objects\n";
    std::cout << "============================================\n\n";

    // ─── Section 1: Object Creation & Constructor ──────────────────────
    {
        std::cout << "--- Section 1: Object Creation & Constructor ---\n";

        // EN: Creating an object = calling the constructor.
        //     The constructor runs immediately when the object is declared.
        //     Memory: object lives on the STACK (automatic storage duration).
        // TR: Nesne oluşturmak = constructor'ı çağırmak.
        //     Constructor, nesne tanımlanınca hemen çalışır.
        //     Bellek: nesne STACK'te yaşar (otomatik ömür süresi).
        TemperatureSensor coolant("COOLANT_01", -40.0, 120.0);
        // EN: ↑ This single line does ALL of this:
        //     1. Allocates sizeof(TemperatureSensor) bytes on stack
        //     2. Calls constructor with ("COOLANT_01", -40.0, 120.0)
        //     3. MIL runs → members directly constructed
        //     4. Constructor body runs → prints creation message
        // TR: ↑ Bu tek satır TÜM bunları yapar:
        //     1. Stack üzerinde sizeof(TemperatureSensor) bayt ayırır
        //     2. Constructor'ı ("COOLANT_01", -40.0, 120.0) ile çağırır
        //     3. MIL çalışır → üyeler doğrudan oluşturulur
        //     4. Constructor gövdesi çalışır → oluşturma mesajı yazdırır

        std::cout << "\n";
    }
    // EN: ↑ coolant goes out of scope here → DESTRUCTOR automatically called
    // TR: ↑ coolant burada scope'dan çıkar → DESTRUCTOR otomatik çağrılır

    std::cout << "\n";

    // ─── Section 2: Encapsulation — Getter/Setter ──────────────────────
    {
        std::cout << "--- Section 2: Encapsulation — Getter/Setter ---\n";

        TemperatureSensor oil("OIL_TEMP", 0.0, 150.0);

```

```cpp
        // EN: We CANNOT access private members directly:
        //     oil.currentTempC_ = 85.5;  ← COMPILE ERROR: 'currentTempC_' is private
        //     This is ENCAPSULATION: data is hidden, access is controlled.
        // TR: private üyelere doğrudan ERİŞEMEYİZ:
        //     oil.currentTempC_ = 85.5;  ← DERLEME HATASI: 'currentTempC_' private
        //     Bu KAPSÜLLEME: veri gizli, erişim kontrollü.

        // EN: Use public setter — validates input before modifying state
        // TR: Public setter kullan — durumu değiştirmeden önce girişi doğrular
        oil.updateTemperature(92.3);
        oil.updateTemperature(105.7);

        // EN: Use const getters to READ private data safely
        // TR: private veriyi güvenle OKUMAK için const getter kullan
        std::cout << "  Oil sensor ID: " << oil.getId() << "\n";
        std::cout << "  Oil temp: " << oil.getTemp() << "C\n";
        std::cout << "  Readings so far: " << oil.getReadingCount() << "\n";

        // EN: Try invalid temperature — setter will reject it
        // TR: Geçersiz sıcaklık dene — setter reddedecek
        oil.updateTemperature(-300.0);  // EN: Below absolute zero → rejected / TR: Mutlak sıfırın altı → reddedilir
        std::cout << "  Temp after invalid attempt: " << oil.getTemp() << "C (unchanged)\n";

        std::cout << "\n";
    }

    std::cout << "\n";

    // ─── Section 3: Behavior Methods ──────────────────────────────────
    {
        std::cout << "--- Section 3: Behavior Methods ---\n";

        TemperatureSensor intake("INTAKE_AIR", -30.0, 60.0);

        // EN: Feed multiple temperature readings and check status
        // TR: Birden fazla sıcaklık okuması ver ve durumu kontrol et
        double readings[] = {25.0, -35.5, 62.0, 40.0};
        for (double temp : readings) {
            intake.updateTemperature(temp);
            intake.reportStatus();
            // EN: ↑ reportStatus() reads internal state and decides:
            //     below min → warning, above max → warning, else → normal
            // TR: ↑ reportStatus() iç durumu okuyup karar verir:
            //     min altı → uyarı, maks üstü → uyarı, yoksa → normal
        }

        std::cout << "\n";
    }

    std::cout << "\n";

    // ─── Section 4: `this` Pointer ────────────────────────────────────
    {
        std::cout << "--- Section 4: this Pointer ---\n";

        TemperatureSensor exhaust("EXHAUST_01", 100.0, 900.0);

        // EN: Case 1: Resolving name shadowing with this->
        //     reassignId has a parameter named sensorId_ which shadows the member
        // TR: Durum 1: this-> ile isim gölgelemeyi çözme
        //     reassignId'nin sensorId_ adında üyeyi gölgeleyen bir parametresi var
        exhaust.reassignId("EXHAUST_LEFT_BANK");

        // EN: Case 2: Method chaining via return *this
        //     Each setter returns the same object, allowing call chaining.
        //     This is the same pattern used by std::cout: cout << a << b << c;
        // TR: Durum 2: return *this ile method chaining
        //     Her setter aynı nesneyi döndürerek zincirleme çağrıya izin verir.
        //     std::cout'un kullandığı kalıpla aynı: cout << a << b << c;
        exhaust.setMinThreshold(150.0).setMaxThreshold(950.0);
```

```
        // EN: ↑ Equivalent to:
        //     TemperatureSensor& ref1 = exhaust.setMinThreshold(150.0); // returns exhaust
        //     ref1.setMaxThreshold(950.0);                              // called on same object
        // TR: ↑ Eşdeğeri:
        //     TemperatureSensor& ref1 = exhaust.setMinThreshold(150.0); // exhaust'ı döndürür
        //     ref1.setMaxThreshold(950.0);                              // aynı nesne üzerinde çağrılır

        exhaust.updateTemperature(850.0);
        exhaust.reportStatus();

        std::cout << "\n";
    }

    std::cout << "\n";

    // ─── Section 5: Multiple Objects — Independence ──────────────────────
    {
        std::cout << "--- Section 5: Multiple Objects — Independence ---\n";

        // EN: Each object has its OWN copy of all member variables.
        //     Modifying one object does NOT affect the other.
        //     They share the same CODE (member functions) but have separate DATA.
        // TR: Her nesne tüm üye değişkenlerin KENDİ kopyasına sahiptir.
        //     Bir nesneyi değiştirmek diğerini ETKİLEMEZ.
        //     Aynı KODU (üye fonksiyonlar) paylaşırlar ama ayrı VERİLERİ vardır.
        TemperatureSensor cabin("CABIN", 15.0, 30.0);
        TemperatureSensor battery("BATTERY", -20.0, 45.0);

        cabin.updateTemperature(22.0);
        battery.updateTemperature(38.5);

        cabin.reportStatus();    // EN: Shows cabin data only / TR: Sadece kabin verisini gösterir
        battery.reportStatus();  // EN: Shows battery data only / TR: Sadece batarya verisini gösterir

        // EN: Changing cabin temp does NOT touch battery temp
        // TR: Kabin sıcaklığını değiştirmek batarya sıcaklığına DOKUNMAZ
        cabin.updateTemperature(33.0);
        std::cout << "  After cabin update:\n";
        cabin.reportStatus();
        battery.reportStatus();  // EN: Still 38.5 — unchanged / TR: Hâlâ 38.5 — değişmedi

        std::cout << "\n";
    }

    std::cout << "\n";

    // ─── Section 6: struct vs class & sizeof ─────────────────────────────
    {
        std::cout << "--- Section 6: struct vs class & sizeof ---\n";

        // EN: struct → POD, all members public by default, aggregate initialization
        //     Aggregate init: no constructor needed, just list values in braces {}
        // TR: struct → POD, tüm üyeler varsayılan public, aggregate başlatma
        //     Aggregate init: constructor gerekmez, süslü parantez içinde değerleri listele {}
        CANFrame frame = {0x1A3, 4, {0xDE, 0xAD, 0xBE, 0xEF, 0, 0, 0, 0}, false};

        std::cout << "  CAN Frame ID: 0x" << std::hex << frame.id << std::dec << "\n";
        std::cout << "  DLC: " << static_cast<int>(frame.length) << "\n";
        // EN: ↑ static_cast<int> because uint8_t would print as a character
        //     (uint8_t is often typedef'd from unsigned char)
        // TR: ↑ static_cast<int> çünkü uint8_t karakter olarak yazdırılır
        //     (uint8_t genellikle unsigned char'dan typedef'tir)

        std::cout << "  Data: ";
        for (int i = 0; i < static_cast<int>(frame.length); ++i) {
            std::cout << "0x" << std::hex << static_cast<int>(frame.data[i]) << " ";
        }
        std::cout << std::dec << "\n";
```

```

        // EN: sizeof shows the actual memory footprint of the type/object
        //     Note: sizeof includes PADDING bytes added by the compiler for alignment!
        //     struct layout in memory is NOT always sum of member sizes.
        // TR: sizeof, tip/nesnenin gerçek bellek ayak izini gösterir
        //     Not: sizeof, derleyicinin hizalama için eklediği PADDING baytlarını içerir!
        //     struct'ın bellekteki düzeni her zaman üye boyutları toplamı DEĞİLDİR.
        std::cout << "  sizeof(CANFrame): " << sizeof(CANFrame) << " bytes\n";
        std::cout << "  sizeof(TemperatureSensor): " << sizeof(TemperatureSensor) << " bytes\n";
        std::cout << "  sizeof(ECUModule): " << sizeof(ECUModule) << " bytes\n";

        std::cout << "\n";
    }

    std::cout << "\n";

    // ─── Section 7: Constructor Overloading ──────────────────────────────
    {
        std::cout << "--- Section 7: Constructor Overloading ---\n";

        // EN: Default constructor → no arguments needed
        //     The compiler selects ECUModule() because no args are provided
        // TR: Varsayılan constructor → argüman gerekmez
        //     Derleyici, argüman verilmediği için ECUModule()'ı seçer
        ECUModule unknown;
        unknown.printInfo();

        // EN: Parameterized constructor → 2 arguments
        //     The compiler selects ECUModule(string, string) based on arg types
        // TR: Parametreli constructor → 2 argüman
        //     Derleyici, argüman tiplerine göre ECUModule(string, string)'i seçer
        ECUModule brakeCtrl("BrakeController", "2.4.1");
        brakeCtrl.printInfo();

        ECUModule steeringCtrl("SteeringAssist", "1.8.0");
        steeringCtrl.printInfo();

        std::cout << "\n";
    }
    // EN: ↑ End of scope → destructors called in REVERSE (LIFO) order:
    //     steeringCtrl destroyed first, then brakeCtrl, then unknown
    //     (last constructed = first destroyed)
    // TR: ↑ Scope sonu → destructor'lar TERS (LIFO) sırada çağrılır:
    //     steeringCtrl önce yıkılır, sonra brakeCtrl, sonra unknown
    //     (son oluşturulan = ilk yıkılan)

    std::cout << "\n";

    // ─── Section 8: LIFO Destruction Order Demo ──────────────────────────
    {
        std::cout << "--- Section 8: LIFO Destruction Order ---\n";
        std::cout << "  Creating 3 sensors in order: A → B → C\n";

        // EN: Objects are constructed in declaration order: A, B, C
        //     But destroyed in REVERSE order: C, B, A  (LIFO)
        //     This matters when objects depend on each other!
        // TR: Nesneler tanım sırasında oluşturulur: A, B, C
        //     Ama TERS sırada yıkılır: C, B, A  (LIFO)
        //     Bu, nesneler birbirine bağımlıyken ÖNEMLİDİR!
        TemperatureSensor sA("A_first", 0, 100);
        TemperatureSensor sB("B_second", 0, 100);
        TemperatureSensor sC("C_third", 0, 100);

        std::cout << "  Now leaving scope — watch destruction order:\n";
    }
```

```
    // EN: ↑ Output will show: C destroyed, B destroyed, A destroyed (LIFO)
    // TR: ↑ Çıktı gösterecek: C yıkıldı, B yıkıldı, A yıkıldı (LIFO)

    std::cout << "\n";

    // ─── Section 9: const Objects ──────────────────────────────────────
    {
        std::cout << "--- Section 9: const Objects ---\n";

        // EN: A const object cannot be modified after construction.
        //     It can ONLY call const member functions.
        //     Attempting to call a non-const function → COMPILE ERROR.
        // TR: const nesne, oluşturulduktan sonra değiştirilemez.
        //     SADECE const üye fonksiyonları çağırabilir.
        //     non-const fonksiyon çağırma girişimi → DERLEME HATASI.
        const TemperatureSensor frozen("FROZEN_REF", -50.0, 50.0);

        // EN: ✓ These work — getId(), getTemp(), reportStatus() are all const
        // TR: ✓ Bunlar çalışır — getId(), getTemp(), reportStatus() hepsi const
        std::cout << "  Frozen sensor ID: " << frozen.getId() << "\n";
        std::cout << "  Frozen sensor temp: " << frozen.getTemp() << "C\n";
        frozen.reportStatus();

        // EN: ✗ These would FAIL to compile if uncommented:
        //     frozen.updateTemperature(10.0);  ← ERROR: non-const function on const object
        //     frozen.reassignId("NEW");        ← ERROR: same reason
        //     This is WHY making getters const matters!
        // TR: ✗ Bunlar yorum işaretleri kaldırılırsa derleme HATASI verir:
        //     frozen.updateTemperature(10.0);  ← HATA: const nesnede non-const fonksiyon
        //     frozen.reassignId("NEW");        ← HATA: aynı sebep
        //     Getter'ları const yapmanın önemli olmasının sebebi BU!

        std::cout << "\n";
    }

    std::cout << "\n==========================================\n";
    std::cout << "   End of Module 02 - Classes & Objects\n";
    std::cout << "==========================================\n";

    return 0;
}
```

► 02. Inheritance Polymorphism

module_02_oop/02_inheritance_polymorphism.cpp

Konu: OOP Part 2: Architecture of Inheritance and Polymorphism — OOP Kısım 2: Kalıtım ve Çok

Teori / Theory

THEORY: What is Inheritance? Why use it?

Software developers hate repeating code (DRY Principle - Don't Repeat Yourself). If we have a 'Dog', a 'Cat', and a 'Bird', they all have an 'age', 'weight', and an 'eat()' function. Instead of writing these properties 3 times, we create a "Base Class" called 'Animal'. The Dog and Cat "inherit" everything from Animal, and only add their unique features (like bark for Dog).

TEORİ: Kalıtım Nedir ve Neden Kullanılır?

Yazılım geliştiriciler aynı kodu tekrar yazmaktan nefret eder (DRY Prensibi). Bir 'Köpek', 'Kedi' ve 'Kuş' sınıfımız varsa, hepsinin bir 'yaş', 'ağırlık' değişkeni ve 'yemekYe()' fonksiyonu olacaktır. Bunları 3 defa ayrı ayrı yazmak yerine 'Hayvan' (Animal) adında bir "Temel Sınıf" (Base Class) yaratırız. Köpek ve Kedi, Hayvan sınıfındaki her şeyi miras alır, üzerine sadece köpeğe özgü şeyleri (havlama gibi) ekleriz.

THEORY: Polymorphism and V-Table Architecture

"Give a command, let the object react according to its true nature." If we have an 'Animal' pointer, but it actually points to a 'Dog', which sound will play if we call `makeSound()`? This is where 'virtual' comes in. 'virtual' tells the compiler to do "Late Binding / Dynamic Dispatch". "Look at the object's REAL type at runtime (using the V-Table) and call the Dog's function."

TEORİ: Çok Biçimlilik ve V-Table

"Bir komut ver, arkadaki nesne kendi doğasına göre tepki versin." Eğer 'Hayvan' tipinde bir işaretçimiz (pointer) varsa, ama bu işaretçi gerçekte bir 'Köpek' nesnesini tutuyorsa, `sesÇıkar()` dediğimizde kimin sesi çıkacak? İşte 'virtual' (sanal) anahtar kelimesi devreye girer. 'virtual', derleyiciye "Program çalışırken nesnenin gerçek tipine bak (V-Table / Sanal Tablo üzerinden) ve Köpeğin fonksiyonunu çağır (Geç Bağlama)" der. Eğer Polymorphism olmasaydı, karşımızdaki nesne gerçekte bir Köpek olsa bile, işaretçi tipi Hayvan olduğu için sıradan bir Hayvan sesi çıkardı.

CPPREF Depth: Object Slicing & V-Table Performance Penalty

"Object Slicing" occurs when you pass a Derived object by VALUE to a Base. The compiler literally "slices off" the extra data! ALWAYS pass polymorphic objects by Pointer (*) or Reference (&). Also, `virtual` functions have a slight CPU cost because they require an extra pointer jump (vptr -> vtable) at runtime, which blocks the compiler from 'inlining' the function.

CPPREF Derinlik: Nesne Dilimlemesi ve V-Tablo Performans Cezası

Eğer bir Köpek nesnesini `void feed(Animal a)` şeklinde Değer Kopyası ile gönderirseniz "Nesne Dilimlenmesi (Object Slicing)" yaşanır, köpeklik özellikleri kesilip atılır! Polimorfizm SADECE pointer/referans `feed(Animal& a)` ile çalışır. Ayrıca `virtual` fonksiyonlar oyun anında ekstra bir pointer araması (vptr) yaptıkları için derleyicinin "inlining" yapmasını engeller (Ufak Hız Kaybı).

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <memory>
#include <vector>

/**
 * @class Animal
 * @brief Base class (Temel Sınıf). Represents generic animal traits. — Genel hayvan
 * özelliklerini barındırır.
 */
class Animal {
public:
  Animal() { std::cout << "[Animal] Created. (Hayvan Doğdu)" << std::endl; }

  /**
   * @brief [THEORY — TEORİ]: The Vital Importance of Virtual Destructor / Virtual Yıkıcının
   * Hayati Önemi
   *
   * EN: If you're building a base class to be inherited from, its DESTRUCTOR MUST BE VIRTUAL!
   * Why? If we say `Animal* pet = new Dog();` and later `delete pet;`, without 'virtual', only
   * the Animal's destructor is called. The Dog's memory becomes a LEAK!
   *
   * TR: Eğer miras alınacak bir temel sınıf yapıyorsanız YIKICI FONKSİYON MUTLAKA VIRTUAL
   * OLMALIDIR. Neden? Eğer biz "Animal* pet = new Dog();" dersek ve ardından "delete pet;"
   * yaparsak komut sınıfı silmeye 'Animal'dan başlar. Eğer virtual demezek, Dog sınıfının
   * yıkıcısı ASLA çağrılmaz ve köpeğe ayırdığımız hafıza sızıntıya (memory leak) uğrar. O yüzden
   * 'virtual' deriz.
   */
  virtual ~Animal() {
    std::cout << "[Animal] Destroyed. (Hayvan Yok Oldu)" << std::endl;
  }

  /**
   * @brief Polymorphic function — Çok biçimli fonksiyon.
   *
   * EN: 'virtual' allows derived classes to override this behavior.
   * TR: 'virtual', bu sınıftan üretilen alt sınıfların bu fonksiyonu ezmesine (override) izin
   * verir.
   */
  virtual void makeSound() const {
    std::cout << "(Silence... Sessizlik...)" << std::endl;
  }
};

/**
 * @class Dog
 * @brief Derived class (Alt — Türetilmiş Sınıf). Inherits from Animal.
 */
// EN: Inheritance syntax
// TR: Kalıtım sözdizimi
class Dog : public Animal {
public:
  Dog() {
    std::cout << "[Dog] Born. (Köpek Hayata Gözlerini Açtı)" << std::endl;
  }

  ~Dog() override {
    std::cout << "[Dog] Destroyed. (Köpek Yok Oldu)" << std::endl;
  }

  /**
   * @brief Overrides the base class functionality — Temel sınıftaki fonksiyonu Köpeklerin
   * doğasına göre ezer.
   *
   * EN: 'override' is optional but HIGHLY RECOMMENDED. It tells the compiler to verify our
   * override.
   * TR: 'override' yazmak derleyicinin bizi denetlemesini sağlar, hata yaparsak uyarır.
   */
  void makeSound() const override {
    std::cout << "Dog says: Woof! Woof! (Hav! Hav!)" << std::endl;
  }
};
```

```cpp

/**
 * @class Cat
 * @brief Another Derived class — Başka bir türetilmiş sınıf.
 */
class Cat : public Animal {
public:
  Cat() { std::cout << "[Cat] Born. (Kedi Doğdu)" << std::endl; }
  ~Cat() override {
    std::cout << "[Cat] Destroyed. (Kedi Yok Oldu)" << std::endl;
  }

  void makeSound() const override {
    std::cout << "Cat says: Meow! (Miyav!)" << std::endl;
  }
};

// ═══════════════════════════════════════════════════════════════════════════════
// EN: IDiagnosticService — Pure Abstract Interface (Automotive Diagnostics)
// TR: IDiagnosticService — Saf Soyut Arayüz (Otomotiv Teşhis)
// ═══════════════════════════════════════════════════════════════════════════════

/**
 * @class IDiagnosticService
 * @brief EN: Pure abstract interface for automotive diagnostic protocols. Any class implementing
 * this MUST provide all three methods.
 * TR: Otomotiv teşhis protokolleri için saf soyut arayüz. Bu arayüzü uygulayan her sınıf üç
 * metodu da sağlamalıdır.
 */
class IDiagnosticService {
public:
    // EN: Pure virtual — every implementation must provide a service name
    // TR: Saf sanal — her uygulama bir servis adı sağlamalı
    virtual std::string getName() const = 0;

    // EN: Pure virtual — runs a diagnostic check and returns pass/fail
    // TR: Saf sanal — teşhis kontrolü çalıştırır ve geçti/kaldı döner
    virtual bool runDiagnostic() = 0;

    // EN: Virtual destructor defaulted — required for safe polymorphic deletion
    // TR: Varsayılan sanal yıkıcı — güvenli polimorfik silme için gerekli
    virtual ~IDiagnosticService() = default;
};

// ─── OBD2Service — Concrete Implementation ─────────────────────────────────
// EN: OBD-II (On-Board Diagnostics) standard used in all modern vehicles
// TR: OBD-II (Araç İçi Teşhis) tüm modern araçlarda kullanılan standart

class OBD2Service : public IDiagnosticService {
private:
    std::string vehicleVIN_;

public:
    explicit OBD2Service(const std::string& vin) : vehicleVIN_(vin) {
        std::cout << "[OBD2Service] Initialized for VIN: " << vehicleVIN_ << std::endl;
    }

    std::string getName() const override {
        return "OBD-II Diagnostic (VIN: " + vehicleVIN_ + ")";
    }

    bool runDiagnostic() override {
        std::cout << "[OBD2] Scanning DTCs (Diagnostic Trouble Codes) for "
                  << vehicleVIN_ << "..." << std::endl;
        std::cout << "[OBD2] Result: No fault codes detected. PASS." << std::endl;
        return true;
    }
};
```

```cpp

// ─── CANDiagService — Another Concrete Implementation ──────────────────────────
// EN: CAN Bus (Controller Area Network) diagnostics for vehicle ECU networks
// TR: CAN Bus (Denetleyici Alan Ağı) araç ECU ağları için teşhis

class CANDiagService : public IDiagnosticService {
private:
    int busSpeed_;

public:
    explicit CANDiagService(int busSpeedKbps) : busSpeed_(busSpeedKbps) {
        std::cout << "[CANDiagService] Initialized at " << busSpeed_
                  << " kbps" << std::endl;
    }

    std::string getName() const override {
        return "CAN Bus Diagnostic (" + std::to_string(busSpeed_) + " kbps)";
    }

    bool runDiagnostic() override {
        std::cout << "[CAN] Probing CAN bus at " << busSpeed_
                  << " kbps..." << std::endl;
        std::cout << "[CAN] Result: Bus load 42%, all nodes responsive. PASS." << std::endl;
        return true;
    }
};

// ─── FinalConfig — Demonstrates 'final' keyword on a class ─────────────────────
// EN: A class marked 'final' cannot be inherited from. Useful for sealed configuration objects
// that must not be extended.
// TR: 'final' ile işaretlenen bir sınıftan miras alınamaz. Genişletilmemesi gereken kapalı
// yapılandırma nesneleri için kullanışlıdır.

class FinalConfig final {
private:
    std::string configName_;
    int configValue_;

public:
    FinalConfig(const std::string& name, int value)
        : configName_(name), configValue_(value) {}

    void print() const {
        std::cout << "Config: " << configName_ << " = " << configValue_ << std::endl;
    }
};

// EN: The following would cause a compile error:
// TR: Aşağıdaki derleme hatası verir: class DerivedConfig : public FinalConfig {};  // ERROR:
// cannot derive from 'final' class

// ─── Shape / Circle / FixedCircle — override and final on methods ──────────────
// EN: Demonstrates 'override' for correctness and 'final' to prevent further overriding
// TR: Doğruluk için 'override' ve daha fazla ezmeyi önlemek için 'final' gösterir

class Shape {
public:
    virtual double area() const {
        return 0.0;
    }
    virtual ~Shape() = default;
};

```

```cpp
class Circle : public Shape {
protected:
    double radius_;

public:
    explicit Circle(double r) : radius_(r) {}

    // EN: 'override' ensures we are truly overriding a base virtual method. If we misspelled
    // 'area' as 'Area', the compiler catches it.
    // TR: 'override', gerçekten bir temel sanal metodu ezdiğimizi garanti eder. 'area' yerine
    // 'Area' yazarsak derleyici yakalar.
    double area() const override {
        return 3.14159265358979 * radius_ * radius_;
    }
};

class FixedCircle final : public Circle {
public:
    explicit FixedCircle(double r) : Circle(r) {}

    // EN: 'final' on this method means NO class inheriting from FixedCircle can override area()
    // again. (But FixedCircle itself is also 'final', so no class can inherit from it at all.)
    // TR: Bu metod üzerindeki 'final', FixedCircle'dan miras alan hiçbir sınıfın area()'yı
    // tekrar ezemeyeceği anlamına gelir.
    double area() const final {
        return 3.14159265358979 * radius_ * radius_;
    }
};

int main() {
  std::cout << "=== OOP Part 2: Polymorphism (Çok Biçimlilik) ===\n" << std::endl;

  // EN: Pointers are Base (Animal), but objects are Derived.
  // TR: İşaretçimiz Base (Animal), ama işaret etiğimiz şeyler gerçek nesneler.
  Animal *pet1 = new Dog();
  std::cout << "---" << std::endl;
  Animal *pet2 = new Cat();

  std::cout << "\n[POLYMORPHISM IN ACTION / ÇOK BİÇİMLİLİK DEVREDE]" << std::endl;

  // EN: Thanks to the V-Table, the REAL types will speak!
  // TR: V-Table sayesinde gerçek tiplerine göre ses çıkaracaklar!
  pet1->makeSound();
  pet2->makeSound();

  std::cout << "\n[CLEANUP / TEMİZLİK]" << std::endl;
  // EN: Virtual Destructor guarantees perfect teardown. Memory Leak = 0!
  // TR: Virtual Destructor sayesinde MÜKEMMEL bir yıkım sırası izlenecek. Memory Leak 0!
  delete pet1;
  std::cout << "---" << std::endl;
  delete pet2;

  // ═══════════════════════════════════════════════════════════════════════════════
  // EN: ADVANCED POLYMORPHISM — Automotive Diagnostic Service Demos
  // TR: GELİŞMİŞ POLİMORFİZM — Otomotiv Teşhis Servisi Gösterimleri
  // ═══════════════════════════════════════════════════════════════════════════════

  std::cout << "\n\n=== ADVANCED POLYMORPHISM: AUTOMOTIVE DIAGNOSTICS ===\n" << std::endl;

  // ─── 1. Abstract Interface — IDiagnosticService ──────────────────────────────
  // EN: We cannot instantiate IDiagnosticService directly (it has pure virtual methods). We
  // create concrete implementations through base pointers.
  // TR: IDiagnosticService doğrudan örneklenemez (saf sanal metotları var). Temel işaretçiler
  // aracılığıyla somut uygulamalar oluştururuz.
  std::cout << "--- Demo: Abstract Interface — IDiagnosticService ---" << std::endl;
  {
      IDiagnosticService* svc1 = new OBD2Service("WVWZZZ3CZWE123456");
      IDiagnosticService* svc2 = new CANDiagService(500);

      std::cout << "\nService 1: " << svc1->getName() << std::endl;
      svc1->runDiagnostic();

      std::cout << "\nService 2: " << svc2->getName() << std::endl;
      svc2->runDiagnostic();

```

```
        // EN: Virtual destructor ensures proper cleanup through base pointer
        // TR: Sanal yıkıcı, temel işaretçi aracılığıyla doğru temizliği sağlar
        delete svc1;
        delete svc2;
    }
    std::cout << std::endl;

    // ─── 2. final Class — FinalConfig ──────────────────────────────────────────────
    // EN: FinalConfig is marked 'final' — attempting to inherit from it would produce a
    // compile-time error. This is useful for sealed ECU configuration objects that must remain
    // unextended.
    // TR: FinalConfig 'final' olarak işaretlenmiş — ondan miras almaya çalışmak derleme zamanı
    // hatası üretir. Genişletilmemesi gereken kapalı ECU yapılandırma nesneleri için
    // kullanışlıdır.
    std::cout << "--- Demo: final Class — FinalConfig ---" << std::endl;
    {
        FinalConfig ecuConfig("MaxRPM", 7500);
        ecuConfig.print();

        FinalConfig brakeConfig("ABSThreshold_mbar", 4200);
        brakeConfig.print();

        // EN: Uncommenting the following line would cause: error: cannot derive from 'final' base
        // 'FinalConfig'
        // TR: Aşağıdaki satırın yorumunu kaldırmak şu hataya neden olur: error: 'final' temel
        // 'FinalConfig'ten türetilemez class ExtendedConfig : public FinalConfig {};  // COMPILE
        // ERROR!
    }
    std::cout << std::endl;

    // ─── 3. override Keyword — Shape / Circle ──────────────────────────────────────
    // EN: 'override' is a C++11 keyword that tells the compiler: "I intend to override a virtual
    // function in the base class. If I'm wrong (e.g., wrong signature), give me a compile error."
    // This prevents silent bugs.
    // TR: 'override', derleyiciye "Temel sınıftaki bir sanal fonksiyonu ezmeyi amaçlıyorum.
    // Yanlışsam derleme hatası ver" der. Sessiz hataları önler.
    std::cout << "--- Demo: override Keyword — Shape/Circle ---" << std::endl;
    {
        Shape baseShape;
        Circle wheel(17.5);
        FixedCircle hubcap(7.0);

        // EN: Polymorphism through base pointer — correct area() called each time
        // TR: Temel işaretçi aracılığıyla polimorfizm — her seferinde doğru area() çağrılır
        Shape* shapes[] = { &baseShape, &wheel, &hubcap };
        const char* names[] = { "Base Shape", "Wheel (Circle r=17.5)", "Hubcap (FixedCircle r=7.0)" };

        for (int i = 0; i < 3; ++i) {
            std::cout << names[i] << " area = " << shapes[i]->area() << std::endl;
        }
    }
    std::cout << std::endl;

    // ─── 4. Polymorphic Container — vector<unique_ptr<IDiagnosticService>> ─────────
    // EN: The idiomatic C++ way to manage polymorphic objects: store them in a vector of
    // unique_ptr to the base interface. No manual delete needed. This is how modern automotive
    // middleware (AUTOSAR Adaptive) manages service instances.
    // TR: Polimorfik nesneleri yönetmenin idiomatik C++ yolu: temel arayüze unique_ptr vektöründe
    // saklamak. Manuel delete gerekmez. Modern otomotiv ara yazılımı (AUTOSAR Adaptive) servis
    // örneklerini böyle yönetir.
    std::cout << "--- Demo: Polymorphic Container — vector<unique_ptr> ---" << std::endl;
    {
        std::vector<std::unique_ptr<IDiagnosticService>> diagnostics;

        diagnostics.push_back(std::make_unique<OBD2Service>("TMBJB9NE1L0123456"));
        diagnostics.push_back(std::make_unique<CANDiagService>(250));
        diagnostics.push_back(std::make_unique<OBD2Service>("WBA3A5C55CF256789"));
        diagnostics.push_back(std::make_unique<CANDiagService>(1000));

        std::cout << "\nRunning all diagnostics in polymorphic container:" << std::endl;
        std::cout << "================================================" << std::endl;

        int passCount = 0;
        int totalCount = 0;
        for (const auto& svc : diagnostics) {
            std::cout << "\n>> " << svc->getName() << std::endl;
```

```
            if (svc->runDiagnostic()) {
                ++passCount;
            }
            ++totalCount;
        }

        std::cout << "\n==============================================" << std::endl;
        std::cout << "Diagnostic Summary: " << passCount << "/" << totalCount
                  << " passed." << std::endl;

        // EN: When 'diagnostics' vector goes out of scope, unique_ptr automatically deletes each
        // service. Zero memory leaks.
        // TR: 'diagnostics' vektörü kapsam dışına çıkınca unique_ptr her servisi otomatik siler.
        // Sıfır bellek sızıntısı.
    }

    return 0;
}
```

► 03. Rule Of Three Five

module_02_oop/03_rule_of_three_five.cpp

Konu: OOP Part 3: Memory Leaks and "Rule of 3 — 5" — OOP Kisim 3: Bellek Sizintilari ve

Teori / Theory

THEORY: Why does the Rule of 0/3/5 exist?

When you create a basic Class, the C++ compiler automatically generates 3 things for you:
1. A default Destructor
2. A default Copy Constructor (Sensor s2 = s1;)
3. A default Copy Assignment Operator (p2 = p1;)
This is fine for simple variables (ints, floats). BUT! If your class has a POINTER that allocates memory dynamically (using `new` or `malloc`), the default copy performs a "Shallow Copy". Both objects will point to the SAME memory! When they are destroyed, they will "Double Free" and crash!

TEORİ: 0/3/5 Kuralı neden var? (Sığ Kopya Krizi)

Temel bir sınıf yazdığınızda, C++ derleyicisi otomatik olarak 3 şey üretir:
1. Varsayılan Yıkıcı (Destructor)
2. Varsayılan Kopya Yapıcı (Copy Constructor) (Sensor s2 = s1;)
3. Varsayılan Kopya Atama Operatörü (Copy Assignment) (p2 = p1;)
Basit değişkenler (int, float) için bu yeterlidir. AMA! Sınıfınızda dinamik bellek tahsis eden (`new` veya `malloc` kullanan) bir POINTER varsa, varsayılan kopya "Sığ Kopya (Shallow Copy)" yapar. Her iki nesne de AYNI belleği işaret eder! Yıkıldıklarında "Double Free" olur ve program çöker!

RULE OF 3

"If a class needs a user-defined Destructor (i.e. manual `delete`), then you MUST also define these two yourself:"
1. Destructor
2. Copy Constructor
3. Copy Assignment Operator

3'LU KURAL

"Eğer bir sınıfın kullanıcı tanımlı bir Yıkıcıya (yani manuel `delete`) ihtiyacı varsa, şu ikisini de KENDİNİZ tanımlamalısınız:"
1. Yıkıcı (Destructor)
2. Kopya Yapıcı (Copy Constructor)
3. Kopya Atama Operatörü (Copy Assignment Operator)

CPPREF Depth: The Rule of 5 and `noexcept` Move

C++11 introduced Move Constructors and Move Assignment (Rule of 5). CppReference strongly mandates: YOU MUST mark your Move functions as `noexcept`! If `noexcept` is missing, `std::vector` reallocation will refuse to Move your objects and will fall back to painfully slow Deep Copies!

CPPREF Derinlik: 5'li Kural ve `noexcept` Taşıma

C++11 ile "Move (Tasima)" mimarisi geldi ve Kural 5'e cikti. CppReference der ki: Tasima (Move) metotlarinizi KESINLIKLE `noexcept` (Hatalara Kapali) olarak isaretlemelisiniz! Eger eksikse, `std::vector` kapasitesini artirirken korkuya kapilir ve Tasima yapmak yerine objelerinizi tek tek Kopyalar (Program %70 yavaslar)!

Kaynak Kod / Source Code

```cpp
#include <cstring>
#include <iostream>
#include <utility>

/**
 * @class StringBox
 * @brief EN: Custom primitive string class that manages dynamic memory.
 * TR: Dinamik bellek yoneten ilkel metin sinifi.
 */
class StringBox {
private:
  // EN: Dynamically allocated text
  // TR: Dinamik RAM
  char *data;
  size_t length;

public:
  // 1. Constructor
  StringBox(const char *sourceText) {
    length = std::strlen(sourceText);
    // EN: Dynamic heap allocation
    // TR: Heap uzerinde RAM
    data = new char[length +
                    1];
    std::strcpy(data, sourceText);
    std::cout << "[Constructor] Memory allocated for: " << data << " (Hafiza ayrildi)" << std::endl;
  }

  // EN: --- RULE OF 3 BEGINS! DESTRUCTOR is user-defined ---
  // TR: --- 3'LÜ KURAL BAŞLIYOR! YIKICI (DESTRUCTOR) kullanıcı tanımlıdır ---
  // EN: [1/3] RULE 1: DESTRUCTOR
  // TR: [1/3] KURAL 1: YIKICI (DESTRUCTOR)
  ~StringBox() {
    std::cout << "[Destructor] Deleting: " << data << " (Siliniyor)" << std::endl;
    delete[] data;
  }
```

```cpp

  // EN: --- IF WE WROTE DELETE ABOVE, WE MUST HAND-WRITE THESE ---
  // TR: --- YUKARIDA DELETE YAZDIYSAN, BUNLARI DA KENDİN YAZMAK ZORUNDASIN ---

  // EN: [2/3] RULE 2: COPY CONSTRUCTOR (Deep Copy)
  // TR: [2/3] KURAL 2: KOPYA OLUŞTURUCU (Derin Kopya)
  // EN: Example: StringBox B = A;
  // TR: Ornek: StringBox B = A;
  StringBox(const StringBox &other) {
    length = other.length;
    std::cout << "[Copy Constructor] Deep copying: " << other.data << " (Derin kopyalandi)" <<
        std::endl;

    // EN: NEVER do 'data = other.data' (Shallow Copy disaster)! Allocate NEW RAM!
    // TR: Asla 'data = other.data' yapmayin! Yeni RAM ac!
    data = new char[length + 1];
    std::strcpy(data, other.data);
  }

  // EN: [3/3] RULE 3: COPY ASSIGNMENT OPERATOR
  // TR: [3/3] KURAL 3: KOPYA ATAMA OPERATÖRÜ
  // EN: Example: B = A; (B already exists, must free old content first!)
  // TR: Ornek: B = A; (B zaten yasiyorken eski icerigi silinmeli!)
  StringBox &operator=(const StringBox &other) {
    // EN: Self-assignment guard
    // TR: Kendine atama korumasi
    if (this == &other)
      return *this;

    std::cout << "[Copy Assignment] Overwriting with: " << other.data << " (Uzerine yazildi)" <<
        std::endl;

    // EN: Free old memory first
    // TR: Eski bellek once silinmeli
    delete[] data;

    length = other.length;
    data = new char[length + 1];
    std::strcpy(data, other.data);

    return *this;
  }

  void print() const {
    std::cout << "Box Content (Kutu Icerigi): " << data << std::endl;
  }
};

// ==========================================================================================
// EN: ECUFirmwareBuffer — Full Rule of 5 Demonstration (Automotive Domain)
// TR: ECUFirmwareBuffer — Tam 5'li Kural Gösterimi (Otomotiv Alanı)
// ==========================================================================================

/**
 * @class ECUFirmwareBuffer
 * @brief EN: Simulates a firmware data buffer for an automotive ECU (Electronic Control Unit).
 * Demonstrates Rule of 5 with move semantics.
 * TR: Otomotiv ECU (Elektronik Kontrol Ünitesi) için firmware veri tamponunu simüle eder. Move
 * semantics ile 5'li Kuralı gösterir.
 */
class ECUFirmwareBuffer {
private:
    char* data_;
    size_t size_;
    std::string label_;

public:
    // EN: Parameterized Constructor — allocates firmware buffer on the heap
    // TR: Parametreli Yapıcı — firmware tamponunu heap üzerinde ayırır
    ECUFirmwareBuffer(size_t size, const std::string& label)
        : data_(new char[size]), size_(size), label_(label) {
        std::memset(data_, 0, size_);
        std::cout << "[Constructor] ECU buffer '" << label_
                  << "' allocated (" << size_ << " bytes)" << std::endl;
    }
```

```cpp

    // ─── 1. Destructor ──────────────────────────────────────────────────────────
    // EN: [1/5] Destructor — releases the heap-allocated firmware buffer
    // TR: [1/5] Yıkıcı — heap'te ayrılan firmware tamponunu serbest bırakır
    ~ECUFirmwareBuffer() {
        if (data_) {
            std::cout << "[Destructor] Deleting ECU buffer '" << label_
                      << "' (" << size_ << " bytes freed)" << std::endl;
            delete[] data_;
        } else {
            std::cout << "[Destructor] ECU buffer '" << label_
                      << "' was moved-from, nothing to free" << std::endl;
        }
    }

    // ─── 2. Copy Constructor (Deep Copy) ────────────────────────────────────────
    // EN: [2/5] Copy Constructor — performs a deep copy of the firmware buffer
    // TR: [2/5] Kopya Yapıcı — firmware tamponunun derin kopyasını yapar
    ECUFirmwareBuffer(const ECUFirmwareBuffer& other)
        : data_(new char[other.size_]), size_(other.size_), label_(other.label_ + "_copy") {
        std::memcpy(data_, other.data_, size_);
        std::cout << "[Copy Constructor] Deep copied '" << other.label_
                  << "' -> '" << label_ << "' (" << size_ << " bytes)" << std::endl;
    }

    // ─── 3. Copy Assignment Operator (Copy-and-Swap Idiom) ──────────────────────
    // EN: [3/5] Copy Assignment — uses copy-and-swap for exception safety
    // TR: [3/5] Kopya Atama — istisna güvenliği için copy-and-swap kullanır
    ECUFirmwareBuffer& operator=(const ECUFirmwareBuffer& other) {
        if (this == &other) return *this;
        std::cout << "[Copy Assignment] Overwriting '" << label_
                  << "' with '" << other.label_ << "'" << std::endl;

        // EN: Create a temporary deep copy, then swap resources
        // TR: Geçici derin kopya oluştur, ardından kaynakları takas et
        ECUFirmwareBuffer temp(other);
        std::swap(data_, temp.data_);
        std::swap(size_, temp.size_);
        std::swap(label_, temp.label_);
        return *this;
    }

    // ─── 4. Move Constructor (C++11) ────────────────────────────────────────────
    // EN: [4/5] Move Constructor — steals resources from source, sets it to null
    // TR: [4/5] Taşıma Yapıcı — kaynağın kaynaklarını çalar, kaynağı null yapar
    // EN: noexcept is CRITICAL: without it, std::vector refuses to move and falls back to slow
    // deep copies during reallocation!
    // TR: noexcept KRİTİKTİR: olmadan std::vector taşıma yapmayı reddeder ve yeniden tahsiste
    // yavaş derin kopyalara döner!
    ECUFirmwareBuffer(ECUFirmwareBuffer&& other) noexcept
        : data_(other.data_), size_(other.size_), label_(std::move(other.label_)) {
        std::cout << "[Move Constructor] Moved resources from '" << label_
                  << "' (" << size_ << " bytes stolen)" << std::endl;
        other.data_ = nullptr;
        other.size_ = 0;
    }

    // ─── 5. Move Assignment Operator (C++11) ────────────────────────────────────
    // EN: [5/5] Move Assignment — swaps/steals resources from source
    // TR: [5/5] Taşıma Atama — kaynağın kaynaklarını takas eder/çalar
    ECUFirmwareBuffer& operator=(ECUFirmwareBuffer&& other) noexcept {
        std::cout << "[Move Assignment] Moving '" << other.label_
                  << "' into '" << label_ << "'" << std::endl;
        if (this != &other) {
            // EN: Release our current resources first
            // TR: Önce mevcut kaynaklarımızı serbest bırak
            delete[] data_;

            // EN: Steal resources from the source
            // TR: Kaynaklarını kaynaktan çal
            data_ = other.data_;
            size_ = other.size_;
            label_ = std::move(other.label_);

```

```
            // EN: Nullify the source so its destructor is safe
            // TR: Kaynağı null yap ki yıkıcısı güvenli olsun
            other.data_ = nullptr;
            other.size_ = 0;
        }
        return *this;
    }

    // EN: Print current buffer info
    // TR: Mevcut tampon bilgisini yazdır
    void print() const {
        std::cout << "ECU Buffer Label: " << label_
                  << " | Size: " << size_
                  << " | Data ptr: " << static_cast<const void*>(data_) << std::endl;
    }
};

// EN: Factory function — may trigger RVO (Return Value Optimization) or move
// TR: Fabrika fonksiyonu — RVO (Dönüş Değeri Optimizasyonu) veya taşıma tetikler
ECUFirmwareBuffer createECUBuffer(size_t size, const std::string& label) {
    ECUFirmwareBuffer buf(size, label);
    return buf;
}

int main() {
  std::cout << "=== MODULE 2: RULE OF 3 / 5 AND MEMORY MANAGEMENT ===\n" << std::endl;

  StringBox originalFile("Top_Secret_Password");
  originalFile.print();

  // EN: Copy Constructor is invoked here! If we hadn't written Rule of 3 and the compiler's
  // default Shallow Copy ran, 'copyFile' would only steal the pointer (address) of 'Top_Secret'.
  // When main() exits and 'originalFile' is destroyed, that address becomes invalid. Then
  // 'copyFile' tries to delete the SAME address again (Double Free) and OS assassinates the
  // program with SIGSEGV!
  // TR: Copy Constructor burada cagrildi!
  StringBox copyFile = originalFile;

  std::cout << "----------------------" << std::endl;
  copyFile.print();

  // EN: Both are safely destroyed as separate physical memory blocks.
  // TR: Ikisi de ayri fiziksel bellek bloklari olarak guvenle yikildi.

  // ═══════════════════════════════════════════════════════════════════════════════
  // EN: RULE OF 5 — ECUFirmwareBuffer Demonstrations
  // TR: 5'Lİ KURAL — ECUFirmwareBuffer Gösterimleri
  // ═══════════════════════════════════════════════════════════════════════════════

  std::cout << "\n\n=== RULE OF 5: ECU FIRMWARE BUFFER DEMOS ===\n" << std::endl;

  // ─── 1. Move Construction ──────────────────────────────────────────────────
  // EN: The Move Constructor is invoked when initializing from a temporary.
  // TR: Geçici bir nesneden başlatılırken Taşıma Yapıcı çağrılır.
  std::cout << "--- Demo: Rule of 5 — Move Construction ---" << std::endl;
  {
      ECUFirmwareBuffer engineECU(1024, "EngineECU_v3.2");
      engineECU.print();

      // EN: Move constructor steals engineECU's guts — engineECU becomes empty
      // TR: Taşıma yapıcı engineECU'nun içini çalar — engineECU boş kalır
      ECUFirmwareBuffer transferredECU(std::move(engineECU));
      std::cout << "After move construction:" << std::endl;
      transferredECU.print();
      engineECU.print();
  }
  std::cout << std::endl;
```

```cpp

    // ─── 2. Move Assignment ──────────────────────────────────────────────────────────
    // EN: Move Assignment transfers ownership between two existing objects.
    // TR: Taşıma Atama, iki mevcut nesne arasında sahipliği aktarır.
    std::cout << "--- Demo: Rule of 5 — Move Assignment ---" << std::endl;
    {
        ECUFirmwareBuffer absModule(512, "ABS_Controller_v1.0");
        ECUFirmwareBuffer escModule(256, "ESC_Module_v2.1");
        absModule.print();
        escModule.print();

        // EN: Move assignment — escModule steals absModule's resources
        // TR: Taşıma atama — escModule, absModule'un kaynaklarını çalar
        escModule = std::move(absModule);
        std::cout << "After move assignment:" << std::endl;
        escModule.print();
        absModule.print();
    }
    std::cout << std::endl;

    // ─── 3. std::move Explicit Transfer ──────────────────────────────────────────────
    // EN: std::move does NOT move anything — it merely casts an lvalue to an rvalue reference,
    // enabling the move constructor/assignment to be called.
    // TR: std::move hiçbir şeyi taşımaz — yalnızca bir lvalue'yu rvalue referansına dönüştürerek
    // taşıma yapıcının/atamanın çağrılmasını sağlar.
    std::cout << "--- Demo: Rule of 5 — std::move Explicit Transfer ---" << std::endl;
    {
        ECUFirmwareBuffer transmissionECU(2048, "Transmission_TCU_v4.0");
        transmissionECU.print();

        // EN: After std::move, transmissionECU is in a valid-but-unspecified state
        // TR: std::move sonrası transmissionECU geçerli-ama-belirtilmemiş durumda
        ECUFirmwareBuffer newOwner = std::move(transmissionECU);
        std::cout << "New owner after explicit std::move:" << std::endl;
        newOwner.print();

        // EN: transmissionECU is now hollow — data_ is nullptr
        // TR: transmissionECU artık boş — data_ nullptr
        std::cout << "Original after std::move (moved-from state):" << std::endl;
        transmissionECU.print();
    }
    std::cout << std::endl;

    // ─── 4. Factory Function (RVO / Move) ────────────────────────────────────────────
    // EN: When returning a local object, the compiler may apply RVO (Return Value Optimization)
    // and construct directly in the caller's space. If RVO is not applied, the move constructor is
    // used instead of a deep copy.
    // TR: Yerel bir nesne döndürülürken, derleyici RVO (Dönüş Değeri Optimizasyonu) uygulayabilir
    // ve doğrudan çağıranın alanında oluşturur. RVO uygulanmazsa, derin kopya yerine taşıma yapıcı
    // kullanılır.
    std::cout << "--- Demo: Rule of 5 — Factory Function (RVO/Move) ---" << std::endl;
    {
        ECUFirmwareBuffer airbagECU = createECUBuffer(4096, "Airbag_SRS_v5.1");
        std::cout << "Factory-created buffer:" << std::endl;
        airbagECU.print();
    }

    return 0;
}
```

► 04. Multiple Inheritance

module_02_oop/04_multiple_inheritance.cpp

Konu: OOP Part 4: Multiple Inheritance & The Diamond Problem — Çoklu Kalıtım ve Elmas

Teori / Theory

THEORY: Why is Multiple Inheritance Banned in Many Languages?

C++ is one of the very few languages that allows a class to have TWO or more Base classes. `class FlyingCar : public Car, public Airplane`. This sounds cool but causes a deadly issue known as the "Diamond Problem". Java and C# banned this entirely, relying on Interfaces instead.

TEORİ: Çoklu Kalıtım Neden Yasaktır?

C++, bir sınıfın AYNI ANDA İKİ farklı temel sınıftan (Base) miras almasına olanak tanıyan nadir dillerdendir. Örneğin Uçan Arabamız (FlyingCar) var; hem KaradaGiden hem de HavadaUcan sınıflarından kalıtım alır. Ancak bu harika görünen özellik, "Elmas Problemi (Diamond Problem)" denen felaketi doğurur. Java veya C# bunu tamamen yasaklamış, yerine `Interface` (Arayüz) zorunluluğu getirmiştir.

THEORY: The Diamond Problem & Virtual Inheritance

Imagine: A Base class 'Device' exists (with an int 'power').
- 'Printer' inherits from Device.
- 'Scanner' inherits from Device.
- 'Copier' inherits from BOTH Printer and Scanner! Result: 'Copier' now contains TWO sets of 'power'! A waste of RAM and ambiguous. C++ solves this using `virtual public` inheritance: "Only make 1 copy of Device."

TEORİ: Elmas Problemi ve Virtual Kalıtım

Düşünün: 'Alet' (Device) adında bir Ana sınıf var (içinde 'int power' eklentisi olsun).
- 'Yazici' (Printer), Alet'ten miras alır.
- 'Tarayici' (Scanner), Alet'ten miras alır.
- 'HepsiBirArada' (Copier), hem Yazici'dan hem Tarayici'dan miras alır!

Sonuç: HepsiBirArada nesnesinin içinde İKİ ADET 'power' değişkeni oluşur! Bir RAM israfı ve Belirsizlik! C++ bunu çözmek için `virtual public` kalıtımını icat etti. "Güç değişkeninden sadece 1 adet kopyala."

CPPREF Depth: The Diamond Problem and Virtual Base Classes — Memory Layout

With normal (non-virtual) multiple inheritance, each path to a shared base class creates a separate sub-object. If both Printer and Scanner inherit from Device without `virtual`, Copier contains TWO Device sub-objects — sizeof(Copier) includes both. Virtual inheritance changes the ABI layout fundamentally. Instead of embedding the base sub-object directly, each virtual-inheriting class stores a VBASE OFFSET (or vbase pointer) inside its vtable. This offset tells the runtime where the single shared virtual base resides relative to the current sub-object. The shared Device is placed at the END of the most-derived object's memory layout. Construction order with virtual bases: ALL virtual bases are constructed FIRST, in left-to-right, depth-first order — and only by the MOST DERIVED class's constructor (not intermediate classes). Then non-virtual bases are constructed left-to-right, then the class itself. sizeof() increases with virtual inheritance due to the hidden vptr or vbase-offset pointers added by the compiler. On a typical 64-bit ABI, each virtual inheritance level adds 8 bytes per sub-object for the vptr. This is why `dynamic_cast` across virtual bases is more expensive — it must traverse offsets stored in the vtable to locate the target.

CPPREF Derinlik: Elmas Problemi ve Sanal Temel Sınıflar — Bellek Düzeni

Normal (sanal olmayan) çoklu kalıtımda, paylaşılan bir temel sınıfa giden her yol ayrı bir alt nesne oluşturur. Printer ve Scanner, Device'tan `virtual` olmadan miras alırsa, Copier İKİ Device alt nesnesi içerir. Sanal kalıtım ABI düzenini temelden değiştirir. Temel alt nesneyi doğrudan gömmek yerine, sanal miras alan her sınıf vtable'ında bir VBASE OFFSET (sanal taban ofseti) saklar. Bu ofset, paylaşılan sanal tabanın geçerli alt nesneye göre nerede bulunduğunu çalışma zamanında belirtir. Paylaşılan Device, en türetilmiş nesnenin bellek düzeninin SONUNA yerleştirilir. Sanal tabanlarda yapılandırma sırası: TÜM sanal tabanlar İLK ÖNCE, soldan sağa ve derinlik öncelikli sırada — yalnızca EN TÜRETİLMİŞ sınıfın yapıcısı tarafından oluşturulur. Sonra sanal olmayan tabanlar soldan sağa yapılandırılır, ardından sınıfın kendisi. sizeof() sanal kalıtımla artar çünkü derleyici gizli vptr veya vbase-ofset işaretçileri ekler. Tipik bir 64-bit ABI'de her sanal kalıtım seviyesi alt nesne başına 8 bayt ekler.

→ Ref: https://en.cppreference.com/w/cpp/language/derived_class

Kaynak Kod / Source Code

```cpp
#include <iostream>

// EN: 1. TOP LEVEL ROOT CLASS
// TR: 1. KÖK SINIF
class Device {
public:
  int powerConsumption;
  Device() : powerConsumption(100) {
    std::cout << "[Device] Root created. / Kök üretildi." << std::endl;
  }
};

// EN: 2. SUBCLASS 1 (Virtual Inheritance!)
// TR: 2. ALT SINIF 1 (Virtual Kalıtım!)
// EN: If we do not say 'virtual' here, the final child class will copy 'powerConsumption' twice!
// TR: Eğer 'virtual' demezsek, en aşağıdaki sentez cihaz kendi içinde 'powerConsumption'dan İKİ
// KOPYA üretir.
class Printer : virtual public Device {
public:
  Printer() {
    std::cout << "[Printer] Print capability added. / Yazdırma yeteneği eklendi." << std::endl;
  }
  void print() { std::cout << "Printing... / Baskı yapılıyor..." << std::endl; }
};

// EN: 3. SUBCLASS 2 (Virtual Inheritance!)
// TR: 3. ALT SINIF 2 (Virtual Kalıtım!)
class Scanner : virtual public Device {
public:
  Scanner() {
    std::cout << "[Scanner] Scan capability added. / Tarama yeteneği eklendi." << std::endl;
  }
  void scan() { std::cout << "Scanning... / Belge taranıyor..." << std::endl; }
};

// EN: 4. THE NIGHTMARE: MULTIPLE INHERITANCE
// TR: 4. KABUS: ÇOKLU KALITIM
class AllInOneCopier : public Printer, public Scanner {
public:
  AllInOneCopier() {
    std::cout << "[AllInOneCopier] Synthesis ready! / Sentez cihaz hazır!" << std::endl;
  }
```

```cpp

  void showPower() {
    // EN: If we hadn't used 'virtual' inheritance above, 'powerConsumption' would be AMBIGUOUS
    // and the code WOULD NOT COMPILE. (It wouldn't know which 'power' to access).
    //
    // TR: Eğer yukarıda 'virtual' kullanmasaydık, 'powerConsumption' AMBIGUOUS (Belirsiz)
    // olacaktı ve kod DERLENMEYECEKTİ (Hangisinin 'power'ına ulaşmak istediğimizi
    // bilemeyecekti).
    std::cout << "Power Consumption (Güç Tüketimi): " << powerConsumption <<
        " Watt (Diamond Problem Solved / Problem çözüldü!)" << std::endl;
  }
};

int main() {
  std::cout << "=== MODULE 2: MULTIPLE INHERITANCE (DIAMOND PROBLEM) ===\n" << std::endl;

  AllInOneCopier machine;

  std::cout << "\nMachine Tests / Makine Testleri:" << std::endl;
  machine.print();
  machine.scan();
  machine.showPower();

  return 0;
}
```

► 05. Encapsulation Abstraction

module_02_oop/05_encapsulation_abstraction.cpp

Konu: OOP Part 5: Encapsulation & Abstraction — OOP Kısım 5: Kapsülleme ve Soyutlama

Teori / Theory

THEORY: The 4 Pillars of OOP

1. Inheritance -> Prevents code duplication (DRY).
2. Polymorphism -> Provides dynamic flexibility through the V-Table.
3. Encapsulation -> Data hiding and security shield.
4. Abstraction -> Hides the "How" and only shows the "What".

TEORİ: 4 Büyük OOP Sütunu

1. Kalıtım (Inheritance) -> Kod tekrarını önler (DRY).
2. Çok Biçimlilik (Polymorphism) -> V-Table ile dinamik esneklik sağlar.
3. Kapsülleme (Encapsulation) -> Veri gizleme ve güvenlik kalkanıdır.
4. Soyutlama (Abstraction) -> Sadece "Neyin" yapılacağını gösterir, "Nasıl" yapıldığını gizler.

THEORY: Why is Encapsulation INDISPENSABLE?

Imagine a 'BankAccount' class. If the 'balance' variable is public, ANYONE from anywhere in the code can do: `account.balance = 1000000;`. This breaks reality! Money should only change through `deposit()` or `withdraw()`. Encapsulation hides the data (`private`) and exposes safety checks (`public` methods).

TEORİ: Kapsülleme Neden Vazgeçilmezdir?

Bir 'BankaHesabi' sınıfı düşünün. Eğer 'bakiye' değişkeni 'public' (açık) olursa, kodun herhangi bir yerinden herhangi biri şunu yazabilir: `hesap.bakiye = 1000000;`. Mevcut gerçekliği yıkar! Para SADECE yasal yollarla `paraYatir()` veya `paraCek()` fonksiyonları üzerinden değişmelidir. Kapsülleme, veriyi (`private`) gizler ve dünyaya sadece bizim yazdığımız güvenlik fonksiyonlarını (`public`) sunar (Getter ve Setter mantığı).

THEORY: What is Abstraction?

When you drive a car, do you need to know the piston angles or spark plug timings? No. You just press the gas pedal (Interface). That's Abstraction. "Hide the complex internal mechanism, expose a simple interface." "Interface Classes" created with Pure Virtual Functions (= 0) are the peak of this.

TEORİ: Soyutlama Nedir?

Bir araba kullanırken "Motorun piston açılarını" veya "Bujilerin ateşleme milisaniyesini" bilmek zorunda mısınız? Hayır. Sadece gaza (Interface/Arayüz) basarsınız. Abstraction (Soyutlama) budur. "Karmaşık iç mekanizmayı gizle, sadece kullanımı basit olan dış yüzü göster." Saf Sanal Fonksiyonlar (= 0) ile yaratılan "Interface Sınıflar" bunun zirvesidir.

CPPREF Depth: Access Specifiers and Friend — Compiler Enforcement vs Runtime

Access specifiers (`public`, `private`, `protected`) are COMPILE-TIME ONLY checks. The compiler rejects `obj.privateVar` in source code, but at runtime ALL memory is flat and accessible — a raw pointer cast can read any byte. This is by design: C++ trusts the programmer after compilation. Security comes from the type system, not from memory hiding. `friend` is not a design flaw — it is a CONTROLLED GATE. When two classes are tightly coupled by design (e.g., an iterator and its container, or `operator<<` for a class), `friend` grants precise access without exposing internals to the entire world via public getters. The PIMPL (Pointer to IMPLementation) idiom achieves ABI-stable encapsulation: the public header contains only a forward-declared Impl struct and a `std::unique_ptr<Impl>`. Changing Impl's members does NOT change the public header's sizeof — downstream code need not recompile. This is critical in shared libraries and automotive ECU software where ABI breaks cause fleet-wide update cascades. Note: `protected` is weaker than `private` — any derived class in any translation unit can access protected members, making it a wider trust boundary than many developers realize.

CPPREF Derinlik: Erişim Belirteçleri ve Friend — Derleyici Zorlama vs Çalışma Zamanı

Erişim belirteçleri (`public`, `private`, `protected`) YALNIZCA DERLEME ZAMANI kontrolleridir. Derleyici kaynak kodda `obj.privateVar` erişimini reddeder, ancak çalışma zamanında TÜM bellek düzdür ve erişilebilirdir — ham bir işaretçi dönüşümü herhangi bir baytı okuyabilir. Bu tasarım gereğidir: C++ derleme sonrasında programcıya güvenir. `friend` bir tasarım hatası değildir — KONTROLLÜ bir GEÇİTTİR. İki sınıf tasarım gereği sıkı bağlıysa (örn. yineleyici ve konteyneri veya bir sınıf için `operator<<`), `friend` tüm dünyaya public getter'lar ile iç yapıyı açmadan kesin erişim sağlar. PIMPL deyimi ABI-kararlı kapsülleme sağlar: public başlık yalnızca ileri bildirilmiş bir Impl yapısı ve `std::unique_ptr<Impl>` içerir. Impl'in üyelerini değiştirmek public başlığın sizeof değerini DEĞİŞTİRMEZ — bağımlı kodun yeniden derlenmesi gerekmez. Not: `protected`, `private`'den daha zayıftır — herhangi bir çeviri birimindeki türetilmiş sınıf protected üyelere erişebilir.

→ Ref: https://en.cppreference.com/w/cpp/language/access

Kaynak Kod / Source Code

```
#include <iostream>
#include <string>

// EN: --- ENCAPSULATION EXAMPLE ---
// TR: --- KAPSÜLLEME ÖRNEĞİ ---
class BankAccount {
private: // DATA HIDING / VERİ GİZLEME
  std::string ownerName;
  double balance;

```

```
11    // EN: Internal private helper function. CANNOT be called from outside. (Abstraction)
12    // TR: Özel bir yardımcı fonksiyondur. Dışarıdan ÇAĞIRILAMAZ. (Soyutlama içerir)
13    bool isTransactionValid(double amount) const { return amount > 0; }
14
15  public:
16    BankAccount(std::string name, double initialDeposit)
17        : ownerName(name), balance(initialDeposit) {}
18
19    // EN: GETTER method retrieving the encapsulated real data (Read-Only access):
20    // TR: Kapsüllenen veriyi dışarı (sadece OKUMAK için) çıkaran GETTER metodu:
21    double getBalance() const {
22      // [Look, but don't touch! / Göster, ama dokundurma!]
23      return balance;
24    }
25
26    // EN: SETTER method modifying the encapsulated data through rules:
27    // TR: Kapsüllenen veriyi kurallarıyla değiştiren SETTER / İşlem metodu:
28    void deposit(double amount) {
29      if (isTransactionValid(amount)) {
30        balance += amount;
31        std::cout << "[Encapsulation] Safe deposit successful. Amount: " << amount <<
32            " / Güvenli para yatırma işlemi başarılı." << std::endl;
33      } else {
34        std::cout << "[ERROR/HATA] Cannot deposit zero or negative funds! (Sıfır "
35                    "veya eksi yatırılamaz!)"
36                  << std::endl;
37      }
38    }
39  };
40
41  // --- ABSTRACTION EXAMPLE / SOYUTLAMA ÖRNEĞİ ---
42
43  /**
44   * @class IEspressoMachine
45   * @brief EN: 'I' prefix indicates it is an Interface.
46   * TR: 'I' ön eki Interface (Arayüz) olduğunu belirtir.
47   *
48   * EN: Contains a "Pure Virtual" function (= 0). This class has NO body. You CANNOT create an
49   * object like 'new IEspressoMachine()'. It is just a "CONTRACT".
50   *
51   * TR: Saf Sanal Fonksiyon (Pure Virtual) içerir. Yani bu sınıfın BEDENİ yoktur. Ondan 'new
52   * IEspressoMachine()' diye nesne ÜRETEMEZSİNİZ. O sadece bir "TAAHHÜTNAME"dir.
53   */
54  class IEspressoMachine {
55  public:
56    virtual ~IEspressoMachine() = default;
57
58    // EN: '= 0' means just show the rule (interface), no body!
59    // TR: '= 0' demek SADECE kuralı (arayüzü) göster, gövdesi yok demektir!
60    virtual void makeCoffee() = 0;
61  };
62
63  // EN: The concrete class implementing the real mechanism (the 'How' details).
64  // TR: Gerçek mekanizmayı uygulayan (Nasıl çalıştığını yazan) Sınıf.
65  class ModernCoffeeMaker : public IEspressoMachine {
66  private:
67    void grindBeans() {
68      std::cout << "1. Grinding beans (0.3mm)... / Çekirdekler öğütülüyor..." << std::endl;
69    }
70    void boilWater() {
71      std::cout << "2. Boiling water to 94C... / Su 94 dereceye isitiliyor..." << std::endl;
72    }
73    void filterEspresso() {
74      std::cout << "3. Filtering with 9 Bars... / 9 Bar basınç ile süzülüyor!" << std::endl;
75    }
76
77  public:
78    void makeCoffee() override {
79      // EN: The Core of Abstraction! User ONLY calls makeCoffee, they don't see the complex 3
80      // steps.
81      // TR: Soyutlamanın Çekirdeği! Kullanıcı SADECE makeCoffee çağırır, şu 3 karmaşık adımı
82      // BİLMEZ/GÖRMEZ.
83      grindBeans();
84      boilWater();
85      filterEspresso();
86      std::cout << "[Result / Sonuç] Your espresso is ready! (Espresso hazır!)\n" << std::endl;
87    }
```

```
88  };
89
90  int main() {
91    std::cout << "=== MODULE 2: ENCAPSULATION & ABSTRACTION ===\n" << std::endl;
92
93    // EN: 1. Encapsulation Test
94    // TR: 1. Kapsülleme Testi
95    std::cout << "--- 1. ENCAPSULATION (Kapsülleme) ---" << std::endl;
96    BankAccount myAccount("Alice", 1000.0);
97
98    // myAccount.balance = 9999999; // ERROR! private veriye asla dokunamazsın.
99    std::cout << "Current Balance (Mevcut Para): " << myAccount.getBalance() << std::endl;
100
101   // EN: Security shield engages!
102   // TR: Sistem güvenlik kalkanını devreye sokar!
103   myAccount.deposit(-500);
104
105   // 2. Abstraction Test (Soyutlama Testi)
106   std::cout << "\n--- 2. ABSTRACTION (Soyutlama) ---" << std::endl;
107
108   // EN: We use the coffee machine through the IEspressoMachine interface. We don't worry about
109   // pistons, pressure, or temp. We just press the button.
110   // TR: IEspressoMachine arayüzü ile makineyi kullanırız. Pistonlar veya sıcaklık bizi
111   // ilgilendirmez. Sadece düğmeye basarız.
112
113   IEspressoMachine *myMachine = new ModernCoffeeMaker();
114   myMachine->makeCoffee();
115
116   delete myMachine;
117
118   return 0;
119 }
```

► 06. Static And Friends

module_02_oop/06_static_and_friends.cpp

Konu: OOP Part 6: Static Members & Friend Functions — OOP Kısım 6: Statik Üyeler ve Friend

Teori / Theory

THEORY: Static Class Members

A `static` data member belongs to the CLASS itself, not to any individual object. All instances share the same copy. Think of a global configuration register inside an ECU — every software module reads and writes the same register, not a per-module copy. Declaring `static int errorCount;` in a class means there is exactly ONE `errorCount` for every object ever created. Static members must be defined (allocated) outside the class body unless they are `inline` or `constexpr`.

TEORİ: Statik Sınıf Üyeleri

Bir `static` veri üyesi, herhangi bir nesneye değil SINIFIN kendisine aittir. Tüm örnekler aynı kopyayı paylaşır. Bir ECU içindeki global konfigürasyon kaydını düşünün — her yazılım modülü aynı kaydı okur ve yazar, her modülün ayrı kopyası yoktur. Sınıf içinde `static int errorCount;` tanımladığınızda, oluşturulan tüm nesneler için TAM OLARAK BİR `errorCount` vardır. Statik üyeler, `inline` veya `constexpr` olmadıkça sınıf gövdesinin dışında tanımlanmalıdır (bellek ayrılmalıdır).

THEORY: Static Member Functions

A `static` member function can be called WITHOUT creating an object: `VehicleTelemetry::getFleetCount()`. It has NO `this` pointer, so it can only access other static members. Use it for factory methods, counters, or utility operations that logically belong to the class but need no instance.

TEORİ: Statik Üye Fonksiyonlar

Bir `static` üye fonksiyon, nesne oluşturmadan çağrılabilir: `VehicleTelemetry::getFleetCount()`. `this` işaretçisi YOKTUR, bu yüzden yalnızca diğer statik üyelere erişebilir. Fabrika metotları, sayaçlar veya mantıksal olarak sınıfa ait olan ancak bir örneğe ihtiyaç duymayan yardımcı işlemler için kullanılır.

THEORY: Friend Functions

A `friend` function is declared inside a class but is NOT a member. It can access all private and protected members. Think of a diagnostic tool that plugs into an ECU's OBD-II port — it needs direct access to private sensor readings and DTC logs that normal external code cannot see. The class explicitly grants trust by declaring `friend void diagnosticDump(...)`.

TEORİ: Friend (Arkadaş) Fonksiyonlar

Bir `friend` fonksiyon sınıf içinde bildirilir ancak üye DEĞİLDİR. Tüm private ve protected üyelere erişebilir. Bir ECU'nun OBD-II portuna takılan bir teşhis aracını düşünün — normal harici kodun göremeyeceği özel sensör okumalarına ve DTC kayıtlarına doğrudan erişmesi gerekir. Sınıf, `friend void diagnosticDump(...)` bildirerek güveni açıkça verir.

THEORY: Friend Classes

When you write `friend class CalibrationTool;` inside `Engine`, EVERY member function of `CalibrationTool` can access `Engine`'s private data. This is like a factory calibration station that must tune private engine parameters (fuel maps, timing offsets) — access that regular drivers never have. Friendship is NOT symmetric: Engine trusts CalibrationTool, but NOT vice versa. Friendship is NOT inherited.

TEORİ: Friend (Arkadaş) Sınıflar

`Engine` içinde `friend class CalibrationTool;` yazdığınızda, `CalibrationTool`'un TÜM üye fonksiyonları `Engine`'in private verilerine erişebilir. Bu, özel motor parametrelerini (yakıt haritaları, zamanlama ofsetleri) ayarlaması gereken bir fabrika kalibrasyon istasyonu gibidir — normal sürücülerin asla sahip olmadığı bir erişim. Arkadaşlık SİMETRİK DEĞİLDİR: Engine CalibrationTool'a güvenir, ama tersi geçerli değildir. Arkadaşlık miras ALINMAZ.

THEORY: When to Use

1. USE `friend` when two classes are tightly coupled BY DESIGN (e.g., an iterator and its container, a serializer and its data class).
2. DO NOT use `friend` to bypass poor encapsulation. If you need many friends, your design is probably wrong — add proper public/protected API.
3. `static` is great for counters, caches, and singletons. Beware of initialization order across translation units (the "static init fiasco").

Abuse Warning / TEORİ: Ne Zaman Kullanılır / Kötüye Kullanım Uyarısı

1. `friend` kullanın: İki sınıf TASARIM GEREĞİ sıkı bağlıysa (örn. bir yineleyici ve konteyneri, bir serileştirici ve veri sınıfı).
2. `friend` KULLANMAYIN: Zayıf kapsüllemeyi atlatmak için. Çok fazla friend'e ihtiyacınız varsa tasarımınız muhtemelen yanlıştır — uygun public/protected API ekleyin.
3. `static`, sayaçlar, önbellekler ve tekil nesneler (singleton) için harikadır. Çeviri birimleri arasında başlatma sırası konusunda dikkatli olun ("static init fiasco" sorunu).

CPPREF Depth: Static Members — Linkage, Initialization Order, and Thread Safety

Static class data members have EXTERNAL LINKAGE by default — the linker must find exactly one definition across all translation units (ODR). If you declare `static int count;` in a header included by 3 .cpp files without `inline`, you get a linker error: multiple definitions. The "Static Initialization Order Fiasco": globals and namespace-scope statics across different TUs have NO GUARANTEED construction order. If static A in file1.cpp depends on static B in file2.cpp, B might not be initialized yet → undefined behavior. The classic fix: use a local static inside a function ("Construct On First Use" idiom). C++11 §6.7 guarantees that local static variables are initialized in a THREAD-SAFE manner — the compiler inserts a hidden guard variable and mutex-like mechanism (often a spinlock or CAS). This makes the Meyer's Singleton pattern safe: `static T& instance() { static T t; return t; }` C++17 introduced `inline static` members, allowing in-class definition of static data members without a separate .cpp definition. This also guarantees a single definition across TUs — the linker deduplicates. `constexpr static` members are implicitly inline since C++17.

CPPREF Derinlik: Statik Üyeler — Bağlantı, Başlatma Sırası ve İş Parçacığı Güvenliği

Statik sınıf veri üyeleri varsayılan olarak DIŞ BAĞLANTIYA sahiptir — bağlayıcı tüm çeviri birimleri arasında tam olarak bir tanım bulmalıdır (ODR). Bir başlıkta `static int count;` bildirip 3 .cpp dosyasına dahil ederseniz `inline` olmadan bağlayıcı hatası alırsınız. "Statik Başlatma Sırası Felaketi": farklı çeviri birimlerindeki global ve namespace kapsamlı statikler GUARANTEED yapılandırma sırasına sahip DEĞİLDİR. Dosya1'deki statik A, dosya2'deki statik B'ye bağlıysa, B henüz başlatılmamış olabilir → tanımsız davranış. Klasik çözüm: fonksiyon içi yerel statik kullanmak ("İlk Kullanımda Oluştur" deyimi). C++11 §6.7, yerel statik değişkenlerin İŞ PARÇACIĞI GÜVENLİ biçimde başlatılacağını garanti eder — derleyici gizli bir koruma değişkeni ve mutex benzeri mekanizma ekler. Bu, Meyer's Singleton kalıbını güvenli kılar: `static T& instance() { static T t; return t; }` C++17, `inline static` üyeleri tanıttı ve sınıf içi tanıma izin verdi. `constexpr static` üyeler C++17'den beri örtük olarak inline'dır.

→ Ref: https://en.cppreference.com/w/cpp/language/static

Kaynak Kod / Source Code

```cpp
#include <iostream>   // EN: Standard I/O for console output
#include <string>     // EN: std::string for VIN and identifiers
#include <cassert>    // EN: Runtime assertions for fleet-size guard

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 1: VehicleTelemetry — static members & friend function
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Forward declaration of friend function so the class can reference it.
// TR: Sınıfın referans verebilmesi için friend fonksiyonun önceden bildirimi.
class VehicleTelemetry;
void diagnosticDump(const VehicleTelemetry& vt);

class VehicleTelemetry {
public:
    // EN: Static constexpr — compile-time constant shared by all instances.
    // TR: Static constexpr — tüm örnekler tarafından paylaşılan derleme zamanı sabiti.
    static constexpr int MAX_FLEET_SIZE = 100;

    // EN: Constructor registers vehicle into the fleet.
    // TR: Yapıcı, aracı filoya kaydeder.
    explicit VehicleTelemetry(std::string vin, double speed = 0.0)
        : vin_(std::move(vin)), lastSpeed_(speed) {
        assert(totalVehiclesTracked_ < MAX_FLEET_SIZE &&
               "Fleet capacity exceeded!");
        ++totalVehiclesTracked_;
    }
```

```cpp

    // EN: Destructor unregisters vehicle from the fleet.
    // TR: Yıkıcı, aracı filodan çıkarır.
    ~VehicleTelemetry() {
        --totalVehiclesTracked_;
    }

    // EN: Static member function — callable without an object.
    // TR: Statik üye fonksiyon — nesne olmadan çağrılabilir.
    static int getFleetCount() {
        return totalVehiclesTracked_;
    }

    // EN: Regular member to update speed telemetry.
    // TR: Hız telemetrisini güncellemek için normal üye fonksiyon.
    void updateSpeed(double newSpeed) {
        lastSpeed_ = newSpeed;
    }

    // EN: Grant friend access to the free function diagnosticDump.
    // TR: diagnosticDump serbest fonksiyonuna friend erişimi ver.
    friend void diagnosticDump(const VehicleTelemetry& vt);

private:
    // EN: Shared counter — one copy for all VehicleTelemetry objects.
    // TR: Paylaşılan sayaç — tüm VehicleTelemetry nesneleri için tek kopya.
    static int totalVehiclesTracked_;

    // EN: Per-instance private data: VIN and last recorded speed (km/h).
    // TR: Örneğe özel private veri: VIN ve son kaydedilen hız (km/sa).
    std::string vin_;
    double lastSpeed_;
};

// EN: Out-of-class definition of the static member (required for non-inline statics).
// TR: Statik üyenin sınıf dışı tanımı (inline olmayan statikler için gereklidir).
int VehicleTelemetry::totalVehiclesTracked_ = 0;

// ─── 1.1 Friend Function Implementation ──────────────────────────────────────────────

// EN: diagnosticDump can read private vin_ and lastSpeed_ because it is a friend.
// TR: diagnosticDump, friend olduğu için private vin_ ve lastSpeed_'e erişebilir.
void diagnosticDump(const VehicleTelemetry& vt) {
    std::cout << "[DIAG] VIN: " << vt.vin_
              << " | Speed: " << vt.lastSpeed_ << " km/h"
              << " | Fleet size: " << VehicleTelemetry::totalVehiclesTracked_
              << "\n";
}

// ═══════════════════════════════════════════════════════════════════════════════════════
// SECTION 2: Engine + CalibrationTool — friend class
// ═══════════════════════════════════════════════════════════════════════════════════════

// EN: Forward declaration so Engine can declare CalibrationTool as a friend.
// TR: Engine'in CalibrationTool'u friend olarak bildirebilmesi için önceden bildirim.
class CalibrationTool;

class Engine {
public:
    // EN: Construct with default factory parameters.
    // TR: Varsayılan fabrika parametreleriyle oluştur.
    explicit Engine(std::string ecuId)
        : ecuId_(std::move(ecuId)),
          fuelMapOffset_(0.0),
          timingAdvanceDeg_(12.0),
          boostPressureBar_(1.0) {}

    // EN: Public read-only interface — safe for any caller.
    // TR: Genel salt-okunur arayüz — herhangi bir çağrıcı için güvenli.
    void printStatus() const {
        std::cout << "[ENGINE " << ecuId_ << "] "
                  << "FuelOffset=" << fuelMapOffset_
                  << " | Timing=" << timingAdvanceDeg_ << "deg"
                  << " | Boost=" << boostPressureBar_ << "bar\n";
    }
```

```

    // EN: Grant full private access to CalibrationTool.
    // TR: CalibrationTool'a tam private erişim ver.
    friend class CalibrationTool;

private:
    std::string ecuId_;
    double fuelMapOffset_;     // EN: fuel-map trim offset
    double timingAdvanceDeg_;  // EN: ignition timing in degrees
    double boostPressureBar_;  // EN: turbo boost pressure in bar
};

class CalibrationTool {
public:
    // EN: Adjust fuel map offset — accesses Engine's private member directly.
    // TR: Yakıt haritası ofsetini ayarla — Engine'in private üyesine doğrudan erişir.
    static void adjustFuelMap(Engine& engine, double offset) {
        std::cout << "[CAL] Adjusting fuel map offset from "
                  << engine.fuelMapOffset_ << " to " << offset << "\n";
        engine.fuelMapOffset_ = offset;
    }

    // EN: Adjust ignition timing — accesses Engine's private member directly.
    // TR: Ateşleme zamanlamasını ayarla — Engine'in private üyesine doğrudan erişir.
    static void adjustTiming(Engine& engine, double degrees) {
        std::cout << "[CAL] Adjusting timing advance from "
                  << engine.timingAdvanceDeg_ << " to " << degrees << "deg\n";
        engine.timingAdvanceDeg_ = degrees;
    }

    // EN: Adjust boost pressure — accesses Engine's private member directly.
    // TR: Turbo basıncını ayarla — Engine'in private üyesine doğrudan erişir.
    static void adjustBoost(Engine& engine, double bar) {
        std::cout << "[CAL] Adjusting boost pressure from "
                  << engine.boostPressureBar_ << " to " << bar << "bar\n";
        engine.boostPressureBar_ = bar;
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 3: main() — Demonstrations
// ═══════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "=== Module 02 | Lesson 06: Static Members & Friends ===\n\n";

    // ── Demo 1: Static counter increments with each vehicle ──────────────
    // EN: Each VehicleTelemetry construction bumps the shared counter.
    // TR: Her VehicleTelemetry oluşturulduğunda paylaşılan sayaç artar.
    std::cout << "--- Demo 1: Static counter ---\n";
    std::cout << "Fleet count before any vehicle: "
              << VehicleTelemetry::getFleetCount() << "\n";

    VehicleTelemetry v1("WBA3A5C55DF123456", 85.0);
    std::cout << "After v1: fleet count = "
              << VehicleTelemetry::getFleetCount() << "\n";

    VehicleTelemetry v2("WVWZZZ3CZWE987654", 120.5);
    std::cout << "After v2: fleet count = "
              << VehicleTelemetry::getFleetCount() << "\n";

    {
        VehicleTelemetry v3("TMBJB9NE1L0123456");
        std::cout << "After v3 (scoped): fleet count = "
                  << VehicleTelemetry::getFleetCount() << "\n";
    }
    // EN: v3 destroyed when scope ends — counter decrements.
    // TR: Kapsam sona erdiğinde v3 yok edilir — sayaç azalır.
    std::cout << "After v3 destroyed: fleet count = "
              << VehicleTelemetry::getFleetCount() << "\n\n";
```

```

    // ─── Demo 2: Static method called without an object ──────────────────────────
    // EN: No object required — call directly on the class.
    // TR: Nesne gerekmez — doğrudan sınıf üzerinden çağrılır.
    std::cout << "--- Demo 2: Static method without object ---\n";
    std::cout << "VehicleTelemetry::getFleetCount() = "
              << VehicleTelemetry::getFleetCount() << "\n\n";

    // ─── Demo 3: Friend function accessing private data ──────────────────────────
    // EN: diagnosticDump reads private vin_ and lastSpeed_ directly.
    // TR: diagnosticDump, private vin_ ve lastSpeed_'i doğrudan okur.
    std::cout << "--- Demo 3: Friend function ---\n";
    v1.updateSpeed(92.3);
    diagnosticDump(v1);
    diagnosticDump(v2);
    std::cout << "\n";

    // ─── Demo 4: Friend class CalibrationTool modifying Engine ───────────────────
    // EN: CalibrationTool accesses Engine's private internals to tune params.
    // TR: CalibrationTool, parametreleri ayarlamak için Engine'in private içeriğine erişir.
    std::cout << "--- Demo 4: Friend class CalibrationTool ---\n";
    Engine myEngine("ECU-2026-TDI");
    std::cout << "Before calibration:\n";
    myEngine.printStatus();

    CalibrationTool::adjustFuelMap(myEngine, 2.5);
    CalibrationTool::adjustTiming(myEngine, 14.0);
    CalibrationTool::adjustBoost(myEngine, 1.8);

    std::cout << "After calibration:\n";
    myEngine.printStatus();
    std::cout << "\n";

    // ─── Demo 5: constexpr static constant usage ─────────────────────────────────
    // EN: MAX_FLEET_SIZE is a compile-time constant available without an instance.
    // TR: MAX_FLEET_SIZE, bir örneğe ihtiyaç duymadan kullanılabilen derleme zamanı sabitidir.
    std::cout << "--- Demo 5: constexpr static constant ---\n";
    std::cout << "Maximum fleet capacity: "
              << VehicleTelemetry::MAX_FLEET_SIZE << " vehicles\n";
    std::cout << "Current fleet utilization: "
              << VehicleTelemetry::getFleetCount() << " / "
              << VehicleTelemetry::MAX_FLEET_SIZE << "\n\n";

    std::cout << "=== All demos completed successfully. ===\n";
    return 0;
}
```

► 07. Nested And Local Classes

module_02_oop/07_nested_and_local_classes.cpp

Konu: OOP Part 7: Nested & Local Classes — OOP Kısım 7: İç İçe ve Yerel Sınıflar

Teori / Theory

THEORY: Nested Classes

A nested class is a class defined inside another class. The inner class logically belongs to the outer class and is scoped within it. Think of a CAN message: the Header (arbitration ID, DLC, flags) only makes sense as part of a CanMessage. Defining Header inside CanMessage keeps it tightly coupled and prevents polluting the global namespace.

TEORİ: İç İçe Sınıflar (Nested Classes)

İç içe sınıf (nested class), başka bir sınıfın içinde tanımlanan sınıftır. İç sınıf, mantıksal olarak dış sınıfa aittir ve onun kapsamında yer alır. Bir CAN mesajı düşünün: Header (arbitrasyon ID, DLC, bayraklar) yalnızca bir CanMessage'ın parçası olarak anlam taşır. Header'ı CanMessage içinde tanımlamak, sıkı bağlılığı korur ve global isim alanını kirletmez.

THEORY: Access Rules

Since C++11, a nested class can access the private and protected members of its enclosing class (given an instance of the outer class). The outer class, however, has NO special access to the nested class's private members. This asymmetry is important for encapsulation.

TEORİ: Erişim Kuralları

C++11'den beri, iç içe sınıf, dış sınıfın private ve protected üyelerine erişebilir (dış sınıfın bir örneği verildiğinde). Ancak dış sınıfın, iç sınıfın private üyelerine özel bir erişimi YOKTUR. Bu asimetri, kapsülleme açısından önemlidir.

THEORY: Local Classes

A local class is defined inside a function body. Its scope is limited to that function — it cannot be used outside. Local classes cannot access the function's automatic (local) variables but can access static locals and global variables. Use cases are rare; lambdas often replace them.

TEORİ: Yerel Sınıflar (Local Classes)

Yerel sınıf, bir fonksiyon gövdesi içinde tanımlanan sınıftır. Kapsamı o fonksiyonla sınırlıdır — dışarıda kullanılamaz. Yerel sınıflar, fonksiyonun otomatik (yerel) değişkenlerine erişemez ama statik yerel ve global değişkenlere erişebilir. Kullanım alanı nadirdir; lambdalar genellikle onların yerini alır.

THEORY: PIMPL Idiom

PIMPL (Pointer to IMPLementation) hides implementation details behind a pointer to a private nested class/struct (often called Impl). The public class header only forward-declares Impl, keeping compile-time dependencies minimal. This is widely used in automotive ECU codebases to stabilize ABIs.

TEORİ: PIMPL Kalıbı

PIMPL (Pointer to IMPLementation), uygulama detaylarını özel bir iç sınıf/yapıya (genellikle Impl olarak adlandırılır) yönelik bir işaretçinin arkasına gizler. Public sınıf başlığı yalnızca Impl'i ileri bildirir ve derleme zamanı bağımlılıklarını minimum tutar. Otomotiv ECU kod tabanlarında ABI kararlılığı sağlamak için yaygın olarak kullanılır.

CPPREF Depth: Nested Class Access Rules and the Pimpl Compilation Firewall

Before C++11, nested classes had NO special access to the enclosing class's private members — they were treated like any other class. The C++11 standard (§11.7) changed this: a nested class is now a MEMBER of the enclosing class and can access all private/protected names (given an instance of the outer class). This is what makes the Pimpl idiom clean. The PIMPL compilation firewall works because the public header only contains `class Impl;` (forward declaration) and `std::unique_ptr<Impl>`. Since unique_ptr's destructor requires Impl to be COMPLETE (it calls `delete`), you MUST define the outer class's destructor in the .cpp file where Impl is fully defined. Forgetting this causes a compile error: "incomplete type" in the unique_ptr destructor instantiation. PIMPL reduces transitive #include dependencies dramatically. A change to Impl's data members or private methods does NOT change the public header → no recompilation of dependent translation units. In large automotive codebases (millions of LOC), this saves hours of build time. Performance cost: every call to an Impl method is an extra pointer indirection, and Impl lives on the heap (cache-unfriendly). For hot paths, consider "fast Pimpl" with aligned storage or a fixed buffer.

CPPREF Derinlik: İç İçe Sınıf Erişim Kuralları ve Pimpl Derleme Güvenlik Duvarı

C++11 öncesinde iç içe sınıflar, çevreleyen sınıfın private üyelerine ÖZEL bir erişime sahip değildi. C++11 standardı (§11.7) bunu değiştirdi: iç içe sınıf artık çevreleyen sınıfın bir ÜYESİDİR ve tüm private/protected isimlere erişebilir (dış sınıfın bir örneği verildiğinde). PIMPL derleme güvenlik duvarı şöyle çalışır: public başlık yalnızca `class Impl;` (ileri bildirim) ve `std::unique_ptr<Impl>` içerir. unique_ptr'nin yıkıcısı Impl'in TAM olmasını gerektirir (`delete` çağırır), bu yüzden dış sınıfın yıkıcısını Impl'in tamamen tanımlandığı .cpp dosyasında tanımlamanız GEREKİR. Bunu unutmak derleme hatasına neden olur. PIMPL, geçişli #include bağımlılıklarını dramatik biçimde azaltır. Impl'in veri üyelerindeki değişiklik public başlığı DEĞİŞTİRMEZ → bağımlı çeviri birimlerinin yeniden derlenmesi gerekmez. Büyük otomotiv kod tabanlarında bu, saatlerce derleme süresinden tasarruf sağlar. Performans maliyeti: her Impl metodu çağrısı ekstra bir işaretçi yönlendirmesidir ve Impl heap'te yaşar (önbellek dostu değil).

→ Ref: https://en.cppreference.com/w/cpp/language/nested_types

Kaynak Kod / Source Code

```cpp
#include <iostream>   // EN: Standard I/O for console output
#include <string>     // EN: String class for text handling
#include <vector>     // EN: Dynamic array container
#include <array>      // EN: Fixed-size array container
#include <cstdint>    // EN: Fixed-width integer types (uint32_t, uint8_t)
#include <memory>     // EN: Smart pointers (unique_ptr) for PIMPL
#include <algorithm>  // EN: std::count_if for validation demo

```

```cpp
// ═══════════════════════════════════════════════════════════════════════════
// 1. CanMessage with Nested Header and Payload Classes
// ═══════════════════════════════════════════════════════════════════════════

// EN: A CAN message class with nested Header and Payload — mimics a real CAN 2.0 frame.
// TR: İç içe Header ve Payload içeren bir CAN mesaj sınıfı — gerçek bir CAN 2.0 çerçevesini
// taklit eder.
class CanMessage {
public:
    // ─── 1a. Nested Header ──────────────────────────────────────────────────
    class Header {
    public:
        Header() = default;
        Header(uint32_t id, uint8_t dlc, bool is_extended, bool is_rtr)
            : arb_id_(id), dlc_(dlc), extended_frame_(is_extended), rtr_(is_rtr) {}

        uint32_t getArbId()       const { return arb_id_; }
        uint8_t  getDlc()         const { return dlc_; }
        bool     isExtended()     const { return extended_frame_; }
        bool     isRtr()          const { return rtr_; }

        void setArbId(uint32_t id)      { arb_id_ = id; }
        void setDlc(uint8_t dlc)        { dlc_ = (dlc <= 8) ? dlc : 8; }
        void setExtended(bool ext)      { extended_frame_ = ext; }
        void setRtr(bool rtr)           { rtr_ = rtr; }

    private:
        uint32_t arb_id_        = 0x000;
        uint8_t  dlc_           = 0;
        bool     extended_frame_ = false;
        bool     rtr_           = false;
    };

    // ─── 1b. Nested Payload ─────────────────────────────────────────────────
    class Payload {
    public:
        Payload() { data_.fill(0x00); }

        void setByte(size_t index, uint8_t value) {
            if (index < 8) data_[index] = value;
        }
        uint8_t getByte(size_t index) const {
            return (index < 8) ? data_[index] : 0;
        }
        const std::array<uint8_t, 8>& raw() const { return data_; }

    private:
        std::array<uint8_t, 8> data_;
    };

    // ─── 1c. CanMessage public interface ────────────────────────────────────
    CanMessage() = default;
    CanMessage(uint32_t id, uint8_t dlc) {
        header_.setArbId(id);
        header_.setDlc(dlc);
    }

    Header&       header()        { return header_; }
    const Header& header()  const { return header_; }
    Payload&      payload()       { return payload_; }
    const Payload& payload() const { return payload_; }

    void print() const {
        std::cout << "CAN Frame [ID=0x" << std::hex << header_.getArbId()
                  << std::dec << " DLC=" << static_cast<int>(header_.getDlc())
                  << " EXT=" << header_.isExtended()
                  << " RTR=" << header_.isRtr() << "] Data: ";
        for (uint8_t i = 0; i < header_.getDlc(); ++i) {
            std::cout << "0x" << std::hex
                      << static_cast<int>(payload_.getByte(i)) << " ";
        }
        std::cout << std::dec << "\n";
    }
```

```

private:
    Header  header_;
    Payload payload_;
};

// ═══════════════════════════════════════════════════════════════════════════════
// 2. ECUDiagnosticSession with PIMPL-like Nested Impl
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Diagnostic session class hiding internal state via a nested Impl struct.
// TR: İç içe Impl yapısı ile iç durumu gizleyen teşhis oturumu sınıfı.
class ECUDiagnosticSession {
public:
    ECUDiagnosticSession();
    ~ECUDiagnosticSession();

    // EN: Public interface — delegates to Impl
    // TR: Public arayüz — Impl'e delege eder
    void startSession(uint16_t session_id);
    bool authenticate(const std::string& key);
    void storeDtc(const std::string& dtc_code);
    std::vector<std::string> readDtcs() const;
    void printStatus() const;

private:
    // EN: Forward-declared nested struct holds all private state
    // TR: İleri bildirimli iç yapı, tüm özel durumu tutar
    struct Impl;
    std::unique_ptr<Impl> pimpl_;
};

// ─── 2a. Impl definition ──────────────────────────────────────────────────────
struct ECUDiagnosticSession::Impl {
    uint16_t session_id       = 0;
    bool     authenticated    = false;
    std::vector<std::string> dtc_buffer;

    static constexpr const char* SECRET_KEY = "ECU_AUTH_2026";
};

ECUDiagnosticSession::ECUDiagnosticSession()
    : pimpl_(std::make_unique<Impl>()) {}

ECUDiagnosticSession::~ECUDiagnosticSession() = default;

void ECUDiagnosticSession::startSession(uint16_t session_id) {
    pimpl_->session_id    = session_id;
    pimpl_->authenticated = false;
    pimpl_->dtc_buffer.clear();
    std::cout << "[ECU] Session 0x" << std::hex << session_id
              << std::dec << " started.\n";
}

bool ECUDiagnosticSession::authenticate(const std::string& key) {
    pimpl_->authenticated = (key == Impl::SECRET_KEY);
    std::cout << "[ECU] Authentication "
              << (pimpl_->authenticated ? "SUCCESS" : "FAILED") << "\n";
    return pimpl_->authenticated;
}

void ECUDiagnosticSession::storeDtc(const std::string& dtc_code) {
    if (!pimpl_->authenticated) {
        std::cout << "[ECU] ERROR: Not authenticated. Cannot store DTC.\n";
        return;
    }
    pimpl_->dtc_buffer.push_back(dtc_code);
}

std::vector<std::string> ECUDiagnosticSession::readDtcs() const {
    if (!pimpl_->authenticated) {
        std::cout << "[ECU] ERROR: Not authenticated. Cannot read DTCs.\n";
        return {};
    }
    return pimpl_->dtc_buffer;
}
```

```cpp

void ECUDiagnosticSession::printStatus() const {
    std::cout << "[ECU] Session=0x" << std::hex << pimpl_->session_id
              << std::dec
              << " Auth=" << (pimpl_->authenticated ? "YES" : "NO")
              << " DTCs=" << pimpl_->dtc_buffer.size() << "\n";
}

// ==========================================================================
// 3. Local Class inside a Function
// ==========================================================================

// EN: Demonstrates a local class used for single-purpose CAN traffic validation.
// TR: Tek amaçlı CAN trafik doğrulaması için kullanılan yerel sınıfı gösterir.
void validateCanTraffic(const std::vector<CanMessage>& traffic) {
    // EN: Local struct — only visible within this function
    // TR: Yerel yapı — yalnızca bu fonksiyon içinde görünür
    struct FrameValidator {
        static bool isValid(const CanMessage& msg) {
            // EN: Standard CAN ID must be <= 0x7FF, DLC must be 1-8
            // TR: Standart CAN ID <= 0x7FF olmalı, DLC 1-8 arasında olmalı
            if (msg.header().isExtended()) return true;  // skip extended check
            return msg.header().getArbId() <= 0x7FF &&
                   msg.header().getDlc() >= 1 &&
                   msg.header().getDlc() <= 8;
        }
    };

    auto valid_count = std::count_if(traffic.begin(), traffic.end(),
                                     FrameValidator::isValid);
    std::cout << "[Validator] " << valid_count << " / " << traffic.size()
              << " frames are valid.\n";

    for (size_t i = 0; i < traffic.size(); ++i) {
        std::cout << "  Frame " << i << ": "
                  << (FrameValidator::isValid(traffic[i]) ? "OK" : "INVALID")
                  << "\n";
    }
}

// ==========================================================================
// 4. Nested Class Accessing Outer's Private Members
// ==========================================================================

// EN: Demonstrates that a nested class can access private members of the outer class.
// TR: İç sınıfın, dış sınıfın private üyelerine erişebildiğini gösterir.
class SensorHub {
public:
    // EN: Nested Inspector can read SensorHub's private calibration_offset_
    // TR: İç Inspector, SensorHub'ın private calibration_offset_ alanını okuyabilir
    class Inspector {
    public:
        static void diagnose(const SensorHub& hub) {
            std::cout << "[Inspector] Hub '" << hub.name_
                      << "' calibration offset = " << hub.calibration_offset_
                      << ", sensor count = " << hub.readings_.size() << "\n";
        }
    };

    explicit SensorHub(std::string name, double offset = 0.0)
        : name_(std::move(name)), calibration_offset_(offset) {}

    void addReading(double value) { readings_.push_back(value + calibration_offset_); }

    void printReadings() const {
        std::cout << "[SensorHub:" << name_ << "] Readings: ";
        for (double r : readings_) std::cout << r << " ";
        std::cout << "\n";
    }

private:
    std::string name_;
    double calibration_offset_;
    std::vector<double> readings_;
};
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════════
// main() — Demos
// ═══════════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "=== Demo 1: CanMessage with Nested Header & Payload ===\n";
    // EN: Construct a CAN message, set header fields, fill payload, print
    // TR: Bir CAN mesajı oluştur, başlık alanlarını ayarla, veriyi doldur, yazdır
    CanMessage msg(0x1A3, 4);
    msg.payload().setByte(0, 0xDE);
    msg.payload().setByte(1, 0xAD);
    msg.payload().setByte(2, 0xBE);
    msg.payload().setByte(3, 0xEF);
    msg.print();

    // EN: Access nested type from outside via scope resolution
    // TR: İç türe dışarıdan kapsam çözücü ile erişim
    CanMessage::Header standalone_hdr(0x7DF, 8, false, false);
    std::cout << "Standalone Header ID: 0x" << std::hex
              << standalone_hdr.getArbId() << std::dec << "\n\n";

    std::cout << "=== Demo 2: ECUDiagnosticSession (PIMPL) ===\n";
    // EN: Create session, authenticate, store and read DTCs
    // TR: Oturum oluştur, kimlik doğrula, DTC kaydet ve oku
    ECUDiagnosticSession diag;
    diag.startSession(0x01);
    diag.printStatus();

    diag.storeDtc("P0301");   // EN: Should fail — not authenticated
    diag.authenticate("WRONG_KEY");
    diag.authenticate("ECU_AUTH_2026");
    diag.printStatus();

    diag.storeDtc("P0301");
    diag.storeDtc("P0420");
    diag.storeDtc("U0100");

    auto dtcs = diag.readDtcs();
    std::cout << "[ECU] Stored DTCs: ";
    for (const auto& d : dtcs) std::cout << d << " ";
    std::cout << "\n\n";

    std::cout << "=== Demo 3: Local Class — validateCanTraffic() ===\n";
    // EN: Build a small traffic sample with valid and invalid frames
    // TR: Geçerli ve geçersiz çerçevelerden oluşan küçük bir trafik örneği oluştur
    std::vector<CanMessage> traffic;
    traffic.emplace_back(0x100, 8);   // valid
    traffic.emplace_back(0x7FF, 1);   // valid (max standard ID)
    traffic.emplace_back(0x800, 4);   // invalid (ID > 0x7FF for standard)
    traffic.emplace_back(0x050, 0);   // invalid (DLC = 0)
    validateCanTraffic(traffic);
    std::cout << "\n";

    std::cout << "=== Demo 4: Nested Class Accessing Outer's Private ===\n";
    // EN: Inspector accesses SensorHub's private name_ and calibration_offset_
    // TR: Inspector, SensorHub'ın private name_ ve calibration_offset_ alanlarına erişir
    SensorHub hub("WheelSpeed_FL", 0.05);
    hub.addReading(1200.0);
    hub.addReading(1205.5);
    hub.addReading(1198.3);
    hub.printReadings();
    SensorHub::Inspector::diagnose(hub);

    return 0;
}
```

► 08. Rtti Typeid

module_02_oop/08_rtti_typeid.cpp

Konu: OOP Part 8: RTTI & typeid — OOP Kısım 8: RTTI ve typeid

Teori / Theory

THEORY: What is RTTI?

RTTI (Run-Time Type Information) is a C++ mechanism that allows the program to determine the actual (dynamic) type of a polymorphic object at runtime. When you have a base-class pointer or reference, RTTI lets you ask: "What is the REAL type hiding behind this pointer?" Two primary tools enable this: the `typeid` operator and `dynamic_cast`.

TEORİ: RTTI Nedir?

RTTI (Çalışma Zamanı Tür Bilgisi), C++'ın polimorfik bir nesnenin gerçek (dinamik) türünü çalışma zamanında belirlemeye yarayan mekanizmasıdır. Elinizde bir temel sınıf işaretçisi veya referansı varsa, RTTI size "Bu işaretçinin arkasında gizlenen GERÇEK tür nedir?" sorusunu sorma imkânı tanır. Bunu sağlayan iki temel araç vardır: `typeid` operatörü ve `dynamic_cast`.

THEORY: typeid Operator

`typeid(expr)` returns a `const std::type_info&`. For polymorphic types (classes with at least one virtual function), it evaluates the runtime type. For non-polymorphic types, it returns the static (compile-time) type. You can compare two type_info objects with `==` and `!=`, and call `.name()` to get an implementation-defined mangled name, or `.hash_code()` for fast comparisons in hash tables.

TEORİ: typeid Operatörü

`typeid(ifade)` bir `const std::type_info&` döndürür. Polimorfik türlerde (en az bir virtual fonksiyona sahip sınıflarda) çalışma zamanı türünü verir. Polimorfik olmayan türlerde ise derleme zamanı (statik) türünü döndürür. İki type_info nesnesini `==` ve `!=` ile karşılaştırabilir, `.name()` ile platforma göre değişen bir isim, `.hash_code()` ile hızlı hash değeri alabilirsiniz.

THEORY: dynamic_cast & Performance

`dynamic_cast<Derived*>(basePtr)` is the safe downcast mechanism. It returns nullptr on failure (for pointers) or throws `std::bad_cast` (for references). RTTI adds a small cost: each polymorphic class carries extra type metadata in its vtable. Compilers offer `-fno-rtti` to disable it entirely (saves binary size, but typeid and dynamic_cast become unavailable).

TEORİ: dynamic_cast ve Performans

`dynamic_cast<Türetilmiş*>(temelPtr)` güvenli alt-türe dönüşüm (downcast) mekanizmasıdır. Başarısız olursa pointer'larda nullptr, referanslarda `std::bad_cast` istisnası döndürür. RTTI küçük bir maliyet ekler: her polimorfik sınıf, vtable'ında ekstra tür bilgisi taşır. Derleyiciler `-fno-rtti` bayrağıyla tamamen devre dışı bırakmaya izin verir (binary boyutu küçülür ama typeid ve dynamic_cast kullanılamaz hale gelir).

THEORY: When to Use RTTI

Prefer virtual dispatch or std::variant/std::visit over RTTI when possible. RTTI is appropriate for plugin systems, serialization, debug logging, or when the set of derived types is open-ended. Overusing typeid checks is often a code smell — it bypasses polymorphism entirely.

TEORİ: RTTI Ne Zaman Kullanılmalı

Mümkün olduğunda RTTI yerine sanal fonksiyon dağıtımını (virtual dispatch) veya std::variant/std::visit tercih edin. RTTI; eklenti sistemleri, serileştirme, hata ayıklama günlüğü veya türetilmiş tür kümesinin açık uçlu olduğu durumlarda uygundur. typeid kontrollerini aşırı kullanmak genellikle bir tasarım hatasıdır — polimorfizmi tamamen devre dışı bırakır.

CPPREF Depth: RTTI Cost and Alternatives — When dynamic_cast Hurts Performance

RTTI requires a vtable — it ONLY works on polymorphic types (classes with at least one virtual function). Applying `typeid` to a non- polymorphic type yields the static (compile-time) type, not the runtime type — a subtle but critical distinction. `dynamic_cast` traverses the inheritance graph at runtime. The cost is O(depth) in the inheritance tree — in deep hierarchies with virtual bases, this involves following vbase offsets through the vtable. In tight loops processing millions of objects, this becomes measurable overhead. The `-fno-rtti` compiler flag (GCC/Clang) disables RTTI entirely, removing type_info structures from every polymorphic class. This typically saves ~5% binary size. However, `typeid` and `dynamic_cast` become compile errors, and libraries compiled WITH rtti cannot be mixed safely with code compiled WITHOUT it (ODR violations in type_info). Alternatives to RTTI-based dispatch:
1. Enum-based type tags — manual but fast O(1) switch dispatch.
2. Visitor pattern (double dispatch) — type-safe, no RTTI needed.
3. std::variant + std::visit — compile-time checked, zero overhead.
4. CRTP (Curiously Recurring Template Pattern) — static polymorphism. In automotive (MISRA C++), real-time, and embedded domains, RTTI is often BANNED due to unpredictable runtime cost and binary bloat.

CPPREF Derinlik: RTTI Maliyeti ve Alternatifleri — dynamic_cast Ne Zaman Performansı Düşürür

RTTI, vtable gerektirir — YALNIZCA polimorfik türlerde (en az bir virtual fonksiyona sahip sınıflarda) çalışır. Polimorfik olmayan bir türe `typeid` uygulamak statik (derleme zamanı) türünü verir — ince ama kritik bir fark. `dynamic_cast` çalışma zamanında kalıtım grafiğini dolaşır. Maliyet, kalıtım ağacında O(derinlik) kadardır — sanal tabanlı derin hiyerarşilerde vtable üzerinden vbase ofsetlerini takip etmeyi içerir. Milyonlarca nesneyi işleyen sıkı döngülerde bu ölçülebilir bir ek yük olur. `-fno-rtti` derleyici bayrağı (GCC/Clang) RTTI'yi tamamen devre dışı bırakır ve her polimorfik sınıftan type_info yapılarını kaldırır. Bu tipik olarak ~%5 binary boyutu tasarrufu sağlar. Ancak `typeid` ve `dynamic_cast` derleme hatasına dönüşür. RTTI tabanlı dağıtıma alternatifler:
1. Enum tabanlı tür etiketleri — manuel ama hızlı O(1) switch.
2. Visitor kalıbı (çift dağıtım) — tür güvenli, RTTI gerekmez.
3. std::variant + std::visit — derleme zamanında kontrollü, sıfır ek yük.
4. CRTP — statik polimorfizm. Otomotivde (MISRA C++), gerçek zamanlı ve gömülü alanlarda RTTI, öngörülemeyen çalışma zamanı maliyeti ve binary şişmesi nedeniyle sıklıkla YASAKLANIR.

→ Ref: https://en.cppreference.com/w/cpp/language/typeid

Kaynak Kod / Source Code

```cpp
#include <iostream>   // EN: Standard I/O for console output
#include <string>     // EN: std::string for type descriptions
#include <typeinfo>   // EN: typeid, std::type_info, std::bad_cast
#include <memory>     // EN: std::unique_ptr for polymorphic ownership
#include <vector>     // EN: std::vector for sensor collection

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 1: Polymorphic Sensor Hierarchy (Automotive Domain)
// ═══════════════════════════════════════════════════════════════════════════

// EN: Base class with virtual functions — makes the hierarchy polymorphic.
// TR: Sanal fonksiyonlara sahip temel sınıf — hiyerarşiyi polimorfik yapar.
class Sensor {
public:
    virtual ~Sensor() = default;

    virtual std::string getType() const { return "GenericSensor"; }

    virtual double read() const = 0;
};

// ─── 1.1 TemperatureSensor ─────────────────────────────────────────────────
class TemperatureSensor : public Sensor {
    double celsius_;
public:
    explicit TemperatureSensor(double c) : celsius_(c) {}

    std::string getType() const override { return "TemperatureSensor"; }
    double read() const override { return celsius_; }

    // EN: Derived-specific method — only accessible after downcast.
    // TR: Türetilmiş sınıfa özgü metot — yalnızca downcast sonrası erişilir.
    double toFahrenheit() const { return celsius_ * 9.0 / 5.0 + 32.0; }
};

// ─── 1.2 PressureSensor ────────────────────────────────────────────────────
class PressureSensor : public Sensor {
    double kpa_;
public:
    explicit PressureSensor(double kpa) : kpa_(kpa) {}

    std::string getType() const override { return "PressureSensor"; }
    double read() const override { return kpa_; }

    // EN: Convert kilopascals to PSI (pounds per square inch).
    // TR: Kilopascal değerini PSI birimine dönüştürür.
    double toPSI() const { return kpa_ * 0.145038; }
};

// ─── 1.3 OxygenSensor ──────────────────────────────────────────────────────
class OxygenSensor : public Sensor {
    double voltage_;
public:
    explicit OxygenSensor(double v) : voltage_(v) {}

    std::string getType() const override { return "OxygenSensor"; }
    double read() const override { return voltage_; }

    // EN: Check if the air-fuel mixture is running lean (voltage < 0.45V).
    // TR: Hava-yakıt karışımının fakir (lean) olup olmadığını kontrol eder.
    bool isLean() const { return voltage_ < 0.45; }
};

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 2: Non-Polymorphic Type (for comparison)
// ═══════════════════════════════════════════════════════════════════════════

// EN: A simple struct without virtual functions — typeid returns static type.
// TR: Virtual fonksiyonu olmayan basit bir yapı — typeid statik tür döndürür.
struct DTCCode {
    int code;
    std::string description;
};
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 3: Production Functions Using RTTI
// ═══════════════════════════════════════════════════════════════════════════

// ─── 3.1 identifySensor ──────────────────────────────────────────────────
// EN: Uses typeid to print the actual runtime type of a sensor.
// TR: Sensörün gerçek çalışma zamanı türünü yazdırmak için typeid kullanır.
void identifySensor(const Sensor& s) {
    std::cout << "  typeid name   : " << typeid(s).name() << "\n";
    std::cout << "  hash_code     : " << typeid(s).hash_code() << "\n";
    std::cout << "  getType()     : " << s.getType() << "\n";
}

// ─── 3.2 processSpecificSensor ───────────────────────────────────────────
// EN: Uses dynamic_cast to safely downcast and call derived-specific methods.
// TR: Güvenli downcast yapıp türetilmiş sınıfa özgü metotları çağırır.
void processSpecificSensor(Sensor* s) {
    if (auto* temp = dynamic_cast<TemperatureSensor*>(s)) {
        std::cout << "  [TemperatureSensor] " << temp->read() << " °C = "
                  << temp->toFahrenheit() << " °F\n";
    } else if (auto* pres = dynamic_cast<PressureSensor*>(s)) {
        std::cout << "  [PressureSensor] " << pres->read() << " kPa = "
                  << pres->toPSI() << " PSI\n";
    } else if (auto* oxy = dynamic_cast<OxygenSensor*>(s)) {
        std::cout << "  [OxygenSensor] " << oxy->read() << " V -> "
                  << (oxy->isLean() ? "Lean" : "Rich") << " mixture\n";
    } else {
        std::cout << "  [Unknown sensor type]\n";
    }
}

// ─── 3.3 demonstrateTypeComparison ───────────────────────────────────────
// EN: Shows typeid equality comparison between objects.
// TR: Nesneler arasında typeid eşitlik karşılaştırmasını gösterir.
void demonstrateTypeComparison(const Sensor& a, const Sensor& b) {
    if (typeid(a) == typeid(b)) {
        std::cout << "  Result: SAME runtime type ("
                  << typeid(a).name() << ")\n";
    } else {
        std::cout << "  Result: DIFFERENT runtime types ("
                  << typeid(a).name() << " vs " << typeid(b).name() << ")\n";
    }
}

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 4: main() — Demonstrations
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "=== Module 02 OOP | 08: RTTI & typeid ===\n\n";

    // ─── Demo 1: typeid on different sensor types through base pointer ──
    std::cout << "--- Demo 1: typeid through base pointer ---\n";
    std::vector<std::unique_ptr<Sensor>> sensors;
    sensors.push_back(std::make_unique<TemperatureSensor>(92.5));
    sensors.push_back(std::make_unique<PressureSensor>(101.3));
    sensors.push_back(std::make_unique<OxygenSensor>(0.72));

    for (const auto& s : sensors) {
        std::cout << "Sensor [" << s->getType() << "]:\n";
        identifySensor(*s);
        std::cout << "\n";
    }
```

```

    // ─── Demo 2: typeid equality comparison ──────────────────────────────────
    std::cout << "--- Demo 2: typeid equality comparison ---\n";
    TemperatureSensor coolantTemp(88.0);
    TemperatureSensor intakeTemp(35.0);
    PressureSensor boostPressure(150.0);

    std::cout << "Comparing coolantTemp vs intakeTemp:\n";
    demonstrateTypeComparison(coolantTemp, intakeTemp);

    std::cout << "Comparing coolantTemp vs boostPressure:\n";
    demonstrateTypeComparison(coolantTemp, boostPressure);
    std::cout << "\n";

    // ─── Demo 3: dynamic_cast safe downcast ──────────────────────────────────
    std::cout << "--- Demo 3: dynamic_cast safe downcast ---\n";
    Sensor* engineTemp = &coolantTemp;
    processSpecificSensor(engineTemp);

    Sensor* manifold = &boostPressure;
    processSpecificSensor(manifold);

    OxygenSensor lambdaProbe(0.38);
    Sensor* exhaust = &lambdaProbe;
    processSpecificSensor(exhaust);
    std::cout << "\n";

    // ─── Demo 4: dynamic_cast failure (returns nullptr) ──────────────────────
    std::cout << "--- Demo 4: dynamic_cast failure ---\n";
    Sensor* sensorPtr = &boostPressure;
    auto* asTemp = dynamic_cast<TemperatureSensor*>(sensorPtr);
    if (asTemp == nullptr) {
        std::cout << "  dynamic_cast<TemperatureSensor*> on PressureSensor "
                  << "returned nullptr (safe failure)\n";
    }

    // EN: Reference cast throws std::bad_cast on failure.
    // TR: Referans dönüşümü başarısızlıkta std::bad_cast fırlatır.
    try {
        [[maybe_unused]] auto& asOxy =
            dynamic_cast<OxygenSensor&>(*sensorPtr);
    } catch (const std::bad_cast& e) {
        std::cout << "  dynamic_cast<OxygenSensor&> threw: "
                  << e.what() << "\n";
    }
    std::cout << "\n";

    // ─── Demo 5: Polymorphic vs non-polymorphic typeid ───────────────────────
    std::cout << "--- Demo 5: Polymorphic vs non-polymorphic typeid ---\n";

    // EN: Polymorphic — typeid resolves actual runtime type.
    // TR: Polimorfik — typeid gerçek çalışma zamanı türünü çözer.
    Sensor* basePtr = &coolantTemp;
    std::cout << "  Declared type : Sensor*\n";
    std::cout << "  typeid(*basePtr): " << typeid(*basePtr).name()
              << "  (runtime type)\n";

    // EN: Non-polymorphic — typeid always returns static (declared) type.
    // TR: Polimorfik olmayan — typeid her zaman statik (bildirilen) türü döner.
    DTCCode dtc{420, "Catalyst Temp Below Threshold"};
    const DTCCode& dtcRef = dtc;
    std::cout << "  Declared type : const DTCCode&\n";
    std::cout << "  typeid(dtcRef)  : " << typeid(dtcRef).name()
              << "  (static type — no virtual)\n";
    std::cout << "\n";

```

```
203     // ─── Demo 6: type_info::name() and type_info::hash_code() ──────────────────────────────
204     std::cout << "--- Demo 6: type_info name() & hash_code() ---\n";
205     const std::type_info& ti1 = typeid(TemperatureSensor);
206     const std::type_info& ti2 = typeid(PressureSensor);
207     const std::type_info& ti3 = typeid(TemperatureSensor);
208
209     std::cout << "  TemperatureSensor name : " << ti1.name() << "\n";
210     std::cout << "  PressureSensor name    : " << ti2.name() << "\n";
211     std::cout << "  TemperatureSensor hash : " << ti1.hash_code() << "\n";
212     std::cout << "  PressureSensor hash    : " << ti2.hash_code() << "\n";
213     std::cout << "  ti1 == ti3 (same type) : "
214               << (ti1 == ti3 ? "true" : "false") << "\n";
215     std::cout << "  ti1 == ti2 (diff type) : "
216               << (ti1 == ti2 ? "true" : "false") << "\n";
217
218     std::cout << "\n=== RTTI & typeid demonstration complete ===\n";
219     return 0;
220 }
```

Modül 3: Modern C++ — Güvenlik ve Performans

Güvenlik ve Performans

6 kaynak dosya · Standart: C++17/C++23

C++17/C++23

► 01. Smart Pointers

module_03_modern_cpp/01_smart_pointers.cpp

Konu: Modern C++: Smart Pointers and RAII — Akıllı İşaretçiler ve RAII (Resource Acquisition

Teori / Theory

THEORY: Why Do We Need Smart Pointers?

In old C++ (before C++11), developers managed memory dynamically using `new` and `delete`. This was a nightmare. If a function threw an exception or returned early, `delete` was skipped -> Memory Leak! If we deleted a pointer twice -> Double Free Crash!

TEORİ: Neden Akıllı İşaretçilere İhtiyacımız Var?

Eski C++'da (C++11 öncesi), geliştiriciler belleği `new` ve `delete` ile tamamen manuel yönetirdi. Bu büyük bir kabustu. Bir fonksiyon hata fırlatır (exception) veya erken `return` ederse, `delete` komutu atlanırdı -> Bellek Sızıntısı (Memory Leak). Aynı pointer'ı yanlışlıkla iki defa silersek -> Çift Silme Çöküşü (Double Free Crash).

THEORY: What is RAII?

The backbone principle of C++. "When a resource (memory, file, socket) is Acquired, bind it to a local/stack Object. When that Object goes out of scope ({...}), its Destructor automatically runs and safely frees the resource." Smart Pointers are pure RAII. They silently call `delete` for you.
1. std::unique_ptr: UNIQUE Ownership. Cannot be copied (only moved). "I am the sole owner of this memory. If I die, it dies."
2. std::shared_ptr: SHARED Ownership. Counts how many people hold the memory (Reference Counting). When the count hits 0, the memory vanishes.
3. std::weak_ptr: A "watcher" for shared_ptr that doesn't increment the count, preventing circular reference loops!

TEORİ: RAII Nedir?

C++'ın belkemiği prensibi. "Bir kaynak (bellek, dosya, ağ soketi) alındığı anda (Acquisition) onu yerel/stack'te yaşayan bir nesneye bağla. O nesne yaşam alanından ({...} scope) çıktığında otomatik olarak Yıkıcısı (Destructor) çalışsın ve kaynağı güvenle serbest bıraksın." Akıllı işaretçiler tam olarak RAII'dir. Arka planda senin için sessizce `delete` çağırırlar.
1. std::unique_ptr: Sahipliği TEKTİR. Kopyalanamaz, sadece taşınabilir. "Bu belleğin tek sahibi benim. Ben ölürsem, o da ölür."
2. std::shared_ptr: Sahipliği PAYLAŞILIR. Belleği kaç kişinin tuttuğunu sayar (Referans Sayacı). Sayaç 0 olunca belleği uçurur.
3. std::weak_ptr: shared_ptr sayacını artırmadan gözcülük yapan, dairesel referans kilitlenmelerini çözen türdür.

CPPREF Depth: `std::make_shared` vs `new` & The Control Block

CppReference dictates: Always use `std::make_shared` instead of `std::shared_ptr<T>(new T())`. Why? Because `shared_ptr` needs TWO things:
1. The actual object.
2. A "Control Block" (holds the counter). Using `new` allocates these separately (2 RAM jumps). `make_shared` allocates them in ONE single contiguous memory block (Faster Cache Hits, No memory leaps).

CPPREF Derinlik: `std::make_shared` vs `new` ve Kontrol Bloğu

CppReference emreder: Daima `std::make_shared` kullanın. Çünkü `shared_ptr` RAM'de 2 şeye ihtiyaç duyar:
1. Asıl Nesne.
2. Sayaç Bloğu (Control Block). Eğer `new` derseniz işletim sistemi iki farklı yere gider (Yavaş). `make_shared` ise Nesne'yi ve Sayacı BİTİŞİK TEK RAM BLOĞUNA yazar (CPU Önbellek, Cache Hit Hızı %100).

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdio>
#include <string>
// EN: Required for smart pointers
// TR: Akıllı işaretçiler için zorunlu
#include <memory>

/**
 * @class HeavyResource
 * @brief EN: Represents a memory-heavy object like a 4K texture or a gigabyte DB. TR: 4K doku
 * veya megabaytlık bir veritabanı gibi ağır nesneleri temsil eder.
 */
class HeavyResource {
public:
  HeavyResource() {
    std::cout << "[HeavyResource] Acquired from OS. (Kaynak İşletim "
                 "Sisteminden Alındı)"
              << std::endl;
  }
  ~HeavyResource() {
    std::cout << "[HeavyResource] Destroyed safely! (Kaynak Güvenle Yok "
                 "Edildi!) -> NO LEAK"
              << std::endl;
  }
  void render() {
    std::cout << "Rendering 4K Textures... / 4K Dokular İşleniyor..." << std::endl;
  }
};
```

```cpp

int main() {
  std::cout << "=== MODERN C++: SMART POINTERS & RAII ===\n" << std::endl;

  std::cout << "--- 1. Raw Pointer (Legacy System - Bad Practice / Eski Sistem "
               "- Kötü Pratik) ---"
            << std::endl;
  HeavyResource *rawPointer = new HeavyResource();

  // EN: If an exception happens here, 'delete' is skipped -> Memory Leak!
  // TR: Eğer burada hata fırlayıp delete adımını atlarsak RAM dolar taşar!
  delete rawPointer;

  std::cout << "\n--- 2. std::unique_ptr (C++14 Way - Best Practice / C++14 "
               "Yolu - En İyi Pratik) ---"
            << std::endl;
  {
    // EN: Scope begins! `std::make_unique()` is faster and SAFER than calling `new`.
    // TR: Scope Başlıyor! `std::make_unique()` kullanmak, doğrudan `new` kullanmaktan çok daha
    // GÜVENLİDİR.
    std::unique_ptr<HeavyResource> uniquePtr = std::make_unique<HeavyResource>();
    uniquePtr->render();

    // auto uniquePtr2 = uniquePtr; // ERROR! Compiler forbids it. Ownership is unique. / HATA!
    // KOPYALANAMAZ.

    // EN: Allowed! Ownership transferred via Move Semantics.
    // TR: İzin verilir. Sahiplik devredildi (Taşıma Semantiği).
    std::unique_ptr<HeavyResource> movedPtr = std::move(uniquePtr);

  } // EN: Scope ends. movedPtr dies. Destructor AUTO-RUNS!
    // TR: Süslü parantez bitti. SCOPE DIŞINA çıktık. movedPtr YOK EDİLDİ. Yıkıcı OTO-ÇALIŞTI!

  std::cout << "\n--- 3. std::shared_ptr (Shared Ownership / Paylaşılan Sahiplik) ---" << std::endl;
  {
    // Count = 1 (Sayaç 1)
    std::shared_ptr<HeavyResource> shared1 = std::make_shared<HeavyResource>();
    std::cout << "Owners (Sahip Sayısı): " << shared1.use_count() << std::endl;

    {
      // EN: Copied! Count = 2
      // TR: Kopya ALINABİLİR! Sayaç 2
      std::shared_ptr<HeavyResource> shared2 = shared1;
      std::cout << "Owners in lower scope (Alt Scope Sahip Sayısı): " << shared1.use_count() <<
          std::endl;
    } // EN: shared2 dies. Count drops to 1. Object still lives.
      // TR: shared2 öldü. Sayaç tekrar 1'e düştü (Nesne halen yaşıyor).

    std::cout << "Owners after lower scope (Alt Scope Bittiğinde Sahip Sayısı): " <<
        shared1.use_count() << std::endl;
  } // EN: shared1 dies. Count becomes 0. OBJECT DESTROYED!
    // TR: shared1 öldü. Sayaç 0 oldu. NESNE TAMAMEN YOK EDİLDİ!

  // ==========================================================================
  // PART 4: Custom Deleter / Özel Silici
  // ==========================================================================
  std::cout << "\n--- 4. Custom Deleter (unique_ptr with FILE* / FILE* ile Özel Silici) ---"
            << std::endl;
  {
    // EN: unique_ptr can manage non-new resources with a custom deleter.
    //     Classic example: C FILE* must be closed with fclose(), not delete.
    // TR: unique_ptr, özel silici ile new olmayan kaynakları yönetebilir.
    //     Klasik örnek: C FILE* fclose() ile kapatılmalı, delete ile değil.
    auto fileDeleter = [](FILE* fp) {
      if (fp) {
        std::fclose(fp);
        std::cout << "  Custom deleter: FILE* closed / Özel silici: FILE* kapatıldı" << std::endl;
      }
    };
```

```cpp

    {
      std::unique_ptr<FILE, decltype(fileDeleter)> filePtr(
          std::fopen("/tmp/smart_ptr_test.txt", "w"), fileDeleter);
      if (filePtr) {
        std::fputs("Hello from unique_ptr with custom deleter!\n", filePtr.get());
        std::cout << "  Written to /tmp/smart_ptr_test.txt" << std::endl;
      }
    } // EN: fileDeleter auto-called here — fclose(), not delete!
      // TR: fileDeleter burada otomatik çağrılır — fclose(), delete değil!

    // EN: Another pattern: array deleter (C++11 style, C++14 has make_unique<T[]>)
    // TR: Diğer desen: dizi silici (C++11 stili, C++14'te make_unique<T[]> var)
    auto arrayDeleter = [](int* p) {
      std::cout << "  Custom deleter: array deleted / Özel silici: dizi silindi" << std::endl;
      delete[] p;
    };
    {
      std::unique_ptr<int, decltype(arrayDeleter)> arr(new int[5]{10, 20, 30, 40, 50},
                                                       arrayDeleter);
      std::cout << "  Array[2] = " << arr.get()[2] << std::endl;
    } // EN: arrayDeleter auto-called — delete[], not delete!
  }

  // ═══════════════════════════════════════════════════════════════════════
  // PART 5: enable_shared_from_this
  // ═══════════════════════════════════════════════════════════════════════
  std::cout << "\n--- 5. enable_shared_from_this ---" << std::endl;
  {
    // EN: Problem: Inside a method, you need a shared_ptr to `this`.
    //     BAD:  shared_ptr<Foo>(this)  — creates SECOND control block → double free!
    //     GOOD: inherit enable_shared_from_this<Foo>, call shared_from_this().
    // TR: Sorun: Bir metod içinde `this`'e shared_ptr gerekiyor.
    //     KÖTÜ:  shared_ptr<Foo>(this) — İKİNCİ kontrol bloğu oluşturur → çift silme!
    //     İYİ:  enable_shared_from_this<Foo>'dan miras al, shared_from_this() çağır.

    struct EventEmitter : public std::enable_shared_from_this<EventEmitter> {
      std::string name;

      explicit EventEmitter(const std::string& n) : name(n) {}

      std::shared_ptr<EventEmitter> getShared() {
        return shared_from_this();
      }

      void describe() const {
        std::cout << "  EventEmitter: " << name << std::endl;
      }
    };

    auto emitter = std::make_shared<EventEmitter>("SensorHub");
    std::cout << "  use_count before: " << emitter.use_count() << std::endl;

    {
      // EN: Safe! Same control block, count incremented.
      // TR: Güvenli! Aynı kontrol bloğu, sayaç artırıldı.
      auto alias = emitter->getShared();
      std::cout << "  use_count with alias: " << emitter.use_count() << std::endl;
      alias->describe();
    }

    std::cout << "  use_count after alias dies: " << emitter.use_count() << std::endl;
  }
```

```cpp

    // ═══════════════════════════════════════════════════════════════════════
    // PART 6: Aliasing Constructor / Takma Ad Yapıcı
    // ═══════════════════════════════════════════════════════════════════════
    std::cout << "\n--- 6. Aliasing Constructor (shared_ptr to member / Üye'ye shared_ptr) ---"
              << std::endl;
    {
      // EN: The aliasing constructor creates a shared_ptr that shares ownership
      //     with another shared_ptr, but points to a DIFFERENT object (e.g., a member).
      //     Signature: shared_ptr<T>(shared_ptr<U> owner, T* ptr)
      //     The owner keeps the parent alive; ptr is what you dereference.
      // TR: Takma ad yapıcı, başka bir shared_ptr ile sahipliği paylaşan ama
      //     FARKLI bir nesneye (örn. üyeye) işaret eden shared_ptr oluşturur.

      struct Engine {
        int horsepower = 450;
      };

      struct Car {
        std::string model = "M3 GTR";
        Engine engine;
      };

      auto car = std::make_shared<Car>();
      std::cout << "  car use_count: " << car.use_count() << std::endl;

      // EN: Aliasing: enginePtr shares ownership with car, but points to car->engine.
      //     As long as enginePtr is alive, the entire Car stays alive.
      // TR: Takma ad: enginePtr sahipliği car ile paylaşır, ama car->engine'e işaret eder.
      std::shared_ptr<Engine> enginePtr(car, &car->engine);
      std::cout << "  car use_count after alias: " << car.use_count() << std::endl;
      std::cout << "  enginePtr->horsepower: " << enginePtr->horsepower << std::endl;

      car.reset(); // EN: We release car, but enginePtr keeps it alive
                   // TR: car'ı bırakıyoruz ama enginePtr hayatta tutuyor
      std::cout << "  After car.reset(), enginePtr->horsepower: "
                << enginePtr->horsepower << std::endl;
      std::cout << "  (Car still alive because enginePtr holds ownership)"
                << std::endl;
    }

    // ═══════════════════════════════════════════════════════════════════════
    // PART 7: weak_ptr — Breaking Circular References
    // ═══════════════════════════════════════════════════════════════════════
    std::cout << "\n--- 7. weak_ptr (Circular Reference Breaker / Dairesel Referans Kırıcı) ---"
              << std::endl;
    {
      // EN: Without weak_ptr, two objects holding shared_ptr to each other
      //     create a circular reference — neither is ever freed (memory leak).
      // TR: weak_ptr olmadan, birbirine shared_ptr tutan iki nesne dairesel
      //     referans oluşturur — ikisi de asla serbest bırakılmaz (bellek sızıntısı).

      struct Node {
        std::string name;
        std::weak_ptr<Node> partner;  // EN: weak, not shared! / TR: shared değil, weak!

        explicit Node(const std::string& n) : name(n) {
          std::cout << "  Node " << name << " created" << std::endl;
        }
        ~Node() {
          std::cout << "  Node " << name << " destroyed" << std::endl;
        }
      };

      {
        auto nodeA = std::make_shared<Node>("A");
        auto nodeB = std::make_shared<Node>("B");

        nodeA->partner = nodeB;  // EN: weak reference, count NOT incremented
        nodeB->partner = nodeA;  // TR: weak referans, sayaç ARTTIRILMAZ

        std::cout << "  A use_count: " << nodeA.use_count()
                  << ", B use_count: " << nodeB.use_count() << std::endl;

```

```
      // EN: Access partner through weak_ptr — must lock() first
      // TR: partner'a weak_ptr üzerinden erişim — önce lock() çağrılmalı
      if (auto locked = nodeA->partner.lock()) {
        std::cout << "  A's partner: " << locked->name << std::endl;
      }

      // EN: Check if partner is still alive
      // TR: Ortağın hâlâ hayatta olup olmadığını kontrol et
      std::cout << "  B's partner expired? " << std::boolalpha
                << nodeA->partner.expired() << std::endl;
    } // EN: Both destroyed! No circular leak because weak_ptr doesn't hold ownership.
      // TR: İkisi de yok edildi! weak_ptr sahiplik tutmadığı için dairesel sızıntı yok.
  }

  return 0;
}
```

► 02. Lambdas

module_03_modern_cpp/02_lambdas.cpp

Konu: Modern C++: Lambda Expressions — From Zero to Mastery — Lambda İfadeleri — Sıfırdan

Teori / Theory

THEORY: What is a Lambda? (Lambdas for Dummies)

Imagine you need to tell the ECU diagnostic system: "Filter only the fault codes above severity 3." Before C++11 you had to write a separate named function somewhere far away and pass a pointer to it. With a Lambda you write the rule RIGHT WHERE you need it — inline, compact, readable.
A Lambda is an ANONYMOUS (unnamed) function object that the compiler generates for you behind the scenes. Think of it as a disposable mini-function you define on the spot. Lambdas are the cornerstone of modern STL algorithms (std::sort, std::for_each, std::find_if, std::transform, ...).

TEORİ: Lambda Nedir? (Sıfırdan Anlatım)

ECU arıza teşhis sistemine: "Sadece önem derecesi 3 üzeri hata kodlarını filtrele" demek istediğinizi düşünün. C++11 öncesi uzaklarda ayrı bir fonksiyon yazıp onun adresini (pointer) göndermeniz gerekirdi. Lambda ile kuralı TAM İHTİYAÇ DUYDUĞUNUZ YERE, satır içi, kompakt ve okunabilir şekilde yazarsınız.
Lambda, derleyicinin arka planda sizin için ürettiği İSİMSİZ (anonim) bir fonksiyon nesnesidir. Onu tek kullanımlık, yerinde tanımlanan mini-fonksiyon olarak düşünün. Lambdalar modern STL algoritmalarının (std::sort, std::for_each, std::find_if, std::transform, ...) temel taşıdır.

THEORY: Lambda Anatomy — The Full Syntax

The complete Lambda syntax is:
[capture_list] (parameters) mutable -> return_type { body }
- [capture_list] : HOW to grab outside variables (see next theory).
- (parameters) : Input arguments, just like a normal function.
- mutable : (Optional) Lets you modify captured-by-value copies.
- -> return_type : (Optional) Explicit return type. Usually auto-deduced.
- { body } : The actual code.
Minimal Lambda: [](){} — captures nothing, takes nothing, does nothing. When there are no parameters you may even omit (): []{}.

TEORİ: Lambda Anatomisi — Tam Sözdizimi

Lambda'nın tam sözdizimi:
[yakalama_listesi] (parametreler) mutable -> dönüş_tipi { gövde }
- [yakalama_listesi] : Dış değişkenleri NASIL yakalayacağınız.
- (parametreler) : Girdi argümanları, normal fonksiyon gibi.
- mutable : (Opsiyonel) Değer ile yakalanan kopyaları değiştirmenize izin verir.
- -> dönüş_tipi : (Opsiyonel) Açık dönüş tipi. Genelde otomatik çıkarılır.
- { gövde } : Gerçek kod.

En Basit Lambda: [](){} — hiçbir şey yakalamaz, almaz, yapmaz. Parametre yoksa () bile atlanabilir: []{}.

THEORY: Capture List Deep Dive

The capture list is what makes Lambdas special. It controls how outside variables enter the Lambda's body:
[] — Capture nothing. Pure function, no outside variables.
[=] — Capture ALL outside variables by VALUE (read-only copies).
[&] — Capture ALL outside variables by REFERENCE (modify originals).
[x] — Capture only 'x' by value.
[&x] — Capture only 'x' by reference.
[=, &x] — Capture everything by value EXCEPT 'x' by reference.
[&, x] — Capture everything by reference EXCEPT 'x' by value.
[this] — Capture the enclosing object's 'this' pointer (C++11).
[*this] — Capture a COPY of the enclosing object (C++17).
[x = expr] — Init-capture: create a new variable 'x' initialized to 'expr' (C++14). Great for move-capturing!
Golden Rule: Capture only what you need. Prefer explicit captures [x, &y] over blanket [=] or [&] for clarity and safety.

TEORİ: Yakalama Listesi Detaylı İnceleme

Yakalama listesi Lambda'ları özel kılan şeydir. Dış değişkenlerin Lambda gövdesine nasıl gireceğini kontrol eder:
[] — Hiçbir şey yakalama. Saf fonksiyon, dış değişken yok.
[=] — Tüm dış değişkenleri DEĞER ile yakala (salt okunur kopya).
[&] — Tüm dış değişkenleri REFERANS ile yakala (orijinali değiştir).
[x] — Sadece 'x' değişkenini değer ile yakala.
[&x] — Sadece 'x' değişkenini referans ile yakala.
[=, &x] — Her şeyi değer ile, SADECE 'x' referans ile yakala.
[&, x] — Her şeyi referans ile, SADECE 'x' değer ile yakala.
[this] — Kapsayan nesnenin 'this' işaretçisini yakala (C++11).
[*this] — Kapsayan nesnenin KOPYASInı yakala (C++17).
[x = ifade] — Başlatıcılı yakalama: 'ifade' ile başlatılan yeni 'x' değişkeni oluştur (C++14). Move-capture için mükemmel!
Altın Kural: Sadece ihtiyacınız olanı yakalayın. Netlik ve güvenlik için toptan [=] veya [&] yerine açık yakalama [x, &y] tercih edin.

CPPREF Depth: Dangling Captures & Lifetime Traps

cppreference.com/w/cpp/language/lambda warns: If a Lambda captures local variables by REFERENCE [&] and then OUTLIVES the scope where those variables were defined (e.g., returned from a function, stored in a container, passed to std::thread), you get a DANGLING REFERENCE — Undefined Behavior! The Lambda holds an address to memory that no longer exists.
Safe rule: If the Lambda escapes the current scope, capture by VALUE [=] or use init-capture [x = std::move(obj)] to own the data.

CPPREF Derinlik: Sarkan Yakalamalar ve Ömür Tuzakları

cppreference.com/w/cpp/language/lambda uyarır: Lambda yerel değişkenleri REFERANS [&] ile yakalayıp, bu değişkenlerin tanımlandığı kapsamın dışına çıkarsa (fonksiyondan dönülürse, container'a saklanırsa, std::thread'e verilirse), SARKAN REFERANS (Dangling Reference) oluşur — Tanımsız Davranış! Lambda artık var olmayan belleğe işaret eder.
Güvenli kural: Lambda mevcut kapsamdan dışarı çıkıyorsa DEĞER [=] ile yakalayın veya init-capture [x = std::move(obj)] ile veriyi sahiplenin.

CPPREF Depth: std::function — Type-Erased Lambda Storage

cppreference.com/w/cpp/utility/functional/function documents: std::function<ReturnType(Args...)> is a general-purpose polymorphic function wrapper. It can store ANY callable: a Lambda, a function pointer, a functor, or std::bind result. Use it when you need to store a Lambda in a member variable, a container, or pass it across API boundaries. Be aware: it has overhead (heap allocation, virtual dispatch). For performance-critical automotive loops, prefer templates or auto.

CPPREF Derinlik: std::function — Tip Silmeli Lambda Saklama

cppreference.com/w/cpp/utility/functional/function belgeliyor: std::function<DönüşTipi(Argümanlar...)> genel amaçlı polimorfik fonksiyon sarmalayıcıdır. HERHANGİ bir çağrılabiliri saklayabilir: Lambda, fonksiyon işaretçisi, functor veya std::bind sonucu. Lambda'yı üye değişkende, container'da saklamanız veya API sınırları arasında geçirmeniz gerektiğinde kullanın. Dikkat: maliyeti vardır (heap tahsisi, sanal dağıtım). Performans-kritik otomotiv döngülerinde template veya auto tercih edin.

THEORY: Generic Lambdas (C++14) & Template Lambdas (C++20)

C++14 introduced generic Lambdas: use 'auto' in the parameter list and the compiler generates a templated operator() internally. This is perfect for writing ECU-agnostic utility functions.
C++20 goes further: you can write explicit template parameter lists on Lambdas: []<typename T>(T value){ ... }. This gives you SFINAE/concepts power directly inside a Lambda — useful for compile-time filtering of sensor types in embedded systems.

TEORİ: Jenerik Lambdalar (C++14) ve Şablon Lambdalar (C++20)

C++14 jenerik Lambda'ları tanıttı: parametre listesinde 'auto' kullanın, derleyici içeride şablonlu operator() üretir. Bu, ECU-bağımsız yardımcı fonksiyonlar yazmak için mükemmeldir.
C++20 daha ileri gider: Lambda'larda açık şablon parametre listesi yazabilirsiniz: []<typename T>(T value){ ... }. Bu size doğrudan Lambda içinde SFINAE/konsept gücü verir — gömülü sistemlerde sensör tiplerini derleme zamanında filtrelemek için kullanışlıdır.

CPPREF Depth: Lambda as Constexpr (C++17) & Consteval (C++20)

cppreference.com/w/cpp/language/lambda documents: Since C++17, Lambdas are implicitly constexpr if their body satisfies constexpr requirements. You can compute values at COMPILE TIME using Lambdas — zero runtime cost. In automotive ECUs with limited flash, this means look-up tables, CRC checksums, and protocol constants can be Lambda-computed at compile time.
C++20 allows `consteval` Lambdas that MUST execute at compile time.

CPPREF Derinlik: constexpr Lambda (C++17) ve consteval (C++20)

cppreference.com/w/cpp/language/lambda belgeliyor: C++17'den beri Lambda'lar gövdesi constexpr gereksinimlerini karşılıyorsa otomatik olarak constexpr'dir. Lambda'lar ile DERLEME ZAMANINDA değer hesaplayabilirsiniz — sıfır çalışma zamanı maliyeti. Sınırlı flash'e sahip otomotiv ECU'larında bu, arama tabloları, CRC sağlama toplamları ve protokol sabitlerinin derleme zamanında Lambda ile hesaplanabileceği anlamına gelir.
C++20, derleme zamanında çalışMASI GEREKEN consteval Lambda'lara izin verir.

Kaynak Kod / Source Code

```cpp
#include <algorithm>  // std::sort, std::for_each, std::find_if, std::transform
#include <cstdint>    // uint8_t, uint16_t
#include <functional> // std::function
#include <iostream>
#include <numeric>    // std::accumulate
#include <string>
#include <vector>

// ─── Automotive Helper Structures ──────────────────────────────────────────────

// EN: Diagnostic Trouble Code (DTC) — core data in any automotive ECU.
// TR: Arıza Teşhis Kodu (DTC) — her otomotiv ECU'sunun temel verisi.
struct DTC {
  uint16_t code;        // e.g. 0x0171
  std::string name;     // e.g. "EGT Sensor Open Circuit"
  uint8_t severity;     // 1 = info, 2 = warning, 3 = critical, 4 = fatal
  bool active;          // currently present?
};

// EN: Sensor reading from a vehicle bus (CAN/LIN).
// TR: Araç veri yolundan (CAN/LIN) gelen sensör okuması.
struct SensorReading {
  std::string sensor;
  double value;
  std::string unit;
};

int main() {
  std::cout << "=== MODULE 3/02: LAMBDA EXPRESSIONS — FULL GUIDE ===\n"
            << std::endl;

  // ═══════════════════════════════════════════════════════════════════════════
  // 1. THE SIMPLEST LAMBDA / EN BASIT LAMBDA
  // ═══════════════════════════════════════════════════════════════════════════
  // EN: A Lambda with no capture, no parameters. Just prints a message.
  // TR: Yakalama yok, parametre yok. Sadece bir mesaj yazdırır.

  auto greet = []() {
    std::cout << "1. ECU Boot: Lambda initialized successfully!" << std::endl;
  };
  // EN: Call the Lambda
  // TR: Lambda'yı çağır
  greet();

  // EN: Even shorter — omit () when there are no parameters:
  // TR: Daha kısa — parametre yoksa () atlanabilir:
  auto greetShort = [] {
    std::cout << "   (Same Lambda without parentheses)\n" << std::endl;
  };
  greetShort();
```

```cpp

    // ═══════════════════════════════════════════════════════════════════════════
    // 2. LAMBDA WITH PARAMETERS / PARAMETRELI LAMBDA
    // ═══════════════════════════════════════════════════════════════════════════
    // EN: Lambdas accept parameters just like regular functions. Here we convert a raw ADC value
    // to a temperature (automotive formula).
    // TR: Lambda'lar normal fonksiyonlar gibi parametre alır. Ham ADC değerini sıcaklığa çeviren
    // otomotiv formülü.

    auto adcToTemperature = [](int rawAdc) -> double {
      // EN: Simplified EGT sensor formula: T = (raw * 0.1) - 40.0
      // TR: Basitleştirilmiş EGT sensör formülü: T = (raw * 0.1) - 40.0
      return (rawAdc * 0.1) - 40.0;
    };

    int rawValue = 850;
    std::cout << "2. ADC Raw=" << rawValue
              << " -> Temperature=" << adcToTemperature(rawValue)
              << " C\n" << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════
    // 3. CAPTURE BY VALUE [=] / DEGER ILE YAKALAMA [=]
    // ═══════════════════════════════════════════════════════════════════════════
    // EN: The Lambda copies the outside variable's VALUE at the moment of creation. Changes to the
    // original do NOT affect the Lambda's copy.
    // TR: Lambda oluşturulma anında dış değişkenin DEĞERİNİ kopyalar. Orijinaldeki değişiklikler
    // Lambda'nın kopyasını ETKİLEMEZ.

    // EN: Critical and above
    // TR: Kritik ve üzeri
    int severityThreshold = 3;
    auto isCritical = [=](const DTC& dtc) {
      return dtc.severity >= severityThreshold && dtc.active;
    };

    // EN: Even if we change threshold AFTER creating the Lambda, the Lambda still sees the OLD
    // value (3), because it captured a COPY.
    // TR: Lambda'yı oluşturduktan SONRA threshold'u değiştirsek bile, Lambda HÂL ESKİ değeri (3)
    // görür, çünkü KOPYAyı yakaladı.
    severityThreshold = 999;

    DTC testDtc{0x0171, "EGT Sensor Open", 4, true};
    std::cout << "3. Capture by value: DTC 0x0171 critical? "
              << (isCritical(testDtc) ? "YES" : "NO")
              << " (threshold was 3 at capture time)\n" << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════
    // 4. CAPTURE BY REFERENCE [&] / REFERANS ILE YAKALAMA [&]
    // ═══════════════════════════════════════════════════════════════════════════
    // EN: The Lambda gets a REFERENCE to the outside variable. Any change the Lambda makes
    // modifies the ORIGINAL. Also, if the outside changes, the Lambda sees the new value.
    // TR: Lambda dış değişkene REFERANS alır. Lambda'nın yaptığı her değişiklik ORİJİNALİ
    // değiştirir. Dışarıda değişiklik olursa Lambda yeni değeri görür.

    int activeFaultCount = 0;
    std::vector<DTC> faultLog = {
        {0x0100, "Coolant Temp High",   3, true},
        {0x0200, "Oil Pressure Low",    4, true},
        {0x0300, "Battery Voltage",     2, false},
        {0x0400, "Throttle Pos Sensor", 3, true},
        {0x0500, "O2 Sensor Circuit",   2, true},
        {0x0600, "Knock Sensor Range",  4, false},
        {0x0700, "Misfire Cylinder 2",  3, true},
    };

    // EN: Count active faults by capturing counter BY REFERENCE.
    // TR: Sayacı REFERANS ile yakalayarak aktif arızaları say.
    std::for_each(faultLog.begin(), faultLog.end(),
                  [&activeFaultCount](const DTC& dtc) {
      if (dtc.active) {
        activeFaultCount++;
      }
    });
    std::cout << "4. Capture by reference: Active faults = "
              << activeFaultCount << "\n" << std::endl;
```

```cpp

    // ═══════════════════════════════════════════════════════════════════════════════
    // 5. MIXED CAPTURE [=, &x] / KARISIK YAKALAMA [=, &x]
    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: Best practice — capture everything by value for safety, but explicitly capture specific
    // variables by reference when you need to modify them.
    // TR: En iyi pratik — güvenlik için her şeyi değer ile yakala, ama değiştirmen gereken belirli
    // değişkenleri açıkça referans ile yakala.

    int criticalCount = 0;
    int warningCount = 0;
    int critThreshold = 3;

    std::for_each(faultLog.begin(), faultLog.end(),
                  [=, &criticalCount, &warningCount](const DTC& dtc) {
      if (!dtc.active) return;
      if (dtc.severity >= critThreshold) {
        // EN: Modified via reference
        // TR: Referansla değiştirilir
        criticalCount++;
      } else {
        warningCount++;
      }
    });
    std::cout << "5. Mixed capture: Critical=" << criticalCount
              << ", Warning=" << warningCount << "\n" << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // 6. MUTABLE LAMBDA / DEGISTIRILEBILIR LAMBDA
    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: By default, a Lambda captured by VALUE cannot modify its copies (they are const). The
    // 'mutable' keyword lifts this restriction. Note: It modifies the COPY, not the original!
    // TR: Varsayılan olarak DEĞER ile yakalanan kopya const'tır, değiştirilemez. 'mutable' bu
    // kısıtlamayı kaldırır. Not: KOPYAyı değiştirir, orijinali DEĞİL!

    int sequenceId = 1000;
    auto generateId = [sequenceId]() mutable -> int {
      // EN: Each call increments the Lambda's internal copy.
      // TR: Her çağrıda Lambda'nın iç kopyası artar.
      return sequenceId++;
    };

    std::cout << "6. Mutable Lambda (sequence generator):" << std::endl;
    std::cout << "   ID: " << generateId() << std::endl; // 1000
    std::cout << "   ID: " << generateId() << std::endl; // 1001
    std::cout << "   ID: " << generateId() << std::endl; // 1002
    std::cout << "   Original sequenceId still = " << sequenceId
              << "\n" << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // 7. INIT CAPTURE / MOVE CAPTURE (C++14) / BASLATICI YAKALAMA
    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: C++14 allows creating new variables inside the capture list itself. This is essential
    // for MOVE SEMANTICS — you can move a unique_ptr or a large buffer into the Lambda without
    // copying.
    // TR: C++14 yakalama listesi içinde yeni değişken oluşturmaya izin verir. Bu, TAŞIMA SEMANTİĞİ
    // için kritiktir — unique_ptr veya büyük bir tampon belleği kopyalamadan Lambda'ya
    // taşıyabilirsiniz.

    std::string rawFrame = "CAN_ID:0x7E8 DATA:41 0C 1A F0";
    auto processFrame = [frame = std::move(rawFrame)]() {
      std::cout << "7. Init-capture (moved): Processing frame: "
                << frame << std::endl;
    };
    processFrame();
    std::cout << "   Original rawFrame is now empty: \""
              << rawFrame << "\"\n" << std::endl;

```

```cpp
    // ═══════════════════════════════════════════════════════════════════════════
    // 8. LAMBDA WITH STL ALGORITHMS / STL ALGORITMALARI ILE LAMBDA
    // ═══════════════════════════════════════════════════════════════════════════
    // EN: This is where Lambdas truly shine in automotive software.
    // TR: Lambda'ların otomotiv yazılımında gerçekten parladığı yer.

    // --- 8a. std::sort — Sort DTCs by severity (descending) ---
    // EN: Custom sorting: highest severity first for prioritized display.
    // TR: Özel sıralama: öncelikli görüntüleme için en yüksek önem derecesi önce.
    std::vector<DTC> sortedFaults = faultLog;
    std::sort(sortedFaults.begin(), sortedFaults.end(),
              [](const DTC& a, const DTC& b) {
      return a.severity > b.severity;
    });
    std::cout << "8a. std::sort by severity (descending):" << std::endl;
    for (const auto& dtc : sortedFaults) {
      std::cout << "    [" << static_cast<int>(dtc.severity) << "] 0x"
                << std::hex << dtc.code << std::dec
                << " " << dtc.name << std::endl;
    }

    // --- 8b. std::find_if — Find first fatal DTC ---
    // EN: Search for the first DTC with severity == 4 (fatal).
    // TR: Önem derecesi == 4 (ölümcül) olan ilk DTC'yi ara.
    auto it = std::find_if(faultLog.begin(), faultLog.end(),
                           [](const DTC& dtc) {
      return dtc.severity == 4;
    });
    if (it != faultLog.end()) {
      std::cout << "\n8b. std::find_if: First fatal DTC -> 0x"
                << std::hex << it->code << std::dec
                << " " << it->name << std::endl;
    }

    // --- 8c. std::transform — Extract fault names into a string vector ---
    // EN: Map DTCs to their names (functional-style projection).
    // TR: DTC'leri isimlerine dönüştür (fonksiyonel-stil projeksiyon).
    std::vector<std::string> faultNames(faultLog.size());
    std::transform(faultLog.begin(), faultLog.end(), faultNames.begin(),
                   [](const DTC& dtc) { return dtc.name; });
    std::cout << "\n8c. std::transform: DTC names extracted ("
              << faultNames.size() << " items)" << std::endl;

    // --- 8d. std::accumulate — Sum all severity scores ---
    // EN: Total severity score across all DTCs (risk assessment).
    // TR: Tüm DTC'lerin toplam önem puanı (risk değerlendirmesi).
    int totalSeverity = std::accumulate(
        faultLog.begin(), faultLog.end(), 0,
        [](int sum, const DTC& dtc) {
      return sum + static_cast<int>(dtc.severity);
    });
    std::cout << "8d. std::accumulate: Total severity score = "
              << totalSeverity << "\n" << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════
    // 9. GENERIC LAMBDA (C++14 auto) / JENERIK LAMBDA
    // ═══════════════════════════════════════════════════════════════════════════
    // EN: Use 'auto' parameters to create Lambdas that work with ANY type. The compiler generates
    // a templated operator() behind the scenes.
    // TR: HERHANGİ bir tip ile çalışan Lambda'lar oluşturmak için 'auto' parametre kullanın.
    // Derleyici arka planda şablonlu operator() üretir.

    auto printReading = [](const auto& r) {
      std::cout << "   Sensor: " << r.sensor
                << " = " << r.value << " " << r.unit << std::endl;
    };

    std::vector<SensorReading> readings = {
        {"Coolant Temp",  92.5, "C"},
        {"Engine RPM",  3500.0, "rpm"},
        {"Battery",      13.8, "V"},
        {"Fuel Pressure", 3.2, "bar"},
    };

```

```cpp
    std::cout << "9. Generic Lambda (auto) — Sensor Readings:" << std::endl;
    std::for_each(readings.begin(), readings.end(), printReading);
    std::cout << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // 10. std::function — STORING LAMBDAS / LAMBDA SAKLAMA
    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: auto is great for local use, but if you need to store a Lambda in a class member,
    // container, or pass it through an API, use std::function. Trade-off: small overhead (type
    // erasure + possible heap allocation).
    // TR: auto yerel kullanım için harikadır, ama Lambda'yı sınıf üyesinde, container'da
    // saklamanız veya API'den geçirmeniz gerekirse std::function kullanın. Takas: küçük maliyet
    // (tip silme + olası heap tahsisi).

    // EN: ECU callback registry pattern — store different validation rules.
    // TR: ECU geri-çağırma kayıt deseni — farklı doğrulama kurallarını sakla.
    std::vector<std::function<bool(const DTC&)>> validators;

    validators.push_back([](const DTC& d) { return d.active; });
    validators.push_back([](const DTC& d) { return d.severity >= 3; });
    validators.push_back([](const DTC& d) { return d.code < 0x0500; });

    DTC sample{0x0400, "Throttle Pos Sensor", 3, true};
    std::cout << "10. std::function validators for DTC 0x0400:" << std::endl;
    int passCount = 0;
    for (size_t i = 0; i < validators.size(); ++i) {
      bool result = validators[i](sample);
      std::cout << "    Validator " << (i + 1) << ": "
                << (result ? "PASS" : "FAIL") << std::endl;
      if (result) passCount++;
    }
    std::cout << "    Passed " << passCount << "/"
              << validators.size() << "\n" << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // 11. LAMBDA AS FUNCTION PARAMETER (HIGHER-ORDER FUNCTIONS) LAMBDA'YI FONKSIYON PARAMETRESI
    // OLARAK GECIRME
    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: You can write your own functions that ACCEPT Lambdas as parameters. This is the
    // "Higher-Order Function" pattern — a function that takes another function as an argument.
    // Using templates avoids std::function overhead.
    // TR: Lambda'ları parametre olarak KABUL EDEN kendi fonksiyonlarınızı yazabilirsiniz. Bu
    // "Yüksek Dereceli Fonksiyon (Higher-Order Function)" desenidir. Template kullanmak
    // std::function maliyetinden kaçınır.

    // EN: Using a Lambda with std::count_if for filtered counting.
    // TR: Filtrelenmiş sayma için std::count_if ile Lambda kullanımı.
    auto activeCount = std::count_if(faultLog.begin(), faultLog.end(),
                                     [](const DTC& d) { return d.active; });
    auto severeActive = std::count_if(faultLog.begin(), faultLog.end(),
                                      [](const DTC& d) {
      return d.active && d.severity >= 3;
    });
    std::cout << "11. Higher-order: Active DTCs = " << activeCount
              << ", Severe+Active = " << severeActive << "\n" << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // 12. IMMEDIATELY INVOKED LAMBDA EXPRESSION (IILE) ANINDA CAGIRILAN LAMBDA IFADESI
    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: Call a Lambda immediately at the point of definition. Useful for complex initialization
    // of const variables.
    // TR: Lambda'yı tanımlandığı noktada hemen çağırın. const değişkenlerin karmaşık başlatılması
    // için kullanışlıdır.

    const std::string ecuStatus = [&]() -> std::string {
      int fatal = 0;
      for (const auto& d : faultLog) {
        if (d.active && d.severity == 4) fatal++;
      }
      if (fatal > 0) return "LIMP MODE";
      if (activeFaultCount > 3) return "DEGRADED";
      return "NORMAL";
```

```
340     }(); // EN: Note the () at the end — immediate invocation!
341          // TR: Sondaki () — hemen çağırma!
342
343     std::cout << "12. IILE: ECU Status = " << ecuStatus << "\n" << std::endl;
344
345     // ═══════════════════════════════════════════════════════════════════════════════════
346     // 13. CONSTEXPR LAMBDA (C++17) / DERLEME-ZAMANI LAMBDA
347     // ═══════════════════════════════════════════════════════════════════════════════════
348     // EN: Since C++17, Lambdas can be constexpr — computed at compile time. Perfect for automotive
349     // look-up tables and protocol constants.
350     // TR: C++17'den beri Lambda'lar constexpr olabilir — derleme zamanında hesaplanır. Otomotiv
351     // arama tabloları ve protokol sabitleri için ideal.
352
353     constexpr auto crcByte = [](uint8_t data) constexpr -> uint8_t {
354       // EN: Simplified CRC-8 step for demonstration.
355       // TR: Gösterim amaçlı basitleştirilmiş CRC-8 adımı.
356       uint8_t crc = data;
357       for (int i = 0; i < 8; ++i) {
358         if (crc & 0x80U) {
359           crc = static_cast<uint8_t>((crc << 1U) ^ 0x07U);
360         } else {
361           crc = static_cast<uint8_t>(crc << 1U);
362         }
363       }
364       return crc;
365     };
366
367     constexpr uint8_t checksum = crcByte(0xA5);
368     std::cout << "13. constexpr Lambda: CRC-8(0xA5) = 0x"
369               << std::hex << static_cast<int>(checksum) << std::dec
370               << " (computed at compile time!)\n" << std::endl;
371
372     // ═══════════════════════════════════════════════════════════════════════════════════
373     // 14. RECURSIVE LAMBDA (via std::function) / OZYINELEMELI LAMBDA
374     // ═══════════════════════════════════════════════════════════════════════════════════
375     // EN: Lambdas cannot directly call themselves (they have no name). To make a recursive Lambda,
376     // wrap it in std::function so it can reference itself through the captured variable.
377     // TR: Lambda'lar kendilerini doğrudan çağıramaz (isimleri yoktur). Özyinelemeli Lambda yapmak
378     // için std::function ile sarmalayın, böylece yakalanan değişken üzerinden kendine referans
379     // verebilir.
380
381     // EN: Compute checksum of a DTC code tree (recursive severity sum).
382     // TR: DTC kodu ağacının sağlama toplamını hesapla (özyinelemeli önem toplamı).
383     std::function<int(const std::vector<DTC>&, size_t)> recursiveSum =
384         [&recursiveSum](const std::vector<DTC>& dtcs, size_t idx) -> int {
385       if (idx >= dtcs.size()) return 0;
386       return static_cast<int>(dtcs[idx].severity) +
387              recursiveSum(dtcs, idx + 1);
388     };
389
390     std::cout << "14. Recursive Lambda: Total severity (recursive) = "
391               << recursiveSum(faultLog, 0) << "\n" << std::endl;
392
393     // ═══════════════════════════════════════════════════════════════════════════════════
394     // 15. LAMBDA RETURNING A LAMBDA (FACTORY PATTERN) LAMBDA DONDUREN LAMBDA (FABRIKA DESENI)
395     // ═══════════════════════════════════════════════════════════════════════════════════
396     // EN: A Lambda can return ANOTHER Lambda. This is a powerful functional programming pattern —
397     // creating specialized filters at runtime.
398     // TR: Bir Lambda BAŞKA bir Lambda döndürebilir. Bu güçlü bir fonksiyonel programlama desenidir
399     // — çalışma zamanında özelleşmiş filtreler oluşturur.
400
401     auto makeSeverityFilter = [](uint8_t minSev) {
402       return [minSev](const DTC& dtc) {
403         return dtc.active && dtc.severity >= minSev;
404       };
405     };
406
407     auto criticalFilter = makeSeverityFilter(4); // fatal only
408     auto warningFilter = makeSeverityFilter(2);  // warning and above
409
410     std::cout << "15. Lambda factory:" << std::endl;
411     std::cout << "    Oil Pressure Low fatal? "
412               << (criticalFilter(faultLog[1]) ? "YES" : "NO") << std::endl;
413     std::cout << "    Battery Voltage warning+? "
414               << (warningFilter(faultLog[2]) ? "YES" : "NO")
415               << " (inactive, so NO)\n" << std::endl;
416
```

```
417     // ═══════════════════════════════════════════════════════════════════════════
418     // SUMMARY / OZET
419     // ═══════════════════════════════════════════════════════════════════════════
420     std::cout << "=== LAMBDA CHEAT SHEET ===" << std::endl;
421     std::cout << "  [](){}        — Simplest Lambda" << std::endl;
422     std::cout << "  [=]           — Capture all by value" << std::endl;
423     std::cout << "  [&]           — Capture all by reference" << std::endl;
424     std::cout << "  [x, &y]       — Explicit mixed capture" << std::endl;
425     std::cout << "  [x=move(v)]   — Init/move capture (C++14)" << std::endl;
426     std::cout << "  mutable       — Modify value-captured copies" << std::endl;
427     std::cout << "  auto params   — Generic Lambda (C++14)" << std::endl;
428     std::cout << "  constexpr     — Compile-time Lambda (C++17)" << std::endl;
429     std::cout << "  std::function — Type-erased storage" << std::endl;
430     std::cout << "  IILE [](){}() — Immediate invocation" << std::endl;
431     std::cout << "\nAll examples use Automotive/Embedded domain data." << std::endl;
432
433     return 0;
434   }
```

► 03. Move Semantics Rvalue

module_03_modern_cpp/03_move_semantics_rvalue.cpp

Konu: Modern C++: Move Semantics & R-Values — Taşıma Semantiği ve L-Value vs R-Value (C++11

Teori / Theory

THEORY: Why are C++03 and C++11 practically "different languages"?

Before C++11, returning a HUGE object (like a 4GB Vector) from a function forced the compiler to perfectly COPY all 4GB of data byte by byte to the new variable, and then immediately DESTROY the old 4GB vector inside the function.
C++11 introduced "Move Semantics" (std::move). Instead of copying 4GB, it steals the internal pointer of the old vector, gives it to the new one, and leaves the old one empty. Cost: O(N) -> O(1). 4GB data copied in 1 nanosecond!

TEORİ: Büyük C++ Atılımı

C++11 öncesinde, bir fonksiyonun içinden devasa büyüklükte (örneğin 4 GB'lik) bir `vector` döndürdüğünüzde, derleyici bu 4 GB'ı bayt bayt yeni değişkene KOPYALAR (RAM ikiye katlanır), sonra fonksiyon içindeki o orijinal 4 GB'ı ÇÖPE ATARDI. Bu tam bir performans katliamıydı.
C++11 "Move Semantics" (Taşıma Semantiği) getirdi. Artık 4 GB'ı kopyalamıyoruz. "Madem fonksiyonun içindeki nesne birazdan ölecek, onun içindeki işçiyi (pointer'ı) ÇALALIM (Steal), yeni değişkene teslim edelim, ölecek olanın içini boşaltıp (null) cenazesini kaldıralım." Maliyet: O(N)'den -> O(1)'e inmiştir. 1 Nanosaniyede 4 GB veri taşınır!

THEORY: L-Value vs R-Value

1. L-Value (Left Value): Has a permanent memory address, a name, and can sit on the left of an equal sign. (`int x = 5;` -> 'x' is an L-Value).
2. R-Value (Right Value): A temporary, nameless object born to die. (`5` or the unassigned return of `factory()` is an R-Value).
3. L-Value Reference (`&`): Classic reference.
4. R-Value Reference (`&&`): C++11 feature. Specifically catches TEMPORARY, DYING objects so you can "Steal/Move" their internals!

TEORİ: Sol Taraf vs Sağ Taraf Değeri

1. L-Value (Left Value): RAM'de yer edinen, bir ismi (adresi) olan ve eşittirin solunda olabilen nesne. (`int x = 5;` -> 'x' bir L-Value'dir).
2. R-Value (Right Value): Geçici, ismi olmayan, şipşak yaratılıp ölen nesne. (`5` veya `factory()` fonksiyonunun havada kalan dönüşü R-Value'dir).
3. L-Value Reference (`&`): Klasik C++ referansı (Sadece kalıcıları tutar).
4. R-Value Reference (`&&`): C++11 Devrimi! GEÇİCİ, ÖLECEK olan nesneleri yakalar ve onların donanımlarını "çalmanızı/taşımanızı" sağlar!

CPPREF Depth: The Secret of `std::move` - It DOES NOT move anything!

CppReference warns developers continuously: `std::move()` DOES NOT ACTUALLY MOVE DATA. Yes, really. It is just an unconditional `static_cast<T&&>(obj)`. It merely "re-labels" an L-Value into an R-Value explicitly, so the compiler is "tricked" into calling the Move Constructor instead of the Copy Constructor. The actual moving/stealing is done inside the Move Constructor!

CPPREF Derinlik: `std::move`'un Sırrı - Aslında Hiçbir Şeyi TAŞIMAZ!

CppReference'in en büyük sırrı: `std::move()` ASLA BİR ŞEY TAŞIMAZ! Evet.. Okyanus kadar veriniz olsa da, o sadece şipşak bir `static_cast<T&&>(obje)` çevirimidir. Yaptığı tek iş bir L-Value nesnesinin etiketini söküp "Bu artık R-Value (Ölecek) bir nesnedir" yazmaktır. Böylece derleyici kandırılır ve (Kopya) yerine (Taşıma) fonksiyonunu çağırır. Asıl hırsızlık/taşıma Move Constructor'ın süslü parantezlerinin içinde yapılır!

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <string>
#include <utility>
#include <vector>

class HeavyBuffer {
private:
  int size;
  int *data;

public:
  // [1] Normal Constructor / Normal Yapıcı
  HeavyBuffer(int bufferSize) : size(bufferSize) {
    data = new int[size];
    std::cout << "[Constructor] Allocated heavy data of size: " << size << " (Ağır veri üretildi.)"
        << std::endl;
  }

  // [2] DESTRUCTOR / YIKICI
  ~HeavyBuffer() {
    std::cout << "[Destructor] Memory wiped. Size: " << size << " (Bellek yıkıldı.)" << std::endl;
    delete[] data;
  }

  // [3] COPY CONSTRUCTOR (The Old, Slow, Expensive Way / Eski Yöntem - Yavaş ve
  // Pahalı)
  HeavyBuffer(const HeavyBuffer &other) : size(other.size) {
    std::cout << "\033[31m[COPY Constructor]\033[0m COPYING BYTE BY BYTE! Too "
                 "heavy... (Satır satır kopyalanıyor!)"
              << std::endl;
    data = new int[size];
    for (int i = 0; i < size; ++i)
      data[i] = other.data[i];
  }

```

```
36    // [4] MOVE CONSTRUCTOR (The C++11 Revolution using R-Value Reference '&&')
37    // EN: "other" is a temporary object. So we DO NOT make it "const". We will empty its guts
38    // (Steal).
39    // TR: "other" geçici bir nesnedir. O yüzden "const" YAPMIYORUZ, içini boşaltacağız
40    // (Hırsızlık).
41    HeavyBuffer(HeavyBuffer &&other) noexcept
42        : size(other.size), data(other.data) {
43
44      // [STEP 1]: We stole 'other's internal pointer and adopted it! (Zero
45      // Copying) [ADIM 1]: 'other'ın pointerını ('data') çaldık ve kendimize verdik. O(1) hız!
46      std::cout << "\033[32m[MOVE Constructor]\033[0m DATA STOLEN (Moved)! O(1) "
47                   "Speed... (Veri çalındı!)"
48                << std::endl;
49
50      // [STEP 2]: POST-THEFT. We MUST nullify 'other' so when it dies, it doesn't
51      // delete OUR stolen memory. [ADIM 2]: HIRSIZLIK SONRASI. 'other'ı nullptr yapmalıyız ki, o
52      // yıkılırken bizim çalınan RAM'imizi çökertmesin.
53      other.data = nullptr;
54      other.size = 0;
55    }
56  };
57
58  /**
59   * @brief EN: A factory function creating a massive temporary memory chunk.
60   * TR: Devasa geçici bir bellek üreten fonksiyon.
61   */
62  HeavyBuffer factory() {
63    HeavyBuffer temp(1000000); // 1 Million elements / 1 Milyon eleman
64    // EN: Temp is an R-Value! Returns a dying object.
65    // TR: Temp fonksiyondan çıkarken ölür! (R-Value)
66    return temp;
67  }
68
69  // ═══════════════════════════════════════════════════════════════════════════════════
70  // EN: PERFECT FORWARDING — Passing arguments without losing their value category
71  // TR: MÜKEMMEL YÖNLENDİRME — Argümanları değer kategorisini kaybetmeden aktarma
72  // ═══════════════════════════════════════════════════════════════════════════════════
73
74  // ─── 1. Helper Overloads for Forwarding Demo ──────────────────────────────────────
75  // EN: Two overloads that reveal whether the caller passed an L-value or R-value.
76  // TR: Çağıranın L-value mi yoksa R-value mi gönderdiğini ortaya çıkaran iki aşırı yükleme.
77
78  void process(const std::string& signal) {
79      // EN: Binds to L-values (named, persistent objects).
80      // TR: L-value'lere (isimli, kalıcı nesnelere) bağlanır.
81      std::cout << "  [process] L-value received: " << signal << std::endl;
82  }
83
84  void process(std::string&& signal) {
85      // EN: Binds to R-values (temporaries, dying objects).
86      // TR: R-value'lere (geçici, ölecek nesnelere) bağlanır.
87      std::cout << "  [process] R-value received: " << signal << std::endl;
88  }
89
90  // ─── 2. Forwarding Reference (Universal Reference) ────────────────────────────────
91  // EN: T&& in a template context is NOT an rvalue reference — it is a "forwarding reference"
92  // (also called universal reference). It deduces T as lvalue-ref or rvalue-ref depending on what
93  // the caller passes. std::forward<T>(arg) preserves the original value category.
94  // TR: Şablon bağlamında T&& bir rvalue referansı DEĞİLDİR — "yönlendirme referansı" (evrensel
95  // referans) adını alır. T, çağıranın gönderdiğine göre lvalue-ref veya rvalue-ref olarak
96  // çıkarılır. std::forward<T>(arg) orijinal değer kategorisini korur.
97
98  template <typename T>
99  void relay(T&& arg) {
100     // EN: std::forward keeps lvalue as lvalue, rvalue as rvalue.
101     // TR: std::forward, lvalue'yi lvalue, rvalue'yi rvalue olarak korur.
102     process(std::forward<T>(arg));
103 }
```

```cpp

// ─── 3. SensorPacket — Variadic Perfect Forwarding ──────────────────────────────────────────
// EN: Automotive ECU sensor data packet built via factory with zero unnecessary copies.
// TR: Otomotiv ECU sensör veri paketi — fabrika ile sıfır gereksiz kopya ile oluşturulur.

class SensorPacket {
private:
    std::string sensor_id_;
    double value_;
    std::string unit_;

public:
    SensorPacket(std::string id, double val, std::string unit)
        : sensor_id_(std::move(id)), value_(val), unit_(std::move(unit)) {
        std::cout << "  [SensorPacket] Constructed: " << sensor_id_
                  << " = " << value_ << " " << unit_ << std::endl;
    }

    friend std::ostream& operator<<(std::ostream& os, const SensorPacket& sp) {
        os << "[" << sp.sensor_id_ << ": " << sp.value_ << " " << sp.unit_ << "]";
        return os;
    }
};

// EN: Variadic template factory — every argument is perfectly forwarded into the SensorPacket
// constructor. No extra copy or move happens.
// TR: Değişken sayıda argümanlı şablon fabrikası — her argüman SensorPacket yapıcısına mükemmel
// şekilde yönlendirilir. Fazladan kopya/taşıma olmaz.

template <typename... Args>
SensorPacket createSensorPacket(Args&&... args) {
    return SensorPacket(std::forward<Args>(args)...);
}

// ═══════════════════════════════════════════════════════════════════════════════════════════

int main() {
  std::cout << "=== MODULE 3: MOVE SEMANTICS & R-VALUE (&&) ===\n" << std::endl;

  std::cout << "\n--- TEST 1: THE OLD C++ COPY (Wasted Power/Sokağa Atılan Güç) ---" << std::endl;
  HeavyBuffer bufferOne(50000);

  // EN: bufferOne is an L-Value (has a name). The compiler IS FORCED to perform a deep COPY.
  // TR: bufferOne bir L-Value'dir (ismi var). Derleyici MECBUREN derin Satır Satır Kopyalama
  // Yapar.
  HeavyBuffer copyBuffer = bufferOne;

  std::cout << "\n--- TEST 2: USING STD::MOVE (Forcible move / Zorla Taşıma) ---" << std::endl;
  // EN: Sometimes, even though bufferOne is an L-Value, we want to say "I'm done with you,
  // transfer to someone else". `std::move(bufferOne)` casts it to an R-Value (`&&`), triggering
  // the MOVE Constructor!
  // TR: Bazen bufferOne L-Value olsa da "Öl, donanımını yeniye aktar" demek isteriz.
  // `std::move(bufferOne)`, objeyi zorla R-Value'ye (`&&`) dönüştürür.
  HeavyBuffer movedBuffer = std::move(bufferOne);
  // WARNING (DİKKAT)! bufferOne is now an empty "shell". Its data is stolen. Do not use it
  // again!

  std::cout << "\n--- TEST 3: FACTORY (R-Value Passage / Fabrika R-Value Geçişi) ---" << std::endl;
  // EN: factory() returns a nameless (R-Value) object. Compiler detects this and triggers Move!
  // TR: factory() isimsiz geçici (R-Value) üretir. Derleyici bunu görür ve hemen Move (Taşıma)
  // tetikler!
  HeavyBuffer fastBuffer = factory();

  // ═══════════════════════════════════════════════════════════════════════════════════════════
  // EN: Demo 4 — Perfect Forwarding: relay() dispatches to correct overload
  // TR: Demo 4 — Mükemmel Yönlendirme: relay() doğru aşırı yüklemeyi çağırır
  // ═══════════════════════════════════════════════════════════════════════════════════════════
  std::cout << "\n--- DEMO 4: PERFECT FORWARDING (Mükemmel Yönlendirme) ---" << std::endl;

  std::string canSignal = "CAN_SPEED_0x1A3";

  // EN: canSignal is an L-value → relay deduces T = std::string&
  // TR: canSignal bir L-value → relay, T = std::string& olarak çıkarır
  std::cout << "  Passing L-value:" << std::endl;
  relay(canSignal);

```

```
    // EN: std::string{...} produces a temporary R-value → relay deduces T = std::string
    // TR: std::string{...} geçici R-value üretir → relay, T = std::string olarak çıkarır
    std::cout << "  Passing R-value:" << std::endl;
    relay(std::string{"CAN_RPM_0x0C6"});

    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: Demo 5 — Universal Reference vs Rvalue Reference
    // TR: Demo 5 — Evrensel Referans ile Rvalue Referansı Farkı
    // ═══════════════════════════════════════════════════════════════════════════════
    std::cout << "\n--- DEMO 5: UNIVERSAL REF vs RVALUE REF ---" << std::endl;

    // EN: KEY DISTINCTION: template<typename T> void foo(T&& x);   // <-- UNIVERSAL (forwarding)
    // ref void bar(std::string&& x);               // <-- plain RVALUE reference
    //
    // Universal ref (T&&) can bind to BOTH lvalue AND rvalue. Plain rvalue ref (Type&&) can ONLY
    // bind to rvalue.
    //
    // TR: KRİTİK FARK: template<typename T> void foo(T&& x);   // <-- EVRENSEL referans void
    // bar(std::string&& x);               // <-- düz RVALUE referansı
    //
    // Evrensel referans (T&&) hem lvalue hem rvalue'ya bağlanabilir. Düz rvalue ref (Tip&&)
    // YALNIZCA rvalue'ya bağlanabilir.

    // EN: Demonstrating that relay (universal ref) accepts both:
    // TR: relay (evrensel ref) her ikisini de kabul eder:
    relay(canSignal);                           // L-value OK
    relay(std::string{"TEMP_COOLANT_0xF4"});   // R-value OK

    // EN: A plain rvalue ref function would REJECT canSignal (l-value).
    // TR: Düz rvalue ref fonksiyon canSignal'i (l-value) REDDEDERDİ.
    std::cout << "  (Plain rvalue ref cannot accept L-values — compile error!)" << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: Demo 6 — Variadic Perfect Forwarding: createSensorPacket factory
    // TR: Demo 6 — Değişken Argümanlı Mükemmel Yönlendirme: Fabrika
    // ═══════════════════════════════════════════════════════════════════════════════
    std::cout << "\n--- DEMO 6: VARIADIC PERFECT FORWARDING (Fabrika) ---" << std::endl;

    // EN: All arguments are forwarded directly into the SensorPacket constructor.
    // TR: Tüm argümanlar doğrudan SensorPacket yapıcısına yönlendirilir.
    auto throttle  = createSensorPacket("THROTTLE_POS", 73.5, "percent");
    auto oilTemp   = createSensorPacket(std::string{"OIL_TEMP"}, 92.1, std::string{"celsius"});

    std::cout << "  Packet 1: " << throttle << std::endl;
    std::cout << "  Packet 2: " << oilTemp  << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: Demo 7 — std::forward vs std::move: when to use which?
    // TR: Demo 7 — std::forward vs std::move: hangisi ne zaman kullanılır?
    // ═══════════════════════════════════════════════════════════════════════════════
    std::cout << "\n--- DEMO 7: std::forward vs std::move ---" << std::endl;

    // EN: RULE OF THUMB: • std::move  → Use when you OWN the object and are DONE with it.
    // Unconditionally casts to rvalue. The source becomes empty. • std::forward → Use ONLY inside
    // templates with forwarding references (T&&). Conditionally casts: preserves lvalue/rvalue
    // category.
    //
    // TR: ALTIN KURAL: • std::move  → Nesneye SAHİPSENİZ ve artık İŞİNİZ BİTTİYSE kullanın.
    // Koşulsuz olarak rvalue'ya dönüştürür. Kaynak boşalır. • std::forward → YALNIZCA T&&
    // yönlendirme referanslı şablonlarda kullanın. Koşullu dönüştürme yapar: lvalue/rvalue
    // kategorisini korur.

    std::string ecu_msg = "ECU_DIAG_REQUEST";

    // EN: std::move — we are transferring ownership deliberately.
    // TR: std::move — sahipliği kasıtlı olarak devrediyoruz.
    std::string transferred = std::move(ecu_msg);
    std::cout << "  After std::move, original is empty: '" << ecu_msg << "'" << std::endl;
    std::cout << "  Transferred holds: '" << transferred << "'" << std::endl;

    // EN: std::forward — only meaningful inside relay() / template context. Outside templates,
    // prefer std::move for owned objects.
    // TR: std::forward — yalnızca relay() / şablon bağlamında anlamlıdır. Şablonların dışında,
    // sahip olduğunuz nesneler için std::move tercih edin.
    std::cout << "  (std::forward is used inside relay() — see Demo 4 output)" << std::endl;

```

```
256     // ═══════════════════════════════════════════════════════════════════════════════
257     // EN: Demo 8 — Reference Collapsing Rules / Referans Çökme Kuralları
258     // TR: Demo 8 — Referans Çökme Kuralları
259     // ═══════════════════════════════════════════════════════════════════════════════
260     std::cout << "\n--- DEMO 8: REFERENCE COLLAPSING RULES ---" << std::endl;
261
262     // EN: The C++ standard defines reference collapsing rules. When a template T&& is deduced:
263     //
264     //      ┌──────────────────┬──────────────┬──────────────────────────────────┐
265     //      │ T deduced as     │ T&&  becomes │ Explanation                      │
266     //      ├──────────────────┼──────────────┼──────────────────────────────────┤
267     //      │ Widget           │ Widget&&     │ Rvalue ref (caller passed rvalue)│
268     //      │ Widget&          │ Widget&      │ & + && = & (lvalue collapse)     │
269     //      │ Widget&&         │ Widget&&     │ && + && = && (rvalue stays)      │
270     //      └──────────────────┴──────────────┴──────────────────────────────────┘
271     //
272     //     Rule: If EITHER reference is an lvalue reference (&), result is &.
273     //           Only && + && = &&.
274     //
275     // TR: C++ standardı referans çökme (collapsing) kurallarını tanımlar:
276     //     Kural: Referanslardan BİRİ lvalue referansı (&) ise sonuç & olur.
277     //            Yalnızca && + && = && verir.
278
279     // EN: Demonstrate with type deduction
280     // TR: Tip çıkarımı ile gösterim
281     auto showRefType = [](auto&& val) {
282       // EN: If val binds to lvalue, T=string& → T&&=string& (collapsed)
283       //     If val binds to rvalue, T=string  → T&&=string&&
284       // TR: val lvalue'ya bağlanırsa, T=string& → T&&=string& (çökme)
285       //     val rvalue'ya bağlanırsa, T=string  → T&&=string&&
286       if constexpr (std::is_lvalue_reference_v<decltype(val)>) {
287         std::cout << "  Received as LVALUE reference (&)" << std::endl;
288       } else {
289         std::cout << "  Received as RVALUE reference (&&)" << std::endl;
290       }
291     };
292
293     std::string lval = "I am an lvalue";
294     showRefType(lval);                      // T = string&  → T&& = string&   (& + && = &)
295     showRefType(std::string{"I am rvalue"}); // T = string   → T&& = string&& (&& stays)
296     showRefType(std::move(lval));           // T = string   → T&& = string&& (xvalue → rvalue)
297
298     // ═══════════════════════════════════════════════════════════════════════════════
299     // EN: Demo 9 — Universal Reference vs Rvalue Reference: The Distinction
300     // TR: Demo 9 — Evrensel Referans ile Rvalue Referans Arasındaki Fark
301     // ═══════════════════════════════════════════════════════════════════════════════
302     std::cout << "\n--- DEMO 9: UNIVERSAL REF vs RVALUE REF DISTINCTION ---" << std::endl;
303
304     // EN: KEY DISTINCTION — memorize this:
305     //
306     //      ┌──────────────────────────────┬──────────────┬──────────────┐
307     //      │ Syntax                       │ What it is   │ Binds to     │
308     //      ├──────────────────────────────┼──────────────┼──────────────┤
309     //      │ void f(Widget&& w)           │ Rvalue reference │ Rvalues ONLY │
310     //      │ template<class T> f(T&& w)   │ Forwarding ref   │ BOTH lval & rval │
311     //      │ auto&& x = expr;             │ Forwarding ref   │ BOTH lval & rval │
312     //      │ Widget&& w = expr;           │ Rvalue reference │ Rvalues ONLY │
313     //      │ void f(vector<T>&& v)        │ Rvalue reference │ Rvalues ONLY │
314     //      └──────────────────────────────┴──────────────┴──────────────┘
315     //
316     //     The ONLY way to get a forwarding reference:
317     //     (1) T&& where T is deduced, OR (2) auto&&
318     //     vector<T>&& is NOT a forwarding ref because T is not deduced from &&.
319     //
320     // TR: ANAHTAR AYRIM:
321     //     Yönlendirme referansı elde etmenin TEK yolu:
322     //     (1) T'nin çıkarıldığı T&& VEYA (2) auto&&
323     //     vector<T>&& yönlendirme referansı DEĞİLDİR.
324
```

```cpp
    // EN: auto&& is a forwarding reference — deduces from initializer
    // TR: auto&& bir yönlendirme referansıdır — başlatıcıdan çıkarım yapar
    auto&& fwdRef1 = lval;                          // auto = string& → auto&& = string&
    auto&& fwdRef2 = std::string{"temp"};            // auto = string  → auto&& = string&&
    std::cout << "  auto&& from lvalue: "
              << (std::is_lvalue_reference_v<decltype(fwdRef1)> ? "lvalue ref" : "rvalue ref")
              << std::endl;
    std::cout << "  auto&& from rvalue: "
              << (std::is_lvalue_reference_v<decltype(fwdRef2)> ? "lvalue ref" : "rvalue ref")
              << std::endl;

    std::cout << "\n=== Program Ending (Destructors Will Run / Yıkıcılar "
                 "Çalışacak) ===\n"
              << std::endl;

    return 0;
}
```

► 04. Modern Types

module_03_modern_cpp/04_modern_types.cpp

Konu: Modern C++: New Data Types (C++17) — Modern Veri Tipleri: optional, variant, any,

Teori / Theory

THEORY: Why Do We Lose Points in Code Reviews?

In old C++ code, if a function failed, it usually returned "-1", "NULL" or "nullptr" (Magic Numbers). This is dangerous because what if "-1" is the ACTUAL valid answer? With C++17, things became extremely safe, similar to Rust.
1. std::optional<T> : Contains a value of type "T" OR it is empty (nullopt).
2. std::variant<T, Y> : A type-safe version of C's "union". Holds only ONE type at a time but allocates memory equal to the BIGGEST mapped type (RAM saver).
3. std::any : Holds "anything" (like variables in Python or JS). Heavy overhead.
4. std::string_view : A lightweight purely observational "window" to look at a string WITHOUT copying it!

TEORİ: Neden Puan Kaybederiz?

Eski C++ kodlarında bir fonksiyon başarılı olamazsa genellikle "-1", "NULL" veya "nullptr" döndürülürdü (Magic Numbers/Sihirli Sayılar). Bu tehlikelidir çünkü kişi gerçekten -1 değerini bulmuş olabilir! C++17 ile beraber işler "Rust" dillerine benzer şekilde muhteşem güvenli hale geldi.
1. std::optional<T> : İçinde "T" tipi bir değer YA VARDIR YA DA YOKTUR (nullopt).
2. std::variant<T, Y> : C'deki "union"ın güvenli halidir. Aynı anda tek tip tutar, RAM cimrisidir.
3. std::any : "İçine ne atarsan at" tipidir. Maliyetlidir, zorda kalınmadıkça kullanılmaz.
4. std::string_view : Bir metni kopyalamadan (yeni RAM tüketmeden) SADECE BAKMA penceresidir.

CPPREF Depth: auto Type Deduction Rules — Template Argument Deduction in Disguise

`auto` uses the same rules as template argument deduction (§7.1.7.4 / §13.10.2). When you write `auto x = expr;`, the compiler deduces the type as if `x` were a function template parameter: references and top-level const/volatile are stripped. This means:
• `const int& r = n; auto a = r;` → a is `int`, NOT `const int&`.
• `auto x = {1,2,3};` deduces `std::initializer_list<int>` (special rule).
• `auto x{42};` in C++17 deduces `int` (changed from init-list in C++11). `decltype(auto)` preserves the EXACT type including references:
• `decltype(auto) y = r;` → y is `const int&`. Parenthesized id-expressions change decltype semantics:
• `decltype(x)` → declared type of x.
• `decltype((x))` → always an lvalue reference (T&), because (x) is an lvalue expression, not an id-expression! This subtle difference is a common source of bugs in return type deduction with `decltype(auto)` — returning `(localVar)` gives a dangling reference.

CPPREF Derinlik: auto Tür Çıkarım Kuralları — Şablon Argüman Çıkarımının Gizli Yüzü

`auto` şablon argüman çıkarımıyla (template argument deduction) aynı kuralları kullanır. `auto x = expr;` yazıldığında derleyici, x'i bir şablon parametresiymiş gibi çıkarır: referanslar ve üst-düzey const/volatile atılır.
• `const int& r = n; auto a = r;` → a tipi `int`'tir, `const int&` DEĞİL.
• `auto x = {1,2,3};` → `std::initializer_list<int>` çıkarılır.
• `auto x{42};` → C++17'de `int` (C++11'de init-list'ti, değişti). `decltype(auto)` referanslar dahil TAM türü korur:
• `decltype(auto) y = r;` → y tipi `const int&`. Parantezli ifadeler decltype semantiğini değiştirir:
• `decltype(x)` → x'in bildirim türü.
• `decltype((x))` → her zaman lvalue referans (T&)! Çünkü (x) bir lvalue ifadesidir, id-expression değildir. Bu ince fark, `decltype(auto)` dönüş türüyle `(localVar)` döndürürken sarkan referans (dangling reference) hatasına yol açar.

Kaynak Kod / Source Code

```cpp
#include <any>
#include <iostream>
#include <optional>
#include <string>
#include <string_view>
#include <variant>

// 1. std::optional (C++17) Example: How to securely handle "Data Not Found" without MAGIC
// NUMBERS? (Veri bulamama durumu "-1" kullanılmadan nasıl çözülür?)
std::optional<int> findAgeByName(const std::string &name) {
  if (name == "Alice")
    return 25;
  if (name == "Bob")
    return 30;

  // EN: Instead of -1, we return 'nullopt' (no value)!
  // TR: Bulamadıysak -1 döndürmek yerine C++17'deki harika "nullopt" döneriz!
  return std::nullopt;
}

// 2. std::string_view (C++17) Example: Preventing massive RAM copies line by line!
// TR: Satır satır metin kopyalamayı önlemek! Eğer std::string alsaydık, o devasa metni RAM'e
// kopyalardık.
void printFirst5Chars(std::string_view text) {
  if (text.length() >= 5)
    std::cout << "[StringView] " << text.substr(0, 5) << "..." << std::endl;
}
```

```cpp

int main() {
  std::cout << "=== MODULE 3: MODERN TYPES (C++17) ===\n" << std::endl;

  // --- TEST 1: std::optional ---
  std::cout << "--- 1. std::optional ---" << std::endl;
  // Not in database / Veritabanında yok.
  auto ageResult = findAgeByName("Charlie");

  if (ageResult.has_value()) { // or just: if (ageResult)
    std::cout << "Age Found (Bulunan Yaş): " << ageResult.value() << std::endl;
  } else {
    std::cout << "Person DOES NOT EXIST (Bu isimde biri YOK. '-1' hatası "
                  "yapmadan güvenle çözüldü.)"
              << std::endl;
  }

  // --- TEST 2: std::variant ---
  std::cout << "\n--- 2. std::variant ---" << std::endl;
  // EN: An automotive diagnostic result. Can hold EITHER an int error code OR a string
  // description.
  // TR: Bir otomotiv teşhis sonucu. SADECE Ya hata kodu (int) ya açıklama (string) taşır!
  std::variant<int, std::string> diagnosticResult;

  diagnosticResult = 300; // Int Mode (DTC numeric code)
  std::cout << "DTC Code (Hata Kodu): " << std::get<int>(diagnosticResult) << std::endl;

  // Switch to String immediately (Old data vanishes/Yok olur)
  diagnosticResult = "P0300 - Random Misfire";
  std::cout << "DTC Description (Açıklama): " << std::get<std::string>(diagnosticResult) <<
      std::endl;

  // --- TEST 3: std::any ---
  std::cout << "\n--- 3. std::any ---" << std::endl;
  // EN: Like 'var' in JS. Heavy performance hit. DO NOT USE unless desperate.
  // TR: Dinamik tür! Performansı çok kötüdür, ZORUNDA KALMADIKÇA KULLANILMAZ.
  std::any anything = 3.14;
  anything = std::string("Dynamic Word / Dinamik Kelime");
  std::cout << "Content (Dinamik Yapı İçeriği): " << std::any_cast<std::string>(anything) <<
      std::endl;

  // --- TEST 4: std::string_view ---
  std::cout << "\n--- 4. std::string_view ---" << std::endl;
  const char *massiveText = "This is a very large text block. Copying it is suicide.";
  // TR: C Tipi metin veya C++ string fark etmez, RAM'de KOPYA ÜRETMEDEN okur!
  printFirst5Chars(massiveText);

  return 0;
}
```

► 05. Structured Bindings Optional

module_03_modern_cpp/05_structured_bindings_optional.cpp

Konu: Structured Bindings, std::optional, std::variant, std::any — Yapısal Bağlamalar ve Modern C++17 Sözcük Türleri

Teori / Theory

THEORY: Structured Bindings (C++17)

Structured bindings allow decomposing an object into its constituent elements using the syntax: auto [a, b, c] = expression; They work with std::pair, std::tuple, std::array, and any aggregate type (struct/class with all public non-static data members). In automotive ECU software, structured bindings dramatically simplify iteration over maps of sensor data, DTC records, and calibration tables.

TEORİ: Yapısal Bağlamalar (C++17)

Yapısal bağlamalar, bir nesneyi bileşenlerine ayırmayı sağlar: auto [a, b, c] = ifade; sözdizimi kullanılır. std::pair, std::tuple, std::array ve tüm üyeleri public olan herhangi bir aggregate türle (struct/class) çalışır. Otomotiv ECU yazılımında, sensör haritaları, DTC kayıtları ve kalibrasyon tablolarının dolaşımını büyük ölçüde basitleştirir.

THEORY: std::optional (C++17)

std::optional<T> represents a value that may or may not be present. It is a safer alternative to raw pointers, sentinel values (-1, NaN), or out-parameters for expressing "no result". Key API: has_value(), value(), value_or(default), operator*, operator bool. In automotive: a sensor may be offline, a DTC lookup may find nothing, a calibration parameter may be unconfigured — all modeled by optional.

TEORİ: std::optional (C++17)

std::optional<T>, var olabilecek veya olmayabilecek bir değeri temsil eder. Ham işaretçiler, sentinel değerler (-1, NaN) veya çıkış parametreleri yerine "sonuç yok" ifade etmek için daha güvenli bir alternatiftir. Temel API: has_value(), value(), value_or(varsayılan), operator*, operator bool. Otomotifte: sensör çevrimdışı, DTC araması sonuçsuz, kalibrasyon parametresi yapılandırılmamış olabilir — hepsi optional ile modellenir.

THEORY: std::variant & std::any (C++17)

std::variant<T1,T2,...> is a type-safe union — it holds exactly one of the listed types at any time. Access via std::get<T>, std::get<I>, or std::visit() with a visitor (often the "overloaded" lambda pattern). std::any is a type-safe void* — it can hold any copy-constructible type and is retrieved via std::any_cast<T>. Throws std::bad_any_cast on mismatch.
When to use which: optional → "might not exist" (nullable value semantics) variant → "one of several known types" (closed type set) any → "truly dynamic type" (open type set, use sparingly)

TEORİ: std::variant & std::any (C++17)

std::variant<T1,T2,...> tür-güvenli birleşimdir — listelenen türlerden her zaman tam olarak birini tutar. std::get<T>, std::get<I> veya std::visit() ile ziyaretçi deseni kullanılarak erişilir. std::any, tür-güvenli void* karşılığıdır — kopyalanabilir herhangi bir türü tutar, std::any_cast<T> ile alınır. Uyumsuzlukta std::bad_any_cast fırlatır.
Hangisi ne zaman kullanılır: optional → "olmayabilir" (null olabilen değer semantiği) variant → "bilinen birkaç türden biri" (kapalı tür kümesi) any → "gerçekten dinamik tür" (açık tür kümesi, dikkatli kullanılmalı)

CPPREF Depth: std::optional Deep Dive

`std::optional<T>` stores `T` in-place using aligned storage — there is NO heap allocation. Its size equals sizeof(T) + 1 byte boolean flag + padding for alignment. For example:
• `optional<int>` is typically 8 bytes (4 int + 1 flag + 3 pad).
• `optional<char>` is typically 2 bytes (1 char + 1 flag). `optional<optional<T>>` is valid and semantically different from `optional<T>`: the outer layer can be engaged holding a disengaged inner optional — two distinct "empty" states. Accessing a disengaged optional via `operator*` or `operator->` is undefined behavior (no exception thrown!). Use `value()` for checked access — it throws `std::bad_optional_access`. C++23 adds monadic operations inspired by functional programming:
• `and_then(f)` — flat-map: f returns optional<U>
• `transform(f)` — map: f returns U, wrapped in optional<U>
• `or_else(f)` — called only when disengaged These enable clean chaining: `opt.and_then(parse).transform(validate)` without nested if-checks.

CPPREF Derinlik: std::optional Derinlemesine

`std::optional<T>`, T'yi hizalı depolama (aligned storage) ile yerinde saklar — heap tahsisi YOKTUR. Boyutu = sizeof(T) + 1 bayt bayrak + hizalama dolgusu. Örneğin:
• `optional<int>` genelde 8 bayttır (4 int + 1 bayrak + 3 dolgu).
• `optional<char>` genelde 2 bayttır. `optional<optional<T>>` geçerlidir ve `optional<T>`'den farklıdır: dış katman, boş bir iç optional tutabilir — iki ayrı "boş" durum. Boş bir optional'a `operator*` ile erişmek tanımsız davranıştır (istisna FIRLATMAZ!). Kontrollü erişim için `value()` kullanılır — `std::bad_optional_access` fırlatır. C++23 fonksiyonel programlamadan esinlenen monadik işlemler ekler:
• `and_then(f)` — f, optional<U> döner (flat-map)
• `transform(f)` — f, U döner, optional<U> olarak sarılır (map)
• `or_else(f)` — yalnızca boşken çağrılır Böylece iç içe if kontrolü olmadan zincirleme yapılabilir: `opt.and_then(parse).transform(validate)`.

→ Ref: https://en.cppreference.com/w/cpp/utility/optional
→ Ref: https://en.cppreference.com/w/cpp/utility/variant
→ Ref: https://en.cppreference.com/w/cpp/utility/any

Kaynak Kod / Source Code

```cpp
#include <iostream>     // EN: Standard I/O
#include <string>       // EN: std::string
#include <map>          // EN: std::map for sensor readings
#include <optional>     // EN: std::optional — nullable value wrapper
#include <variant>      // EN: std::variant — type-safe union
#include <any>          // EN: std::any — type-erased container
#include <tuple>        // EN: std::tuple for structured bindings demo
#include <array>        // EN: std::array for structured bindings demo
#include <vector>       // EN: std::vector for parameter storage
#include <stdexcept>    // EN: std::runtime_error, std::bad_any_cast

// ═══════════════════════════════════════════════════════════════════════════
// 1. STRUCTURED BINDINGS WITH std::map — SENSOR READINGS
// ═══════════════════════════════════════════════════════════════════════════

// EN: Build a map of automotive sensor names to their current readings (volts).
// TR: Otomotiv sensör adlarını mevcut okumalarına (volt) eşleyen bir harita oluşturur.
std::map<std::string, double> getEngineSensorReadings() {
    return {
        {"CoolantTemp",    92.3},
        {"OilPressure",     3.8},
        {"ThrottlePos",    45.0},
        {"IntakeAirTemp",  38.7},
        {"BatteryVoltage", 13.9}
    };
}
```

```cpp

// EN: Print all sensor readings using structured bindings in a range-for loop.
// TR: Tüm sensör okumalarını yapısal bağlamalar ile range-for döngüsünde yazdırır.
void printSensorMap(const std::map<std::string, double>& sensorMap) {
    std::cout << "  Sensor readings:\n";
    for (const auto& [name, value] : sensorMap) {
        std::cout << "    " << name << " = " << value << "\n";
    }
}

// ═══════════════════════════════════════════════════════════════════════════════
// 2. STRUCTURED BINDINGS WITH STRUCT — ECU DIAGNOSTIC RESULT
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Diagnostic Trouble Code result from an ECU self-test.
// TR: ECU öz-test sonucundan gelen Arıza Teşhis Kodu (DTC) sonucu.
struct ECUDiagResult {
    int         dtcCode;        // EN: DTC numeric code (e.g., 301 = misfire cyl 1)
    std::string description;    // EN: Human-readable fault description
    bool        isPending;      // EN: true if fault is intermittent / not confirmed
};

// EN: Simulate reading a diagnostic result from ECU.
// TR: ECU'dan teşhis sonucu okumayı simüle eder.
ECUDiagResult getLatestDiagResult() {
    return {301, "Misfire detected — Cylinder 1", true};
}

// ═══════════════════════════════════════════════════════════════════════════════
// 3. std::optional — SENSOR READING THAT MAY BE ABSENT
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Represents a single sensor reading with metadata.
// TR: Meta verili tek bir sensör okumasını temsil eder.
struct SensorReading {
    std::string name;
    double      value;
    std::string unit;
};

// EN: Read sensor by ID. Returns std::nullopt if the sensor is offline.
// TR: Sensörü ID ile okur. Sensör çevrimdışıysa std::nullopt döndürür.
std::optional<SensorReading> readSensor(int sensorId) {
    // EN: Simulate: even IDs are online, odd IDs are offline.
    // TR: Simülasyon: çift ID'ler çevrimiçi, tek ID'ler çevrimdışı.
    if (sensorId % 2 == 0) {
        return SensorReading{"Sensor_" + std::to_string(sensorId),
                             25.0 + sensorId * 1.5,
                             "°C"};
    }
    return std::nullopt;
}

// ─── 3a. Optional Chaining — Multiple Sensors Must Be Online ─────────────────

// EN: Perform a combined diagnostic only if ALL required sensors are online.
// TR: Birleşik teşhisi yalnızca TÜM gerekli sensörler çevrimiçiyse gerçekleştirir.
void combinedDiagnostic(int id1, int id2, int id3) {
    auto s1 = readSensor(id1);
    auto s2 = readSensor(id2);
    auto s3 = readSensor(id3);

    if (s1 && s2 && s3) {
        // EN: All sensors online — proceed with combined check.
        // TR: Tüm sensörler çevrimiçi — birleşik kontrole devam et.
        std::cout << "  Combined diagnostic OK:\n";
        std::cout << "    " << s1->name << " = " << s1->value << " " << s1->unit << "\n";
        std::cout << "    " << s2->name << " = " << s2->value << " " << s2->unit << "\n";
        std::cout << "    " << s3->name << " = " << s3->value << " " << s3->unit << "\n";
    } else {
```

```
        // EN: At least one sensor is offline — report which ones.
        // TR: En az bir sensör çevrimdışı — hangilerini raporla.
        std::cout << "  Combined diagnostic FAILED — offline sensors:";
        if (!s1) std::cout << " Sensor_" << id1;
        if (!s2) std::cout << " Sensor_" << id2;
        if (!s3) std::cout << " Sensor_" << id3;
        std::cout << "\n";
    }
}

// ═══════════════════════════════════════════════════════════════════════════════
// 4. std::variant — POLYMORPHIC CAN MESSAGE PAYLOADS
// ═══════════════════════════════════════════════════════════════════════════════

// EN: CAN bus messages can carry different payload types depending on the signal.
// TR: CAN veri yolu mesajları, sinyale bağlı olarak farklı yük türleri taşıyabilir.
using CANPayload = std::variant<int, double, std::string>;

// EN: Overloaded helper — combines multiple lambdas into a single visitor.
// TR: Overloaded yardımcısı — birden fazla lambdayı tek bir ziyaretçide birleştirir.
template<class... Ts> struct overloaded : Ts... { using Ts::operator()...; };
template<class... Ts> overloaded(Ts...) -> overloaded<Ts...>;

// EN: Decode and display a CAN payload using std::visit + overloaded lambdas.
// TR: CAN yükünü std::visit + overloaded lambdalar ile çöz ve görüntüle.
void decodeCANPayload(const std::string& signalName, const CANPayload& payload) {
    std::cout << "    Signal [" << signalName << "]: ";
    std::visit(overloaded{
        [](int val) {
            // EN: Integer payload — e.g., gear position, RPM count.
            // TR: Tam sayı yükü — ör. vites konumu, RPM sayısı.
            std::cout << "INT = " << val << "\n";
        },
        [](double val) {
            // EN: Floating-point payload — e.g., temperature, voltage.
            // TR: Kayan nokta yükü — ör. sıcaklık, voltaj.
            std::cout << "DOUBLE = " << val << "\n";
        },
        [](const std::string& val) {
            // EN: String payload — e.g., VIN, diagnostic text.
            // TR: Metin yükü — ör. VIN, teşhis metni.
            std::cout << "STRING = \"" << val << "\"\n";
        }
    }, payload);
}

// ═══════════════════════════════════════════════════════════════════════════════
// 5. std::any — DYNAMIC ECU PARAMETER STORAGE
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Generic parameter store for ECU calibration — keys are string names, values can be any
// type (int thresholds, double gains, string labels).
// TR: ECU kalibrasyon için genel parametre deposu — anahtarlar string adlar, değerler herhangi
// bir tür olabilir (int eşikler, double kazançlar, string etiketler).
using ECUParamStore = std::map<std::string, std::any>;

// EN: Safely retrieve a parameter from the store with type checking.
// TR: Parametre deposundan tür kontrolüyle güvenli bir şekilde parametre alır.
template<typename T>
std::optional<T> getParam(const ECUParamStore& store, const std::string& key) {
    auto it = store.find(key);
    if (it == store.end()) {
        return std::nullopt;
    }
    try {
        return std::any_cast<T>(it->second);
    } catch (const std::bad_any_cast&) {
        // EN: Type mismatch — requested type doesn't match stored type.
        // TR: Tür uyumsuzluğu — istenen tür depolanan türle eşleşmiyor.
        return std::nullopt;
    }
}
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════════
// main()
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "=== Module 03 | 05: Structured Bindings, optional, variant, any ===\n\n";

    // ─── Demo 1: Structured Bindings — pair, tuple, struct, map ──────────────────

    std::cout << "[Demo 1] Structured Bindings\n";

    // EN: 1a — Decompose an std::pair (e.g., map insert result).
    // TR: 1a — std::pair ayrıştırma (ör. map ekleme sonucu).
    std::map<std::string, int> dtcCounts;
    auto [iter, inserted] = dtcCounts.emplace("P0301", 1);
    std::cout << "  Pair: key=" << iter->first
              << ", inserted=" << std::boolalpha << inserted << "\n";

    // EN: 1b — Decompose an std::tuple (e.g., ECU firmware version).
    // TR: 1b — std::tuple ayrıştırma (ör. ECU yazılım sürümü).
    std::tuple<int, int, int> firmwareVersion{2, 14, 7};
    auto [major, minor, patch] = firmwareVersion;
    std::cout << "  Tuple: firmware v" << major << "." << minor << "." << patch << "\n";

    // EN: 1c — Decompose an std::array.
    // TR: 1c — std::array ayrıştırma.
    std::array<double, 3> pidGains{0.8, 0.1, 0.05};
    auto [kp, ki, kd] = pidGains;
    std::cout << "  Array: PID gains Kp=" << kp << " Ki=" << ki << " Kd=" << kd << "\n";

    // EN: 1d — Decompose a user-defined struct.
    // TR: 1d — Kullanıcı tanımlı struct ayrıştırma.
    auto [code, desc, pending] = getLatestDiagResult();
    std::cout << "  Struct: DTC " << code << " — \"" << desc
              << "\" pending=" << std::boolalpha << pending << "\n";

    // EN: 1e — Iterate over a map with structured bindings.
    // TR: 1e — Yapısal bağlamalarla harita üzerinde dolaşma.
    auto sensors = getEngineSensorReadings();
    printSensorMap(sensors);

    std::cout << "\n";

    // ─── Demo 2: std::optional — Sensor Read (present / absent) ──────────────────

    std::cout << "[Demo 2] std::optional — Sensor Read\n";

    for (int id : {2, 3, 6, 7}) {
        auto reading = readSensor(id);
        if (reading.has_value()) {
            // EN: Sensor is online — display value.
            // TR: Sensör çevrimiçi — değeri göster.
            std::cout << "  Sensor " << id << ": " << reading->name
                      << " = " << reading->value << " " << reading->unit << "\n";
        } else {
            // EN: Sensor is offline — use value_or as fallback demonstration.
            // TR: Sensör çevrimdışı — value_or ile varsayılan değer gösterimi.
            SensorReading fallback{"N/A", 0.0, ""};
            auto safe = reading.value_or(fallback);
            std::cout << "  Sensor " << id << ": OFFLINE (fallback name=\""
                      << safe.name << "\")\n";
        }
    }

    std::cout << "\n";

    // ─── Demo 3: Optional Chaining — Multiple Sensors ────────────────────────────

    std::cout << "[Demo 3] Optional Chaining — Combined Diagnostic\n";

    // EN: All even IDs → all online → success.
    // TR: Tüm çift ID'ler → hepsi çevrimiçi → başarılı.
    combinedDiagnostic(2, 4, 6);

```

```
    // EN: Mixed IDs → some offline → failure with report.
    // TR: Karışık ID'ler → bazıları çevrimdışı → raporlu başarısızlık.
    combinedDiagnostic(2, 3, 6);

    std::cout << "\n";

    // ─── Demo 4: std::variant — CAN Payload with std::visit ──────────────────────────

    std::cout << "[Demo 4] std::variant — CAN Message Payloads\n";

    // EN: Simulate CAN bus messages with different payload types.
    // TR: Farklı yük türleriyle CAN veri yolu mesajlarını simüle eder.
    std::vector<std::pair<std::string, CANPayload>> canMessages = {
        {"GearPosition",    3},
        {"CoolantTemp",     91.7},
        {"VIN",             std::string("WVWZZZ3CZWE123456")},
        {"EngineRPM",       3200},
        {"BatteryVoltage",  13.8}
    };

    for (const auto& [signal, payload] : canMessages) {
        decodeCANPayload(signal, payload);
    }

    // EN: Check which type is currently held using index().
    // TR: index() kullanarak şu anda hangi türün tutulduğunu kontrol et.
    CANPayload sample = 42;
    std::cout << "  Variant index for int(42): " << sample.index() << "\n";
    sample = 3.14;
    std::cout << "  Variant index for double(3.14): " << sample.index() << "\n";
    sample = std::string("test");
    std::cout << "  Variant index for string: " << sample.index() << "\n";

    std::cout << "\n";

    // ─── Demo 5: std::any — Generic Parameter Map ────────────────────────────────────

    std::cout << "[Demo 5] std::any — ECU Parameter Store\n";

    ECUParamStore ecuParams;
    ecuParams["MaxRPM"]           = 7000;
    ecuParams["IdleTargetRPM"]    = 800;
    ecuParams["FuelTrimGain"]     = 1.05;
    ecuParams["CalibrationLabel"] = std::string("CAL_2026_Q1");

    // EN: Safe retrieval with correct type.
    // TR: Doğru türle güvenli erişim.
    if (auto rpm = getParam<int>(ecuParams, "MaxRPM")) {
        std::cout << "  MaxRPM = " << *rpm << "\n";
    }

    if (auto gain = getParam<double>(ecuParams, "FuelTrimGain")) {
        std::cout << "  FuelTrimGain = " << *gain << "\n";
    }

    if (auto label = getParam<std::string>(ecuParams, "CalibrationLabel")) {
        std::cout << "  CalibrationLabel = \"" << *label << "\"\n";
    }

    // EN: Wrong type → returns nullopt instead of crashing.
    // TR: Yanlış tür → çökmek yerine nullopt döndürür.
    if (auto bad = getParam<std::string>(ecuParams, "MaxRPM")) {
        std::cout << "  MaxRPM as string = " << *bad << "\n";
    } else {
        std::cout << "  MaxRPM as string: type mismatch (nullopt)\n";
    }

```

```
    // EN: Missing key → returns nullopt.
    // TR: Eksik anahtar → nullopt döndürür.
    if (auto missing = getParam<int>(ecuParams, "NonExistentParam")) {
        std::cout << "  NonExistentParam = " << *missing << "\n";
    } else {
        std::cout << "  NonExistentParam: not found (nullopt)\n";
    }

    std::cout << "\n";

    // ─── Demo 6: Comparison — When to Use optional vs variant vs any ──────────────────────────

    std::cout << "[Demo 6] Comparison — optional vs variant vs any\n\n";

    // EN: optional — "might not exist": a sensor reading that could be absent.
    // TR: optional — "olmayabilir": mevcut olmayabilecek bir sensör okuması.
    std::optional<double> oilTemp = 105.3;
    std::optional<double> exhaustTemp = std::nullopt;
    std::cout << "  optional<double> oilTemp: "
              << oilTemp.value_or(-1.0) << "\n";
    std::cout << "  optional<double> exhaustTemp: "
              << exhaustTemp.value_or(-1.0) << " (default = sensor offline)\n";

    // EN: variant — "one of several known types": a CAN signal value.
    // TR: variant — "bilinen birkaç türden biri": bir CAN sinyal değeri.
    std::variant<int, double, std::string> canSignal = 42;
    std::cout << "  variant<int,double,string>: holds index "
              << canSignal.index() << " (int)\n";
    canSignal = std::string("DIAGNOSTIC_OK");
    std::cout << "  variant<int,double,string>: holds index "
              << canSignal.index() << " (string)\n";

    // EN: any — "truly dynamic type": use when type set is open/unknown.
    // TR: any — "gerçekten dinamik tür": tür kümesi açık/bilinmediğinde kullanılır.
    std::any dynParam = 3.14;
    std::cout << "  any: type = " << dynParam.type().name()
              << ", value = " << std::any_cast<double>(dynParam) << "\n";
    dynParam = std::string("runtime_label");
    std::cout << "  any: type = " << dynParam.type().name()
              << ", value = \"" << std::any_cast<std::string>(dynParam) << "\"\n";

    std::cout << "\n  Rule of thumb:\n";
    std::cout << "    optional  → value may be absent (nullable semantics)\n";
    std::cout << "    variant   → value is one of N known types (closed set)\n";
    std::cout << "    any       → value is truly dynamic (open set, use sparingly)\n";

    std::cout << "\n=== End of Module 03 | 05 ===\n";
    return 0;
}
```

► 06. Cpp23 Features

module_03_modern_cpp/06_cpp23_features.cpp

Konu: C++23 Key Features — C++23 Önemli Özellikler

Teori / Theory

THEORY: C++23 — The Next Standard

C++23 brings significant quality-of-life and power features:

```
┌──────────────────────────┬──────────────────────────────────────────┐
│ Feature                  │ Header / Syntax                          │
├──────────────────────────┼──────────────────────────────────────────┤
│ std::expected<T,E>       │ <expected> — Error handling without throw  │
│ std::print / println     │ <print> — Type-safe I/O (fmtlib-based)     │
│ std::generator<T>        │ <generator> — Standard coroutine generator │
│ Deducing this            │ void f(this auto& self) — explicit self    │
│ if consteval             │ Compile-time vs runtime branching          │
│ std::mdspan              │ <mdspan> — Multidimensional view           │
│ std::flat_map/flat_set   │ <flat_map> — Cache-friendly sorted map     │
│ [[assume(expr)]]         │ Optimizer hint                             │
│ std::stacktrace          │ <stacktrace> — Runtime call stack          │
│ Multidimensional operator │ a[1,2,3] subscript                        │
│ size_t literals          │ 42uz (std::size_t literal)                 │
└──────────────────────────┴──────────────────────────────────────────┘
```

TEORİ: C++23 — Yeni Standart

C++23 önemli yaşam kalitesi ve güç özellikleri getirir:

```
┌──────────────────────────┬──────────────────────────────────────────┐
│ Özellik                  │ Başlık / Sözdizimi                       │
├──────────────────────────┼──────────────────────────────────────────┤
│ std::expected<T,E>       │ <expected> — throw'suz hata yönetimi       │
│ std::print / println     │ <print> — Tip güvenli G/Ç (fmtlib tabanlı) │
│ std::generator<T>        │ <generator> — Standart coroutine üretici   │
│ Deducing this            │ void f(this auto& self) — açık self        │
│ if consteval             │ Derleme zamanı / çalışma zamanı dallanma   │
│ std::mdspan              │ <mdspan> — Çok boyutlu görünüm             │
│ std::flat_map/flat_set   │ <flat_map> — Önbellek dostu sıralı map     │
│ [[assume(expr)]]         │ Optimize edici ipucu                       │
│ std::stacktrace          │ <stacktrace> — Çalışma zamanı çağrı yığını │
│ Çok boyutlu operatör     │ a[1,2,3] alt indis                         │
│ size_t sabitleri         │ 42uz (std::size_t sabiti)                  │
└──────────────────────────┴──────────────────────────────────────────┘
```

Compiler support (GCC 13): std::expected ✓, deducing this ✓, if consteval ✓. Print/generator/mdspan require GCC 14+.

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <expected>
#include <string>
#include <vector>
#include <type_traits>
#include <cmath>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: std::expected<T,E> — Error Handling Without Exceptions
// ═══════════════════════════════════════════════════════════════════════════

// EN: Error type for sensor operations
// TR: Sensör işlemleri için hata tipi
enum class SensorError {
    NotCalibrated,
    OutOfRange,
    Disconnected
};

std::string to_string(SensorError e) {
    switch (e) {
        case SensorError::NotCalibrated: return "NotCalibrated";
        case SensorError::OutOfRange:    return "OutOfRange";
        case SensorError::Disconnected:  return "Disconnected";
    }
    return "Unknown";
}

// EN: Functions returning expected instead of throwing
// TR: throw yerine expected döndüren fonksiyonlar
std::expected<double, SensorError> readSensor(int id) {
    if (id < 0) return std::unexpected(SensorError::Disconnected);
    if (id > 100) return std::unexpected(SensorError::OutOfRange);
    return 25.0 + static_cast<double>(id) * 0.5;  // simulated reading
}

std::expected<double, SensorError> calibrate(double raw) {
    if (raw < 0.0) return std::unexpected(SensorError::NotCalibrated);
    return raw * 1.05;  // apply 5% calibration factor
}

std::expected<std::string, SensorError> formatReading(double calibrated) {
    return "Temperature: " + std::to_string(calibrated) + "°C";
}

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: Deducing This
// ═══════════════════════════════════════════════════════════════════════════

// EN: "Deducing this" (explicit object parameter) lets a member function
//     deduce the type of *this, eliminating the need for const/non-const
//     overload pairs and enabling CRTP without inheritance.
//     NOTE: Requires GCC 14+ / Clang 18+ / MSVC 19.36+.
//     We demonstrate the PATTERN here with traditional code + reference syntax.
// TR: "Deducing this" (açık nesne parametresi), üye fonksiyonun *this'in
//     tipini çıkarmasını sağlar. GCC 14+ gerektirir. Deseni burada gösteriyoruz.

struct SensorData {
    std::vector<double> readings = {23.5, 24.1, 25.0, 22.8};

    // EN: Traditional C++ — two overloads needed:
    // TR: Geleneksel C++ — iki aşırı yükleme gerekir:
    //     double& front() { return readings.front(); }
    //     const double& front() const { return readings.front(); }
    //
    // C++23 deducing this — SINGLE function replaces both:
    //     auto&& front(this auto& self) { return self.readings.front(); }
    //     Deduces const/non-const from caller automatically!

    double& front() { return readings.front(); }
    const double& front() const { return readings.front(); }
    size_t size() const { return readings.size(); }
};
```

```cpp

// EN: CRTP-like without inheritance — deducing this reference
//     struct Printable {
//         void print(this auto const& self) {
//             std::cout << self.to_string();  // calls derived's method
//         }
//     };
//     struct Temp : Printable { string to_string() const { ... } };
//     Temp t(36.6); t.print();  // No virtual dispatch!
// TR: Miras olmadan CRTP benzeri — deducing this referansı

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: if consteval
// ═══════════════════════════════════════════════════════════════════════════

// EN: consteval forces compile-time evaluation. if consteval branches
//     between compile-time and runtime code paths.
// TR: consteval derleme zamanı değerlendirmeyi zorlar. if consteval
//     derleme zamanı ve çalışma zamanı kod yolları arasında dallanır.

consteval int compile_time_sqrt_approx(int n) {
    // EN: Simple integer sqrt — only available at compile time
    // TR: Basit tamsayı karekökü — yalnızca derleme zamanında kullanılabilir
    int result = 0;
    while ((result + 1) * (result + 1) <= n) ++result;
    return result;
}

constexpr double flexible_sqrt(double x) {
    if consteval {
        // EN: At compile-time: use consteval-compatible algorithm
        // TR: Derleme zamanında: consteval uyumlu algoritma kullan
        double guess = x / 2.0;
        for (int i = 0; i < 20; ++i) {
            guess = (guess + x / guess) / 2.0;
        }
        return guess;
    } else {
        // EN: At runtime: use optimized library function
        // TR: Çalışma zamanında: optimize edilmiş kütüphane fonksiyonu kullan
        return std::sqrt(x);
    }
}

// ═══════════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 03 - C++23 Features\n";
    std::cout << "============================================\n\n";

    // ─── Demo 1: std::expected basics ──────────────────────────────────
    {
        std::cout << "--- Demo 1: std::expected<T,E> ---\n";

        auto r1 = readSensor(42);
        auto r2 = readSensor(-1);
        auto r3 = readSensor(200);

        // EN: Check with has_value() or bool conversion
        // TR: has_value() veya bool dönüşümü ile kontrol et
        if (r1) {
            std::cout << "  Sensor 42: " << r1.value() << "°C\n";
        }
        if (!r2) {
            std::cout << "  Sensor -1: ERROR " << to_string(r2.error()) << "\n";
        }
        if (!r3) {
            std::cout << "  Sensor 200: ERROR " << to_string(r3.error()) << "\n";
        }

        // EN: value_or() — default on error
        // TR: value_or() — hata durumunda varsayılan değer
        double safe = r2.value_or(-999.0);
        std::cout << "  value_or(-999): " << safe << "\n\n";
    }
```

```

    // ─── Demo 2: expected monadic operations ──────────────────────────
    {
        std::cout << "--- Demo 2: expected Monadic Chaining ---\n";

        // EN: and_then chains operations that return expected
        //     transform maps the value (like optional::transform)
        //     or_else handles the error case
        // TR: and_then, expected döndüren işlemleri zincirler
        //     transform değeri dönüştürür
        //     or_else hata durumunu ele alır

        auto pipeline = readSensor(42)
            .and_then(calibrate)
            .transform([](double v) { return v + 0.5; })
            .and_then(formatReading);

        if (pipeline) {
            std::cout << "  OK: " << pipeline.value() << "\n";
        }

        // EN: Error pipeline — and_then short-circuits on error
        // TR: Hata hattı — and_then hata durumunda kısa devre yapar
        auto errPipeline = readSensor(-1)
            .and_then(calibrate)
            .and_then(formatReading)
            .or_else([](SensorError e) -> std::expected<std::string, SensorError> {
                return "FALLBACK: error was " + to_string(e);
            });

        if (errPipeline) {
            std::cout << "  Recovered: " << errPipeline.value() << "\n";
        }
        std::cout << "\n";
    }

    // ─── Demo 3: Deducing This ────────────────────────────────────────
    {
        std::cout << "--- Demo 3: Deducing This (reference) ---\n";

        SensorData data;
        const SensorData cdata;

        // EN: Traditional — needs two overloads. Deducing this replaces both:
        // TR: Geleneksel — iki aşırı yükleme gerekir. Deducing this ikisinin yerini alır:
        //     auto&& front(this auto& self) { return self.readings.front(); }
        std::cout << "  Mutable front: " << data.front() << "\n";
        std::cout << "  Const front: " << cdata.front() << "\n";
        std::cout << "  Size: " << data.size() << "\n";

        data.front() = 99.9;
        std::cout << "  After modify: " << data.front() << "\n";

        // EN: Deducing this syntax (GCC 14+):
        // TR: Deducing this sözdizimi (GCC 14+):
        std::cout << "\n  Deducing this eliminates overload pairs:\n";
        std::cout << "    // Before (2 functions):\n";
        std::cout << "    T& get() { return data_; }\n";
        std::cout << "    const T& get() const { return data_; }\n";
        std::cout << "    // After (1 function, C++23):\n";
        std::cout << "    auto&& get(this auto& self) { return self.data_; }\n\n";
    }

    // ─── Demo 4: if consteval ─────────────────────────────────────────
    {
        std::cout << "--- Demo 4: if consteval ---\n";

        // EN: Compile-time evaluation
        // TR: Derleme zamanlı değerlendirme
        constexpr double ct_sqrt = flexible_sqrt(144.0);
        std::cout << "  constexpr sqrt(144) = " << ct_sqrt << "\n";

```

```
        // EN: Runtime evaluation — uses std::sqrt
        // TR: Çalışma zamanlı değerlendirme — std::sqrt kullanır
        double runtime_val = 144.0;
        double rt_sqrt = flexible_sqrt(runtime_val);
        std::cout << "  runtime sqrt(144) = " << rt_sqrt << "\n";

        // EN: consteval integer sqrt
        // TR: consteval tamsayı karekökü
        constexpr int isqrt = compile_time_sqrt_approx(144);
        static_assert(isqrt == 12, "compile-time sqrt check");
        std::cout << "  consteval isqrt(144) = " << isqrt << "\n\n";
    }

    // ─── Demo 5: C++23 Features Reference (not yet in GCC 13) ──────────
    {
        std::cout << "--- Demo 5: C++23 Reference (compiler-dependent) ---\n";
        std::cout << "  std::print / std::println (GCC 14+, <print>):\n";
        std::cout << "    std::println(\"Sensor {}: {:.2f}C\", id, temp);\n";
        std::cout << "    // Type-safe, no format specifiers like printf\n\n";

        std::cout << "  std::generator<T> (GCC 14+, <generator>):\n";
        std::cout << "    std::generator<int> fibonacci() {\n";
        std::cout << "        int a=0, b=1;\n";
        std::cout << "        while (true) { co_yield a; auto t=a; a=b; b+=t; }\n";
        std::cout << "    }\n";
        std::cout << "    for (auto v : fibonacci() | views::take(10)) { ... }\n\n";

        std::cout << "  std::mdspan (GCC 14+, <mdspan>):\n";
        std::cout << "    int data[12];\n";
        std::cout << "    std::mdspan m(data, 3, 4);  // 3x4 matrix view\n";
        std::cout << "    m[1, 2] = 42;  // multi-dimensional subscript\n\n";

        std::cout << "  [[assume(expr)]] — optimizer hint:\n";
        std::cout << "    void process(int x) {\n";
        std::cout << "        [[assume(x > 0)]];  // UB if false, but optimizer uses it\n";
        std::cout << "    }\n\n";

        std::cout << "  std::flat_map / std::flat_set:\n";
        std::cout << "    See module_04_advanced/03_stl_containers_deepdive.cpp Demo 8\n\n";

        std::cout << "  Multidimensional subscript operator:\n";
        std::cout << "    template<class T> struct Matrix {\n";
        std::cout << "        T& operator[](size_t r, size_t c) { return data_[r*cols_+c]; }\n";
        std::cout << "    };\n";
        std::cout << "    m[2, 3] = 42;  // C++23: multiple args to operator[]\n\n";

        std::cout << "  size_t / signed size_t literals:\n";
        std::cout << "    auto x = 42uz;   // std::size_t\n";
        std::cout << "    auto y = 42z;    // std::ssize_t (signed)\n\n";
    }

    std::cout << "============================================\n";
    std::cout << "   End of C++23 Features\n";
    std::cout << "============================================\n";

    return 0;
}
```

Modül 4: Templates & STL — Şablonlar ve STL

Şablonlar ve STL

9 kaynak dosya · Standart: C++17

C++17

► 01. Templates Stl

module_04_advanced/01_templates_stl.cpp

Konu: Advanced: Templates and STL — İleri Seviye: Şablonlar (Derleme Zamanı Büyüsü) ve STL

Teori / Theory

THEORY: Why Were Templates Invented?

Imagine you want to find the maximum of two numbers. You write a function `max(int a, int b)`. But tomorrow, you need `max(double a, double b)`. Next week, `max(string a, string b)`. Should you write 100 overloaded functions? NO! In C++, Templates are "Code that writes code". You write ONE template, and the compiler generates the specific versions (int, float, custom class) FOR YOU at Compile-Time. This is called "Template Instantiation". The beauty is that the same logic applies to ANY type that supports the operations used inside the template — this is called "Structural Typing" or "Duck Typing at Compile-Time".
There are two kinds of templates:
1. Function Templates: Generic functions like `findMax<T>(T a, T b)`.
2. Class Templates: Generic classes like `Box<T>`, `std::vector<T>`.
The compiler performs "Type Deduction" — if you call `findMax(3, 5)` it automatically deduces `T = int`. For class templates (before C++17 CTAD), you must specify the type explicitly: `Box<int> b(42);`

TEORİ: Şablonlar (Templates) Neden Yaratıldı?

İki sayının maksimumunu bulan bir `max(int a, int b)` yazdın. Yarın `max(double, double)` gerekti, haftaya `max(string, string)`. 100 farklı fonksiyon mu yazacaksın? HAYIR! C++'da Şablonlar (Templates) "Kod Yazan Kodlardır". Sen sadece tek bir T (Zaman/Tip) şablonu yazarsın, DERLEYİCİ program derlenirken (Compile-Time) arka planda senin için o tiplere özel fonksiyonları (veya sınıfları) fiziksel olarak üretir. Buna "Şablon Somutlaştırma (Template Instantiation)" denir. Şablon içinde kullanılan operasyonları destekleyen HER tip ile çalışır — buna derleme zamanı "Ördek Tipleme" denir (Duck Typing at Compile-Time).
İki çeşit şablon vardır:
1. Fonksiyon Şablonları: `findMax<T>(T a, T b)` gibi jenerik fonksiyonlar.
2. Sınıf Şablonları: `Box<T>`, `std::vector<T>` gibi jenerik sınıflar.
Derleyici "Tip Çıkarımı (Type Deduction)" yapar — `findMax(3, 5)` çağrısında otomatik olarak `T = int` anlar. Sınıf şablonlarında (C++17 CTAD öncesi) tipi açıkça belirtmelisin: `Box<int> b(42);`

THEORY: The Standard Template Library (STL)

The STL is one of the greatest engineering marvels. It has three pillars:
1. Containers (vector, map, set, list, deque) -> Store data.
2. Iterators (begin(), end(), rbegin()) -> Object-oriented pointers.
3. Algorithms (std::sort, std::find, std::transform) -> Process data.
The key insight: Algorithms don't know about containers. Containers don't know about algorithms. They communicate ONLY through iterators! This decoupling means `std::sort` works on vector, array, deque — anything with random-access iterators. And `std::find` works on ANY container with forward iterators.
Common STL containers and their use cases:
1. `std::vector<T>`: Dynamic array, contiguous memory, fast random access. Best for: most general-purpose sequential data.
2. `std::map<K,V>`: Red-black tree, sorted key-value pairs, O(log n) lookup. Best for: ordered dictionaries.
3. `std::unordered_map<K,V>`: Hash table, O(1) average lookup. Best for: fast key-value lookups without ordering.
4. `std::set<T>`: Sorted unique elements. Best for: membership testing.
5. `std::deque<T>`: Double-ended queue, fast push/pop at both ends.

TEORİ: Standart Kütüphane ve Algoritması

STL (Standard Template Library) dünyanın en harika mühendislik ürünlerinden biridir. Üç temel bacağı vardır:
1. Konteynerler (Containers: vector, map, set, list, deque -> Veriyi depolar)
2. İteratörler (Iterators: begin(), end(), rbegin() -> Nesne odaklı ptr)
3. Algoritmalar (Algorithms: std::sort, std::find, std::transform)
Kilit bilgi: Algoritmalar konteynerleri bilmez. Konteynerler algoritmaları bilmez. Sadece İteratörler aracılığıyla iletişim kurarlar! Bu ayrışma sayesinde `std::sort` vector, array, deque üzerinde — random-access iteratörü olan her şeyde çalışır. `std::find` ise forward iteratörü olan HER konteynerde çalışır.
Yaygın STL konteynerleri ve kullanım alanları:
1. `std::vector<T>`: Dinamik dizi, bitişik bellek, hızlı rastgele erişim.
2. `std::map<K,V>`: Kırmızı-siyah ağaç, sıralı anahtar-değer, O(log n).
3. `std::unordered_map<K,V>`: Hash tablo, ortalama O(1) arama.
4. `std::set<T>`: Sıralı benzersiz elemanlar. Üyelik testi için ideal.
5. `std::deque<T>`: Çift uçlu kuyruk, her iki uçta hızlı push/pop.

THEORY: Template Specialization

Sometimes a generic template doesn't work well for a specific type. For example, comparing C-strings with `>` compares pointer addresses, not content! You can provide a "Template Specialization" — a custom version for a specific type. The compiler will prefer the specialized version when that type is used. This is explicit (full) specialization. There's also "Partial Specialization" for class templates where you specialize some parameters.

TEORİ: Şablon Özelleştirme

Bazen jenerik bir şablon belirli bir tip için düzgün çalışmaz. Örneğin, C-string'leri `>` ile karşılaştırmak pointer adreslerini karşılaştırır, içeriği değil! "Şablon Özelleştirme (Template Specialization)" ile belirli bir tipe özel versiyon yazabilirsin. Derleyici o tip kullanıldığında özel versiyonu tercih eder. Bu "tam (full) özelleştirme"dir. Sınıf şablonlarında bazı parametreleri özelleştiren "Kısmi Özelleştirme (Partial Specialization)" de vardır.

CPPREF Depth: Template Instantiation & Code Bloat

Templates are NOT real functions. They are blueprints. If you call `findMax` with 50 different data types `(int, float, char, SensorData, ECUConfig...)`, the compiler physically generates 50 DIFFERENT functions in the final `.exe` binary. This causes "Code Bloat" (Massive file size). Use them wisely! You can mitigate bloat with:
1. `extern template` declarations (C++11): Tell the compiler NOT to instantiate a template in this translation unit.
2. Thin template wrappers over non-template implementations.
3. Limiting the number of distinct type arguments.

CPPREF Derinlik: Şablon Somutlaştırma ve Kod Şişmesi

Şablonlar (Templates) gerçek fonksiyon DEĞİLDİR. Eğer siz `findMax` metodunu 50 farklı veri tipiyle çağırırsanız, derleyici arkaplanda 50 FARKLI fiziksel fonksiyon kopyası (Instantiations) oluşturur ve `.exe` dosyanızın boyutu devasa şişer! (Buna Code Bloat denir). Şişmeyi azaltmak için:
1. `extern template` bildirimleri (C++11): Derleyiciye bu çeviri biriminde şablonu somutlaştırma demeyi sağlar.
2. Şablon olmayan uygulamalar üzerine ince şablon sarmalayıcılar yazmak.
3. Farklı tip argüman sayısını sınırlamak.

Kaynak Kod / Source Code

```cpp
#include <algorithm> // EN: std::sort, std::find, std::transform
#include <cstring>   // EN: strcmp for C-string specialization
#include <iostream>
#include <map>
#include <numeric> // EN: std::accumulate
#include <string>
#include <vector>

// ─── 1. Function Template (Fonksiyon Şablonu) ──────────────────────────────────────

/**
 * @brief Generic Template Function — Jenerik Şablon Fonksiyonu
 *
 * EN: A template parameter `typename T` stands in for "any data type". The compiler deduces T
 * from the arguments automatically.
 * TR: T, "herhangi bir veri tipi" anlamına gelen kılıftır (Placeholder). Derleyici T'yi
 * argümanlardan otomatik çıkarır.
 */
template <typename T> T findMax(T a, T b) {
  return (a > b) ? a : b;
}

// ─── 2. Template Specialization (Şablon Özelleştirme) ──────────────────────────────

/**
 * @brief Specialization for C-strings — C-string İçin Özelleştirme
 *
 * EN: Without this, `findMax("apple", "zebra")` compares pointer addresses! This specialization
 * uses strcmp to compare actual string content.
 * TR: Bu olmazsa `findMax("apple", "zebra")` pointer adreslerini karşılaştırır! Bu özelleştirme
 * strcmp ile gerçek string içeriğini karşılaştırır.
 */
template <> const char *findMax<const char *>(const char *a, const char *b) {
  return (std::strcmp(a, b) > 0) ? a : b;
}
```

```cpp

// ─── 3. Class Template (Sınıf Şablonu) ──────────────────────────────────────

/**
 * @class Box
 * @brief Generic Class Template with Multiple Operations
 *
 * EN: A generic box that can hold any type. Demonstrates class templates with member functions,
 * operator overloading, and type-safe containers.
 * TR: Herhangi bir tipi tutabilen jenerik bir kutu. Sınıf şablonlarını üye fonksiyonlar,
 * operatör yükleme ve tip-güvenli konteynerlerle gösterir.
 */
template <typename DataType> class Box {
private:
  DataType content;

public:
  explicit Box(DataType initData) : content(initData) {}
  DataType peek() const { return content; }

  // EN: Comparison operator so we can compare Box objects
  // TR: Box nesnelerini karşılaştırabilmek için karşılaştırma operatörü
  bool operator>(const Box &other) const { return content > other.content; }

  // EN: Stream output operator (friend function)
  // TR: Akış çıktı operatörü (arkadaş fonksiyon)
  friend std::ostream &operator<<(std::ostream &os, const Box &b) {
    os << "Box[" << b.content << "]";
    return os;
  }
};

// ─── 4. Multi-Parameter Template (Çok Parametreli Şablon) ────────────────────

/**
 * @brief Template with multiple type parameters — Çoklu tip parametreli şablon
 *
 * EN: Templates can have more than one type parameter. This Pair class holds two values of
 * potentially different types, like std::pair.
 * TR: Şablonlar birden fazla tip parametresine sahip olabilir. Bu Pair sınıfı, std::pair gibi
 * farklı tipte iki değer tutar.
 */
template <typename First, typename Second> struct Pair {
  First first;
  Second second;

  Pair(First f, Second s) : first(f), second(s) {}

  void print() const {
    std::cout << "(" << first << ", " << second << ")";
  }
};

int main() {
  std::cout << "=== ADVANCED C++: TEMPLATES & STL DEEP DIVE ===\n"
            << std::endl;

  // ─── 1. Function Templates with Type Deduction ──────────────────────────
  std::cout << "--- 1. Function Templates (Sablon Fonksiyonlar) ---"
            << std::endl;

  // EN: Explicit type specification vs auto deduction
  // TR: Açık tip belirtme vs otomatik çıkarım
  std::cout << "Max of (10, 20): " << findMax<int>(10, 20)
            << "  [explicit <int>]" << std::endl;
  std::cout << "Max of (3.14, 2.71): " << findMax(3.14, 2.71)
            << "  [auto deduced double]" << std::endl;
  std::cout << "Max of strings: "
            << findMax(std::string("Apple"), std::string("Zebra"))
            << "  [auto deduced string]" << std::endl;

```

```
107     // EN: Template specialization for C-strings in action
108     // TR: C-string şablon özelleştirmesi iş başında
109     const char *result = findMax("apple", "zebra");
110     std::cout << "Max of C-strings: " << result
111               << "  [specialization with strcmp]" << std::endl;
112     std::cout << std::endl;
113
114     // ─── 2. Class Templates ──────────────────────────────────────────────────────
115     std::cout << "--- 2. Class Templates (Sablon Siniflar) ---" << std::endl;
116
117     Box<int> intBox(999);
118     Box<std::string> strBox("Secret Message / Gizli Mesaj");
119     Box<double> dblBox(3.14159);
120
121     std::cout << "Int box: " << intBox << std::endl;
122     std::cout << "String box: " << strBox << std::endl;
123     std::cout << "Double box: " << dblBox << std::endl;
124
125     // EN: Using findMax with Box objects (operator> is defined)
126     // TR: Box nesneleriyle findMax kullanımı (operator> tanımlı)
127     Box<int> boxA(42);
128     Box<int> boxB(99);
129     std::cout << "Max box: " << findMax(boxA, boxB) << std::endl;
130     std::cout << std::endl;
131
132     // ─── 3. Multi-Parameter Templates ────────────────────────────────────────────
133     std::cout << "--- 3. Multi-Parameter Pair (Coklu Tip Sablonu) ---"
134               << std::endl;
135
136     Pair<std::string, int> student("Ali", 95);
137     Pair<int, double> measurement(42, 98.6);
138     std::cout << "Student: ";
139     student.print();
140     std::cout << std::endl;
141     std::cout << "Measurement: ";
142     measurement.print();
143     std::cout << "\n" << std::endl;
144
145     // ─── 4. STL Container + Iterator + Algorithm Combo ───────────────────────────
146     std::cout << "--- 4. STL: Container + Iterator + Algorithm ---" << std::endl;
147
148     // EN: Vector container with algorithms
149     // TR: Vector konteyneri ve algoritmalar
150     std::vector<int> data = {42, 1, 99, 15, -3, 77, 23};
151     std::cout << "Original: ";
152     for (int v : data) {
153       std::cout << v << " ";
154     }
155     std::cout << std::endl;
156
157     // EN: std::sort uses iterators to work with ANY random-access container
158     // TR: std::sort iteratörleri kullanarak rastgele erişimli HER konteynerde çalışır
159     std::sort(data.begin(), data.end());
160     std::cout << "Sorted:   ";
161     for (int v : data) {
162       std::cout << v << " ";
163     }
164     std::cout << std::endl;
165
166     // EN: std::find returns an iterator to the found element or end()
167     // TR: std::find bulunan elemana iterator döndürür, yoksa end()
168     auto it = std::find(data.begin(), data.end(), 42);
169     if (it != data.end()) {
170       std::cout << "Found 42 at index: "
171                 << std::distance(data.begin(), it) << std::endl;
172     }
173
174     // EN: std::accumulate sums all elements (from <numeric>)
175     // TR: std::accumulate tüm elemanları toplar (<numeric>'ten)
176     int sum = std::accumulate(data.begin(), data.end(), 0);
177     std::cout << "Sum of all elements: " << sum << std::endl;
178
179     // EN: std::transform applies a function to each element
180     // TR: std::transform her elemana bir fonksiyon uygular
181     std::vector<int> doubled(data.size());
182     std::transform(data.begin(), data.end(), doubled.begin(),
183                    [](int x) { return x * 2; });
```

```
184     std::cout << "Doubled:  ";
185     for (int v : doubled) {
186       std::cout << v << " ";
187     }
188     std::cout << "\n" << std::endl;
189
190     // ─── 5. STL Map (Associative Container) ──────────────────────────────────────────
191     std::cout << "--- 5. STL Map (Iliskisel Konteyner) ---" << std::endl;
192
193     // EN: std::map stores sorted key-value pairs (Red-Black Tree)
194     // TR: std::map sıralı anahtar-değer çiftleri saklar (Kırmızı-Siyah Ağaç)
195     std::map<std::string, int> scores;
196     scores["Alice"] = 95;
197     scores["Bob"] = 87;
198     scores["Charlie"] = 92;
199     scores["Diana"] = 98;
200
201     // EN: Iterate over map (automatically sorted by key)
202     // TR: Map üzerinde döngü (anahtara göre otomatik sıralı)
203     std::cout << "Student scores (sorted by name):" << std::endl;
204     for (const auto &[name, score] : scores) {
205       std::cout << "  " << name << ": " << score << std::endl;
206     }
207
208     // EN: Find a specific key
209     // TR: Belirli bir anahtarı bul
210     auto mapIt = scores.find("Bob");
211     if (mapIt != scores.end()) {
212       std::cout << "Bob's score: " << mapIt->second << std::endl;
213     }
214     std::cout << std::endl;
215
216     // ─── 6. STL Algorithm Chaining (Algoritma Zincirleme) ────────────────────────────
217     std::cout << "--- 6. Algorithm Chaining (Algoritma Zincirleme) ---"
218               << std::endl;
219
220     // EN: Count elements greater than 20 using std::count_if with a lambda
221     // TR: Lambda ile 20'den büyük elemanları say (std::count_if)
222     std::vector<int> values = {5, 12, 35, 8, 42, 99, 3, 17, 50};
223     auto countAbove20 = std::count_if(values.begin(), values.end(),
224                                       [](int x) { return x > 20; });
225     std::cout << "Elements > 20: " << countAbove20 << std::endl;
226
227     // EN: Partition — move elements satisfying a predicate to the front
228     // TR: Partition — koşulu sağlayan elemanları öne taşı
229     std::partition(values.begin(), values.end(),
230                    [](int x) { return x % 2 == 0; });
231     std::cout << "After partition (evens first): ";
232     for (int v : values) {
233       std::cout << v << " ";
234     }
235     std::cout << std::endl;
236
237     // EN: Find min and max in one pass
238     // TR: Tek geçişte min ve max bul
239     auto [minIt, maxIt] = std::minmax_element(values.begin(), values.end());
240     std::cout << "Min: " << *minIt << ", Max: " << *maxIt << std::endl;
241
242     return 0;
243   }
```

► 02. Functors Callbacks

module_04_advanced/02_functors_callbacks.cpp

Konu: Advanced STL: Functors and Callbacks — İleri STL: Obje Fonksiyonlar (Functors) ve Geri

Teori / Theory

THEORY: Why Are Standard Functions Insufficient? (What is a Functor?)

A regular function holds NO state (no memory of past calls). A "Functor" (Function Object) is a Class that acts like a function by overloading `operator()`. It can have internal variables (state), a Constructor, and can be passed to STL algorithms like `std::sort` or `std::for_each`.

TEORİ: Neden Sıradan Fonksiyonlar Yetersiz Kalır?

Alışageldiğimiz sıradan fonksiyonların "Hafızası" (State/Durum) yoktur. Kaç kere çağırıldığı, içine konan parametrelerin geçmişi vs silinir. "Functor" (Fonksiyon Objesi), sanki bir fonksiyonmuş gibi davranan ama aslında bir "Sınıf" (Class) olan yapılardır. Bunu `operator()` ezerek (overload) yapar. İçinde kendi "private" hafızasını tutabilir! Özel bir devrimdir.

THEORY: Callbacks and std::function

In HMI (Head-Up Display) or Networking, when a button is pressed, you don't know exactly which logic should run at compile time. You "donate" a function to that button. This is called a "Callback". Using `#include <functional>`, `std::function` acts as a universal vessel capable of storing ANY callable (Lambdas, Functors, raw functions) like a variable!

TEORİ: Callbacks ve std::function

Bir butona tıklandığında (UI programlama) HANGİ fonksiyonun çalışacağını önceden bilemezsiniz. O butona bir fonksiyon "bağışlarsınız". Buna "Callback" (Geri Çağırma) denir. `<functional>` kütüphanesi ile `std::function` kullanarak her türlü fonksiyonu (lambda, functor, normal fonk) bir C++ değişkeni gibi saklayabilirsiniz.

CPPREF Depth: `std::function` CPU Penalty & Type Erasure (Tip Silinmesi)

`std::function` achieves this magic using "Type Erasure". However, if the lambda/functor you store inside it is too large (usually > 16-32 bytes), it FAILS the "Small Object Optimization (SOO)" and silently calls `new` to allocate heap memory! In a rendering loop (60 FPS), this silent allocation can murder system performance!

CPPREF Derinlik: `std::function` CPU Cezası ve Tip Silinmesi

`std::function` bu esnekliği "Type Erasure (Tip Silinmesi)" ile yapar. Fakat içine koyduğunuz Lambda objesi çok büyükse (Örneğin bir sürü şeyi [=] ile hapsettiyse), "Küçük Nesne Optimizasyonunu (SOO)" aşar ve o okyanus kadar objeyi saklamak için arkaplanda gizlice `new` ile Heap(RAM) ayırır! Saniyede 60 kere çalışan bir ECU kontrol döngüsünde `std::function` yüzünden gizlice RAM alınıp silinmesi performansı FELÇ eder!

Kaynak Kod / Source Code

```cpp
#include <algorithm>
#include <functional> // EN: std::function
#include <iostream>
#include <vector>

/**
 * @class DeductorFunctor
 * @brief EN: A Stateful Functor evaluating ongoing calculations.
 * TR: Özel bir durum (State) tutan Obje Fonksiyon (Functor). Her çağırılışta kendisinden önceki
 * bakiyeyi HATIRLAR (Stateful)!
 */
class DeductorFunctor {
private:
  // EN: STATE (Memory of past calls)
  // TR: STATE (Durum - Fonksiyon Hafızası)
  int balance;
public:
  DeductorFunctor(int initialBalance) : balance(initialBalance) {}

  // EN: Overloading the parenthesis () operator! Now this CLASS can be called like a FUNCTION!
  // TR: Parantez () operatörü aşırı yüklendi! (Overloaded). Artık bu SINIF, bir FONKSİYON gibi
  // `banka(10);` şeklinde ÇAĞRILABİLİR!
  void operator()(int amount) {
    balance -= amount;
    std::cout << amount << " spent (harcandı). Remaining Balance (Kalan bakiye): " << balance <<
        std::endl;
  }
};

/**
 * @brief EN: GUI or Networking Simulation. "Give me a function to execute later!"
 * TR: GUI veya Ağ simülasyonu. "Bana çalışması bitince bir fonksiyon ver, çağıracağım!"
 * @param callback Çalıştırılacak olay. `std::function` sayesinde Lambda, Functor fark etmez!
 */
void executeEvent(std::function<void(int)> callback) {
  std::cout << "\n[SYSTEM] Event triggered! Starting Callback... (Olay "
                "tetiklendi! Geri çağırma başlıyor...)"
            << std::endl;
  // EN: Running the injected function internally.
  // TR: Bize verilen fonksiyonu kendi içimizde çalıştırdık.
  callback(50);
}

int main() {
  std::cout << "=== MODULE 4: FUNCTORS & CALLBACKS (std::function) ===\n" << std::endl;

  // 1. Functor Test (Stateful Function Magic / Hafızalı Fonksiyon Büyüsü)
  std::cout << "--- 1. FUNCTORS (Stateful Functions) ---" << std::endl;
  DeductorFunctor bankSystem(100);

  std::cout << ">> Used exactly like a function, but it's an Object! "
                "(Fonksiyon gibi çağrılıyor):"
            << std::endl;
  // EN: 1st Call: Balance is 80. (It remembers!)
  // TR: 1. Çağrı: Bakiye 80. (Hatırlıyor!)
  bankSystem( 20);
  // EN: 2nd Call: Balance is 65.
  // TR: 2. Çağrı: Bakiye 65.
  bankSystem(15);

  // 2. Callbacks Test
  std::cout << "\n--- 2. CALLBACKS (std::function) ---" << std::endl;

  // EN: Lambda Carrier
  // TR: Sıradan fonk yerine LAMBDA taşıyıcısı (std::function) kullanıyoruz.
  std::function<void(int)> myLambda = [](int x) {
    std::cout << "Lambda executed! Given value (Lambda çalıştı, değer): " << x << std::endl;
  };

```

```
    executeEvent(myLambda);

    // EN: Callback via Functor
    // TR: Functor ile callback (Sınıfı veriyoruz, operator() sayesinde çağırılıyor)
    executeEvent(bankSystem);

    return 0;
}
```

► 03. Stl Containers Deepdive

module_04_advanced/03_stl_containers_deepdive.cpp

Konu: Advanced STL: Container Architectures & Big-O — İleri STL: Konteyner Mimarileri ve

Teori / Theory

THEORY: When to Use Which Container? (Software Engineering Interviews)

Choosing the wrong STL container can make a program 10,000x slower.
- std::vector : Fast random access [O(1)]. Slow to insert at the front [O(N)]. Under the hood, it's just a dynamic array. Best default choice because of CPU CACHE.
- std::list : Doubly Linked List. Fast to insert anywhere [O(1)]. TERRIBLE at searching or random access [O(N)]. Nodes are scattered in RAM.
- std::map / unordered_map : Key-Value stores. map uses a Red-Black Tree [O(logN)]. unordered_map uses a Hash Table [O(1)].

TEORİ: Hangi Konteyner Ne Zaman?

Yazılım Mimarisi Mülakatlarında asıl soru "Neyi yazdığın" değil, "Veriyi nereye koyduğun"dur.
- std::vector : %90 ihtimalle doğru cevaptır. RAM'de yan yana (Contiguous) durur. CPU Cache (Önbellek) dostudur. Okuması roket hızındadır O(1). Ama dizinin başına eleman İTERSENİZ tüm dizi kayacağı için çok yavaştır O(N).
- std::list : Bağlı Listelerdir (Linked List). Elemanlar RAM'de darmadağın durur. Araya eleman sokuşturmak harikadır O(1). Ama 50. elemanı bulmak O(N)'dir.
- std::map vs std::unordered_map : Sözlük yapıları (Key-Value/Anahtar-Değer). map: Veriyi sürekli alfabetik/sayısal sıralayarak (Kırmızı-Siyah Ağaç) tutar O(logN). unordered_map: Sırasız, Hash (Şifreleme) tablosudur. Rokettir O(1).

THEORY: Vector Capacity vs Size Confusion

`size` is the actual elements inside (e.g., 5 elements). `capacity` is the pre-allocated RAM hardware given by OS (e.g., 8 spaces). If capacity fills up, Vector doubles it `(8*2 = 16) ` and physically MOVES to a new RAM address (Reallocation). This is costly! Solution: `reserve()`.

TEORİ: Kapasite vs Boyut Kargaşası

Vector'ün Boyutu (Size) içindeki gerçek eleman sayısıdır. Capacity (Kapasite) ise işletim sisteminden çekrilmiş RAM potansiyelidir. Eğer Kapasite dolar ve siz itmeye devam ederseniz, Vector kendi kapasitesini 2 KATINA ÇIKARARAK yep yeni bir fiziksel RAM adresine kopyalanır ve Taşınır (Reallocation). Bu korkunç yavaştır! Çözüm: `reserve()` fonksiyonu ile baştan yer ayırtmak.

CPPREF Depth: The Dreaded Iterator Invalidation HACK!

What happens if you have a pointer (Iterator) looking at `vector[5]`, but you push a new element and the vector reallocates to a new RAM address? CppReference says: ALL ITERATORS ARE INVALIDATED! Your pointer is now looking at dead memory (Garbage). Same happens if you `erase()` from the middle of a Vector. ALWAYS check cppreference for the "Iterator invalidation" table before modifying a container while looping over it!

CPPREF Derinlik: Korkunç İteratör Geçersizleşmesi (İteratör Çöküşü)!

Ya eğer elinizde `vector`'ün 5. elemanını gösteren bir pointer/İteratör varsa, ve siz yeni eleman ekleyip vector'ün kapasitesini doldurduğunuz için vector komple YENİ BİR RAM ADRESİNE taşınırsa? CppReference açıkça yazar: "TÜM İTERATÖRLER GEÇERSİZ KALIR (Invalidated)". Sizin o
5. elemana bakan pointerınız çöpte sallanıp (Dangling) oyunu çökertir (SIGSEGV). Bir STL kabı içinde döngü yaparken eleman siliyorsanız, daima CppRef Invalidation tablosuna bakmalısınız!

Kaynak Kod / Source Code

```cpp
#include <algorithm>
#include <array>
#include <deque>
#include <iostream>
#include <list>
#include <map>
#include <memory_resource>
#include <numeric>
#include <tuple>
#include <unordered_map>
#include <vector>

int main() {
  std::cout << "=== MODULE 4: STL CONTAINERS DEEP DIVE ===\n" << std::endl;

  // --- 1. VECTOR CAPACITY & REALLOCATION COST (Vektor Kapasite Çilesi) ---
  std::cout << "--- 1. std::vector Capacity & Reallocation ---" << std::endl;
  std::vector<int> numbers;

  // EN: Cost of not using reserve():
  // TR: reserve() kullanmamanın "Yeniden Taşınma (Realloc)" maliyeti:
  int migrationCount = 0;
  size_t oldCapacity = numbers.capacity();

  for (int i = 0; i < 20; i++) {
    numbers.push_back(i);
    // EN: If capacity changes, it means the entire vector moved to a new memory block!
    // TR: Eğer kapasite değişirse, tüm vector donanımsal olarak yeni bir RAM alanına fırlatıldı
    // demektir!
    if (numbers.capacity() != oldCapacity) {
      std::cout << "Reallocation (Veri Taşındı)! New Capacity (Yeni Kapasite): " <<
          numbers.capacity() << std::endl;
      oldCapacity = numbers.capacity();
      migrationCount++;
    }
  }

  std::cout << "For just 20 elements, Vector physically moved in RAM " << migrationCount <<
      " times!" << std::endl;
  std::cout << "> SOLUTION (ÇÖZÜM): `numbers.reserve(20);` to prepare RAM at "
                "the start.\n"
            << std::endl;

  // --- 2. MAP (Tree) vs UNORDERED_MAP (Hash Table) ---
  std::cout << "--- 2. Dictionary (Key-Value) Selection ---" << std::endl;

```

```cpp
    // EN: Backed by Red-Black Tree. Maintains order. O(logN).
    // TR: Arkasında Kırmızı-Siyah Ağaç yatar. Sürekli veriyi sıralı tutar. O(logN).
    std::map<std::string, int> orderedDictionary;
    orderedDictionary["Zebra"] = 1;
    orderedDictionary["Bear"] = 2;
    orderedDictionary["Cat"] = 3;

    std::cout << "std::map (Ordered Tree / Sıralı Ağaç - O(logN) ) :" << std::endl;
    for (auto const &[key, val] : orderedDictionary) {
      // EN: Output will AUTOMATICALLY be Alphabetical! (Bear, Cat, Zebra)
      // TR: Çıktı OTOMATİK olarak Alfabetik sıralıdır! (Bear, Cat, Zebra)
      std::cout << key << ":" << val << " ";
    }

    // EN: Backed by Hash Table. Unpredictable order. O(1) Speed!
    // TR: Arkasında Hash C-Tabloları yatar. Sıralama garantisi YOKTUR. O(1) Roket Hızı!
    std::cout << "\n\nstd::unordered_map (Hash Table / Sırasız - O(1) ) :" << std::endl;
    std::unordered_map<std::string, int> hashDictionary;
    hashDictionary["Zebra"] = 1;
    hashDictionary["Bear"] = 2;
    hashDictionary["Cat"] = 3;

    for (auto const &[key, val] : hashDictionary) {
      // EN: Output order is NOT guaranteed. It only focuses on speed.
      // TR: Çıktı sıralama garantisi VERMEZ. Sadece hıza odaklanır.
      std::cout << key << ":" << val << " ";
    }
    std::cout << "\n" << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: Demo 3 — std::array: Fixed-Size Wheel Speed Sensors
    // TR: Demo 3 — std::array: Sabit Boyutlu Tekerlek Hız Sensörleri
    // ═══════════════════════════════════════════════════════════════════════════════
    std::cout << "--- 3. std::array — Fixed-Size Wheel Speed Sensors ---" << std::endl;

    // EN: std::array<T, N> lives entirely on the stack — zero heap allocation.
    // TR: std::array<T, N> tamamen stack üzerinde yaşar — heap tahsisi sıfırdır.
    std::array<double, 4> wheelSpeedKmh = {82.5, 81.9, 83.1, 80.7};

    // ── 3a. Compile-Time Size ──────────────────────────────────────────────────
    // EN: Size is known at compile time — no runtime overhead.
    // TR: Boyut derleme zamanında bilinir — çalışma zamanı ek maliyeti yoktur.
    std::cout << "Wheel count (compile-time): " << wheelSpeedKmh.size() << std::endl;
    static_assert(wheelSpeedKmh.size() == 4, "ECU expects exactly 4 wheel sensors");

    // ── 3b. Bounds Checking with .at() ─────────────────────────────────────────
    // EN: .at() throws std::out_of_range if index is invalid — safer than [].
    // TR: .at() geçersiz indekste std::out_of_range fırlatır — []'den güvenlidir.
    std::cout << "Front-Left  (FL) speed: " << wheelSpeedKmh.at(0) << " km/h" << std::endl;
    std::cout << "Front-Right (FR) speed: " << wheelSpeedKmh.at(1) << " km/h" << std::endl;
    std::cout << "Rear-Left   (RL) speed: " << wheelSpeedKmh.at(2) << " km/h" << std::endl;
    std::cout << "Rear-Right  (RR) speed: " << wheelSpeedKmh.at(3) << " km/h" << std::endl;

    // ── 3c. STL Algorithm Compatibility ────────────────────────────────────────
    // EN: std::array is fully compatible with STL algorithms (sort, find, etc.).
    // TR: std::array, STL algoritmalarıyla (sort, find vb.) tam uyumludur.
    std::sort(wheelSpeedKmh.begin(), wheelSpeedKmh.end());
    std::cout << "Sorted speeds (ascending): ";
    for (const auto &spd : wheelSpeedKmh) {
      std::cout << spd << " ";
    }
    std::cout << std::endl;

    auto it = std::find(wheelSpeedKmh.begin(), wheelSpeedKmh.end(), 83.1);
    if (it != wheelSpeedKmh.end()) {
      std::cout << "Found 83.1 km/h at index: "
                << std::distance(wheelSpeedKmh.begin(), it) << std::endl;
    }
    std::cout << std::endl;
```

```cpp

    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: Demo 4 — std::tuple: Heterogeneous ECU Diagnostic Data
    // TR: Demo 4 — std::tuple: Heterojen ECU Arıza Teşhis Verisi
    // ═══════════════════════════════════════════════════════════════════════════════
    std::cout << "--- 4. std::tuple — ECU Diagnostic Packet ---" << std::endl;

    // EN: Tuple groups different types together — perfect for multi-field ECU frames.
    // TR: Tuple farklı tipleri bir arada gruplar — çoklu alan içeren ECU çerçeveleri için
    // idealdir.
    std::tuple<int, std::string, double> ecuDiag(0x1A3F, "Throttle Sensor Drift", 4.87);

    // ─── 4a. Access with std::get<> ──────────────────────────────────────────────
    // EN: std::get<Index> retrieves the element at compile-time index.
    // TR: std::get<Index> derleme zamanı indeksiyle elemana erişir.
    std::cout << "Error Code : 0x" << std::hex << std::get<0>(ecuDiag) << std::dec << std::endl;
    std::cout << "Description: " << std::get<1>(ecuDiag) << std::endl;
    std::cout << "Voltage    : " << std::get<2>(ecuDiag) << " V" << std::endl;

    // ─── 4b. Unpacking with std::tie ─────────────────────────────────────────────
    // EN: std::tie binds tuple elements to existing variables by reference.
    // TR: std::tie, tuple elemanlarını mevcut değişkenlere referansla bağlar.
    int errCode{};
    std::string errDesc;
    double voltage{};
    std::tie(errCode, errDesc, voltage) = ecuDiag;
    std::cout << "[tie] Code=0x" << std::hex << errCode << std::dec
              << "  Desc=" << errDesc << "  V=" << voltage << std::endl;

    // ─── 4c. Structured Bindings (C++17) ─────────────────────────────────────────
    // EN: Structured bindings provide the cleanest syntax — one-liner unpacking.
    // TR: Yapısal bağlamalar en temiz sözdizimini sunar — tek satırda açma.
    auto [code, desc, volt] = ecuDiag;
    std::cout << "[structured binding] Code=0x" << std::hex << code << std::dec
              << "  Desc=" << desc << "  V=" << volt << "\n" << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: Demo 5 — Container Selection Guide (Automotive Context)
    // TR: Demo 5 — Konteyner Seçim Rehberi (Otomotiv Bağlamı)
    // ═══════════════════════════════════════════════════════════════════════════════
    std::cout << "--- 5. Container Selection Guide ---" << std::endl;
    std::cout << R"(
    +-------------------+------------------+-------------------------------------+
    | Container         | Complexity       | Automotive Use Case                 |
    +-------------------+------------------+-------------------------------------+
    | std::vector       | O(1) access      | Sensor stream buffers, CAN frames   |
    | std::array        | O(1), stack only | Fixed wheel/axle sensor arrays      |
    | std::map          | O(logN) ordered  | DTC code registry (sorted by ID)    |
    | std::unordered_map| O(1) hash        | Real-time signal lookup by name     |
    | std::tuple        | compile-time     | Grouping heterogeneous ECU fields   |
    +-------------------+------------------+-------------------------------------+
  )" << std::endl;

    // ─── Note: std::span (C++20) ─────────────────────────────────────────────────
    // EN: std::span<T> is a non-owning, lightweight view over contiguous memory (e.g., a raw
    // array, std::vector, or std::array). It does NOT copy data. Think of it as a "window" into an
    // existing buffer — ideal for passing sensor data slices to processing functions without
    // ownership transfer. Requires C++20: compile with -std=c++20.
    // TR: std::span<T>, bitişik bellekteki veriye sahiplenmeyen hafif bir görünümdür (raw dizi,
    // std::vector veya std::array üzerinde). Veriyi KOPYALAMAZ. Mevcut bir tampona açılan bir
    // "pencere" gibi düşünün — sensör veri dilimlerini sahiplik devretmeden işleme fonksiyonlarına
    // geçirmek için idealdir. C++20 gerektirir: -std=c++20 ile derleyin.

    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: Demo 6 — std::deque: Double-Ended Queue Internals
    // TR: Demo 6 — std::deque: Çift Uçlu Kuyruk İç Yapısı
    // ═══════════════════════════════════════════════════════════════════════════════
    std::cout << "--- 6. std::deque: Double-Ended Queue ---" << std::endl;
    {
      // EN: deque stores data in CHUNKS (blocks). Not contiguous like vector.
      //     O(1) push_front AND push_back (vector has O(n) push_front).
      //     Iterators invalidated only on insert/erase in the middle.
      // TR: deque veriyi PARÇALAR (bloklar) halinde saklar. vector gibi bitişik değil.
      //     O(1) push_front VE push_back (vector'ün push_front'u O(n)).
      std::deque<std::string> canBus;

```

```
192       // EN: Efficient at both ends — ideal for sliding window / FIFO buffer
193       // TR: Her iki uçta verimli — kayan pencere / FIFO tampon için ideal
194       canBus.push_back("FRAME_0x100");
195       canBus.push_back("FRAME_0x101");
196       canBus.push_back("FRAME_0x102");
197       canBus.push_front("FRAME_0x0FF");  // O(1) — vector can't do this efficiently
198
199       std::cout << "  Front: " << canBus.front()
200                 << ", Back: " << canBus.back()
201                 << ", Size: " << canBus.size() << std::endl;
202
203       canBus.pop_front();  // O(1) dequeue
204       std::cout << "  After pop_front, Front: " << canBus.front() << std::endl;
205
206       // EN: Random access — same as vector, O(1)
207       // TR: Rastgele erişim — vector ile aynı, O(1)
208       std::cout << "  canBus[1] = " << canBus[1] << std::endl;
209
210       // EN: Memory layout: array of pointers to fixed-size chunks
211       //     ┌──────────┐    ┌──────────┐
212       //     │ chunk 0 │────>│ A B C D  │
213       //     │ chunk 1 │────>│ E F G H  │
214       //     │ chunk 2 │────>│ I _ _ _  │
215       //     └──────────┘    └──────────┘
216       //     Unlike vector, no full reallocation on growth — just add a new chunk.
217       // TR: Bellek yerleşimi: sabit boyutlu parçalara işaretçi dizisi.
218       //     vector'den farklı olarak, büyümede tam yeniden ayırma yok.
219       std::cout << "  (deque: O(1) both ends, random access, chunk-based memory)\n" << std::endl;
220     }
221
222     // ═══════════════════════════════════════════════════════════════════════════════
223     // EN: Demo 7 — std::pmr: Polymorphic Memory Resources
224     // TR: Demo 7 — std::pmr: Polimorfik Bellek Kaynakları
225     // ═══════════════════════════════════════════════════════════════════════════════
226     std::cout << "--- 7. std::pmr: Polymorphic Memory Resources ---" << std::endl;
227     {
228       // EN: std::pmr containers use a runtime-configurable memory resource.
229       //     Use cases: arena allocators, stack buffers, embedded systems.
230       //     Key benefit: swap allocator at runtime without changing container type.
231       // TR: std::pmr konteynerleri çalışma zamanında yapılandırılabilir bellek kaynağı kullanır.
232       //     Kullanım alanları: arena ayırıcılar, yığın tamponları, gömülü sistemler.
233
234       // EN: Stack-based buffer — zero heap allocations for small data
235       // TR: Yığın tabanlı tampon — küçük veriler için sıfır heap ayırma
236       char stackBuffer[1024];
237       std::pmr::monotonic_buffer_resource pool{stackBuffer, sizeof(stackBuffer)};
238
239       std::pmr::vector<int> sensorReadings{&pool};
240       for (int i = 0; i < 20; ++i) {
241         sensorReadings.push_back(i * 10);
242       }
243       std::cout << "  pmr::vector with stack buffer: " << sensorReadings.size()
244                 << " elements, sum=" << std::accumulate(sensorReadings.begin(),
245                                                         sensorReadings.end(), 0)
246                 << std::endl;
247
248       // EN: pmr::string uses same allocator
249       // TR: pmr::string aynı bellek yöneticisini kullanır
250       std::pmr::string msg{"ECU_DIAGNOSTIC_OK", &pool};
251       std::cout << "  pmr::string: " << msg << " (length=" << msg.size() << ")" << std::endl;
252
253       // EN: Advantages over std::allocator:
254       //     1. No template parameter differences — pmr::vector<int> is always same type
255       //     2. Arena/pool allocation — super fast for many short-lived objects
256       //     3. No heap fragmentation — monotonic_buffer_resource just bumps a pointer
257       //     4. Embedded-friendly — can use static buffers
258       // TR: std::allocator'a göre avantajları:
259       //     1. Şablon parametresi farkı yok — pmr::vector<int> her zaman aynı tip
260       //     2. Arena/havuz ayırma — çok sayıda kısa ömürlü nesne için süper hızlı
261       //     3. Heap parçalanması yok
262       //     4. Gömülü dostu — statik tamponlar kullanabilir
263       std::cout << "  (pmr: runtime allocator, arena-friendly, zero-heap possible)\n" << std::endl;
264     }
```

```

  // ═══════════════════════════════════════════════════════════════════════════════
  // EN: Demo 8 — C++23 flat_map/flat_set Reference
  // TR: Demo 8 — C++23 flat_map/flat_set Referansı
  // ═══════════════════════════════════════════════════════════════════════════════
  std::cout << "--- 8. C++23 flat_map / flat_set Reference ---" << std::endl;
  {
    // EN: std::flat_map and std::flat_set (C++23, <flat_map> / <flat_set>) store
    //     elements in sorted CONTIGUOUS memory (vector-backed). Benefits:
    //     - Cache-friendly iteration (unlike std::map's tree nodes)
    //     - Lower memory overhead (no node pointers)
    //     - Better for read-heavy, small-to-medium sized maps
    //
    //     #include <flat_map>
    //     std::flat_map<int, std::string> dtc_codes;
    //     dtc_codes[0x100] = "Engine Overheat";
    //     dtc_codes[0x200] = "Low Oil Pressure";
    //     // Iteration is FAST — contiguous memory, cache-line friendly
    //     for (const auto& [code, desc] : dtc_codes) { ... }
    //
    //      ┌──────────────┬──────────────┬──────────────┬──────────────────┐
    //      │ Container    │ Insert O()   │ Lookup O()   │ Iteration        │
    //      ├──────────────┼──────────────┼──────────────┼──────────────────┤
    //      │ std::map     │ O(log N)     │ O(log N)     │ Slow (tree nodes)│
    //      │ flat_map     │ O(N) amort.  │ O(log N)     │ FAST (contiguous)│
    //      │ unordered_map│ O(1) amort.  │ O(1) amort.  │ Unordered        │
    //      └──────────────┴──────────────┴──────────────┴──────────────────┘
    //
    // TR: std::flat_map ve std::flat_set (C++23) elemanları sıralı BİTİŞİK
    //     bellekte (vector-destekli) saklar. Önbellek dostu iterasyon.

    // EN: Simulate flat_map behavior with sorted vector of pairs
    // TR: flat_map davranışını sıralı çift vektörüyle simüle et
    std::vector<std::pair<int, std::string>> flatDtc = {
        {0x100, "Engine Overheat"},
        {0x150, "Coolant Low"},
        {0x200, "Low Oil Pressure"},
        {0x300, "ABS Fault"}
    };
    std::sort(flatDtc.begin(), flatDtc.end());

    // EN: Binary search — O(log N), same as flat_map::find
    // TR: İkili arama — O(log N), flat_map::find ile aynı
    auto found = std::lower_bound(flatDtc.begin(), flatDtc.end(),
                                  std::make_pair(0x200, std::string{}),
                                  [](const auto& a, const auto& b) {
                                      return a.first < b.first;
                                  });
    if (found != flatDtc.end() && found->first == 0x200) {
        std::cout << "  Found 0x200: " << found->second << std::endl;
    }

    std::cout << "  Flat iteration (contiguous): ";
    for (const auto& [dtcCode, dtcDesc] : flatDtc) {
        (void)dtcDesc;
        std::cout << "0x" << std::hex << dtcCode << std::dec << " ";
    }
    std::cout << "\n  (C++23 std::flat_map — use sorted vector until compiler supports it)\n"
              << std::endl;
  }

  return 0;
}
```

► 04. Iterators And Adaptors

module_04_advanced/04_iterators_and_adaptors.cpp

Konu: Templates & STL: Iterators, Categories and Adaptors — Şablonlar ve STL: İteratörler,

Teori / Theory

THEORY: Iterator Categories — The 5 Ranks of STL Access

1. Input Iterator Read-only, single-pass, forward-only. Like reading sensor data from a CAN bus — once you read a frame, it's gone. You can only move forward and read each value exactly once. Supports: ++it, *it (read), it == it2, it != it2
2. Output Iterator Write-only, single-pass, forward-only. Like writing commands to an actuator — you push data out, you can't read it back. Supports: ++it, *it = val (write)
3. Forward Iterator Read+Write, multi-pass, forward-only. Like iterating over a linked list of ECU diagnostic trouble codes (DTCs). Supports: Everything Input + multi-pass guarantee
4. Bidirectional Iterator Forward + backward traversal. Like scrolling through a vehicle's event log — you can go forward and backward through entries. Supports: Everything Forward + --it Used by: std::list, std::set, std::map
5. Random Access Iterator Full pointer arithmetic. Like accessing any sensor value in an array by index — instant O(1) jump to any position. Supports: Everything Bidirectional + it+n, it-n, it[n], it<it2 Used by: std::vector, std::deque, std::array, raw pointers

TEORİ: İteratör Kategorileri — STL Erişiminin 5 Kademesi

1. Giriş İteratörü Salt okunur, tek geçişli, sadece ileri. CAN bus'tan sensör verisi okumak gibi — bir kare okunduktan sonra gider. Sadece ileri gidip her değeri tam olarak bir kez okuyabilirsiniz. Destekler: ++it, *it (okuma), it == it2, it != it2
2. Çıkış İteratörü Salt yazılır, tek geçişli, sadece ileri. Bir aktüatöre komut yazmak gibi — veri gönderirsiniz, geri okuyamazsınız. Destekler: ++it, *it = val (yazma)
3. İleri İteratörü Okuma+Yazma, çok geçişli, sadece ileri. ECU arıza kodları (DTC) bağlı listesinde gezinmek gibi. Destekler: Input'un tamamı + çok geçiş garantisi
4. Çift Yönlü İteratör İleri + geri gezinme. Araç olay günlüğünde gezinmek gibi — ileri ve geri gidebilirsiniz. Destekler: Forward'ın tamamı + --it Kullanan: std::list, std::set, std::map
5. Rastgele Erişim İteratörü Tam pointer aritmetiği. Bir dizideki herhangi bir sensör değerine indeksle erişmek gibi — herhangi bir konuma anında O(1) atlama. Destekler: Bidirectional'ın tamamı + it+n, it-n, it[n], it<it2 Kullanan: std::vector, std::deque, std::array, ham pointerlar

HIERARCHY

Random Access ⊃ Bidirectional ⊃ Forward ⊃ Input ⊃ Output Each higher category includes all capabilities of those below it. std::sort → Requires Random Access std::find → Input is sufficient std::reverse → Requires Bidirectional

HİYERARŞİ

Random Access ⊃ Bidirectional ⊃ Forward ⊃ Input ⊃ Output Her üst kategori, altındakilerin tüm yeteneklerini içerir. std::sort → Random Access gerektirir std::find → Input yeterlidir std::reverse → Bidirectional gerektirir

CPPREF Depth: Iterator Categories and Their Guarantees

The C++ standard defines 5 iterator categories, each a superset of the previous:
1. InputIterator — single-pass read (==, !=, *, ++)
2. OutputIterator — single-pass write (*it = val, ++)
3. ForwardIterator — multi-pass read/write (can revisit elements)
4. BidirectionalIterator — adds backward traversal (--)
5. RandomAccessIterator — O(1) jump: it+n, it-n, it[n], <, > `std::distance(first, last)` is O(1) for RandomAccess iterators (pointer subtraction) but O(n) for all others (incremental ++). `std::advance(it, n)` similarly dispatches on category. Iterator invalidation rules vary critically by container:
• vector: insert/push_back may invalidate ALL iterators (realloc).
• deque: insert at front/back invalidates iterators but not refs.
• list/set/map: insert NEVER invalidates existing iterators. C++20 introduces `std::contiguous_iterator` — guarantees elements are adjacent in memory (e.g., vector, array, string).

CPPREF Derinlik: İteratör Kategorileri ve Garantileri

C++ standardı 5 iteratör kategorisi tanımlar, her biri öncekinin üst kümesidir:
1. InputIterator — tek geçişli okuma (==, !=, *, ++)
2. OutputIterator — tek geçişli yazma (*it = val, ++)
3. ForwardIterator — çok geçişli okuma/yazma (tekrar ziyaret edilebilir)
4. BidirectionalIterator — geri gidiş eklenir (--)
5. RandomAccessIterator — O(1) atlama: it+n, it-n, it[n], <, > `std::distance(first, last)` RandomAccess için O(1)'dir (işaretçi çıkarma), diğerleri için O(n)'dir (sıralı ++). İteratör geçersizleşme kuralları konteynere göre değişir:
• vector: insert/push_back TÜM iteratörleri geçersiz kılabilir (realloc).
• deque: baş/sona ekleme iteratörleri geçersiz kılar ama referansları değil.
• list/set/map: ekleme mevcut iteratörleri ASLA geçersiz kılmaz. C++20 `std::contiguous_iterator` ekler — elemanların bellekte bitişik olduğunu garanti eder (ör. vector, array, string).

Kaynak Kod / Source Code

```cpp
#include <algorithm>    // EN: STL algorithms
#include <deque>        // EN: Double-ended queue
#include <forward_list> // EN: Singly-linked list
#include <iostream>     // EN: Standard I/O
#include <iterator>     // EN: Iterator adaptors
#include <list>         // EN: Doubly-linked list
#include <numeric>      // EN: std::iota, std::accumulate
#include <set>          // EN: Ordered set
#include <sstream>      // EN: String stream
#include <string>       // EN: std::string
#include <vector>       // EN: Dynamic array

// ═══════════════════════════════════════════════════════════════════════════
// [PRODUCTION CODE / ÜRETİM KODU]
// ═══════════════════════════════════════════════════════════════════════════

// ─── 1. Demonstrating Iterator Categories ──────────────────────────────

// EN: A function that REQUIRES Random Access Iterators (like std::sort).
// TR: Random Access İteratör GEREKTİREN bir fonksiyon (std::sort gibi).
template <typename RandomIt>
void printSorted(RandomIt first, RandomIt last) {
    std::vector<typename std::iterator_traits<RandomIt>::value_type> temp(first, last);
    std::sort(temp.begin(), temp.end());
    for (const auto& val : temp) {
        std::cout << val << " ";
    }
    std::cout << "\n";
}
```

```cpp

// EN: A function that works with ANY Input Iterator (minimum requirement).
// TR: Herhangi bir Input İteratör ile çalışan fonksiyon (minimum gereksinim).
template <typename InputIt>
typename std::iterator_traits<InputIt>::value_type
sumRange(InputIt first, InputIt last) {
    typename std::iterator_traits<InputIt>::value_type total{};
    while (first != last) {
        total += *first;
        ++first;
    }
    return total;
}

// ─── 2. Custom Iterator — SensorRange ──────────────────────────────────────────

// EN: A custom forward iterator that generates sensor readings in a range. Simulates reading
// sequential sensor addresses (e.g., temperature registers 0x100, 0x104, 0x108 ... on an ECU
// memory map).
// TR: Belirli bir aralıkta sensör okumaları üreten özel forward iterator. Sıralı sensör
// adreslerini okumayı simüle eder (örn. ECU bellek haritasında 0x100, 0x104, 0x108 ... sıcaklık
// kaydedicileri).
class SensorRange {
   public:
    // EN: Forward Iterator implementation
    // TR: Forward İteratör uygulaması
    class Iterator {
       public:
        // EN: Required type aliases for std::iterator_traits
        // TR: std::iterator_traits için gerekli tip tanımları
        using iterator_category = std::forward_iterator_tag;
        using value_type = int;
        using difference_type = std::ptrdiff_t;
        using pointer = const int*;
        using reference = const int&;

        explicit Iterator(int val) : current_(val) {}

        reference operator*() const { return current_; }
        pointer operator->() const { return &current_; }

        Iterator& operator++() {
            // EN: 4-byte aligned sensor addresses
            // TR: 4-byte hizalı sensör adresleri
            current_ += 4;
            return *this;
        }

        Iterator operator++(int) {
            Iterator tmp = *this;
            ++(*this);
            return tmp;
        }

        bool operator==(const Iterator& other) const {
            return current_ >= other.current_;
        }

        bool operator!=(const Iterator& other) const {
            return !(*this == other);
        }

       private:
        int current_;
    };

    SensorRange(int start, int end) : start_(start), end_(end) {}

    Iterator begin() const { return Iterator(start_); }
    Iterator end() const { return Iterator(end_); }

   private:
    int start_;
    int end_;
};
```

```cpp

// ─── 3. Container Adaptor Demonstrations ──────────────────────────────────────

// EN: Collect diagnostic trouble codes (DTCs) using back_inserter. back_inserter automatically
// calls push_back on the target container.
// TR: Arıza kodlarını (DTC) back_inserter ile topla. back_inserter hedef konteynerde otomatik
// push_back çağırır.
std::vector<std::string> collectActiveDTCs(
    const std::vector<std::string>& allCodes,
    const std::string& prefix) {
    std::vector<std::string> result;
    std::copy_if(allCodes.begin(), allCodes.end(),
                 std::back_inserter(result),
                 [&prefix](const std::string& code) {
                     return code.substr(0, prefix.size()) == prefix;
                 });
    return result;
}

// ═══════════════════════════════════════════════════════════════════════════════
// [EXAMPLES / ÖRNEKLER]
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "=== MODULE 4: ITERATORS & ADAPTORS ===\n"
              << "=== MODÜL 4: İTERATÖRLER VE ADAPTÖRLER ===\n\n";

    // ─── 1. Iterator Categories in Action ─────────────────────────────────────
    {
        std::cout << "--- 1. Iterator Categories ---\n";

        std::vector<int> speeds = {120, 85, 200, 60, 150};

        // EN: vector iterators are Random Access — supports std::sort
        // TR: vector iteratörleri Random Access — std::sort destekler
        std::cout << "Sorted speeds: ";
        printSorted(speeds.begin(), speeds.end());

        // EN: sumRange works with ANY iterator (Input is enough)
        // TR: sumRange herhangi bir iteratör ile çalışır (Input yeterli)
        std::cout << "Sum of speeds: " << sumRange(speeds.begin(), speeds.end()) << "\n";

        // EN: list iterators are Bidirectional — can reverse but NOT random access
        // TR: list iteratörleri Bidirectional — ters gidebilir ama rastgele erişim YOK
        std::list<int> rpms = {3000, 1500, 4500, 2200};
        // EN: list has its own sort (merge sort) — doesn't need Random Access
        // TR: list'in kendi sıralaması var (merge sort) — Random Access gerektirmez
        rpms.sort();
        std::cout << "Sorted RPMs (list::sort): ";
        for (const auto& r : rpms) std::cout << r << " ";
        std::cout << "\n";

        // EN: forward_list is Forward only — single direction, no size()
        // TR: forward_list sadece Forward — tek yön, size() yok
        std::forward_list<int> temps = {72, 88, 65, 91};
        temps.sort();
        std::cout << "Sorted temps (forward_list): ";
        for (const auto& t : temps) std::cout << t << " ";
        std::cout << "\n\n";
    }

    // ─── 2. Custom Iterator — SensorRange ─────────────────────────────────────
    {
        std::cout << "--- 2. Custom SensorRange Iterator ---\n";

        // EN: Iterate over sensor addresses 0x100 to 0x120 (4-byte steps)
        // TR: 0x100'den 0x120'ye sensör adresleri üzerinde gezin (4-byte adım)
        SensorRange sensors(0x100, 0x120);

        std::cout << "Sensor addresses: ";
        for (int addr : sensors) {
            std::cout << "0x" << std::hex << addr << " ";
        }
        std::cout << std::dec << "\n";
```

```cpp

        // EN: Works with STL algorithms because it satisfies ForwardIterator
        // TR: ForwardIterator'ı karşıladığı için STL algoritmaları ile çalışır
        std::cout << "Count of addresses: "
                  << std::distance(sensors.begin(), sensors.end()) << "\n\n";
    }

    // ─── 3. back_inserter — Dynamic Collection ──────────────────────────────────
    {
        std::cout << "--- 3. back_inserter ---\n";

        std::vector<int> source = {10, 20, 30, 40, 50};
        std::vector<int> dest;

        // EN: back_inserter creates an Output Iterator that calls push_back
        // TR: back_inserter push_back çağıran bir Output İteratör oluşturur
        std::copy(source.begin(), source.end(), std::back_inserter(dest));
        std::cout << "Copied via back_inserter: ";
        for (int v : dest) std::cout << v << " ";
        std::cout << "\n";

        // EN: Transform + back_inserter — double each value into a new vector
        // TR: Transform + back_inserter — her değeri iki katına çıkar
        std::vector<int> doubled;
        std::transform(source.begin(), source.end(),
                       std::back_inserter(doubled),
                       [](int x) { return x * 2; });
        std::cout << "Doubled: ";
        for (int v : doubled) std::cout << v << " ";
        std::cout << "\n\n";
    }

    // ─── 4. inserter — Middle Insertion ─────────────────────────────────────────
    {
        std::cout << "--- 4. inserter (mid-point insertion) ---\n";

        std::vector<int> data = {1, 2, 5, 6};
        std::vector<int> toInsert = {3, 4};

        // EN: Insert {3,4} at position 2 (before element '5')
        // TR: {3,4}'ü 2. pozisyona ekle ('5' elemanından önce)
        auto insertPos = data.begin() + 2;
        std::copy(toInsert.begin(), toInsert.end(),
                  std::inserter(data, insertPos));

        std::cout << "After inserter: ";
        for (int v : data) std::cout << v << " ";
        std::cout << "\n\n";
    }

    // ─── 5. ostream_iterator — Direct Output ────────────────────────────────────
    {
        std::cout << "--- 5. ostream_iterator ---\n";

        std::vector<std::string> ecuNames = {"ECM", "TCM", "BCM", "ABS", "TPMS"};

        // EN: Write directly to cout using an output iterator — no loop needed!
        // TR: Output iterator ile doğrudan cout'a yaz — döngü gerekmez!
        std::cout << "ECU modules: ";
        std::copy(ecuNames.begin(), ecuNames.end(),
                  std::ostream_iterator<std::string>(std::cout, ", "));
        std::cout << "\n\n";
    }
```

```cpp

    // ─── 6. istream_iterator — Parse from String Stream ──────────────────────────
    {
        std::cout << "--- 6. istream_iterator (from stringstream) ---\n";

        // EN: Simulate reading space-separated sensor values from a data stream
        // TR: Veri akışından boşlukla ayrılmış sensör değerlerini okumayı simüle et
        std::istringstream sensorData("72 85 91 68 77");
        std::vector<int> readings{
            std::istream_iterator<int>(sensorData),
            std::istream_iterator<int>()};

        std::cout << "Parsed readings: ";
        for (int r : readings) std::cout << r << " ";
        std::cout << "\n";
        std::cout << "Average: "
                  << sumRange(readings.begin(), readings.end()) /
                         static_cast<int>(readings.size())
                  << "\n\n";
    }

    // ─── 7. Reverse Iterators ────────────────────────────────────────────────────
    {
        std::cout << "--- 7. Reverse Iterators ---\n";

        std::vector<int> faultLog = {101, 203, 305, 407, 509};

        // EN: rbegin()/rend() traverse the container backwards
        // TR: rbegin()/rend() konteyneri geriye doğru gezinir
        std::cout << "Fault log (newest first): ";
        for (auto it = faultLog.rbegin(); it != faultLog.rend(); ++it) {
            std::cout << *it << " ";
        }
        std::cout << "\n";

        // EN: Find last element > 300 using reverse iterators
        // TR: Ters iteratör ile > 300 olan son elemanı bul
        auto rit = std::find_if(faultLog.rbegin(), faultLog.rend(),
                                [](int code) { return code > 300; });
        if (rit != faultLog.rend()) {
            std::cout << "Last fault > 300: " << *rit << "\n";
        }
        std::cout << "\n";
    }

    // ─── 8. std::move_iterator ───────────────────────────────────────────────────
    {
        std::cout << "--- 8. move_iterator ---\n";

        std::vector<std::string> logs = {"[INFO] Start", "[WARN] Temp high",
                                         "[ERR] Overheat", "[INFO] Shutdown"};

        // EN: Move (not copy!) strings from logs into archive
        // TR: Stringleri logs'tan archive'a taşı (kopyalama değil!)
        std::vector<std::string> archive(
            std::make_move_iterator(logs.begin()),
            std::make_move_iterator(logs.end()));

        std::cout << "Archived " << archive.size() << " entries\n";
        std::cout << "Original logs[0] after move: \""
                  << logs[0] << "\" (empty — moved!)\n\n";
    }

    // ─── 9. DTC Filter with back_inserter ────────────────────────────────────────
    {
        std::cout << "--- 9. Automotive DTC Filter ---\n";

        std::vector<std::string> allDTCs = {
            "P0301", "P0420", "B0100", "C0035",
            "P0171", "U0100", "P0442", "B0200"};

```

```
        // EN: Filter only Powertrain codes (start with 'P')
        // TR: Sadece Motor/Şanzıman kodlarını filtrele ('P' ile başlayan)
        auto powertrainDTCs = collectActiveDTCs(allDTCs, "P");

        std::cout << "Powertrain DTCs: ";
        std::copy(powertrainDTCs.begin(), powertrainDTCs.end(),
                  std::ostream_iterator<std::string>(std::cout, " "));
        std::cout << "\n";

        // EN: Filter Body codes (start with 'B')
        // TR: Gövde kodlarını filtrele ('B' ile başlayan)
        auto bodyDTCs = collectActiveDTCs(allDTCs, "B");
        std::cout << "Body DTCs: ";
        std::copy(bodyDTCs.begin(), bodyDTCs.end(),
                  std::ostream_iterator<std::string>(std::cout, " "));
        std::cout << "\n\n";
    }

    // ─── 10. std::iota + Iterators ──────────────────────────────────────────────────────────
    {
        std::cout << "--- 10. std::iota + Iterator Arithmetic ---\n";

        std::vector<int> sensorIDs(10);
        // EN: Fill with sequential IDs: 100, 101, 102, ...
        // TR: Sıralı ID'ler ile doldur: 100, 101, 102, ...
        std::iota(sensorIDs.begin(), sensorIDs.end(), 100);

        // EN: Random Access: jump to 5th element directly
        // TR: Random Access: doğrudan 5. elemana atla
        auto it = sensorIDs.begin() + 4;
        std::cout << "5th sensor ID: " << *it << "\n";

        // EN: Iterator arithmetic: distance between two positions
        // TR: İteratör aritmetiği: iki pozisyon arası mesafe
        auto first = std::find(sensorIDs.begin(), sensorIDs.end(), 103);
        auto last = std::find(sensorIDs.begin(), sensorIDs.end(), 108);
        std::cout << "Distance 103→108: "
                  << std::distance(first, last) << " elements\n";

        // EN: advance() moves an iterator by N steps
        // TR: advance() bir iteratörü N adım ilerletir
        auto walker = sensorIDs.begin();
        std::advance(walker, 7);
        std::cout << "After advance(7): " << *walker << "\n";

        // EN: next()/prev() return a NEW iterator without modifying the original
        // TR: next()/prev() orijinali değiştirmeden YENİ iteratör döndürür
        auto n = std::next(sensorIDs.begin(), 3);
        auto p = std::prev(sensorIDs.end(), 2);
        std::cout << "next(begin,3)=" << *n << "  prev(end,2)=" << *p << "\n";
    }

    std::cout << "\n=== ITERATORS & ADAPTORS COMPLETE ===\n";
    return 0;
}
```

► 05. Algorithms Masterclass

module_04_advanced/05_algorithms_masterclass.cpp

Konu: Templates & STL: Algorithm Masterclass — Şablonlar ve STL: Algoritma Ustalık Rehberi

Teori / Theory

THEORY: STL Algorithms — The Power of Generic Programming

The STL provides 100+ algorithms in <algorithm> and <numeric>. They operate on ITERATORS, not containers — making them universal. This is the "separation of concerns" principle: containers STORE data, algorithms PROCESS data, iterators CONNECT them.
Key algorithm families:

```
┌──────────────────────────────────────────────────────────┐
│ Non-modifying  │ find, count, all_of, any_of, none_of,      │
│                │ for_each, mismatch, equal, search          │
├──────────────────────────────────────────────────────────┤
│ Modifying      │ copy, move, transform, fill, generate,     │
│                │ replace, remove, unique, reverse, rotate   │
├──────────────────────────────────────────────────────────┤
│ Sorting        │ sort, stable_sort, partial_sort,           │
│                │ nth_element, is_sorted                     │
├──────────────────────────────────────────────────────────┤
│ Binary Search  │ lower_bound, upper_bound, equal_range,     │
│                │ binary_search (requires sorted range!)     │
├──────────────────────────────────────────────────────────┤
│ Partitioning   │ partition, stable_partition,               │
│                │ is_partitioned                             │
├──────────────────────────────────────────────────────────┤
│ Numeric        │ accumulate, inner_product, partial_sum,    │
│                │ iota, reduce (C++17), transform_reduce     │
├──────────────────────────────────────────────────────────┤
│ Min/Max        │ min, max, minmax, min_element,             │
│                │ max_element, minmax_element, clamp (C++17) │
└──────────────────────────────────────────────────────────┘
```

TEORİ: STL Algoritmaları — Jenerik Programlamanın Gücü

STL, <algorithm> ve <numeric> başlıklarında 100'den fazla algoritma sunar. Konteynerler üzerinde değil, İTERATÖRLER üzerinde çalışırlar — bu onları evrensel kılar. Bu "sorumlulukların ayrılması" ilkesidir: konteynerler veri DEPOLAR, algoritmalar veri İŞLER, iteratörler onları BAĞLAR.
Temel algoritma aileleri:

```
┌──────────────────────────────────────────────────────────┐
│ Değiştirmeyen  │ find, count, all_of, any_of, none_of,      │
│                │ for_each, mismatch, equal, search          │
├──────────────────────────────────────────────────────────┤
│ Değiştiren     │ copy, move, transform, fill, generate,     │
│                │ replace, remove, unique, reverse, rotate   │
├──────────────────────────────────────────────────────────┤
│ Sıralama       │ sort, stable_sort, partial_sort,           │
│                │ nth_element, is_sorted                     │
├──────────────────────────────────────────────────────────┤
│ İkili Arama    │ lower_bound, upper_bound, equal_range,     │
│                │ binary_search (sıralı aralık gerektirir!)  │
├──────────────────────────────────────────────────────────┤
│ Bölümleme      │ partition, stable_partition,               │
│                │ is_partitioned                             │
├──────────────────────────────────────────────────────────┤
│ Sayısal        │ accumulate, inner_product, partial_sum,    │
│                │ iota, reduce (C++17), transform_reduce     │
├──────────────────────────────────────────────────────────┤
│ Min/Maks       │ min, max, minmax, min_element,             │
│                │ max_element, minmax_element, clamp (C++17) │
└──────────────────────────────────────────────────────────┘
```

CPPREF Depth: Algorithm Complexity Guarantees and Exception Safety

STL algorithms provide strict worst-case complexity guarantees:
• `std::sort` — O(n log n) comparisons. Typically introsort (quicksort + heapsort fallback), guaranteed worst-case O(n log n).
• `std::stable_sort` — O(n $\log^2$n) in-place, or O(n log n) if extra memory is available (merge sort with buffer).
• `std::nth_element` — O(n) average (introselect). After the call, element at nth position is what it would be in a sorted range.
• `std::partial_sort` — O(n log k) where k = middle - first.
Exception safety: algorithms use `std::move_if_noexcept` internally. If the element's move constructor is noexcept, it moves; otherwise it copies to maintain the strong exception guarantee. This is why marking move constructors `noexcept` is critical for performance. Algorithms that write to output iterators (e.g., `std::transform`) provide the basic guarantee: no leaks, but output may be partial. `std::for_each` returns its function object — useful for stateful functors accumulating results during traversal.

CPPREF Derinlik: Algoritma Karmaşıklık Garantileri ve İstisna Güvenliği

STL algoritmaları katı en-kötü-durum karmaşıklık garantisi sunar:
• `std::sort` — O(n log n) karşılaştırma. Genelde introsort (quicksort + heapsort yedek), en kötü durum O(n log n) garantili.
• `std::stable_sort` — Yerinde O(n $\log^2$n), veya ek bellekle O(n log n) (tamponlu merge sort).
• `std::nth_element` — ortalama O(n) (introselect). Çağrı sonrası n. pozisyondaki eleman, sıralı dizideki yerindedir.
• `std::partial_sort` — O(n log k), k = middle - first.
İstisna güvenliği: algoritmalar dahili olarak `std::move_if_noexcept` kullanır. Elemanın taşıma yapıcısı noexcept ise taşır; değilse güçlü istisna garantisini korumak için kopyalar. Bu nedenle taşıma yapıcılarını `noexcept` işaretlemek performans için kritiktir. Çıkış iteratörlerine yazan algoritmalar (ör. `std::transform`) temel garanti sunar: sızıntı olmaz, ama çıktı kısmi kalabilir.

Kaynak Kod / Source Code

```cpp
#include <algorithm>   // EN: STL algorithms
#include <cmath>       // EN: std::abs
#include <functional>  // EN: std::greater, std::multiplies
#include <iostream>    // EN: Standard I/O
#include <iterator>    // EN: ostream_iterator, back_inserter
#include <numeric>     // EN: accumulate, iota, inner_product, partial_sum, reduce
#include <string>      // EN: std::string
#include <vector>      // EN: std::vector

// ═══════════════════════════════════════════════════════════════════════════════
// [PRODUCTION CODE / ÜRETİM KODU]
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Automotive sensor reading structure
// TR: Otomotiv sensör okuma yapısı
struct SensorReading {
    std::string name;
    double value;
    // EN: 1=critical, 5=info
    // TR: 1=kritik, 5=bilgi
    int priority;

    friend std::ostream& operator<<(std::ostream& os, const SensorReading& s) {
        return os << s.name << "(" << s.value << ",P" << s.priority << ")";
    }
};

// ═══════════════════════════════════════════════════════════════════════════════
// [EXAMPLES / ÖRNEKLER]
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "=== MODULE 4: STL ALGORITHMS MASTERCLASS ===\n"
              << "=== MODÜL 4: STL ALGORİTMA USTALIK REHBERİ ===\n\n";

    // ═══════════════════════════════════════════════════════════════════════════
    // [SORTING FAMILY / SIRALAMA AİLESİ]
    // ═══════════════════════════════════════════════════════════════════════════

    // ─── 1. std::sort — Introsort O(N log N) ──────────────────────────────────
    {
        std::cout << "--- 1. std::sort ---\n";

        std::vector<int> speeds = {120, 85, 200, 60, 150, 95, 180};

        // EN: Default ascending sort
        // TR: Varsayılan artan sıralama
        std::sort(speeds.begin(), speeds.end());
        std::cout << "Ascending: ";
        for (int s : speeds) std::cout << s << " ";

        // EN: Descending sort with std::greater
        // TR: std::greater ile azalan sıralama
        std::sort(speeds.begin(), speeds.end(), std::greater<int>());
        std::cout << "\nDescending: ";
        for (int s : speeds) std::cout << s << " ";
        std::cout << "\n\n";
    }

```

```cpp
    // ─── 2. std::stable_sort — Preserves Equal Order ──────────────────────────────
    {
        std::cout << "--- 2. std::stable_sort (preserves relative order) ---\n";

        std::vector<SensorReading> alerts = {
            {"OilTemp", 95.0, 2},    {"Coolant", 102.0, 1},
            {"TireFL", 32.0, 3},     {"BrakeFL", 450.0, 1},
            {"Battery", 12.1, 3},    {"FuelLvl", 25.0, 2}};

        // EN: stable_sort by priority — equal priorities keep original order
        // TR: Önceliğe göre stable_sort — eşit öncelikler sırasını korur
        std::stable_sort(alerts.begin(), alerts.end(),
                         [](const SensorReading& a, const SensorReading& b) {
                             return a.priority < b.priority;
                         });

        std::cout << "By priority (stable): ";
        for (const auto& s : alerts) std::cout << s << " ";
        std::cout << "\n\n";
    }

    // ─── 3. std::partial_sort — Top-N ─────────────────────────────────────────────
    {
        std::cout << "--- 3. std::partial_sort (Top-3 fastest) ---\n";

        std::vector<int> lapTimes = {92, 88, 95, 85, 91, 87, 98, 83};

        // EN: Sort only the first 3 elements — rest is unspecified order More efficient than
        // full sort when you only need top-N!
        // TR: Sadece ilk 3 elemanı sırala — geri kalan belirsiz sırada Sadece ilk-N gerektiğinde
        // tam sıralamadan daha verimli!
        std::partial_sort(lapTimes.begin(), lapTimes.begin() + 3,
                          lapTimes.end());

        std::cout << "Top 3 fastest laps: ";
        for (int i = 0; i < 3; ++i) std::cout << lapTimes[static_cast<size_t>(i)] << "s ";
        std::cout << "\n\n";
    }

    // ─── 4. std::nth_element — Selection ──────────────────────────────────────────
    {
        std::cout << "--- 4. std::nth_element (median finder) ---\n";

        std::vector<int> temps = {72, 88, 65, 91, 78, 85, 70};
        size_t mid = temps.size() / 2;

        // EN: After nth_element: temps[mid] is the median. Elements before it are ≤ it, after
        // are ≥ it (unordered). O(N) average — much faster than O(N log N) full sort!
        // TR: nth_element sonrası: temps[mid] medyan. Öncesindekiler ≤, sonrasındakiler ≥
        // (sırasız). Ortalama O(N) — tam sıralamadan çok daha hızlı!
        std::nth_element(temps.begin(), temps.begin() + static_cast<long>(mid),
                         temps.end());
        std::cout << "Median temperature: " << temps[mid] << "°C\n\n";
    }

    // ═══════════════════════════════════════════════════════════════════════════════
    // [SEARCHING FAMILY / ARAMA AİLESİ]
    // ═══════════════════════════════════════════════════════════════════════════════

    // ─── 5. std::find / std::find_if ──────────────────────────────────────────────
    {
        std::cout << "--- 5. std::find / std::find_if ---\n";

        std::vector<int> errorCodes = {0, 0, 301, 0, 420, 0, 171};

        // EN: find first non-zero error code
        // TR: ilk sıfır olmayan hata kodunu bul
        auto it = std::find_if(errorCodes.begin(), errorCodes.end(),
                               [](int code) { return code != 0; });
        if (it != errorCodes.end()) {
            std::cout << "First error: P0" << *it
                      << " at index " << std::distance(errorCodes.begin(), it)
                      << "\n\n";
        }
    }
```

```cpp

    // ─── 6. Binary Search Family (SORTED ranges only!) ──────────────────────────────
    {
        std::cout << "--- 6. Binary Search (lower/upper_bound, equal_range) ---\n";

        // EN: MUST be sorted for binary search algorithms!
        // TR: Binary search algoritmaları için SIRALANMIŞ olmalı!
        std::vector<int> sortedRPMs = {1000, 1500, 2000, 2500, 3000,
                                       3000, 3000, 3500, 4000, 5000};

        // EN: lower_bound — first element ≥ value (insertion point)
        // TR: lower_bound — değere ≥ ilk eleman (ekleme noktası)
        auto lo = std::lower_bound(sortedRPMs.begin(), sortedRPMs.end(), 3000);
        std::cout << "lower_bound(3000): index "
                  << std::distance(sortedRPMs.begin(), lo) << "\n";

        // EN: upper_bound — first element > value
        // TR: upper_bound — değerden > ilk eleman
        auto up = std::upper_bound(sortedRPMs.begin(), sortedRPMs.end(), 3000);
        std::cout << "upper_bound(3000): index "
                  << std::distance(sortedRPMs.begin(), up) << "\n";

        // EN: equal_range — returns [lower_bound, upper_bound) pair Count of 3000s = upper -
        // lower
        // TR: equal_range — [lower_bound, upper_bound) çifti döndürür
        auto [eqLo, eqUp] = std::equal_range(sortedRPMs.begin(),
                                             sortedRPMs.end(), 3000);
        std::cout << "Count of 3000 RPM readings: "
                  << std::distance(eqLo, eqUp) << "\n";

        // EN: binary_search — just returns true/false (exists or not)
        // TR: binary_search — sadece true/false döndürür (var mı yok mu)
        bool has4500 = std::binary_search(sortedRPMs.begin(),
                                          sortedRPMs.end(), 4500);
        std::cout << "Has 4500 RPM? " << (has4500 ? "yes" : "no") << "\n\n";
    }

    // ═══════════════════════════════════════════════════════════════════════════════
    // [MODIFYING ALGORITHMS / DEĞİŞTİRİCİ ALGORİTMALAR]
    // ═══════════════════════════════════════════════════════════════════════════════

    // ─── 7. std::transform — Map Operation ──────────────────────────────────────
    {
        std::cout << "--- 7. std::transform (map operation) ---\n";

        std::vector<double> celsiusReadings = {22.5, 18.0, 35.2, 0.0, -5.0};
        std::vector<double> fahrenheit;

        // EN: Convert Celsius → Fahrenheit using transform
        // TR: Transform ile Celsius → Fahrenheit dönüşümü
        std::transform(celsiusReadings.begin(), celsiusReadings.end(),
                       std::back_inserter(fahrenheit),
                       [](double c) { return c * 9.0 / 5.0 + 32.0; });

        std::cout << "Fahrenheit: ";
        for (double f : fahrenheit) std::cout << f << " ";
        std::cout << "\n\n";
    }

    // ─── 8. std::remove_if + erase (Erase-Remove Idiom) ─────────────────────────
    {
        std::cout << "--- 8. Erase-Remove Idiom ---\n";

        std::vector<int> readings = {85, -1, 72, -1, 91, 68, -1, 77};

        // EN: remove_if moves "removed" elements to the end, returns new logical end. You MUST
        // call erase() to actually shrink! -1 represents invalid/error readings.
        // TR: remove_if "silinen" elemanları sona taşır, yeni mantıksal sonu döndürür. Gerçekten
        // küçültmek için erase() ÇAĞIRMALISIN!
        auto newEnd = std::remove_if(readings.begin(), readings.end(),
                                     [](int r) { return r < 0; });
        readings.erase(newEnd, readings.end());

        std::cout << "After removing invalid (-1): ";
        for (int r : readings) std::cout << r << " ";
        std::cout << " (size=" << readings.size() << ")\n\n";
    }
```

```cpp

    // ─── 9. std::unique — Remove Consecutive Duplicates ──────────────────────────
    {
        std::cout << "--- 9. std::unique (consecutive duplicates) ---\n";

        std::vector<int> canFrames = {100, 100, 200, 200, 200, 300, 100, 100};

        // EN: unique only removes CONSECUTIVE duplicates — sort first for all!
        // TR: unique sadece ARDIŞIK tekrarları siler — hepsi için önce sırala!
        auto it = std::unique(canFrames.begin(), canFrames.end());
        canFrames.erase(it, canFrames.end());

        std::cout << "After unique: ";
        for (int f : canFrames) std::cout << f << " ";
        std::cout << "\n\n";
    }

    // ─── 10. std::rotate — Circular Shift ────────────────────────────────────────
    {
        std::cout << "--- 10. std::rotate ---\n";

        std::vector<int> buffer = {1, 2, 3, 4, 5, 6, 7, 8};

        // EN: Rotate left by 3 positions: element at position 3 becomes first
        // TR: 3 pozisyon sola döndür: 3. pozisyondaki eleman ilk olur
        std::rotate(buffer.begin(), buffer.begin() + 3, buffer.end());

        std::cout << "After rotate left 3: ";
        for (int b : buffer) std::cout << b << " ";
        std::cout << "\n\n";
    }

    // ═══════════════════════════════════════════════════════════════════════════
    // [PARTITIONING / BÖLÜMLEME]
    // ═══════════════════════════════════════════════════════════════════════════

    // ─── 11. std::partition / std::stable_partition ──────────────────────────────
    {
        std::cout << "--- 11. std::partition ---\n";

        std::vector<SensorReading> data = {
            {"OilTemp", 95.0, 2},  {"Coolant", 110.0, 1},
            {"TireFL", 32.0, 3},   {"BrakeFL", 500.0, 1},
            {"Battery", 12.1, 3},  {"FuelLvl", 15.0, 2}};

        // EN: Partition into critical (priority ≤ 2) and non-critical
        // TR: Kritik (öncelik ≤ 2) ve kritik olmayan olarak böl
        auto boundary = std::stable_partition(
            data.begin(), data.end(),
            [](const SensorReading& s) { return s.priority <= 2; });

        std::cout << "Critical alerts: ";
        for (auto it = data.begin(); it != boundary; ++it) {
            std::cout << it->name << " ";
        }
        std::cout << "\nInfo alerts: ";
        for (auto it = boundary; it != data.end(); ++it) {
            std::cout << it->name << " ";
        }
        std::cout << "\n\n";
    }

    // ═══════════════════════════════════════════════════════════════════════════
    // [NUMERIC ALGORITHMS / SAYISAL ALGORİTMALAR]
    // ═══════════════════════════════════════════════════════════════════════════

    // ─── 12. std::accumulate, std::reduce (C++17) ────────────────────────────────
    {
        std::cout << "--- 12. std::accumulate & std::reduce ---\n";

        std::vector<double> fuelConsumption = {7.2, 8.1, 6.5, 9.0, 7.8};

```

```
        // EN: accumulate — sequential fold (guarantees left-to-right)
        // TR: accumulate — sıralı katlama (soldan sağa garanti)
        double totalFuel = std::accumulate(fuelConsumption.begin(),
                                           fuelConsumption.end(), 0.0);
        std::cout << "Total fuel: " << totalFuel << " L\n";
        std::cout << "Average: " << totalFuel / static_cast<double>(fuelConsumption.size())
                  << " L/100km\n";

        // EN: reduce (C++17) — may reorder operations (parallelizable!) For floating point,
        // reduce may give slightly different results due to reordering. For integers, identical
        // to accumulate.
        // TR: reduce (C++17) — işlemleri yeniden sıralayabilir (paralelleştirilebilir!)
        double totalReduce = std::reduce(fuelConsumption.begin(),
                                         fuelConsumption.end(), 0.0);
        std::cout << "reduce total: " << totalReduce << " L\n\n";
    }

    // ─── 13. std::inner_product — Dot Product ──────────────────────────────────────
    {
        std::cout << "--- 13. std::inner_product ---\n";

        // EN: Weighted sensor scores: score = Σ(value × weight)
        // TR: Ağırlıklı sensör skorları: skor = Σ(değer × ağırlık)
        std::vector<double> values = {95.0, 78.0, 88.0, 92.0};
        std::vector<double> weights = {0.3, 0.2, 0.25, 0.25};

        double weightedScore = std::inner_product(
            values.begin(), values.end(), weights.begin(), 0.0);
        std::cout << "Weighted health score: " << weightedScore << "\n\n";
    }

    // ─── 14. std::partial_sum — Running Total ──────────────────────────────────────
    {
        std::cout << "--- 14. std::partial_sum (cumulative) ---\n";

        std::vector<int> dailyKm = {120, 85, 200, 150, 95};
        std::vector<int> cumulative;

        // EN: Cumulative distance: [120, 205, 405, 555, 650]
        // TR: Kümülatif mesafe: [120, 205, 405, 555, 650]
        std::partial_sum(dailyKm.begin(), dailyKm.end(),
                         std::back_inserter(cumulative));

        std::cout << "Daily km: ";
        for (int d : dailyKm) std::cout << d << " ";
        std::cout << "\nCumulative: ";
        for (int c : cumulative) std::cout << c << " ";
        std::cout << "\n\n";
    }

    // ═══════════════════════════════════════════════════════════════════════════════
    // [PREDICATES & CONDITIONS / KOŞULLAR]
    // ═══════════════════════════════════════════════════════════════════════════════

    // ─── 15. all_of / any_of / none_of ─────────────────────────────────────────────
    {
        std::cout << "--- 15. all_of / any_of / none_of ---\n";

        std::vector<int> tirePressures = {32, 33, 31, 34};

        bool allSafe = std::all_of(tirePressures.begin(), tirePressures.end(),
                                   [](int p) { return p >= 28 && p <= 36; });
        bool anyLow = std::any_of(tirePressures.begin(), tirePressures.end(),
                                  [](int p) { return p < 28; });
        bool noneFlat = std::none_of(tirePressures.begin(), tirePressures.end(),
                                     [](int p) { return p == 0; });

        std::cout << "All tires safe (28-36 PSI)? " << (allSafe ? "yes" : "no") << "\n";
        std::cout << "Any tire low (<28 PSI)? " << (anyLow ? "yes" : "no") << "\n";
        std::cout << "No flat tires? " << (noneFlat ? "yes" : "no") << "\n\n";
    }
```

```cpp

    // ─── 16. std::count / std::count_if ──────────────────────────────────────────
    {
        std::cout << "--- 16. std::count_if ---\n";

        std::vector<int> speedLog = {60, 85, 120, 130, 95, 140, 110, 70};
        int speedLimit = 120;

        auto violations = std::count_if(
            speedLog.begin(), speedLog.end(),
            [speedLimit](int s) { return s > speedLimit; });
        std::cout << "Speed violations (>" << speedLimit << " km/h): "
                  << violations << "\n\n";
    }

    // ═══════════════════════════════════════════════════════════════════════════════
    // [MIN/MAX / MİN/MAKS]
    // ═══════════════════════════════════════════════════════════════════════════════

    // ─── 17. min_element / max_element / minmax_element / clamp ──────────────────
    {
        std::cout << "--- 17. min/max/clamp ---\n";

        std::vector<double> voltages = {12.4, 11.8, 13.2, 12.9, 11.5, 13.8};

        auto [minIt, maxIt] = std::minmax_element(voltages.begin(),
                                                   voltages.end());
        std::cout << "Min voltage: " << *minIt << "V\n";
        std::cout << "Max voltage: " << *maxIt << "V\n";

        // EN: clamp (C++17) — restrict a value to [lo, hi] range
        // TR: clamp (C++17) — değeri [lo, hi] aralığına kısıtla
        double rawThrottle = 1.15;
        double clamped = std::clamp(rawThrottle, 0.0, 1.0);
        std::cout << "Throttle " << rawThrottle << " clamped to [0,1]: "
                  << clamped << "\n\n";
    }

    // ─── 18. std::for_each with Side Effects ─────────────────────────────────────
    {
        std::cout << "--- 18. std::for_each ---\n";

        std::vector<std::string> modules = {"ECM", "TCM", "BCM", "ABS"};

        int index = 0;
        std::for_each(modules.begin(), modules.end(),
                      [&index](const std::string& m) {
                          std::cout << "  [" << index++ << "] " << m << "\n";
                      });
        std::cout << "\n";
    }

    // ─── 19. std::generate — Fill with Computed Values ───────────────────────────
    {
        std::cout << "--- 19. std::generate ---\n";

        std::vector<int> sensorIDs(8);
        int nextID = 1000;

        // EN: Generate sequential sensor IDs starting from 1000
        // TR: 1000'den başlayarak sıralı sensör ID'leri üret
        std::generate(sensorIDs.begin(), sensorIDs.end(),
                      [&nextID]() { return nextID++; });

        std::cout << "Generated IDs: ";
        for (int id : sensorIDs) std::cout << id << " ";
        std::cout << "\n\n";
    }
```

```

    // ─── 20. std::mismatch — Find First Difference ──────────────────────────────
    {
        std::cout << "--- 20. std::mismatch ---\n";

        std::vector<int> expected = {100, 200, 300, 400, 500};
        std::vector<int> actual   = {100, 200, 350, 400, 500};

        // EN: Find where calibration data diverges from expected values
        // TR: Kalibrasyon verilerinin beklenen değerlerden nerede saptığını bul
        auto [expIt, actIt] = std::mismatch(expected.begin(), expected.end(),
                                            actual.begin());
        if (expIt != expected.end()) {
            std::cout << "First mismatch at index "
                      << std::distance(expected.begin(), expIt)
                      << ": expected=" << *expIt
                      << " actual=" << *actIt << "\n";
        }
    }

    std::cout << "\n=== ALGORITHMS MASTERCLASS COMPLETE ===\n";
    return 0;
}
```

► 06. Associative Containers

module_04_advanced/06_associative_containers.cpp

Konu: Templates & STL: Associative & Unordered Containers — Şablonlar ve STL: İlişkisel ve

Teori / Theory

THEORY: Ordered Containers — Red-Black Tree

std::set, std::map, std::multiset, std::multimap Backed by Red-Black Tree (self-balancing BST).
✓ Always sorted by key
✓ O(log N) insert, find, erase
✓ Supports ordered traversal, lower_bound, upper_bound
✗ Slower than hash for pure lookup

TEORİ: Sıralı Konteynerler — Kırmızı-Siyah Ağaç

std::set, std::map, std::multiset, std::multimap Kırmızı-Siyah Ağaç (kendi kendini dengeleyen BST) ile desteklenir.
✓ Her zaman anahtara göre sıralı
✓ O(log N) ekleme, bulma, silme
✓ Sıralı gezinme, lower_bound, upper_bound destekler
✗ Saf arama için hash'ten yavaş

THEORY: Unordered Containers — Hash Table

std::unordered_set, std::unordered_map, std::unordered_multiset/map Backed by Hash Table (bucket array + linked lists).
✓ O(1) average insert, find, erase
✓ Fastest for pure key lookup
✗ No ordering guarantee
✗ Worst case O(N) if many hash collisions
✗ Requires hash function + equality operator

TEORİ: Sırasız Konteynerler — Hash Tablosu

std::unordered_set, std::unordered_map, std::unordered_multiset/map Hash Tablosu (kova dizisi + bağlı listeler) ile desteklenir.
✓ Ortalama O(1) ekleme, bulma, silme
✓ Saf anahtar araması için en hızlı
✗ Sıralama garantisi yok
✗ Çok çarpışma varsa en kötü O(N)
✗ Hash fonksiyonu + eşitlik operatörü gerektirir

THEORY: Container Adaptors

std::stack (LIFO), std::queue (FIFO), std::priority_queue (max-heap). These are NOT real containers — they WRAP an underlying container (default: deque for stack/queue, vector for priority_queue) and restrict its interface.

TEORİ: Konteyner Adaptörleri

std::stack (LIFO), std::queue (FIFO), std::priority_queue (max-heap). Bunlar gerçek konteyner DEĞİL — alttaki bir konteyneri SARARLAR (varsayılan: stack/queue için deque, priority_queue için vector) ve arayüzünü kısıtlarlar.

CPPREF Depth: Red-Black Trees vs Hash Tables — When Order Matters

`std::map` / `std::set` are red-black trees (self-balancing BST):
• All operations O(log n) — insert, find, erase.
• Iteration yields elements in SORTED order by key.
• Iterators are stable: insert/erase never invalidates other iterators. `std::unordered_map` / `std::unordered_set` use hash tables:
• Average O(1) for insert/find/erase, worst-case O(n) (all collide).
• No ordering guarantee — iteration order is implementation-defined.
• Load factor = size / bucket_count. When load_factor > max_load_factor (default 1.0), a REHASH occurs: new bucket array allocated, all elements re-hashed — O(n) and ALL iterators are invalidated! Custom hash: specialize `std::hash<T>` or pass as template argument: `std::unordered_map<K, V, MyHash, MyEqual>` A good hash distributes uniformly. Combine fields with: `seed ^= std::hash<T>{}(val) + 0x9e3779b9 + (seed<<6) + (seed>>2);` C++17 `try_emplace()` avoids constructing the value if key exists — more efficient than `emplace()` for expensive-to-construct values.

CPPREF Derinlik: Kırmızı-Siyah Ağaç vs Hash Tablosu — Sıra Ne Zaman Önemli

`std::map` / `std::set` kırmızı-siyah ağaçtır (dengeli BST):
• Tüm işlemler O(log n) — ekleme, arama, silme.
• İterasyon elemanları anahtara göre SIRALI verir.
• İteratörler kararlıdır: ekleme/silme diğer iteratörleri bozmaz. `std::unordered_map` / `std::unordered_set` hash tablosu kullanır:
• Ortalama O(1), en kötü O(n) (tüm anahtarlar çakışırsa).
• Sıralama garantisi YOKTUR — gezinme sırası uygulama tanımlıdır.
• Yük faktörü = size / bucket_count. max_load_factor (varsayılan 1.0) aşılınca REHASH olur: yeni kova dizisi, tüm elemanlar yeniden hash'lenir — O(n) ve TÜM iteratörler geçersiz olur! Özel hash: `std::hash<T>` özelleştirmesi veya şablon argümanı: `std::unordered_map<K, V, MyHash, MyEqual>` İyi bir hash fonksiyonu düzgün dağılım sağlar. Alanları birleştirmek için: `seed ^= hash(val) + 0x9e3779b9 + (seed<<6) + (seed>>2);` C++17 `try_emplace()` anahtar varsa değer oluşturmaktan kaçınır — pahalı nesnelerde `emplace()`'ten daha verimlidir.

Kaynak Kod / Source Code

```cpp
#include <algorithm>       // EN: STL algorithms
#include <functional>      // EN: std::greater
#include <iostream>        // EN: Standard I/O
#include <map>             // EN: std::map, std::multimap
#include <queue>           // EN: std::queue, std::priority_queue
#include <set>             // EN: std::set, std::multiset
#include <stack>           // EN: std::stack
#include <string>          // EN: std::string
#include <unordered_map>   // EN: std::unordered_map
#include <unordered_set>   // EN: std::unordered_set
#include <vector>          // EN: std::vector

// ═══════════════════════════════════════════════════════════════════════════
// [PRODUCTION CODE / ÜRETİM KODU]
// ═══════════════════════════════════════════════════════════════════════════

// ─── 1. Custom Comparator for std::set ──────────────────────────────────

// EN: Diagnostic Trouble Code (DTC) with severity and timestamp.
// TR: Arıza Kodu (DTC) — şiddet derecesi ve zaman damgası ile.
struct DTC {
    std::string code;     // EN: e.g. "P0301"
    int severity;         // EN: 1=critical..5=info
    int timestampMs;      // EN: milliseconds since boot

    friend std::ostream& operator<<(std::ostream& os, const DTC& d) {
        return os << d.code << "(sev=" << d.severity
                  << ",t=" << d.timestampMs << "ms)";
    }
};

// EN: Custom comparator: sort DTCs by severity first, then by time. This is a FUNCTOR — std::set
// stores it internally.
// TR: Özel karşılaştırıcı: DTC'leri önce şiddete, sonra zamana göre sırala. Bu bir FUNCTOR —
// std::set bunu dahili olarak saklar.
struct DTCSeverityOrder {
    bool operator()(const DTC& a, const DTC& b) const {
        if (a.severity != b.severity) return a.severity < b.severity;
        return a.timestampMs < b.timestampMs;
    }
};

// ─── 2. Custom Hash for unordered_map ───────────────────────────────────

// EN: Sensor key: ECU ID + Sensor Index. We need a custom hash because std::pair doesn't have a
// default hash in C++.
// TR: Sensör anahtarı: ECU ID + Sensör İndeksi. std::pair'in C++'ta varsayılan hash'i olmadığı
// için özel hash gerekir.
struct SensorKey {
    int ecuId;
    int sensorIndex;

    bool operator==(const SensorKey& other) const {
        return ecuId == other.ecuId && sensorIndex == other.sensorIndex;
    }
};

struct SensorKeyHash {
    size_t operator()(const SensorKey& k) const {
        // EN: Combine two hashes using XOR + bit shift (a common technique)
        // TR: İki hash'i XOR + bit kaydırma ile birleştir (yaygın teknik)
        size_t h1 = std::hash<int>{}(k.ecuId);
        size_t h2 = std::hash<int>{}(k.sensorIndex);
        return h1 ^ (h2 << 1);
    }
};
```

```cpp

// ─── 3. Priority Queue for Alert Processing ──────────────────────────────────

// EN: Vehicle alert with priority level.
// TR: Öncelik seviyeli araç uyarısı.
struct VehicleAlert {
    std::string message;
    int priority;  // EN: lower = more urgent

    // EN: priority_queue is max-heap by default, so we invert: higher priority number = LESS
    // urgent → goes to bottom
    // TR: priority_queue varsayılan max-heap, bu yüzden ters çeviriyoruz: yüksek öncelik
    // numarası = DAHA AZ acil → alta gider
    bool operator<(const VehicleAlert& other) const {
        return priority > other.priority;  // min-heap behavior
    }
};

// ═══════════════════════════════════════════════════════════════════════════════
// [EXAMPLES / ÖRNEKLER]
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "=== MODULE 4: ASSOCIATIVE & UNORDERED CONTAINERS ===\n"
              << "=== MODÜL 4: İLİŞKİSEL VE SIRASIZ KONTEYNERLER ===\n\n";

    // ═══════════════════════════════════════════════════════════════════════════
    // [ORDERED CONTAINERS / SIRALI KONTEYNERLER]
    // ═══════════════════════════════════════════════════════════════════════════

    // ─── 1. std::map — Sorted Key-Value ──────────────────────────────────────
    {
        std::cout << "--- 1. std::map (sorted key-value) ---\n";

        std::map<std::string, double> sensorData;

        // EN: Insert via [] operator (creates if not exists)
        // TR: [] operatörü ile ekle (yoksa oluşturur)
        sensorData["OilTemp"] = 92.5;
        sensorData["Coolant"] = 88.0;
        sensorData["Battery"] = 12.6;

        // EN: Insert via insert() — does NOT overwrite existing!
        // TR: insert() ile ekle — mevcut olanın üzerine YAZMAZ!
        auto [it1, inserted1] = sensorData.insert({"OilTemp", 999.0});
        std::cout << "OilTemp insert: " << (inserted1 ? "new" : "exists")
                  << " val=" << it1->second << "\n";

        // EN: insert_or_assign() (C++17) — inserts or overwrites
        // TR: insert_or_assign() (C++17) — ekler veya üzerine yazar
        sensorData.insert_or_assign("OilTemp", 95.0);

        // EN: Traversal is ALWAYS sorted by key (alphabetical for string)
        // TR: Gezinme HER ZAMAN anahtara göre sıralı (string için alfabetik)
        std::cout << "All sensors (sorted):\n";
        for (const auto& [key, val] : sensorData) {
            std::cout << "  " << key << " = " << val << "\n";
        }

        // EN: find() returns iterator — O(log N)
        // TR: find() iterator döndürür — O(log N)
        auto found = sensorData.find("Coolant");
        if (found != sensorData.end()) {
            std::cout << "Found: " << found->first << " = " << found->second << "\n";
        }

        // EN: count() — 0 or 1 for map (useful for existence check)
        // TR: count() — map için 0 veya 1 (varlık kontrolü için kullanışlı)
        std::cout << "Has 'Turbo'? " << sensorData.count("Turbo") << "\n\n";
    }
```

```cpp

    // ─── 2. std::multimap — Multiple Values Per Key ──────────────────────────────
    {
        std::cout << "--- 2. std::multimap (multiple values per key) ---\n";

        std::multimap<int, std::string> errorsByHour;

        // EN: Same key (hour) can have multiple error entries
        // TR: Aynı anahtar (saat) birden fazla hata girişine sahip olabilir
        errorsByHour.insert({9, "P0301 — Cylinder misfire"});
        errorsByHour.insert({9, "P0420 — Catalyst efficiency"});
        errorsByHour.insert({10, "P0171 — Lean mixture"});
        errorsByHour.insert({9, "C0035 — ABS sensor"});

        // EN: equal_range returns [first, last) for a given key
        // TR: equal_range verilen anahtar için [ilk, son) döndürür
        auto [lo, hi] = errorsByHour.equal_range(9);
        std::cout << "Errors at hour 9:\n";
        for (auto it = lo; it != hi; ++it) {
            std::cout << "  " << it->second << "\n";
        }
        std::cout << "Total at hour 9: " << errorsByHour.count(9) << "\n\n";
    }

    // ─── 3. std::set with Custom Comparator ──────────────────────────────────────
    {
        std::cout << "--- 3. std::set<DTC, DTCSeverityOrder> ---\n";

        std::set<DTC, DTCSeverityOrder> dtcQueue;

        dtcQueue.insert({"P0301", 3, 1500});
        dtcQueue.insert({"P0420", 2, 800});
        dtcQueue.insert({"C0035", 1, 200});
        dtcQueue.insert({"B0100", 4, 3000});
        dtcQueue.insert({"P0171", 1, 1200});

        // EN: Iteration is in severity order (1=critical first)
        // TR: Gezinme şiddet sırasında (1=kritik ilk)
        std::cout << "DTCs by severity:\n";
        for (const auto& dtc : dtcQueue) {
            std::cout << "  " << dtc << "\n";
        }
        std::cout << "\n";
    }

    // ─── 4. std::set — Set Operations ────────────────────────────────────────────
    {
        std::cout << "--- 4. Set Operations (union, intersection, diff) ---\n";

        std::set<std::string> mondayDTCs = {"P0301", "P0420", "C0035", "P0171"};
        std::set<std::string> tuesdayDTCs = {"P0420", "B0100", "P0171", "U0100"};

        // EN: set_intersection — DTCs that appeared BOTH days
        // TR: set_intersection — HER İKİ gün görülen DTC'ler
        std::vector<std::string> persistent;
        std::set_intersection(mondayDTCs.begin(), mondayDTCs.end(),
                              tuesdayDTCs.begin(), tuesdayDTCs.end(),
                              std::back_inserter(persistent));
        std::cout << "Persistent DTCs: ";
        for (const auto& d : persistent) std::cout << d << " ";

        // EN: set_difference — Monday only (cleared by Tuesday)
        // TR: set_difference — Sadece Pazartesi (Salı'ya kadar temizlendi)
        std::vector<std::string> clearedDTCs;
        std::set_difference(mondayDTCs.begin(), mondayDTCs.end(),
                            tuesdayDTCs.begin(), tuesdayDTCs.end(),
                            std::back_inserter(clearedDTCs));
        std::cout << "\nCleared DTCs: ";
        for (const auto& d : clearedDTCs) std::cout << d << " ";

```

```
207         // EN: set_union — all unique DTCs across both days
208         // TR: set_union — iki gündeki tüm benzersiz DTC'ler
209         std::set<std::string> allDTCs;
210         std::set_union(mondayDTCs.begin(), mondayDTCs.end(),
211                        tuesdayDTCs.begin(), tuesdayDTCs.end(),
212                        std::inserter(allDTCs, allDTCs.begin()));
213         std::cout << "\nAll unique DTCs: ";
214         for (const auto& d : allDTCs) std::cout << d << " ";
215         std::cout << "\n\n";
216     }
217 
218     // ═══════════════════════════════════════════════════════════════════════════════
219     // [UNORDERED CONTAINERS / SIRASIZ KONTEYNERLER]
220     // ═══════════════════════════════════════════════════════════════════════════════
221 
222     // ─── 5. std::unordered_map — O(1) Lookup ──────────────────────────────────────
223     {
224         std::cout << "--- 5. std::unordered_map (hash table) ---\n";
225 
226         std::unordered_map<std::string, int> ecuModules;
227         // EN: Engine Control Module address
228         // TR: Motor Kontrol Modülü adresi
229         ecuModules["ECM"] = 0x7E0;
230         // EN: Transmission Control Module
231         // TR: Şanzıman Kontrol Modülü
232         ecuModules["TCM"] = 0x7E1;
233         // EN: Anti-lock Braking System
234         // TR: Kilitlenme Önleyici Fren Sistemi
235         ecuModules["ABS"] = 0x7B0;
236         // EN: Body Control Module
237         // TR: Gövde Kontrol Modülü
238         ecuModules["BCM"] = 0x740;
239 
240         // EN: O(1) average lookup — much faster than map for large datasets
241         // TR: Ortalama O(1) arama — büyük veri setleri için map'ten çok daha hızlı
242         std::cout << "ECM address: 0x" << std::hex << ecuModules["ECM"]
243                   << std::dec << "\n";
244 
245         // EN: Bucket information — shows hash table internals
246         // TR: Kova bilgisi — hash tablosu iç yapısını gösterir
247         std::cout << "Buckets: " << ecuModules.bucket_count()
248                   << ", Load factor: " << ecuModules.load_factor()
249                   << "\n\n";
250     }
251 
252     // ─── 6. unordered_map with Custom Hash ────────────────────────────────────────
253     {
254         std::cout << "--- 6. unordered_map with Custom Hash ---\n";
255 
256         std::unordered_map<SensorKey, double, SensorKeyHash> sensorMap;
257 
258         // EN: Key = {ECU_ID, SensorIndex}, Value = reading
259         // TR: Anahtar = {ECU_ID, SensörIndeks}, Değer = okuma
260         sensorMap[{0x7E0, 0}] = 92.5;   // ECM, OilTemp
261         sensorMap[{0x7E0, 1}] = 88.0;   // ECM, Coolant
262         sensorMap[{0x7B0, 0}] = 32.0;   // ABS, WheelSpeedFL
263         sensorMap[{0x7B0, 1}] = 33.0;   // ABS, WheelSpeedFR
264 
265         SensorKey query{0x7B0, 0};
266         auto it = sensorMap.find(query);
267         if (it != sensorMap.end()) {
268             std::cout << "ABS WheelSpeedFL: " << it->second << "\n\n";
269         }
270     }
```

```cpp

    // ─── 7. std::unordered_set — Fast Membership Test ──────────────────────────────
    {
        std::cout << "--- 7. std::unordered_set (fast contains) ---\n";

        // EN: Blacklisted VIN prefixes (known recall vehicles)
        // TR: Kara listedeki VIN ön ekleri (bilinen geri çağırma araçları)
        std::unordered_set<std::string> recallVINs = {
            "1G1YY", "2T1BU", "3FA6P", "5YJ3E", "JN1TB"};

        std::string testVIN = "3FA6P";
        // EN: O(1) average — contains() is C++20, use count() for C++17
        // TR: Ortalama O(1) — contains() C++20, C++17 için count() kullan
        if (recallVINs.count(testVIN) > 0) {
            std::cout << "⚠ VIN " << testVIN << " is in recall list!\n\n";
        }
    }

    // ═══════════════════════════════════════════════════════════════════════════════
    // [CONTAINER ADAPTORS / KONTEYNER ADAPTÖRLERİ]
    // ═══════════════════════════════════════════════════════════════════════════════

    // ─── 8. std::stack — LIFO (Undo History) ───────────────────────────────────────
    {
        std::cout << "--- 8. std::stack (LIFO — undo history) ---\n";

        std::stack<std::string> undoHistory;

        // EN: Each user action is pushed onto the stack
        // TR: Her kullanıcı aksiyonu yığına itilir
        undoHistory.push("Set RPM limit to 6000");
        undoHistory.push("Enable traction control");
        undoHistory.push("Set drive mode to Sport");

        std::cout << "Undo sequence:\n";
        while (!undoHistory.empty()) {
            std::cout << "  Undo: " << undoHistory.top() << "\n";
            undoHistory.pop();
        }
        std::cout << "\n";
    }

    // ─── 9. std::queue — FIFO (Message Queue) ──────────────────────────────────────
    {
        std::cout << "--- 9. std::queue (FIFO — message processing) ---\n";

        std::queue<std::string> canBusQueue;

        // EN: CAN bus messages arrive in order and must be processed FIFO
        // TR: CAN bus mesajları sırayla gelir ve FIFO işlenmelidir
        canBusQueue.push("0x7E0: Engine RPM = 3200");
        canBusQueue.push("0x7B0: Wheel Speed = 85");
        canBusQueue.push("0x740: Door Status = Closed");

        std::cout << "Processing CAN messages:\n";
        while (!canBusQueue.empty()) {
            std::cout << "  → " << canBusQueue.front() << "\n";
            canBusQueue.pop();
        }
        std::cout << "\n";
    }

    // ─── 10. std::priority_queue — Alert Dispatcher ────────────────────────────────
    {
        std::cout << "--- 10. std::priority_queue (alert dispatcher) ---\n";

        std::priority_queue<VehicleAlert> alertQueue;

        // EN: Push alerts with different priorities
        // TR: Farklı önceliklerle uyarılar ekle
        alertQueue.push({"Low washer fluid", 5});
        alertQueue.push({"ENGINE OVERHEATING!", 1});
        alertQueue.push({"Tire pressure low", 3});
        alertQueue.push({"BRAKE FAILURE!", 1});
        alertQueue.push({"Service due in 500 km", 4});

```

```
        // EN: Dequeue processes the most urgent (lowest number) first!
        // TR: Kuyruktan çıkarma en acil olanı (en düşük numara) ilk işler!
        std::cout << "Alert processing order:\n";
        while (!alertQueue.empty()) {
            const auto& a = alertQueue.top();
            std::cout << "  [P" << a.priority << "] " << a.message << "\n";
            alertQueue.pop();
        }
        std::cout << "\n";
    }

    // ─── 11. std::map — Structured Bindings & try_emplace ──────────────────────────────────
    {
        std::cout << "--- 11. C++17: try_emplace & extract ---\n";

        std::map<int, std::string> ecuRegistry;
        ecuRegistry[0x7E0] = "ECM";
        ecuRegistry[0x7B0] = "ABS";

        // EN: try_emplace (C++17) — only constructs value if key doesn't exist More efficient
        // than insert for expensive-to-construct values
        // TR: try_emplace (C++17) — anahtar yoksa değeri oluşturur Pahalı oluşturma maliyetli
        // değerler için insert'ten verimli
        auto [it, ok] = ecuRegistry.try_emplace(0x7E0, "DUPLICATE");
        std::cout << "try_emplace 0x7E0: " << (ok ? "inserted" : "exists")
                  << " -> " << it->second << "\n";

        // EN: extract (C++17) — remove a node without destroying it Can be re-inserted with a
        // different key!
        // TR: extract (C++17) — düğümü yok etmeden çıkar Farklı bir anahtarla tekrar
        // eklenebilir!
        auto node = ecuRegistry.extract(0x7B0);
        if (!node.empty()) {
            // EN: Change key!
            // TR: Anahtarı değiştir!
            node.key() = 0x7B1;
            ecuRegistry.insert(std::move(node));
        }

        std::cout << "After extract+reinsert:\n";
        for (const auto& [addr, name] : ecuRegistry) {
            std::cout << "  0x" << std::hex << addr << std::dec
                      << " → " << name << "\n";
        }
    }

    std::cout << "\n=== ASSOCIATIVE CONTAINERS COMPLETE ===\n";
    return 0;
}
```

► 07. String Operations

module_04_advanced/07_string_operations.cpp

Konu: Templates & STL: String Operations & string_view — Şablonlar ve STL: String İşlemleri

Teori / Theory

THEORY: std::string Internals & string_view

——— std::string Memory Model ——— std::string is a dynamic char array with:
1. Heap-allocated buffer (for strings > SSO threshold).
2. size(): number of characters.
3. capacity(): allocated buffer size (can be > size).
4. SSO (Small String Optimization): strings ≤ ~15-22 chars (implementation-dependent) are stored INSIDE the object itself — no heap allocation! This makes short strings very fast.

——— std::string_view (C++17) ——— A non-owning, read-only view into a string. Like a "window" that points to someone else's data without copying it.
✓ No allocation, no copy — just pointer + size
✓ Can view std::string, const char*, string literals
✓ Perfect for function parameters that only READ strings
✗ DANGER: The viewed string must outlive the string_view! If the original string is destroyed → dangling view!

TEORİ: std::string İç Yapısı ve string_view

——— std::string Bellek Modeli ——— std::string, şu özelliklere sahip dinamik bir char dizisidir:
1. Heap tahsisli tampon (SSO eşiğinden büyük stringler için).
2. size(): karakter sayısı.
3. capacity(): tahsis edilmiş tampon boyutu (size'dan büyük olabilir).
4. SSO (Küçük String Optimizasyonu): ≤ ~15-22 karakter stringler (uygulamaya bağlı) nesnenin KENDİSİNDE depolanr — heap tahsisi yok! Bu kısa stringleri çok hızlı yapar.

——— std::string_view (C++17) ——— Stringe sahiplik almayan, salt okunur bir görünüm. Kopyalamadan başkasının verisine işaret eden bir "pencere" gibi.
✓ Tahsis yok, kopya yok — sadece pointer + boyut
✓ std::string, const char*, string sabitleri görüntüleyebilir
✓ Sadece string OKUYAN fonksiyon parametreleri için mükemmel
✗ TEHLİKE: Görüntülenen string, string_view'dan uzun yaşamalı! Orijinal string yok edilirse → sarkan (dangling) görünüm!

CPPREF Depth: std::string SSO — Small String Optimization

Most standard library implementations use SSO (Small String Optimization): strings up to ~15-22 characters (implementation- dependent) are stored IN-PLACE within the std::string object itself — no heap allocation at all. Only when the string exceeds this threshold does a heap allocation occur. Typical layout: `sizeof(std::string)` is 32 bytes (GCC/libstdc++) with SSO buffer of 15 chars + null terminator. `std::string_view` (C++17) is a non-owning view: just a pointer + length. O(1) construction, never allocates, never copies. DANGER: the viewed data MUST outlive the string_view! Returning a string_view to a local std::string → dangling view → UB. This is one of the most common C++17 bugs. `std::string::reserve(n)` pre-allocates n bytes of capacity. `std::string::shrink_to_fit()` is a non-binding request to free excess capacity. C++20 adds `constexpr std::string` — compile-time string operations with transient allocation.

CPPREF Derinlik: std::string SSO — Küçük String Optimizasyonu

Çoğu standart kütüphane SSO (Small String Optimization) kullanır: ~15-22 karaktere kadar olan stringler std::string nesnesinin KENDİSİNDE saklanır — hiç heap tahsisi yapılmaz. Yalnızca bu eşik aşıldığında heap'ten bellek ayrılır. Tipik düzen: `sizeof(std::string)` GCC/libstdc++ ile 32 bayttır, 15 karakter + null sonlandırıcı SSO tamponuyla. `std::string_view` (C++17) sahipsiz bir görünümdür: sadece bir işaretçi + uzunluk. O(1) inşa, asla tahsis etmez, asla kopyalamaz. TEHLİKE: görüntülenen veri, string_view'dan UZUN YAŞAMALI! Yerel bir std::string'e string_view döndürmek → sarkan görünüm → tanımsız davranış. Bu, en yaygın C++17 hatalarından biridir. `reserve(n)` n bayt kapasite önceden ayırır. `shrink_to_fit()` fazla kapasiteyi serbest bırakma isteğidir (bağlayıcı değil). C++20 `constexpr std::string` ekler — derleme zamanı string işlemleri.

Kaynak Kod / Source Code

```cpp
#include <algorithm>    // EN: STL algorithms
#include <cctype>       // EN: std::toupper, std::tolower, std::isdigit
#include <iostream>     // EN: Standard I/O
#include <sstream>      // EN: std::istringstream, std::ostringstream
#include <string>       // EN: std::string
#include <string_view>  // EN: std::string_view (C++17)
#include <vector>       // EN: std::vector

// ═══════════════════════════════════════════════════════════════════════════
// [PRODUCTION CODE / ÜRETİM KODU]
// ═══════════════════════════════════════════════════════════════════════════

// ─── 1. String Splitting (Tokenization) ──────────────────────────────────

// EN: Split a CAN bus log line by delimiter. No regex needed — just std::string::find() and
// substr(). This is the most common string operation in embedded log parsing.
// TR: CAN bus log satırını ayırıcıya göre böl. Regex gerekmez — sadece std::string::find() ve
// substr(). Gömülü log ayrıştırmada en yaygın string işlemidir.
std::vector<std::string> split(const std::string& str, char delimiter) {
    std::vector<std::string> tokens;
    std::string::size_type start = 0;
    std::string::size_type end = str.find(delimiter);

    while (end != std::string::npos) {
        tokens.push_back(str.substr(start, end - start));
        start = end + 1;
        end = str.find(delimiter, start);
    }
    tokens.push_back(str.substr(start));
    return tokens;
}

// ─── 2. String Replace All ────────────────────────────────────────────────

// EN: Replace all occurrences of a substring. Useful for sanitizing diagnostic messages before
// logging.
// TR: Bir alt dizenin tüm tekrarlarını değiştir. Tanılama mesajlarını loglama öncesi temizlemek
// için kullanışlı.
std::string replaceAll(std::string str, const std::string& from,
                       const std::string& to) {
    if (from.empty()) return str;
    size_t pos = 0;
    while ((pos = str.find(from, pos)) != std::string::npos) {
        str.replace(pos, from.length(), to);
        pos += to.length();
    }
    return str;
}
```

```cpp

// ─── 3. string_view Utility Functions ─────────────────────────────────────────

// EN: Extract the DTC category from a code like "P0301". Uses string_view to avoid any
// allocation — just creates a view into the first character.
// TR: "P0301" gibi bir koddan DTC kategorisini çıkar. Herhangi bir tahsitten kaçınmak için
// string_view kullanır — sadece ilk karaktere bir görünüm oluşturur.
std::string_view getDTCCategory(std::string_view code) {
    // EN: P=Powertrain, B=Body, C=Chassis, U=Network
    // TR: P=Motor/Şanzıman, B=Gövde, C=Şasi, U=Ağ
    if (code.empty()) return "Unknown";
    switch (code[0]) {
        case 'P': return "Powertrain";
        case 'B': return "Body";
        case 'C': return "Chassis";
        case 'U': return "Network";
        default:  return "Unknown";
    }
}

// EN: Trim whitespace from both sides (using string_view — zero copy!)
// TR: Her iki taraftan boşlukları kes (string_view ile — sıfır kopya!)
std::string_view trim(std::string_view sv) {
    while (!sv.empty() && std::isspace(static_cast<unsigned char>(sv.front()))) {
        sv.remove_prefix(1);
    }
    while (!sv.empty() && std::isspace(static_cast<unsigned char>(sv.back()))) {
        sv.remove_suffix(1);
    }
    return sv;
}

// EN: Check if a VIN (Vehicle Identification Number) is valid format. Uses string_view as
// parameter — accepts string, const char*, literal.
// TR: VIN (Araç Kimlik Numarası) formatının geçerli olup olmadığını kontrol et. Parametre olarak
// string_view kullanır — string, const char*, sabit kabul eder.
bool isValidVIN(std::string_view vin) {
    if (vin.size() != 17) return false;
    // EN: VIN must be alphanumeric and not contain I, O, Q
    // TR: VIN alfanümerik olmalı ve I, O, Q içermemeli
    return std::all_of(vin.begin(), vin.end(), [](char c) {
        return std::isalnum(static_cast<unsigned char>(c)) &&
               c != 'I' && c != 'O' && c != 'Q';
    });
}

// ═══════════════════════════════════════════════════════════════════════════════
// [EXAMPLES / ÖRNEKLER]
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "=== MODULE 4: STRING OPERATIONS & STRING_VIEW ===\n"
              << "=== MODÜL 4: STRİNG İŞLEMLERİ VE STRİNG_VIEW ===\n\n";

    // ─── 1. Basic String Operations ─────────────────────────────────────────
    {
        std::cout << "--- 1. Basic String Operations ---\n";

        std::string vin = "1G1YY22G965";

        // EN: Append, concatenate
        // TR: Ekleme, birleştirme
        vin += "123456";
        std::cout << "VIN: " << vin << " (length=" << vin.size() << ")\n";

        // EN: substr — extract a portion
        // TR: substr — bir bölüm çıkar
        std::string wmi = vin.substr(0, 3);     // World Manufacturer ID
        std::string vds = vin.substr(3, 5);     // Vehicle Descriptor Section
        std::cout << "WMI: " << wmi << ", VDS: " << vds << "\n";

        // EN: compare
        // TR: karşılaştır
        std::string other = "1G1YY22G965123456";
        std::cout << "Equal? " << (vin == other ? "yes" : "no") << "\n";
        std::cout << "Compare: " << vin.compare(other) << "\n\n";
    }
```

```cpp

    // ─── 2. Search Operations ──────────────────────────────────────────────────────
    {
        std::cout << "--- 2. String Search ---\n";

        std::string logLine = "[2026-03-06 14:22:05] ECM P0301: Cylinder 1 Misfire Detected";

        // EN: find — returns index of first occurrence (or npos)
        // TR: find — ilk bulunma indeksini döndürür (veya npos)
        auto dtcPos = logLine.find("P0");
        if (dtcPos != std::string::npos) {
            std::string dtcCode = logLine.substr(dtcPos, 5);
            std::cout << "Found DTC: " << dtcCode << " at position " << dtcPos << "\n";
        }

        // EN: rfind — search backwards (find LAST occurrence)
        // TR: rfind — geriye doğru ara (SON tekrarı bul)
        auto lastColon = logLine.rfind(':');
        std::cout << "Text after last colon: " << logLine.substr(lastColon + 2) << "\n";

        // EN: find_first_of — find any character from a set
        // TR: find_first_of — bir kümeden herhangi bir karakteri bul
        auto firstDigit = logLine.find_first_of("0123456789");
        std::cout << "First digit at index: " << firstDigit << "\n";

        // EN: find_first_not_of — find first char NOT in set
        // TR: find_first_not_of — kümede OLMAYAN ilk karakteri bul
        std::string padded = "   trimmed text   ";
        auto contentStart = padded.find_first_not_of(' ');
        auto contentEnd = padded.find_last_not_of(' ');
        std::cout << "Trimmed: \"" << padded.substr(contentStart, contentEnd - contentStart + 1)
                  << "\"\n\n";
    }

    // ─── 3. Modification Operations ────────────────────────────────────────────────
    {
        std::cout << "--- 3. String Modification ---\n";

        std::string msg = "Engine temperature is CRITICAL at 120C";

        // EN: replace a range
        // TR: bir aralığı değiştir
        msg.replace(msg.find("CRITICAL"), 8, "WARNING");
        std::cout << "After replace: " << msg << "\n";

        // EN: erase characters
        // TR: karakterleri sil
        std::string serial = "SN-12345-AB";
        serial.erase(0, 3);  // Remove "SN-"
        std::cout << "Erased prefix: " << serial << "\n";

        // EN: insert at position
        // TR: pozisyona ekle
        serial.insert(0, "VH-");
        std::cout << "Inserted prefix: " << serial << "\n";

        // EN: Transform to uppercase using std::transform
        // TR: std::transform ile büyük harfe dönüştür
        std::string ecuName = "engine_control_module";
        std::transform(ecuName.begin(), ecuName.end(), ecuName.begin(),
                       [](unsigned char c) { return std::toupper(c); });
        std::cout << "Uppercase: " << ecuName << "\n\n";
    }
```

```cpp

    // ─── 4. Split / Tokenize ──────────────────────────────────────────────────
    {
        std::cout << "--- 4. Split (tokenize) ---\n";

        // EN: Parse a CAN log line: "timestamp|ECU_ID|data_bytes"
        // TR: CAN log satırını ayrıştır: "zaman_damgası|ECU_ID|veri_baytları"
        std::string canLog = "14:22:05.123|0x7E0|03 41 0D 7A";

        auto parts = split(canLog, '|');
        std::cout << "Timestamp: " << parts[0] << "\n";
        std::cout << "ECU ID:    " << parts[1] << "\n";
        std::cout << "Data:      " << parts[2] << "\n";

        // EN: Alternative: split with std::istringstream
        // TR: Alternatif: std::istringstream ile böl
        std::string csv = "85,120,72,91,68";
        std::istringstream iss(csv);
        std::string token;
        std::cout << "CSV values: ";
        while (std::getline(iss, token, ',')) {
            std::cout << token << " ";
        }
        std::cout << "\n\n";
    }

    // ─── 5. Replace All ───────────────────────────────────────────────────────
    {
        std::cout << "--- 5. Replace All ---\n";

        std::string template_msg = "Vehicle {VIN} has {DTC_COUNT} active DTCs";
        std::string result = replaceAll(template_msg, "{VIN}", "1G1YY22G965123456");
        result = replaceAll(result, "{DTC_COUNT}", "3");
        std::cout << "Formatted: " << result << "\n\n";
    }

    // ─── 6. std::string_view — Zero-Copy Reading ──────────────────────────────
    {
        std::cout << "--- 6. std::string_view (zero-copy) ---\n";

        // EN: string_view doesn't allocate — just points to existing data
        // TR: string_view tahsis etmez — mevcut veriye işaret eder
        std::string fullLog = "[2026-03-06 14:22:05] P0301 Cylinder 1 Misfire";

        // EN: Create views into different parts WITHOUT copying
        // TR: Kopyalamadan farklı bölümlere görünümler oluştur
        std::string_view dateView(fullLog.data() + 1, 10);   // "2026-03-06"
        std::string_view timeView(fullLog.data() + 12, 8);   // "14:22:05"
        std::string_view codeView(fullLog.data() + 22, 5);   // "P0301"

        std::cout << "Date: " << dateView << "\n";
        std::cout << "Time: " << timeView << "\n";
        std::cout << "Code: " << codeView << "\n";
        std::cout << "Category: " << getDTCCategory(codeView) << "\n\n";

        // EN: string_view works with string literals too (no std::string needed!)
        // TR: string_view string sabitleri ile de çalışır (std::string gerekmez!)
        std::cout << "P0420 category: " << getDTCCategory("P0420") << "\n";
        std::cout << "B0100 category: " << getDTCCategory("B0100") << "\n";
        std::cout << "U0100 category: " << getDTCCategory("U0100") << "\n\n";
    }

    // ─── 7. string_view — trim and validate ───────────────────────────────────
    {
        std::cout << "--- 7. string_view trim & validate ---\n";

        std::string_view padded = "   Hello World   ";
        std::cout << "Before trim: \"" << padded << "\"\n";
        std::cout << "After trim:  \"" << trim(padded) << "\"\n";

```

```
        // EN: VIN validation using string_view (no copies!)
        // TR: string_view ile VIN doğrulama (kopya yok!)
        std::cout << "VIN 'WBA3A5G59DNP12345' valid? "
                  << (isValidVIN("WBA3A5G59DNP12345") ? "yes" : "no") << "\n";
        std::cout << "VIN 'INVALID_SHORT' valid? "
                  << (isValidVIN("INVALID_SHORT") ? "yes" : "no") << "\n";
        std::cout << "VIN 'WBAOI5G59DNP12345' valid? "  // Contains 'O' and 'I'
                  << (isValidVIN("WBAOI5G59DNP12345") ? "yes" : "no") << "\n\n";
    }

    // ─── 8. std::ostringstream — String Building ──────────────────────────────
    {
        std::cout << "--- 8. std::ostringstream (string builder) ---\n";

        // EN: Build a formatted diagnostic report without repeated concatenation
        // TR: Tekrarlanan birleştirme olmadan biçimlendirilmiş teşhis raporu oluştur
        std::ostringstream report;
        report << "=== DIAGNOSTIC REPORT ===\n";
        report << "VIN: 1G1YY22G965123456\n";
        report << "DTCs: P0301, P0420\n";
        report << "Battery: " << 12.6 << "V\n";
        report << "Oil Temp: " << 92 << "°C\n";

        std::string finalReport = report.str();
        std::cout << finalReport;
        std::cout << "Report size: " << finalReport.size() << " bytes\n\n";
    }

    // ─── 9. SSO (Small String Optimization) Proof ─────────────────────────────
    {
        std::cout << "--- 9. SSO (Small String Optimization) ---\n";

        // EN: Likely SSO (no heap)
        // TR: Muhtemelen SSO (heap yok)
        std::string shortStr = "Hi";
        std::string longStr = "This is a much longer string that exceeds SSO threshold";

        std::cout << "Short string:\n";
        std::cout << "  size=" << shortStr.size()
                  << " capacity=" << shortStr.capacity()
                  << " sizeof=" << sizeof(shortStr) << "\n";

        std::cout << "Long string:\n";
        std::cout << "  size=" << longStr.size()
                  << " capacity=" << longStr.capacity()
                  << " sizeof=" << sizeof(longStr) << "\n";

        // EN: sizeof is always the SAME (typically 32 bytes on 64-bit). Short strings fit inside
        // those 32 bytes (SSO). Long strings allocate on the heap — capacity > sizeof.
        // TR: sizeof HER ZAMAN AYNI (genellikle 64-bit'te 32 byte). Kısa stringler bu 32 byte'a
        // sığar (SSO). Uzun stringler heap'te tahsis edilir — capacity > sizeof.
        std::cout << "\n";
    }

    // ─── 10. String Comparison and starts_with (C++20 backport) ────────────────
    {
        std::cout << "--- 10. Prefix/Suffix Check ---\n";

        std::string dtc = "P0301";

        // EN: C++17 approach: compare substr
        // TR: C++17 yaklaşımı: substr karşılaştır
        bool isPowertrain = (dtc.compare(0, 1, "P") == 0);
        bool isMisfire = (dtc.compare(dtc.size() - 2, 2, "01") == 0);

        std::cout << dtc << " isPowertrain? " << (isPowertrain ? "yes" : "no") << "\n";
        std::cout << dtc << " isMisfire? " << (isMisfire ? "yes" : "no") << "\n";

        // EN: Using string_view for efficient prefix matching
        // TR: Verimli önek eşleştirme için string_view kullanma
        std::vector<std::string> allDTCs = {"P0301", "P0420", "B0100", "C0035", "U0100"};
        std::cout << "Powertrain codes: ";
```

```
        for (const auto& code : allDTCs) {
            std::string_view sv(code);
            if (!sv.empty() && sv[0] == 'P') {
                std::cout << code << " ";
            }
        }
        std::cout << "\n";
    }

    std::cout << "\n=== STRING OPERATIONS COMPLETE ===\n";
    return 0;
}
```

► 08. Ranges And Views

module_04_advanced/08_ranges_and_views.cpp

Konu: C++20 Ranges & Views — Educational C++17 Implementation — C++20 Aralıklar ve Görünümler

Teori / Theory

THEORY: C++20 Ranges & Views — The Pipeline Revolution

1. What Are Ranges?

A Range is any type that exposes begin() and end(). C++20 formalises this with concepts like std::ranges::range and std::ranges::random_access_range. Ranges replace the old iterator-pair idiom with a single object that *is* the sequence.

2. Views — Lazy, Non-Owning Wrappers

A View is a lightweight, non-owning wrapper around a range. Views are cheap to copy and never own the underlying data. Key views:
• filter(pred) — keep elements satisfying predicate
• transform(fn) — apply function to each element
• take(n) — first N elements
• drop(n) — skip first N elements
• reverse — reverse iteration order

3. Pipe Syntax

The pipe operator `|` chains views left-to-right, reading like a processing pipeline on a factory floor: sensorData | views::filter(isValid) | views::transform(calibrate) Each stage lazily yields elements to the next — no intermediate containers are created.

4. Why Ranges Matter

• Eliminate temporary containers between algorithm stages
• Improve readability — declarative, not imperative
• Lazy evaluation — compute only what is consumed
• Composable — small pieces snap together like LEGO bricks

5. C++17 Workaround Strategy

Under C++17 we lack <ranges>, so we emulate the pattern: 1. Template helper functions: filterAndTransform<Container, Pred, Fn> 2. Iterator-based LazyRange<T> that defers computation
3. Pipeline builder class that chains stages via std::function All code below compiles with -std=c++17 -Wall -Wextra -Wpedantic.

TEORİ: C++20 Aralıklar ve Görünümler — Boru Hattı Devrimi

1. Aralık Nedir?

Bir Aralık (Range), begin() ve end() sunan herhangi bir tiptir. C++20 bunu std::ranges::range ve std::ranges::random_access_range gibi kavramlarla (concepts) resmîleştirir. Aralıklar, eski yineleyici-çifti deyimini, dizinin *kendisi* olan tek bir nesneyle değiştirir.

2. Görünümler — Tembel, Sahiplik Almayan Sarmalayıcılar

Bir Görünüm (View), aralık etrafındaki hafif, sahiplik almayan bir sarmalayıcıdır. Kopyalaması ucuzdur ve alttaki veriye sahip olmaz:
• filter(pred) — koşulu sağlayan elemanları tut
• transform(fn) — her elemana fonksiyon uygula
• take(n) — ilk N eleman
• drop(n) — ilk N elemanı atla
• reverse — ters sırada yinele

3. Boru Sözdizimi

Boru operatörü `|`, görünümleri soldan sağa zincirler; fabrika bandındaki bir işleme hattı gibi okunur: sensorData | views::filter(isValid) | views::transform(calibrate) Her aşama tembel (lazy) olarak bir sonrakine eleman verir — ara konteyner oluşturulmaz.

4. Aralıklar Neden Önemli

• Algoritma aşamaları arasındaki geçici konteynerleri ortadan kaldırır
• Okunabilirliği artırır — bildirimsel, zorunlu değil
• Tembel değerlendirme — yalnızca tüketilen hesaplanır
• Bileşenleştirilebilir — küçük parçalar LEGO gibi birleşir

5. C++17 Geçici Çözüm Stratejisi

C++17'de <ranges> olmadığından, aşağıdaki stratejiyle öykünüyoruz: 1. Şablon yardımcı fonksiyonlar: filterAndTransform<Container, Pred, Fn> 2. Hesaplamayı erteleyen yineleyici tabanlı LazyRange<T> 3. std::function ile aşamaları zincirleyen boru hattı oluşturucu sınıf Aşağıdaki tüm kod -std=c++17 -Wall -Wextra -Wpedantic ile derlenir.
Standart: C++17 (C++20 sections in comments only) Derleme : g++ -std=c++17 -Wall -Wextra -Wpedantic 08_ranges_and_views.cpp

CPPREF Depth: Lazy Evaluation and View Composition — Zero-Overhead Abstraction

C++20 views are lazy — they produce elements on demand with NO intermediate containers allocated. `views::filter | views::transform` creates a composed view object at COMPILE TIME; the pipeline is a single type encoding the entire chain of operations. Views satisfy O(1) construction and move — they store only the source range reference plus the predicate/transform objects. `views::iota(0)` is an INFINITE range — safe because laziness means elements are generated only when consumed (e.g., via `take(10)`). Borrowed range vs owning range: a borrowed range's iterators remain valid even after the range object is destroyed (e.g., `std::span`, `std::string_view`). Functions like `ranges::find` return `std::ranges::dangling` if given a non-borrowed temporary — compile- time dangling protection! In C++17 (pre-ranges), similar laziness can be achieved with:
• Custom iterator pairs wrapping filter/transform logic
• Pipeline builder classes chaining `std::function` stages
• Coroutine-like generators (Boost.Coroutine2 or custom)

CPPREF Derinlik: Tembel Değerlendirme ve Görünüm Bileşimi — Sıfır Maliyetli Soyutlama

C++20 view'ları tembeldir — elemanları talep üzerine üretir, ara konteyner OLUŞTURMAZ. `views::filter | views::transform` DERLEME ZAMANINDA bileşik bir view nesnesi oluşturur; boru hattı tüm işlem zincirini kodlayan tek bir tiptir. View'lar O(1) inşa ve taşıma garantisi sunar — yalnızca kaynak aralık referansı ile yüklem/dönüşüm nesnelerini saklar. `views::iota(0)` SONSUZ bir aralıktır — tembellik sayesinde elemanlar yalnızca tüketildiğinde üretilir (ör. `take(10)` ile). Ödünç alınmış aralık (borrowed range) vs sahipli aralık: ödünç alınmış aralığın iteratörleri, aralık nesnesi yok edildikten sonra bile geçerli kalır (ör. `std::span`, `std::string_view`). `ranges::find` gibi fonksiyonlar, ödünç alınmamış geçici nesne verilirse `std::ranges::dangling` döner — derleme zamanında sarkan iteratör koruması! C++17'de (ranges öncesi) benzer tembellik şu yollarla sağlanabilir:
• Filtre/dönüşüm mantığını saran özel iteratör çiftleri
• `std::function` aşamalarını zincirleyen boru hattı sınıfları
• Coroutine benzeri üreteçler (Boost.Coroutine2 veya özel)

Kaynak Kod / Source Code

```cpp
#include <algorithm>   // EN: STL algorithms (copy_if, transform)
#include <cstdint>     // EN: Fixed-width integers
#include <functional>  // EN: std::function for pipeline stages
#include <iomanip>     // EN: Output formatting
#include <iostream>    // EN: Console I/O
#include <iterator>    // EN: Iterator utilities
#include <numeric>     // EN: Accumulate, iota
#include <sstream>     // EN: String stream for lazy range
#include <string>      // EN: String operations
#include <vector>      // EN: Dynamic array container

// #include <ranges>  // C++20 — not used; all code below is C++17-compatible

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 1 — filterAndTransform: C++17 Pipeline Helper
// ═══════════════════════════════════════════════════════════════════════════

// EN: Generic filter + transform pipeline — C++17 equivalent of:
// TR: Genel filtre + dönüştürme boru hattı — C++17 karşılığı: C++20: container |
// views::filter(pred) | views::transform(fn)
template <typename Container, typename Predicate, typename Transform>
auto filterAndTransform(const Container& input, Predicate pred, Transform fn) {
    using OutputType = decltype(fn(*input.begin()));
    std::vector<OutputType> result;
    result.reserve(input.size());
    for (const auto& elem : input) {
        if (pred(elem)) {
            result.push_back(fn(elem));
        }
    }
    return result;
}
```

```cpp

// EN: Take the first N elements from a container
// TR: Bir konteynerden ilk N elemanı al
template <typename Container>
auto takeN(const Container& input, std::size_t n) {
    using ValueType = typename Container::value_type;
    std::vector<ValueType> result;
    std::size_t count = 0;
    for (const auto& elem : input) {
        if (count >= n) break;
        result.push_back(elem);
        ++count;
    }
    return result;
}

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 2 — LazyRange: Deferred Computation (C++17 Iterator Approach)
// ═══════════════════════════════════════════════════════════════════════════════

// EN: LazyRange defers filter + transform until iteration — mimics view laziness
// TR: LazyRange, filtre + dönüşümü yinelemeye kadar erteler — görünüm tembelliğini taklit eder
template <typename T>
class LazyRange {
public:
    using FilterFn    = std::function<bool(const T&)>;
    using TransformFn = std::function<T(const T&)>;

    LazyRange(const std::vector<T>& source, FilterFn filter, TransformFn transform)
        : source_(source), filter_(std::move(filter)), transform_(std::move(transform)) {}

    // ─── Inner Iterator ──────────────────────────────────────────────────────
    class Iterator {
    public:
        using iterator_category = std::input_iterator_tag;
        using value_type        = T;
        using difference_type   = std::ptrdiff_t;
        using pointer           = const T*;
        using reference         = const T&;

        Iterator(const LazyRange* owner, std::size_t pos)
            : owner_(owner), pos_(pos) { advance(); }

        T operator*() const { return owner_->transform_(owner_->source_[pos_]); }

        Iterator& operator++() { ++pos_; advance(); return *this; }
        Iterator operator++(int) { auto tmp = *this; ++(*this); return tmp; }

        bool operator==(const Iterator& o) const { return pos_ == o.pos_; }
        bool operator!=(const Iterator& o) const { return !(*this == o); }

    private:
        void advance() {
            while (pos_ < owner_->source_.size() &&
                   !owner_->filter_(owner_->source_[pos_])) {
                ++pos_;
            }
        }
        const LazyRange* owner_;
        std::size_t pos_;
    };

    Iterator begin() const { return Iterator(this, 0); }
    Iterator end()   const { return Iterator(this, source_.size()); }

    // EN: Materialise lazily-computed results into a vector
    // TR: Tembel hesaplanmış sonuçları bir vektöre topla
    std::vector<T> collect() const {
        return std::vector<T>(begin(), end());
    }

private:
    const std::vector<T>& source_;
    FilterFn              filter_;
    TransformFn           transform_;
};
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 3 — SensorPipeline: Multi-Stage Processing Builder
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Represents a raw automotive sensor reading (e.g., throttle position)
// TR: Ham bir otomotiv sensör okumasını temsil eder (ör. gaz kelebeği konumu)
struct SensorReading {
    int      sensorId;    // EN: Unique sensor identifier / TR: Benzersiz sensör tanımlayıcı
    double   rawValue;    // EN: Raw ADC value / TR: Ham ADC değeri
    bool     valid;       // EN: Validity flag / TR: Geçerlilik bayrağı
};

// EN: SensorPipeline chains: filter invalid → scale → clamp → collect
// TR: SensorPipeline zinciri: geçersiz filtrele → ölçekle → sınırla → topla
class SensorPipeline {
public:
    using Stage = std::function<std::vector<double>(const std::vector<double>&)>;

    // EN: Add a processing stage to the pipeline
    // TR: Boru hattına bir işleme aşaması ekle
    SensorPipeline& addStage(Stage stage) {
        stages_.push_back(std::move(stage));
        return *this;
    }

    // EN: Execute all stages sequentially on the input
    // TR: Tüm aşamaları girdi üzerinde sıralı çalıştır
    [[nodiscard]] std::vector<double> execute(const std::vector<double>& input) const {
        auto data = input;
        for (const auto& stage : stages_) {
            data = stage(data);
        }
        return data;
    }

private:
    std::vector<Stage> stages_;
};

// ─── Pipeline Stage Factories ──────────────────────────────────────────────

// EN: Create a filter stage that keeps elements satisfying pred
// TR: Koşulu sağlayan elemanları tutan bir filtre aşaması oluştur
inline auto makeFilterStage(std::function<bool(double)> pred) {
    return [pred = std::move(pred)](const std::vector<double>& in) {
        std::vector<double> out;
        std::copy_if(in.begin(), in.end(), std::back_inserter(out), pred);
        return out;
    };
}

// EN: Create a transform stage that applies fn to each element
// TR: Her elemana fn uygulayan bir dönüşüm aşaması oluştur
inline auto makeTransformStage(std::function<double(double)> fn) {
    return [fn = std::move(fn)](const std::vector<double>& in) {
        std::vector<double> out(in.size());
        std::transform(in.begin(), in.end(), out.begin(), fn);
        return out;
    };
}

// EN: Create a take-N stage that keeps only the first N elements
// TR: Yalnızca ilk N elemanı tutan bir alma aşaması oluştur
inline auto makeTakeStage(std::size_t n) {
    return [n](const std::vector<double>& in) {
        auto count = std::min(n, in.size());
        return std::vector<double>(in.begin(), in.begin() + static_cast<std::ptrdiff_t>(count));
    };
}

```

```cpp
// ═══════════════════════════════════════════════════════════════════════════
// SECTION 4 — Classic STL vs Range-Style Comparison
// ═══════════════════════════════════════════════════════════════════════════

// EN: Classic STL approach — requires intermediate containers
// TR: Klasik STL yaklaşımı — ara konteynerler gerektirir
inline std::vector<double> classicApproach(const std::vector<SensorReading>& readings) {
    // EN: Step 1 — filter valid readings
    // TR: Adım 1 — geçerli okumaları filtrele
    std::vector<SensorReading> validReadings;
    std::copy_if(readings.begin(), readings.end(),
                 std::back_inserter(validReadings),
                 [](const SensorReading& r) { return r.valid; });

    // EN: Step 2 — extract and calibrate raw values
    // TR: Adım 2 — ham değerleri çıkar ve kalibre et
    std::vector<double> calibrated(validReadings.size());
    std::transform(validReadings.begin(), validReadings.end(),
                   calibrated.begin(),
                   [](const SensorReading& r) { return r.rawValue * 0.01; });
    return calibrated;
}

// EN: Range-style approach in C++17 — single pass with filterAndTransform
// TR: C++17'de aralık tarzı yaklaşım — filterAndTransform ile tek geçiş
inline std::vector<double> rangeStyleApproach(const std::vector<SensorReading>& readings) {
    /*  C++20 equivalent / C++20 karşılığı:
     *  auto result = readings
     *      | std::views::filter([](const SensorReading& r) { return r.valid; })
     *      | std::views::transform([](const SensorReading& r) { return r.rawValue * 0.01; });
     */
    return filterAndTransform(
        readings,
        [](const SensorReading& r) { return r.valid; },
        [](const SensorReading& r) { return r.rawValue * 0.01; }
    );
}

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 5 — Utility Helpers
// ═══════════════════════════════════════════════════════════════════════════

// EN: Print a vector with a label
// TR: Bir vektörü etiketle yazdır
template <typename T>
void printVec(const std::string& label, const std::vector<T>& v) {
    std::cout << label << ": [";
    for (std::size_t i = 0; i < v.size(); ++i) {
        if (i > 0) std::cout << ", ";
        std::cout << std::fixed << std::setprecision(2) << v[i];
    }
    std::cout << "]\n";
}

// ═══════════════════════════════════════════════════════════════════════════
// main() — Demonstrations
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "═══════════════════════════════════════════════════════\n";
    std::cout << " C++20 Ranges & Views — C++17 Educational Demos\n";
    std::cout << "═══════════════════════════════════════════════════════\n\n";

    // ─── Demo 1: Filter + Transform Pipeline (C++17 Working) ──────────────
    // EN: Filter valid sensor readings, then calibrate
    // TR: Geçerli sensör okumalarını filtrele, ardından kalibre et
    {
        std::cout << "── Demo 1: Filter + Transform Pipeline ──\n";

        std::vector<SensorReading> readings = {
            {1, 512.0, true},  {2, -10.0, false}, {3, 1023.0, true},
            {4, 0.0, true},    {5, 750.0, false},  {6, 200.0, true}
        };

```

```
253         auto calibrated = filterAndTransform(
254             readings,
255             [](const SensorReading& r) { return r.valid; },
256             [](const SensorReading& r) { return r.rawValue * 0.01; }
257         );
258
259         /*  C++20 equivalent / C++20 karşılığı:
260          *  auto calibrated = readings
261          *      | std::views::filter([](auto& r){ return r.valid; })
262          *      | std::views::transform([](auto& r){ return r.rawValue * 0.01; });
263          */
264
265         printVec("  Calibrated values", calibrated);
266         std::cout << '\n';
267     }
268
269     // ─── Demo 2: C++20 Equivalent (Comment Only) ──────────────────────────────────────
270     // EN: The following C++20 code is shown for reference only
271     // TR: Aşağıdaki C++20 kodu yalnızca referans amaçlıdır
272     {
273         std::cout << "── Demo 2: C++20 Equivalent (in comments) ──\n";
274         std::cout << "  // #include <ranges>\n";
275         std::cout << "  // auto result = sensorData\n";
276         std::cout << "  //     | std::views::filter(isValid)\n";
277         std::cout << "  //     | std::views::transform(calibrate)\n";
278         std::cout << "  //     | std::views::take(5);\n";
279         std::cout << "  (See source comments for full C++20 syntax)\n\n";
280
281         /*  Full C++20 pipeline / Tam C++20 boru hattı:
282          *
283          *  #include <ranges>
284          *  namespace views = std::views;
285          *
286          *  auto isValid   = [](const SensorReading& r) { return r.valid; };
287          *  auto calibrate = [](const SensorReading& r) { return r.rawValue * 0.01; };
288          *
289          *  std::vector<SensorReading> sensorData = { ... };
290          *
291          *  auto result = sensorData
292          *      | views::filter(isValid)
293          *      | views::transform(calibrate)
294          *      | views::take(5);
295          *
296          *  for (double v : result) std::cout << v << '\n';
297          */
298     }
299
300     // ─── Demo 3: Multi-Stage Pipeline — filter → transform → take N ──────────────────
301     // EN: Build a pipeline: discard negatives → multiply by 2 → take first 3
302     // TR: Boru hattı kur: negatifleri at → 2 ile çarp → ilk 3'ü al
303     {
304         std::cout << "── Demo 3: Multi-Stage Pipeline ──\n";
305
306         std::vector<double> rawSignals = {
307             -5.0, 12.0, 3.5, -1.0, 8.8, 22.1, 6.0, -0.5, 15.3
308         };
309
310         /*  C++20 equivalent / C++20 karşılığı:
311          *  auto result = rawSignals
312          *      | views::filter([](double v){ return v >= 0.0; })
313          *      | views::transform([](double v){ return v * 2.0; })
314          *      | views::take(3);
315          */
316
317         SensorPipeline pipeline;
318         pipeline
319             .addStage(makeFilterStage([](double v) { return v >= 0.0; }))
320             .addStage(makeTransformStage([](double v) { return v * 2.0; }))
321             .addStage(makeTakeStage(3));
322
323         auto result = pipeline.execute(rawSignals);
324         printVec("  Pipeline output (first 3 non-negative × 2)", result);
325         std::cout << '\n';
326     }
```

```cpp

    // ─── Demo 4: Lazy Range vs Eager Evaluation ──────────────────────────────────────
    // EN: Compare lazy (deferred) vs eager (immediate) computation
    // TR: Tembel (ertelenmiş) ile istekli (anlık) hesaplamayı karşılaştır
    {
        std::cout << "── Demo 4: Lazy vs Eager Evaluation ──\n";

        std::vector<double> temperatures = {
            -40.0, 25.5, 105.3, 72.0, -10.0, 88.9, 150.0, 36.6
        };

        // EN: Predicate — keep temperatures in valid ECU operating range [-30, 125]
        // TR: Koşul — geçerli ECU çalışma aralığındaki sıcaklıkları tut [-30, 125]
        auto inRange = [](const double& t) { return t >= -30.0 && t <= 125.0; };

        // EN: Transform — Celsius to Fahrenheit
        // TR: Dönüşüm — Celsius'tan Fahrenheit'a
        auto toFahrenheit = [](const double& t) { return t * 9.0 / 5.0 + 32.0; };

        // --- Eager approach (C++17 filterAndTransform) ---
        // EN: Computes all results immediately into a new vector
        // TR: Tüm sonuçları hemen yeni bir vektöre hesaplar
        auto eagerResult = filterAndTransform(temperatures, inRange, toFahrenheit);
        printVec("  Eager (all at once)", eagerResult);

        // --- Lazy approach (LazyRange) ---
        // EN: Nothing is computed until we iterate or collect
        // TR: Yineleyinceye veya toplayana kadar hiçbir şey hesaplanmaz
        LazyRange<double> lazyView(temperatures, inRange, toFahrenheit);

        std::cout << "  Lazy  (first 3)  : [";
        int count = 0;
        for (auto it = lazyView.begin(); it != lazyView.end() && count < 3; ++it, ++count) {
            if (count > 0) std::cout << ", ";
            std::cout << std::fixed << std::setprecision(2) << *it;
        }
        std::cout << "]\n";

        auto lazyAll = lazyView.collect();
        printVec("  Lazy  (collected)", lazyAll);
        std::cout << '\n';
    }

    // ─── Demo 5: Custom Pipeline Builder Pattern ─────────────────────────────────────
    // EN: Demonstrates the builder pattern for composing processing stages
    // TR: İşleme aşamalarını birleştirmek için inşaatçı kalıbını gösterir
    {
        std::cout << "── Demo 5: Custom Pipeline Builder ──\n";

        // EN: Simulated throttle position sensor data (0–4095 ADC range)
        // TR: Simüle edilmiş gaz kelebeği konum sensörü verisi (0–4095 ADC aralığı)
        std::vector<double> throttleADC = {
            150.0, 4200.0, 2048.0, -50.0, 3500.0, 0.0, 1500.0, 4096.0
        };

        /*  C++20 equivalent / C++20 karşılığı:
         *  auto processed = throttleADC
         *      | views::filter([](double v){ return v >= 0.0 && v <= 4095.0; })
         *      | views::transform([](double v){ return (v / 4095.0) * 100.0; })
         *      | views::transform([](double v){ return std::clamp(v, 0.0, 100.0); });
         */

        SensorPipeline throttlePipeline;
        throttlePipeline
            // EN: Stage 1 — filter out-of-range ADC values
            // TR: Aşama 1 — aralık dışı ADC değerlerini filtrele
            .addStage(makeFilterStage([](double v) {
                return v >= 0.0 && v <= 4095.0;
            }))
            // EN: Stage 2 — normalize to percentage (0–100%)
            // TR: Aşama 2 — yüzdeye normalleştir (0–100%)
            .addStage(makeTransformStage([](double v) {
                return (v / 4095.0) * 100.0;
            }))
```

```
            // EN: Stage 3 — clamp to safe throttle range
            // TR: Aşama 3 — güvenli gaz kelebeği aralığına sınırla
            .addStage(makeTransformStage([](double v) {
                return std::max(0.0, std::min(100.0, v));
            }));

        auto throttlePercent = throttlePipeline.execute(throttleADC);
        printVec("  Throttle %", throttlePercent);

        // EN: Compare classic vs range-style on SensorReading data
        // TR: SensorReading verisi üzerinde klasik ile aralık tarzını karşılaştır
        std::vector<SensorReading> mixedReadings = {
            {10, 800.0, true}, {11, -5.0, false}, {12, 400.0, true},
            {13, 950.0, true}, {14, 0.0, false},  {15, 620.0, true}
        };

        auto classicResult = classicApproach(mixedReadings);
        auto rangeResult   = rangeStyleApproach(mixedReadings);

        printVec("  Classic STL result", classicResult);
        printVec("  Range-style result", rangeResult);

        // EN: Verify both approaches produce identical output
        // TR: Her iki yaklaşımın aynı çıktıyı ürettiğini doğrula
        bool match = (classicResult == rangeResult);
        std::cout << "  Results match: " << (match ? "YES" : "NO") << "\n\n";
    }

    std::cout << "═══════════════════════════════════════════════════════\n";
    std::cout << " All demos completed successfully.\n";
    std::cout << "═══════════════════════════════════════════════════════\n";

    return 0;
}
```

► 09. Json And Serialization

module_04_advanced/09_json_and_serialization.cpp

Konu: JSON & Serialization: Veri Serileştirme ve JSON işleme — Data Serialization & JSON Processing

Teori / Theory

THEORY: What Is Serialization?

Serialization is the process of converting in-memory data structures into a format that can be stored (file, database) or transmitted (network, IPC). Deserialization is the reverse: reconstructing objects from that format.

```
┌──────────────────────────────────────────────────────────┐
│ C++ Object  →  serialize()  →  bytes/string  →  file/network  │
│ C++ Object  ←  deserialize() ←  bytes/string  ←  file/network  │
└──────────────────────────────────────────────────────────┘
```

Common formats:

```
┌──────────────┬──────────┬──────────────┬──────────────────┐
│ Format       │ Type     │ Human-read?  │ Use Case         │
├──────────────┼──────────┼──────────────┼──────────────────┤
│ JSON         │ Text     │ Yes          │ Config, REST APIs │
│ XML          │ Text     │ Yes          │ Enterprise, SOAP  │
│ YAML         │ Text     │ Yes          │ Config files      │
│ Protobuf     │ Binary   │ No           │ High-perf RPC (gRPC) │
│ MessagePack  │ Binary   │ No           │ Compact JSON-like │
│ FlatBuffers  │ Binary   │ No           │ Zero-copy access  │
│ Raw binary   │ Binary   │ No           │ Embedded, firmware │
└──────────────┴──────────┴──────────────┴──────────────────┘
```

TEORİ: Serileştirme Nedir?

Serileştirme (serialization), bellekteki veri yapılarını saklanabilir (dosya, veritabanı) veya iletilebilir (ağ, IPC) bir formata dönüştürme işlemidir. Ters serileştirme (deserialization) ise tersidir.

```
┌──────────────────────────────────────────────────────────┐
│ C++ Nesne  →  serialize()    →  bayt/string  →  dosya/ağ    │
│ C++ Nesne  ←  deserialize()  ←  bayt/string  ←  dosya/ağ    │
└──────────────────────────────────────────────────────────┘
```

Yaygın formatlar:

Format	Tür	İnsan okunur?	Kullanım Alanı
JSON	Metin	Evet	Config, REST API'lar
XML	Metin	Evet	Kurumsal, SOAP
YAML	Metin	Evet	Config dosyaları
Protobuf	İkili	Hayır	Yüksek perf. RPC(gRPC)
MessagePack	İkili	Hayır	Kompakt JSON-benzeri
FlatBuffers	İkili	Hayır	Sıfır-kopya erişim
Ham ikili	İkili	Hayır	Gömülü, firmware

THEORY: JSON Format

JSON (JavaScript Object Notation) is the most common text serialization format. It has 6 value types:
- Object: { "key": value, ... } → maps to std::map / struct
- Array: [value, value, ...] → maps to std::vector
- String: "hello" → maps to std::string
- Number: 42, 3.14 → maps to int, double
- Boolean: true, false → maps to bool
- Null: null → maps to std::nullopt

JSON is hierarchical — objects and arrays can nest arbitrarily deep. Whitespace is insignificant (can be minified or pretty-printed).

TEORİ: JSON Formatı

JSON (JavaScript Object Notation) en yaygın metin serileştirme formatıdır. 6 değer tipi vardır:
- Object: { "anahtar": değer, ... } → std::map / struct
- Array: [değer, değer, ...] → std::vector
- String: "merhaba" → std::string
- Number: 42, 3.14 → int, double
- Boolean: true, false → bool
- Null: null → std::nullopt

THEORY: Manual vs Library

In production, use a library like nlohmann/json (header-only, modern C++): #include <nlohmann/json.hpp> using json = nlohmann::json; json j = {{"name", "ECU"}, {"temp", 85.3}}; // create std::string s = j.dump(2); // serialize (pretty) json parsed = json::parse(s); // deserialize
In this file, we implement serialization FROM SCRATCH to teach:
1. How JSON mapping works conceptually
2. How to design to_json / from_json patterns
3. Binary serialization for embedded systems
4. Common pitfalls (endianness, alignment, string escaping)

TEORİ: Elle vs Kütüphane

Üretimde nlohmann/json gibi kütüphane kullan (header-only, modern C++): #include <nlohmann/json.hpp> using json = nlohmann::json; json j = {{"name", "ECU"}, {"temp", 85.3}}; // oluştur std::string s = j.dump(2); // serileştir (güzel format) json parsed = json::parse(s); // ters serileştir
Bu dosyada öğretmek için serileştirmeyi SIFIRDAN uyguluoruz:
1. JSON eşleştirmesinin kavramsal olarak nasıl çalıştığı
2. to_json / from_json desenleri nasıl tasarlanır
3. Gömülü sistemler için ikili serileştirme
4. Yaygın tuzaklar (endianness, hizalama, string kaçış)

THEORY: Binary Serialization Pitfalls

Direct memory dump (memcpy/write struct) seems easy but is DANGEROUS:
1. ENDIANNESS: Big-endian vs little-endian machines interpret bytes differently
2. PADDING: Compilers add padding bytes for alignment — struct size ≠ sum of fields
3. POINTER MEMBERS: Pointers are addresses — meaningless in another process/machine
4. std::string / std::vector: Internal pointers — cannot be memcpy'd
5. VERSIONING: Adding a field breaks all old serialized data
Safe approach: Serialize field-by-field with explicit byte order.

TEORİ: İkili Serileştirme Tuzakları

Doğrudan bellek dökümü (memcpy/yapı yazma) kolay görünür ama TEHLİKELİDİR:
1. ENDIANNESS: Big/little-endian makineler baytları farklı yorumlar
2. PADDING: Derleyiciler hizalama için dolgu baytları ekler
3. POINTER ÜYELERİ: İşaretçiler adrestir — başka işlem/makinede anlamsız
4. std::string / std::vector: İç işaretçiler — memcpy yapılamaz
5. VERSİYONLAMA: Alan eklemek tüm eski serileştirilmiş veriyi bozar

→ Ref: https://github.com/nlohmann/json — nlohmann/json (industry standard)
→ Ref: https://en.cppreference.com/w/cpp/numeric/byteswap — C++23 std::byteswap

Kaynak Kod / Source Code

```
#include <iostream>
#include <string>
#include <sstream>
#include <vector>
#include <map>
#include <variant>
#include <optional>
#include <cstring>
#include <cstdint>
#include <iomanip>
#include <algorithm>
#include <stdexcept>
#include <fstream>

```

```cpp
// ═══════════════════════════════════════════════════════════════════════════
// PART 1: Simple JSON Builder (Serialization)
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: A minimal JSON builder that constructs JSON strings from C++ data.
 *     This teaches the PATTERN used by nlohmann/json without the full library.
 *
 *     Design: Uses std::variant to represent JSON values (type-safe union).
 *     This is similar to how real JSON libraries work internally.
 *
 * TR: C++ verilerinden JSON string'leri oluşturan minimal JSON oluşturucu.
 *     Tam kütüphane olmadan nlohmann/json'ın KALIBINI öğretir.
 *     Tasarım: JSON değerlerini temsil etmek için std::variant kullanır (tip-güvenli birlik).
 */
class JsonValue {
public:
    // EN: The 6 JSON types represented as a std::variant
    //     std::monostate represents JSON null (no value)
    // TR: std::variant olarak temsil edilen 6 JSON tipi
    //     std::monostate JSON null'ı temsil eder (değer yok)
    using Null   = std::monostate;
    using Bool   = bool;
    using Int    = int64_t;
    using Float  = double;
    using String = std::string;
    using Array  = std::vector<JsonValue>;
    using Object = std::map<std::string, JsonValue>;

    using ValueType = std::variant<Null, Bool, Int, Float, String, Array, Object>;

    // EN: Default = null / TR: Varsayılan = null
    JsonValue() : value_(Null{}) {}

    // EN: Constructors for each JSON type
    // TR: Her JSON tipi için constructor'lar
    JsonValue(bool b) : value_(b) {}                        // NOLINT
    JsonValue(int i) : value_(static_cast<int64_t>(i)) {}    // NOLINT
    JsonValue(int64_t i) : value_(i) {}                      // NOLINT
    JsonValue(double d) : value_(d) {}                       // NOLINT
    JsonValue(const char* s) : value_(std::string(s)) {}     // NOLINT
    JsonValue(std::string s) : value_(std::move(s)) {}       // NOLINT
    JsonValue(Array a) : value_(std::move(a)) {}             // NOLINT
    JsonValue(Object o) : value_(std::move(o)) {}            // NOLINT

    // EN: Serialize to JSON string with optional indentation
    //     indent = -1 → compact (no whitespace)
    //     indent >= 0 → pretty-printed with that many spaces per level
    // TR: JSON string'e serileştir, opsiyonel girintileme ile
    //     indent = -1 → kompakt (boşluk yok)
    //     indent >= 0 → o kadar boşlukla güzel yazdırılmış
    std::string dump(int indent = -1) const {
        std::ostringstream oss;
        dump_impl(oss, indent, 0);
        return oss.str();
    }

    // EN: operator[] for object access — creates entries if they don't exist
    // TR: Nesne erişimi için operator[] — yoksa girdi oluşturur
    JsonValue& operator[](const std::string& key) {
        if (!std::holds_alternative<Object>(value_)) {
            value_ = Object{};
        }
        return std::get<Object>(value_)[key];
    }

    // EN: Type checking helpers
    // TR: Tip kontrol yardımcıları
    bool is_null() const { return std::holds_alternative<Null>(value_); }
    bool is_bool() const { return std::holds_alternative<Bool>(value_); }
    bool is_int() const { return std::holds_alternative<Int>(value_); }
    bool is_float() const { return std::holds_alternative<Float>(value_); }
    bool is_string() const { return std::holds_alternative<String>(value_); }
    bool is_array() const { return std::holds_alternative<Array>(value_); }
    bool is_object() const { return std::holds_alternative<Object>(value_); }

```

```cpp
    // EN: Value getters (throw if wrong type)
    // TR: Değer alıcılar (yanlış tip ise fırlatır)
    template<typename T>
    const T& get() const { return std::get<T>(value_); }

private:
    ValueType value_;

    // EN: Escape special characters in JSON strings.
    //     JSON requires: \" \\ \/ \b \f \n \r \t and \uXXXX for control chars.
    //     Forgetting to escape = INVALID JSON = parser rejection.
    // TR: JSON string'lerinde özel karakterleri escape et.
    //     JSON şunları gerektirir: \" \\ \/ \b \f \n \r \t ve kontrol karakterleri \uXXXX.
    //     Escape unutmak = GEÇERSİZ JSON = parser reddi.
    static std::string escape_string(const std::string& s) {
        std::string result;
        result.reserve(s.size() + 2);
        result += '"';
        for (char c : s) {
            switch (c) {
                case '"':  result += "\\\""; break;
                case '\\': result += "\\\\"; break;
                case '\b': result += "\\b";  break;
                case '\f': result += "\\f";  break;
                case '\n': result += "\\n";  break;
                case '\r': result += "\\r";  break;
                case '\t': result += "\\t";  break;
                default:
                    if (static_cast<unsigned char>(c) < 0x20) {
                        // EN: Control character → \u00XX
                        // TR: Kontrol karakteri → \u00XX
                        char buf[8];
                        snprintf(buf, sizeof(buf), "\\u%04x",
                                 static_cast<unsigned int>(static_cast<unsigned char>(c)));
                        result += buf;
                    } else {
                        result += c;
                    }
            }
        }
        result += '"';
        return result;
    }

    void dump_impl(std::ostringstream& oss, int indent, int depth) const {
        std::string pad = (indent >= 0) ? std::string(static_cast<size_t>((depth + 1) * indent), ' ') : "";
        std::string pad_close = (indent >= 0) ? std::string(static_cast<size_t>(depth * indent), ' ') : "";
        std::string nl = (indent >= 0) ? "\n" : "";
        std::string sep = (indent >= 0) ? ", " : ",";

        std::visit([&](auto&& val) {
            using T = std::decay_t<decltype(val)>;
            if constexpr (std::is_same_v<T, Null>) {
                oss << "null";
            } else if constexpr (std::is_same_v<T, Bool>) {
                oss << (val ? "true" : "false");
            } else if constexpr (std::is_same_v<T, Int>) {
                oss << val;
            } else if constexpr (std::is_same_v<T, Float>) {
                oss << std::setprecision(6) << val;
            } else if constexpr (std::is_same_v<T, String>) {
                oss << escape_string(val);
            } else if constexpr (std::is_same_v<T, Array>) {
                oss << "[" << nl;
                for (size_t i = 0; i < val.size(); ++i) {
                    oss << pad;
                    val[i].dump_impl(oss, indent, depth + 1);
                    if (i + 1 < val.size()) oss << ",";
                    oss << nl;
                }
                oss << pad_close << "]";
            } else if constexpr (std::is_same_v<T, Object>) {
                oss << "{" << nl;
```

```
164                 size_t i = 0;
165                 for (const auto& [key, jval] : val) {
166                     oss << pad << escape_string(key) << ": ";
167                     jval.dump_impl(oss, indent, depth + 1);
168                     if (i + 1 < val.size()) oss << ",";
169                     oss << nl;
170                     ++i;
171                 }
172                 oss << pad_close << "}";
173             }
174         }, value_);
175     }
176 };
177
178 // ═══════════════════════════════════════════════════════════════════════════
179 // PART 2: Struct ↔ JSON Mapping (to_json / from_json pattern)
180 // ═══════════════════════════════════════════════════════════════════════════
181
182 /**
183  * EN: This is the PATTERN used by nlohmann/json for automatic struct conversion.
184  *     You define free functions `to_json` and `from_json` for your types.
185  *     The library then automatically converts:
186  *       json j = my_struct;        → calls to_json
187  *       auto s = j.get<MyStruct>(); → calls from_json
188  *
189  *     We replicate this pattern manually to teach the concept.
190  *
191  * TR: nlohmann/json'ın otomatik struct dönüşümü için kullandığı KALIPTIR.
192  *     Tipleriniz için serbest `to_json` ve `from_json` fonksiyonları tanımlarsınız.
193  */
194
195 // EN: Domain struct — represents a sensor reading
196 // TR: Domain yapısı — sensör okumasını temsil eder
197 struct SensorReading {
198     std::string sensor_id;
199     double temperature;
200     uint32_t timestamp;
201     bool is_critical;
202     std::vector<double> history;  // EN: Last N readings / TR: Son N okuma
203 };
204
205 // EN: to_json: Convert SensorReading → JsonValue (serialization)
206 //     This maps each C++ field to a JSON key-value pair.
207 // TR: to_json: SensorReading → JsonValue dönüşümü (serileştirme)
208 //     Her C++ alanını bir JSON anahtar-değer çiftine eşler.
209 JsonValue to_json(const SensorReading& sr) {
210     // EN: Build JSON array from vector
211     // TR: Vector'den JSON dizisi oluştur
212     JsonValue::Array history_arr;
213     history_arr.reserve(sr.history.size());
214     for (double h : sr.history) {
215         history_arr.emplace_back(h);
216     }
217
218     // EN: Build JSON object from struct fields
219     // TR: Struct alanlarından JSON nesnesi oluştur
220     JsonValue::Object obj;
221     obj["sensor_id"] = JsonValue(sr.sensor_id);
222     obj["temperature"] = JsonValue(sr.temperature);
223     obj["timestamp"] = JsonValue(static_cast<int64_t>(sr.timestamp));
224     obj["is_critical"] = JsonValue(sr.is_critical);
225     obj["history"] = JsonValue(std::move(history_arr));
226
227     return JsonValue(std::move(obj));
228 }
229
230 // EN: ECU configuration struct — more complex example
231 // TR: ECU konfigürasyon yapısı — daha karmaşık örnek
232 struct ECUConfig {
233     std::string module_name;
234     std::string firmware_version;
235     int can_bus_speed;           // EN: kbps / TR: kbps
236     bool debug_mode;
237     std::vector<std::string> enabled_features;
238 };
```

```cpp

JsonValue to_json(const ECUConfig& cfg) {
    JsonValue::Array features;
    for (const auto& f : cfg.enabled_features) {
        features.emplace_back(f);
    }

    JsonValue::Object obj;
    obj["module_name"] = JsonValue(cfg.module_name);
    obj["firmware_version"] = JsonValue(cfg.firmware_version);
    obj["can_bus_speed_kbps"] = JsonValue(static_cast<int64_t>(cfg.can_bus_speed));
    obj["debug_mode"] = JsonValue(cfg.debug_mode);
    obj["enabled_features"] = JsonValue(std::move(features));

    return JsonValue(std::move(obj));
}

// ============================================================================
// PART 3: Binary Serialization (Portable, Field-by-Field)
// ============================================================================

/**
 * EN: BinaryWriter safely serializes data to a byte buffer.
 *     Unlike JSON (text-based, ~2x size overhead), binary is compact and fast.
 *     Used in embedded systems, game networking, and protocol implementations.
 *
 *     Key principle: ALWAYS write in a fixed byte order (we use little-endian).
 *     Never dump raw structs — serialization must be field-by-field.
 *
 * TR: BinaryWriter veriyi güvenle bir bayt tamponuna serileştirir.
 *     JSON'dan (metin tabanlı, ~2x boyut yükü) farklı olarak ikili kompakt ve hızlıdır.
 *     Gömülü sistemlerde, oyun ağında ve protokol uygulamalarında kullanılır.
 *
 *     Temel prensip: HER ZAMAN sabit bayt sırasında yaz (little-endian kullanıyoruz).
 *     Asla ham yapıları dökme — serileştirme alan-alan olmalı.
 */
class BinaryWriter {
public:
    // EN: Write a single byte
    // TR: Tek bayt yaz
    void write_u8(uint8_t val) {
        buffer_.push_back(val);
    }

    // EN: Write 16-bit in little-endian byte order
    //     Regardless of host CPU endianness, we always write LSB first.
    // TR: 16-bit'i little-endian bayt sırasında yaz
    //     Host CPU endianness'ından bağımsız, her zaman LSB önce yazılır.
    void write_u16(uint16_t val) {
        buffer_.push_back(static_cast<uint8_t>(val & 0xFF));         // EN: LSB / TR: En az anlamlı bayt
        buffer_.push_back(static_cast<uint8_t>((val >> 8) & 0xFF));  // EN: MSB / TR: En anlamlı bayt
    }

    // EN: Write 32-bit in little-endian
    // TR: 32-bit'i little-endian'da yaz
    void write_u32(uint32_t val) {
        for (int i = 0; i < 4; ++i) {
            buffer_.push_back(static_cast<uint8_t>((val >> (i * 8)) & 0xFF));
        }
    }

    // EN: Write a double (IEEE 754, 8 bytes)
    //     We memcpy to uint64_t to get the raw bytes, then write in LE order.
    //     This is portable across architectures (unlike casting to char*).
    // TR: double yaz (IEEE 754, 8 bayt)
    //     Ham baytları almak için uint64_t'ye memcpy, sonra LE sırasında yaz.
    void write_f64(double val) {
        uint64_t bits;
        std::memcpy(&bits, &val, sizeof(bits));
        for (int i = 0; i < 8; ++i) {
            buffer_.push_back(static_cast<uint8_t>((bits >> (i * 8)) & 0xFF));
        }
    }
```

```cpp

    // EN: Write a string: length prefix (u32) + raw UTF-8 bytes.
    //     Length prefix is crucial — binary format has no delimiter like JSON's quotes.
    //     Without length, the reader doesn't know where the string ends!
    // TR: String yaz: uzunluk öneki (u32) + ham UTF-8 baytları.
    //     Uzunluk öneki kritik — ikili formatın JSON'ın tırnakları gibi sınırlayıcısı yok.
    void write_string(const std::string& s) {
        write_u32(static_cast<uint32_t>(s.size()));
        for (char c : s) {
            buffer_.push_back(static_cast<uint8_t>(c));
        }
    }

    // EN: Write a bool as a single byte (0 = false, 1 = true)
    // TR: bool'u tek bayt olarak yaz (0 = false, 1 = true)
    void write_bool(bool val) {
        buffer_.push_back(val ? 1 : 0);
    }

    const std::vector<uint8_t>& data() const { return buffer_; }
    size_t size() const { return buffer_.size(); }

private:
    std::vector<uint8_t> buffer_;
};

/**
 * EN: BinaryReader reads data back from a byte buffer.
 *     Reads must happen in the EXACT SAME ORDER as writes!
 *     This is why binary protocols need strict documentation.
 *
 * TR: BinaryReader bayt tamponundan veriyi geri okur.
 *     Okumalar yazma ile TAMAMEN AYNI SIRADA yapılmalı!
 *     İkili protokollerin sıkı belgelemeye ihtiyacı bundandır.
 */
class BinaryReader {
public:
    explicit BinaryReader(const std::vector<uint8_t>& data) : data_(data), pos_(0) {}

    uint8_t read_u8() {
        check_bounds(1);
        return data_[pos_++];
    }

    uint16_t read_u16() {
        check_bounds(2);
        uint16_t val = static_cast<uint16_t>(data_[pos_])
                     | (static_cast<uint16_t>(data_[pos_ + 1]) << 8);
        pos_ += 2;
        return val;
    }

    uint32_t read_u32() {
        check_bounds(4);
        uint32_t val = 0;
        for (int i = 0; i < 4; ++i) {
            val |= static_cast<uint32_t>(data_[pos_ + static_cast<size_t>(i)]) << (i * 8);
        }
        pos_ += 4;
        return val;
    }

    double read_f64() {
        check_bounds(8);
        uint64_t bits = 0;
        for (int i = 0; i < 8; ++i) {
            bits |= static_cast<uint64_t>(data_[pos_ + static_cast<size_t>(i)]) << (i * 8);
        }
        pos_ += 8;
        double val;
        std::memcpy(&val, &bits, sizeof(val));
        return val;
    }
```

```cpp

    std::string read_string() {
        uint32_t len = read_u32();
        check_bounds(len);
        std::string s(data_.begin() + static_cast<ptrdiff_t>(pos_),
                      data_.begin() + static_cast<ptrdiff_t>(pos_ + len));
        pos_ += len;
        return s;
    }

    bool read_bool() {
        return read_u8() != 0;
    }

    size_t position() const { return pos_; }
    size_t remaining() const { return data_.size() - pos_; }

private:
    const std::vector<uint8_t>& data_;
    size_t pos_;

    void check_bounds(size_t n) const {
        if (pos_ + n > data_.size()) {
            throw std::runtime_error("BinaryReader: buffer underflow at position "
                                     + std::to_string(pos_));
        }
    }
};

// ============================================================================
// PART 4: Binary Serialization for Domain Types
// ============================================================================

/**
 * EN: Serialize SensorReading to binary format.
 *     Protocol: [sensor_id:string][temperature:f64][timestamp:u32][is_critical:bool]
 *               [history_count:u32][history[0]:f64][history[1]:f64]...
 *
 * TR: SensorReading'i ikili formata serileştir.
 *     Protokol: [sensor_id:string][temperature:f64][timestamp:u32][is_critical:bool]
 *               [history_count:u32][history[0]:f64][history[1]:f64]...
 */
void serialize_binary(BinaryWriter& writer, const SensorReading& sr) {
    writer.write_string(sr.sensor_id);
    writer.write_f64(sr.temperature);
    writer.write_u32(sr.timestamp);
    writer.write_bool(sr.is_critical);
    writer.write_u32(static_cast<uint32_t>(sr.history.size()));
    for (double h : sr.history) {
        writer.write_f64(h);
    }
}

SensorReading deserialize_binary(BinaryReader& reader) {
    SensorReading sr;
    sr.sensor_id = reader.read_string();
    sr.temperature = reader.read_f64();
    sr.timestamp = reader.read_u32();
    sr.is_critical = reader.read_bool();
    uint32_t history_count = reader.read_u32();
    sr.history.resize(history_count);
    for (uint32_t i = 0; i < history_count; ++i) {
        sr.history[i] = reader.read_f64();
    }
    return sr;
}

```

```
// ═══════════════════════════════════════════════════════════════════════════
// PART 5: Simple JSON Parser (Deserialization)
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: A minimal recursive-descent JSON parser.
 *     Demonstrates how JSON parsing works internally:
 *     1. Tokenize: identify strings, numbers, keywords, delimiters
 *     2. Parse: recursively build a tree structure
 *
 *     Production code should use nlohmann/json or simdjson (SIMD-accelerated).
 *     This parser is educational — it handles the core cases but isn't battle-tested.
 *
 * TR: Minimal özyinelemeli-iniş JSON ayrıştırıcı.
 *     JSON ayrıştırmanın dahili olarak nasıl çalıştığını gösterir.
 *     Üretim kodu nlohmann/json veya simdjson (SIMD hızlandırmalı) kullanmalı.
 */
class JsonParser {
public:
    static JsonValue parse(const std::string& input) {
        size_t pos = 0;
        auto result = parse_value(input, pos);
        return result;
    }

private:
    static void skip_whitespace(const std::string& s, size_t& pos) {
        while (pos < s.size() && (s[pos] == ' ' || s[pos] == '\t' ||
               s[pos] == '\n' || s[pos] == '\r')) {
            ++pos;
        }
    }

    static JsonValue parse_value(const std::string& s, size_t& pos) {
        skip_whitespace(s, pos);
        if (pos >= s.size()) throw std::runtime_error("Unexpected end of input");

        char c = s[pos];
        if (c == '"') return parse_string(s, pos);
        if (c == '{') return parse_object(s, pos);
        if (c == '[') return parse_array(s, pos);
        if (c == 't' || c == 'f') return parse_bool(s, pos);
        if (c == 'n') return parse_null(s, pos);
        if (c == '-' || (c >= '0' && c <= '9')) return parse_number(s, pos);

        throw std::runtime_error(std::string("Unexpected character: ") + c);
    }

    static JsonValue parse_string(const std::string& s, size_t& pos) {
        if (s[pos] != '"') throw std::runtime_error("Expected '\"'");
        ++pos;
        std::string result;
        while (pos < s.size() && s[pos] != '"') {
            if (s[pos] == '\\') {
                ++pos;
                if (pos >= s.size()) throw std::runtime_error("Unexpected end in string escape");
                switch (s[pos]) {
                    case '"': result += '"'; break;
                    case '\\': result += '\\'; break;
                    case 'n': result += '\n'; break;
                    case 't': result += '\t'; break;
                    case 'r': result += '\r'; break;
                    default: result += s[pos]; break;
                }
            } else {
                result += s[pos];
            }
            ++pos;
        }
        if (pos >= s.size()) throw std::runtime_error("Unterminated string");
        ++pos; // skip closing "
        return JsonValue(std::move(result));
    }
```

```cpp

    static JsonValue parse_number(const std::string& s, size_t& pos) {
        size_t start = pos;
        bool is_float = false;
        if (s[pos] == '-') ++pos;
        while (pos < s.size() && s[pos] >= '0' && s[pos] <= '9') ++pos;
        if (pos < s.size() && s[pos] == '.') { is_float = true; ++pos; }
        while (pos < s.size() && s[pos] >= '0' && s[pos] <= '9') ++pos;
        if (pos < s.size() && (s[pos] == 'e' || s[pos] == 'E')) {
            is_float = true; ++pos;
            if (pos < s.size() && (s[pos] == '+' || s[pos] == '-')) ++pos;
            while (pos < s.size() && s[pos] >= '0' && s[pos] <= '9') ++pos;
        }
        std::string num_str = s.substr(start, pos - start);
        if (is_float) return JsonValue(std::stod(num_str));
        return JsonValue(static_cast<int64_t>(std::stoll(num_str)));
    }

    static JsonValue parse_bool(const std::string& s, size_t& pos) {
        if (s.compare(pos, 4, "true") == 0) { pos += 4; return JsonValue(true); }
        if (s.compare(pos, 5, "false") == 0) { pos += 5; return JsonValue(false); }
        throw std::runtime_error("Invalid boolean");
    }

    static JsonValue parse_null(const std::string& s, size_t& pos) {
        if (s.compare(pos, 4, "null") == 0) { pos += 4; return JsonValue(); }
        throw std::runtime_error("Invalid null");
    }

    static JsonValue parse_array(const std::string& s, size_t& pos) {
        ++pos; // skip [
        skip_whitespace(s, pos);
        JsonValue::Array arr;
        if (pos < s.size() && s[pos] == ']') { ++pos; return JsonValue(std::move(arr)); }
        while (true) {
            arr.push_back(parse_value(s, pos));
            skip_whitespace(s, pos);
            if (pos < s.size() && s[pos] == ',') { ++pos; skip_whitespace(s, pos); }
            else break;
        }
        if (pos >= s.size() || s[pos] != ']') throw std::runtime_error("Expected ']'");
        ++pos;
        return JsonValue(std::move(arr));
    }

    static JsonValue parse_object(const std::string& s, size_t& pos) {
        ++pos; // skip {
        skip_whitespace(s, pos);
        JsonValue::Object obj;
        if (pos < s.size() && s[pos] == '}') { ++pos; return JsonValue(std::move(obj)); }
        while (true) {
            skip_whitespace(s, pos);
            auto key_val = parse_string(s, pos);
            std::string key = key_val.get<JsonValue::String>();
            skip_whitespace(s, pos);
            if (pos >= s.size() || s[pos] != ':') throw std::runtime_error("Expected ':'");
            ++pos;
            obj[key] = parse_value(s, pos);
            skip_whitespace(s, pos);
            if (pos < s.size() && s[pos] == ',') { ++pos; }
            else break;
        }
        skip_whitespace(s, pos);
        if (pos >= s.size() || s[pos] != '}') throw std::runtime_error("Expected '}'");
        ++pos;
        return JsonValue(std::move(obj));
    }
};
```

```

// ═══════════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "============================================\n";
    std::cout << "   JSON & Serialization\n";
    std::cout << "============================================\n\n";

    // ─── Demo 1: JSON Builder — Struct to JSON ──────────────────────────
    {
        std::cout << "--- Demo 1: Struct → JSON (Serialization) ---\n";

        // EN: Create a sensor reading and convert to JSON
        // TR: Sensör okuması oluştur ve JSON'a dönüştür
        SensorReading reading;
        reading.sensor_id = "EXHAUST_01";
        reading.temperature = 742.5;
        reading.timestamp = 1710300000;
        reading.is_critical = true;
        reading.history = {735.0, 738.2, 740.1, 742.5};

        JsonValue json = to_json(reading);

        // EN: dump(2) → pretty-print with 2 spaces indentation
        // TR: dump(2) → 2 boşluk girintileme ile güzel yazdır
        std::cout << json.dump(2) << "\n\n";

        // EN: dump(-1) → compact format (no whitespace, for transmission)
        // TR: dump(-1) → kompakt format (boşluk yok, iletim için)
        std::cout << "  Compact: " << json.dump() << "\n\n";
    }

    // ─── Demo 2: Nested JSON Object ─────────────────────────────────────
    {
        std::cout << "--- Demo 2: Nested JSON Object ---\n";

        // EN: Build a complex JSON structure with nesting
        // TR: İç içe karmaşık JSON yapısı oluştur
        ECUConfig config;
        config.module_name = "BrakeController";
        config.firmware_version = "2.4.1";
        config.can_bus_speed = 500;
        config.debug_mode = false;
        config.enabled_features = {"ABS", "ESC", "TCS", "HillAssist"};

        JsonValue json = to_json(config);
        std::cout << json.dump(2) << "\n\n";
    }

    // ─── Demo 3: JSON Parser (Deserialization) ──────────────────────────
    {
        std::cout << "--- Demo 3: JSON String → Parsed Object ---\n";

        // EN: Parse a JSON string and access its values
        // TR: JSON string'i ayrıştır ve değerlerine eriş
        std::string json_str = R"({
            "vehicle": "Model-X",
            "year": 2026,
            "electric": true,
            "range_km": 450.5,
            "sensors": ["lidar", "radar", "camera"],
            "config": {
                "autopilot": true,
                "level": 3
            }
        })";

        std::cout << "  Input JSON:\n" << json_str << "\n\n";

        JsonValue parsed = JsonParser::parse(json_str);
        std::cout << "  Re-serialized (pretty):\n" << parsed.dump(2) << "\n\n";

        // EN: Access nested values through the parsed structure
        // TR: Ayrıştırılmış yapı üzerinden iç içe değerlere eriş
        if (parsed.is_object()) {
            auto& obj = parsed.get<JsonValue::Object>();
```

```
            if (obj.count("vehicle") && obj.at("vehicle").is_string()) {
                std::cout << "  Vehicle: " << obj.at("vehicle").get<JsonValue::String>() << "\n";
            }
            if (obj.count("year") && obj.at("year").is_int()) {
                std::cout << "  Year: " << obj.at("year").get<JsonValue::Int>() << "\n";
            }
            if (obj.count("sensors") && obj.at("sensors").is_array()) {
                auto& sensors = obj.at("sensors").get<JsonValue::Array>();
                std::cout << "  Sensors (" << sensors.size() << "): ";
                for (size_t i = 0; i < sensors.size(); ++i) {
                    if (i > 0) std::cout << ", ";
                    std::cout << sensors[i].get<JsonValue::String>();
                }
                std::cout << "\n";
            }
        }
        std::cout << "\n";
    }

    // ─── Demo 4: Binary Serialization ──────────────────────────────────
    {
        std::cout << "--- Demo 4: Binary Serialization ---\n";

        // EN: Serialize the same sensor reading to binary format
        // TR: Aynı sensör okumasını ikili formata serileştir
        SensorReading original;
        original.sensor_id = "OIL_TEMP";
        original.temperature = 92.3;
        original.timestamp = 1710300500;
        original.is_critical = false;
        original.history = {88.1, 89.5, 91.0, 92.3};

        BinaryWriter writer;
        serialize_binary(writer, original);

        std::cout << "  Binary size: " << writer.size() << " bytes\n";
        std::cout << "  JSON size:   " << to_json(original).dump().size() << " bytes\n";
        // EN: ↑ Binary is typically 2-5x smaller than JSON!
        // TR: ↑ İkili genellikle JSON'dan 2-5x daha küçüktür!

        // EN: Show hex dump of first 32 bytes
        // TR: İlk 32 baytın hex dökümünü göster
        std::cout << "  Hex dump (first 32 bytes): ";
        for (size_t i = 0; i < std::min<size_t>(32, writer.size()); ++i) {
            std::cout << std::hex << std::setfill('0') << std::setw(2)
                      << static_cast<int>(writer.data()[i]) << " ";
        }
        std::cout << std::dec << "\n";

        // EN: Deserialize from binary and verify
        // TR: İkiliden ters serileştir ve doğrula
        BinaryReader reader(writer.data());
        SensorReading restored = deserialize_binary(reader);

        std::cout << "\n  Restored from binary:\n";
        std::cout << "    sensor_id:   " << restored.sensor_id << "\n";
        std::cout << "    temperature: " << restored.temperature << "\n";
        std::cout << "    timestamp:   " << restored.timestamp << "\n";
        std::cout << "    is_critical: " << (restored.is_critical ? "true" : "false") << "\n";
        std::cout << "    history:     [";
        for (size_t i = 0; i < restored.history.size(); ++i) {
            if (i > 0) std::cout << ", ";
            std::cout << restored.history[i];
        }
        std::cout << "]\n";

        // EN: Verify data integrity
        // TR: Veri bütünlüğünü doğrula
        bool match = (original.sensor_id == restored.sensor_id) &&
                     (original.temperature == restored.temperature) &&
                     (original.timestamp == restored.timestamp) &&
                     (original.is_critical == restored.is_critical) &&
                     (original.history == restored.history);
        std::cout << "    Integrity check: " << (match ? "PASS ✓" : "FAIL ✗") << "\n";
        std::cout << "    Reader consumed: " << reader.position() << " / "
                  << writer.size() << " bytes (remaining: " << reader.remaining() << ")\n\n";
    }
```

```cpp

    // ─── Demo 5: String Escaping ──────────────────────────────────────
    {
        std::cout << "--- Demo 5: JSON String Escaping ---\n";

        // EN: Demonstrate proper JSON string escaping
        //     Without escaping, special characters break JSON validity.
        // TR: JSON string escape'lemesini göster
        //     Escape olmadan özel karakterler JSON geçerliliğini bozar.
        JsonValue::Object obj;
        obj["path"] = JsonValue("C:\\Users\\nimbus\\data.json");
        obj["message"] = JsonValue("He said \"hello\" and left");
        obj["multiline"] = JsonValue("Line 1\nLine 2\tTabbed");

        JsonValue test(std::move(obj));
        std::cout << test.dump(2) << "\n\n";

        // EN: Parse it back to verify round-trip
        // TR: Gidiş-dönüş doğrulamak için geri ayrıştır
        std::string serialized = test.dump();
        JsonValue reparsed = JsonParser::parse(serialized);
        std::cout << "  Round-trip (parsed back):\n" << reparsed.dump(2) << "\n\n";
    }

    // ─── Demo 6: JSON vs Binary Size Comparison ───────────────────────
    {
        std::cout << "--- Demo 6: Size Comparison (JSON vs Binary) ---\n";

        // EN: Create multiple readings and compare serialization sizes
        // TR: Birden fazla okuma oluşturup serileştirme boyutlarını karşılaştır
        std::vector<SensorReading> readings;
        readings.push_back({"TEMP_01", 85.3, 1710300000, false, {82.1, 83.5, 84.0}});
        readings.push_back({"TEMP_02", 92.1, 1710300001, true, {90.0, 91.2}});
        readings.push_back({"PRESSURE", 2.4, 1710300002, false, {2.3, 2.35, 2.38, 2.4}});

        // EN: JSON size
        // TR: JSON boyutu
        size_t json_total = 0;
        for (const auto& r : readings) {
            json_total += to_json(r).dump().size();
        }

        // EN: Binary size
        // TR: İkili boyut
        BinaryWriter batch_writer;
        for (const auto& r : readings) {
            serialize_binary(batch_writer, r);
        }

        std::cout << "  3 sensor readings:\n";
        std::cout << "    JSON total:   " << json_total << " bytes\n";
        std::cout << "    Binary total: " << batch_writer.size() << " bytes\n";
        double ratio = static_cast<double>(json_total) / static_cast<double>(batch_writer.size());
        std::cout << "    Ratio: JSON is " << std::fixed << std::setprecision(1)
                  << ratio << "x larger than binary\n";
        // EN: Binary is more efficient for embedded systems with limited bandwidth
        //     JSON is better for debugging, configs, and human-readable APIs
        // TR: İkili, sınırlı bant genişliğine sahip gömülü sistemler için daha verimli
        //     JSON, hata ayıklama, konfigürasyon ve insan tarafından okunabilir API'ler için daha iyi
        std::cout << "\n";
    }

    std::cout << "============================================\n";
    std::cout << "   End of JSON & Serialization\n";
    std::cout << "============================================\n";

    return 0;
}
```

Modül 5: Certification Prep — Sertifikasyon Hazırlığı

Sertifikasyon Hazırlığı

7 kaynak dosya · Standart: C++17

C++17

► 01. Undefined Behavior

module_05_certification/01_undefined_behavior.cpp

Konu: CPA/CPP Prep: Undefined Behavior & Tricky Questions — Sertifikasyon Hazırlığı: Tanımsız

Teori / Theory

THEORY: Why do C++ Institute and CppQuiz ask such hard questions?

In C++, unlike managed languages like Java or C#, the compiler assumes YOU know exactly what you are doing. If you access memory out of bounds or use a pointer after it's deleted, C++ DOES NOT stop you. It results in "Undefined Behavior" (UB). Certification exams test your ability to spot these hidden traps.

TEORİ: Sertifika Tuzakları

C++, Java veya C# gibi dillerin aksine kontrolü tamamen size bırakır. Derleyici sizin NE YAPTIĞINIZI KESİN OLARAK BİLDİĞİNİZİ varsayar. Dizi boyutunu aşarsanız veya silinmiş bir belleği okumaya çalışırsanız, C++ sizi durdurmaz (hata vermez). Buna "Undefined Behavior" (Tanımsız Davranış - UB) denir. CPA/CPP ve CppQuiz sınavları tam olarak bu gizli tuzakları görme yeteneğinizi test eder.

THEORY: Examples of Undefined Behavior

1. Dangling Pointers: Pointing to an address that has already been freed.
2. Out of Bounds: Reading the 10th element of a 5-element array.
3. Uninitialized Variables: Reading a local variable containing garbage memory.
4. Signed Integer Overflow: Adding 1 to the absolute MAX value of an `int`.

TEORİ: Tanımsız Davranış Örnekleri

1. Dangling (Sarkan) Pointers: Silinmiş bir adresi işaret eden pointer (Use after free).
2. Sınır Aşımı (Out of Bounds): 5 elemanlı dizinin 10. elemanını okumak.
3. İlklenmemiş Değişken (Uninitialized): İçine değer atanmamış çöp belleği okumak.
4. İşaretli Taşma (Signed Overflow): Maksimum sınırındaki int değerine 1 eklemek (C++'ta UB'dir).

CPPREF Depth: Unsequenced Modification HACK!

CppReference explicitly forbids modifying a variable and reading it twice in the same sequence point (e.g., `cout << i++ << ++i;`). The C++ standard DOES NOT GUARANTEE which one executes first. Depending on the compiler (GCC vs Clang), it might output completely different numbers, or crash!

CPPREF Derinlik: Sırasız Değişiklik (Sırasız Değişim)!

CppReference, bir değişkeni aynı satırda (sekans noktasında) hem okuyup hem de değiştirmeyi (Örn: `cout << i++ << ++i;`) kesinlikle YASAKLAR (UB'dir). C++ standardı derleyicilere sağdan mı yoksa soldan mı hesaplayacaklarını söylemez! Mac'te farklı, Windows'ta farklı sonuç çıkar. Oyun state'leriniz darmadağın olur!

Kaynak Kod / Source Code

```cpp
#include <iostream>

void testDanglingPointer() {
  std::cout << "\n--- Danger Zone: Dangling Pointer (Sarkan İşaretçi) ---" << std::endl;
  int *ptr = new int(100);
  std::cout << "Original Value (Orijinal Değer): " << *ptr << std::endl;

  // EN: Memory is freed.
  // TR: Belleği iade ettik.
  delete ptr;

  // DANGER (TEHLİKE)!
  // EN: 'ptr' still points to that address, but it belongs to the OS now.
  // TR: 'ptr' hala o adresi gösteriyor ama adres artık bize ait değil.
  //
  // *ptr = 500; // UNDEFINED BEHAVIOR! (Do not uncomment! / Yorumdan çıkarmayın!)

  // Correct Way (Doğru Yönem):
  // EN: Safer!
  // TR: Artık hiçbir yeri göstermiyor. Güvenli!
  ptr = nullptr;
  std::cout << "Pointer safely nullified. (İşaretçi sıfırlandı.)" << std::endl;
}

void testUninitializedVariable() {
  std::cout << "\n--- Danger Zone: Uninitialized Variable (İlklenmemiş Değişken) ---" << std::endl;
  // EN: Memory allocated, but holds generic GARBAGE.
  // TR: Sadece hafızada yer ayrıldı, içi çöp.
  int mysteriousNumber;

  // std::cout << "Value: " << mysteriousNumber << std::endl; // UNDEFINED BEHAVIOR: Just reading
  // it is UB!
  // EN: Now initialized.
  // TR: Ancak değer atandıktan sonra okunabilir.
  mysteriousNumber = 42;
  std::cout << "Initialized safely (Güvenle ilk değer atandı): " << mysteriousNumber << std::endl;
}

int main() {
  std::cout << "=== MODULE 5: CERTIFICATION & CPPQUIZ TRAPS ===\n" << std::endl;
  std::cout << "EN: Exams ask you to READ code and spot hidden DANGER ZONES, "
               "not write them!"
            << std::endl;
  std::cout << "TR: Sertifika sınavları kodu YAZMANIZI değil, OKUMANIZI ve "
               "gizli tehlikeleri bulmanızı ister!\n"
            << std::endl;

  testDanglingPointer();
  testUninitializedVariable();

  // 3. Trap: Object Slicing (Nesne Dilimlenmesi)
  class Base {
  public:
    virtual void print() { std::cout << "Base"; }
  };
  class Derived : public Base {
  public:
    void print() override { std::cout << "Derived"; }
  };

```

```
61    Derived myDerived;
62
63    // EN: ERROR! Specific traits were sliced off! (Object Slicing)
64    // TR: HATA! Özel olan kısımlar "dilimlenip" kesilip atıldı. (Nesne Dilimlenmesi)
65    Base slicedBase = myDerived;
66
67    std::cout << "\nObject Slicing Test: ";
68    // EN: Prints "Base"! Polymorphism FAILED.
69    // TR: "Base" yazar! Polimorfizm çalışmaz!
70    slicedBase.print();
71
72    std::cout << " (Polymorphism FAILED due to Value Copy. Reference was needed!)" << std::endl;
73
74    return 0;
75  }
```

► 02. Object Slicing Virtual

module_05_certification/02_object_slicing_virtual.cpp

Konu: CPA/CPP Prep: Object Slicing and Virtual Tables — Nesne Dilimlenmesi ve Sanal Tablolar

Teori / Theory

THEORY: What is Object Slicing?

Polymorphism (Dynamic Binding) ONLY works over Pointers (*) or References (&). If you directly assign a Derived object to a Base object (Pass by Value), the compiler literally "slices off" all the new variables and overridden functions that belong to the Derived class. The object becomes a pure Base object!

TEORİ: Nesne Dilimlenmesi Nedir?

Çok biçimlilik (Polimorfizm) SADECE İşaretçiler (*) veya Referanslar (&) üzerinden çalışır. Eğer türetilmiş (Derived) bir nesneyi doğrudan temel (Base) bir nesneye ("Değer ile", yani Pass by Value) atarsanız, derleyici Alt sınıfa ait olan TÜM yeni değişkenleri ve ezilmiş (overridden) fonksiyonları kelimenin tam anlamıyla "bıçakla kesip / dilimleyip" çöpe atar. O nesne artık saf bir Base nesnesine dönüşür!

THEORY: Why does it slice? (Memory Architecture)

In C++, objects are contiguous blocks of memory.
- Base Class: 4 Bytes
- Derived Class: 8 Bytes If you say `Base b = Derived();`, the compiler cannot fit 8 Bytes into a 4 Byte box. It HAS to SLICE the overflowing parts off.

TEORİ: Neden Dilimleniyor?

C++'ta nesneler RAM'de yan yana duran kutucuklardır.
- Base Sınıf: 4 Bayt (sadece yaş)
- Derived Sınıf: 8 Bayt (hem yaş hem maaş) Eğer `Base b = Derived();` derseniz, derleyici 4 baytlık küçük kutunun içine, 8 baytlık dev nesneyi sığdıramaz. Mecburen taflan kısmı kesip atar (Dilimler)!

CPPREF Depth: Slicing Destroys RTTI

Slicing doesn't just cut off memory; it permanently DESTROYS the object's dynamic type identity (typeid / RTTI pointer). The sliced object's hidden `vptr` is forcibly rewritten to point to the Base class's `vtable`. It is completely unrecoverable!

CPPREF Derinlik: Dilimleme RTTI'yı Yok Eder (Yazılımsal Kimlik Katliamı)

Dilimleme sadece fazla byte'ları kesmez; nesnenin dinamik RTTI kimliğini (typeid) SONSUZA DEK YOK EDER. Gizli Sanal İşaretçi (`vptr`), zorla Base sınıfın `vtable`'ına (sanal tablosuna) bağlanacak şekilde değiştirilir. Orijinal nesneyi bir daha asla geri getiremezsiniz!

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <string>

class Document {
public:
  std::string name;
  Document(std::string docName) : name(docName) {}

  // EN: Virtual function is the key to polymorphism.
  // TR: Sanal (Virtual) fonksiyon, polimorfizmin anahtarıdır.
  virtual void print() const {
    std::cout << "[Base] Printing generic Document: " << name << std::endl;
  }
};

class PDFDocument : public Document {
public:
  // EN: Specific to subclass.
  // TR: Alt sınıfa Özel bir değişken.
  int securityProtocol;

  PDFDocument(std::string docName, int proto)
      : Document(docName), securityProtocol(proto) {}

  // EN: Overriding the base function.
  // TR: Fonksiyonu ezmek (Override).
  void print() const override {
    std::cout << "[Derived] Printing PDF: " << name << " | Security Protocol (Güvenlik Protokolü): "
        << securityProtocol << std::endl;
  }
};

// EN: 1. UNSAFE: Pass by Value -> SLICED!
// TR: 1. GÜVENSİZ: Değer ile Geçiş -> DİLİMLENİR!
void sloppyPrint(Document doc) {
  // EN: This object is no longer a PDF. It is just a regular Document.
  // TR: Bu nesne artık bir PDF değil. Sadece sıradan bir Document (Dilimlenmiş).
  doc.print();
}

// EN: 2. SAFE: Pass by Reference -> NO SLICING!
// TR: 2. GÜVENLİ: Referans ile Geçiş -> Polimorfizm!
void securePrint(const Document &doc) {
  // EN: The V-Table is consulted. The true owner (PDF) method is called.
  // TR: V-Table'a (Sanal Tabloya) bakılarak asıl sahibin (PDF) metodu çağırılır.
  doc.print();
}

int main() {
  std::cout << "=== MODULE 5: OBJECT SLICING (NESNE DİLİMLENMESİ) ===\n" << std::endl;

  PDFDocument topSecretFile("NuclearCodes.pdf", 256);

```

```
54    // --- TEST 1: THE SLICING DISASTER / DİLİMLENME FELAKETİ ---
55    std::cout << "1. Pass by Value (Sliced Object / Dilimlenen Nesne):" << std::endl;
56    std::cout << "-> INTERVIEW TRAP: Compiler gives NO ERROR, but calls the "
57                 "wrong function!"
58              << std::endl;
59    std::cout << "-> MÜLAKAT TUZAĞI: Derleyici hata (Error) VERMEZ ancak yanlış "
60                 "fonksiyonu çağırır!"
61              << std::endl;
62    sloppyPrint(topSecretFile);
63
64    std::cout << "\n-------------------------------------------------\n" << std::endl;
65
66    // --- TEST 2: CORRECT & SAFE POLYMORPHISM / DOĞRU POLİMORFİZM ---
67    std::cout << "2. Pass by Reference (Real Polymorphism / Gerçek Polimorfizm):" << std::endl;
68    std::cout << "-> INTACT OBJECT: The V-Table recognized the true owner!" << std::endl;
69    std::cout << "-> NESNE BÜTÜN: Sanal Tablo (V-Table) asıl sahibini tanıdı!" << std::endl;
70    securePrint(topSecretFile);
71
72    std::cout << "\n-------------------------------------------------\n" << std::endl;
73
74    // EN: A classic CppQuiz Question:
75    // TR: Klasik CppQuiz Sorusu:
76    std::cout << "3. Direct Assignment (Doğrudan Atama / Slicing):" << std::endl;
77    Document standardDoc = topSecretFile;
78    standardDoc.print();
79    // std::cout << standardDoc.securityProtocol; // ERROR! Won't compile because it was sliced
80    // off! (Erişilemez)
81
82    return 0;
83  }
```

► 03. Type Casting

module_05_certification/03_type_casting.cpp

Konu: CPA/CPP Prep: Type Casting — Sertifikasyon: C++ Tip Dönüşüm Operatörleri

Teori / Theory

THEORY: Why are C-Style Casts (int) banned in Modern C++?

In C, you could force any type into any other type using `(int) x` or `(char*) ptr`. This tells the compiler: "Shut up, I know what I'm doing." But it's violent. It breaks constness, converts pointers blindly, and causes severe run-time crashes. C++ introduced 4 safe, intentional, and searchable cast operators.

TEORİ: Eski Cast Neden Yasak?

C dilinde, `(int)x` diyerek her türlü veriyi (örneğin bir kedi nesnesini integer'a) zorla dönüştürebilirsiniz. Derleyiciye "Sus, ne yaptığımı biliyorum" demektir. Ama bu bir felakettir; nesne dilimlenmelerine, `const` zırhının kırılmasına yol açar. C++, derleyicinin "niyetinizi anlaması" ve sizi koruması için 4 ayrı dönüştürücü çıkardı. C-Stili dönüşüm kullananlar mülakatlarda genellikle anında eksi puan alır!

THEORY: The 4 Modern C++ Casts

C++ replaced the dangerous C-style `(int)x` cast with 4 explicit, searchable, and intentional cast operators. Each has a specific purpose:

---

1. static_cast<Type>(val)

The "logical/ideal" cast. Used for well-defined conversions like float → int, int → enum, base* → derived* (when you know the type). Checked at COMPILE-TIME. Zero runtime cost. This is the cast you use 95% of the time.

---

2. dynamic_cast<Type>(val)

The "safe polymorphism" cast. Used to safely downcast a base class pointer/reference to a derived class. Checks the V-Table at RUN-TIME (RTTI). If the cast is invalid, returns nullptr (pointer) or throws std::bad_cast (reference). Slower than static_cast but SAFE.

---

3. const_cast<Type>(val)

Strips away `const` or `volatile` qualifiers. Used ONLY when interfacing with old C APIs that forgot to mark parameters as `const`. Modifying a truly const object through const_cast is UNDEFINED BEHAVIOR. Use with extreme caution!

---

4. reinterpret_cast<Type>(val)

The "brute force" cast. Reinterprets the raw bit pattern of one type as another. No checks, no safety net. Used almost exclusively in low-level hardware/driver programming and memory-mapped I/O. Casting unrelated pointer types and dereferencing them triggers Strict Aliasing UB. Only safe targets: `char*`, `unsigned char*`, `std::byte*`. FORBIDDEN in application-level code!

TEORİ: 4 Modern C++ Dönüştürücü

C++, tehlikeli C-stili `(int)x` dönüşümünü 4 açık, aranabilir ve niyeti belirten dönüştürücü operatörle değiştirdi. Her birinin amacı farklıdır:

---

1. static_cast<Type>(val)

"Mantıklı/ideal" dönüşüm. float → int, int → enum, base* → derived* gibi iyi tanımlanmış dönüşümler için kullanılır (tipi bildiğinizde). DERLEME ZAMANINDA kontrol edilir. Çalışma zamanı maliyeti sıfırdır. Zamanın %95'inde bu cast'i kullanırsınız.

---

2. dynamic_cast<Type>(val)

"Güvenli polimorfizm" dönüşümü. Üst sınıf pointer/referansını alt sınıfa güvenle dönüştürmek için kullanılır. ÇALIŞMA ZAMANINDA V-Table üzerinden kontrol eder (RTTI). Geçersiz dönüşümde nullptr (pointer) veya std::bad_cast (referans) döner. static_cast'ten yavaş ama GÜVENLİDİR.

---

3. const_cast<Type>(val)

`const` veya `volatile` niteleyicilerini kaldırır. SADECE `const` parametreyi unutmuş eski C API'leriyle çalışırken kullanılır. Gerçekten const olan bir nesneyi const_cast ile değiştirmek TANIMSIZ DAVRANISTIR (UB). Son derece dikkatli kullanın!

---

4. reinterpret_cast<Type>(val)

"Kaba kuvvet" dönüşümü. Bir tipin ham bit desenini başka bir tip olarak yorumlar. Kontrol yok, güvenlik ağı yok. Neredeyse yalnızca düşük seviye donanım/sürücü programlama ve bellek-eşlemeli I/O'da kullanılır. İlişkisiz pointer tiplerini dönüştürüp okumak Strict Aliasing UB tetikler. Güvenli hedefler: `char*`, `unsigned char*`, `std::byte*`. Uygulama seviyesi kodda YASAKTIR!

CPPREF Depth: `dynamic_cast` Cost & `reinterpret_cast` Strict Aliasing UB

`dynamic_cast` performs a string-search on the Run-Time Type Information (RTTI) tree, making it EXTREMELY SLOW compared to `static_cast`. Use it sparingly! Also, `reinterpret_cast` frequently triggers "Strict Aliasing" Undefined Behavior if you cast two unrelated pointers and dereference them. Only cast to `char*` or `std::byte*`.

CPPREF Derinlik: `dynamic_cast` Maliyeti ve `reinterpret_cast` Sıkı Örtüşme UB

`dynamic_cast`, asıl kimliği bulmak için RTTI ağacında text araması yapar. Bu yüzden `static_cast`e kıyasla DEHŞET YAVAŞTIR (Oyun döngüsünde anlık FPS düşüşü yaratır). Ek olarak, `reinterpret_cast` ile dönüştürülmüş uyumsuz tipleri `*ptr` okumak Strict Aliasing kuralını kırıp UB (Tanımsız Davranış) çökmelerine sebep olur. Sadece `char*` veya `std::byte*` ile güvenle donanım okuyabilirsiniz.

Kaynak Kod / Source Code

```cpp
#include <iostream>

class Organism {
public:
  virtual ~Organism() {}
// EN: Virtual Destructor REQUIRED for V-Table (Polymorphism)
// TR: V-Table için şart!
};

class Human : public Organism {
public:
  void run() { std::cout << "Human is running! (İnsan koşuyor!)" << std::endl; }
};

class Plant : public Organism {};

void cStyle_VS_staticCast() {
  double pi = 3.14159;

  // EN: Old C-way (Bad Habit) - Unclear intention and unsearchable.
  // TR: Eski C Yöntemi (Kötü Alışkanlık) - Güvenliği belirsiz.
  int oldInt = (int)pi;

  // EN: Modern C++ way (static_cast) - Explicit intention. Compiler verifies it.
  // TR: Modern C++ yöntemi (static_cast) - Niyet belli. Derleyici onaylar.
  int modernInt = static_cast<int>(pi);

  std::cout << "C-Style cast: " << oldInt << " | static_cast: " << modernInt << std::endl;
}

void unlockDynamicCast() {
  Organism *unknownEntity1 = new Human();
  Organism *unknownEntity2 = new Plant();

  std::cout << "\n--- DYNAMIC CAST (Polymorphic Safety / Güvenlik) ---" << std::endl;

  // EN: [INTERVIEW QUESTION]: I have an Organism*. If it's truly a Human, I want to make it run.
  // If it's a Plant, I do nothing. How do I do this without crashing?
  //
  // TR: [MÜLAKAT SORUSU]: Elimde Organism* var, ama "İnsan" ise koşturacağım. "Bitki" ise hiçbir
  // şey yapmayacağım. Nasıl güvenle kontrol ederim?

  // EN: dynamic_cast queries the V-Table! If it matches `Human*`, it returns the pointer. Else,
  // `nullptr`.
  // TR: dynamic_cast gider V-Table'dan sorar! İnsansa verir, değilse "nullptr" döndürür! Çökme
  // engellenir.

  Human *isReallyHuman1 = dynamic_cast<Human *>(unknownEntity1);
```

```
  // EN: C++17 allows "if (auto p = dynamic_cast...)"
  // TR: C++17, "if (auto p = dynamic_cast...)" kullanımına izin verir.
  if (isReallyHuman1 != nullptr) {
    std::cout << "[SUCCESS / BAŞARI] 1st Entity is Human!" << std::endl;
    isReallyHuman1->run();
  }

  Human *isReallyHuman2 = dynamic_cast<Human *>(unknownEntity2);
  if (isReallyHuman2 == nullptr) {
    std::cout << "[REJECTED / REDDEDİLDİ] 2nd Entity is NOT Human (Probably Plant). "
            "CRASH PREVENTED! (Çöküş önlendi!)"
        << std::endl;
  }

  delete unknownEntity1;
  delete unknownEntity2;
}

int main() {
  std::cout << "=== MODULE 5: MODERN TYPE CASTING ===\n" << std::endl;

  unlockDynamicCast();

  std::cout << "\n--- reinterpret_cast (Danger Zone / Tehlikeli Bölge) ---" << std::endl;
  // EN: Do not use this unless hacking hardware/drivers. It tells the OS to treat a meaningless
  // number as a RAM address.
  // TR: Donanım programlama harici YASAKTIR. Integer bir sayıyı, RAM adresi imiş gibi yutturmaya
  // denir.

  long hardwareAddress = 0x7FFAABBCCDD;
  int *fakePointer = reinterpret_cast<int *>(hardwareAddress);

  std::cout << "Integer masked as a Pointer (Sayı Pointer yapıldı): " << fakePointer <<
      " (If you dereference this, PROGRAM CRASHES! / İçini okursanız ÇÖKER!)" << std::endl;

  return 0;
}
```

► 04. Exceptions Handling

module_05_certification/04_exceptions_handling.cpp

Konu: CPA/CPP Prep: Exception Handling in C++ — Sertifikasyon: İstisnai Durum (Hata) Yönetimi

Teori / Theory

THEORY: if-else Error Checking vs Exceptions

In old C systems, when an error occurred, functions returned `-1` or `false`. The danger was "Ignoring the Error Value". A developer might forget to check it.
In C++, the `try / catch / throw` mechanism exists. If an error is thrown (`throw`), the program starts tearing itself back down the Stack (Stack Unwinding) until it finds a `catch` block. If you never catch it, it CRASHES (`std::terminate`).

TEORİ: if-else Hata Kontrolü vs İstisnalar

Eski C sistemlerinde bir hata oluştuğunda fonksiyon `-1` veya `false` döndürürdü. Bu yöntem "Hata değerinin yoksayılması / unutulması (Ignore)" riskini taşırdı.
C++'ta `try / catch / throw` mekanizması vardır. Bir hata fırlatılırsa (`throw`), o hatayı Tutan (`catch`) bir yer bulana kadar program kendi kendini Stack'ten (Yığın) geriye doğru söker (Buna "Stack Unwinding" denir). Yakalamazsanız program tamamen GÖÇER (std::terminate).

THEORY: The `noexcept` Keyword (Interview Question)

C++11 introduced `noexcept` (The guarantee of NOT throwing). It tells the compiler: "I swear this function is perfect. It will NEVER throw." The compiler therefore removes all background try-catch safety netting, making the code INCREDIBLY FAST! Often used in 'Move Constructors'.

TEORİ: noexcept Anahtar Kelimesi

C++11 ile `noexcept` (Exception fırlatmama sözü/garantisi) geldi. Derleyiciye: "Söz veriyorum, bu fonksiyonum hatasız çalışacak, hiç hata fırlatmayacak" der. Derleyici bunu görünce tüm hata kontrol (try-catch) koruma zırhlarını kaldırır ve KODU İNANILMAZ HIZLANDIRIR! Genellikle 'Move Constructor' (Taşıma Yıkıcıları) ve 'Destructor' (Yıkıcılar) için yazılır.
DİKKAT: Eğer `noexcept` verip içinde gizlice `throw` yaparsanız (sözünüzü bozduğunuz an), C++ programı ANINDA infaz eder (terminate).

CPPREF Depth: The 4 Levels of Exception Safety Guarantees

CppReference expects professional C++ code to provide one of these guarantees:
1. Nothrow Guarantee: Will NEVER throw (like `noexcept`).
2. Strong Guarantee: If an operation fails, the state is rolled back to exactly what it was before the call (Commit or Rollback - like Databases).
3. Basic Guarantee: If it fails, no memory is leaked, but the object might be in an altered (but valid) state.
4. No Guarantee: Fails and corrupts/leaks memory. (Unacceptable in modern C++).

CPPREF Derinlik: 4 İstisna Güvenliği Garanti Seviyesi

CppReference, profesyonel kodun şu 4 Özel Güvenlik Seviyesinden birini vadetmesini bekler:
1. Nothrow: Asla hata fırlatmaz (`noexcept`).
2. Güçlü Teminat (Strong): İşlem başarısız olursa, her şey işlem öncesi haline geri döner. Hiç dokunulmamış gibi olur (Veritabanlarındaki Rollback gibi).
3. Temel Teminat (Basic): Hata olursa RAM sızdırmaz, çökmeleri engeller ama nesne o anki bozuk (fakat güvenli) bir halde kalabilir.
4. Garantisiz: Hata olursa RAM sızdırır ve programı çökertir (Modern C++'ta Yasak!)

Kaynak Kod / Source Code

```cpp
#include <iostream>
// EN: Contains many built-in standard exceptions
// TR: Birçok yerleşik standart hata sınıfını içerir
#include <stdexcept>

// EN: 1. Custom Exception Class
// TR: 1. Özel Hata Sınıfı
// EN: Usually inherited from std::exception.
// TR: Genelde std::exception'dan miras alınarak yapılır.
class BankException : public std::exception {
public:
  // EN: Overriding what() to provide a custom message. Must be `noexcept`.
  // TR: what() fonksiyonunu ezerek kendi mesajımızı veriyoruz. (`noexcept` zorunludur).
  const char *what() const noexcept override {
    return "Insufficient Funds! (Bakiye Yetersiz!)";
  }
};

class BankAccount {
private:
  int balance;

public:
  BankAccount(int initialBalance) : balance(initialBalance) {}

```

```cpp
  // EN: 2. A function that THROWS an Exception
  // TR: 2. Hata fırlatan fonksiyon.
  void deposit(int amount) {
    if (amount < 0) {
      // EN: Throw! Execution stops instantly. Program searches for 'catch'.
      // TR: Fırlat (throw)! Program akışı ANINDA KESİLİR, en yakın 'catch' bloğu aranır.
      throw std::invalid_argument( "Cannot deposit negative money! (Negatif para yatırılamaz!)");
    }
    balance += amount;
  }

  void withdraw(int amount) {
    if (amount > balance) {
      // EN: Throwing our CUSTOM Object!
      // TR: Kendi Özel Objemizi fırlattık.
      throw BankException();
    }
    balance -= amount;
  }

  // EN: 3. NOEXCEPT GUARANTEE (Performance Boost)
  // TR: 3. PERFORMANS ARTIŞI
  int getBalance() const noexcept {
    // EN: So simple, it's impossible to throw. Compiler optimizes it to O(1) raw speed.
    // TR: Bu o kadar basit bir işlem ki, hata fırlatması ihtimali SIFIR. Derleyici güvenlik
    // ağlarını siler, en hızlı şekilde (O(1)) C gibi derler.
    return balance;
  }
};

int main() {
  std::cout << "=== MODULE 5: EXCEPTION HANDLING (try/catch/noexcept) ===\n" << std::endl;

  BankAccount myAccount(100);

  // EN: RULE: Trap risky zones (where a throw might happen) inside a 'try' block!
  // TR: KURAL: Riskli bölgeleri (throw olma ihtimali olanları) 'try' içine hapset!
  try {
    std::cout << "Transaction 1 in progress (İşlem 1 yazılıyor)..." << std::endl;
    myAccount.deposit(50); // Balance: 150

    std::cout << "Transaction 2 in progress (İşlem 2 yapılıyor)..." << std::endl;
    // CRASH! / HATA FIRLIYOR! (Aşağıdaki işlemler iptal)
    myAccount.withdraw( 1000);

    std::cout << "You will never see this line! (Bu satırı göremeyeceksiniz! "
                 "Kod kırıldı.)"
              << std::endl;
    myAccount.deposit(-5);

  }
  // EN: catch blocks are Polymorphic. Put the most specific ones FIRST, the generic ones LAST.
  // TR: catch blokları 'Polimorfik'tir. En spesifik (özel) hatayı başa, en geneli sona
  // (Fallback) yazın.

  catch (const BankException &e) {
    // EN: We caught our custom exception by reference (&).
    // TR: Özel Banka sınıfımızı referans(&) ile yakaladık.
    std::cout << "\n[ERROR CAUGHT - BANK]: " << e.what() << std::endl;
  } catch (const std::invalid_argument &e) {
    // EN: Standard argument error.
    // TR: std içinden gelen Argüman hatası.
    std::cout << "\n[ERROR CAUGHT - ARGUMENT]: " << e.what() << std::endl;
  } catch (const std::exception &e) {
    // EN: The ultimate fallback catcher for any other std::exception.
    // TR: Yukarıdaki özel durumlara uymayan başka BİLİNMEYEN std hataları EN SON BURAYA düşer.
    std::cout << "\n[ERROR CAUGHT - GENERIC]: " << e.what() << std::endl;
  }

  std::cout << "\nProgram did not crash! Life goes on! (Program çökmedi!)" << std::endl;
  std::cout << "Current Balance [noexcept] (Mevcut Bakiye): " << myAccount.getBalance() <<
      std::endl;

  return 0;
}
```

► 05. Const Correctness

module_05_certification/05_const_correctness.cpp

Konu: CPA/CPP Prep: const Correctness in C++ — Sertifikasyon: const Doğruluğu (Sabitlik

Teori / Theory

THEORY: const Member Functions

A const member function promises NOT to modify the object's state. When you mark a method as `const`, the compiler enforces that no data member is modified through `this`. Only const member functions can be called on const objects — calling a non-const method on a const object is a compile-time error.

TEORİ: const Üye Fonksiyonlar

Bir const üye fonksiyon, nesnenin durumunu DEĞİŞTİRMEYECEĞİNE söz verir. Bir metodu `const` olarak işaretlediğinizde, derleyici `this` üzerinden hiçbir veri üyesinin değiştirilmediğini garanti eder. const nesneler üzerinde yalnızca const üye fonksiyonlar çağrılabilir — const bir nesne üzerinde non-const metot çağırmak derleme hatasıdır.

THEORY: The mutable Keyword

`mutable` allows a data member to be modified even inside a const member function. This is used for caches, mutexes, and lazy initialization — things that do NOT change the logical/observable state of the object.

TEORİ: mutable Anahtar Kelimesi

`mutable`, bir veri üyesinin const üye fonksiyon içinde bile değiştirilmesine izin verir. Önbellekler (cache), mutex'ler ve tembel başlatma (lazy init) gibi nesnenin mantıksal/gözlemlenebilir durumunu DEĞİŞTİRMEYEN öğeler için kullanılır.

THEORY: const_cast Dangers

`const_cast` can remove the const qualifier, but removing const from data that was ORIGINALLY declared const is Undefined Behavior! The compiler may place truly-const data in read-only memory. The only legitimate use is interfacing with legacy C APIs that forgot const.

TEORİ: const_cast Tehlikeleri

`const_cast`, const niteleyicisini kaldırabilir, ancak ASLINDA const olarak tanımlanmış veriden const'u kaldırmak Tanımsız Davranıştır (UB)! Derleyici gerçek const verileri salt-okunur belleğe yerleştirebilir. Tek meşru kullanımı, const'u unutan eski C API'leriyle arayüzlemedir.

THEORY: East-const vs West-const

`const int x` (west-const) and `int const x` (east-const) are identical. East-const reads naturally right-to-left: "x is a const int". With pointers it matters: `int const* p` (pointer to const int) vs `int* const p` (const pointer to int). East-const is consistent.

TEORİ: Doğu-const vs Batı-const

`const int x` (batı-const) ile `int const x` (doğu-const) aynıdır. Doğu-const sağdan sola doğal okunur: "x bir const int'tir". Pointer'larda fark önemlidir: `int const* p` (const int'e pointer) vs `int* const p` (int'e const pointer). Doğu-const tutarlıdır.

THEORY: const Propagation & API Design

Passing by `const&` avoids copies while guaranteeing immutability. A well-designed API marks every parameter and method const unless mutation is required. "const is a contract" — it documents intent and enables compiler optimizations. const propagation means if you receive a const ref, everything you pass it to must also accept const.

TEORİ: const Yayılımı ve API

`const&` ile geçiş, kopyalamayı önlerken değişmezliği garanti eder. İyi tasarlanmış bir API, mutasyon gerektirmedikçe her parametreyi ve metodu const olarak işaretler. "const bir sözleşmedir" — niyeti belgeler ve derleyici optimizasyonlarını mümkün kılar. const yayılımı, bir const ref aldığınızda, onu aktardığınız her yerin de const kabul etmesi gerektiği anlamına gelir.

CPPREF Depth: Const Propagation Through Pointers and the Mutable Escape Hatch

`const` on a member function means `this` is `const T*`. BUT: pointers held inside the class are only "shallow const" — `const` on `this` does NOT propagate through `T* member_`. The member pointer itself cannot be reseated, but the pointee remains fully mutable. This is a well-known hole in the const system. `std::experimental::propagate_const<T>` wraps a pointer/smart-pointer and makes `operator*` / `operator->` propagate the constness of the wrapper to the pointee, closing the gap. `mutable` bypasses const entirely — it was designed for mutexes, caches, and lazy initialization (things that do not affect observable state). Overusing `mutable` silently violates the const-correctness contract. Thread-safe const (§17.6.5.9): the standard library assumes `const` member functions are safe to call concurrently. If your `const` methods mutate shared mutable state without synchronization, you have broken this contract and invoked undefined behavior in any STL algorithm.

CPPREF Derinlik: Pointer'lar Üzerinden Const Yayılımı ve Mutable Kaçış Yolu

Bir üye fonksiyondaki `const`, `this`'in `const T*` olduğu anlamına gelir. ANCAK: sınıf içindeki pointer'lar yalnızca "sığ const"tur — `this` üzerindeki `const`, `T* member_` boyunca YAYILMAZ. Pointer'ın kendisi yeniden atanamaz ama gösterdiği veri tamamen değiştirilebilir. `std::experimental::propagate_const<T>` bu açığı kapatır — wrapper'ın const'luğunu pointee'ye yayar. `mutable` ise const'u tamamen atlar; mutex, önbellek ve tembel başlatma için tasarlanmıştır (gözlemlenebilir durumu etkilemeyen şeyler). Aşırı kullanımı const sözleşmesini ihlal eder. İş parçacığı güvenli const (§17.6.5.9): standart kütüphane, `const` üye fonksiyonların eşzamanlı çağrılmasının güvenli olduğunu varsayar. `const` metodlarınız senkronizasyon olmadan paylaşılan mutable durumu değiştiriyorsa, bu sözleşmeyi bozmuş ve herhangi bir STL algoritmasında tanımsız davranışı başlatmış olursunuz.

Kaynak Kod / Source Code

```cpp
#include <iostream>   // EN: Standard I/O for console output
#include <string>     // EN: String class for text data
#include <mutex>      // EN: Mutex for thread-safe const access pattern
#include <cmath>      // EN: Math functions for expensive computation demo

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 1: const and non-const Overloads — SensorCalibration
// EN: Demonstrates overload resolution based on const-ness of the object
// TR: Nesnenin const olup olmadığına göre aşırı yükleme çözünürlüğünü gösterir
// ═══════════════════════════════════════════════════════════════════════════════

class SensorCalibration {
private:
    double offset_ = 0.0;
    // EN: Sensor name used in automotive ECU calibration
    // TR: Otomotiv ECU kalibrasyon sensör adı
    std::string sensorName_ = "DefaultSensor";

public:
    explicit SensorCalibration(std::string name, double offset)
        : offset_(offset), sensorName_(std::move(name)) {}

    // EN: const overload — returns by const reference (read-only access)
    // TR: const aşırı yükleme — const referans döndürür (salt okunur erişim)
    const double& getValue() const {
        std::cout << "  [const getValue() called]\n";
        return offset_;
    }

    // EN: non-const overload — returns by reference (allows modification)
    // TR: non-const aşırı yükleme — referans döndürür (değiştirmeye izin verir)
    double& getValue() {
        std::cout << "  [non-const getValue() called]\n";
        return offset_;
    }

    const std::string& getName() const { return sensorName_; }
};

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 2: mutable Cache — ExpensiveCalculation
// EN: mutable allows caching inside a const method (logical constness)
// TR: mutable, const metot içinde önbelleklemeye izin verir (mantıksal sabitlik)
// ═══════════════════════════════════════════════════════════════════════════════

class ExpensiveCalculation {
private:
    double input_;
    // EN: mutable — these can change even in a const context
    // TR: mutable — const bağlamda bile değişebilir
    mutable double cachedResult_ = 0.0;
    mutable bool dirty_ = true;

    // EN: Simulates a heavy computation (e.g., aerodynamic drag coefficient)
    // TR: Ağır bir hesaplamayı simüle eder (ör. aerodinamik sürtünme katsayısı)
    double computeExpensive() const {
        std::cout << "  [Performing expensive calculation...]\n";
        return std::sin(input_) * std::cos(input_) * 42.0;
    }

public:
    explicit ExpensiveCalculation(double input) : input_(input) {}

    // EN: const method but updates cache via mutable members
    // TR: const metot ama mutable üyeler ile önbelleği günceller
    double getResult() const {
        if (dirty_) {
            cachedResult_ = computeExpensive();
            dirty_ = false;
        }
        return cachedResult_;
    }
```

```cpp

    void setInput(double input) {
        input_ = input;
        dirty_ = true;
    }
};

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 3: ThreadSafeCounter with mutable std::mutex
// EN: mutable mutex allows locking in const methods for thread-safe reads
// TR: mutable mutex, const metotlarda thread-güvenli okuma için kilitlemeye izin verir
// ═══════════════════════════════════════════════════════════════════════════════

class ThreadSafeCounter {
private:
    int count_ = 0;
    // EN: mutable so we can lock even in const member functions
    // TR: mutable, böylece const üye fonksiyonlarda bile kilitleyebiliriz
    mutable std::mutex mtx_;

public:
    // EN: const method — reads the counter (thread-safe pattern)
    // TR: const metot — sayacı okur (thread-güvenli desen)
    int getCount() const {
        std::lock_guard<std::mutex> lock(mtx_);
        return count_;
    }

    void increment() {
        std::lock_guard<std::mutex> lock(mtx_);
        ++count_;
    }
};

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 4: const Propagation Chain — VehicleConfig
// EN: Demonstrates const flowing through multiple function layers
// TR: const'un birden fazla fonksiyon katmanından geçişini gösterir
// ═══════════════════════════════════════════════════════════════════════════════

struct VehicleConfig {
    std::string model = "EV-500";
    int maxSpeedKmh = 250;
    double batteryCapacityKwh = 75.0;
};

// ─── 4a. Low-level reader ──────────────────────────────────────────────────
// EN: Accepts const ref — cannot modify the config
// TR: const ref kabul eder — yapılandırmayı değiştiremez
void printMaxSpeed(const VehicleConfig& config) {
    std::cout << "  Max speed: " << config.maxSpeedKmh << " km/h\n";
}

// ─── 4b. Mid-level validator ───────────────────────────────────────────────
// EN: Also takes const ref — propagates const guarantee downward
// TR: O da const ref alır — const garantisini aşağıya yayar
void validateConfig(const VehicleConfig& config) {
    std::cout << "  Validating config for: " << config.model << "\n";
    printMaxSpeed(config);
}

// ─── 4c. Top-level entry point ─────────────────────────────────────────────
// EN: Entire chain preserves const — no accidental mutation possible
// TR: Tüm zincir const'u korur — kazara değişiklik imkansız
void processConfig(const VehicleConfig& config) {
    std::cout << "  Processing vehicle configuration...\n";
    validateConfig(config);
}

```

```
// ═══════════════════════════════════════════════════════════════════════════
// SECTION 5: const_cast Danger (demonstrated safely in comments)
// EN: Removing const from originally-const data is Undefined Behavior!
// TR: Aslında const olan veriden const kaldırmak Tanımsız Davranıştır!
// ═══════════════════════════════════════════════════════════════════════════

// EN: BAD PRACTICE — DO NOT DO THIS IN PRODUCTION:
// TR: KÖTÜ PRATİK — BUNU ÜRETİMDE YAPMAYIN:
//
// const int engineCode = 42; int& ref = const_cast<int&>(engineCode);   // UB if we write to
// ref! ref = 99;  // UNDEFINED BEHAVIOR — compiler may have placed 42 in ROM
//
// EN: The only acceptable use: calling a C API that takes non-const but promises not to modify:
// TR: Tek kabul edilebilir kullanım: non-const alan ama değiştirmeyeceğine söz veren bir C
// API'sini çağırmak:
//
// void legacy_c_api(char* str);   // C function, doesn't modify str const std::string msg =
// "ECU_OK"; legacy_c_api(const_cast<char*>(msg.c_str()));  // OK if API truly read-only

// ═══════════════════════════════════════════════════════════════════════════
// MAIN — Demonstrations
// ═══════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "═══════════════════════════════════════════════════\n";
    std::cout << " const Correctness — C++ Certification Prep\n";
    std::cout << "═══════════════════════════════════════════════════\n\n";

    // ─── Demo 1: const vs non-const overload resolution ──────────────────
    // EN: The compiler picks the overload based on the const-ness of the object
    // TR: Derleyici, nesnenin const olup olmadığına göre aşırı yüklemeyi seçer
    std::cout << "--- Demo 1: const vs non-const overload ---\n";
    {
        SensorCalibration sensor("TirePressure", 2.5);
        const SensorCalibration constSensor("Throttle", 1.0);

        // EN: Non-const object → non-const overload chosen
        // TR: Non-const nesne → non-const aşırı yükleme seçilir
        std::cout << "Mutable sensor:\n";
        sensor.getValue() = 3.0;
        std::cout << "  Updated value: " << sensor.getValue() << "\n";

        // EN: const object → const overload chosen (cannot assign!)
        // TR: const nesne → const aşırı yükleme seçilir (atama yapılamaz!)
        std::cout << "Const sensor:\n";
        double val = constSensor.getValue();
        std::cout << "  Read value: " << val << "\n";
        // constSensor.getValue() = 5.0;  // ERROR: assignment to const ref
    }

    // ─── Demo 2: mutable cache — lazy computation ────────────────────────
    // EN: const method computes on first call, returns cache on second
    // TR: const metot ilk çağrıda hesaplar, ikincide önbellekten döndürür
    std::cout << "\n--- Demo 2: mutable cache (lazy computation) ---\n";
    {
        const ExpensiveCalculation calc(1.5);
        std::cout << "First call:\n";
        std::cout << "  Result: " << calc.getResult() << "\n";
        std::cout << "Second call (cached):\n";
        std::cout << "  Result: " << calc.getResult() << "\n";
    }

    // ─── Demo 3: const reference chains ──────────────────────────────────
    // EN: const flows from top-level to every nested function call
    // TR: const, üst seviyeden her iç içe fonksiyon çağrısına akar
    std::cout << "\n--- Demo 3: const propagation chain ---\n";
    {
        const VehicleConfig config{"EV-900", 300, 100.0};
        processConfig(config);
    }
```

```

    // ─── Demo 4: const object restrictions ──────────────────────────────────────
    // EN: Shows what you CAN and CANNOT do with a const object
    // TR: const bir nesne ile yapabileceklerinizi ve yapamayacaklarınızı gösterir
    std::cout << "\n--- Demo 4: const object restrictions ---\n";
    {
        const ThreadSafeCounter counter;
        std::cout << "  Count (const access): " << counter.getCount() << "\n";
        // counter.increment();  // ERROR: cannot call non-const on const object

        ThreadSafeCounter mutableCounter;
        mutableCounter.increment();
        mutableCounter.increment();
        mutableCounter.increment();
        std::cout << "  Count (after 3 increments): "
                  << mutableCounter.getCount() << "\n";
    }

    // ─── Demo 5: East-const vs west-const ───────────────────────────────────────
    // EN: Both styles are identical — east-const reads right-to-left
    // TR: Her iki stil de aynıdır — doğu-const sağdan sola okunur
    std::cout << "\n--- Demo 5: East-const vs West-const ---\n";
    {
        // EN: West-const style (traditional)
        // TR: Batı-const stili (geleneksel)
        const int westSpeed = 120;

        // EN: East-const style (reads right-to-left: "x is a const int")
        // TR: Doğu-const stili (sağdan sola okunur: "x bir const int'tir")
        int const eastSpeed = 120;

        std::cout << "  West-const: " << westSpeed << "\n";
        std::cout << "  East-const: " << eastSpeed << "\n";
        std::cout << "  Identical? " << (westSpeed == eastSpeed ? "YES" : "NO")
                  << "\n";

        // EN: Where it really matters — pointers
        // TR: Asıl farkın önemli olduğu yer — pointer'lar
        int speed = 200;
        int const* ptrToConst = &speed;      // pointer to const int
        int* const constPtr   = &speed;      // const pointer to int

        // *ptrToConst = 300;  // ERROR: can't modify through pointer-to-const
        *constPtr = 300;       // OK: the pointer is const, not the data
        std::cout << "  constPtr modification: speed = " << speed << "\n";
        (void)ptrToConst;
    }

    std::cout << "\n═══════════════════════════════════════════════════════\n";
    std::cout << " All const correctness demos completed successfully!\n";
    std::cout << "═══════════════════════════════════════════════════════\n";

    return 0;
}
```

► 06. Interview Gotchas

module_05_certification/06_interview_gotchas.cpp

Konu: CPA/CPP Prep: C++ Interview Gotchas & Tricky Behavior — Sertifikasyon: C++ Mülakat

Teori / Theory

THEORY: Most Vexing Parse

`SensorReading sr(TemperatureSource());` looks like object construction, but the compiler parses it as a FUNCTION DECLARATION: a function `sr` that takes a function-pointer parameter and returns `SensorReading`. This is the "Most Vexing Parse" — the C++ grammar always prefers a declaration over an expression. Fix: use brace initialization `{}`.

TEORİ: En Sinir Bozucu Ayrıştırma

`SensorReading sr(TemperatureSource());` nesne oluşturma gibi görünür, ama derleyici bunu bir FONKSİYON BİLDİRİMİ olarak ayrıştırır: `sr` adında, fonksiyon-pointer parametresi alan ve `SensorReading` döndüren bir fonksiyon. Bu "Most Vexing Parse"tır — C++ grameri bildirim olabilecek her şeyi bildirim olarak yorumlar. Çözüm: süslü parantez `{}`.

THEORY: RVO

When a function returns a local object by value, the compiler can eliminate the copy/move entirely — constructing the object directly in the caller's memory. Named RVO (NRVO) applies to named local variables. C++17 mandates RVO for prvalues (unnamed temporaries). NRVO is not guaranteed by the standard but virtually all major compilers apply it.

NRVO (Return Value Optimization) / TEORİ: Dönüş Değeri Opt.

Bir fonksiyon yerel bir nesneyi değer ile döndürdüğünde, derleyici kopyalama/taşımayı tamamen ortadan kaldırabilir — nesneyi doğrudan çağıranın belleğinde oluşturur. İsimli RVO (NRVO) isimli yerel değişkenlere uygulanır. C++17, prvalue'lar (isimsiz geçiciler) için RVO'yu zorunlu kılar. NRVO standart tarafından garanti edilmez ama neredeyse tüm büyük derleyiciler uygular.

THEORY: Dangling References

Returning a reference to a local variable is ALWAYS dangling — the local is destroyed when the function returns. Returning a reference to a temporary has the same problem. Temporaries die at end of the full expression, EXCEPT when bound to a `const&` or `&&` in the same scope, which extends their lifetime to match the reference's scope.

TEORİ: Sarkan (Dangling) Referanslar

Yerel bir değişkene referans döndürmek HER ZAMAN sarkan (dangling) olur — yerel, fonksiyon döndüğünde yok edilir. Geçiciye referans döndürmek de aynı sorundur. Geçiciler tam ifadenin sonunda ölür, ANCAK aynı kapsamda `const&` veya `&&`'ye bağlanırlarsa ömürleri referansın kapsamına kadar uzatılır.

THEORY: Evaluation Order

Before C++17, in `f(a(), b())` the order of evaluating `a()` and `b()` was UNSPECIFIED — either could run first! C++17 guarantees: chained function calls (e.g., `a.b(x).c(y)`) are evaluated left-to-right, and the object expression is evaluated before its arguments.

TEORİ: Değerlendirme Sırası

C++17 öncesinde, `f(a(), b())` ifadesinde `a()` ve `b()` değerlendirme sırası BELİRTİLMEMİŞTİ — hangisi önce çalışır bilinmezdi! C++17 garantisi: zincirleme fonksiyon çağrılarında (ör. `a.b(x).c(y)`) soldan sağa değerlendirilir ve nesne ifadesi argümanlarından önce değerlendirilir.

THEORY: Implicit Conversions & explicit

A single-argument constructor acts as an implicit conversion operator! `Voltage v = 3.3;` silently constructs a Voltage from a double. This causes subtle bugs. Mark constructors `explicit` to prevent this. `explicit` is one of the most important defensive keywords in C++.

TEORİ: Örtülü Dönüşümler

Tek argümanlı yapıcı, örtülü bir dönüşüm operatörü gibi davranır! `Voltage v = 3.3;` sessizce bir double'dan Voltage oluşturur. Bu sinsi hatalara yol açar. Yapıcıları `explicit` olarak işaretleyerek önleyin. `explicit`, C++'taki en önemli savunma anahtar kelimelerden biridir.

CPPREF Depth: RVO, NRVO and Copy Elision — Guaranteed vs Optional

C++17 guarantees RVO for prvalue returns — the copy/move constructor need not even exist! The object is materialized directly in the caller's storage. NRVO (Named Return Value Optimization) applies to named local variables returned by value; it is NOT guaranteed by the standard but every major compiler (GCC, Clang, MSVC) performs it. CRITICAL PITFALL: writing `return std::move(local);` PREVENTS NRVO because the expression is no longer an lvalue naming a local — the compiler sees an xvalue and must use the move constructor instead of eliding entirely. Never `std::move` a return value of a local object. Copy elision also applies to throw/catch: throwing a local by value can elide the copy into the exception object. In practice, returning by value is often FASTER than returning through an output parameter or pointer, because elision eliminates all copies while an output param forces at least one construction + assignment.

CPPREF Derinlik: RVO, NRVO ve Kopya Eleme — Garantili vs İsteğe Bağlı

C++17, prvalue dönüşleri için RVO'yu garanti eder — kopya/taşıma yapıcısının var olması bile gerekmez! Nesne doğrudan çağıranın belleğinde oluşturulur. NRVO (İsimli Dönüş Değeri Optimizasyonu) değer ile döndürülen isimli yerel değişkenlere uygulanır; standart tarafından garanti EDİLMEZ ama tüm büyük derleyiciler uygular. KRİTİK TUZAK: `return std::move(local);` yazmak NRVO'yu ENGELLER çünkü ifade artık bir yerel'i isimlendiren lvalue değildir — derleyici bir xvalue görür ve tamamen elemek yerine taşıma yapıcısını kullanmak zorunda kalır. Asla yerel nesnenin dönüş değerini `std::move` ile sarmalamayın. Kopya eleme throw/catch için de geçerlidir. Pratikte, değer ile dönüş genellikle çıktı parametresi veya pointer ile dönüşten DAHA HIZLIDIR çünkü eleme tüm kopyaları ortadan kaldırır.

Kaynak Kod / Source Code

```cpp
#include <iostream>   // EN: Standard I/O for console output
#include <string>     // EN: String class for text data
#include <vector>     // EN: Dynamic array for sensor data

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 1: Most Vexing Parse — ECUConfig
// EN: Parentheses cause the compiler to see a function declaration, not a ctor
// TR: Parantezler derleyicinin nesne değil fonksiyon bildirimi görmesine yol açar
// ═══════════════════════════════════════════════════════════════════════════════

struct DataSource {
    // EN: Simulates an automotive data bus (CAN/LIN)
    // TR: Otomotiv veri yolunu simüle eder (CAN/LIN)
    int channelId = 1;
};

class ECUConfig {
    std::string name_;
    int channel_;
public:
    // EN: Default constructor
    // TR: Varsayılan yapıcı
    ECUConfig() : name_("DefaultECU"), channel_(0) {
        std::cout << "  ECUConfig() default ctor\n";
    }

    // EN: Constructor taking a DataSource
    // TR: DataSource alan yapıcı
    explicit ECUConfig(DataSource src)
        : name_("ECU_CH" + std::to_string(src.channelId)),
          channel_(src.channelId) {
        std::cout << "  ECUConfig(DataSource) ctor — channel "
                  << channel_ << "\n";
    }

    const std::string& getName() const { return name_; }
};

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 2: RVO/NRVO Demo — SensorPacket with Noisy Ctors
// EN: Tracks every copy/move to prove they are elided by the compiler
// TR: Derleyicinin kopyalama/taşımayı ortadan kaldırdığını kanıtlamak için izler
// ═══════════════════════════════════════════════════════════════════════════════

class SensorPacket {
    std::string tag_;
    double value_;
public:
    explicit SensorPacket(std::string tag, double val)
        : tag_(std::move(tag)), value_(val) {
        std::cout << "  [SensorPacket CONSTRUCTED: " << tag_ << "]\n";
    }

    SensorPacket(const SensorPacket& other)
        : tag_(other.tag_), value_(other.value_) {
        std::cout << "  [SensorPacket COPY ctor: " << tag_ << "]\n";
    }

    SensorPacket(SensorPacket&& other) noexcept
        : tag_(std::move(other.tag_)), value_(other.value_) {
        std::cout << "  [SensorPacket MOVE ctor: " << tag_ << "]\n";
    }

    SensorPacket& operator=(const SensorPacket&) = default;
    SensorPacket& operator=(SensorPacket&&) noexcept = default;
    ~SensorPacket() = default;

    friend std::ostream& operator<<(std::ostream& os, const SensorPacket& sp) {
        return os << sp.tag_ << " = " << sp.value_;
    }
};
```

```cpp

// EN: Returns a local object by value — NRVO should elide the copy/move
// TR: Yerel nesneyi değer ile döndürür — NRVO kopyayı/taşımayı ortadan kaldırır
SensorPacket createPacket() {
    SensorPacket packet("EngineTemp", 92.5);
    return packet;
}

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 3: explicit Constructor — Voltage
// EN: Prevents dangerous implicit double → Voltage conversions
// TR: Tehlikeli örtülü double → Voltage dönüşümlerini engeller
// ═══════════════════════════════════════════════════════════════════════════════

class Voltage {
    double volts_;
public:
    // EN: explicit prevents: Voltage v = 3.3; (implicit conversion)
    // TR: explicit şunu engeller: Voltage v = 3.3; (örtülü dönüşüm)
    explicit Voltage(double v) : volts_(v) {}

    double get() const { return volts_; }

    friend std::ostream& operator<<(std::ostream& os, const Voltage& v) {
        return os << v.volts_ << "V";
    }
};

// EN: Without explicit, this function could be called with a plain double!
// TR: explicit olmasaydı, bu fonksiyona düz bir double ile çağrı yapılabilirdi!
void checkVoltage(const Voltage& v) {
    std::cout << "  Checking voltage: " << v << "\n";
    if (v.get() > 14.0) {
        std::cout << "  WARNING: Over-voltage detected!\n";
    }
}

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 4: Dangling Reference Anti-Pattern (safe demonstration)
// EN: Shows the WRONG way and the RIGHT way to return data from functions
// TR: Fonksiyonlardan veri döndürmenin YANLIŞ ve DOĞRU yolunu gösterir
// ═══════════════════════════════════════════════════════════════════════════════

// EN: WRONG — returns reference to a local (dangling after return!)
// TR: YANLIŞ — yerel değişkene referans döndürür (dönüşten sonra sarkık!)
// const std::string& getBrandDangling() {
//     std::string brand = "AutomotiveCorp";
//     return brand;  // DANGLING: brand is destroyed here!
// }

// EN: RIGHT — return by value (RVO will optimize it anyway)
// TR: DOĞRU — değer ile döndür (RVO zaten optimize edecek)
std::string getBrandSafe() {
    std::string brand = "AutomotiveCorp";
    return brand;
}

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 5: Evaluation Order Demo (C++17)
// EN: Demonstrates guaranteed left-to-right evaluation in chained calls
// TR: Zincirleme çağrılarda garantili soldan sağa değerlendirmeyi gösterir
// ═══════════════════════════════════════════════════════════════════════════════

class DiagnosticChain {
    std::vector<std::string> log_;
public:
    // EN: Each method returns *this for chaining, logs order of calls
    // TR: Her metot zincirleme için *this döndürür, çağrı sırasını loglar
    DiagnosticChain& checkEngine(int code) {
        log_.push_back("Engine[" + std::to_string(code) + "]");
        return *this;
    }

    DiagnosticChain& checkBrakes(int code) {
        log_.push_back("Brakes[" + std::to_string(code) + "]");
        return *this;
    }
```

```cpp

    DiagnosticChain& checkTransmission(int code) {
        log_.push_back("Transmission[" + std::to_string(code) + "]");
        return *this;
    }

    void printLog() const {
        std::cout << "  Evaluation order: ";
        for (size_t i = 0; i < log_.size(); ++i) {
            if (i > 0) std::cout << " -> ";
            std::cout << log_[i];
        }
        std::cout << "\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════════════
// MAIN — Demonstrations
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "═══════════════════════════════════════════════════════\n";
    std::cout << " Interview Gotchas — C++ Certification Prep\n";
    std::cout << "═══════════════════════════════════════════════════════\n\n";

    // ─── Demo 1: Most Vexing Parse ─────────────────────────────────────────
    // EN: Parentheses = function declaration! Braces = correct construction.
    // TR: Parantez = fonksiyon bildirimi! Süslü parantez = doğru oluşturma.
    std::cout << "--- Demo 1: Most Vexing Parse ---\n";
    {
        // EN: BROKEN — this declares a FUNCTION, not an object:
        // TR: BOZUK — bu bir NESNE değil, FONKSİYON bildirir: ECUConfig config(DataSource());
        // // Function declaration!

        // EN: FIX 1 — brace initialization (preferred in modern C++)
        // TR: ÇÖZÜM 1 — süslü parantez başlatma (modern C++'ta tercih edilir)
        ECUConfig config1{DataSource()};
        std::cout << "  config1 name: " << config1.getName() << "\n";

        // EN: FIX 2 — extra parentheses around argument
        // TR: ÇÖZÜM 2 — argüman etrafında ekstra parantez
        ECUConfig config2((DataSource()));
        std::cout << "  config2 name: " << config2.getName() << "\n";

        // EN: FIX 3 — named variable for the argument
        // TR: ÇÖZÜM 3 — argüman için isimli değişken
        DataSource src;
        ECUConfig config3(src);
        std::cout << "  config3 name: " << config3.getName() << "\n";
    }

    // ─── Demo 2: RVO — zero copies ─────────────────────────────────────────
    // EN: Noisy constructors prove that no copy or move happens
    // TR: Gürültülü yapıcılar kopya veya taşıma olmadığını kanıtlar
    std::cout << "\n--- Demo 2: RVO/NRVO (Return Value Optimization) ---\n";
    {
        std::cout << "Creating packet via function return:\n";
        SensorPacket pkt = createPacket();
        std::cout << "  Packet: " << pkt << "\n";
        std::cout << "  (If no COPY/MOVE printed above, NRVO worked!)\n";
    }

    // ─── Demo 3: explicit prevents implicit conversion ─────────────────────
    // EN: Without explicit, checkVoltage(3.3) would silently compile
    // TR: explicit olmasaydı, checkVoltage(3.3) sessizce derlenir
    std::cout << "\n--- Demo 3: explicit constructor ---\n";
    {
        Voltage batteryVoltage(12.6);
        checkVoltage(batteryVoltage);

        // EN: This would fail to compile — no implicit double → Voltage:
        // TR: Bu derlenmez — örtülü double → Voltage dönüşümü yok: checkVoltage(3.3);  // ERROR:
        // no implicit conversion

```

```
        // EN: Must be explicit:
        // TR: Açık olmalı:
        checkVoltage(Voltage(14.5));
    }

    // ─── Demo 4: Temporary lifetime extension ──────────────────────────────────────
    // EN: Binding a temporary to const& extends its lifetime
    // TR: Bir geçiciyi const&'ye bağlamak ömrünü uzatır
    std::cout << "\n--- Demo 4: Temporary lifetime extension ---\n";
    {
        // EN: Safe — return by value, stored in local variable
        // TR: Güvenli — değer ile dönüş, yerel değişkende saklanır
        std::string brand = getBrandSafe();
        std::cout << "  Brand (by value): " << brand << "\n";

        // EN: Safe — const& extends the temporary's lifetime to this scope
        // TR: Güvenli — const& geçicinin ömrünü bu kapsama kadar uzatır
        const std::string& tempRef = getBrandSafe();
        std::cout << "  Brand (const& to temp): " << tempRef << "\n";

        // EN: Safe — auto&& (forwarding ref) also extends lifetime
        // TR: Güvenli — auto&& (yönlendirme ref) de ömrü uzatır
        auto&& fwdRef = getBrandSafe();
        std::cout << "  Brand (auto&& to temp): " << fwdRef << "\n";
    }

    // ─── Demo 5: Evaluation order — C++17 left-to-right ────────────────────────────
    // EN: Chained member function calls are evaluated left-to-right in C++17
    // TR: Zincirleme üye fonksiyon çağrıları C++17'de soldan sağa değerlendirilir
    std::cout << "\n--- Demo 5: Evaluation order (C++17) ---\n";
    {
        DiagnosticChain diag;
        // EN: C++17 guarantees: checkEngine → checkBrakes → checkTransmission
        // TR: C++17 garantisi: checkEngine → checkBrakes → checkTransmission
        diag.checkEngine(101)
            .checkBrakes(202)
            .checkTransmission(303);
        diag.printLog();
    }

    std::cout << "\n═══════════════════════════════════════════════════════════\n";
    std::cout << " All interview gotcha demos completed successfully!\n";
    std::cout << "═══════════════════════════════════════════════════════════\n";

    return 0;
}
```

► 07. Error Handling Advanced

module_05_certification/07_error_handling_advanced.cpp

Konu: Advanced Error Handling — İleri Seviye Hata Yönetimi

Teori / Theory

THEORY: Error Handling Strategies in C++

C++ offers multiple error handling approaches. Each has trade-offs:

```
┌──────────────────────┬──────────┬──────────┬──────────────────────┐
│ Approach             │ Speed    │ Safety   │ When to Use          │
├──────────────────────┼──────────┼──────────┼──────────────────────┤
│ Exceptions           │ Zero-cost│ High     │ Truly exceptional    │
│                      │ (happy   │          │ (file not found, OOM) │
│                      │  path)   │          │                      │
├──────────────────────┼──────────┼──────────┼──────────────────────┤
│ Return codes (int)   │ Fast     │ Low      │ C interop, embedded  │
│                      │          │ (ignored) │ (no exceptions)      │
├──────────────────────┼──────────┼──────────┼──────────────────────┤
│ std::error_code      │ Fast     │ Medium   │ System/OS errors     │
│                      │          │          │ (errno, HRESULT)     │
├──────────────────────┼──────────┼──────────┼──────────────────────┤
│ std::optional<T>     │ Fast     │ High     │ May or may not have  │
│                      │          │          │ a value (no error)   │
├──────────────────────┼──────────┼──────────┼──────────────────────┤
│ std::expected<T,E>   │ Fast     │ High     │ Value-or-error,      │
│ (C++23)              │          │          │ Rust Result<T,E> like │
└──────────────────────┴──────────┴──────────┴──────────────────────┘
```

TEORİ: C++'ta Hata Yönetimi Stratejileri

C++ birden fazla hata yönetimi yaklaşımı sunar. Her birinin ödünleşimleri vardır:

```
┌──────────────────────┬──────────┬──────────┬──────────────────────┐
│ Yaklaşım             │ Hız      │ Güvenlik │ Ne Zaman Kullanılır  │
├──────────────────────┼──────────┼──────────┼──────────────────────┤
│ İstisnalar           │ Sıfır    │ Yüksek   │ Gerçekten istisnai   │
│                      │ maliyet  │          │ (dosya yok, OOM)     │
│                      │ (mutlu   │          │                      │
│                      │  yol)    │          │                      │
├──────────────────────┼──────────┼──────────┼──────────────────────┤
│ Dönüş kodları (int)  │ Hızlı    │ Düşük    │ C interop, gömülü    │
│                      │          │ (yoksay.) │ (istisna desteği yok) │
├──────────────────────┼──────────┼──────────┼──────────────────────┤
│ std::error_code      │ Hızlı    │ Orta     │ Sistem/OS hataları   │
│                      │          │          │ (errno, HRESULT)     │
├──────────────────────┼──────────┼──────────┼──────────────────────┤
│ std::optional<T>     │ Hızlı    │ Yüksek   │ Değer olabilir veya  │
│                      │          │          │ olmayabilir (hata yok)│
├──────────────────────┼──────────┼──────────┼──────────────────────┤
│ std::expected<T,E>   │ Hızlı    │ Yüksek   │ Değer-veya-hata,     │
│ (C++23)              │          │          │ Rust Result<T,E> bnzr │
└──────────────────────┴──────────┴──────────┴──────────────────────┘
```

THEORY: std::error_code

The <system_error> framework provides:
std::error_code — A lightweight, non-throwing error container:
- Holds a numeric code + category pointer
- Category identifies the error domain (system, generic, custom)
- Pattern: function returns error_code, caller checks
std::error_condition — Platform-independent error representation std::error_category — Base class for error domains std::system_error — Exception that wraps error_code
Why system_error over raw int?
- Type-safe: can't mix errno with HRESULT
- Portable: std::errc maps to platform-specific codes
- Composable: Custom categories for your domain

TEORİ: std::error_category

<system_error> çerçevesi şunları sağlar:
std::error_code — Hafif, istisna fırlatmayan hata konteyneri:
- Sayısal kod + kategori işaretçisi tutar
- Kategori, hata alanını tanımlar (sistem, genel, özel)
- Desen: fonksiyon error_code döndürür, çağıran kontrol eder
std::error_condition — Platform bağımsız hata temsili std::error_category — Hata alanları için temel sınıf std::system_error — error_code saran istisna
Neden ham int yerine system_error?
- Tip güvenli: errno ile HRESULT karıştırılamaz
- Taşınabilir: std::errc platforma özgü kodlara eşlenir
- Birleştirilebilir: Kendi alanınız için özel kategoriler

THEORY: std::expected<T,E> — The Modern Way (C++23)

std::expected<T,E> holds either a T value or an E error:
auto result = parse_sensor_data(raw); if (result.has_value()) { use(result.value()); // or *result } else { handle(result.error()); // The error }
Monadic operations (C++23): result.and_then(f) — If value, apply f (returns expected) result.transform(f) — If value, apply f (returns wrapped value) result.or_else(f) — If error, apply f (returns expected)
This enables functional-style chaining: auto final = read_file("data.bin") .and_then(parse_json) .transform(extract_temperature) .or_else(log_and_default);

TEORİ: std::expected<T,E> — Modern Yaklaşım (C++23)

std::expected<T,E> ya T değeri ya da E hatası tutar:
auto result = parse_sensor_data(raw); if (result.has_value()) { use(result.value()); // veya *result } else { handle(result.error()); // Hata }
Monadik işlemler (C++23): result.and_then(f) — Değer varsa, f uygula (expected döndürür) result.transform(f) — Değer varsa, f uygula (sarılı değer döndürür) result.or_else(f) — Hata varsa, f uygula (expected döndürür)
Bu, fonksiyonel tarzda zincirleme sağlar: auto final = read_file("data.bin") .and_then(parse_json) .transform(extract_temperature) .or_else(log_and_default);

Kaynak Kod / Source Code

```
#include <iostream>
#include <string>
#include <vector>
#include <system_error>
#include <variant>
#include <optional>
#include <functional>
#include <cmath>
```

```cpp
#include <cerrno>
#include <cstring>
#include <cstdint>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: std::error_code and Custom Error Categories
// ═══════════════════════════════════════════════════════════════════════════

// EN: Define a custom error category for our sensor subsystem
// TR: Sensör alt sistemi için özel hata kategorisi tanımla
enum class SensorError {
    Success = 0,
    Timeout = 1,
    OutOfRange = 2,
    CalibrationFailed = 3,
    HardwareFault = 4
};

// EN: Custom error category — describes our error domain
// TR: Özel hata kategorisi — hata alanımızı tanımlar
class SensorErrorCategory : public std::error_category {
public:
    const char* name() const noexcept override {
        return "SensorError";
    }

    std::string message(int ev) const override {
        switch (static_cast<SensorError>(ev)) {
            case SensorError::Success:           return "Success";
            case SensorError::Timeout:           return "Sensor read timeout";
            case SensorError::OutOfRange:        return "Value out of valid range";
            case SensorError::CalibrationFailed: return "Calibration failed";
            case SensorError::HardwareFault:     return "Hardware fault detected";
            default:                             return "Unknown sensor error";
        }
    }
};

// EN: Global category instance (singleton pattern)
// TR: Global kategori örneği (tekil kalıp)
const SensorErrorCategory& sensor_error_category() {
    static SensorErrorCategory instance;
    return instance;
}

std::error_code make_error_code(SensorError e) {
    return {static_cast<int>(e), sensor_error_category()};
}

// EN: Register SensorError with std::error_code system
// TR: SensorError'u std::error_code sistemiyle kaydet
namespace std {
    template<>
    struct is_error_code_enum<SensorError> : true_type {};
}

// EN: Function that returns error_code instead of throwing
// TR: İstisna fırlatmak yerine error_code döndüren fonksiyon
std::error_code read_sensor(int sensor_id, double& value) {
    if (sensor_id < 0) {
        return SensorError::HardwareFault;
    }
    if (sensor_id > 10) {
        return SensorError::Timeout;
    }
    value = 25.0 + static_cast<double>(sensor_id) * 1.5;
    if (value > 35.0) {
        return SensorError::OutOfRange;
    }
    return {};  // EN: Default-constructed = success / TR: Varsayılan = başarılı
}

```

```cpp
// ═══════════════════════════════════════════════════════════════════════════
// PART 2: Expected<T,E> — C++23 Polyfill
// ═══════════════════════════════════════════════════════════════════════════

// EN: Simplified std::expected polyfill for C++17 demonstration
//     Real C++23 std::expected has more features, but the pattern is the same
// TR: C++17 gösterimi için basitleştirilmiş std::expected polyfill'i
//     Gerçek C++23 std::expected daha fazla özelliğe sahiptir
template<typename T, typename E>
class Expected {
public:
    // EN: Value constructors — use in_place_index to disambiguate when T==E
    // TR: Değer kurucu — T==E olduğunda belirsizliği gidermek için in_place_index kullan
    Expected(T value) : data_(std::in_place_index<0>, std::move(value)) {}

    // EN: Error constructor — use tag type to disambiguate
    // TR: Hata kurucu — belirsizliği gidermek için etiket tipi kullan
    struct unexpect_t {};
    Expected(unexpect_t, E error) : data_(std::in_place_index<1>, std::move(error)) {}

    // EN: Static factory for errors
    // TR: Hatalar için statik fabrika
    static Expected unexpected(E error) {
        return Expected(unexpect_t{}, std::move(error));
    }

    bool has_value() const { return data_.index() == 0; }
    explicit operator bool() const { return has_value(); }

    T& value() { return std::get<0>(data_); }
    const T& value() const { return std::get<0>(data_); }

    E& error() { return std::get<1>(data_); }
    const E& error() const { return std::get<1>(data_); }

    T& operator*() { return value(); }
    const T& operator*() const { return value(); }

    // EN: Monadic operations (C++23 style)
    // TR: Monadik işlemler (C++23 tarzı)
    // and_then: If value, apply f (f returns Expected<U,E>)
    template<typename F>
    auto and_then(F&& f) -> decltype(f(std::declval<T>())) {
        if (has_value()) return f(value());
        return decltype(f(std::declval<T>()))::unexpected(error());
    }

    // transform: If value, apply f (wraps result in Expected)
    template<typename F>
    auto transform(F&& f) -> Expected<decltype(f(std::declval<T>())), E> {
        using U = decltype(f(std::declval<T>()));
        if (has_value()) return Expected<U, E>(f(value()));
        return Expected<U, E>::unexpected(error());
    }

    // or_else: If error, apply f
    template<typename F>
    Expected or_else(F&& f) {
        if (has_value()) return *this;
        return f(error());
    }

private:
    std::variant<T, E> data_;
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: Practical Error Handling Pipeline
// ═══════════════════════════════════════════════════════════════════════════

struct SensorData {
    double temperature;
    double pressure;
};
```

```cpp

// EN: Each function in the pipeline returns Expected
// TR: Pipeline'daki her fonksiyon Expected döndürür
Expected<std::string, std::string> read_raw_data(int sensor_id) {
    if (sensor_id < 0)
        return Expected<std::string, std::string>::unexpected("Invalid sensor ID");
    if (sensor_id > 100)
        return Expected<std::string, std::string>::unexpected("Sensor timeout");
    return std::string("temp=25.3,pressure=1013.25");
}

Expected<SensorData, std::string> parse_data(const std::string& raw) {
    // EN: Simple parsing simulation
    // TR: Basit ayrıştırma simülasyonu
    if (raw.empty())
        return Expected<SensorData, std::string>::unexpected("Empty data");
    return SensorData{25.3, 1013.25};
}

Expected<double, std::string> validate_temperature(const SensorData& data) {
    if (data.temperature < -40.0 || data.temperature > 150.0)
        return Expected<double, std::string>::unexpected("Temperature out of range");
    return data.temperature;
}

// ═══════════════════════════════════════════════════════════════════════════
// PART 4: Exception Safety Guarantee Levels
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: Exception Safety Guarantees — Every C++ developer must know these:
 *
 *      ┌──────────────────────┬──────────────────────────────────────────┐
 *      │ Guarantee Level      │ Promise                                  │
 *      ├──────────────────────┼──────────────────────────────────────────┤
 *      │ No-throw             │ Operation NEVER throws. Declared noexcept.│
 *      │                      │ Examples: destructors, swap, move operations│
 *      ├──────────────────────┼──────────────────────────────────────────┤
 *      │ Strong (rollback)    │ If exception thrown, state is unchanged (as if│
 *      │                      │ operation never happened). Copy-and-swap idiom.│
 *      ├──────────────────────┼──────────────────────────────────────────┤
 *      │ Basic                │ No resource leaks, invariants preserved, but│
 *      │                      │ state may have changed. Minimum acceptable.│
 *      ├──────────────────────┼──────────────────────────────────────────┤
 *      │ No guarantee         │ Could leak, corrupt state. Unacceptable!  │
 *      └──────────────────────┴──────────────────────────────────────────┘
 */

// EN: Strong guarantee example: copy-and-swap idiom
// TR: Güçlü garanti örneği: kopyala-ve-takas deyimi
class SensorArray {
public:
    explicit SensorArray(size_t n) : data_(n, 0.0) {}

    // EN: Strong guarantee: If push_back throws, old state preserved
    // TR: Güçlü garanti: push_back fırlatırsa, eski durum korunur
    void add_reading_strong(double value) {
        auto copy = data_;              // 1. Copy current state
        copy.push_back(value);          // 2. Modify copy (may throw)
        data_.swap(copy);               // 3. Swap (noexcept!) — COMMIT
    }

    // EN: Basic guarantee: State is valid but modified on exception
    // TR: Temel garanti: İstisna durumunda durum geçerli ama değişmiş olabilir
    void add_reading_basic(double value) {
        data_.push_back(value);  // May throw (reallocation), but vector
                                 // provides basic guarantee internally
    }

    size_t size() const noexcept { return data_.size(); }

private:
    std::vector<double> data_;
};
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════
// PART 5: noexcept Deep Dive
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: noexcept is a CONTRACT and an OPTIMIZATION:
 *
 *     1. noexcept on destructors: ALWAYS (default since C++11)
 *     2. noexcept on move operations: CRITICAL for vector optimization
 *        - vector::push_back uses move IF move ctor is noexcept
 *        - Otherwise falls back to copy (for strong guarantee)
 *     3. noexcept on swap: Always (swap should never fail)
 *     4. noexcept(expr): Conditional noexcept based on expression
 *
 *     Performance impact:
 *     - noexcept functions: compiler can elide stack unwinding code
 *     - vector reallocation: move vs copy (can be 100x difference!)
 *
 * TR: noexcept bir SÖZLEŞME ve OPTİMİZASYONdur:
 *     1. Yıkıcılarda: HER ZAMAN
 *     2. Taşıma işlemlerinde: KRİTİK (vector optimizasyonu için)
 *     3. Swap'ta: Her zaman
 *     4. noexcept(expr): İfadeye dayalı koşullu noexcept
 */

class EcuFirmware {
    std::string version_;
    std::vector<uint8_t> binary_;

public:
    EcuFirmware(std::string ver, std::vector<uint8_t> bin)
        : version_(std::move(ver)), binary_(std::move(bin)) {}

    // EN: noexcept move — vector will use move during reallocation
    // TR: noexcept move — vector yeniden boyutlandırmada move kullanacak
    EcuFirmware(EcuFirmware&& other) noexcept
        : version_(std::move(other.version_))
        , binary_(std::move(other.binary_)) {}

    EcuFirmware& operator=(EcuFirmware&& other) noexcept {
        version_ = std::move(other.version_);
        binary_ = std::move(other.binary_);
        return *this;
    }

    // EN: Conditional noexcept — noexcept if string comparison is noexcept
    // TR: Koşullu noexcept — string karşılaştırması noexcept ise noexcept
    bool is_same_version(const EcuFirmware& other) const
        noexcept(noexcept(version_ == other.version_)) {
        return version_ == other.version_;
    }

    const std::string& version() const noexcept { return version_; }
    size_t binary_size() const noexcept { return binary_.size(); }
};

// ═══════════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 05 - Advanced Error Handling\n";
    std::cout << "============================================\n\n";

    // ─── Demo 1: std::error_code with Custom Category ──────────────────
    {
        std::cout << "--- Demo 1: std::error_code & Custom Category ---\n";

        // EN: Success case
        // TR: Başarılı durum
        double value = 0.0;
        std::error_code ec = read_sensor(3, value);
        std::cout << "    Sensor 3: " << (ec ? "FAIL" : "OK")
                  << ", value=" << value << "\n";

```

```
        // EN: Error cases
        // TR: Hata durumları
        ec = read_sensor(-1, value);
        std::cout << "    Sensor -1: " << ec.message()
                  << " (category: " << ec.category().name()
                  << ", code: " << ec.value() << ")\n";

        ec = read_sensor(15, value);
        std::cout << "    Sensor 15: " << ec.message() << "\n";

        // EN: System error codes (standard errno mapping)
        // TR: Sistem hata kodları (standart errno eşlemesi)
        auto sys_ec = std::make_error_code(std::errc::no_such_file_or_directory);
        std::cout << "    System: " << sys_ec.message()
                  << " (category: " << sys_ec.category().name() << ")\n\n";
    }

    // ─── Demo 2: Expected<T,E> Monadic Chaining ──────────────────────
    {
        std::cout << "--- Demo 2: Expected<T,E> Monadic Chaining ---\n";

        // EN: Success pipeline: read → parse → validate
        // TR: Başarı pipeline'ı: oku → ayrıştır → doğrula
        auto result = read_raw_data(5)
            .and_then(parse_data)
            .and_then(validate_temperature);

        if (result) {
            std::cout << "    Pipeline OK: temperature = " << *result << " C\n";
        } else {
            std::cout << "    Pipeline FAIL: " << result.error() << "\n";
        }

        // EN: Error pipeline — first error short-circuits
        // TR: Hata pipeline'ı — ilk hata kısa devre yapar
        auto err_result = read_raw_data(-1)
            .and_then(parse_data)
            .and_then(validate_temperature);

        if (!err_result) {
            std::cout << "    Error pipeline: " << err_result.error() << "\n";
        }

        // EN: transform example — convert temperature to string
        // TR: transform örneği — sıcaklığı string'e dönüştür
        auto str_result = read_raw_data(5)
            .and_then(parse_data)
            .and_then(validate_temperature)
            .transform([](double temp) {
                return std::to_string(temp) + " C";
            });

        if (str_result) {
            std::cout << "    Transform result: " << *str_result << "\n";
        }

        // EN: or_else example — provide fallback on error
        // TR: or_else örneği — hata durumunda yedek değer sağla
        auto fallback_result = read_raw_data(-1)
            .and_then(parse_data)
            .and_then(validate_temperature)
            .or_else([](const std::string& err) -> Expected<double, std::string> {
                std::cout << "    or_else handler: " << err << " → using default\n";
                return 20.0;  // EN: Fallback value
            });

        if (fallback_result) {
            std::cout << "    Fallback value: " << *fallback_result << " C\n\n";
        }
    }
```

```cpp

    // ─── Demo 3: Exception Safety Guarantees ──────────────────────────
    {
        std::cout << "--- Demo 3: Exception Safety Guarantees ---\n";
        SensorArray arr(3);
        arr.add_reading_strong(42.0);
        arr.add_reading_basic(43.0);
        std::cout << "    SensorArray size after 2 adds: " << arr.size() << "\n";

        std::cout << "    add_reading_strong: Copy-and-swap → strong guarantee\n";
        std::cout << "    add_reading_basic:  Direct push_back → basic guarantee\n\n";

        std::cout << "    Guarantee levels:\n";
        std::cout << "      No-throw:  destructors, swap, move (noexcept)\n";
        std::cout << "      Strong:    copy-and-swap, commit-or-rollback\n";
        std::cout << "      Basic:     no leaks, invariants hold, state may change\n";
        std::cout << "      None:      UNACCEPTABLE — fix this!\n\n";
    }

    // ─── Demo 4: noexcept Impact on Containers ────────────────────────
    {
        std::cout << "--- Demo 4: noexcept and Container Optimization ---\n";
        std::cout << "    EcuFirmware move ctor noexcept: "
                  << std::boolalpha
                  << std::is_nothrow_move_constructible<EcuFirmware>::value << "\n";
        std::cout << "    std::string move ctor noexcept: "
                  << std::is_nothrow_move_constructible<std::string>::value << "\n";
        std::cout << "    std::vector<int> move noexcept: "
                  << std::is_nothrow_move_constructible<std::vector<int>>::value << "\n\n";

        // EN: With noexcept move → vector uses move during reallocation
        //     Without noexcept → vector copies (for strong exception guarantee)
        // TR: noexcept move ile → vector yeniden boyutlamada move kullanır
        //     noexcept olmadan → vector kopyalar (güçlü istisna garantisi için)
        std::vector<EcuFirmware> firmwares;
        firmwares.reserve(2);  // EN: Start small to force reallocation
        firmwares.emplace_back("v1.0", std::vector<uint8_t>(1024));
        firmwares.emplace_back("v2.0", std::vector<uint8_t>(2048));
        firmwares.emplace_back("v3.0", std::vector<uint8_t>(4096));  // Triggers realloc

        std::cout << "    Stored " << firmwares.size() << " firmwares\n";
        std::cout << "    (noexcept move → vector used MOVE during realloc)\n";
        std::cout << "    (without noexcept → vector would COPY → much slower!)\n\n";
    }

    // ─── Demo 5: Error Handling Strategy Decision ─────────────────────
    {
        std::cout << "--- Demo 5: Error Handling Strategy Guide ---\n";
        std::cout << "  ┌──────────────────────────┬──────────────────────────────┐\n";
        std::cout << "  │ Situation                │ Use                          │\n";
        std::cout << "  ├──────────────────────────┼──────────────────────────────┤\n";
        std::cout << "  │ Programming errors       │ assert() / preconditions     │\n";
        std::cout << "  │ Truly exceptional        │ throw exceptions             │\n";
        std::cout << "  │ Expected failures        │ std::expected<T,E> (C++23)   │\n";
        std::cout << "  │ Missing value            │ std::optional<T>             │\n";
        std::cout << "  │ OS/system errors         │ std::error_code              │\n";
        std::cout << "  │ Embedded / no-except     │ Return codes + error_code    │\n";
        std::cout << "  │ Chained operations       │ expected + and_then/transform│\n";
        std::cout << "  └──────────────────────────┴──────────────────────────────┘\n\n";
    }

    std::cout << "============================================\n";
    std::cout << "   End of Advanced Error Handling\n";
    std::cout << "============================================\n";

    return 0;
}
```

Modül 6: Industry Standards — Endüstri Standartları

Endüstri Standartları

11 kaynak dosya · Standart: C++17

C++17

► 01. Concurrency Threads

module_06_industry_standards/01_concurrency_threads.cpp

Konu: Industry Standards: Threading and Concurrency — Endüstri Standartları: İş Parçacıkları

Teori / Theory

THEORY: Why Multithreading?

Single-threaded programs execute one line at a time. If a video is rendering and taking 5 minutes, the entire UI freezes. Modern CPUs have 8, 16, or 32 cores. `std::thread` allows C++ to split work across these cores simultaneously.

TEORİ: Neden Çoklu İş Parçacığı?

Tek iş parçacıklı (single-threaded) programlar aynı anda tek satır okur. Eğer bir video render ediliyorsa ve 5 dakika sürüyorsa, bütün kullanıcı arayüzü (UI) donar. Modern işlemciler 16-32 çekirdeklidir. C++11 ile gelen `std::thread`, donanımın tüm gücünü sömürerek işleri yan yana (aynı anda) fiziksel olarak çalıştırmamızı sağlar.

THEORY: Race Condition and Mutex

Two threads trying to write to the SAME variable at the EXACT same nanosecond causes a "Race Condition" (corruption). We must protect shared data. `std::mutex` acts like a bathroom key. Only one thread can hold the key, modify data, and return the key. Others wait at the door.

TEORİ: Yarış Durumu ve Mutex Kilidi

İki ayrı Thread'in saniyenin milyarda biri hızla aynı değişkene (aynı RAM adresine) YAZMAYA çalışmasına 'Race Condition' (Yarış Durumu) denir. Veri anında bozulur. Bunu çözmek için `std::mutex` (Mutual Exclusion / Karşılıklı Dışlama) kullanırız. Mutex bir "Kilitli Kapı Anahtarı" gibidir. Bir Thread anahtarı alır (`lock()`), veriyi değiştirir ve işi bitince anahtarı bırakır (`unlock()`). Diğer Thread'ler kapıda beklemek zorundadır.

THEORY: std::atomic — Lock-Free Thread Safety

For simple types (int, bool, pointers), `std::atomic<T>` provides thread-safe reads and writes WITHOUT needing a mutex. The CPU guarantees the operation completes in a single indivisible step. This is dramatically faster than mutex for counters, flags, and simple shared state. Operations like `fetch_add()`, `compare_exchange_strong()`, and `store()/load()` are all atomic.

TEORİ: std::atomic — Kilitsiz Thread Güvenliği

Basit tipler (int, bool, pointer) için `std::atomic<T>`, mutex'e gerek kalmadan thread-güvenli okuma/yazma sağlar. CPU, işlemin TEK bölünmez adımda tamamlanacağını garanti eder. Sayaçlar, bayraklar ve basit paylaşımlı durum için mutex'ten DRAMATIK şekilde hızlıdır. `fetch_add()`, `compare_exchange_strong()`, `store()/load()` gibi işlemler atomiktir.

THEORY: std::condition_variable — Thread Signaling

Sometimes a thread needs to WAIT until a certain condition is met (e.g., "wait until the queue has data"). Busy-waiting (spin loop) wastes CPU. `std::condition_variable` lets a thread sleep (zero CPU) until another thread calls `notify_one()` or `notify_all()`. This is the foundation of Producer-Consumer patterns.

TEORİ: std::condition_variable — Thread Sinyalleşmesi

Bazen bir thread belirli bir koşul gerçekleşene kadar BEKLEMELİDİR (örn. "kuyrukta veri olana kadar bekle"). Sürekli döngüde kontrol etmek (spin loop) CPU israfıdır. `std::condition_variable`, bir thread'in sıfır CPU ile uyumasına ve başka bir thread `notify_one()` veya `notify_all()` çağırana kadar beklemesine olanak tanır. Producer-Consumer kalıbının temelidir.

CPPREF Depth: Data Race is PURE Undefined Behavior!

CppReference explicitly states: A Data Race isn't just "getting the wrong math result". It is strictly Undefined Behavior (UB)! The compiler is allowed to completely optimize away your loop, crash the program, or print garbage. Even READING a variable while another thread is WRITING to it is a Data Race!

CPPREF Derinlik: Veri Yarışı SAF Tanımsız Davranıştır!

Data Race (Veri Yarışı) C++'ta sadece "Yanlış matematiksel sonuç çıkması" demek değildir. CppReference'a göre BU KESİN BİR UB'DİR (Tanımsız Davranış)! Derleyici kodun o kısmını tamamen silebilir, çökertebilir. Bir Thread yazarken diğerinin sadece OKUMASI bile UB'dir (Yarı yazılmış bit'leri -Torn Reads- okur).

CPPREF Depth: std::atomic Memory Ordering — The Hidden Complexity

By default, `std::atomic` uses `memory_order_seq_cst` (sequentially consistent), the STRONGEST and slowest ordering. For performance-critical code, weaker orderings like `memory_order_relaxed` (no ordering guarantees), `memory_order_acquire/release` (one-directional fence) exist. Using the wrong ordering leads to subtle, platform-specific bugs that only appear on ARM/weak-memory architectures. cppreference.com/w/cpp/atomic/memory_order

CPPREF Derinlik: std::atomic Bellek Sıralaması — Gizli Karmaşıklık

Varsayılan olarak `std::atomic`, `memory_order_seq_cst` (sıralı tutarlı) kullanır — en GÜÇLÜ ve en yavaş sıralama. Performans-kritik kodlar için `memory_order_relaxed` (sıralama garantisi yok), `memory_order_acquire/ release` (tek yönlü çit) gibi zayıf sıralamalar vardır. Yanlış sıralama, yalnızca ARM/zayıf-bellek mimarilerinde ortaya çıkan ince, platforma özgü hatalara yol açar. cppreference.com/w/cpp/atomic/memory_order

CPPREF Depth: Spurious Wakeups in condition_variable

`condition_variable::wait()` can wake up WITHOUT anyone calling `notify_*()`. This is called a "Spurious Wakeup" and is allowed by the POSIX and C++ standards. Always use the predicate overload: `cv.wait(lock, [&]{ return !queue.empty(); })` to handle this. The lambda is re-checked every time the thread wakes, spurious or not. cppreference.com/w/cpp/thread/condition_variable/wait

CPPREF Derinlik: condition_variable'da Sahte Uyanmalar

`condition_variable::wait()`, kimse `notify_*()` çağırmadan UYANABİLİR. Buna "Sahte Uyanma (Spurious Wakeup)" denir ve POSIX ile C++ standartları tarafından izin verilir. Her zaman predicate (koşul) aşırı yüklemesini kullanın: `cv.wait(lock, [&]{ return !queue.empty(); })`. Lambda, thread her uyandığında (sahte veya değil) yeniden kontrol edilir. cppreference.com/w/cpp/thread/condition_variable/wait

Kaynak Kod / Source Code

```cpp
#include <atomic>
#include <condition_variable>
#include <iostream>
#include <mutex>
#include <queue>
#include <thread>
#include <vector>

// EN: Shared Key
// TR: Ortak Kilit
std::mutex mtx;
int sharedBankAccount = 0;

/**
 * @brief EN: A task to be run by multiple threads at the same time.
 * TR: Aynı anda birden fazla iş parçacığı tarafından koşturulacak görev.
 */
void depositMoney(int id, int amount) {
  // EN: Thread ID for identification
  // TR: Thread kimliği
  (void)id;
  for (int i = 0; i < 10000; i++) {
    // [THEORY: std::lock_guard & RAII / TEORİ: std::lock_guard ve RAII]
    // EN: lock_guard works like unique_ptr (RAII). It locks the mutex when entering the scope
    // {...} and AUTOMATICALLY unlocks it when the scope ends. No risk of Deadlocks even if an
    // exception is thrown!
    //
    // TR: lock_guard tıpkı unique_ptr gibidir (RAII prensibi). Süslü parantez {...} içine
    // girerken Mutex'i otomatik kilitler (lock). Süslü parantez bitince otomatik olarak kilidi
    // AÇAR (unlock). Exception fırlasa bile sistem kilitlenmez (Deadlock olmaz!)

    std::lock_guard<std::mutex> lock(mtx);

    // EN: CRITICAL SECTION. Only ONE thread is allowed here at any given time.
    // TR: KRİTİK BÖLGE. Burada aynı anda sadece TEK bir thread islem yapabilir.
    sharedBankAccount += amount;

  // EN: lock_guard dies here and returns the key.
  // TR: lock_guard burada yıkılır ve anahtarı devreder.
  }
}
```

```

// ═══════════════════════════════════════════════════════════════════════════════
// [DEMO 2: std::atomic - Lock-Free Counter / Kilitsiz Sayaç]
// ═══════════════════════════════════════════════════════════════════════════════
std::atomic<int> atomicCounter{0};

void atomicIncrement(int iterations) {
  for (int i = 0; i < iterations; i++) {
    // EN: fetch_add is ONE atomic CPU instruction — no mutex needed!
    // TR: fetch_add TEK atomik CPU komutu — mutex gerekmez!
    atomicCounter.fetch_add(1, std::memory_order_relaxed);
  }
}

// ═══════════════════════════════════════════════════════════════════════════════
// [DEMO 3: Producer-Consumer with condition_variable]
// ═══════════════════════════════════════════════════════════════════════════════
std::queue<int> taskQueue;
std::mutex queueMtx;
std::condition_variable cv;
bool producerDone = false;

/**
 * @brief EN: Producer pushes tasks into the queue and notifies consumers.
 * TR: Üretici, kuyruğa görev ekler ve tüketicileri bilgilendirir.
 */
void producer(int taskCount) {
  for (int i = 1; i <= taskCount; i++) {
    {
      std::lock_guard<std::mutex> lock(queueMtx);
      taskQueue.push(i);
      std::cout << "[PRODUCER] Pushed task #" << i << std::endl;
    }
    // EN: Wake ONE waiting consumer thread
    // TR: Bekleyen BİR tüketiciyi uyandır
    cv.notify_one();
    std::this_thread::sleep_for(std::chrono::milliseconds(100));
  }
  {
    std::lock_guard<std::mutex> lock(queueMtx);
    producerDone = true;
  }
  // EN: Wake ALL remaining consumers to let them exit.
  // TR: Kalan TÜM tüketicileri çıkabilmeleri için uyandır.
  cv.notify_all();
}

/**
 * @brief EN: Consumer waits for tasks and processes them.
 * TR: Tüketici görev bekler ve işler.
 */
void consumer(int id) {
  while (true) {
    std::unique_lock<std::mutex> lock(queueMtx);
    // EN: Wait with predicate to handle SPURIOUS WAKEUPS safely!
    // TR: Sahte uyanmalara karşı predicate ile bekle!
    cv.wait(lock, [] { return !taskQueue.empty() || producerDone; });

    if (taskQueue.empty() && producerDone) {
      // EN: No more work
      // TR: İş kalmadı
      break;
    }

    int task = taskQueue.front();
    taskQueue.pop();
    // EN: Release lock before processing
    // TR: İşlemden önce kilidi bırak
    lock.unlock();

    std::cout << "[CONSUMER " << id << "] Processing task #" << task << std::endl;
  }
}
```

```cpp

int main() {
  std::cout << "=== MODULE 6: INDUSTRY STANDARDS - CONCURRENCY ===\n" << std::endl;

  // ==========================================================================================
  // PART 1: Mutex-Protected Bank Account (Original Demo)
  // ==========================================================================================
  std::cout << "--- PART 1: std::mutex + std::lock_guard ---\n" << std::endl;
  std::cout << "Main Thread started (Ana cekirdek acildi).\n" << std::endl;

  std::vector<std::thread> workers;
  for (int i = 0; i < 10; i++) {
    workers.push_back(std::thread(depositMoney, i, 1));
  }

  std::cout << "10 Threads LAUNCHED! Running massively in the background...\n"
               "(10 Thread Firlatildi! Arka planda devasa hizla calisiyorlar...)\n"
            << std::endl;

  for (auto &w : workers) {
    w.join();
  }

  std::cout << "Expected Bank Account: 100000" << std::endl;
  std::cout << "Actual Bank Account:   " << sharedBankAccount << std::endl;

  if (sharedBankAccount == 100000) {
    std::cout << "=> [SUCCESS] std::mutex prevented the Race Condition!\n" << std::endl;
  }

  // ==========================================================================================
  // PART 2: std::atomic — Lock-Free Counter
  // ==========================================================================================
  // EN: 10 threads each increment an atomic counter 10000 times. No mutex needed! The CPU
  // handles atomicity at the hardware level.
  // TR: 10 thread her biri atomik sayacı 10000 kez artırır. Mutex gerekmez! CPU atomikliği
  // donanım seviyesinde halleder.
  std::cout << "--- PART 2: std::atomic (Lock-Free Counter) ---\n" << std::endl;

  std::vector<std::thread> atomicWorkers;
  for (int i = 0; i < 10; i++) {
    atomicWorkers.emplace_back(atomicIncrement, 10000);
  }
  for (auto &w : atomicWorkers) {
    w.join();
  }

  std::cout << "Expected atomic counter: 100000" << std::endl;
  std::cout << "Actual atomic counter:   " << atomicCounter.load() << std::endl;

  if (atomicCounter.load() == 100000) {
    std::cout << "=> [SUCCESS] std::atomic is lock-free and correct!\n" << std::endl;
  }

  // ==========================================================================================
  // PART 3: Producer-Consumer with condition_variable
  // ==========================================================================================
  // EN: Classic multi-threaded pattern: 1 producer pushes tasks, 2 consumers process them.
  // condition_variable coordinates without busy-waiting.
  // TR: Klasik çoklu-thread kalıbı: 1 üretici görev ekler, 2 tüketici işler. condition_variable
  // meşgul-bekleme olmadan koordine eder.
  std::cout << "--- PART 3: Producer-Consumer (condition_variable) ---\n"
            << std::endl;

  std::thread prod(producer, 6);
  std::thread cons1(consumer, 1);
  std::thread cons2(consumer, 2);

  prod.join();
  cons1.join();
  cons2.join();

  std::cout << "\n=> Producer-Consumer pattern completed with zero busy-waiting!"
            << std::endl;

  return 0;
}
```

► 02. Cmake Basics.Txt

module_06_industry_standards/02_cmake_basics.txt

Kaynak Kod / Source Code

```
# CMakeLists.txt (Example / Ornek)
# ---------------------------------------------------------------------
# [THEORY: Why Do We Use CMake? / TEORİ: TEORİ: Neden CMake Kullanıyoruz?]
# EN:
# Compiling C++ code manually like `g++ a.cpp b.cpp -o main` works for small tasks.
# But what if you have hundreds of files, or external libraries written by someone
# else like "nlohmann_json" or "Google_Test"? It becomes impossible!
# CMake is a PERFECT "meta-build" tool. It allows your project to be compiled with
# the EXACT SAME STANDARD on Windows (Visual Studio), Linux (Make/Ninja), or Mac (Xcode).
#
# TR:
# C++ projelerinde her dosyayi "g++ a.cpp b.cpp -o main" seklinde derlemek kücük
# projelerde calisir. Ancak yüzlerce dosyaniz ve baska birinin yazdigi "Google_Test"
# gibi kütüphaneleriniz varsa bu mümkün degildir!
# CMake, projeyi her isletim sisteminde ayni standartta hatasız derlememizi
# saglayan MUKEMMEL bir meta-derleme aracidir. Endüstri standardidir.
# ---------------------------------------------------------------------

# 1. Minimum CMake Version Required
# EN: Tells the system the lowest acceptable CMake version.
# TR: Projenin gerektirdigi minimum CMake sürümü.
cmake_minimum_required(VERSION 3.10)

# 2. Project Declaration
# EN: Setting the Project Name, Version, and the Language (CXX = C++).
# TR: Proje Adi, Versiyonu ve Dil Belirleme (CXX).
project(ModernCppMastery VERSION 1.0.0 LANGUAGES CXX)

# 3. Setting C++ Standards (Crucial!)
# EN: Forcing the compiler to use C++17 rules strictly.
# TR: Hangi C++ Standardini kullanacagimizi derleyiciye dayatiyoruz (Zorunlu kiliyoruz).
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED True)

# 4. Source Files & Executable Target
# EN: "Create an application named MainApp utilizing these source files."
# TR: Kodlarimizin bulundugu baglantilari ve olusacak calistirilabilir dosyanin adini belirliyoruz.
# add_executable(MainApp src/main.cpp src/player.cpp)

# 5. External Libraries (Dependencies)
# EN: If we downloaded a package via vcpkg or conan, we bind it here.
# TR: Eger disaridan (vcpkg / conan yardimiyla) bir kütüphane indirilmisse onu kodumuzla birlestiririz.
# find_package(GTest REQUIRED)
# target_link_libraries(MainApp PRIVATE GTest::gtest_main)
```

► 03. Unit Testing Gtest

module_06_industry_standards/03_unit_testing_gtest.cpp

Konu: Industry Standards: Unit Testing (TDD) with Google Test Simulation — Endüstri

Teori / Theory

THEORY: Why is Unit Testing the Peak of Professionalism?

Writing code is easy. Proving that it works perfectly under every condition is engineering. If you change a function today, how do you know you didn't break something you wrote 3 months ago? "Test-Driven Development (TDD)" and frameworks like "Google Test (GTest)" automate this. The cycle:
1) Write a FAILING test,
2) Write minimal code to pass,
3) Refactor.

TEORİ: TDD Nedir?

Kod yazmak kolaydir. O kodun her kosulda (%100) calistigini "kanitlamak" ise yazılım mühendisligidir. Bugün bir fonksiyonda degisiklik yaptiginizda, 3 ay önce yazilan modulleri bozmadiginizi nereden biliyorsunuz? İste "Test-Driven Development (TDD)" yani Test Güdümlü Gelistirme burada devreye girer. "Google Test" (GTest), cogu savunma sanayi, otomotiv veya HFT (Finans) sirketinin C++ kodunu test etmek icin kullandigi standard kütüphanedir. Döngü: 1) BAŞARISIZ test yaz, 2) Geçecek minimum kodu yaz, 3) Refactor et.

THEORY: TEST() and TEST_F() Macros

In real GTest, `TEST(SuiteName, TestName)` defines a standalone test case. `TEST_F(FixtureName, TestName)` defines a test that uses a "Test Fixture" — a class with `SetUp()` and `TearDown()` methods that run before and after every test. Fixtures share setup logic (like creating database connections) across multiple tests without code duplication.

TEORİ: TEST() ve TEST_F() Makroları

Gerçek GTest'te `TEST(SüitAdı, TestAdı)` bağımsız bir test tanımlar. `TEST_F(FixtureAdı, TestAdı)` ise bir "Test Fixture" kullanan test tanımlar — her testten önce ve sonra çalışan `SetUp()` ve `TearDown()` metodlarına sahip bir sınıf. Fixture'lar kurulum mantığını (veritabanı bağlantısı oluşturma gibi) birden fazla test arasında kod tekrarı olmadan paylaşır.

THEORY: EXPECT vs ASSERT

`ASSERT_EQ(a, b)` — if fails, test STOPS immediately. Use for critical preconditions. `EXPECT_EQ(a, b)` — if fails, test CONTINUES running remaining checks (reports all failures at end). In production, `EXPECT_*` is preferred so you see ALL failures in one run, not just the first one.

TEORİ: EXPECT vs ASSERT Farkı

`ASSERT_EQ(a, b)` — başarısız olursa test ANINDA durur. Kritik ön koşullar için kullanılır. `EXPECT_EQ(a, b)` — başarısız olursa test çalışmaya DEVAM eder (tüm hataları sonda raporlar). Üretimde `EXPECT_*` tercih edilir çünkü tek çalıştırmada TÜM hataları görürsünüz.

THEORY: Dependency Injection for Testability

Tightly coupled code (class directly calls database/network) is UNTESTABLE because you can't run the test without an actual server. Solution: inject dependencies through interfaces (abstract classes). In tests, you pass a "Mock" or "Stub" instead of the real implementation. The class under test doesn't know the difference!

TEORİ: Test Edilebilirlik için Bağımlılık Enjeksiyonu

Sıkı bağlı kod (sınıf doğrudan veritabanı/ağ çağırır) TEST EDİLEMEZDİR çünkü gerçek sunucu olmadan test çalıştıramazsınız. Çözüm: bağımlılıkları arayüzler (soyut sınıflar) aracılığıyla enjekte edin. Testlerde gerçek uygulama yerine "Mock" veya "Stub" geçirin. Test edilen sınıf farkı bilmez!

CPPREF Depth: GTest Assertion Macros Reference

Key GTest macros from the official documentation:
1. `EXPECT_EQ(a, b)` / `ASSERT_EQ(a, b)` — equality.
2. `EXPECT_NE(a, b)` — not equal.
3. `EXPECT_LT(a, b)` / `EXPECT_GT(a, b)` — less/greater.
4. `EXPECT_TRUE(cond)` / `EXPECT_FALSE(cond)` — boolean.
5. `EXPECT_THROW(stmt, ExType)` — expects exception.
6. `EXPECT_NEAR(a, b, eps)` — floating point comparison.
7. `EXPECT_THAT(val, matcher)` — GMock matchers. google.github.io/googletest/reference/assertions.html

CPPREF Derinlik: GTest Assertion Makroları Referansı

Resmi dokümantasyondan temel GTest makroları:
1. `EXPECT_EQ(a, b)` / `ASSERT_EQ(a, b)` — eşitlik.
2. `EXPECT_NE(a, b)` — eşit değil.
3. `EXPECT_LT(a, b)` / `EXPECT_GT(a, b)` — küçük/büyük.
4. `EXPECT_TRUE(cond)` / `EXPECT_FALSE(cond)` — boolean.
5. `EXPECT_THROW(stmt, ExType)` — istisna beklentisi.
6. `EXPECT_NEAR(a, b, eps)` — kayan nokta karşılaştırma.
7. `EXPECT_THAT(val, matcher)` — GMock eşleştiriciler. google.github.io/googletest/reference/assertions.html

CPPREF Depth: GMock — Interface Mocking for Isolated Tests

Google Mock (GMock) allows creating mock classes that override virtual methods with `MOCK_METHOD(ReturnType, Name, (Args), (Qualifiers))`. You can then set expectations: `EXPECT_CALL(mock, method()).Times(1)`. GMock verifies that the expected calls were made. This enables testing code WITHOUT any real external dependencies (network, database, filesystem). google.github.io/googletest/gmock_for_dummies.html

CPPREF Derinlik: GMock — İzole Testler için Arayüz Taklit Etme

Google Mock (GMock), sanal metodları `MOCK_METHOD(DönüşTipi, Ad, (Argümanlar), (Nitelikler))` ile geçersiz kılan taklit sınıflar oluşturmaya olanak tanır. Beklentiler ayarlayabilirsiniz: `EXPECT_CALL(mock, metod()).Times(1)`. GMock beklenen çağrıların yapılıp yapılmadığını doğrular. Bu, gerçek harici bağımlılıklar olmadan (ağ, veritabanı, dosya sistemi) kod test etmeyi mümkün kılar. google.github.io/googletest/gmock_for_dummies.html
@warning EN: Never test your own code by "just running and looking". Tests MUST be automated (CI/CD)! TR: Asla kendi yazdiginiz kodu sadece "calistirarak ve gözle bakarak" test etmeyin. Testler OTOMATIK kosmalidir!

Kaynak Kod / Source Code

```cpp
#include <cmath>
#include <functional>
#include <iostream>
#include <memory>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>

// ==========================================================================
// [GTest SIMULATION FRAMEWORK / GTest SİMÜLASYON ÇATISI]
// ==========================================================================
// EN: We simulate GTest macros (TEST, EXPECT_EQ, EXPECT_TRUE, EXPECT_THROW, EXPECT_NEAR) without
// needing the actual library.
// TR: Gerçek kütüphane gerekmeden GTest makrolarını simüle ediyoruz.

namespace MiniGTest {

struct TestResult {
  std::string suiteName;
  std::string testName;
  bool passed = true;
  std::vector<std::string> failures;
};

std::vector<TestResult> allResults;
TestResult *currentTest = nullptr;

void registerTest(const std::string &suite, const std::string &name,
                  std::function<void()> fn) {
  allResults.push_back({suite, name, true, {}});
  currentTest = &allResults.back();
  try {
    fn();
  } catch (const std::exception &e) {
    currentTest->passed = false;
    currentTest->failures.push_back(std::string("EXCEPTION: ") + e.what());
  }
}

void expectEq(int expected, int actual, const char *file, int line) {
  if (expected != actual) {
    currentTest->passed = false;
    std::ostringstream oss;
    oss << file << ":" << line << " EXPECT_EQ failed: expected=" << expected
        << " actual=" << actual;
    currentTest->failures.push_back(oss.str());
  }
}

void expectTrue(bool cond, const char *file, int line) {
  if (!cond) {
    currentTest->passed = false;
    std::ostringstream oss;
    oss << file << ":" << line << " EXPECT_TRUE failed";
    currentTest->failures.push_back(oss.str());
  }
}

void expectFalse(bool cond, const char *file, int line) {
  if (cond) {
    currentTest->passed = false;
    std::ostringstream oss;
    oss << file << ":" << line << " EXPECT_FALSE failed";
    currentTest->failures.push_back(oss.str());
  }
}

void expectNear(double a, double b, double eps, const char *file, int line) {
  if (std::fabs(a - b) > eps) {
    currentTest->passed = false;
    std::ostringstream oss;
    oss << file << ":" << line << " EXPECT_NEAR failed: |" << a << " - " << b
        << "| > " << eps;
    currentTest->failures.push_back(oss.str());
```

```
  }
}

void printSummary() {
  int passed = 0;
  int failed = 0;
  for (const auto &r : allResults) {
    std::cout << (r.passed ? "\033[32m[ PASSED ]\033[0m "
                           : "\033[31m[ FAILED ]\033[0m ")
              << r.suiteName << "." << r.testName << std::endl;
    if (!r.passed) {
      for (const auto &f : r.failures) {
        std::cout << "           " << f << std::endl;
      }
      failed++;
    } else {
      passed++;
    }
  }
  std::cout << "\n========================================" << std::endl;
  std::cout << "[ SUMMARY ] " << passed << " passed, " << failed << " failed, "
            << (passed + failed) << " total." << std::endl;
}

} // namespace MiniGTest

// Macros simulating real GTest interface
#define TEST(suite, name)                                                    \
  void suite##_##name();                                                     \
  struct suite##_##name##_reg {                                              \
    suite##_##name##_reg() {                                                 \
      MiniGTest::registerTest(#suite, #name, suite##_##name);                \
    }                                                                        \
  } suite##_##name##_instance;                                               \
  void suite##_##name()

#define EXPECT_EQ(expected, actual)                                          \
  MiniGTest::expectEq(expected, actual, __FILE__, __LINE__)
#define EXPECT_TRUE(cond) MiniGTest::expectTrue(cond, __FILE__, __LINE__)
#define EXPECT_FALSE(cond) MiniGTest::expectFalse(cond, __FILE__, __LINE__)
#define EXPECT_NEAR(a, b, eps)                                              \
  MiniGTest::expectNear(a, b, eps, __FILE__, __LINE__)

#define EXPECT_THROW(stmt, ExType)                                          \
  do {                                                                       \
    bool caught = false;                                                     \
    try {                                                                    \
      stmt;                                                                  \
    } catch (const ExType &) {                                               \
      caught = true;                                                         \
    } catch (...) {                                                          \
    }                                                                        \
    if (!caught) {                                                           \
      MiniGTest::currentTest->passed = false;                                \
      MiniGTest::currentTest->failures.push_back(                            \
          std::string(__FILE__) + ":" + std::to_string(__LINE__) +           \
          " EXPECT_THROW: no " #ExType " thrown");                           \
    }                                                                        \
  } while (0)

// ==========================================================================
// [PRODUCTION CODE / ÜRETİM KODU]
// ==========================================================================

// --- Brake Force Calculator (Automotive) ---
int calculateBrakeForce(int pedalPressure, int brakeWear) {
  int force = pedalPressure - brakeWear;
  return (force < 0) ? 0 : force;
}

// --- Temperature Converter ---
double celsiusToFahrenheit(double c) { return c * 9.0 / 5.0 + 32.0; }

// --- Safe Division with Exception ---
double safeDivide(double a, double b) {
  if (b == 0.0) {
    throw std::invalid_argument("Division by zero!");
```

```
153     }
154     return a / b;
155   }
156
157   // ═══════════════════════════════════════════════════════════════════════════════
158   // [DEPENDENCY INJECTION / BAĞIMLILIK ENJEKSİYONU]
159   // ═══════════════════════════════════════════════════════════════════════════════
160   // EN: Interface (abstract class) for a payment gateway. Production code uses RealPaymentGateway;
161   // tests use MockPaymentGateway.
162   // TR: Ödeme ağ geçidi için arayüz (soyut sınıf). Üretim kodu RealPaymentGateway kullanır;
163   // testler MockPaymentGateway kullanır.
164
165   class IPaymentGateway {
166   public:
167     virtual ~IPaymentGateway() = default;
168     virtual bool processPayment(double amount) = 0;
169     virtual double getBalance() const = 0;
170   };
171
172   // EN: Mock implementation for testing — no real payment processing!
173   // TR: Test için taklit uygulama — gerçek ödeme işlemi yok!
174   class MockPaymentGateway : public IPaymentGateway {
175     double balance_;
176     int callCount_ = 0;
177
178   public:
179     explicit MockPaymentGateway(double balance) : balance_(balance) {}
180     bool processPayment(double amount) override {
181       callCount_++;
182       if (amount > balance_) return false;
183       balance_ -= amount;
184       return true;
185     }
186     double getBalance() const override { return balance_; }
187     int getCallCount() const { return callCount_; }
188   };
189
190   // EN: The class under test — takes IPaymentGateway via constructor injection.
191   // TR: Test edilen sınıf — IPaymentGateway'i yapıcı enjeksiyonu ile alır.
192   class OrderProcessor {
193     IPaymentGateway &gateway_;
194
195   public:
196     explicit OrderProcessor(IPaymentGateway &gw) : gateway_(gw) {}
197     bool placeOrder(double price) {
198       if (price <= 0) return false;
199       return gateway_.processPayment(price);
200     }
201   };
202
203   // ═══════════════════════════════════════════════════════════════════════════════
204   // [TEST CASES — Simulating Real GTest TEST() Macros]
205   // ═══════════════════════════════════════════════════════════════════════════════
206
207   // --- Suite: BrakeForceTest ---
208   TEST(BrakeForceTest, NormalBraking) {
209     EXPECT_EQ(20, calculateBrakeForce(50, 30));
210   }
211
212   TEST(BrakeForceTest, WornBrakesAbsorbFully) {
213     EXPECT_EQ(0, calculateBrakeForce(10, 50));
214   }
215
216   TEST(BrakeForceTest, NewBrakePads) {
217     EXPECT_EQ(100, calculateBrakeForce(100, 0));
218   }
219
220   TEST(BrakeForceTest, EqualPressureAndWear) {
221     EXPECT_EQ(0, calculateBrakeForce(30, 30));
222   }
223
224   // --- Suite: TemperatureTest ---
225   TEST(TemperatureTest, BoilingPoint) {
226     EXPECT_NEAR(212.0, celsiusToFahrenheit(100.0), 0.001);
227   }
228
```

```
229 TEST(TemperatureTest, FreezingPoint) {
230   EXPECT_NEAR(32.0, celsiusToFahrenheit(0.0), 0.001);
231 }
232
233 TEST(TemperatureTest, BodyTemperature) {
234   EXPECT_NEAR(98.6, celsiusToFahrenheit(37.0), 0.1);
235 }
236
237 // --- Suite: DivisionTest ---
238 TEST(DivisionTest, NormalDivision) {
239   EXPECT_NEAR(5.0, safeDivide(10.0, 2.0), 0.001);
240 }
241
242 TEST(DivisionTest, DivisionByZeroThrows) {
243   EXPECT_THROW(safeDivide(10.0, 0.0), std::invalid_argument);
244 }
245
246 // --- Suite: PaymentTest (Dependency Injection + Mock) ---
247 TEST(PaymentTest, SuccessfulOrder) {
248   MockPaymentGateway mock(100.0);
249   OrderProcessor processor(mock);
250   EXPECT_TRUE(processor.placeOrder(30.0));
251   EXPECT_NEAR(70.0, mock.getBalance(), 0.001);
252 }
253
254 TEST(PaymentTest, InsufficientFunds) {
255   MockPaymentGateway mock(20.0);
256   OrderProcessor processor(mock);
257   EXPECT_FALSE(processor.placeOrder(50.0));
258   EXPECT_NEAR(20.0, mock.getBalance(), 0.001);
259 }
260
261 TEST(PaymentTest, InvalidPrice) {
262   MockPaymentGateway mock(100.0);
263   OrderProcessor processor(mock);
264   EXPECT_FALSE(processor.placeOrder(-10.0));
265   EXPECT_EQ(0, mock.getCallCount());
266 }
267
268 TEST(PaymentTest, MockCallCount) {
269   MockPaymentGateway mock(1000.0);
270   OrderProcessor processor(mock);
271   processor.placeOrder(10.0);
272   processor.placeOrder(20.0);
273   processor.placeOrder(30.0);
274   EXPECT_EQ(3, mock.getCallCount());
275 }
276
277 // ==========================================================================
278 // [MAIN — Run All Tests / Tüm Testleri Çalıştır]
279 // ==========================================================================
280 int main() {
281   std::cout << "=== MODULE 6: GOOGLE TEST (GTEST) SIMULATION ===\n"
282             << std::endl;
283   std::cout << "Running " << MiniGTest::allResults.size() << " tests...\n"
284             << std::endl;
285   MiniGTest::printSummary();
286   return 0;
287 }
```

► 04. Valgrind Profiling

module_06_industry_standards/04_valgrind_profiling.cpp

Konu: Industry Standards: Memory Profiling and Valgrind — Profiling ve Valgrind ile Bellek

Teori / Theory

THEORY: Why Use Valgrind and Profiling?

Writing code that compiles without errors is NOT enough. If your program consumes 1 Megabyte of RAM extra every hour, it will crash your server after a week (OOM - Out of Memory). 'Valgrind' is a magical Linux tool. You run your program "inside" Valgrind, and it tracks EVERY SINGLE BYTE of memory you allocated and checks if you freed it.

TEORİ: Neden Valgrind Kullanılır?

Hata vermeden (0 error) derlenen bir kod yazmak YETERLİ DEĞİLDİR! C++ programınız her saat fazladan 1 MB RAM tüketirse (Memory Leak - Bellek Sızıntısı), o program 1 hafta sonra sunucuyu patlatır (Out of Memory - OOM).
Linux tarafında (Kurumsal firmalarda %99 Unix tabanlı iş yapılır) "Valgrind" isimli inanılmaz bir dedektif programı vardır. C++ executable (çalıştırılabilir) dosyanızı normal `./main` diye çalıştırmak yerine `valgrind ./main` diye çalıştırırsınız. Valgrind, programınızın etrafını sarı şeritlerle çevirir ve istediğiniz HER BİR BAYT belleği not alır. Program kapanırken "Sen işletim sisteminden şu kadar bellek aldın ama şu kadarını geri iade etmemişsin, RAM'in kanayan yeri şurasıdır!" der.

HOW TO APPLY? / NASIL UYGULANIR?

Add the "-g" flag while compiling to inject line numbers into the binary.

Derlerken (Compile) kod satır numaralarını içine gömmek için "-g" bayrağı eklenir:
1. Compile: `g++ -g -o leak_test 04_valgrind_profiling.cpp`
2. Execute: `valgrind --leak-check=full ./leak_test`

CPPREF Depth: Sanitizer Internals — How ASan, TSan, UBSan Actually Work

ASan (AddressSanitizer) uses shadow memory with a 1:8 mapping — every 8 bytes of application memory have 1 byte of shadow tracking whether that region is fully accessible, partially accessible, or poisoned. Typical slowdown is ~2x with ~3x memory overhead. TSan (ThreadSanitizer) uses happens-before analysis backed by vector clocks to detect data races; it incurs ~5-15x slowdown and 5-10x memory overhead. UBSan (UndefinedBehaviorSanitizer) instruments arithmetic and type operations at compile time (signed overflow, null dereference, misaligned access) with minimal runtime overhead. ASan and TSan CANNOT be combined in the same binary — they use incompatible shadow memory layouts. MSan (MemorySanitizer) detects reads of uninitialized memory; also incompatible with ASan. In CI pipelines, run separate builds for each sanitizer: one ASan+UBSan build and one TSan build.

CPPREF Derinlik: Sanitizer İç Yapısı — ASan, TSan, UBSan Nasıl Çalışır?

ASan (AddressSanitizer) 1:8 eşlemeli gölge bellek kullanır — uygulama belleğinin her 8 baytı için 1 bayt gölge, o bölgenin tamamen erişilebilir, kısmen erişilebilir veya zehirlenmiş olduğunu izler. Tipik yavaşlama ~2x, bellek ek yükü ~3x'tir. TSan (ThreadSanitizer) veri yarışlarını tespit etmek için vektör saatleriyle desteklenen happens-before analizi kullanır; ~5-15x yavaşlama ve 5-10x bellek ek yükü getirir. UBSan (UndefinedBehavior Sanitizer) aritmetik ve tür işlemlerini derleme zamanında enstrümante eder (işaretli taşma, null referans, hizasız erişim) — minimal çalışma zamanı ek yükü vardır. ASan ve TSan aynı binary'de BİRLEŞTİRİLEMEZ — uyumsuz gölge bellek düzenleri kullanırlar. MSan (MemorySanitizer) başlatılmamış bellek okumalarını tespit eder; o da ASan ile uyumsuzdur. CI pipeline'larında her sanitizer için ayrı derleme çalıştırın: bir ASan+UBSan, bir TSan.

Kaynak Kod / Source Code

```cpp
#include <cstring>
#include <iostream>
#include <memory>
#include <string>
#include <vector>

// ═══════════════════════════════════════════════════════════════════════════════
// EN: AddressSanitizer (ASan) — Demonstrates what ASan targets.
// TR: AddressSanitizer (ASan) — ASan'ın tespit ettiği hataları gösterir.
// ═══════════════════════════════════════════════════════════════════════════════
// EN: ASan is a compile-time instrumentation flag: -fsanitize=address. It detects
// heap-buffer-overflow, stack-buffer-overflow, use-after-free, and memory leaks at ~2x runtime
// cost (vs Valgrind's ~20x slowdown). Compile: g++ -g -fsanitize=address -o asan_test file.cpp
// TR: ASan, derleme zamanı enstrümantasyon bayrağıdır: -fsanitize=address. Heap tampon taşması,
// stack tampon taşması, serbest bırakılmış bellek erişimi ve bellek sızıntılarını ~2x çalışma
// maliyetiyle tespit eder (Valgrind'in ~20x yavaşlamasına karşılık). Derleme: g++ -g
// -fsanitize=address -o asan_test dosya.cpp
void demonstrateASanTargets() {
  std::cout << "\n--- ASan Target Patterns (Commented — triggering would crash!) ---\n";

  // ─── Heap-Buffer-Overflow ──────────────────────────────────────────────────
  // EN: Writing past allocated heap memory. ASan catches this immediately.
  // TR: Tahsis edilen heap belleğinin ötesine yazma. ASan bunu anında yakalar. DANGEROUS — DO
  // NOT UNCOMMENT: int *buf = new int[10]; buf[10] = 0xDEAD;  // heap-buffer-overflow! delete[]
  // buf;
  std::cout << "[ASan] Heap-buffer-overflow: writing buf[10] on a 10-element array\n";

  // ─── Stack-Buffer-Overflow ─────────────────────────────────────────────────
  // EN: Overrunning a local stack array. ASan instruments stack frames too.
  // TR: Yerel stack dizisini taşırma. ASan stack çerçevelerini de izler. DANGEROUS — DO NOT
  // UNCOMMENT: int stack_arr[5]; stack_arr[5] = 42;  // stack-buffer-overflow!
  std::cout << "[ASan] Stack-buffer-overflow: stack_arr[5] on a 5-element array\n";

  // ─── Use-After-Free ────────────────────────────────────────────────────────
  // EN: Accessing memory after delete. Classic dangling pointer.
  // TR: Delete sonrası belleğe erişim. Klasik sarkan işaretçi. DANGEROUS — DO NOT UNCOMMENT: int
  // *p = new int(7); delete p; *p = 99;  // use-after-free!
  std::cout << "[ASan] Use-after-free: dereferencing deleted pointer\n";
}

```

```cpp
// ═══════════════════════════════════════════════════════════════════════════════
// EN: ThreadSanitizer (TSan) — Data Race Detection
// TR: ThreadSanitizer (TSan) — Veri Yarışı Tespiti
// ═══════════════════════════════════════════════════════════════════════════════
// EN: TSan is enabled with -fsanitize=thread. It detects data races where two threads access the
// same memory without synchronization and at least one is a write. CANNOT be used simultaneously
// with ASan. Compile: g++ -g -fsanitize=thread -o tsan_test file.cpp
// TR: TSan, -fsanitize=thread ile etkinleştirilir. İki iş parçacığının senkronizasyon olmadan
// aynı belleğe eriştiği ve en az birinin yazma yaptığı veri yarışlarını tespit eder. ASan ile
// aynı anda KULLANILAMAZ. Derleme: g++ -g -fsanitize=thread -o tsan_test dosya.cpp
//
// EN: Example data race pattern (DO NOT run without TSan):
// TR: Örnek veri yarışı deseni (TSan olmadan ÇALIŞTIRMAYIN): int sharedCounter = 0; // Thread 1:
// sharedCounter++;  // WRITE — no lock! // Thread 2:  sharedCounter++;  // WRITE — no lock! =>
// DATA RACE
//
// EN: Fix: protect with std::mutex:
// TR: Çözüm: std::mutex ile koruma: std::mutex mtx; // Thread 1:  { std::lock_guard<std::mutex>
// lk(mtx); sharedCounter++; } // Thread 2:  { std::lock_guard<std::mutex> lk(mtx);
// sharedCounter++; }

// ═══════════════════════════════════════════════════════════════════════════════
// EN: UndefinedBehaviorSanitizer (UBSan) — Catches Silent Killers
// TR: UndefinedBehaviorSanitizer (UBSan) — Sessiz Katilleri Yakalar
// ═══════════════════════════════════════════════════════════════════════════════
// EN: UBSan is enabled with -fsanitize=undefined. It catches signed integer overflow, null
// pointer dereference, misaligned access, and more. These are "undefined behavior" — compiler
// may do ANYTHING. Compile: g++ -g -fsanitize=undefined -o ubsan_test file.cpp
// TR: UBSan, -fsanitize=undefined ile etkinleştirilir. İşaretli tamsayı taşması, null işaretçi
// referansı, hizalama hataları vb. yakalar. Bunlar "tanımsız davranış"tır — derleyici HERHANGİ
// bir şey yapabilir. Derleme: g++ -g -fsanitize=undefined -o ubsan_test dosya.cpp
int potentialSignedOverflow(int a, int b) {
  // EN: If a + b exceeds INT_MAX, this is undefined behavior. UBSan will report: "signed integer
  // overflow: X + Y".
  // TR: Eğer a + b, INT_MAX'ı aşarsa bu tanımsız davranıştır. UBSan şunu raporlar: "signed
  // integer overflow: X + Y".
  return a + b;
}

// ═══════════════════════════════════════════════════════════════════════════════
// EN: RAII Approach — Valgrind Reports 0 Leaks
// TR: RAII Yaklaşımı — Valgrind 0 Sızıntı Raporlar
// ═══════════════════════════════════════════════════════════════════════════════
void properRAII_VectorApproach() {
  // EN: std::vector manages heap memory automatically. When the function returns, the vector
  // destructor frees all memory — zero leaks.
  // TR: std::vector heap belleği otomatik yönetir. Fonksiyon döndüğünde vector yıkıcısı tüm
  // belleği serbest bırakır — sıfır sızıntı.
  std::vector<int> sensorBuffer(1000000);
  sensorBuffer[0] = 42;
  std::cout << "[RAII Vector] Data loaded: " << sensorBuffer[0]
            << " — memory auto-freed on scope exit" << std::endl;
}

void properRAII_UniquePtrApproach() {
  // EN: std::unique_ptr owns the resource exclusively. Automatically calls delete[] when it goes
  // out of scope — impossible to leak.
  // TR: std::unique_ptr kaynağa münhasır sahiptir. Kapsam dışına çıkınca otomatik delete[]
  // çağırır — sızıntı imkansız.
  auto canBuffer = std::make_unique<int[]>(1000000);
  canBuffer[0] = 0xCAFE;
  std::cout << "[RAII unique_ptr] CAN frame loaded: 0x" << std::hex
            << canBuffer[0] << std::dec
            << " — unique_ptr frees on scope exit" << std::endl;
}

void theLeakyFunction() {
  // EN: Allocating 1 million integers (approx 4 MB of RAM).
  // TR: 1 Milyon tam sayı (yaklasık 4 MB RAM tüketimi) istiyoruz.
  int *databaseSimulation = new int[1000000];

  databaseSimulation[0] = 42;
  std::cout << "[Valgrind Test] Data loaded (Veri Yüklendi): " << databaseSimulation[0] <<
      std::endl;

```

```
116     // EN: OH NO! We exit the function WITHOUT calling 'delete[]'! This is a disaster. When the
117     // function ends, the pointer 'databaseSimulation' vanishes, but the 4MB memory it grabbed
118     // remains occupied forever as a ZOMBIE.
119     //
120     // TR: AMAN TANRIM! DELETE YAPMADAN FONKSİYONDAN CIKIYORUZ! İşte burası bir felakettir. Zira
121     // fonksiyon kapandığında 'databaseSimulation' isimli işaretçi YOK OLUR ama RAM'de ayırdığı o
122     // devasa 4MB alan sahipsiz (Zombi) şekilde sistem yeniden başlayana kadar havada kalır!
123     // (MEMORY LEAK).
124   }
125
126   int main() {
127     std::cout << "=== MODULE 6: VALGRIND & MEMORY PROFILING ===\n" << std::endl;
128
129     std::cout << "[SYSTEM] WARNING: Triggering 4MB Memory Leak intentionally...\n";
130     std::cout << "[SYSTEM] DİKKAT: Bilerek 4MB Bellek sızıntısı tetikleniyor...\n";
131
132     theLeakyFunction();
133
134     std::cout << "\n[SYSTEM] Function ended. But 4 Megabytes of garbage remains in RAM." << std::endl;
135     std::cout << "[SYSTEM] Fonksiyon bitti ama 4MB çöp kaldı. Bunu bulan "
136                  "dedektif Valgrind aracıdır!"
137               << std::endl;
138
139     // ==========================================================================================
140     // EN: How to fix this in Modern C++? NEVER use RAW POINTERS inside 'theLeakyFunction'. Use
141     // `std::vector<int>` or `std::unique_ptr` instead! When they go out of scope, they instantly
142     // wipe the 4MB data (RAII rules supreme).
143     //
144     // TR: Modern C++ ile Nasıl Düzeltilir? 'theLeakyFunction' içerisinde ASLA ÇIPLAK İŞARETÇİ
145     // KULLANMA. `std::vector<int>` veya akıllı işaretçi kullan. Cünkü std::vector kapsam dışına
146     // cıktığında içindeki o 4MB datayı saniyesinde sisteme iade eder! (RAII)
147     // ==========================================================================================
148
149     // ==========================================================================================
150     // EN: Demo 2 — ASan Target Patterns
151     // TR: Demo 2 — ASan Hedef Desenleri
152     // ==========================================================================================
153     std::cout << "\n--- Demo 2: AddressSanitizer (ASan) Targets ---" << std::endl;
154     demonstrateASanTargets();
155
156     // ==========================================================================================
157     // EN: Demo 3 — Proper RAII: Vector vs Raw Pointer
158     // TR: Demo 3 — Doğru RAII: Vector vs Ham İşaretçi
159     // ==========================================================================================
160     std::cout << "\n--- Demo 3: RAII Vector Approach (0 Leaks Under Valgrind) ---" << std::endl;
161     properRAII_VectorApproach();
162
163     // ==========================================================================================
164     // EN: Demo 4 — unique_ptr Automatic Cleanup
165     // TR: Demo 4 — unique_ptr Otomatik Temizlik
166     // ==========================================================================================
167     std::cout << "\n--- Demo 4: unique_ptr Automatic Cleanup ---" << std::endl;
168     properRAII_UniquePtrApproach();
169
170     // ==========================================================================================
171     // EN: Demo 5 — UBSan: Signed Overflow Example
172     // TR: Demo 5 — UBSan: İşaretli Taşma Örneği
173     // ==========================================================================================
174     std::cout << "\n--- Demo 5: UBSan Signed Overflow ---" << std::endl;
175     // EN: Safe call — no overflow here. Compile with -fsanitize=undefined to instrument and catch
176     // overflows automatically.
177     // TR: Güvenli çağrı — burada taşma yok. Taşmaları otomatik yakalamak için -fsanitize=undefined
178     // ile derleyin.
179     int safeResult = potentialSignedOverflow(100, 200);
180     std::cout << "100 + 200 = " << safeResult << " (safe, no overflow)" << std::endl;
181     std::cout << "Compile with -fsanitize=undefined to catch cases like INT_MAX + 1\n";
182
```

```
  // ═══════════════════════════════════════════════════════════════════════════
  // EN: Demo 6 — Sanitizer & Profiling Compilation Flags Reference
  // TR: Demo 6 — Sanitizer & Profiling Derleme Bayrakları Referansı
  // ═══════════════════════════════════════════════════════════════════════════
  std::cout << "\n--- Demo 6: Sanitizer Compilation Flags Reference ---" << std::endl;
  std::cout << R"(
  +----------+---------------------------------+-------------------------------------+
  | Tool     | Compile Flag                    | Detects                             |
  +----------+---------------------------------+-------------------------------------+
  | ASan     | -fsanitize=address              | Buffer overflow, use-after-free, leak|
  | TSan     | -fsanitize=thread               | Data races between threads          |
  | UBSan    | -fsanitize=undefined            | Signed overflow, null deref, align  |
  | MSan     | -fsanitize=memory (clang)       | Uninitialized memory reads          |
  +----------+---------------------------------+-------------------------------------+
  | Valgrind | valgrind --leak-check=full ./bin | All leaks, ~20x slower              |
  | Perf     | perf record ./bin; perf report  | CPU hotspots, cache misses          |
  | Callgrind| valgrind --tool=callgrind ./bin | Function-level profiling            |
  +----------+---------------------------------+-------------------------------------+
)" << std::endl;

  // ═══════════════════════════════════════════════════════════════════════════
  // EN: Practical Profiling Workflow (Automotive ECU Software)
  // TR: Pratik Profiling İş Akışı (Otomotiv ECU Yazılımı)
  // ═══════════════════════════════════════════════════════════════════════════
  // EN: Step 1 — Compile with debug symbols: g++ -g -O0 -o ecu_test main.cpp
  // TR: Adım 1 — Debug sembolleriyle derle: g++ -g -O0 -o ecu_test main.cpp
  //
  // EN: Step 2 — Run under Valgrind Memcheck for full leak report:
  // TR: Adım 2 — Tam sızıntı raporu için Valgrind Memcheck altında çalıştır: valgrind
  // --leak-check=full --show-leak-kinds=all ./ecu_test
  //
  // EN: Step 3 — Compile with ASan for faster turnaround (~2x vs 20x):
  // TR: Adım 3 — Daha hızlı geri dönüş için ASan ile derle (~2x vs 20x): g++ -g
  // -fsanitize=address -o ecu_test main.cpp && ./ecu_test
  //
  // EN: Step 4 — Profile CPU hotspots with perf:
  // TR: Adım 4 — CPU darboğazlarını perf ile profilleyin: perf record -g ./ecu_test && perf
  // report
  //
  // EN: Step 5 — Function-level profiling with Callgrind:
  // TR: Adım 5 — Fonksiyon düzeyinde profil için Callgrind: valgrind --tool=callgrind ./ecu_test
  // kcachegrind callgrind.out.*   (GUI visualization)

  return 0;
}
```

► 05. Async Futures

module_06_industry_standards/05_async_futures.cpp

Konu: Industry Standards — Asynchronous Code: std::async, std::future, std::promise,

Teori / Theory

THEORY: Why std::async instead of std::thread?

When you launch `std::thread`, it runs completely parallel, but it does NOT easily RETURN a value back to the main thread. Also, managing raw threads can be messy. `std::async` is a higher-level tool. It runs a task in the background and returns a "TICKET" called `std::future`. You can continue doing your work on the main thread, and when you finally need the answer, you cash in the ticket (`ticket.get()`).

TEORİ: std::async vs std::thread

`std::thread` ile boyut açtığımızda harika bir şekilde paralel çalışır (ANCAK), işi bitirdiğinde Ana Thread'e (Main) "Sonucu (Return değeri)" geri yollaması büyük bir eziyettir. C++11 ile `std::async` geldi.
`std::async` arka planda bir thread acar, işi o thread'e verir ve sana garantör olarak bir "Fiş / Bilet / Kupon" (`std::future`) uzatır. "Sen git kendi isine bak, işlemler bitince bana bu biletle geleceksin ve ben sana hesapladığım değeri geri vereceğim." der. Eğer sen bileti tahsil etmeye `ticket.get()` ile gidersen ve arka plandaki is HENÜZ BİTMEMİŞSE, Main Thread o noktada işlemi beklemeye (BLOKE OLMAYA) gecer!

THEORY: std::promise — The Manual Push Channel

`std::async` automatically decides which thread runs the work and delivers the result. But sometimes YOU want to control when and what value gets pushed. `std::promise<T>` is a write-end channel: you call `promise.set_value(42)` from any thread, and the paired `std::future<T>` (obtained via `promise.get_future()`) receives it. Think of it as a one-shot mailbox: the sender drops a letter, the receiver picks it up.

TEORİ: std::promise — Manuel Değer İtme Kanalı

`std::async` işi otomatik planlar. Ancak bazen HANGİ değerin, HANGİ anda gönderileceğini SİZ kontrol etmek istersiniz. `std::promise<T>` bir "yazma ucu" kanalıdır. Herhangi bir thread'den `promise.set_value(42)` dersiniz, eşleştirilmiş `std::future<T>` (promise.get_future() ile alınan) o değeri alır. Tek kullanımlık posta kutusu gibidir: gönderici mektubu atar, alıcı kutudan çeker.

THEORY: std::packaged_task — Callable Wrapper with Future

`std::packaged_task<R(Args...)>` wraps ANY callable (function, lambda, functor) and automatically connects it to a `std::future`. When you invoke the packaged_task (via `operator()`), the result is stored in the future. Unlike `std::async`, YOU decide exactly when and on which thread the task executes — giving you full scheduling control (perfect for thread pools!).

TEORİ: std::packaged_task — Future Bağlantılı Çağrılabilir Sarmalayıcı

`std::packaged_task<R(Args...)>` herhangi bir çağrılabilir nesneyi (fonksiyon, lambda, functor) sarar ve otomatik olarak bir `std::future` ile eşleştirir. packaged_task'ı `operator()` ile çağırdığınızda sonuç future'a yazılır. `std::async`'den farkı: görevi NE ZAMAN ve HANGİ thread'de çalıştıracağınıza SİZ karar verirsiniz — Thread Pool mimarisi için ideal!

THEORY: std::shared_future — Multiple Readers

`std::future::get()` can only be called ONCE (the ticket is torn). `std::shared_future<T>` allows MULTIPLE threads to call `.get()` and each receives a copy of the result. Created from `future.share()`. Useful when a single computation feeds many consumers (e.g., config loading shared by 10 worker threads).

TEORİ: std::shared_future — Çoklu Okuyucu

`std::future::get()` yalnızca BİR KEZ çağrılabilir (bilet yırtılır). `std::shared_future<T>` ise BİRDEN FAZLA thread'in `.get()` çağırmasına izin verir ve her biri sonucun bir kopyasını alır. `future.share()` ile oluşturulur. Tek bir hesaplamanın birçok tüketiciye dağıtılması gerektiğinde kullanılır (örneğin, 10 işçi thread'e paylaşılan config yüklemesi).

CPPREF Depth: The Silent Block of `std::future` Destructor

A massive trap! If you use `std::async(std::launch::async, ...)` but don't assign the returned ticket to a variable, the temporary `std::future` dies immediately at the semicolon. Its Destructor WILL BLOCK THE MAIN THREAD until the async task finishes! So it completely defeats the purpose of async!

CPPREF Derinlik: `std::future` Yıkıcısının Sessiz Bloğu

Devasa bir mülakat tuzağı! Eğer `std::async(std::launch::async, ...)` ile asenkron iş başlatır ama dönen o Bileti (future) bir değişkene atamazsanız, isimsiz geçici future nesnesi o satır sonunda anında ölür. CppRef kurallarına göre o Future'ın Destructor'ı, arka plandaki iş bitene kadar ANA THREAD'İ KİLİTLER VE BEKLER! Yani asenkron yaptığınızı sanarken kodu tamamen senkronize bloke etmiş olursunuz!

CPPREF Depth: std::promise::set_exception — Error Propagation Across Threads

If a worker thread encounters an error, it can call `promise.set_exception(std::current_exception())`. The consumer thread calling `future.get()` will then receive the exception as if it was thrown locally. This is the ONLY safe way to propagate exceptions across thread boundaries. cppreference.com/w/cpp/thread/promise/set_exception

CPPREF Derinlik: std::promise::set_exception — Thread'ler Arası Hata İletimi

Eğer işçi thread bir hatayla karşılaşırsa, `promise.set_exception(std::current_exception())` çağırabilir. Tüketici thread `future.get()` çağırdığında, istisna sanki yerel olarak fırlatılmış gibi alınır. Bu, thread sınırları arasında istisna iletmenin TEK güvenli yoludur. cppreference.com/w/cpp/thread/promise/set_exception

CPPREF Depth: std::packaged_task is MoveOnly — No Copying!

`std::packaged_task` is move-only, just like `std::unique_ptr`. You cannot copy it. To pass it into a `std::thread`, you MUST use `std::move(task)`. Attempting to copy triggers a deleted copy-constructor error at compile time. cppreference.com/w/cpp/thread/packaged_task

CPPREF Derinlik: std::packaged_task MoveOnly'dir — Kopyalanamaz!

`std::packaged_task`, tıpkı `std::unique_ptr` gibi yalnızca taşınabilir (move-only). Kopyalanamaz. Bir `std::thread`'e geçirmek için `std::move(task)` kullanmanız ZORUNLUDUR. Kopyalama girişimi derleme zamanında silinmiş kopya yapıcı hatası verir. cppreference.com/w/cpp/thread/packaged_task

Kaynak Kod / Source Code

```cpp
#include <atomic>
#include <chrono> // EN: Time simulations
#include <condition_variable>
#include <functional>
#include <future>
#include <iostream>
#include <mutex>
#include <queue>
#include <thread>
#include <vector>

/**
 * @brief EN: Heavy function simulating a network/database request.
 * TR: Ağdan / veritabanından veri ceker gibi bekleyen agir fonksiyon.
 * @return 200 (OK status) after 3 seconds.
 */
int heavyServerRequest() {
  std::cout << "[WORKER THREAD] Accessing Server... ETA: 3 Seconds (Sunucuya "
               "erisiliyor...)"
            << std::endl;
  std::this_thread::sleep_for(std::chrono::seconds(3));

  std::cout << "[WORKER THREAD] Server Responded / Sunucu yanit verdi: [OK - 200]" << std::endl;

  // EN: We are actually RETURNING a value! You can't easily do this with std::thread.
  // TR: Dikkat, değer DONDURUYORUZ! std::thread ile bunu basitce yakalayamazsiniz.
  return 200;
}

// ==========================================================================================
// [DEMO 2: std::promise - Manual Value Push / Elle Değer İtme]
// ==========================================================================================
/**
 * @brief EN: Worker receives a promise and pushes the result manually.
 * TR: İşçi bir promise alır ve sonucu elle gönderir.
 */
void workerWithPromise(std::promise<std::string> prom) {
  std::cout << "[PROMISE WORKER] Computing secret key..." << std::endl;
  std::this_thread::sleep_for(std::chrono::seconds(1));
  // EN: Push the value into the channel — the future on the other side wakes.
  // TR: Değeri kanala it — diğer uçtaki future uyanır.
  prom.set_value("SECRET-KEY-XJ9-ALPHA");
}

// ==========================================================================================
// [DEMO 3: std::packaged_task - Deferred Callable / Ertelenmiş Görev]
// ==========================================================================================
/**
 * @brief EN: A pure function to be wrapped in packaged_task.
 * TR: packaged_task içine sarılacak saf fonksiyon.
 */
int computeChecksum(int a, int b) {
  std::this_thread::sleep_for(std::chrono::milliseconds(500));
  return (a * 31) ^ (b * 17);
}

int main() {
  std::cout << "=== MODULE 6: ADVANCED CONCURRENCY (ASYNC / FUTURE / "
               "PROMISE / PACKAGED_TASK / SHARED_FUTURE) ===\n"
            << std::endl;

  // ==========================================================================================
  // PART 1: std::async + std::future (Original Demo)
  // ==========================================================================================
  std::cout << "--- PART 1: std::async + std::future ---\n" << std::endl;

  auto startTime = std::chrono::high_resolution_clock::now();

```

```cpp
    // EN: std::launch::async -> Forces it to run strictly on a new separate thread. "Work in the
    // background and give me a ticket!"
    // TR: "Arka planda çalış, bana bilet (future) fırlat!" std::launch::async -> Kesinlikle yeni
    // bir çekirdekte çalıştır demektir.
    std::future<int> ticket = std::async(std::launch::async, heavyServerRequest);

    // EN: Main Thread continues working!
    // TR: Ana döngü bos durmuyor!
    std::cout << "\n[MAIN] Task sent asynchronously! Meanwhile, UI is rendering "
                 "frames..."
              << std::endl;

    for (int i = 1; i <= 3; i++) {
      std::cout << "[MAIN] Drawing Frame (Kare cizimi...) " << i << std::endl;
      std::this_thread::sleep_for(std::chrono::seconds(1));
    }

    // EN: Once `.get()` is called: if the heavy code isn't finished yet, MAIN FREEZES and WAITS!
    // TR: ".get()" cagirildiginda eğer ağır iş henuz isini bitirmemisse, ANA PROGRAM DONAR!
    int result = ticket.get();

    auto endTime = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> totalTime = endTime - startTime;

    std::cout << "\n--- ASYNC RESULTS / SONUCLAR ---" << std::endl;
    std::cout << "Received Server Code (Gelen Kod): " << result << std::endl;
    std::cout << "Total Elapsed Time: " << totalTime.count() << " seconds.\n"
              << std::endl;

    // ==========================================================================================
    // PART 2: std::promise + std::future (Manual Push)
    // ==========================================================================================
    // EN: Create a promise-future pair. The promise is the "write end", the future is the "read
    // end". We send the promise to a worker thread.
    // TR: Bir promise-future çifti oluştur. Promise "yazma ucu", future "okuma ucu". Promise'i
    // işçi thread'e gönderiyoruz.
    std::cout << "--- PART 2: std::promise (Manual Channel) ---\n" << std::endl;

    std::promise<std::string> secretPromise;
    std::future<std::string> secretFuture = secretPromise.get_future();

    // EN: We MUST move the promise into the thread (it's move-only).
    // TR: Promise taşınabilir-only; std::move ZORUNLU.
    std::thread promThread(workerWithPromise, std::move(secretPromise));

    std::cout << "[MAIN] Waiting for secret key from worker..." << std::endl;
    std::string key = secretFuture.get(); // Blocks until set_value()
    std::cout << "[MAIN] Received key: " << key << "\n" << std::endl;

    promThread.join();

    // ==========================================================================================
    // PART 3: std::packaged_task (Deferred Execution)
    // ==========================================================================================
    // EN: Wrap computeChecksum in a packaged_task. We get the future BEFORE running the task, then
    // move the task into a thread.
    // TR: computeChecksum'u packaged_task ile sar. Görevi çalıştırmadan ÖNCE future'ı al, ardından
    // görevi thread'e taşı.
    std::cout << "--- PART 3: std::packaged_task (Deferred Callable) ---\n"
              << std::endl;

    std::packaged_task<int(int, int)> task(computeChecksum);
    std::future<int> checksumFuture = task.get_future();

    // EN: Move the task into a thread (packaged_task is move-only like unique_ptr).
    // TR: Görevi thread'e taşı (packaged_task unique_ptr gibi move-only).
    std::thread taskThread(std::move(task), 42, 99);

    std::cout << "[MAIN] packaged_task dispatched. Doing other work..." << std::endl;
    int checksum = checksumFuture.get(); // Blocks until task completes
    std::cout << "[MAIN] Checksum result: " << checksum << "\n" << std::endl;

    taskThread.join();

```

```
143     // ═══════════════════════════════════════════════════════════════════════════════
144     // PART 4: std::shared_future (Multiple Readers)
145     // ═══════════════════════════════════════════════════════════════════════════════
146     // EN: One computation, many consumers. shared_future allows multiple .get() calls from
147     // different threads.
148     // TR: Tek hesaplama, çok tüketici. shared_future birden fazla thread'in .get() çağırmasına
149     // izin verir.
150     std::cout << "--- PART 4: std::shared_future (Multiple Readers) ---\n"
151               << std::endl;
152
153     std::shared_future<int> sharedResult =
154         std::async(std::launch::async, []() -> int {
155           std::cout << "[SHARED WORKER] Loading config..." << std::endl;
156           std::this_thread::sleep_for(std::chrono::milliseconds(500));
157           // EN: Config value
158           // TR: Ayar değeri
159           return 9999;
160         // EN: .share() converts future -> shared_future
161         // TR: .share() future'ı shared_future'a dönüştürür
162         }).share();
163
164     // EN: Multiple threads read the same result simultaneously.
165     // TR: Birden fazla thread aynı sonucu eşzamanlı okur.
166     std::vector<std::thread> readers;
167     for (int i = 0; i < 3; i++) {
168       readers.emplace_back([sharedResult, i]() {
169         int val = sharedResult.get(); // All 3 threads can call get()!
170         std::cout << "[READER " << i << "] Config loaded: " << val << std::endl;
171       });
172     }
173
174     for (auto &r : readers) {
175       r.join();
176     }
177
178     std::cout << "\n=> std::shared_future allows MULTIPLE .get() calls — unlike "
179                  "std::future which tears the ticket on first use!"
180               << std::endl;
181
182     // ═══════════════════════════════════════════════════════════════════════════════
183     // PART 5: std::jthread and stop_token (C++20)
184     // ═══════════════════════════════════════════════════════════════════════════════
185     // EN: std::jthread = "joining thread". Automatically calls .join() in destructor.
186     //     Also supports cooperative cancellation via stop_token/stop_source.
187     //     No more forgetting .join() — the #1 cause of std::terminate crashes!
188     // TR: std::jthread = "birleşen thread". Yıkıcıda otomatik .join() çağırır.
189     //     stop_token/stop_source ile işbirlikçi iptal desteği de sunar.
190     std::cout << "\n--- PART 5: std::jthread & stop_token (C++20) ---\n" << std::endl;
191
192     {
193       // EN: jthread destructor auto-joins — no need for explicit .join()
194       // TR: jthread yıkıcısı otomatik birleşir — açık .join() gerekmez
195       std::jthread worker([](std::stop_token stoken) {
196         int count = 0;
197         while (!stoken.stop_requested()) {
198           std::cout << "  [jthread] Working cycle " << count++ << std::endl;
199           std::this_thread::sleep_for(std::chrono::milliseconds(50));
200           if (count >= 5) break;  // EN: Safety limit for demo
201         }
202         std::cout << "  [jthread] Stopped after " << count << " cycles" << std::endl;
203       });
204
205       // EN: request_stop() signals the thread to finish cooperatively
206       // TR: request_stop() thread'e işbirlikçi olarak bitmesini sinyaller
207       std::this_thread::sleep_for(std::chrono::milliseconds(200));
208       worker.request_stop();
209       std::cout << "  [main] Stop requested" << std::endl;
210       // EN: No .join() needed — jthread destructor handles it!
211       // TR: .join() gereksiz — jthread yıkıcısı halleder!
212     }
213
214     std::cout << "  [main] jthread auto-joined on scope exit\n" << std::endl;
```

```cpp

    // ═══════════════════════════════════════════════════════════════════════════════
    // PART 6: stop_callback — Registering Cancellation Handlers
    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: stop_callback registers a function that runs when stop is requested.
    //     Useful for cleanup: closing sockets, flushing buffers, etc.
    // TR: stop_callback, durdurma istendiğinde çalışan bir fonksiyon kaydeder.
    std::cout << "--- PART 6: stop_callback (C++20) ---\n" << std::endl;
    {
      std::jthread sensor([](std::stop_token stoken) {
        // EN: Register callback that fires on stop
        // TR: Durdurulduğunda tetiklenen geri çağrı kaydet
        std::stop_callback cb(stoken, []() {
          std::cout << "  [callback] Cleanup: flushing sensor buffer..." << std::endl;
        });

        int readings = 0;
        while (!stoken.stop_requested() && readings < 3) {
          std::cout << "  [sensor] Reading " << readings++ << std::endl;
          std::this_thread::sleep_for(std::chrono::milliseconds(50));
        }
      });

      std::this_thread::sleep_for(std::chrono::milliseconds(100));
      sensor.request_stop();
      // EN: callback fires immediately when stop is requested
      // TR: durdurma istendiğinde geri çağrı hemen tetiklenir
    }
    std::cout << "  [main] sensor jthread cleaned up\n" << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // PART 7: Simple Thread Pool Pattern
    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: A thread pool reuses a fixed number of threads for many tasks.
    //     Instead of creating/destroying threads per task (expensive), tasks go
    //     into a queue and idle workers pick them up.
    // TR: Thread havuzu, birçok görev için sabit sayıda thread'i yeniden kullanır.
    //     Görev başına thread oluşturmak/yok etmek yerine (pahalı), görevler
    //     kuyruğa girer ve boştaki işçiler alır.
    std::cout << "--- PART 7: Simple Thread Pool Pattern ---\n" << std::endl;
    {
      std::mutex queueMtx;
      std::condition_variable cv;
      std::queue<std::function<void()>> tasks;
      std::atomic<bool> poolDone{false};
      std::atomic<int> completed{0};

      // EN: Create 4 worker threads
      // TR: 4 işçi thread oluştur
      std::vector<std::jthread> pool;
      for (int i = 0; i < 4; ++i) {
        pool.emplace_back([&, i](std::stop_token) {
          while (true) {
            std::function<void()> job;
            {
              std::unique_lock<std::mutex> lk(queueMtx);
              cv.wait(lk, [&] { return !tasks.empty() || poolDone.load(); });
              if (poolDone.load() && tasks.empty()) break;
              job = std::move(tasks.front());
              tasks.pop();
            }
            job();
            completed++;
          }
        });
      }
```

```cpp

    // EN: Submit 12 tasks
    // TR: 12 görev gönder
    {
      std::lock_guard<std::mutex> lk(queueMtx);
      for (int i = 0; i < 12; ++i) {
        tasks.push([i]() {
          std::this_thread::sleep_for(std::chrono::milliseconds(20));
        });
      }
    }
    cv.notify_all();

    // EN: Wait for tasks then shut down pool
    // TR: Görevleri bekle sonra havuzu kapat
    while (completed.load() < 12) {
      std::this_thread::sleep_for(std::chrono::milliseconds(10));
    }
    poolDone = true;
    cv.notify_all();
    pool.clear();  // EN: jthread destructors auto-join

    std::cout << "  Thread pool: 4 workers processed " << completed.load()
              << " tasks" << std::endl;
    std::cout << "  (jthread auto-joins on clear — no manual join needed)\n" << std::endl;
  }

  return 0;
}
```

► 06. Debugging Techniques

module_06_industry_standards/06_debugging_techniques.cpp

Konu: Industry Standards: Debugging Techniques & Defensive Programming — Endüstri

Teori / Theory

THEORY: Debugging, Assertions & Defensive Programming

1. GDB Basics

GDB is the GNU Debugger — the primary debugging tool on Linux. Compile with `-g` to include debug symbols: g++ -g -std=c++17 -o prog 06_debugging_techniques.cpp Key commands: — break main — set breakpoint at main() — run — start execution — next (n) — step over to next line — step (s) — step into function call — print var — inspect variable value — backtrace (bt)— show call stack — watch var — break when variable changes (watchpoint)

2. Core Dumps

Enable core dumps: `ulimit -c unlimited` When a program crashes (SIGSEGV, SIGABRT), the OS writes a core file. Analyze: `gdb ./program core` — see exact crash location and variables.

3. Static Analysis & Sanitizers

Compiler warnings (-Wall -Wextra -Wpedantic) are the first line of defense. Address/Thread/UB sanitizers catch bugs at runtime: — -fsanitize=address — buffer overflow, use-after-free — -fsanitize=undefined — signed overflow, null deref, alignment — -fsanitize=thread — data races in multithreaded code

4. assert() vs static_assert()

assert(expr) — runtime check; aborts if expr is false. Disabled by defining NDEBUG. static_assert(expr, msg) — compile-time check; zero runtime cost. Use for type traits, sizes, alignment guarantees.

5. Defensive Programming

— Preconditions — validate inputs before use. — Postconditions — verify outputs before returning. — Invariants — maintain class state consistency across all methods.
[[nodiscard]] — force callers to check return values.

6. Logging Levels

— TRACE — finest granularity, every function entry/exit — DEBUG — diagnostic information for developers — INFO — general operational messages — WARN — potentially harmful situations — ERROR — error events, program can continue — FATAL — severe error, program must abort

TEORİ: Hata Ayıklama, Onaylamalar ve Savunmacı Programlama

GDB, GNU Hata Ayıklayıcısıdır — Linux'ta temel hata ayıklama aracıdır. Hata ayıklama sembollerini eklemek için `-g` ile derleyin: g++ -g -std=c++17 -o prog 06_debugging_techniques.cpp Temel komutlar: — break main — main()'de kırılma noktası koy — run — çalıştırmayı başlat — next (n) — sonraki satıra geç — step (s) — fonksiyon çağrısına gir — print var — değişken değerini incele — backtrace (bt)— çağrı yığınını göster — watch var — değişken değişince dur (izleme noktası)

2. Çekirdek Dökümleri

Çekirdek dökümünü etkinleştir: `ulimit -c unlimited` Program çöktüğünde (SIGSEGV, SIGABRT), işletim sistemi bir core dosyası yazar. Analiz: `gdb ./program core` — tam çökme noktasını ve değişkenleri gör.

Derleyici uyarıları (-Wall -Wextra -Wpedantic) ilk savunma hattıdır. Adres/İş parçacığı/TB temizleyicileri çalışma zamanında hata yakalar: — -fsanitize=address — tampon taşması, serbest bırakılmış kullanımı — -fsanitize=undefined — işaretli taşma, null ref, hizalama — -fsanitize=thread — çok iş parçacıklı kodda veri yarışları

assert(ifade) — çalışma zamanı kontrolü; ifade false ise iptal eder. NDEBUG tanımlanarak devre dışı bırakılır. static_assert(ifade, msg) — derleme zamanı kontrolü; çalışma zamanı maliyeti sıfır. Tip özellikleri, boyutlar, hizalama garantileri için kullanılır.

5. Savunmacı Programlama

— Ön koşullar — kullanmadan önce girdileri doğrula. — Son koşullar — dönmeden önce çıktıları doğrula. — Değişmezler — tüm yöntemlerde sınıf durum tutarlılığını koru.
[[nodiscard]] — çağıranları dönüş değerini kontrol etmeye zorla.

6. Günlük Seviyeleri

— TRACE — en ince ayrıntı, her fonksiyon giriş/çıkış — DEBUG — geliştiriciler için tanılama bilgisi — INFO — genel operasyonel mesajlar — WARN — potansiyel zararlı durumlar — ERROR — hata olayları, program devam edebilir — FATAL — ağır hata, program iptal etmeli
Standart: C++17 Derleme : g++ -std=c++17 -Wall -Wextra -Wpedantic 06_debugging_techniques.cpp

CPPREF Depth: GDB Advanced — Conditional Breakpoints, Watchpoints, and Reverse Debugging

4. assert() vs static_assert()

`break func if x > 100` sets a conditional breakpoint — execution stops only when the condition is true, invaluable for debugging iteration N of a million-iteration loop. `watch variable` sets a hardware watchpoint; the CPU halts when the watched memory location changes value. `rwatch` triggers on read access, `awatch` on any access. GDB reverse debugging: `record` starts recording, then `reverse-next`, `reverse-step`, and `reverse-continue` replay execution backwards — you can step BACK to find where a variable was corrupted. `gdb -tui` launches a terminal UI with source code pane. Core dump analysis: `thread apply all bt` prints all thread backtraces — essential for multithreaded crash investigation. Python scripting inside GDB lets you write custom pretty-printers for complex types (e.g., printing a red-black tree as a sorted list). `set pagination off` and `set logging on` for scripted batch debugging.

CPPREF Derinlik: İleri GDB — Koşullu Kırılma Noktaları, İzleme Noktaları ve Geri Hata Ayıklama

`break func if x > 100` koşullu kırılma noktası koyar — yürütme yalnızca koşul doğru olduğunda durur; milyon iterasyonlu bir döngüde N. iterasyonu hata ayıklamak için paha biçilmezdir. `watch variable` donanım izleme noktası koyar; izlenen bellek konumunun değeri değiştiğinde CPU durur. `rwatch` okuma erişiminde, `awatch` herhangi bir erişimde tetiklenir. GDB geri hata ayıklama: `record` kaydı başlatır, ardından `reverse-next`, `reverse-step` ve `reverse-continue` yürütmeyi GERİYE doğru oynatır — bir değişkenin nerede bozulduğunu bulmak için GERİ adım atabilirsiniz. `gdb -tui` kaynak kodlu terminal arayüzü başlatır. Çekirdek dökümü analizi: `thread apply all bt` tüm iş parçacığı geri izlerini yazdırır — çok iş parçacıklı çökme araştırması için zorunludur. GDB içinde Python betikleme, karmaşık tipler için özel pretty-printer yazmanızı sağlar.

Kaynak Kod / Source Code

```cpp
#include <cassert>     // EN: Runtime assertions
#include <chrono>      // EN: Timestamp generation
#include <cstdint>     // EN: Fixed-width integers
#include <functional>  // EN: std::function for invariant checks
#include <iomanip>     // EN: Output formatting
#include <iostream>    // EN: Console I/O
#include <sstream>     // EN: String stream for log formatting
#include <string>      // EN: String operations
#include <type_traits> // EN: Type traits for static_assert

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 1 — ECULogger: Singleton Logger with Levels
// ═══════════════════════════════════════════════════════════════════════════

// EN: Log severity levels for automotive ECU diagnostics
// TR: Otomotiv ECU tanılaması için günlük önem seviyeleri
enum class LogLevel : std::uint8_t {
    TRACE = 0,
    DEBUG = 1,
    INFO  = 2,
    WARN  = 3,
    ERROR = 4,
    FATAL = 5
};
```

```

// EN: Convert LogLevel enum to string representation
// TR: LogLevel enum'ını metin temsiline dönüştür
inline const char* logLevelToString(LogLevel level) {
    switch (level) {
        case LogLevel::TRACE: return "TRACE";
        case LogLevel::DEBUG: return "DEBUG";
        case LogLevel::INFO:  return "INFO ";
        case LogLevel::WARN:  return "WARN ";
        case LogLevel::ERROR: return "ERROR";
        case LogLevel::FATAL: return "FATAL";
    }
    return "?????";
}

// EN: Singleton logger — centralized logging for ECU software
// TR: Tekil günlükçü — ECU yazılımı için merkezi günlükleme
class ECULogger {
public:
    // EN: Get the singleton instance
    // TR: Tekil örneği al
    static ECULogger& instance() {
        static ECULogger logger;
        return logger;
    }

    // EN: Set the minimum log level — messages below this are suppressed
    // TR: Minimum günlük seviyesini ayarla — altındaki mesajlar bastırılır
    void setLevel(LogLevel level) { minLevel_ = level; }

    // EN: Log a message with level, file and line information
    // TR: Seviye, dosya ve satır bilgisiyle bir mesaj günlükle
    void log(LogLevel level, const char* file, int line, const std::string& msg) {
        if (level < minLevel_) return;

        auto now = std::chrono::system_clock::now();
        auto epoch = now.time_since_epoch();
        auto millis = std::chrono::duration_cast<std::chrono::milliseconds>(epoch).count();

        std::cout << "[" << std::setw(13) << millis << "ms] "
                  << "[" << logLevelToString(level) << "] "
                  << file << ":" << line << " — " << msg << "\n";
    }

    // EN: Deleted copy/move — singleton pattern
    // TR: Silinen kopya/taşıma — tekil kalıp
    ECULogger(const ECULogger&)            = delete;
    ECULogger& operator=(const ECULogger&) = delete;
    ECULogger(ECULogger&&)                 = delete;
    ECULogger& operator=(ECULogger&&)      = delete;

private:
    ECULogger() = default;
    LogLevel minLevel_ = LogLevel::TRACE;
};

// ─── Convenience Macros ──────────────────────────────────────────────────────

// EN: LOG macro captures __FILE__ and __LINE__ automatically
// TR: LOG makrosu __FILE__ ve __LINE__ bilgisini otomatik yakalar
#define ECU_LOG(level, msg) \
    ECULogger::instance().log(level, __FILE__, __LINE__, msg)

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 2 — ECU_ASSERT: Enhanced Assertion with Logging
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Enhanced assert macro — logs the failure before aborting
// TR: Geliştirilmiş onaylama makrosu — iptal etmeden önce hatayı günlükler
#define ECU_ASSERT(cond, msg)                                                \
```

```
    do {                                                                    \
        if (!(cond)) {                                                      \
            ECU_LOG(LogLevel::FATAL,                                        \
                    std::string("ASSERTION FAILED: ") + #cond + " — " + msg); \
            assert((cond) && (msg));                                        \
        }                                                                   \
    } while (false)

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 3 — static_assert Examples: Compile-Time Guarantees
// ═══════════════════════════════════════════════════════════════════════════

// EN: Automotive CAN frame — strict layout required for hardware DMA
// TR: Otomotiv CAN çerçevesi — donanım DMA için katı yerleşim gerekli
struct CANFrame {
    std::uint32_t id;         // EN: CAN identifier (11 or 29 bit)
    std::uint8_t  dlc;        // EN: Data length code (0–8)
    std::uint8_t  data[8];    // EN: Payload bytes
    std::uint8_t  padding[3]; // EN: Alignment padding
};

// EN: Compile-time checks — zero runtime cost
// TR: Derleme zamanı kontrolleri — sıfır çalışma zamanı maliyeti
static_assert(sizeof(CANFrame) == 16, "CANFrame must be exactly 16 bytes for DMA");
static_assert(std::is_trivially_copyable_v<CANFrame>,
              "CANFrame must be trivially copyable for memcpy/DMA");
static_assert(alignof(CANFrame) <= 4, "CANFrame alignment must be at most 4 bytes");

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 4 — Precondition / Postcondition Pattern
// ═══════════════════════════════════════════════════════════════════════════

// EN: Compute throttle percentage from raw ADC value with pre/post checks
// TR: Ham ADC değerinden gaz kelebeği yüzdesini ön/son kontrollerle hesapla
[[nodiscard]] inline double computeThrottlePercent(int rawADC) {
    // EN: Precondition — raw ADC must be in valid 12-bit range
    // TR: Ön koşul — ham ADC geçerli 12-bit aralığında olmalı
    ECU_ASSERT(rawADC >= 0 && rawADC <= 4095,
               "rawADC out of 12-bit range [0, 4095]");

    double percent = (static_cast<double>(rawADC) / 4095.0) * 100.0;

    // EN: Postcondition — result must be in [0, 100]
    // TR: Son koşul — sonuç [0, 100] aralığında olmalı
    ECU_ASSERT(percent >= 0.0 && percent <= 100.0,
               "Throttle percent out of [0, 100] range");

    return percent;
}

// EN: Validate engine RPM — returns clamped value with [[nodiscard]]
// TR: Motor devrini doğrula — [[nodiscard]] ile sınırlanmış değer döndürür
[[nodiscard]] inline int clampRPM(int rpm) {
    // EN: Precondition — RPM should be non-negative from sensor
    // TR: Ön koşul — sensörden gelen RPM negatif olmamalı
    ECU_ASSERT(rpm >= 0, "RPM cannot be negative");

    constexpr int kMaxRPM = 8000;
    if (rpm > kMaxRPM) {
        ECU_LOG(LogLevel::WARN,
                "RPM " + std::to_string(rpm) + " exceeds max, clamping to " +
                std::to_string(kMaxRPM));
        rpm = kMaxRPM;
    }
    return rpm;
}

```

```cpp
// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 5 — InvariantChecker: RAII Guard for Class Invariants
// ═══════════════════════════════════════════════════════════════════════════════

// EN: RAII guard that checks a class invariant when the scope exits
// TR: Kapsam çıkışında sınıf değişmezini kontrol eden RAII koruyucusu
class InvariantChecker {
public:
    using CheckFn = std::function<bool()>;

    // EN: Construct with an invariant check function and description
    // TR: Bir değişmez kontrol fonksiyonu ve açıklama ile oluştur
    InvariantChecker(CheckFn check, std::string description)
        : check_(std::move(check)), desc_(std::move(description)) {}

    // EN: On destruction, verify the invariant holds
    // TR: Yıkımda, değişmezin geçerli olduğunu doğrula
    ~InvariantChecker() {
        if (!check_()) {
            ECU_LOG(LogLevel::ERROR,
                    "INVARIANT VIOLATED: " + desc_);
        }
    }

    InvariantChecker(const InvariantChecker&)            = delete;
    InvariantChecker& operator=(const InvariantChecker&) = delete;

private:
    CheckFn     check_;
    std::string desc_;
};

// EN: Simple battery monitor class that must maintain voltage invariant
// TR: Voltaj değişmezini koruması gereken basit akü izleme sınıfı
class BatteryMonitor {
public:
    explicit BatteryMonitor(double voltage) : voltage_(voltage) {}

    // EN: Update voltage with automatic invariant checking on scope exit
    // TR: Kapsam çıkışında otomatik değişmez kontrolüyle voltajı güncelle
    void updateVoltage(double newVoltage) {
        InvariantChecker guard(
            [this]() { return voltage_ >= 0.0 && voltage_ <= 16.0; },
            "Battery voltage must be in [0.0, 16.0]V"
        );
        voltage_ = newVoltage;
    }

    [[nodiscard]] double voltage() const { return voltage_; }

private:
    double voltage_;
};

// ═══════════════════════════════════════════════════════════════════════════════
// main() — Demonstrations
// ═══════════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "═══════════════════════════════════════════════════════\n";
    std::cout << " Debugging Techniques & Defensive Programming Demos\n";
    std::cout << "═══════════════════════════════════════════════════════\n\n";

```

```cpp
    // ─── Demo 1: ECULogger — Log at Different Levels ──────────────────────────────
    // EN: Demonstrate logging at various severity levels
    // TR: Çeşitli önem seviyelerinde günlüklemeyi göster
    {
        std::cout << "── Demo 1: ECULogger — Log Levels ──\n";

        auto& logger = ECULogger::instance();
        logger.setLevel(LogLevel::TRACE);

        ECU_LOG(LogLevel::TRACE, "Engine control loop iteration 42");
        ECU_LOG(LogLevel::DEBUG, "Throttle ADC raw = 2048");
        ECU_LOG(LogLevel::INFO,  "ECU initialised — firmware v3.2.1");
        ECU_LOG(LogLevel::WARN,  "Coolant temperature approaching limit: 98°C");
        ECU_LOG(LogLevel::ERROR, "CAN bus timeout on node 0x1A — retrying");

        // EN: Raise minimum level to suppress TRACE and DEBUG
        // TR: TRACE ve DEBUG'ı bastırmak için minimum seviyeyi yükselt
        logger.setLevel(LogLevel::INFO);
        ECU_LOG(LogLevel::TRACE, "This TRACE message will be suppressed");
        ECU_LOG(LogLevel::DEBUG, "This DEBUG message will be suppressed");
        ECU_LOG(LogLevel::INFO,  "Only INFO and above are visible now");
        std::cout << '\n';
    }

    // ─── Demo 2: ECU_ASSERT — Precondition Checking ───────────────────────────────
    // EN: Validate inputs using ECU_ASSERT macro
    // TR: Girdileri ECU_ASSERT makrosuyla doğrula
    {
        std::cout << "── Demo 2: ECU_ASSERT — Precondition Checking ──\n";

        // EN: Valid input — should pass assertions
        // TR: Geçerli girdi — onaylamaları geçmeli
        double throttle = computeThrottlePercent(2048);
        std::cout << "  Throttle at ADC 2048: "
                  << std::fixed << std::setprecision(1)
                  << throttle << "%\n";

        throttle = computeThrottlePercent(0);
        std::cout << "  Throttle at ADC 0:    " << throttle << "%\n";

        throttle = computeThrottlePercent(4095);
        std::cout << "  Throttle at ADC 4095: " << throttle << "%\n";

        // EN: Would trigger assertion (uncomment to test):
        // TR: Onaylamayı tetikler (test etmek için yorum işaretini kaldırın): double bad =
        // computeThrottlePercent(-100);  // ASSERTION FAILED
        std::cout << '\n';
    }

    // ─── Demo 3: static_assert — Compile-Time Guarantees ──────────────────────────
    // EN: These checks happen at compile time; if they fail, build stops
    // TR: Bu kontroller derleme zamanında olur; başarısız olursa derleme durur
    {
        std::cout << "── Demo 3: static_assert — Compile-Time Checks ──\n";

        std::cout << "  sizeof(CANFrame)  = " << sizeof(CANFrame)  << " bytes\n";
        std::cout << "  alignof(CANFrame) = " << alignof(CANFrame) << " bytes\n";
        std::cout << "  is_trivially_copyable: "
                  << std::boolalpha
                  << std::is_trivially_copyable_v<CANFrame> << "\n";

        // EN: Additional compile-time checks
        // TR: Ek derleme zamanı kontrolleri
        static_assert(sizeof(int) >= 4, "int must be at least 32 bits");
        static_assert(std::is_same_v<decltype(CANFrame::id), std::uint32_t>,
                      "CAN ID must be uint32_t");

        std::cout << "  All static_assert checks passed at compile time.\n\n";
    }
```

```cpp

    // ─── Demo 4: InvariantChecker — Automatic Postcondition Verify ──────────────────────────
    // EN: RAII guard automatically checks invariant when scope exits
    // TR: RAII koruyucusu kapsam çıkışında otomatik olarak değişmezi kontrol eder
    {
        std::cout << "── Demo 4: InvariantChecker — RAII Guard ──\n";

        BatteryMonitor battery(12.6);
        std::cout << "  Initial voltage: " << battery.voltage() << "V\n";

        // EN: Normal update — invariant holds
        // TR: Normal güncelleme — değişmez geçerli
        battery.updateVoltage(14.2);
        std::cout << "  After charge:    " << battery.voltage() << "V\n";

        // EN: Edge-case update — invariant still holds
        // TR: Sınır değer güncelleme — değişmez hâlâ geçerli
        battery.updateVoltage(0.0);
        std::cout << "  Deep discharge:  " << battery.voltage() << "V\n";

        // EN: Violation — InvariantChecker logs error on scope exit
        // TR: İhlal — InvariantChecker kapsam çıkışında hata günlükler
        battery.updateVoltage(18.5);
        std::cout << "  Over-voltage:    " << battery.voltage()
                  << "V (invariant violated!)\n\n";
    }

    // ─── Demo 5: Defensive Coding Patterns ──────────────────────────────────────────────────
    // EN: Demonstrate [[nodiscard]], clamp, and safe parameter handling
    // TR: [[nodiscard]], sınırlama ve güvenli parametre işlemeyi göster
    {
        std::cout << "── Demo 5: Defensive Coding Patterns ──\n";

        // EN: [[nodiscard]] ensures the caller uses the return value
        // TR: [[nodiscard]] çağıranın dönüş değerini kullanmasını sağlar
        int safeRPM = clampRPM(6500);
        std::cout << "  clampRPM(6500)  = " << safeRPM << "\n";

        safeRPM = clampRPM(9500);
        std::cout << "  clampRPM(9500)  = " << safeRPM << " (clamped)\n";

        // EN: Trying to ignore [[nodiscard]] result causes compiler warning:
        // TR: [[nodiscard]] sonucunu yok saymak derleyici uyarısına neden olur: clampRPM(5000);
        // // WARNING: ignoring return value with 'nodiscard'

        // EN: Defensive null/range checking pattern
        // TR: Savunmacı null/aralık kontrol kalıbı
        auto safeDivide = [](double a, double b) -> double {
            ECU_ASSERT(b != 0.0, "Division by zero in safeDivide");
            return a / b;
        };

        double ratio = safeDivide(100.0, 3.0);
        std::cout << "  safeDivide(100, 3) = "
                  << std::fixed << std::setprecision(4)
                  << ratio << "\n";

        std::cout << '\n';
    }

    std::cout << "═══════════════════════════════════════════════════════════\n";
    std::cout << " All demos completed successfully.\n";
    std::cout << "═══════════════════════════════════════════════════════════\n";

    return 0;
}
```

► 07. Git For Cpp Devs.Md

module_06_industry_standards/07_git_for_cpp_devs.md

Kaynak Kod / Source Code

```
# CLion build directories
out/                  # VS Code CMake output
.cache/               # ccache, clangd cache

# ─── IDE Files / IDE Dosyaları ──────────────────────────────
.vscode/              # VS Code settings (optional: you may want to keep some)
.idea/                # CLion / IntelliJ
*.swp                 # Vim swap files
*.swo
*~                    # Emacs backup files
.vs/                  # Visual Studio

# ─── CMake Generated / CMake Tarafından Oluşturulan ─────────────
CMakeCache.txt
CMakeFiles/
cmake_install.cmake
Makefile              # Only if CMake-generated! Remove this line if you use raw Make
compile_commands.json # Regenerated by CMake, but some prefer to track it
CTestTestfile.cmake
_deps/                # FetchContent downloads

# ─── Dependency Management / Bağımlılık Yönetimi ────────────────
vcpkg_installed/
conan/
_deps/

# ─── Testing & Coverage / Test ve Kapsam ────────────────────────
*.gcda                # GCC coverage data
*.gcno                # GCC coverage notes
*.gcov                # Coverage output
coverage/
Testing/              # CTest output

# ─── Documentation / Dokümantasyon ──────────────────────────────
docs/html/            # Doxygen output (if generated, not committed)
docs/latex/

# ─── OS Files / İşletim Sistemi Dosyaları ───────────────────────
.DS_Store             # macOS
Thumbs.db             # Windows
```

---

3. Branch Stratejisi / Branching Strategy

3.1 Feature Branch Workflow

```
main (or master)
 │
 ├── develop                  ← Integration branch (optional)
 │    ├── feature/can-parser   ← Feature branches
 │    ├── feature/ecu-diag
 │    └── feature/simd-opt
 │
 ├── release/v2.1             ← Release preparation
 │
 └── hotfix/memory-leak       ← Emergency production fixes
```

```bash
# EN: Create a feature branch / TR: Bir feature branch oluştur
git checkout -b feature/can-parser

# EN: Work on your feature (multiple commits are fine)
# TR: Feature üzerinde çalış (birden fazla commit yapılabilir)
git add src/can_parser.cpp include/can_parser.hpp
git commit -m "feat(can): add basic CAN frame parsing"
```

```

git add src/can_parser.cpp tests/test_can_parser.cpp
git commit -m "feat(can): add extended frame (29-bit ID) support"

# EN: Push feature branch to remote / TR: Feature branch'i uzak sunucuya gönder
git push -u origin feature/can-parser

# EN: When done, merge back to main (or create PR)
# TR: Bitince, main'e birleştir (veya PR oluştur)
git checkout main
git merge --no-ff feature/can-parser    # --no-ff creates a merge commit
git branch -d feature/can-parser         # Delete the feature branch
```

3.2 Merge vs Rebase

```bash
# EN: MERGE: Creates a merge commit, preserves full branch history
#     Good for: Feature branches merged via PR, keeping context
# TR: MERGE: Birleştirme commit'i oluşturur, tam dal geçmişini korur

git checkout main
git merge feature/can-parser

# EN: REBASE: Replays your commits on top of main, linear history
#     Good for: Cleaning up local work before merging, bisect-friendly
#     WARNING: Never rebase commits that are already pushed and shared!
# TR: REBASE: Commit'lerinizi main'in üstüne yeniden oynatır, düz geçmiş
#     UYARI: Zaten gönderilmiş ve paylaşılmış commit'leri asla rebase etmeyin!

git checkout feature/can-parser
git rebase main
git checkout main
git merge feature/can-parser    # Fast-forward merge after rebase
```

Altın Kural / Golden Rule:
> EN: **Never rebase commits that have been pushed to a shared branch.**
> TR: **Paylaşılan bir dala gönderilmiş commit'leri asla rebase etmeyin.**

---

4. Conflict Resolution / Çakışma Çözme

```bash
# EN: When a merge fails due to conflicts:
# TR: Bir birleştirme çakışmalar nedeniyle başarısız olduğunda:

git merge feature/can-parser
# Auto-merging src/can_parser.cpp
# CONFLICT (content): Merge conflict in src/can_parser.cpp
# Automatic merge failed; fix conflicts and then commit the result.

git status   # See which files have conflicts

# EN: Open the conflicted file — you'll see markers:
# TR: Çakışan dosyayı aç — işaretçileri göreceksin:
```

```cpp
// EN: Conflict markers in the file:
<<<<<<< HEAD
void parse_frame(const CANFrame& frame) {
    // Current branch's version (HEAD = main)
=======
```

```
void parse_frame(CANFrame frame) {
    // Incoming branch's version (feature/can-parser)
>>>>>>> feature/can-parser
```

```bash
# EN: After manually resolving:
# TR: Manuel olarak çözdükten sonra:
git add src/can_parser.cpp       # Mark as resolved
git commit                       # Complete the merge (default message is fine)
```

Çakışma Önleme İpuçları / Conflict Prevention Tips:
1. **Pull sıkça / Pull frequently:** `git pull --rebase origin main` ile güncel kal
2. **Küçük commit'ler / Small commits:** Daha az çakışma riski
3. **Header/source ayrımı / Header/source separation:** `.hpp` interface, `.cpp` implementasyon
4. **Forward declarations:** `#include` bağımlılıklarını azalt
5. **clang-format:** Herkes aynı formatı kullanırsa boşluk çakışması olmaz

---

5. Git Hooks / Git Kancaları

```bash
# EN: Git hooks are scripts that run automatically at certain points.
#     Stored in .git/hooks/ (local) or managed via tools like pre-commit.
# TR: Git kancaları belirli noktalarda otomatik çalışan betiklerdir.
```

5.1 pre-commit: clang-format

```bash
#!/bin/bash
# .git/hooks/pre-commit — Auto-format C++ files before commit
# EN: This hook runs clang-format on all staged .cpp/.hpp files
# TR: Bu kanca, tüm staged .cpp/.hpp dosyalarına clang-format uygular

STAGED_FILES=$(git diff --cached --name-only --diff-filter=ACM | grep -E '\.(cpp|hpp|h|cc|cxx)$')

if [ -n "$STAGED_FILES" ]; then
    echo "Running clang-format on staged C++ files..."
    for FILE in $STAGED_FILES; do
        clang-format -i "$FILE"
        git add "$FILE"      # Re-stage formatted file
    done
fi
```

5.2 pre-commit: clang-tidy

```bash
#!/bin/bash
# .git/hooks/pre-commit — Static analysis check
# EN: Runs clang-tidy on changed files and blocks commit if issues found
# TR: Değişen dosyalarda clang-tidy çalıştırır, sorun varsa commit'i engeller

STAGED_FILES=$(git diff --cached --name-only --diff-filter=ACM | grep -E '\.(cpp|cc|cxx)$')

if [ -n "$STAGED_FILES" ]; then
    echo "Running clang-tidy..."
    ERRORS=0
    for FILE in $STAGED_FILES; do
        if ! clang-tidy "$FILE" -- -std=c++17 2>/dev/null; then
            ERRORS=1
        fi
    done
    if [ "$ERRORS" -ne 0 ]; then
        echo "ERROR: clang-tidy found issues. Fix them before committing."
        exit 1
    fi
fi
```

5.3 pre-push: Build & Test

```bash
#!/bin/bash
```

```
# .git/hooks/pre-push — Build and test before push
# EN: Ensures code compiles and tests pass before pushing
# TR: Göndermeden önce kodun derlendiğini ve testlerin geçtiğini garanti eder

echo "Running build and tests before push..."
cd build && cmake --build . --target all && ctest --output-on-failure
if [ $? -ne 0 ]; then
    echo "ERROR: Build or tests failed. Push aborted."
    exit 1
fi
echo "All checks passed. Pushing..."
```

5.4 Modern Alternative: pre-commit Framework

```yaml
# .pre-commit-config.yaml — Industry-standard hook management
# Install: pip install pre-commit && pre-commit install

repos:
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.6.0
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
      - id: check-merge-conflict

  - repo: https://github.com/pocc/pre-commit-hooks  # C++ specific
    rev: v1.3.5
    hooks:
      - id: clang-format
        args: [--style=file]    # Uses .clang-format in project root
      - id: clang-tidy
        args: [-p=build]        # Uses compile_commands.json from build/
      - id: cppcheck
```

---

6. Submodules ve Dependency Management / Alt Modüller ve Bağımlılık Yönetimi

6.1 Git Submodules

```bash
# EN: Add an external library as a submodule
# TR: Harici bir kütüphaneyi alt modül olarak ekle
git submodule add https://github.com/google/googletest.git extern/googletest
git submodule add https://github.com/nlohmann/json.git extern/json

# EN: Clone a project with submodules
# TR: Alt modüllü bir projeyi klonla
git clone --recurse-submodules https://github.com/user/project.git

# EN: If already cloned, initialize submodules
# TR: Zaten klonlandıysa, alt modülleri başlat
git submodule update --init --recursive

# EN: Update submodule to latest upstream version
# TR: Alt modülü son upstream sürümüne güncelle
cd extern/googletest
git checkout main && git pull
cd ../..
git add extern/googletest
git commit -m "build: update googletest to latest"
```

```cmake
# CMakeLists.txt — Using submodule in CMake
add_subdirectory(extern/googletest)
target_link_libraries(my_tests PRIVATE gtest_main)
```

6.2 Alternatives to Submodules / Alt Modül Alternatifleri

```
┌──────────────────┬──────────────────────────────────────────────────────┐
│ Method           │ Pros / Cons                                          │
```

```
├──────────────────┼──────────────────────────────────────────────────────┤
│ Git Submodules   │ □ Exact version pinning. □ Confusing workflow        │
│ FetchContent     │ □ CMake-native, auto-download. □ Re-downloads often  │
│ vcpkg            │ □ Large package registry. □ Microsoft-centric        │
│ Conan            │ □ Python-based, flexible. □ Learning curve           │
│ Header-only      │ □ Just copy .hpp file. □ No version management       │
│ System packages  │ □ apt/brew install. □ Version may differ             │
└──────────────────┴──────────────────────────────────────────────────────┘
```

```cmake
# CMake FetchContent — Modern alternative to submodules
include(FetchContent)
FetchContent_Declare(
    googletest
    GIT_REPOSITORY https://github.com/google/googletest.git
    GIT_TAG        v1.14.0  # Pin to specific version!
)
FetchContent_MakeAvailable(googletest)
```

---

7. Faydalı Git Komutları / Useful Git Commands

```bash
# ─── History & Investigation / Geçmiş ve İnceleme ──────────────
git log --oneline --graph --all          # Visual branch history
git log --oneline -20                     # Last 20 commits
git log --author="Murat" --since="1 week ago"  # Filter by author/date
git blame src/engine.cpp                  # Who wrote each line?
git bisect start                          # Binary search for bug-introducing commit

# ─── Stashing / Geçici Saklama ──────────────────────────────────
git stash                                 # Save uncommitted work temporarily
git stash pop                             # Restore stashed work
git stash list                            # List all stashes
git stash drop stash@{0}                  # Remove a specific stash

# ─── Undoing Changes / Değişiklikleri Geri Alma ─────────────────
git checkout -- src/main.cpp              # Discard unstaged changes in file
git reset HEAD src/main.cpp               # Unstage a file
git reset --soft HEAD~1                   # Undo last commit, keep changes staged
git revert abc1234                        # Create new commit that undoes abc1234

# ─── Tagging & Releases / Etiketleme ve Sürümler ────────────────
git tag v1.0.0                            # Lightweight tag
git tag -a v1.0.0 -m "Release 1.0.0"     # Annotated tag (recommended)
git push origin v1.0.0                    # Push tag to remote
git push origin --tags                    # Push all tags

# ─── Cleaning / Temizlik ────────────────────────────────────────
git clean -fd                             # Remove untracked files and directories
git gc                                    # Garbage collect (compress objects)
```

---

8. C++ Spesifik İpuçları / C++-Specific Tips

8.1 compile_commands.json Tracking

```bash
# EN: compile_commands.json is generated by CMake and used by clangd/clang-tidy.
#     Some teams track it, some don't. Best practice:
# TR: compile_commands.json CMake tarafından oluşturulur, clangd/clang-tidy kullanır.

# Option 1: Track it (easier for teammates to get IDE support immediately)
# Option 2: Add to .gitignore and regenerate (cleaner, avoids merge conflicts)

# Generate with CMake:
cmake -B build -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
ln -sf build/compile_commands.json .
```

8.2 Large Binary Files

```bash
# EN: C++ projects sometimes have large test data, firmware images, etc.
#     Use Git LFS (Large File Storage) for these:
# TR: C++ projeleri bazen büyük test verileri, firmware görüntüleri vb. içerir.

git lfs install
git lfs track "*.bin"         # Firmware images
git lfs track "*.hex"         # Intel HEX files
git lfs track "*.elf"         # ELF binaries
git lfs track "*.dbc"         # CAN database files
git lfs track "*.a2l"         # ASAM MCD-2MC calibration files
git add .gitattributes        # LFS tracking info stored here
```

8.3 Useful Git Aliases for C++ Devs

```bash
# Add to ~/.gitconfig:
[alias]
    # Quick status
    s = status -sb

    # Pretty log
    lg = log --oneline --graph --decorate --all -20

    # Show what changed in last commit
    last = log -1 --stat

    # List files changed in a commit
    files = diff-tree --no-commit-id --name-only -r

    # Force-rebuild after checkout (useful for CMake)
    rebuild = !cd build && cmake --build . --clean-first

    # Find all TODO/FIXME in tracked files
    todos = grep -n -i 'TODO\\|FIXME\\|HACK\\|XXX'
```

Özet / Summary

Konu / Topic	Anahtar Kavram / Key Concept
Temel döngü / Basic cycle	`add → commit → push → pull`
Commit mesajları	Conventional Commits: `type(scope): description`
.gitignore	`*.o`, `build/`, IDE dosyaları, `*.gcda`
Branch stratejisi	`main` + `feature/*` + `hotfix/*` + `release/*`
Merge vs Rebase	Merge: tarih korur. Rebase: düz geçmiş. Asla paylaşılanı rebase etme
Conflict çözme	`<<<<<<<`, `=======`, `>>>>>>>` işaretlerini düzenle
Git hooks	`pre-commit`: format + lint; `pre-push`: build + test
Submodules	Harici kitaplıklar için. Alternatif: FetchContent, vcpkg
C++ spesifik	`compile_commands.json`, Git LFS, `.clang-format`

► 08. Cmake Deepdive.Txt

module_06_industry_standards/08_cmake_deepdive.txt

Kaynak Kod / Source Code

```
# CMake Deep Dive — Derinlemesine CMake Rehberi
# ═══════════════════════════════════════════════════════════════════════════
#
# EN: This file is a comprehensive reference for modern CMake (3.20+).
#     It covers target-based commands, FetchContent, presets, cross-compilation,
#     CPack, CTest, generator expressions, and best practices.
#
# TR: Bu dosya modern CMake (3.20+) için kapsamlı bir referanstır.
#     Target tabanlı komutlar, FetchContent, presetler, çapraz derleme,
#     CPack, CTest, generator ifadeleri ve en iyi uygulamaları kapsar.
#
# Date: 2026
# © 2026 Murat Mecit KAHRAMANLI. All rights reserved.
# License: Code: Apache License 2.0 | Book content: CC BY-NC-SA 4.0
# ═══════════════════════════════════════════════════════════════════════════

# ═══════════════════════════════════════════════════════════════════════════
# PART 1: MODERN TARGET-BASED CMAKE
# ═══════════════════════════════════════════════════════════════════════════
#
# [THEORY: Target-Based vs Old-Style CMake / TEORİ: Hedef Tabanlı vs Eski Stil CMake]
#
# EN: OLD-STYLE CMake (before 3.0) used global variables:
#     include_directories(/usr/include/mylib)    ← Affects ALL targets
#     link_libraries(mylib)                     ← Affects ALL targets
#     add_definitions(-DUSE_FEATURE)            ← Affects ALL targets
#
#     MODERN CMake (3.0+) is TARGET-BASED:
#     target_include_directories(myapp PRIVATE ...)  ← Only affects myapp
#     target_link_libraries(myapp PRIVATE mylib)     ← Only affects myapp
#     target_compile_definitions(myapp PRIVATE ...)  ← Only affects myapp
#
#     The key insight: Everything is a TARGET. Libraries are targets.
#     Executables are targets. When you link target A to target B,
#     B's PUBLIC properties propagate to A automatically.
#
# TR: ESKİ CMake (3.0 öncesi) global değişkenler kullanıyordu.
#     MODERN CMake (3.0+) TARGET TABANLIDIR. Her şey bir hedef (target).
#
# [THEORY: Visibility Keywords / TEORİ: Görünürlük Anahtar Kelimeleri]
#
# EN:
#     ┌──────────┬──────────────────────────────────────────────────┐
#     │ Keyword  │ Who sees it?                                     │
#     ├──────────┼──────────────────────────────────────────────────┤
#     │ PRIVATE  │ Only the target itself                           │
#     │ PUBLIC   │ The target AND anyone who links to it            │
#     │ INTERFACE│ Only those who link to it (not the target itself)│
#     └──────────┴──────────────────────────────────────────────────┘
#
#     Example: If library "math_lib" has:
#       target_include_directories(math_lib PUBLIC include/)
#     Then any target linking to math_lib automatically gets include/ too.
#
# TR:
#     ┌──────────┬──────────────────────────────────────────────────┐
#     │ Anahtar  │ Kim görür?                                       │
#     ├──────────┼──────────────────────────────────────────────────┤
#     │ PRIVATE  │ Sadece hedefin kendisi                           │
#     │ PUBLIC   │ Hedef VE ona bağlanan herkes                     │
#     │ INTERFACE│ Sadece ona bağlananlar (hedefin kendisi değil)   │
#     └──────────┴──────────────────────────────────────────────────┘
#

# Example: Complete Modern CMake Project
# ─────────────────────────────────────────────────────────────────────────

cmake_minimum_required(VERSION 3.20)
project(ecu_firmware
    VERSION 2.1.0
    DESCRIPTION "Automotive ECU Firmware"
    LANGUAGES CXX
```

```
70  )
71
72  # EN: Set C++ standard at project level (targets can override)
73  # TR: Proje seviyesinde C++ standardını ayarla (hedefler geçersiz kılabilir)
74  set(CMAKE_CXX_STANDARD 20)
75  set(CMAKE_CXX_STANDARD_REQUIRED ON)
76  set(CMAKE_CXX_EXTENSIONS OFF)   # EN: No GNU extensions (strict ISO) / TR: GNU uzantıları yok
77
78  # EN: Export compile_commands.json for clangd/clang-tidy
79  # TR: clangd/clang-tidy için compile_commands.json oluştur
80  set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
81
82  # ── Library target ──
83
84  # EN: Create a STATIC library from source files
85  # TR: Kaynak dosyalardan STATİK kütüphane oluştur
86  add_library(can_stack STATIC
87      src/can_driver.cpp
88      src/can_parser.cpp
89      src/can_transport.cpp
90  )
91
92  # EN: target_include_directories: Tell the compiler where to find headers
93  #     PUBLIC = both this library and its consumers need these headers
94  # TR: Derleyiciye başlık dosyalarını nerede bulacağını söyle
95  target_include_directories(can_stack
96      PUBLIC
97          $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
98          $<INSTALL_INTERFACE:include>
99      PRIVATE
100         ${CMAKE_CURRENT_SOURCE_DIR}/src  # EN: Internal headers / TR: İç başlıklar
101 )
102
103 # EN: target_compile_features: Require specific C++ features
104 #     This is BETTER than set(CMAKE_CXX_STANDARD) for libraries
105 # TR: Belirli C++ özelliklerini gerekli kıl — kütüphaneler için daha iyi
106 target_compile_features(can_stack PUBLIC cxx_std_20)
107
108 # EN: target_compile_options: Add warning flags
109 # TR: Uyarı bayraklarını ekle
110 target_compile_options(can_stack PRIVATE
111     -Wall -Wextra -Wpedantic -Wshadow -Wconversion
112     $<$<CONFIG:Debug>:-g -O0 -fsanitize=address>
113     $<$<CONFIG:Release>:-O3 -DNDEBUG>
114 )
115
116 # EN: target_compile_definitions: Preprocessor defines
117 # TR: Önişlemci tanımları
118 target_compile_definitions(can_stack
119     PUBLIC
120         ECU_PROJECT_VERSION="${PROJECT_VERSION}"
121     PRIVATE
122         CAN_INTERNAL_DEBUG     # EN: Only visible inside can_stack / TR: Sadece can_stack içinde
     görünür
123 )
124
125 # ── Executable target ──
126
127 add_executable(ecu_app src/main.cpp)
128
129 # EN: Linking to can_stack propagates its PUBLIC include dirs,
130 #     compile features, and definitions automatically
131 # TR: can_stack'e bağlanmak onun PUBLIC include dizinlerini,
132 #     derleme özelliklerini ve tanımlarını otomatik yayar
133 target_link_libraries(ecu_app PRIVATE can_stack)
134
135
136 # ═══════════════════════════════════════════════════════════════════════
137 # PART 2: FETCHCONTENT — Dependency Management
138 # ═══════════════════════════════════════════════════════════════════════
139 #
140 # [THEORY: FetchContent vs Submodules vs Package Managers / TEORİ: FetchContent, Submodüller ve
     Paket Yöneticileri]
141 #
142 # EN: FetchContent (CMake 3.11+) downloads dependencies at configure time.
143 #     No need for git submodules, apt install, or manual downloads.
144 #
```

```
#      ┌──────────────────┬──────────────────────────────────────────┐
#      │ Method           │ Trade-offs                               │
#      ├──────────────────┼──────────────────────────────────────────┤
#      │ FetchContent     │ ✓ Auto-download, version pinning         │
#      │                  │ ✗ Re-downloads on clean build, slow first config │
#      │ Git Submodules   │ ✓ Version pinning, local copy            │
#      │                  │ ✗ Confusing workflow, easy to get wrong  │
#      │ vcpkg            │ ✓ Pre-built binaries, large registry     │
#      │                  │ ✗ Extra tool, Microsoft-centric          │
#      │ Conan            │ ✓ Flexible, multi-platform, binary caching │
#      │                  │ ✗ Python dependency, learning curve      │
#      │ System packages  │ ✓ Zero setup (apt install)               │
#      │                  │ ✗ Version pinning impossible, not reproducible │
#      └──────────────────┴──────────────────────────────────────────┘
#
# TR: FetchContent (CMake 3.11+), bağımlılıkları yapılandırma zamanında indirir.

include(FetchContent)

# EN: Download GoogleTest at configure time
# TR: GoogleTest'i yapılandırma zamanında indir
FetchContent_Declare(
    googletest
    GIT_REPOSITORY https://github.com/google/googletest.git
    GIT_TAG        v1.14.0       # EN: Always pin to specific version! / TR: Her zaman sürüm sabitle!
    GIT_SHALLOW    TRUE          # EN: Don't download full history / TR: Tam geçmişi indirme
)

# EN: Download nlohmann/json header-only library
# TR: nlohmann/json başlık-yalnız kütüphanesini indir
FetchContent_Declare(
    nlohmann_json
    GIT_REPOSITORY https://github.com/nlohmann/json.git
    GIT_TAG        v3.11.3
    GIT_SHALLOW    TRUE
)

# EN: Download fmt (formatting library, predecessor of std::format)
# TR: fmt kütüphanesini indir (std::format'ın öncüsü)
FetchContent_Declare(
    fmt
    GIT_REPOSITORY https://github.com/fmtlib/fmt.git
    GIT_TAG        10.2.1
    GIT_SHALLOW    TRUE
)

# EN: MakeAvailable downloads + adds to build (calls add_subdirectory internally)
# TR: MakeAvailable indirir + derlemeye ekler (dahili olarak add_subdirectory çağırır)
FetchContent_MakeAvailable(googletest nlohmann_json fmt)

# EN: Now you can link to them like any other target:
# TR: Artık bunlara diğer hedefler gibi bağlanabilirsin:
#   target_link_libraries(ecu_app PRIVATE nlohmann_json::nlohmann_json)
#   target_link_libraries(ecu_app PRIVATE fmt::fmt)
#   target_link_libraries(test_app PRIVATE GTest::gtest_main)


# ═══════════════════════════════════════════════════════════════════════
# PART 3: CMAKE PRESETS (CMakePresets.json)
# ═══════════════════════════════════════════════════════════════════════
#
# [THEORY: What Are CMake Presets? / TEORİ: CMake Presetleri Nedir?]
#
# EN: CMake Presets (3.19+) replace long command-line arguments with a JSON file.
#     Instead of: cmake -B build -DCMAKE_BUILD_TYPE=Debug -DCMAKE_CXX_COMPILER=g++-12 ...
#     You write:  cmake --preset debug
#
#     Presets file goes in project root: CMakePresets.json (committed to VCS)
#     User overrides go in: CMakeUserPresets.json (in .gitignore!)
#
# TR: CMake Presetleri uzun komut satırı argümanlarını JSON dosyası ile değiştirir.
#
# Example CMakePresets.json:
# ─────────────────────────────────────────────────────────────────────
# {
#     "version": 6,
#     "cmakeMinimumRequired": { "major": 3, "minor": 25, "patch": 0 },
```

```
#
#     "configurePresets": [
#         {
#             "name": "base",
#             "hidden": true,
#             "generator": "Ninja",
#             "binaryDir": "${sourceDir}/build/${presetName}",
#             "cacheVariables": {
#                 "CMAKE_CXX_STANDARD": "20",
#                 "CMAKE_EXPORT_COMPILE_COMMANDS": "ON"
#             }
#         },
#         {
#             "name": "debug",
#             "displayName": "Debug (GCC, ASan)",
#             "inherits": "base",
#             "cacheVariables": {
#                 "CMAKE_BUILD_TYPE": "Debug",
#                 "CMAKE_CXX_FLAGS": "-fsanitize=address -fno-omit-frame-pointer"
#             }
#         },
#         {
#             "name": "release",
#             "displayName": "Release (GCC, LTO)",
#             "inherits": "base",
#             "cacheVariables": {
#                 "CMAKE_BUILD_TYPE": "Release",
#                 "CMAKE_INTERPROCEDURAL_OPTIMIZATION": "ON"
#             }
#         },
#         {
#             "name": "cross-arm",
#             "displayName": "Cross-compile for ARM Cortex-M4",
#             "inherits": "base",
#             "toolchainFile": "${sourceDir}/cmake/toolchain-arm-none-eabi.cmake",
#             "cacheVariables": {
#                 "CMAKE_BUILD_TYPE": "Release"
#             }
#         }
#     ],
#
#     "buildPresets": [
#         { "name": "debug",   "configurePreset": "debug" },
#         { "name": "release", "configurePreset": "release" }
#     ],
#
#     "testPresets": [
#         {
#             "name": "debug-tests",
#             "configurePreset": "debug",
#             "output": { "outputOnFailure": true },
#             "execution": { "jobs": 4 }
#         }
#     ]
# }
#
# Usage / Kullanım:
#   cmake --preset debug          # Configure
#   cmake --build --preset debug  # Build
#   ctest --preset debug-tests    # Test


# ═══════════════════════════════════════════════════════════════════════════
# PART 4: CROSS-COMPILATION TOOLCHAIN FILES
# ═══════════════════════════════════════════════════════════════════════════
#
# [THEORY: Cross-Compilation / TEORİ: Çapraz Derleme]
#
# EN: Cross-compilation = building code on one platform (host) to run on
#     another (target). Common in automotive: develop on x86 Linux,
#     deploy to ARM Cortex-M4 ECU.
#
#     A toolchain file tells CMake:
#     - Which compiler to use (arm-none-eabi-gcc)
#     - What the target system is (Generic = bare metal)
#     - Where to find libraries for the target
#
```

```
# TR: Çapraz derleme = bir platformda (ana makine) derleyip başka bir
#     platformda (hedef) çalıştırmak. Otomotivde yaygın: x86 Linux'ta
#     geliştir, ARM Cortex-M4 ECU'ya dağıt.
#
# Example: cmake/toolchain-arm-none-eabi.cmake
# ──────────────────────────────────────────────────────────────────────────
# set(CMAKE_SYSTEM_NAME Generic)        # EN: Bare-metal (no OS)
# set(CMAKE_SYSTEM_PROCESSOR arm)
#
# set(CMAKE_C_COMPILER   arm-none-eabi-gcc)
# set(CMAKE_CXX_COMPILER arm-none-eabi-g++)
# set(CMAKE_ASM_COMPILER arm-none-eabi-gcc)
#
# # EN: Cortex-M4 with hardware FPU
# set(CMAKE_C_FLAGS   "-mcpu=cortex-m4 -mthumb -mfloat-abi=hard -mfpu=fpv4-sp-d16")
# set(CMAKE_CXX_FLAGS "${CMAKE_C_FLAGS} -fno-exceptions -fno-rtti")
#
# # EN: Don't try to link during compiler detection (bare metal has no stdlib)
# set(CMAKE_TRY_COMPILE_TARGET_TYPE STATIC_LIBRARY)
#
# # EN: Sysroot for target libraries
# set(CMAKE_FIND_ROOT_PATH /opt/arm-toolchain/sysroot)
# set(CMAKE_FIND_ROOT_PATH_MODE_PROGRAM NEVER)
# set(CMAKE_FIND_ROOT_PATH_MODE_LIBRARY ONLY)
# set(CMAKE_FIND_ROOT_PATH_MODE_INCLUDE ONLY)
#
# Usage / Kullanım:
#   cmake -B build-arm -DCMAKE_TOOLCHAIN_FILE=cmake/toolchain-arm-none-eabi.cmake
#   cmake --build build-arm


# ══════════════════════════════════════════════════════════════════════════
# PART 5: CPACK — Packaging
# ══════════════════════════════════════════════════════════════════════════
#
# [THEORY: CPack / TEORİ: Paketleme]
#
# EN: CPack creates distributable packages (DEB, RPM, ZIP, NSIS installer)
#     from your CMake project. No separate packaging scripts needed.
#
# TR: CPack, CMake projenizden dağıtılabilir paketler (DEB, RPM, ZIP) oluşturur.

# EN: Install rules tell CMake what goes into the package
# TR: Kurulum kuralları CMake'e pakete neyin gideceğini söyler
install(TARGETS ecu_app
    RUNTIME DESTINATION bin
)
install(TARGETS can_stack
    ARCHIVE DESTINATION lib
    LIBRARY DESTINATION lib
)
install(DIRECTORY include/
    DESTINATION include
    FILES_MATCHING PATTERN "*.hpp"
)

# EN: CPack configuration
# TR: CPack yapılandırması
set(CPACK_PACKAGE_NAME "ecu-firmware")
set(CPACK_PACKAGE_VERSION ${PROJECT_VERSION})
set(CPACK_PACKAGE_DESCRIPTION_SUMMARY "ECU Firmware Package")
set(CPACK_PACKAGE_CONTACT "developer@example.com")

# EN: Generate DEB package (Debian/Ubuntu)
# TR: DEB paketi oluştur (Debian/Ubuntu)
set(CPACK_GENERATOR "DEB;TGZ;ZIP")
set(CPACK_DEBIAN_PACKAGE_DEPENDS "libc6 (>= 2.35)")

include(CPack)

# Usage / Kullanım:
#   cmake --build build --target package
#   → ecu-firmware-2.1.0-Linux.deb
#   → ecu-firmware-2.1.0-Linux.tar.gz
#   → ecu-firmware-2.1.0-Linux.zip


```

```
# ═══════════════════════════════════════════════════════════════════════════
# PART 6: CTEST — Testing Integration
# ═══════════════════════════════════════════════════════════════════════════
#
# [THEORY: CTest / TEORİ: Test Entegrasyonu]
#
# EN: CTest is CMake's built-in test runner. It works with ANY test framework
#     (GoogleTest, Catch2, doctest, or plain executables).
#
# TR: CTest, CMake'in yerleşik test çalıştırıcısıdır. HERHANGİ bir test
#     çerçevesiyle çalışır.

enable_testing()

# EN: Method 1: Manual test registration
# TR: Yöntem 1: Manuel test kaydı
add_executable(test_can_parser tests/test_can_parser.cpp)
target_link_libraries(test_can_parser PRIVATE can_stack GTest::gtest_main)
add_test(NAME CanParserTests COMMAND test_can_parser)

# EN: Method 2: GoogleTest auto-discovery (recommended)
#     Scans the test executable for TEST() macros and registers each one
# TR: Yöntem 2: GoogleTest otomatik keşif (önerilen)
include(GoogleTest)
gtest_discover_tests(test_can_parser
    PROPERTIES
        TIMEOUT 30           # EN: 30 second timeout per test
        LABELS "unit"        # EN: Label for filtering
)

# EN: Method 3: Custom test with arguments
# TR: Yöntem 3: Argümanlı özel test
add_test(
    NAME IntegrationTest_CAN
    COMMAND ecu_app --test-mode --bus=vcan0
    WORKING_DIRECTORY ${CMAKE_BINARY_DIR}
)
set_tests_properties(IntegrationTest_CAN PROPERTIES
    TIMEOUT 60
    LABELS "integration"
    ENVIRONMENT "CAN_BITRATE=500000"
)

# Usage / Kullanım:
#   cd build && ctest                        # Run all tests
#   ctest --output-on-failure               # Show output only if tests fail
#   ctest -L unit                            # Run only tests labeled "unit"
#   ctest -L integration                     # Run only integration tests
#   ctest -j $(nproc)                        # Run tests in parallel
#   ctest --rerun-failed                     # Rerun only failed tests


# ═══════════════════════════════════════════════════════════════════════════
# PART 7: GENERATOR EXPRESSIONS
# ═══════════════════════════════════════════════════════════════════════════
#
# [THEORY: Generator Expressions / TEORİ: Üreteci İfadeleri]
#
# EN: Generator expressions ($<...>) are evaluated at BUILD TIME (not configure time).
#     They allow conditional behavior based on build configuration, platform, etc.
#
#     Key generator expressions:
#     ┌──────────────────────────────────────┬──────────────────────────────────┐
#     │ Expression                           │ Meaning                          │
#     ├──────────────────────────────────────┼──────────────────────────────────┤
#     │ $<CONFIG:Debug>                      │ True if Debug build              │
#     │ $<CONFIG:Release>                    │ True if Release build            │
#     │ $<PLATFORM_ID:Linux>                 │ True if building on Linux        │
#     │ $<CXX_COMPILER_ID:GNU>               │ True if using GCC                │
#     │ $<BOOL:${MY_VAR}>                    │ True if MY_VAR is truthy         │
#     │ $<BUILD_INTERFACE:...>               │ Only during build (not install)  │
#     │ $<INSTALL_INTERFACE:...>             │ Only after install               │
#     │ $<TARGET_FILE:target>                │ Full path to target's output file │
#     │ $<IF:cond,true_val,false_val>        │ Conditional value (CMake 3.8+)   │
#     └──────────────────────────────────────┴──────────────────────────────────┘
#
#
# TR: Generator ifadeleri ($<...>) DERLEME ZAMANINDA değerlendirilir.
```

```
453
454 # EN: Example: Different flags for Debug vs Release
455 # TR: Örnek: Debug ve Release için farklı bayraklar
456 target_compile_options(can_stack PRIVATE
457     $<$<CONFIG:Debug>:-g -O0 -DDEBUG_MODE>
458     $<$<CONFIG:Release>:-O3 -DNDEBUG -march=native>
459 )
460
461 # EN: Example: Compiler-specific flags
462 # TR: Örnek: Derleyiciye özel bayraklar
463 target_compile_options(can_stack PRIVATE
464     $<$<CXX_COMPILER_ID:GNU>:-Wlogical-op -Wduplicated-cond>
465     $<$<CXX_COMPILER_ID:Clang>:-Wno-unused-lambda-capture>
466     $<$<CXX_COMPILER_ID:MSVC>:/W4 /permissive->
467 )
468
469 # EN: Example: Platform-specific source files
470 # TR: Örnek: Platforma özel kaynak dosyaları
471 target_sources(can_stack PRIVATE
472     $<$<PLATFORM_ID:Linux>:src/can_socketcan.cpp>
473     $<$<PLATFORM_ID:Windows>:src/can_peak.cpp>
474 )
475
476
477 # ═══════════════════════════════════════════════════════════════════════
478 # PART 8: BEST PRACTICES SUMMARY
479 # ═══════════════════════════════════════════════════════════════════════
480 #
481 # EN: Modern CMake Best Practices:
482 #
483 #     DO:
484 #     □ Use target_* commands (not global include_directories, etc.)
485 #     □ Specify PRIVATE/PUBLIC/INTERFACE correctly
486 #     □ Use FetchContent or package managers for dependencies
487 #     □ Use CMakePresets.json for reproducible builds
488 #     □ Use generator expressions for config-dependent behavior
489 #     □ Pin dependency versions (GIT_TAG v1.14.0, not GIT_TAG main)
490 #     □ Use CMAKE_EXPORT_COMPILE_COMMANDS for IDE/tooling support
491 #     □ Use target_compile_features (cxx_std_20) for portability
492 #
493 #     DON'T:
494 #     □ Use include_directories() — use target_include_directories()
495 #     □ Use link_libraries() — use target_link_libraries()
496 #     □ Use add_definitions() — use target_compile_definitions()
497 #     □ Set CMAKE_CXX_FLAGS directly — use target_compile_options()
498 #     □ Use file(GLOB ...) for source files (fragile, misses new files)
499 #     □ Use GIT_TAG main/master (non-reproducible builds!)
500 #     □ Hardcode paths (/usr/lib/...) — use find_package() or FetchContent
501 #
502 # TR: Modern CMake En İyi Uygulamaları:
503 #     YAP: target_* komutları kullan, FetchContent kullan, sürüm sabitle
504 #     YAPMA: Global komutlar kullanma, CMAKE_CXX_FLAGS doğrudan ayarlama,
505 #            file(GLOB) kullanma, ana dalı GIT_TAG olarak verme
506 #
507 #
508 # ═══════════════════════════════════════════════════════════════════════
509 # QUICK REFERENCE: Common CMake Commands
510 # ═══════════════════════════════════════════════════════════════════════
511 #
512 # ┌──────────────────────────────────────────┬──────────────────────────┐
513 # │ Command                                  │ Purpose                  │
514 # ├──────────────────────────────────────────┼──────────────────────────┤
515 # │ cmake_minimum_required(VERSION x.y)      │ Minimum CMake version    │
516 # │ project(name VERSION x.y LANGUAGES CXX)  │ Declare project          │
517 # │ add_executable(target src1.cpp src2.cpp) │ Create executable target │
518 # │ add_library(target STATIC/SHARED src.cpp) │ Create library target   │
519 # │ target_include_directories(t SCOPE dirs) │ Add include paths        │
520 # │ target_link_libraries(t SCOPE libs)      │ Link to libraries        │
521 # │ target_compile_definitions(t SCOPE defs) │ Add preprocessor defines │
522 # │ target_compile_options(t SCOPE flags)    │ Add compiler flags       │
523 # │ target_compile_features(t SCOPE features) │ Require C++ standard features │
524 # │ find_package(Pkg REQUIRED)               │ Find installed package   │
525 # │ FetchContent_Declare(name ...)           │ Declare downloadable dependency │
526 # │ FetchContent_MakeAvailable(name)         │ Download and add to build │
527 # │ install(TARGETS/FILES/DIRECTORY ...)     │ Install rules for packaging │
528 # │ enable_testing() + add_test(...)         │ Enable CTest             │
529 # │ include(GoogleTest)                      │ GoogleTest integration   │
```

```
530 # │ gtest_discover_tests(target)                │ Auto-register tests                          │
531 # └─────────────────────────────────────────────┴──────────────────────────────────────────────┘
```

► 09. Mutex Lock Deepdive

module_06_industry_standards/09_mutex_lock_deepdive.cpp

Konu: Mutex & Lock Deep Dive — Mutex ve Kilit Mekanizmaları Derinlemesine

Teori / Theory

THEORY: C++ Lock Guard Hierarchy

C++ provides multiple lock wrappers with increasing flexibility:

```
┌──────────────────────┬──────────────────────────────────────────┐
│ Type                 │ Use Case                                 │
├──────────────────────┼──────────────────────────────────────────┤
│ lock_guard           │ Simple RAII lock, no manual unlock/relock.    │
│                      │ Locks in ctor, unlocks in dtor. Fastest.      │
├──────────────────────┼──────────────────────────────────────────┤
│ unique_lock          │ Deferred locking, try_lock, timed_lock,       │
│                      │ manual unlock/relock. Required for cond_var.  │
├──────────────────────┼──────────────────────────────────────────┤
│ scoped_lock (C++17)  │ Locks MULTIPLE mutexes atomically without     │
│                      │ deadlock (uses deadlock avoidance algorithm). │
├──────────────────────┼──────────────────────────────────────────┤
│ shared_lock (C++17)  │ Shared (reader) access with shared_mutex.     │
│                      │ Multiple readers, exclusive writer.           │
└──────────────────────┴──────────────────────────────────────────┘
```

TEORİ: C++ Kilit Koruyucu Hiyerarşisi

C++ artan esneklikle birden fazla kilit sarmalayıcı sağlar:

```
┌──────────────────────┬──────────────────────────────────────────┐
│ Tür                  │ Kullanım Alanı                           │
├──────────────────────┼──────────────────────────────────────────┤
│ lock_guard           │ Basit RAII kilit, elle açma/tekrar kilitleme  │
│                      │ yok. Ctor'da kilitler, dtor'da açar. En hızlı.│
├──────────────────────┼──────────────────────────────────────────┤
│ unique_lock          │ Ertelenmiş kilitleme, try_lock, zamanlı kilit,│
│                      │ elle açma/tekrar kilitl. cond_var için ŞART.  │
├──────────────────────┼──────────────────────────────────────────┤
│ scoped_lock (C++17)  │ Birden fazla mutex'i atomik olarak kilitler   │
│                      │ deadlock yok (kaçınma algoritması kullanır).  │
├──────────────────────┼──────────────────────────────────────────┤
│ shared_lock (C++17)  │ shared_mutex ile paylaşımlı (okuyucu) erişim. │
│                      │ Çoklu okuyucu, tekli yazıcı.                  │
└──────────────────────┴──────────────────────────────────────────┘
```

THEORY: std::unique_lock

unique_lock is the Swiss Army knife of locks:
unique_lock<mutex> lk(mtx); // Lock immediately unique_lock<mutex> lk(mtx, defer_lock); // Don't lock yet unique_lock<mutex> lk(mtx, try_to_lock); // Try, don't block unique_lock<mutex> lk(mtx, adopt_lock); // Already locked
lk.lock(); // Lock manually (after defer_lock) lk.unlock(); // Unlock manually (e.g., for long non-critical section) lk.try_lock(); // Non-blocking attempt lk.owns_lock(); // Check if currently locked
WHY use unique_lock over lock_guard?
1. condition_variable::wait() REQUIRES unique_lock
2. You need to unlock early (minimize lock scope)
3. You need try_lock or timed_lock
4. You need to transfer lock ownership (move)

TEORİ: std::unique_lock

unique_lock kilitlerde İsviçre Çakısı'dır:
unique_lock<mutex> lk(mtx); // Hemen kilitle unique_lock<mutex> lk(mtx, defer_lock); // Henüz kilitleme unique_lock<mutex> lk(mtx, try_to_lock); // Dene, bloklanma unique_lock<mutex> lk(mtx, adopt_lock); // Zaten kilitli
lk.lock(); // Elle kilitle (defer_lock sonrası) lk.unlock(); // Elle aç (uzun kritik olmayan bölüm için) lk.try_lock(); // Bloklanmayan deneme lk.owns_lock(); // Şu an kilitli mi kontrol et
NEDEN lock_guard yerine unique_lock?
1. condition_variable::wait() unique_lock GEREKTİRİR
2. Erken açma ihtiyacınız var (kilit kapsamını minimize et)
3. try_lock veya zamanlı kilit gerekiyor
4. Kilit sahipliğini transfer etmeniz gerekiyor (move)

THEORY: std::shared_mutex — Reader-Writer Lock

shared_mutex (C++17) allows:
- Multiple simultaneous READERS (shared_lock)
- Exclusive single WRITER (unique_lock or lock_guard)
Perfect for: caches, config stores, sensor data buffers where reads are much more frequent than writes.
Read path: shared_lock<shared_mutex> lk(mtx); // Multiple ok Write path: unique_lock<shared_mutex> lk(mtx); // Exclusive

Scenario	Readers	Writer
Read-only	N (∞)	0 — all readers proceed concurrently
Write	0	1 — writer waits for readers to end
Read + Write	blocked	1 — readers wait until writer done

TEORİ: std::shared_mutex — Okuyucu-Yazıcı Kilidi

shared_mutex (C++17) şunlara izin verir:
- Birden fazla eşzamanlı OKUYUCU (shared_lock)
- Özel tekli YAZICI (unique_lock veya lock_guard)

İdeal kullanım: önbellekler, config depoları, sensör veri tamponları — okumaların yazmalardan çok daha sık olduğu durumlar.
Okuma yolu: shared_lock<shared_mutex> lk(mtx); // Çoklu OK Yazma yolu: unique_lock<shared_mutex> lk(mtx); // Özel (tekli)

Senaryo	Okuyucu	Yazıcı
Salt okuma	N (∞)	0 — tüm okuyucular eşzamanlı devam
Yazma	0	1 — yazıcı okuyucuların bitmesini bk.
Okuma + Yazma	bloklanır	1 — okuyucular yazıcı bitene dek bk.

THEORY: std::scoped_lock — Deadlock-Free Multi-Lock

scoped_lock (C++17) locks multiple mutexes atomically using a deadlock avoidance algorithm (std::lock under the hood).
DEADLOCK EXAMPLE (without scoped_lock): Thread 1: lock(A) → lock(B) // A then B Thread 2: lock(B) → lock(A) // B then A → DEADLOCK!
scoped_lock solution: Thread 1: scoped_lock lk(A, B); // Always locks in safe order Thread 2: scoped_lock lk(A, B); // Same — no deadlock possible
Rule: ALWAYS prefer scoped_lock when locking multiple mutexes.

TEORİ: std::scoped_lock — Kilitlenmesiz Çoklu Kilit

scoped_lock (C++17) birden fazla mutex'i atomik olarak kilitler. Deadlock önleme algoritması kullanır (altta std::lock).
DEADLOCK ÖRNEĞİ (scoped_lock olmadan): Thread 1: lock(A) → lock(B) // Önce A sonra B Thread 2: lock(B) → lock(A) // Önce B sonra A → DEADLOCK!
scoped_lock çözümü: Thread 1: scoped_lock lk(A, B); // Her zaman güvenli sırada kilitler Thread 2: scoped_lock lk(A, B); // Aynı — deadlock imkansız
Kural: Birden fazla mutex kilitlerken HER ZAMAN scoped_lock tercih et.

THEORY: std::recursive_mutex

recursive_mutex allows the same thread to lock multiple times. It maintains a lock count — unlock must be called the same number of times.
When to use: Functions that call each other, all needing the lock. WARNING: Recursive mutex is often a design smell! Consider refactoring:
- Extract locked helper (private, no lock) + public locking wrapper
- This avoids the overhead and complexity of recursive_mutex

TEORİ: std::recursive_mutex — Özyinelemeli Mutex

recursive_mutex aynı thread'in birden fazla kez kilitlemesine izin verir. Kilit sayacı tutar — unlock aynı sayıda çağrılmalıdır.
Ne zaman kullanılır: Birbirini çağıran fonksiyonlar, hepsi kilide ihtiyaç duyar. UYARI: Recursive mutex genellikle tasarım kokusu! Yeniden yapılandırmayı düşün:
- Kilitli yardımcı (private, kilitsiz) + genel kilitleme sarmalayıcı
- Bu recursive_mutex'in ek yükünü ve karmaşıklığını önler

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <mutex>
#include <shared_mutex>
#include <thread>
#include <vector>
#include <string>
#include <chrono>
#include <map>
#include <atomic>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: std::unique_lock — Flexible Locking
// ═══════════════════════════════════════════════════════════════════════════

// EN: Sensor data buffer with deferred locking
// TR: Ertelenmiş kilitleme ile sensör veri tamponu
class SensorBuffer {
public:
    void update(double value) {
        // EN: Deferred locking — lock only when needed
        // TR: Ertelenmiş kilitleme — sadece gerektiğinde kilitle
        std::unique_lock<std::mutex> lk(mtx_, std::defer_lock);

        // EN: Some non-critical preprocessing (no lock needed here)
        // TR: Kritik olmayan ön işleme (burada kilide gerek yok)
        double processed = value * 1.8 + 32.0;  // Celsius → Fahrenheit

        // EN: Now lock for the critical section
        // TR: Şimdi kritik bölüm için kilitle
        lk.lock();
        buffer_.push_back(processed);
        last_update_ = std::chrono::steady_clock::now();
        // EN: lk automatically unlocks when it goes out of scope
        // TR: lk kapsam dışına çıkınca otomatik olarak kilidi açar
    }

    double average() const {
        std::lock_guard<std::mutex> lk(mtx_);
        if (buffer_.empty()) return 0.0;
        double sum = 0.0;
        for (auto v : buffer_) sum += v;
        return sum / static_cast<double>(buffer_.size());
    }

    size_t size() const {
        std::lock_guard<std::mutex> lk(mtx_);
        return buffer_.size();
    }

    bool try_update(double value) {
        // EN: try_to_lock — don't block if mutex is busy
        // TR: try_to_lock — mutex meşgulse bloke olma
        std::unique_lock<std::mutex> lk(mtx_, std::try_to_lock);
        if (!lk.owns_lock()) {
            return false;  // EN: Mutex busy, caller can retry or skip
        }
        buffer_.push_back(value);
        return true;
    }

private:
    mutable std::mutex mtx_;
    std::vector<double> buffer_;
    std::chrono::steady_clock::time_point last_update_;
};

```

```cpp
// ═══════════════════════════════════════════════════════════════════════════
// PART 2: std::shared_mutex — Reader-Writer Lock
// ═══════════════════════════════════════════════════════════════════════════

// EN: Configuration store — reads are frequent, writes are rare
// TR: Konfigürasyon deposu — okuma sık, yazma nadir
class ConfigStore {
public:
    void set(const std::string& key, const std::string& value) {
        // EN: Exclusive lock for writing — blocks ALL readers and writers
        // TR: Yazmak için özel kilit — TÜM okuyucu ve yazıcıları durdurur
        std::unique_lock<std::shared_mutex> lk(mtx_);
        config_[key] = value;
        write_count_++;
    }

    std::string get(const std::string& key) const {
        // EN: Shared lock for reading — multiple readers can proceed
        // TR: Okumak için paylaşımlı kilit — birden fazla okuyucu ilerleyebilir
        std::shared_lock<std::shared_mutex> lk(mtx_);
        read_count_++;
        auto it = config_.find(key);
        return (it != config_.end()) ? it->second : "(not found)";
    }

    size_t read_ops() const { return read_count_.load(); }
    size_t write_ops() const { return write_count_.load(); }

private:
    mutable std::shared_mutex mtx_;
    std::map<std::string, std::string> config_;
    mutable std::atomic<size_t> read_count_{0};
    std::atomic<size_t> write_count_{0};
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: std::scoped_lock — Deadlock-Free Multi-Lock
// ═══════════════════════════════════════════════════════════════════════════

// EN: Bank accounts — transfer requires locking TWO mutexes
// TR: Banka hesapları — transfer İKİ mutex'i kilitlemeyi gerektirir
class BankAccount {
public:
    explicit BankAccount(std::string name, double balance)
        : name_(std::move(name)), balance_(balance) {}

    // EN: SAFE transfer using scoped_lock — no deadlock!
    // TR: scoped_lock kullanarak GÜVENLİ transfer — deadlock yok!
    static void transfer(BankAccount& from, BankAccount& to, double amount) {
        // EN: scoped_lock atomically locks BOTH mutexes in a safe order
        //     Even if another thread calls transfer(to, from, ...) simultaneously
        // TR: scoped_lock her iki mutex'i de güvenli sırada atomik olarak kilitler
        std::scoped_lock lock(from.mtx_, to.mtx_);

        if (from.balance_ >= amount) {
            from.balance_ -= amount;
            to.balance_ += amount;
        }
    }

    double balance() const {
        std::lock_guard<std::mutex> lk(mtx_);
        return balance_;
    }
    const std::string& name() const { return name_; }

private:
    std::string name_;
    mutable std::mutex mtx_;
    double balance_;
};

```

```cpp
// ═══════════════════════════════════════════════════════════════════════
// PART 4: std::recursive_mutex — And Why to Avoid It
// ═══════════════════════════════════════════════════════════════════════

// EN: BAD DESIGN — needs recursive_mutex because public functions call each other
// TR: KÖTÜ TASARIM — genel fonksiyonlar birbirini çağırdığı için recursive_mutex gerekir
class SensorLoggerBad {
public:
    void log(const std::string& msg) {
        std::lock_guard<std::recursive_mutex> lk(mtx_);
        logs_.push_back(msg);
    }

    void log_with_count(const std::string& msg) {
        // EN: This calls log(), which also locks mtx_ → needs recursive_mutex!
        // TR: Bu log()'u çağırır, o da mtx_'i kilitler → recursive_mutex gerekir!
        std::lock_guard<std::recursive_mutex> lk(mtx_);
        log(msg + " (count: " + std::to_string(logs_.size()) + ")");
    }

    size_t count() const {
        std::lock_guard<std::recursive_mutex> lk(mtx_);
        return logs_.size();
    }

private:
    mutable std::recursive_mutex mtx_;
    std::vector<std::string> logs_;
};

// EN: GOOD DESIGN — refactored to avoid recursive_mutex
// TR: İYİ TASARIM — recursive_mutex kullanmadan yeniden yapılandırılmış
class SensorLoggerGood {
public:
    void log(const std::string& msg) {
        std::lock_guard<std::mutex> lk(mtx_);
        log_impl(msg);  // EN: Delegate to unlocked helper
    }

    void log_with_count(const std::string& msg) {
        std::lock_guard<std::mutex> lk(mtx_);
        log_impl(msg + " (count: " + std::to_string(logs_.size()) + ")");
    }

    size_t count() const {
        std::lock_guard<std::mutex> lk(mtx_);
        return logs_.size();
    }

private:
    // EN: Private helper — assumes lock is already held (no locking!)
    // TR: Private yardımcı — kilidin zaten alındığını varsayar (kilitleme yok!)
    void log_impl(const std::string& msg) {
        logs_.push_back(msg);
    }

    mutable std::mutex mtx_;
    std::vector<std::string> logs_;
};

// ═══════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 06 - Mutex & Lock Deep Dive\n";
    std::cout << "============================================\n\n";

    // ─── Demo 1: unique_lock Deferred & Try Locking ────────────────────
    {
        std::cout << "--- Demo 1: std::unique_lock ---\n";
        SensorBuffer buffer;

        // EN: Multiple threads updating sensor buffer
        // TR: Birden fazla thread sensör tamponunu günceller
        std::vector<std::thread> threads;
```

```
        for (int i = 0; i < 5; ++i) {
            threads.emplace_back([&buffer, i]() {
                for (int j = 0; j < 100; ++j) {
                    buffer.update(20.0 + static_cast<double>(i) + static_cast<double>(j) * 0.1);
                }
            });
        }
        for (auto& t : threads) t.join();

        std::cout << "    Buffer size: " << buffer.size() << " readings\n";
        std::cout << "    Average (Fahrenheit): " << buffer.average() << "\n";

        // EN: Demonstrate try_lock
        // TR: try_lock gösterimi
        int success = 0, fail = 0;
        std::vector<std::thread> try_threads;
        std::mutex count_mtx;
        for (int i = 0; i < 10; ++i) {
            try_threads.emplace_back([&]() {
                for (int j = 0; j < 50; ++j) {
                    bool ok = buffer.try_update(99.9);
                    std::lock_guard<std::mutex> lk(count_mtx);
                    if (ok) success++; else fail++;
                }
            });
        }
        for (auto& t : try_threads) t.join();
        std::cout << "    try_update: " << success << " success, "
                  << fail << " skipped (mutex busy)\n\n";
    }

    // ── Demo 2: shared_mutex Reader-Writer ──────────────────────────
    {
        std::cout << "--- Demo 2: std::shared_mutex (Reader-Writer) ---\n";
        ConfigStore config;

        // EN: Writer thread — updates config periodically
        // TR: Yazıcı thread — konfigürasyonu periyodik olarak günceller
        std::thread writer([&config]() {
            for (int i = 0; i < 10; ++i) {
                config.set("sensor_rate", std::to_string(100 + i * 10));
                std::this_thread::sleep_for(std::chrono::microseconds(100));
            }
        });

        // EN: Reader threads — read config frequently (much more than writes)
        // TR: Okuyucu thread'ler — config'i sık okur (yazmadan çok daha fazla)
        std::vector<std::thread> readers;
        for (int i = 0; i < 5; ++i) {
            readers.emplace_back([&config]() {
                for (int j = 0; j < 100; ++j) {
                    [[maybe_unused]] auto val = config.get("sensor_rate");
                    std::this_thread::sleep_for(std::chrono::microseconds(10));
                }
            });
        }

        writer.join();
        for (auto& t : readers) t.join();

        std::cout << "    Read ops:  " << config.read_ops() << "\n";
        std::cout << "    Write ops: " << config.write_ops() << "\n";
        std::cout << "    Final sensor_rate: " << config.get("sensor_rate") << "\n";
        std::cout << "    (shared_lock allows concurrent reads!)\n\n";
    }
```

```

    // ─── Demo 3: scoped_lock Deadlock-Free Transfer ──────────────────
    {
        std::cout << "--- Demo 3: std::scoped_lock (Deadlock-Free) ---\n";
        BankAccount alice("Alice", 1000.0);
        BankAccount bob("Bob", 1000.0);

        std::cout << "    Before: Alice=" << alice.balance()
                  << ", Bob=" << bob.balance() << "\n";

        // EN: Two threads doing opposite transfers — scoped_lock prevents deadlock
        // TR: İki thread zıt transfer yapar — scoped_lock deadlock'u önler
        std::thread t1([&]() {
            for (int i = 0; i < 1000; ++i) {
                BankAccount::transfer(alice, bob, 1.0);
            }
        });
        std::thread t2([&]() {
            for (int i = 0; i < 1000; ++i) {
                BankAccount::transfer(bob, alice, 1.0);
            }
        });
        t1.join();
        t2.join();

        std::cout << "    After:  Alice=" << alice.balance()
                  << ", Bob=" << bob.balance() << "\n";
        std::cout << "    Total preserved: " << (alice.balance() + bob.balance())
                  << " (should be 2000)\n\n";
    }

    // ─── Demo 4: recursive_mutex vs Refactored Design ────────────────
    {
        std::cout << "--- Demo 4: recursive_mutex vs Good Design ---\n";

        // EN: BAD: uses recursive_mutex
        // TR: KÖTÜ: recursive_mutex kullanır
        SensorLoggerBad bad_logger;
        bad_logger.log_with_count("Sensor reading");
        bad_logger.log("Direct log");
        std::cout << "    Bad logger (recursive_mutex): "
                  << bad_logger.count() << " entries\n";

        // EN: GOOD: refactored, uses regular mutex
        // TR: İYİ: yeniden yapılandırılmış, normal mutex kullanır
        SensorLoggerGood good_logger;
        good_logger.log_with_count("Sensor reading");
        good_logger.log("Direct log");
        std::cout << "    Good logger (regular mutex):  "
                  << good_logger.count() << " entries\n";

        std::cout << "\n    recursive_mutex: Same thread can lock multiple times\n";
        std::cout << "    BETTER: Extract unlocked helper + locking wrapper\n\n";
    }

    // ─── Demo 5: Lock Type Decision Table ────────────────────────────
    {
        std::cout << "--- Demo 5: Lock Type Decision Table ---\n";
        std::cout << "  ┌──────────────────────┬──────────────────────────────────┐\n";
        std::cout << "  │ Scenario             │ Use                              │\n";
        std::cout << "  ├──────────────────────┼──────────────────────────────────┤\n";
        std::cout << "  │ Simple critical sect.│ lock_guard (fastest, simplest)   │\n";
        std::cout << "  │ condition_variable   │ unique_lock (REQUIRED)           │\n";
        std::cout << "  │ Try without blocking │ unique_lock + try_to_lock        │\n";
        std::cout << "  │ Unlock early         │ unique_lock + lk.unlock()        │\n";
        std::cout << "  │ Lock 2+ mutexes      │ scoped_lock (deadlock-free)      │\n";
        std::cout << "  │ Many readers, 1 writer│ shared_mutex + shared_lock      │\n";
        std::cout << "  │ Recursive call chain │ Refactor! (or recursive_mutex)   │\n";
        std::cout << "  └──────────────────────┴──────────────────────────────────┘\n\n";
    }

    std::cout << "==========================================\n";
    std::cout << "   End of Mutex & Lock Deep Dive\n";
    std::cout << "==========================================\n";

    return 0;
}
```

► 10. Advanced Logging

module_06_industry_standards/10_advanced_logging.cpp

Konu: Advanced Logging — İleri Seviye Günlükleme

Teori / Theory

THEORY: Production Logging Requirements

Production logging goes far beyond std::cout. Requirements:
1. LOG LEVELS: Debug < Info < Warning < Error < Critical
2. THREAD SAFETY: Multiple threads logging simultaneously
3. LOG ROTATION: Don't fill the disk — rotate by size or time
4. STRUCTURED LOGGING: Machine-parseable (JSON) + human-readable
5. ASYNC LOGGING: Don't block hot paths for I/O
6. SOURCE LOCATION: File, line, function — automatically
7. SINKS: Console, file, syslog, network — configurable

Library	Description
spdlog	Fast, header-only, async. THE standard.
glog (Google)	LOG(INFO), CHECK(), VLOG(). Thread-safe.
Boost.Log	Full-featured, configurable. Heavy.
plog	Header-only, portable, tiny.
Custom (this file)	Zero-dependency learning implementation.

TEORİ: Üretim Günlükleme Gereksinimleri

Üretim günlükleme std::cout'un çok ötesindedir. Gereksinimler:
1. LOG SEVİYELERİ: Debug < Info < Warning < Error < Critical
2. THREAD GÜVENLİĞİ: Birden fazla thread eşzamanlı günlükleme
3. GÜNLÜK ROTASYONU: Diski doldurma — boyut/zamana göre döndür
4. YAPILANDIRILMIŞ LOGLAMA: Makine-ayrıştırılabilir (JSON) + insan-okunabilir
5. ASENKRON LOGLAMA: Sıcak yolları G/Ç için bloke etme
6. KAYNAK KONUMU: Dosya, satır, fonksiyon — otomatik olarak
7. ÇIKIŞ HEDEFLERİ: Konsol, dosya, syslog, ağ — yapılandırılabilir

Kütüphane	Açıklama
spdlog	Hızlı, header-only, asenkron. STANDART.
glog (Google)	LOG(INFO), CHECK(), VLOG(). Thread-güvenli.
Boost.Log	Tam özellikli, yapılandırılabilir. Ağır.
plog	Header-only, taşınabilir, küçük.
Custom (bu dosya)	Sıfır bağımlılık öğrenme implementasyonu.

THEORY: spdlog — The Industry Standard

spdlog quick reference (CMake: FetchContent or find_package):
// Create logger with rotating file sink auto logger = spdlog::rotating_logger_mt("ecu_log", // Logger name "logs/ecu.log", // File path 1024 * 1024 * 5, // 5 MB max size 3 // Keep 3 rotated files);
// Log with format args (fmtlib-style) logger->info("Sensor {} reading: {:.2f}C", sensor_id, temp); logger->warn("Temperature above threshold: {}C", temp); logger->error("Sensor {} communication failed", sensor_id);
// Set pattern spdlog::set_pattern("[%Y-%m-%d %H:%M:%S.%e] [%l] [thread %t] %v");
// Async logging (background thread for I/O) spdlog::init_thread_pool(8192, 1); // Queue size, 1 worker thread auto async_logger = spdlog::create_async<spdlog::sinks::rotating_file_sink_mt>("async_ecu", "logs/ecu.log", 5*1024*1024, 3);

TEORİ: spdlog — Endüstri Standardı

spdlog hızlı referans (CMake: FetchContent veya find_package):
// Dönen dosya sink'i ile logger oluştur auto logger = spdlog::rotating_logger_mt("ecu_log", // Logger adı "logs/ecu.log", // Dosya yolu 1024 * 1024 * 5, // 5 MB maks boyut 3 // 3 döndürülmüş dosya tut);
// Format argümanları ile logla (fmtlib-tarzı) logger->info("Sensor {} reading: {:.2f}C", sensor_id, temp); logger->warn("Temperature above threshold: {}C", temp); logger->error("Sensor {} communication failed", sensor_id);
// Desen ayarla spdlog::set_pattern("[%Y-%m-%d %H:%M:%S.%e] [%l] [thread %t] %v");
// Asenkron loglama (G/Ç için arka plan threadi) spdlog::init_thread_pool(8192, 1); // Kuyruk boyutu, 1 işçi thread auto async_logger = spdlog::create_async<spdlog::sinks::rotating_file_sink_mt>("async_ecu", "logs/ecu.log", 5*1024*1024, 3);

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <fstream>
#include <string>
#include <sstream>
#include <mutex>
#include <thread>
#include <queue>
#include <condition_variable>
#include <chrono>
#include <iomanip>
#include <vector>
#include <functional>
#include <filesystem>
#include <atomic>
#include <cstdint>

namespace fs = std::filesystem;

// ═══════════════════════════════════════════════════════════════════════
// PART 1: Log Level System
// ═══════════════════════════════════════════════════════════════════════

enum class LogLevel : uint8_t {
    Debug    = 0,
    Info     = 1,
    Warning  = 2,
    Error    = 3,
    Critical = 4
};
```

```cpp

const char* to_string(LogLevel level) {
    switch (level) {
        case LogLevel::Debug:    return "DEBUG";
        case LogLevel::Info:     return "INFO ";
        case LogLevel::Warning:  return "WARN ";
        case LogLevel::Error:    return "ERROR";
        case LogLevel::Critical: return "CRIT ";
        default:                 return "?????";
    }
}

// EN: ANSI color codes for console output
// TR: Konsol çıktısı için ANSI renk kodları
const char* level_color(LogLevel level) {
    switch (level) {
        case LogLevel::Debug:    return "\033[36m";   // Cyan
        case LogLevel::Info:     return "\033[32m";   // Green
        case LogLevel::Warning:  return "\033[33m";   // Yellow
        case LogLevel::Error:    return "\033[31m";   // Red
        case LogLevel::Critical: return "\033[1;31m"; // Bold Red
        default:                 return "\033[0m";
    }
}

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: Structured Log Entry
// ═══════════════════════════════════════════════════════════════════════════

struct LogEntry {
    LogLevel level;
    std::string message;
    std::string file;
    int line;
    std::thread::id thread_id;
    std::chrono::system_clock::time_point timestamp;

    // EN: Format as human-readable text
    // TR: İnsan okunabilir metin olarak biçimlendir
    std::string format_text() const {
        auto time = std::chrono::system_clock::to_time_t(timestamp);
        auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(
            timestamp.time_since_epoch()) % 1000;

        std::ostringstream oss;
        oss << std::put_time(std::localtime(&time), "%Y-%m-%d %H:%M:%S")
            << "." << std::setfill('0') << std::setw(3) << ms.count()
            << " [" << to_string(level) << "]"
            << " [" << thread_id << "]"
            << " " << message;
        return oss.str();
    }

    // EN: Format as JSON (structured logging)
    // TR: JSON olarak biçimlendir (yapılandırılmış günlükleme)
    std::string format_json() const {
        auto time = std::chrono::system_clock::to_time_t(timestamp);
        auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(
            timestamp.time_since_epoch()) % 1000;

        std::ostringstream oss;
        oss << "{\"timestamp\":\"";
        oss << std::put_time(std::localtime(&time), "%Y-%m-%dT%H:%M:%S");
        oss << "." << std::setfill('0') << std::setw(3) << ms.count() << "\"";
        oss << ",\"level\":\"" << to_string(level) << "\"";
        oss << ",\"thread\":\"" << thread_id << "\"";
        oss << ",\"file\":\"" << file << ":" << line << "\"";
        oss << ",\"message\":\"" << message << "\"}";
        return oss.str();
    }
};

```

```cpp
// ═══════════════════════════════════════════════════════════════════════════
// PART 3: Async Logger with Queue
// ═══════════════════════════════════════════════════════════════════════════

// EN: Async logger — logs are buffered in a queue, written by background thread
// TR: Asenkron logger — günlükler kuyrukta tamponlanır, arka plan thread'i yazar
class AsyncLogger {
public:
    explicit AsyncLogger(const std::string& name,
                         LogLevel min_level = LogLevel::Debug,
                         size_t max_queue = 4096)
        : name_(name), min_level_(min_level), max_queue_(max_queue),
          running_(true) {
        worker_ = std::thread(&AsyncLogger::worker_loop, this);
    }

    ~AsyncLogger() {
        {
            std::lock_guard<std::mutex> lk(mtx_);
            running_ = false;
        }
        cv_.notify_one();
        if (worker_.joinable()) worker_.join();

        // EN: Flush remaining entries
        // TR: Kalan girişleri boşalt
        while (!queue_.empty()) {
            process_entry(queue_.front());
            queue_.pop();
        }
    }

    void log(LogLevel level, const std::string& msg,
             const char* file = "", int line = 0) {
        if (level < min_level_) return;

        LogEntry entry{level, msg, file, line,
                       std::this_thread::get_id(),
                       std::chrono::system_clock::now()};

        std::lock_guard<std::mutex> lk(mtx_);
        if (queue_.size() < max_queue_) {
            queue_.push(std::move(entry));
            cv_.notify_one();
        } else {
            dropped_++;
        }
    }

    void set_console_output(bool enabled) { console_enabled_ = enabled; }
    void set_json_format(bool enabled) { json_format_ = enabled; }
    size_t dropped_count() const { return dropped_.load(); }
    size_t processed_count() const { return processed_.load(); }

private:
    void worker_loop() {
        while (true) {
            LogEntry entry;
            {
                std::unique_lock<std::mutex> lk(mtx_);
                cv_.wait(lk, [this] { return !queue_.empty() || !running_; });
                if (!running_ && queue_.empty()) break;
                entry = std::move(queue_.front());
                queue_.pop();
            }
            process_entry(entry);
        }
    }
```

```

    void process_entry(const LogEntry& entry) {
        if (console_enabled_) {
            std::cout << level_color(entry.level)
                      << entry.format_text()
                      << "\033[0m\n";
        }
        processed_++;
    }

    std::string name_;
    LogLevel min_level_;
    size_t max_queue_;
    bool console_enabled_ = true;
    bool json_format_ = false;
    std::atomic<bool> running_;
    std::atomic<size_t> dropped_{0};
    std::atomic<size_t> processed_{0};

    std::mutex mtx_;
    std::condition_variable cv_;
    std::queue<LogEntry> queue_;
    std::thread worker_;
};

// EN: Convenience macro with source location
// TR: Kaynak konumu ile kolaylık makrosu
#define LOG(logger, level, msg) \
    (logger).log(level, msg, __FILE__, __LINE__)

// ═══════════════════════════════════════════════════════════════════════
// PART 4: Log Rotation
// ═══════════════════════════════════════════════════════════════════════

// EN: Rotating file logger — rotates when file exceeds max size
// TR: Dönen dosya logger'ı — dosya maksimum boyutu aştığında döner
class RotatingFileLogger {
public:
    RotatingFileLogger(const std::string& base_path,
                       size_t max_size_bytes,
                       size_t max_files)
        : base_path_(base_path)
        , max_size_(max_size_bytes)
        , max_files_(max_files)
        , current_size_(0) {
        open_file();
    }

    ~RotatingFileLogger() {
        if (file_.is_open()) file_.close();
    }

    void write(const LogEntry& entry) {
        std::lock_guard<std::mutex> lk(mtx_);
        auto text = entry.format_text() + "\n";
        auto len = text.size();

        if (current_size_ + len > max_size_) {
            rotate();
        }

        file_ << text;
        file_.flush();
        current_size_ += len;
    }

    size_t rotation_count() const { return rotation_count_; }

private:
    void open_file() {
        if (file_.is_open()) file_.close();
        file_.open(base_path_, std::ios::app);
        if (fs::exists(base_path_)) {
            current_size_ = fs::file_size(base_path_);
        }
    }
```

```

    void rotate() {
        file_.close();
        // EN: Shift existing rotated files: .2 → .3, .1 → .2
        // TR: Mevcut dönmüş dosyaları kaydır: .2 → .3, .1 → .2
        for (size_t i = max_files_ - 1; i > 0; --i) {
            auto src = base_path_ + "." + std::to_string(i);
            auto dst = base_path_ + "." + std::to_string(i + 1);
            if (fs::exists(src)) {
                std::error_code ec;
                fs::rename(src, dst, ec);
            }
        }
        // EN: Current → .1
        // TR: Mevcut → .1
        if (fs::exists(base_path_)) {
            std::error_code ec;
            fs::rename(base_path_, base_path_ + ".1", ec);
        }
        // EN: Remove excess files
        // TR: Fazla dosyaları sil
        auto excess = base_path_ + "." + std::to_string(max_files_);
        if (fs::exists(excess)) {
            std::error_code ec;
            fs::remove(excess, ec);
        }
        current_size_ = 0;
        file_.open(base_path_, std::ios::out);
        rotation_count_++;
    }

    std::string base_path_;
    size_t max_size_;
    size_t max_files_;
    size_t current_size_;
    size_t rotation_count_ = 0;
    std::ofstream file_;
    std::mutex mtx_;
};

// ═══════════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 06 - Advanced Logging\n";
    std::cout << "============================================\n\n";

    // ─── Demo 1: Log Levels & Formatting ──────────────────────────────
    {
        std::cout << "--- Demo 1: Log Levels & Formatting ---\n";

        LogEntry entry{LogLevel::Info, "Engine started successfully",
                       "ecu_main.cpp", 42,
                       std::this_thread::get_id(),
                       std::chrono::system_clock::now()};

        std::cout << "  Text: " << entry.format_text() << "\n";
        std::cout << "  JSON: " << entry.format_json() << "\n\n";

        // EN: Show all log levels with colors
        // TR: Tüm günlük seviyelerini renklerle göster
        for (auto lvl : {LogLevel::Debug, LogLevel::Info, LogLevel::Warning,
                         LogLevel::Error, LogLevel::Critical}) {
            std::cout << "  " << level_color(lvl) << "[" << to_string(lvl) << "]"
                      << " Sample message at this level\033[0m\n";
        }
        std::cout << "\n";
    }
```

```cpp

    // ── Demo 2: Async Logger ──────────────────────────────────────
    {
        std::cout << "--- Demo 2: Async Logger ---\n";

        AsyncLogger logger("ecu_async", LogLevel::Info);

        // EN: Multiple threads logging simultaneously
        // TR: Birden fazla thread eşzamanlı günlükleme
        std::vector<std::thread> threads;
        for (int i = 0; i < 4; ++i) {
            threads.emplace_back([&logger, i]() {
                for (int j = 0; j < 5; ++j) {
                    LOG(logger, LogLevel::Info,
                        "Thread " + std::to_string(i) + " msg " + std::to_string(j));
                }
            });
        }
        for (auto& t : threads) t.join();

        // EN: Give worker thread time to process
        // TR: İşçi thread'e işleme zamanı ver
        std::this_thread::sleep_for(std::chrono::milliseconds(100));

        std::cout << "  Processed: " << logger.processed_count()
                  << ", Dropped: " << logger.dropped_count() << "\n\n";
    }

    // ── Demo 3: Log Rotation ──────────────────────────────────────
    {
        std::cout << "--- Demo 3: Log Rotation ---\n";

        auto log_path = fs::temp_directory_path() / "ecu_rotating.log";
        {
            // EN: Create rotating logger: 200 bytes per file, keep 3 rotated files
            // TR: Dönen logger: dosya başına 200 bayt, 3 dönmüş dosya tut
            RotatingFileLogger rot_logger(log_path.string(), 200, 3);

            for (int i = 0; i < 20; ++i) {
                LogEntry entry{LogLevel::Info,
                               "Sensor reading #" + std::to_string(i) + ": 25.3C",
                               "sensors.cpp", 100 + i,
                               std::this_thread::get_id(),
                               std::chrono::system_clock::now()};
                rot_logger.write(entry);
            }
            std::cout << "  Rotations: " << rot_logger.rotation_count() << "\n";
        }

        // EN: Show rotated files
        // TR: Dönmüş dosyaları göster
        for (int i = 0; i <= 3; ++i) {
            auto p = (i == 0) ? log_path : fs::path(log_path.string() + "." + std::to_string(i));
            if (fs::exists(p)) {
                std::cout << "  " << p.filename().string()
                          << ": " << fs::file_size(p) << " bytes\n";
            }
        }

        // EN: Cleanup
        // TR: Temizlik
        for (int i = 0; i <= 3; ++i) {
            auto p = (i == 0) ? log_path : fs::path(log_path.string() + "." + std::to_string(i));
            std::error_code ec;
            fs::remove(p, ec);
        }
        std::cout << "\n";
    }
```

```
383
384     // ─── Demo 4: spdlog Reference ──────────────────────────────────────
385     {
386         std::cout << "--- Demo 4: spdlog Quick Reference ---\n";
387         std::cout << "  CMakeLists.txt:\n";
388         std::cout << "    FetchContent_Declare(spdlog\n";
389         std::cout << "      GIT_REPOSITORY https://github.com/gabime/spdlog\n";
390         std::cout << "      GIT_TAG v1.13.0)\n";
391         std::cout << "    FetchContent_MakeAvailable(spdlog)\n";
392         std::cout << "    target_link_libraries(app PRIVATE spdlog::spdlog)\n\n";
393         std::cout << "  Usage:\n";
394         std::cout << "    #include <spdlog/spdlog.h>\n";
395         std::cout << "    #include <spdlog/sinks/rotating_file_sink.h>\n\n";
396         std::cout << "    auto logger = spdlog::rotating_logger_mt(\n";
397         std::cout << "        \"ecu\", \"logs/ecu.log\", 5*1024*1024, 3);\n";
398         std::cout << "    logger->info(\"Sensor {}: {:.2f}C\", id, temp);\n";
399         std::cout << "    logger->set_level(spdlog::level::debug);\n\n";
400         std::cout << "  Async:\n";
401         std::cout << "    spdlog::init_thread_pool(8192, 1);\n";
402         std::cout << "    auto async = spdlog::create_async<rotating_file_sink_mt>(\n";
403         std::cout << "        \"async_ecu\", \"logs/ecu.log\", 5*1024*1024, 3);\n\n";
404     }
405
406     // ─── Demo 5: Logging Best Practices ────────────────────────────────
407     {
408         std::cout << "--- Demo 5: Logging Best Practices ---\n";
409         std::cout << "  ┌──────────────────────────────┬──────────────────────────────────┐\n";
410         std::cout << "  │ Practice                     │ Why                              │\n";
411         std::cout << "  ├──────────────────────────────┼──────────────────────────────────┤\n";
412         std::cout << "  │ Use log levels properly      │ Filter noise, find issues fast   │\n";
413         std::cout << "  │ Include context (IDs, values) │ Debug without reproducing        │\n";
414         std::cout << "  │ Structured format (JSON)     │ Machine-parseable, searchable    │\n";
415         std::cout << "  │ Async logging in hot paths   │ Don't block business logic       │\n";
416         std::cout << "  │ Rotate by size + count       │ Don't fill disk                  │\n";
417         std::cout << "  │ Thread ID in log entries     │ Trace concurrent execution       │\n";
418         std::cout << "  │ Never log secrets/passwords  │ Security compliance              │\n";
419         std::cout << "  │ Rate-limit repeated errors   │ Don't flood on known issue       │\n";
420         std::cout << "  └──────────────────────────────┴──────────────────────────────────┘\n\n";
421     }
422
423     std::cout << "============================================\n";
424     std::cout << "   End of Advanced Logging\n";
425     std::cout << "============================================\n";
426
427     return 0;
428 }
```

► 11. Gtest Advanced

module_06_industry_standards/11_gtest_advanced.cpp

Konu: Advanced Google Test Patterns — İleri GTest Desenleri

Teori / Theory

THEORY: What This File Adds

The basic GTest simulation (03_unit_testing_gtest.cpp) covers TEST(), EXPECT_EQ/TRUE/FALSE/NEAR/THROW, and dependency injection with mocks. This file adds ADVANCED patterns that real-world projects require:

```
┌──────────────────────────┬──────────────────────────────────┐
│ Feature                  │ GTest API                        │
├──────────────────────────┼──────────────────────────────────┤
│ Test Fixtures (SetUp/Down)  │ TEST_F(Fixture, Name)            │
│ Parameterized Tests      │ TEST_P + INSTANTIATE_TEST_SUITE_P  │
│ Typed Tests (template)   │ TYPED_TEST / TYPED_TEST_SUITE    │
│ Death Tests              │ EXPECT_DEATH(stmt, regex)        │
│ GMock EXPECT_CALL        │ EXPECT_CALL(obj, Method()).Times(N)  │
│ GMock Matchers           │ Eq, Gt, Contains, HasSubstr, _   │
│ Code Coverage            │ gcov + lcov + genhtml            │
└──────────────────────────┴──────────────────────────────────┘
```

TEORİ: Bu Dosyanın Eklediği

Temel GTest simülasyonu (03_unit_testing_gtest.cpp) TEST(), EXPECT_EQ/TRUE/FALSE/NEAR/THROW ve bağımlılık enjeksiyonunu kapsar. Bu dosya gerçek projelerin GEREKTİRDİĞİ ileri desenleri ekler:

```
┌──────────────────────────┬──────────────────────────────────┐
│ Özellik                  │ GTest API                        │
├──────────────────────────┼──────────────────────────────────┤
│ Test Fixture (SetUp/Down)  │ TEST_F(Fixture, Name)            │
│ Parametrize Testler      │ TEST_P + INSTANTIATE_TEST_SUITE_P  │
│ Tipli Testler (şablon)   │ TYPED_TEST / TYPED_TEST_SUITE    │
│ Ölüm Testleri            │ EXPECT_DEATH(stmt, regex)        │
│ GMock EXPECT_CALL        │ EXPECT_CALL(obj, Method()).Times(N)  │
│ GMock Matcher'ı          │ Eq, Gt, Contains, HasSubstr, _   │
│ Kod Kapsama              │ gcov + lcov + genhtml            │
└──────────────────────────┴──────────────────────────────────┘
```

THEORY: Test Fixtures

A TEST FIXTURE is a class that provides shared setup/teardown logic for multiple tests. In real GTest:
class SensorTest : public ::testing::Test { protected: void SetUp() override { sensor_ = new Sensor(42); } void TearDown() override { delete sensor_; } Sensor* sensor_; }; TEST_F(SensorTest, ReadsTemperature) { EXPECT_GT(sensor_->read(), 0); }
SetUp() runs before EACH test, TearDown() after EACH test. The fixture is DESTROYED and RECONSTRUCTED between tests — each test gets a clean state. This prevents test interdependence.

TEORİ: Test Fikstürleri

TEST FIXTURE, birden fazla testin paylaştığı kurulum/temizleme mantığı sağlayan sınıftır. Gerçek GTest'te:
class SensorTest : public ::testing::Test { protected: void SetUp() override { sensor_ = new Sensor(42); } void TearDown() override { delete sensor_; } Sensor* sensor_; }; TEST_F(SensorTest, ReadsTemperature) { EXPECT_GT(sensor_->read(), 0); }
SetUp() HER testten önce, TearDown() HER testten sonra çalışır. Fixture testler arasında YIKILIP YENİDEN OLUŞTURULUR — her test temiz bir durumla başlar. Bu testler arası bağımlılığı önler.

THEORY: Parameterized Tests

Parameterized tests run the SAME test logic with DIFFERENT inputs. Instead of writing 10 nearly-identical TEST() cases, write ONE TEST_P() and feed it data:
class FizzBuzzTest : public ::testing::TestWithParam<std::pair<int,string>> {}; TEST_P(FizzBuzzTest, ProducesCorrectOutput) { auto [input, expected] = GetParam(); EXPECT_EQ(fizzBuzz(input), expected); } INSTANTIATE_TEST_SUITE_P(Defaults, FizzBuzzTest, ::testing::Values(std::make_pair(1, "1"), std::make_pair(3, "Fizz"), std::make_pair(5, "Buzz"), std::make_pair(15, "FizzBuzz")));

TEORİ: Parametrize Testler

Parametrize testler AYNI test mantığını FARKLI girdilerle çalıştırır. 10 neredeyse aynı TEST() yazmak yerine, TEK bır TEST_P() yazıp veri beslersin:
class FizzBuzzTest : public ::testing::TestWithParam<std::pair<int,string>> {}; TEST_P(FizzBuzzTest, ProducesCorrectOutput) { auto [input, expected] = GetParam(); EXPECT_EQ(fizzBuzz(input), expected); } INSTANTIATE_TEST_SUITE_P(Defaults, FizzBuzzTest, ::testing::Values(std::make_pair(1, "1"), std::make_pair(3, "Fizz"), std::make_pair(5, "Buzz"), std::make_pair(15, "FizzBuzz")));

THEORY: GMock & EXPECT_CALL

GMock goes beyond simple stubs. With EXPECT_CALL you can verify:
- HOW MANY times a method was called (Times(n), AtLeast(1), AtMost(3))
- WHAT arguments were passed (Eq(42), Gt(0), _)
- WHAT to return (Return(true), WillOnce/WillRepeatedly)
- CALL ORDER (InSequence, After)
class MockSensor : public ISensor { public: MOCK_METHOD(int, read, (), (override)); MOCK_METHOD(bool, calibrate, (int), (override)); }; TEST(Controller, ReadsAndCalibrates) { MockSensor mock; EXPECT_CALL(mock, read()).Times(2).WillRepeatedly(Return(25)); EXPECT_CALL(mock, calibrate(Gt(0))).WillOnce(Return(true)); Controller ctrl(mock); ctrl.loop(); }

TEORİ: GMock ve EXPECT_CALL

GMock basit stub'ların ötesine geçer. EXPECT_CALL ile doğrulama:
- KAÇ KEZ çağrıldı (Times(n), AtLeast(1), AtMost(3))
- HANGİ argümanlarla çağrıldı (Eq(42), Gt(0), _)
- NE döndürdü (Return(true), WillOnce/WillRepeatedly)
- ÇAĞRI SIRASI (InSequence, After)
class MockSensor : public ISensor { public: MOCK_METHOD(int, read, (), (override)); MOCK_METHOD(bool, calibrate, (int), (override)); }; TEST(Controller, ReadsAndCalibrates) { MockSensor mock; EXPECT_CALL(mock, read()).Times(2).WillRepeatedly(Return(25)); EXPECT_CALL(mock, calibrate(Gt(0))).WillOnce(Return(true)); Controller ctrl(mock); ctrl.loop(); }

THEORY: Code Coverage

Coverage measures which code lines/branches tests actually execute:
Compile with coverage instrumentation g++ --coverage -fprofile-arcs -ftest-coverage -o test test.cpp ./test gcov test.cpp # Generate .gcov files lcov --capture --directory . --output-file coverage.info genhtml coverage.info --output-directory coverage_html
Coverage types:
- Line coverage: which lines executed (aim ≥80%)
- Branch coverage: which if/else paths taken (aim ≥70%)
- Function coverage: which functions called (aim 100%)
CMake integration: target_compile_options(test PRIVATE --coverage) target_link_options(test PRIVATE --coverage)

TEORİ: Kod Kapsama

Kapsama, testlerin hangi kod satırlarını/dallarını çalıştırdığını ölçer:
Kapsama enstrümantasyonu ile derle g++ --coverage -fprofile-arcs -ftest-coverage -o test test.cpp ./test gcov test.cpp # .gcov dosyaları üret lcov --capture --directory . --output-file coverage.info genhtml coverage.info --output-directory coverage_html
Kapsama türleri:
- Satır kapsama: hangi satırlar çalıştı (hedef ≥%80)
- Dal kapsama: hangi if/else yolları alındı (hedef ≥%70)
- Fonksiyon kapsama: hangi fonksiyonlar çağrıldı (hedef %100)
CMake entegrasyonu: target_compile_options(test PRIVATE --coverage) target_link_options(test PRIVATE --coverage)

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <functional>
#include <sstream>
#include <cmath>
#include <memory>
#include <tuple>
#include <stdexcept>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: Extended MiniGTest Framework
// ═══════════════════════════════════════════════════════════════════════════

namespace MiniGTest2 {

struct TestResult {
    std::string suite;
    std::string name;
    bool passed = true;
    std::vector<std::string> failures;
};

std::vector<TestResult> results;
TestResult* current = nullptr;

void addResult(const std::string& suite, const std::string& name) {
    results.push_back({suite, name, true, {}});
    current = &results.back();
}

void fail(const std::string& msg) {
    if (current) {
        current->passed = false;
        current->failures.push_back(msg);
    }
}

void expectEq(int expected, int actual, const char* file, int line) {
    if (expected != actual) {
        std::ostringstream oss;
        oss << file << ":" << line << " EXPECT_EQ: expected=" << expected
            << " actual=" << actual;
        fail(oss.str());
    }
}

void expectStrEq(const std::string& expected, const std::string& actual,
                 const char* file, int line) {
    if (expected != actual) {
        std::ostringstream oss;
        oss << file << ":" << line << " EXPECT_STREQ: expected=\"" << expected
            << "\" actual=\"" << actual << "\"";
        fail(oss.str());
    }
}
```

```

void expectTrue(bool cond, const char* file, int line) {
    if (!cond) {
        std::ostringstream oss;
        oss << file << ":" << line << " EXPECT_TRUE failed";
        fail(oss.str());
    }
}

void expectGt(int a, int b, const char* file, int line) {
    if (!(a > b)) {
        std::ostringstream oss;
        oss << file << ":" << line << " EXPECT_GT: " << a << " not > " << b;
        fail(oss.str());
    }
}

void expectNear(double a, double b, double eps, const char* file, int line) {
    if (std::fabs(a - b) > eps) {
        std::ostringstream oss;
        oss << file << ":" << line << " EXPECT_NEAR: |" << a << " - " << b
            << "| > " << eps;
        fail(oss.str());
    }
}

void printSummary() {
    int passed = 0, failed = 0;
    for (const auto& r : results) {
        std::cout << (r.passed ? "  \033[32m[PASSED]\033[0m "
                               : "  \033[31m[FAILED]\033[0m ")
                  << r.suite << "." << r.name << "\n";
        if (!r.passed) {
            for (const auto& f : r.failures)
                std::cout << "            " << f << "\n";
            failed++;
        } else {
            passed++;
        }
    }
    std::cout << "\n  Summary: " << passed << " passed, " << failed
              << " failed, " << (passed + failed) << " total.\n";
}

} // namespace MiniGTest2

// Macros
#define EXPECT_EQ2(e,a)    MiniGTest2::expectEq(e,a,__FILE__,__LINE__)
#define EXPECT_STREQ2(e,a) MiniGTest2::expectStrEq(e,a,__FILE__,__LINE__)
#define EXPECT_TRUE2(c)    MiniGTest2::expectTrue(c,__FILE__,__LINE__)
#define EXPECT_GT2(a,b)    MiniGTest2::expectGt(a,b,__FILE__,__LINE__)
#define EXPECT_NEAR2(a,b,e) MiniGTest2::expectNear(a,b,e,__FILE__,__LINE__)

// ═══════════════════════════════════════════════════════════════════════
// PART 2: Production Code Under Test
// ═══════════════════════════════════════════════════════════════════════

// EN: Sensor simulator for fixture & parameterized test demos
// TR: Fixture ve parametrize test demoları için sensör simülatörü
class Sensor {
public:
    explicit Sensor(int id) : id_(id), value_(0), calibrated_(false) {}

    void calibrate(int offset) {
        offset_ = offset;
        calibrated_ = true;
    }

    int read() const {
        if (!calibrated_) return -1;
        return value_ + offset_;
    }

```

```
130     void setValue(int v) { value_ = v; }
131     int id() const { return id_; }
132     bool isCalibrated() const { return calibrated_; }
133
134 private:
135     int id_;
136     int value_ = 0;
137     int offset_ = 0;
138     bool calibrated_ = false;
139 };
140
141 // EN: FizzBuzz for parameterized test demo
142 // TR: Parametrize test demosu için FizzBuzz
143 std::string fizzBuzz(int n) {
144     if (n % 15 == 0) return "FizzBuzz";
145     if (n % 3 == 0) return "Fizz";
146     if (n % 5 == 0) return "Buzz";
147     return std::to_string(n);
148 }
149
150 // EN: Interface for GMock simulation
151 // TR: GMock simülasyonu için arayüz
152 class IDataSource {
153 public:
154     virtual ~IDataSource() = default;
155     virtual int fetchValue(int key) = 0;
156     virtual bool isConnected() const = 0;
157 };
158
159 // EN: Class under test — depends on IDataSource
160 // TR: Test edilen sınıf — IDataSource'a bağımlı
161 class DataProcessor {
162 public:
163     explicit DataProcessor(IDataSource& ds) : ds_(ds) {}
164
165     int processKey(int key) {
166         if (!ds_.isConnected()) return -1;
167         int val = ds_.fetchValue(key);
168         return val * 2;
169     }
170
171 private:
172     IDataSource& ds_;
173 };
174
175 // ==========================================================================
176 // MAIN
177 // ==========================================================================
178 int main() {
179     std::cout << "============================================\n";
180     std::cout << "   Module 06 - Advanced GTest Patterns\n";
181     std::cout << "============================================\n\n";
182
183     // ─── Demo 1: Test Fixture (SetUp / TearDown) ──────────────────────
184     {
185         std::cout << "--- Demo 1: Test Fixture (SetUp / TearDown) ---\n";
186
187         // EN: Simulate TEST_F — fixture class with SetUp/TearDown
188         // TR: TEST_F simülasyonu — SetUp/TearDown ile fixture sınıfı
189         struct SensorFixture {
190             std::unique_ptr<Sensor> sensor;
191
192             void SetUp() {
193                 sensor = std::make_unique<Sensor>(42);
194                 sensor->calibrate(5);
195                 sensor->setValue(100);
196                 // EN: Each test starts with calibrated sensor reading 105
197                 // TR: Her test kalibre edilmiş sensörle (105 okuması) başlar
198             }
199
200             void TearDown() {
201                 sensor.reset();
202                 // EN: Clean up resources after each test
203                 // TR: Her testten sonra kaynakları temizle
204             }
205         };
206
```

```
        // TEST_F(SensorFixture, ReadsCalibratedValue)
        {
            SensorFixture f;
            f.SetUp();
            MiniGTest2::addResult("SensorFixture", "ReadsCalibratedValue");
            EXPECT_EQ2(105, f.sensor->read());
            f.TearDown();
        }

        // TEST_F(SensorFixture, IsCalibrated)
        {
            SensorFixture f;
            f.SetUp();
            MiniGTest2::addResult("SensorFixture", "IsCalibrated");
            EXPECT_TRUE2(f.sensor->isCalibrated());
            f.TearDown();
        }

        // TEST_F(SensorFixture, IdMatches)
        {
            SensorFixture f;
            f.SetUp();
            MiniGTest2::addResult("SensorFixture", "IdMatches");
            EXPECT_EQ2(42, f.sensor->id());
            f.TearDown();
        }

        // TEST_F(SensorFixture, RecalibrateWithNewOffset)
        {
            SensorFixture f;
            f.SetUp();
            MiniGTest2::addResult("SensorFixture", "RecalibrateWithNewOffset");
            f.sensor->calibrate(10);
            EXPECT_EQ2(110, f.sensor->read());
            f.TearDown();
        }

        std::cout << "  (4 fixture tests — SetUp/TearDown per test)\n\n";

        std::cout << "  Real GTest equivalent:\n";
        std::cout << "    class SensorFixture : public ::testing::Test {\n";
        std::cout << "    protected:\n";
        std::cout << "        void SetUp() override { sensor_ = new Sensor(42); }\n";
        std::cout << "        void TearDown() override { delete sensor_; }\n";
        std::cout << "        Sensor* sensor_;\n";
        std::cout << "    };\n";
        std::cout << "    TEST_F(SensorFixture, Reads) { EXPECT_GT(sensor_->read(), 0); }\n\n";
    }

    // ─── Demo 2: Parameterized Tests (TEST_P) ──────────────────────────
    {
        std::cout << "--- Demo 2: Parameterized Tests (TEST_P) ---\n";

        // EN: Test data table — one TEST_P runs with each pair
        // TR: Test veri tablosu — her çift için tek TEST_P çalışır
        struct FizzBuzzParam {
            int input;
            std::string expected;
        };

        std::vector<FizzBuzzParam> params = {
            {1, "1"}, {2, "2"}, {3, "Fizz"}, {4, "4"},
            {5, "Buzz"}, {6, "Fizz"}, {10, "Buzz"},
            {15, "FizzBuzz"}, {30, "FizzBuzz"}, {7, "7"}
        };

        // Simulate TEST_P + INSTANTIATE_TEST_SUITE_P
        for (const auto& p : params) {
            std::string testName = "Input_" + std::to_string(p.input);
            MiniGTest2::addResult("FizzBuzzParam", testName);
            EXPECT_STREQ2(p.expected, fizzBuzz(p.input));
        }

```

```
        std::cout << "  (10 parameterized tests from data table)\n\n";

        std::cout << "  Real GTest equivalent:\n";
        std::cout << "    class FizzBuzzTest : public ::testing::TestWithParam<\n";
        std::cout << "        std::pair<int, std::string>> {};\n";
        std::cout << "    TEST_P(FizzBuzzTest, Produces) {\n";
        std::cout << "        auto [in, exp] = GetParam();\n";
        std::cout << "        EXPECT_EQ(fizzBuzz(in), exp);\n";
        std::cout << "    }\n";
        std::cout << "    INSTANTIATE_TEST_SUITE_P(Values, FizzBuzzTest,\n";
        std::cout << "        ::testing::Values(make_pair(3,\"Fizz\"), ...));\n\n";
    }

    // ─── Demo 3: Death Tests ──────────────────────────────────────────
    {
        std::cout << "--- Demo 3: Death Tests (EXPECT_DEATH) ---\n";

        // EN: Death tests verify that code crashes/aborts as expected.
        //     In real GTest: EXPECT_DEATH(func(), "regex matching stderr");
        //     We simulate by catching exceptions (since we can't fork here).
        // TR: Death testler kodun beklendiği gibi crash/abort ettiğini doğrular.

        // Simulate EXPECT_DEATH for assertion-like behavior
        auto expectDeath = [](std::function<void()> fn,
                              const std::string& testName) {
            MiniGTest2::addResult("DeathTest", testName);
            bool died = false;
            try {
                fn();
            } catch (...) {
                died = true;
            }
            if (!died) {
                MiniGTest2::fail("Expected death/exception but survived");
            }
        };

        // EN: Null pointer dereference simulation (throws instead of segfault)
        // TR: Null pointer başvurusu simülasyonu (segfault yerine exception fırlatır)
        expectDeath([]() {
            throw std::runtime_error("null dereference");
        }, "NullDereferenceAborts");

        // EN: Out of range access
        // TR: Aralık dışı erişim
        expectDeath([]() {
            std::vector<int> v = {1, 2, 3};
            (void)v.at(100);
        }, "OutOfRangeAborts");

        // EN: Division by zero throws
        // TR: Sıfıra bölme exception fırlatır
        expectDeath([]() {
            double b = 0.0;
            if (b == 0.0) throw std::logic_error("div by zero");
            volatile double r = 1.0 / b;
            (void)r;
        }, "DivisionByZeroAborts");

        std::cout << "  (3 death tests — all 'died' as expected)\n\n";

        std::cout << "  Real GTest equivalent:\n";
        std::cout << "    TEST(DeathTest, NullDeref) {\n";
        std::cout << "        EXPECT_DEATH(dereference(nullptr), \".*\");\n";
        std::cout << "    }\n";
        std::cout << "    // Runs in forked subprocess on Linux\n";
        std::cout << "    // Use EXPECT_EXIT for exit-code checks\n\n";
    }
```

```

    // ─── Demo 4: GMock EXPECT_CALL Simulation ──────────────────────────
    {
        std::cout << "--- Demo 4: GMock EXPECT_CALL Simulation ---\n";

        // EN: Mock that records calls, returns configurable values,
        //     and verifies expectations — simulating GMock behavior.
        // TR: Çağrıları kaydeden, yapılandırılabilir değerler döndüren
        //     ve beklentileri doğrulayan mock — GMock davranışını simüle eder.

        class MockDataSource : public IDataSource {
        public:
            // EN: Configure return values (WillOnce/WillRepeatedly equivalent)
            // TR: Dönüş değerlerini yapılandır (WillOnce/WillRepeatedly karşılığı)
            void whenFetchValue(int returnVal) { fetch_return_ = returnVal; }
            void whenIsConnected(bool val) { connected_ = val; }

            int fetchValue(int key) override {
                fetch_calls_.push_back(key);
                return fetch_return_;
            }

            bool isConnected() const override {
                connect_check_count_++;
                return connected_;
            }

            // EN: Verification methods (EXPECT_CALL equivalent)
            // TR: Doğrulama metotları (EXPECT_CALL karşılığı)
            size_t fetchCallCount() const { return fetch_calls_.size(); }
            const std::vector<int>& fetchCallArgs() const { return fetch_calls_; }
            size_t connectCheckCount() const { return connect_check_count_; }

        private:
            int fetch_return_ = 0;
            bool connected_ = true;
            std::vector<int> fetch_calls_;
            mutable size_t connect_check_count_ = 0;
        };

        // Test: connected, fetches twice
        {
            MockDataSource mock;
            mock.whenIsConnected(true);
            mock.whenFetchValue(42);

            DataProcessor proc(mock);
            MiniGTest2::addResult("GMockSim", "ConnectedProcessesTwice");

            int r1 = proc.processKey(1);
            int r2 = proc.processKey(2);

            EXPECT_EQ2(84, r1);       // 42 * 2
            EXPECT_EQ2(84, r2);       // 42 * 2
            // EXPECT_CALL(mock, fetchValue(_)).Times(2)
            EXPECT_EQ2(2, static_cast<int>(mock.fetchCallCount()));
            // EXPECT_CALL(mock, isConnected()).Times(2)
            EXPECT_EQ2(2, static_cast<int>(mock.connectCheckCount()));
        }

        // Test: disconnected, never fetches
        {
            MockDataSource mock;
            mock.whenIsConnected(false);

            DataProcessor proc(mock);
            MiniGTest2::addResult("GMockSim", "DisconnectedNeverFetches");

            int r = proc.processKey(1);
            EXPECT_EQ2(-1, r);
            // EXPECT_CALL(mock, fetchValue(_)).Times(0)
            EXPECT_EQ2(0, static_cast<int>(mock.fetchCallCount()));
        }

```

```
422         // Test: verify argument values
423         {
424             MockDataSource mock;
425             mock.whenIsConnected(true);
426             mock.whenFetchValue(10);
427 
428             DataProcessor proc(mock);
429             MiniGTest2::addResult("GMockSim", "VerifiesArgValues");
430 
431             proc.processKey(100);
432             proc.processKey(200);
433             proc.processKey(300);
434 
435             const auto& args = mock.fetchCallArgs();
436             EXPECT_EQ2(3, static_cast<int>(args.size()));
437             EXPECT_EQ2(100, args[0]);
438             EXPECT_EQ2(200, args[1]);
439             EXPECT_EQ2(300, args[2]);
440         }
441 
442         std::cout << "  (3 mock tests — call count, connectivity, arg verification)\n\n";
443 
444         std::cout << "  Real GMock equivalent:\n";
445         std::cout << "    class MockDataSource : public IDataSource {\n";
446         std::cout << "    public:\n";
447         std::cout << "        MOCK_METHOD(int, fetchValue, (int), (override));\n";
448         std::cout << "        MOCK_METHOD(bool, isConnected, (), (const, override));\n";
449         std::cout << "    };\n";
450         std::cout << "    TEST(Processor, CallsCorrectly) {\n";
451         std::cout << "        MockDataSource mock;\n";
452         std::cout << "        EXPECT_CALL(mock, isConnected()).WillRepeatedly(Return(true));\n";
453         std::cout << "        EXPECT_CALL(mock, fetchValue(100)).WillOnce(Return(42));\n";
454         std::cout << "    }\n\n";
455     }
456 
457     // ─── Demo 5: Code Coverage Reference ──────────────────────────────
458     {
459         std::cout << "--- Demo 5: Code Coverage & CI Pipeline ---\n";
460         std::cout << "  Compile with coverage:\n";
461         std::cout << "    g++ --coverage -fprofile-arcs -ftest-coverage \\\n";
462         std::cout << "        -o test_binary unit_tests.cpp\n\n";
463         std::cout << "  Run and analyze:\n";
464         std::cout << "    ./test_binary\n";
465         std::cout << "    gcov unit_tests.cpp        # .gcov text report\n";
466         std::cout << "    lcov --capture -d . -o cov.info\n";
467         std::cout << "    genhtml cov.info -o cov_html/  # HTML report\n\n";
468         std::cout << "  CMake integration:\n";
469         std::cout << "    option(ENABLE_COVERAGE \"Coverage\" OFF)\n";
470         std::cout << "    if(ENABLE_COVERAGE)\n";
471         std::cout << "        target_compile_options(tests PRIVATE --coverage)\n";
472         std::cout << "        target_link_options(tests PRIVATE --coverage)\n";
473         std::cout << "    endif()\n\n";
474         std::cout << "  CI/CD (GitHub Actions snippet):\n";
475         std::cout << "    - run: cmake -B build -DENABLE_COVERAGE=ON\n";
476         std::cout << "    - run: cmake --build build && cd build && ctest\n";
477         std::cout << "    - run: lcov --capture -d build -o cov.info\n";
478         std::cout << "    - run: genhtml cov.info -o coverage/\n";
479         std::cout << "    - uses: codecov/codecov-action@v3\n\n";
480 
481         std::cout << "  ┌──────────────────────┬──────────┬──────────────────────┐\n";
482         std::cout << "  │ Coverage Type        │ Target   │ Tool                 │\n";
483         std::cout << "  ├──────────────────────┼──────────┼──────────────────────┤\n";
484         std::cout << "  │ Line coverage        │ ≥ 80%    │ gcov, lcov           │\n";
485         std::cout << "  │ Branch coverage      │ ≥ 70%    │ lcov --rc branch=1   │\n";
486         std::cout << "  │ Function coverage    │ 100%     │ gcov -f              │\n";
487         std::cout << "  │ MC/DC (automotive)   │ 100%     │ BullseyeCoverage, Cantata│\n";
488         std::cout << "  └──────────────────────┴──────────┴──────────────────────┘\n\n";
489     }
```

```
490
491     // ─── Print Results ──────────────────────────────────────────────
492     std::cout << "==========================================\n";
493     std::cout << "   Test Results\n";
494     std::cout << "==========================================\n";
495     MiniGTest2::printSummary();
496
497     std::cout << "\n==========================================\n";
498     std::cout << "   End of Advanced GTest Patterns\n";
499     std::cout << "==========================================\n";
500
501     return 0;
502 }
```

Modül 7: Under the Hood — OS Derinlikleri

OS Derinlikleri

11 kaynak dosya · Standart: C++17

C++17

► 01. Vtable Memory Layout

module_07_under_the_hood/01_vtable_memory_layout.cpp

Konu: Under the Hood: V-Table (Virtual Table) Memory Cost Proof — Donanım Seviyesi: V-Table

Teori / Theory

THEORY: Polymorphism is NOT Free! (What is a Vptr?)

Polymorphism requires a mechanism to look up the correct function at RUNTIME. This is done using a "Virtual Table" (V-Table). When a class has AT LEAST ONE `virtual` function, the compiler silently inserts a completely hidden pointer into your class called `vptr` (Virtual Pointer). This pointer points to the V-Table. Since it is a pointer, it adds exactly 8 Bytes (on a 64-bit OS) to the size of EVERY SINGLE OBJECT you create!

TEORİ: Polymorphism Bedava Değildir!

Polimorfizm ("Sanal" metodlar), C++'ın doğru fonksiyonu ÇALIŞMA ZAMANINDA donanımsal Gerçek: 64-bit bir işletim sisteminde pointer'lar TAM 8 BAYT yer kaplar. Yere düşen bir yağmur damlası (Particle) sınıfında `virtual` kullanırsanız, milyonlarca damlanın her biri fazladan 8 bayt şişerek RAM'i bitirir!

CPPREF Depth: The Devirtualization Optimization

CppReference notes that modern compilers (GCC/Clang) try to perform "Devirtualization". If the compiler can PROVE at compile-time the exact type of the polymorphic object, it bypasses the V-Table lookup entirely and calls the function directly! (Massive speed boost). Using the `final` keyword helps the compiler prove this.

CPPREF Derinlik: Sanallaştırma Giderme Optimizasyonu (Sanal Çözme)

CppReference'a göre modern derleyiciler (GCC/Clang) "Sanal Çözme (Devirtualization)" adı verilen bir optimizasyon dener. Eğer derleyici, o nesnenin kimliğini derleme anında (Compile-Time) KESİN olarak kanıtlarsa, o yavaş V-Table aramasını tamamen çöpe atar ve doğrudan düz fonksiyon gibi çağırır! (Devasa FPS artışı). Sınıflara veya fonksiyonlara `final` kelimesini yazmak derleyiciye bu kanıtı sunar.

Kaynak Kod / Source Code

```cpp
#include <iostream>

// EN: 1. Plain Old Data Class. NO virtual keywords here.
// TR: Düz (Plain Old Data) Sınıf. İçinde virtual HİÇBİR ŞEY yok.
class CheapCharacter {
public:
  int health;  // 4 Bytes
  float speed; // 4 Bytes
  // Expected Total (Beklenen Toplam): 8 Bytes.
```

```cpp

  // EN: Regular function. Does NOT take up RAM space inside the object. Resolved at
  // Compile-Time.
  // TR: Sıradan fonksiyon. RAM'de (nesne içinde) yer KAPLAMAZ. Derleme zamanında çözülür.
  void jump() { std::cout << "Jump! (Zıpla!)" << std::endl; }
};

// EN: 2. Polymorphic Class (Contains a V-Table!).
// TR: Polimorfik (Sanal Tablolu) Sınıf. Sadece bir 'virtual' eklendiği için...
class ExpensiveCharacter {
public:
  int health;  // 4 Bytes
  float speed; // 4 Bytes
  // Expected Total: 8 Bytes...

  // EN: BUT! The moment we say "virtual", the compiler secretly adds a `void* vptr;` behind the
  // scenes! vptr = 8 Bytes (on 64-bit systems). Total: 8 + 8 = 16 Bytes! (100% RAM overhead!)
  //
  // TR: ANCAK! "virtual" dendiği an, derleyici gizli bir 'void* vptr;' satırını GİZLİCE EKLER!
  // vptr = 8 Bayt (64-bit sistemlerde). Toplam: 8 + 8 = 16 Bayt bekliyoruz! (%100 RAM israfı!)
  virtual void jump() {
    std::cout << "Flying Jump! (Uçarak Zıpla!)" << std::endl;
  }
};

int main() {
  std::cout << "=== MODULE 7: UNDER THE HOOD (V-TABLE PROOF) ===\n" << std::endl;

  std::cout << "[64-Bit OS Memory Estimation]" << std::endl;
  std::cout << "1 int + 1 float = Normally occupies 8 Bytes." << std::endl;

  std::cout << "\n----------------------------------------" << std::endl;
  std::cout << "1. CheapCharacter (NO Polymorphism): " << sizeof(CheapCharacter) << " Bytes." <<
      std::endl;
  std::cout << "-> Only Health (4) + Speed (4). Functions are free." << std::endl;
  std::cout << "-> (Sadece Health ve Speed. Fonksiyon bellekte bedavadır.)" << std::endl;

  std::cout << "\n----------------------------------------" << std::endl;
  std::cout << "2. ExpensiveCharacter (HAS Polymorphism): " << sizeof(ExpensiveCharacter) <<
      " Bytes!" << std::endl;
  std::cout << "-> WHY 16 BYTES? Because an 8-Byte 'vptr' (Virtual Pointer) "
              "was injected!"
            << std::endl;
  std::cout << "-> (NEDEN 16 BAYT OLDU? Çünkü 8 bayt 'vptr' pointer eklendi!)" << std::endl;

  std::cout << "\n[ARCHITECTURAL CONCLUSION / MİMARİ SONUÇ]:" << std::endl;
  std::cout << "EN: If you instantiate 1 Million sensor objects in an ECU "
            << std::endl;
  std::cout << "    DO NOT use 'virtual' in the base class. It wastes RAM and "
              "causes Cache Misses!"
            << std::endl;
  std::cout << "TR: Eğer bir ECU'da 1 milyon sensör nesnesi üreteceksiniz," << std::endl;
  std::cout << "    Merminin Base sınıfında 'virtual' KULLANMAYIN donanımı yorar." << std::endl;

  return 0;
}
```

► 02. Sigsegv And Raii

module_07_under_the_hood/02_sigsegv_and_raii.cpp

Konu: Under the Hood: Segmentation Fault (SIGSEGV) & Prevention via RAII — İşletim Sistemi

Teori / Theory

THEORY: Why do we get SIGSEGV (Crash)?

The OS allocates specific RAM pages for your C++ program. If you try to read or write to a memory address that the OS DID NOT GIVE YOU (like `nullptr`, or an array index that is out of bounds), the hardware's MMU (Memory Management Unit) sends an interrupt to the OS. The OS immediately ASSASSINATES your program with a signal called `SIGSEGV` (Segmentation Violation).

TEORİ: Neden SIGSEGV (Çöktü) Hatası Alırız?

İşletim Sistemi (OS), C++ programınıza sadece belirli bir RAM hafızası (Sayfa) verir. Eğer size ait OLMAYAN bir adrese (Örn: Donanımın adresine veya 'nullptr' içi boş adrese) dokunmaya, okumaya veya yazmaya çalışırsanız, işlemcinin Donanımsal Bellek Yöneticisi (MMU) bunu yakalar. İşletim sistemi anında programınıza suikast düzenler ve `SIGSEGV` (Segmentation Violation) sinyali yollayarak UYGULAMAYI ZORLA KAPATIR!

THEORY: RAII - How it saves lives!

RAII = "Resource Acquisition Is Initialization". Manual `new` and `delete` leads to 90% of human errors. If you forget `delete`, you get a Memory Leak. If you delete twice, you get a Double Free Crash. RAII dictates: "Entrust the Resource (RAM/File/Mutex) to a C++ Object. When the object goes out of scope {...}, its Destructor AUTOMATICALLY CLEANS IT UP." (e.g., std::unique_ptr, std::vector).

TEORİ: RAII Nedir ve Hayatımızı Nasıl Kurtarır?

RAII = "Kaynak Edinimi Başlatmadır". Manuel `new` ve `delete` kullanımı hataların %90'ına yol açar. `delete` yapmayı unutursanız Bellek Sızıntısı (Memory Leak) olur. İki kere silerseniz Çift Silme Hatası (Double Free/SIGSEGV) olur. RAII der ki: "RAM/Dosya/Kilit kaynağını C++ Objelerine emanet et. Nesne süslü parantez dışına çıkıp öldüğünde, KENDİ KENDİNİ OTOMATİK SİLSİN." (Örn: std::unique_ptr, std::vector).

CPPREF Depth: OS Core Dumps (Program Cesetleri)

When SIGSEGV happens, Linux doesn't just kill the program. It violently dumps the entire RAM state (Memory map, registers) into a massive file called a "Core Dump" (The corpse of your program). You can later load this Core Dump into GDB (`gdb ./app core`) to do an autopsy and see exactly which line of C++ triggered the assassination!

CPPREF Derinlik: İşletim Sistemi Bellek Dökümleri (Program Cesetleri)

SIGSEGV olduğunda Linux sadece programı kapatıp geçmez. Çöktüğü andaki tüm İşlemci yazmaçlarını (Registers) ve RAM haritasını "Core Dump" (Çekirdek Dökümü - Programın Cesedi) denen devasa bir dosyaya yazar. Daha sonra GDB hata ayıklayıcısına bu cesedi verip (`gdb ./uygulama core`) otopsi yapabilir, programınızın tam olarak hangi C++ satırından vurulduğunu görebilirsiniz!

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <memory>
#include <vector>

// EN: Dangerous legacy C-style function. WARNING: SIGSEGV RISK!
// TR: Tehlikeli (Eski C Tarzi) bir fonksiyon. DİKKAT: SIGSEGV RİSKİ!
void dangerousMemoryAccess() {
  int *garbageAddress = nullptr; // Address 0x0000000
  // EN: Suppress warning — demo only
  // TR: Uyarıyı bastır — yalnızca demo
  (void)garbageAddress;

  // EN: BURIED SIGSEGV TRAP: If you uncomment the line below, the program is ASSASSINATED
  // INSTANTLY!
  // TR: GÖMÜLÜ SIGSEGV TUZAĞI: Aşağıdaki satır açılırsa Program ANINDA İNFAZ EDİLİR!
  //
  // *garbageAddress = 55; // SIGSEGV (Segmentation Fault: 11)!!!

  // EN: Why? Because the OS did not grant us permission to write to Address 0.
  // TR: Neden? Cünkü İşletim Sistemi bize 0. (Yokluk) adresine yazmamız için HAK VERMEDİ.
}

// EN: Memory Leak Trap (What happens without RAII?)
// TR: Memory Leak Tuzağı. (RAII kullanılmazsa ne olur?)
void createAndForget() {
  // EN: Stole 2000 Bytes from OS!
  // TR: OS'den 2000 Bayt çaldık!
  int *array = new int[500];

  // EN: Imagine an error in the control loop
  // TR: Kontrol döngüsünde hata çıktı.
  bool randomErrorOccurred = true;

  if (randomErrorOccurred) {
    // EN: WARNING! We escape before reaching 'delete[] array'. RAM is lost forever (Leak).
    // TR: DİKKAT! 'delete[] array;' koduna asla gelemeden fonksiyondan kaçtık! RAM sızdı.
    return;
  }

  // EN: Would have freed it if no error happened...
  // TR: Hata olmasaydı silinecekti...
  delete[] array;
}

// EN: ------ THE ARCHITECTURE OF MODERN C++: RAII ------
// TR: ------ MÜKEMMEL C++ MİMARİSİ: RAII ------
class ActuatorController {
public:
  ActuatorController() {
    std::cout << "[RAII] Actuator initialized (Memory Acquired). / Aktüatör başlatıldı." <<
        std::endl;
  }
  ~ActuatorController() {
    std::cout << "[RAII] Actuator went out of scope! AUTO-RELEASED!!" <<
        "\n       (Aktüatör kapsam dışına çıktı! OTOMATIK SERBEST BIRAKILIYOR!)" << std::endl;
  }
};

int main() {
  std::cout << "=== MODULE 7: SIGSEGV (SEGMENTATION FAULT) & RAII ===\n" << std::endl;

  std::cout << "\n1. Risk of SIGSEGV (SIGSEGV Riski ve Neden Çıkar?)" << std::endl;
  std::cout << "EN: The OS won't let you write to unowned RAM. It shoots you "
               "with SIGSEGV."
            << std::endl;
  std::cout << "TR: İşletim Sistemi, hakkiniz olmayan RAM adresine yazmaniza "
               "anında SIGSEGV vurur."
            << std::endl;

  dangerousMemoryAccess();

  std::cout << "\n2. The RAII Solution (RAII Çözümü)" << std::endl;

```

```cpp
  // EN: Creating a Scope { ... }
  // TR: Süslü parantez ile (Scope / Kapsam) yaratıyoruz.
  {
    // EN: STEP 1: Smart Pointer is the backbone of C++14 RAII. NO NEW, NO DELETE.
    // TR: ADIM 1: "Smart Pointer", C++14'ün RAII silahidir. Asla `new` veya `delete`
    // kullanılmaz.
    std::unique_ptr<ActuatorController> safeObject = std::make_unique<ActuatorController>();

    // EN: STEP 2: std::vector is historically a RAII container itself.
    // TR: ADIM 2: std::vector doğuştan bir RAII'dir.
    std::vector<int> smartArray(500, 10); // 2000 Bytes managed automatically!

    std::cout << "  -> Inside the Scope, everything is fine... (Kapsam içindeyiz...)" << std::endl;

    bool suddenCodeCrash = true;
    if (suddenCodeCrash) {
      std::cout << "  -> ERROR BROKE THE FLOW! ESCAPING! (HATA İLE KOD "
                   "KIRILDI! KACIYORUZ!)"
                << std::endl;
      // EN: Without RAII, breaking here would mean severe leaking...
      // TR: Eğer RAII olmasaydı 'return/throw' yüzünden RAM sızacaktı...
    }

  // EN: THIS IS RAII! The moment this bracket closes, EVERYTHING is returned to OS
  // automatically!
  // TR: İşTE RAII! Parantez kapandığı an, İşletim Sisteminden alınan HDD/RAM her şey İADE
  // EDİLİR!
  }

  std::cout << "  -> Returned outside safely with 0 Bytes leaked. (Dışarıya "
               "Sifir Cöple dönüldü.)"
            << std::endl;

  return 0;
}
```

► 03. Fork Vs Thread Pool

module_07_under_the_hood/03_fork_vs_thread_pool.cpp

Konu: Under the Hood: Posix fork() vs std::thread & Thread Pooling — İşletim Sistemi: Process

Teori / Theory

THEORY: Process vs Thread

In Linux (POSIX), `fork()` creates a completely new PROCESS. It CLONES the entire memory space of the parent program. Processes do not share memory. If Process A changes a variable, Process B does not see it! (High security, HIGH cost, communication requires "Pipes/IPC").
`std::thread` creates a THREAD within the same Process. All threads SHARE the exact same RAM (Heap, Global vars). If Thread A changes a variable, Thread B sees it immediately! (Low cost, HIGH DANGER of Race Conditions -> Requires Mutex).

TEORİ: İşlem (Process) vs İş Parçacığı (Thread)

İşletim Sistemi Mimarisinde `fork()` (Linux'a özeldir) kocaman bir ağaçtır. Sen `fork` çağırdığında programın RAM'deki her şeyinin (3 GB bile olsa) İşletim Sistemi tarafından İKİNCİ BİR KOPYASI yaratılır! İki paralel evren oluşur. Birbirlerinin değişkenlerini GÖREMEZLER! (Güvenlik devasa, Bedel devasa).
`std::thread` (C++11) ise aynı evin içinde yeni bir odadır. Tüm Thread'ler AYNI SALONU (RAM / Heap Bellek) paylaşırlar. Bir Thread masadaki (RAM) değeri değiştirirse, diğeri anında görür! (Bedel yok denecek kadar azdır ama Race Condition (Yarış Durumu) çıkmaması için `std::mutex` Kilidi mecburidir).

THEORY: Thread Pooling Architecture

Constantly "Creating a Thread, assigning work, and Destroying the Thread" is slow even at the OS level. Professional automotive middleware (AUTOSAR) or servers (Node.js) create a "Thread Pool" on startup. They keep heavily sleeping workers alive. When there is work, they wake them up, no creation cost overhead!

TEORİ: İş Parçacığı Havuzu

Sürekli "Thread Yarat, iş ver, Thread'i Yık" döngüsü işletim sistemi seviyesinde bile yavaştır. Profesyonel otomotiv ara katmanları (AUTOSAR) veya sunucular (Node.js) uygulama açılışında bir "Thread Pool" (İş Parçacığı Havuzu) açar. 8 boş çekirdek için 8 tane "aç uyuyan işçi" Thread yaratılır. İş oldukça onlara yollanır, bitince uyumaya geri dönerler. ASLA baştan oluşturulmaz (Zero overhead).

CPPREF Depth: Context Switching Penalty

The OS Scheduler switches between running threads 1000 times a second. This is called a "Context Switch" and it is EXPENSIVE! A thread has to save all its CPU registers to RAM, and load the next thread's registers. If you spawn 1000 threads on an 8-core CPU, the OS spends more time Context Switching than actually executing your code (Thrashing)!

CPPREF Derinlik: Bağlam Değişimi Cezası

İşletim Sistemi Zamanlayıcısı (OS Scheduler) saniyede 1000 kere koşan Thread'ler arasında geçiş yapar. Buna "Context Switch (Bağlam Değiştirme)" denir ve ÇOK PAHALIDIR! Bir Thread tüm işlemci yazmaçlarını RAM'e kaydeder, diğeri de RAM'den işlemciye yüklenir. 8 Core bir CPU'da 1000 tane Thread açarsanız, donanım kodunuzu çalıştırmaktan çok sadece Thread'ler arası geçiş yaparak alev alır ve sistem kitlenir (Thrashing/Donma)! Thread Pool bu yüzden şarttır.

Kaynak Kod / Source Code

```cpp
#include <iostream>
// EN: For waiting on Linux processes
// TR: Linux süreçlerini beklemek için
#include <sys/wait.h>
#include <thread>
// EN: POSIX standard (for fork). Does NOT exist on Windows!
// TR: POSIX standardı (fork için). Windows'ta YOKTUR!
#include <unistd.h>

// EN: A variable sitting in the global heap space.
// TR: Global uzayda oturan ortak bir değişken.
int sharedVariable = 100;

void threadWorkerTask(int id) {
  // EN: All Threads SEE and MODIFY the EXACT SAME variable!
  // TR: Tüm İplikler (Threads) Aynı "sharedVariable"ı GÖRÜR VE DEĞİŞTİRİR!
  sharedVariable += 50;
  std::cout << "[THREAD " << id << "] I am a C++ Thread! Shared RAM value is now: " <<
      sharedVariable << std::endl;
}

int main() {
  std::cout << "=== MODULE 7: FORK (PROCESS) VS THREAD (IS PARCACIGI) ===\n" << std::endl;

  std::cout << "--- 1. C++11 std::thread (Lightweight, Shared Memory / Hafif "
               "ve Ortak) ---"
            << std::endl;
  sharedVariable = 100;

  std::thread t1(threadWorkerTask, 1);
  std::thread t2(threadWorkerTask, 2);

  t1.join();
```

```
34      t2.join();
35
36      std::cout << "[MAIN] Threads finished. Did the shared variable TRULY change "
37                   "globally?\n"
38                << "[MAIN] Degisken sistemde GERCEKTEN DEĞİŞTİ Mİ?: "
39                << sharedVariable << std::endl;
40      // EN: Output will be 200! (Shared Memory)
41      // TR: Çıktı 200 olacaktır! (Ortak Bellek)
42
43      std::cout << "\n--------------------------------------------------\n";
44
45      std::cout << "--- 2. Linux POSIX fork() (Heavy, Cloned Universes / Ağır, "
46                   "Paralel Evrenler) ---"
47                << std::endl;
48      sharedVariable = 100; // Reset / Sıfırlıyoruz.
49
50      // EN: Fork SPLITS the OS execution in two. It returns once as the Parent, once as the Child.
51      // TR: Fork işletim sistemini 2'ye yarar. İşletim sistemi aynı satırda ikiz kod döndürür.
52      pid_t processID = fork();
53
54      if (processID < 0) {
55        std::cerr << "Fork failed! (Fork fiyaskosu!)" << std::endl;
56        return 1;
57      } else if (processID == 0) {
58        // === CHILD PROCESS ZONE (THE PARALLEL UNIVERSE) / ÇOCUK BÖLGESİ ===
59        // EN: The child is born with an exact 1:1 photostatic copy of the RAM.
60        // TR: Çocuk, babasının anılarının (RAM'in) 1'e 1 fotokopisine sahip başlar!
61        sharedVariable += 500;
62        std::cout << "[FORK: CHILD PROCESS / ÇOCUK] My pID: " << getpid() <<
63            ". In my parallel universe, sharedVariable is (Benim "
64                     "evrenimde değer): "
65                  << sharedVariable << std::endl;
66        // EN: Child commits suicide.
67        // TR: Çocuk işini zorla bitirip "exit" yapar.
68        exit(0);
69
70      } else {
71        // === PARENT PROCESS ZONE (THE ORIGINAL UNIVERSE) / BABA (ASIL) BÖLGE ===
72        // EN: processID > 0 means it's the ID of the son that was just born.
73        // TR: processID > 0 demek, Oğlumun doğduğu pID numarasıdır.
74
75        // EN: MUST WAIT for the child to die, otherwise it becomes a Zombie Process.
76        // TR: Çocuğun paralel evrendeki işini bitirmesini Bekliyoruz (Zombi Proces olmasın diye).
77        wait(NULL);
78
79        std::cout << "[FORK: PARENT PROCESS / BABA] Child finished, I woke up." << std::endl;
80        std::cout << "[FORK: PARENT PROCESS / BABA] Let's see if the child's +500 "
81                     "affected MY UNIVERSE?"
82                  << std::endl;
83        std::cout << "[RESULT / SONUÇ] Parent's sharedVariable is (Babanın "
84                     "evrenindeki değer): "
85                  << sharedVariable << " !!!\n"
86                  << std::endl;
87
88        // EN: Output will STILL be 100! The OS blocked the memory via Virtual Memory walls.
89        // TR: Cıktı Hala 100 kalıcaktır! İşletim Sistemi aralarına Duvar (Virtual Memory
90        // Segregation) cektì!
91      }
92
93      std::cout << "\n[ARCHITECTURAL CONCLUSION / MİMARİ SONUÇ]:" << std::endl;
94      std::cout << "A-) 'fork()' (Process) is ultra safe but HEAVY. Data exchange "
95                   "(Pipes) is a nightmare."
96                << std::endl;
97      std::cout << "B-) 'std::thread' runs extremely FAST but causes Race Conditions." << std::endl;
98      std::cout << "C-) 'THREAD POOL' (Havuz) is the modern industry secret. Keep "
99                   "threads alive and waiting!"
100                << std::endl;
101
102     return 0;
103   }
```

► 04. Memory Model And Cache

module_07_under_the_hood/04_memory_model_and_cache.cpp

Konu: EN: CPU cache hierarchy, memory layout, false sharing, and cache-friendly design

Teori / Theory

THEORY: CPU Cache Hierarchy & Memory Model

Modern CPUs do not access main memory (RAM) directly for every read/write. Instead, a hierarchy of small, fast caches sits between the core and RAM:
1. L1 cache — per-core, ~32-64 KB, ~1-4 cycle latency.
2. L2 cache — per-core, ~256 KB-1 MB, ~10-20 cycles.
3. L3 cache — shared across cores, ~4-64 MB, ~30-50 cycles.
4. Main RAM — ~100-300 cycles latency. Data is transferred in cache lines, typically 64 bytes at a time.

TEORİ: CPU Önbellek Hiyerarşisi ve Bellek Modeli

Modern CPU'lar her okuma/yazma için doğrudan ana belleğe (RAM) erişmez. Bunun yerine, çekirdek ile RAM arasında küçük ve hızlı önbellekler bulunur:
1. L1 önbellek — çekirdek başına, ~32-64 KB, ~1-4 döngü gecikme.
2. L2 önbellek — çekirdek başına, ~256 KB-1 MB, ~10-20 döngü.
3. L3 önbellek — çekirdekler arası paylaşımlı, ~4-64 MB, ~30-50 döngü.
4. Ana RAM — ~100-300 döngü gecikme. Veriler önbellek satırları halinde, genellikle 64 bayt olarak aktarılır.

THEORY: Spatial & Temporal Locality

Spatial Locality — When data is accessed, the entire cache line (64B) is fetched. Contiguous memory (std::vector) is fast because the next element is already in cache. Scattered memory (std::list, pointer-chasing) causes cache misses.
Temporal Locality — Recently accessed data stays in cache and is fast to re-read. Hot loops over small data sets benefit enormously from temporal locality.

TEORİ: Uzamsal ve Zamansal Yerellik

Uzamsal Yerellik — Veriye erişildiğinde, tüm önbellek satırı (64B) getirilir. Bitişik bellek (std::vector) hızlıdır çünkü sonraki eleman zaten önbellektedir. Dağınık bellek (std::list, işaretçi takibi) önbellek kaçırmalarına neden olur.
Zamansal Yerellik — Son erişilen veri önbellekte kalır ve tekrar okumak hızlıdır. Küçük veri setleri üzerindeki sıcak döngüler zamansal yerellikten büyük fayda görür.

THEORY: False Sharing & SoA vs AoS

False Sharing — When two threads write to different variables on the SAME cache line, hardware forces constant invalidation ("cache line ping-pong"). Fix: pad each variable to its own 64-byte cache line with alignas(64).
SoA vs AoS — AoS (Array of Structs): struct { float x,y,z; } arr[N]; Nice OOP, but iterating one field loads unused fields into cache. SoA (Struct of Arrays): struct { float x[N], y[N], z[N]; }; Better cache use when iterating one field across all elements.
alignas keyword — alignas(N) forces a variable/struct to start at an N-byte boundary. Use alignas(64) to ensure struct occupies its own cache line.

TEORİ: Yanlış Paylaşım ve SoA vs AoS

Yanlış Paylaşım — İki iş parçacığı AYNI önbellek satırındaki farklı değişkenlere yazarsa, donanım sürekli geçersiz kılma yapar ("önbellek satırı ping-pong"). Çözüm: alignas(64) ile her değişkeni kendi 64 baytlık satırına hizalayın.
SoA vs AoS — AoS (Yapı Dizisi): struct { float x,y,z; } arr[N]; Güzel OOP, ama tek alan iterasyonunda kullanılmayan alanlar da önbelleğe yüklenir. SoA (Dizi Yapısı): struct { float x[N], y[N], z[N]; }; Tüm elemanlar üzerinde tek alan iterasyonunda daha iyi önbellek kullanımı.
alignas anahtar kelimesi — alignas(N) bir değişkeni/yapıyı N-bayt sınırına hizalamaya zorlar. Yapının kendi önbellek satırını kaplaması için alignas(64) kullanın.

CPPREF Depth: C++ Memory Model — Sequential Consistency and Acquire-Release Semantics

C++11 defines a formal memory model (§1.10) that governs how threads observe writes to shared variables. `memory_order_seq_cst` (the default for std::atomic operations) provides a total order visible to all threads — the simplest mental model but the most expensive on weakly-ordered architectures (ARM, POWER). `memory_order_acquire` / `memory_order_release` establish a happens-before relationship between a store-release and a load-acquire on the SAME atomic variable, without imposing a full fence. `memory_order_relaxed` guarantees only atomicity — no ordering at all — useful for pure counters. False sharing occurs when two independent atomics (or hot variables) reside on the same cache line (typically 64 bytes), forcing cores to bounce the line back and forth. C++17 exposes `std::hardware_destructive_interference_size` so you can pad/align to avoid it. Always profile before weakening memory orders; incorrect relaxed code creates data races that surface only under heavy load.

CPPREF Derinlik: C++ Bellek Modeli — Sıralı Tutarlılık ve Acquire-Release Semantiği

C++11 iş parçacıklarının paylaşılan değişkenlere yapılan yazmaları nasıl gözlemlediğini yöneten resmi bir bellek modeli tanımlar (§1.10). `memory_order_seq_cst` tüm iş parçacıklarına görünür toplam sıra sağlar — en basit model ama zayıf sıralı mimarilerde (ARM, POWER) en pahalısı. `memory_order_acquire/release` AYNI atomik değişken üzerinde store-release ile load-acquire arasında happens-before ilişkisi kurar; tam çit gerektirmez. `memory_order_relaxed` yalnızca atomiklik garanti eder, sıralama yoktur — saf sayaçlar için uygundur. Yanlış paylaşım (false sharing), iki bağımsız atomik aynı önbellek satırında (genellikle 64 bayt) bulunduğunda çekirdeklerin satırı sürekli birbirine göndermesine yol açar. C++17 bunu önlemek için `std::hardware_destructive_interference_size` sunar.

Kaynak Kod / Source Code

```cpp
#include <iostream>      // EN: Standard I/O
#include <vector>        // EN: Contiguous dynamic array
#include <list>          // EN: Doubly-linked list for comparison
#include <chrono>        // EN: High-resolution timing
#include <thread>        // EN: Thread support for false sharing demo
#include <numeric>       // EN: std::accumulate, std::iota
#include <algorithm>     // EN: std::generate
#include <cstdint>       // EN: Fixed-width integer types
#include <cstddef>       // EN: std::size_t, alignof

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 1: Data Structures — AoS vs SoA (Automotive Sensors)
// ═══════════════════════════════════════════════════════════════════════════

// EN: AoS — each sensor reading is one struct with all fields together
// TR: AoS — her sensör okuması tüm alanları bir arada tutan bir yapıdır
struct SensorAoS {
    float temp;
    float pressure;
    float voltage;
};
```

```cpp

// EN: SoA — separate arrays for each field, better cache line utilization
// TR: SoA — her alan için ayrı dizi, daha iyi önbellek satırı kullanımı
struct SensorSoA {
    std::vector<float> temp;
    std::vector<float> pressure;
    std::vector<float> voltage;

    // EN: Resize all channels at once
    // TR: Tüm kanalları aynı anda yeniden boyutlandır
    void resize(std::size_t n) {
        temp.resize(n);
        pressure.resize(n);
        voltage.resize(n);
    }
};

// ─── 1. False-Sharing Prevention with alignas ──────────────────────────────────

// EN: Without padding — both counters likely share a cache line
// TR: Dolgu olmadan — iki sayaç muhtemelen aynı önbellek satırını paylaşır
struct NaiveCounter {
    std::uint64_t counter_a = 0;
    std::uint64_t counter_b = 0;
};

// EN: With padding — each counter occupies its own 64-byte cache line
// TR: Dolgu ile — her sayaç kendi 64 baytlık önbellek satırını kaplar
struct alignas(64) PaddedCounter {
    std::uint64_t value = 0;
};

// ─── 2. ECU Configuration with Cache-Line Alignment ────────────────────────────

// EN: ECU config aligned to cache line boundary for DMA / shared-memory use
// TR: DMA / paylaşımlı bellek kullanımı için önbellek satırına hizalı ECU yapılandırması
struct alignas(64) EcuConfig {
    std::uint32_t ecu_id;
    float max_rpm;
    float idle_rpm;
    float fuel_trim;
    std::uint8_t  cylinder_count;
    bool          turbo_enabled;
    // EN: Padding is implicit — alignas ensures 64-byte total boundary
    // TR: Dolgu örtüktür — alignas 64 baytlık toplam sınırı garanti eder
};

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 2: Benchmark Utilities
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Simple RAII timer that prints elapsed time on destruction
// TR: Yıkımda geçen süreyi yazdıran basit RAII zamanlayıcı
class ScopedTimer {
public:
    explicit ScopedTimer(const char* label)
        : label_(label), start_(std::chrono::high_resolution_clock::now()) {}

    ~ScopedTimer() {
        auto end = std::chrono::high_resolution_clock::now();
        auto us  = std::chrono::duration_cast<std::chrono::microseconds>(end - start_).count();
        std::cout << "  [" << label_ << "] " << us << " us\n";
    }

    // EN: Non-copyable, non-movable
    // TR: Kopyalanamaz, taşınamaz
    ScopedTimer(const ScopedTimer&)            = delete;
    ScopedTimer& operator=(const ScopedTimer&) = delete;

private:
    const char* label_;
    std::chrono::high_resolution_clock::time_point start_;
};
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 3: Demonstration Functions
// ═══════════════════════════════════════════════════════════════════════════════

// ─── 3. AoS vs SoA Benchmark ─────────────────────────────────────────────────

// EN: Sum all temperature readings — AoS version (loads pressure & voltage too)
// TR: Tüm sıcaklık okumalarını topla — AoS versiyonu (basınç ve voltajı da yükler)
double sumTemperaturesAoS(const std::vector<SensorAoS>& sensors) {
    double total = 0.0;
    for (const auto& s : sensors) {
        total += static_cast<double>(s.temp);
    }
    return total;
}

// EN: Sum all temperature readings — SoA version (only touches temp array)
// TR: Tüm sıcaklık okumalarını topla — SoA versiyonu (sadece temp dizisine dokunur)
double sumTemperaturesSoA(const SensorSoA& sensors) {
    double total = 0.0;
    for (const auto& t : sensors.temp) {
        total += static_cast<double>(t);
    }
    return total;
}

// ─── 4. Vector vs List Iteration ─────────────────────────────────────────────

// EN: Iterate and sum elements — tests spatial locality difference
// TR: Elemanları dolaş ve topla — uzamsal yerellik farkını test eder
void benchmarkVectorVsList(std::size_t n) {
    std::cout << "\n--- Vector vs List Iteration (n=" << n << ") ---\n";

    // EN: Fill a vector with sequential values
    // TR: Vektörü sıralı değerlerle doldur
    std::vector<int> vec(n);
    std::iota(vec.begin(), vec.end(), 0);

    // EN: Fill a list with the same values
    // TR: Listeyi aynı değerlerle doldur
    std::list<int> lst(vec.begin(), vec.end());

    volatile long long sink = 0;

    {
        ScopedTimer t("vector sum");
        long long sum = 0;
        for (const auto& v : vec) { sum += v; }
        sink = sum;
    }

    {
        ScopedTimer t("list   sum");
        long long sum = 0;
        for (const auto& v : lst) { sum += v; }
        sink = sum;
    }

    (void)sink;
}

// ─── 5. False Sharing Demonstration ──────────────────────────────────────────

static constexpr std::size_t kIncrements = 50'000'000;

// EN: Two threads hammer the same NaiveCounter — false sharing likely
// TR: İki iş parçacığı aynı NaiveCounter'ı döver — yanlış paylaşım olası
void falseSharingNaive() {
    NaiveCounter nc;

    auto inc_a = [&]() {
        for (std::size_t i = 0; i < kIncrements; ++i) { ++nc.counter_a; }
    };
```

```
169     auto inc_b = [&]() {
170         for (std::size_t i = 0; i < kIncrements; ++i) { ++nc.counter_b; }
171     };
172
173     ScopedTimer t("naive  (shared line)");
174     std::thread t1(inc_a);
175     std::thread t2(inc_b);
176     t1.join();
177     t2.join();
178 }
179
180 // EN: Two threads on separate PaddedCounters — no false sharing
181 // TR: İki iş parçacığı ayrı PaddedCounter'larda — yanlış paylaşım yok
182 void falseSharingPadded() {
183     PaddedCounter pa;
184     PaddedCounter pb;
185
186     auto inc_a = [&]() {
187         for (std::size_t i = 0; i < kIncrements; ++i) { ++pa.value; }
188     };
189     auto inc_b = [&]() {
190         for (std::size_t i = 0; i < kIncrements; ++i) { ++pb.value; }
191     };
192
193     ScopedTimer t("padded (own line)   ");
194     std::thread t1(inc_a);
195     std::thread t2(inc_b);
196     t1.join();
197     t2.join();
198 }
199
200 // ─── 6. Master Demonstration ──────────────────────────────────────────────────
201
202 void demonstrateCacheEffects() {
203     constexpr std::size_t N = 2'000'000;
204
205     // EN: Prepare AoS data
206     // TR: AoS verisini hazırla
207     std::vector<SensorAoS> aos(N);
208     for (std::size_t i = 0; i < N; ++i) {
209         aos[i].temp     = static_cast<float>(i) * 0.01f;
210         aos[i].pressure = static_cast<float>(i) * 0.02f;
211         aos[i].voltage  = static_cast<float>(i) * 0.03f;
212     }
213
214     // EN: Prepare SoA data
215     // TR: SoA verisini hazırla
216     SensorSoA soa;
217     soa.resize(N);
218     for (std::size_t i = 0; i < N; ++i) {
219         soa.temp[i]     = static_cast<float>(i) * 0.01f;
220         soa.pressure[i] = static_cast<float>(i) * 0.02f;
221         soa.voltage[i]  = static_cast<float>(i) * 0.03f;
222     }
223
224     std::cout << "--- AoS vs SoA Temperature Sum (N=" << N << ") ---\n";
225
226     volatile double result = 0.0;
227
228     {
229         ScopedTimer t("AoS temp sum");
230         result = sumTemperaturesAoS(aos);
231     }
232     {
233         ScopedTimer t("SoA temp sum");
234         result = sumTemperaturesSoA(soa);
235     }
236
237     (void)result;
238
239     // EN: Vector vs list comparison
240     // TR: Vektör ile liste karşılaştırması
241     benchmarkVectorVsList(N);
242
243     // EN: False sharing comparison
244     // TR: Yanlış paylaşım karşılaştırması
245     std::cout << "\n--- False Sharing: Naive vs Padded ---\n";
```

```
    falseSharingNaive();
    falseSharingPadded();
}

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 4: main() — All Demos
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "═══════════════════════════════════════════\n";
    std::cout << " Module 07 — Memory Model & Cache Effects\n";
    std::cout << "═══════════════════════════════════════════\n\n";

    // ─── Demo 1: sizeof and alignment checks ─────────────────────────────────
    std::cout << "--- Demo 1: sizeof / alignof ---\n";

    std::cout << "  sizeof(SensorAoS)   = " << sizeof(SensorAoS)   << " bytes\n";
    std::cout << "  alignof(SensorAoS)  = " << alignof(SensorAoS)  << " bytes\n";
    std::cout << "  sizeof(NaiveCounter) = " << sizeof(NaiveCounter) << " bytes\n";
    std::cout << "  sizeof(PaddedCounter)= " << sizeof(PaddedCounter)<< " bytes\n";
    std::cout << "  alignof(PaddedCounter)=" << alignof(PaddedCounter)<< " bytes\n";
    std::cout << "  sizeof(EcuConfig)   = " << sizeof(EcuConfig)   << " bytes\n";
    std::cout << "  alignof(EcuConfig)  = " << alignof(EcuConfig)  << " bytes\n\n";

    // EN: Verify PaddedCounter is on its own cache line
    // TR: PaddedCounter'ın kendi önbellek satırında olduğunu doğrula
    static_assert(sizeof(PaddedCounter) >= 64,
                  "PaddedCounter must be at least 64 bytes for cache-line isolation");
    static_assert(alignof(PaddedCounter) == 64,
                  "PaddedCounter must be aligned to 64-byte boundary");

    // ─── Demo 2: AoS vs SoA benchmark ────────────────────────────────────────
    std::cout << "--- Demo 2 & 3 & 4: Cache Effect Benchmarks ---\n";
    demonstrateCacheEffects();

    // ─── Demo 5: alignas applied to ECU config struct ────────────────────────
    std::cout << "\n--- Demo 5: ECU Config Alignment ---\n";

    EcuConfig ecu{};
    ecu.ecu_id         = 0xA0B1;
    ecu.max_rpm        = 7500.0f;
    ecu.idle_rpm       = 800.0f;
    ecu.fuel_trim      = 1.02f;
    ecu.cylinder_count = 4;
    ecu.turbo_enabled  = true;

    // EN: Show that the struct address is 64-byte aligned
    // TR: Yapı adresinin 64 bayta hizalı olduğunu göster
    auto addr = reinterpret_cast<std::uintptr_t>(&ecu);
    std::cout << "  EcuConfig address  : 0x" << std::hex << addr << std::dec << "\n";
    std::cout << "  64-byte aligned?   : " << ((addr % 64 == 0) ? "YES" : "NO") << "\n";
    std::cout << "  ecu_id             : 0x" << std::hex << ecu.ecu_id << std::dec << "\n";
    std::cout << "  max_rpm            : " << ecu.max_rpm << "\n";
    std::cout << "  idle_rpm           : " << ecu.idle_rpm << "\n";
    std::cout << "  fuel_trim          : " << ecu.fuel_trim << "\n";
    std::cout << "  cylinder_count     : " << static_cast<int>(ecu.cylinder_count) << "\n";
    std::cout << "  turbo_enabled      : " << std::boolalpha << ecu.turbo_enabled << "\n";

    std::cout << "\n═══════════════════════════════════════════\n";
    std::cout << " All demos complete.\n";
    std::cout << "═══════════════════════════════════════════\n";

    return 0;
}
```

► 05. Compilation Pipeline

module_07_under_the_hood/05_compilation_pipeline.cpp

Konu: EN: C++ Compilation Pipeline — From Source to Executable

Teori / Theory

THEORY: C++ Compilation Pipeline

The C++ build process transforms human-readable source into machine code through four distinct stages. Understanding this pipeline is critical for diagnosing linker errors, managing build times, and writing robust firmware for automotive ECUs.

— Stage 1: PREPROCESSING — The preprocessor (cpp) runs before actual compilation. It handles:

• #include — textual inclusion of header files
• #define / #ifdef / #ifndef — macro expansion & conditional compilation
• #pragma — compiler-specific directives
• __FILE__, __LINE__, __func__ — predefined macros In automotive, conditional compilation selects HW-specific code paths (e.g., ECU_VARIANT_A vs ECU_VARIANT_B) from one codebase.

— Stage 2: COMPILATION — The compiler (cc1plus) translates each preprocessed translation unit (TU) into an object file (.o). Each .cpp file is ONE translation unit. Templates are instantiated per-TU, which can cause code bloat if many TUs instantiate the same template with the same types.

— Stage 3: LINKING — The linker (ld) merges object files and resolves symbol references. The One Definition Rule (ODR) states: every used entity must have exactly one definition across all TUs. Violations cause undefined behavior or linker errors. `extern` declares a symbol without defining it; the definition lives elsewhere. `inline` functions/variables are ODR-exempt — the linker picks one copy. C++17 inline variables enable header-only definitions of global state. `static` at file/namespace scope gives internal linkage — the symbol is invisible outside its TU.

— Stage 4: LOADING — The OS loader maps the executable into memory, resolves dynamic library symbols, sets up the stack, and transfers control to _start → main(). In automotive, ECU firmware is often flashed directly to ROM; the "loader" is the bootloader that copies code to execution addresses.

— Header-Only Libraries — Placing all code in headers is convenient (no separate .cpp to compile) but can increase build times and binary size (code bloat) since every TU that includes the header gets its own copy of non-inline definitions.

TEORİ: C++ Derleme Hattı

C++ derleme süreci, insan tarafından okunabilir kaynağı makine koduna dört ayrı aşamada dönüştürür. Bu hattı anlamak, bağlayıcı hatalarını teşhis etmek, derleme sürelerini yönetmek ve otomotiv ECU'ları için sağlam yazılım geliştirmek açısından kritiktir.

— Aşama 1: ÖN İŞLEME — Ön işlemci (cpp), gerçek derlemeden önce çalışır:

• #include — başlık dosyalarının metin olarak dahil edilmesi
• #define / #ifdef / #ifndef — makro açılımı ve koşullu derleme
• #pragma — derleyiciye özel direktifler
• __FILE__, __LINE__, __func__ — önceden tanımlı makrolar Otomotivde koşullu derleme, tek kod tabanından donanıma özel yolları seçer.

— Aşama 2: DERLEME — Derleyici, ön işlenmiş her çeviri birimini (TU) nesne dosyasına (.o) dönüştürür. Her .cpp dosyası BİR çeviri birimidir. Şablonlar her TU'da ayrı ayrı örneklenir; bu, aynı şablonun birden fazla TU'da kullanılması durumunda kod şişmesine yol açabilir.

— Aşama 3: BAĞLAMA — Bağlayıcı (ld), nesne dosyalarını birleştirir ve sembol referanslarını çözümler. Tek Tanım Kuralı (ODR): her kullanılan varlığın tüm TU'lar boyunca tam olarak bir tanımı olmalıdır. `extern` bir sembolü tanımlamadan bildirir. `inline` fonksiyonlar/değişkenler ODR'den muaftır — bağlayıcı tek bir kopya seçer. C++17 inline değişkenler, başlık-yalnız tanımları mümkün kılar. Dosya/ad-alanı kapsamında `static`, iç bağlantı sağlar.

— Aşama 4: YÜKLEME — İşletim sistemi yükleyicisi çalıştırılabilir dosyayı belleğe eşler, dinamik kütüphane sembollerini çözümler, yığını kurar ve kontrolü _start → main()'e aktarır. Otomotivde ECU yazılımı genellikle doğrudan ROM'a yazılır; "yükleyici" kodu çalıştırma adreslerine kopyalayan önyükleyicidir.

— Başlık-Yalnız Kütüphaneler — Tüm kodu başlık dosyalarına koymak pratiktir ancak derleme süresini ve ikili boyutunu artırabilir; başlığı dahil eden her TU, inline olmayan tanımların kendi kopyasını alır.

CPPREF Depth: One Definition Rule (ODR) — The Silent Killer of Large Projects

The One Definition Rule states every entity must have exactly one definition across all Translation Units. An ODR violation is undefined behavior — and no diagnostic is required! Inline functions must have identical token-for-token definitions in every TU; otherwise the linker silently picks one and discards the rest. Header-only libraries rely on the inline/template ODR exemption: the definition may appear in multiple TUs provided every copy is identical. In C++17, `inline` on a variable or function means "allow multiple definitions" — it does NOT mean "please inline this call". Link-Time Optimization (LTO) can detect some ODR violations that normal linking misses. C++20 Modules eliminate the textual inclusion model entirely, making ODR violations structurally impossible for module-exported entities.

CPPREF Derinlik: Tek Tanım Kuralı (ODR) — Büyük Projelerin Sessiz Katili

Tek Tanım Kuralı her varlığın tüm Çeviri Birimlerinde tam olarak bir tanımı olmasını gerektirir. ODR ihlali tanımsız davranıştır — ve derleyicinin uyarı vermesi zorunlu DEĞİLDİR! Inline fonksiyonlar her TU'da birebir aynı tanıma sahip olmalıdır; aksi halde bağlayıcı sessizce birini seçer. Başlık-yalnız kütüphaneler inline/template ODR muafiyetine dayanır. C++17'de `inline` bir değişken veya fonksiyonda "birden fazla tanıma izin ver" demektir — "bu çağrıyı satır içi yap" değil. LTO bazı ODR ihlallerini yakalayabilir. C++20 Modülleri dâhil etme modelini tamamen kaldırarak modül-dışa-aktarılan varlıklar için ODR ihlallerini yapısal olarak imkânsız kılar.

Kaynak Kod / Source Code

```
#include <iostream>   // EN: Standard I/O for console output
#include <string>     // EN: std::string for text handling
#include <sstream>    // EN: std::ostringstream for string building
#include <cstdint>    // EN: Fixed-width integer types (uint16_t, etc.)
#include <vector>     // EN: Dynamic array container
#include <array>      // EN: Fixed-size array container
#include <typeinfo>   // EN: typeid for RTTI demonstration

```

```
// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 1: INLINE CONSTEXPR AUTOMOTIVE CONFIGURATION
// ═══════════════════════════════════════════════════════════════════════════════

// EN: inline constexpr values can be defined in headers safely (ODR-exempt).
// TR: inline constexpr değerler, başlık dosyalarında güvenle tanımlanabilir (ODR muaf).

// ─── 1.1 Engine Parameters ─────────────────────────────────────────────────

inline constexpr uint16_t MAX_RPM           = 7500;
// EN: Maximum engine revolutions per minute
// TR: Maksimum motor devir sayısı (d/dk)

inline constexpr uint16_t IDLE_RPM          = 800;
// EN: Idle engine speed target
// TR: Rölanti devir hedefi

inline constexpr float    MIN_VOLTAGE       = 9.0f;
// EN: Minimum battery voltage before ECU triggers low-voltage mode
// TR: ECU'nun düşük gerilim moduna geçtiği minimum akü voltajı

inline constexpr float    MAX_COOLANT_TEMP  = 110.0f;
// EN: Coolant temperature threshold for overheating alarm (°C)
// TR: Aşırı ısınma alarmı için soğutma suyu sıcaklık eşiği (°C)

inline constexpr uint8_t  NUM_CYLINDERS     = 4;
// EN: Number of engine cylinders
// TR: Motor silindir sayısı

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 2: PREPROCESSOR — CONDITIONAL COMPILATION
// ═══════════════════════════════════════════════════════════════════════════════

// EN: ECU_DEBUG enables verbose diagnostics; undef for production builds.
// TR: ECU_DEBUG ayrıntılı tanılamayı etkinleştirir; üretim derlemelerinde kaldırılır.

#define ECU_DEBUG

#ifdef ECU_DEBUG
inline constexpr bool IS_DEBUG_BUILD = true;
#else
inline constexpr bool IS_DEBUG_BUILD = false;
#endif

// ─── 2.1 ECU_LOG Macro ─────────────────────────────────────────────────────

// EN: Logging macro using __FILE__, __LINE__, __func__ predefined macros.
// TR: __FILE__, __LINE__, __func__ önceden tanımlı makrolarını kullanan günlük makrosu.

#define ECU_LOG(msg)                                                          \
    do {                                                                      \
        if constexpr (IS_DEBUG_BUILD) {                                       \
            std::cout << "[ECU_LOG] " << __FILE__                             \
                      << ":" << __LINE__                                      \
                      << " (" << __func__ << ") — " << msg << "\n";          \
        }                                                                     \
    } while (0)

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 3: STATIC AT NAMESPACE SCOPE — INTERNAL LINKAGE
// ═══════════════════════════════════════════════════════════════════════════════

// EN: 'static' at namespace scope gives internal linkage — the variable exists only in this
// translation unit. Another TU can define its own copy without causing an ODR violation.
// TR: Ad-alanı kapsamında 'static', iç bağlantı sağlar — değişken yalnızca bu çeviri biriminde
// bulunur. Başka bir TU, ODR ihlali olmadan kendi kopyasını tanımlayabilir.

static int s_ecu_boot_count = 0;

// EN: Each TU that includes such a definition gets its own independent copy.
// TR: Böyle bir tanımı içeren her TU, kendi bağımsız kopyasını alır.

static void increment_boot_count() {
    ++s_ecu_boot_count;
}
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 4: TEMPLATE INSTANTIATION — CODE GENERATION & BLOAT
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Each unique template instantiation generates its own copy of code. If SensorBuffer<float,
// 64> and SensorBuffer<double, 64> are both used, the compiler emits TWO complete class
// implementations.
// TR: Her benzersiz şablon örneklemesi kendi kod kopyasını üretir. SensorBuffer<float, 64> ve
// SensorBuffer<double, 64> birlikte kullanılırsa, derleyici İKİ tam sınıf gerçeklemesi üretir.

template <typename T, std::size_t Capacity>
class SensorBuffer {
public:
    // EN: Push a sensor reading into the circular buffer.
    // TR: Dairesel tampona bir sensör okuma değeri ekler.
    void push(T value) {
        buffer_[write_idx_ % Capacity] = value;
        ++write_idx_;
    }

    // EN: Get the most recent reading.
    // TR: En son okuma değerini döndürür.
    T latest() const {
        return (write_idx_ > 0)
            ? buffer_[(write_idx_ - 1) % Capacity]
            : T{};
    }

    // EN: Return how many elements have been pushed (may exceed Capacity).
    // TR: Kaç öğe eklendiğini döndürür (Capacity'yi aşabilir).
    std::size_t total_writes() const { return write_idx_; }

    // EN: Return the fixed capacity of this buffer.
    // TR: Bu tamponun sabit kapasitesini döndürür.
    static constexpr std::size_t capacity() { return Capacity; }

private:
    std::array<T, Capacity> buffer_{};
    std::size_t write_idx_ = 0;
};

// ─── 4.1 Extern Declaration Example ──────────────────────────────────────────

// EN: 'extern' keyword declares a variable/function defined elsewhere. In a multi-TU project,
// the definition would be in exactly one .cpp file. Here we provide the definition immediately
// (single-TU demo).
// TR: 'extern' anahtar kelimesi, başka yerde tanımlanan bir değişkeni bildirir. Çok TU'lu bir
// projede tanım tam olarak bir .cpp dosyasında olur. Burada tanımı hemen sağlıyoruz (tek TU
// demosu).

extern const char* const ECU_FIRMWARE_VERSION;
const char* const ECU_FIRMWARE_VERSION = "v3.2.1-AUTOSAR";

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 5: MAIN — DEMONSTRATIONS
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "╔═══════════════════════════════════════════════════════╗\n";
    std::cout << "║  Module 07 — 05: C++ Compilation Pipeline             ║\n";
    std::cout << "║  EN: From Source Code to Running Binary               ║\n";
    std::cout << "║  TR: Kaynak Koddan Çalışan İkili Dosyaya              ║\n";
    std::cout << "╚═══════════════════════════════════════════════════════╝\n\n";

    // ─── Demo 1: Preprocessor Macros & Conditional Compilation ──────────────

    std::cout << "── Demo 1: Preprocessor & Conditional Compilation ──\n";
    // EN: Show whether ECU_DEBUG is active at compile time.
    // TR: Derleme zamanında ECU_DEBUG'ın aktif olup olmadığını gösterir.

    std::cout << "  IS_DEBUG_BUILD = " << std::boolalpha << IS_DEBUG_BUILD << "\n";

#ifdef ECU_DEBUG
    std::cout << "  [ECU_DEBUG] Diagnostic mode ACTIVE — extra checks enabled.\n";
```

```
    // EN: In production, #undef ECU_DEBUG removes this entire block.
    // TR: Üretimde, #undef ECU_DEBUG bu bloğun tamamını kaldırır.
#endif

    std::cout << "  Firmware: " << ECU_FIRMWARE_VERSION << "\n\n";

    // ─── Demo 2: ECU_LOG Macro with File/Line/Func Info ──────────────────────────

    std::cout << "── Demo 2: ECU_LOG Macro ──\n";
    // EN: ECU_LOG expands __FILE__, __LINE__, __func__ at the call site.
    // TR: ECU_LOG, çağrı noktasında __FILE__, __LINE__, __func__ değerlerini açar.

    ECU_LOG("Engine control module initialized");
    ECU_LOG("CAN bus ready — baud rate 500kbps");
    ECU_LOG("Sensor calibration complete");
    std::cout << "\n";

    // ─── Demo 3: inline constexpr Configuration Values ───────────────────────────

    std::cout << "── Demo 3: Inline Constexpr Configuration ──\n";
    // EN: These values are compile-time constants with no storage overhead.
    // TR: Bu değerler, depolama maliyeti olmayan derleme zamanı sabitleridir.

    std::cout << "  MAX_RPM          = " << MAX_RPM << " rpm\n";
    std::cout << "  IDLE_RPM         = " << IDLE_RPM << " rpm\n";
    std::cout << "  MIN_VOLTAGE      = " << MIN_VOLTAGE << " V\n";
    std::cout << "  MAX_COOLANT_TEMP = " << MAX_COOLANT_TEMP << " °C\n";
    std::cout << "  NUM_CYLINDERS    = " << static_cast<int>(NUM_CYLINDERS) << "\n";

    // EN: Simulate an RPM range check using the constants.
    // TR: Sabitler kullanarak bir RPM aralığı denetimi simüle eder.
    uint16_t current_rpm = 3200;
    if (current_rpm >= IDLE_RPM && current_rpm <= MAX_RPM) {
        std::cout << "  RPM " << current_rpm << " is within operating range.\n";
    }
    std::cout << "\n";

    // ─── Demo 4: Static Namespace-Scope Variable Behavior ────────────────────────

    std::cout << "── Demo 4: Static Internal Linkage ──\n";
    // EN: s_ecu_boot_count has internal linkage — it only exists in this TU. In a multi-file
    // project, each TU would have its own independent copy.
    // TR: s_ecu_boot_count iç bağlantıya sahiptir — yalnızca bu TU'da bulunur. Çok dosyalı bir
    // projede her TU kendi bağımsız kopyasına sahip olur.

    std::cout << "  Boot count before: " << s_ecu_boot_count << "\n";
    increment_boot_count();
    increment_boot_count();
    increment_boot_count();
    std::cout << "  Boot count after 3 increments: " << s_ecu_boot_count << "\n";
    std::cout << "  (Each TU would maintain its own static copy)\n\n";

    // ─── Demo 5: Template Instantiation & Code Bloat ─────────────────────────────

    std::cout << "── Demo 5: Template Instantiation Sizes ──\n";
    // EN: Different instantiations create separate code — observe sizes.
    // TR: Farklı örneklemeler ayrı kod oluşturur — boyutları gözlemleyin.

    SensorBuffer<float, 32>    rpm_buffer;
    SensorBuffer<double, 32>   temp_buffer;
    SensorBuffer<uint16_t, 64> voltage_buffer;
    SensorBuffer<float, 128>   accel_buffer;

```

```
    // EN: Push some sample automotive sensor data.
    // TR: Örnek otomotiv sensör verisi ekler.
    rpm_buffer.push(3200.0f);
    rpm_buffer.push(3450.5f);
    temp_buffer.push(92.3);
    voltage_buffer.push(1380);
    accel_buffer.push(0.98f);

    std::cout << "  sizeof SensorBuffer<float,   32>  = "
              << sizeof(rpm_buffer)     << " bytes\n";
    std::cout << "  sizeof SensorBuffer<double,  32>  = "
              << sizeof(temp_buffer)    << " bytes\n";
    std::cout << "  sizeof SensorBuffer<uint16_t,64>  = "
              << sizeof(voltage_buffer) << " bytes\n";
    std::cout << "  sizeof SensorBuffer<float,  128>  = "
              << sizeof(accel_buffer)   << " bytes\n";

    std::cout << "\n  Latest RPM reading:     " << rpm_buffer.latest() << " rpm\n";
    std::cout << "  Latest coolant temp:    " << temp_buffer.latest() << " °C\n";
    std::cout << "  Latest voltage (raw):   " << voltage_buffer.latest() << "\n";
    std::cout << "  Accel buffer capacity:  " << accel_buffer.capacity() << "\n";

    // EN: Each unique <T, N> combination generates a full class in the object file. With 4
    // instantiations, 4 copies of push(), latest(), etc. exist.
    // TR: Her benzersiz <T, N> kombinasyonu nesne dosyasında tam bir sınıf üretir. 4 örnekleme
    // ile push(), latest() vb. fonksiyonların 4 kopyası bulunur.

    std::cout << "\n  4 instantiations = 4× full class code in the object file.\n";
    std::cout << "  This is why header-only template libraries can cause bloat.\n";

    std::cout << "\n═══════════════════════════════════════════════════════════════\n";
    std::cout << "  Pipeline: .cpp → Preprocessor → Compiler → Linker → Binary\n";
    std::cout << "                        ↑ macros     ↑ .o files  ↑ executable\n";
    std::cout << "═══════════════════════════════════════════════════════════════\n";

    return 0;
}
```

► 06. Abi And Name Mangling

module_07_under_the_hood/06_abi_and_name_mangling.cpp

Konu: EN: ABI (Application Binary Interface) and C++ Name Mangling — binary contracts, symbol

Teori / Theory

THEORY: ABI and Name Mangling

ABI (Application Binary Interface) — The Binary-Level Contract An ABI defines how compiled code interacts at the binary level: calling conventions, register usage, struct layout, vtable ordering, and how function names become symbols. Linked object files must agree on the ABI or linker errors / crashes will follow.
Name Mangling — Encoding Overloaded Signatures The compiler "mangles" each function into a unique symbol. Under the Itanium ABI: foo(int)
→ _Z3fooi | foo(int,float) → _Z3fooif Each overload gets a distinct mangled name so the linker resolves the correct one.
extern "C" — Disabling Name Mangling for Interoperability `extern "C" { ... }` tells the compiler to use C linkage: no mangling. Essential for C interop, shared libraries (.so/.dll), and dlopen/dlsym plugin APIs.
Compiler ABI Incompatibility MSVC, GCC, and Clang each use different mangling schemes. Mixing compilers (or major versions) can cause linker failures. In automotive ECU software, the entire toolchain must be locked to ensure ABI consistency.
Tools: `c++filt` demangles symbols (echo _Z3fooi | c++filt → foo(int)). `nm` lists symbols in object files. Combine: nm mylib.so | c++filt
Shared Libraries: exposing a C ABI (extern "C") for .so files ensures any compiler version can link your functions — standard for UDS, bootloaders, CAN layers.

TEORİ: ABI ve İsim Karıştırma

ABI (Uygulama İkili Arayüzü) — İkili Düzey Sözleşme ABI, derlenmiş kodun ikili düzeyde nasıl etkileştiğini tanımlar: çağrı kuralları, yazmaç kullanımı, struct düzeni, vtable sıralaması ve fonksiyon adlarının sembollere dönüşümü. Bağlanan nesne dosyaları ABI üzerinde anlaşmalıdır.
İsim Karıştırma — Aşırı Yüklenmiş İmzaları Kodlama Derleyici her fonksiyonu benzersiz bir sembole "karıştırır". Itanium ABI altında: foo(int)
→ _Z3fooi | foo(int,float) → _Z3fooif Her aşırı yükleme farklı bir karıştırılmış isim alır; bağlayıcı doğru olanı çözümler.
extern "C" — Birlikte Çalışabilirlik İçin İsim Karıştırmayı Devre Dışı Bırakma `extern "C" { ... }` derleyiciye C bağlantısı kullanmasını söyler: karıştırma yok. C birlikte çalışma, paylaşımlı kütüphaneler ve dlopen/dlsym eklenti API'leri için gerekli.
Derleyici ABI Uyumsuzluğu MSVC, GCC ve Clang farklı karıştırma şemaları kullanır. Derleyici karışımı bağlayıcı hatalarına neden olabilir. Otomotiv ECU yazılımında araç zinciri kilitlenmelidir.
Araçlar: `c++filt` sembolleri çözümler. `nm` sembolleri listeler.
Paylaşımlı Kütüphaneler: .so dosyaları için C ABI sunmak, herhangi bir derleyici sürümünün fonksiyonlarınızı bağlamasını sağlar — UDS, önyükleyiciler, CAN katmanları.

CPPREF Depth: ABI Stability and the Itanium C++ ABI

Most Linux/Unix compilers (GCC, Clang) follow the Itanium C++ ABI, which standardises name mangling, vtable layout, exception handling, and RTTI representation. Name mangling encodes: namespace, enclosing class, function name, parameter types (including cv-qualifiers and references), and template arguments. `extern "C"` suppresses mangling so C code can link the symbol directly. ABI breaks happen when: struct/class layout changes (adding or reordering members), virtual function order changes, or the exception model changes. Compiling with `-fvisibility=hidden` and explicitly exporting only public symbols (`__attribute__((visibility("default")))`) produces smaller and faster shared libraries — fewer relocations, faster `dlopen()`. This is the standard practice for plugin architectures.

CPPREF Derinlik: ABI Kararlılığı ve Itanium C++ ABI'si

Çoğu Linux/Unix derleyicisi (GCC, Clang) isim bozma, vtable düzeni, istisna işleme ve RTTI temsilini standartlaştıran Itanium C++ ABI'sini takip eder. İsim bozma şunu kodlar: ad alanı, kapsayan sınıf, fonksiyon adı, parametre tipleri (cv-niteleyiciler ve referanslar dahil) ve şablon argümanları. `extern "C"` bozulmayı bastırır, böylece C kodu sembolü doğrudan bağlayabilir. ABI kırılmaları şu durumlarda olur: yapı/sınıf düzeni değiştiğinde, sanal fonksiyon sırası değiştiğinde veya istisna modeli değiştiğinde. `-fvisibility=hidden` ile derleyip yalnızca genel sembolleri açıkça dışa aktarmak daha küçük ve hızlı paylaşımlı kütüphaneler üretir — daha az yer değiştirme, daha hızlı `dlopen()`.

Kaynak Kod / Source Code

```cpp
#include <iostream>    // EN: Standard I/O for console output
#include <string>      // EN: std::string for sensor name lookups
#include <functional>  // EN: std::function for modern callback wrappers
#include <typeinfo>    // EN: typeid for RTTI-based type name inspection
#include <cstdio>      // EN: C-style printf for extern "C" callback demos
#include <array>       // EN: std::array for plugin dispatch table

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 1: C++ Overloaded Functions — Name Mangling in Action
// ═══════════════════════════════════════════════════════════════════════════

// EN: Three overloads share the name "readSensor" — the compiler mangles each into a unique
// symbol.
// TR: Üç aşırı yükleme "readSensor" adını paylaşır — derleyici her birini benzersiz bir sembole
// karıştırır.

// ── 1a. Read sensor by numeric ID ──────────────────────────────────────
int readSensor(int id) {
    // EN: Simulates reading raw ADC value from sensor channel
    // TR: Sensör kanalından ham ADC değeri okumayı simüle eder
    int raw_value = 1000 + id * 42;
    std::cout << "  [readSensor(int)] Channel " << id
              << " → raw value: " << raw_value << "\n";
    return raw_value;
}

// ── 1b. Read sensor by ID with calibration factor ──────────────────────
float readSensor(int id, float calibration) {
    // EN: Applies calibration offset to the sensor reading
    // TR: Sensör okumasına kalibrasyon ofseti uygular
    float calibrated = static_cast<float>(1000 + id * 42) * calibration;
    std::cout << "  [readSensor(int, float)] Channel " << id
              << " with cal=" << calibration
              << " → calibrated: " << calibrated << "\n";
    return calibrated;
}

// ── 1c. Read sensor by name (string lookup) ────────────────────────────
int readSensor(const std::string& name) {
    // EN: Simulates named sensor lookup (e.g., from config database)
    // TR: İsimli sensör aramasını simüle eder (ör. yapılandırma veritabanından)
    int resolved_value = static_cast<int>(name.length()) * 100 + 55;
    std::cout << "  [readSensor(string)] Sensor \"" << name
              << "\" → resolved value: " << resolved_value << "\n";
    return resolved_value;
}

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 2: extern "C" Block — C-Compatible ECU API Stubs
// ═══════════════════════════════════════════════════════════════════════════

// EN: extern "C" prevents name mangling — symbols are exported as plain C names.
// TR: extern "C" isim karıştırmayı engeller — semboller düz C adları olarak dışa aktarılır.

extern "C" {

int ecu_init() {
    // EN: Initialize ECU hardware abstraction layer (stub)
    // TR: ECU donanım soyutlama katmanını başlat (saplama)
    std::cout << "  [ecu_init] ECU initialized (firmware v2.4.1)\n";
    return 0;
}

void ecu_shutdown() {
    // EN: Graceful ECU shutdown — flush CAN buffers, store NVM
    // TR: Düzgün ECU kapatma — CAN arabelleklerini boşalt, NVM'yi kaydet
    std::cout << "  [ecu_shutdown] ECU shutdown complete\n";
}
```

```

int ecu_read_sensor(int channel) {
    // EN: Read raw sensor value via C-linkage API (no mangling)
    // TR: C bağlantılı API ile ham sensör değerini oku (karıştırma yok)
    int value = 2048 + channel * 17;
    std::cout << "  [ecu_read_sensor] Channel " << channel
              << " → value: " << value << "\n";
    return value;
}

} // extern "C"

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 3: C-Style Callback Pattern — Diagnostic Service
// ═══════════════════════════════════════════════════════════════════════════

// EN: Function pointer typedef for diagnostic callbacks (C-compatible signature)
// TR: Tanılama geri çağrıları için fonksiyon işaretçisi typedef'i (C uyumlu imza)
typedef void (*DiagCallback)(int code, const char* msg);

// EN: Global callback storage (simulates registration in a diagnostic manager)
// TR: Global geri çağrı deposu (tanı yöneticisinde kayıt işlemini simüle eder)
static DiagCallback g_diag_callback = nullptr;

void registerCallback(DiagCallback cb) {
    // EN: Register a diagnostic callback function pointer
    // TR: Bir tanılama geri çağrı fonksiyon işaretçisini kaydet
    g_diag_callback = cb;
    std::cout << "  [registerCallback] Callback registered at address: "
              << reinterpret_cast<void*>(cb) << "\n";
}

void triggerDiagnostic(int fault_code, const char* description) {
    // EN: Trigger the registered diagnostic callback if available
    // TR: Varsa kayıtlı tanılama geri çağrısını tetikle
    if (g_diag_callback) {
        g_diag_callback(fault_code, description);
    } else {
        std::cout << "  [triggerDiagnostic] No callback registered!\n";
    }
}

// EN: Sample callback implementations (would be in separate modules in production)
// TR: Örnek geri çağrı uygulamaları (üretimde ayrı modüllerde olurdu)
void onDtcDetected(int code, const char* msg) {
    std::printf("  [DTC Handler] Fault 0x%04X: %s\n", code, msg);
}

void onDtcLogged(int code, const char* msg) {
    std::printf("  [DTC Logger]  Fault 0x%04X stored to NVM: %s\n", code, msg);
}

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 4: Simulated Plugin System — Function Pointer Dispatch Table
// ═══════════════════════════════════════════════════════════════════════════

// EN: Plugin entry point signature — compatible with dlsym/GetProcAddress patterns
// TR: Eklenti giriş noktası imzası — dlsym/GetProcAddress kalıplarıyla uyumlu
using PluginFunc = int (*)(const char* command);

// EN: Simulated plugin implementations (in real systems, loaded from .so files)
// TR: Simüle edilmiş eklenti uygulamaları (gerçek sistemlerde .so dosyalarından yüklenir)
int pluginDiagnostics(const char* command) {
    std::cout << "  [Plugin:Diagnostics] Executing: " << command << "\n";
    return 0;
}

int pluginFlasher(const char* command) {
    std::cout << "  [Plugin:Flasher] Executing: " << command << "\n";
    return 0;
}

int pluginCalibration(const char* command) {
    std::cout << "  [Plugin:Calibration] Executing: " << command << "\n";
    return 0;
}
```

```

// EN: Plugin dispatch table — maps slot indices to function pointers
// TR: Eklenti dağıtım tablosu — yuva indekslerini fonksiyon işaretçilerine eşler
struct PluginEntry {
    const char*  name;
    PluginFunc   func;
};

// ═══════════════════════════════════════════════════════════════════════════════
// MAIN — Demonstration Scenarios
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "═══════════════════════════════════════════════════════\n";
    std::cout << " ABI & Name Mangling — Automotive ECU Examples\n";
    std::cout << "═══════════════════════════════════════════════════════\n\n";

    // ── Demo 1: Overloaded Function Calls ──────────────────────────────────
    // EN: Each call resolves to a different mangled symbol at link time
    // TR: Her çağrı bağlama zamanında farklı bir karıştırılmış sembole çözümlenir
    std::cout << "── Demo 1: Overloaded readSensor() calls ──\n";
    readSensor(3);
    readSensor(3, 1.05f);
    readSensor(std::string("throttle_position"));
    std::cout << "\n";

    // ── Demo 2: extern "C" Function Calls ──────────────────────────────────
    // EN: These symbols have no mangling — plain C names in the symbol table
    // TR: Bu sembollerin karıştırması yok — sembol tablosunda düz C adları
    std::cout << "── Demo 2: extern \"C\" ECU API calls ──\n";
    int init_status = ecu_init();
    std::cout << "  Init status: " << init_status << "\n";
    ecu_read_sensor(0);
    ecu_read_sensor(5);
    ecu_shutdown();
    std::cout << "\n";

    // ── Demo 3: Register and Trigger C-Style Callbacks ─────────────────────
    // EN: Function pointer callbacks — standard pattern for C-compatible diagnostic interfaces
    // TR: Fonksiyon işaretçisi geri çağrıları — C uyumlu tanı arayüzleri için standart kalıp
    std::cout << "── Demo 3: Diagnostic callback registration ──\n";
    registerCallback(onDtcDetected);
    triggerDiagnostic(0xC410, "Catalyst temperature above threshold");

    registerCallback(onDtcLogged);
    triggerDiagnostic(0xB112, "CAN bus off — ECU communication lost");
    std::cout << "\n";

    // ── Demo 4: Function Pointer Table (Simulated Plugin Dispatch) ─────────
    // EN: In real systems, these pointers come from dlsym() on loaded .so files.
    // TR: Gerçek sistemlerde bu işaretçiler yüklenen .so dosyalarındaki dlsym()'den gelir.
    std::cout << "── Demo 4: Plugin dispatch table ──\n";
    std::array<PluginEntry, 3> plugins = {{
        {"Diagnostics",  pluginDiagnostics},
        {"Flasher",      pluginFlasher},
        {"Calibration",  pluginCalibration}
    }};

    for (const auto& plugin : plugins) {
        std::cout << "  Dispatching to plugin [" << plugin.name << "]:\n";
        int result = plugin.func("self_test");
        std::cout << "    → Return code: " << result << "\n";
    }
    std::cout << "\n";
```

```

    // ─── Demo 5: typeid Names of Different Overloads ──────────────────────────────
    // EN: typeid reveals the compiler's internal type representation for each overload.
    // TR: typeid her aşırı yükleme için derleyicinin dahili tür temsilini gösterir.
    std::cout << "── Demo 5: typeid of overloaded function pointers ──\n";
    int   (*fp_int)(int)                    = readSensor;
    float (*fp_int_float)(int, float)       = readSensor;
    int   (*fp_string)(const std::string&)  = readSensor;

    std::cout << "  readSensor(int)          → typeid: "
              << typeid(fp_int).name() << "\n";
    std::cout << "  readSensor(int, float)   → typeid: "
              << typeid(fp_int_float).name() << "\n";
    std::cout << "  readSensor(string)       → typeid: "
              << typeid(fp_string).name() << "\n";
    std::cout << "\n  Addresses (all distinct due to mangling):\n";
    std::cout << "    readSensor(int):        " << reinterpret_cast<void*>(fp_int) << "\n";
    std::cout << "    readSensor(int, float): " << reinterpret_cast<void*>(fp_int_float) << "\n";
    std::cout << "    readSensor(string):     " << reinterpret_cast<void*>(fp_string) << "\n";

    std::cout << "\n═══════════════════════════════════════════════════════\n";
    std::cout << " Tip: compile and inspect mangled symbols with:\n";
    std::cout << "   nm ./06_abi_and_name_mangling | grep readSensor\n";
    std::cout << "   nm ./06_abi_and_name_mangling | grep ecu_ \n";
    std:cout << "   nm ./06_abi_and_name_mangling | c++filt | grep readSensor\n";
    std::cout << "═══════════════════════════════════════════════════════\n";

    return 0;
}
```

► 07. Inline Assembly

module_07_under_the_hood/07_inline_assembly.cpp

Konu: Inline Assembly: Doğrudan CPU ile Konuşma — Talking Directly to the CPU

Teori / Theory

THEORY: What Is Inline Assembly?

Inline assembly lets you embed raw CPU instructions inside C++ code. GCC/Clang use the extended asm syntax:
asm volatile("instruction" : output operands // =r(var) → result goes to var : input operands // "r"(var) → var loaded into register : clobber list); // registers modified by the asm
Keywords:
- `asm` or `__asm__` → introduces assembly block
- `volatile` → tells compiler: "don't optimize this away!" Without volatile, compiler may DELETE the asm if it thinks the output is unused.
- Constraints: "r" = any register, "m" = memory, "i" = immediate

TEORİ: Inline Assembly Nedir?

Inline assembly, C++ kodu içine doğrudan CPU talimatları gömmenizi sağlar. GCC/Clang genişletilmiş asm sözdizimini kullanır:
asm volatile("talimat" : çıkış operandlar // =r(var) → sonuç var'a gider : giriş operandlar // "r"(var) → var yazmaça yüklenir : bozulan liste); // asm tarafından değiştirilen yazmaçlar
Anahtar sözcükler:
- `asm` veya `__asm__` → assembly bloğunu başlatır
- `volatile` → derleyiciye "bunu optimize etme!" der. volatile olmadan derleyici çıkış kullanılmıyorsa asm'yi SİLEBİLİR.
- Kısıtlamalar: "r" = herhangi yazmaç, "m" = bellek, "i" = anlık değer

THEORY: Why Usually AVOID Inline Assembly

Inline assembly should be a LAST RESORT. Reasons to avoid:
1. NON-PORTABLE: x86 asm won't work on ARM (automotive ECUs often use ARM!)
2. OPTIMIZER BLIND: Compiler can't optimize across asm boundaries
3. ERROR-PRONE: Wrong constraints → silent data corruption
4. MAINTENANCE NIGHTMARE: Few developers can read/write assembly
5. COMPILER IS USUALLY BETTER: Modern compilers (GCC -O2/O3) generate better machine code than most humans can write
Alternatives (prefer these):
- Compiler intrinsics: __builtin_popcount, __builtin_clz, __builtin_expect
- SIMD intrinsics: _mm256_add_ps (portable across compilers)
- Compiler hints: [[likely]], [[unlikely]], __attribute__((hot/cold))
When inline asm IS justified:
- Accessing CPU features with no intrinsic (CPUID, RDTSC, MSR registers)
- Implementing OS kernel primitives (context switch, interrupt handling)
- Hardware-specific device drivers
- Cryptographic constant-time operations (prevent compiler optimization)

TEORİ: Neden Genellikle KAÇINILMALI

Inline assembly SON ÇARE olmalıdır. Kaçınılmasının nedenleri:
1. TAŞINABİLİR DEĞİL: x86 asm ARM'de çalışmaz (otomotiv ECU'ları genelde ARM!)
2. OPTİMİZATÖR KÖR: Derleyici asm sınırları arasında optimize edemez
3. HATAYA AÇIK: Yanlış kısıtlamalar → sessiz veri bozulması
4. BAKIM KABUSSU: Az geliştirici assembly okuyup yazabilir
5. DERLEYİCİ GENELDE DAHA İYİ: Modern derleyiciler (GCC -O2/O3) çoğu insandan daha iyi makine kodu üretir
Alternatifler (bunları tercih et):
- Derleyici intrinsikleri: __builtin_popcount, __builtin_clz, __builtin_expect
- SIMD intrinsikleri: _mm256_add_ps (derleyiciler arası taşınabilir)
- Derleyici ipuçları: [[likely]], [[unlikely]], __attribute__((hot/cold))
Inline asm'in HAKLI olduğu durumlar:
- İntrinsik olmayan CPU özelliklerine erişim (CPUID, RDTSC, MSR yazmaçları)
- OS kernel ilkellerini uygulama (bağlam değiştirme, kesme yönetimi)
- Donanıma özel aygıt sürücüleri
- Kriptografik sabit-zamanlı işlemler (derleyici optimizasyonunu önleme)

THEORY: x86-64 Register Quick Reference

x86-64 has 16 general-purpose 64-bit registers:

Register	Convention
RAX	Return value, accumulator
RBX	Callee-saved (preserved across calls)
RCX	4th argument (Windows) / counter
RDX	3rd argument (Linux) / high part of multiply
RSI	2nd argument (Linux) / source index
RDI	1st argument (Linux) / destination index
RSP	Stack pointer (DO NOT touch in inline asm!)
RBP	Base/frame pointer
R8-R15	Additional registers (R8-R11 caller-saved, R12-R15 callee)

Sub-registers: RAX(64) → EAX(32) → AX(16) → AL(8)/AH(8)

TEORİ: x86-64 Yazmaç Hızlı Başvuru

x86-64'te 16 genel amaçlı 64-bit yazmaç vardır:

```
┌─────────┬──────────────────────────────────────────────────────┐
│ Yazmaç  │ Gelenek                                              │
├─────────┼──────────────────────────────────────────────────────┤
│ RAX     │ Dönüş değeri, toplam yazmaç                          │
│ RBX     │ Callee-saved (çağrılar boyunca korunur)              │
│ RCX     │ 4. argüman (Windows) / sayaç                         │
│ RDX     │ 3. argüman (Linux) / çarpmanın üst kısmı             │
│ RSI     │ 2. argüman (Linux) / kaynak indeksi                  │
│ RDI     │ 1. argüman (Linux) / hedef indeksi                   │
│ RSP     │ Yığın işaretçisi (inline asm'de DOKUNMA!)            │
│ RBP     │ Taban/çerçeve işaretçisi                             │
│ R8-R15  │ Ek yazmaçlar (R8-R11 caller-saved, R12-R15 callee-saved)  │
└─────────┴──────────────────────────────────────────────────────┘
```

Alt-yazmaçlar: RAX(64) → EAX(32) → AX(16) → AL(8)/AH(8)

→ Ref: https://gcc.gnu.org/onlinedocs/gcc/Extended-Asm.html
→ Ref: https://www.felixcloutier.com/x86/ — x86 instruction set reference

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <cstring>
#include <chrono>
#include <string>
#include <array>
#include <vector>

// ═══════════════════════════════════════════════════════════════════════
// PART 1: Basic Inline Assembly — Arithmetic
// ═══════════════════════════════════════════════════════════════════════

/**
 * EN: The simplest possible inline asm: add two integers.
 *
 *     Breakdown of: asm volatile("addl %2, %0" : "=r"(result) : "0"(a), "r"(b));
 *
 *     "addl %2, %0"    → AT&T syntax: addl SOURCE, DEST  (l = 32-bit long)
 *     : "=r"(result)   → OUTPUT: result is written ("=") to any register ("r")
 *     : "0"(a)         → INPUT:  load 'a' into the SAME register as operand 0 (result)
 *       "r"(b)         → INPUT:  load 'b' into any register
 *     (no clobbers)    → We didn't modify any registers beyond the output
 *
 *     %0, %1, %2 refer to operands in order: result, a, b
 *
 * TR: Mümkün olan en basit inline asm: iki sayıyı topla.
 *     "addl %2, %0"    → AT&T sözdizimi: addl KAYNAK, HEDEF
 *     "=r"(result)     → ÇIKIŞ: result bir yazmaca yazılır
 *     "0"(a)           → GİRİŞ: 'a'yı operand 0 ile aynı yazmaca yükle
 *     "r"(b)           → GİRİŞ: 'b'yi herhangi yazmaca yükle
 */
int asm_add(int a, int b) {
    int result;
    asm volatile(
        "addl %2, %0"       // EN: addl = add 32-bit integers / TR: addl = 32-bit tam sayıları topla
        : "=r"(result)       // EN: Output constraint / TR: Çıkış kısıtlaması
        : "0"(a), "r"(b)    // EN: Input constraints / TR: Giriş kısıtlamaları
    );
    return result;
}
```

```cpp

/**
 * EN: Multiply using x86 MUL instruction.
 *     MUL is special: it always uses RAX as one operand and puts the result in RDX:RAX.
 *     For 32-bit: EAX * operand → EDX:EAX (64-bit result)
 *
 * TR: x86 MUL talimatı ile çarpma.
 *     MUL özeldir: her zaman RAX'ı operand olarak kullanır, sonucu RDX:RAX'a koyar.
 */
uint64_t asm_multiply(uint32_t a, uint32_t b) {
    uint32_t low, high;
    asm volatile(
        "mull %2"                // EN: EAX * %2 → EDX:EAX / TR: EAX * %2 → EDX:EAX
        : "=a"(low), "=d"(high) // EN: "a" = EAX register, "d" = EDX register
        : "r"(b), "0"(a)        // EN: b in any reg, a loaded into EAX (operand 0)
    );
    return (static_cast<uint64_t>(high) << 32) | low;
}

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: RDTSC — Reading the CPU Cycle Counter
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: RDTSC (Read Time-Stamp Counter) reads the CPU's cycle counter.
 *     This is a 64-bit counter that increments with every CPU clock cycle.
 *     It's useful for micro-benchmarking (measuring exact cycle counts).
 *
 *     The instruction returns: EDX:EAX (high 32 bits : low 32 bits)
 *
 *     WARNING: RDTSC can be reordered by the CPU! For precise measurement:
 *     - Use RDTSCP (serializing version) or
 *     - Use CPUID before RDTSC (acts as a barrier)
 *     - Or use std::chrono::high_resolution_clock (portable!)
 *
 * TR: RDTSC (Read Time-Stamp Counter) CPU'nun döngü sayacını okur.
 *     Her CPU saat döngüsünde artan 64-bit'lik bir sayaçtır.
 *     Mikro-kıyaslama (hassas döngü sayısı ölçme) için kullanışlıdır.
 *
 *     UYARI: RDTSC CPU tarafından yeniden sıralanabilir! Kesin ölçüm için:
 *     - RDTSCP (serileştiren sürüm) veya CPUID bariyeri kullanın
 *     - Veya std::chrono::high_resolution_clock kullanın (taşınabilir!)
 */
uint64_t rdtsc() {
    uint32_t lo, hi;
    asm volatile(
        "rdtsc"              // EN: Read Time-Stamp Counter → EDX:EAX
        : "=a"(lo), "=d"(hi) // EN: lo = EAX, hi = EDX
        :
        : "memory"           // EN: Clobber: tell compiler memory may have changed
                             // TR: Bozulan: derleyiciye bellek değişmiş olabilir de
    );
    return (static_cast<uint64_t>(hi) << 32) | lo;
}

/**
 * EN: RDTSCP is the serializing version of RDTSC.
 *     It guarantees all previous instructions have completed before reading.
 *     Also returns the processor ID in ECX (useful for multi-core profiling).
 *
 * TR: RDTSCP, RDTSC'nin serileştiren sürümüdür.
 *     Okumadan önce önceki tüm talimatların tamamlandığını garanti eder.
 */
uint64_t rdtscp() {
    uint32_t lo, hi, aux;
    asm volatile(
        "rdtscp"
        : "=a"(lo), "=d"(hi), "=c"(aux) // EN: EAX, EDX, ECX / TR: EAX, EDX, ECX
        :
        : "memory"
    );
    return (static_cast<uint64_t>(hi) << 32) | lo;
}
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: CPUID — Querying CPU Features
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: CPUID is a special instruction that returns information about the CPU:
 *     - Brand string, vendor (Intel/AMD)
 *     - Feature flags (SSE, AVX, AES, etc.)
 *     - Cache sizes, core count
 *
 *     Input: EAX = leaf number (which info to query)
 *     Output: EAX, EBX, ECX, EDX = results
 *
 * TR: CPUID, CPU hakkında bilgi döndüren özel bir talimattır:
 *     - Marka, üretici (Intel/AMD)
 *     - Özellik bayrakları (SSE, AVX, AES vb.)
 *     - Önbellek boyutları, çekirdek sayısı
 */
struct CpuidResult {
    uint32_t eax, ebx, ecx, edx;
};

CpuidResult cpuid(uint32_t leaf, uint32_t subleaf = 0) {
    CpuidResult r;
    asm volatile(
        "cpuid"
        : "=a"(r.eax), "=b"(r.ebx), "=c"(r.ecx), "=d"(r.edx)
        : "a"(leaf), "c"(subleaf)
        // EN: CPUID clobbers many things, but we listed all outputs, so no extra clobbers needed
        // TR: CPUID birçok şeyi bozar, ama tüm çıkışları listeledik, ekstra bozulan gerekmez
    );
    return r;
}

/**
 * EN: Get CPU vendor string (12 characters: "GenuineIntel" or "AuthenticAMD")
 *     CPUID leaf 0 returns: EBX:EDX:ECX as a 12-byte string
 *     Note: The order is EBX, EDX, ECX (not EBX, ECX, EDX!)
 *
 * TR: CPU üretici string'ini al (12 karakter: "GenuineIntel" veya "AuthenticAMD")
 *     CPUID yaprak 0: EBX:EDX:ECX olarak 12 baytlık string döndürür
 */
std::string get_cpu_vendor() {
    auto r = cpuid(0);
    char vendor[13];
    std::memcpy(vendor + 0, &r.ebx, 4);
    std::memcpy(vendor + 4, &r.edx, 4);  // EN: Yes, EDX before ECX! (x86 quirk)
    std::memcpy(vendor + 8, &r.ecx, 4);  // TR: Evet, ECX'ten önce EDX! (x86 tuhaflığı)
    vendor[12] = '\0';
    return std::string(vendor);
}

/**
 * EN: Get CPU brand string (48 characters, e.g., "Intel(R) Core(TM) i7-...")
 *     Requires CPUID leaves 0x80000002, 0x80000003, 0x80000004
 *     Each leaf returns 16 bytes (EAX+EBX+ECX+EDX), total 48 bytes.
 *
 * TR: CPU marka string'ini al (48 karakter, ör: "Intel(R) Core(TM) i7-...")
 *     CPUID yaprakları 0x80000002, 0x80000003, 0x80000004 gerektirir
 */
std::string get_cpu_brand() {
    char brand[49];
    for (uint32_t i = 0; i < 3; ++i) {
        auto r = cpuid(0x80000002 + i);
        std::memcpy(brand + i * 16 + 0, &r.eax, 4);
        std::memcpy(brand + i * 16 + 4, &r.ebx, 4);
        std::memcpy(brand + i * 16 + 8, &r.ecx, 4);
        std::memcpy(brand + i * 16 + 12, &r.edx, 4);
    }
    brand[48] = '\0';
    // EN: Trim leading spaces (Intel pads with spaces)
    // TR: Baştaki boşlukları kırp (Intel boşluklarla doldurur)
    std::string result(brand);
    size_t start = result.find_first_not_of(' ');
    return (start != std::string::npos) ? result.substr(start) : result;
}
```

```

// ═══════════════════════════════════════════════════════════════════════════
// PART 4: Compiler Intrinsics — The PREFERRED Alternative
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: Compiler intrinsics are functions that map directly to CPU instructions
 *     but are understood by the compiler (so it can optimize around them).
 *     They are cross-platform within GCC/Clang family and MUCH safer than asm.
 *
 *     Common GCC/Clang intrinsics:
 *
 *     ┌──────────────────────────┬──────────────────────────────────────┐
 *     │ Intrinsic                │ What It Does                         │
 *     ├──────────────────────────┼──────────────────────────────────────┤
 *     │ __builtin_popcount(x)    │ Count number of 1-bits in x          │
 *     │ __builtin_clz(x)         │ Count Leading Zeros (from MSB)       │
 *     │ __builtin_ctz(x)         │ Count Trailing Zeros (from LSB)      │
 *     │ __builtin_expect(x, v)   │ Branch prediction hint (x likely == v)  │
 *     │ __builtin_bswap32(x)     │ Byte-swap 32 bits (endian conversion)   │
 *     │ __builtin_prefetch(p)    │ Prefetch memory at address p into cache │
 *     │ __builtin_unreachable()  │ Tell compiler this code path is impossible │
 *     └──────────────────────────┴──────────────────────────────────────┘
 *
 * TR: Derleyici intrinsic'leri doğrudan CPU talimatlarına eşlenen ama derleyici
 *     tarafından anlaşılan fonksiyonlardır (böylece etraflarında optimize edebilir).
 *     asm'den ÇOK daha güvenli ve GCC/Clang ailesinde çapraz platformdur.
 */
void demo_intrinsics() {
    uint32_t x = 0b00101100;  // EN: Binary literal (C++14) / TR: İkili literali (C++14)

    std::cout << "  x = 0b00101100 (decimal " << x << ")\n";
    std::cout << "    __builtin_popcount(x)  = " << __builtin_popcount(x)
              << " (set bits: 3)\n";
    // EN: popcount counts the number of 1-bits → 00101100 has three 1s
    // TR: popcount, 1-bit sayısını sayar → 00101100'da üç tane 1 var

    std::cout << "    __builtin_clz(x)       = " << __builtin_clz(x)
              << " (leading zeros from bit 31)\n";
    // EN: CLZ = Count Leading Zeros from the most significant bit
    //     0b00000000_00000000_00000000_00101100 → 26 leading zeros
    // TR: CLZ = En anlamlı bitten itibaren öndeki sıfırları say

    std::cout << "    __builtin_ctz(x)       = " << __builtin_ctz(x)
              << " (trailing zeros from bit 0)\n";
    // EN: CTZ = Count Trailing Zeros from LSB → ...00101100 → 2 trailing zeros
    // TR: CTZ = LSB'den itibaren sondaki sıfırları say

    uint32_t y = 0xDEADBEEF;
    std::cout << "\n  y = 0xDEADBEEF\n";
    std::cout << "    __builtin_bswap32(y)   = 0x" << std::hex
              << __builtin_bswap32(y) << std::dec << "\n";
    // EN: bswap32 reverses byte order: DE-AD-BE-EF → EF-BE-AD-DE
    //     This is used for endian conversion (same as htonl/ntohl!)
    // TR: bswap32 bayt sırasını tersine çevirir: DE-AD-BE-EF → EF-BE-AD-DE

    // EN: Branch prediction hint
    // TR: Dal tahmini ipucu
    int value = 42;
    if (__builtin_expect(value == 42, 1)) {  // EN: We expect this is TRUE / TR: TRUE bekliyoruz
        std::cout << "\n  __builtin_expect: predicted correctly (value is 42)\n";
    }
}

// ═══════════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 07 - Inline Assembly & Intrinsics\n";
    std::cout << "============================================\n\n";

    // ─── Demo 1: Basic Arithmetic via asm ──────────────────────────────
    {
        std::cout << "--- Demo 1: Inline Assembly Arithmetic ---\n";
        std::cout << "  asm_add(25, 17)      = " << asm_add(25, 17) << "\n";
        std::cout << "  asm_multiply(12, 34) = " << asm_multiply(12, 34) << "\n";
```

```
        // EN: For comparison — compiler generates the SAME instructions with plain C++:
        //     int result = a + b;  →  addl %esi, %eax (or similar)
        //     This shows asm isn't needed for basic math!
        // TR: Karşılaştırma için — derleyici düz C++ ile AYNI talimatları üretir:
        //     Bu, temel matematik için asm'ın gerekmediğini gösterir!
        std::cout << "  (Plain C++ would generate the same machine code)\n\n";
    }

    // ─── Demo 2: RDTSC Cycle Counter ──────────────────────────────────
    {
        std::cout << "--- Demo 2: RDTSC — CPU Cycle Counter ---\n";

        // EN: Measure how many CPU cycles a simple operation takes
        // TR: Basit bir işlemin kaç CPU döngüsü sürdüğünü ölç
        uint64_t start = rdtscp();
        volatile int sum = 0;  // EN: volatile prevents optimizer from removing the loop
        for (int i = 0; i < 1000; ++i) {
            sum += i;
        }
        uint64_t end = rdtscp();

        std::cout << "  Loop 0..999 sum:\n";
        std::cout << "    RDTSC start:  " << start << "\n";
        std::cout << "    RDTSC end:    " << end << "\n";
        std::cout << "    Cycles taken: " << (end - start) << "\n";
        std::cout << "    (~" << (end - start) / 1000 << " cycles per iteration)\n";

        // EN: Compare with std::chrono — MORE PORTABLE, uses the SAME counter
        // TR: std::chrono ile karşılaştır — DAHA TAŞINABİLİR, AYNI sayacı kullanır
        auto t1 = std::chrono::high_resolution_clock::now();
        volatile int sum2 = 0;
        for (int i = 0; i < 1000; ++i) {
            sum2 += i;
        }
        auto t2 = std::chrono::high_resolution_clock::now();
        auto ns = std::chrono::duration_cast<std::chrono::nanoseconds>(t2 - t1).count();
        std::cout << "    std::chrono says: " << ns << " ns\n";
        std::cout << "    (Prefer std::chrono for portable benchmarks!)\n\n";
    }

    // ─── Demo 3: CPUID — CPU Information ──────────────────────────────
    {
        std::cout << "--- Demo 3: CPUID — CPU Information ---\n";

        std::cout << "  CPU Vendor: " << get_cpu_vendor() << "\n";
        std::cout << "  CPU Brand:  " << get_cpu_brand() << "\n";

        // EN: Check feature flags (leaf 1, ECX and EDX)
        // TR: Özellik bayraklarını kontrol et (yaprak 1, ECX ve EDX)
        auto features = cpuid(1);

        // EN: Feature bit positions from Intel SDM Vol.2A, Table 3-10
        // TR: Intel SDM Vol.2A, Tablo 3-10'dan özellik bit pozisyonları
        bool has_sse2   = (features.edx >> 26) & 1;
        bool has_sse41  = (features.ecx >> 19) & 1;
        bool has_sse42  = (features.ecx >> 20) & 1;
        bool has_avx    = (features.ecx >> 28) & 1;
        bool has_aes    = (features.ecx >> 25) & 1;
        bool has_popcnt = (features.ecx >> 23) & 1;

        std::cout << "  Features:\n";
        std::cout << "    SSE2:   " << (has_sse2   ? "YES" : "NO") << "\n";
        std::cout << "    SSE4.1: " << (has_sse41  ? "YES" : "NO") << "\n";
        std::cout << "    SSE4.2: " << (has_sse42  ? "YES" : "NO") << "\n";
        std::cout << "    AVX:    " << (has_avx    ? "YES" : "NO") << "\n";
        std::cout << "    AES-NI: " << (has_aes    ? "YES" : "NO") << "\n";
        std::cout << "    POPCNT: " << (has_popcnt ? "YES" : "NO") << "\n";

        // EN: Check AVX2 (leaf 7, subleaf 0, EBX bit 5)
        // TR: AVX2 kontrol et (yaprak 7, alt yaprak 0, EBX bit 5)
        auto ext_features = cpuid(7, 0);
        bool has_avx2 = (ext_features.ebx >> 5) & 1;
        std::cout << "    AVX2:   " << (has_avx2 ? "YES" : "NO") << "\n\n";
    }
```

```
    // ─── Demo 4: Compiler Intrinsics ──────────────────────────────────
    {
        std::cout << "--- Demo 4: Compiler Intrinsics (Preferred Alternative) ---\n";
        demo_intrinsics();
        std::cout << "\n";
    }

    // ─── Demo 5: Constraint Types Summary ─────────────────────────────
    {
        std::cout << "--- Demo 5: Assembly Constraint Reference ---\n";
        std::cout << "   ┌──────────────┬──────────────────────────────────────┐\n";
        std::cout << "   │ Constraint   │ Meaning                              │\n";
        std::cout << "   ├──────────────┼──────────────────────────────────────┤\n";
        std::cout << "   │ \"r\"         │ Any general-purpose register         │\n";
        std::cout << "   │ \"a\"         │ EAX/RAX register specifically        │\n";
        std::cout << "   │ \"b\"         │ EBX/RBX register specifically        │\n";
        std::cout << "   │ \"c\"         │ ECX/RCX register specifically        │\n";
        std::cout << "   │ \"d\"         │ EDX/RDX register specifically        │\n";
        std::cout << "   │ \"m\"         │ Memory operand (address)             │\n";
        std::cout << "   │ \"i\"         │ Immediate integer constant           │\n";
        std::cout << "   │ \"0\"..\"9\"   │ Same register as operand N           │\n";
        std::cout << "   │ \"=\"         │ Output-only (write)                  │\n";
        std::cout << "   │ \"+\"         │ Read-write (both input and output)   │\n";
        std::cout << "   │ \"memory\"    │ Clobber: asm may read/write any memory │\n";
        std::cout << "   │ \"cc\"        │ Clobber: asm modifies condition flags │\n";
        std::cout << "   └──────────────┴──────────────────────────────────────┘\n\n";
    }

    std::cout << "============================================\n";
    std::cout << "   End of Inline Assembly & Intrinsics\n";
    std::cout << "============================================\n";

    return 0;
}
```

► 08. Simd Intrinsics

module_07_under_the_hood/08_simd_intrinsics.cpp

Konu: SIMD: Single Instruction, Multiple Data — Vektörel İşleme

Teori / Theory

THEORY: What Is SIMD?

SIMD lets a single CPU instruction operate on MULTIPLE data elements simultaneously. Instead of adding two numbers, you add 4, 8, or 16 at once.

```
┌──────────────────────────────────────────────────────────┐
│ Scalar (without SIMD):          SIMD (SSE: 128-bit):     │
│   a[0] + b[0] = c[0]             a[0..3] + b[0..3] = c[0..3]  │
│   a[1] + b[1] = c[1]             (single instruction!)    │
│   a[2] + b[2] = c[2]                                      │
│   a[3] + b[3] = c[3]           AVX (256-bit): 8 floats at once │
│   (4 instructions)             AVX-512: 16 floats at once │
└──────────────────────────────────────────────────────────┘
```

x86 SIMD evolution:
- SSE (1999): 128-bit registers (XMM0-XMM15), 4 floats
- SSE2 (2001): + 64-bit doubles, integer SIMD
- SSE4 (2006): + blend, round, string processing
- AVX (2011): 256-bit registers (YMM0-YMM15), 8 floats
- AVX2 (2013): + 256-bit integers
- AVX-512 (2016): 512-bit registers (ZMM0-ZMM31), 16 floats

TEORİ: SIMD Nedir?

SIMD, tek bir CPU talimatının aynı anda BİRDEN FAZLA veri elemanı üzerinde çalışmasını sağlar. İki sayı toplamak yerine 4, 8 veya 16 sayı birden toplanır.

```
┌──────────────────────────────────────────────────────────┐
│ Skaler (SIMD'siz):              SIMD (SSE: 128-bit):     │
│   a[0] + b[0] = c[0]              a[0..3] + b[0..3] = c[0..3]  │
│   a[1] + b[1] = c[1]              (tek talimat!)           │
│   a[2] + b[2] = c[2]                                       │
│   a[3] + b[3] = c[3]            AVX (256-bit): 8 float birden  │
│   (4 talimat)                   AVX-512: 16 float birden   │
└──────────────────────────────────────────────────────────┘
```

x86 SIMD evrimi:
- SSE (1999): 128-bit yazmaçlar (XMM0-XMM15), 4 float
- SSE2 (2001): + 64-bit double, tamsayı SIMD
- SSE4 (2006): + blend, round, string işleme
- AVX (2011): 256-bit yazmaçlar (YMM0-YMM15), 8 float
- AVX2 (2013): + 256-bit tamsayılar
- AVX-512 (2016): 512-bit yazmaçlar (ZMM0-ZMM31), 16 float

THEORY: Intrinsics vs Auto-Vectorization

Two ways to use SIMD:
1. COMPILER AUTO-VECTORIZATION (preferred): Write normal C++ loops → compiler generates SIMD automatically Enabled by: -O2/-O3, -march=native, -ftree-vectorize Works BEST with: simple loops, contiguous memory, no dependencies
2. EXPLICIT INTRINSICS (when compiler can't auto-vectorize): Use Intel intrinsics functions (_mm_*, _mm256_*, _mm512_*) Full control, but: non-portable, hard to read, hard to maintain
Rule of thumb: ALWAYS try auto-vectorization first. Use intrinsics only when benchmarks prove the compiler isn't vectorizing.

TEORİ: İntrinsikler vs Oto-Vektörleştirme

SIMD kullanmanın iki yolu:
1. DERLEYİCİ OTO-VEKTÖRLEŞTİRME (tercih edilen): Normal C++ döngüleri yaz → derleyici SIMD'i otomatik üretir Etkinleştirme: -O2/-O3, -march=native, -ftree-vectorize EN İYİ çalışır: basit döngüler, bitisik bellek, bağımlılık yok
2. AÇIK İNTRİNSİKLER (derleyici vektörleştiremediğinde): Intel intrinsics fonksiyonlarını kullan (_mm_*, _mm256_*, _mm512_*) Tam kontrol, ama: taşınamaz, okunması zor, bakımı zor
Altın kural: HER ZAMAN önce oto-vektörleştirmeyi dene. İntrinsikleri sadece benchmark'lar derleyicinin vektörleştirmediğini kanıtlarsa kullan.

THEORY: Memory Alignment

SIMD instructions require (or strongly prefer) aligned memory:
- SSE: 16-byte aligned (_mm_load_ps requires 16-byte alignment)
- AVX: 32-byte aligned (_mm256_load_ps requires 32-byte alignment)
- Use `alignas(N)` to align stack/static arrays
- Use aligned allocators for heap memory
- Unaligned loads (_mm_loadu_ps) work but may be slower on older CPUs (On modern CPUs, aligned/unaligned performance is nearly identical)

TEORİ: Bellek Hizalama

SIMD talimatları hizalanmış bellek gerektirir (veya şiddetle tercih eder):
- SSE: 16-bayt hizalı (_mm_load_ps 16-bayt hizalama gerektirir)
- AVX: 32-bayt hizalı (_mm256_load_ps 32-bayt hizalama gerektirir)
- Yığın/statik dizileri hizalamak için `alignas(N)` kullanın
- Heap belleği için hizalı ayırıcılar kullanın
- Hizasız yüklemeler (_mm_loadu_ps) çalışır ama eski CPU'larda daha yavaş (Modern CPU'larda hizalı/hizasız performans neredeyse aynıdır)

→ Ref: https://www.intel.com/content/www/us/en/docs/intrinsics-guide/
→ Ref: https://gcc.gnu.org/onlinedocs/gcc/x86-Options.html

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <chrono>
#include <array>
#include <numeric>
#include <iomanip>
#include <cmath>

// EN: SIMD intrinsic headers — each includes the previous generation
//     <immintrin.h> includes everything: SSE, SSE2, ..., AVX, AVX2, AVX-512
// TR: SIMD intrinsic başlık dosyaları — her biri önceki nesli içerir
#include <immintrin.h>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: SSE — 128-bit SIMD (4 floats / 2 doubles at once)
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: SSE intrinsic naming convention:
 *     _mm_<operation>_<type>
 *
 *     Types: ps = packed single (4×float)
 *            pd = packed double (2×double)
 *            epi32 = packed 32-bit integer
 *            si128 = 128-bit integer
 *
 *     Operations: add, sub, mul, div, load, store, set, setzero, sqrt, min, max
 *
 *     Example: _mm_add_ps = Packed Single-precision Add → adds 4 floats at once
 *
 * TR: SSE intrinsic adlandırma kuralı:
 *     _mm_<işlem>_<tür>
 *     Türler: ps = paketli tek hassasiyet (4×float), pd = paketli çift (2×double)
 */

/**
 * EN: Add two arrays of 4 floats using SSE intrinsics.
 *     This demonstrates the basic load → operate → store pattern:
 *     1. _mm_load_ps: Load 4 floats from memory into __m128 register
 *     2. _mm_add_ps:  Add two __m128 registers (4 floats simultaneously)
 *     3. _mm_store_ps: Store __m128 register back to memory
 *
 * TR: SSE intrinsikleri ile 4 float'lık iki diziyi topla.
 *     Temel yükle → işle → sakla kalıbı gösterilir.
 */
```

```cpp
void sse_add_example() {
    // EN: alignas(16) ensures 16-byte alignment for SSE loads
    //     Without this, _mm_load_ps would crash (SIGSEGV) on some CPUs!
    // TR: alignas(16), SSE yükleme için 16-bayt hizalamayı garanti eder
    alignas(16) float a[4] = {1.0f, 2.0f, 3.0f, 4.0f};
    alignas(16) float b[4] = {5.0f, 6.0f, 7.0f, 8.0f};
    alignas(16) float c[4];

    // EN: Load 4 floats from a[] and b[] into 128-bit SSE registers
    // TR: a[] ve b[]'den 4 float'ı 128-bit SSE yazmaçlarına yükle
    __m128 va = _mm_load_ps(a);   // va = [1.0, 2.0, 3.0, 4.0]
    __m128 vb = _mm_load_ps(b);   // vb = [5.0, 6.0, 7.0, 8.0]

    // EN: Add 4 pairs simultaneously — ONE instruction for 4 additions!
    // TR: 4 çifti aynı anda topla — 4 toplama için TEK talimat!
    __m128 vc = _mm_add_ps(va, vb);  // vc = [6.0, 8.0, 10.0, 12.0]

    // EN: Store result back to memory
    // TR: Sonucu belleğe geri sakla
    _mm_store_ps(c, vc);

    std::cout << "  SSE _mm_add_ps: [";
    for (int i = 0; i < 4; ++i) {
        if (i > 0) std::cout << ", ";
        std::cout << c[i];
    }
    std::cout << "]\n";
}

/**
 * EN: SSE provides rich math operations beyond basic arithmetic.
 *     This function demonstrates several in one place.
 *
 * TR: SSE temel aritmetiğin ötesinde zengin matematik işlemleri sağlar.
 */
void sse_operations_showcase() {
    alignas(16) float a[4] = {16.0f, 9.0f, 25.0f, 4.0f};
    alignas(16) float b[4] = {3.0f, 7.0f, 2.0f, 8.0f};
    alignas(16) float result[4];

    __m128 va = _mm_load_ps(a);
    __m128 vb = _mm_load_ps(b);

    // EN: Square root of each element / TR: Her elemanın karekökü
    __m128 vsqrt = _mm_sqrt_ps(va);
    _mm_store_ps(result, vsqrt);
    std::cout << "  sqrt([16,9,25,4])    = [" << result[0] << ", " << result[1]
              << ", " << result[2] << ", " << result[3] << "]\n";

    // EN: Element-wise minimum / TR: Eleman bazlı minimum
    __m128 vmin = _mm_min_ps(va, vb);
    _mm_store_ps(result, vmin);
    std::cout << "  min(a,b)             = [" << result[0] << ", " << result[1]
              << ", " << result[2] << ", " << result[3] << "]\n";

    // EN: Element-wise maximum / TR: Eleman bazlı maksimum
    __m128 vmax = _mm_max_ps(va, vb);
    _mm_store_ps(result, vmax);
    std::cout << "  max(a,b)             = [" << result[0] << ", " << result[1]
              << ", " << result[2] << ", " << result[3] << "]\n";

    // EN: Multiply then add: (a * b) + c — one operation, better precision
    //     FMA (Fused Multiply-Add) avoids one rounding step
    // TR: Çarp sonra topla: (a * b) + c — tek işlem, daha iyi hassasiyet
    alignas(16) float addend[4] = {1.0f, 1.0f, 1.0f, 1.0f};
    __m128 vadd = _mm_load_ps(addend);
    __m128 vfma = _mm_fmadd_ps(va, vb, vadd);  // (a*b) + 1
    _mm_store_ps(result, vfma);
    std::cout << "  fma(a*b+1)           = [" << result[0] << ", " << result[1]
              << ", " << result[2] << ", " << result[3] << "]\n";
}

```

```
// ═══════════════════════════════════════════════════════════════════════════
// PART 2: AVX — 256-bit SIMD (8 floats / 4 doubles at once)
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: AVX doubles SSE's width: 256-bit registers process 8 floats at once.
 *     Naming: _mm256_<operation>_<type>
 *
 *     Important difference from SSE:
 *     - AVX registers (YMM) are split into two 128-bit "lanes"
 *     - Some operations don't cross lane boundaries (shuffle, permute)
 *     - This is a common source of bugs!
 *
 * TR: AVX, SSE'nin genişliğini ikiye katlar: 256-bit yazmaçlar aynı anda 8 float işler.
 *     Önemli fark: AVX yazmaçları iki 128-bit "şerit"e bölünmüştür.
 */
void avx_add_example() {
    // EN: alignas(32) for AVX's 256-bit = 32-byte alignment requirement
    // TR: AVX'in 256-bit = 32-bayt hizalama gereksinimi için alignas(32)
    alignas(32) float a[8] = {1.0f, 2.0f, 3.0f, 4.0f, 5.0f, 6.0f, 7.0f, 8.0f};
    alignas(32) float b[8] = {10.0f, 20.0f, 30.0f, 40.0f, 50.0f, 60.0f, 70.0f, 80.0f};
    alignas(32) float c[8];

    __m256 va = _mm256_load_ps(a);   // EN: Load 8 floats into 256-bit YMM register
    __m256 vb = _mm256_load_ps(b);
    __m256 vc = _mm256_add_ps(va, vb);  // EN: 8 additions in one instruction!
    _mm256_store_ps(c, vc);

    std::cout << "  AVX _mm256_add_ps: [";
    for (int i = 0; i < 8; ++i) {
        if (i > 0) std::cout << ", ";
        std::cout << c[i];
    }
    std::cout << "]\n";
}

/**
 * EN: AVX integer operations (AVX2 required).
 *     Processes 8 × 32-bit integers simultaneously.
 *
 * TR: AVX tam sayı işlemleri (AVX2 gerektirir).
 *     Aynı anda 8 × 32-bit tam sayı işler.
 */
void avx2_integer_example() {
    alignas(32) int32_t a[8] = {1, 2, 3, 4, 5, 6, 7, 8};
    alignas(32) int32_t b[8] = {10, 20, 30, 40, 50, 60, 70, 80};
    alignas(32) int32_t c[8];

    __m256i va = _mm256_load_si256(reinterpret_cast<const __m256i*>(a));
    __m256i vb = _mm256_load_si256(reinterpret_cast<const __m256i*>(b));
    __m256i vc = _mm256_add_epi32(va, vb);  // EN: 8 integer adds at once
    _mm256_store_si256(reinterpret_cast<__m256i*>(c), vc);

    std::cout << "  AVX2 _mm256_add_epi32: [";
    for (int i = 0; i < 8; ++i) {
        if (i > 0) std::cout << ", ";
        std::cout << c[i];
    }
    std::cout << "]\n";
}

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: Benchmark — Scalar vs SIMD
// ═══════════════════════════════════════════════════════════════════════════

static constexpr size_t BENCH_SIZE = 1024 * 1024;  // EN: 1M elements / TR: 1M eleman

/**
 * EN: Scalar (non-SIMD) vector addition — baseline for comparison.
 *     The compiler MAY auto-vectorize this with -O2, so we use
 *     -fno-tree-vectorize in the benchmark to see the true scalar speed.
 *     (Or compare with -O2 to see how well the compiler auto-vectorizes.)
 *
 * TR: Skaler (SIMD olmayan) vektör toplama — karşılaştırma temeli.
 */
void __attribute__((noinline)) scalar_add(const float* a, const float* b, float* c, size_t n) {
```

```
194     for (size_t i = 0; i < n; ++i) {
195         c[i] = a[i] + b[i];
196     }
197 }
198
199 /**
200  * EN: AVX-vectorized addition — 8 floats per iteration.
201  *     Processes 8 elements per loop iteration (256 bits / 32 bits per float).
202  *     Handles tail elements (remainder after 8-element chunks) with scalar code.
203  *
204  * TR: AVX ile vektörleştirilmiş toplama — yineleme başına 8 float.
205  *     Kalan elemanlar (8'lik gruplardan arta kalanlar) skaler kodla işlenir.
206  */
207 void __attribute__((noinline)) avx_add(const float* a, const float* b, float* c, size_t n) {
208     size_t i = 0;
209     // EN: Main loop: process 8 floats per iteration
210     // TR: Ana döngü: her yinelemede 8 float işle
211     for (; i + 8 <= n; i += 8) {
212         __m256 va = _mm256_loadu_ps(a + i);  // EN: loadu = unaligned load (safe for any address)
213         __m256 vb = _mm256_loadu_ps(b + i);
214         __m256 vc = _mm256_add_ps(va, vb);
215         _mm256_storeu_ps(c + i, vc);
216     }
217     // EN: Handle remaining elements that don't fill a full 8-element chunk
218     // TR: Tam 8 elemanlık grubu doldurmayan kalan elemanları işle
219     for (; i < n; ++i) {
220         c[i] = a[i] + b[i];
221     }
222 }
223
224 struct BenchResult {
225     double scalar_ms;
226     double simd_ms;
227     double speedup;
228 };
229
230 /**
231  * EN: Benchmark function that runs both versions multiple times and reports average.
232  * TR: Her iki sürümü birden çok kez çalıştırıp ortalamayı raporlayan kıyaslama fonksiyonu.
233  */
234 BenchResult run_benchmark() {
235     // EN: Allocate aligned arrays on heap using aligned_alloc
236     //     aligned_alloc(alignment, size) — size must be multiple of alignment
237     // TR: aligned_alloc ile yığında hizalanmış diziler ayır
238     constexpr size_t align = 32;
239     constexpr size_t bytes = BENCH_SIZE * sizeof(float);
240
241     float* a = static_cast<float*>(std::aligned_alloc(align, bytes));
242     float* b = static_cast<float*>(std::aligned_alloc(align, bytes));
243     float* c = static_cast<float*>(std::aligned_alloc(align, bytes));
244
245     // EN: Initialize with some data / TR: Verilerle başlat
246     for (size_t i = 0; i < BENCH_SIZE; ++i) {
247         a[i] = static_cast<float>(i) * 0.5f;
248         b[i] = static_cast<float>(i) * 0.25f;
249     }
250
251     constexpr int ITERATIONS = 100;
252
253     // EN: Benchmark scalar version
254     // TR: Skaler sürümü benchmark'la
255     auto t1 = std::chrono::high_resolution_clock::now();
256     for (int iter = 0; iter < ITERATIONS; ++iter) {
257         scalar_add(a, b, c, BENCH_SIZE);
258     }
259     auto t2 = std::chrono::high_resolution_clock::now();
260     double scalar_ms = std::chrono::duration<double, std::milli>(t2 - t1).count() / ITERATIONS;
261
262     // EN: Quick correctness check / TR: Hızlı doğruluk kontrolü
263     float expected_last = static_cast<float>(BENCH_SIZE - 1) * 0.75f;
264     float got_scalar = c[BENCH_SIZE - 1];
265
266     // EN: Benchmark SIMD version
267     // TR: SIMD sürümü benchmark'la
268     t1 = std::chrono::high_resolution_clock::now();
```

```
    for (int iter = 0; iter < ITERATIONS; ++iter) {
        avx_add(a, b, c, BENCH_SIZE);
    }
    t2 = std::chrono::high_resolution_clock::now();
    double simd_ms = std::chrono::duration<double, std::milli>(t2 - t1).count() / ITERATIONS;

    float got_simd = c[BENCH_SIZE - 1];

    std::free(a);
    std::free(b);
    std::free(c);

    // EN: Verify both produce same result / TR: İkisinin aynı sonucu ürettiğini doğrula
    if (std::abs(got_scalar - expected_last) > 0.01f || std::abs(got_simd - expected_last) > 0.01f) {
        std::cout << "  WARNING: Correctness check failed!\n";
    }

    return {scalar_ms, simd_ms, scalar_ms / simd_ms};
}

// ═══════════════════════════════════════════════════════════════════════════
// PART 4: Auto-Vectorization Tips
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: Patterns that help auto-vectorization:
 *
 *     ✅ GOOD (vectorizer-friendly):
 *     - Simple counted loops: for (int i=0; i<n; ++i)
 *     - Contiguous memory access: a[i], *(ptr + i)
 *     - No cross-iteration dependencies
 *     - Use __restrict__ to promise no aliasing
 *     - Use -O2 -march=native -ftree-vectorize
 *
 *     ❌ BAD (prevents auto-vectorization):
 *     - Pointer aliasing: c[i] = a[i] + b[i] where c could overlap a or b
 *     - Function calls inside loop (unless simple inline functions)
 *     - Data-dependent branches: if (a[i] > 0) c[i] = a[i];
 *     - Cross-iteration dependencies: a[i] = a[i-1] + 1
 *     - Complex control flow in loop body
 *
 * TR: Oto-vektörleştirmeye yardımcı kalıplar:
 *     ✅ İYİ: Basit sayılı döngüler, bitişik bellek erişimi, bağımlılık yok
 *     ❌ KÖTÜ: İşaretçi örtüşmesi, döngü içi fonksiyon çağrısı, veri bağımlı dallar
 */
void __attribute__((noinline)) autovec_friendly(
    float* __restrict__ c,
    const float* __restrict__ a,
    const float* __restrict__ b,
    size_t n)
{
    // EN: `__restrict__` tells the compiler: a, b, c point to non-overlapping memory
    //     This is the C++ equivalent of C99's `restrict` keyword
    //     Without it, compiler must assume c[i] might alias a[j] or b[j]
    // TR: `__restrict__` derleyiciye söyler: a, b, c örtüşmeyen belleğe işaret eder
    for (size_t i = 0; i < n; ++i) {
        c[i] = a[i] * b[i] + a[i];  // EN: Should auto-vectorize with -O2
    }
}

// ═══════════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 07 - SIMD: Vectorized Computing\n";
    std::cout << "============================================\n\n";

    // ─── Demo 1: SSE Basics ──────────────────────────────────────────
    {
        std::cout << "--- Demo 1: SSE (128-bit) Operations ---\n";
        sse_add_example();
        sse_operations_showcase();
        std::cout << "\n";
    }
```

```cpp

    // ── Demo 2: AVX Basics ──────────────────────────────────────────
    {
        std::cout << "--- Demo 2: AVX/AVX2 (256-bit) Operations ---\n";
        avx_add_example();
        avx2_integer_example();
        std::cout << "\n";
    }

    // ── Demo 3: Benchmark ───────────────────────────────────────────
    {
        std::cout << "--- Demo 3: Benchmark — Scalar vs AVX (1M floats) ---\n";
        auto result = run_benchmark();
        std::cout << std::fixed << std::setprecision(3);
        std::cout << "  Scalar add: " << result.scalar_ms << " ms\n";
        std::cout << "  AVX add:    " << result.simd_ms << " ms\n";
        std::cout << "  Speedup:    " << result.speedup << "x\n";
        std::cout << "  (Theoretical max for AVX: 8x, real-world limited by memory bandwidth)\n\n";
    }

    // ── Demo 4: Alignment Importance ────────────────────────────────
    {
        std::cout << "--- Demo 4: Memory Alignment ---\n";

        alignas(32) float aligned_arr[8];
        float unaligned_arr[8];  // EN: May not be 32-byte aligned / TR: 32-bayt hizalı olmayabilir

        std::cout << "  alignas(32) array address: " << static_cast<void*>(aligned_arr)
                  << " (aligned: " << (reinterpret_cast<uintptr_t>(aligned_arr) % 32 == 0 ? "YES" : "NO") << ")\n";
        std::cout << "  regular array address:     " << static_cast<void*>(unaligned_arr)
                  << " (32-byte aligned: " << (reinterpret_cast<uintptr_t>(unaligned_arr) % 32 == 0 ? "YES" : "NO") << ")\n";

        std::cout << "\n  Register sizes:\n";
        std::cout << "    __m128 (SSE):  " << sizeof(__m128) << " bytes = " << sizeof(__m128) * 8 << " bits\n";
        std::cout << "    __m256 (AVX):  " << sizeof(__m256) << " bytes = " << sizeof(__m256) * 8 << " bits\n";
        std::cout << "    __m256i (AVX2): " << sizeof(__m256i) << " bytes = " << sizeof(__m256i) * 8 << " bits\n\n";
    }

    // ── Demo 5: Auto-Vectorization Report ───────────────────────────
    {
        std::cout << "--- Demo 5: Auto-Vectorization Tips ---\n";
        std::cout << "  Compile flags to enable auto-vectorization:\n";
        std::cout << "    g++ -O2 -march=native -ftree-vectorize\n\n";
        std::cout << "  Diagnostic flags to SEE what was vectorized:\n";
        std::cout << "    g++ -fopt-info-vec-optimized     (successful)\n";
        std::cout << "    g++ -fopt-info-vec-missed        (failures + reasons)\n";
        std::cout << "    g++ -fopt-info-vec-all           (everything)\n\n";
        std::cout << "  Key rules for vectorizable loops:\n";
        std::cout << "    1. Use __restrict__ to prevent aliasing assumptions\n";
        std::cout << "    2. Keep loop body simple (no function calls if possible)\n";
        std::cout << "    3. Access memory contiguously (a[i], not a[i*stride])\n";
        std::cout << "    4. Avoid cross-iteration dependencies\n";
        std::cout << "    5. Use -march=native to target YOUR CPU's SIMD level\n\n";

        // EN: Quick demo of auto-vectorized function
        // TR: Oto-vektörleştirilmiş fonksiyonun hızlı demosu
        alignas(32) float a[8] = {1, 2, 3, 4, 5, 6, 7, 8};
        alignas(32) float b[8] = {2, 2, 2, 2, 2, 2, 2, 2};
        alignas(32) float c[8];
        autovec_friendly(c, a, b, 8);
        std::cout << "  autovec_friendly (a*b+a): [";
        for (int i = 0; i < 8; ++i) {
            if (i > 0) std::cout << ", ";
            std::cout << c[i];
        }
        std::cout << "]\n\n";
    }
```

```
    // ─── Demo 6: Common Intrinsics Reference ──────────────────────────
    {
        std::cout << "--- Demo 6: Common SIMD Intrinsics Reference ---\n";
        std::cout << "  ┌──────────────────────────┬──────────────────────────────────┐\n";
        std::cout << "  │ Intrinsic                │ Description                      │\n";
        std::cout << "  ├──────────────────────────┼──────────────────────────────────┤\n";
        std::cout << "  │ _mm_load_ps(p)           │ Load 4 aligned floats (SSE)      │\n";
        std::cout << "  │ _mm_loadu_ps(p)          │ Load 4 unaligned floats (SSE)    │\n";
        std::cout << "  │ _mm256_load_ps(p)        │ Load 8 aligned floats (AVX)      │\n";
        std::cout << "  │ _mm_add_ps(a,b)          │ Add 4 floats (SSE)               │\n";
        std::cout << "  │ _mm256_add_ps(a,b)       │ Add 8 floats (AVX)               │\n";
        std::cout << "  │ _mm_mul_ps(a,b)          │ Multiply 4 floats (SSE)          │\n";
        std::cout << "  │ _mm_fmadd_ps(a,b,c)      │ Fused multiply-add: a*b+c (FMA)  │\n";
        std::cout << "  │ _mm_sqrt_ps(a)           │ Square root 4 floats (SSE)       │\n";
        std::cout << "  │ _mm256_setzero_ps()      │ Set all 8 floats to 0 (AVX)      │\n";
        std::cout << "  │ _mm256_set1_ps(x)        │ Broadcast x to all 8 slots (AVX) │\n";
        std::cout << "  │ _mm_cmpgt_ps(a,b)        │ Compare greater, returns mask (SSE)│\n";
        std::cout << "  │ _mm256_blend_ps(a,b,mask)│ Select elements by mask (AVX)    │\n";
        std::cout << "  └──────────────────────────┴──────────────────────────────────┘\n\n";
    }

    std::cout << "==========================================\n";
    std::cout << "   End of SIMD Intrinsics\n";
    std::cout << "==========================================\n";

    return 0;
}
```

► 09. Cpp20 Modules

module_07_under_the_hood/09_cpp20_modules.cpp

Konu: C++20 Modules: #include'ın Sonu — The End of #include

Teori / Theory

THEORY: What Are C++20 Modules?

C++20 Modules replace the 40-year-old #include system with a modern, compiled import mechanism. Key differences:

```
┌──────────────────────┬──────────────────────────────┐
│ Feature              │ #include (Headers)    │ import (Modules)    │
├──────────────────────┼──────────────────────┼──────────────────┤
│ Mechanism            │ Textual copy-paste    │ Pre-compiled binary │
│ Build speed          │ Slow (re-parsed each  │ Fast (parsed once,  │
│                      │ TU = O(N²) work)      │ imported as binary) │
│ Macro leakage        │ YES (all #defines     │ NO (macros don't    │
│                      │ leak everywhere)      │ cross module border)│
│ Include order matters│ YES (notorious bugs)  │ NO                  │
│ ODR violations       │ Common                │ Impossible by design│
│ Symbol visibility    │ Everything exported   │ Only explicit export│
│ Tooling support      │ Mature                │ Improving (2024+)   │
└──────────────────────┴──────────────────────┴──────────────────┘
```

TEORİ: C++20 Modülleri Nedir?

C++20 Modülleri, 40 yıllık #include sistemini modern, derlenmiş bir import mekanizmasıyla değiştirir. Temel farklar:

```
┌──────────────────────┬──────────────────────┬──────────────────────┐
│ Özellik              │ #include (Başlıklar)  │ import (Modüller)    │
├──────────────────────┼──────────────────────┼──────────────────────┤
│ Mekanizma            │ Metinsel kopyala-yap. │ Ön-derlenmiş ikili   │
│ Derleme hızı         │ Yavaş (her TU'da     │ Hızlı (bir kez       │
│                      │ yeniden ayrıştırılır) │ ayrıştır, ikili al)  │
│ Makro sızması        │ EVET (tüm #define'lar │ HAYIR (makrolar      │
│                      │ her yere sızar)      │ modül sınırını geçmz) │
│ Include sırası önemli│ EVET (meşhur hatalar) │ HAYIR                │
│ ODR ihlalleri        │ Sık                  │ Tasarım gereği imknsız │
│ Sembol görünürlüğü   │ Her şey dışa aktarılır│ Sadece açık export   │
│ Araç desteği         │ Olgun                │ Gelişiyor (2024+)    │
└──────────────────────┴──────────────────────┴──────────────────────┘
```

THEORY: Module Syntax

A module consists of:
1. MODULE INTERFACE UNIT (.cppm/.ixx):
- Declares what the module exports (public API)
- Contains `export module name;` at the top
- Only exported entities are visible to importers
2. MODULE IMPLEMENTATION UNIT (.cpp):
- Contains `module name;` (no export keyword)
- Can access non-exported internals
- Implementation details hidden from importers
Basic syntax:

```
// ── math_utils.cppm (Interface Unit) ──
```

export module math_utils; // Declare exported module
export int add(int a, int b); // Exported: visible to importers export double pi = 3.14159; // Exported variable
int internal_helper(int x); // NOT exported: module-private

```
// ── math_utils.cpp (Implementation Unit) ──
```

module math_utils; // Implementation of math_utils
int add(int a, int b) { return a + b; } int internal_helper(int x) { return x * x; }

```
// ── main.cpp (Consumer) ──
```

import math_utils; // Import the module int main() { int result = add(3, 4); // OK: add is exported // internal_helper(5); // ERROR: not exported! }

TEORİ: Modül Sözdizimi

Bir modül şunlardan oluşur:
1. MODÜL ARAYÜZ BİRİMİ (.cppm/.ixx):
- Modülün neyi dışa aktardığını bildirir (genel API)
- Tepede `export module isim;` içerir
- Sadece dışa aktarılan varlıklar içe aktaranlara görünür
2. MODÜL UYGULAMA BİRİMİ (.cpp):
- `module isim;` içerir (export anahtar kelimesi yok)
- Dışa aktarılmayan içselliklere erişebilir
- Uygulama detayları içe aktaranlardan gizlidir
Temel sözdizimi:

```
// — math_utils.cppm (Arayüz Birimi) —
```

export module math_utils; // Dışa aktarılan modülü bildir
export int add(int a, int b); // Dışa aktarıldı: içe aktaranlara görünür export double pi = 3.14159; // Dışa aktarılan değişken
int internal_helper(int x); // Dışa AKTARILMADI: modül-özel

```
// — math_utils.cpp (Uygulama Birimi) —
```

module math_utils; // math_utils'in uygulaması
int add(int a, int b) { return a + b; } int internal_helper(int x) { return x * x; }

```
// — main.cpp (Tüketici) —
```

import math_utils; // Modülü içe aktar int main() { int result = add(3, 4); // OK: add dışa aktarıldı // internal_helper(5); // HATA: dışa aktarılmadı! }

THEORY: Module Partitions

Large modules can be split into partitions:

```
// — math:basic.cppm —
```

export module math:basic; // Partition ":basic" of module "math" export int add(int a, int b);

```
// — math:advanced.cppm —
```

export module math:advanced; // Partition ":advanced" export double sqrt_approx(double x);

```
// — math.cppm (Primary Interface) —
```

export module math; export import :basic; // Re-export basic partition export import :advanced; // Re-export advanced partition

```
// — main.cpp —
```

import math; // Gets everything from both partitions // import math:basic; // Can also import specific partition

TEORİ: Modül Bölümleri

Büyük modüller bölümlere ayrılabilir (partition):

```
// — math:basic.cppm —
```

export module math:basic; // "math" modülünün ":basic" bölümü export int add(int a, int b);

```
// — math:advanced.cppm —
```

export module math:advanced; // ":advanced" bölümü export double sqrt_approx(double x);

```
// — math.cppm (Birincil Arayüz) —
```

export module math; export import :basic; // basic bölümünü yeniden dışa aktar export import :advanced; // advanced bölümünü yeniden dışa aktar

```
// — main.cpp —
```

import math; // Her iki bölümden her şeyi alır // import math:basic; // Belirli bölümü de içe aktarabilir

THEORY: Header Units

Header units bridge the gap between headers and modules:
import <iostream>; // Import standard library header as header unit import <vector>; // Faster than #include (pre-compiled) import "legacy.h"; // Import your own headers as header units
Benefits:
- Faster builds (header parsed once, cached as binary)
- No macro leakage from the header unit (mostly)
- Gradual migration path: change #include → import one at a time
Limitations:
- Not all headers can be imported as header units
- Macro-heavy headers (e.g., Windows.h) may not work
- Compiler support varies (GCC/Clang still evolving)

TEORİ: Başlık Birimleri

Başlık birimleri, başlıklar ve modüller arasında köprü kurar:
import <iostream>; // Standart kütüphane başlığını başlık birimi olarak içe aktar import <vector>; // #include'dan daha hızlı (ön-derlenmiş) import "legacy.h"; // Kendi başlıklarınızı başlık birimi olarak içe aktar
Faydaları:
- Daha hızlı derleme (başlık bir kez ayrıştırılır, ikili olarak önbelleğe alınır)
- Başlık biriminden makro sızması yok (çoğunlukla)
- Kademeli göç yolu: #include → import'u birer birer değiştir
Sınırlamalar:
- Tüm başlıklar başlık birimi olarak içe aktarılamaz
- Makro ağırlıklı başlıklar (ör. Windows.h) çalışmayabilir
- Derleyici desteği değişkenlik gösterir (GCC/Clang hala gelişiyor)

THEORY: module :private Fragment

The private fragment lets you hide implementation in the interface file:
export module sensor;
export class Sensor { public: double read(); private: double calibration_; };
module :private; // Everything below is hidden from importers // AND doesn't affect ABI (changes don't cause recompile)
double Sensor::read() { return calibration_ * 0.95; // Implementation detail }

TEORİ: module :private Parçası

Private parça, arayüz dosyasında uygulama detaylarını gizlemenizi sağlar:
export module sensor;
export class Sensor { public: double read(); private: double calibration_; };
module :private; // Aşağıdaki her şey içe aktaranlardan gizlenir // VE ABI'yi etkilemez (değişiklikler yeniden derleme yapmaz)
double Sensor::read() { return calibration_ * 0.95; // Uygulama detayı }

THEORY: CMake Module Support

CMake 3.28+ supports C++20 modules natively:
cmake_minimum_required(VERSION 3.28) project(my_project CXX) set(CMAKE_CXX_STANDARD 20) add_library(math_lib) target_sources(math_lib PUBLIC FILE_SET CXX_MODULES FILES src/math.cppm # Module interface src/math_basic.cppm # Partition)
add_executable(app src/main.cpp) target_link_libraries(app PRIVATE math_lib)
Key CMake additions:
- `FILE_SET CXX_MODULES` — tells CMake these are module files
- CMake handles build order (interfaces must compile before consumers)
- Ninja generator works best (handles module dependency scanning)

TEORİ: CMake Modül Desteği

CMake 3.28+ C++20 modüllerini doğal olarak destekler:
cmake_minimum_required(VERSION 3.28) project(my_project CXX) set(CMAKE_CXX_STANDARD 20) add_library(math_lib) target_sources(math_lib PUBLIC FILE_SET CXX_MODULES FILES src/math.cppm # Modül arayüzü src/math_basic.cppm # Bölüm)
add_executable(app src/main.cpp) target_link_libraries(app PRIVATE math_lib)
Temel CMake eklemeleri:
- `FILE_SET CXX_MODULES` — CMake'e bunların modül dosyaları olduğunu söyler
- CMake derleme sırasını yönetir (arayüzler tþketicilerden önce derlenmeli)
- Ninja üreteci en iyi çalışır (modül bağımlılık taramasını yapar)

THEORY: Migration Strategy

Migrating from #include to import — recommended approach:
Phase 1: PREPARATION
- Ensure code compiles with C++20 (-std=c++20)
- Use CMake 3.28+ with Ninja generator
- Identify "leaf" libraries (no dependents to break)
Phase 2: HEADER UNITS (low risk)
- Replace `#include <vector>` with `import <vector>;`
- Replace `#include <string>` with `import <string>;`
- Immediate build speedup, minimal risk
Phase 3: NEW CODE AS MODULES
- Write all new components as modules
- Existing code still uses #include
Phase 4: GRADUAL CONVERSION
- Convert leaf libraries first (no reverse dependencies)
- Work inward toward the dependency graph center
- Keep both module and header interface temporarily (wrapper module)
Wrapper module pattern during transition:
// legacy_wrapper.cppm — wraps existing headers as a module export module legacy_wrapper; // We can still use #include inside a module! // (They won't leak to importers) #include "legacy_sensor.h" #include "legacy_can.h"
export using legacy::Sensor; export using legacy::CANFrame;

TEORİ: Göç Stratejisi

#include'dan import'a göç — önerilen yaklaşım:
Faz 1: HAZIRLIK
- Kodun C++20 ile derlendiğinden emin ol (-std=c++20)
- CMake 3.28+ Ninja üreteci ile kullan
- "Yaprak" kütüphaneleri belirle (kıracak bağımlı yok)
Faz 2: BAŞLIK BİRİMLERİ (düşük risk)
- `#include <vector>` → `import <vector>;` olarak değiştir
- `#include <string>` → `import <string>;` olarak değiştir
- Anında derleme hızlanması, minimum risk
Faz 3: YENİ KOD MODÜL OLARAK
- Tüm yeni bileşenleri modül olarak yaz
- Mevcut kod hala #include kullanır
Faz 4: KADEMELİ DÖNÜŞÜM
- Önce yaprak kütüphaneleri dönüştür (ters bağımlılık yok)
- Bağımlılık grafiğinin merkezine doğru ilerle
- Geçiçi olarak hem modül hem başlık arayüzünü koru (sarmalayıcı modül)
Geçiş sırasında sarmalayıcı modül deseni:
// legacy_wrapper.cppm — mevcut başlıkları modül olarak sarar export module legacy_wrapper; // Modül içinde hala #include kullanabiliriz! // (içe aktaranlara sızmaz) #include "legacy_sensor.h" #include "legacy_can.h" export using legacy::Sensor; export using legacy::CANFrame;

THEORY: Build Time Comparison

Real-world build time improvements with modules:

Scenario	Headers	Modules	Speedup
50 TUs, each includes <iostream>, <vector>, <string>, <map>	45 sec	12 sec	3.8x
200 TUs, project headers (lots of template code)	180 sec	35 sec	5.1x
Incremental (1 file changed) (header included by many)	12 sec	3 sec	4.0x

Why modules are faster:
1. Parsed ONCE, stored as BMI (Binary Module Interface)
2. No redundant parsing across TUs (headers parsed N times = $O(N^2)$)
3. No macro/include-order dependencies to invalidate caches

TEORİ: Derleme Süresi Karşılaştırması

Modüllerle gerçek derleme süresi iyileştirmeleri:

```
┌──────────────────────────┬──────────┬──────────┬──────────┐
│ Senaryo                  │ Başlıklar │ Modüller  │ Hızlanma  │
├──────────────────────────┼──────────┼──────────┼──────────┤
│ 50 TU, her biri          │ 45 sn    │ 12 sn    │ 3.8x     │
│   <iostream>, <vector>,  │          │          │          │
│   <string>, <map> içerir │          │          │          │
│                          │          │          │          │
│ 200 TU, proje başlıkları │ 180 sn   │ 35 sn    │ 5.1x     │
│   (çok sayıda şablon kod)│          │          │          │
│                          │          │          │          │
│ Artımlı (1 dosya değişti)│ 12 sn    │ 3 sn     │ 4.0x     │
│   (başlık çok yerde dahil)│         │          │          │
└──────────────────────────┴──────────┴──────────┴──────────┘
```

Modüller neden daha hızlı:
1. Bir kez ayrıştırılır, BMI (Binary Module Interface) olarak saklanır
2. TU'lar arasında tekrarlı ayrıştırma yok (başlıklar N kez = $O(N^2)$)
3. Önbellekleri geçersiz kılan makro/include-sırası bağımlılıkları yok

→ Ref: https://en.cppreference.com/w/cpp/language/modules
→ Ref: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1103r3.pdf

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <chrono>
#include <functional>
#include <memory>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: Simulating Module-Like Design in C++20 (without actual modules)
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: Since compiling actual modules requires a full build system setup,
 *     we simulate module patterns here using namespaces with explicit
 *     "export" (public) vs "internal" (private) separation.
 *
 *     In a real module:
 *       export class Sensor { ... };      → module_api::Sensor
 *       class InternalHelper { ... };     → module_impl::InternalHelper
 *
 * TR: Gerçek modül derlemesi tam bir yapı sistemi kurulumu gerektirdiğinden,
 *     burada ad alanları kullanarak modül kalıplarını simüle ediyoruz.
 */

// EN: This namespace represents what would be `export`-ed from a module
// TR: Bu ad alanı, bir modülden `export` edilecek olanları temsil eder
namespace sensor_module {

    // EN: EXPORTED types — visible to anyone who `import`s the module
    // TR: DIŞA AKTARILAN türler — modülü `import` eden herkes görebilir
    struct SensorReading {
        double value;
        std::string unit;
        uint64_t timestamp_ms;
    };
```

```cpp

    // EN: EXPORTED class / TR: DIŞA AKTARILAN sınıf
    class TemperatureSensor {
    public:
        explicit TemperatureSensor(std::string id)
            : id_(std::move(id)), calibration_offset_(0.0) {}

        SensorReading read() const {
            // EN: Simulate reading / TR: Okuma simülasyonu
            double raw = 22.5 + calibration_offset_;
            return {raw, "°C", current_time_ms()};
        }

        const std::string& id() const { return id_; }
        void set_calibration(double offset) { calibration_offset_ = offset; }

    private:
        std::string id_;
        double calibration_offset_;

        static uint64_t current_time_ms() {
            auto now = std::chrono::steady_clock::now();
            return static_cast<uint64_t>(
                std::chrono::duration_cast<std::chrono::milliseconds>(
                    now.time_since_epoch()).count());
        }
    };

    // EN: EXPORTED function / TR: DIŞA AKTARILAN fonksiyon
    std::string format_reading(const SensorReading& r) {
        return r.unit + ": " + std::to_string(r.value)
               + " @ " + std::to_string(r.timestamp_ms) + "ms";
    }

}  // namespace sensor_module

// EN: This namespace represents module-INTERNAL symbols (NOT exported)
// TR: Bu ad alanı modül-İÇ sembollerini temsil eder (DIŞA AKTARILMAZ)
namespace sensor_module_impl {

    // EN: Internal helper — would NOT be visible to importers in a real module
    // TR: İç yardımcı — gerçek modülde importerlara görünmez
    double apply_moving_average(double current, double previous, double alpha = 0.3) {
        return alpha * current + (1.0 - alpha) * previous;
    }

}  // namespace sensor_module_impl


// ═══════════════════════════════════════════════════════════════════════════
// PART 2: Demonstrating Module Partition Pattern
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: Simulating module partitions using nested namespaces:
 *     - ecu_module::diagnostics corresponds to `module ecu:diagnostics`
 *     - ecu_module::communication corresponds to `module ecu:communication`
 *     - ecu_module (primary) re-exports both
 *
 * TR: İç içe ad alanları ile modül bölümlerini simüle etme.
 */
namespace ecu_module {

    // EN: Partition: ecu:diagnostics / TR: Bölüm: ecu:diagnostics
    namespace diagnostics {
        enum class DTCStatus : uint8_t {
            Inactive = 0x00,
            Active   = 0x01,
            Pending  = 0x02,
            Confirmed = 0x03
        };

        struct DiagnosticTroubleCode {
            uint32_t code;
            DTCStatus status;
            std::string description;
        };
```

```

        std::string dtc_to_string(const DiagnosticTroubleCode& dtc) {
            std::string status_str;
            switch (dtc.status) {
                case DTCStatus::Inactive:  status_str = "INACTIVE"; break;
                case DTCStatus::Active:    status_str = "ACTIVE"; break;
                case DTCStatus::Pending:   status_str = "PENDING"; break;
                case DTCStatus::Confirmed: status_str = "CONFIRMED"; break;
            }
            return "DTC-" + std::to_string(dtc.code) + " [" + status_str + "] "
                   + dtc.description;
        }
    }

    // EN: Partition: ecu:communication / TR: Bölüm: ecu:communication
    namespace communication {
        struct CANMessage {
            uint32_t id;
            uint8_t data[8];
            uint8_t length;
        };

        void pack_dtc_into_can(const diagnostics::DiagnosticTroubleCode& dtc,
                               CANMessage& msg) {
            msg.id = 0x7DF;  // EN: OBD-II functional address
            msg.length = 4;
            msg.data[0] = 0x03;  // EN: Number of data bytes
            msg.data[1] = 0x19;  // EN: Read DTC by status
            msg.data[2] = static_cast<uint8_t>(dtc.status);
            msg.data[3] = static_cast<uint8_t>(dtc.code & 0xFF);
        }
    }

}  // namespace ecu_module

// ═══════════════════════════════════════════════════════════════════════
// PART 3: Module :private Fragment Simulation
// ═══════════════════════════════════════════════════════════════════════

/**
 * EN: The `module :private;` fragment allows implementation in the interface
 *     file — but changes to it DON'T cause consumers to recompile.
 *     We simulate this by separating the class declaration from its definition.
 *
 * TR: `module :private;` parçası, arayüz dosyasında uygulamaya izin verir —
 *     ama değişiklikler tüketicilerin yeniden derlenmesine neden OLMAZ.
 */
namespace config_module {

    // EN: "Interface" part — this is what consumers depend on (ABI)
    // TR: "Arayüz" kısmı — tüketicilerin bağımlı olduğu kısım (ABI)
    class ConfigManager {
    public:
        ConfigManager();
        void set(const std::string& key, const std::string& value);
        std::string get(const std::string& key, const std::string& default_val = "") const;
        size_t size() const;
        void print_all() const;

    private:
        // EN: Using PIMPL would be even better for ABI stability
        // TR: ABI kararlılığı için PIMPL kullanmak daha iyi olurdu
        struct Entry {
            std::string key;
            std::string value;
        };
        std::vector<Entry> entries_;
    };

    // ── module :private; equivalent ──
    // EN: Everything below is implementation — changes here DON'T affect
    //     consumers' ABI or force recompilation in a real module.
    // TR: Aşağıdaki her şey uygulama — buradaki değişiklikler gerçek bir
    //     modülde tüketicilerin ABI'sini etkilemez veya yeniden derlemez.

    ConfigManager::ConfigManager() = default;

```

```
    void ConfigManager::set(const std::string& key, const std::string& value) {
        for (auto& e : entries_) {
            if (e.key == key) { e.value = value; return; }
        }
        entries_.push_back({key, value});
    }

    std::string ConfigManager::get(const std::string& key, const std::string& default_val) const {
        for (const auto& e : entries_) {
            if (e.key == key) return e.value;
        }
        return default_val;
    }

    size_t ConfigManager::size() const { return entries_.size(); }

    void ConfigManager::print_all() const {
        for (const auto& e : entries_) {
            std::cout << "    " << e.key << " = " << e.value << "\n";
        }
    }

}  // namespace config_module

// ═══════════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "═══════════════════════════════════════════\n";
    std::cout << "   Module 07 - C++20 Modules Deep Dive\n";
    std::cout << "═══════════════════════════════════════════\n\n";

    // ─── Demo 1: Module Export vs Internal ──────────────────────────
    {
        std::cout << "--- Demo 1: Module Export vs Internal ---\n";

        // EN: Using "exported" API / TR: "Dışa aktarılan" API'yi kullanma
        sensor_module::TemperatureSensor sensor("TEMP_ECU_01");
        sensor.set_calibration(0.5);
        auto reading = sensor.read();

        std::cout << "  Sensor ID: " << sensor.id() << "\n";
        std::cout << "  Reading:   " << sensor_module::format_reading(reading) << "\n";

        // EN: Internal helper — in a real module, this would NOT compile from outside!
        // TR: İç yardımcı — gerçek modülde dışarıdan DERLENMEZDİ!
        double smoothed = sensor_module_impl::apply_moving_average(reading.value, 21.0);
        std::cout << "  Smoothed:  " << smoothed << " (internal helper — would fail with real modules)\n\n";

        std::cout << "  Module syntax equivalent:\n";
        std::cout << "    export module sensor;                    // Interface\n";
        std::cout << "    export class TemperatureSensor { ... };  // Exported\n";
        std::cout << "    double apply_moving_average(...);        // NOT exported\n\n";
    }

    // ─── Demo 2: Module Partitions ──────────────────────────────────
    {
        std::cout << "--- Demo 2: Module Partitions ---\n";

        ecu_module::diagnostics::DiagnosticTroubleCode dtc{
            0x0123,
            ecu_module::diagnostics::DTCStatus::Active,
            "Coolant temp sensor circuit"
        };

        std::cout << "  " << ecu_module::diagnostics::dtc_to_string(dtc) << "\n";

        ecu_module::communication::CANMessage msg{};
        ecu_module::communication::pack_dtc_into_can(dtc, msg);
        std::cout << "  Packed into CAN ID: 0x" << std::hex << msg.id << std::dec
                  << ", length: " << static_cast<int>(msg.length) << "\n\n";

        std::cout << "  Module syntax equivalent:\n";
        std::cout << "    export module ecu:diagnostics;   // Partition 1\n";
        std::cout << "    export module ecu:communication;  // Partition 2\n";
        std::cout << "    export module ecu;                // Primary interface\n";
```

```
        std::cout << "    export import :diagnostics;       // Re-export\n";
        std::cout << "    export import :communication;     // Re-export\n\n";
    }

    // ── Demo 3: module :private Fragment ──────────────────────────────
    {
        std::cout << "--- Demo 3: module :private (Implementation Hiding) ---\n";

        config_module::ConfigManager cfg;
        cfg.set("ecu.variant", "ECU_VARIANT_A");
        cfg.set("can.baudrate", "500000");
        cfg.set("diag.timeout_ms", "5000");

        std::cout << "  Config entries (" << cfg.size() << "):\n";
        cfg.print_all();
        std::cout << "  Lookup: diag.timeout_ms = " << cfg.get("diag.timeout_ms") << "\n";
        std::cout << "  Lookup: missing_key = \"" << cfg.get("missing_key", "N/A") << "\"\n\n";

        std::cout << "  Module syntax equivalent:\n";
        std::cout << "    export module config;\n";
        std::cout << "    export class ConfigManager { ... };  // Public ABI\n";
        std::cout << "    module :private;                      // Below = hidden impl\n";
        std::cout << "    void ConfigManager::set(...) { ... } // Changes don't recompile consumers\n\n";
    }

    // ── Demo 4: Header Units ──────────────────────────────────────────
    {
        std::cout << "--- Demo 4: Header Units Reference ---\n";
        std::cout << "  Traditional:     #include <iostream>\n";
        std::cout << "  Header unit:     import <iostream>;    // Faster, no macro leakage\n";
        std::cout << "  Own header:      import \"my_header.h\"; // Gradual migration\n\n";
        std::cout << "  Compiler flags:\n";
        std::cout << "    GCC:   g++ -std=c++20 -fmodules-ts main.cpp\n";
        std::cout << "    Clang: clang++ -std=c++20 -fmodules main.cpp\n";
        std::cout << "    MSVC:  cl /std:c++20 /interface /TP module.ixx\n\n";
    }

    // ── Demo 5: Migration Decision Tree ───────────────────────────────
    {
        std::cout << "--- Demo 5: Migration Decision Tree ---\n";
        std::cout << "  ┌── Is it a NEW component?\n";
        std::cout << "  │   YES → Write as a module from day one\n";
        std::cout << "  │   NO  → Continue with headers for now\n";
        std::cout << "  │\n";
        std::cout << "  ├── Is build time a bottleneck?\n";
        std::cout << "  │   YES → Start with header units (import <vector>;)\n";
        std::cout << "  │   NO  → Lower priority for migration\n";
        std::cout << "  │\n";
        std::cout << "  ├── Does the component have many dependents?\n";
        std::cout << "  │   YES → Migrate last (too risky early)\n";
        std::cout << "  │   NO  → Good candidate for early migration\n";
        std::cout << "  │\n";
        std::cout << "  └── Is the header macro-heavy?\n";
        std::cout << "      YES → Keep as header (modules don't export macros)\n";
        std::cout << "      NO  → Safe to convert to module\n\n";
    }

    // ── Demo 6: Complete Module File Examples ─────────────────────────
    {
        std::cout << "--- Demo 6: Complete Module File Examples ---\n\n";

        std::cout << "  === Example: math_utils.cppm (Interface) ===\n";
        std::cout << "  export module math_utils;\n";
        std::cout << "  \n";
        std::cout << "  export namespace math {\n";
        std::cout << "      constexpr double pi = 3.14159265358979;\n";
        std::cout << "      constexpr double e  = 2.71828182845904;\n";
        std::cout << "      \n";
        std::cout << "      double sin(double x);\n";
        std::cout << "      double cos(double x);\n";
        std::cout << "      \n";
        std::cout << "      template<typename T>\n";
        std::cout << "      T clamp(T value, T low, T high) {\n";
        std::cout << "          return (value < low) ? low : (value > high) ? high : value;\n";
        std::cout << "      }\n";
        std::cout << "  }\n\n";
```

```

        std::cout << "  === Example: math_utils.cpp (Implementation) ===\n";
        std::cout << "  module math_utils;         // No 'export' keyword\n";
        std::cout << "  #include <cmath>           // OK: #include inside module impl\n";
        std::cout << "  \n";
        std::cout << "  namespace math {\n";
        std::cout << "      double sin(double x) { return std::sin(x); }\n";
        std::cout << "      double cos(double x) { return std::cos(x); }\n";
        std::cout << "  }\n\n";

        std::cout << "  === Example: CMakeLists.txt ===\n";
        std::cout << "  cmake_minimum_required(VERSION 3.28)\n";
        std::cout << "  project(my_project CXX)\n";
        std::cout << "  set(CMAKE_CXX_STANDARD 20)\n";
        std::cout << "  add_library(math_lib)\n";
        std::cout << "  target_sources(math_lib\n";
        std::cout << "      PUBLIC FILE_SET CXX_MODULES FILES\n";
        std::cout << "          src/math_utils.cppm)\n";
        std::cout << "  target_sources(math_lib PRIVATE\n";
        std::cout << "      src/math_utils.cpp)\n\n";
    }

    std::cout << "==========================================\n";
    std::cout << "   End of C++20 Modules\n";
    std::cout << "==========================================\n";

    return 0;
}
```

► 10. Linking Deepdive

module_07_under_the_hood/10_linking_deepdive.cpp

Konu: Linking: Static, Dynamic, LTO — Bağlama Derinlemesine

Teori / Theory

THEORY: What Is Linking?

Linking is the FINAL stage of the build pipeline. The linker (ld) takes multiple object files (.o) and combines them into one executable or library.
Compilation produces .o files with UNRESOLVED symbols: main.o: calls "parse_frame" but doesn't define it parser.o: defines "parse_frame" Linker's job: match every call to its definition → executable.
Two types of linking:
1. STATIC LINKING: Library code is COPIED into the executable at build time
2. DYNAMIC LINKING: Library code stays in a separate .so/.dll file, loaded at runtime by the OS's dynamic linker (ld-linux.so)

TEORİ: Bağlama Nedir?

Bağlama, derleme hattının SON aşamasıdır. Bağlayıcı (ld) birden fazla nesne dosyasını (.o) alıp tek bir çalıştırılabilir veya kütüphane oluşturur.
Derleme ÇÖZÜLMEMİŞ sembollerle .o dosyaları üretir: main.o: "parse_frame" çağırır ama tanımlamaz parser.o: "parse_frame" tanımlar Bağlayıcının görevi: her çağrıyı tanımıyla eşle → çalıştırılabilir.
İki tür bağlama:
1. STATİK BAĞLAMA: Kütüphane kodu derleme zamanında exe'ye KOPYALANIR
2. DİNAMİK BAĞLAMA: Kütüphane kodu ayrı .so/.dll dosyasında kalır, çalışma zamanında OS'un dinamik bağlayıcısı (ld-linux.so) tarafından yüklenir

THEORY: Static vs Dynamic Libraries

Feature	Static (.a / .lib)	Dynamic (.so / .dll)
How it works	Code copied into exe	Code loaded at runtime
File size	Larger executable	Smaller exe + .so files
Deployment	Single file (easy)	Must ship .so files too
Update library	Must RECOMPILE app	Just replace .so file
Memory (N apps)	N copies in memory	ONE copy shared by all
Load time	Faster (no runtime link)	Slightly slower
Symbol visibility	All symbols included	Only exported symbols
Use case	Embedded/firmware, tests	Plugins, system libs

TEORİ: Statik vs Dinamik Kütüphaneler

Özellik	Statik (.a / .lib)	Dinamik (.so / .dll)
Nasıl çalışır	Kod exe'ye kopyalanır	Kod çal. zamanında yükl.
Dosya boyutu	Büyük çalıştırılabilir	Küçük exe + .so dosyaları
Dağıtım	Tek dosya (kolay)	.so dosyaları da gerekli
Kütüphane güncelle	Uygulamayı YENİDEN DERLE	Sadece .so'yu değiştir
Bellek (N uygulama)	Bellekte N kopya	Hepsi BİR kopyayı paylaşır
Yükleme süresi	Daha hızlı (ç.z. bağlamaø)	Biraz daha yavaş
Sembol görünürlüğü	Tüm semboller dahil	Sadece dışa aktarılanlar
Kullanım alanı	Gömülü/firmware, testler	Eklentiler, sistem kütph.

THEORY: Creating Static Libraries

Step 1: Compile source to object files g++ -c -o math.o math.cpp g++ -c -o utils.o utils.cpp
Step 2: Archive objects into .a file using `ar` ar rcs libmymath.a math.o utils.o ↑ r=replace, c=create, s=index (for faster linking)
Step 3: Link executable against the static library g++ -o app main.cpp -L. -lmymath ↑ -L. = look in current dir, -lmymath = link libmymath.a
CMake: add_library(mymath STATIC math.cpp utils.cpp) target_link_libraries(app PRIVATE mymath)

TEORİ: Statik Kütüphane Oluşturma

Adım 1: Kaynağı nesne dosyalarına derle g++ -c -o math.o math.cpp g++ -c -o utils.o utils.cpp
Adım 2: `ar` ile nesneleri .a dosyasına arşivle ar rcs libmymath.a math.o utils.o ↑ r=değiştir, c=oluştur, s=indeks (daha hızlı bağlama için)
Adım 3: Statik kütüphaneye karşı çalıştırılabiliri bağla g++ -o app main.cpp -L. -lmymath ↑ -L. = mevcut dizinde ara, -lmymath = libmymath.a'yı bağla
CMake: add_library(mymath STATIC math.cpp utils.cpp) target_link_libraries(app PRIVATE mymath)

THEORY: Creating Dynamic Libraries

Step 1: Compile with -fPIC (Position-Independent Code) g++ -c -fPIC -o math.o math.cpp ↑ -fPIC = code can run at ANY address (required for .so)
Step 2: Create shared library g++ -shared -o libmymath.so math.o utils.o
Step 3: Link executable g++ -o app main.cpp -L. -lmymath
Step 4: Runtime — OS must find the .so file! export LD_LIBRARY_PATH=.:$LD_LIBRARY_PATH (temporary) sudo ldconfig (system-wide) g++ -o app main.cpp -L. -lmymath -Wl,-rpath,. (embed path)
CMake: add_library(mymath SHARED math.cpp utils.cpp)

TEORİ: Dinamik Kütüphane Oluşturma

Adım 1: -fPIC ile derle (Pozisyondan Bağımsız Kod) g++ -c -fPIC -o math.o math.cpp ↑ -fPIC = kod HERHANGİ bir adreste çalışabilir (.so için gerekli)
Adım 2: Paylaşımlı kütüphane oluştur g++ -shared -o libmymath.so math.o utils.o
Adım 3: Çalıştırılabiliri bağla g++ -o app main.cpp -L. -lmymath
Adım 4: Çalışma zamanı — OS .so dosyasını bulmalı! export LD_LIBRARY_PATH=.:$LD_LIBRARY_PATH (geçici) sudo ldconfig (sistem çapında) g++ -o app main.cpp -L. -lmymath -Wl,-rpath,. (yolu göm)
CMake: add_library(mymath SHARED math.cpp utils.cpp)

THEORY: Symbol Visibility

By default on Linux, ALL symbols in a .so are exported (visible). This is a security and performance problem. Best practice: hide everything by default, export only what's needed.
Method 1: Compiler flag g++ -fvisibility=hidden -shared -o lib.so Then mark exported functions: __attribute__((visibility("default"))) void my_api_function();
Method 2: Linker version script g++ -shared -Wl,--version-script=exports.map -o lib.so # exports.map: { global: my_api_*; local: *; };
Method 3: CMake macro (portable) #if defined(_WIN32) #define MY_API __declspec(dllexport) #elif defined(__GNUC__) #define MY_API __attribute__((visibility("default"))) #else #define MY_API #endif

TEORİ: Sembol Görünürlüğü

Linux'ta varsayılan olarak .so'daki TÜM semboller dışa aktarılır (görünür). Bu bir güvenlik ve performans problemidir. En iyi uygulama: varsayılan olarak gizle, sadece gerekenleri dışa aktar.
Yöntem 1: Derleyici bayrağı g++ -fvisibility=hidden -shared -o lib.so Sonra dışa aktarılan fonksiyonları işaretle: __attribute__((visibility("default"))) void my_api_function();
Yöntem 2: Bağlayıcı sürüm betiği g++ -shared -Wl,--version-script=exports.map -o lib.so # exports.map: { global: my_api_*; local: *; };
Yöntem 3: CMake makrosu (taşınabilir) #if defined(_WIN32) #define MY_API __declspec(dllexport) #elif defined(__GNUC__) #define MY_API __attribute__((visibility("default"))) #else #define MY_API #endif

THEORY: rpath vs LD_LIBRARY_PATH vs ldconfig

Three ways the dynamic linker finds .so files at runtime:
1. LD_LIBRARY_PATH (environment variable):
- Checked first by ld-linux.so
- Temporary, per-session
- DON'T use in production (security risk, fragile)
2. rpath (embedded in executable):
- Stored in ELF header of the executable
- Set at link time: -Wl,-rpath,/usr/local/lib
- $ORIGIN trick: -Wl,-rpath,'$ORIGIN/../lib' (relative to exe)
- BEST for application-specific libraries
3. ldconfig (system cache):
- Reads /etc/ld.so.conf and /etc/ld.so.conf.d/*.conf
- Builds cache at /etc/ld.so.cache
- BEST for system-wide libraries
- Run: sudo ldconfig after installing .so to /usr/local/lib

TEORİ: rpath, LD_LIBRARY_PATH ve ldconfig Karşılaştırması

Dinamik bağlayıcının çalışma zamanında .so dosyalarını bulmasının 3 yolu:
1. LD_LIBRARY_PATH (ortam değişkeni):
- ld-linux.so tarafından önce kontrol edilir
- Geçici, oturum başına
- Üretimde KULLANMA (güvenlik riski, kırılgan)
2. rpath (exe'ye gömülü):
- Çalıştırılabilirin ELF başlığında saklanır
- Bağlama zamanında ayarlanır: -Wl,-rpath,/usr/local/lib
- $ORIGIN hilesi: -Wl,-rpath,'$ORIGIN/../lib' (exe'ye göre bağıl)
- Uygulamaya özel kütüphaneler için EN İYİ
3. ldconfig (sistem önbelleği):
- /etc/ld.so.conf ve /etc/ld.so.conf.d/*.conf okur
- /etc/ld.so.cache'de önbellek oluşturur
- Sistem çapında kütüphaneler için EN İYİ
- .so'yu /usr/local/lib'e yükledikten sonra: sudo ldconfig

THEORY: Library Versioning (soname)

Linux shared libraries use a versioning scheme:
libmymath.so.2.1.0 ← Real file (MAJOR.MINOR.PATCH) libmymath.so.2 ← Symlink (soname = MAJOR version) libmymath.so ← Symlink (development link)
Rules:
- MAJOR: ABI break (existing code must recompile)
- MINOR: New features, backward compatible
- PATCH: Bug fixes only
Setting soname: g++ -shared -Wl,-soname,libmymath.so.2 -o libmymath.so.2.1.0 ln -s libmymath.so.2.1.0 libmymath.so.2 ln -s libmymath.so.2 libmymath.so
CMake handles this automatically: set_target_properties(mymath PROPERTIES VERSION 2.1.0 SOVERSION 2)

TEORİ: Kütüphane Sürümleme

Linux paylaşımlı kütüphaneleri sürümleme şeması kullanır:
libmymath.so.2.1.0 ← Gerçek dosya (MAJOR.MINOR.PATCH) libmymath.so.2 ← Sembolik bağ (soname = MAJOR sürüm) libmymath.so ← Sembolik bağ (geliştirme bağlantısı)
Kurallar:
- MAJOR: ABI kırılması (mevcut kod yeniden derlenmeli)
- MINOR: Yeni özellikler, geriye uyumlu
- PATCH: Sadece hata düzeltmeleri

soname ayarlama: g++ -shared -Wl,-soname,libmymath.so.2 -o libmymath.so.2.1.0 ln -s libmymath.so.2.1.0 libmymath.so.2 ln -s libmymath.so.2 libmymath.so
CMake bunu otomatik yönetir: set_target_properties(mymath PROPERTIES VERSION 2.1.0 SOVERSION 2)

THEORY: LTO — Link-Time Optimization

LTO lets the compiler optimize ACROSS translation unit boundaries. Normally, the compiler optimizes each .cpp file independently. With LTO, the linker sees ALL code and can:
- Inline functions across .cpp files
- Remove unused code more aggressively (dead code elimination)
- Propagate constants across files
- Devirtualize virtual calls when the type is known

Types of LTO:

```
┌──────────┬──────────────────────────────────────────┐
│ Type     │ Description                              │
├──────────┼──────────────────────────────────────────┤
│ Full LTO │ All TUs merged into one, then optimized  │
│ (-flto)  │ □ Best optimization □ Slow, high memory (~2x)  │
│          │                                          │
│ Thin LTO │ TUs optimized in parallel with cross-TU info  │
│ (-flto=thin│ □ Fast, parallel □ Slightly less optimal  │
└──────────┴──────────────────────────────────────────┘
```

Usage: g++ -flto -O2 -c main.cpp math.cpp # Compile with LTO g++ -flto -O2 main.o math.o -o app # Link with LTO
Thin LTO (Clang only): clang++ -flto=thin -O2 -c main.cpp math.cpp clang++ -flto=thin -O2 main.o math.o -o app
CMake: set(CMAKE_INTERPROCEDURAL_OPTIMIZATION ON) # Enables -flto # Or per-target: set_target_properties(app PROPERTIES INTERPROCEDURAL_OPTIMIZATION ON)
Typical improvements: 5–20% faster, 10–30% smaller binary.

TEORİ: Bağlama Zamanı Optimizasyonu

LTO, derleyicinin çeviri birimi sınırları ARASINDA optimize etmesini sağlar. Normalde derleyici her .cpp dosyasını bağımsız optimize eder. LTO ile bağlayıcı TÜM kodu görür ve şunları yapabilir:
- .cpp dosyaları arasında fonksiyon satır içi açma
- Kullanılmayan kodu daha agresif kaldırma (ölü kod eleme)
- Dosyalar arasında sabit yayma
- Tür biliniyorsa sanal çağrıları doğrudanlaştırma

LTO türleri:

```
┌──────────┬──────────────────────────────────────────┐
│ Tür      │ Açıklama                                 │
├──────────┼──────────────────────────────────────────┤
│ Tam LTO  │ Tüm TU'lar birleştirilir, sonra optimize edilir  │
│ (-flto)  │ En iyi optimizasyon. Yavaş, yüksek bellek (~2x)  │
│          │                                          │
│ İnce LTO │ TU'lar çapraz-TU bilgisiyle paralel optimize edilir │
│(-flto=thin)│ Hızlı, paralel. Biraz daha az optimal    │
└──────────┴──────────────────────────────────────────┘
```

Kullanım: g++ -flto -O2 -c main.cpp math.cpp # LTO ile derle g++ -flto -O2 main.o math.o -o app # LTO ile bağla
İnce LTO (sadece Clang): clang++ -flto=thin -O2 -c main.cpp math.cpp clang++ -flto=thin -O2 main.o math.o -o app
CMake: set(CMAKE_INTERPROCEDURAL_OPTIMIZATION ON) # -flto'yu etkinleştirir # Veya hedef başına: set_target_properties(app PROPERTIES INTERPROCEDURAL_OPTIMIZATION ON)
Tipik iyileştirmeler: %5-20 daha hızlı, %10-30 daha küçük ikili.

→ Ref: https://gcc.gnu.org/wiki/LinkTimeOptimization
→ Ref: https://man7.org/linux/man-pages/man8/ldconfig.8.html

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <string>
#include <cstdlib>
#include <dlfcn.h>    // EN: dlopen/dlsym for runtime dynamic loading / TR: Çalışma zamanı dinamik yükleme

// ═══════════════════════════════════════════════════════════════════════
// PART 1: Static Library Simulation
// ═══════════════════════════════════════════════════════════════════════

/**
 * EN: These would normally live in a separate .cpp file compiled into libmath.a
 *     We keep them here for single-file demonstration.
 *
 * TR: Bunlar normalde ayrı bir .cpp'de libmath.a'ya derlenir.
 *     Tek dosya gösterimi için burada tutuyoruz.
 */
namespace static_lib_demo {

    // EN: "Library" functions — imagine these are in math_lib.cpp → libmath.a
    // TR: "Kütüphane" fonksiyonları — bunların math_lib.cpp → libmath.a'da olduğunu düşünün
    int add(int a, int b) { return a + b; }
    int multiply(int a, int b) { return a * b; }

```

```
24      void demonstrate() {
25          std::cout << "  Static library symbol resolution:\n";
26          std::cout << "    add(10, 20)      = " << add(10, 20) << "\n";
27          std::cout << "    multiply(6, 7)   = " << multiply(6, 7) << "\n\n";
28
29          std::cout << "  Build commands:\n";
30          std::cout << "    # Step 1: Compile to .o\n";
31          std::cout << "    g++ -c -o math.o math.cpp\n";
32          std::cout << "    # Step 2: Archive into .a\n";
33          std::cout << "    ar rcs libmath.a math.o\n";
34          std::cout << "    # Step 3: Link\n";
35          std::cout << "    g++ -o app main.cpp -L. -lmath\n\n";
36      }
37  }
38
39  // ═══════════════════════════════════════════════════════════════════════
40  // PART 2: Dynamic Library Runtime Loading (dlopen/dlsym)
41  // ═══════════════════════════════════════════════════════════════════════
42
43  /**
44   * EN: dlopen/dlsym let you load shared libraries AT RUNTIME (not at link time).
45   *     This is the foundation of plugin systems:
46   *
47   *     void* handle = dlopen("libplugin.so", RTLD_LAZY);
48   *     auto func = (func_ptr)dlsym(handle, "create_plugin");
49   *     Plugin* p = func();
50   *     // ... use plugin ...
51   *     dlclose(handle);
52   *
53   *     Flags for dlopen:
54   *     - RTLD_LAZY:  Resolve symbols on first use (faster startup)
55   *     - RTLD_NOW:   Resolve all symbols immediately (safer, catches errors early)
56   *     - RTLD_GLOBAL: Make symbols available to subsequently loaded libraries
57   *
58   * TR: dlopen/dlsym, paylaşımlı kütüphaneleri ÇALIŞMA ZAMANINDA yüklemenizi sağlar.
59   *     Bu, eklenti (plugin) sistemlerinin temelidir.
60   */
61  void demonstrate_dlopen() {
62      std::cout << "  Runtime dynamic loading (dlopen/dlsym):\n";
63
64      // EN: Try to load libm.so (math library — always available on Linux)
65      // TR: libm.so'yu yüklemeyi dene (matematik kütüphanesi — Linux'ta her zaman mevcut)
66      void* handle = dlopen("libm.so.6", RTLD_LAZY);
67      if (!handle) {
68          std::cout << "    dlopen failed: " << dlerror() << "\n";
69          return;
70      }
71
72      // EN: Look up the cos() function by name
73      // TR: cos() fonksiyonunu isimle ara
74      using cos_func_t = double(*)(double);
75      cos_func_t cos_fn = reinterpret_cast<cos_func_t>(dlsym(handle, "cos"));
76
77      const char* error = dlerror();
78      if (error) {
79          std::cout << "    dlsym failed: " << error << "\n";
80          dlclose(handle);
81          return;
82      }
83
84      // EN: Call the dynamically loaded function
85      // TR: Dinamik olarak yüklenen fonksiyonu çağır
86      double result = cos_fn(0.0);
87      std::cout << "    Loaded cos() from libm.so.6 via dlsym\n";
88      std::cout << "    cos(0.0) = " << result << "\n";
89
90      dlclose(handle);
91      std::cout << "    dlclose() called — library unloaded\n\n";
92  }
93
```

```
// ================================================================================
// PART 3: Symbol Visibility Example
// ================================================================================

/**
 * EN: Demonstrating visibility attributes.
 *     In a real shared library compiled with -fvisibility=hidden:
 *     - Functions WITHOUT the attribute are hidden (internal only)
 *     - Functions WITH __attribute__((visibility("default"))) are exported
 *
 * TR: Görünürlük öznitelikleri gösterme.
 *     -fvisibility=hidden ile derlenen gerçek paylaşımlı kütüphanede:
 *     - Öznitelik OLMAYAN fonksiyonlar gizlidir
 *     - __attribute__((visibility("default"))) OLAN fonksiyonlar dışa aktarılır
 */

// EN: Cross-platform export macro
// TR: Platformlar arası dışa aktarma makrosu
#if defined(_WIN32)
    #define MY_API __declspec(dllexport)
#elif defined(__GNUC__)
    #define MY_API __attribute__((visibility("default")))
#else
    #define MY_API
#endif

// EN: EXPORTED — visible to users of the library
// TR: DIŞA AKTARILAN — kütüphane kullanıcılarına görünür
MY_API int public_api_function(int x) {
    return x * 2;
}

// EN: HIDDEN — internal implementation, not visible outside the library
// TR: GİZLİ — iç uygulama, kütüphane dışında görünmez
static int internal_helper(int x) {
    return x + 1;
}

// ================================================================================
// PART 4: Inspecting Binaries
// ================================================================================

/**
 * EN: Essential commands for inspecting compiled binaries:
 *
 *     # See symbols in a binary or library:
 *     nm -C libmath.a           # -C = demangle C++ names
 *     nm -D libmath.so          # -D = dynamic symbols only
 *
 *     # See shared library dependencies:
 *     ldd ./my_app              # Lists all required .so files
 *     readelf -d ./my_app       # ELF dynamic section (rpath, soname)
 *
 *     # See symbol visibility in a .so:
 *     objdump -T libmath.so     # Dynamic symbol table
 *
 *     # See rpath embedded in executable:
 *     readelf -d ./my_app | grep -i path
 *
 *     # Check if LTO was used:
 *     readelf -S main.o | grep gnu.lto
 *
 * TR: Derlenmiş ikili dosyaları incelemek için temel komutlar (yukarıda).
 */

// ================================================================================
// MAIN
// ================================================================================
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 07 - Linking Deep Dive\n";
    std::cout << "============================================\n\n";

```

```cpp
    // ── Demo 1: Static Library ──────────────────────────────────────
    {
        std::cout << "--- Demo 1: Static Library (.a) ---\n";
        static_lib_demo::demonstrate();
    }

    // ── Demo 2: Dynamic Loading ─────────────────────────────────────
    {
        std::cout << "--- Demo 2: Dynamic Loading (dlopen/dlsym) ---\n";
        demonstrate_dlopen();
    }

    // ── Demo 3: Symbol Visibility ───────────────────────────────────
    {
        std::cout << "--- Demo 3: Symbol Visibility ---\n";
        std::cout << "  public_api_function(21) = " << public_api_function(21) << " (exported)\n";
        std::cout << "  internal_helper(21)     = " << internal_helper(21) << " (hidden/static)\n\n";

        std::cout << "  Visibility macro:\n";
        std::cout << "    #define MY_API __attribute__((visibility(\"default")))\n";
        std::cout << "    Compile with: g++ -fvisibility=hidden -shared ...\n\n";
    }

    // ── Demo 4: LTO Reference ───────────────────────────────────────
    {
        std::cout << "--- Demo 4: LTO (Link-Time Optimization) ---\n";
        std::cout << "  Full LTO:\n";
        std::cout << "    g++ -flto -O2 -c main.cpp lib.cpp\n";
        std::cout << "    g++ -flto -O2 main.o lib.o -o app\n\n";
        std::cout << "  Thin LTO (Clang):\n";
        std::cout << "    clang++ -flto=thin -O2 -c main.cpp lib.cpp\n";
        std::cout << "    clang++ -flto=thin -O2 main.o lib.o -o app\n\n";
        std::cout << "  CMake:\n";
        std::cout << "    set(CMAKE_INTERPROCEDURAL_OPTIMIZATION ON)\n\n";
        std::cout << "  Typical gains: 5-20% runtime, 10-30% smaller binary\n\n";
    }

    // ── Demo 5: Binary Inspection Commands ──────────────────────────
    {
        std::cout << "--- Demo 5: Binary Inspection Commands ---\n";
        std::cout << "  ┌──────────────────────────────┬──────────────────────────────┐\n";
        std::cout << "  │ Command                      │ What it shows                │\n";
        std::cout << "  ├──────────────────────────────┼──────────────────────────────┤\n";
        std::cout << "  │ nm -C libmath.a              │ All symbols (demangled)      │\n";
        std::cout << "  │ nm -D libmath.so             │ Dynamic (exported) symbols   │\n";
        std::cout << "  │ ldd ./app                    │ Shared library dependencies  │\n";
        std::cout << "  │ readelf -d ./app             │ ELF dynamic section (rpath etc) │\n";
        std::cout << "  │ objdump -T libmath.so        │ Dynamic symbol table         │\n";
        std::cout << "  │ readelf -S main.o | grep lto │ Check if LTO was used        │\n";
        std::cout << "  │ file ./app                   │ Binary type, architecture    │\n";
        std::cout << "  │ size ./app                   │ Section sizes (text/data/bss) │\n";
        std::cout << "  └──────────────────────────────┴──────────────────────────────┘\n\n";
    }

    // ── Demo 6: Library Search Order ────────────────────────────────
    {
        std::cout << "--- Demo 6: Dynamic Linker Search Order ---\n";
        std::cout << "  1. DT_RPATH in executable (deprecated, use DT_RUNPATH)\n";
        std::cout << "  2. LD_LIBRARY_PATH environment variable\n";
        std::cout << "  3. DT_RUNPATH in executable (-Wl,-rpath,...)\n";
        std::cout << "  4. /etc/ld.so.cache (built by ldconfig)\n";
        std::cout << "  5. /lib and /usr/lib (default paths)\n\n";
        std::cout << "  Best practice for deployment:\n";
        std::cout << "    Use rpath with $ORIGIN for portable apps:\n";
        std::cout << "    g++ -o app main.o -Wl,-rpath,'$ORIGIN/../lib' -L./lib -lmylib\n";
        std::cout << "    (finds .so relative to executable location)\n\n";
    }

    std::cout << "============================================\n";
    std::cout << "   End of Linking Deep Dive\n";
    std::cout << "============================================\n";

    return 0;
}
```

► 11. Low Level Optimization

module_07_under_the_hood/11_low_level_optimization.cpp

Konu: Low-Level Optimization — Düşük Seviye Optimizasyon

Teori / Theory

THEORY: Branch Prediction & Hints

Modern CPUs predict branch outcomes to keep the pipeline full. Wrong predictions cost ~15-20 cycles (pipeline flush).
C++20 attributes:
[[likely]] — This branch is probably taken (hot path)
[[unlikely]] — This branch is probably NOT taken (error/cold path)
GCC/Clang builtin (pre-C++20): __builtin_expect(expr, expected_value) if (__builtin_expect(error_occurred, 0)) { ... } // expect false
Real impact: 5-15% speedup on branch-heavy code (parsers, validators)

TEORİ: Dal Tahmini ve İpuçları

Modern CPU'lar pipeline'ı dolu tutmak için dal sonuçlarını tahmin eder. Yanlış tahmin ~15-20 döngüye mal olur (pipeline boşaltma).
C++20 öznitelikleri:
[[likely]] — Bu dal muhtemelen alınır (sıcak yol)
[[unlikely]] — Bu dal muhtemelen ALINMAZ (hata/soğuk yol)
GCC/Clang yerleşiği (C++20 öncesi): __builtin_expect(expr, beklenen_deger) if (__builtin_expect(hata_olustu, 0)) { ... } // false bekle
Gerçek etki: Dal yoğun kodda %5-15 hızlanma (ayrıştırıcı, doğrulayıcı)

THEORY: Profile-Guided Optimization (PGO)

PGO uses REAL runtime data to optimize:
Step 1: Build instrumented binary g++ -fprofile-generate -O2 main.cpp -o main_instr
Step 2: Run with representative workload ./main_instr < typical_input.txt # Generates .gcda profiles
Step 3: Rebuild with profile data g++ -fprofile-use -O2 main.cpp -o main_optimized
What PGO optimizes:
- Branch prediction hints (based on actual branch frequencies)
- Function inlining (inline frequently-called functions)
- Code layout (hot code close together for cache)
- Loop unrolling (based on actual iteration counts)
- Register allocation (based on actual variable usage)
Typical improvement: 10-30% for branch-heavy code

TEORİ: Profil Güdümlü Optimizasyon (PGO)

PGO GERÇEK çalışma zamanı verileri kullanarak optimize eder:
Adım 1: Enstrümante ikili oluştur g++ -fprofile-generate -O2 main.cpp -o main_instr
Adım 2: Temsili iş yükü ile çalıştır ./main_instr < typical_input.txt # .gcda profilleri üretir
Adım 3: Profil verileriyle yeniden derle g++ -fprofile-use -O2 main.cpp -o main_optimized
PGO'nun optimize ettiği şeyler:
- Dal tahmini ipuçları (gerçek dal frekanslarına göre)
- Fonksiyon satır içi açma (sık çağrılan fonksiyonlar)
- Kod yerleşimi (sıcak kod önbellek için yan yana)
- Döngü açma (gerçek iterasyon sayılarına göre)
- Yazmaç tahsisi (gerçek değişken kullanımına göre)
Tipik iyileşme: Dal yoğun kodda %10-30

THEORY: Hot/Cold Path Separation

Keep hot (frequently executed) code compact and together:
HOT PATH: COLD PATH:
- Normal execution flow - Error handling
- Inner loops - Logging/debug output
- Frequently accessed data - Initialization (one-time)
- Short functions (inlined) - Exception handling

Techniques:
1. __attribute__((hot)) / __attribute__((cold)) — GCC/Clang
2. [[likely]] / [[unlikely]] — C++20
3. Move error strings to separate function (keeps hot path small)
4. Avoid large inline functions on error paths
5. Use noinline on cold functions to keep icache pressure low

TEORİ: Sıcak/Soğuk Yol Ayrımı

Sıcak (sık çalıştırılan) kodu kompakt ve bir arada tut:
SICAK YOL: SOĞUK YOL:
- Normal yürütme akışı - Hata yönetimi
- İç döngüler - Loglama/debug çıktısı
- Sık erişilen veri - Başlatma (tek seferlik)
- Kısa fonksiyonlar (satır içi) - İstisna yönetimi

Teknikler:
1. __attribute__((hot)) / __attribute__((cold)) — GCC/Clang
2. [[likely]] / [[unlikely]] — C++20
3. Hata dizelerini ayrı fonksiyona taşı (sıcak yol küçük kalır)
4. Hata yollarında büyük inline fonksiyonlardan kaçın
5. Soğuk fonksiyonlarda noinline kullan (icache basıncını düşük tut)

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <numeric>
#include <cstdint>
#include <algorithm>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: [[likely]] / [[unlikely]] Attributes (C++20)
// ═══════════════════════════════════════════════════════════════════════════

// EN: Sensor validation with branch hints
// TR: Dal ipuçları ile sensör doğrulama
int validate_sensor_likely(int value) {
    if (value >= 0 && value <= 1023) [[likely]] {
        // EN: Normal case — sensor value in valid range (99% of the time)
        // TR: Normal durum — sensör değeri geçerli aralıkta (zamanın %99'u)
        return value;
    } else [[unlikely]] {
        // EN: Error case — out of range (rarely happens)
        // TR: Hata durumu — aralık dışı (nadiren olur)
        return -1;
    }
}

// EN: Switch with likely hint
// TR: likely ipucu ile switch
enum class PacketType : uint8_t { Data = 0, Heartbeat = 1, Error = 2, Config = 3 };

void process_packet(PacketType type) {
    switch (type) {
        case PacketType::Data: [[likely]]
```

```
            // EN: 90% of packets are data
            // TR: Paketlerin %90'ı veridir
            break;
        case PacketType::Heartbeat:
            break;
        case PacketType::Error: [[unlikely]]
            // EN: Errors are rare
            // TR: Hatalar nadirdir
            break;
        case PacketType::Config: [[unlikely]]
            break;
    }
}

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: __builtin_expect (Pre-C++20)
// ═══════════════════════════════════════════════════════════════════════════

// EN: GCC/Clang builtin for branch prediction hints
// TR: Dal tahmini ipuçları için GCC/Clang yerleşik fonksiyonu
#define LIKELY(x)   __builtin_expect(!!(x), 1)
#define UNLIKELY(x) __builtin_expect(!!(x), 0)

int validate_sensor_builtin(int value) {
    if (LIKELY(value >= 0 && value <= 1023)) {
        return value;
    } else {
        return -1;
    }
}

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: Hot/Cold Path Separation
// ═══════════════════════════════════════════════════════════════════════════

// EN: __attribute__((cold)) — tells compiler this is rarely called
// TR: __attribute__((cold)) — derleyiciye bunun nadiren çağrıldığını söyler
__attribute__((cold, noinline))
void handle_error(int sensor_id, int value) {
    // EN: Error handling code — large, but rarely executed
    //     Using noinline keeps it out of the hot path's icache footprint
    // TR: Hata yönetimi kodu — büyük ama nadiren çalıştırılır
    std::cerr << "ERROR: Sensor " << sensor_id << " value " << value
              << " out of range!\n";
}

// EN: __attribute__((hot)) — tells compiler to optimize aggressively
// TR: __attribute__((hot)) — derleyiciye agresif optimize etmesini söyler
__attribute__((hot))
int process_sensor_reading(int sensor_id, int raw_value) {
    // EN: HOT PATH — validate and convert (runs millions of times)
    // TR: SICAK YOL — doğrula ve dönüştür (milyonlarca kez çalışır)
    if (raw_value >= 0 && raw_value <= 1023) [[likely]] {
        return raw_value * 100 / 1024;  // Scale to 0-100
    }
    // EN: COLD PATH — error handling (separated to keep hot path small)
    // TR: SOĞUK YOL — hata yönetimi (sıcak yolu küçük tutmak için ayrılmış)
    handle_error(sensor_id, raw_value);
    return -1;
}

// ═══════════════════════════════════════════════════════════════════════════
// PART 4: Compiler Intrinsics for Bit Operations
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: Compiler intrinsics compile to single CPU instructions.
 *     Much faster than library implementations.
 *
 *     ┌──────────────────────────┬──────────────────────────────────────┐
 *     │ Intrinsic                │ Purpose                              │
 *     ├──────────────────────────┼──────────────────────────────────────┤
 *     │ __builtin_popcount(x)    │ Count set bits (population count)    │
 *     │ __builtin_clz(x)         │ Count leading zeros                  │
 *     │ __builtin_ctz(x)         │ Count trailing zeros                 │
 *     │ __builtin_ffs(x)         │ Find first set bit (1-indexed)       │
 *     │ __builtin_parity(x)      │ Parity of set bits (odd=1, even=0)   │
```

```
111  *     │ __builtin_bswap32(x)    │ Byte swap (endian conversion)       │
112  *     │ __builtin_prefetch(p)   │ Prefetch memory into cache          │
113  *     └─────────────────────────┴─────────────────────────────────────┘
114  *
115  * TR: Derleyici intrinsic'leri tek CPU talimatına derlenir.
116  *     Kütüphane uygulamalarından çok daha hızlıdır.
117  */
118
119 void demo_intrinsics() {
120     uint32_t val = 0b1010'1100'0011'0000;  // 0xAC30
121
122     std::cout << "    Value: 0x" << std::hex << val << std::dec << "\n";
123     std::cout << "    popcount:      " << __builtin_popcount(val) << " bits set\n";
124     std::cout << "    clz:           " << __builtin_clz(val) << " leading zeros\n";
125     std::cout << "    ctz:           " << __builtin_ctz(val) << " trailing zeros\n";
126     std::cout << "    ffs:           " << __builtin_ffs(static_cast<int>(val))
127               << " (first set bit, 1-indexed)\n";
128     std::cout << "    parity:        " << __builtin_parity(val)
129               << " (" << (__builtin_parity(val) ? "odd" : "even") << " bits)\n";
130     std::cout << "    bswap32:       0x" << std::hex << __builtin_bswap32(val)
131               << std::dec << " (endian swap)\n";
132 }
133
134 // ═══════════════════════════════════════════════════════════════════════════
135 // PART 5: Prefetch for Sequential Access
136 // ═══════════════════════════════════════════════════════════════════════════
137
138 /**
139  * EN: __builtin_prefetch(addr, rw, locality)
140  *     - addr: pointer to memory to prefetch
141  *     - rw: 0 = read, 1 = write
142  *     - locality: 0 = non-temporal (use once), 3 = keep in all caches
143  *
144  *     Useful for: sequential array processing where you know you'll need
145  *     future elements before you get to them.
146  *
147  * TR: __builtin_prefetch(adr, oku_yaz, yerellik)
148  *     Ardışık dizi işlemlerinde, gelecek elementlere ihtiyaç duyacağınızı
149  *     bildiğiniz durumlarda faydalıdır.
150  */
151
152 // ═══════════════════════════════════════════════════════════════════════════
153 // MAIN
154 // ═══════════════════════════════════════════════════════════════════════════
155 int main() {
156     std::cout << "============================================\n";
157     std::cout << "   Module 07 - Low-Level Optimization\n";
158     std::cout << "============================================\n\n";
159
160     // ─── Demo 1: [[likely]] / [[unlikely]] ──────────────────────────
161     {
162         std::cout << "--- Demo 1: [[likely]] / [[unlikely]] ---\n";
163         std::cout << "    validate(512)  = " << validate_sensor_likely(512) << " (valid)\n";
164         std::cout << "    validate(2000) = " << validate_sensor_likely(2000) << " (invalid)\n";
165         std::cout << "    validate(-5)   = " << validate_sensor_likely(-5) << " (invalid)\n\n";
166     }
167
168     // ─── Demo 2: Branch Prediction Benchmark ────────────────────────
169     {
170         std::cout << "--- Demo 2: Branch Prediction Impact ---\n";
171
172         constexpr size_t N = 10'000'000;
173         std::vector<int> data(N);
174
175         // EN: Fill with values 0-1023 (valid) + occasional -1 (invalid)
176         // TR: 0-1023 (geçerli) değerlerle doldur + ara sıra -1 (geçersiz)
177         std::mt19937 rng(42);
178         std::uniform_int_distribution<int> dist(0, 1023);
179         for (auto& v : data) v = dist(rng);
180         // EN: Sprinkle 0.1% invalid values
181         // TR: %0.1 geçersiz değer serpiştir
182         for (size_t i = 0; i < N / 1000; ++i) {
183             data[rng() % N] = -1;
184         }
```

```cpp

        // EN: Sorted data (predictable branches)
        // TR: Sıralı veri (öngörülebilir dallanma)
        auto sorted = data;
        std::sort(sorted.begin(), sorted.end());

        // EN: Benchmark unsorted (unpredictable)
        // TR: Sırasız veriyi benchmark'la (öngörülemez)
        auto t1 = std::chrono::high_resolution_clock::now();
        volatile int sum1 = 0;
        for (auto v : data) {
            if (v >= 0) sum1 += v;
        }
        auto t2 = std::chrono::high_resolution_clock::now();

        // EN: Benchmark sorted (predictable)
        // TR: Sıralı veriyi benchmark'la (öngörülebilir)
        auto t3 = std::chrono::high_resolution_clock::now();
        volatile int sum2 = 0;
        for (auto v : sorted) {
            if (v >= 0) sum2 += v;
        }
        auto t4 = std::chrono::high_resolution_clock::now();

        auto us1 = std::chrono::duration_cast<std::chrono::microseconds>(t2 - t1).count();
        auto us2 = std::chrono::duration_cast<std::chrono::microseconds>(t4 - t3).count();

        std::cout << "    Unsorted (random branches): " << us1 << " us\n";
        std::cout << "    Sorted (predictable):       " << us2 << " us\n";
        std::cout << "    (Sorted may be faster due to branch prediction)\n\n";
    }

    // ─── Demo 3: Hot/Cold Path Separation ──────────────────────────────
    {
        std::cout << "--- Demo 3: Hot/Cold Path Separation ---\n";
        std::cout << "    process_sensor_reading(1, 512) = "
                  << process_sensor_reading(1, 512) << "%\n";
        std::cout << "    process_sensor_reading(2, 1023) = "
                  << process_sensor_reading(2, 1023) << "%\n";
        std::cout << "    process_sensor_reading(3, -5)  = ";
        int err_result = process_sensor_reading(3, -5);
        std::cout << err_result << " (error)\n\n";

        std::cout << "    Hot path: validate + scale (inline, optimized)\n";
        std::cout << "    Cold path: handle_error (noinline, out of icache)\n\n";
    }

    // ─── Demo 4: Compiler Intrinsics ───────────────────────────────────
    {
        std::cout << "--- Demo 4: Compiler Intrinsics ---\n";
        demo_intrinsics();
        std::cout << "\n";
    }

    // ─── Demo 5: Prefetch Demo ─────────────────────────────────────────
    {
        std::cout << "--- Demo 5: Prefetch Demo ---\n";

        constexpr size_t N = 10'000'000;
        std::vector<int> data(N);
        std::iota(data.begin(), data.end(), 0);

        // EN: Without prefetch
        // TR: Prefetch olmadan
        auto t1 = std::chrono::high_resolution_clock::now();
        long long sum1 = 0;
        for (size_t i = 0; i < N; ++i) {
            sum1 += data[i];
        }
        auto t2 = std::chrono::high_resolution_clock::now();
```

```
255
256         // EN: With prefetch (prefetch 16 elements ahead)
257         // TR: Prefetch ile (16 eleman ilerisini önbellek'le)
258         auto t3 = std::chrono::high_resolution_clock::now();
259         long long sum2 = 0;
260         for (size_t i = 0; i < N; ++i) {
261             if (i + 16 < N) {
262                 __builtin_prefetch(&data[i + 16], 0, 0);
263             }
264             sum2 += data[i];
265         }
266         auto t4 = std::chrono::high_resolution_clock::now();
267
268         auto us1 = std::chrono::duration_cast<std::chrono::microseconds>(t2 - t1).count();
269         auto us2 = std::chrono::duration_cast<std::chrono::microseconds>(t4 - t3).count();
270
271         std::cout << "    Without prefetch: " << us1 << " us (sum=" << sum1 << ")\n";
272         std::cout << "    With prefetch:    " << us2 << " us (sum=" << sum2 << ")\n";
273         std::cout << "    (Sequential access already prefetched by HW;\n";
274         std::cout << "     prefetch helps more with random access patterns)\n\n";
275     }
276
277     // ─── Demo 6: PGO & Optimization Reference ──────────────────────────
278     {
279         std::cout << "--- Demo 6: PGO & Optimization Reference ---\n";
280         std::cout << "  Profile-Guided Optimization:\n";
281         std::cout << "    g++ -fprofile-generate -O2 app.cpp -o app_instr\n";
282         std::cout << "    ./app_instr < typical_input.txt\n";
283         std::cout << "    g++ -fprofile-use -O2 app.cpp -o app_optimized\n\n";
284         std::cout << "   ┌──────────────────────────────┬──────────────────────────────┐\n";
285         std::cout << "   │ Technique                    │ When to Use                  │\n";
286         std::cout << "   ├──────────────────────────────┼──────────────────────────────┤\n";
287         std::cout << "   │ [[likely]]/[[unlikely]]      │ Known hot/cold branches      │\n";
288         std::cout << "   │ __builtin_expect             │ Pre-C++20 branch hints       │\n";
289         std::cout << "   │ __attribute__((hot/cold))    │ Function-level hint          │\n";
290         std::cout << "   │ __builtin_prefetch           │ Random access patterns       │\n";
291         std::cout << "   │ __builtin_popcount,clz,ctz   │ Bit operations (1 instruction)│\n";
292         std::cout << "   │ PGO (-fprofile-use)          │ After profiling real workload │\n";
293         std::cout << "   │ LTO (-flto)                  │ Cross-TU optimization        │\n";
294         std::cout << "   │ -O3 -march=native            │ Maximum local optimization   │\n";
295         std::cout << "   └──────────────────────────────┴──────────────────────────────┘\n\n";
296     }
297
298     std::cout << "==========================================\n";
299     std::cout << "   End of Low-Level Optimization\n";
300     std::cout << "==========================================\n";
301
302     return 0;
303 }
```

Modül 8: C++ Core Guidelines — Üst Düzey Kurallar

Üst Düzey Kurallar

9 kaynak dosya · Standart: C++17

C++17

► 01. Philosophy And Interfaces

module_08_core_guidelines/01_philosophy_and_interfaces.cpp

Konu: C++ Core Guidelines: Philosophy & Interfaces — C++ Çekirdek Kuralları: Felsefe ve

Teori / Theory

INTRODUCTION: What are C++ Core Guidelines?

The "C++ Core Guidelines" are a set of tried-and-true rules about coding in C++. Written by Bjarne Stroustrup (creator of C++) and Herb Sutter, they aim to help developers write MUCH Safer, Cleaner, and Faster modern C++.

GİRİŞ: C++ Core Guidelines Nedir?

"C++ Core Guidelines", C++'ın yaratıcısı Bjarne Stroustrup ve ISO C++ Komitesi tarafından yazılan, Modern C++'ın nasıl DOĞRU, GÜVENLİ ve PERFORMANSLI yazılacağını anlatan resmi bir anayasadır. Sektördeki en iyi Mimarlar bu kurallara uyar.

P.1: Express ideas directly in code

Compilers don't read comments. If you have to write a comment to explain WHAT a loop does, name your variables better or use Standard Library (STL) algorithms.

P.1: Fikirlerinizi Koda Doğrudan Dökün

Derleyiciler yorum satırlarını (comments) okumazlar. Bir döngünün NE yaptığını anlatmak için satırlarca yoruma ihtiyaç duyuyorsanız, kodun iskeleti kötüdür. Standart Kütüphane (STL) metodlarını (Örn: std::find_if) kullanın. İşin NİYETİNİ belli edin!

P.4 & P.5: Compile-Time Checking

"Prefer compile-time checking to run-time checking." If you can calculate or check a rule during COMPILATION, do it! Use `constexpr` and `static_assert()`. A bug caught at compile-time costs $0. At runtime, it causes a crash.

Derleme Zamanı Kontrolleri

"Derleme zamanını Çalışma zamanına (Runtime) tercih edin." Eğer bir değeri oyun çalışmadan (Build alırken) absolut hesaplayabiliyorsan, `constexpr` ve `static_assert()` kullan! Derlemede yakalanan bir hatanın maliyeti SIFIRDIR, çalışırken yakalanan bir hata programı (veya sunucuyu) çökertir.

CPPREF Depth: Type Safety as Zero-Cost Abstraction — Strong Typedefs

C++ Core Guidelines I.4: "Make interfaces precisely and strongly typed". Primitive obsession — using bare `int` for meters, seconds, and IDs — lets the compiler silently accept nonsensical expressions like `meters + seconds`. Strong typedefs (a thin wrapper class with explicit constructors) enforce correctness at ZERO runtime cost because the optimizer elides the wrapper entirely. `enum class` prevents implicit conversions between unrelated enumerations. `[[nodiscard]]` forces callers to use (not ignore) a return value — essential for error codes. In C++20, Concepts formalize interface contracts: `template<Sortable T>` replaces unnamed SFINAE with readable, diagnosable constraints.

CPPREF Derinlik: Sıfır Maliyetli Soyutlama Olarak Tip Güvenliği — Güçlü Typedef'ler

C++ Çekirdek Kılavuzu I.4: "Arayüzleri kesin ve güçlü tipli yapın". İlkel tip takıntısı — metre, saniye ve kimlik için çıplak `int` kullanmak — derleyicinin `metre + saniye` gibi anlamsız ifadeleri sessizce kabul etmesine yol açar. Güçlü typedef'ler (açık kuruculu ince sarmalayıcı sınıf) SIFIR çalışma zamanı maliyetiyle doğruluğu zorlar çünkü optimizer sarmalayıcıyı tamamen eler. `enum class` ilgisiz numaralandırmalar arası örtük dönüşümü engeller. `[[nodiscard]]` çağıranı dönüş değerini kullanmaya zorlar. C++20'de Kavramlar arayüz sözleşmelerini resmileştirir.

Kaynak Kod / Source Code

```cpp
#include <algorithm> // for std::find_if
#include <iostream>
#include <memory>
#include <string>
#include <vector>

// ---------------------------------------------------------------------------------------------
// [P.1] Express Intent (Kötü vs İyi Örnek)
// ---------------------------------------------------------------------------------------------
class ECU {
public:
  std::string name;
  bool isOnline = true;
};

void checkECUStatus_Bad(const std::vector<ECU> &ecus, const std::string &targetName) {
  // BAD (Kötü): C-Style manual loop. Hard to catch the intent at a glance.
  // TR: KÖTÜ: Geleneksel döngü. Okuması ve arkasındaki "Niyeti" anlamak vakit kaybettirir.
  bool found = false;
  for (size_t i = 0; i < ecus.size(); ++i) {
    if (ecus[i].name == targetName) {
      found = true;
      break;
    }
  }
  (void)found; // EN: Suppress — intentional bad example
}

void checkECUStatus_Good(const std::vector<ECU> &ecus, const std::string &targetName) {
  // GOOD (İyi - C++ Core Guidelines): Use STL algorithm! Intent is 100% clear immediately: "Find
  // if any ECU matches".
  // TR: İYİ: STL Algoritması kullanımı. Niyet çok açıktır: "Bir Koşula Göre Ara ve Bul
  // (find_if)".
  auto it = std::find_if(ecus.begin(), ecus.end(),
      [&](const ECU &e) { return e.name == targetName; });

```

```
37    if (it != ecus.end()) {
38      std::cout << "[Intent/Niyet: STL] ECU '" << targetName << "' found!" << std::endl;
39    }
40  }
41
42  // ---------------------------------------------------------------------------------------------
43  // [P.4 & P.5] Compile-Time Checking (`constexpr` and `static_assert`)
44  // ---------------------------------------------------------------------------------------------
45  // EN: Because of `constexpr`, compiler calculates this during BUILD time (0 CPU cost at
46  // runtime).
47  // TR: `constexpr` sayesinde CPU bu fonksiyonun hesabını BUILD (Derleme) anında tamamlar.
48  constexpr int calculateMaxSensorChannels(int baseChannels, int expansionModules) {
49    return baseChannels + (expansionModules * 2);
50  }
51
52  int main() {
53    std::cout << "=== MODULE 8: C++ CORE GUIDELINES - PHILOSOPHY & INTERFACES ===\n" << std::endl;
54
55    // 1. Express Intent demonstration
56    std::vector<ECU> vehicleECUs = {
57        {"BCM", true}, {"Powertrain", false}, {"HVAC", true}};
58    checkECUStatus_Good(vehicleECUs, "Powertrain");
59
60    // 2. Compile-Time Magic
61    // EN: This is NOT calculated at runtime! Replaced directly with `20` in the ASM binary. TR: Bu
62    // işlem çalışırken (Runtime) YAPILMAZ! Derleyici satırı Assembly kodunda doğrudan `20` yapar.
63    constexpr int maxChannels = calculateMaxSensorChannels(10, 5);
64
65    // EN: static_assert checks rules ALWAYS DURING COMPILE. If false -> Red Compiler Error! TR:
66    // static_assert kuralları DERLEME AŞAMASINDA kontrol eder. Yanlışsa IDE direkt kırmızıyı
67    // basar, Build alınmaz.
68    static_assert(maxChannels >= 10, "[FATAL ERROR] Sensor channels cannot be less than 10!");
69
70    std::cout << "\nMax Sensor Channels (Calculated purely at Compile Time!): " << maxChannels <<
71        std::endl;
72
73    std::cout << "\n[ARCHITECTURAL CONCLUSION / MİMARİ SONUÇ]:" << std::endl;
74    std::cout << "- EN: Write code for the COMPILER to optimize and HUMANS to read!" << std::endl;
75    std::cout << "- TR: İnsanların rahat okuyacağı (STL kullanarak niyet belli eden), "
76            "Derleyicinin uçuracağı (constexpr) mimariler kurun!"
77        << std::endl;
78
79    // ═══════════════════════════════════════════════════════════════════════════════════════
80    // EN: I.4 — Make interfaces precisely and strongly typed
81    // TR: I.4 — Arayüzleri kesin ve güçlü-tipli yapın
82    // ═══════════════════════════════════════════════════════════════════════════════════════
83
84    // ─── 1. BAD: Weakly typed interface ──────────────────────────────────────────────────
85    // EN: What does 'unit' mean? 0 = KPH? 1 = MPH? Magic numbers everywhere.
86    // TR: 'unit' ne demek? 0 = KPH mi? 1 = MPH mi? Anlaşılmaz büyü sayıları.
87    //
88    // void setSpeed(int speed, int unit);   // BAD — caller can pass anything setSpeed(120, 3);
89    // // What is 3? Compiles fine!
90
91    // ─── 2. GOOD: Strongly typed interface ───────────────────────────────────────────────
92    // EN: Type-safe enum makes invalid calls impossible at compile time.
93    // TR: Tip-güvenli enum, geçersiz çağrıları derleme zamanında engeller.
94    enum class SpeedUnit { KPH, MPH };
95
96    auto setSpeed = [](double speed, SpeedUnit unit) {
97      const char* label = (unit == SpeedUnit::KPH) ? "km/h" : "mph";
98      std::cout << "[I.4 GOOD] Speed set to " << speed << " " << label << std::endl;
99    };
100
```

```
101     setSpeed(120.0, SpeedUnit::KPH);
102     // setSpeed(120.0, 2);  // EN: Won't compile! Type safety enforced.
103                             // TR: Derlenmez! Tip güvenliği sağlandı.
104
105     // ═══════════════════════════════════════════════════════════════════════════════
106     // EN: I.11 — Never transfer ownership by a raw pointer
107     // TR: I.11 — Ham işaretçi ile sahiplik aktarımı ASLA yapılmaz
108     // ═══════════════════════════════════════════════════════════════════════════════
109
110     // ─── 3. BAD: Raw pointer ownership transfer ─────────────────────────────────
111     // EN: Who owns this pointer? Caller? Callee? Nobody knows → leak.
112     // TR: Bu işaretçinin sahibi kim? Çağıran mı? Çağrılan mı? → sızıntı.
113     //
114     // ECU* createECU() { return new ECU{"Raw", true}; }  // BAD
115
116     // ─── 4. GOOD: unique_ptr ownership transfer ─────────────────────────────────
117     // EN: Ownership is explicit. unique_ptr is the ONLY owner, auto-deletes.
118     // TR: Sahiplik açıkça bellidir. unique_ptr TEK sahiptir, otomatik siler.
119     auto createECU_Safe = []() -> std::unique_ptr<ECU> {
120       return std::make_unique<ECU>(ECU{"SmartECU", true});
121     };
122
123     auto safeEcu = createECU_Safe();
124     std::cout << "[I.11 GOOD] Owned ECU: " << safeEcu->name
125               << " (unique_ptr, no leak possible)" << std::endl;
126
127     return 0;
128   }
```

► 02. Resource Management Rule Of Zero

module_08_core_guidelines/02_resource_management_rule_of_zero.cpp

Konu: C++ Core Guidelines: Resource Management & Rule of Zero — Kaynak Yönetimi Mimarisi

Teori / Theory

INTRODUCTION: Rule of Zero & RAII

The Guidelines say: "Manage resources automatically using resource handles and RAII."
What is the "Rule of Zero"? If your class only manages standard modern resources (like `std::string`, `std::vector`, `std::unique_ptr`), you SHOULD NOT write a Destructor (`~MyClass()`), Copy Constructor, or Assignment Operator. The compiler's default automatically handles memory perfectly. ZERO custom memory-management code = ZERO memory leak bugs!

GİRİŞ: Sıfır Kuralı ve RAII

Çekirdek Kurallar der ki: "Kaynakları (Belleği) otomatik olarak RAII kılıflarıyla yönet!"
"Sıfır Kuralı (Rule of Zero)" Nedir? Eğer sizin yazdığınız sınıf, belleği kendi yöneten akıllı nesneler (std::vector, std::string) kuruyorsa, BAŞKA HİÇBİR ŞEY YAZMAYIN! Derleyicinin üreteceği otomatik Yıkıcı/Kurucu kodlar zaten KUSURSUZ çalışır. O (Sıfır) kod yazarsanız = 0 (Sıfır) Bellek Kaçağı üretirsiniz!

R.10: Avoid malloc() and free()

`malloc()` does not call Constructors! It grabs raw, dumb bytes from RAM. `free()` does not call Destructors! It leaves sub-resources orphaned. NEVER use them.

R.10: Malloc ve Free KESİNLİKLE YASAK

Eski C fonksiyonu `malloc()`, C++ Sınıflarını "Constructor" çağırmadan ucube gibi yaşama döndürür. `free()` ise Sınıf içindeki işaretçileri umursamadan öldürür ve RAM çöp olur.

CPPREF Depth: Rule of Zero — Let the Compiler Write Your Special Members

If every data member is an RAII type (std::string, std::vector, std::unique_ptr, std::shared_ptr), the compiler-generated destructor, copy constructor, copy assignment, move constructor, and move assignment are all correct — you need to write NONE of them. This is the Rule of Zero. The moment you define ANY special member (even a destructor), the compiler suppresses implicit generation of the move operations. Writing `= default` explicitly requests compiler generation and documents your intent. The Rule of Zero dramatically reduces bugs because hand-written special members are a top source of resource leaks, double-frees, and slicing errors. Reserve the Rule of Five for low-level resource handles only.

CPPREF Derinlik: Sıfır Kuralı — Özel Üyeleri Derleyiciye Bırakın

Tüm veri üyeleri RAII tipi (std::string, std::vector, std::unique_ptr, std::shared_ptr) ise derleyicinin ürettiği yıkıcı, kopyalama kurucusu, kopyalama ataması, taşıma kurucusu ve taşıma ataması zaten doğrudur — HİÇBİRİNİ yazmanız gerekmez (Sıfır Kuralı). HERHANGİ bir özel üye tanımladığınız anda (yıkıcı bile olsa) derleyici taşıma işlemlerini otomatik üretmez. `= default` derleyici üretimini açıkça talep eder. Sıfır Kuralı hataları büyük ölçüde azaltır çünkü elle yazılan özel üyeler kaynak sızıntısı, çift serbest bırakma ve dilimleme hatalarının en büyük kaynağıdır.

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <vector>

// ---------------------------------------------------------------------------------------------
// [C.20, R.1] THE RULE OF ZERO (Sıfır Kuralı Mimarisi)
// ---------------------------------------------------------------------------------------------
// EN: This class flawlessly manages its own memory thanks to the Rule of Zero.
// TR: Bu sınıf, 'Sıfır Kuralı' sayesinde kendi belleğini kusursuz yönetir.
class VehicleConfigManager {
private:
  std::string vehicleId;                  // EN: Self-managed (RAII)
  std::vector<std::string> ecuModuleNames; // EN: Self-managed (RAII)

public:
  // Simple constructor to initialize data.
  // TR: Constructor sadece ilk değer ataması içindir.
  VehicleConfigManager(const std::string &id) : vehicleId(id) {
    std::cout << "[VehicleConfig] Initialized: " << vehicleId << std::endl;
  }

  void addECU(const std::string &name) { ecuModuleNames.push_back(name); }

  // EN: NOTICE WHAT IS MISSING? - NO `~VehicleConfigManager()` Destructor! - NO `new` or
  // `delete` anywhere! -> When this object naturally dies, `std::vector` and `std::string` clean
  // themselves up. 100% Leak-Proof!
  //
  // TR: FARK ETTİNİZ Mİ NE EKSİK? - Yıkıcı `~VehicleConfigManager()` fonksiyonu YOK! - El ile
  // yazılmış `new` veya `delete` YOK! -> Obje ölürken, içindeki objeler (vector, string) C++
  // mantığıyla OS'ye iade edilir.
};

int main() {
  std::cout << "=== MODULE 8: C++ CORE GUIDELINES - RULE OF ZERO & RESOURCE "
               "MGMT ===\n"
            << std::endl;

  {
    std::cout << "-> Entering local scope... (Kapsama giriliyor...)" << std::endl;

    // 1. Applying Rule of Zero
    VehicleConfigManager myCar("VIN_WBAPH5C55BA271234");
    myCar.addECU("BCM");
    myCar.addECU("Powertrain");

    // 2. Applying R.11 (Avoid new and delete explicitly)
```

```
48      // EN: NEVER write -> VehicleConfigManager* v = new VehicleConfigManager(...);
49      // TR: ASLA yazma -> VehicleConfigManager* v = new VehicleConfigManager(...);
50
51      std::unique_ptr<VehicleConfigManager> smartConfig =
52          std::make_unique<VehicleConfigManager>("VIN_JH4DB7660SS001234");
53
54    } // EN: BOTH 'myMatch' and 'smartMatchLocation' auto-destruct HERE safely.
55      // TR: Buradaki '}' anında, akıllı işaretçi ve stack objesi muazzam bir sessizlikle iade
56      // edilir.
57
58    std::cout << "-> Left local scope safely. No manual memory management required!\n"
59              "-> (Sıfır sızıntı! Amale gibi delete[] kontrolü yapmaya gerek "
60              "kalmadı!)\n"
61          << std::endl;
62
63    std::cout << "\n[ARCHITECTURAL CONCLUSION / MİMARİ SONUÇ]:" << std::endl;
64    std::cout << "- EN: C++ Core Guidelines states: 'Rule of Zero'. Fall back "
65                 "entirely on std::vector and Smart Pointers."
66              << std::endl;
67    std::cout << "- TR: (Manuel bellek işine hiç girmemeyi (Rule of Zero) emreder!)" << std::endl;
68
69    // ═══════════════════════════════════════════════════════════════════════════════
70    // EN: R.1 — Manage resources automatically using RAII
71    // TR: R.1 — Kaynakları RAII kullanarak otomatik yönet
72    // ═══════════════════════════════════════════════════════════════════════════════
73
74    // ─── 1. RAII FileHandle wrapper ──────────────────────────────────────────────
75    // EN: Constructor acquires, Destructor releases. No manual close needed.
76    // TR: Constructor edinir, Destructor serbest bırakır. Manuel kapatma yok.
77    struct FileHandle {
78      std::string filename;
79      bool isOpen = false;
80
81      explicit FileHandle(const std::string& fname)
82          : filename(fname), isOpen(true) {
83        std::cout << "[R.1 RAII] Opened file: " << filename << std::endl;
84      }
85
86      ~FileHandle() {
87        if (isOpen) {
88          isOpen = false;
89          std::cout << "[R.1 RAII] Auto-closed file: " << filename << std::endl;
90        }
91      }
92
93      // EN: Delete copy to enforce single ownership (move-only resource).
94      // TR: Tek sahiplik için kopyalamayı sil (sadece taşınabilir kaynak).
95      FileHandle(const FileHandle&) = delete;
96      FileHandle& operator=(const FileHandle&) = delete;
97    };
98
99    // ─── 2. Demo: automatic cleanup even on early return ─────────────────────────
100   // EN: Even if we exit the scope early, destructor is guaranteed to run.
101   // TR: Kapsamdan erken çıksak bile, yıkıcı çalışması garanti edilir.
102   {
103     FileHandle logFile("telemetry_log.csv");
104     std::cout << "[R.1 Demo] Processing file: " << logFile.filename << std::endl;
105
106     bool earlyExit = true;
107     if (earlyExit) {
108       std::cout << "[R.1 Demo] Early return from scope!" << std::endl;
109       // EN: ~FileHandle() runs automatically RIGHT HERE at '}'
110       // TR: ~FileHandle() otomatik olarak '}' noktasında çalışır
111     }
112   } // EN: Destructor fires here — file is auto-closed, no leak.
113     // TR: Yıkıcı burada çalışır — dosya otomatik kapatılır, sızıntı yok.
114
115   std::cout << "[R.1 Demo] Scope exited — FileHandle cleaned up automatically!" << std::endl;
116
117   return 0;
118 }
```

► 03. Error Handling

module_08_core_guidelines/03_error_handling.cpp

Konu: C++ Core Guidelines: Error Handling — Hata Yönetimi

Teori / Theory

E.2 & E.3: Use Exceptions for Errors

"Throw an exception to signal that a function can't perform its assigned task." C++ Core Guidelines heavily prefers Exceptions (`throw std::runtime_error`) over returning Error Codes (like `-1` or `false`). Error codes get ignored, leading to silent bugs. Exceptions CANNOT be ignored unless explicitly caught (`catch`).

Hatalar için Exception Kullanın

"Eğer bir fonksiyon işini yapamıyorsa (Örn: Dosya bulunamadı), Exception (İstisna) fırlat." C++ Mimarisi, eski C-tarzı "-1" veya "false" döndüren hata kodlarından nefret eder. Çünkü hata kodlarını yazılımcılar kolayca görmezden gelir. Exception (`throw`) ise yakalanmazsa (`catch`) programı durdurur. Sessiz (Zombie) bug'lara izin vermez!

E.15: Throw by value, catch by reference

ALWAYS throw standard exceptions by value (`throw MyError();`) and catch them by const reference (`catch (const MyError& e)`). This prevents "Object Slicing".

Değer ile Fırlat, Referans ile Yakala

Exception fırlatırken obje oluşturarak (değer) fırlat: `throw std::invalid_argument("hata");` Yakalarken ASLA değer olarak yakalama, her zaman **Referans** ile yakala: `catch(const std::exception& e)` Bu sayede "Object Slicing" engellenir ve C++ polimorfizmi çalışır.

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <memory>
#include <stdexcept>
#include <string>

// EN: BAD PRACTICE: Using Error Codes (C-Style)
// TR: KÖTÜ PRATİK: C-Tarzı Hata Kodları (Return ile kontrol). Yazılımcı bunu unutabilir!
int openDatabaseOldWay(const std::string &address) {
  if (address.empty())
    return -1; // -1 means FATAL ERROR (Bulunamadı)
  return 0;    // 0 means Success
}

// EN: GOOD PRACTICE: Using Exceptions (C++ Core Guidelines E.2)
// TR: MÜKEMMEL PRATİK: Exception! Sessizce geçilmesine / unutulmasına C++ derleyicisi asla izin
// vermez!
void openDatabaseModern(const std::string &address) {
  if (address.empty()) {
    // EN: E.15: Throw by value!
    // TR: Mimar kuralı: Sadece objeyi fırlat
    throw std::invalid_argument(
        "FATAL: Database address cannot be empty!"
        " (Db adresi bos olamaz!)");
  }
  std::cout << "[Database] Successfully connected to: " << address << std::endl;
}
```

```

// EN: [[nodiscard]] makes the compiler WARN if the return value is ignored.
// TR: [[nodiscard]] dönüş değeri göz ardı edilirse derleyiciyi UYARIR.
[[nodiscard]] int getErrorCode() {
  return -1; // EN: Simulated error code
             // TR: Simüle edilmiş hata kodu
}

int main() {
  std::cout << "=== MODULE 8: C++ CORE GUIDELINES - ERROR HANDLING ===\n" << std::endl;

  // 1. DANGER OF ERROR CODES (Sessiz Zombileşen Hatalar)
  // EN: I forgot to check the return value (-1). My program continues
  // TR: Gelen -1 değerini `if` içine almayı unuttum. Program ZOMBİ olarak devam ediyor!
  openDatabaseOldWay( "");

  // 2. SAFETY OF EXCEPTIONS (Exception Güvenliği, Fail-Fast Yaklasimi)
  try {
    // EN: This will immediately blast an error and JUMP to catch!
    // TR: Bu fonksiyon anında fırlatma yapar ve Alt satırları okumadan 'catch' bloğuna DÜŞER!
    openDatabaseModern("");
    std::cout << "This line will NEVER be printed." << std::endl;
  }
  // EN: CABLE-RULE (E.15): CATCH BY CONST REFERENCE! (Referans ile Yakala)
  // TR: Object Slicing olmasın diye her zaman 'const &' ile polimorfik olarak yakalıyoruz!
  catch (const std::invalid_argument &e) {
    std::cerr << "[Core Guidelines Exception Catcher] -> " << e.what() << std::endl;
  }

```

```
56    // ═══════════════════════════════════════════════════════════════════════════════
57    // [E.12 & E.16: Destructors and noexcept / Yıkıcılar ve noexcept Güvencesi]
58    // ═══════════════════════════════════════════════════════════════════════════════
59    // EN: Destructors (`~MyClass()`) and memory deallocation functions MUST NEVER THROW
60    // EXCEPTIONS! If an exception happens inside a destructor during another exception unwinding,
61    // the C++ runtime immediately terminates (`std::terminate`).
62    //
63    // TR: Yıkıcı fonksiyonlarda (`~MyClass`), ASLA Exception fırlatılamaz. Eğer fırlatılırsa C++
64    // sistemi o saniye İntihar/İptal eder (`std::terminate`). Modern C++'da tüm yıkıcılar
65    // varsayılan olarak `noexcept` (Hata Fırlatmaz) güvencesindedir.
66    // ═══════════════════════════════════════════════════════════════════════════════
67    //
68    // ═══════════════════════════════════════════════════════════════════════════════
69      // PPREF DEPTH: The Brutality of `std::terminate()` / CPPREF DERİNLİK: `std::terminate()`
70    // Acımasızlığı]
71    // ═══════════════════════════════════════════════════════════════════════════════
72    // EN: `std::terminate()` is the absolute end of the line. It doesn't call destructors, it
73    // doesn't free memory, it doesn't close files. It violently kills the process from the OS
74    // level (`abort()`). This happens if: 1. You throw an exception and no `catch` block exists
75    // anywhere to catch it. 2. An exception throws INSIDE a destructor while another exception is
76    // currently flying (Stack Unwinding collision). 3. A `noexcept` function explicitly throws.
77    //
78    // TR: `std::terminate()` bir programın yaşayabileceği en vahşi ölümdür. Açık olan dosyaları
79    // kapatmaz, RAM'i temizlemez (Destructor çağırmadan fişi çeker). 3 durumda ortaya çıkar: 1.
80    // Fırlattığınız hatayı kimse (catch ile) yakalamazsa, 2. Bir hata havada uçarken (Stack geri
81    // sarılırken), Destructor'ın içinden İKİNCİ bir hata daha fırlatırsanız (Çarpışma), 3.
82    // `noexcept` verdiğiniz fonksiyonun içinde gizlice hata fırlatırsanız. Program anında OS
83    // seviyesinde vurulur (`abort()`).
84    // ═══════════════════════════════════════════════════════════════════════════════
85
86    // ═══════════════════════════════════════════════════════════════════════════════
87    // EN: E.6 — Use RAII to prevent leaks in error paths
88    // TR: E.6 — Hata yollarında sızıntıları önlemek için RAII kullanın
89    // ═══════════════════════════════════════════════════════════════════════════════
90
91    // ─── 1. BAD: Raw pointer leaks on exception ─────────────────────────────────
92    // EN: If process() throws, the delete never runs → memory leak.
93    // TR: process() hata fırlatırsa delete çalışmaz → bellek sızıntısı.
94    //
95    // int* data = new int[1024]; process(data);      // If this throws... delete[] data;      //
96    // ...this line is NEVER reached! LEAK!
97
98    // ─── 2. GOOD: unique_ptr + exception = no leak ──────────────────────────────
99    // EN: unique_ptr guarantees cleanup even when exceptions fly.
100   // TR: unique_ptr, exception fırlatılsa bile temizliği garanti eder.
101   try {
102     auto safeData = std::make_unique<int[]>(1024);
103     safeData[0] = 42;
104     std::cout << "[E.6 GOOD] safeData[0] = " << safeData[0] << std::endl;
105
106     // EN: Simulate an error after allocation
107     // TR: Tahsis sonrası bir hata simüle et
108     throw std::runtime_error("Simulated processing error!");
109
110     // EN: This line is skipped, but unique_ptr still frees memory.
111     // TR: Bu satır atlanır ama unique_ptr yine de belleği serbest bırakır.
112   } catch (const std::runtime_error& e) {
113     std::cerr << "[E.6 Caught] " << e.what()
114               << " (unique_ptr freed memory automatically!)" << std::endl;
115   }
116
117   // ─── 3. [[nodiscard]]: Force callers to check return values ─────────────────
118   // EN: [[nodiscard]] makes the compiler WARN if the return value is ignored.
119   // TR: [[nodiscard]] dönüş değeri göz ardı edilirse derleyiciyi UYARIR. (getErrorCode() defined
120   // above main as a [[nodiscard]] free function)
121
122   // EN: Good — caller checks the result. Compiler is happy.
123   // TR: İyi — çağıran sonucu kontrol eder. Derleyici mutludur.
124   int errCode = getErrorCode();
125   if (errCode != 0) {
126     std::cout << "[nodiscard Demo] Error code received: " << errCode << std::endl;
127   }
```

```

  // EN: Bad — ignoring the return would trigger a compiler warning:
  // TR: Kötü — dönüşü yok saymak derleyici uyarısı tetikler: getErrorCode();  // WARNING:
  // ignoring return value with [[nodiscard]]

  return 0;
}
```

► 04. Concurrency And Performance

module_08_core_guidelines/04_concurrency_and_performance.cpp

Konu: C++ Core Guidelines: Concurrency & Performance Rules — Eşzamanlılık ve Performans

Teori / Theory

CP.20: Use RAII Locks, Never touch raw mutexes

"Never use plain lock() and unlock(). Use RAII." If you manually call `mtx.lock()` and subsequently throw an exception before reaching `mtx.unlock()`, your underlying mutex NEVER unlocks. Your server will FREEZE FOREVER (Deadlock). Always use `std::lock_guard` or `std::scoped_lock` (C++17).

Çıplak Mutex yasaktır, RAII kullan!

"ASLA çıplak mtx.lock() ve mtx.unlock() KULLANMAYIN." Eğer manuel `lock()` yaparsanız ve hemen alt satırda bir Exception (Hata) fırlatılırsa kod `unlock()` kısmına asırlar geçse varamaz. Tüm sunucunuz sonsuza dek kitlenir (Deadlock). Kilit mekanizmasını bir Scope (Kapsam) içine kilitleyen `std::lock_guard` kullanın.

Per.1 & Per.2: Premature Optimization

"Don't optimize without reason." & "Don't optimize prematurely." C++ Programmers love doing bitwise magic or raw pointer arithmetic to save 2 nanoseconds. 99% of the time, the compiler's optimizer flag (`-O3`) does it much better and safer. First, write CLEAN, INTENTIONAL, READABLE CODE. Second, use a PROFILER (like Valgrind/Callgrind). Third, ONLY optimize the bottleneck!

Erken Optimizasyon (Şeytanın İşi)

"Sebepsiz yere optimizasyon yapma!" ve "Erken optimizasyon bütün kötülüklerin anasıdır." C++ Mimarisi: 3 nanosaniye kazanmak için okunması imkansız Bitwise kodlar YAZMAYIN! * Halbuki `-O3` (Optimizasyon Bayrağı) ile çalışan C++ Derleyicisi sizden 100 kat akıllıdır. Önce TEMİZ, NİYETİ BELLİ OLAN KOD yaz (STL kullan). Sonra Valgrind Profiler ile ölç. Ve SADECE Oyunun Cidden Kasma Yaptığı yerleri (Bottleneck) optimize et!

CPPREF Depth: Hardware Concurrency limit & Cache Coherence!

CppReference warns against creating 1000 threads for performance. Use `std::thread::hardware_concurrency()` to find the exact number of physical/ logical cores you have. Spawning more threads than CPU cores causes extreme lag. Also, multiple threads modifying variables physically close together in RAM causes "False Sharing" (L1/L2 Cache Coherence ping-pong), drastically hurting performance!

CPPREF Derinlik: Donanım Eşzamanlılık Sınırı ve Önbellek Tutarlılığı!

CppReference'in dev uyarısı: Performans için "1000 tane Thread açayım" derseniz sistem çöker. Gerçek fiziksel ve mantıksal çekirdek sayısını öğrenmek için `std::thread::hardware_concurrency()` kullanılmalıdır (Maksimum thread limiti). İkinci Felaket: Birden çok Thread, RAM'de birbirine çok yakın duran iki farklı değişkene sürekli yazıp bozarsa, İşlemci Ön Bellekleri (L1/L2 Cache) çıldırır. Sürekli birbirlerinin verisini geçersiz kılarlar ("False Sharing" / Cache Coherence). Performans 10 kat düşer!

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <mutex>
#include <thread>
#include <vector>

std::mutex databaseLock;
int globalOnlineECUs = 0;

void registerECU_Bad() {
  // EN: CP.20 VIOLATION: Manual Lock/Unlock. VERY DANGEROUS!
  // TR: CORE GUIDELINE İHLALİ: Manuel Kilit. EĞER ARADA HATA ÇIKARSA SİSTEM ÇÖKER!
  databaseLock.lock();

  globalOnlineECUs++;
  // If a 'throw' happens exactly here, unlock() is silently skipped -> FATAL DEADLOCK!

  databaseLock.unlock();
}

void registerECU_Good() {
  // EN: C++ CORE GUIDELINES (CP.20): Use RAII (std::lock_guard explicitly)
  // TR: MÜKEMMEL KOD (RAII): Kapsama girince Mutex Kilitlenir, süslü parantez kapanınca Otomatik
  // Açılır!
  std::lock_guard<std::mutex> lock(databaseLock); // Automatic Safe Locker!

  globalOnlineECUs++;
  // EN: Even if an Exception throws here, RAII strictly un-winds the lock back to OS. No
  // deadlocks.
  // TR: Burada Hata/Exception atılsa dahi C++ sistemi kilidi garantili olarak İşletim sistemine
  // iade eder.
}

int main() {
  std::cout << "=== MODULE 8: C++ CORE GUIDELINES - CONCURRENCY & PERFORMANCE ===\n" << std::endl;

  std::vector<std::thread> workers;

  // EN: Launch 10 Threads simultaneously to register ECU modules.
  // TR: 10 Thread Aynı Anda ECU Kayıt (Simülasyon). Race Condition'ı RAII Lock Guard ile
  // engelliyoruz.
  for (int i = 0; i < 10; ++i) {
    workers.push_back(std::thread(registerECU_Good));
  }

  // EN: Join all threads (Wait for them to finish properly).
  // TR: Tüm çalışma grupları birleşene (Bitiş) kadar ana sunucuyu bekle.
  for (auto &w : workers) {
    w.join();
  }

  std::cout << "All ECUs Registered Safely! Total Online: " << globalOnlineECUs << std::endl;

  std::cout << "\n[ARCHITECTURAL CONCLUSION / MİMARİ SONUÇ]:" << std::endl;
  std::cout << "1. Threads + Shared Global Data = CATASTROPHE. Always use "
               "std::lock_guard / std::scoped_lock."
            << std::endl;
  std::cout << "2. PERFORMANCE MYTH: Let the '-O3' generic compiler flags do "
               "the 'clever' micro-optimizations!"
            << std::endl;
```

```cpp

    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: Per.7 — Design to enable optimization (data layout matters)
    // TR: Per.7 — Optimizasyonu mümkün kılacak şekilde tasarla (veri düzeni önemlidir)
    // ═══════════════════════════════════════════════════════════════════════════════

    // EN: C++ has no standard `restrict` keyword (C99 feature). Some compilers offer
    // `__restrict__` as an extension, but it's non-portable.
    // TR: C++'da standart bir `restrict` anahtar kelimesi yoktur (C99 özelliği). Bazı derleyiciler
    // `__restrict__` uzantısını sunar ama taşınabilir değildir.

    // ─── 1. Avoid unnecessary copies in range-for loops ──────────────────────────
    // EN: Using `auto` (value) copies every element. Use `const auto&` instead.
    // TR: `auto` (değer) her elemanı kopyalar. Bunun yerine `const auto&` kullanın.

    std::vector<std::string> sensorNames = {
        "Temperature", "Pressure", "Humidity", "Altitude", "Speed"};

    // EN: BAD — copies every string (expensive for large objects).
    // TR: KÖTÜ — her string'i kopyalar (büyük nesneler için pahalıdır). for (auto name :
    // sensorNames) { /* copy per iteration */ }

    // EN: GOOD — zero-copy read, compiler can auto-vectorize.
    // TR: İYİ — sıfır kopyalı okuma, derleyici otomatik vektörleştirebilir.
    std::cout << "\n[Per.7] Sensor names (const auto& — zero copy):" << std::endl;
    for (const auto& name : sensorNames) {
      std::cout << "  -> " << name << std::endl;
    }

    std::cout << "[Per.7] Always use 'const auto&' in range-for when you only read!"
              << std::endl;

    return 0;
  }
```

► 05. Smart Pointer Guidelines

module_08_core_guidelines/05_smart_pointer_guidelines.cpp

Konu: C++ Core Guidelines: Smart Pointer Ownership Rules — Akıllı İşaretçi Sahiplik Kuralları

Teori / Theory

[R.20: Use unique_ptr to represent exclusive ownership / Özel sahiplik için unique_ptr kullanın]

When exactly ONE entity owns a resource, use `std::unique_ptr`. It has ZERO runtime overhead compared to a raw pointer — the compiler generates identical machine code. The destructor automatically frees the resource when the owning scope exits, preventing every possible memory leak.

Bir kaynağın yalnızca TEK bir sahibi varsa `std::unique_ptr` kullanın. Ham işaretçiye (raw pointer) kıyasla SIFIR çalışma zamanı maliyeti vardır — derleyici birebir aynı makine kodunu üretir. Sahip olan kapsam çıkınca yıkıcı kaynağı otomatik serbest bırakır, bellek sızıntısı imkansızlaşır.

[R.21: Prefer unique_ptr over shared_ptr unless you need shared ownership / Paylaşımlı sahiplik gerekmiyorsa unique_ptr tercih edin]

`shared_ptr` carries atomic reference-count overhead (control block + two atomic increments/decrements per copy). Default to `unique_ptr`; promote to `shared_ptr` ONLY when multiple owners genuinely need the same resource.

`shared_ptr` atomik referans sayacı yükü taşır (kontrol bloğu + her kopyada iki atomik artış/azalış). Varsayılan olarak `unique_ptr` kullanın; SADECE birden fazla sahip aynı kaynağa gerçekten ihtiyaç duyduğunda `shared_ptr`'ye terfi ettirin.

R.22: Use make_shared / make_unique — Exception safety + single allocation

`std::make_unique<T>(args)` and `std::make_shared<T>(args)` prevent memory leaks in complex expressions and, for shared_ptr, fuse the control block with the object into ONE allocation (cache-friendly, fewer calls to the allocator).

`make_unique` ve `make_shared` karmaşık ifadelerde bellek sızıntısını önler. `make_shared` ayrıca kontrol bloğu ile nesneyi TEK bir bellek tahsisinde birleştirir (önbellek dostu, daha az tahsis çağrısı).

R.24: Aliasing constructor — shared_ptr to a sub-object

The aliasing constructor `shared_ptr<T>(owner, &owner->member)` lets you hand out a pointer to a MEMBER while keeping the PARENT alive. No extra allocation. The control block is shared with the parent.

Aliasing constructor `shared_ptr<T>(sahip, &sahip->uye)` ile ANA nesneyi canlı tutarken bir ÜYE'ye işaretçi verebilirsiniz. Ekstra tahsis yok, kontrol bloğu ana nesneyle paylaşılır.

R.30: Take smart pointers as parameters ONLY to express lifetime semantics

If a function only READS or USES an object, take `const T&` or `T*`. Accepting `unique_ptr<T>` means "I will take ownership." Accepting `shared_ptr<T>` means "I may share ownership." Passing smart pointers when you don't need ownership semantics pollutes the API and forces callers into a specific allocation strategy.

Bir fonksiyon yalnızca nesneyi OKUYORSA veya KULLANIYORSA `const T&` veya `T*` alın. `unique_ptr<T>` kabul etmek "sahipliği devralıyorum" demek, `shared_ptr<T>` kabul etmek "sahipliği paylaşabilirim" demektir. Sahiplik semantiği gerekmediğinde akıllı işaretçi geçmek API'yi kirletir.

CPPREF Depth: shared_ptr Control Block and Aliasing Constructor Internals

`std::shared_ptr<T>` internally holds TWO pointers: one to the managed object and one to a control block. The control block stores the strong reference count, the weak reference count, the deleter, and (optionally) the allocator. `std::make_shared<T>(args...)` performs a SINGLE heap allocation for both the object and the control block — more cache-friendly and fewer allocator calls versus `shared_ptr<T>(new T)`. The aliasing constructor `shared_ptr<U>(shared_ptr<T>, U*)` shares the control block of T but points to a different object U — useful for pointing to a member of a shared object without extra ref counting. `enable_shared_from_this<T>` embeds a hidden `weak_ptr<T>`; calling `shared_from_this()` on an object NOT managed by a `shared_ptr` is UB.

CPPREF Derinlik: shared_ptr Kontrol Bloğu ve Aliasing Kurucu İç Yapısı

`std::shared_ptr<T>` dahili olarak İKİ işaretçi tutar: biri yönetilen nesneye, biri kontrol bloğuna. Kontrol bloğu güçlü referans sayısını, zayıf referans sayısını, siliciyi ve (isteğe bağlı) ayırıcıyı saklar. `std::make_shared<T>(args...)` nesne ve kontrol bloğu için TEK yığın tahsisi yapar — `shared_ptr<T>(new T)`'ye kıyasla daha önbellek dostu. Aliasing kurucu `shared_ptr<U>(shared_ptr<T>, U*)` T'nin kontrol bloğunu paylaşır ama farklı bir U nesnesine işaret eder. `enable_shared_from_this` gizli bir `weak_ptr<T>` gömer; `shared_ptr` tarafından yönetilmeyen bir nesnede `shared_from_this()` çağırmak tanımsız davranıştır.

Kaynak Kod / Source Code

```cpp
#include <chrono>   // EN: high_resolution_clock for timing benchmarks
#include <iostream> // EN: standard I/O
#include <memory>   // EN: smart pointers (unique_ptr, shared_ptr, weak_ptr)
#include <string>   // EN: std::string
#include <vector>   // EN: std::vector

// ═══════════════════════════════════════════════════════════════════════════
// PRODUCTION CODE — Automotive Smart Pointer Architecture
// ═══════════════════════════════════════════════════════════════════════════

// ─── 1. unique_ptr Factory Pattern ─────────────────────────────────────

enum class ModuleType { BCM, Powertrain, ADAS, Infotainment };

class ECUModule {
public:
    std::string name;
    int firmwareVersion;

    ECUModule(std::string n, int fw) : name(std::move(n)), firmwareVersion(fw) {}
    virtual ~ECUModule() = default;

    virtual void selfTest() const {
        std::cout << "  [ECU] " << name << " v" << firmwareVersion
                  << " — generic self-test PASS" << std::endl;
    }
};

class ADASModule : public ECUModule {
public:
    ADASModule() : ECUModule("ADAS", 3) {}
    void selfTest() const override {
        std::cout << "  [ADAS] Camera + LiDAR calibration — PASS" << std::endl;
    }
};

// EN: R.20 + R.22 — factory returns unique_ptr (exclusive ownership)
// TR: R.20 + R.22 — fabrika unique_ptr döndürür (özel sahiplik)
std::unique_ptr<ECUModule> createModule(ModuleType type) {
    switch (type) {
        case ModuleType::BCM:
            return std::make_unique<ECUModule>("BCM", 7);
        case ModuleType::Powertrain:
            return std::make_unique<ECUModule>("Powertrain", 12);
        case ModuleType::ADAS:
            return std::make_unique<ADASModule>();
        case ModuleType::Infotainment:
            return std::make_unique<ECUModule>("Infotainment", 5);
    }
    return nullptr;
}
```

```cpp

// ─── 2. shared_ptr — Shared Sensor Data Pool ──────────────────────────────────

struct SensorDataPool {
    std::vector<double> temperatures;
    std::vector<double> pressures;

    SensorDataPool() : temperatures(128, 0.0), pressures(64, 0.0) {
        std::cout << "  [SensorDataPool] Allocated (128 temp + 64 pressure slots)"
                  << std::endl;
    }
    ~SensorDataPool() {
        std::cout << "  [SensorDataPool] Released — all consumers done" << std::endl;
    }
};

// ─── 3. Aliasing Constructor — Engine / Turbo ─────────────────────────────────

struct Turbo {
    int boostPSI = 22;
};

struct Engine {
    std::string model = "V6-TwinTurbo";
    Turbo turbo;
};

// ─── 4. R.30 — Correct function signatures ────────────────────────────────────

// EN: GOOD — function only reads the object, takes const reference
// TR: İYİ — fonksiyon sadece nesneyi okur, const referans alır
void diagnose(const ECUModule& module) {
    std::cout << "  [Diag] Running diagnostics on: " << module.name << std::endl;
    module.selfTest();
}

// EN: GOOD — function takes ownership, accepts unique_ptr by value
// TR: İYİ — fonksiyon sahipliği devralır, unique_ptr değer olarak kabul eder
void installModule(std::unique_ptr<ECUModule> module) {
    std::cout << "  [Install] Installed " << module->name
              << " (fw v" << module->firmwareVersion << ")" << std::endl;
    // EN: module is automatically destroyed at end of scope
    // TR: module kapsam sonunda otomatik yok edilir
}

// ─── 5. weak_ptr Observer — Cycle Breaking ────────────────────────────────────

class SensorObserver {
    std::weak_ptr<SensorDataPool> pool_;
    std::string observerName_;

public:
    SensorObserver(std::shared_ptr<SensorDataPool> pool, std::string name)
        : pool_(pool), observerName_(std::move(name)) {}

    void readLatest() const {
        // EN: lock() promotes weak_ptr to shared_ptr if the object is alive
        // TR: lock() nesne hayattaysa weak_ptr'yi shared_ptr'ye yükseltir
        if (auto sp = pool_.lock()) {
            std::cout << "  [" << observerName_ << "] Pool alive — temp slots: "
                      << sp->temperatures.size() << std::endl;
        } else {
            std::cout << "  [" << observerName_ << "] Pool EXPIRED — nothing to read"
                      << std::endl;
        }
    }
};

// ═══════════════════════════════════════════════════════════════════════════════
// MAIN — Demonstrations
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "=== MODULE 8: C++ CORE GUIDELINES — SMART POINTER OWNERSHIP ===\n"
              << std::endl;
```

```cpp

    // ─── Demo 1: unique_ptr Factory & Ownership Transfer ──────────────────────────
    std::cout << "[Demo 1] unique_ptr Factory & std::move\n";
    {
        auto bcm = createModule(ModuleType::BCM);
        auto adas = createModule(ModuleType::ADAS);

        // EN: R.30 — diagnose does NOT take ownership, receives const ref
        // TR: R.30 — diagnose sahiplik almaz, const referans alır
        diagnose(*bcm);
        diagnose(*adas);

        // EN: Transfer ownership via std::move — bcm is now nullptr
        // TR: std::move ile sahiplik devri — bcm artık nullptr
        installModule(std::move(bcm));
        if (!bcm) {
            std::cout << "  bcm is nullptr after std::move (ownership transferred)\n";
        }
    }

    std::cout << std::endl;

    // ─── Demo 2: shared_ptr Reference Counting ────────────────────────────────────
    std::cout << "[Demo 2] shared_ptr Reference Counting\n";
    {
        auto pool = std::make_shared<SensorDataPool>();
        std::cout << "  use_count after creation  : " << pool.use_count() << std::endl;

        {
            // EN: Second consumer shares ownership
            // TR: İkinci tüketici sahipliği paylaşır
            auto consumer2 = pool;
            std::cout << "  use_count with 2 consumers: " << pool.use_count() << std::endl;

            auto consumer3 = pool;
            std::cout << "  use_count with 3 consumers: " << pool.use_count() << std::endl;
        }
        // EN: consumer2 and consumer3 destroyed — count drops back to 1
        // TR: consumer2 ve consumer3 yok edildi — sayaç 1'e düştü
        std::cout << "  use_count after inner scope: " << pool.use_count() << std::endl;
    }

    std::cout << std::endl;

    // ─── Demo 3: Aliasing Constructor ─────────────────────────────────────────────
    std::cout << "[Demo 3] Aliasing Constructor (R.24)\n";
    {
        auto engine = std::make_shared<Engine>();

        // EN: Aliasing — turboPtr points to engine->turbo but shares ownership
        // TR: Aliasing — turboPtr engine->turbo'yu gösterir ama sahipliği paylaşır
        std::shared_ptr<Turbo> turboPtr(engine, &engine->turbo);

        std::cout << "  engine use_count : " << engine.use_count() << std::endl;
        std::cout << "  turbo boost (PSI): " << turboPtr->boostPSI << std::endl;

        engine.reset();
        // EN: Engine object still alive because turboPtr holds shared ownership
        // TR: Engine nesnesi hâlâ hayatta çünkü turboPtr paylaşımlı sahiplik tutuyor
        std::cout << "  turbo boost after engine.reset(): " << turboPtr->boostPSI
                  << " (engine kept alive by alias)" << std::endl;
    }

    std::cout << std::endl;

    // ─── Demo 4: weak_ptr Observer — Detect Expired Objects ───────────────────────
    std::cout << "[Demo 4] weak_ptr Observer Pattern\n";
    {
        SensorObserver observer(nullptr, "LateBinder");

        {
            auto pool = std::make_shared<SensorDataPool>();
            observer = SensorObserver(pool, "TempMonitor");

```

```
            // EN: Pool is alive — observer can read
            // TR: Pool hayatta — gözlemci okuyabilir
            observer.readLatest();
        }
        // EN: Pool is destroyed — weak_ptr detects expiration
        // TR: Pool yok edildi — weak_ptr süresinin dolduğunu tespit eder
        observer.readLatest();
    }

    std::cout << std::endl;

    // ─── Demo 5: Performance — raw vs unique vs shared creation timing ──────────────────────────
    std::cout << "[Demo 5] Performance Comparison (creation timing)\n";
    {
        constexpr int N = 500000;

        // EN: Raw pointer allocation
        // TR: Ham işaretçi tahsisi
        auto t0 = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < N; ++i) {
            auto* p = new ECUModule("raw", i);
            delete p;
        }
        auto t1 = std::chrono::high_resolution_clock::now();

        // EN: unique_ptr allocation (make_unique)
        // TR: unique_ptr tahsisi (make_unique)
        auto t2 = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < N; ++i) {
            auto p = std::make_unique<ECUModule>("unique", i);
        }
        auto t3 = std::chrono::high_resolution_clock::now();

        // EN: shared_ptr allocation (make_shared)
        // TR: shared_ptr tahsisi (make_shared)
        auto t4 = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < N; ++i) {
            auto p = std::make_shared<ECUModule>("shared", i);
        }
        auto t5 = std::chrono::high_resolution_clock::now();

        auto rawMs = std::chrono::duration_cast<std::chrono::milliseconds>(t1 - t0).count();
        auto uniqMs = std::chrono::duration_cast<std::chrono::milliseconds>(t3 - t2).count();
        auto shrMs = std::chrono::duration_cast<std::chrono::milliseconds>(t5 - t4).count();

        std::cout << "  " << N << " allocations each:\n";
        std::cout << "    raw new/delete : " << rawMs << " ms\n";
        std::cout << "    make_unique    : " << uniqMs << " ms\n";
        std::cout << "    make_shared    : " << shrMs << " ms\n";
        std::cout << "  (unique ≈ raw; shared slightly slower due to control block)"
                  << std::endl;
    }

    std::cout << "\n[ARCHITECTURAL CONCLUSION / MİMARİ SONUÇ]:" << std::endl;
    std::cout << "1. Default to unique_ptr (zero overhead, exclusive ownership)." << std::endl;
    std::cout << "2. Use shared_ptr ONLY when multiple owners genuinely exist." << std::endl;
    std::cout << "3. Never pass smart pointers to functions that don't need ownership."
              << std::endl;

    return 0;
}
```

► 06. Class Design Guidelines

module_08_core_guidelines/06_class_design_guidelines.cpp

Konu: C++ Core Guidelines: Class Design Rules — Sınıf Tasarım Kuralları

Teori / Theory

C.20: Rule of Zero — Avoid defining default operations if you can

If your class members are ALL RAII types (std::string, std::vector, std::unique_ptr), the compiler-generated destructor, copy, and move operations are ALREADY CORRECT. Writing them manually is error-prone and redundant. Let the compiler do its job.

Sınıfınızın tüm üyeleri RAII türlerinden (std::string, std::vector, std::unique_ptr) oluşuyorsa, derleyicinin otomatik ürettiği yıkıcı, kopya ve taşıma işlemleri ZATEN DOĞRUDUR. Manuel yazmak hata kaynağıdır ve gereksizdir. Derleyicinin işini yapmasına izin verin.

C.21: Define or =delete ALL special members if you define any

If you write a custom destructor, copy constructor, or copy assignment, you almost certainly need to define (or delete) ALL of them. This is the "Rule of Five" in modern C++. Partial definitions lead to subtle bugs like double-free or missing move semantics.

Özel bir yıkıcı, kopya kurucu veya kopya atama yazıyorsanız, neredeyse kesinlikle HEPSİNİ tanımlamanız (veya silmeniz) gerekir. Bu modern C++'da "Beşli Kuralı"dır. Kısmi tanımlamalar çift serbest bırakma veya eksik taşıma semantiği gibi ince hatalara yol açar.

C.35: A base class destructor should be public+virtual or protected+non-virtual

If you delete through a base pointer, the destructor MUST be virtual. Otherwise the derived destructor never runs — resource leak. If deletion through base is not intended, make the destructor protected and non-virtual.

Bir temel sınıf işaretçisi üzerinden silme yapıyorsanız yıkıcı KESİNLİKLE sanal (virtual) olmalıdır. Aksi halde türetilmiş yıkıcı asla çalışmaz — kaynak sızıntısı. Temel sınıf üzerinden silme amaçlanmıyorsa yıkıcıyı korumalı (protected) ve sanal olmayan yapın.

C.67: Polymorphic class should suppress copying

Copying a polymorphic object through a base pointer causes "Object Slicing" — the derived part is silently chopped off. Delete copy operations or make them protected. Use clone() (virtual copy) if deep copying is needed.

Polimorfik bir nesneyi temel işaretçi üzerinden kopyalamak "Object Slicing"e neden olur — türetilmiş kısım sessizce kesilir. Kopya işlemlerini silin veya korumalı yapın. Derin kopya gerekiyorsa clone() kullanın.

C.128 / C.129: virtual → override → final & Interface vs Implementation

Mark every overriding function `override` so the compiler catches typos. Use `final` to prevent further overriding of a specific method or to seal an entire class. Separate INTERFACE inheritance (pure virtuals) from IMPLEMENTATION inheritance (shared base logic) — Liskov Substitution Principle: every subtype must be usable wherever the base type is expected.

Her geçersiz kılan fonksiyonu `override` ile işaretleyin, derleyici yazım hatalarını yakalasın. Belirli bir metodu veya tüm sınıfı mühürlemek için `final` kullanın. ARAYÜZ kalıtımını (saf sanal) UYGULAMA kalıtımından (ortak temel mantık) ayırın — Liskov İkame Prensibi: her alt tür, temel türün beklendiği her yerde kullanılabilir olmalıdır.

CPPREF Depth: Liskov Substitution Principle in C++ — Beyond Theory

LSP states that if S is a subtype of T, objects of type T may be replaced with objects of type S without altering program correctness. Common violations include: strengthening preconditions (derived demands more than base promised), weakening postconditions (derived delivers less than base guaranteed), and changing exception specifications. The `override` keyword (C++11) enforces exact signature matching with the base — a misspelled or mistyped override becomes a compile error. `final` prevents further overriding of a particular virtual method or prevents deriving from a class entirely. Composition over inheritance: prefer has-a (member) relationships to is-a (inheritance) except when runtime polymorphism through interfaces (pure virtual base) is needed. A well-designed hierarchy has a thin interface base and concrete leaves.

CPPREF Derinlik: Liskov İkame Prensibi — C++'ta Teorinin Ötesi

LSP, S tipi T'nin alt tipi ise T nesnelerinin S ile yer değiştirilebileceğini ve program doğruluğunun bozulmaması gerektiğini belirtir. Yaygın ihlaller: ön-koşulları güçlendirmek (türetilmiş sınıf tabandan fazlasını talep eder), son-koşulları zayıflatmak (türetilmiş sınıf tabandan azını garanti eder), istisna belirtimlerini değiştirmek. `override` (C++11) taban ile tam imza eşleşmesini zorlar. `final` belirli bir sanal fonksiyonun daha fazla geçersiz kılınmasını veya sınıftan türetilmesini engeller. Kalıtım yerine bileşim: çalışma zamanı çok biçimliliği (saf sanal taban) dışında has-a (üye) ilişkilerini is-a (kalıtım) ilişkisine tercih edin.

Kaynak Kod / Source Code

```cpp
#include <iostream> // EN: standard I/O
#include <memory>   // EN: smart pointers for RAII members
#include <string>   // EN: std::string
#include <vector>   // EN: std::vector

// ═══════════════════════════════════════════════════════════════════════════
// PRODUCTION CODE — Automotive Class Design
// ═══════════════════════════════════════════════════════════════════════════

// ─── 1. Rule of Zero: SafeVehicleData ──────────────────────────────────

// EN: C.20 — all members are RAII types; no user-defined special members needed
// TR: C.20 — tüm üyeler RAII türlerinden; kullanıcı tanımlı özel üye gerekmez
class SafeVehicleData {
public:
    std::string vin;
    std::vector<double> sensorReadings;
    std::unique_ptr<std::string> diagnosticLog;

    // EN: A regular constructor is fine — Rule of Zero still applies
    // TR: Normal bir kurucu sorun değil — Sıfır Kuralı hâlâ geçerli
    SafeVehicleData(std::string v, std::vector<double> readings)
        : vin(std::move(v)),
          sensorReadings(std::move(readings)),
          diagnosticLog(std::make_unique<std::string>("Initial log entry")) {}

```

```
27      void print() const {
28          std::cout << "  VIN: " << vin
29                    << " | Sensors: " << sensorReadings.size()
30                    << " | Log: " << (diagnosticLog ? *diagnosticLog : "null")
31                    << std::endl;
32      }
33  };
34
35  // ─── 2. Abstract Interface: IDiagnosticProtocol ──────────────────────────────────────
36
37  // EN: C.129 — pure interface (no data, all pure virtual)
38  // TR: C.129 — saf arayüz (veri yok, hepsi saf sanal)
39  class IDiagnosticProtocol {
40  public:
41      virtual ~IDiagnosticProtocol() = default;
42
43      // EN: C.67 — suppress copying on polymorphic base
44      // TR: C.67 — polimorfik temel sınıfta kopyalamayı engelle
45      IDiagnosticProtocol(const IDiagnosticProtocol&) = delete;
46      IDiagnosticProtocol& operator=(const IDiagnosticProtocol&) = delete;
47
48      virtual std::string protocolName() const = 0;
49      virtual bool connect(const std::string& target) = 0;
50      virtual std::string readDTC() const = 0;
51      virtual void clearDTC() = 0;
52
53  protected:
54      IDiagnosticProtocol() = default;
55      IDiagnosticProtocol(IDiagnosticProtocol&&) = default;
56      IDiagnosticProtocol& operator=(IDiagnosticProtocol&&) = default;
57  };
58
59  // ─── 3. Concrete: OBD2Protocol ───────────────────────────────────────────────────────
60
61  class OBD2Protocol : public IDiagnosticProtocol {
62      std::string connectedECU_;
63      bool connected_ = false;
64
65  public:
66      OBD2Protocol() = default;
67
68      // EN: C.128 — every override explicitly marked
69      // TR: C.128 — her geçersiz kılma açıkça işaretlendi
70      std::string protocolName() const override { return "OBD-II (ISO 15765)"; }
71
72      bool connect(const std::string& target) override {
73          connectedECU_ = target;
74          connected_ = true;
75          std::cout << "  [OBD2] Connected to " << target << std::endl;
76          return true;
77      }
78
79      std::string readDTC() const override {
80          if (!connected_) return "NOT_CONNECTED";
81          return "P0301 — Cylinder 1 Misfire Detected";
82      }
83
84      void clearDTC() override {
85          std::cout << "  [OBD2] DTCs cleared on " << connectedECU_ << std::endl;
86      }
87  };
88
89  // ─── 4. Concrete: CANProtocol ────────────────────────────────────────────────────────
90
91  class CANProtocol : public IDiagnosticProtocol {
92      std::string busName_;
93      bool connected_ = false;
94
95  public:
96      CANProtocol() = default;
97
98      std::string protocolName() const override { return "CAN 2.0B (ISO 11898)"; }
99
```

```cpp
    bool connect(const std::string& target) override {
        busName_ = target;
        connected_ = true;
        std::cout << "  [CAN] Joined bus: " << target << std::endl;
        return true;
    }

    std::string readDTC() const override {
        if (!connected_) return "NOT_CONNECTED";
        return "U0100 — Lost Communication with ECM/PCM";
    }

    void clearDTC() override {
        std::cout << "  [CAN] DTCs cleared on bus " << busName_ << std::endl;
    }
};

// ─── 5. Final Class: HardcodedConfig ──────────────────────────────────────────

// EN: C.128 final — sealed class, no further derivation allowed
// TR: C.128 final — mühürlü sınıf, daha fazla türetme yasak
class HardcodedConfig final {
    int baudRate_;
    std::string protocol_;

public:
    HardcodedConfig(int baud, std::string proto)
        : baudRate_(baud), protocol_(std::move(proto)) {}

    void print() const {
        std::cout << "  [Config] " << protocol_ << " @ " << baudRate_
                  << " baud (FINAL — immutable design)" << std::endl;
    }
};

// ─── 6. Deleted Copy: SingletonECU ────────────────────────────────────────────

// EN: C.21 + C.67 — singleton must not be copied or moved
// TR: C.21 + C.67 — tekil nesne kopyalanamaz veya taşınamaz
class SingletonECU {
    std::string ecuId_;

    // EN: Private constructor — only accessible via instance()
    // TR: Özel kurucu — yalnızca instance() üzerinden erişilebilir
    explicit SingletonECU(std::string id) : ecuId_(std::move(id)) {}

public:
    // EN: Delete ALL copy and move operations (C.21)
    // TR: TÜM kopya ve taşıma işlemlerini sil (C.21)
    SingletonECU(const SingletonECU&) = delete;
    SingletonECU& operator=(const SingletonECU&) = delete;
    SingletonECU(SingletonECU&&) = delete;
    SingletonECU& operator=(SingletonECU&&) = delete;

    static SingletonECU& instance() {
        static SingletonECU inst("MasterECU-001");
        return inst;
    }

    void identify() const {
        std::cout << "  [Singleton] I am: " << ecuId_
                  << " (one and only instance)" << std::endl;
    }
};

// ═══════════════════════════════════════════════════════════════════════════════
// Helper: Polymorphic dispatch through interface pointer
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Liskov Substitution — works with ANY IDiagnosticProtocol subtype
// TR: Liskov İkamesi — HERHANGİ bir IDiagnosticProtocol alt türüyle çalışır
void runDiagnosticSession(IDiagnosticProtocol& proto, const std::string& target) {
    std::cout << "  Protocol : " << proto.protocolName() << std::endl;
    proto.connect(target);
    std::cout << "  DTC Read : " << proto.readDTC() << std::endl;
    proto.clearDTC();
}
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════════
// MAIN — Demonstrations
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "=== MODULE 8: C++ CORE GUIDELINES — CLASS DESIGN ===\n"
              << std::endl;

    // ─── Demo 1: Rule of Zero ──────────────────────────────────────────────
    std::cout << "[Demo 1] Rule of Zero — SafeVehicleData\n";
    {
        SafeVehicleData car1("WBA12345678901234", {36.5, 37.1, 38.0});
        car1.print();

        // EN: Move semantics work automatically — no user-defined move ctor
        // TR: Taşıma semantiği otomatik çalışır — kullanıcı tanımlı taşıma yok
        SafeVehicleData car2 = std::move(car1);
        std::cout << "  After move:\n";
        std::cout << "    car2 -> ";
        car2.print();
        std::cout << "    car1 -> ";
        car1.print();
    }

    std::cout << std::endl;

    // ─── Demo 2: Polymorphism through Interface ────────────────────────────
    std::cout << "[Demo 2] IDiagnosticProtocol — Polymorphic Dispatch\n";
    {
        OBD2Protocol obd2;
        CANProtocol can;

        std::cout << "  --- OBD2 Session ---\n";
        runDiagnosticSession(obd2, "EngineCU");

        std::cout << "  --- CAN Session ---\n";
        runDiagnosticSession(can, "CAN-HS-Bus");
    }

    std::cout << std::endl;

    // ─── Demo 3: Final Class ───────────────────────────────────────────────
    std::cout << "[Demo 3] final class — HardcodedConfig\n";
    {
        HardcodedConfig cfg(500000, "CAN-FD");
        cfg.print();

        // EN: Attempting `class Derived : public HardcodedConfig {}` would be a compile error
        // because HardcodedConfig is final.
        // TR: `class Turetilmis : public HardcodedConfig {}` denemek derleme hatası verir çünkü
        // HardcodedConfig final'dır.
        std::cout << "  (Any attempt to inherit from HardcodedConfig → compile error)"
                  << std::endl;
    }

    std::cout << std::endl;

    // ─── Demo 4: Deleted Copy Operations ───────────────────────────────────
    std::cout << "[Demo 4] Deleted Copy — SingletonECU\n";
    {
        SingletonECU& ecu = SingletonECU::instance();
        ecu.identify();

        SingletonECU& ecu2 = SingletonECU::instance();
        ecu2.identify();

        // EN: Both references point to the exact same object
        // TR: Her iki referans da tam olarak aynı nesneyi gösterir
        std::cout << "  Same instance? "
                  << (&ecu == &ecu2 ? "YES" : "NO") << std::endl;

```

```
        // EN: Uncommenting the next line would cause a compile error: SingletonECU copy = ecu;
        // // DELETED — C.21 enforced
        // TR: Aşağıdaki satırı açmak derleme hatası verir: SingletonECU kopya = ecu; // SİLİNMİŞ
        // — C.21 uygulanmış
    }

    std::cout << std::endl;

    // ─── Demo 5: override and final on Virtual Methods ──────────────────────────────
    std::cout << "[Demo 5] override / final Keywords\n";
    {
        // EN: Demonstrate that override catches errors at compile time
        // TR: override'ın derleme zamanında hataları yakaladığını göster
        std::unique_ptr<IDiagnosticProtocol> proto =
            std::make_unique<OBD2Protocol>();
        std::cout << "  Protocol via base ptr: " << proto->protocolName() << std::endl;

        auto proto2 = std::make_unique<CANProtocol>();
        std::cout << "  Protocol via base ptr: " << proto2->protocolName() << std::endl;

        // EN: Both correctly dispatch through virtual table thanks to override
        // TR: override sayesinde her ikisi de sanal tablo aracılığıyla doğru gönderilir
        std::cout << "  (Removing 'override' keyword would hide typo-bugs silently)"
                  << std::endl;
    }

    std::cout << "\n[ARCHITECTURAL CONCLUSION / MİMARİ SONUÇ]:" << std::endl;
    std::cout << "1. Rule of Zero: Let RAII members handle resource management." << std::endl;
    std::cout << "2. Pure interfaces + override/final = safe polymorphic hierarchies." << std::endl;
    std::cout << "3. Delete copy/move on singletons and polymorphic bases." << std::endl;

    return 0;
}
```

► 07. Api Design

module_08_core_guidelines/07_api_design.cpp

Konu: API Design in Modern C++ — Modern C++'da API Tasarımı

Teori / Theory

THEORY: Principles of Good C++ API Design

A well-designed API should be:
1. EASY TO USE CORRECTLY — intuitive names, strong types
2. HARD TO USE INCORRECTLY — compile-time errors > runtime errors
3. STABLE — ABI/API breaks are documented, versioned
4. MINIMAL — expose only what users need
5. CONSISTENT — follow patterns from std library

TEORİ: İyi C++ API Tasarımının İlkeleri

İyi tasarlanmış bir API şunları sağlamalıdır:
1. DOĞRU KULLANMAK KOLAY — sezgisel isimler, güçlü türler
2. YANLIŞ KULLANMAK ZOR — derleme zamanı hataları > çalışma zamanı hataları
3. KARARLI — ABI/API kırılmaları belgelenir, sürümlenir
4. MİNİMAL — sadece kullanıcıların ihtiyaç duyduğunu aç
5. TUTARLI — std kütüphanesinden kalıpları izle

→ Ref: C++ Core Guidelines: https://isocpp.github.io/CppCoreGuidelines/

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <string>
#include <memory>
#include <stdexcept>
#include <vector>
#include <type_traits>
#include <functional>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: Strong Types / NamedType Pattern
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: PROBLEM: Weak types lead to "primitive obsession" bugs:
 *       void connect(string host, int port, int timeout_ms, int retry_count);
 *       connect("server", 5000, 8080, 3);  // BUG: port and timeout swapped!
 *       Compiles fine but the bug is silent. The types are too weak.
 *
 *     SOLUTION: Strong types (NamedType) wrap primitives with distinct types:
 *       connect(Host{"server"}, Port{8080}, Timeout{5000ms}, RetryCount{3});
 *       connect(Host{"server"}, Timeout{5000ms}, Port{8080}, RetryCount{3});
 *       // ↑ COMPILE ERROR: Port and Timeout are different types!
 *
 * TR: SORUN: Zayıf türler sessiz hatalara yol açar (port ve timeout karıştırılır).
 *     ÇÖZÜM: Güçlü türler (NamedType) ilkel türleri farklı türlerle sarar.
 */
template<typename T, typename Tag>
class StrongType {
public:
    explicit StrongType(T value) : value_(std::move(value)) {}
    T& get() { return value_; }
    const T& get() const { return value_; }
private:
    T value_;
};

// EN: Define distinct types — they're all `int` underneath but incompatible
// TR: Farklı türler tanımla — hepsi altta `int` ama birbirleriyle uyumsuz
using Port       = StrongType<int, struct PortTag>;
using TimeoutMs  = StrongType<int, struct TimeoutTag>;
using RetryCount = StrongType<int, struct RetryTag>;

struct ConnectionConfig {
    std::string host;
    Port port;
    TimeoutMs timeout;
    RetryCount retries;
};

// EN: Now the API is impossible to misuse — types enforce correct ordering
// TR: Artık API'yi yanlış kullanmak imkansız — türler doğru sırayı zorlar
bool connect_to_server(const ConnectionConfig& config) {
    std::cout << "    Connecting to " << config.host
              << ":" << config.port.get()
              << " (timeout=" << config.timeout.get() << "ms"
              << ", retries=" << config.retries.get() << ")\n";
    return true;
}

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: PIMPL (Pointer to Implementation)
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: PIMPL (Pointer to IMPLementation) hides implementation details behind
 *     a pointer to a forward-declared class. Benefits:
 *
 *     1. ABI STABILITY: Changing private members doesn't break binary compatibility
 *        → Users don't need to recompile when implementation changes
 *     2. COMPILE FIREWALL: Header doesn't include implementation headers
 *        → Faster builds (fewer transitive includes)
 *     3. CLEAN HEADER: Public header shows only the API, no private details
 *
 *     Cost: One extra heap allocation + pointer indirection (usually negligible)
 *
```

```
 *     Pattern:
 *       // sensor.hpp (PUBLIC HEADER — shipped to users)
 *       class Sensor {
 *       public:
 *           Sensor();
 *           ~Sensor();
 *           double read();
 *       private:
 *           struct Impl;                      // Forward declaration only!
 *           std::unique_ptr<Impl> pimpl_;     // Pointer to hidden impl
 *       };
 *
 *       // sensor.cpp (PRIVATE — not shipped to users)
 *       struct Sensor::Impl {
 *           double calibration;
 *           FilterAlgorithm filter;  // Heavy internal dependency
 *           DatabaseConnection db;   // Users don't need to know about this!
 *       };
 *
 * TR: PIMPL, uygulama detaylarını ileri bildirilen bir sınıfa işaretçi arkasında gizler.
 *     1. ABI KARARLILIĞI: Özel üyeleri değiştirmek ikili uyumluluğu bozmaz
 *     2. DERLEME GÜVENLİK DUVARI: Başlık, uygulama başlıklarını içermez
 *     3. TEMİZ BAŞLIK: Genel başlık yalnızca API'yi gösterir
 */
class ECUDiagnostics {
public:
    ECUDiagnostics(const std::string& ecu_name);
    ~ECUDiagnostics();

    // EN: Move-only (because of unique_ptr) / TR: Sadece taşınabilir (unique_ptr yüzünden)
    ECUDiagnostics(ECUDiagnostics&& other) noexcept;
    ECUDiagnostics& operator=(ECUDiagnostics&& other) noexcept;

    // EN: Public API — stable ABI / TR: Genel API — kararlı ABI
    void run_diagnostics();
    int get_dtc_count() const;
    std::string get_summary() const;

private:
    struct Impl;                      // EN: Forward declaration / TR: İleri bildirim
    std::unique_ptr<Impl> pimpl_;     // EN: Hidden implementation / TR: Gizli uygulama
};

// EN: Implementation — normally in .cpp file, NOT in header
// TR: Uygulama — normalde .cpp dosyasında, başlıkta DEĞİL
struct ECUDiagnostics::Impl {
    std::string ecu_name;
    int dtc_count = 0;
    std::vector<std::string> log;

    void internal_scan() {
        log.push_back("Scanning ECU: " + ecu_name);
        dtc_count = 3;  // EN: Simulated / TR: Simüle edilmiş
        log.push_back("Found " + std::to_string(dtc_count) + " DTCs");
    }
};

ECUDiagnostics::ECUDiagnostics(const std::string& ecu_name)
    : pimpl_(std::make_unique<Impl>()) {
    pimpl_->ecu_name = ecu_name;
}

ECUDiagnostics::~ECUDiagnostics() = default;
ECUDiagnostics::ECUDiagnostics(ECUDiagnostics&&) noexcept = default;
ECUDiagnostics& ECUDiagnostics::operator=(ECUDiagnostics&&) noexcept = default;

void ECUDiagnostics::run_diagnostics() { pimpl_->internal_scan(); }
int ECUDiagnostics::get_dtc_count() const { return pimpl_->dtc_count; }

std::string ECUDiagnostics::get_summary() const {
    std::string result;
    for (const auto& entry : pimpl_->log) {
        result += "    " + entry + "\n";
    }
    return result;
}
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: [[deprecated]] Attribute & API Versioning
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: Inline namespaces enable API versioning without breaking existing code:
 *
 *     namespace mylib {
 *         inline namespace v2 {   // Current default version
 *             void process(int x);
 *         }
 *         namespace v1 {          // Old version still available
 *             [[deprecated("Use v2::process instead")]]
 *             void process(int x);
 *         }
 *     }
 *
 *     mylib::process(42);        // Calls v2::process (inline namespace = default)
 *     mylib::v1::process(42);    // Still compiles but warns "deprecated"
 *
 * TR: Inline ad alanları, mevcut kodu bozmadan API sürümlemeyi sağlar.
 */
namespace sensor_api {

    // EN: Version 1 — old API (deprecated but still accessible)
    // TR: Sürüm 1 — eski API (kullanımdan kaldırılmış ama erişilebilir)
    namespace v1 {
        [[deprecated("Use sensor_api::read_temperature() instead (v2 with units)")]]
        double read_temp(int sensor_id) {
            return 22.5 + static_cast<double>(sensor_id) * 0.1;
        }
    }

    // EN: Version 2 — current API (inline = default when using sensor_api::)
    // TR: Sürüm 2 — mevcut API (inline = sensor_api:: kullanıldığında varsayılan)
    inline namespace v2 {
        struct Temperature {
            double value;
            std::string unit;
        };

        Temperature read_temperature(int sensor_id) {
            return {22.5 + static_cast<double>(sensor_id) * 0.1, "°C"};
        }
    }
}

// ═══════════════════════════════════════════════════════════════════════════
// PART 4: Tag Dispatch & Overload Sets
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: Tag dispatch uses empty struct types to select function overloads
 *     at compile time. This avoids runtime branching and makes intent clear.
 *
 * TR: Tag dispatch, derleme zamanında fonksiyon aşırı yüklemelerini seçmek
 *     için boş struct türleri kullanır.
 */
namespace tags {
    struct Celsius {};
    struct Fahrenheit {};
    struct Kelvin {};
}

// EN: Same function name, different behavior based on tag type
// TR: Aynı fonksiyon adı, tag türüne göre farklı davranış
double convert_to_celsius(double value, tags::Celsius) {
    return value;  // EN: Already Celsius / TR: Zaten Celsius
}

double convert_to_celsius(double value, tags::Fahrenheit) {
    return (value - 32.0) * 5.0 / 9.0;
}
```

```cpp

double convert_to_celsius(double value, tags::Kelvin) {
    return value - 273.15;
}

// ═══════════════════════════════════════════════════════════════════════════
// PART 5: Header-Only Library Pattern
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: Header-only libraries put EVERYTHING in headers (no .cpp files).
 *     Popular examples: nlohmann/json, Catch2, Eigen.
 *
 *     Rules for header-only libraries:
 *     1. All functions must be `inline` (to avoid ODR violations)
 *     2. Templates are inherently inline (no keyword needed)
 *     3. Use `inline` for non-template functions and variables
 *     4. Consider a single-header approach (#include "lib.hpp")
 *
 * TR: Header-only kütüphaneler HER ŞEYİ başlıklara koyar (.cpp yok).
 *     Tüm fonksiyonlar `inline` olmalıdır (ODR ihlallerini önlemek için).
 */
namespace header_only_lib {

    // EN: `inline` is required for non-template functions in headers
    //     Without it: multiple definition error when included from 2+ .cpp files
    // TR: Başlıklarda şablon olmayan fonksiyonlar için `inline` gereklidir
    inline double clamp(double value, double low, double high) {
        return (value < low) ? low : (value > high) ? high : value;
    }

    // EN: Templates are implicitly inline — no keyword needed
    // TR: Şablonlar örtük olarak inline — anahtar kelime gerekmez
    template<typename Container>
    auto sum(const Container& c) {
        using T = typename Container::value_type;
        T result{};
        for (const auto& elem : c) result += elem;
        return result;
    }

    // EN: C++17 inline variables — header-safe global state
    // TR: C++17 inline değişkenler — başlık dosyası güvenli global durum
    inline constexpr double VERSION = 2.0;
}

// ═══════════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 08 - API Design in Modern C++\n";
    std::cout << "============================================\n\n";

    // ─── Demo 1: Strong Types ──────────────────────────────────────────
    {
        std::cout << "--- Demo 1: Strong Types (NamedType Pattern) ---\n";

        // EN: BEFORE (dangerous): connect("host", 5000, 8080, 3)
        //     Port and timeout easily swapped with no compiler error!
        // TR: ÖNCE (tehlikeli): port ve timeout kolayca karıştırılır!
        std::cout << "  Before (weak types): connect(host, port?, timeout?, retries?)\n";

        // EN: AFTER (safe): types ENFORCE correct usage
        // TR: SONRA (güvenli): türler doğru kullanımı ZORLAR
        ConnectionConfig cfg{
            "ecu-gateway.local",
            Port{8080},
            TimeoutMs{5000},
            RetryCount{3}
        };
        std::cout << "  After (strong types):\n";
        connect_to_server(cfg);
```

```

        // EN: This would NOT compile:
        // TR: Bu derlenmez:
        //   ConnectionConfig bad{"host", TimeoutMs{5000}, Port{8080}, RetryCount{3}};
        //   Error: cannot convert TimeoutMs to Port
        std::cout << "  (Swapping Port/Timeout gives COMPILE ERROR!)\n\n";
    }

    // ── Demo 2: PIMPL ──────────────────────────────────────────────
    {
        std::cout << "--- Demo 2: PIMPL (Pointer to Implementation) ---\n";
        ECUDiagnostics diag("ECU_ENGINE_01");
        diag.run_diagnostics();

        std::cout << "  DTC count: " << diag.get_dtc_count() << "\n";
        std::cout << "  Log:\n" << diag.get_summary();
        std::cout << "  sizeof(ECUDiagnostics) = " << sizeof(ECUDiagnostics)
                  << " bytes (just a unique_ptr!)\n\n";
    }

    // ── Demo 3: API Versioning ─────────────────────────────────────
    {
        std::cout << "--- Demo 3: API Versioning (inline namespace) ---\n";

        // EN: Default (v2) — called without version prefix
        // TR: Varsayılan (v2) — sürüm öneki olmadan çağrılır
        auto temp = sensor_api::read_temperature(1);
        std::cout << "  sensor_api::read_temperature(1) = "
                  << temp.value << " " << temp.unit << " (v2 — current)\n";

        // EN: Explicit v1 — compiles with deprecation warning
        // TR: Açık v1 — kullanımdan kaldırma uyarısıyla derlenir
        // double old_temp = sensor_api::v1::read_temp(1);
        std::cout << "  sensor_api::v1::read_temp(1) → [[deprecated]] warning\n\n";
    }

    // ── Demo 4: Tag Dispatch ───────────────────────────────────────
    {
        std::cout << "--- Demo 4: Tag Dispatch ---\n";
        std::cout << "  100°F → " << convert_to_celsius(100.0, tags::Fahrenheit{}) << " °C\n";
        std::cout << "  373K  → " << convert_to_celsius(373.15, tags::Kelvin{}) << " °C\n";
        std::cout << "  25°C  → " << convert_to_celsius(25.0, tags::Celsius{}) << " °C\n";
        std::cout << "  (No runtime branching — overload resolved at compile time)\n\n";
    }

    // ── Demo 5: Header-Only Library ────────────────────────────────
    {
        std::cout << "--- Demo 5: Header-Only Library Pattern ---\n";
        std::cout << "  clamp(150, 0, 100) = " << header_only_lib::clamp(150, 0, 100) << "\n";

        std::vector<double> data = {1.5, 2.5, 3.5, 4.5};
        std::cout << "  sum({1.5, 2.5, 3.5, 4.5}) = " << header_only_lib::sum(data) << "\n";
        std::cout << "  Library version: " << header_only_lib::VERSION << "\n\n";

        std::cout << "  Rules for header-only libraries:\n";
        std::cout << "    1. All functions: inline (ODR compliance)\n";
        std::cout << "    2. Templates: inherently inline\n";
        std::cout << "    3. Variables: inline constexpr (C++17)\n";
        std::cout << "    4. Classes: define in namespace, methods inline\n\n";
    }

    // ── Demo 6: API Design Checklist ───────────────────────────────
    {
        std::cout << "--- Demo 6: API Design Checklist ---\n";
        std::cout << "  □ Use strong types to prevent argument swapping\n";
        std::cout << "  □ PIMPL for ABI stability and compilation firewall\n";
        std::cout << "  □ [[deprecated]] for graceful API evolution\n";
        std::cout << "  □ inline namespace for API versioning\n";
        std::cout << "  □ Tag dispatch for zero-cost type-based dispatch\n";
        std::cout << "  □ const& for read-only params, value for to-be-moved\n";
        std::cout << "  □ [[nodiscard]] for functions where ignoring return is bug\n";
        std::cout << "  □ explicit constructors to prevent implicit conversions\n";
        std::cout << "  □ noexcept on move operations and destructors\n";
        std::cout << "  □ Header-only + inline for simple distribution\n\n";
    }

```

```
    std::cout << "==========================================\n";
    std::cout << "   End of API Design\n";
    std::cout << "==========================================\n";

    return 0;
}
```

► 08. Maintainable Cpp

module_08_core_guidelines/08_maintainable_cpp.cpp

Konu: Maintainable C++ in Big Projects — Büyük Projelerde Sürdürülebilir C++

Teori / Theory

THEORY: What Makes C++ Hard to Maintain?

Large C++ codebases (100K+ lines) face unique challenges:
1. COMPILE TIMES: A single header change can trigger rebuilds of 1000s of files
2. DEPENDENCY HELL: Circular includes, tight coupling, transitive dependencies
3. BUILD COMPLEXITY: Platform-specific code, conditional compilation (#ifdef soup)
4. IMPLICIT BEHAVIOR: Implicit conversions, copy vs move, ADL surprises
5. ABI FRAGILITY: Changing a class layout breaks binary compatibility
Key strategies:
- Modular architecture with clear dependency graphs
- Forward declarations to minimize includes
- PIMPL for API boundaries
- Static analysis (clang-tidy, cppcheck) in CI
- Code review checklists specific to C++

TEORİ: C++ Bakımını Zorlaştıran Nedir?

Büyük C++ projeleri (100K+ satır) benzersiz zorluklarla karşılaşır:
1. DERLEME SÜRELERİ: Tek bir başlık değişikliği binlerce dosyanın yeniden derlenmesini tetikler
2. BAĞIMLILIK KARMAŞASI: Dairesel include'lar, sıkı bağlama, geçişken bağımlılıklar
3. YAPI KARMAŞIKLIĞI: Platforma özel kod, koşullu derleme (#ifdef çorbası)
4. ÖRTÜK DAVRANIŞ: Örtük dönüşümler, copy vs move, ADL sürprizleri
5. ABI KIRILGANLIĞI: Sınıf düzenini değiştirmek ikili uyumluluğu bozar
Temel stratejiler:
- Net bağımlılık grafikleriyle modüler mimari
- Include'ları azaltmak için ileri bildirimler (forward declarations)
- API sınırlarında PIMPL
- CI'da statik analiz (clang-tidy, cppcheck)
- C++'a özel kod gözden geçirme kontrol listeleri

THEORY: Modular Architecture

Organize code into layers with clear dependency rules:

```
┌──────────────────────────────────────────────────┐
│                  Application Layer                  │
│    (main.cpp, CLI, GUI — depends on everything)     │
├──────────────────────────────────────────────────┤
│                  Service Layer                      │
│    (Business logic, orchestration)                  │
├──────────────────────────────────────────────────┤
│                  Domain Layer                       │
│    (Core types, interfaces — minimal dependencies)  │
├──────────────────────────────────────────────────┤
│                  Infrastructure Layer               │
│    (Database, network, file I/O — adapters)         │
├──────────────────────────────────────────────────┤
│                  Platform Layer                     │
│    (OS abstractions, HAL for embedded)              │
└──────────────────────────────────────────────────┘
```

Rules:
- Dependencies flow DOWNWARD only (no upward calls)
- Use interfaces (abstract classes) at layer boundaries
- Domain layer has ZERO external dependencies

TEORİ: Modüler Mimari

Kodu net bağımlılık kurallarıyla katmanlara düzenle:

```
┌──────────────────────────────────────────────────┐
│                Uygulama Katmanı                     │
│   (main.cpp, CLI, GUI — her şeye bağımlı)           │
├──────────────────────────────────────────────────┤
│                  Servis Katmanı                     │
│   (İş mantığı, orkestrasyon)                        │
├──────────────────────────────────────────────────┤
│                  Alan Katmanı                       │
│   (Temel tipler, arayüzler — minimum bağımlılık)    │
├──────────────────────────────────────────────────┤
│               Altyapı Katmanı                       │
│   (Veritabanı, ağ, dosya G/Ç — adaptörler)          │
├──────────────────────────────────────────────────┤
│               Platform Katmanı                      │
│   (İşletim sistemi soyutlamaları, gömülü için HAL)  │
└──────────────────────────────────────────────────┘
```

Kurallar:
- Bağımlılıklar SADECE AŞAĞIYA doğru akar (yukarı çağrı yok)
- Katman sınırlarında arayüzler (soyut sınıflar) kullan
- Alan katmanının SIFIR harici bağımlılığı olmalı

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <functional>
#include <algorithm>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: Dependency Inversion with Interfaces
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: DEPENDENCY INVERSION PRINCIPLE (DIP):
 *     "High-level modules should not depend on low-level modules.
 *      Both should depend on abstractions."
 *
 *     Instead of:  DiagService → Database        (tight coupling)
 *     Use:         DiagService → IRepository ← Database  (loose coupling)
 *
 *     Benefits:
 *     - Swap implementations without changing business logic
 *     - Test with mock implementations
 *     - Compile only the interface, not the implementation
 *
 * TR: BAĞIMLILIK TERSİNE ÇEVİRME İLKESİ (DIP):
 *     "Üst seviye modüller alt seviye modüllere bağımlı olmamalıdır.
 *      İkisi de soyutlamalara bağımlı olmalıdır."
 */

// EN: DOMAIN LAYER — Pure interface, no implementation details
// TR: ALAN KATMANI — Saf arayüz, uygulama detayı yok
struct DiagnosticRecord {
    uint32_t dtc_code;
    std::string description;
    bool active;
};

class IDiagRepository {
public:
    virtual ~IDiagRepository() = default;
    virtual void store(const DiagnosticRecord& record) = 0;
    virtual std::vector<DiagnosticRecord> get_active() const = 0;
    virtual size_t count() const = 0;
};

// EN: INFRASTRUCTURE LAYER — Concrete implementation
// TR: ALTYAPI KATMANI — Somut uygulama
class InMemoryDiagRepository : public IDiagRepository {
public:
    void store(const DiagnosticRecord& record) override {
        records_.push_back(record);
    }

    std::vector<DiagnosticRecord> get_active() const override {
        std::vector<DiagnosticRecord> result;
        std::copy_if(records_.begin(), records_.end(), std::back_inserter(result),
                     [](const auto& r) { return r.active; });
        return result;
    }

    size_t count() const override { return records_.size(); }

private:
    std::vector<DiagnosticRecord> records_;
};

// EN: SERVICE LAYER — Business logic, depends ONLY on interface
// TR: SERVİS KATMANI — İş mantığı, SADECE arayüze bağımlı
class DiagnosticService {
public:
    // EN: Constructor injection — the standard DI pattern in C++
    //     Takes interface by reference (caller owns the object)
    // TR: Constructor enjeksiyonu — C++'ta standart DI kalıbı
    explicit DiagnosticService(IDiagRepository& repo)
        : repo_(repo) {}
```

```

    void report_fault(uint32_t code, const std::string& desc) {
        repo_.store({code, desc, true});
    }

    void clear_fault(uint32_t code, const std::string& desc) {
        repo_.store({code, desc, false});
    }

    void print_active_faults() const {
        auto active = repo_.get_active();
        std::cout << "    Active faults (" << active.size() << "):\n";
        for (const auto& r : active) {
            std::cout << "      DTC-" << r.dtc_code << ": " << r.description << "\n";
        }
    }

private:
    IDiagRepository& repo_;  // EN: Reference to interface, not concrete class
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: Include Hygiene & Forward Declarations
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: INCLUDE HYGIENE — Rules for fast builds in large projects:
 *
 *     1. FORWARD DECLARE whenever possible (instead of #include):
 *        class Sensor;  // Forward declaration — no include needed!
 *        void process(Sensor* s);  // Only pointer/reference → forward decl OK
 *
 *     2. INCLUDE WHAT YOU USE (IWYU):
 *        - Include the header that DEFINES what you use
 *        - Don't rely on transitive includes (they can change)
 *        - Tool: include-what-you-use (Google IWYU)
 *
 *     3. USE #pragma once or include guards:
 *        #pragma once                  // Modern compilers (simpler)
 *        #ifndef MY_HEADER_HPP         // Portable (works everywhere)
 *        #define MY_HEADER_HPP
 *        ...
 *        #endif
 *
 *     4. MINIMIZE header contents:
 *        - Put implementation in .cpp
 *        - Use PIMPL for complex classes
 *        - Forward declare in headers, include in .cpp
 *
 *     Build time impact of includes:
 *
 *     ┌──────────────────────┬──────────┬──────────────────────┐
 *     │ Header               │ Lines    │ Impact per TU        │
 *     ├──────────────────────┼──────────┼──────────────────────┤
 *     │ <iostream>           │ ~28,000  │ Heavy — avoid in headers! │
 *     │ <vector>             │ ~12,000  │ Medium               │
 *     │ <string>             │ ~7,000   │ Medium               │
 *     │ <cstdint>            │ ~50      │ Light — use freely   │
 *     │ Forward declaration  │ 1        │ Zero cost            │
 *     └──────────────────────┴──────────┴──────────────────────┘
 *
 * TR: INCLUDE HİJYENİ — Büyük projelerde hızlı derleme için kurallar:
 *     1. Mümkün olduğunda İLERİ BİLDİRİM kullan (#include yerine)
 *     2. KULLANDIĞINI INCLUDE ET (IWYU)
 *     3. #pragma once veya include korumaları kullan
 *     4. Başlık içeriklerini MİNİMİZE et
 */

```

```
// ═══════════════════════════════════════════════════════════════════════════
// PART 3: Static Analysis Integration
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: Static analysis catches bugs WITHOUT running the code.
 *     Essential tools for C++ projects:
 *
 *     1. clang-tidy — THE most important C++ linter
 *        - Checks: modernize-*, bugprone-*, performance-*, readability-*
 *        - Config file: .clang-tidy in project root
 *        - IDE integration: VS Code, CLion, Vim
 *        - Run: clang-tidy -p build src/main.cpp
 *
 *     2. cppcheck — Additional checker
 *        - Catches different bugs than clang-tidy
 *        - Run: cppcheck --enable=all --std=c++17 src/
 *
 *     3. Compiler warnings (FREE static analysis!):
 *        g++ -Wall -Wextra -Wpedantic -Wshadow -Wconversion
 *            -Wnon-virtual-dtor -Wold-style-cast -Wcast-align
 *            -Wunused -Woverloaded-virtual -Wnull-dereference
 *            -Wdouble-promotion -Wformat=2
 *
 *     4. AddressSanitizer (runtime, but catches memory bugs):
 *        g++ -fsanitize=address -fno-omit-frame-pointer -g
 *
 * TR: Statik analiz, kodu ÇALIŞTIRMADAN hataları yakalar.
 *     C++ projeleri için temel araçlar: clang-tidy, cppcheck, derleyici uyarıları.
 */

// EN: Example .clang-tidy configuration
// TR: Örnek .clang-tidy yapılandırması
/*
    Checks: >
      -*,
      bugprone-*,
      modernize-*,
      performance-*,
      readability-*,
      cppcoreguidelines-*,
      -modernize-use-trailing-return-type,
      -readability-magic-numbers
    WarningsAsErrors: ''
    HeaderFilterRegex: '.*'
    FormatStyle: 'file'
*/

// ═══════════════════════════════════════════════════════════════════════════
// PART 4: Code Review Checklist for C++
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: C++-Specific Code Review Checklist:
 *
 *     MEMORY & OWNERSHIP:
 *     [ ] No raw new/delete (use smart pointers)
 *     [ ] No dangling references or pointers
 *     [ ] Container elements not invalidated during iteration
 *     [ ] Move semantics used where appropriate
 *     [ ] RAII used for all resources (files, locks, sockets)
 *
 *     CORRECTNESS:
 *     [ ] No undefined behavior (signed overflow, null deref, etc.)
 *     [ ] No narrowing conversions (use static_cast or gsl::narrow)
 *     [ ] Rule of 0/3/5 followed (destructors, copy, move)
 *     [ ] Virtual destructors on base classes
 *     [ ] const correctness (const methods, const params)
 *
 *     PERFORMANCE:
 *     [ ] Pass large objects by const& (not by value)
 *     [ ] std::move used for sinks (parameters that will be stored)
 *     [ ] reserve() called on vectors with known sizes
 *     [ ] No unnecessary copies in range-for loops (use const auto&)
 *
 *     API DESIGN:
 *     [ ] explicit on single-arg constructors
```

```
 *     [ ] [[nodiscard]] on important return values
 *     [ ] noexcept on move operations and destructors
 *     [ ] Strong types for same-typed parameters
 *
 *     BUILD & MAINTENANCE:
 *     [ ] No new compiler warnings introduced
 *     [ ] Includes minimized (forward declarations used)
 *     [ ] Tests written for new/changed functionality
 *     [ ] No #define macros (use constexpr/templates instead)
 *
 * TR: C++'a Özgü Kod İnceleme Kontrol Listesi (yukarıda).
 */

// ═══════════════════════════════════════════════════════════════════════════
// PART 5: Large-Scale Refactoring Strategies
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: Strategies for refactoring large C++ codebases:
 *
 *     1. STRANGLER FIG PATTERN:
 *        - Don't rewrite everything at once
 *        - Wrap old code with new interface
 *        - Gradually move callers to new interface
 *        - Remove old code when unused
 *
 *     2. EXTRACT INTERFACE:
 *        - Add abstract base class above concrete class
 *        - Move callers to use the interface
 *        - Now multiple implementations possible
 *
 *     3. PIMPL MIGRATION:
 *        - Add Impl struct to class
 *        - Move private members to Impl
 *        - Now header changes don't recompile users
 *
 *     4. COMPILE-TIME SEAM (for testing):
 *        - Template the class on its dependencies
 *        - In production: real implementations
 *        - In tests: mock implementations
 *
 * TR: Büyük C++ projelerini yeniden yapılandırma stratejileri:
 *     1. BOĞUCU İNCİR KALIPBI: Eski kodu yeni arayüzle sar, kademeli geçiş
 *     2. ARAYÜZ ÇIKAR: Somut sınıfın üstüne soyut temel sınıf ekle
 *     3. PIMPL GEÇİŞİ: Özel üyeleri Impl struct'ına taşı
 *     4. DERLEME ZAMANI DİKİŞİ: Test için şablon tabanlı bağımlılık enjeksiyonu
 */

// EN: Example of compile-time seam for testability
// TR: Test edilebilirlik için derleme zamanı dikişi örneği
template<typename Repository>
class TestableService {
public:
    explicit TestableService(Repository& repo) : repo_(repo) {}

    void add_record(uint32_t code, const std::string& desc) {
        repo_.store({code, desc, true});
    }

    size_t total() const { return repo_.count(); }

private:
    Repository& repo_;
};

// EN: In tests: use a mock repository that tracks calls
// TR: Testlerde: çağrıları izleyen mock repository kullan
class MockDiagRepository {
public:
    void store(const DiagnosticRecord& record) {
        stored_records.push_back(record);
    }
    std::vector<DiagnosticRecord> get_active() const { return {}; }
    size_t count() const { return stored_records.size(); }
```

```

    // EN: Test helpers / TR: Test yardımcıları
    std::vector<DiagnosticRecord> stored_records;
};

// ═══════════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "═══════════════════════════════════════════\n";
    std::cout << "   Module 08 - Maintainable C++\n";
    std::cout << "═══════════════════════════════════════════\n\n";

    // ─── Demo 1: Dependency Inversion ──────────────────────────────────
    {
        std::cout << "--- Demo 1: Dependency Inversion ---\n";

        InMemoryDiagRepository repo;
        DiagnosticService service(repo);

        service.report_fault(0x0100, "Engine coolant temp too high");
        service.report_fault(0x0201, "Oxygen sensor circuit open");
        service.clear_fault(0x0050, "Battery voltage low (cleared)");

        service.print_active_faults();
        std::cout << "    Total records: " << repo.count() << "\n\n";

        std::cout << "    Architecture:\n";
        std::cout << "      DiagnosticService → IDiagRepository (interface)\n";
        std::cout << "                          ↑\n";
        std::cout << "      InMemoryDiagRepository (implementation)\n";
        std::cout << "      (Can swap to DatabaseRepo, FileRepo, etc.)\n\n";
    }

    // ─── Demo 2: Compile-Time Seam (Testing) ───────────────────────────
    {
        std::cout << "--- Demo 2: Compile-Time Seam for Testing ---\n";

        // EN: Production: uses real repository
        // TR: Üretim: gerçek repository kullanır
        InMemoryDiagRepository real_repo;
        TestableService<InMemoryDiagRepository> prod_service(real_repo);
        prod_service.add_record(0x0100, "Engine fault");
        std::cout << "    Production service records: " << prod_service.total() << "\n";

        // EN: Test: uses mock repository (tracks calls)
        // TR: Test: mock repository kullanır (çağrıları izler)
        MockDiagRepository mock_repo;
        TestableService<MockDiagRepository> test_service(mock_repo);
        test_service.add_record(0x0200, "Test fault");
        test_service.add_record(0x0300, "Another fault");
        std::cout << "    Mock service records: " << test_service.total() << "\n";
        std::cout << "    Mock captured DTC: " << mock_repo.stored_records[0].dtc_code << "\n\n";
    }

    // ─── Demo 3: Static Analysis Commands ──────────────────────────────
    {
        std::cout << "--- Demo 3: Static Analysis Integration ---\n";
        std::cout << "  clang-tidy:\n";
        std::cout << "    clang-tidy -p build src/*.cpp          # Analyze all sources\n";
        std::cout << "    clang-tidy --fix -p build src/main.cpp # Auto-fix issues\n\n";
        std::cout << "  cppcheck:\n";
        std::cout << "    cppcheck --enable=all --std=c++17 -I include/ src/\n\n";
        std::cout << "  Compiler warnings (add to CMake):\n";
        std::cout << "    -Wall -Wextra -Wpedantic -Wshadow -Wconversion\n";
        std::cout << "    -Wnon-virtual-dtor -Wold-style-cast -Wformat=2\n\n";
        std::cout << "  CI pipeline:\n";
        std::cout << "    1. clang-format --dry-run --Werror (formatting)\n";
        std::cout << "    2. clang-tidy -p build (static analysis)\n";
        std::cout << "    3. cmake --build build (compilation)\n";
        std::cout << "    4. ctest --test-dir build (testing)\n";
        std::cout << "    5. valgrind / ASan (memory checking)\n\n";
    }
```

```

    // ─── Demo 4: Build Time Optimization ──────────────────────────────
    {
        std::cout << "--- Demo 4: Build Time Optimization ---\n";
        std::cout << "  ┌──────────────────────────────────────┬──────────────────────────────────┐\n";
        std::cout << "  │ Technique                            │ Impact                           │\n";
        std::cout << "  ├──────────────────────────────────────┼──────────────────────────────────┤\n";
        std::cout << "  │ Forward declarations                 │ Huge (avoids transitive incl.)   │\n";
        std::cout << "  │ PIMPL pattern                        │ Large (breaks header dependency)│\n";
        std::cout << "  │ ccache (compiler cache)              │ Huge for rebuild (90%+ cache)    │\n";
        std::cout << "  │ Ninja generator (vs Make)            │ Medium (better parallelism)      │\n";
        std::cout << "  │ Precompiled headers (PCH)            │ Large (parse STL once)           │\n";
        std::cout << "  │ C++20 Modules                        │ Very large (no re-parsing)       │\n";
        std::cout << "  │ Unity builds (UNITY_BUILD)           │ Large (fewer TU compilations)    │\n";
        std::cout << "  │ -j$(nproc) (parallel build)          │ Linear speedup with cores        │\n";
        std::cout << "  └──────────────────────────────────────┴──────────────────────────────────┘\n\n";
    }

    std::cout << "==========================================\n";
    std::cout << "   End of Maintainable C++\n";
    std::cout << "==========================================\n";

    return 0;
}
```

► 09. Clean Code Reference.Md

module_08_core_guidelines/09_clean_code_reference.md

Kaynak Kod / Source Code

```
# Clean Code Reference for C++ — C++ İçin Temiz Kod Referansı

© 2026 Murat Mecit KAHRAMANLI. All rights reserved.
Code: Apache License 2.0 | Book content: CC BY-NC-SA 4.0

---

## 1. Naming Conventions / İsimlendirme Kuralları

### EN: Common C++ Naming Styles

| Element            | Style                | Example                          |
|--------------------|----------------------|----------------------------------|
| Class/Struct       | PascalCase           | `SensorManager`                  |
| Function/Method    | camelCase or snake   | `readSensor()` / `read_sensor()` |
| Variable           | camelCase or snake   | `sensorValue` / `sensor_value`   |
| Constant           | kPascal or UPPER     | `kMaxRetries` / `MAX_RETRIES`    |
| Enum               | PascalCase           | `enum class SensorState`         |
| Enum value         | PascalCase           | `SensorState::Active`            |
| Namespace          | lowercase            | `namespace sensors`              |
| Template param     | PascalCase           | `template<typename ValueType>`   |
| Private member     | trailing underscore  | `int count_;`                    |
| Header guard       | UPPER_SNAKE          | `SENSOR_MANAGER_HPP`             |
| File name          | snake_case           | `sensor_manager.cpp`             |

### TR: Yaygın C++ İsimlendirme Stilleri
- Sınıf/Struct: PascalCase (`SensorManager`)
- Fonksiyon: camelCase veya snake_case
- Sabit: kPascal veya UPPER_SNAKE
- Private üye: sondaki alt çizgi (`count_`)
```

Important Rules / Önemli Kurallar

1. **Names should reveal intent** — İsimler amacı açıklamalı
   - BAD: `int d;  // elapsed time in days`
   - GOOD: `int elapsedDays;`
2. **Avoid abbreviations** — Kısaltmalardan kaçın
   - BAD: `int tpc;  // temperature processing count`
   - GOOD: `int temperatureProcessingCount;`
3. **One letter names only for loops** — Tek harfli isimler sadece döngülerde
   - OK: `for (int i = 0; i < n; ++i)`
4. **Bool names should be questions** — Bool isimleri soru olmalı
   - `bool isActive;`, `bool hasPermission;`, `bool shouldRetry;`

---

2. Function Design / Fonksiyon Tasarımı

EN: Function Length & Complexity

Metric	Target	Max Acceptable
Lines per function	≤ 20	40
Parameters	≤ 3	5
Cyclomatic complexity	≤ 5	10
Nesting depth	≤ 2	4

TR: Fonksiyon Uzunluğu ve Karmaşıklığı
- Fonksiyon başına satır: ≤ 20 (maks 40)
- Parametreler: ≤ 3 (maks 5)

Rules / Kurallar

1. **Do ONE thing** — Fonksiyon BİR şey yapsın
   ```cpp
   // BAD: Does validation AND processing AND logging
   void handleSensorData(int raw) { ... }

   // GOOD: Split into focused functions
   bool validateReading(int raw);
   double processReading(int raw);
   void logReading(double processed);
   ```

2. **Use early return to reduce nesting** — Erken dönüş ile yuvalamayı azalt
   ```cpp
   // BAD: Deep nesting
   void process(Sensor* s) {
       if (s != nullptr) {
           if (s->isActive()) {
               if (s->hasData()) {
                   // actual logic buried deep
               }
           }
       }
   }

   // GOOD: Guard clauses
   void process(Sensor* s) {
       if (!s) return;
       if (!s->isActive()) return;
       if (!s->hasData()) return;
       // actual logic at top level
   }
   ```

3. **Parameter passing rules** — Parametre geçirme kuralları
   ```cpp
   void observe(const Widget& w);       // Input: const ref
   void mutate(Widget& w);              // Input-output: ref
   void sink(Widget w);                 // Consuming: by value
   Widget create();                     // Output: return by value
   void optional(Widget* w);            // Optional: pointer (nullable)
   ```

---

3. Code Smells to Avoid / Kaçınılacak Kod Kokuları

```

### EN: Common C++ Code Smells

| Smell                     | Fix                                                |
|---------------------------|----------------------------------------------------|
| Raw `new`/`delete`        | Use `make_unique`/`make_shared`                    |
| C-style casts `(int)x`    | Use `static_cast<int>(x)`                          |
| `#define` constants       | Use `constexpr` or `const`                         |
| `#define` functions       | Use `inline` functions or templates                |
| Magic numbers             | Use named constants: `constexpr int kMaxRetry = 3;`|
| God class (1000+ lines)   | Split into focused classes with SRP                |
| Long parameter lists      | Use structs/builder pattern                        |
| Comments explaining what  | Make code self-documenting (better names)          |
| `using namespace std;`    | Never in headers! OK sparingly in .cpp             |
| Implicit conversions      | Use `explicit` on constructors                     |
| Output parameters         | Return by value (thanks to RVO/NRVO)               |
| `void*`                   | Use templates, `std::variant`, or `std::any`       |
| `errno`-style errors      | Use exceptions or `std::expected`                  |

### TR: Yaygın C++ Kod Kokuları
- Ham `new`/`delete` → `make_unique`/`make_shared` kullan
- C-tarzı dönüşüm → `static_cast` kullan
- `#define` sabitleri → `constexpr` kullan
- Sihirli sayılar → isimli sabitler kullan

---

## 4. Header File Best Practices / Başlık Dosyası En İyi Uygulamaları

```cpp
#pragma once  // or include guards

// 1. Standard library headers
#include <string>
#include <vector>

// 2. Third-party library headers
#include <fmt/core.h>

// 3. Project headers
#include "sensors/temperature.hpp"
#include "utils/math_helpers.hpp"

namespace ecu::sensors {

// Forward declarations (instead of #include)
class PressureSensor;

// Class declaration (minimal — implementation in .cpp)
class TemperatureSensor {
public:
    explicit TemperatureSensor(int channel);
    double read() const;

private:
    struct Impl;                    // PIMPL for heavy private members
    std::unique_ptr<Impl> impl_;
};

}  // namespace ecu::sensors
```

Rules / Kurallar
1. **#pragma once** at the top of every header
2. **Include order**: standard → third-party → project
3. **Forward declare** in headers, include in .cpp
4. **No `using namespace`** in headers — EVER
5. **Minimize** what headers expose (PIMPL, forward decl)

---

5. clang-format Configuration / clang-format Yapılandırması

Recommended `.clang-format` file / Önerilen `.clang-format` dosyası

```yaml
# .clang-format
```
```

```
---
Language: Cpp
BasedOnStyle: Google
IndentWidth: 4
ColumnLimit: 100
AccessModifierOffset: -4
AlignAfterOpenBracket: Align
AllowShortFunctionsOnASingleLine: Inline
AllowShortIfStatementsOnASingleLine: Never
AllowShortLoopsOnASingleLine: false
BreakBeforeBraces: Attach
NamespaceIndentation: None
PointerAlignment: Left
SortIncludes: CaseInsensitive
SpaceBeforeParens: ControlStatements
IncludeBlocks: Regroup
IncludeCategories:
  - Regex: '^<[a-z_]+>'
    Priority: 1
  - Regex: '^<'
    Priority: 2
  - Regex: '^"'
    Priority: 3
...
```

Usage / Kullanım
```bash
# Format a file
clang-format -i src/main.cpp

# Check formatting (CI)
clang-format --dry-run --Werror src/*.cpp

# Format all C++ files in project
find src/ -name "*.cpp" -o -name "*.hpp" | xargs clang-format -i
```

---

6. C++ Core Guidelines Quick Reference

Most Important Rules / En Önemli Kurallar

Rule	Description
F.1	Package meaningful operations as functions
F.3	Keep functions short and simple
F.4	If function may need compile-time eval, use constexpr
F.16	Input: `const T&` for cheap-to-move, `T` for sink
C.2	Use class if has invariant; struct if members vary
C.9	Minimize member exposure
C.35	Base class needs virtual dtor OR protected non-virtual
C.48	Prefer `{}` init for constants, `=` for others
R.1	Manage resources with RAII
R.3	Raw pointer = non-owning
R.11	Avoid calling `new`/`delete` explicitly
ES.1	Prefer standard library over hand-written code
ES.5	Keep scopes small
ES.23	Prefer `{}` init to avoid narrowing
ES.49	Don't use casts; if must, use named casts
P.1	Express ideas directly in code
P.3	Express intent
I.11	Never transfer ownership by raw pointer

---

7. Modern C++ Idioms Checklist / Modern C++ Deyimleri Kontrol Listesi

Before Committing / Commit Etmeden Önce

- [ ] **RAII everywhere** — No resource leaks (files, locks, memory)
- [ ] **const correct** — `const` on methods, `const&` on parameters
- [ ] **explicit constructors** — Single-arg and implicit conversion ctors
- [ ] **noexcept on move** — Move ctor/assignment are noexcept
- [ ] **[[nodiscard]]** — On functions where ignoring return = likely bug
- [ ] **auto wisely** — Use auto for complex types, not for int/double
```
```

```
- [ ] **Range-for with const auto&** — `for (const auto& item : container)`
- [ ] **Smart pointers** — No raw `new`/`delete`
- [ ] **No narrowing** — Use `static_cast` or `gsl::narrow`
- [ ] **Rule of 0/3/5** — Default, or all three/five
- [ ] **Scoped enums** — `enum class` not plain `enum`
- [ ] **nullptr** — Not `NULL` or `0` for pointers
- [ ] **constexpr** — For compile-time computable values
- [ ] **Structured bindings** — `auto [key, value] = pair;`
- [ ] **std::string_view** — For non-owning string references
```

Modül 9: Advanced Mechanics — İleri Mekanikler

İleri Mekanikler

8 kaynak dosya · Standart: C++20

C++20

► 01. Bit Manipulation And Alignment

module_09_advanced_mechanics/01_bit_manipulation_and_alignment.cpp

Konu: Advanced Mechanics: Bit Manipulation & Memory Alignment — İleri Teknikler: Bit

Teori / Theory

1. Bitwise Manipulation

Hardware interfaces, embedded systems, and automotive ECU firmware rely on Bit Masking. Doing math at the bit level (`&`, `|`, `^`, `<<`, `>>`) is the absolute fastest way a CPU can operate. Standard C++ also provides `<bitset>` to make handling flags easier.

Bit Düzeyinde İşlemler

Donanım arayüzleri, gömülü sistemler ve otomotiv ECU firmware'leri "Bit Maskeleme" (Bit Masking) kurallarına dayanır. Bitlerle çalışmak (`VE`, `VEYA`, `XOR`, `Kaydırma`) CPU'nun dünyadaki en hızlı yaptığı matematiktir. Saf matematiksel hesapları bir kenara bırakıp bit mantığıyla bellek veya hız kazanımı sağlanır.

2. Memory Alignment & Padding (alignas)

Data structures are automatically "padded" by the compiler to align perfectly within the CPU's cache lines. If you organize variables poorly inside a `struct`, you can waste up to 50% of your RAM due to invisible Padding Bytes! We can control this using `alignas` or by ordering members largest-to-smallest.

Bellek Hizalaması

Sınıf veya Yapılar (Struct) bellekte rastgele yan yana durmazlar. C++ Derleyicisi verileri işlemcinin L1/L2 önbelleğine tam uyumlu oturtmak için aralarına gizli boşluklar (Padding) serpiştirir. Değişkenleri kötü sıralarsanız RAM israfı yaparsınız. Bunu kontrol etmek için `sizeof` bilincine ve `alignas` kuralına ihtiyacımız var.

CPPREF Depth: alignas, alignof, and the Strict Aliasing Rule

`alignof(T)` returns the alignment requirement of type T in bytes. `alignas(N)` overrides the default alignment, forcing a variable or type to be aligned to at least N bytes. The Strict Aliasing Rule (§6.10) states: accessing an object through a pointer of incompatible type is Undefined Behavior. Exception: `char*`, `unsigned char*`, and `std::byte*` may alias any type. GCC's `-fno-strict-aliasing` disables this optimization at the cost of performance. `std::aligned_storage` is deprecated in C++23 — prefer `alignas` with a `std::byte` array.

CPPREF Derinlik: alignas, alignof ve Katı Örtüşme Kuralı

`alignof(T)`, T türünün hizalama gereksinimini bayt olarak döndürür. `alignas(N)` varsayılan hizalamayı geçersiz kılarak değişkeni en az N bayta hizalar. Katı Örtüşme Kuralı (§6.10): bir nesneye uyumsuz türde bir işaretçi üzerinden erişmek Tanımsız Davranıştır. İstisna: `char*`, `unsigned char*` ve `std::byte*` her türe örtüşebilir. GCC'nin `-fno-strict-aliasing` seçeneği bu optimizasyonu performans pahasına devre dışı bırakır. `std::aligned_storage` C++23'te kullanımdan kaldırıldı — `alignas` + `std::byte` dizisi tercih edin.

Kaynak Kod / Source Code

```cpp
#include <bitset>
#include <iostream>

// ---------------------------------------------------------------------------------------------
// [SECTION 1] RAM WASTING vs OPTIMAL STRUCT (Padding/Hizalama İsrafı)
// ---------------------------------------------------------------------------------------------
// EN: Order: 1 byte, 8 bytes, 4 bytes.
// TR: Derleyici aralara boşluk (Padding) ekleneceği için 13 Byte yerine 24 Byte kaplar!
struct WastedMemoryStruct {
  char a;   // 1 byte (+7 bytes of hidden padding!) -> 8 byte block
  double b; // 8 bytes                              -> 8 byte block
  int c;    // 4 bytes (+4 bytes of hidden padding!)-> 8 byte block
}; // Total Size = 24 Bytes (Almost 50% Memory Wasted!)

// EN: Order: Largest to Smallest (8, 4, 1).
// TR: En büyükten en küçüğe sıralandığı için derleyici boşluk koymaz! İşlemci rahat eder.
struct OptimizedStruct {
  double b; // 8 bytes
  int c;    // 4 bytes
  char a;   // 1 byte (+3 bytes padding at the end)
}; // Total Size = 16 Bytes (Much better!)

// EN: Forcing absolute cache-line alignment (e.g. 64-bytes for AVX/SIMD instructions) TR:
// Gelişmiş donanım hızlandırmaları için objeyi kesinlikle 64-Byte katsayısına zorlamak:
struct alignas(64) SimdVector {
  float x, y, z, w; // Genelde matematik veya SIMD motorları için kullanılır.
};

// ---------------------------------------------------------------------------------------------
// [SECTION 2] BITWISE FLAGS vs BOOLEANS
// ---------------------------------------------------------------------------------------------
// EN: Instead of using 8 `bool` variables (which take 8 Full Bytes), use 8 Bits (1 Byte). TR: 8
// tane `bool` değişkeni açmak 8 Byte yer kaplar. 8 biti olan tek 1 Byte ile hepsini kontrol
// etmek:
const unsigned char FLAG_RUNNING = 1 << 0;    // 00000001 (Engine Running)
const unsigned char FLAG_ABS_ACTIVE = 1 << 1; // 00000010 (ABS Active)
const unsigned char FLAG_LIGHTS_ON = 1 << 2;  // 00000100 (Headlights On)
const unsigned char FLAG_DOOR_OPEN = 1 << 3;  // 00001000 (Door Open)

int main() {
  std::cout << "=== MODULE 9: BIT MANIPULATION & ALIGNMENT ===\n" << std::endl;

  // --- ALIGNMENT TESTS ---
  std::cout << "[MEMORY ALIGNMENT]" << std::endl;
  std::cout << "Size of WastedMemoryStruct: " << sizeof(WastedMemoryStruct) << " bytes" <<
      std::endl;
  std::cout << "Size of OptimizedStruct   : " << sizeof(OptimizedStruct) << " bytes" << std::endl;
  std::cout << "Size of alignas(64) Vector: " << sizeof(SimdVector) << " bytes\n" << std::endl;

  // --- BITWISE TESTS (C-Style) ---
  std::cout << "[BIT MANIPULATION (C-Style)]" << std::endl;
  unsigned char ecuStatusFlags = 0; // 00000000

  // Set flags (OR `|`) / Bayrakları Yak (VEYA)
  ecuStatusFlags |= FLAG_RUNNING;
  ecuStatusFlags |= FLAG_LIGHTS_ON;

  // Check flags (AND `&`) / Bayrakları Kontrol Et (VE)
  if (ecuStatusFlags & FLAG_RUNNING) {
    std::cout << "- ECU Engine is Running!" << std::endl;
  }

```

```
63    // Toggle flags (XOR `^`) / Bayrakları Tersine Çevir (XOR)
64    ecuStatusFlags ^= FLAG_LIGHTS_ON; // Turns headlights flag OFF
65
66    // --- BITWISE TESTS (Modern C++ std::bitset) ---
67    std::cout << "\n[BIT MANIPULATION (Modern C++ <bitset>)]" << std::endl;
68    // EN: Modern C++ provides a safe object wrapper around bits.
69    // TR: C++ bitset ile manuel 0000'larla uğraşmanızı engeller, OOP yapısına çeker.
70    std::bitset<8> modernECUStatus;
71    modernECUStatus.set(0); // Bit 0 (Engine Running) -> True
72    modernECUStatus.set(3); // Bit 3 (Brake Active) -> True
73
74    std::cout << "Current 8-Bit Status: " << modernECUStatus << std::endl;
75    std::cout << "Is brake active? : " << (modernECUStatus.test(3) ? "Yes" : "No") << std::endl;
76
77    return 0;
78  }
```

► 02. Operator Overloading

module_09_advanced_mechanics/02_operator_overloading.cpp

Konu: Advanced Mechanics: Operator Overloading — İleri Teknikler: Operatör Aşırı Yükleme

Teori / Theory

Operator Overloading (Operatörlerin Ezilmesi

C++ allows us to modify what standard operators (`+`, `-`, `==`, `<<`, `()`) actually do when used with our custom Classes. This allows our custom Types to feel and act exactly like built-in native C++ types (`int`, `float`).

Aşırı Yüklenmesi)

C++ dili, kendi yarattığımız devasa Sınıfları (Objects) tıpkı bir `int` veya `float` gibi toplama `+`, çıkarma `-`, karılaştırma `==` operatörleriyle kullanabilmemiz için Operatör Ezme (Aşırı Yükleme) yetkisi verir.

Why overload `<<`?

If you type `std::cout << myObject;`, the compiler throws a FATAL ERROR. Because it doesn't know how to "Print" your custom object. By overloading `operator<<` as a `friend` function, we teach `std::ostream` how to read your object.

Neden Sol Kaydırma `<<` ezilir?

Eğer bir objeye `std::cout << obje` derseniz derleyici küfür eder. Çünkü objeyi ekrana nasıl basacağını bilemez. `operator<<` fonksiyonunu sınıfın içine bir `friend` (dost fonksiyon) olarak aşılarsak kütüphane objemizi şakır şakır okur.

CPPREF Depth: Operator Overloading — ADL, Hidden Friends, and the Spaceship

Argument-Dependent Lookup (ADL) finds `operator<<` in the namespace of the operand type, even without a `using` directive. The "hidden friend" idiom defines `friend operator==` inline inside the class — it is found only via ADL and prevents implicit conversions on the LHS operand. C++20 `operator<=>` (the "spaceship operator") auto-generates all six comparison operators (`<`, `>`, `<=`, `>=`, `==`, `!=`). Return types: `std::strong_ordering` (total order, substitutable), `std::weak_ordering` (total order, not substitutable), and `std::partial_ordering` (permits incomparable values, e.g., NaN).

CPPREF Derinlik: Operatör Aşırı Yükleme — ADL, Gizli Dostlar ve Uzay Gemisi

Argümana Bağlı Arama (ADL), `using` direktifi olmadan bile `operator<<` fonksiyonunu operandın ad alanında bulur. "Gizli dost" deyimi, sınıf içinde `friend operator==` tanımlar — yalnızca ADL ile bulunur ve sol operandda örtük dönüşümleri engeller. C++20 `operator<=>` ("uzay gemisi operatörü") altı karşılaştırma operatörünü otomatik üretir. Dönüş türleri: `std::strong_ordering` (tam sıralama, ikame edilebilir), `std::weak_ordering` (tam sıralama, ikame edilemez) ve `std::partial_ordering` (karşılaştırılamaz değerlere izin verir, ör: NaN).

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <string>
#include <tuple>

// EN: A 2D Math Vector (Not std::vector array, but a mathematical Coordinate System) TR: 2D
// Matematik Vektörü (Dizilerdeki vector değil, fizikteki XYZ vektörü).
class Vector2D {
private:
  float x;
  float y;

public:
  Vector2D(float inputX = 0, float inputY = 0) : x(inputX), y(inputY) {}

  // ---------------------------------------------------------------------------------------------
  // [1] Operator Overloading for Addition `+` (Toplama Operatörü)
  // ---------------------------------------------------------------------------------------------
  // EN: Tells C++ what happens when we do: vectorA + vectorB
  // TR: vecA + vecB yaparsak arka planda tam olarak ne olacağını C++'a öğretiyoruz.
  Vector2D operator+(const Vector2D &other) const {
    return Vector2D(this->x + other.x, this->y + other.y);
  }

  // ---------------------------------------------------------------------------------------------
  // [2] Operator Overloading for Equality `==` (Eşitlik Operatörü)
  // ---------------------------------------------------------------------------------------------
  bool operator==(const Vector2D &other) const {
    // EN: Note: Float comparison requires an epsilon gap. simplified here.
    // TR: Normalde float kıyası risklidir (1.000001), ama eğitim için basit tutuyoruz.
    return (this->x == other.x && this->y == other.y);
  }

  // ---------------------------------------------------------------------------------------------
  // [3] Operator Overloading for Function Call `()` (Functor Mimarisinin
  // Temeli)
  // ---------------------------------------------------------------------------------------------
  // EN: Allows an OBJECT to be called exactly like a FUNCTION. `myObject()`
  // TR: Bir OBJEYİ, sanki sıradan bir fonksiyonmuş gibi çağırmayı (`obje()`) sağlar.
  void operator()() const {
    std::cout << "[Functor Call] Vector coordinates: (" << x << ", " << y << ")" << std::endl;
  }

  // ---------------------------------------------------------------------------------------------
  // [4] STREAM OPERATOR OVERLOADING (`<<`) -> Using `friend`
  // ---------------------------------------------------------------------------------------------
  // EN: Stream operators CANNOT be normal member functions! They must be global or friends. TR:
  // Cout/Ostream fonksiyonları dışarıdan geldiği için `friend` anahtar kelimesi ile Sınıfımızın
  // 'Mahrem (Private)' bölgelerine erişim izni vermemiz gerekir!
  friend std::ostream &operator<<(std::ostream &out, const Vector2D &vec) {
    out << "[" << vec.x << ", " << vec.y << "]";
    // EN: We return out to chain calls: `cout << v1 << v2;` (Zincirleme)
    // TR: Zincirleme çağrılar için out döndürürüz: `cout << v1 << v2;`
    return out;
  }
};
```

```

// ═══════════════════════════════════════════════════════════════════════════════
// EN: SensorReading — All 6 comparison operators via std::tie idiom
// TR: SensorReading — std::tie deyimiyle 6 karşılaştırma operatörünün tamamı
// ═══════════════════════════════════════════════════════════════════════════════

class SensorReading {
private:
    double timestamp_;
    double value_;
    std::string sensor_id_;

public:
    SensorReading(double ts, double val, std::string id)
        : timestamp_(ts), value_(val), sensor_id_(std::move(id)) {}

    // ─── 1. Equality == ──────────────────────────────────────────────────────
    // EN: Two readings are equal when both timestamp and value match.
    // TR: Zaman damgası ve değer eşleştiğinde iki okuma eşittir.
    bool operator==(const SensorReading& o) const {
        return std::tie(timestamp_, value_) == std::tie(o.timestamp_, o.value_);
    }

    // ─── 2. Inequality != ────────────────────────────────────────────────────
    // EN: Defined in terms of == for consistency.
    // TR: Tutarlılık için == ile tanımlanır.
    bool operator!=(const SensorReading& o) const {
        return !(*this == o);
    }

    // ─── 3. Less-than < ──────────────────────────────────────────────────────
    // EN: Lexicographic comparison via std::tie — first by timestamp, then value.
    // TR: std::tie ile sözlüksel karşılaştırma — önce zaman damgası, sonra değer.
    bool operator<(const SensorReading& o) const {
        return std::tie(timestamp_, value_) < std::tie(o.timestamp_, o.value_);
    }

    // ─── 4. Greater-than > ───────────────────────────────────────────────────
    // EN: Flip the operands of < for > .
    // TR: > için < operandları ters çevrilir.
    bool operator>(const SensorReading& o) const {
        return o < *this;
    }

    // ─── 5. Less-or-equal <= ─────────────────────────────────────────────────
    // EN: Not greater means less-or-equal.
    // TR: Büyük değilse küçük-eşittir.
    bool operator<=(const SensorReading& o) const {
        return !(o < *this);
    }

    // ─── 6. Greater-or-equal >= ──────────────────────────────────────────────
    // EN: Not less means greater-or-equal.
    // TR: Küçük değilse büyük-eşittir.
    bool operator>=(const SensorReading& o) const {
        return !(*this < o);
    }

    // ─── Stream output operator ──────────────────────────────────────────────
    // EN: Prints sensor reading in automotive diagnostic format.
    // TR: Sensör okumasını otomotiv teşhis formatında yazdırır.
    friend std::ostream& operator<<(std::ostream& os, const SensorReading& sr) {
        os << "[" << sr.sensor_id_ << " @" << sr.timestamp_
           << "s : " << sr.value_ << "]";
        return os;
    }

    // ─── C++20 Spaceship Operator (Reference Only) ───────────────────────────
    // EN: In C++20, a SINGLE declaration replaces all 6 comparison operators: auto
    // operator<=>(const SensorReading&) const = default; The compiler auto-generates ==, !=, <,
    // >, <=, >= from this one line. It performs member-wise lexicographic comparison in
    // declaration order.
    // TR: C++20'de TEK bir bildirim 6 karşılaştırma operatörünün hepsini değiştirir: auto
    // operator<=>(const SensorReading&) const = default; Derleyici bu tek satırdan ==, !=, <, >,
    // <=, >= operatörlerini üretir. Üye bazında, bildirim sırasına göre sözlüksel karşılaştırma
    // yapar.
};
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════════
// EN: EventCounter — Prefix and Postfix increment operators
// TR: EventCounter — Önek ve Sonek artırma operatörleri
// ═══════════════════════════════════════════════════════════════════════════════

class EventCounter {
private:
    int count_;
    std::string event_name_;

public:
    EventCounter(std::string name, int initial = 0)
        : count_(initial), event_name_(std::move(name)) {}

    // ─── Prefix ++counter (returns reference to incremented self) ──────────────
    // EN: Prefix increment — increments first, then returns the new value.
    // TR: Önek artırma — önce artırır, sonra yeni değeri döndürür.
    EventCounter& operator++() {
        ++count_;
        return *this;
    }

    // ─── Postfix counter++ (int dummy distinguishes from prefix) ───────────────
    // EN: Postfix increment — the dummy 'int' parameter tells the compiler this is postfix.
    // Returns the OLD value, then increments.
    // TR: Sonek artırma — 'int' kukla parametresi derleyiciye sonek olduğunu bildirir. ESKİ
    // değeri döndürür, sonra artırır.
    EventCounter operator++(int) {
        EventCounter old = *this;
        ++count_;
        return old;
    }

    friend std::ostream& operator<<(std::ostream& os, const EventCounter& ec) {
        os << "[" << ec.event_name_ << ": " << ec.count_ << "]";
        return os;
    }
};

// ═══════════════════════════════════════════════════════════════════════════════

int main() {
  std::cout << "=== MODULE 9: OPERATOR OVERLOADING ===\n" << std::endl;

  Vector2D pointVelocity_A(5.5f, -2.0f);
  Vector2D pointVelocity_B(2.0f, 4.0f);

  // 1. Addition Overload (O-Overload of +)
  // TR: Sanki int veya float imiş gibi doğal toplama yapar!
  Vector2D result = pointVelocity_A + pointVelocity_B;

  // 2. Stream Overload (O-Overload of <<)
  // TR: `<<` ezildiği (overload edildiği) için obje anında string algısı yaratır!
  std::cout << "Velocity A : " << pointVelocity_A << std::endl;
  std::cout << "Velocity B : " << pointVelocity_B << std::endl;
  std::cout << "Result (A+B): " << result << std::endl;

  // 3. Equality Overload (O-Overload of ==)
  if (pointVelocity_A == pointVelocity_B) {
    std::cout << "Both vectors are totally equal!" << std::endl;
  } else {
    std::cout << "Vectors are pointing to different directions." << std::endl;
  }

  // 4. Function Overload / Functor Mimari Testi (O-Overload of ())
  std::cout << "\nTriggering Functor Object Call:" << std::endl;
  // EN: Looks like a function, but it is actually the OBJECT acting like a function.
  // TR: Fonksiyon gibi görünür ama aslında NESNE fonksiyon gibi davranır.
  result();
```

```

    // ═══════════════════════════════════════════════════════════════════════════
    // EN: Demo 5 — All 6 comparison operators on SensorReading
    // TR: Demo 5 — SensorReading üzerinde 6 karşılaştırma operatörü
    // ═══════════════════════════════════════════════════════════════════════════
    std::cout << "\n--- DEMO 5: SENSOR READING COMPARISONS ---" << std::endl;

    SensorReading rpm_early(1.00, 3200.0, "RPM_SENSOR");
    SensorReading rpm_late (2.50, 3200.0, "RPM_SENSOR");
    SensorReading rpm_same (1.00, 3200.0, "RPM_SENSOR");

    std::cout << "  A: " << rpm_early << std::endl;
    std::cout << "  B: " << rpm_late  << std::endl;
    std::cout << "  C: " << rpm_same  << std::endl;

    // EN: Testing all 6 comparison operators.
    // TR: 6 karşılaştırma operatörünün tümünü test ediyoruz.
    std::cout << "  A == C : " << (rpm_early == rpm_same  ? "true" : "false") << std::endl;
    std::cout << "  A != B : " << (rpm_early != rpm_late  ? "true" : "false") << std::endl;
    std::cout << "  A <  B : " << (rpm_early <  rpm_late  ? "true" : "false") << std::endl;
    std::cout << "  B >  A : " << (rpm_late  >  rpm_early ? "true" : "false") << std::endl;
    std::cout << "  A <= C : " << (rpm_early <= rpm_same  ? "true" : "false") << std::endl;
    std::cout << "  B >= A : " << (rpm_late  >= rpm_early ? "true" : "false") << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════
    // EN: Demo 6 — Prefix vs Postfix increment on EventCounter
    // TR: Demo 6 — EventCounter üzerinde önek vs sonek artırma
    // ═══════════════════════════════════════════════════════════════════════════
    std::cout << "\n--- DEMO 6: PREFIX vs POSTFIX INCREMENT ---" << std::endl;

    EventCounter brakeEvents("BRAKE_PRESS", 0);
    std::cout << "  Initial : " << brakeEvents << std::endl;

    // EN: Prefix — increments then returns the NEW value.
    // TR: Önek — artırır, sonra YENİ değeri döndürür.
    std::cout << "  ++counter: " << ++brakeEvents << std::endl;
    std::cout << "  After    : " << brakeEvents   << std::endl;

    // EN: Postfix — returns OLD value, then increments.
    // TR: Sonek — ESKİ değeri döndürür, sonra artırır.
    std::cout << "  counter++: " << brakeEvents++ << std::endl;
    std::cout << "  After    : " << brakeEvents   << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════
    // EN: Demo 7 — Stream output operator for SensorReading
    // TR: Demo 7 — SensorReading için akış çıkış operatörü
    // ═══════════════════════════════════════════════════════════════════════════
    std::cout << "\n--- DEMO 7: STREAM OUTPUT FOR SENSOR READING ---" << std::endl;

    SensorReading coolant(0.50, 87.3,  "COOLANT_TEMP");
    SensorReading battery(0.50, 13.8,  "BATTERY_VOLT");
    SensorReading oil    (1.20, 102.5, "OIL_PRESSURE");

    // EN: The overloaded << operator prints in automotive diagnostic format.
    // TR: Aşırı yüklenmiş << operatörü otomotiv teşhis formatında yazdırır.
    std::cout << "  " << coolant << std::endl;
    std::cout << "  " << battery << std::endl;
    std::cout << "  " << oil     << std::endl;

    // EN: Chained stream output — works because operator<< returns ostream&.
    // TR: Zincirleme akış çıkışı — operator<< ostream& döndürdüğü için çalışır.
    std::cout << "  Chained: " << coolant << " | " << battery << " | " << oil << std::endl;

    return 0;
}
```

► 03. Advanced Pointers

module_09_advanced_mechanics/03_advanced_pointers.cpp

Konu: Advanced Mechanics: Pointers to Member Functions & std::invoke — Üye Fonksiyon ve

Teori / Theory

Pointers to Data Members

Regular pointers (`int*`) point to a specific memory address AT RUNTIME. Pointers to class members (`int ECUModule::*`) don't point to RAM until they are combined with an actual instantiated Object! They just hold an "offset" inside.

Bir Sınıfın Değişkenini İşaret Etmek

Normal Pointerlar (`int*`) çalışma zamanında RAM'de kesin bir adres tutar. (0x1A2...) Ama Sınıf Üye Pointerları (`int ECUModule::*`), RAM adresi DEĞİL, Değişkenin veya Fonksiyonun Sınıfın içindeki göreceli (Offset) indeksini/formülünü tutar. Bunu çağırmak için GERÇEK BİR OBJEYE (instance) ihtiyaç duyarız.

Pointers to Member Functions

Used massively in Automotive Middleware (like AUTOSAR Adaptive) or UI frameworks (Qt Signals/Slots) to store a "Function call to be executed later on an object". Modern C++17 provides `std::invoke` which makes the ugly syntax beautiful.

Otomotiv Middleware Delegasyonu İçin

AUTOSAR Adaptive içindeki Delegate'ler veya Qt UI sinyalleri bu prensiple çalışır. Bir ECU modülünün bir komutu tetiklemesi "işlemini" bir Pointer içine kaydederiz ve olay tetiklendiğinde "O objenin üzerindeki kaydettiğimiz bu fonksiyonu" çalıştırırız.

CPPREF Depth: Pointer Arithmetic, Provenance, and std::launder

Pointer arithmetic is defined only within an array (including one-past- the-end). Dereferencing a pointer outside these bounds is UB. Pointer provenance: even if two pointers hold the same numeric address, they may not be interchangeable — the optimizer tracks "where" a pointer came from. `std::launder` (C++17) creates a new pointer to an object at a given address, bypassing provenance tracking. It is needed after placement-new overwrites an object whose type contains non-trivial `const` or reference members, because the compiler may cache the old values through the original pointer.

CPPREF Derinlik: İşaretçi Aritmetiği, Köken ve std::launder

İşaretçi aritmetiği yalnızca bir dizi içinde (sondan-bir-sonrası dahil) tanımlıdır. Bu sınırlar dışındaki bir işaretçinin referansını kaldırmak UB'dir. İşaretçi kökeni: iki işaretçi aynı sayısal adresi taşısa bile birbirinin yerine kullanılamayabilir — derleyici işaretçinin "nereden geldiğini" izler. `std::launder` (C++17), verilen bir adresteki nesneye yeni bir işaretçi oluşturarak köken takibini atlar. Placement-new ile `const` veya referans üyeler içeren bir nesne üzerine yazıldığında gereklidir, çünkü derleyici eski değerleri orijinal işaretçi üzerinden önbelleğe almış olabilir.

Kaynak Kod / Source Code

```cpp
#include <functional> // for std::invoke (C++17)
#include <iostream>
#include <string>

class ECUModule {
public:
  std::string name;
  int messageBudget;
  int uptimeHours;

  // Constructor
  ECUModule(std::string n, int budget, int uptime)
      : name(std::move(n)), messageBudget(budget), uptimeHours(uptime) {}

  // Target Member Function 1
  void sendCANFrame(int priority) {
    if (messageBudget >= priority) {
      messageBudget -= priority;
      std::cout << "[" << name << "] CAN frame sent (Priority: " << priority
                << "). Budget left: " << messageBudget << std::endl;
    } else {
      std::cout << "[" << name << "] CAN budget exhausted!" << std::endl;
    }
  }

  // Target Member Function 2
  void activateDiagMode(int durationSecs) {
    std::cout << "[" << name << "] Diagnostic mode activated for "
              << durationSecs << " seconds." << std::endl;
  }
};

int main() {
  std::cout << "=== MODULE 9: POINTERS TO MEMBERS (ADVANCED) ===\n" << std::endl;

  ECUModule bcm("BCM_BodyControl", 50, 100);
  ECUModule hvac("HVAC_Climate", 10, 50);

  // ==========================================================================
  // 1. Pointer to a DATA MEMBER (Veri değişkeni işaretçisi)
  // ==========================================================================
  // EN: The ugly C++ syntax: Define a pointer that CAN hold an 'int' belonging to 'ECUModule'.
  // TR: Ucube Klasik C++ Syntaxı: 'ECUModule'in içindeki bir 'int' değişkeni tutabilen formül.
  int ECUModule::*pDataMetric = &ECUModule::messageBudget;

  // Accesses messageBudget!
  std::cout << "[DATA POINTER]: BCM Budget:  " << bcm.*pDataMetric << std::endl;
  // Same pointer, different object!
  std::cout << "[DATA POINTER]: HVAC Budget: " << hvac.*pDataMetric << std::endl;

  // EN: Change the pointer to point to 'uptimeHours' instead of 'messageBudget'.
  // TR: Formülü değiştir. Artık Mesaj Bütçesi değil, 'Çalışma Süresi' değerini tutar.
  pDataMetric = &ECUModule::uptimeHours;
  std::cout << "[DATA POINTER]: BCM Uptime:  " << bcm.*pDataMetric << "\n" << std::endl;

  // ==========================================================================
  // 2. Pointer to a MEMBER FUNCTION (Üye Fonksiyon işaretçisi + Delegate Mantığı)
  // ==========================================================================
  // EN: The UGLIEST Syntax known to mankind. -> return_type (Class::*pointerName)(args); TR: C++
  // insanlığa karşı bir suç olan Syntax: void (ECUModule::*pAction)(int);
  void (ECUModule::*pAction)(int) = &ECUModule::sendCANFrame;

  // EN: Native execution (Requires the painful `.*` or `->*` operators)
  // TR: Geleneksel/Korkunç yürütme şekli (Parantez cehennemi!)
  (bcm.*pAction)(10); // BCM sends a CAN frame!

  // EN: Wait! Let's change the "Action" mapping to diagnostic mode instead!
  // TR: Dur! Eylem atamasını değiştir, aynı pointer artık 'Diagnostic' fonksiyonunu kaydetsin!
  pAction = &ECUModule::activateDiagMode;
  (hvac.*pAction)(5); // HVAC activates diagnostic mode!

```

```
    // ==========================================================================
    // 3. MODERN C++17 `std::invoke` (The Hero / Kurtarıcı)
    // ==========================================================================
    // EN: std::invoke completely solves the horrible syntax above.
    // TR: std::invoke, C++ Komitesinin bu korkunç `.*` saçmalığını yok etmek için C++17'de yazdığı
    // çözümdür.
    std::cout << "\n[C++17 std::invoke MAGIC]" << std::endl;

    // Just invoke (Execute) the functionPointer ON the specific object WITH its arguments! TR:
    // Çağır (Invoke et) -> Neyi? pAction'ı -> Neyin Üstünde? bcm -> Hangi Argümanla? 15 BCM
    // activates diagnostic mode using modern magic.
    std::invoke(pAction, bcm, 15);

    // Also works naturally with Member Data (Değişkenlerde de aynı temizlik geçerlidir) Gets
    // HVAC's uptime smoothly!
    int currentParam = std::invoke(pDataMetric, hvac);
    std::cout << "Invoked HVAC Uptime: " << currentParam << std::endl;

    return 0;
  }
```

► 04. Constant Evaluation

module_09_advanced_mechanics/04_constant_evaluation.cpp

Konu: Advanced Mechanics: C++20 Constants (constexpr vs consteval vs constinit) — Sabit

Teori / Theory

1. constexpr

A `constexpr` function CAN be evaluated at compile-time (if all inputs are known). But if inputs are dynamic (from user), it naturally degrades to a normal run-time function! It is basically saying: "Compiler, please do this beforehand IF you can."

Derleme Zamanı VEYA Çalışma Zamanı Mimarisi (C++11/14)

`constexpr` fonksiyon, derleme (Build) anında işlemcinin hesaplayabileceği bir emirdir. ANCAK değişkenler kullanıcı elinden dinamik olarak gelirse, derleyici mırın kırın etmez, "Tamam bunu oyun içerisinde Runtime'da normal fonksiyon gibi yapacağım" der. Yani hem derlemede hem oyun anında esnek çalışır.

2. consteval

A `consteval` function (Immediate Function) MUST strictly be evaluated at compile-time! If the compiler cannot determine the inputs beforehand, it throws a HARD BUILD ERROR. Use this when you absolutely CANNOT afford calculation overhead in the final binary execution.

YALNIZCA Derleme Zamanı Şartı (C++20)

`consteval` "Hemen Hesapla!" (C++20) anlamına gelir. Eğer bu fonksiyon derleme aşamasında bitmezse Oyun asla derlenemez. Askeri/Cerrahi robot yazılımlarında, hızın sıfır kayıp (0 overhead) yaşaması istenen kriptolu hash / konfigürasyon işlemlerinde mecburidir.

3. constinit

The `constinit` keyword guarantees that a static/global variable is fully initialized during compilation (Constant Initialization). It solves the infamous "Static Initialization Order Fiasco" in massive C++ engines. The variable CAN be mutable (changeable) later!

Statik Değerlerin Temiz Başlangıcı (C++20)

`constinit` kelimesi Global veya Static değişkenleri hedefler. Sadece "Oyun açılmadan önce ilk değerini" derleme aşamasında sabitler. Ama `const` DEĞİLDİR! Yani oyun akarken sonradan değiştirilebilir! C++ motorlarındaki meşhur Global çökme bug'larını kökünden engeller.

CPPREF Depth: constexpr vs consteval vs constinit — Compile-Time Guarantees

`constexpr` (C++11): function/variable CAN be evaluated at compile time if all inputs are constant; otherwise degrades to runtime. `consteval` (C++20): "immediate function" — MUST be evaluated at compile time; runtime call is a hard error. `constinit` (C++20): the variable must be initialized at compile time, but may be modified at runtime — solves the static initialization order fiasco. `if consteval` (C++23) tests whether execution is currently in a constexpr evaluation context. Constexpr dynamic allocations (C++20): `new`/`delete` are allowed inside constexpr if fully deallocated by the end of evaluation.

CPPREF Derinlik: constexpr vs consteval vs constinit — Derleme Zamanı Garantileri

`constexpr` (C++11): tüm girdiler sabit ise derleme zamanında değerlendirilebilir; aksi halde çalışma zamanına düşer. `consteval` (C++20): "anlık fonksiyon" — derleme zamanında hesaplanmak ZORUNDADIR; çalışma zamanı çağrısı derleme hatasıdır. `constinit` (C++20): değişken derleme zamanında başlatılmalıdır ama çalışma zamanında değiştirilebilir — statik başlatma sırası fiyaskosunu çözer. `if consteval` (C++23), yürütmenin şu anda constexpr bağlamında olup olmadığını test eder. Constexpr dinamik tahsisler (C++20): `new`/`delete` değerlendirme sonunda tam serbest bırakılırsa constexpr içinde kullanılabilir.

Kaynak Kod / Source Code

```cpp
#include <iostream>

// ==========================================================================
// [constexpr] function (Flex / Esnek)
// ==========================================================================
constexpr int squareConstexpr(int n) { return n * n; }

// ==========================================================================
// [consteval] function (Strict Compile-Time / ASLA Çalışma Zamanına Kalmaz)
// ==========================================================================
// C++20 Feature! Only available if compiled with -std=c++20
consteval int squareConsteval(int n) {
  return n * n; // Strictly must be replaced by the answer before ECU boots!
}

// ==========================================================================
// [constinit] Variable (No Runtime startup cost, but Mutable)
// ==========================================================================
// Calculated at compile time (100).
constinit int GlobalConfigurationScale = squareConstexpr(10);
// Notice it is an `int` not `const int`. It can be mutated later!
// TR: Oyun başlamadan önce bu (100) oldu. Oyun oynanırken 500 yapılabilir!

int main() {
  std::cout << "=== MODULE 9: CONSTANTS DEEP DIVE (C++20) ===\n" << std::endl;

  // --- 1. constexpr (ESNEK TEST) ---
  // TR: Parametre sabit (5), o halde 25 olarak Derleme Zamaninda bitir ve Binaya gom!
  constexpr int compileTimeResult = squareConstexpr(5);

  int dynamicVar = 8;
  // TR: Parametre dışarıdan/dosyadan geldi, derleyici pes etti, bunu ÇALIŞMA (Run) zamanında
  // hesapladı!
  int runTimeResult = squareConstexpr(dynamicVar);

```

```
36    // --- 2. consteval (KUSURSUZ ASKER TESTİ) ---
37    // TR: Bu %100 Build aşamasında Assembly'e gömüldü. Maliyet 0 CPU.
38    int immediateResult = squareConsteval(6); // 36 is baked directly!
39
40    std::cout << "constexpr(5)=" << compileTimeResult << " | constexpr(" << dynamicVar << ")=" <<
41        runTimeResult << " | consteval(6)=" << immediateResult << std::endl;
42
43    // [FATAL ERROR]: This would completely crash the compilation! int
44    // failedAttempt = squareConsteval(dynamicVar);
45    // EN: The compiler cannot know what `dynamicVar` is at compile time, so `consteval` throws an
46    // error!
47    // TR: Derleyici dynamicVar'ın kaç olduğunu bilemez, C++20'ye göre cinayet!
48
49    // --- 3. constinit (GLOBAL/STATIC TEST) ---
50    std::cout << "Original constinit global Config Scale: " << GlobalConfigurationScale << std::endl;
51    // EN: Still mutable! It just started without runtime cost.
52    // TR: Başlangıç değeri 100 olarak derlemeden geldi ama sonra değiştirebildik!
53    GlobalConfigurationScale = 450;
54    std::cout << "Mutated Global Config Scale           : " << GlobalConfigurationScale << std::endl;
55
56    std::cout << "\n[ARCHITECTURAL RECAP]:" << std::endl;
57    std::cout << "1. Use 'constexpr' by default. (It falls back to runtime "
58                "gently). // Genelde bunu kullan."
59              << std::endl;
60    std::cout << "2. Use 'consteval' when overhead is forbidden. // Hız kaybı "
61                "ölümcül ise bunu kullan."
62              << std::endl;
63    std::cout << "3. Use 'constinit' on Globals/Statics to prevent startup "
64                "fiascos. // Globalleri güvene al."
65              << std::endl;
66
67    return 0;
68  }
```

► 05. Type Traits And Concepts

module_09_advanced_mechanics/05_type_traits_and_concepts.cpp

Konu: Advanced Mechanics: Type Traits (SFINAE) & C++20 Concepts — Kısıtlandırılmış Şablonlar

Teori / Theory

1. Type Traits & SFINAE (Substitution Failure Is Not An Error) - Eski C++ (C++11/14)

When using C++ Templates (`template <typename T>`), `T` can be ANYTHING (an `int`, a `std::string`, or an `ECUModule`). To restrict `T` (e.g., "Only accept numbers!"), modern C++ uses `<type_traits>` like `std::is_arithmetic_v<T>` combined with `std::enable_if`. If a substitution fails, the compiler simply ignores the overload rather than throwing an error (SFINAE!).

C++ Şablonlarını (Templates) kullanırken, Giren 'T' (Type) kelimesi HER ŞEY olabilir. Bir adam oraya 'Helikopter' sınıfını bile atabilir. Derleyiciyi kısıtlamak ("Buraya sadece rakam girebilir!") için Type Traits ve SFINAE denilen ürkütücü derecede karmaşık, ve efsanevi `std::enable_if` mekanizması kullanılırdı.

2. Concepts: The Savior of C++20

SFINAE is notoriously unreadable and generates 500-page long compiler error logs. C++20 introduced `requires` and `Concepts` to completely replace SFINAE! Concepts allow us to write clean, extremely readable restrictions on Templates.

Kurtarıcı Kavramlar (Modern C++20)

SFINAE mekanizması korkunç uzun hatalar fırlatmasıyla meşhurdur. C++ Komitesi, C++20 ile birlikte "Oluşumları/Kavramları (Concepts)" piyasaya sürdü! Artık `requires` yazarak kodun tamı tamına ne kabul ettiğini %100 okunabilirlikle derleyiciye izah edebiliyoruz. 500 sayılık hatalar yerine "Bu tipe izin yok!" alırız.

CPPREF Depth: Type Traits Implementation — How is_same and conditional Work Under the Hood

`is_same<T,U>`: primary template inherits `false_type`; partial specialization `is_same<T,T>` inherits `true_type`. `conditional<B,T,F>`: primary template defines `type = T`; specialization for `false` defines `type = F`. `enable_if<true, T>` defines member `type = T`; `enable_if<false>` has no `type` — this triggers SFINAE when used in template parameter deduction. `void_t<Ts...>` equals `void` if all `Ts` are well-formed — this is the detection idiom for checking members. C++20 concepts replace most SFINAE patterns with cleaner syntax: `requires` clauses, concept definitions, and constrained `auto`.

CPPREF Derinlik: Tür Özelliklerinin İç Yüzü — is_same ve conditional Nasıl Çalışır

`is_same<T,U>`: birincil şablon `false_type`'dan türer; kısmi özelleştirme `is_same<T,T>` ise `true_type`'dan türer. `conditional<B,T,F>`: birincil şablon `type = T` tanımlar; `false` özelleştirmesi `type = F` tanımlar. `enable_if<true, T>` `type = T` üyesini tanımlar; `enable_if<false>` hiçbir `type` içermez — şablon parametre çıkarımında SFINAE'yi tetikler. `void_t<Ts...>`, tüm `Ts` geçerliyse `void`'e eşittir — üye tespit deyimi (detection idiom) budur. C++20 kavramları (concepts), SFINAE kalıplarının çoğunu daha temiz bir sözdizimi ile değiştirir: `requires` cümleleri, kavram tanımları ve kısıtlı `auto`.

Kaynak Kod / Source Code

```cpp
#include <concepts> // Modern C++20 Concepts!
#include <iostream>
#include <string>
#include <type_traits> // Old C++11 traits

class Helicopter {}; // Dummy object / Eğlence için ucube obje

// ==========================================================================
// [APPROACH 1: The OLD SFINAE Way (C++14) - Extremely Ugly and Cryptic]
// ==========================================================================
// EN: "This function only exists if T is an arithmetic (number) type."
// TR: "Bu fonksiyon SADECE 'T' bir sayı türüyse kabul edilsin, yoksa yok sayılsın."
template <typename T, typename std::enable_if_t<std::is_arithmetic_v<T>, bool> = true>
void addTaxes_SFINAE(T price) {
  std::cout << "[SFINAE (Old)] New Price: " << (price * 1.18) << std::endl;
}

// ==========================================================================
// [APPROACH 2: The NEW C++20 Concept Way - Beautiful and Readable!]
// ==========================================================================
// EN: "This function REQUIRES 'T' to be a floating-point number (double/float)." TR: "Bu şablon
// FONKSİYON, T'nin ONDALIKLI SAYI (float, double) olmasını EMREDER! (requires)"
template <typename T>
  requires std::floating_point<T>
void addTaxes_Concept(T price) {
  std::cout << "[C++20 Concept] Floating Tax Calculation: " << (price * 1.18) << std::endl;
}

// EN: We can even create our Custom Concepts!
// TR: Kendi emrimizi / kısıtlamamızı özel Concept kelimesiyle (Mimarisi) yaratabiliriz!
template <typename T>
concept MustBePrintable = requires(T x) {
  // If this code compiles, the concept is met! (Bu derlenirse kabul!)
  std::cout << x;
};

```

```
// Target structure that enforces OUR custom concept. (Bizim konseptimiz!)
template <MustBePrintable T> void customPrinter(T data) {
  std::cout << "[Custom Concept Printer]: > " << data << std::endl;
}

int main() {
  std::cout << "=== MODULE 9: TYPE TRAITS & CONCEPTS (C++20) ===\n" << std::endl;

  Helicopter apache;
  (void)apache; // EN: Used only for SFINAE rejection demo

  // --- 1. SFINAE LOGIC (C++11/14) ---
  addTaxes_SFINAE(100);  // int is arithmetic. OK! (int izinlidir)
  addTaxes_SFINAE(45.5); // double is arithmetic. OK! (double izinlidir)
  // addTaxes_SFINAE("10");     // SFINAE REJECTS (String is not arithmetic)
  // addTaxes_SFINAE(apache);   // SFINAE REJECTS (Helicopter cannot pay taxes)

  // --- 2. C++20 CONCEPTS LOGIC --- addTaxes_Concept(100);    // CONCEPT REJECTS / BUILD ERROR!
  // (100 is int, NOT a float!)
  addTaxes_Concept(45.5f); // OK! (float is floating_point)
  addTaxes_Concept(20.90); // OK! (double is floating_point)

  // --- 3. CUSTOM CONCEPT IMPLEMENTATION ---
  customPrinter(999);           // Integers can be printed via std::cout
  customPrinter("Hello World"); // Strings can be printed via std::cout
  // customPrinter(apache);      // WILL CAUSE AN ERROR! (No overload for << on Helicopter)

  std::cout << "\n[ARCHITECTURAL CONCLUSION / MİMARİ SONUÇ]:" << std::endl;
  std::cout << "- SFINAE and enable_if are obsolete. Move purely to C++20 "
               "`requires` & `Concepts`."
            << std::endl;
  std::cout << "- C++20 Kavramları, karmaşık Template kodlarının "
               "okunabilirliğini C#'taki Generics seviyesine taşır!"
            << std::endl;

  return 0;
}
```

► 06. Sfinae And Tag Dispatch

module_09_advanced_mechanics/06_sfinae_and_tag_dispatch.cpp

Konu: SFINAE, Tag Dispatch & if constexpr — Compile-Time Overload Selection

Teori / Theory

1. SFINAE — "Substitution Failure Is Not An Error"

THEORY

When the compiler tries to instantiate a function template, it substitutes the provided type arguments into the template signature. If that substitution produces an invalid type or expression, the compiler does NOT emit a hard error — it simply removes that overload from the candidate set. This is the SFINAE principle. Combined with std::enable_if, we can conditionally enable/disable overloads based on type properties at compile time.

TEORİ

Derleyici bir fonksiyon şablonunu somutlaştırmaya çalışırken, sağlanan tür argümanlarını şablon imzasına yerleştirir. Bu yerleştirme geçersiz bir tür veya ifade üretirse, derleyici hata VERMEZ — o aşırı yüklemeyi (overload) aday kümesinden sessizce çıkarır. Bu SFINAE prensibidir.

2. std::enable_if<condition, T>

enable_if conditionally defines a nested type `type` only when `condition` is true. When false, the substitution fails (SFINAE) and the overload is discarded. This lets us create separate function overloads for integral vs floating-point vs custom types — all resolved at compile time.

enable_if, yalnızca koşul doğru olduğunda iç içe `type` türünü tanımlar. Yanlış olduğunda yerleştirme başarısız olur (SFINAE) ve aşırı yükleme atılır. Böylece tamsayı, ondalıklı ve özel türler için ayrı fonksiyon aşırı yüklemeleri — hepsi derleme zamanında çözülür.

3. Tag Dispatch

Tag dispatch uses empty struct "tags" as function parameters to guide overload resolution at compile time. The caller passes a tag object, and the compiler selects the matching overload. This gives clean, readable compile-time branching without any enable_if clutter.

Etiket dağıtımı, derleme zamanında aşırı yükleme çözümlemesini yönlendirmek için boş yapı "etiketlerini" fonksiyon parametresi olarak kullanır. Çağıran bir etiket nesnesi geçirir, derleyici eşleşen aşırı yüklemeyi seçer.

4. if constexpr (C++17)

C++17 introduced `if constexpr` — a compile-time if-statement. The branch not taken is completely discarded by the compiler (not even type-checked). This is a much cleaner alternative to SFINAE for simple compile-time branching inside a single function template.

C++17 ile gelen `if constexpr`, derleme zamanı if ifadesidir. Alınmayan dal derleyici tarafından tamamen atılır (tür kontrolü bile yapılmaz). Basit derleme zamanı dallanma için SFINAE'den çok daha temiz bir yoldur.

5. Type Traits Used: std::is_integral, std::is_floating_point, std::is_same

The <type_traits> header provides compile-time boolean queries about types. std::is_integral_v<T> is true for int, long, char, etc. std::is_floating_point_v<T> is true for float, double, long double. std::is_same_v<T, U> checks if T and U are the exact same type. These traits are the building blocks of all SFINAE-based dispatch.

<type_traits> başlığı, türler hakkında derleme zamanı sorgulamaları sağlar. Bu özellikler, tüm SFINAE tabanlı dağıtımın yapı taşlarıdır.

CPPREF Depth: SFINAE Contexts and the Expression SFINAE Revolution

SFINAE only operates in the "immediate context" of template argument substitution — errors inside a function body are hard compilation errors, NOT SFINAE. Expression SFINAE (C++11): placing `decltype(expr)` in the return type triggers SFINAE if `expr` is ill-formed. `std::void_t<>` simplifies member detection by mapping any well-formed type list to `void`. `if constexpr` (C++17) eliminates most SFINAE need for branching within a single function body. `requires` expressions (C++20) are the final evolution — readable, composable constraints with clear compiler error messages that replace the arcane enable_if chains.

CPPREF Derinlik: SFINAE Bağlamları ve İfade SFINAE Devrimi

SFINAE yalnızca şablon argüman yerleştirmesinin "yakın bağlamında" çalışır — fonksiyon gövdesindeki hatalar SFINAE değil, kesin derleme hatalarıdır. İfade SFINAE (C++11): dönüş türüne `decltype(expr)` yerleştirildiğinde, `expr` geçersizse SFINAE tetiklenir. `std::void_t<>`, herhangi bir geçerli tür listesini `void`'e eşleyerek üye tespitini kolaylaştırır. `if constexpr` (C++17), tek bir fonksiyon gövdesindeki dallanma için SFINAE ihtiyacını büyük ölçüde ortadan kaldırır. `requires` ifadeleri (C++20) son evrimdir — gizemli enable_if zincirlerinin yerini alan okunabilir, birleştirilebilir kısıtlamalar ve açık hata mesajları.

Kaynak Kod / Source Code

```cpp
#include <cstdint>      // EN: Fixed-width integer types (uint8_t)
#include <iostream>     // EN: Standard I/O for console output
#include <string>       // EN: std::string for text data
#include <type_traits>  // EN: Type traits (is_integral, enable_if, etc.)
#include <sstream>      // EN: String stream for serialization

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 1: SFINAE with std::enable_if — ECU Data Processing
// ═══════════════════════════════════════════════════════════════════════════

// EN: Process integral ECU data (raw ADC counts, register values, CAN IDs)
// TR: Tamsayı ECU verilerini işle (ham ADC sayıları, register değerleri, CAN ID'leri)
template <typename T, std::enable_if_t<std::is_integral_v<T>, int> = 0>
void processECUData(T value) {
    std::cout << "  [INTEGRAL path] Raw ECU register value: 0x"
              << std::hex << value << std::dec
              << " (" << value << " decimal)" << std::endl;
}

// EN: Process floating-point ECU data (calibrated sensor readings, voltages)
// TR: Ondalıklı ECU verilerini işle (kalibre sensör okumaları, voltajlar)
template <typename T, std::enable_if_t<std::is_floating_point_v<T>, int> = 0>
void processECUData(T value) {
    std::cout << "  [FLOAT path] Calibrated sensor reading: "
              << value << " V" << std::endl;
}

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 2: Tag Dispatch — Sensor Data Pipeline
// ═══════════════════════════════════════════════════════════════════════════

// EN: Tag types — empty structs used solely for overload selection
// TR: Etiket türleri — yalnızca aşırı yükleme seçimi için kullanılan boş yapılar
struct RawSensorTag {};
struct CalibratedSensorTag {};

// ─── 2a. Implementation overloads selected by tag ──────────────────────────

// EN: Process raw sensor data (needs filtering, offset correction)
// TR: Ham sensör verisini işle (filtreleme, ofset düzeltmesi gerekir)
template <typename T>
void processSensorData(T value, RawSensorTag) {
    constexpr double offset = 0.33;
    constexpr double gain   = 1.05;
    double corrected = static_cast<double>(value) * gain - offset;
    std::cout << "  [RAW] Input: " << value
              << " -> After offset/gain correction: " << corrected << std::endl;
}

// EN: Process calibrated sensor data (already engineering units)
// TR: Kalibre sensör verisini işle (zaten mühendislik birimlerinde)
template <typename T>
void processSensorData(T value, CalibratedSensorTag) {
    std::cout << "  [CALIBRATED] Direct engineering value: " << value
              << " (no correction needed)" << std::endl;
}
```

```

// ─── 2b. Public dispatch interface ──────────────────────────────────────────

// EN: Caller passes a tag to choose the processing path at compile time
// TR: Çağıran, derleme zamanında işleme yolunu seçmek için bir etiket geçirir
template <typename T, typename Tag>
void dispatchSensor(T value, Tag tag) {
    processSensorData(value, tag);
}

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 3: if constexpr (C++17) — Modern Replacement
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Single function with compile-time branching — no SFINAE boilerplate
// TR: Derleme zamanı dallanmalı tek fonksiyon — SFINAE şablonu gerekmez
template <typename T>
void processModern(T value) {
    if constexpr (std::is_integral_v<T>) {
        std::cout << "  [if constexpr → INTEGRAL] Register: 0x"
                  << std::hex << value << std::dec << std::endl;
    } else if constexpr (std::is_floating_point_v<T>) {
        std::cout << "  [if constexpr → FLOAT] Sensor voltage: "
                  << value << " V" << std::endl;
    } else {
        std::cout << "  [if constexpr → OTHER] Unknown data type, forwarding raw bytes"
                  << std::endl;
    }
}

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 4: Has-Member Detection via SFINAE — .toString() check
// ═══════════════════════════════════════════════════════════════════════════════

// ─── 4a. Detector trait ─────────────────────────────────────────────────────

// EN: Primary template — fallback when T does NOT have .toString()
// TR: Birincil şablon — T'de .toString() YOKSA geri dönüş
template <typename T, typename = void>
struct HasToString : std::false_type {};

// EN: Specialization — enabled only when T has a .toString() method
// TR: Uzmanlık — yalnızca T'de .toString() metodu olduğunda etkinleşir
template <typename T>
struct HasToString<T, std::void_t<decltype(std::declval<T>().toString())>>
    : std::true_type {};

// ─── 4b. Serialize function using the detector ──────────────────────────────

// EN: If T has .toString(), use it; otherwise fall back to stream insertion
// TR: T'de .toString() varsa onu kullan; yoksa akış ekleme operatörüne dön
template <typename T>
auto serialize(const T& val) -> std::string {
    if constexpr (HasToString<T>::value) {
        return val.toString();
    } else {
        std::ostringstream oss;
        oss << val;
        return oss.str();
    }
}

// ─── 4c. Test types ─────────────────────────────────────────────────────────

// EN: Automotive diagnostic trouble code — has .toString()
// TR: Otomotiv arıza kodu — .toString() metodu var
struct DiagnosticCode {
    int code;
    std::string module;

    std::string toString() const {
        return "DTC-" + module + "-" + std::to_string(code);
    }
};
```

```cpp

// EN: Plain voltage reading — no .toString(), relies on operator<<
// TR: Düz voltaj okuması — .toString() yok, operator<< kullanır
struct VoltageReading {
    double volts;

    friend std::ostream& operator<<(std::ostream& os, const VoltageReading& vr) {
        return os << vr.volts << "V";
    }
};

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 5: Compile-Time Error Prevention Demo Types
// ═══════════════════════════════════════════════════════════════════════════════

// EN: A constrained function that only accepts pointer types (e.g., memory-mapped I/O)
// TR: Yalnızca işaretçi türlerini kabul eden kısıtlı fonksiyon (örn. bellek eşlemeli G/Ç)
template <typename T, std::enable_if_t<std::is_pointer_v<T>, int> = 0>
void writeMemoryMappedIO(T addr, uint8_t value) {
    // EN: In production, this would write to a hardware register
    // TR: Üretimde bu, bir donanım kaydına yazardı
    std::cout << "  [MMIO] Writing 0x" << std::hex << static_cast<int>(value)
              << std::dec << " to address " << static_cast<const void*>(addr)
              << std::endl;
}

// ═══════════════════════════════════════════════════════════════════════════════
// MAIN — Demonstrations
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "╔═══════════════════════════════════════════════════════════╗\n";
    std::cout << "║  MODULE 09: SFINAE, Tag Dispatch & if constexpr           ║\n";
    std::cout << "╚═══════════════════════════════════════════════════════════╝\n\n";

    // ─── Demo 1: SFINAE enable_if — integral vs floating-point paths ──────────
    std::cout << "── Demo 1: SFINAE enable_if (ECU Data Processing) ──\n";
    // EN: Integral type → hex register path
    // TR: Tamsayı türü → hex register yolu
    processECUData(0x1A3F);
    // EN: Floating-point type → calibrated sensor path
    // TR: Ondalıklı tür → kalibre sensör yolu
    processECUData(3.28);
    processECUData(255);
    processECUData(1.5f);
    std::cout << std::endl;

    // ─── Demo 2: Tag dispatch — raw vs calibrated sensor data ─────────────────
    std::cout << "── Demo 2: Tag Dispatch (Sensor Pipeline) ──\n";
    // EN: Raw ADC reading needs offset/gain correction
    // TR: Ham ADC okuması ofset/kazanç düzeltmesi gerektirir
    dispatchSensor(1023, RawSensorTag{});
    dispatchSensor(2.48, RawSensorTag{});
    // EN: Calibrated value already in engineering units
    // TR: Kalibre değer zaten mühendislik birimlerinde
    dispatchSensor(98.6, CalibratedSensorTag{});
    dispatchSensor(14.7, CalibratedSensorTag{});
    std::cout << std::endl;

    // ─── Demo 3: if constexpr — modern compile-time branching ─────────────────
    std::cout << "── Demo 3: if constexpr (Modern Replacement) ──\n";
    // EN: Same function, different compile-time branches based on type
    // TR: Aynı fonksiyon, türe göre farklı derleme zamanı dalları
    processModern(0xFF);
    processModern(4.75);
    processModern("CAN_BUS_MSG");
    std::cout << std::endl;

    // ─── Demo 4: Has-member detection — .toString() check ─────────────────────
    std::cout << "── Demo 4: Has-Member Detection (.toString()) ──\n";
    DiagnosticCode dtc{4201, "ECM"};
    VoltageReading vr{12.6};

    // EN: DiagnosticCode has .toString() → uses it
    // TR: DiagnosticCode .toString() metoduna sahip → onu kullanır
    std::cout << "  DiagnosticCode serialized: " << serialize(dtc) << std::endl;
```

```
    // EN: VoltageReading has no .toString() → uses operator<<
    // TR: VoltageReading .toString() yok → operator<< kullanır
    std::cout << "  VoltageReading serialized: " << serialize(vr) << std::endl;
    // EN: Plain int has no .toString() → uses operator<<
    // TR: Düz int .toString() yok → operator<< kullanır
    std::cout << "  int serialized:            " << serialize(42) << std::endl;

    // EN: Compile-time trait verification
    // TR: Derleme zamanı özellik doğrulaması
    static_assert(HasToString<DiagnosticCode>::value,
                  "DiagnosticCode must have .toString()");
    static_assert(!HasToString<VoltageReading>::value,
                  "VoltageReading must NOT have .toString()");
    static_assert(!HasToString<int>::value,
                  "int must NOT have .toString()");
    std::cout << std::endl;

    // ── Demo 5: Compile-time error prevention ──────────────────────────────
    std::cout << "── Demo 5: Compile-Time Error Prevention (MMIO) ──\n";
    uint8_t fake_register = 0;
    // EN: Pointer type → SFINAE allows the call
    // TR: İşaretçi türü → SFINAE çağrıya izin verir
    writeMemoryMappedIO(&fake_register, 0xAB);

    // EN: Uncommenting the line below would cause a compile error — SFINAE removes the overload
    // because int is not a pointer type.
    // TR: Aşağıdaki satırın yorumunu kaldırmak derleme hatasına neden olur — SFINAE, int bir
    // işaretçi türü olmadığı için aşırı yüklemeyi kaldırır. writeMemoryMappedIO(0x4000'0000,
    // 0xAB);  // ERROR: int is not a pointer!

    std::cout << "  (Passing a non-pointer to writeMemoryMappedIO is a compile error)\n";
    std::cout << std::endl;

    // ── Summary ────────────────────────────────────────────────────────────
    std::cout << "╔═══════════════════════════════════════════════════════════╗\n";
    std::cout << "│  SUMMARY / ÖZET                                           │\n";
    std::cout << "╟───────────────────────────────────────────────────────────╢\n";
    std::cout << "│  1. SFINAE + enable_if: powerful but verbose              │\n";
    std::cout << "│  2. Tag dispatch: clean compile-time overload selection   │\n";
    std::cout << "│  3. if constexpr (C++17): simplest for inline branching   │\n";
    std::cout << "│  4. Has-member detection: SFINAE metadata introspection   │\n";
    std::cout << "│  5. For C++20 projects, prefer Concepts over SFINAE       │\n";
    std::cout << "╚═══════════════════════════════════════════════════════════╝\n";

    return 0;
}
```

► 07. Crtp Pattern

module_09_advanced_mechanics/07_crtp_pattern.cpp

Konu: CRTP — Curiously Recurring Template Pattern for Static Polymorphism

Teori / Theory

1. CRTP — Curiously Recurring Template Pattern

THEORY

CRTP is a C++ idiom where a class Derived inherits from a base class template parameterized by Derived itself: class Derived : public Base<Derived> The base class can then static_cast<Derived*>(this) to call methods on the derived class — all resolved at compile time with no virtual dispatch.

TEORİ

CRTP, bir Derived sınıfının kendisi ile parametrize edilmiş bir temel sınıf şablonundan türetildiği C++ deyimidir: class Derived : public Base<Derived> Temel sınıf, türetilmiş sınıftaki metotları çağırmak için static_cast<Derived*>(this) kullanabilir — hepsi derleme zamanında çözülür.

2. Static Polymorphism vs Virtual Dispatch

Virtual functions require a vtable pointer per object and an indirect call through the vtable at runtime. CRTP resolves the call at compile time:
1. No vtable pointer (8 bytes saved per object on 64-bit).
2. No indirect branch (better branch prediction, inlineable by compiler).
3. No heap allocation required for polymorphic behavior. Trade-off: CRTP types are not runtime-polymorphic (no base-class pointer).

Sanal fonksiyonlar nesne başına bir vtable işaretçisi ve çalışma zamanında vtable üzerinden dolaylı çağrı gerektirir. CRTP çağrıyı derleme zamanında çözer:
1. vtable işaretçisi yok (64-bit'te nesne başına 8 bayt tasarruf).
2. Dolaylı dal yok (daha iyi dal tahmini, derleyici tarafından inline edilebilir).
3. Polimorfik davranış için yığın tahsisi gerekmez. Takas: CRTP türleri çalışma zamanında polimorfik değildir (temel sınıf pointer'ı yok).

3. Use Cases: Mixins, Static Interfaces, Expression Templates

1. Mixins: inject reusable functionality (logging, counting, serialization) into unrelated classes without virtual overhead.
2. Static interfaces: enforce that derived classes implement certain methods, checked at compile time via static_cast.
3. Expression templates: build lazy evaluation trees (e.g., Eigen library).

1. Mixin'ler: sanal maliyet olmadan ilgisiz sınıflara yeniden kullanılabilir işlevsellik (loglama, sayma, serileştirme) enjekte eder.
2. Statik arayüzler: türetilmiş sınıfların belirli metotları uygulamasını derleme zamanında zorlar.
3. İfade şablonları: tembel değerlendirme ağaçları oluşturur.

4. Common Pitfall: Calling Derived Methods from Base Constructor

When Base<Derived>'s constructor runs, the Derived object is NOT fully constructed yet. Calling static_cast<Derived*>(this)->method() from the base constructor is undefined behavior! Always call derived methods from regular member functions, never from constructors or destructors.

Base<Derived> yapıcısı çalıştığında, Derived nesnesi henüz tam olarak oluşturulmamıştır. Temel yapıcıdan static_cast<Derived*>(this)->method() çağırmak tanımsız davranıştır! Türetilmiş metotları her zaman normal üye fonksiyonlarından çağırın, asla yapıcı veya yıkıcılardan çağırmayın.

CPPREF Depth: CRTP Pitfalls and the Deducing-This Alternative (C++23)

Pitfall #1: calling `static_cast<Derived*>(this)` in the base constructor — Derived is not yet constructed, invoking its methods is UB. Pitfall #2: forgetting to pass the correct Derived type (copy-paste error such as inheriting from `Base<WrongClass>` instead of `Base<Derived>`). Pitfall #3: CRTP does not support runtime polymorphism — there is no common base-class pointer for heterogeneous collections. C++23 deducing-this (`this auto&& self`) eliminates CRTP for most use cases: the base class method deduces the exact derived type from the implicit object parameter — cleaner syntax, same zero-cost dispatch, and no inheritance required.

CPPREF Derinlik: CRTP Tuzakları ve Deducing-This Alternatifi (C++23)

Tuzak #1: temel yapıcıda `static_cast<Derived*>(this)` çağırmak — Derived henüz oluşturulmamıştır, metotlarını çağırmak UB'dir. Tuzak #2: doğru Derived türünü geçirmeyi unutmak (kopyala-yapıştır hatası, örneğin `Base<Derived>` yerine `Base<YanlışSınıf>`'tan türetme). Tuzak #3: CRTP çalışma zamanı polimorfizmini desteklemez — heterojen koleksiyonlar için ortak bir temel sınıf işaretçisi yoktur. C++23 deducing-this (`this auto&& self`) çoğu kullanım için CRTP'yi ortadan kaldırır: temel sınıf metodu, örtük nesne parametresinden tam türetilmiş türü çıkarır — daha temiz sözdizimi, aynı sıfır maliyetli dağıtım, kalıtım gerekmez.

Kaynak Kod / Source Code

```cpp
#include <chrono>    // EN: High-resolution clock for benchmarking
#include <cstdint>   // EN: Fixed-width integer types
#include <iostream>   // EN: Standard I/O for console output
#include <string>     // EN: std::string for log messages

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 1: Static Polymorphism — SensorBase with CRTP
// ═══════════════════════════════════════════════════════════════════════════

// EN: CRTP base — defines the common algorithm, delegates specifics to Derived
// TR: CRTP tabanı — ortak algoritmayı tanımlar, özellikleri Derived'a devreder
template <typename Derived>
```

```
13  class SensorBase {
14  public:
15      // EN: Template method: read → process → report (algorithm skeleton)
16      // TR: Şablon metodu: oku → işle → raporla (algoritma iskeleti)
17      void readAndProcess() {
18          // EN: static_cast to Derived — resolved at compile time, zero overhead
19          // TR: Derived'a static_cast — derleme zamanında çözülür, sıfır maliyet
20          auto& self = *static_cast<Derived*>(this);
21          double raw = self.doRead();
22          std::cout << "  [" << self.name() << "] Raw: " << raw
23                    << " → Processed: " << raw * self.scaleFactor() << std::endl;
24      }
25  };
26
27  // ─── 1a. TemperatureSensor (NTC thermistor on coolant line) ──────────────────────────
28
29  // EN: Reads coolant temperature via ADC, applies NTC conversion factor
30  // TR: ADC üzerinden soğutma suyu sıcaklığını okur, NTC dönüşüm katsayısı uygular
31  class TemperatureSensor : public SensorBase<TemperatureSensor> {
32  public:
33      double doRead() const { return 2048.0; }
34      double scaleFactor() const { return 0.0489; }
35      const char* name() const { return "CoolantTemp"; }
36  };
37
38  // ─── 1b. PressureSensor (MAP sensor on intake manifold) ──────────────────────────────
39
40  // EN: Reads manifold absolute pressure, applies bar conversion
41  // TR: Manifold mutlak basıncını okur, bar dönüşümü uygular
42  class PressureSensor : public SensorBase<PressureSensor> {
43  public:
44      double doRead() const { return 3200.0; }
45      double scaleFactor() const { return 0.000305; }
46      const char* name() const { return "MAP_Sensor"; }
47  };
48
49  // ═══════════════════════════════════════════════════════════════════════════════════════
50  // SECTION 2: CRTP Mixin — Loggable (adds logging to any class)
51  // ═══════════════════════════════════════════════════════════════════════════════════════
52
53  // EN: Mixin that injects logging capability into any Derived class via CRTP
54  // TR: CRTP aracılığıyla herhangi bir Derived sınıfına loglama yeteneği enjekte eden mixin
55  template <typename Derived>
56  class Loggable {
57  public:
58      void log(const std::string& message) const {
59          // EN: Access derived class's identifier without virtual dispatch
60          // TR: Sanal dağıtım olmadan türetilmiş sınıfın tanımlayıcısına erişim
61          const auto& self = *static_cast<const Derived*>(this);
62          std::cout << "  [LOG:" << self.moduleName() << "] " << message << std::endl;
63      }
64  };
65
66  // EN: ECU module that gets logging functionality for free via CRTP mixin
67  // TR: CRTP mixin aracılığıyla ücretsiz loglama işlevselliği alan ECU modülü
68  class ECUModule : public Loggable<ECUModule> {
69      std::string name_;
70  public:
71      explicit ECUModule(std::string name) : name_(std::move(name)) {}
72      const std::string& moduleName() const { return name_; }
73
74      void initialize() {
75          log("Initialization sequence started");
76          log("Self-test PASSED, module ready");
77      }
78
79      void shutdown() {
80          log("Graceful shutdown initiated");
81      }
82  };
83
```

```cpp
// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 3: CRTP Mixin — InstanceCounter (per-type instance tracking)
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Each Derived type gets its OWN static counter (separate template instantiation)
// TR: Her Derived türü KENDİ statik sayacını alır (ayrı şablon somutlaştırması)
template <typename T>
class InstanceCounter {
    static inline int count_ = 0;  // EN: C++17 inline static
public:
    InstanceCounter()  { ++count_; }
    InstanceCounter(const InstanceCounter&) { ++count_; }
    InstanceCounter(InstanceCounter&&) noexcept { ++count_; }
    ~InstanceCounter() { --count_; }

    static int instanceCount() { return count_; }
};

// EN: Injector hardware driver — tracks how many instances are alive
// TR: Enjektör donanım sürücüsü — kaç örneğin canlı olduğunu izler
class InjectorDriver : public InstanceCounter<InjectorDriver> {
    int cylinder_;
public:
    explicit InjectorDriver(int cyl) : cylinder_(cyl) {}
    int cylinder() const { return cylinder_; }
};

// EN: Ignition coil driver — separate counter from InjectorDriver
// TR: Ateşleme bobini sürücüsü — InjectorDriver'dan ayrı sayaç
class IgnitionCoilDriver : public InstanceCounter<IgnitionCoilDriver> {
    int cylinder_;
public:
    explicit IgnitionCoilDriver(int cyl) : cylinder_(cyl) {}
    int cylinder() const { return cylinder_; }
};

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 4: Virtual Dispatch Baseline (for benchmark comparison)
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Abstract sensor using traditional virtual dispatch
// TR: Geleneksel sanal dağıtım kullanan soyut sensör
class VirtualSensorBase {
public:
    virtual ~VirtualSensorBase() = default;
    virtual double doRead() const = 0;
    virtual double scaleFactor() const = 0;

    double readAndProcess() const {
        return doRead() * scaleFactor();
    }
};

class VirtualTemperatureSensor : public VirtualSensorBase {
public:
    double doRead() const override { return 2048.0; }
    double scaleFactor() const override { return 0.0489; }
};

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 5: Template Method Pattern via CRTP
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Diagnostic routine with fixed steps, customizable per ECU subsystem
// TR: Sabit adımları olan, ECU alt sistemi başına özelleştirilebilir teşhis rutini
template <typename Derived>
```

```cpp
class DiagnosticRoutine {
public:
    void runFullDiagnostic() {
        auto& self = *static_cast<Derived*>(this);
        std::cout << "  [DIAG] Starting diagnostic for: " << self.subsystemName() << std::endl;
        std::cout << "  [DIAG] Step 1 — Pre-check:  " << (self.preCheck()  ? "OK" : "FAIL") <<
            std::endl;
        std::cout << "  [DIAG] Step 2 — Core test:   " << (self.coreTest()  ? "OK" : "FAIL") <<
            std::endl;
        std::cout << "  [DIAG] Step 3 — Post-check:  " << (self.postCheck() ? "OK" : "FAIL") <<
            std::endl;
        std::cout << "  [DIAG] Diagnostic complete." << std::endl;
    }
};

// EN: Fuel system diagnostic — implements the three customizable steps
// TR: Yakıt sistemi teşhisi — üç özelleştirilebilir adımı uygular
class FuelSystemDiag : public DiagnosticRoutine<FuelSystemDiag> {
public:
    const char* subsystemName() const { return "FuelSystem"; }
    bool preCheck()  const { return true;  }
    bool coreTest()  const { return true;  }
    bool postCheck() const { return true;  }
};

// EN: Brake system diagnostic
// TR: Fren sistemi teşhisi
class BrakeSystemDiag : public DiagnosticRoutine<BrakeSystemDiag> {
public:
    const char* subsystemName() const { return "BrakeSystem"; }
    bool preCheck()  const { return true;  }
    bool coreTest()  const { return false; }
    bool postCheck() const { return true;  }
};

// ═══════════════════════════════════════════════════════════════════════════════
// MAIN — Demonstrations
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "╔═══════════════════════════════════════════════════════════╗\n";
    std::cout << "║  MODULE 09: CRTP — Curiously Recurring Template Pattern    ║\n";
    std::cout << "╚═══════════════════════════════════════════════════════════╝\n\n";

    // ── Demo 1: Static polymorphism via SensorBase CRTP ──────────────────────
    std::cout << "── Demo 1: Static Polymorphism (SensorBase CRTP) ──\n";
    TemperatureSensor tempSensor;
    PressureSensor    mapSensor;
    // EN: Same interface, different implementations — no vtable involved
    // TR: Aynı arayüz, farklı uygulamalar — vtable yok
    tempSensor.readAndProcess();
    mapSensor.readAndProcess();
    std::cout << std::endl;

    // ── Demo 2: Loggable mixin — ECUModule gets logging for free ─────────────
    std::cout << "── Demo 2: Loggable Mixin (ECU Module) ──\n";
    ECUModule engineControl("EngineCtrl");
    ECUModule absModule("ABS_Unit");
    // EN: Logging injected via CRTP, no virtual dispatch overhead
    // TR: CRTP aracılığıyla enjekte edilen loglama, sanal dağıtım yükü yok
    engineControl.initialize();
    absModule.initialize();
    engineControl.shutdown();
    std::cout << std::endl;

    // ── Demo 3: InstanceCounter — separate counts per type ───────────────────
    std::cout << "── Demo 3: InstanceCounter (Per-Type Tracking) ──\n";
    InjectorDriver inj1(1), inj2(2), inj3(3), inj4(4);
    IgnitionCoilDriver coil1(1), coil2(2);
    std::cout << "  InjectorDriver instances:     "
              << InjectorDriver::instanceCount() << std::endl;
    std::cout << "  IgnitionCoilDriver instances:  "
              << IgnitionCoilDriver::instanceCount() << std::endl;
    {
```

```
        // EN: Create temporary instances inside a scope
        // TR: Bir kapsam içinde geçici örnekler oluştur
        InjectorDriver temp5(5);
        IgnitionCoilDriver temp3(3);
        std::cout << "  [inner scope] Injectors: "
                  << InjectorDriver::instanceCount()
                  << ", Coils: " << IgnitionCoilDriver::instanceCount() << std::endl;
    }
    // EN: Temporaries destroyed — count decremented
    // TR: Geçiciler yok edildi — sayaç azaltıldı
    std::cout << "  [after scope] Injectors: "
              << InjectorDriver::instanceCount()
              << ", Coils: " << IgnitionCoilDriver::instanceCount() << std::endl;
    std::cout << std::endl;

    // ── Demo 4: Performance comparison — CRTP vs virtual dispatch ──────────────────────
    std::cout << "── Demo 4: CRTP vs Virtual Dispatch Benchmark ──\n";
    constexpr int iterations = 10'000'000;
    volatile double sink = 0.0;

    // EN: CRTP benchmark (compile-time dispatch, inlineable)
    // TR: CRTP karşılaştırması (derleme zamanı dağıtımı, satır içi yapılabilir)
    {
        TemperatureSensor crtp_sensor;
        auto start = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < iterations; ++i) {
            auto& base = static_cast<SensorBase<TemperatureSensor>&>(crtp_sensor);
            // EN: Inline-able static dispatch
            // TR: Satır içi yapılabilen statik dağıtım
            sink = crtp_sensor.doRead() * crtp_sensor.scaleFactor();
            (void)base;
        }
        auto end = std::chrono::high_resolution_clock::now();
        auto crtp_us = std::chrono::duration_cast<std::chrono::microseconds>(end - start).count();
        std::cout << "  CRTP dispatch:    " << crtp_us << " µs (" << iterations << " iterations)"
            << std::endl;
    }

    // EN: Virtual dispatch benchmark (runtime indirect call)
    // TR: Sanal dağıtım karşılaştırması (çalışma zamanı dolaylı çağrı)
    {
        VirtualTemperatureSensor virt_sensor;
        VirtualSensorBase* base_ptr = &virt_sensor;
        auto start = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < iterations; ++i) {
            // EN: Indirect call through vtable
            // TR: Vtable üzerinden dolaylı çağrı
            sink = base_ptr->readAndProcess();
        }
        auto end = std::chrono::high_resolution_clock::now();
        auto virt_us = std::chrono::duration_cast<std::chrono::microseconds>(end - start).count();
        std::cout << "  Virtual dispatch: " << virt_us << " µs (" << iterations << " iterations)"
            << std::endl;
    }
    (void)sink;
    std::cout << std::endl;

    // ── Demo 5: Template method pattern via CRTP ────────────────────────────────────────
    std::cout << "── Demo 5: Template Method Pattern via CRTP ──\n";
    FuelSystemDiag  fuelDiag;
    BrakeSystemDiag brakeDiag;
    // EN: Same runFullDiagnostic() algorithm, different per-subsystem steps
    // TR: Aynı runFullDiagnostic() algoritması, alt sisteme göre farklı adımlar
    fuelDiag.runFullDiagnostic();
    std::cout << std::endl;
    brakeDiag.runFullDiagnostic();
    std::cout << std::endl;

```

```
    // ─── Summary ──────────────────────────────────────────────────────────────────────────────
    std::cout << "╔═══════════════════════════════════════════════════════════════╗\n";
    std::cout << "║  SUMMARY / ÖZET                                                ║\n";
    std::cout << "╟═══════════════════════════════════════════════════════════════╢\n";
    std::cout << "║  1. CRTP: static polymorphism — zero vtable overhead           ║\n";
    std::cout << "║  2. Mixin: inject reusable behavior via CRTP inheritance       ║\n";
    std::cout << "║  3. InstanceCounter: per-type tracking with CRTP + static      ║\n";
    std::cout << "║  4. Benchmark: CRTP enables inlining, virtual does not         ║\n";
    std::cout << "║  5. Template method: CRTP enforces interface at compile        ║\n";
    std::cout << "╚═══════════════════════════════════════════════════════════════╝\n";

    return 0;
}
```

► 08. Template Metaprogramming

module_09_advanced_mechanics/08_template_metaprogramming.cpp

Konu: Template Metaprogramming Deep Dive — Şablon Metaprogramlama Derinlemesine

Teori / Theory

THEORY: Template Metaprogramming (TMP)

Template metaprogramming = computation at COMPILE TIME using templates. The compiler IS the interpreter. Templates ARE the programming language.
Key concepts:
1. VARIADIC TEMPLATES — Templates with variable number of args (T...)
2. FOLD EXPRESSIONS (C++17) — Reduce parameter packs with operators
3. INDEX SEQUENCES — std::index_sequence for compile-time indexing
4. RECURSIVE EXPANSION — Base case + pack expansion pattern
5. constexpr if (C++17) — Compile-time branching

```
┌──────────────────────┬──────────────────────────────────────┐
│ Runtime Programming  │ Template Metaprogramming             │
├──────────────────────┼──────────────────────────────────────┤
│ if/else              │ if constexpr / SFINAE / concepts     │
│ for loop             │ Recursive template instantiation     │
│ int x = 42;          │ template<int X> struct Val {};       │
│ function()           │ Template specialization              │
│ array[i]             │ std::tuple_element + index_sequence  │
│ sum += x             │ Fold expression (... + args)         │
└──────────────────────┴──────────────────────────────────────┘
```

TEORİ: Şablon Metaprogramlama

Şablon metaprogramlama = şablonlar kullanılarak DERLEME ZAMANINDA hesaplama. Derleyici yorumlayıcıdır. Şablonlar programlama dilidir.
Temel kavramlar:
1. VARIADIC TEMPLATES — Değişken sayıda argümanlı şablonlar (T...)
2. FOLD EXPRESSIONS (C++17) — Parametre paketlerini operatörlerle indirgeme
3. INDEX SEQUENCES — Derleme zamanı indeksleme için std::index_sequence
4. RECURSIVE EXPANSION — Temel durum + paket genişletme deseni
5. constexpr if (C++17) — Derleme zamanı dallanma

```
┌──────────────────────┬──────────────────────────────────────┐
│ Çalışma Zamanı Prog. │ Şablon Metaprogramlama               │
├──────────────────────┼──────────────────────────────────────┤
│ if/else              │ if constexpr / SFINAE / concepts     │
│ for döngüsü          │ Özyinelemeli şablon örneklemesi      │
│ int x = 42;          │ template<int X> struct Val {};       │
│ function()           │ Şablon uzmanlaşması                  │
│ array[i]             │ std::tuple_element + index_sequence  │
│ sum += x             │ Katlama ifadesi (... + args)         │
└──────────────────────┴──────────────────────────────────────┘
```

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <string>
#include <tuple>
#include <array>
#include <type_traits>
#include <utility>
#include <cstdint>

// ═══════════════════════════════════════════════════════════════════════
// PART 1: Variadic Templates — Pack Expansion
// ═══════════════════════════════════════════════════════════════════════

/**
 * EN: Variadic templates accept any number of template arguments.
 *     Parameter pack T... is expanded by the compiler at each instantiation.
 *
 *     template<typename... Args>    // Template parameter pack
 *     void foo(Args... args) {}     // Function parameter pack
 *
 *     Pack expansion:
 *       f(args...)           → f(arg1, arg2, arg3)
 *       f(process(args)...)  → f(process(arg1), process(arg2), ...)
 *       sizeof...(Args)      → Number of types in the pack
 *
 * TR: Variadic şablonlar herhangi sayıda şablon argümanını kabul eder.
 *     Parametre paketi T... derleyici tarafından her örneklemede genişletilir.
 */

// EN: Type-safe printf using variadic templates (recursive unpacking)
// TR: Variadic şablonlar kullanan tip güvenli printf (özyinelemeli açma)

// EN: Base case — no more arguments
// TR: Temel durum — daha fazla argüman yok
void type_safe_print(const char* fmt) {
    while (*fmt) {
        if (*fmt == '{' && *(fmt + 1) == '}') {
            throw std::runtime_error("Too few arguments for format string");
        }
        std::cout << *fmt++;
    }
}
```

```cpp

// EN: Recursive case — process one argument, recurse for rest
// TR: Özyinelemeli durum — bir argümanı işle, geri kalan için tekrarla
template<typename T, typename... Rest>
void type_safe_print(const char* fmt, const T& first, const Rest&... rest) {
    while (*fmt) {
        if (*fmt == '{' && *(fmt + 1) == '}') {
            std::cout << first;
            type_safe_print(fmt + 2, rest...);  // EN: Recurse with remaining args
            return;
        }
        std::cout << *fmt++;
    }
}

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: Fold Expressions (C++17)
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: Fold expressions reduce a parameter pack with a binary operator.
 *     Four forms:
 *
 *     (args op ...)      → Unary right fold: (a1 op (a2 op (a3 op a4)))
 *     (... op args)      → Unary left fold:  (((a1 op a2) op a3) op a4)
 *     (args op ... op init) → Binary right fold
 *     (init op ... op args) → Binary left fold
 *
 *     Supported operators: + - * / % ^ & | ~ = < > << >>
 *                          += -= *= /= %= ^= &= |= <<= >>=
 *                          == != <= >= && || , .* ->*
 *
 * TR: Fold ifadeleri parametre paketini ikili operatörle indirger.
 *     Dört form: tekli sağ/sol, ikili sağ/sol katlama.
 */

// EN: Sum all values (unary left fold)
// TR: Tüm değerleri topla (tekli sol katlama)
template<typename... Args>
auto fold_sum(Args... args) {
    return (... + args);  // ((a1 + a2) + a3) + ...
}

// EN: All true? (unary left fold with &&)
// TR: Hepsi doğru mu? (tekli sol katlama, &&)
template<typename... Args>
bool fold_all(Args... args) {
    return (... && args);
}

// EN: Any true? (unary left fold with ||)
// TR: Herhangi biri doğru mu? (tekli sol katlama, ||)
template<typename... Args>
bool fold_any(Args... args) {
    return (... || args);
}

// EN: Print with separator using comma fold expression
// TR: Virgül katlama ifadesi ile ayırıcılı yazdırma
template<typename... Args>
void print_all(const Args&... args) {
    ((std::cout << args << " "), ...);  // Comma fold: execute for each arg
    std::cout << "\n";
}

// EN: Concatenate strings using fold
// TR: Katlama kullanarak stringleri birleştir
template<typename... Args>
std::string fold_concat(const Args&... args) {
    return (std::string{} + ... + std::string(args));
}
```

```

// EN: Check if ALL types satisfy a condition (compile-time fold)
// TR: TÜM tiplerin bir koşulu sağlayıp sağlamadığını kontrol et
template<typename... Ts>
constexpr bool all_arithmetic = (... && std::is_arithmetic_v<Ts>);

template<typename... Ts>
constexpr bool any_floating = (... || std::is_floating_point_v<Ts>);

// ==========================================================================
// PART 3: std::index_sequence / std::integer_sequence
// ==========================================================================

/**
 * EN: std::index_sequence<0, 1, 2, ...> is a compile-time sequence of indices.
 *     Used to "iterate" at compile time, especially with tuples and arrays.
 *
 *     std::make_index_sequence<N> → std::index_sequence<0, 1, ..., N-1>
 *     std::index_sequence_for<Types...> → index_sequence for parameter pack
 *
 * TR: std::index_sequence<0, 1, 2, ...> derleme zamanında indeks dizisidir.
 *     Özellikle tuple'lar ve dizilerle "iterasyon" yapmak için kullanılır.
 */

// EN: Print all elements of a tuple using index_sequence
// TR: index_sequence kullanarak tuple'ın tüm elemanlarını yazdır
template<typename Tuple, size_t... Is>
void print_tuple_impl(const Tuple& t, std::index_sequence<Is...>) {
    ((std::cout << (Is == 0 ? "" : ", ") << std::get<Is>(t)), ...);
}

template<typename... Ts>
void print_tuple(const std::tuple<Ts...>& t) {
    std::cout << "(";
    print_tuple_impl(t, std::index_sequence_for<Ts...>{});
    std::cout << ")";
}

// EN: Transform each tuple element through a function
// TR: Her tuple elemanını bir fonksiyonla dönüştür
template<typename F, typename Tuple, size_t... Is>
auto transform_tuple_impl(F&& f, const Tuple& t, std::index_sequence<Is...>) {
    return std::make_tuple(f(std::get<Is>(t))...);
}

template<typename F, typename... Ts>
auto transform_tuple(F&& f, const std::tuple<Ts...>& t) {
    return transform_tuple_impl(std::forward<F>(f), t,
                                std::index_sequence_for<Ts...>{});
}

// EN: Compile-time array creation using index_sequence
// TR: index_sequence kullanarak derleme zamanında dizi oluşturma
template<size_t... Is>
constexpr auto make_squares_impl(std::index_sequence<Is...>) {
    return std::array<size_t, sizeof...(Is)>{{(Is * Is)...}};
}

template<size_t N>
constexpr auto make_squares() {
    return make_squares_impl(std::make_index_sequence<N>{});
}

```

```cpp
// ═══════════════════════════════════════════════════════════════════════════
// PART 4: constexpr if (C++17) — Compile-Time Branching
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: if constexpr evaluates at COMPILE TIME. The false branch is discarded
 *     (not even type-checked). This replaces many SFINAE patterns.
 *
 *     Before (SFINAE):
 *       template<typename T, enable_if_t<is_integral_v<T>, int> = 0>
 *       T process(T x) { return x * 2; }
 *
 *     After (if constexpr):
 *       template<typename T>
 *       auto process(T x) {
 *           if constexpr (is_integral_v<T>) return x * 2;
 *           else return x + 0.5;
 *       }
 *
 * TR: if constexpr DERLEME ZAMANINDA değerlendirilir.
 *     Yanlış dal atılır (tip kontrolü bile yapılmaz).
 *     Birçok SFINAE kalıbının yerini alır.
 */

// EN: Type-dependent processing using if constexpr
// TR: if constexpr kullanarak tipe bağımlı işleme
template<typename T>
std::string describe_type(const T& value) {
    if constexpr (std::is_integral_v<T>) {
        return "integer: " + std::to_string(value);
    } else if constexpr (std::is_floating_point_v<T>) {
        return "float: " + std::to_string(value);
    } else if constexpr (std::is_same_v<T, std::string>) {
        return "string: \"" + value + "\"";
    } else {
        return "unknown type";
    }
}

// EN: Compile-time factorial using if constexpr (cleaner than specialization)
// TR: if constexpr kullanan derleme zamanı faktöriyel
template<int N>
constexpr int factorial() {
    if constexpr (N <= 1) return 1;
    else return N * factorial<N - 1>();
}

// ═══════════════════════════════════════════════════════════════════════════
// PART 5: Practical TMP Patterns
// ═══════════════════════════════════════════════════════════════════════════

// EN: Type list — a list of types as a single type
// TR: Tip listesi — tek bir tip olarak tiplerin listesi
template<typename... Ts>
struct TypeList {
    static constexpr size_t size = sizeof...(Ts);
};

// EN: Get Nth type from a TypeList
// TR: TypeList'ten N'inci tipi al
template<size_t N, typename List>
struct TypeAt;

template<size_t N, typename Head, typename... Tail>
struct TypeAt<N, TypeList<Head, Tail...>> : TypeAt<N - 1, TypeList<Tail...>> {};

template<typename Head, typename... Tail>
struct TypeAt<0, TypeList<Head, Tail...>> {
    using type = Head;
};

```

```cpp
template<size_t N, typename List>
using TypeAt_t = typename TypeAt<N, List>::type;

// EN: Check if a type is in a TypeList
// TR: Bir tipin TypeList'te olup olmadığını kontrol et
template<typename T, typename List>
struct Contains;

template<typename T, typename... Ts>
struct Contains<T, TypeList<Ts...>> {
    static constexpr bool value = (... || std::is_same_v<T, Ts>);
};

// EN: Compile-time string hash (FNV-1a)
// TR: Derleme zamanı string hash (FNV-1a)
constexpr uint32_t fnv1a_hash(const char* s, size_t len) {
    uint32_t hash = 2166136261u;
    for (size_t i = 0; i < len; ++i) {
        hash ^= static_cast<uint32_t>(static_cast<unsigned char>(s[i]));
        hash *= 16777619u;
    }
    return hash;
}

constexpr uint32_t operator""_hash(const char* s, size_t len) {
    return fnv1a_hash(s, len);
}

// ═══════════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 09 - Template Metaprogramming\n";
    std::cout << "============================================\n\n";

    // ─── Demo 1: Variadic Templates ──────────────────────────────────
    {
        std::cout << "--- Demo 1: Variadic Templates ---\n";
        std::cout << "    type_safe_print: ";
        type_safe_print("Sensor {}: temp={}C, status={}\n", 42, 25.3, "OK");

        std::cout << "    sizeof...(pack): ";
        auto count_types = []<typename... Ts>(Ts...) { return sizeof...(Ts); };
        std::cout << count_types(1, 2.0, "three", 'f') << " types in pack\n\n";
    }

    // ─── Demo 2: Fold Expressions ────────────────────────────────────
    {
        std::cout << "--- Demo 2: Fold Expressions (C++17) ---\n";
        std::cout << "    fold_sum(1,2,3,4,5):  " << fold_sum(1, 2, 3, 4, 5) << "\n";
        std::cout << "    fold_sum(1.5, 2.5):   " << fold_sum(1.5, 2.5) << "\n";
        std::cout << "    fold_all(T,T,T):      " << std::boolalpha
                  << fold_all(true, true, true) << "\n";
        std::cout << "    fold_all(T,F,T):      " << fold_all(true, false, true) << "\n";
        std::cout << "    fold_any(F,F,T):      " << fold_any(false, false, true) << "\n";
        std::cout << "    print_all: ";
        print_all(42, 3.14, "hello", 'X');
        std::cout << "    fold_concat: " << fold_concat("Hello", " ", "World", "!") << "\n";

        // EN: Compile-time type checks with fold
        // TR: Katlama ile derleme zamanlı tip kontrolleri
        std::cout << "    all_arithmetic<int,double,float>: "
                  << all_arithmetic<int, double, float> << "\n";
        std::cout << "    all_arithmetic<int,string>:       "
                  << all_arithmetic<int, std::string> << "\n";
        std::cout << "    any_floating<int,double>:         "
                  << any_floating<int, double> << "\n\n";
    }
```

```

    // ─── Demo 3: index_sequence ──────────────────────────────────────
    {
        std::cout << "--- Demo 3: std::index_sequence ---\n";

        auto sensor_data = std::make_tuple(42, "Engine", 95.5, true);
        std::cout << "    Tuple: ";
        print_tuple(sensor_data);
        std::cout << "\n";

        // EN: Transform tuple: double all numeric values
        // TR: Tuple dönüştür: tüm sayısal değerleri ikiye katla
        auto doubled = transform_tuple([](auto x) {
            if constexpr (std::is_arithmetic_v<std::decay_t<decltype(x)>>)
                return x * 2;
            else
                return x;  // non-numeric: pass through
        }, std::make_tuple(10, 20.5, 30));
        std::cout << "    Doubled tuple: ";
        print_tuple(doubled);
        std::cout << "\n";

        // EN: Compile-time array of squares
        // TR: Derleme zamanlı kareler dizisi
        constexpr auto squares = make_squares<8>();
        std::cout << "    Squares[0..7]: ";
        for (auto s : squares) std::cout << s << " ";
        std::cout << "\n\n";
    }

    // ─── Demo 4: if constexpr ────────────────────────────────────────
    {
        std::cout << "--- Demo 4: if constexpr ---\n";
        std::cout << "    describe(42):     " << describe_type(42) << "\n";
        std::cout << "    describe(3.14):   " << describe_type(3.14) << "\n";
        std::cout << "    describe(string): " << describe_type(std::string("hello")) << "\n";

        std::cout << "    factorial<5>: " << factorial<5>() << "\n";
        std::cout << "    factorial<10>: " << factorial<10>() << "\n";
        static_assert(factorial<5>() == 120, "5! = 120");
        static_assert(factorial<10>() == 3628800, "10! = 3628800");
        std::cout << "    (static_assert passed at compile time!)\n\n";
    }

    // ─── Demo 5: TypeList & Compile-Time Hash ────────────────────────
    {
        std::cout << "--- Demo 5: TypeList & Compile-Time Hash ---\n";

        using SensorTypes = TypeList<int, float, double, std::string>;
        std::cout << "    SensorTypes::size: " << SensorTypes::size << "\n";

        // EN: TypeAt — get type at index
        // TR: TypeAt — indeksteki tipi al
        static_assert(std::is_same_v<TypeAt_t<0, SensorTypes>, int>);
        static_assert(std::is_same_v<TypeAt_t<2, SensorTypes>, double>);
        std::cout << "    TypeAt<0>: int (verified at compile time)\n";
        std::cout << "    TypeAt<2>: double (verified at compile time)\n";

        // EN: Contains — check if type is in list
        // TR: Contains — tipin listede olup olmadığını kontrol et
        std::cout << "    Contains<float>:  " << std::boolalpha
                  << Contains<float, SensorTypes>::value << "\n";
        std::cout << "    Contains<char>:   "
                  << Contains<char, SensorTypes>::value << "\n";

        // EN: Compile-time hash (usable in switch statements!)
        // TR: Derleme zamanlı hash (switch ifadelerinde kullanılabilir!)
        constexpr auto h1 = "temperature"_hash;
        constexpr auto h2 = "pressure"_hash;
        std::cout << "    \"temperature\" hash: " << h1 << "\n";
        std::cout << "    \"pressure\" hash:    " << h2 << "\n";

        // EN: Can use in switch!
        // TR: switch'te kullanılabilir!
        auto process = [](const char* sensor, size_t len) {
```

```
            switch (fnv1a_hash(sensor, len)) {
                case "temperature"_hash: return "temp handler";
                case "pressure"_hash:    return "press handler";
                default:                 return "unknown";
            }
        };
        std::cout << "    switch(hash(\"temperature\")): "
                  << process("temperature", 11) << "\n\n";
    }

    std::cout << "==========================================\n";
    std::cout << "   End of Template Metaprogramming\n";
    std::cout << "==========================================\n";

    return 0;
}
```

Modül 10: Design Patterns — Tasarım Kalıpları

Tasarım Kalıpları

7 kaynak dosya · Standart: C++20

C++20

► 01. Solid Principles

module_10_design_patterns/01_solid_principles.cpp

Konu: Design Patterns & Architecture: SOLID Principles — Tasarım Kalıpları ve Mimari: SOLID

Teori / Theory

THEORY: What is SOLID?

SOLID is a set of five design principles that make software easier to maintain, extend, and test. Coined by Robert C. Martin ("Uncle Bob"), these principles are the FOUNDATION of professional OOP design. Every senior C++ developer is expected to know and apply them daily.

TEORİ: SOLID Nedir?

SOLID, yazılımı bakımı kolay, genişletilebilir ve test edilebilir kılan beş tasarım prensibidir. Robert C. Martin ("Bob Amca") tarafından formüle edilen bu prensipler profesyonel OOP tasarımının TEMELİDİR. Her kıdemli C++ geliştiricisinin bunları bilmesi ve günlük uygulaması beklenir.

THEORY: S — Single Responsibility Principle (SRP)

A class should have ONE and ONLY ONE reason to change. If a class handles file I/O, business logic, AND formatting — it has THREE reasons to change. Split it into three focused classes. The key question: "What is this class's single job?"

TEORİ: S — Tek Sorumluluk Prensibi

Bir sınıfın değişmek için TEK ve YALNIZCA TEK bir nedeni olmalıdır. Eğer bir sınıf dosya G/Ç, iş mantığı VE biçimlendirme yapıyorsa — üç değişim nedeni vardır. Üç odaklı sınıfa bölün. Anahtar soru: "Bu sınıfın tek görevi nedir?"

THEORY: O — Open/Closed Principle (OCP)

Software entities should be OPEN for extension but CLOSED for modification. Add new behavior by creating NEW classes (inheritance or composition), NOT by editing existing working code. This prevents introducing bugs into stable code while still allowing growth.

TEORİ: O — Açık/Kapalı Prensibi

Yazılım varlıkları genişlemeye AÇIK, değişikliğe KAPALI olmalıdır. Yeni davranış eklemek için mevcut çalışan kodu DEĞİŞTİRMEK yerine YENİ sınıflar (kalıtım veya bileşim) oluşturun. Bu, kararlı kodda hata tanıtmayı önlerken büyümeye izin verir.

THEORY: L — Liskov Substitution Principle (LSP)

Objects of a derived class MUST be substitutable for objects of its base class WITHOUT altering the correctness of the program. If `Square` inherits from `Rectangle` but breaks `setWidth()/setHeight()` behavior, it violates LSP. The derived class must honor the BASE class's contract.

TEORİ: L — Liskov Yerine Geçme Prensibi

Türetilmiş sınıf nesneleri, programın doğruluğunu bozmadan temel sınıf nesnelerinin yerine GEÇEBİLMELİDİR. Eğer `Kare` sınıfı `Dikdörtgen`'den miras alır ama `setWidth()/setHeight()` davranışını bozarsa, LSP'yi ihlal eder. Türetilmiş sınıf, TEMEL sınıfın sözleşmesine uymalıdır.

THEORY: I — Interface Segregation Principle (ISP)

No client should be FORCED to depend on methods it does not use. Instead of one "God Interface" with 20 methods, split into small focused interfaces. A `Printer` shouldn't be forced to implement `scan()` and `fax()` if it only prints. Each interface represents ONE capability.

TEORİ: I — Arayüz Ayrımı Prensibi

Hiçbir istemci, kullanmadığı yöntemlere bağımlı olmaya ZORLANMAMALIDIR. 20 metodlu tek bir "Tanrı Arayüzü" yerine küçük odaklı arayüzlere bölün. Bir `Yazıcı`, yalnızca yazdırıyorsa `tarama()` ve `faks()` uygulamaya zorlanmamalıdır. Her arayüz TEK bir yeteneği temsil eder.

THEORY: D — Dependency Inversion Principle (DIP)

High-level modules should NOT depend on low-level modules. Both should depend on ABSTRACTIONS (interfaces). A `GameEngine` shouldn't know about `OpenGLRenderer` directly — it should depend on an `IRenderer` interface. This makes swapping implementations trivial and enables testing with mocks.

TEORİ: D — Bağımlılık Tersine Çevirme Prensibi

Üst seviye modüller, alt seviye modüllere bağımlı OLMAMALIDIR. İkisi de SOYUTLAMALARA (arayüzler) bağımlı olmalıdır. Bir `OyunMotoru` doğrudan `OpenGLRenderer`'ı bilmemelidir — bir `IRenderer` arayüzüne bağımlı olmalıdır. Bu, uygulama değiştirmeyi kolay ve mock ile test etmeyi mümkün kılar.

CPPREF Depth: Pure Virtual Functions and Abstract Classes

SOLID heavily relies on abstract classes (interfaces). In C++, a class with at least one `= 0` pure virtual function is abstract and CANNOT be instantiated. The derived class MUST override all pure virtuals or it also becomes abstract. A virtual destructor is essential in base classes to avoid undefined behavior when deleting through base pointers. cppreference.com/w/cpp/language/abstract_class

CPPREF Derinlik: Saf Sanal Fonksiyonlar ve Soyut Sınıflar

SOLID, soyut sınıflara (arayüzler) yoğun şekilde dayanır. C++'ta en az bir `= 0` saf sanal fonksiyonu olan sınıf soyuttur ve ÖRNEKLENEMez. Türetilmiş sınıf tüm saf sanal fonksiyonları geçersiz kılMALIDIR aksi takdirde kendisi de soyut olur. Temel sınıflarda, temel işaretçi aracılığıyla silerken tanımsız davranıştan kaçınmak için sanal yıkıcı şarttır. cppreference.com/w/cpp/language/abstract_class

CPPREF Depth: std::unique_ptr and Ownership Transfer

DIP implementations typically use `std::unique_ptr<IInterface>` to hold injected dependencies. `unique_ptr` enforces single ownership — the object is destroyed when the pointer goes out of scope. Transfer ownership with `std::move()`. For shared ownership, use `std::shared_ptr`. Never use raw `new`/`delete` in modern C++ — smart pointers are the RAII standard. cppreference.com/w/cpp/memory/unique_ptr

CPPREF Derinlik: std::unique_ptr ve Sahiplik Devri

DIP uygulamaları tipik olarak enjekte edilen bağımlılıkları tutmak için `std::unique_ptr<IArayüz>` kullanır. `unique_ptr` tek sahipliği zorlar — nesne, işaretçi kapsam dışına çıktığında yok edilir. Sahipliği `std::move()` ile devredin. Paylaşımlı sahiplik için `std::shared_ptr` kullanın. Modern C++'ta asla ham `new`/`delete` kullanmayın — akıllı işaretçiler RAII standardıdır. cppreference.com/w/cpp/memory/unique_ptr

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <vector>

// ================================================================================
// [SRP DEMO: Single Responsibility Principle]
// ================================================================================
// EN: BAD: One class doing everything. GOOD: Split into focused classes.
// TR: KÖTÜ: Her şeyi yapan tek sınıf. İYİ: Odaklı sınıflara böl.

// --- BAD DESIGN (SRP violation) ---
class BadReport {
public:
  std::string generateContent() { return "Q4 Sales: $1,200,000"; }
  void saveToFile(const std::string &content) {
    // EN: Mixes business logic with I/O!
    // TR: İş mantığını G/Ç ile karıştırır!
    (void)content;
    std::cout << "[BAD] Saving to file... (mixed responsibility)" << std::endl;
  }
};

// --- GOOD DESIGN (SRP applied) ---
class ReportGenerator {
public:
  std::string generate() { return "Q4 Sales: $1,200,000"; }
};

class ReportSaver {
public:
  void save(const std::string &content) {
    (void)content;
    std::cout << "[GOOD-SRP] Report saved by dedicated saver." << std::endl;
  }
};
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════════
// [OCP DEMO: Open/Closed Principle]
// ═══════════════════════════════════════════════════════════════════════════════
// EN: Add new shapes WITHOUT modifying existing code.
// TR: Mevcut kodu DEĞİŞTİRMEDEN yeni şekiller ekle.

class Shape {
public:
  virtual ~Shape() = default;
  virtual double area() const = 0;
  virtual std::string name() const = 0;
};

class Circle : public Shape {
  double radius_;

public:
  explicit Circle(double r) : radius_(r) {}
  double area() const override { return 3.14159 * radius_ * radius_; }
  std::string name() const override { return "Circle"; }
};

class Rectangle : public Shape {
  double w_, h_;

public:
  Rectangle(double w, double h) : w_(w), h_(h) {}
  double area() const override { return w_ * h_; }
  std::string name() const override { return "Rectangle"; }
};

// EN: NEW shape added without touching Circle or Rectangle!
// TR: Circle veya Rectangle'a dokunmadan YENİ şekil eklendi!
class Triangle : public Shape {
  double base_, height_;

public:
  Triangle(double b, double h) : base_(b), height_(h) {}
  double area() const override { return 0.5 * base_ * height_; }
  std::string name() const override { return "Triangle"; }
};

// EN: This function works with ANY shape — open for extension.
// TR: Bu fonksiyon HERHANGİ bir şekille çalışır — genişlemeye açık.
void printArea(const Shape &s) {
  std::cout << "[OCP] " << s.name() << " area = " << s.area() << std::endl;
}

// ═══════════════════════════════════════════════════════════════════════════════
// [ISP DEMO: Interface Segregation Principle]
// ═══════════════════════════════════════════════════════════════════════════════
// EN: Split "God interface" into small focused interfaces.
// TR: "Tanrı arayüzü"nü küçük odaklı arayüzlere böl.

class IPrintable {
public:
  virtual ~IPrintable() = default;
  virtual void print() = 0;
};

class IScannable {
public:
  virtual ~IScannable() = default;
  virtual void scan() = 0;
};

// EN: SimplePrinter only implements IPrintable — not forced to implement scan!
// TR: SimplePrinter yalnızca IPrintable uygular — scan uygulamaya zorlanmaz!
class SimplePrinter : public IPrintable {
public:
  void print() override {
    std::cout << "[ISP] SimplePrinter printing..." << std::endl;
  }
};
```

```cpp

class MultiFunctionDevice : public IPrintable, public IScannable {
public:
  void print() override {
    std::cout << "[ISP] MultiFunctionDevice printing..." << std::endl;
  }
  void scan() override {
    std::cout << "[ISP] MultiFunctionDevice scanning..." << std::endl;
  }
};

// ==========================================================================================
// [DIP DEMO: Dependency Inversion Principle]
// ==========================================================================================
// EN: High-level DiagnosticsEngine depends on ICommunicationBus interface, not a concrete
// protocol class.
// TR: Üst seviye DiagnosticsEngine, somut sınıfa değil ICommunicationBus arayüzüne bağımlıdır.

class ICommunicationBus {
public:
  virtual ~ICommunicationBus() = default;
  virtual void sendFrame(const std::string &frameData) = 0;
};

class CANBusProtocol : public ICommunicationBus {
public:
  void sendFrame(const std::string &frameData) override {
    std::cout << "[DIP] CAN Bus transmitting: " << frameData << std::endl;
  }
};

class FlexRayProtocol : public ICommunicationBus {
public:
  void sendFrame(const std::string &frameData) override {
    std::cout << "[DIP] FlexRay transmitting: " << frameData << std::endl;
  }
};

// EN: DiagnosticsEngine doesn't know or care which bus protocol it uses!
// TR: DiagnosticsEngine hangi veri yolu protokolünü kullandığını bilMEZ!
class DiagnosticsEngine {
  std::unique_ptr<ICommunicationBus> bus_;

public:
  explicit DiagnosticsEngine(std::unique_ptr<ICommunicationBus> b)
      : bus_(std::move(b)) {}
  void transmitDTC(const std::string &dtcCode) { bus_->sendFrame(dtcCode); }
};

int main() {
  std::cout << "=== MODULE 10: SOLID PRINCIPLES ===\n" << std::endl;

  // --- SRP Demo ---
  std::cout << "--- SRP: Single Responsibility ---\n" << std::endl;
  ReportGenerator gen;
  ReportSaver saver;
  std::string content = gen.generate();
  saver.save(content);
  std::cout << "Report content: " << content << "\n" << std::endl;

  // --- OCP Demo ---
  std::cout << "--- OCP: Open/Closed ---\n" << std::endl;
  Circle c(5.0);
  Rectangle r(4.0, 6.0);
  Triangle t(3.0, 8.0);
  printArea(c);
  printArea(r);
  printArea(t);
  std::cout << std::endl;

  // --- LSP Demo (via OCP shapes — all substitutable) ---
  std::cout << "--- LSP: Liskov Substitution ---\n" << std::endl;
  std::vector<std::unique_ptr<Shape>> shapes;
  shapes.push_back(std::make_unique<Circle>(3.0));
  shapes.push_back(std::make_unique<Rectangle>(2.0, 5.0));
  shapes.push_back(std::make_unique<Triangle>(4.0, 6.0));
```

```
188     for (const auto &s : shapes) {
189       std::cout << "[LSP] " << s->name() << " -> " << s->area() << std::endl;
190     }
191     std::cout << std::endl;
192
193     // --- ISP Demo ---
194     std::cout << "--- ISP: Interface Segregation ---\n" << std::endl;
195     SimplePrinter sp;
196     MultiFunctionDevice mfd;
197     sp.print();
198     mfd.print();
199     mfd.scan();
200     std::cout << std::endl;
201
202     // --- DIP Demo ---
203     std::cout << "--- DIP: Dependency Inversion ---\n" << std::endl;
204     DiagnosticsEngine engine1(std::make_unique<CANBusProtocol>());
205     DiagnosticsEngine engine2(std::make_unique<FlexRayProtocol>());
206     engine1.transmitDTC("P0300 - Random Misfire Detected");
207     engine2.transmitDTC("C0035 - Left Front Wheel Speed Sensor");
208
209     std::cout << "\n=> All 5 SOLID principles demonstrated!" << std::endl;
210
211     return 0;
212   }
```

► 02. Creational Patterns

module_10_design_patterns/02_creational_patterns.cpp

Konu: Design Patterns: Creational — Singleton, Factory, Builder, Abstract Factory /

Teori / Theory

THEORY: Creational Patterns Overview

Creational Design Patterns control HOW objects are created. Instead of scattering `new` calls throughout your codebase, these patterns centralize and encapsulate object creation. This makes code more flexible, testable, and decoupled from concrete types.

TEORİ: Oluşturucu Kalıplar

Oluşturucu Tasarım Kalıpları nesnelerin NASIL oluşturulduğunu kontrol eder. Kod tabanı boyunca `new` çağrılarını dağıtmak yerine, bu kalıplar nesne oluşturmayı merkezileştirir ve kapsüller. Bu, kodu daha esnek, test edilebilir ve somut tiplerden bağımsız kılar.

THEORY: Singleton — One Instance, Global Access

The Singleton pattern ensures a class has EXACTLY ONE instance and provides a global point of access to it. Common use cases: logging systems, configuration managers, hardware drivers. The modern C++11 approach uses a static local variable inside a method — the compiler guarantees thread-safe initialization ("Magic Static" / Scott Meyers' Singleton).

TEORİ: Singleton — Tek Örnek, Global Erişim

Singleton kalıbı, bir sınıfın TAM OLARAK TEK bir örneğe sahip olmasını ve ona küresel erişim noktası sağlamasını garanti eder. Yaygın kullanım: loglama sistemleri, yapılandırma yöneticileri, donanım sürücüleri. Modern C++11 yaklaşımı, bir metodun içindeki statik yerel değişken kullanır — derleyici thread-güvenli başlatmayı garanti eder ("Sihirli Statik" / Scott Meyers' Singleton).

THEORY: Factory Method — Delegate Creation to Subclasses

The Factory Method defines an interface for creating objects but lets subclasses decide WHICH class to instantiate. The client code calls a factory function that returns a base class pointer — it never knows the concrete type. This follows the Open/Closed Principle: adding a new product requires only a new subclass, no modification of existing code.

TEORİ: Fabrika Metodu — Oluşturmayı Alt Sınıflara Devret

Fabrika Metodu, nesne oluşturmak için bir arayüz tanımlar ama HANGİ sınıfın örnekleneceğine alt sınıfların karar vermesine izin verir. İstemci kodu, temel sınıf işaretçisi döndüren bir fabrika fonksiyonu çağırır — somut tipi asla bilmez. Bu, Açık/Kapalı Prensibine uyar: yeni ürün eklemek yalnızca yeni bir alt sınıf gerektirir, mevcut kodda değişiklik gerekmez.

THEORY: Builder — Step-by-Step Complex Object Construction

The Builder pattern separates the CONSTRUCTION of a complex object from its REPRESENTATION. Instead of a constructor with 15 parameters, you chain `.setX().setY().setZ().build()` calls. Each method returns `*this` for fluent API chaining. The Builder validates the object before returning it.

TEORİ: Builder — Adım Adım Karmaşık Nesne İnşası

Builder kalıbı, karmaşık bir nesnenin İNŞASINI TEMSİLİNDEN ayırır. 15 parametreli yapıcı yerine `.setX().setY().setZ().build()` çağrılarını zincirlersiniz. Her metod akıcı API zincirleme için `*this` döndürür. Builder, nesneyi döndürmeden önce doğrular.

THEORY: Abstract Factory — Family of Related Objects

The Abstract Factory provides an interface for creating FAMILIES of related objects without specifying their concrete classes. Example: a GUI toolkit factory that creates buttons, checkboxes, and menus that all match the same theme (Windows vs. macOS). The client uses the factory interface and never sees concrete implementations.

TEORİ: Soyut Fabrika — İlişkili Nesneler Ailesi

Soyut Fabrika, somut sınıflarını belirtmeden İLİŞKİLİ nesneler AİLELERİ oluşturmak için bir arayüz sağlar. Örnek: hepsi aynı temaya uyan düğmeler, onay kutuları ve menüler oluşturan bir GUI araç takımı fabrikası (Windows vs. macOS). İstemci fabrika arayüzünü kullanır ve somut uygulamaları asla görmez.

CPPREF Depth: Static Local Variable Thread Safety (C++11)

Since C++11, initialization of a function-local static variable is guaranteed to be thread-safe. The compiler inserts a hidden guard variable to prevent double-initialization even if multiple threads call the function simultaneously. This is called "Magic Statics" and is the recommended way to implement Singleton in modern C++. cppreference.com/w/cpp/language/storage_duration#Static_local_variables

CPPREF Derinlik: Statik Yerel Değişken Thread Güvenliği (C++11)

C++11'den beri, fonksiyon-yerel statik değişkeninin başlatılması thread-güvenli olarak GARANTİ edilir. Derleyici, birden fazla thread fonksiyonu aynı anda çağırsa bile çift başlatmayı önlemek için gizli bir koruma değişkeni ekler. Buna "Sihirli Statikler" denir ve modern C++'ta Singleton uygulamanın önerilen yoludur. cppreference.com/w/cpp/language/storage_duration#Static_local_variables

CPPREF Depth: std::unique_ptr as Factory Return Type

Modern factories return `std::unique_ptr<Base>` instead of raw pointers. This enforces RAII: the caller automatically owns the object and it's destroyed when the unique_ptr goes out of scope. No manual `delete` needed. `std::make_unique<T>(args...)` is preferred over `new T(args...)` as it's exception-safe and clearer in intent. cppreference.com/w/cpp/memory/unique_ptr

CPPREF Derinlik: Fabrika Dönüş Tipi Olarak std::unique_ptr

Modern fabrikalar ham işaretçi yerine `std::unique_ptr<Temel>` döndürür. Bu RAII'yi zorlar: çağıran otomatik olarak nesneye sahip olur ve unique_ptr kapsam dışına çıktığında yok edilir. Manuel `delete` gerekmez. `std::make_unique<T>(args...)`, `new T(args...)` yerine tercih edilir çünkü istisna-güvenli ve niyette daha açıktır. cppreference.com/w/cpp/memory/unique_ptr

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <memory>
#include <sstream>
#include <string>
#include <vector>

// ================================================================================
// [1. SINGLETON PATTERN — Thread-Safe "Magic Static"]
// ================================================================================

class Logger {
  Logger() { std::cout << "[Singleton] Logger created (only once)." << std::endl; }
  Logger(const Logger &) = delete;
  Logger &operator=(const Logger &) = delete;

public:
  // EN: C++11 guarantees thread-safe initialization of static locals.
  // TR: C++11, statik yerel değişkenlerin thread-güvenli başlatmasını garanti eder.
  static Logger &getInstance() {
    static Logger instance; // Created exactly ONCE ("Magic Static")
    return instance;
  }

  void log(const std::string &msg) {
    std::cout << "[LOG] " << msg << std::endl;
  }
};

// ================================================================================
// [2. FACTORY METHOD PATTERN]
// ================================================================================

// EN: Abstract ECU diagnostic module — each ECU type runs its own check.
// TR: Soyut ECU teşhis modülü — her ECU tipi kendi kontrolünü çalıştırır.
class DiagnosticModule {
public:
  virtual ~DiagnosticModule() = default;
  virtual void runDiagnostic() const = 0;
  virtual std::string moduleName() const = 0;
};
```

```cpp

class BodyControlModule : public DiagnosticModule {
public:
  void runDiagnostic() const override {
    std::cout << "BCM: Checking door locks, windows, lighting relays..."
              << std::endl;
  }
  std::string moduleName() const override { return "BodyControlModule"; }
};

class PowertrainECU : public DiagnosticModule {
public:
  void runDiagnostic() const override {
    std::cout << "Powertrain: Verifying fuel injection, ignition timing..."
              << std::endl;
  }
  std::string moduleName() const override { return "PowertrainECU"; }
};

class ClimateControlUnit : public DiagnosticModule {
public:
  void runDiagnostic() const override {
    std::cout << "HVAC: Testing compressor, blower motor, cabin temp sensor..."
              << std::endl;
  }
  std::string moduleName() const override { return "ClimateControlUnit"; }
};

// EN: Factory function — client never uses 'new' directly.
// TR: Fabrika fonksiyonu — istemci asla doğrudan 'new' kullanmaz.
std::unique_ptr<DiagnosticModule> createModule(const std::string &type) {
  if (type == "bcm") return std::make_unique<BodyControlModule>();
  if (type == "powertrain") return std::make_unique<PowertrainECU>();
  if (type == "hvac") return std::make_unique<ClimateControlUnit>();
  return nullptr;
}

// ═══════════════════════════════════════════════════════════════════════════
// [3. BUILDER PATTERN — Fluent API for Complex Objects]
// ═══════════════════════════════════════════════════════════════════════════

class HttpRequest {
  std::string method_;
  std::string url_;
  std::string body_;
  int timeout_;

  // EN: Private constructor — only Builder can create.
  // TR: Özel yapıcı — yalnızca Builder oluşturabilir.
  HttpRequest(std::string m, std::string u, std::string b, int t)
      : method_(std::move(m)), url_(std::move(u)), body_(std::move(b)),
        timeout_(t) {}

public:
  class Builder {
    std::string method_ = "GET";
    std::string url_;
    std::string body_;
    int timeout_ = 30;

  public:
    Builder &setMethod(const std::string &m) { method_ = m; return *this; }
    Builder &setUrl(const std::string &u) { url_ = u; return *this; }
    Builder &setBody(const std::string &b) { body_ = b; return *this; }
    Builder &setTimeout(int t) { timeout_ = t; return *this; }

    HttpRequest build() {
      return HttpRequest(method_, url_, body_, timeout_);
    }
  };

  void display() const {
    std::cout << "[Builder] " << method_ << " " << url_;
    if (!body_.empty()) std::cout << " | Body: " << body_;
    std::cout << " | Timeout: " << timeout_ << "s" << std::endl;
  }
};
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════════
// [4. ABSTRACT FACTORY PATTERN — Family of Related Objects]
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Abstract product interfaces
// TR: Soyut ürün arayüzleri
class IButton {
public:
  virtual ~IButton() = default;
  virtual void render() const = 0;
};

class ICheckbox {
public:
  virtual ~ICheckbox() = default;
  virtual void render() const = 0;
};

// --- Windows family ---
class WindowsButton : public IButton {
public:
  void render() const override {
    std::cout << "[Win] Rendering Windows-style button." << std::endl;
  }
};
class WindowsCheckbox : public ICheckbox {
public:
  void render() const override {
    std::cout << "[Win] Rendering Windows-style checkbox." << std::endl;
  }
};

// --- macOS family ---
class MacButton : public IButton {
public:
  void render() const override {
    std::cout << "[Mac] Rendering macOS-style button." << std::endl;
  }
};
class MacCheckbox : public ICheckbox {
public:
  void render() const override {
    std::cout << "[Mac] Rendering macOS-style checkbox." << std::endl;
  }
};

// EN: Abstract Factory interface
// TR: Soyut Fabrika arayüzü
class IGUIFactory {
public:
  virtual ~IGUIFactory() = default;
  virtual std::unique_ptr<IButton> createButton() const = 0;
  virtual std::unique_ptr<ICheckbox> createCheckbox() const = 0;
};

class WindowsFactory : public IGUIFactory {
public:
  std::unique_ptr<IButton> createButton() const override {
    return std::make_unique<WindowsButton>();
  }
  std::unique_ptr<ICheckbox> createCheckbox() const override {
    return std::make_unique<WindowsCheckbox>();
  }
};

class MacFactory : public IGUIFactory {
public:
  std::unique_ptr<IButton> createButton() const override {
    return std::make_unique<MacButton>();
  }
  std::unique_ptr<ICheckbox> createCheckbox() const override {
    return std::make_unique<MacCheckbox>();
  }
};
```

```cpp

// EN: Client code works with ANY factory — no concrete types mentioned.
// TR: İstemci kodu HERHANGİ bir fabrikayla çalışır — somut tip bahsedilmez.
void buildUI(const IGUIFactory &factory) {
  auto btn = factory.createButton();
  auto chk = factory.createCheckbox();
  btn->render();
  chk->render();
}

int main() {
  std::cout << "=== MODULE 10: CREATIONAL DESIGN PATTERNS ===\n" << std::endl;

  // --- 1. Singleton ---
  std::cout << "--- SINGLETON ---\n" << std::endl;
  Logger::getInstance().log("Application started.");
  Logger::getInstance().log("Processing data...");
  Logger::getInstance().log("Shutdown complete.");
  std::cout << std::endl;

  // --- 2. Factory Method ---
  std::cout << "--- FACTORY METHOD ---\n" << std::endl;
  std::vector<std::string> ecuList = {"bcm", "bcm", "powertrain", "hvac"};
  for (const auto &type : ecuList) {
    auto mod = createModule(type);
    if (mod) {
      std::cout << "Loaded: " << mod->moduleName() << " -> ";
      mod->runDiagnostic();
    }
  }
  std::cout << std::endl;

  // --- 3. Builder ---
  std::cout << "--- BUILDER ---\n" << std::endl;
  auto req1 = HttpRequest::Builder()
                  .setMethod("POST")
                  .setUrl("https://api.example.com/users")
                  .setBody(R"({"name":"Ahmet"})")
                  .setTimeout(10)
                  .build();
  auto req2 = HttpRequest::Builder()
                  .setUrl("https://api.example.com/health")
                  .build(); // Uses defaults: GET, timeout=30
  req1.display();
  req2.display();
  std::cout << std::endl;

  // --- 4. Abstract Factory ---
  std::cout << "--- ABSTRACT FACTORY ---\n" << std::endl;
  WindowsFactory wf;
  MacFactory mf;
  std::cout << "Building Windows UI:" << std::endl;
  buildUI(wf);
  std::cout << "Building macOS UI:" << std::endl;
  buildUI(mf);

  std::cout << "\n=> All creational patterns demonstrated!" << std::endl;

  return 0;
}
```

► 03. Behavioral Patterns

module_10_design_patterns/03_behavioral_patterns.cpp

Konu: Design Patterns: Behavioral — Observer, Strategy, Command, State — Davranışsal Kalıplar

Teori / Theory

THEORY: Behavioral Patterns Overview

Behavioral patterns focus on HOW objects communicate and distribute responsibilities. They define the protocols for interaction between objects, making systems more flexible and easier to extend. These patterns are the backbone of event-driven architectures, ECU communication lines, and UI frameworks.

TEORİ: Davranışsal Kalıplar

Davranışsal kalıplar, nesnelerin NASIL iletişim kurduğuna ve sorumlulukları nasıl dağıttığına odaklanır. Nesneler arasındaki etkileşim protokollerini tanımlar, sistemleri daha esnek ve genişletmesi kolay kılar. Bu kalıplar olay-güdümlü mimarilerin, ECU iletişim hatlarının ve HMI çerçevelerinin omurgasıdır.

THEORY: Observer — Publish/Subscribe Event System

The Observer pattern defines a one-to-many dependency: when one object (Subject) changes state, ALL registered observers are notified automatically. The subject doesn't know concrete observer types — it only knows the `IObserver` interface. Used in: ECU event buses, HMI notification systems, CAN signal dispatchers, sensor data pipelines.

TEORİ: Observer — Yayınla/Abone Ol Olay Sistemi

Observer kalıbı bir-çoğa bağımlılık tanımlar: bir nesne (Konu) durum değiştirdiğinde, TÜM kayıtlı gözlemciler otomatik bilgilendirilir. Konu, somut gözlemci tiplerini bilmez — yalnızca `IObserver` arayüzünü bilir. Kullanım: ECU olay veri yolları, HMI bildirim sistemleri, CAN sinyal dağıtıcıları, sensör veri hatları.

THEORY: Strategy — Swappable Algorithms at Runtime

The Strategy pattern encapsulates a family of algorithms behind an interface and makes them interchangeable at runtime. Instead of `if/else` chains selecting behavior, you inject the desired strategy object. The context delegates the algorithm call to the strategy. Adding new algorithms requires ZERO modification of existing code (Open/Closed Principle).

TEORİ: Strategy — Çalışma Zamanında Değiştirilebilir Algoritmalar

Strategy kalıbı, bir algoritma ailesini bir arayüz arkasında kapsüller ve çalışma zamanında değiştirilebilir kılar. Davranış seçen `if/else` zincirleri yerine istenen strateji nesnesi enjekte edilir. Bağlam, algoritma çağrısını stratejiye delege eder. Yeni algoritma eklemek mevcut kodda SIFIR değişiklik gerektirir (Açık/Kapalı Prensibi).

THEORY: Command — Encapsulate Request as Object

The Command pattern turns a request/action into a standalone object with all necessary information. This enables: queuing commands, logging them, implementing undo/redo, and executing them later. Each command implements an `execute()` method. The invoker doesn't know what the command does — it just calls `execute()`.

TEORİ: Command — İsteği Nesne Olarak Kapsülle

Command kalıbı, bir istek/eylemi gerekli tüm bilgilerle bağımsız bir nesneye dönüştürür. Bu şunları mümkün kılar: komutları kuyruğa alma, loglama, geri alma/yineleme uygulamak ve sonra çalıştırma. Her komut bir `execute()` metodu uygular. Çağırıcı komutun ne yaptığını bilmez — sadece `execute()` çağırır.

THEORY: State — Object Behavior Changes with Internal State

The State pattern lets an object change its behavior when its internal state changes. Instead of massive `switch/case` blocks, each state is a separate class with its own behavior. State transitions happen by replacing the current state object. The context object delegates all calls to the current state.

TEORİ: State — Nesne Davranışı İç Durumla Değişir

State kalıbı, bir nesnenin iç durumu değiştiğinde davranışını değiştirmesine olanak tanır. Devasa `switch/case` blokları yerine her durum, kendi davranışına sahip ayrı bir sınıftır. Durum geçişleri mevcut durum nesnesini değiştirerek gerçekleşir. Bağlam nesnesi tüm çağrıları mevcut duruma delege eder.

CPPREF Depth: std::function as Strategy Container

In modern C++, you don't always need a full class hierarchy for Strategy. `std::function<ReturnType(Args...)>` can hold any callable (function pointer, lambda, functor, member function via std::bind). This is lighter than virtual dispatch for simple strategies. However, `std::function` has type-erasure overhead (~40 bytes + potential heap allocation for large callables). cppreference.com/w/cpp/utility/functional/function

CPPREF Derinlik: Strateji Kapsayıcı Olarak std::function

Modern C++'ta Strategy için her zaman tam bir sınıf hiyerarşisine ihtiyacınız yok. `std::function<DönüşTipi(Argümanlar...)>` herhangi bir çağrılabiliryi tutabilir (fonksiyon işaretçisi, lambda, functor, std::bind ile üye fonksiyon). Basit stratejiler için sanal fonksiyon dağıtımından daha hafiftir. Ancak `std::function`'ın tip-silme maliyeti vardır (~40 bayt + büyük çağrılabilirler için potansiyel heap tahsisi). cppreference.com/w/cpp/utility/functional/function

CPPREF Depth: std::variant as Type-Safe State Machine

C++17's `std::variant` can replace virtual-dispatch State pattern with a zero-overhead, value-based approach. Each state is a struct, the variant holds one at a time. `std::visit` with a visitor dispatches to the correct state handler at compile-time. No heap allocation, no virtual table — pure stack-based state machine. cppreference.com/w/cpp/utility/variant

CPPREF Derinlik: Tip-Güvenli Durum Makinesi Olarak std::variant

C++17'nin `std::variant`'ı, sanal-dağıtımlı State kalıbını sıfır maliyetli, değer-tabanlı bir yaklaşımla değiştirebilir. Her durum bir struct, variant aynı anda birini tutar. Bir visitor ile `std::visit`, derleme zamanında doğru durum işleyicisine yönlendirir. Heap tahsisi yok, sanal tablo yok — tamamen yığın-tabanlı durum makinesi. cppreference.com/w/cpp/utility/variant

Kaynak Kod / Source Code

```cpp
#include <functional>
#include <iostream>
#include <memory>
#include <string>
#include <vector>

// ═══════════════════════════════════════════════════════════════════════════
// [1. OBSERVER PATTERN]
// ═══════════════════════════════════════════════════════════════════════════

class IObserver {
public:
  virtual ~IObserver() = default;
  virtual void onNotify(const std::string &event) = 0;
};

class EventBus {
  std::vector<IObserver *> observers_;

public:
  void subscribe(IObserver *obs) { observers_.push_back(obs); }

  void notify(const std::string &event) {
    for (auto *obs : observers_) {
      obs->onNotify(event);
    }
  }
};

class FaultLogger : public IObserver {
public:
  void onNotify(const std::string &event) override {
    if (event == "ENGINE_OVERHEAT") {
      std::cout << "[FaultLog] Logging DTC P0217 — Engine Overheat."
                << std::endl;
    } else if (event == "MAINTENANCE_DUE") {
      std::cout << "[FaultLog] Logging scheduled maintenance reminder."
                << std::endl;
    }
  }
};

class ClusterDisplay : public IObserver {
public:
  void onNotify(const std::string &event) override {
    if (event == "ENGINE_OVERHEAT") {
      std::cout << "[Cluster] Showing RED overheat warning on dashboard!"
                << std::endl;
    } else if (event == "MAINTENANCE_DUE") {
      std::cout << "[Cluster] Showing wrench icon — service needed."
                << std::endl;
    }
  }
};

class TelematicsReporter : public IObserver {
public:
  void onNotify(const std::string &event) override {
    if (event == "ENGINE_OVERHEAT") {
      std::cout << "[Telematics] Uploading overheat snapshot to cloud."
                << std::endl;
    }
  }
};

```

```
// ═══════════════════════════════════════════════════════════════════════════
// [2. STRATEGY PATTERN]
// ═══════════════════════════════════════════════════════════════════════════

class ISortStrategy {
public:
  virtual ~ISortStrategy() = default;
  virtual void sort(std::vector<int> &data) const = 0;
  virtual std::string name() const = 0;
};

class BubbleSort : public ISortStrategy {
public:
  void sort(std::vector<int> &data) const override {
    for (size_t i = 0; i < data.size(); i++) {
      for (size_t j = 0; j + 1 < data.size() - i; j++) {
        if (data[j] > data[j + 1]) std::swap(data[j], data[j + 1]);
      }
    }
  }
  std::string name() const override { return "BubbleSort"; }
};

class SelectionSort : public ISortStrategy {
public:
  void sort(std::vector<int> &data) const override {
    for (size_t i = 0; i < data.size(); i++) {
      size_t minIdx = i;
      for (size_t j = i + 1; j < data.size(); j++) {
        if (data[j] < data[minIdx]) minIdx = j;
      }
      std::swap(data[i], data[minIdx]);
    }
  }
  std::string name() const override { return "SelectionSort"; }
};

class InsertionSort : public ISortStrategy {
public:
  void sort(std::vector<int> &data) const override {
    for (size_t i = 1; i < data.size(); i++) {
      int key = data[i];
      size_t j = i;
      while (j > 0 && data[j - 1] > key) {
        data[j] = data[j - 1];
        j--;
      }
      data[j] = key;
    }
  }
  std::string name() const override { return "InsertionSort"; }
};

// EN: Context class — delegates sorting to injected strategy.
// TR: Bağlam sınıfı — sıralamayı enjekte edilen stratejiye delege eder.
class Sorter {
  std::unique_ptr<ISortStrategy> strategy_;

public:
  void setStrategy(std::unique_ptr<ISortStrategy> s) {
    strategy_ = std::move(s);
  }
  void doSort(std::vector<int> &data) {
    if (strategy_) {
      std::cout << "[Strategy] Using " << strategy_->name() << ": ";
      strategy_->sort(data);
      for (int v : data) std::cout << v << " ";
      std::cout << std::endl;
    }
  }
};

```

```
// ═══════════════════════════════════════════════════════════════════════════
// [3. COMMAND PATTERN — Undo/Redo Support]
// ═══════════════════════════════════════════════════════════════════════════

class ICommand {
public:
  virtual ~ICommand() = default;
  virtual void execute() = 0;
  virtual void undo() = 0;
  virtual std::string description() const = 0;
};

class TextEditor {
  std::string text_;

public:
  void insert(const std::string &s) { text_ += s; }
  void removeLast(size_t n) {
    if (n <= text_.size()) text_.erase(text_.size() - n, n);
  }
  const std::string &getText() const { return text_; }
};

class InsertTextCommand : public ICommand {
  TextEditor &editor_;
  std::string text_;

public:
  InsertTextCommand(TextEditor &e, std::string t)
      : editor_(e), text_(std::move(t)) {}
  void execute() override { editor_.insert(text_); }
  void undo() override { editor_.removeLast(text_.size()); }
  std::string description() const override {
    return "Insert '" + text_ + "'";
  }
};

class CommandHistory {
  std::vector<std::unique_ptr<ICommand>> history_;

public:
  void executeCommand(std::unique_ptr<ICommand> cmd) {
    cmd->execute();
    std::cout << "[Command] Executed: " << cmd->description() << std::endl;
    history_.push_back(std::move(cmd));
  }
  void undoLast() {
    if (!history_.empty()) {
      auto &cmd = history_.back();
      cmd->undo();
      std::cout << "[Command] Undone: " << cmd->description() << std::endl;
      history_.pop_back();
    }
  }
};

// ═══════════════════════════════════════════════════════════════════════════
// [4. STATE PATTERN — Traffic Light State Machine]
// ═══════════════════════════════════════════════════════════════════════════

class TrafficLight; // Forward declaration

class ITrafficState {
public:
  virtual ~ITrafficState() = default;
  virtual void handle(TrafficLight &context) = 0;
  virtual std::string color() const = 0;
};

class TrafficLight {
  std::unique_ptr<ITrafficState> state_;

public:
  explicit TrafficLight(std::unique_ptr<ITrafficState> initial)
      : state_(std::move(initial)) {}

  void setState(std::unique_ptr<ITrafficState> s) { state_ = std::move(s); }
```

```

  void next() {
    std::cout << "[State] Current: " << state_->color() << " -> ";
    state_->handle(*this);
  }
};

class GreenState : public ITrafficState {
public:
  void handle(TrafficLight &context) override;
  std::string color() const override { return "GREEN"; }
};

class YellowState : public ITrafficState {
public:
  void handle(TrafficLight &context) override;
  std::string color() const override { return "YELLOW"; }
};

class RedState : public ITrafficState {
public:
  void handle(TrafficLight &context) override {
    std::cout << "switching to GREEN" << std::endl;
    context.setState(std::make_unique<GreenState>());
  }
  std::string color() const override { return "RED"; }
};

void GreenState::handle(TrafficLight &context) {
  std::cout << "switching to YELLOW" << std::endl;
  context.setState(std::make_unique<YellowState>());
}

void YellowState::handle(TrafficLight &context) {
  std::cout << "switching to RED" << std::endl;
  context.setState(std::make_unique<RedState>());
}

int main() {
  std::cout << "=== MODULE 10: BEHAVIORAL DESIGN PATTERNS ===\n" << std::endl;

  // --- 1. Observer ---
  std::cout << "--- OBSERVER (Vehicle Event Bus) ---\n" << std::endl;
  EventBus bus;
  FaultLogger faultLog;
  ClusterDisplay cluster;
  TelematicsReporter telematics;
  bus.subscribe(&faultLog);
  bus.subscribe(&cluster);
  bus.subscribe(&telematics);
  bus.notify("ENGINE_OVERHEAT");
  std::cout << std::endl;
  bus.notify("MAINTENANCE_DUE");
  std::cout << std::endl;

  // --- 2. Strategy ---
  std::cout << "--- STRATEGY (Sorting) ---\n" << std::endl;
  Sorter sorter;
  std::vector<int> data1 = {5, 2, 8, 1, 9};
  std::vector<int> data2 = {7, 3, 6, 4, 0};
  std::vector<int> data3 = {12, 11, 15, 13, 14};

  sorter.setStrategy(std::make_unique<BubbleSort>());
  sorter.doSort(data1);
  sorter.setStrategy(std::make_unique<SelectionSort>());
  sorter.doSort(data2);
  sorter.setStrategy(std::make_unique<InsertionSort>());
  sorter.doSort(data3);
  std::cout << std::endl;

```

```
    // --- 3. Command (Undo/Redo) ---
    std::cout << "--- COMMAND (Text Editor with Undo) ---\n" << std::endl;
    TextEditor editor;
    CommandHistory history;

    history.executeCommand(
        std::make_unique<InsertTextCommand>(editor, "Hello"));
    history.executeCommand(
        std::make_unique<InsertTextCommand>(editor, " World"));
    history.executeCommand(
        std::make_unique<InsertTextCommand>(editor, "!"));
    std::cout << "Text: \"" << editor.getText() << "\"" << std::endl;

    history.undoLast(); // Removes "!"
    std::cout << "After undo: \"" << editor.getText() << "\"" << std::endl;
    history.undoLast(); // Removes " World"
    std::cout << "After undo: \"" << editor.getText() << "\"\n" << std::endl;

    // --- 4. State (Traffic Light) ---
    std::cout << "--- STATE (Traffic Light) ---\n" << std::endl;
    TrafficLight light(std::make_unique<GreenState>());
    for (int i = 0; i < 6; i++) {
      light.next(); // GREEN->YELLOW->RED->GREEN->YELLOW->RED
    }

    std::cout << "\n=> All behavioral patterns demonstrated!" << std::endl;

    return 0;
  }
```

► 04. Structural Patterns

module_10_design_patterns/04_structural_patterns.cpp

Konu: Design Patterns: Structural — Adapter, Decorator, Facade, Composite, Proxy — Yapısal

Teori / Theory

THEORY: Structural Patterns Overview

Structural patterns deal with HOW classes and objects are COMPOSED to form larger structures. They simplify complex relationships, adapt incompatible interfaces, and add new behavior without altering existing code. Think of them as "architectural connectors" that glue components together.

TEORİ: Yapısal Kalıplar

Yapısal kalıplar, sınıfların ve nesnelerin daha büyük yapılar oluşturmak için NASIL BİRLEŞTİRİLDİĞİ ile ilgilenir. Karmaşık ilişkileri basitleştirir, uyumsuz arayüzleri uyarlar ve mevcut kodu değiştirmeden yeni davranış ekler. Bileşenleri birbirine yapıştıran "mimari bağlayıcılar" olarak düşünün.

THEORY: Adapter — Make Incompatible Interfaces Work Together

The Adapter pattern wraps an existing class with an incompatible interface so it matches the interface the client expects. Like a power adapter that converts EU plugs to US outlets. The adapted class doesn't need ANY modification — the adapter handles the translation.

TEORİ: Adapter — Uyumsuz Arayüzleri Birlikte Çalıştır

Adapter kalıbı, uyumsuz bir arayüze sahip mevcut sınıfı, istemcinin beklediği arayüze uyacak şekilde sarar. AB fişlerini ABD prizlerine dönüştüren güç adaptörü gibi. Uyarlanan sınıf HİÇBİR değişiklik gerektirmez — adaptör çeviriyi halleder.

THEORY: Decorator — Add Behavior Dynamically

The Decorator pattern wraps an object to ADD new behavior without modifying the original class. Decorators share the same interface as the wrapped object, so they can be stacked (chained). Example: adding encryption, compression, and logging to a data stream — each decorator wraps the previous one.

TEORİ: Decorator — Davranışı Dinamik Olarak Ekle

Decorator kalıbı, orijinal sınıfı değiştirmeden yeni davranış EKLEMEK için bir nesneyi sarar. Dekoratörler sarılan nesneyle aynı arayüzü paylaşır, böylece üst üste yığılabilir (zincirlenir). Örnek: bir veri akışına şifreleme, sıkıştırma ve loglama ekleme — her dekoratör öncekini sarar.

THEORY: Facade — Simplified Interface to Complex System

The Facade pattern provides a SIMPLIFIED interface to a complex subsystem consisting of many classes. Instead of clients managing 10 different objects, they talk to ONE facade that orchestrates everything behind the scenes. Common in automotive middleware, media frameworks, and library wrappers.

TEORİ: Facade — Karmaşık Sisteme Basitleştirilmiş Arayüz

Facade kalıbı, birçok sınıftan oluşan karmaşık bir alt sisteme BASİTLEŞTİRİLMİŞ bir arayüz sağlar. İstemciler 10 farklı nesneyi yönetmek yerine, perde arkasında her şeyi orkestra eden TEK bir facade ile konuşurlar. Oyun motorlarında, medya çerçevelerinde ve kütüphane sarmalayıcılarda yaygındır.

THEORY: Composite — Treat Individual and Group Objects Uniformly

The Composite pattern lets you compose objects into tree structures and treat individual objects and compositions UNIFORMLY through the same interface. A `File` and a `Folder` (containing files and subfolders) both implement `getSize()`. The client doesn't care if it's a leaf or a branch.

TEORİ: Composite — Tekil ve Grup Nesneleri Aynı Şekilde Ele Al

Composite kalıbı, nesneleri ağaç yapılarına birleştirmenize ve tekil nesnelerle kompozisyonları aynı arayüz aracılığıyla AYNI ŞEKİLDE ele almanıza olanak tanır. Bir `Dosya` ve bir `Klasör` (dosya ve alt klasörler içeren) her ikisi de `getSize()` uygular. İstemci yaprak mı dal mı olduğunu umursamaz.

CPPREF Depth: Multiple Inheritance for Mixin-Style Decorator Chains

C++ supports multiple inheritance, which enables mixin-style decorators where behavior is composed by inheriting from multiple small classes. Each mixin adds ONE capability. Virtual inheritance resolves the diamond problem when mixins share a common base. In practice, prefer composition-based decorators (wrapping via constructor) over inheritance-based ones for better flexibility. cppreference.com/w/cpp/language/derived_class

CPPREF Derinlik: Mixin-Tarzı Dekoratör Zincirleri için Çoklu Kalıtım

C++ çoklu kalıtımı destekler, bu da davranışın birden fazla küçük sınıftan kalıtımla birleştirildiği mixin-tarzı dekoratörleri mümkün kılar. Her mixin TEK bir yetenek ekler. Sanal kalıtım, mixin'ler ortak bir temeli paylaştığında elmas problemini çözer. Pratikte, daha iyi esneklik için kalıtım-tabanlı yerine bileşim-tabanlı dekoratörleri (yapıcı ile sarma) tercih edin. cppreference.com/w/cpp/language/derived_class

CPPREF Depth: std::shared_ptr for Shared Composite Ownership

In Composite pattern implementations, child nodes can be shared among multiple parents (e.g., symbolic links, DAG structures). Use `std::shared_ptr` for shared ownership where multiple owners keep the object alive. Use `std::weak_ptr` to break circular references. For strict tree hierarchies (single parent), `std::unique_ptr` is preferred for zero overhead. cppreference.com/w/cpp/memory/shared_ptr

CPPREF Derinlik: Paylaşımlı Composite Sahipliği için std::shared_ptr

Composite kalıp uygulamalarında, çocuk düğümler birden fazla ebeveyn arasında paylaşılabilir (örn. sembolik bağlantılar, DAG yapıları). Birden fazla sahibin nesneyi canlı tuttuğu paylaşımlı sahiplik için `std::shared_ptr` kullanın. Dairesel referansları kırmak için `std::weak_ptr` kullanın. Katı ağaç hiyerarşileri (tek ebeveyn) için sıfır maliyetli `std::unique_ptr` tercih edilir. cppreference.com/w/cpp/memory/shared_ptr

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <memory>
#include <numeric>
#include <string>
#include <vector>

// ==========================================================================
// [1. ADAPTER PATTERN]
// ==========================================================================

// EN: Legacy library we CANNOT modify (third-party).
// TR: Değiştiremeyeceğimiz eski kütüphane (üçüncü taraf).
class LegacyXMLParser {
public:
  std::string parseXML(const std::string &xml) {
    return "[XML Parsed] " + xml;
  }
};

// EN: Interface our system expects.
// TR: Sistemimizin beklediği arayüz.
class IDataParser {
public:
  virtual ~IDataParser() = default;
  virtual std::string parse(const std::string &data) = 0;
};

// EN: Adapter wraps the legacy class to match our interface.
// TR: Adaptör, eski sınıfı bizim arayüzümüze uyacak şekilde sarar.
class XMLParserAdapter : public IDataParser {
  LegacyXMLParser legacy_;

public:
  std::string parse(const std::string &data) override {
    return legacy_.parseXML(data);
  }
};

class JSONParser : public IDataParser {
public:
  std::string parse(const std::string &data) override {
    return "[JSON Parsed] " + data;
  }
};

```

```cpp
// ═══════════════════════════════════════════════════════════════════════════════
// [2. DECORATOR PATTERN — Stacking Behaviors]
// ═══════════════════════════════════════════════════════════════════════════════

class IMessage {
public:
  virtual ~IMessage() = default;
  virtual std::string getContent() const = 0;
};

class PlainMessage : public IMessage {
  std::string text_;

public:
  explicit PlainMessage(std::string t) : text_(std::move(t)) {}
  std::string getContent() const override { return text_; }
};

// EN: Base decorator — wraps an IMessage.
// TR: Temel dekoratör — bir IMessage sarar.
class MessageDecorator : public IMessage {
protected:
  std::unique_ptr<IMessage> wrapped_;

public:
  explicit MessageDecorator(std::unique_ptr<IMessage> m)
      : wrapped_(std::move(m)) {}
};

class EncryptedMessage : public MessageDecorator {
public:
  using MessageDecorator::MessageDecorator;
  std::string getContent() const override {
    return "[ENCRYPTED]{" + wrapped_->getContent() + "}";
  }
};

class CompressedMessage : public MessageDecorator {
public:
  using MessageDecorator::MessageDecorator;
  std::string getContent() const override {
    return "[COMPRESSED]{" + wrapped_->getContent() + "}";
  }
};

class LoggedMessage : public MessageDecorator {
public:
  using MessageDecorator::MessageDecorator;
  std::string getContent() const override {
    std::cout << "[LOG] Message accessed." << std::endl;
    return wrapped_->getContent();
  }
};

// ═══════════════════════════════════════════════════════════════════════════════
// [3. FACADE PATTERN — Simplified Interface]
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Complex subsystem classes (Instrument Cluster subsystems).
// TR: Karmaşık alt sistem sınıfları (Gösterge paneli alt sistemleri).
class AudioSubsystem {
public:
  void playChime(const std::string &chime) {
    std::cout << "  [Audio] Playing chime: " << chime << std::endl;
  }
};

class SensorFusionEngine {
public:
  void calibrateSensors(int sensorCount) {
    std::cout << "  [SensorFusion] Calibrating " << sensorCount
              << " sensors" << std::endl;
  }
};
```

```cpp

class DisplayRenderer {
public:
  void setResolution(int w, int h) {
    std::cout << "  [Display] Cluster resolution: " << w << "x" << h
              << std::endl;
  }
};

class CANGateway {
public:
  void connectBus(const std::string &busName) {
    std::cout << "  [CAN] Connected to " << busName << " bus" << std::endl;
  }
};

// EN: Facade — ONE method to initialize the entire instrument cluster.
// TR: Facade — Tüm gösterge panelini başlatmak için TEK metod.
class InstrumentClusterFacade {
  AudioSubsystem audio_;
  SensorFusionEngine sensors_;
  DisplayRenderer display_;
  CANGateway can_;

public:
  void startCluster(const std::string &vehicleId) {
    std::cout << "[Facade] Initializing instrument cluster for "
              << vehicleId << "..." << std::endl;
    display_.setResolution(1920, 720);
    sensors_.calibrateSensors(12);
    audio_.playChime("startup_chime.wav");
    can_.connectBus("Vehicle CAN-HS");
    std::cout << "[Facade] Cluster ready!" << std::endl;
  }
};

// ═══════════════════════════════════════════════════════════════════════════
// [4. COMPOSITE PATTERN — File System Tree]
// ═══════════════════════════════════════════════════════════════════════════

class IFileComponent {
public:
  virtual ~IFileComponent() = default;
  virtual int getSize() const = 0;
  virtual void display(int depth = 0) const = 0;
  virtual std::string getName() const = 0;
};

class File : public IFileComponent {
  std::string name_;
  int size_;

public:
  File(std::string n, int s) : name_(std::move(n)), size_(s) {}
  int getSize() const override { return size_; }
  std::string getName() const override { return name_; }
  void display(int depth) const override {
    for (int i = 0; i < depth; i++) std::cout << "  ";
    std::cout << "□ " << name_ << " (" << size_ << " KB)" << std::endl;
  }
};

class Folder : public IFileComponent {
  std::string name_;
  std::vector<std::unique_ptr<IFileComponent>> children_;

public:
  explicit Folder(std::string n) : name_(std::move(n)) {}

  void add(std::unique_ptr<IFileComponent> child) {
    children_.push_back(std::move(child));
  }

```

```
193     int getSize() const override {
194       return std::accumulate(children_.begin(), children_.end(), 0,
195                              [](int sum, const auto &c) { return sum + c->getSize(); });
196     }
197
198     std::string getName() const override { return name_; }
199
200     void display(int depth) const override {
201       for (int i = 0; i < depth; i++) std::cout << "  ";
202       std::cout << "□ " << name_ << " (" << getSize() << " KB total)" << std::endl;
203       for (const auto &child : children_) {
204         child->display(depth + 1);
205       }
206     }
207   };
208
209   int main() {
210     std::cout << "=== MODULE 10: STRUCTURAL DESIGN PATTERNS ===\n" << std::endl;
211
212     // --- 1. Adapter ---
213     std::cout << "--- ADAPTER ---\n" << std::endl;
214     XMLParserAdapter xmlAdapter;
215     JSONParser jsonParser;
216     std::cout << xmlAdapter.parse("<user>Ahmet</user>") << std::endl;
217     std::cout << jsonParser.parse(R"({"name":"Ahmet"})") << "\n" << std::endl;
218
219     // --- 2. Decorator (Stacking) ---
220     std::cout << "--- DECORATOR (Stacking) ---\n" << std::endl;
221     // EN: Stack: Plain -> Compressed -> Encrypted -> Logged
222     // TR: Yığın: Düz -> Sıkıştırılmış -> Şifrelenmiş -> Loglanmış
223     auto msg = std::make_unique<PlainMessage>("Hello, World!");
224     auto compressed = std::make_unique<CompressedMessage>(std::move(msg));
225     auto encrypted = std::make_unique<EncryptedMessage>(std::move(compressed));
226     auto logged = std::make_unique<LoggedMessage>(std::move(encrypted));
227     std::cout << "Result: " << logged->getContent() << "\n" << std::endl;
228
229     // --- 3. Facade ---
230     std::cout << "--- FACADE ---\n" << std::endl;
231     InstrumentClusterFacade cluster;
232     cluster.startCluster("VIN_WBAPH5C55BA271234");
233     std::cout << std::endl;
234
235     // --- 4. Composite (File System) ---
236     std::cout << "--- COMPOSITE (File System Tree) ---\n" << std::endl;
237     auto root = std::make_unique<Folder>("project");
238     auto src = std::make_unique<Folder>("src");
239     src->add(std::make_unique<File>("main.cpp", 12));
240     src->add(std::make_unique<File>("utils.cpp", 8));
241     auto docs = std::make_unique<Folder>("docs");
242     docs->add(std::make_unique<File>("README.md", 3));
243     root->add(std::move(src));
244     root->add(std::move(docs));
245     root->add(std::make_unique<File>("Makefile", 1));
246     root->display(0);
247
248     std::cout << "\n=> All structural patterns demonstrated!" << std::endl;
249
250     return 0;
251   }
```

► 05. Testable Design

module_10_design_patterns/05_testable_design.cpp

Konu: Design Patterns: Testable Design — CRTP, Type Erasure, Policy-Based Design — Test

Teori / Theory

THEORY: CRTP — Curiously Recurring Template Pattern

CRTP is a technique where a class inherits from a template that takes ITSELF as the template parameter: `class Derived : public Base<Derived>`. This enables "static polymorphism" — the base class calls methods on Derived at COMPILE TIME without any virtual function overhead (no vtable, no indirection). Used for: mixin injection, compile-time interfaces, expression templates, and the "static interface" pattern.

TEORİ: CRTP — Merakla Tekrarlanan Şablon Kalıbı

CRTP, bir sınıfın KENDİSİNİ şablon parametresi olarak alan bir şablondan miras aldığı bir tekniktir: `class Derived : public Base<Derived>`. Bu "statik polimorfizm" sağlar — temel sınıf Derived üzerindeki metodları DERLEME ZAMANINDA hiçbir sanal fonksiyon maliyeti olmadan çağırır (vtable yok, dolaylı erişim yok). Kullanım: mixin enjeksiyonu, derleme zamanı arayüzleri, ifade şablonları ve "statik arayüz" kalıbı.

THEORY: Type Erasure — Polymorphism Without Inheritance

Type Erasure hides concrete types behind a uniform interface WITHOUT requiring inheritance from a common base class. It combines templates, a polymorphic wrapper class, and internal virtual dispatch. The standard library uses this in `std::function`, `std::any`, and `std::format`. The pattern: store any type-erased object, access it through a fixed interface. "Duck typing in C++".

TEORİ: Type Erasure — Kalıtımsız Polimorfizm

Type Erasure, ortak temel sınıftan kalıtım gerektirmeden somut tipleri tekdüze bir arayüz arkasına gizler. Şablonları, polimorfik bir sarmalayıcı sınıf ve dahili sanal dağıtımı birleştirir. Standart kütüphane bunu `std::function`, `std::any` ve `std::format`'ta kullanır. Kalıp: tip silinmiş herhangi bir nesneyi depola, sabit bir arayüz aracılığıyla eriş. "C++'ta Duck Typing".

THEORY: Policy-Based Design — Compile-Time Strategy Pattern

Policy-Based Design uses template parameters as "policies" that define behavior at COMPILE TIME. Instead of runtime virtual dispatch (Strategy pattern), the behavior is baked into the type itself. Zero runtime overhead! Popularized by Andrei Alexandrescu's "Modern C++ Design". You can mix and match policies freely: `Widget<ThreadSafe, JSONLogging, PoolAllocator>`.

TEORİ: Policy-Based Design — Derleme Zamanı Strateji Kalıbı

Policy-Based Design, davranışı DERLEME ZAMANINDA tanımlayan "politikalar" olarak şablon parametreleri kullanır. Çalışma zamanı sanal dağıtım (Strategy kalıbı) yerine davranış tipin kendisine gömülür. Sıfır çalışma zamanı maliyeti! Andrei Alexandrescu'nun "Modern C++ Design" ile popülerleşti. Politikaları serbestçe karıştırabilirsiniz: `Widget<ThreadSafe, JSONLogging, PoolAllocator>`.

THEORY: Compile-Time Polymorphism vs Runtime Polymorphism

Runtime polymorphism (virtual functions) selects behavior at RUNTIME via vtable lookup (~1 cache miss per call). Compile-time polymorphism (CRTP, templates, concepts) selects behavior at COMPILE TIME with zero overhead. Trade-off: compile-time = faster execution, longer compile times, larger binary; runtime = slower execution, faster compilation, smaller binary. Embedded & HPC prefer compile-time; plugin architectures need runtime.

TEORİ: Derleme Zamanı vs Çalışma Zamanı Polimorfizmi

Çalışma zamanı polimorfizmi (sanal fonksiyonlar) vtable araması ile ÇALIŞMA ZAMANINDA davranış seçer (~çağrı başına 1 önbellek kaçırma). Derleme zamanı polimorfizmi (CRTP, şablonlar, concepts) DERLEME ZAMANINDA sıfır maliyetle davranış seçer. Takas: derleme zamanı = daha hızlı çalışma, daha uzun derleme, daha büyük binary; çalışma zamanı = daha yavaş çalışma, daha hızlı derleme, daha küçük binary. Gömülü & HPC derleme zamanını tercih eder; eklenti mimarileri çalışma zamanına ihtiyaç duyar.

CPPREF Depth: static_cast<Derived*>(this) in CRTP — The Core Mechanism

In CRTP, the base class accesses derived methods via `static_cast<Derived*>(this)->method()`. This is safe because the base class KNOWS (via the template parameter) that `this` is actually a `Derived` object. The cast is resolved at compile time with zero cost. However, calling a derived method that doesn't exist causes a compile error — functioning as a static interface checker. cppreference.com/w/cpp/language/static_cast

CPPREF Derinlik: CRTP'de static_cast<Derived*>(this) — Temel Mekanizma

CRTP'de temel sınıf, türetilmiş metodlara `static_cast<Derived*>(this)->method()` ile erişir. Bu güvenlidir çünkü temel sınıf (şablon parametresi aracılığıyla) `this`'in aslında bir `Derived` nesnesi olduğunu BİLİR. Dönüşüm derleme zamanında sıfır maliyetle çözülür. Ancak var olmayan bir türetilmiş metodu çağırmak derleme hatası verir — statik arayüz denetleyicisi olarak işlev görür. cppreference.com/w/cpp/language/static_cast

CPPREF Depth: std::any, std::function — Standard Library Type Erasure

`std::any` (C++17) can hold ANY copyable type and retrieve it with `std::any_cast<T>`. It stores small objects inline (SBO - Small Buffer Optimization, typically 16-32 bytes) and allocates larger ones on the heap. `std::function` uses the same technique for callable objects. Both incur type-erasure overhead but provide maximum flexibility. cppreference.com/w/cpp/utility/any

CPPREF Derinlik: std::any, std::function — Standart Kütüphane Type Erasure

`std::any` (C++17) herhangi bir kopyalanabilir tipi tutabilir ve `std::any_cast<T>` ile alınır. Küçük nesneleri satır içi depolar (SBO - Küçük Tampon Optimizasyonu, tipik 16-32 bayt) ve büyükleri heap'te tahsis eder. `std::function` çağrılabilir nesneler için aynı tekniği kullanır. İkisi de tip-silme maliyetine sahiptir ama maksimum esneklik sağlar. cppreference.com/w/cpp/utility/any

Kaynak Kod / Source Code

```cpp
#include <cmath>
#include <iostream>
#include <memory>
#include <string>
#include <vector>

// ═══════════════════════════════════════════════════════════════════════════
// [1. CRTP — Curiously Recurring Template Pattern]
```

```
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Base class uses static_cast to call Derived methods at compile time.
// TR: Temel sınıf, Derived metodlarını derleme zamanında çağırmak için static_cast kullanır.
template <typename Derived> class Shape {
public:
  double area() const {
    // EN: Compile-time dispatch — no vtable!
    // TR: Derleme zamanı dağıtım — vtable yok!
    return static_cast<const Derived *>(this)->areaImpl();
  }

  std::string name() const {
    return static_cast<const Derived *>(this)->nameImpl();
  }

  void print() const {
    std::cout << "[CRTP] " << name() << " area = " << area() << std::endl;
  }
};

class CRTPCircle : public Shape<CRTPCircle> {
  double radius_;

public:
  explicit CRTPCircle(double r) : radius_(r) {}
  double areaImpl() const { return 3.14159 * radius_ * radius_; }
  std::string nameImpl() const { return "Circle"; }
};

class CRTPSquare : public Shape<CRTPSquare> {
  double side_;

public:
  explicit CRTPSquare(double s) : side_(s) {}
  double areaImpl() const { return side_ * side_; }
  std::string nameImpl() const { return "Square"; }
};

// EN: Static polymorphism — works with any Shape<T> at compile time.
// TR: Statik polimorfizm — derleme zamanında herhangi bir Shape<T> ile çalışır.
template <typename T> void printShape(const Shape<T> &shape) {
  shape.print();
}

// ═══════════════════════════════════════════════════════════════════════════════
// [2. TYPE ERASURE — Polymorphism Without Inheritance]
// ═══════════════════════════════════════════════════════════════════════════════
// EN: Any object that has a `draw()` method can be stored in our Drawable container — WITHOUT
// inheriting from any base class! "Duck typing".
// TR: `draw()` metodu olan herhangi bir nesne Drawable kapsayıcımızda depolanabilir — herhangi
// bir temel sınıftan miras almadan! "Duck typing".

class Drawable {
  struct Concept {
    virtual ~Concept() = default;
    virtual void draw() const = 0;
    virtual std::unique_ptr<Concept> clone() const = 0;
  };

  template <typename T> struct Model : Concept {
    T obj_;
    explicit Model(T o) : obj_(std::move(o)) {}
    void draw() const override { obj_.draw(); }
    std::unique_ptr<Concept> clone() const override {
      return std::make_unique<Model>(obj_);
    }
  };

  std::unique_ptr<Concept> pimpl_;

public:
  template <typename T>
  Drawable(T x) : pimpl_(std::make_unique<Model<T>>(std::move(x))) {}

  Drawable(const Drawable &other) : pimpl_(other.pimpl_->clone()) {}
  Drawable &operator=(const Drawable &other) {
```

```
    pimpl_ = other.pimpl_->clone();
    return *this;
  }
  Drawable(Drawable &&) = default;
  Drawable &operator=(Drawable &&) = default;

  void draw() const { pimpl_->draw(); }
};

// EN: These classes DON'T inherit from anything — but they have draw()!
// TR: Bu sınıflar hiçbir şeyden miras ALMAZ — ama draw() var!
struct CANMessage {
  void draw() const {
    std::cout << "[TypeErasure] Rendering CAN frame on bus monitor."
              << std::endl;
  }
};

struct LINFrame {
  std::string label;
  void draw() const {
    std::cout << "[TypeErasure] Rendering LIN frame: " << label << std::endl;
  }
};

struct DiagnosticReport {
  std::string text;
  void draw() const {
    std::cout << "[TypeErasure] Rendering DTC report: " << text << std::endl;
  }
};

// ═══════════════════════════════════════════════════════════════════════════
// [3. POLICY-BASED DESIGN — Compile-Time Strategy]
// ═══════════════════════════════════════════════════════════════════════════

// --- Logging Policies ---
struct ConsoleLog {
  static void log(const std::string &msg) {
    std::cout << "[ConsoleLog] " << msg << std::endl;
  }
};

struct SilentLog {
  static void log(const std::string &) {} // No output
};

// --- Validation Policies ---
struct StrictValidation {
  static bool validate(int value) { return value > 0 && value < 1000; }
  static std::string policyName() { return "Strict"; }
};

struct RelaxedValidation {
  static bool validate(int) { return true; } // Accept everything
  static std::string policyName() { return "Relaxed"; }
};

// EN: Widget class configured by policies at COMPILE TIME.
// TR: Widget sınıfı DERLEME ZAMANINDA politikalarla yapılandırılır.
template <typename LogPolicy, typename ValidationPolicy> class Widget {
  int value_ = 0;

public:
  void setValue(int v) {
    if (ValidationPolicy::validate(v)) {
      value_ = v;
      LogPolicy::log("Value set to " + std::to_string(v) +
                     " [" + ValidationPolicy::policyName() + "]");
    } else {
      LogPolicy::log("Validation FAILED for " + std::to_string(v) +
                     " [" + ValidationPolicy::policyName() + "]");
    }
  }
  int getValue() const { return value_; }
};
```

```

// ═══════════════════════════════════════════════════════════════════════════════
// [4. COMPILE-TIME INTERFACE CHECKING WITH CRTP]
// ═══════════════════════════════════════════════════════════════════════════════
// EN: CRTP can enforce that Derived implements required methods — compile error if missing. This
// acts as a "static interface" / concept.
// TR: CRTP, Derived'ın gerekli metodları uygulamasını zorunlu kılabilir — eksikse derleme
// hatası. "Statik arayüz" / concept olarak çalışır.

template <typename Derived> class Serializable {
public:
  std::string serialize() const {
    return static_cast<const Derived *>(this)->serializeImpl();
  }
};

class Config : public Serializable<Config> {
  std::string key_;
  std::string value_;

public:
  Config(std::string k, std::string v)
      : key_(std::move(k)), value_(std::move(v)) {}

  // EN: Required by Serializable CRTP — compile error if missing!
  // TR: Serializable CRTP tarafından gerekli — eksikse derleme hatası!
  std::string serializeImpl() const {
    return key_ + "=" + value_;
  }
};

int main() {
  std::cout << "=== MODULE 10: TESTABLE & ADVANCED DESIGN PATTERNS ===\n"
            << std::endl;

  // --- 1. CRTP (Static Polymorphism) ---
  std::cout << "--- CRTP (Static Polymorphism) ---\n" << std::endl;
  CRTPCircle circle(5.0);
  CRTPSquare square(4.0);
  printShape(circle);
  printShape(square);
  std::cout << std::endl;

  // --- 2. Type Erasure ---
  std::cout << "--- TYPE ERASURE (Duck Typing) ---\n" << std::endl;
  // EN: Store completely unrelated types in the same container!
  // TR: Tamamen ilişkisiz tipleri aynı kapsayıcıda depola!
  std::vector<Drawable> canvas;
  canvas.emplace_back(CANMessage{});
  canvas.emplace_back(LINFrame{"Door_Status"});
  canvas.emplace_back(DiagnosticReport{"P0300 — Random Misfire"});

  for (const auto &drawable : canvas) {
    drawable.draw();
  }
  std::cout << std::endl;

  // --- 3. Policy-Based Design ---
  std::cout << "--- POLICY-BASED DESIGN ---\n" << std::endl;

  // EN: Strict validation + Console logging
  // TR: Sıkı doğrulama + Konsol günlükleme
  Widget<ConsoleLog, StrictValidation> strictWidget;
  strictWidget.setValue(42);     // OK
  strictWidget.setValue(-1);     // FAILS validation
  strictWidget.setValue(99999);  // FAILS validation

  std::cout << std::endl;

```

```
    // EN: Relaxed validation + Silent logging (zero overhead!)
    // TR: Esnek doğrulama + Sessiz günlükleme (sıfır ek yük!)
    Widget<SilentLog, RelaxedValidation> relaxedWidget;
    relaxedWidget.setValue(-999); // Accepted silently
    std::cout << "Relaxed widget value: " << relaxedWidget.getValue() << std::endl;

    std::cout << std::endl;

    // --- 4. CRTP as Static Interface (Serializable) ---
    std::cout << "--- CRTP STATIC INTERFACE ---\n" << std::endl;
    Config cfg("database_host", "localhost:5432");
    std::cout << "Serialized config: " << cfg.serialize() << std::endl;

    std::cout << "\n=> CRTP, Type Erasure, and Policy-Based Design demonstrated!"
              << std::endl;

    return 0;
}
```

► 06. Pattern Decision Guide

module_10_design_patterns/06_pattern_decision_guide.cpp

Konu: Pattern Decision Guide — When to Use, When NOT to Use, Comparisons, Missing Patterns /

Teori / Theory

THEORY: Why a Decision Guide?

Knowing HOW to implement a pattern is only half the battle. The harder skill is knowing WHEN to use it, WHEN NOT to use it, and which pattern to choose among similar alternatives. This file provides:
1. Anti-pattern warnings.
2. Head-to-head comparisons between confusing pairs.
3. Before/after refactoring examples.
4. Two missing patterns: Proxy and Prototype.

TEORİ: Neden Bir Karar Rehberi?

Bir kalıbı NASIL uygulayacağınızı bilmek savaşın sadece yarısıdır. Daha zor beceri, NE ZAMAN kullanacağınızı, NE ZAMAN KULLANMAYACAĞINIZI ve benzer alternatifler arasında hangisini seçeceğinizi bilmektir. Bu dosya şunları sunar:
1. Anti-pattern uyarıları.
2. Kafa karıştıran çiftler arası karşılaştırmalar.
3. Öncesi/sonrası refactoring örnekleri.
4. Eksik iki kalıp: Proxy ve Prototype.

THEORY: Singleton — Anti-Pattern Warnings

Singleton is the MOST overused and MOST criticized design pattern. Before using it, ask:
✓ USE when: there's a physical constraint (one GPU, one log file, one config file)
✗ DON'T USE when: you just want "global access" — that's a global variable in disguise
✗ DON'T USE when: you need testability — Singleton makes mocking nearly impossible
✗ DON'T USE when: constructor has side effects (DB connections, file I/O)
⚠ WARNING: Singleton hides dependencies. Functions that use Logger::getInstance() have an invisible dependency that doesn't appear in their parameter list, violating Dependency Inversion Principle (DIP). Prefer injecting the logger as a reference parameter.

TEORİ: Singleton — Anti-Pattern Uyarıları

Singleton en ÇOK aşırı kullanılan ve en ÇOK eleştirilen tasarım kalıbıdır. Kullanmadan önce sorun:
✓ KULLAN: fiziksel kısıt varsa (tek GPU, tek log dosyası, tek config dosyası)
✗ KULLANMA: sadece "global erişim" istiyorsan — bu kılık değiştirmiş global değişkendir
✗ KULLANMA: test edilebilirlik gerekiyorsa — Singleton mock'lamayı neredeyse imkansız kılar
✗ KULLANMA: constructor yan etkilere sahipse (DB bağlantısı, dosya I/O)
⚠ UYARI: Singleton bağımlılıkları gizler. Logger::getInstance() kullanan fonksiyonların parametre listesinde görünmeyen gizli bağımlılığı vardır — DIP ihlali.

THEORY: Observer — Memory Leak & Lifetime Risks

Observer pattern has a dangerous trap: DANGLING OBSERVERS. If an observer is destroyed without unsubscribing from the subject, the subject holds a dangling pointer. Next notify() call → undefined behavior (crash, corruption). Solutions:
• Use weak_ptr instead of raw pointers in the observer list
• RAII-based subscription: destructor automatically unsubscribes
• Use std::function + token-based unsubscribe (return an ID on subscribe)
⚠ WARNING: The simple EventBus in 03_behavioral_patterns.cpp uses raw pointers — this is intentionally minimal for teaching. Production code MUST handle observer lifetime.

TEORİ: Observer — Bellek Sızıntısı ve Ömür Riskleri

Observer kalıbının tehlikeli tuzağı: SARKAN GÖZLEMCİLER. Bir gözlemci konudan aboneliğini iptal etmeden yok edilirse, konu sarkan işaretçi tutar. Sonraki notify() çağrısı → tanımsız davranış (çökme, bozulma). Çözümler:
• Gözlemci listesinde ham işaretçi yerine weak_ptr kullan
• RAII tabanlı abonelik: yıkıcı otomatik aboneliği iptal eder
• std::function + token tabanlı abonelik iptali (subscribe'da ID döndür)

THEORY: Decorator — Complexity Cost

Each Decorator layer adds a virtual function call and a unique_ptr indirection. Stacking 5+ decorators creates a deep chain that's hard to debug (stack traces become unreadable).
✓ USE when: you need 2-4 combinable behaviors at runtime
✗ DON'T USE when: all combinations are known at compile time (use templates/CRTP instead)
✗ DON'T USE when: you need to inspect the inner object (decorators obscure the wrapped type) Alternative: Policy-Based Design (see 05_testable_design.cpp) provides compile-time decoration with ZERO runtime overhead.

TEORİ: Decorator — Karmaşıklık Maliyeti

Her Decorator katmanı sanal fonksiyon çağrısı ve unique_ptr yönlendirmesi ekler. 5+ dekoratör yığmak, debug'ı zor derin zincir oluşturur (stack trace'ler okunamaz hale gelir).
✓ KULLAN: çalışma zamanında 2-4 birleştirilebilir davranış gerektiğinde
✗ KULLANMA: tüm kombinasyonlar derleme zamanında biliniyorsa (CRTP/template kullan)
✗ KULLANMA: iç nesneyi incelemeniz gerekiyorsa (dekoratörler sarılı tipi gizler)

THEORY: Factory Method vs Abstract Factory — When Which?

These two are the most confused pair: FACTORY METHOD: Creates ONE product. Uses inheritance — subclass overrides the factory method. The factory IS the base class itself.
→ Use when: you have a single product hierarchy and creation logic varies by subclass. ABSTRACT FACTORY: Creates a FAMILY of related products. Uses composition — the factory is a separate object injected into the client.
→ Use when: you need multiple related objects that must be from the same "theme" (e.g., WindowsButton + WindowsCheckbox, never WindowsButton + MacCheckbox). RULE OF THUMB: "If creating ONE thing → Factory Method. If creating a SET → Abstract Factory."

TEORİ: Fabrika Metodu vs Soyut Fabrika — Hangisi Ne Zaman?

Bu ikisi en çok karıştırılan çifttir: FABRİKA METODU: TEK ürün oluşturur. Kalıtım kullanır — alt sınıf fabrika metodunu override eder. Fabrika temel sınıfın KENDİSİDİR.
→ Kullan: tek ürün hiyerarşiniz varsa ve oluşturma mantığı alt sınıfa göre değişiyorsa. SOYUT FABRİKA: İLİŞKİLİ ürünler AİLESİ oluşturur. Bileşim kullanır — fabrika istemciye enjekte edilen ayrı bir nesnedir.
→ Kullan: aynı "tema"dan olması gereken birden fazla ilişkili nesne gerektiğinde. KURAL: "TEK şey oluşturuyorsan → Fabrika Metodu. SET oluşturuyorsan → Soyut Fabrika."

THEORY: Strategy vs State — The Subtle Difference

Both patterns look identical in structure (context + interface + concrete classes). The difference is INTENT: STRATEGY: The CLIENT chooses which algorithm to use. The object doesn't change state — the behavior is externally selected. Example: sorting algorithm selection. STATE: The OBJECT ITSELF changes its behavior based on internal transitions. The client just calls `next()` — the state decides the next state. Example: traffic light cycling. KEY TEST: "Does the object decide its own transitions?" → State. "Does the caller pick the behavior?" → Strategy.

TEORİ: Strategy vs State — İnce Fark

Her iki kalıp yapısal olarak aynı görünür (bağlam + arayüz + somut sınıflar). Fark NİYETTEDİR: STRATEGY: İSTEMCİ hangi algoritmayı kullanacağını seçer. Nesne durum değiştirmez — davranış dışarıdan seçilir. Örnek: sıralama algoritması seçimi. STATE: NESNENİN KENDİSİ iç geçişlere göre davranışını değiştirir. İstemci sadece `next()` çağırır — durum bir sonraki durumu belirler. Örnek: trafik ışığı döngüsü. TEST: "Nesne kendi geçişlerine mi karar veriyor?" → State. "Çağıran mı davranışı seçiyor?" → Strategy.

THEORY: Adapter vs Facade — Wrapping for Different Reasons

Both "wrap" something, but the purpose differs: ADAPTER: Makes ONE incompatible interface match ANOTHER existing interface. The client already has an interface it expects — the adapter translates.
→ "I have a square peg and a round hole — I need an adapter." FACADE: Simplifies a COMPLEX SUBSYSTEM into one easy-to-use interface. There's no pre-existing interface to match — you're creating a new simplified API.
→ "I have 10 subsystems — I want ONE startCluster() call." ADAPTER = interface translation. FACADE = interface simplification.

TEORİ: Adapter vs Facade — Farklı Amaçlarla Sarma

İkisi de "sarar" ama amaç farklıdır: ADAPTER: TEK uyumsuz arayüzü mevcut BAŞKA bir arayüzle eşleştirir. İstemci zaten beklediği bir arayüze sahiptir — adaptör çeviri yapar. FACADE: KARMAŞIK ALT SİSTEMİ kullanımı kolay tek bir arayüze basitleştirir. Eşleştirilecek önceden var arayüz yoktur — yeni basitleştirilmiş API oluştururuz. ADAPTER = arayüz çevirisi. FACADE = arayüz basitleştirmesi.

CPPREF Depth: std::unique_ptr and Prototype clone()

The Prototype pattern uses a `clone()` method that returns a deep copy. In modern C++, the return type should be `std::unique_ptr<Base>` to enforce clear ownership. The base class declares `virtual std::unique_ptr<Base> clone() const = 0;` and each derived class returns `std::make_unique<Derived>(*this)`. This relies on the copy constructor — ensure Rule of Five compliance if the class manages resources. Covariant return types don't work with smart pointers, so the return type must be the BASE type. cppreference.com/w/cpp/memory/unique_ptr

CPPREF Derinlik: std::unique_ptr ve Prototype clone()

Prototype kalıbı derin kopya döndüren bir `clone()` metodu kullanır. Modern C++'ta dönüş tipi net sahiplik için `std::unique_ptr<Temel>` olmalıdır. Temel sınıf `virtual std::unique_ptr<Temel> clone() const = 0;` bildirir ve her türetilmiş sınıf `std::make_unique<Türetilmiş>(*this)` döndürür. Bu kopyalama yapıcısına dayanır — sınıf kaynak yönetiyorsa Beşin Kuralı uyumunu sağlayın. Kovaryant dönüş tipleri akıllı işaretçilerle çalışmaz, bu nedenle dönüş tipi TEMEL tip olmalıdır.

CPPREF Depth: Proxy vs Smart Pointer — Overlapping Responsibility

C++'s smart pointers (unique_ptr, shared_ptr) are themselves a form of Proxy pattern! They intercept access to the raw pointer and add behavior (reference counting, automatic deletion). When implementing a custom Proxy, consider whether a smart pointer with a custom deleter already solves the problem. Custom Proxy is needed when you want: access control, lazy initialization, logging, caching, or remote communication (RPC/gRPC stub). cppreference.com/w/cpp/memory/shared_ptr

CPPREF Derinlik: Proxy vs Akıllı İşaretçi — Örtüşen Sorumluluk

C++'ın akıllı işaretçileri (unique_ptr, shared_ptr) kendileri bir Proxy kalıbı biçimidir! Ham işaretçiye erişimi yakalar ve davranış ekler (referans sayımı, otomatik silme). Özel Proxy uygularken, özel silici ile akıllı işaretçinin sorunu zaten çözüp çözmediğini düşünün. Özel Proxy şunlar için gereklidir: erişim kontrolü, tembel başlatma, loglama, önbellekleme veya uzak iletişim (RPC/gRPC stub).

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <vector>

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 1: PROTOTYPE PATTERN — Deep Cloning
// Eksik kalıp: Prototype (clone ile derin kopyalama)
// ═══════════════════════════════════════════════════════════════════════════

// EN: Abstract base — all prototypes must implement clone().
// TR: Soyut temel — tüm prototipler clone() uygulamalıdır.
class VehicleConfig {
public:
  virtual ~VehicleConfig() = default;
  virtual std::unique_ptr<VehicleConfig> clone() const = 0;
  virtual void display() const = 0;
  virtual std::string type() const = 0;
};

// EN: Concrete prototype — full deep copy via copy constructor.
// TR: Somut prototip — kopyalama yapıcısı ile tam derin kopya.
class ElectricVehicleConfig : public VehicleConfig {
  std::string model_;
  int batteryCapacity_; // kWh
  bool hasAutopilot_;

```

```cpp
public:
  ElectricVehicleConfig(std::string model, int battery, bool autopilot)
      : model_(std::move(model)), batteryCapacity_(battery),
        hasAutopilot_(autopilot) {}

  // EN: clone() returns unique_ptr<Base> — covariant returns don't work with smart ptrs.
  // TR: clone() unique_ptr<Temel> döndürür — kovaryant dönüş akıllı işaretçilerle çalışmaz.
  std::unique_ptr<VehicleConfig> clone() const override {
    return std::make_unique<ElectricVehicleConfig>(*this); // Uses copy ctor
  }

  void display() const override {
    std::cout << "  EV Config: " << model_ << " | " << batteryCapacity_
              << " kWh | Autopilot: " << (hasAutopilot_ ? "Yes" : "No")
              << std::endl;
  }

  std::string type() const override { return "ElectricVehicle"; }
};

class HybridVehicleConfig : public VehicleConfig {
  std::string model_;
  int engineCC_;
  int batteryCapacity_;

public:
  HybridVehicleConfig(std::string model, int cc, int battery)
      : model_(std::move(model)), engineCC_(cc), batteryCapacity_(battery) {}

  std::unique_ptr<VehicleConfig> clone() const override {
    return std::make_unique<HybridVehicleConfig>(*this);
  }

  void display() const override {
    std::cout << "  Hybrid Config: " << model_ << " | " << engineCC_
              << "cc + " << batteryCapacity_ << " kWh" << std::endl;
  }

  std::string type() const override { return "HybridVehicle"; }
};

// EN: Prototype Registry — store templates, clone on demand.
// TR: Prototip Kaydı — şablonları depola, talep üzerine klonla.
class VehicleConfigRegistry {
  std::vector<std::pair<std::string, std::unique_ptr<VehicleConfig>>>
      prototypes_;

public:
  void registerPrototype(const std::string &name,
                         std::unique_ptr<VehicleConfig> proto) {
    prototypes_.emplace_back(name, std::move(proto));
  }

  std::unique_ptr<VehicleConfig> create(const std::string &name) const {
    for (const auto &[key, proto] : prototypes_) {
      if (key == name) return proto->clone();
    }
    return nullptr;
  }
};

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 2: PROXY PATTERN — Lazy Loading & Access Control
// Eksik kalıp: Proxy (tembel yükleme ve erişim kontrolü)
// ═══════════════════════════════════════════════════════════════════════════

// EN: Subject interface — both real object and proxy implement this.
// TR: Konu arayüzü — hem gerçek nesne hem proxy bunu uygular.
class IDatabase {
public:
  virtual ~IDatabase() = default;
  virtual std::string query(const std::string &sql) = 0;
  virtual bool isConnected() const = 0;
};
```

```cpp

// EN: Real (heavy) object — expensive to create (simulates DB connection).
// TR: Gerçek (ağır) nesne — oluşturması pahalı (DB bağlantısını simüle eder).
class RealDatabase : public IDatabase {
  std::string connectionString_;
  bool connected_ = false;

public:
  explicit RealDatabase(std::string connStr)
      : connectionString_(std::move(connStr)) {
    // EN: Simulates expensive connection setup.
    // TR: Pahalı bağlantı kurulumunu simüle eder.
    std::cout << "  [RealDB] Connecting to " << connectionString_
              << " (heavy operation)..." << std::endl;
    connected_ = true;
  }

  std::string query(const std::string &sql) override {
    return "[RealDB] Result for: " + sql;
  }

  bool isConnected() const override { return connected_; }
};

// EN: Proxy — Lazy initialization: creates RealDatabase only on first query().
//     Also adds access control: denies queries if user role is insufficient.
// TR: Proxy — Tembel başlatma: RealDatabase'i yalnızca ilk query()'de oluşturur.
//     Ayrıca erişim kontrolü ekler: kullanıcı rolü yetersizse sorguları reddeder.
class DatabaseProxy : public IDatabase {
  std::string connectionString_;
  std::string userRole_;
  mutable std::unique_ptr<RealDatabase> realDb_; // Lazy

public:
  DatabaseProxy(std::string connStr, std::string role)
      : connectionString_(std::move(connStr)), userRole_(std::move(role)) {
    std::cout << "  [Proxy] Created (no DB connection yet — lazy)." << std::endl;
  }

  std::string query(const std::string &sql) override {
    // EN: Access control — only "admin" and "analyst" can query.
    // TR: Erişim kontrolü — yalnızca "admin" ve "analyst" sorgulayabilir.
    if (userRole_ != "admin" && userRole_ != "analyst") {
      return "[Proxy] ACCESS DENIED for role: " + userRole_;
    }

    // EN: Lazy initialization — create real DB only when needed.
    // TR: Tembel başlatma — gerçek DB yalnızca gerektiğinde oluşturulur.
    if (!realDb_) {
      std::cout << "  [Proxy] First query — initializing real database..."
                << std::endl;
      realDb_ = std::make_unique<RealDatabase>(connectionString_);
    }

    // EN: Logging (Proxy can also add logging, caching, metrics...).
    // TR: Loglama (Proxy loglama, önbellekleme, metrik de ekleyebilir...).
    std::cout << "  [Proxy] Forwarding query: " << sql << std::endl;
    return realDb_->query(sql);
  }

  bool isConnected() const override {
    return realDb_ && realDb_->isConnected();
  }
};

```

```cpp
// ═══════════════════════════════════════════════════════════════════════════
// SECTION 3: BEFORE/AFTER — Strategy Pattern Refactoring
// Öncesi/Sonrası — Strategy kalıbı ile refactoring
// ═══════════════════════════════════════════════════════════════════════════

// EN: BEFORE (Bad) — behavior selected by if/else chain.
//     Adding a new discount type requires modifying this function (violates OCP).
// TR: ÖNCESİ (Kötü) — davranış if/else zinciriyle seçilir.
//     Yeni indirim tipi eklemek bu fonksiyonu değiştirmeyi gerektirir (OCP ihlali).
namespace before {
double applyDiscount(double price, const std::string &type) {
  if (type == "student") return price * 0.80;
  if (type == "military") return price * 0.75;
  if (type == "employee") return price * 0.70;
  // EN: Every new discount type → modify this function. Fragile!
  // TR: Her yeni indirim tipi → bu fonksiyonu değiştir. Kırılgan!
  return price;
}
} // namespace before

// EN: AFTER (Good) — Strategy pattern, open for extension, closed for modification.
// TR: SONRASI (İyi) — Strategy kalıbı, genişlemeye açık, değişikliğe kapalı.
namespace after {
class IDiscountStrategy {
public:
  virtual ~IDiscountStrategy() = default;
  virtual double apply(double price) const = 0;
  virtual std::string name() const = 0;
};

class StudentDiscount : public IDiscountStrategy {
public:
  double apply(double price) const override { return price * 0.80; }
  std::string name() const override { return "Student (20% off)"; }
};

class MilitaryDiscount : public IDiscountStrategy {
public:
  double apply(double price) const override { return price * 0.75; }
  std::string name() const override { return "Military (25% off)"; }
};

class EmployeeDiscount : public IDiscountStrategy {
public:
  double apply(double price) const override { return price * 0.70; }
  std::string name() const override { return "Employee (30% off)"; }
};

// EN: Adding VIPDiscount requires ZERO modification of existing code — just a new class.
// TR: VIPDiscount eklemek mevcut kodda SIFIR değişiklik gerektirir — sadece yeni sınıf.
class VIPDiscount : public IDiscountStrategy {
public:
  double apply(double price) const override { return price * 0.60; }
  std::string name() const override { return "VIP (40% off)"; }
};

double applyDiscount(double price, const IDiscountStrategy &strategy) {
  return strategy.apply(price);
}
} // namespace after

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 4: BEFORE/AFTER — Observer with Safe Lifetime (weak_ptr)
// Öncesi/Sonrası — Güvenli ömürlü Observer (weak_ptr)
// ═══════════════════════════════════════════════════════════════════════════

// EN: Safe Observer using weak_ptr — expired observers are automatically skipped.
// TR: weak_ptr kullanan güvenli Observer — süresi dolan gözlemciler otomatik atlanır.
class SafeObserver : public std::enable_shared_from_this<SafeObserver> {
public:
  virtual ~SafeObserver() = default;
  virtual void onEvent(const std::string &event) = 0;
};
```

```cpp

class SafeEventBus {
  std::vector<std::weak_ptr<SafeObserver>> observers_;

public:
  void subscribe(std::shared_ptr<SafeObserver> obs) {
    observers_.push_back(obs); // Stores weak_ptr (won't prevent destruction)
  }

  void notify(const std::string &event) {
    // EN: Iterate and lock — skip expired observers, no dangling pointers.
    // TR: İtera et ve kilitle — süresi dolan gözlemcileri atla, sarkan işaretçi yok.
    auto it = observers_.begin();
    while (it != observers_.end()) {
      if (auto locked = it->lock()) {
        locked->onEvent(event);
        ++it;
      } else {
        // EN: Observer was destroyed — remove from list.
        // TR: Gözlemci yok edildi — listeden kaldır.
        it = observers_.erase(it);
      }
    }
  }

  size_t subscriberCount() const { return observers_.size(); }
};

class DashboardWidget : public SafeObserver {
  std::string name_;

public:
  explicit DashboardWidget(std::string n) : name_(std::move(n)) {}
  void onEvent(const std::string &event) override {
    std::cout << "  [" << name_ << "] Received: " << event << std::endl;
  }
  ~DashboardWidget() override {
    std::cout << "  [" << name_ << "] Destroyed (auto-unsubscribed)."
              << std::endl;
  }
};

```

```
// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 5: QUICK REFERENCE — Pattern Selection Cheat Sheet
// Hızlı Referans — Kalıp Seçim Tablosu
// ═══════════════════════════════════════════════════════════════════════════════

//   ┌──────────────────────┬──────────────────────────────────────────────────┐
//   │ Problem              │ Pattern to Use                                   │
//   ├──────────────────────┼──────────────────────────────────────────────────┤
//   │ One global instance  │ Singleton (prefer DI / Dependency Injection first!) │
//   │ Hide concrete type   │ Factory Method                                   │
//   │ Family of objects    │ Abstract Factory                                 │
//   │ Complex constructor  │ Builder                                          │
//   │ Clone existing obj   │ Prototype                                        │
//   │ Wrap legacy API      │ Adapter                                          │
//   │ Simplify subsystem   │ Facade                                           │
//   │ Stack behaviors      │ Decorator                                        │
//   │ Tree structure       │ Composite                                        │
//   │ Lazy/access control  │ Proxy                                            │
//   │ Event notification   │ Observer                                         │
//   │ Swappable algorithm  │ Strategy                                         │
//   │ Undo/redo actions    │ Command                                          │
//   │ State machine        │ State                                            │
//   ├──────────────────────┼──────────────────────────────────────────────────┤
//   │ Compile-time strat.  │ Policy-Based Design / CRTP (see 05_testable)     │
//   │ Type erasure         │ std::any / hand-rolled (see 05_testable)         │
//   └──────────────────────┴──────────────────────────────────────────────────┘

// ═══════════════════════════════════════════════════════════════════════════════
// MAIN — Demonstrations
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
  std::cout << "=== MODULE 10: PATTERN DECISION GUIDE ===\n" << std::endl;

  // ── 1. Prototype ──
  std::cout << "--- PROTOTYPE (Clone & Registry) ---\n" << std::endl;

  VehicleConfigRegistry registry;
  registry.registerPrototype(
      "ev_standard",
      std::make_unique<ElectricVehicleConfig>("Model S", 100, true));
  registry.registerPrototype(
      "hybrid_base",
      std::make_unique<HybridVehicleConfig>("Camry Hybrid", 2500, 18));

  // EN: Clone from registry — no knowledge of concrete types needed.
  // TR: Kayıttan klonla — somut tip bilgisi gerekmez.
  auto car1 = registry.create("ev_standard");
  auto car2 = registry.create("ev_standard"); // Another clone of the same template
  auto car3 = registry.create("hybrid_base");

  std::cout << "Cloned configs:" << std::endl;
  if (car1) car1->display();
  if (car2) car2->display();
  if (car3) car3->display();
  std::cout << std::endl;

  // ── 2. Proxy ──
  std::cout << "--- PROXY (Lazy Init + Access Control) ---\n" << std::endl;

  // EN: Proxy created — no DB connection yet (lazy).
  // TR: Proxy oluşturuldu — henüz DB bağlantısı yok (tembel).
  DatabaseProxy adminDb("postgresql://prod:5432", "admin");
  DatabaseProxy guestDb("postgresql://prod:5432", "guest");

  std::cout << "\nAdmin queries:" << std::endl;
  std::cout << adminDb.query("SELECT * FROM sensors") << std::endl;
  std::cout << adminDb.query("SELECT * FROM diagnostics") << std::endl;

  std::cout << "\nGuest tries to query:" << std::endl;
  std::cout << guestDb.query("SELECT * FROM secrets") << std::endl;
  std::cout << std::endl;

  // ── 3. Before/After — Strategy ──
  std::cout << "--- BEFORE/AFTER: Strategy Refactoring ---\n" << std::endl;

  double price = 100.0;
```

```

    // BEFORE: if/else chain
    std::cout << "BEFORE (if/else): student=" << before::applyDiscount(price, "student")
              << ", military=" << before::applyDiscount(price, "military") << std::endl;

    // AFTER: Strategy pattern — new VIPDiscount added with ZERO modification
    after::StudentDiscount student;
    after::MilitaryDiscount military;
    after::VIPDiscount vip; // NEW type — no existing code changed

    std::cout << "AFTER  (Strategy): " << student.name() << "="
              << after::applyDiscount(price, student)
              << ", " << military.name() << "="
              << after::applyDiscount(price, military)
              << ", " << vip.name() << "="
              << after::applyDiscount(price, vip) << std::endl;
    std::cout << std::endl;

    // ── 4. Safe Observer (weak_ptr) ──
    std::cout << "--- SAFE OBSERVER (weak_ptr lifetime) ---\n" << std::endl;

    SafeEventBus safeBus;
    auto speedometer = std::make_shared<DashboardWidget>("Speedometer");
    auto tachometer = std::make_shared<DashboardWidget>("Tachometer");

    safeBus.subscribe(speedometer);
    safeBus.subscribe(tachometer);

    std::cout << "Subscribers: " << safeBus.subscriberCount() << std::endl;
    safeBus.notify("SPEED_UPDATE: 120 km/h");

    // EN: Destroy tachometer — it auto-unsubscribes (weak_ptr expires).
    // TR: Tachometer'ı yok et — otomatik abonelik iptali (weak_ptr süresi dolar).
    std::cout << "\nDestroying tachometer..." << std::endl;
    tachometer.reset();

    std::cout << "Notifying after destruction:" << std::endl;
    safeBus.notify("SPEED_UPDATE: 80 km/h");
    std::cout << "Subscribers after cleanup: " << safeBus.subscriberCount()
              << std::endl;

    // ── 5. Decision Summary ──
    std::cout << "\n--- PATTERN SELECTION SUMMARY ---\n" << std::endl;
    std::cout << "  Factory Method vs Abstract Factory: Single product vs Product family"
              << std::endl;
    std::cout << "  Strategy vs State: Caller picks vs Object transitions internally"
              << std::endl;
    std::cout << "  Adapter vs Facade: Interface translation vs Interface simplification"
              << std::endl;
    std::cout << "  Decorator vs CRTP: Runtime stacking vs Compile-time composition"
              << std::endl;
    std::cout << "  Singleton: Think twice — prefer Dependency Injection!" << std::endl;

    std::cout << "\n=> Pattern decision guide complete!" << std::endl;

    return 0;
}
```

► 07. Embedded Automotive Patterns

module_10_design_patterns/07_embedded_automotive_patterns.cpp

Konu: Embedded & Automotive Design Patterns — Gömülü ve Otomotiv Tasarım Kalıpları

Teori / Theory

THEORY: GoF vs Embedded Patterns

Classic GoF design patterns assume a desktop/server environment: heap allocation is cheap, virtual dispatch overhead is negligible, and memory is abundant. In embedded and automotive systems, these assumptions BREAK:

Assumption	Desktop/Server	Embedded/Automotive
Heap allocation	Fast, abundant	Forbidden or costly
Virtual dispatch	~ns overhead, fine	Predictability matters
Memory	GB available	KB-MB (MCU: 32-512KB)
Exceptions	Common error handling	Often disabled (-fno-)
RTTI (typeid/dyn_cast)	Available	Usually disabled
Dynamic containers	vector, map freely	Static buffers only
Thread model	std::thread, pthreads	RTOS tasks, ISRs

This file covers patterns specifically designed for resource-constrained, real-time, and safety-critical environments (AUTOSAR, RTOS, bare-metal).

TEORİ: GoF ve Gömülü Kalıplar Farkı

Klasik GoF tasarım kalıpları masaüstü/sunucu ortamını varsayar: heap tahsisi ucuzdur, sanal dağıtım maliyeti ihmal edilebilir ve bellek boldur. Gömülü ve otomotiv sistemlerde bu varsayımlar KIRILIR:

Varsayım	Masaüstü/Sunucu	Gömülü/Otomotiv
Heap tahsisi	Hızlı, bol	Yasak veya maliyetli
Sanal dağıtım	~ns maliyet, sorunsuz	Öngörülebilirlik önemli
Bellek	GB mevcut	KB-MB (MCU: 32-512KB)
İstisnalar	Yaygın hata yönetimi	Genellikle devre dışı
RTTI (typeid/dyn_cast)	Mevcut	Genellikle devre dışı
Dinamik konteyner	vector, map serbestçe	Sadece statik tamponlar
Thread modeli	std::thread, pthreads	RTOS görevleri, ISR'ler

Bu dosya, kaynak kısıtlı, gerçek zamanlı ve güvenlik-kritik ortamlara (AUTOSAR, RTOS, bare-metal) özel tasarlanmış kalıpları kapsar.

THEORY: Memory Pool Pattern

In safety-critical embedded systems (ASIL-B and above), dynamic memory allocation (malloc/new) is either FORBIDDEN or severely restricted after initialization. Reasons:
1. Fragmentation: repeated alloc/free fragments memory → eventual OOM
2. Non-deterministic timing: malloc's worst-case time is unbounded
3. No virtual memory: MCUs have no MMU, OOM = hard fault
4. MISRA C++ Rule 21.3: "shall not use dynamic memory after init phase"
Memory Pool allocates a fixed block at startup and serves objects from it:

```
┌──────────────────────────────────────────────┐
│ Pool<Sensor, 16>                             │
│ ┌────┬────┬────┬────┬────┬────┬────┬────┬── ···  │
│ │ S0 │ S1 │FREE│ S3 │FREE│FREE│ S6 │FREE│        │
│ └────┴────┴────┴────┴────┴────┴────┴────┴── ···  │
│ Free list: [2] → [4] → [5] → [7] → ···       │
└──────────────────────────────────────────────┘
```

Uses placement new to construct objects in pre-allocated storage.

TEORİ: Bellek Havuzu Kalıbı

Güvenlik-kritik gömülü sistemlerde (ASIL-B ve üzeri), dinamik bellek tahsisi (malloc/new) başlatma sonrası ya YASAKTIR ya da ciddi şekilde kısıtlıdır. Nedenler:
1. Parçalanma: tekrarlanan alloc/free belleği parçalar → sonunda OOM
2. Belirsiz zamanlama: malloc'un en kötü durum süresi sınırsızdır
3. Sanal bellek yok: MCU'larda MMU yok, OOM = hard fault
4. MISRA C++ Kuralı 21.3: "init aşamasından sonra dinamik bellek kullanılmamalı"
Bellek Havuzu, başlangıçta sabit bir blok tahsis eder ve nesneleri oradan sunar:

```
┌──────────────────────────────────────────────┐
│ Pool<Sensör, 16>                             │
│ ┌────┬────┬────┬────┬────┬────┬────┬────┬── ···  │
│ │ S0 │ S1 │BOŞ │ S3 │BOŞ │BOŞ │ S6 │BOŞ │        │
│ └────┴────┴────┴────┴────┴────┴────┴────┴── ···  │
│ Serbest liste: [2] → [4] → [5] → [7] → ···   │
└──────────────────────────────────────────────┘
```

Placement new ile önceden tahsis edilmiş depolamada nesne oluşturur.

THEORY: Active Object Pattern

The Active Object pattern decouples METHOD INVOCATION from METHOD EXECUTION. Instead of calling a method directly (which blocks the caller), the request is placed in a message queue. A dedicated thread processes requests sequentially.

```
┌──────────┐    enqueue()    ┌────────────────┐   dequeue()   ┌──────────┐
│  Caller  │ ──────────────→ │  Message Queue │ ────────────→ │ Worker   │
│ (Client) │                 │  (thread-safe) │               │ Thread   │
└──────────┘                 └────────────────┘               └──────────┘
```

Why use it in embedded/automotive?
• AUTOSAR Runnables are essentially Active Objects (triggered by OS schedule)
• ISR → Task communication: ISR enqueues, task dequeues safely
• Deterministic ordering: messages processed FIFO
• Thread safety: only the worker thread accesses shared state

TEORİ: Aktif Nesne Kalıbı

Aktif Nesne kalıbı, METOT ÇAĞRISINI METOT YÜRÜTMESINDEN ayırır. Bir metodu doğrudan çağırmak yerine (çağıranı bloklar), istek bir mesaj kuyruğuna yerleştirilir. Özel bir thread istekleri sırayla işler.

```
┌──────────┐    enqueue()    ┌──────────────┐   dequeue()   ┌──────────┐
│ Çağıran  │ ──────────────→ │ Mesaj Kuyruğu │ ────────────→ │ Worker   │
│ (İstemci)│                 │(thread-güvenli)│              │ Thread   │
└──────────┘                 └──────────────┘               └──────────┘
```

Gömülü/otomotivde neden kullanılır?
• AUTOSAR Runnable'lar esasen Aktif Nesnelerdir (OS zamanlayıcısı tarafından tetiklenir)
• ISR → Task iletişimi: ISR kuyruğa ekler, task güvenle kuyruğundan alır
• Belirleyici sıralama: mesajlar FIFO işlenir
• Thread güvenliği: yalnızca worker thread paylaşılan duruma erişir

THEORY: Embedded State Machine

State machines are ubiquitous in embedded systems. Every ECU mode controller, communication protocol stack, and peripheral driver is fundamentally an FSM.
Two main implementation approaches:

Approach	Characteristics
Switch-Case FSM	Simple, inline, fast. Hard to scale.
Table-Driven FSM	Data-driven, extensible, MISRA-friendly.
	State × Event → {Action, NextState}

Table-driven FSMs are preferred in automotive (AUTOSAR BSW uses them internally):
- Transition table is const data → stored in FLASH, not RAM
- Adding states/events doesn't change the dispatch logic
- Easy to auto-generate from UML state diagrams
- MISRA C++ 2023 compliant (no complex control flow)

TEORİ: Gömülü Durum Makinesi

Durum makineleri gömülü sistemlerde her yerdedir. Her ECU mod kontrolcüsü, iletişim protokol yığını ve çevre birimi sürücüsü temelde bir FSM'dir.
İki ana uygulama yaklaşımı:

Yaklaşım	Özellikler
Switch-Case FSM	Basit, satır içi, hızlı. Ölçeklemesi zor.
Tablo Güdümlü FSM	Veri güdümlü, genişletilebilir, MISRA uyumlu.
	Durum × Olay → {Aksiyon, SonrakiDurum}

Tablo güdümlü FSM'ler otomotivde tercih edilir (AUTOSAR BSW dahili olarak kullanır):
- Geçiş tablosu const veridir → FLASH'ta saklanır, RAM'da değil
- Durum/olay eklemek dağıtım mantığını değiştirmez
- UML durum diyagramlarından otomatik oluşturulması kolaydır
- MISRA C++ 2023 uyumlu (karmaşık kontrol akışı yok)

THEORY: Double Buffer & Hardware Proxy

Double Buffer: Used when one entity writes data while another reads it concurrently. Two buffers alternate roles — while the reader uses buffer A, the writer fills buffer B. On swap, roles reverse. ISR writes to back buffer; main loop reads front buffer. No locks needed if swap is atomic (single pointer/index swap).
Hardware Proxy: Wraps raw register access behind an OOP interface. Instead of: *(volatile uint32_t*)0x40020000 |= (1 << 5); You write: gpio.set_pin(5); Uses CRTP + volatile to achieve zero-cost abstraction over MMIO registers.

TEORİ: Çift Tampon ve Donanım Vekili

Çift Tampon: Bir taraf veri yazarken diğer taraf eşzamanlı okuyorsa kullanılır. İki tampon rol değiştirir — okuyucu A tamponu kullanırken, yazıcı B tamponunu doldurur. Takas'ta roller tersine döner. ISR arka tampona yazar; ana döngü ön tamponu okur. Takas atomik ise (tek pointer/indeks takası) kilit gerekmez.
Donanım Vekili: Ham register erişimini OOP arayüzüyle sarar. Şu yerine: *(volatile uint32_t*)0x40020000 |= (1 << 5); Şunu yazarsın: gpio.set_pin(5); CRTP + volatile ile MMIO register'ları üzerinde sıfır maliyetli soyutlama sağlar.

→ Ref: https://en.cppreference.com/w/cpp/language/new — placement new
→ Ref: https://en.cppreference.com/w/cpp/atomic/atomic — std::atomic for lock-free patterns
→ Ref: https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rr-newdelete — R.10/R.11

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <cstring>
#include <array>
#include <new>             // EN: placement new / TR: placement new
#include <functional>
#include <queue>
#include <mutex>
#include <condition_variable>
#include <thread>
#include <atomic>
#include <chrono>
#include <string>
#include <optional>
#include <cassert>
#include <iomanip>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: Static Memory Pool<T, N> — Heap-Free Object Management
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: A fixed-size object pool that allocates from a static array. No heap, no fragmentation.
 *     Uses a free-list (singly linked via indices) for O(1) allocate/deallocate.
 *     placement new constructs objects in-place; explicit destructor call destroys them.
 *
 *     MISRA-friendly: no dynamic allocation after init, deterministic timing.
 *     Suitable for RTOS tasks, ISR-safe if wrapped with a spinlock.
 *
 * TR: Statik diziden tahsis yapan sabit boyutlu nesne havuzu. Heap yok, parçalanma yok.
 *     O(1) tahsis/serbest bırakma için serbest liste (indekslerle tek bağlı) kullanır.
 *     placement new nesneleri yerinde oluşturur; açık destructor çağrısı yıkar.
 *
 *     MISRA uyumlu: init sonrası dinamik tahsis yok, belirleyici zamanlama.
 *     RTOS görevleri için uygun, spinlock ile sarılırsa ISR güvenli.
 */
template<typename T, std::size_t N>
```

```cpp
class MemoryPool {
    static_assert(N > 0, "Pool size must be > 0");

    // EN: Each slot is a union: either holds a T object or a free-list index.
    //     When free, 'next_free' points to the next available slot.
    //     When occupied, 'storage' holds the constructed T.
    // TR: Her yuva bir union: ya T nesnesi ya da serbest liste indeksi tutar.
    //     Boşken 'next_free' sonraki uygun yuvayı gösterir.
    //     Doluyken 'storage' inşa edilmiş T'yi tutar.
    struct Slot {
        alignas(T) std::byte storage[sizeof(T)];
        std::size_t next_free;
        bool occupied = false;
    };

    std::array<Slot, N> slots_;
    std::size_t free_head_;     // EN: Head of the free list / TR: Serbest listenin başı
    std::size_t active_count_;  // EN: Number of allocated objects / TR: Tahsis edilmiş nesne sayısı

public:
    MemoryPool() : free_head_(0), active_count_(0) {
        // EN: Initialize free list: slot[0] → slot[1] → ··· → slot[N-1] → SENTINEL
        // TR: Serbest listeyi başlat: slot[0] → slot[1] → ··· → slot[N-1] → NÖBETÇİ
        for (std::size_t i = 0; i < N; ++i) {
            slots_[i].next_free = i + 1;  // EN: Last element points to N (sentinel) / TR: Son eleman N'yi gösterir (nöbetçi)
            slots_[i].occupied = false;
        }
    }

    ~MemoryPool() {
        // EN: Destroy all still-alive objects — prevent resource leaks
        // TR: Hâlâ yaşayan tüm nesneleri yık — kaynak sızıntısını önle
        for (std::size_t i = 0; i < N; ++i) {
            if (slots_[i].occupied) {
                object_at(i)->~T();
            }
        }
    }

    // EN: Non-copyable, non-movable (owns raw storage)
    // TR: Kopyalanamaz, taşınamaz (ham depolamaya sahip)
    MemoryPool(const MemoryPool&) = delete;
    MemoryPool& operator=(const MemoryPool&) = delete;

    /// EN: Construct a T in the pool. Returns pointer or nullptr if full.
    /// TR: Havuzda bir T oluştur. Doluysa nullptr döndürür.
    template<typename... Args>
    T* allocate(Args&&... args) {
        if (free_head_ >= N) return nullptr;  // EN: Pool exhausted / TR: Havuz dolu

        std::size_t idx = free_head_;
        free_head_ = slots_[idx].next_free;   // EN: Advance free list head / TR: Serbest listeyi ilerlet
        slots_[idx].occupied = true;
        ++active_count_;

        // EN: placement new — construct T in pre-allocated storage
        // TR: placement new — önceden ayrılmış depolamada T oluştur
        return new (slots_[idx].storage) T(std::forward<Args>(args)...);
    }

    /// EN: Destroy object and return its slot to the free list.
    /// TR: Nesneyi yık ve yuvasını serbest listeye döndür.
    void deallocate(T* ptr) {
        if (!ptr) return;

        // EN: Find slot index from pointer arithmetic
        // TR: İşaretçi aritmetiğinden yuva indeksi bul
        auto* byte_ptr = reinterpret_cast<std::byte*>(ptr);
        std::size_t idx = 0;
        bool found = false;
```

```
        for (std::size_t i = 0; i < N; ++i) {
            if (slots_[i].storage == byte_ptr) {
                idx = i;
                found = true;
                break;
            }
        }

        if (!found || !slots_[idx].occupied) return;  // EN: Not from this pool / TR: Bu havuzdan
 değil

        ptr->~T();                            // EN: Explicit destructor call / TR: Açık destructor
 çağrısı
        slots_[idx].occupied = false;
        slots_[idx].next_free = free_head_;   // EN: Push to front of free list / TR: Serbest
 listenin başına ekle
        free_head_ = idx;
        --active_count_;
    }

    [[nodiscard]] std::size_t active() const { return active_count_; }
    [[nodiscard]] std::size_t capacity() const { return N; }
    [[nodiscard]] std::size_t available() const { return N - active_count_; }

private:
    T* object_at(std::size_t idx) {
        return reinterpret_cast<T*>(slots_[idx].storage);
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: Double Buffer for Sensor Data — ISR/DMA Safe
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: Double buffer for concurrent producer/consumer without locks.
 *     The producer writes to the "back" buffer; the consumer reads the "front" buffer.
 *     swap() atomically exchanges the two buffer indices.
 *
 *     Real-world use: DMA fills back buffer with ADC samples while the main loop
 *     processes the front buffer. After DMA complete interrupt → swap().
 *
 *     Template parameters:
 *       T = data type (e.g., SensorFrame)
 *       N = number of elements per buffer (e.g., 8 ADC channels)
 *
 * TR: Kilitsiz eşzamanlı üretici/tüketici için çift tampon.
 *     Üretici "arka" tampona yazar; tüketici "ön" tamponu okur.
 *     swap() iki tampon indeksini atomik olarak değiştirir.
 *
 *     Gerçek kullanım: DMA, arka tamponu ADC örnekleriyle doldururken ana döngü
 *     ön tamponu işler. DMA tamamlama kesmesinden sonra → swap().
 */
template<typename T, std::size_t N>
class DoubleBuffer {
    std::array<T, N> buffer_a_{};
    std::array<T, N> buffer_b_{};
    std::atomic<int> active_{0};  // EN: 0=A is front, 1=B is front / TR: 0=A ön, 1=B ön

public:
    /// EN: Get read-only reference to the front (consumer) buffer.
    /// TR: Ön (tüketici) tamponuna salt okunur referans al.
    [[nodiscard]] const std::array<T, N>& front() const {
        return (active_.load(std::memory_order_acquire) == 0) ? buffer_a_ : buffer_b_;
    }

    /// EN: Get writable reference to the back (producer) buffer.
    /// TR: Arka (üretici) tamponuna yazılabilir referans al.
    [[nodiscard]] std::array<T, N>& back() {
        return (active_.load(std::memory_order_acquire) == 0) ? buffer_b_ : buffer_a_;
    }

```

```
    /// EN: Atomically swap front and back buffers. Call this after producer finishes writing.
    /// TR: Ön ve arka tamponları atomik olarak takas et. Üretici yazmayı bitirdikten sonra çağır.
    void swap() {
        int expected = active_.load(std::memory_order_relaxed);
        int desired = 1 - expected;
        active_.compare_exchange_strong(expected, desired, std::memory_order_release);
    }
};

// ==============================================================================
// PART 3: Active Object with Message Queue
// ==============================================================================

/**
 * EN: Active Object runs tasks asynchronously in a dedicated thread.
 *     Callers enqueue std::function<void()> messages; the worker thread processes them FIFO.
 *     Thread-safe: the queue is protected by a mutex + condition variable.
 *
 *     This is the foundation for:
 *     - AUTOSAR Runnables (OS-scheduled functions without shared state)
 *     - CAN message dispatchers (ISR enqueues, task processes)
 *     - Command pattern with deferred execution
 *
 * TR: Aktif Nesne, görevleri özel bir thread'de asenkron olarak çalıştırır.
 *     Çağıranlar std::function<void()> mesajları kuyruğa ekler; worker thread FIFO işler.
 *     Thread güvenli: kuyruk mutex + koşul değişkeniyle korunur.
 *
 *     Bunun temeli:
 *     - AUTOSAR Runnable'lar (paylaşılan durum olmadan OS-zamanlanmış fonksiyonlar)
 *     - CAN mesaj dağıtıcıları (ISR kuyruğa ekler, task işler)
 *     - Ertelenmiş yürütme ile Command kalıbı
 */
class ActiveObject {
    std::queue<std::function<void()>> queue_;
    std::mutex mutex_;
    std::condition_variable cv_;
    std::thread worker_;
    std::atomic<bool> running_{true};

public:
    ActiveObject() : worker_([this] { run(); }) {}

    ~ActiveObject() {
        // EN: Signal the worker to stop and wait for it to finish
        // TR: Worker'a durmasını sinyal ver ve bitmesini bekle
        {
            std::lock_guard<std::mutex> lock(mutex_);
            running_ = false;
        }
        cv_.notify_one();
        if (worker_.joinable()) worker_.join();
    }

    // EN: Non-copyable (owns a thread)
    // TR: Kopyalanamaz (bir thread'e sahip)
    ActiveObject(const ActiveObject&) = delete;
    ActiveObject& operator=(const ActiveObject&) = delete;

    /// EN: Enqueue a task for asynchronous execution.
    /// TR: Asenkron yürütme için bir görev kuyruğa ekle.
    void enqueue(std::function<void()> task) {
        {
            std::lock_guard<std::mutex> lock(mutex_);
            queue_.push(std::move(task));
        }
        cv_.notify_one();
    }

    /// EN: Wait until all pending tasks are processed.
    /// TR: Tüm bekleyen görevler işlenene kadar bekle.
    void flush() {
        // EN: Enqueue a synchronization task and wait on its completion
        // TR: Bir senkronizasyon görevi kuyruğa ekle ve tamamlanmasını bekle
        std::mutex m;
        std::condition_variable cv;
        bool done = false;

```

```cpp
        enqueue([&] {
            std::lock_guard<std::mutex> lock(m);
            done = true;
            cv.notify_one();
        });

        std::unique_lock<std::mutex> lock(m);
        cv.wait(lock, [&] { return done; });
    }

private:
    void run() {
        while (true) {
            std::function<void()> task;
            {
                std::unique_lock<std::mutex> lock(mutex_);
                cv_.wait(lock, [this] { return !queue_.empty() || !running_; });

                if (!running_ && queue_.empty()) return;

                task = std::move(queue_.front());
                queue_.pop();
            }
            task();  // EN: Execute outside the lock / TR: Kilidin dışında çalıştır
        }
    }
};

// ============================================================================
// PART 4: Table-Driven State Machine — MISRA Compliant
// ============================================================================

/**
 * EN: A generic table-driven FSM where the transition table is const data.
 *     State × Event → { guard(), action(), next_state }
 *     The dispatch loop is fixed; only the table changes per application.
 *
 *     This is the pattern used in AUTOSAR Basic Software (BSW) modules:
 *     - BswM (BSW Mode Manager)
 *     - ComM (Communication Manager)
 *     - EcuM (ECU State Manager)
 *
 * TR: Geçiş tablosunun const veri olduğu genel tablo güdümlü FSM.
 *     Durum × Olay → { guard(), aksiyon(), sonraki_durum }
 *     Dağıtım döngüsü sabittir; uygulama başına yalnızca tablo değişir.
 *
 *     AUTOSAR Temel Yazılım (BSW) modüllerinde kullanılan kalıp:
 *     - BswM (BSW Mod Yöneticisi)
 *     - ComM (İletişim Yöneticisi)
 *     - EcuM (ECU Durum Yöneticisi)
 */

// EN: ECU operating states / TR: ECU çalışma durumları
enum class EcuState : uint8_t {
    STARTUP,      // EN: Power-on initialization / TR: Güç açılış başlatma
    NORMAL,       // EN: Normal operation mode / TR: Normal çalışma modu
    DIAGNOSTIC,   // EN: Diagnostic session active / TR: Teşhis oturumu aktif
    SLEEP,        // EN: Low-power sleep mode / TR: Düşük güç uyku modu
    COUNT         // EN: Number of states (sentinel) / TR: Durum sayısı (nöbetçi)
};

// EN: Events that trigger state transitions / TR: Durum geçişlerini tetikleyen olaylar
enum class EcuEvent : uint8_t {
    INIT_COMPLETE,     // EN: Initialization finished / TR: Başlatma tamamlandı
    DIAG_REQUEST,      // EN: Diagnostic request received / TR: Teşhis isteği alındı
    DIAG_TIMEOUT,      // EN: Diagnostic session timed out / TR: Teşhis oturumu zaman aşımı
    SLEEP_REQUEST,     // EN: Sleep mode requested (ignition off) / TR: Uyku modu istendi
    WAKEUP,            // EN: Wake-up signal (CAN, LIN, timer) / TR: Uyanma sinyali
    COUNT              // EN: Number of events (sentinel) / TR: Olay sayısı (nöbetçi)
};
```

```cpp

// EN: Convert enum to string for logging / TR: Log için enum'u string'e dönüştür
const char* to_string(EcuState s) {
    constexpr const char* names[] = {"STARTUP", "NORMAL", "DIAGNOSTIC", "SLEEP"};
    return names[static_cast<uint8_t>(s)];
}

const char* to_string(EcuEvent e) {
    constexpr const char* names[] = {
        "INIT_COMPLETE", "DIAG_REQUEST", "DIAG_TIMEOUT", "SLEEP_REQUEST", "WAKEUP"
    };
    return names[static_cast<uint8_t>(e)];
}

// EN: Transition entry: guard + action + next state
// TR: Geçiş girdisi: koruma + aksiyon + sonraki durum
struct Transition {
    using Guard  = bool(*)();    // EN: Pre-condition check / TR: Ön koşul kontrolü
    using Action = void(*)();    // EN: Side effect / TR: Yan etki

    Guard  guard;                // EN: nullptr = always true / TR: nullptr = her zaman doğru
    Action action;               // EN: nullptr = no action / TR: nullptr = aksiyon yok
    EcuState next_state;
};

// EN: No-op transition (stay in current state)
// TR: İşlemsiz geçiş (mevcut durumda kal)
constexpr Transition NO_TRANSITION{nullptr, nullptr, EcuState::COUNT};

// EN: Action functions — side effects of state transitions
// TR: Aksiyon fonksiyonları — durum geçişlerinin yan etkileri
void action_start_normal()   { std::cout << "    [ACTION] Starting normal operation — peripherals enabled\n"; }
void action_enter_diag()     { std::cout << "    [ACTION] Entering diagnostic session — UDS active\n"; }
void action_exit_diag()      { std::cout << "    [ACTION] Exiting diagnostic — returning to normal\n"; }
void action_enter_sleep()    { std::cout << "    [ACTION] Entering sleep — CAN transceiver to standby\n"; }
void action_wakeup()         { std::cout << "    [ACTION] Wake-up detected — re-initializing\n"; }

// EN: Guard functions — conditions that must be true for transition
// TR: Koruma fonksiyonları — geçiş için doğru olması gereken koşullar
bool guard_always_true() { return true; }

// EN: Transition table — the CORE of the table-driven FSM
//     Indexed: transition_table[state][event]
// TR: Geçiş tablosu — tablo güdümlü FSM'nin ÇEKİRDEĞİ
//     İndeksleme: transition_table[durum][olay]
constexpr std::size_t STATE_COUNT = static_cast<std::size_t>(EcuState::COUNT);
constexpr std::size_t EVENT_COUNT = static_cast<std::size_t>(EcuEvent::COUNT);

// EN: We initialize transition table at runtime because function pointers aren't constexpr
// TR: Fonksiyon işaretçileri constexpr olmadığından geçiş tablosu çalışma zamanında başlatılır
Transition transition_table[STATE_COUNT][EVENT_COUNT];

void init_transition_table() {
    // EN: Fill with NO_TRANSITION (stay in current state for unhandled events)
    // TR: NO_TRANSITION ile doldur (işlenmeyen olaylarda mevcut durumda kal)
    for (std::size_t s = 0; s < STATE_COUNT; ++s)
        for (std::size_t e = 0; e < EVENT_COUNT; ++e)
            transition_table[s][e] = NO_TRANSITION;

    // EN: STARTUP state transitions / TR: STARTUP durumu geçişleri
    transition_table[0][0] = {guard_always_true, action_start_normal, EcuState::NORMAL};

    // EN: NORMAL state transitions / TR: NORMAL durumu geçişleri
    transition_table[1][1] = {guard_always_true, action_enter_diag,   EcuState::DIAGNOSTIC};
    transition_table[1][3] = {guard_always_true, action_enter_sleep,  EcuState::SLEEP};

    // EN: DIAGNOSTIC state transitions / TR: DIAGNOSTIC durumu geçişleri
    transition_table[2][2] = {guard_always_true, action_exit_diag,    EcuState::NORMAL};
    transition_table[2][3] = {guard_always_true, action_enter_sleep,  EcuState::SLEEP};

```

```
    // EN: SLEEP state transitions / TR: SLEEP durumu geçişleri
    transition_table[3][4] = {guard_always_true, action_wakeup,       EcuState::STARTUP};
}

/// EN: The FSM dispatch function — processes one event.
/// TR: FSM dağıtım fonksiyonu — bir olay işler.
EcuState fsm_dispatch(EcuState current, EcuEvent event) {
    auto s = static_cast<std::size_t>(current);
    auto e = static_cast<std::size_t>(event);

    const auto& t = transition_table[s][e];

    // EN: If no transition defined, stay in current state
    // TR: Geçiş tanımlı değilse mevcut durumda kal
    if (t.next_state == EcuState::COUNT) return current;

    // EN: Check guard condition / TR: Koruma koşulunu kontrol et
    if (t.guard && !t.guard()) return current;

    // EN: Execute action / TR: Aksiyonu çalıştır
    if (t.action) t.action();

    std::cout << "    [FSM] " << to_string(current) << " --(" << to_string(event)
              << ")--> " << to_string(t.next_state) << "\n";

    return t.next_state;
}

// ═══════════════════════════════════════════════════════════════════════════
// PART 5: Hardware Proxy Pattern — CRTP + volatile
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: Wraps raw memory-mapped register access behind a type-safe OOP interface.
 *     Uses CRTP so derived classes specify the register base address and layout.
 *     volatile ensures the compiler doesn't optimize away register reads/writes.
 *
 *     On a real MCU, base_address would be e.g., 0x40020000 (GPIO port A on STM32).
 *     Here we simulate with a static array, but the interface is identical.
 *
 * TR: Ham bellek eşlemeli register erişimini tip güvenli OOP arayüzüyle sarar.
 *     CRTP kullanır, böylece türetilmiş sınıflar register temel adresini ve düzenini belirtir.
 *     volatile, derleyicinin register okuma/yazmaları optimize etmemesini sağlar.
 *
 *     Gerçek MCU'da base_address ör. 0x40020000 (STM32'de GPIO port A) olurdu.
 *     Burada statik dizi ile simüle ediyoruz ama arayüz aynı.
 */
template<typename Derived>
class RegisterBase {
protected:
    // EN: Read a 32-bit register at given offset from base
    // TR: Tabandan verilen offset'teki 32-bit register'ı oku
    [[nodiscard]] uint32_t read_reg(std::size_t offset) const {
        const volatile uint32_t* reg = static_cast<const Derived*>(this)->reg_ptr(offset);
        return *reg;
    }

    // EN: Write a 32-bit value to register at given offset
    // TR: Verilen offset'teki register'a 32-bit değer yaz
    void write_reg(std::size_t offset, uint32_t value) {
        volatile uint32_t* reg = static_cast<Derived*>(this)->reg_ptr(offset);
        *reg = value;
    }

    // EN: Set specific bits (read-modify-write)
    // TR: Belirli bitleri ayarla (oku-değiştir-yaz)
    void set_bits(std::size_t offset, uint32_t mask) {
        write_reg(offset, read_reg(offset) | mask);
    }

    // EN: Clear specific bits
    // TR: Belirli bitleri temizle
    void clear_bits(std::size_t offset, uint32_t mask) {
        write_reg(offset, read_reg(offset) & ~mask);
    }
};
```

```cpp

/**
 * EN: Simulated GPIO port — demonstrates the Hardware Proxy pattern.
 *     In production code, regs_ would be removed and reg_ptr() would return
 *     reinterpret_cast<volatile uint32_t*>(BASE_ADDRESS + offset).
 *
 * TR: Simüle edilmiş GPIO portu — Donanım Vekili kalıbını gösterir.
 *     Üretim kodunda regs_ kaldırılır ve reg_ptr()
 *     reinterpret_cast<volatile uint32_t*>(BASE_ADDRESS + offset) döndürür.
 */
class GpioPort : public RegisterBase<GpioPort> {
    // EN: Register offsets (matching STM32 GPIO layout)
    // TR: Register offset'leri (STM32 GPIO düzenine uygun)
    static constexpr std::size_t MODER  = 0;  // EN: Mode register / TR: Mod register'ı
    static constexpr std::size_t ODR    = 1;  // EN: Output data register / TR: Çıkış veri register'ı
    static constexpr std::size_t IDR    = 2;  // EN: Input data register / TR: Giriş veri register'ı
    static constexpr std::size_t BSRR   = 3;  // EN: Bit set/reset register / TR: Bit ayarla/sıfırla register'ı

    // EN: Simulated register file (in production: MMIO at fixed address)
    // TR: Simüle register dosyası (üretimde: sabit adreste MMIO)
    uint32_t regs_[4] = {0, 0, 0, 0};

    friend class RegisterBase<GpioPort>;
    volatile uint32_t* reg_ptr(std::size_t offset) {
        return reinterpret_cast<volatile uint32_t*>(&regs_[offset]);
    }
    const volatile uint32_t* reg_ptr(std::size_t offset) const {
        return reinterpret_cast<const volatile uint32_t*>(&regs_[offset]);
    }

public:
    /// EN: Configure pin as output (set 2 mode bits to 01)
    /// TR: Pin'i çıkış olarak yapılandır (2 mod bitini 01 yap)
    void configure_output(uint8_t pin) {
        assert(pin < 16);
        uint32_t shift = static_cast<uint32_t>(pin) * 2;
        uint32_t mode = read_reg(MODER);
        mode &= ~(0x3u << shift);    // EN: Clear 2 bits / TR: 2 biti temizle
        mode |=  (0x1u << shift);    // EN: Set to output (01) / TR: Çıkışa ayarla (01)
        write_reg(MODER, mode);
    }

    /// EN: Set pin HIGH using the bit set/reset register (atomic on real HW)
    /// TR: Pin'i YÜKSEK yap — bit ayarla/sıfırla register kullan (gerçek HW'da atomik)
    void set_pin(uint8_t pin) {
        assert(pin < 16);
        write_reg(BSRR, 1u << pin);    // EN: Write to lower 16 bits = SET / TR: Alt 16 bite yaz = AYARLA
        set_bits(ODR, 1u << pin);       // EN: Also update ODR for simulation / TR: Simülasyon için ODR'yi de güncelle
    }

    /// EN: Set pin LOW
    /// TR: Pin'i DÜŞÜK yap
    void clear_pin(uint8_t pin) {
        assert(pin < 16);
        write_reg(BSRR, 1u << (pin + 16));  // EN: Write to upper 16 bits = RESET / TR: Üst 16 bite yaz = SIFIRLA
        clear_bits(ODR, 1u << pin);
    }

    /// EN: Read pin state from output data register
    /// TR: Çıkış veri register'ından pin durumunu oku
    [[nodiscard]] bool read_pin(uint8_t pin) const {
        assert(pin < 16);
        return (read_reg(ODR) & (1u << pin)) != 0;
    }

    /// EN: Read all output pins as bitmask
    /// TR: Tüm çıkış pin'lerini bit maskesi olarak oku
    [[nodiscard]] uint32_t read_all() const {
        return read_reg(ODR);
    }
};
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════
// PART 6: Real-World — ECU Mode Manager (Integrating All Patterns)
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: Combines the patterns above into a realistic ECU scenario:
 *     - MemoryPool for CAN message objects
 *     - DoubleBuffer for sensor readings
 *     - ActiveObject for async task processing
 *     - Table-Driven FSM for ECU state management
 *     - GpioPort for hardware indicator (LED)
 *
 * TR: Yukarıdaki kalıpları gerçekçi bir ECU senaryosunda birleştirir:
 *     - CAN mesaj nesneleri için MemoryPool
 *     - Sensör okumaları için DoubleBuffer
 *     - Asenkron görev işleme için ActiveObject
 *     - ECU durum yönetimi için Tablo Güdümlü FSM
 *     - Donanım göstergesi (LED) için GpioPort
 */
struct CanMessage {
    uint32_t id;
    uint8_t dlc;
    std::array<uint8_t, 8> data;

    friend std::ostream& operator<<(std::ostream& os, const CanMessage& msg) {
        os << "CAN[0x" << std::hex << std::setfill('0') << std::setw(3) << msg.id
           << std::dec << " DLC=" << static_cast<int>(msg.dlc) << " Data=";
        for (int i = 0; i < msg.dlc && i < 8; ++i) {
            os << std::hex << std::setw(2) << static_cast<int>(msg.data[static_cast<std::size_t>(i)])
;
            if (i < msg.dlc - 1) os << ":";
        }
        os << std::dec << "]";
        return os;
    }
};

struct SensorReading {
    float temperature;
    float voltage;
    uint32_t timestamp_ms;
};

// ═══════════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 10 — Embedded & Automotive Patterns\n";
    std::cout << "============================================\n\n";

    // ─── Demo 1: Static Memory Pool ──────────────────────────────────────
    {
        std::cout << "--- Demo 1: Static Memory Pool<CanMessage, 8> ---\n";
        // EN: Create a pool of 8 CAN messages — no heap allocation after this point
        // TR: 8 CAN mesajlık havuz oluştur — bu noktadan sonra heap tahsisi yok
        MemoryPool<CanMessage, 8> pool;
        std::cout << "  Pool capacity: " << pool.capacity()
                  << ", available: " << pool.available() << "\n";

        // EN: Allocate some CAN messages from the pool
        // TR: Havuzdan bazı CAN mesajları tahsis et
        auto* msg1 = pool.allocate(CanMessage{0x1A3, 8, {0xDE, 0xAD, 0xBE, 0xEF, 0x00, 0xFF, 0x12,
0x34}});
        auto* msg2 = pool.allocate(CanMessage{0x7DF, 3, {0x02, 0x01, 0x0D, 0, 0, 0, 0, 0}});
        auto* msg3 = pool.allocate(CanMessage{0x100, 2, {0x55, 0xAA, 0, 0, 0, 0, 0, 0}});

        std::cout << "  Allocated 3 messages. Active: " << pool.active()
                  << ", Available: " << pool.available() << "\n";

        if (msg1) std::cout << "  msg1: " << *msg1 << "\n";
        if (msg2) std::cout << "  msg2: " << *msg2 << "\n";
        if (msg3) std::cout << "  msg3: " << *msg3 << "\n";

```

```cpp
        // EN: Return msg2 to the pool — slot is recycled
        // TR: msg2'yi havuza geri ver — yuva geri dönüştürülür
        pool.deallocate(msg2);
        std::cout << "  After deallocating msg2: Active=" << pool.active()
                  << ", Available=" << pool.available() << "\n";

        // EN: Allocate a new message — reuses the freed slot
        // TR: Yeni mesaj tahsis et — serbest bırakılan yuvayı yeniden kullanır
        auto* msg4 = pool.allocate(CanMessage{0x200, 4, {0x01, 0x02, 0x03, 0x04, 0, 0, 0, 0}});
        if (msg4) std::cout << "  msg4 (reused slot): " << *msg4 << "\n";
        std::cout << "\n";
    }

    // ── Demo 2: Double Buffer ──────────────────────────────────────
    {
        std::cout << "--- Demo 2: Double Buffer for Sensor Data ---\n";
        DoubleBuffer<SensorReading, 4> sensor_buf;

        // EN: Simulate ISR/DMA: write to back buffer
        // TR: ISR/DMA simülasyonu: arka tampona yaz
        auto& back = sensor_buf.back();
        back[0] = {85.3f, 12.1f, 1000};
        back[1] = {90.1f, 11.9f, 1001};
        back[2] = {76.5f, 12.3f, 1002};
        back[3] = {88.8f, 12.0f, 1003};
        std::cout << "  [Producer] Wrote 4 sensor readings to back buffer\n";

        // EN: Swap buffers — now the producer's data becomes readable
        // TR: Tamponları takas et — üreticinin verisi artık okunabilir
        sensor_buf.swap();
        std::cout << "  [Swap] Buffers exchanged\n";

        // EN: Consumer reads from front buffer
        // TR: Tüketici ön tampondan okur
        const auto& front = sensor_buf.front();
        for (std::size_t i = 0; i < 4; ++i) {
            std::cout << "  [Consumer] Sensor[" << i << "]: temp="
                      << front[i].temperature << "°C, voltage="
                      << front[i].voltage << "V, ts=" << front[i].timestamp_ms << "ms\n";
        }
        std::cout << "\n";
    }

    // ── Demo 3: Active Object ──────────────────────────────────────
    {
        std::cout << "--- Demo 3: Active Object (Async Task Queue) ---\n";
        ActiveObject worker;

        // EN: Enqueue 5 tasks — they execute asynchronously in the worker thread
        // TR: 5 görev kuyruğa ekle — worker thread'de asenkron yürütülür
        for (int i = 1; i <= 5; ++i) {
            worker.enqueue([i] {
                std::cout << "    [Worker] Processing task #" << i
                          << " (thread: " << std::this_thread::get_id() << ")\n";
            });
        }

        // EN: Flush ensures all enqueued tasks complete before we proceed
        // TR: Flush, devam etmeden önce tüm kuyruktaki görevlerin tamamlanmasını sağlar
        worker.flush();

        std::cout << "  [Main] All tasks completed. Main thread: "
                  << std::this_thread::get_id() << "\n\n";
    }
```

```cpp

    // ─── Demo 4: Table-Driven State Machine ──────────────────────────────
    {
        std::cout << "--- Demo 4: ECU Table-Driven State Machine ---\n";
        // EN: Initialize the transition table once
        // TR: Geçiş tablosunu bir kez başlat
        init_transition_table();

        // EN: Simulate ECU lifecycle: STARTUP → NORMAL → DIAGNOSTIC → NORMAL → SLEEP → STARTUP
        // TR: ECU yaşam döngüsünü simüle et
        EcuState state = EcuState::STARTUP;
        std::cout << "  Initial state: " << to_string(state) << "\n\n";

        // EN: Event sequence simulating a typical ECU session
        // TR: Tipik bir ECU oturumunu simüle eden olay dizisi
        EcuEvent events[] = {
            EcuEvent::INIT_COMPLETE,   // EN: STARTUP → NORMAL / TR: STARTUP → NORMAL
            EcuEvent::DIAG_REQUEST,    // EN: NORMAL → DIAGNOSTIC / TR: NORMAL → DIAGNOSTIC
            EcuEvent::DIAG_TIMEOUT,    // EN: DIAGNOSTIC → NORMAL / TR: DIAGNOSTIC → NORMAL
            EcuEvent::SLEEP_REQUEST,   // EN: NORMAL → SLEEP / TR: NORMAL → SLEEP
            EcuEvent::WAKEUP,          // EN: SLEEP → STARTUP / TR: SLEEP → STARTUP
            EcuEvent::INIT_COMPLETE,   // EN: STARTUP → NORMAL / TR: STARTUP → NORMAL
        };

        for (auto event : events) {
            std::cout << "  Event: " << to_string(event) << "\n";
            state = fsm_dispatch(state, event);
            std::cout << "  Current state: " << to_string(state) << "\n\n";
        }
    }

    // ─── Demo 5: Hardware Proxy ──────────────────────────────────────────
    {
        std::cout << "--- Demo 5: Hardware Proxy (GPIO Port) ---\n";
        GpioPort gpio;

        // EN: Configure pins 0, 5, 13 as outputs (like on STM32 Nucleo: LED on pin 5)
        // TR: Pin 0, 5, 13'ü çıkış olarak yapılandır (STM32 Nucleo gibi: LED pin 5'te)
        gpio.configure_output(0);
        gpio.configure_output(5);
        gpio.configure_output(13);

        // EN: Set/clear pins like controlling real hardware LEDs
        // TR: Gerçek donanım LED'lerini kontrol eder gibi pin'leri ayarla/temizle
        gpio.set_pin(0);
        gpio.set_pin(5);
        gpio.set_pin(13);
        std::cout << "  After set_pin(0,5,13): ODR = 0x"
                  << std::hex << gpio.read_all() << std::dec << "\n";
        std::cout << "  Pin 5 state: " << (gpio.read_pin(5) ? "HIGH" : "LOW") << "\n";

        gpio.clear_pin(5);
        std::cout << "  After clear_pin(5): ODR = 0x"
                  << std::hex << gpio.read_all() << std::dec << "\n";
        std::cout << "  Pin 5 state: " << (gpio.read_pin(5) ? "HIGH" : "LOW") << "\n";
        std::cout << "\n";
    }

    // ─── Demo 6: Integrated ECU Scenario ─────────────────────────────────
    {
        std::cout << "--- Demo 6: Integrated ECU Scenario ---\n";
        std::cout << "  Combining: MemoryPool + DoubleBuffer + ActiveObject + FSM + GPIO\n\n";

        // EN: Setup components / TR: Bileşenleri hazırla
        MemoryPool<CanMessage, 16> can_pool;
        DoubleBuffer<SensorReading, 2> sensor_buf;
        ActiveObject ecu_task;
        GpioPort status_led;
        status_led.configure_output(5);  // EN: Status LED on pin 5 / TR: Durum LED'i pin 5'te

        init_transition_table();
        EcuState ecu_state = EcuState::STARTUP;

```

```
751         // EN: Phase 1: Init complete → NORMAL, LED ON
752         // TR: Faz 1: Init tamamlandı → NORMAL, LED AÇIK
753         ecu_task.enqueue([&] {
754             ecu_state = fsm_dispatch(ecu_state, EcuEvent::INIT_COMPLETE);
755             status_led.set_pin(5);
756             std::cout << "    [ECU] LED ON — system running\n";
757         });
758 
759         // EN: Phase 2: Receive CAN message, allocate from pool
760         // TR: Faz 2: CAN mesajı al, havuzdan tahsis et
761         ecu_task.enqueue([&] {
762             auto* msg = can_pool.allocate(CanMessage{0x7DF, 3, {0x02, 0x01, 0x0D, 0, 0, 0, 0, 0}});
763             if (msg) std::cout << "    [ECU] Received: " << *msg << "\n";
764         });
765 
766         // EN: Phase 3: Update sensor data via double buffer
767         // TR: Faz 3: Çift tampon ile sensör verisini güncelle
768         ecu_task.enqueue([&] {
769             auto& back = sensor_buf.back();
770             back[0] = {95.2f, 13.8f, 2000};
771             back[1] = {42.1f, 5.0f,  2001};
772             sensor_buf.swap();
773             const auto& front = sensor_buf.front();
774             std::cout << "    [ECU] Engine temp: " << front[0].temperature
775                       << "°C, Battery: " << front[1].voltage << "V\n";
776         });
777 
778         // EN: Phase 4: Sleep request → LED OFF
779         // TR: Faz 4: Uyku isteği → LED KAPALI
780         ecu_task.enqueue([&] {
781             ecu_state = fsm_dispatch(ecu_state, EcuEvent::SLEEP_REQUEST);
782             status_led.clear_pin(5);
783             std::cout << "    [ECU] LED OFF — entering sleep\n";
784         });
785 
786         ecu_task.flush();
787         std::cout << "  Final ECU state: " << to_string(ecu_state) << "\n";
788         std::cout << "  CAN pool active: " << can_pool.active() << "/" << can_pool.capacity() <<
    "\n";
789     }
790 
791     std::cout << "\n==========================================\n";
792     std::cout << "   End of Module 10 — Embedded Patterns\n";
793     std::cout << "==========================================\n";
794 
795     return 0;
796 }
```

Modül 11: Embedded & Real-Time — Gömülü Sistemler

Gömülü Sistemler

5 kaynak dosya · Standart: C++20

C++20

► 01. Volatile Hardware

module_11_embedded_systems/01_volatile_hardware.cpp

Konu: Embedded & Systems: volatile Keyword, Memory-Mapped I/O, ISR Concepts — volatile

Teori / Theory

THEORY: What is volatile?

The `volatile` qualifier tells the compiler: "Do NOT optimize reads or writes to this variable — its value can change at ANY time without the program's knowledge." This happens in three critical situations:
1) Memory-mapped hardware registers (GPIO pins, timers, ADC results)
2) Variables modified by Interrupt Service Routines (ISR)
3) Variables shared with DMA controllers or external hardware Without `volatile`, the compiler may cache the value in a CPU register and never re-read it from RAM — missing hardware changes entirely.

TEORİ: volatile Nedir?

`volatile` niteleyicisi derleyiciye şunu söyler: "Bu değişkene yapılan okumaları veya yazmaları OPTİMİZE ETME — değeri programın bilgisi dışında HERHANGİ bir anda değişebilir." Bu üç kritik durumda olur:
1) Bellek eşlemeli donanım kayıtları (GPIO pinleri, zamanlayıcılar, ADC)
2) Kesme Hizmet Rutinleri (ISR) tarafından değiştirilen değişkenler
3) DMA denetleyicileri veya harici donanım ile paylaşılan değişkenler `volatile` olmadan derleyici değeri CPU kaydında önbelleğe alabilir ve RAM'den asla tekrar okumayabilir — donanım değişikliklerini tamamen kaçırır.

THEORY: Memory-Mapped I/O — Hardware as Memory Addresses

In embedded systems, hardware peripherals are controlled by reading from and writing to specific MEMORY ADDRESSES. Writing `0x01` to address `0x40021000` might enable a GPIO clock. The CPU bus routes these addresses directly to hardware registers instead of RAM. These addresses are always accessed through `volatile` pointers to prevent compiler optimization.

TEORİ: Bellek Eşlemeli G/Ç — Bellek Adresi Olarak Donanım

Gömülü sistemlerde donanım çevre birimleri, belirli BELLEK ADRESLERİNE okuma ve yazma yaparak kontrol edilir. `0x40021000` adresine `0x01` yazmak bir GPIO saatini etkinleştirebilir. CPU veri yolu bu adresleri RAM yerine doğrudan donanım kayıtlarına yönlendirir. Bu adresler, derleyici optimizasyonunu önlemek için her zaman `volatile` işaretçiler aracılığıyla erişilir.

THEORY: Interrupt Service Routines (ISR)

An ISR is a function called by the hardware (CPU interrupt controller) when an external event occurs (button press, timer overflow, UART data received). ISRs preempt the main program at any instruction boundary. ISR rules:
1. Keep it SHORT.
2. No heap allocation.
3. No blocking calls.
4. Use `volatile` for shared variables.
5. Consider atomic types for multi-byte values.

TEORİ: Kesme Hizmet Rutinleri

ISR, harici bir olay gerçekleştiğinde (düğme basma, zamanlayıcı taşması, UART veri alma) donanım (CPU kesme denetleyicisi) tarafından çağrılan fonksiyondur. ISR'lar ana programı herhangi bir komut sınırında keser. ISR kuralları:
1. KISA tut.
2. Heap tahsisi yok.
3. Blokleyici çağrı yok.
4. Paylaşılan değişkenler için `volatile` kullan.
5. Çok baytlı değerler için atomik tipler düşün.

CPPREF Depth: volatile Does NOT Mean Thread-Safe!

A critical misconception: `volatile` in C++ does NOT provide atomicity or memory ordering guarantees. It prevents compiler optimization but says NOTHING about CPU cache coherency or instruction reordering. For multi-threaded code, use `std::atomic<T>` instead. `volatile` is ONLY for hardware I/O and ISR communication. In Java/C#, `volatile` has different (stronger) semantics — don't confuse them! cppreference.com/w/cpp/language/cv

CPPREF Derinlik: volatile Thread-Güvenli Demek DEĞİLDİR!

Kritik bir yanlış anlama: C++'ta `volatile` atomiklik veya bellek sıralama garantisi SAĞLAMAZ. Derleyici optimizasyonunu önler ama CPU önbellek tutarlılığı veya komut yeniden sıralama hakkında HİÇBİR ŞEY söylemez. Çoklu-thread kod için bunun yerine `std::atomic<T>` kullanın. `volatile` YALNIZCA donanım G/Ç ve ISR iletişimi içindir. Java/C#'ta `volatile` farklı (daha güçlü) anlamlara sahiptir — karıştırmayın! cppreference.com/w/cpp/language/cv

CPPREF Depth: volatile and const Can Coexist

`const volatile int* reg = (const volatile int*)0x40000000;` means: "I promise not to WRITE to this register, but its value can still CHANGE due to hardware." This is common for read-only status registers (e.g., ADC result, timer counter). The compiler won't optimize away reads, and the programmer cannot accidentally write to it. cppreference.com/w/cpp/language/cv

CPPREF Derinlik: volatile ve const Bir Arada Olabilir

`const volatile int* reg = (const volatile int*)0x40000000;` şu anlama gelir: "Bu kayda YAZMA sözü veriyorum, ama değeri donanım nedeniyle hâlâ DEĞİŞEBİLİR." Bu, salt okunur durum kayıtları (ADC sonucu, zamanlayıcı sayacı) için yaygındır. Derleyici okumaları optimize etmez ve programcı yanlışlıkla yazamaz. cppreference.com/w/cpp/language/cv

Kaynak Kod / Source Code

```cpp
#include <cstdint>
#include <iostream>

// ═══════════════════════════════════════════════════════════════════════════════
// [DEMO 1: volatile Prevents Compiler Optimization]
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Without volatile, the compiler could optimize this to "while(true)" because it sees no
// code path that changes 'flag'.
// TR: volatile olmadan derleyici bunu "while(true)" olarak optimize edebilir çünkü 'flag'ı
// değiştiren hiçbir kod yolu görmez.
volatile bool hardwareFlag = false;

void simulateISR() {
  // EN: Simulated ISR — in real embedded, this would be triggered by hardware.
  // TR: Simüle ISR — gerçek gömülüde bu donanım tarafından tetiklenirdi.
  hardwareFlag = true;
  std::cout << "[ISR] Hardware flag SET to true." << std::endl;
}

// ═══════════════════════════════════════════════════════════════════════════════
// [DEMO 2: Memory-Mapped I/O Simulation]
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Simulate memory-mapped hardware registers.
// TR: Bellek eşlemeli donanım kayıtlarını simüle et.
struct GPIO_Registers {
  volatile uint32_t MODER;   // Mode register
  volatile uint32_t ODR;     // Output data register
  volatile uint32_t IDR;     // Input data register (read-only in real HW)
  volatile uint32_t BSRR;   // Bit set/reset register
};

// EN: In real embedded, this would be a hardware address like 0x40020000.
// TR: Gerçek gömülüde bu 0x40020000 gibi bir donanım adresi olurdu.
GPIO_Registers simulated_gpio = {0, 0, 0, 0};

void configureGPIO() {
  // EN: Set pin 5 to output mode (bits 11:10 = 01 for output).
  // TR: Pin 5'i çıkış moduna ayarla (bitler 11:10 = 01 çıkış için).
  simulated_gpio.MODER |= (1U << 10);
  simulated_gpio.MODER &= ~(1U << 11);
  std::cout << "[GPIO] Pin 5 configured as OUTPUT. MODER=0x"
            << std::hex << simulated_gpio.MODER << std::dec << std::endl;
}

void setGPIOPin(uint32_t pin) {
  // EN: BSRR: writing to lower 16 bits SETS, upper 16 bits RESETS.
  // TR: BSRR: alt 16 bite yazma AYARLAR, üst 16 bit SIFIRLAR.
  simulated_gpio.BSRR = (1U << pin);
  simulated_gpio.ODR |= (1U << pin);
  std::cout << "[GPIO] Pin " << pin << " SET HIGH. ODR=0x"
            << std::hex << simulated_gpio.ODR << std::dec << std::endl;
}

void resetGPIOPin(uint32_t pin) {
  simulated_gpio.BSRR = (1U << (pin + 16U));
  simulated_gpio.ODR &= ~(1U << pin);
  std::cout << "[GPIO] Pin " << pin << " SET LOW. ODR=0x"
            << std::hex << simulated_gpio.ODR << std::dec << std::endl;
}
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════════
// [DEMO 3: ISR-Safe Shared Variable Pattern]
// ═══════════════════════════════════════════════════════════════════════════════

// EN: volatile + flag pattern for ISR-main communication.
// TR: ISR-ana program iletişimi için volatile + bayrak kalıbı.
volatile uint32_t adcResult = 0;
volatile bool adcReady = false;

void simulateADC_ISR() {
  // EN: Simulates ADC conversion complete interrupt.
  // TR: ADC dönüşüm tamamlandı kesmesini simüle eder.
  adcResult = 3300; // 3.3V in millivolts
  adcReady = true;
  std::cout << "[ADC ISR] Conversion complete: " << adcResult << " mV"
            << std::endl;
}

// ═══════════════════════════════════════════════════════════════════════════════
// [DEMO 4: const volatile — Read-Only Hardware Register]
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Simulating a read-only hardware status register.
// TR: Salt okunur donanım durum kaydını simüle etme.
volatile uint32_t hwStatusReg = 0x0000'00FF;

int main() {
  std::cout << "=== MODULE 11: VOLATILE & HARDWARE I/O ===\n" << std::endl;

  // --- Demo 1: volatile flag ---
  std::cout << "--- DEMO 1: volatile Flag ---\n" << std::endl;
  std::cout << "[Main] Flag before ISR: " << std::boolalpha << hardwareFlag
            << std::endl;
  simulateISR();
  if (hardwareFlag) {
    std::cout << "[Main] Flag detected as TRUE — ISR communication works!\n"
              << std::endl;
  }

  // --- Demo 2: Memory-Mapped GPIO ---
  std::cout << "--- DEMO 2: Memory-Mapped GPIO Simulation ---\n" << std::endl;
  configureGPIO();
  setGPIOPin(5);  // Turn on LED
  setGPIOPin(13); // Turn on buzzer
  resetGPIOPin(5); // Turn off LED
  std::cout << std::endl;

  // --- Demo 3: ISR-Safe ADC Pattern ---
  std::cout << "--- DEMO 3: ISR-Safe ADC Read ---\n" << std::endl;
  std::cout << "[Main] Waiting for ADC..." << std::endl;
  simulateADC_ISR(); // In real code, this happens asynchronously
  if (adcReady) {
    uint32_t voltage = adcResult;
    adcReady = false;
    std::cout << "[Main] Read ADC: " << voltage << " mV (" << (voltage / 1000.0)
              << " V)\n"
              << std::endl;
  }

  // --- Demo 4: const volatile ---
  std::cout << "--- DEMO 4: const volatile (Read-Only Register) ---\n"
            << std::endl;
  const volatile uint32_t *statusReg = &hwStatusReg;
  // *statusReg = 0; // COMPILE ERROR: cannot write to const volatile!
  std::cout << "[Main] Status register value: 0x" << std::hex << *statusReg
            << std::dec << std::endl;
  std::cout << "=> const volatile: compiler re-reads every time, but "
               "writing is forbidden at compile time."
            << std::endl;

  std::cout << "\n=> volatile, Memory-Mapped I/O, and ISR patterns "
               "demonstrated!" << std::endl;

  return 0;
}
```

► 02. Bitfields Protocols

module_11_embedded_systems/02_bitfields_protocols.cpp

Konu: Embedded & Systems: Bitfields, Protocol Parsing, Packed Structs — Bit Alanları,

Teori / Theory

THEORY: Bitfields — Packing Data into Minimal Space

C++ bitfields allow struct members to occupy a specific number of BITS instead of whole bytes. `uint8_t flag : 1;` uses exactly 1 bit. This is critical in embedded systems where every byte matters: network protocol headers, hardware register layouts, command packets over CAN/SPI/I2C buses. A 32-bit register might contain 8 different fields packed into one word.

TEORİ: Bit Alanları — Veriyi Minimum Alana Paketleme

C++ bit alanları, yapı üyelerinin tam baytlar yerine belirli sayıda BİT kaplamasına izin verir. `uint8_t flag : 1;` tam olarak 1 bit kullanır. Bu, her baytın önemli olduğu gömülü sistemlerde kritiktir: ağ protokolü başlıkları, donanım kayıt düzenleri, CAN/SPI/I2C veri yolları üzerinden komut paketleri. Bir 32-bit kayıt, tek bir kelimeye paketlenmiş 8 farklı alan içerebilir.

THEORY: Protocol Parsing with Structs

Network protocols (TCP/IP, CAN, USB) define exact byte layouts for packets. In C++, we can overlay a struct onto raw byte buffers to parse them directly — zero-copy parsing! The struct fields map directly to protocol fields. This is dramatically faster than parsing byte-by-byte. Requirements: `#pragma pack(1)` or `__attribute__((packed))` to disable compiler padding.

TEORİ: Yapılarla Protokol Ayrıştırma

Ağ protokolleri (TCP/IP, CAN, USB) paketler için kesin bayt düzenleri tanımlar. C++'ta ham bayt tamponlarına bir yapı bindirerek bunları doğrudan ayrıştırabiliriz — sıfır-kopya ayrıştırma! Yapı alanları doğrudan protokol alanlarına eşlenir. Bu, bayt bayt ayrıştırmadan dramatik şekilde hızlıdır. Gereksinimler: derleyici dolgusunu devre dışı bırakmak için `#pragma pack(1)` veya `__attribute__((packed))`.

THEORY: Endianness — Byte Order Matters

CPUs store multi-byte values in either "Little-Endian" (LSB first, x86, ARM) or "Big-Endian" (MSB first, network order). When parsing network protocols, you MUST convert between network byte order (big-endian) and host byte order using `htons()/ntohs()` (16-bit) and `htonl()/ntohl()` (32-bit). Ignoring endianness corrupts multi-byte fields across platforms.

TEORİ: Endianness — Bayt Sırası Önemlidir

CPU'lar çok baytlı değerleri "Little-Endian" (LSB önce, x86, ARM) veya "Big-Endian" (MSB önce, ağ sırası) olarak depolar. Ağ protokollerini ayrıştırırken, ağ bayt sırası (big-endian) ile ana makine bayt sırası arasında `htons()/ntohs()` (16-bit) ve `htonl()/ntohl()` (32-bit) kullanarak dönüştürmeniz ZORUNLUDUR. Endianness'ı görmezden gelmek platformlar arasında çok baytlı alanları bozar.

CPPREF Depth: Implementation-Defined Behavior of Bitfields

The C++ standard leaves several bitfield aspects implementation-defined:
1. Layout ordering (MSB-first or LSB-first within a byte).
2. Whether bitfields can span storage unit boundaries.
3. Alignment of bitfield storage units. This means bitfield layout is NOT portable across compilers! For cross-platform protocol parsing, prefer explicit bit manipulation (`& mask`, `>> shift`) over bitfields.
⚠ cppreference.com/w/cpp/language/bit_field

CPPREF Derinlik: Bit Alanlarının Uygulamaya Bağlı Davranışı

C++ standardı birçok bit alanı yönünü uygulamaya bağlı bırakır:
1. Düzen sıralaması (bir bayt içinde MSB-önce veya LSB-önce).
2. Bit alanlarının depolama birimi sınırlarını geçip geçemeyeceği.
3. Bit alanı depolama birimlerinin hizalaması. Bu, bit alanı düzeninin derleyiciler arasında TAŞINABİLİR OLMADIĞI anlamına gelir! Platformlar arası protokol ayrıştırma için bit alanları yerine açık bit manipülasyonu (`& maske`, `>> kaydırma`) tercih edin.
⚠ cppreference.com/w/cpp/language/bit_field

CPPREF Depth: std::byte and Type-Safe Byte Manipulation (C++17)

C++17 introduced `std::byte` as a distinct type for raw byte manipulation. Unlike `char` or `unsigned char`, `std::byte` only supports bitwise operations — no arithmetic! This prevents accidentally treating raw bytes as numbers. `std::byte b{0xFF}; auto v = std::to_integer<uint8_t>(b);` cppreference.com/w/cpp/types/byte

CPPREF Derinlik: std::byte ve Tip-Güvenli Bayt Manipülasyonu (C++17)

C++17, ham bayt manipülasyonu için ayrı bir tip olarak `std::byte`'ı tanıttı. `char` veya `unsigned char`'dan farklı olarak `std::byte` yalnızca bitsel işlemleri destekler — aritmetik yok! Bu, ham baytların yanlışlıkla sayı olarak işlenmesini önler. `std::byte b{0xFF}; auto v = std::to_integer<uint8_t>(b);` cppreference.com/w/cpp/types/byte

Kaynak Kod / Source Code

```cpp
#include <cstdint>
#include <cstring>
#include <iomanip>
#include <iostream>

// ═══════════════════════════════════════════════════════════════════════════
// [1. BITFIELD DEMO — Hardware Register Layout]
// ═══════════════════════════════════════════════════════════════════════════

// EN: Simulates a UART control register with bitfields.
// TR: Bit alanlarıyla bir UART kontrol kaydını simüle eder.
struct UART_CR {
  uint32_t UE : 1;     // UART Enable
  uint32_t RE : 1;     // Receiver Enable
  uint32_t TE : 1;     // Transmitter Enable
  uint32_t IDLEIE : 1; // IDLE Interrupt Enable
  uint32_t RXNEIE : 1; // RX Not Empty Interrupt Enable
  uint32_t TCIE : 1;   // Transmission Complete Interrupt
  uint32_t TXEIE : 1;  // TX Empty Interrupt
  uint32_t PEIE : 1;   // Parity Error Interrupt
  uint32_t PS : 1;     // Parity Selection (0=Even, 1=Odd)
  uint32_t PCE : 1;    // Parity Control Enable
  uint32_t reserved : 22; // Remaining bits
};
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════════
// [2. PACKED STRUCT — Protocol Header (CAN Bus Frame)]
// ═══════════════════════════════════════════════════════════════════════════════

// EN: #pragma pack(1) prevents compiler from inserting padding bytes.
// TR: #pragma pack(1) derleyicinin dolgu baytları eklemesini önler.
#pragma pack(push, 1)
struct CANFrame {
  uint32_t id : 11;     // CAN Standard ID (11 bits)
  uint32_t rtr : 1;     // Remote Transmission Request
  uint32_t ide : 1;     // Identifier Extension
  uint32_t dlc : 4;     // Data Length Code (0-8)
  uint32_t reserved : 15;
  uint8_t data[8];      // Payload (max 8 bytes)
};
#pragma pack(pop)

// ═══════════════════════════════════════════════════════════════════════════════
// [3. MANUAL BIT MANIPULATION — Portable Protocol Parsing]
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Explicit bit manipulation is portable across all compilers.
// TR: Açık bit manipülasyonu tüm derleyicilerde taşınabilirdir.
struct SensorPacket {
  uint8_t raw[4]; // 4 bytes of raw data

  // EN: Extract sensor ID (bits 31:24 of first word)
  // TR: Sensör kimliği çıkar (ilk kelimenin bitleri 31:24)
  uint8_t getSensorId() const { return raw[0]; }

  // EN: Extract temperature (bits 23:12, 12-bit signed value)
  // TR: Sıcaklık çıkar (bitler 23:12, 12-bit işaretli değer)
  int16_t getTemperature() const {
    uint16_t val =
        static_cast<uint16_t>((static_cast<uint16_t>(raw[1]) << 4) |
                              (static_cast<uint16_t>(raw[2]) >> 4));
    // EN: Sign extend 12-bit to 16-bit
    // TR: 12-bit'i 16-bit'e işaret genişlet
    if (val & 0x800U) val |= 0xF000U;
    return static_cast<int16_t>(val);
  }

  // EN: Extract battery level (bits 11:4, 8-bit value, 0-255)
  // TR: Pil seviyesi çıkar (bitler 11:4, 8-bit değer, 0-255)
  uint8_t getBatteryLevel() const {
    return static_cast<uint8_t>(((raw[2] & 0x0FU) << 4) |
                                (raw[3] >> 4));
  }

  // EN: Extract alert flags (bits 3:0)
  // TR: Uyarı bayrakları çıkar (bitler 3:0)
  uint8_t getAlertFlags() const {
    return static_cast<uint8_t>(raw[3] & 0x0FU);
  }
};

// ═══════════════════════════════════════════════════════════════════════════════
// [4. ENDIANNESS DEMONSTRATION]
// ═══════════════════════════════════════════════════════════════════════════════

bool isLittleEndian() {
  uint16_t val = 0x0001;
  // EN: If first byte is 0x01, the LSB is stored first (little-endian).
  // TR: İlk bayt 0x01 ise, LSB önce depolanır (little-endian).
  uint8_t firstByte = 0;
  std::memcpy(&firstByte, &val, 1);
  return firstByte == 0x01;
}

uint16_t swapBytes16(uint16_t val) {
  return static_cast<uint16_t>((val >> 8) | (val << 8));
}
```

```cpp

uint32_t swapBytes32(uint32_t val) {
  return ((val >> 24) & 0xFFU) | ((val >> 8) & 0xFF00U) |
         ((val << 8) & 0xFF0000U) | ((val << 24) & 0xFF000000U);
}

int main() {
  std::cout << "=== MODULE 11: BITFIELDS & PROTOCOL PARSING ===\n" << std::endl;

  // --- Demo 1: Bitfield Register ---
  std::cout << "--- DEMO 1: UART Control Register (Bitfield) ---\n" << std::endl;
  UART_CR uart{};
  uart.UE = 1;    // Enable UART
  uart.RE = 1;    // Enable receiver
  uart.TE = 1;    // Enable transmitter
  uart.RXNEIE = 1; // Enable RX interrupt

  std::cout << "UART Enable:    " << uart.UE << std::endl;
  std::cout << "Receiver:       " << uart.RE << std::endl;
  std::cout << "Transmitter:    " << uart.TE << std::endl;
  std::cout << "RX Interrupt:   " << uart.RXNEIE << std::endl;
  std::cout << "sizeof(UART_CR)=" << sizeof(UART_CR) << " bytes\n" << std::endl;

  // --- Demo 2: CAN Frame ---
  std::cout << "--- DEMO 2: CAN Bus Frame (Packed Struct) ---\n" << std::endl;
  CANFrame frame{};
  frame.id = 0x123;    // Standard CAN ID
  frame.rtr = 0;       // Data frame (not remote)
  frame.ide = 0;       // Standard (not extended)
  frame.dlc = 3;       // 3 bytes of data
  frame.data[0] = 0xAA;
  frame.data[1] = 0xBB;
  frame.data[2] = 0xCC;

  std::cout << "CAN ID:   0x" << std::hex << frame.id << std::endl;
  std::cout << "DLC:      " << std::dec << frame.dlc << std::endl;
  std::cout << "Data:     ";
  for (int i = 0; i < frame.dlc; i++) {
    std::cout << "0x" << std::hex << std::setw(2) << std::setfill('0')
              << static_cast<int>(frame.data[i]) << " ";
  }
  std::cout << std::dec << "\nsizeof(CANFrame)=" << sizeof(CANFrame)
            << " bytes\n"
            << std::endl;

  // --- Demo 3: Manual Bit Parsing ---
  std::cout << "--- DEMO 3: Portable Bit Manipulation ---\n" << std::endl;
  SensorPacket packet{};
  // EN: Sensor ID=0x42, Temp=+25 (0x019), Battery=200, Alerts=0x05
  // TR: Sensör ID=0x42, Sıcaklık=+25 (0x019), Pil=200, Uyarılar=0x05
  packet.raw[0] = 0x42;
  packet.raw[1] = 0x01;
  packet.raw[2] = 0x9C;
  packet.raw[3] = 0x85;

  std::cout << "Sensor ID:     0x" << std::hex
            << static_cast<int>(packet.getSensorId()) << std::dec << std::endl;
  std::cout << "Temperature:   " << packet.getTemperature() << " C" << std::endl;
  std::cout << "Battery:       " << static_cast<int>(packet.getBatteryLevel())
            << "/255" << std::endl;
  std::cout << "Alert Flags:   0x" << std::hex
            << static_cast<int>(packet.getAlertFlags()) << std::dec << "\n"
            << std::endl;

```

```
162     // --- Demo 4: Endianness ---
163     std::cout << "--- DEMO 4: Endianness ---\n" << std::endl;
164     std::cout << "System is: "
165               << (isLittleEndian() ? "Little-Endian" : "Big-Endian") << std::endl;
166     uint16_t val16 = 0x1234;
167     uint32_t val32 = 0xDEADBEEF;
168     std::cout << "0x" << std::hex << val16 << " swapped = 0x"
169               << swapBytes16(val16) << std::endl;
170     std::cout << "0x" << val32 << " swapped = 0x" << swapBytes32(val32)
171               << std::dec << std::endl;
172
173     std::cout << "\n=> Bitfields, packed structs, protocol parsing, and "
174                  "endianness demonstrated!" << std::endl;
175
176     return 0;
177   }
```

► 03. Embedded Constraints

module_11_embedded_systems/03_embedded_constraints.cpp

Konu: Embedded & Systems: No-Heap Allocation, Static Buffers, constexpr, Placement New /

Teori / Theory

THEORY: No-Heap Programming — Why malloc/new Are Banned

In safety-critical embedded systems (automotive, medical, aerospace), dynamic memory allocation (malloc/new) is BANNED because:
1. Heap fragmentation can exhaust memory unpredictably.
2. Allocation time is non-deterministic — bad for real-time systems.
3. Heap metadata overhead wastes scarce RAM.
4. Memory leaks in long-running firmware are catastrophic. Instead, all memory is statically allocated at compile time using fixed-size arrays, stack variables, and memory pools (placement new).

TEORİ: Yığınsız Programlama — malloc/new Neden Yasaklıdır

Güvenlik-kritik gömülü sistemlerde (otomotiv, tıbbi, havacılık) dinamik bellek tahsisi (malloc/new) YASAKLIDIR çünkü:
1. Yığın parçalanması belleği öngörülemez şekilde tüketebilir.
2. Tahsis süresi belirleyici değildir — gerçek zamanlı sistemler icin kötü.
3. Yığın meta veri ek yükü kıt RAM'i israf eder.
4. Uzun süre çalışan donanım yazılımında bellek sızıntıları felakettir. Bunun yerine, tüm bellek derleme zamanında sabit boyutlu diziler, yığıt değişkenleri ve bellek havuzları (placement new) kullanılarak statik olarak tahsis edilir.

THEORY: constexpr — Compile-Time Computation

`constexpr` forces computation to happen at compile time, producing zero runtime overhead. In embedded, this means:
1. CRC/checksum lookup tables computed at compile time.
2. Baud rate divisor calculations resolved before flashing.
3. Trigonometric lookup tables baked into firmware ROM. C++17 allows `constexpr if` for compile-time branching, and C++20 allows `consteval` to GUARANTEE compile-time-only execution.

TEORİ: constexpr — Derleme Zamanı Hesaplama

`constexpr` hesaplamayı derleme zamanında gerçekleşmeye zorlar, sıfır çalışma zamanı ek yükü üretir. Gömülü sistemlerde bu şu anlama gelir:
1. CRC/sağlama toplamı arama tabloları derleme zamanında hesaplanır.
2. Baud hızı bölücü hesaplamaları flaşlamadan önce çözümlenir.
3. Trigonometrik arama tabloları donanım yazılımı ROM'una gömülür. C++17 derleme zamanı dallanma için `constexpr if`'e izin verir ve C++20 yalnızca derleme zamanı yürütmeyi GARANTİ etmek için `consteval`'e izin verir.

THEORY: Placement New — Object Construction in Pre-Allocated Memory

`placement new` constructs an object at a specific memory address without allocating. Syntax: `new (address) Type(args);` This is essential for:
1. Memory pools with fixed-size blocks.
2. DMA buffer management.
3. Constructing objects on stack-allocated aligned buffers. You must call the destructor manually: `obj->~Type();`

TEORİ: Placement New — Önceden Tahsis Edilmiş Bellekte Nesne Oluşturma

`placement new` tahsis yapmadan belirli bir bellek adresinde nesne oluşturur. Sözdizimi: `new (adres) Tip(argümanlar);` Bu, şunlar için gereklidir:
1. Sabit boyutlu bloklara sahip bellek havuzları.
2. DMA tampon yönetimi.
3. Yığıt-tahsisli hizalanmış tamponlarda nesne oluşturma. Yıkıcıyı manuel olarak çağırmanız gerekir: `obj->~Type();`

CPPREF Depth: constexpr Specifier Rules and Limits

A `constexpr` function must:
1. Have a literal return type.
2. Not contain goto, asm, or non-literal variables (pre-C++20). C++14 relaxed most restrictions (loops, multiple statements). C++20 allows virtual constexpr, try-catch, dynamic_cast. `consteval` (C++20) guarantees the function is ONLY called at compile time — if forced at runtime, it's a compilation error. `constinit` ensures static storage duration variables are initialized at compile time.
⚠ cppreference.com/w/cpp/language/constexpr

CPPREF Derinlik: constexpr Belirleyici Kuralları ve Sınırları

Bir `constexpr` fonksiyon:
1. Literal dönüş tipine sahip olmalı.
2. goto, asm veya literal olmayan değişkenler içermemeli (C++20 öncesi). C++14 çoğu kısıtlamayı gevşetti (döngüler, çoklu ifadeler). C++20 sanal constexpr, try-catch, dynamic_cast'e izin verir. `consteval` (C++20) fonksiyonun YALNIZCA derleme zamanında çağrıldığını garanti eder. `constinit` statik depolama süresine sahip değişkenlerin derleme zamanında başlatılmasını sağlar.
⚠ cppreference.com/w/cpp/language/constexpr

CPPREF Depth: Placement New and Object Lifetime

Placement new does NOT allocate memory — it only calls the constructor at a given address. The programmer is responsible for:
1. Ensuring proper alignment via `alignas` or `std::aligned_storage`.
2. Manually calling the destructor before reusing memory.
3. NOT calling `delete` on placement-new objects. `std::launder` (C++17) is needed after placement new to access the object through old pointers when the type has const/reference members.
⚠ cppreference.com/w/cpp/language/new#Placement_new

CPPREF Derinlik: Placement New ve Nesne Yaşam Süresi

Placement new bellek TAHSİS ETMEZ — yalnızca verilen adreste yapıcıyı çağırır. Programcı şunlardan sorumludur:
1. `alignas` veya `std::aligned_storage` ile uygun hizalamayı sağlama.
2. Belleği yeniden kullanmadan önce yıkıcıyı manuel çağırma.
3. Placement-new nesnelerinde `delete` ÇAĞIRMAMAK. `std::launder` (C++17) const/referans üyeleri olan türlerde eski işaretçiler aracılığıyla nesneye erişmek için placement new sonrası gereklidir.
⚠ cppreference.com/w/cpp/language/new#Placement_new

Kaynak Kod / Source Code

```cpp
#include <cstdint>
#include <cstring>
#include <iostream>
#include <new>      // placement new
#include <array>
#include <type_traits>

// ═══════════════════════════════════════════════════════════════════════════
// [1. STATIC RING BUFFER — No-Heap Circular Buffer]
// ═══════════════════════════════════════════════════════════════════════════

// EN: A fixed-size ring buffer using no dynamic allocation.
// TR: Dinamik tahsis kullanmayan sabit boyutlu halka tampon.
template <typename T, std::size_t N>
class StaticRingBuffer {
  std::array<T, N> buffer_{};
  std::size_t head_ = 0;
  std::size_t tail_ = 0;
  std::size_t count_ = 0;

public:
  bool push(const T& item) {
    if (count_ == N) return false; // Full
    buffer_[tail_] = item;
    tail_ = (tail_ + 1) % N;
    ++count_;
    return true;
  }

  bool pop(T& item) {
    if (count_ == 0) return false; // Empty
    item = buffer_[head_];
    head_ = (head_ + 1) % N;
    --count_;
    return true;
  }

  [[nodiscard]] std::size_t size() const { return count_; }
  [[nodiscard]] bool empty() const { return count_ == 0; }
  [[nodiscard]] bool full() const { return count_ == N; }
};
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════════
// [2. CONSTEXPR — Compile-Time CRC-8 Table]
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Generate a CRC-8 lookup table entirely at compile time.
// TR: CRC-8 arama tablosunu tamamen derleme zamanında oluştur.
constexpr uint8_t CRC8_POLY = 0x07;

constexpr std::array<uint8_t, 256> generateCRC8Table() {
  std::array<uint8_t, 256> table{};
  for (int i = 0; i < 256; ++i) {
    uint8_t crc = static_cast<uint8_t>(i);
    for (int bit = 0; bit < 8; ++bit) {
      if (crc & 0x80U) {
        crc = static_cast<uint8_t>((crc << 1) ^ CRC8_POLY);
      } else {
        crc = static_cast<uint8_t>(crc << 1);
      }
    }
    table[static_cast<std::size_t>(i)] = crc;
  }
  return table;
}

// EN: Table is fully resolved at compile time — zero runtime cost.
// TR: Tablo tamamen derleme zamanında çözümlenir — sıfır çalışma zamanı maliyeti.
constexpr auto CRC8_TABLE = generateCRC8Table();

constexpr uint8_t computeCRC8(const uint8_t* data, std::size_t len) {
  uint8_t crc = 0x00;
  for (std::size_t i = 0; i < len; ++i) {
    crc = CRC8_TABLE[static_cast<std::size_t>(crc ^ data[i])];
  }
  return crc;
}

// ═══════════════════════════════════════════════════════════════════════════════
// [3. CONSTEXPR — Baud Rate Divisor]
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Compute UART baud rate divisor at compile time.
// TR: UART baud hızı bölücüsünü derleme zamanında hesapla.
constexpr uint32_t computeBaudDivisor(uint32_t clockHz, uint32_t baudRate) {
  return (clockHz + baudRate / 2) / baudRate; // Rounded division
}

// EN: Resolved at compile time — no runtime division needed.
// TR: Derleme zamanında çözümlenir — çalışma zamanı bölmeye gerek yok.
constexpr uint32_t UART_DIVISOR_115200 = computeBaudDivisor(16000000, 115200);
constexpr uint32_t UART_DIVISOR_9600   = computeBaudDivisor(16000000, 9600);

// ═══════════════════════════════════════════════════════════════════════════════
// [4. PLACEMENT NEW — Memory Pool]
// ═══════════════════════════════════════════════════════════════════════════════

// EN: A simple fixed-size memory pool using placement new.
// TR: Placement new kullanan basit sabit boyutlu bellek havuzu.
struct Sensor {
  uint16_t id;
  float value;
  bool active;

  Sensor(uint16_t sensorId, float val)
      : id(sensorId), value(val), active(true) {}

  void print() const {
    std::cout << "  Sensor[" << id << "] value=" << value
              << " active=" << active << std::endl;
  }

  ~Sensor() {
    std::cout << "  ~Sensor[" << id << "] destroyed" << std::endl;
  }
};
```

```cpp

// EN: Pool with compile-time capacity, no heap allocation.
// TR: Derleme zamanı kapasiteli havuz, yığın tahsisi yok.
template <typename T, std::size_t MaxObjects>
class StaticPool {
  // EN: Aligned raw storage for objects.
  // TR: Nesneler için hizalanmış ham depolama.
  alignas(T) uint8_t storage_[MaxObjects * sizeof(T)]{};
  bool occupied_[MaxObjects]{};

public:
  template <typename... Args>
  T* allocate(Args&&... args) {
    for (std::size_t i = 0; i < MaxObjects; ++i) {
      if (!occupied_[i]) {
        occupied_[i] = true;
        T* ptr = reinterpret_cast<T*>(&storage_[i * sizeof(T)]);
        new (ptr) T(std::forward<Args>(args)...); // placement new
        return ptr;
      }
    }
    return nullptr; // Pool exhausted
  }

  void deallocate(T* ptr) {
    auto addr = reinterpret_cast<uint8_t*>(ptr);
    auto base = &storage_[0];
    if (addr < base || addr >= base + MaxObjects * sizeof(T)) return;

    std::size_t idx = static_cast<std::size_t>(addr - base) / sizeof(T);
    if (occupied_[idx]) {
      ptr->~T(); // Manual destructor call
      occupied_[idx] = false;
    }
  }

  [[nodiscard]] std::size_t usedCount() const {
    std::size_t count = 0;
    for (std::size_t i = 0; i < MaxObjects; ++i) {
      if (occupied_[i]) ++count;
    }
    return count;
  }
};

// ═══════════════════════════════════════════════════════════════════════════════
// [5. STATIC LOOKUP TABLE — Compile-Time Sine Approximation]
// ═══════════════════════════════════════════════════════════════════════════════

constexpr double PI = 3.14159265358979323846;

constexpr double constexprSin(double x) {
  // EN: Taylor series sin(x) = x - x^3/3! + x^5/5! - x^7/7! + ...
  // TR: Taylor serisi sin(x) = x - x^3/3! + x^5/5! - x^7/7! + ...
  double result = 0.0;
  double term = x;
  for (int n = 1; n <= 15; n += 2) {
    result += term;
    term *= -x * x / static_cast<double>((n + 1) * (n + 2));
  }
  return result;
}

// EN: 16-entry sine table, built at compile time.
// TR: Derleme zamanında oluşturulan 16 girişli sinüs tablosu.
constexpr std::array<double, 16> generateSineTable() {
  std::array<double, 16> table{};
  for (int i = 0; i < 16; ++i) {
    table[static_cast<std::size_t>(i)] =
        constexprSin(2.0 * PI * static_cast<double>(i) / 16.0);
  }
  return table;
}

```

```cpp
constexpr auto SINE_TABLE = generateSineTable();

int main() {
  std::cout << "=== MODULE 11: EMBEDDED CONSTRAINTS ===\n" << std::endl;

  // --- Demo 1: Static Ring Buffer ---
  std::cout << "--- DEMO 1: Static Ring Buffer (No Heap) ---\n" << std::endl;
  StaticRingBuffer<int, 4> rb;
  rb.push(10);
  rb.push(20);
  rb.push(30);
  rb.push(40);
  std::cout << "Buffer full? " << (rb.full() ? "YES" : "NO") << std::endl;
  std::cout << "Push(50) success? " << (rb.push(50) ? "YES" : "NO") << std::endl;

  int val = 0;
  rb.pop(val);
  std::cout << "Popped: " << val << " | Size: " << rb.size() << "\n" << std::endl;

  // --- Demo 2: Compile-Time CRC-8 ---
  std::cout << "--- DEMO 2: Constexpr CRC-8 Table ---\n" << std::endl;
  const uint8_t testData[] = {0x01, 0x02, 0x03, 0x04, 0x05};
  uint8_t crc = computeCRC8(testData, 5);
  std::cout << "CRC-8 of {01,02,03,04,05} = 0x" << std::hex
            << static_cast<int>(crc) << std::dec << std::endl;
  std::cout << "CRC table[0x00]=" << static_cast<int>(CRC8_TABLE[0])
            << " table[0xFF]=" << static_cast<int>(CRC8_TABLE[255])
            << "\n" << std::endl;

  // --- Demo 3: Baud Rate Divisor ---
  std::cout << "--- DEMO 3: Compile-Time Baud Divisor ---\n" << std::endl;
  std::cout << "16MHz / 115200 baud => divisor=" << UART_DIVISOR_115200
            << std::endl;
  std::cout << "16MHz / 9600 baud   => divisor=" << UART_DIVISOR_9600
            << "\n" << std::endl;

  // --- Demo 4: Placement New Memory Pool ---
  std::cout << "--- DEMO 4: Static Memory Pool (Placement New) ---\n"
            << std::endl;
  StaticPool<Sensor, 4> pool;
  Sensor* s1 = pool.allocate(static_cast<uint16_t>(1), 23.5f);
  Sensor* s2 = pool.allocate(static_cast<uint16_t>(2), 45.1f);
  Sensor* s3 = pool.allocate(static_cast<uint16_t>(3), 67.8f);
  std::cout << "Pool used: " << pool.usedCount() << "/4" << std::endl;
  if (s1) s1->print();
  if (s2) s2->print();
  if (s3) s3->print();

  std::cout << "\nDeallocating sensor 2..." << std::endl;
  pool.deallocate(s2);
  std::cout << "Pool used: " << pool.usedCount() << "/4" << std::endl;

  Sensor* s4 = pool.allocate(static_cast<uint16_t>(4), 99.9f);
  if (s4) s4->print();
  std::cout << "Pool used: " << pool.usedCount() << "/4" << std::endl;

  // EN: Clean up remaining objects.
  // TR: Kalan nesneleri temizle.
  pool.deallocate(s1);
  pool.deallocate(s3);
  pool.deallocate(s4);

  // --- Demo 5: Compile-Time Sine Table ---
  std::cout << "\n--- DEMO 5: Constexpr Sine Lookup Table ---\n" << std::endl;
  std::cout << "16-point sine table (computed at compile time):" << std::endl;
  for (std::size_t i = 0; i < SINE_TABLE.size(); ++i) {
    std::cout << "  sin(" << i << "*22.5deg) = ";
    if (SINE_TABLE[i] >= 0) std::cout << " ";
    std::cout << SINE_TABLE[i] << std::endl;
  }

  std::cout << "\n=> No-heap, constexpr, memory pool, placement new "
               "demonstrated!" << std::endl;

  return 0;
}
```

► 04. Rtos Concurrency

module_11_embedded_systems/04_rtos_concurrency.cpp

Konu: Embedded & Systems: RTOS Patterns, Priority Inversion, Lock-Free Structures — RTOS

Teori / Theory

THEORY: RTOS Task Model — Cooperative vs Preemptive

Real-Time Operating Systems (FreeRTOS, AUTOSAR OS, QNX) manage tasks with fixed priorities. Preemptive scheduling: a higher-priority task instantly preempts a lower-priority one. Cooperative: tasks yield voluntarily. In automotive ECUs, critical tasks (brake ABS, airbag) run at highest priority. HMI/Infotainment tasks run at lower priority. Tasks communicate via message queues, semaphores, and shared memory.

TEORİ: RTOS Görev Modeli — İşbirlikli vs Öncelikli

Gerçek Zamanlı İşletim Sistemleri (FreeRTOS, AUTOSAR OS, QNX) görevleri sabit önceliklerle yönetir. Öncelikli zamanlama: yüksek öncelikli görev anında düşük öncelikli görevi keser. İşbirlikli: görevler gönüllü olarak CPU'yu bırakır. Otomotiv ECU'larda kritik görevler (fren ABS, hava yastığı) en yüksek öncelikte çalışır. HMI/Bilgi-eğlence görevleri daha düşük öncelikte çalışır. Görevler mesaj kuyrukları, semaforlar ve paylaşılan bellek aracılığıyla iletişim kurar.

THEORY: Priority Inversion — The Mars Pathfinder Bug

Priority Inversion occurs when a high-priority task waits for a low-priority task holding a shared mutex — while a medium-priority task preempts the low-priority one, BLOCKING the high-priority task indefinitely. This caused the Mars Pathfinder reboot loop in 1997. Solutions:
1. Priority Inheritance: temporarily boost the low-priority task's priority.
2. Priority Ceiling: mutex has a pre-assigned ceiling priority.
3. Lock-free structures: eliminate mutexes entirely.

TEORİ: Öncelik Tersine Çevrilmesi — Mars Pathfinder Hatası

Öncelik Tersine Çevrilmesi, yüksek öncelikli görev paylaşılan bir mutex'i tutan düşük öncelikli görevi beklediğinde oluşur — orta öncelikli görev düşük öncelikliyi keserken, yüksek öncelikli görevi SÜRESİZ BLOKE EDİYOR. Bu, 1997'de Mars Pathfinder yeniden başlatma döngüsüne neden oldu. Çözümler:
1. Öncelik Kalıtımı: düşük öncelikli görevin önceliğini geçici olarak yükselt.
2. Öncelik Tavanı: mutex'in önceden atanmış bir tavan önceliği vardır.
3. Kilitsiz yapılar: mutex'leri tamamen ortadan kaldır.

THEORY: Lock-Free Programming — CAS Operations

Lock-free data structures use atomic Compare-And-Swap (CAS) instead of mutexes. `std::atomic::compare_exchange_weak/strong` atomically: reads the current value, compares it to "expected," and if equal, swaps to "desired." This eliminates deadlocks, priority inversion, and lock contention. Used extensively in automotive middleware (AUTOSAR COM), sensor data pipelines, and real-time audio processing.

TEORİ: Kilitsiz Programlama — CAS İşlemleri

Kilitsiz veri yapıları mutex'ler yerine atomik Karşılaştır-Ve-Değiştir (CAS) kullanır. `std::atomic::compare_exchange_weak/strong` atomik olarak: mevcut değeri okur, "beklenen" ile karşılaştırır ve eşitse "istenen" ile değiştirir. Bu, kilitlenmeleri, öncelik tersine çevrilmesini ve kilit çekişmesini ortadan kaldırır. Otomotiv ara yazılımında (AUTOSAR COM), sensör veri hatlarında ve gerçek zamanlı ses işlemede yoğun olarak kullanılır.

CPPREF Depth: std::atomic::compare_exchange_weak vs strong

`compare_exchange_weak` may fail spuriously (return false even if current == expected) on some architectures (ARM LL/SC). It MUST be used in a loop. `compare_exchange_strong` never fails spuriously but may be slower on LL/SC platforms. Weak is preferred in CAS loops, strong for single-shot operations. Both take memory_order parameters for fine-grained control. cppreference.com/w/cpp/atomic/atomic/compare_exchange

CPPREF Derinlik: std::atomic::compare_exchange_weak vs strong

`compare_exchange_weak` bazı mimarilerde (ARM LL/SC) sahte başarısız olabilir (current == expected olsa bile false döner). Bir döngü içinde kullanılması ZORUNLUDUR. `compare_exchange_strong` asla sahte başarısız olmaz ancak LL/SC platformlarda daha yavaş olabilir. CAS döngülerinde weak, tek seferlik işlemlerde strong tercih edilir. cppreference.com/w/cpp/atomic/atomic/compare_exchange

CPPREF Depth: std::memory_order — Acquire/Release Semantics

Memory ordering controls how atomic operations are reordered by CPU/ compiler. `memory_order_relaxed`: no ordering guarantees (fastest, counters). `memory_order_acquire`: no reads/writes can move BEFORE this load. `memory_order_release`: no reads/writes can move AFTER this store. Acquire-Release pair synchronizes: if thread A releases and thread B acquires the same atomic, B sees all writes A made before the release. `memory_order_seq_cst` (default): full sequential consistency — slowest but safest. In embedded, relaxed+acquire/release is heavily used. cppreference.com/w/cpp/atomic/memory_order

CPPREF Derinlik: std::memory_order — Acquire/Release Semantiği

Bellek sıralaması, atomik işlemlerin CPU/derleyici tarafından nasıl yeniden sıralandığını kontrol eder. `memory_order_relaxed`: sıralama garantisi yok (en hızlı, sayaçlar). `memory_order_acquire`: bu yüklemeden ÖNCE hiçbir okuma/yazma taşınamaz. `memory_order_release`: bu depolamadan SONRA hiçbir okuma/yazma taşınamaz. Acquire-Release çifti senkronize eder. `memory_order_seq_cst` (varsayılan): tam sıralı tutarlılık — en yavaş ama en güvenli. Gömülü sistemlerde relaxed+acquire/release yoğun kullanılır. cppreference.com/w/cpp/atomic/memory_order

Kaynak Kod / Source Code

```cpp
#include <atomic>
#include <chrono>
#include <cstdint>
#include <functional>
#include <iostream>
#include <mutex>
#include <queue>
#include <string>
#include <thread>
#include <vector>

```

```
// ═══════════════════════════════════════════════════════════════════════════
// [1. RTOS TASK SIMULATION — Priority-Based Scheduler]
// ═══════════════════════════════════════════════════════════════════════════

// EN: Simulates RTOS-style tasks with priorities (like FreeRTOS).
// TR: Öncelikli RTOS tarzı görevleri simüle eder (FreeRTOS gibi).
struct RTOSTask {
  std::string name;
  int priority; // Higher = more important
  std::function<void()> taskFunc;
};

class SimpleScheduler {
  std::vector<RTOSTask> tasks_;
  std::mutex mtx_;

public:
  void addTask(const std::string& name, int priority,
               std::function<void()> func) {
    std::lock_guard<std::mutex> lock(mtx_);
    tasks_.push_back({name, priority, std::move(func)});
  }

  // EN: Execute tasks in priority order (highest first).
  // TR: Görevleri öncelik sırasına göre çalıştır (en yüksek önce).
  void runAll() {
    std::lock_guard<std::mutex> lock(mtx_);
    // Sort by priority (descending)
    std::sort(tasks_.begin(), tasks_.end(),
              [](const RTOSTask& a, const RTOSTask& b) {
                return a.priority > b.priority;
              });

    for (const auto& task : tasks_) {
      std::cout << "  [SCHED] Running '" << task.name
                << "' (priority=" << task.priority << ")" << std::endl;
      task.taskFunc();
    }
    tasks_.clear();
  }
};

// ═══════════════════════════════════════════════════════════════════════════
// [2. PRIORITY INVERSION DEMONSTRATION]
// ═══════════════════════════════════════════════════════════════════════════

// EN: Shows how priority inversion can occur with shared mutexes.
// TR: Paylaşılan mutex'lerle öncelik tersine çevrilmesinin nasıl oluştuğunu gösterir.

std::mutex sharedResourceMtx;
std::atomic<bool> inversionDemo{true};

void lowPriorityTask() {
  std::cout << "  [LOW] Acquiring shared resource..." << std::endl;
  std::lock_guard<std::mutex> lock(sharedResourceMtx);
  std::cout << "  [LOW] Got resource. Working (simulating slow I/O)..."
            << std::endl;
  // EN: Simulate slow work while holding the lock.
  // TR: Kilidi tutarken yavaş çalışmayı simüle et.
  std::this_thread::sleep_for(std::chrono::milliseconds(80));
  std::cout << "  [LOW] Done. Releasing resource." << std::endl;
}

```

```cpp
void highPriorityTask() {
  // EN: Small delay so low-priority grabs the lock first.
  // TR: Düşük öncelikli görev kilidi ilk alsın diye küçük gecikme.
  std::this_thread::sleep_for(std::chrono::milliseconds(10));
  std::cout << "  [HIGH] Trying to acquire shared resource..." << std::endl;
  auto start = std::chrono::steady_clock::now();
  std::lock_guard<std::mutex> lock(sharedResourceMtx);
  auto end = std::chrono::steady_clock::now();
  auto waitMs = std::chrono::duration_cast<std::chrono::milliseconds>(
                    end - start).count();
  std::cout << "  [HIGH] Got resource after waiting " << waitMs << "ms!"
            << std::endl;
  std::cout << "  [HIGH] => Priority inversion: HIGH waited for LOW!"
            << std::endl;
}

// ==========================================================================
// [3. LOCK-FREE SPSC QUEUE (Single Producer, Single Consumer)]
// ==========================================================================

// EN: Lock-free SPSC queue using atomic indices — used in automotive sensor pipelines (ADC →
// DSP), CAN message buffers, and audio ring buffers.
// TR: Atomik indeksler kullanan kilitsiz SPSC kuyruğu — otomotiv sensör hatlarında (ADC → DSP),
// CAN mesaj tamponlarında kullanılır.
template <typename T, std::size_t Capacity>
class LockFreeSPSCQueue {
  std::array<T, Capacity> buffer_{};
  std::atomic<std::size_t> readIdx_{0};
  std::atomic<std::size_t> writeIdx_{0};

public:
  bool tryPush(const T& item) {
    std::size_t currWrite = writeIdx_.load(std::memory_order_relaxed);
    std::size_t nextWrite = (currWrite + 1) % Capacity;
    // EN: If next write would hit read, queue is full.
    // TR: Sonraki yazma okumaya çarparsa, kuyruk doludur.
    if (nextWrite == readIdx_.load(std::memory_order_acquire)) {
      return false; // Full
    }
    buffer_[currWrite] = item;
    writeIdx_.store(nextWrite, std::memory_order_release);
    return true;
  }

  bool tryPop(T& item) {
    std::size_t currRead = readIdx_.load(std::memory_order_relaxed);
    if (currRead == writeIdx_.load(std::memory_order_acquire)) {
      return false; // Empty
    }
    item = buffer_[currRead];
    readIdx_.store((currRead + 1) % Capacity,
                   std::memory_order_release);
    return true;
  }
};

// ==========================================================================
// [4. ATOMIC CAS — Lock-Free Counter with Backoff]
// ==========================================================================

// EN: Demonstrates compare_exchange_weak in a CAS loop with exponential backoff — common in
// lock-free embedded counters.
// TR: Üstel geri çekilme ile CAS döngüsünde compare_exchange_weak'i gösterir — kilitsiz gömülü
// sayaçlarda yaygındır.

std::atomic<int> lockFreeCounter{0};

void casIncrement(int iterations) {
```

```
143     for (int i = 0; i < iterations; ++i) {
144       int expected = lockFreeCounter.load(std::memory_order_relaxed);
145       while (!lockFreeCounter.compare_exchange_weak(
146           expected, expected + 1, std::memory_order_release,
147           std::memory_order_relaxed)) {
148         // EN: CAS failed — expected was updated, retry automatically.
149         // TR: CAS başarısız — expected güncellendi, otomatik yeniden dene.
150       }
151     }
152   }
153
154   int main() {
155     std::cout << "=== MODULE 11: RTOS CONCURRENCY PATTERNS ===\n" << std::endl;
156
157     // --- Demo 1: Priority-Based Scheduler ---
158     std::cout << "--- DEMO 1: RTOS Priority Scheduler ---\n" << std::endl;
159     SimpleScheduler scheduler;
160     scheduler.addTask("HMI_Render", 2, []() {
161       std::cout << "    -> Rendering instrument cluster gauges" << std::endl;
162     });
163     scheduler.addTask("ABS_BrakeControl", 10, []() {
164       std::cout << "    -> ABS brake actuation — CRITICAL" << std::endl;
165     });
166     scheduler.addTask("Climate_FanSpeed", 3, []() {
167       std::cout << "    -> Adjusting HVAC fan to target temperature" << std::endl;
168     });
169     scheduler.addTask("Airbag_Deploy", 15, []() {
170       std::cout << "    -> Airbag deployment check — HIGHEST PRIORITY"
171                 << std::endl;
172     });
173     scheduler.addTask("CAN_Logger", 1, []() {
174       std::cout << "    -> Logging CAN bus messages to flash" << std::endl;
175     });
176     scheduler.runAll();
177
178     // --- Demo 2: Priority Inversion ---
179     std::cout << "\n--- DEMO 2: Priority Inversion ---\n" << std::endl;
180     std::cout << "Scenario: LOW holds mutex, HIGH must wait:" << std::endl;
181     std::thread tLow(lowPriorityTask);
182     std::thread tHigh(highPriorityTask);
183     tLow.join();
184     tHigh.join();
185
186     // --- Demo 3: Lock-Free SPSC Queue ---
187     std::cout << "\n--- DEMO 3: Lock-Free SPSC Queue ---\n" << std::endl;
188     LockFreeSPSCQueue<int, 8> sensorQueue;
189
190     // EN: Producer: ADC sampling interrupt pushes sensor readings.
191     // TR: Üretici: ADC örnekleme kesmesi sensör okumalarını iter.
192     std::thread producer([&sensorQueue]() {
193       for (int i = 1; i <= 10; ++i) {
194         int sensorVal = 1000 + i * 50; // Simulated EGT value in °C
195         if (sensorQueue.tryPush(sensorVal)) {
196           std::cout << "  [PRODUCER] Pushed EGT reading: " << sensorVal
197                     << " C" << std::endl;
198         } else {
199           std::cout << "  [PRODUCER] Queue full! Dropped: " << sensorVal
200                     << std::endl;
201         }
202         std::this_thread::sleep_for(std::chrono::milliseconds(5));
203       }
204     });
205
206     // EN: Consumer: DSP task processes sensor readings.
207     // TR: Tüketici: DSP görevi sensör okumalarını işler.
208     std::thread consumer([&sensorQueue]() {
209       int val = 0;
210       int count = 0;
```

```
    while (count < 10) {
      if (sensorQueue.tryPop(val)) {
        std::cout << "  [CONSUMER] Processed EGT: " << val << " C"
                  << (val > 1300 ? " ** WARNING: OVERHEAT! **" : "")
                  << std::endl;
        ++count;
      }
      std::this_thread::sleep_for(std::chrono::milliseconds(8));
    }
  });

  producer.join();
  consumer.join();

  // --- Demo 4: Lock-Free CAS Counter ---
  std::cout << "\n--- DEMO 4: Lock-Free CAS Counter ---\n" << std::endl;
  lockFreeCounter.store(0);
  constexpr int THREADS = 4;
  constexpr int ITERS_PER_THREAD = 10000;
  std::vector<std::thread> workers;
  workers.reserve(THREADS);
  for (int t = 0; t < THREADS; ++t) {
    workers.emplace_back(casIncrement, ITERS_PER_THREAD);
  }
  for (auto& w : workers) w.join();

  std::cout << "  Expected: " << THREADS * ITERS_PER_THREAD << std::endl;
  std::cout << "  Actual:   " << lockFreeCounter.load() << std::endl;
  std::cout << "  Match:    "
            << (lockFreeCounter.load() == THREADS * ITERS_PER_THREAD
                    ? "YES (lock-free is correct!)"
                    : "NO (bug!)")
            << std::endl;

  std::cout << "\n=> RTOS scheduling, priority inversion, lock-free queue, "
               "CAS counter demonstrated!" << std::endl;

  return 0;
}
```

► 05. Advanced Parallelism

module_11_embedded_systems/05_advanced_parallelism.cpp

Konu: Embedded & Systems: Thread Pools, C++17 Parallel Algorithms, Work Stealing — İş

Teori / Theory

THEORY: Thread Pools — Why Not Spawn Per Task

Creating a new `std::thread` for each task is expensive: OS must allocate stack memory (~1-8MB), register the thread with the scheduler, and tear it down when done. In embedded/automotive systems with limited resources, this is unacceptable. A thread pool pre-creates N worker threads that continuously pull tasks from a shared queue. This gives: (1) bounded resource usage, (2) amortized creation cost, (3) predictable latency. AUTOSAR Adaptive Platform uses thread pools for service-oriented RPCs.

TEORİ: İş Parçacığı Havuzları — Neden Görev Başına Oluşturma Yapılmaz

Her görev için yeni bir `std::thread` oluşturmak pahalıdır: OS yığıt belleği (~1-8MB) ayırmalı, iş parçacığını zamanlayıcıya kaydetmeli ve bittiğinde yıkmalıdır. Sınırlı kaynaklara sahip gömülü/otomotiv sistemlerde bu kabul edilemez. Bir iş parçacığı havuzu, sürekli olarak paylaşılan bir kuyruktan görev çeken N işçi iş parçacığı önceden oluşturur. Bu şunları verir: (1) sınırlı kaynak kullanımı, (2) amortize oluşturma maliyeti, (3) öngörülebilir gecikme.

THEORY: C++17 Parallel Algorithms — std::execution Policies

C++17 added execution policies to STL algorithms: `std::execution::seq` (sequential), `std::execution::par` (parallel), `std::execution::par_unseq` (parallel + vectorized). This means existing `std::sort`, `std::transform`, `std::reduce` can run in parallel with just one extra argument! In automotive: parallel map data processing, multiple sensor fusion, batch diagnostics. Note: requires linking `-ltbb` on GCC (Intel TBB backend).

TEORİ: C++17 Paralel Algoritmalar — std::execution Politikaları

C++17, STL algoritmalarına yürütme politikaları ekledi: `std::execution::seq` (sıralı), `std::execution::par` (paralel), `std::execution::par_unseq` (paralel + vektörize). Bu, mevcut `std::sort`, `std::transform`, `std::reduce`'un tek bir ek argümanla paralel çalışabileceği anlamına gelir! Otomotivde: paralel harita veri işleme, çoklu sensör füzyonu, toplu tanılama. Not: GCC'de `-ltbb` bağlantısı gerektirir (Intel TBB arka ucu).

THEORY: Work Stealing — Load-Balanced Parallelism

In a basic thread pool, if one task is much longer than others, its worker stays busy while others idle. Work stealing solves this: each worker has its own deque. When a worker's deque is empty, it "steals" a task from another worker's deque (from the BACK, to minimize contention). Intel TBB, Java ForkJoinPool, and Tokio (Rust) all use work stealing. In automotive: useful for parallel sensor data pipelines with varying latencies.

TEORİ: İş Çalma — Yük-Dengeli Paralellik

Temel bir iş parçacığı havuzunda, bir görev diğerlerinden çok uzunsa, işçisi meşgul kalırken diğerleri boşta kalır. İş çalma bunu çözer: her işçinin kendi deque'ü vardır. Bir işçinin deque'ü boşaldığında, başka bir işçinin deque'ünden (çekişmeyi en aza indirmek için ARKADAN) bir görev "çalar". Intel TBB, Java ForkJoinPool ve Tokio (Rust) hepsi iş çalma kullanır.

CPPREF Depth: std::execution Policy Tag Types

`std::execution::seq` — no parallelism, same as before. Elements accessed in order on calling thread. `std::execution::par` — may execute in parallel on unspecified threads but elements are NOT interleaved within a thread. `std::execution::par_unseq` — may execute in parallel AND vectorized (SIMD). Element access functions must NOT use mutexes or blocking synchronization! `std::execution::unseq` (C++20) — vectorized on the calling thread only. cppreference.com/w/cpp/algorithm/execution_policy_tag

CPPREF Derinlik: std::execution Politika Etiket Tipleri

`std::execution::seq` — paralellik yok, öncekiyle aynı. `std::execution::par` — belirtilmemiş iş parçacıklarında paralel yürütülebilir ancak elemanlar bir iş parçacığı içinde serpiştirilmez. `std::execution::par_unseq` — paralel VE vektörize (SIMD) yürütülebilir. Eleman erişim fonksiyonları mutex veya blokleyici senkronizasyon KULLANMAMALIDIR! `std::execution::unseq` (C++20) — yalnızca çağıran iş parçacığında vektörize. cppreference.com/w/cpp/algorithm/execution_policy_tag

CPPREF Depth: std::reduce vs std::accumulate — Parallel Reduction

`std::accumulate` (C++98) is strictly left-fold: `((init op a) op b) op c`. Cannot be parallelized because order is fixed. `std::reduce` (C++17) is unordered: the operation can be applied in any order, enabling parallel execution. The operation MUST be associative and commutative (e.g., `+`, `*`, `max`). For floating-point, `reduce` may give slightly different results due to different summation order (affects rounding). `std::transform_reduce` is the parallel map-reduce primitive. cppreference.com/w/cpp/algorithm/reduce

CPPREF Derinlik: std::reduce vs std::accumulate — Paralel İndirgeme

`std::accumulate` (C++98) kesinlikle sol-katlama: sıra sabittir. Paralelleştirilemez. `std::reduce` (C++17) sırasızdır: işlem herhangi bir sırada uygulanabilir, paralel yürütmeyi mümkün kılar. İşlem ilişkisel ve değişmeli OLMALIDIR (ör. `+`, `*`, `max`). Kayan nokta için `reduce` farklı toplama sırası nedeniyle biraz farklı sonuçlar verebilir. `std::transform_reduce` paralel map-reduce ilkesidir. cppreference.com/w/cpp/algorithm/reduce

Kaynak Kod / Source Code

```cpp
#include <algorithm>
#include <atomic>
#include <chrono>
#include <condition_variable>
#include <cstdint>
#include <functional>
#include <iostream>
#include <mutex>
#include <numeric>
#include <queue>
#include <string>
#include <thread>
#include <vector>

// ═══════════════════════════════════════════════════════════════════════════════
// [1. THREAD POOL IMPLEMENTATION]
// ═══════════════════════════════════════════════════════════════════════════════

// EN: A production-quality thread pool for embedded task dispatching.
// TR: Gömülü görev dağıtımı için üretim kalitesinde iş parçacığı havuzu.
class ThreadPool {
  std::vector<std::thread> workers_;
  std::queue<std::function<void()>> taskQueue_;
  std::mutex mtx_;
  std::condition_variable cv_;
  std::atomic<bool> shutdown_{false};

```

```
public:
  explicit ThreadPool(std::size_t numThreads) {
    for (std::size_t i = 0; i < numThreads; ++i) {
      workers_.emplace_back([this]() { workerLoop(); });
    }
  }

  ~ThreadPool() {
    shutdown_.store(true, std::memory_order_release);
    cv_.notify_all();
    for (auto& w : workers_) {
      if (w.joinable()) w.join();
    }
  }

  // EN: Submit a task to the pool.
  // TR: Havuza bir görev gönder.
  void submit(std::function<void()> task) {
    {
      std::lock_guard<std::mutex> lock(mtx_);
      taskQueue_.push(std::move(task));
    }
    cv_.notify_one();
  }

  // EN: Wait until all queued tasks are completed.
  // TR: Kuyruktaki tüm görevler tamamlanana kadar bekle.
  void waitAll() {
    std::unique_lock<std::mutex> lock(mtx_);
    cv_.wait(lock, [this]() {
      return taskQueue_.empty();
    });
    // EN: Small grace period for in-flight tasks to complete.
    // TR: Devam eden görevlerin tamamlanması için küçük bekleme süresi.
    lock.unlock();
    std::this_thread::sleep_for(std::chrono::milliseconds(50));
  }

private:
  void workerLoop() {
    while (true) {
      std::function<void()> task;
      {
        std::unique_lock<std::mutex> lock(mtx_);
        cv_.wait(lock, [this]() {
          return shutdown_.load(std::memory_order_acquire) ||
                 !taskQueue_.empty();
        });
        if (shutdown_.load(std::memory_order_acquire) &&
            taskQueue_.empty()) {
          return;
        }
        task = std::move(taskQueue_.front());
        taskQueue_.pop();
      }
      task();
      cv_.notify_all(); // Wake waitAll()
    }
  }
};

// ==========================================================================
// [2. SIMULATED SENSOR FUSION — Parallel Processing]
// ==========================================================================

// EN: Simulate processing multiple automotive sensor streams in parallel.
// TR: Birden fazla otomotiv sensör akışını paralel olarak işlemeyi simüle et.
struct SensorReading {
  std::string sensorName;
  double value;
  std::string unit;
};

```

```cpp
SensorReading processSensor(const std::string& name, double rawValue,
                            double calibrationOffset) {
  // EN: Simulate some computation (filtering, calibration).
  // TR: Bazı hesaplamaları simüle et (filtreleme, kalibrasyon).
  std::this_thread::sleep_for(std::chrono::milliseconds(20));
  double calibrated = rawValue + calibrationOffset;
  std::string unit = (name.find("EGT") != std::string::npos)   ? "C"
                     : (name.find("RPM") != std::string::npos)  ? "rpm"
                     : (name.find("MAP") != std::string::npos)  ? "kPa"
                     : (name.find("AFR") != std::string::npos)  ? "lambda"
                                                                : "raw";
  return {name, calibrated, unit};
}

// ==========================================================================================
// [3. MANUAL std::reduce EQUIVALENT — Portable Sum]
// ==========================================================================================

// EN: Since parallel std::execution requires -ltbb and may not be available, we demonstrate the
// concept with manual parallel reduction.
// TR: Paralel std::execution -ltbb gerektirdiğinden ve mevcut olmayabileceğinden kavramı manuel
// paralel indirgeme ile gösteriyoruz.

double parallelReduce(const std::vector<double>& data, std::size_t numThreads) {
  std::vector<double> partialSums(numThreads, 0.0);
  std::vector<std::thread> threads;
  std::size_t chunkSize = data.size() / numThreads;

  for (std::size_t t = 0; t < numThreads; ++t) {
    std::size_t start = t * chunkSize;
    std::size_t end = (t == numThreads - 1) ? data.size() : start + chunkSize;
    threads.emplace_back(
        [&data, &partialSums, t, start, end]() {
          double sum = 0.0;
          for (std::size_t i = start; i < end; ++i) {
            sum += data[i];
          }
          partialSums[t] = sum;
        });
  }
  for (auto& th : threads) th.join();

  // EN: Final sequential merge of partial sums.
  // TR: Kısmi toplamların son sıralı birleştirmesi.
  double total = 0.0;
  for (double ps : partialSums) total += ps;
  return total;
}

int main() {
  std::cout << "=== MODULE 11: ADVANCED PARALLELISM ===\n" << std::endl;

  // --- Demo 1: Thread Pool ---
  std::cout << "--- DEMO 1: Thread Pool (4 workers) ---\n" << std::endl;
  {
    ThreadPool pool(4);
    std::mutex printMtx;

    // EN: Submit automotive ECU tasks to the pool.
    // TR: Otomotiv ECU görevlerini havuza gönder.
    std::vector<std::string> ecuTasks = {
        "BCM: Check door lock status",
        "Cluster: Update speedometer needle",
        "HVAC: Read cabin temperature sensor",
        "Powertrain: Calculate fuel injection timing",
        "HMI: Render navigation overlay",
        "Chassis: Process wheel speed ABS data",
        "Telematics: Upload OBD-II snapshot",
        "BCM: Toggle headlight relay"};

```

```
171       for (const auto& taskDesc : ecuTasks) {
172         pool.submit([&printMtx, taskDesc]() {
173           std::lock_guard<std::mutex> lock(printMtx);
174           std::cout << "  [Worker " << std::this_thread::get_id()
175                     << "] " << taskDesc << std::endl;
176         });
177       }
178       pool.waitAll();
179     }
180     std::cout << "  Thread pool destroyed — all workers joined.\n" << std::endl;
181
182     // --- Demo 2: Parallel Sensor Fusion ---
183     std::cout << "--- DEMO 2: Parallel Sensor Fusion ---\n" << std::endl;
184     {
185       struct SensorJob {
186         std::string name;
187         double rawVal;
188         double offset;
189       };
190
191       std::vector<SensorJob> jobs = {
192           {"EGT_Cyl1", 720.0, +2.5},  {"EGT_Cyl2", 735.0, -1.2},
193           {"EGT_Cyl3", 710.0, +0.8},  {"EGT_Cyl4", 745.0, -0.5},
194           {"RPM_Crank", 3200.0, 0.0},  {"MAP_Intake", 85.0, +1.1},
195           {"AFR_Wideband", 0.98, +0.02}};
196
197       std::vector<SensorReading> results(jobs.size());
198       std::vector<std::thread> threads;
199
200       auto startTime = std::chrono::steady_clock::now();
201       for (std::size_t i = 0; i < jobs.size(); ++i) {
202         threads.emplace_back([&jobs, &results, i]() {
203           results[i] = processSensor(jobs[i].name, jobs[i].rawVal,
204                                      jobs[i].offset);
205         });
206       }
207       for (auto& t : threads) t.join();
208       auto endTime = std::chrono::steady_clock::now();
209       auto elapsed = std::chrono::duration_cast<std::chrono::milliseconds>(
210                          endTime - startTime).count();
211
212       for (const auto& r : results) {
213         std::cout << "  " << r.sensorName << " = " << r.value << " "
214                   << r.unit << std::endl;
215       }
216       std::cout << "  Parallel time: " << elapsed << "ms"
217                 << " (sequential would be ~" << jobs.size() * 20 << "ms)\n"
218                 << std::endl;
219     }
220
221     // --- Demo 3: Parallel Reduction ---
222     std::cout << "--- DEMO 3: Parallel Reduction ---\n" << std::endl;
223     {
224       constexpr std::size_t DATA_SIZE = 1000000;
225       std::vector<double> sensorData(DATA_SIZE);
226       for (std::size_t i = 0; i < DATA_SIZE; ++i) {
227         sensorData[i] = static_cast<double>(i % 100) * 0.01;
228       }
229
230       // EN: Sequential sum using std::accumulate.
231       // TR: std::accumulate kullanarak sıralı toplam.
232       auto t1 = std::chrono::steady_clock::now();
233       double seqSum = std::accumulate(sensorData.begin(), sensorData.end(), 0.0);
234       auto t2 = std::chrono::steady_clock::now();
235       auto seqMs = std::chrono::duration_cast<std::chrono::microseconds>(
236                        t2 - t1).count();
237
```

```
238     // EN: Parallel sum using manual thread-based reduction.
239     // TR: Manuel iş parçacığı tabanlı indirgeme kullanarak paralel toplam.
240     auto t3 = std::chrono::steady_clock::now();
241     double parSum = parallelReduce(sensorData, 4);
242     auto t4 = std::chrono::steady_clock::now();
243     auto parMs = std::chrono::duration_cast<std::chrono::microseconds>(
244                      t4 - t3).count();
245
246     std::cout << "  Sequential sum: " << seqSum << " (" << seqMs << " us)"
247               << std::endl;
248     std::cout << "  Parallel sum:   " << parSum << " (" << parMs << " us)"
249               << std::endl;
250     std::cout << "  Match: "
251               << (std::abs(seqSum - parSum) < 0.001 ? "YES" : "DRIFT")
252               << std::endl;
253   }
254
255   std::cout << "\n--- DEMO 4: std::execution Policy Reference ---\n"
256             << std::endl;
257   std::cout << "  // With -ltbb, these would work:" << std::endl;
258   std::cout << "  // std::sort(std::execution::par, v.begin(), v.end());"
259             << std::endl;
260   std::cout << "  // std::reduce(std::execution::par, v.begin(), v.end());"
261             << std::endl;
262   std::cout << "  // std::transform_reduce(std::execution::par_unseq, ...);"
263             << std::endl;
264   std::cout << "  // See cppreference.com/w/cpp/algorithm/execution_policy_tag"
265             << std::endl;
266
267   std::cout << "\n=> Thread pool, sensor fusion, parallel reduction "
268                "demonstrated!" << std::endl;
269
270   return 0;
271 }
```

Modül 12: Coroutines — Eşyordamlar

Eşyordamlar

2 kaynak dosya · Standart: C++20

C++20

► 01. Coroutine Fundamentals

module_12_coroutines/01_coroutine_fundamentals.cpp

Konu: C++20 Coroutines: Temeller ve Mekanizma — Fundamentals and Mechanics

Teori / Theory

THEORY: What Are Coroutines?

A coroutine is a function that can SUSPEND its execution and RESUME later. Unlike a regular function that runs from start to finish, a coroutine can:
- Pause midway (yield a value or wait for something)
- Resume from exactly where it left off (local variables preserved!)
- Pause and resume multiple times before final completion

Think of it like reading a book: you can put a bookmark, do something else, and come back to exactly where you stopped. Regular functions are like eating a meal — you start and finish in one sitting.

TEORİ: Coroutine Nedir?

Coroutine, çalışmasını ASKIya alıp daha sonra devam edebilen bir fonksiyondur. Baştan sona çalışan normal bir fonksiyondan farklı olarak:
- Ortada duraklar (değer üretir veya bir şeyi bekler)
- Kaldığı yerden tam olarak devam eder (yerel değişkenler korunur!)
- Son tamamlanmadan önce birçok kez duraklar ve devam eder

Kitap okumak gibi düşün: ayraç koyup başka şey yapıp kaldığın yere dönebilirsin. Normal fonksiyonlar yemek yemek gibi — başlarsın ve bir oturumda bitirirsin.

THEORY: The Three Magic Keywords

A function becomes a coroutine if its body contains ANY of these keywords:

```
┌──────────────┬──────────────────────────────────────────────────────┐
│ Keyword      │ What It Does                                         │
├──────────────┼──────────────────────────────────────────────────────┤
│ co_await     │ Suspends execution, waits for an async operation     │
│              │ (like "await" in Python/JS/C#)                       │
├──────────────┼──────────────────────────────────────────────────────┤
│ co_yield     │ Suspends and produces a value (lazy generator)       │
│              │ Equivalent to co_await promise.yield_value(expr)     │
├──────────────┼──────────────────────────────────────────────────────┤
│ co_return    │ Completes the coroutine and optionally returns a value  │
│              │ Unlike regular return: also destroys the coroutine   │
│              │ frame's promise and does final cleanup               │
└──────────────┴──────────────────────────────────────────────────────┘
```

IMPORTANT: You CANNOT mix `return` and `co_return` in the same function! If the function uses any co_* keyword, it's a coroutine — use co_return.

TEORİ: ÜÇ SİHİRLİ ANAHTAR KELİME

Bir fonksiyon, gövdesinde bu anahtar kelimelerden HERHANGİ BİRİ varsa coroutine olur:

```
┌──────────────┬──────────────────────────────────────────────────────┐
│ Anahtar      │ Ne Yapar                                             │
├──────────────┼──────────────────────────────────────────────────────┤
│ co_await     │ Çalışmayı askıya alır, asenkron işlemi bekler        │
├──────────────┼──────────────────────────────────────────────────────┤
│ co_yield     │ Askıya alır ve değer üretir (tembel üretici)         │
├──────────────┼──────────────────────────────────────────────────────┤
│ co_return    │ Coroutine'i tamamlar, isteğe bağlı değer döndürür    │
└──────────────┴──────────────────────────────────────────────────────┘
```

ÖNEMLİ: Aynı fonksiyonda `return` ve `co_return` KARIŞTIRILAMAZ!

THEORY: Coroutine Anatomy

When compiler sees a co_* keyword, it transforms the function:
1. COROUTINE FRAME (heap-allocated*):
- Holds local variables, parameters, promise object
- Persists between suspensions (that's why locals survive!)
- *Can be heap-elided (HALO optimization) if lifetime is bounded
2. PROMISE TYPE (promise_type):
- Controls coroutine behavior at key points
- Like a "policy class" the compiler consults
- You define: initial_suspend, final_suspend, yield_value, etc.
3. COROUTINE HANDLE (std::coroutine_handle<Promise>):
- A non-owning pointer to the coroutine frame
- Used to RESUME or DESTROY the coroutine from outside
- handle.resume() → continues execution
- handle.destroy() → deallocates the frame
Compiler transformation (simplified):

```
┌──────────────────────────────────────────────────┐
│  Your Code          →  Compiler Generates        │
├──────────────────────────────────────────────────┤
│  Generator count()  →  1. Allocate coroutine frame      │
│  {                  →  2. Construct promise object      │
│    co_yield 1;      →  3. Call initial_suspend()        │
│    co_yield 2;      →  4. Execute body with suspensions │
│  }                  →  5. Call final_suspend()          │
│                     →  6. Destroy frame (if needed)     │
└──────────────────────────────────────────────────┘
```

TEORİ: Coroutine Anatomisi

Derleyici co_* anahtar kelimesini görünce fonksiyonu dönüştürür:
1. COROUTINE FRAME (heap'te ayrılır*):
- Yerel değişkenleri, parametreleri, promise nesnesini tutar
- Askıya almalar arasında kalıcıdır (yerel değişkenlerin yaşama sebebi!)
- *HALO optimizasyonu ile heap kullanımı elenebilir
2. PROMISE TYPE (promise_type):
- Kritik noktalarda coroutine davranışını kontrol eder
- Derleyicinin danıştığı bir "politika sınıfı" gibi
- Siz tanımlarsınız: initial_suspend, final_suspend, yield_value, vb.
3. COROUTINE HANDLE (std::coroutine_handle<Promise>):
- Coroutine frame'e sahip olmayan (non-owning) işaretçi
- Dışarıdan coroutine'i DEVAM ETTİRMEK veya YOK ETMEK için kullanılır
- handle.resume() → çalışmaya devam eder
- handle.destroy() → frame'i serbest bırakır
Derleyici dönüşümü (basitleştirilmiş):

```
┌──────────────────────────────────────────────────┐
│  Sizin Kodunuz      →  Derleyicinin Ürettikleri  │
├──────────────────────────────────────────────────┤
│  Generator count()  →  1. Coroutine frame tahsis et       │
│  {                  →  2. Promise nesnesi oluştur         │
│    co_yield 1;      →  3. initial_suspend() çağrısı       │
│    co_yield 2;      →  4. Gövdeyi askıya almalarla çalıştır│
│  }                  →  5. final_suspend() çağrısı         │
│                     →  6. Frame'i yok et (gerekirse)      │
└──────────────────────────────────────────────────┘
```

THEORY: Awaitable Concept

co_await operates on "awaitable" objects. An awaitable must have 3 methods:
bool await_ready() → Should we suspend at all? (false = yes, suspend) void await_suspend(h) → What to do when suspending (schedule, store handle) T await_resume() → What value to produce when resumed
Built-in awaitables:
- std::suspend_always → await_ready() returns false (always suspends)
- std::suspend_never → await_ready() returns true (never suspends)

TEORİ: Awaitable Kavramı

co_await "awaitable" nesneler üzerinde çalışır. 3 metodu olmalı:
bool await_ready() → Askıya almalı mıyız? (false = evet, askıya al) void await_suspend(h) → Askıya alınırken ne yapılmalı T await_resume() → Devam edilince hangi değer üretilmeli

→ Ref: https://en.cppreference.com/w/cpp/language/coroutines
→ Ref: https://lewissbaker.github.io/ — "Asymmetric Transfer" blog (excellent)

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <coroutine>   // EN: C++20 coroutine support / TR: C++20 coroutine desteği
#include <string>
#include <vector>
#include <cstdint>
#include <optional>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: Generator<T> — A Complete Coroutine Type From Scratch
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: Generator<T> is the simplest useful coroutine type: a lazy sequence producer.
 *     It produces values one at a time via co_yield, pausing between each one.
 *     The caller pulls values with next() — no value is computed until requested.
 *
 *     This is the "Hello World" of coroutines. We build it from scratch to show
 *     exactly what the compiler expects.
 *
 * TR: Generator<T> en basit kullanışlı coroutine tipidir: tembel dizi üretici.
 *     co_yield ile her seferinde bir değer üretir, aralarında duraklar.
 *     Çağıran next() ile değer çeker — istenene kadar hesaplama yapılmaz.
 *
 *     Bu coroutine'lerin "Merhaba Dünya"sıdır. Derleyicinin tam olarak ne
 *     beklediğini göstermek için sıfırdan inşa ediyoruz.
 */
template<typename T>
class Generator {
public:
    // ── promise_type: The "policy" that tells the compiler how this coroutine behaves ──
    // EN: The compiler looks for a nested type called `promise_type` inside the return type.
    //     This is NOT optional — without it, compilation fails.
    //     Each method answers a specific question the compiler asks:
    //
    // TR: Derleyici, dönüş tipinin içinde `promise_type` adlı iç içe bir tip arar.
    //     Bu opsiyonel DEĞİLDİR — olmadan derleme başarısız olur.
    //     Her metot, derleyicinin sorduğu belirli bir soruyu yanıtlar:
    struct promise_type {
        T current_value;             // EN: Stores the last co_yield'd value / TR: Son co_yield edilen değeri tutar
        std::exception_ptr exception; // EN: Captures exceptions from coroutine / TR: Coroutine'den gelen istisnaları yakalar

```

```
42          // EN: Q: "What object should be returned to the caller?"
43          //     A: A Generator wrapping a handle to this coroutine.
44          // TR: S: "Çağırana hangi nesne döndürülmeli?"
45          //     C: Bu coroutine'e bir handle saran bir Generator.
46          Generator get_return_object() {
47              return Generator{std::coroutine_handle<promise_type>::from_promise(*this)};
48          }
49
50          // EN: Q: "Should the coroutine suspend BEFORE running any user code?"
51          //     A: YES (suspend_always) — lazy start, nothing runs until first next() call.
52          //     If we said suspend_never, the coroutine would run eagerly to first co_yield.
53          // TR: S: "Coroutine, kullanıcı kodu çalıştırmadan ÖNCE askıya alınmalı mı?"
54          //     C: EVET (suspend_always) — tembel başlangıç, ilk next() çağrısına kadar çalışmaz.
55          std::suspend_always initial_suspend() noexcept { return {}; }
56
57          // EN: Q: "Should the coroutine suspend AFTER it finishes (at final_suspend)?"
58          //     A: YES — we suspend so the caller can detect completion and clean up the handle.
59          //     If we said suspend_never, the frame auto-destroys and our handle dangles!
60          //     WARNING: final_suspend MUST be noexcept (standard requirement).
61          // TR: S: "Coroutine bittikten SONRA askıya alınmalı mı?"
62          //     C: EVET — çağıran tamamlanmayı algılayıp handle'ı temizleyebilsin.
63          //     suspend_never desek frame otomatik yıkılır ve handle sarkar!
64          //     UYARI: final_suspend noexcept OLMALIDIR (standart gereksiniml).
65          std::suspend_always final_suspend() noexcept { return {}; }
66
67          // EN: Q: "A co_yield expression was hit — what should happen?"
68          //     A: Store the value and suspend (suspend_always).
69          //     The caller will read current_value after resume returns to them.
70          // TR: S: "co_yield ifadesine ulaşıldı — ne olmalı?"
71          //     C: Değeri sakla ve askıya al (suspend_always).
72          std::suspend_always yield_value(T value) {
73              current_value = std::move(value);
74              return {};
75          }
76
77          // EN: Q: "co_return was hit (or end of function reached) — any return value?"
78          //     A: No value for generators — void return.
79          // TR: S: "co_return'e ulaşıldı — dönüş değeri?"
80          //     C: Generator'lar için değer yok — void dönüş.
81          void return_void() {}
82
83          // EN: Q: "An unhandled exception escaped the coroutine body — what to do?"
84          //     A: Capture it. The caller can rethrow it later if needed.
85          // TR: S: "Coroutine gövdesinden yakalanmamış istisna çıktı — ne yapılmalı?"
86          //     C: Yakala. Çağıran gerekirse sonra yeniden fırlatabilir.
87          void unhandled_exception() {
88              exception = std::current_exception();
89          }
90      };
91
92      // ── Generator class body ──
93
94      // EN: Type alias for the coroutine handle specific to our promise
95      // TR: Promise'imize özgü coroutine handle için tür takma adı
96      using handle_type = std::coroutine_handle<promise_type>;
97
98      // EN: Constructor — takes ownership of the coroutine handle
99      // TR: Constructor — coroutine handle'ın sahipliğini alır
100     explicit Generator(handle_type h) : handle_(h) {}
101
102     // EN: Destructor — MUST destroy the handle to free the coroutine frame!
103     //     If you forget this, the frame LEAKS (heap memory not freed).
104     // TR: Destructor — Coroutine frame'i serbest bırakmak için handle'ı yıkmalı!
105     //     Unutursan frame SIZDIRIR (heap belleği serbest bırakılmaz).
106     ~Generator() {
107         if (handle_) handle_.destroy();
108     }
109
110     // EN: Move-only type (coroutine ownership cannot be shared)
111     // TR: Sadece taşınabilir tip (coroutine sahipliği paylaşılamaz)
112     Generator(const Generator&) = delete;
113     Generator& operator=(const Generator&) = delete;
114     Generator(Generator&& other) noexcept : handle_(other.handle_) {
115         other.handle_ = nullptr;
116     }
```

```cpp
    Generator& operator=(Generator&& other) noexcept {
        if (this != &other) {
            if (handle_) handle_.destroy();
            handle_ = other.handle_;
            other.handle_ = nullptr;
        }
        return *this;
    }

    // EN: Advance the coroutine to the next co_yield point and return the value.
    //     Returns std::optional<T> — empty if coroutine is finished.
    //
    //     Flow: next() → handle_.resume() → coroutine runs until co_yield → suspends
    //           → control returns here → read promise().current_value
    //
    // TR: Coroutine'i sonraki co_yield noktasına ilerlet ve değeri döndür.
    //     std::optional<T> döndürür — coroutine bittiyse boş.
    std::optional<T> next() {
        if (!handle_ || handle_.done()) return std::nullopt;

        handle_.resume();  // EN: Continue coroutine until next co_yield or end
                           // TR: Sonraki co_yield veya sona kadar coroutine'i devam ettir

        if (handle_.done()) return std::nullopt;

        // EN: Check for exceptions that escaped the coroutine
        // TR: Coroutine'den kaçan istisnaları kontrol et
        if (handle_.promise().exception) {
            std::rethrow_exception(handle_.promise().exception);
        }

        return handle_.promise().current_value;
    }

private:
    handle_type handle_;
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: Simple Task — co_await + co_return
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: Task<T> represents an asynchronous computation that produces a single value.
 *     Unlike Generator (which yields many values), Task produces ONE result via co_return.
 *     This is the coroutine equivalent of std::future<T>.
 *
 * TR: Task<T> tek bir değer üreten asenkron hesaplamayı temsil eder.
 *     Generator'dan (birçok değer üreten) farklı olarak, Task co_return ile TEK sonuç üretir.
 *     std::future<T>'nin coroutine karşılığıdır.
 */
template<typename T>
class Task {
public:
    struct promise_type {
        T result;
        std::exception_ptr exception;

        Task get_return_object() {
            return Task{std::coroutine_handle<promise_type>::from_promise(*this)};
        }

        // EN: Lazy start — suspend initially, don't run until explicitly resumed
        // TR: Tembel başlangıç — başlangıçta askıya al, açıkça devam ettirilene kadar çalıştırma
        std::suspend_always initial_suspend() noexcept { return {}; }

        // EN: Suspend at end so caller can read the result before frame is destroyed
        // TR: Çağıran, frame yıkılmadan sonucu okuyabilsin diye sonunda askıya al
        std::suspend_always final_suspend() noexcept { return {}; }

        // EN: co_return value; → store the result
        // TR: co_return value; → sonucu sakla
        void return_value(T value) {
            result = std::move(value);
        }

```

```
        void unhandled_exception() {
            exception = std::current_exception();
        }
    };

    using handle_type = std::coroutine_handle<promise_type>;

    explicit Task(handle_type h) : handle_(h) {}
    ~Task() { if (handle_) handle_.destroy(); }

    Task(const Task&) = delete;
    Task& operator=(const Task&) = delete;
    Task(Task&& other) noexcept : handle_(other.handle_) { other.handle_ = nullptr; }
    Task& operator=(Task&& other) noexcept {
        if (this != &other) {
            if (handle_) handle_.destroy();
            handle_ = other.handle_;
            other.handle_ = nullptr;
        }
        return *this;
    }

    // EN: Run the coroutine to completion and return the result.
    //     Loop because the coroutine may have MULTIPLE co_await suspension points.
    //     Each resume() advances to the next co_await or co_return.
    // TR: Coroutine'i tamamlanana kadar çalıştır ve sonucu döndür.
    //     Coroutine birden fazla co_await askıya alma noktasına sahip olabilir.
    //     Her resume() sonraki co_await veya co_return'e ilerler.
    T get() {
        while (!handle_.done()) {
            handle_.resume();
        }
        if (handle_.promise().exception) {
            std::rethrow_exception(handle_.promise().exception);
        }
        return std::move(handle_.promise().result);
    }

private:
    handle_type handle_;
};

// ============================================================================
// PART 3: Coroutine Functions (the actual coroutines!)
// ============================================================================

// ── 3a: Simple counter generator ──

/**
 * EN: The simplest possible generator: counts from 'start' to 'end'.
 *     Each co_yield suspends the coroutine and makes the value available to the caller.
 *     When the function body ends, the coroutine finishes (return_void is called).
 *
 *     Execution flow step-by-step for count_up(1, 3):
 *       1. Caller calls count_up(1, 3) → coroutine frame allocated on heap
 *       2. initial_suspend() returns suspend_always → coroutine suspends immediately
 *       3. Generator object returned to caller
 *       4. Caller calls next() → handle_.resume()
 *       5. Coroutine runs: i=1, hits co_yield 1 → yield_value(1) stores 1, suspends
 *       6. Control returns to caller, next() returns 1
 *       7. Caller calls next() → handle_.resume()
 *       8. Coroutine continues: i=2, hits co_yield 2 → stores 2, suspends
 *       9. ... and so on until i > end, function body ends, return_void + final_suspend
 *
 * TR: Mümkün olan en basit generator: 'start'tan 'end'e kadar sayar.
 *     Her co_yield coroutine'i askıya alır ve değeri çağırana sunar.
 */
Generator<int> count_up(int start, int end) {
    // EN: This looks like a normal loop, but co_yield makes it a coroutine!
    //     The loop "pauses" at each co_yield and the caller gets the value.
    // TR: Normal bir döngüye benziyor, ama co_yield onu coroutine yapıyor!
    //     Döngü her co_yield'da "duraklar" ve çağıran değeri alır.
    for (int i = start; i <= end; ++i) {
        co_yield i;
```

```
        // EN: ↑ Execution suspends here. Local variable 'i' is preserved in the frame.
        //     When next() is called again, execution continues from the line AFTER co_yield.
        // TR: ↑ Çalışma burada askıya alınır. Yerel değişken 'i' frame'de korunur.
        //     next() tekrar çağrılınca çalışma co_yield'DAN SONRAKİ satırdan devam eder.
    }
    // EN: Function body ends → return_void() is called → final_suspend() suspends
    // TR: Fonksiyon gövdesi biter → return_void() çağrılır → final_suspend() askıya alır
}

// ── 3b: Fibonacci generator (infinite!) ──

/**
 * EN: An INFINITE generator — it never returns, just keeps yielding Fibonacci numbers.
 *     This is perfectly fine! The caller decides when to stop pulling values.
 *     This is the power of lazy evaluation: compute only what you need.
 *     Memory usage: O(1) — only 3 variables in the coroutine frame (a, b, temp).
 *     Compare to pre-computing: storing N Fibonacci numbers needs O(N) memory.
 *
 * TR: SONSUZ bir generator — asla dönmez, Fibonacci sayıları üretmeye devam eder.
 *     Sorun değil! Çağıran ne zaman değer çekmeyi durduracağına karar verir.
 *     Tembel değerlendirmenin gücü: sadece ihtiyacın olanı hesapla.
 *     Bellek kullanımı: O(1) — frame'de sadece 3 değişken (a, b, temp).
 */
Generator<uint64_t> fibonacci() {
    uint64_t a = 0, b = 1;
    while (true) {                // EN: Infinite loop — but suspends at each yield!
        co_yield a;               // TR: Sonsuz döngü — ama her yield'da askıya alınır!
        uint64_t temp = a + b;
        a = b;
        b = temp;
    }
    // EN: This line is never reached — the coroutine runs forever (or until destroyed)
    // TR: Bu satıra asla ulaşılmaz — coroutine sonsuza kadar çalışır (veya yıkılana dek)
}

// ── 3c: Filtered generator — composing coroutines ──

/**
 * EN: Generator that filters another generator's output - only even numbers.
 *     This shows coroutine COMPOSITION: one coroutine wrapping another.
 *     The inner generator produces values, our coroutine filters and re-yields.
 *
 * TR: Başka bir generator'ın çıktısını filtreleyen generator — sadece çift sayılar.
 *     Coroutine BİLEŞİMİNİ gösterir: bir coroutine başkasını sarar.
 */
Generator<uint64_t> even_fibonacci(int count) {
    auto fib = fibonacci();
    int found = 0;
    while (found < count) {
        auto val = fib.next();
        if (!val.has_value()) break;  // EN: Inner generator finished / TR: İç generator bitti
        if (*val % 2 == 0) {
            co_yield *val;
            ++found;
        }
    }
}

// ── 3d: String tokenizer generator ──

/**
 * EN: Real-world use case: tokenize a string lazily.
 *     Instead of splitting the entire string upfront (allocating a vector of strings),
 *     we yield one token at a time. For huge strings, this saves memory.
 *
 * TR: Gerçek dünya kullanımı: string'i tembel olarak tokenize et.
 *     Tüm string'i önceden bölmek yerine (string vektörü ayırmak),
 *     her seferinde bir token üretiriz. Büyük string'ler için bellek tasarrufu.
 */
Generator<std::string> tokenize(const std::string& input, char delimiter) {
    std::string token;
    for (char ch : input) {
```

```
339         if (ch == delimiter) {
340             if (!token.empty()) {
341                 co_yield token;
342                 token.clear();
343             }
344         } else {
345             token += ch;
346         }
347     }
348     if (!token.empty()) {
349         co_yield token;   // EN: Don't forget the last token! / TR: Son token'ı unutma!
350     }
351 }
352
353 // ── 3e: Task-based coroutine — co_return demo ──
354
355 /**
356  * EN: A Task coroutine that does a "computation" and returns a single result.
357  *     Uses co_return instead of co_yield.
358  *     In real code, before co_return you'd co_await some async I/O.
359  *
360  * TR: "Hesaplama" yapan ve tek sonuç döndüren Task coroutine.
361  *     co_yield yerine co_return kullanır.
362  *     Gerçek kodda co_return öncesinde asenkron I/O co_await edersiniz.
363  */
364 // EN: IMPORTANT: Coroutine parameters MUST be taken by VALUE, not by reference!
365 //     Why? Because initial_suspend() suspends BEFORE the body runs.
366 //     If a param is a reference to a temporary, that temporary dies while
367 //     the coroutine is suspended → dangling reference → undefined behavior!
368 //     Rule: Always pass by value to coroutines (the compiler copies into the frame).
369 //
370 // TR: ÖNEMLİ: Coroutine parametreleri referans DEĞİL, DEĞER ile alınmalı!
371 //     Neden? initial_suspend() gövde çalışmadan ÖNCE askıya alır.
372 //     Parametre geçici bir nesneye referans ise, o geçici coroutine askıya alınırken
373 //     ölür → sarkan referans → tanımsız davranış!
374 //     Kural: Coroutine'lere her zaman değer ile geç (derleyici frame'e kopyalar).
375 Task<std::string> compute_sensor_report(std::string sensorId, double temp) {
376     // EN: In a real system, this could co_await a database query or network call
377     // TR: Gerçek sistemde burası veritabanı sorgusu veya ağ çağrısı co_await edebilirdi
378     std::string report = "Sensor " + sensorId + ": " + std::to_string(temp) + "C";
379
380     if (temp > 100.0) {
381         report += " [CRITICAL]";
382     } else if (temp > 75.0) {
383         report += " [WARNING]";
384     } else {
385         report += " [NORMAL]";
386     }
387
388     co_return report;  // EN: Complete the Task with this value
389                        // TR: Task'ı bu değerle tamamla
390 }
391
392 // ═══════════════════════════════════════════════════════════════════════
393 // PART 4: Custom Awaitable — Understanding co_await Mechanics
394 // ═══════════════════════════════════════════════════════════════════════
395
396 /**
397  * EN: A custom awaitable that simulates a "delay" or async operation.
398  *     When co_await is used on this, the three methods are called:
399  *       1. await_ready() → "Is the result available immediately?"
400  *       2. await_suspend(handle) → "Suspend the coroutine, here's the handle"
401  *       3. await_resume() → "Coroutine was resumed, what's the result?"
402  *
403  * TR: "Gecikme" veya asenkron işlemi simüle eden özel bir awaitable.
404  *     co_await bu üzerinde kullanılınca üç metot çağrılır:
405  *       1. await_ready() → "Sonuç hemen mevcut mu?"
406  *       2. await_suspend(handle) → "Coroutine'i askıya al, handle burada"
407  *       3. await_resume() → "Coroutine devam etti, sonuç ne?"
408  */
409 struct SimulatedDelay {
410     std::string operation_name;
411     int result_value;
412
```

```cpp
    // EN: "Is the result immediately available?" → No, we always need to "wait"
    // TR: "Sonuç hemen mevcut mu?" → Hayır, her zaman "beklememiz" gerekiyor
    bool await_ready() const noexcept {
        std::cout << "    [await_ready] '" << operation_name << "' — not ready, will suspend\n";
        return false;  // EN: false = suspend / TR: false = askıya al
    }

    // EN: "The coroutine is being suspended. What should we do?"
    //     We could schedule work on a thread pool, register with an event loop, etc.
    //     For this demo, we just print a message.
    //     IMPORTANT: We can choose to:
    //       - void return → always suspend
    //       - bool return → false = don't actually suspend (optimization)
    //       - coroutine_handle return → symmetric transfer to another coroutine
    // TR: "Coroutine askıya alınıyor. Ne yapmalıyız?"
    //     İş parçacığı havuzunda iş zamanlayabilir, olay döngüsüne kaydedebiliriz.
    void await_suspend(std::coroutine_handle<> /*handle*/) const noexcept {
        std::cout << "    [await_suspend] '" << operation_name << "' — simulating async work...\n";
        // EN: In real code: store the handle, schedule resume on completion
        // TR: Gerçek kodda: handle'ı sakla, tamamlanınca resume zamanla
    }

    // EN: "The coroutine is being resumed. What value should co_await produce?"
    //     This return value becomes the result of the `co_await expr` expression.
    // TR: "Coroutine devam ettiriliyor. co_await hangi değeri üretmeli?"
    int await_resume() const noexcept {
        std::cout << "    [await_resume] '" << operation_name << "' — result = " << result_value << "\n";
        return result_value;
    }
};

/**
 * EN: A coroutine that uses our custom awaitable to demonstrate co_await mechanics.
 *     Each co_await suspends the coroutine and resumes it (in our simple case, immediately).
 *
 * TR: co_await mekanizmasını göstermek için özel awaitable kullanan coroutine.
 */
Task<int> sensor_pipeline() {
    std::cout << "  [coroutine] Starting sensor pipeline...\n";

    // EN: co_await SimulatedDelay{...} calls:
    //     1. SimulatedDelay::await_ready() → false → suspend
    //     2. SimulatedDelay::await_suspend(handle) → prints message
    //     3. (Later, when resumed) SimulatedDelay::await_resume() → returns value
    // TR: co_await SimulatedDelay{...} şunları çağırır:
    //     1. await_ready() → false → askıya al
    //     2. await_suspend(handle) → mesaj yazdır
    //     3. (Devam ettirilince) await_resume() → değer döndür
    int raw_value = co_await SimulatedDelay{"ADC Read", 2048};
    std::cout << "  [coroutine] Got raw ADC value: " << raw_value << "\n";

    int calibrated = co_await SimulatedDelay{"Calibration", raw_value / 4};
    std::cout << "  [coroutine] Calibrated value: " << calibrated << "\n";

    co_return raw_value + calibrated;
}

// ============================================================================
// MAIN
// ============================================================================
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 12 - Coroutine Fundamentals\n";
    std::cout << "============================================\n\n";

```

```
    // ── Demo 1: Simple Counter Generator ──────────────────────────────
    {
        std::cout << "--- Demo 1: Simple Counter Generator (co_yield) ---\n";
        // EN: count_up(1, 5) creates a coroutine that yields 1, 2, 3, 4, 5
        //     Each next() call resumes the coroutine, which runs until the next co_yield
        // TR: count_up(1, 5) 1, 2, 3, 4, 5 üreten bir coroutine oluşturur
        //     Her next() çağrısı coroutine'i devam ettirir, sonraki co_yield'a kadar çalışır
        auto counter = count_up(1, 5);
        while (auto val = counter.next()) {
            std::cout << "  Got: " << *val << "\n";
        }
        std::cout << "  Generator exhausted.\n\n";
    }

    // ── Demo 2: Infinite Fibonacci Generator ──────────────────────────
    {
        std::cout << "--- Demo 2: Infinite Fibonacci Generator ---\n";
        // EN: fibonacci() is an INFINITE generator — it never stops on its own.
        //     We decide to pull only 15 values. Lazy evaluation at its finest!
        //     No array is pre-computed. Each value calculated on-demand.
        // TR: fibonacci() SONSUZ bir generator — kendiliğinden asla durmaz.
        //     Sadece 15 değer çekmeye karar veriyoruz. Tembel değerlendirmenin en iyisi!
        auto fib = fibonacci();
        std::cout << "  First 15 Fibonacci numbers:\n  ";
        for (int i = 0; i < 15; ++i) {
            auto val = fib.next();
            if (val) std::cout << *val << " ";
        }
        std::cout << "\n\n";
        // EN: fib destructor destroys the coroutine frame — no leak!
        // TR: fib destructor'ı coroutine frame'i yıkar — sızıntı yok!
    }

    // ── Demo 3: Composed Generator — Even Fibonacci ───────────────────
    {
        std::cout << "--- Demo 3: Coroutine Composition — Even Fibonacci ---\n";
        // EN: even_fibonacci wraps fibonacci and filters: only yields even numbers
        //     This demonstrates coroutine composition — generators calling generators
        // TR: even_fibonacci, fibonacci'yi sarlayıp filtreler: sadece çift sayıları üretir
        auto even_fib = even_fibonacci(5);
        std::cout << "  First 5 even Fibonacci numbers:\n  ";
        while (auto val = even_fib.next()) {
            std::cout << *val << " ";
        }
        std::cout << "\n\n";
    }

    // ── Demo 4: String Tokenizer Generator ────────────────────────────
    {
        std::cout << "--- Demo 4: Lazy String Tokenizer ---\n";
        // EN: Real-world pattern: tokenize a CAN bus log line lazily
        //     Each token is produced only when requested — no vector<string> allocation
        // TR: Gerçek dünya kalıbı: CAN bus log satırını tembelce tokenize et
        std::string can_log = "1A3:8:DE:AD:BE:EF:00:FF:12:34";
        auto tokens = tokenize(can_log, ':');
        int token_num = 1;
        while (auto tok = tokens.next()) {
            std::cout << "  Token " << token_num++ << ": " << *tok << "\n";
        }
        std::cout << "\n";
    }

    // ── Demo 5: Task with co_return ───────────────────────────────────
    {
        std::cout << "--- Demo 5: Task with co_return ---\n";
```

```
        // EN: Task produces a SINGLE value via co_return (not co_yield)
        //     .get() resumes the coroutine and retrieves the result
        // TR: Task co_return ile TEK değer üretir (co_yield değil)
        //     .get() coroutine'i devam ettirir ve sonucu alır
        auto report1 = compute_sensor_report("EXHAUST_01", 110.5);
        auto report2 = compute_sensor_report("COOLANT_02", 65.0);
        auto report3 = compute_sensor_report("OIL_TEMP", 88.3);

        std::cout << "  " << report1.get() << "\n";
        std::cout << "  " << report2.get() << "\n";
        std::cout << "  " << report3.get() << "\n";
        std::cout << "\n";
    }

    // ─── Demo 6: Custom Awaitable — co_await Mechanics ──────────────────
    {
        std::cout << "--- Demo 6: Custom Awaitable — co_await Mechanics ---\n";
        // EN: This demonstrates the 3-step co_await process:
        //     await_ready → await_suspend → (resume) → await_resume
        //     Our SimulatedDelay always suspends, then returns a value on resume.
        //
        //     In real-world code, await_suspend would register the coroutine handle
        //     with an I/O event loop or thread pool, and resume() would be called
        //     later when the async operation completes.
        //
        // TR: co_await'in 3 adımlı sürecini gösterir:
        //     await_ready → await_suspend → (devam ettir) → await_resume
        //     Gerçek kodda await_suspend, handle'ı I/O olay döngüsüne kaydeder
        //     ve asenkron işlem tamamlanınca resume() çağrılır.
        auto pipeline = sensor_pipeline();
        // EN: pipeline is a Task. get() calls resume() which drives the coroutine.
        //     Inside, each co_await creates a suspension point.
        //     After await_suspend, we call resume again, which triggers await_resume.
        // TR: pipeline bir Task. get(), resume()'ı çağırarak coroutine'i yürütür.

        // EN: We need to manually resume through all suspension points in our simple case
        // TR: Basit durumumuzda tüm askıya alma noktalarından manuel geçmeliyiz
        int result = pipeline.get();
        std::cout << "  [main] Pipeline final result: " << result << "\n";
        std::cout << "\n";
    }

    // ─── Demo 7: Generator as Range-like Iteration ──────────────────────
    {
        std::cout << "--- Demo 7: Collecting Generator to Vector ---\n";
        // EN: You can collect all generator values into a container if needed
        //     This shows the bridge between lazy and eager evaluation
        // TR: Gerekirse tüm generator değerlerini bir konteyner'a toplayabilirsin
        //     Tembel ve hevesli değerlendirme arasındaki köprüyü gösterir
        auto counter = count_up(10, 20);
        std::vector<int> collected;
        while (auto val = counter.next()) {
            collected.push_back(*val);
        }
        std::cout << "  Collected " << collected.size() << " values: [";
        for (size_t i = 0; i < collected.size(); ++i) {
            if (i > 0) std::cout << ", ";
            std::cout << collected[i];
        }
        std::cout << "]\n\n";
    }

    std::cout << "============================================\n";
    std::cout << "   End of Module 12 - Coroutine Fundamentals\n";
    std::cout << "============================================\n";

    return 0;
}
```

► 02. Async Coroutine Patterns

module_12_coroutines/02_async_coroutine_patterns.cpp

Konu: Advanced Async Coroutine Patterns — Gelişmiş Asenkron Coroutine Kalıpları

Teori / Theory

THEORY: Symmetric Transfer

When one coroutine awaits another, a naive implementation does: caller.suspend() → callee.resume() → ... → callee.suspend() → caller.resume() This nests coroutine frames on the system STACK. Chain enough coroutines and you get STACK OVERFLOW — one of the hardest coroutine bugs.
Symmetric Transfer solves this: instead of returning to the caller and then resuming the next coroutine, await_suspend returns a coroutine_handle that the COMPILER resumes directly — no stack growth!

```
┌──────────────────────────┬──────────────────────────────────────────┐
│ await_suspend return type │ Behavior                                 │
├──────────────────────────┼──────────────────────────────────────────┤
│ void                     │ Suspend unconditionally, return to resumer  │
│ bool                     │ true→suspend, false→don't suspend        │
│ coroutine_handle<>       │ SYMMETRIC TRANSFER: suspend current,     │
│                          │ resume returned handle (no stack growth)    │
└──────────────────────────┴──────────────────────────────────────────┘
```

std::noop_coroutine() returns a handle that does nothing when resumed. Used as a "return to caller" sentinel in the symmetric transfer chain.

TEORİ: Simetrik Transfer

Bir coroutine başka birini beklediğinde, naif uygulama şunu yapar: çağıran.askı() → çağrılan.devam() → ... → çağrılan.askı() → çağıran.devam() Bu, coroutine frame'lerini sistem YIĞINına (stack) yerleştirir. Yeterince coroutine zincirleyin → YIĞIN TAŞMASI — en zor coroutine hatalarından biri.
Simetrik Transfer bunu çözer: çağırana dönüp sonraki coroutine'i devam ettirmek yerine, await_suspend doğrudan DERLEYİCİNİN devam ettirdiği bir coroutine_handle döndürür — yığın büyümesi yok!

```
┌──────────────────────────┬──────────────────────────────────────────┐
│ await_suspend dönüş tipi  │ Davranış                                 │
├──────────────────────────┼──────────────────────────────────────────┤
│ void                     │ Koşulsuz askıya al, yeniden başlatıcıya dön │
│ bool                     │ true→askıya al, false→askıya alma        │
│ coroutine_handle<>       │ SİMETRİK TRANSFER: mevcut'u askıya al,   │
│                          │ döndürülen handle'i devam ettir (yığın yok) │
└──────────────────────────┴──────────────────────────────────────────┘
```

std::noop_coroutine() devam ettirildiğinde hiçbir şey yapmayan bir handle döndürür. Simetrik transfer zincirinde "çağırana dön" nöbetçisi olarak kullanılır.

THEORY: Coroutine Exception Handling

Exceptions in coroutines follow specific rules:
1. If an exception escapes the coroutine BODY:
→ promise.unhandled_exception() is called
→ Typically stores it: current_exception_ = std::current_exception()
→ Then final_suspend() runs (coroutine is at final suspend point)
2. The CALLER retrieves the exception later (when calling .get() or similar):
→ std::rethrow_exception(current_exception_) propagates it
3. If an exception happens DURING co_await (in await_suspend):
→ The coroutine is NOT suspended (it was in the process of suspending)
→ The exception propagates to the CALLER immediately

Where exception occurs	What happens
Coroutine body	unhandled_exception() stores it
co_await (await_suspend)	Propagates to caller immediately
co_await (await_resume)	Propagates inside coroutine body
promise constructor	Coroutine frame destroyed, throws

TEORİ: Coroutine'de Hata Yönetimi

Coroutine'lerdeki istisnalar belirli kuralları takip eder:
1. İstisna coroutine GÖVDESİNDEN kaçarsa:
→ promise.unhandled_exception() çağrılır
→ Genellikle saklar: current_exception_ = std::current_exception()
→ Sonra final_suspend() çalışır
2. ÇAĞIRAN istisnayı daha sonra alır (.get() veya benzeri çağrısında):
→ std::rethrow_exception(current_exception_) yayar
3. İstisna co_await SIRASINDA (await_suspend'de) olursa:
→ Coroutine askıya ALINMAZ
→ İstisna ÇAĞIRANA hemen yayılır *

İstisnanın oluştuğu yer	Ne olur
Coroutine gövdesi	unhandled_exception() saklar
co_await (await_suspend)	Çağırana hemen yayılır
co_await (await_resume)	Coroutine gövdesinde yayılır
promise constructor	Coroutine frame yok edilir, fırlatır

THEORY: Coroutine Memory & Lifetime

Coroutine frames are typically HEAP allocated. The compiler MAY elide this allocation via HALO (Heap Allocation eLision Optimization) if:
- The coroutine's lifetime is bounded by the caller
- The frame size is known at compile time
- The compiler can prove the coroutine doesn't outlive the frame holder

Critical lifetime pitfalls:

Pitfall	Consequence
Reference parameter to coroutine	Dangling reference after caller returns (frame outlives caller)
Lambda capturing local by ref	Dangling ref when lambda used as coroutine body
Destroying handle before final_suspend	Memory leak (frame not freed) OR double free (if promise owns)
Forgetting to destroy handle	Memory leak of coroutine frame

Best practice: Coroutine parameters should be taken BY VALUE. The coroutine frame copies them, so they survive suspension.

TEORİ: Bellek ve Yaşam Süresi

Coroutine frame'leri tipik olarak HEAP'te tahsis edilir. Derleyici bu tahsisi HALO (Heap Allocation eLision Optimization) ile eleyebilir:
- Coroutine'in yaşam süresi çağıran tarafından sınırlıysa
- Frame boyutu derleme zamanında biliniyorsa
- Derleyici coroutine'in frame tutucudan uzun yaşamadığını kanıtlayabilirse

Kritik yaşam süresi tuzakları:

Tuzak	Sonuç
Coroutine'e referans parametre	Çağıran döndükten sonra sarkan referans (frame çağırandan uzun)
Referansla yakalayan lambda	Coroutine gövdesi olarak kullanılırsa sarkan ref
Handle'ı final_suspend'den önce yok etmek	Bellek sızıntısı (frame serbest bırakılmaz) VEYA çift serbest
Handle'ı yok etmeyi unutmak	Coroutine frame bellek sızıntısı

En iyi uygulama: Coroutine parametreleri DEĞER ile alınmalıdır. Coroutine frame onları kopyalar, böylece askıya almadan sonra yaşarlar.

THEORY: Async Event Loop

A coroutine scheduler is an event loop that manages multiple coroutines. When a coroutine suspends, the scheduler picks the next ready coroutine. When an async operation completes, the scheduler resumes the waiting coroutine.
This is the foundation of all async frameworks (Boost.Asio, libuv, tokio):

```
┌──────────────────────────────────────────────────────────────┐
│ Scheduler (Event Loop)                                       │
│ ┌──────────────┐ ┌──────────────┐ ┌──────────────┐           │
│ │ Coroutine A │ │ Coroutine B │ │ Coroutine C │ ← Ready Queue  │
│ └──────┬───────┘ └──────┬───────┘ └──────┬───────┘           │
│        │                │                │                   │
│   co_await          co_await        co_yield                 │
│   (I/O wait)        (timer)         (value)                  │
│        │                │                │                   │
│   ┌────▼────────────────▼────────────────▼────┐              │
│   │          event_loop.run()                 │              │
│   │   Pick next ready → resume → repeat       │              │
│   └───────────────────────────────────────────┘              │
└──────────────────────────────────────────────────────────────┘
```

TEORİ: Asenkron Olay Döngüsü

Bir coroutine zamanlayıcısı, birden fazla coroutine'i yöneten bir olay döngüsüdür. Bir coroutine askıya alındığında, zamanlayıcı bir sonraki hazır coroutine'i seçer. Asenkron bir işlem tamamlandığında, bekleyen coroutine'i devam ettirir.
Bu, tüm async çerçevelerinin (Boost.Asio, libuv, tokio) temelidir:

```
┌──────────────────────────────────────────────────────────────┐
│ Zamanlayıcı (Olay Döngüsü)                                   │
│ ┌──────────────┐ ┌──────────────┐ ┌──────────────┐           │
│ │ Coroutine A │ │ Coroutine B │ │ Coroutine C │ ← Hazır Kuyruğu │
│ └──────┬───────┘ └──────┬───────┘ └──────┬───────┘           │
│        │                │                │                   │
│   co_await          co_await        co_yield                 │
│   (I/O bekle)       (zamanlayıcı)    (değer)                 │
│        │                │                │                   │
│   ┌────▼────────────────▼────────────────▼──────┐            │
│   │          olay_dongusu.run()                 │            │
│   │   Sonraki hazırı seç → devam ettir → tekrarla│            │
│   └─────────────────────────────────────────────┘            │
└──────────────────────────────────────────────────────────────┘
```

→ Ref: https://en.cppreference.com/w/cpp/coroutine/coroutine_handle
→ Ref: https://en.cppreference.com/w/cpp/coroutine/noop_coroutine
→ Ref: https://lewissbaker.github.io/2020/05/11/understanding_symmetric_transfer

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <coroutine>
#include <exception>
#include <stdexcept>
#include <queue>
#include <vector>
#include <functional>
#include <string>
#include <optional>
#include <variant>
#include <atomic>
#include <chrono>
```

```
13  #include <cassert>
14
15  // ==============================================================================
16  // PART 1: Async Task with Exception Propagation
17  // ==============================================================================
18
19  /**
20   * EN: Task<T> — An async coroutine type that supports:
21   *     - co_return to set a result value
22   *     - Exception propagation from coroutine body to caller
23   *     - Symmetric transfer to avoid stack overflow in coroutine chains
24   *     - RAII: the coroutine frame is destroyed when the Task goes out of scope
25   *
26   *     Unlike a Generator (which yields multiple values), a Task produces ONE result
27   *     and may throw an exception. The caller retrieves the result via .result().
28   *
29   * TR: Task<T> — Şunları destekleyen asenkron coroutine türü:
30   *     - co_return ile sonuç değeri ayarlama
31   *     - Coroutine gövdesinden çağırana istisna yayma
32   *     - Coroutine zincirlerinde yığın taşmasını önlemek için simetrik transfer
33   *     - RAII: Task kapsam dışına çıktığında coroutine frame yok edilir
34   *
35   *     Generator'dan farklı olarak (birden fazla değer üretir), Task TEK sonuç üretir
36   *     ve istisna fırlatabilir. Çağıran sonucu .result() ile alır.
37   */
38  template<typename T>
39  class Task {
40  public:
41      struct promise_type {
42          /// EN: Stores the result: either a T value or an exception_ptr
43          /// TR: Sonucu saklar: ya bir T değeri ya da exception_ptr
44          std::variant<std::monostate, T, std::exception_ptr> result_;
45
46          /// EN: Handle of the coroutine that is awaiting THIS task (the continuation)
47          /// TR: BU görevi bekleyen coroutine'in handle'ı (devam)
48          std::coroutine_handle<> continuation_ = std::noop_coroutine();
49
50          /// EN: Create the Task object from the promise
51          /// TR: Promise'ten Task nesnesini oluştur
52          Task get_return_object() {
53              return Task{std::coroutine_handle<promise_type>::from_promise(*this)};
54          }
55
56          /// EN: Don't start executing until awaited (lazy start)
57          ///     This is critical for symmetric transfer to work correctly.
58          /// TR: Beklenene kadar çalışmaya başlama (tembel başlatma)
59          ///     Simetrik transferin doğru çalışması için kritik.
60          std::suspend_always initial_suspend() noexcept { return {}; }
61
62          /// EN: At final suspend, use symmetric transfer to resume the awaiting coroutine.
63          ///     Returns a coroutine_handle (not void!) → compiler does symmetric transfer.
64          /// TR: Son askıda, bekleyen coroutine'i devam ettirmek için simetrik transfer kullan.
65          ///     void değil coroutine_handle döndürür → derleyici simetrik transfer yapar.
66          struct FinalAwaiter {
67              bool await_ready() noexcept { return false; }
68
69              /// EN: This is where symmetric transfer happens — we don't return to
70              ///     the resumer, we DIRECTLY resume the continuation coroutine.
71              /// TR: Simetrik transfer burada gerçekleşir — yeniden başlatıcıya
72              ///     dönmeyiz, doğrudan devam coroutine'ini devam ettiririz.
73              std::coroutine_handle<> await_suspend(
74                  std::coroutine_handle<promise_type> h) noexcept {
75                  return h.promise().continuation_;
76              }
77
78              void await_resume() noexcept {}
79          };
80
81          FinalAwaiter final_suspend() noexcept { return {}; }
82
83          /// EN: Store the return value / TR: Dönüş değerini sakla
84          void return_value(T value) {
85              result_.template emplace<1>(std::move(value));
86          }
87
```

```cpp
        /// EN: Capture the exception for later propagation to the caller.
        ///     std::current_exception() captures the active exception.
        /// TR: İstisnayı çağırana daha sonra yayma için yakala.
        ///     std::current_exception() aktif istisnayı yakalar.
        void unhandled_exception() {
            result_.template emplace<2>(std::current_exception());
        }
    };

    using handle_type = std::coroutine_handle<promise_type>;

    // EN: RAII: destroy the coroutine frame when Task is destroyed
    // TR: RAII: Task yok edildiğinde coroutine frame'i yok et
    explicit Task(handle_type h) : handle_(h) {}
    ~Task() { if (handle_) handle_.destroy(); }

    // EN: Move-only (transferring frame ownership)
    // TR: Sadece taşınabilir (frame sahipliğini transfer eder)
    Task(Task&& other) noexcept : handle_(other.handle_) { other.handle_ = nullptr; }
    Task& operator=(Task&& other) noexcept {
        if (this != &other) {
            if (handle_) handle_.destroy();
            handle_ = other.handle_;
            other.handle_ = nullptr;
        }
        return *this;
    }
    Task(const Task&) = delete;
    Task& operator=(const Task&) = delete;

    /// EN: Make Task co_awaitable — this lets you write: auto val = co_await someTask();
    /// TR: Task'ı co_await edilebilir yap — şöyle yazabilirsiniz: auto val = co_await someTask();
    struct Awaiter {
        handle_type handle;

        bool await_ready() noexcept {
            // EN: If the coroutine is already done, no need to suspend
            // TR: Coroutine zaten bittiyse askıya almaya gerek yok
            return handle.done();
        }

        /// EN: Symmetric transfer: store continuation handle, then resume the task.
        ///     Returning handle means: "resume THIS task's coroutine next"
        /// TR: Simetrik transfer: devam handle'ını sakla, sonra görevi devam ettir.
        ///     Handle döndürmek: "sonra BU görevin coroutine'ini devam ettir" demek
        std::coroutine_handle<> await_suspend(std::coroutine_handle<> awaiting) noexcept {
            handle.promise().continuation_ = awaiting;
            return handle;
        }

        /// EN: Retrieve the result (or rethrow the stored exception)
        /// TR: Sonucu al (veya saklanan istisnayı yeniden fırlat)
        T await_resume() {
            auto& result = handle.promise().result_;
            if (std::holds_alternative<std::exception_ptr>(result)) {
                std::rethrow_exception(std::get<std::exception_ptr>(result));
            }
            return std::move(std::get<T>(result));
        }
    };

    Awaiter operator co_await() noexcept { return Awaiter{handle_}; }

    /// EN: Synchronous retrieval — for calling from non-coroutine code (e.g., main)
    ///     Resumes the coroutine and returns the result or rethrows exception.
    /// TR: Senkron alma — coroutine olmayan koddan çağırmak için (ör. main)
    ///     Coroutine'i devam ettirir ve sonucu döndürür veya istisnayı yeniden fırlatır.
    T result() {
        if (!handle_.done()) {
            handle_.resume();
        }
        auto& r = handle_.promise().result_;
        if (std::holds_alternative<std::exception_ptr>(r)) {
            std::rethrow_exception(std::get<std::exception_ptr>(r));
        }
        return std::move(std::get<T>(r));
    }
```

```cpp

private:
    handle_type handle_;
};

/// EN: Specialization for void return type / TR: void dönüş tipi için özelleştirme
template<>
class Task<void> {
public:
    struct promise_type {
        std::exception_ptr exception_;
        std::coroutine_handle<> continuation_ = std::noop_coroutine();

        Task get_return_object() {
            return Task{std::coroutine_handle<promise_type>::from_promise(*this)};
        }
        std::suspend_always initial_suspend() noexcept { return {}; }

        struct FinalAwaiter {
            bool await_ready() noexcept { return false; }
            std::coroutine_handle<> await_suspend(
                std::coroutine_handle<promise_type> h) noexcept {
                return h.promise().continuation_;
            }
            void await_resume() noexcept {}
        };
        FinalAwaiter final_suspend() noexcept { return {}; }

        void return_void() {}
        void unhandled_exception() { exception_ = std::current_exception(); }
    };

    using handle_type = std::coroutine_handle<promise_type>;

    explicit Task(handle_type h) : handle_(h) {}
    ~Task() { if (handle_) handle_.destroy(); }
    Task(Task&& o) noexcept : handle_(o.handle_) { o.handle_ = nullptr; }
    Task& operator=(Task&& o) noexcept {
        if (this != &o) { if (handle_) handle_.destroy(); handle_ = o.handle_; o.handle_ = nullptr; }
        return *this;
    }
    Task(const Task&) = delete;
    Task& operator=(const Task&) = delete;

    struct Awaiter {
        handle_type handle;
        bool await_ready() noexcept { return handle.done(); }
        std::coroutine_handle<> await_suspend(std::coroutine_handle<> awaiting) noexcept {
            handle.promise().continuation_ = awaiting;
            return handle;
        }
        void await_resume() {
            if (handle.promise().exception_)
                std::rethrow_exception(handle.promise().exception_);
        }
    };
    Awaiter operator co_await() noexcept { return Awaiter{handle_}; }

    void result() {
        if (!handle_.done()) handle_.resume();
        if (handle_.promise().exception_)
            std::rethrow_exception(handle_.promise().exception_);
    }

private:
    handle_type handle_;
};

```

```cpp
// ═══════════════════════════════════════════════════════════════════════════
// PART 2: Coroutine Scheduler (Round-Robin)
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: A simple round-robin coroutine scheduler. Coroutines yield control back
 *     to the scheduler which then picks the next one. This demonstrates how
 *     async frameworks (Boost.Asio, libuv) work under the hood.
 *
 *     How it works:
 *     1. Coroutines call co_await scheduler.suspend() to yield control
 *     2. The scheduler puts the yielding coroutine back in the ready queue
 *     3. The scheduler resumes the next coroutine in the queue
 *     4. When all coroutines are done, the loop exits
 *
 * TR: Basit bir round-robin coroutine zamanlayıcısı. Coroutine'ler kontrolü
 *     zamanlayıcıya geri verir, o da bir sonrakini seçer. Bu, async çerçevelerinin
 *     (Boost.Asio, libuv) perde arkasında nasıl çalıştığını gösterir.
 *
 *     Nasıl çalışır:
 *     1. Coroutine'ler kontrolü vermek için co_await scheduler.suspend() çağırır
 *     2. Zamanlayıcı, veren coroutine'i hazır kuyruğuna geri koyar
 *     3. Zamanlayıcı, kuyruktaki sonraki coroutine'i devam ettirir
 *     4. Tüm coroutine'ler bittiğinde döngü çıkar
 */
class Scheduler {
    std::queue<std::coroutine_handle<>> ready_queue_;

public:
    /// EN: A custom awaitable that puts the current coroutine back in the queue
    ///     and yields control to the scheduler.
    /// TR: Mevcut coroutine'i kuyruğa geri koyan ve kontrolü zamanlayıcıya
    ///     veren özel bir awaitable.
    struct SchedulerAwaiter {
        Scheduler& scheduler;

        bool await_ready() noexcept { return false; }  // EN: Always suspend / TR: Her zaman askıya al

        /// EN: Save the current coroutine handle and return to the scheduler.
        ///     The scheduler will pick the next coroutine to run.
        /// TR: Mevcut coroutine handle'ını kaydet ve zamanlayıcıya dön.
        ///     Zamanlayıcı çalıştırılacak sonraki coroutine'i seçecek.
        void await_suspend(std::coroutine_handle<> h) {
            scheduler.ready_queue_.push(h);
        }

        void await_resume() noexcept {}
    };

    /// EN: Returns an awaitable that yields control to the scheduler
    /// TR: Kontrolü zamanlayıcıya veren bir awaitable döndürür
    SchedulerAwaiter suspend() { return SchedulerAwaiter{*this}; }

    /// EN: Schedule a coroutine for execution (add to ready queue)
    /// TR: Bir coroutine'i çalıştırma için zamanla (hazır kuyruğuna ekle)
    void schedule(std::coroutine_handle<> h) {
        ready_queue_.push(h);
    }

    /// EN: Run until all coroutines complete (the event loop)
    /// TR: Tüm coroutine'ler tamamlanana kadar çalış (olay döngüsü)
    void run() {
        while (!ready_queue_.empty()) {
            auto h = ready_queue_.front();
            ready_queue_.pop();

            if (!h.done()) {
                h.resume();
            }
        }
    }

    [[nodiscard]] std::size_t pending() const { return ready_queue_.size(); }
};

```

```cpp
/// EN: ScheduledTask — a coroutine type designed to work with the Scheduler.
///     Unlike Task<T>, this one auto-registers with the scheduler at initial_suspend.
/// TR: ScheduledTask — Zamanlayıcı ile çalışmak üzere tasarlanmış coroutine türü.
///     Task<T>'den farklı olarak, initial_suspend'de zamanlayıcıya otomatik kaydolur.
class ScheduledTask {
public:
    struct promise_type {
        ScheduledTask get_return_object() {
            return ScheduledTask{std::coroutine_handle<promise_type>::from_promise(*this)};
        }

        /// EN: Lazy start — scheduler will resume us when ready
        /// TR: Tembel başlatma — zamanlayıcı hazır olunca devam ettirecek
        std::suspend_always initial_suspend() noexcept { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }
        void return_void() {}
        void unhandled_exception() { std::terminate(); }
    };

    using handle_type = std::coroutine_handle<promise_type>;

    explicit ScheduledTask(handle_type h) : handle_(h) {}
    ~ScheduledTask() { if (handle_) handle_.destroy(); }

    ScheduledTask(ScheduledTask&& o) noexcept : handle_(o.handle_) { o.handle_ = nullptr; }
    ScheduledTask& operator=(ScheduledTask&&) = delete;
    ScheduledTask(const ScheduledTask&) = delete;
    ScheduledTask& operator=(const ScheduledTask&) = delete;

    /// EN: Get the underlying coroutine handle for scheduling
    /// TR: Zamanlama için altta yatan coroutine handle'ını al
    [[nodiscard]] handle_type handle() const { return handle_; }

private:
    handle_type handle_;
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: Async Pipeline (Producer → Transform → Consumer)
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: Demonstrates a coroutine pipeline where data flows through stages:
 *     Producer → Transform → Consumer
 *     Each stage is a coroutine. The pipeline is driven by the scheduler.
 *
 *     This models real-world data processing:
 *     - Sensor data → Filter → Display (automotive dashboard)
 *     - Network packet → Parse → Business logic (server)
 *     - Audio samples → DSP → Output (audio processing)
 *
 * TR: Verinin aşamalardan aktığı coroutine pipeline gösterir:
 *     Üretici → Dönüştür → Tüketici
 *     Her aşama bir coroutine'dir. Pipeline zamanlayıcı tarafından yürütülür.
 *
 *     Gerçek dünya veri işlemesini modeller:
 *     - Sensör verisi → Filtre → Gösterim (otomotiv gösterge paneli)
 *     - Ağ paketi → Ayrıştır → İş mantığı (sunucu)
 *     - Ses örnekleri → DSP → Çıkış (ses işleme)
 */

/// EN: A simple channel for passing data between coroutines (single-producer, single-consumer)
/// TR: Coroutine'ler arası veri geçişi için basit kanal (tek üretici, tek tüketici)
template<typename T>
class Channel {
    std::queue<T> buffer_;
    std::coroutine_handle<> waiting_consumer_ = nullptr;
    bool closed_ = false;

public:
    /// EN: Send a value into the channel / TR: Kanala bir değer gönder
    void send(T value) {
        buffer_.push(std::move(value));
```

```
        if (waiting_consumer_ && !waiting_consumer_.done()) {
            auto consumer = waiting_consumer_;
            waiting_consumer_ = nullptr;
            consumer.resume();
        }
    }

    /// EN: Close the channel — no more values will be sent
    /// TR: Kanalı kapat — daha fazla değer gönderilmeyecek
    void close() { closed_ = true; }

    /// EN: Check if there's a value to receive / TR: Alınacak değer var mı kontrol et
    [[nodiscard]] bool has_value() const { return !buffer_.empty(); }
    [[nodiscard]] bool is_closed() const { return closed_ && buffer_.empty(); }

    /// EN: Try to receive a value (non-blocking) / TR: Değer almayı dene (bloklamayan)
    std::optional<T> try_receive() {
        if (buffer_.empty()) return std::nullopt;
        T val = std::move(buffer_.front());
        buffer_.pop();
        return val;
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 4: Cancellation Token Pattern
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: A CancellationToken allows cooperative cancellation of coroutines.
 *     The caller sets the token; the coroutine checks it at each suspension point.
 *
 *     This is the same pattern used in:
 *     - .NET CancellationToken
 *     - Kotlin coroutine cancellation (isActive check)
 *     - Automotive ECU: diagnostic session timeout → cancel running diagnostics
 *
 *     Key principle: cancellation is COOPERATIVE, not preemptive.
 *     The coroutine must voluntarily check the token and exit gracefully.
 *
 * TR: CancellationToken, coroutine'lerin işbirlikçi iptali sağlar.
 *     Çağıran token'ı ayarlar; coroutine her askıya alma noktasında kontrol eder.
 *
 *     Aynı kalıp şuralarda kullanılır:
 *     - .NET CancellationToken
 *     - Kotlin coroutine iptali (isActive kontrolü)
 *     - Otomotiv ECU: teşhis oturumu zaman aşımı → çalışan teşhisleri iptal et
 *
 *     Temel ilke: iptal İŞBİRLİKÇİDİR, öncelikli değildir.
 *     Coroutine gönüllü olarak token'ı kontrol etmeli ve zarif bir şekilde çıkmalıdır.
 */
class CancellationToken {
    std::atomic<bool> cancelled_{false};

public:
    /// EN: Request cancellation / TR: İptal iste
    void cancel() { cancelled_.store(true, std::memory_order_release); }

    /// EN: Check if cancellation was requested / TR: İptal istenip istenmediğini kontrol et
    [[nodiscard]] bool is_cancelled() const {
        return cancelled_.load(std::memory_order_acquire);
    }

    /// EN: Reset for reuse / TR: Yeniden kullanım için sıfırla
    void reset() { cancelled_.store(false, std::memory_order_release); }
};

/// EN: Awaitable that throws if cancellation is requested — use in coroutine body:
///     co_await check_cancellation(token);
/// TR: İptal istenirse fırlatan awaitable — coroutine gövdesinde kullan:
///     co_await check_cancellation(token);
struct CancellationAwaitable {
    const CancellationToken& token;

    bool await_ready() { return false; }

    /// EN: Check cancellation at the suspension point. If cancelled, don't suspend
```

```
    /// TR: Askıya alma noktasında iptali kontrol et. İptal edildiyse askıya alma
    bool await_suspend(std::coroutine_handle<>) {
        // EN: Return false → don't actually suspend, continue to await_resume
        //     Return true → suspend (yield control point for scheduler)
        // TR: false döndür → askıya alma, await_resume'a devam et
        //     true döndür → askıya al (zamanlayıcı için kontrol verme noktası)
        return !token.is_cancelled();
    }

    void await_resume() {
        if (token.is_cancelled()) {
            throw std::runtime_error("Coroutine cancelled");
        }
    }
};

CancellationAwaitable check_cancellation(const CancellationToken& token) {
    return CancellationAwaitable{token};
}

// ═══════════════════════════════════════════════════════════════════════════
// PART 5: Real-World Example — Async Sensor Polling (Automotive)
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: Simulates an automotive scenario where multiple sensors are polled
 *     asynchronously using coroutines. Each sensor reading is a coroutine
 *     that "awaits" I/O completion. The scheduler runs them cooperatively.
 *
 *     Scenario: ECU reads 3 sensors (temperature, pressure, RPM) in parallel.
 *     Each sensor has different latency. The coroutine scheduler interleaves them.
 *
 * TR: Birden fazla sensörün coroutine'ler kullanılarak asenkron olarak sorgulandığı
 *     otomotiv senaryosunu simüle eder. Her sensör okuması, I/O tamamlanmasını
 *     "bekleyen" bir coroutine'dir. Zamanlayıcı bunları işbirlikçi olarak çalıştırır.
 *
 *     Senaryo: ECU, 3 sensörü (sıcaklık, basınç, RPM) paralel okur.
 *     Her sensörün farklı gecikmesi var. Coroutine zamanlayıcısı araya girer.
 */
struct SensorData {
    std::string name;
    double value;
    std::string unit;
};

// ═══════════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 12 — Async Coroutine Patterns\n";
    std::cout << "============================================\n\n";

    // ─── Demo 1: Task<T> with Exception Propagation ──────────────────────
    {
        std::cout << "--- Demo 1: Task<T> with Exception Propagation ---\n";

        // EN: A coroutine that computes a value successfully
        // TR: Başarıyla değer hesaplayan bir coroutine
        auto compute_square = [](int x) -> Task<int> {
            std::cout << "  [coroutine] Computing " << x << " squared...\n";
            co_return x * x;
        };

        auto task1 = compute_square(7);
        std::cout << "  Result: " << task1.result() << "\n";

        // EN: A coroutine that throws an exception
        // TR: İstisna fırlatan bir coroutine
        auto failing_task = []() -> Task<int> {
            std::cout << "  [coroutine] About to throw...\n";
            throw std::runtime_error("Sensor read timeout!");
            co_return 0;  // EN: Never reached / TR: Asla ulaşılmaz
        };

```

```cpp
        auto task2 = failing_task();
        try {
            auto val = task2.result();
            std::cout << "  Unexpected result: " << val << "\n";
        } catch (const std::runtime_error& e) {
            // EN: Exception was stored in the promise and re-thrown at result()
            // TR: İstisna promise'te saklandı ve result()'ta yeniden fırlatıldı
            std::cout << "  Caught exception: " << e.what() << "\n";
        }

        // EN: A Task<void> coroutine
        // TR: Task<void> coroutine
        auto void_task = []() -> Task<void> {
            std::cout << "  [coroutine] Void task executing...\n";
            co_return;
        };

        auto task3 = void_task();
        task3.result();
        std::cout << "  Void task completed successfully.\n\n";
    }

    // ─── Demo 2: Chained Tasks (Symmetric Transfer) ──────────────────
    {
        std::cout << "--- Demo 2: Chained Tasks (Symmetric Transfer) ---\n";

        // EN: inner_task is co_awaited by outer_task — symmetric transfer prevents stack growth
        // TR: inner_task, outer_task tarafından co_await edilir — simetrik transfer yığın büyümesini önler
        auto inner_task = []() -> Task<int> {
            std::cout << "  [inner] Computing value...\n";
            co_return 42;
        };

        auto outer_task = [&inner_task]() -> Task<std::string> {
            std::cout << "  [outer] Awaiting inner task...\n";
            int val = co_await inner_task();
            std::cout << "  [outer] Got " << val << " from inner task\n";
            co_return "Answer is " + std::to_string(val);
        };

        auto result_task = outer_task();
        std::cout << "  Final result: " << result_task.result() << "\n";

        // EN: Exception propagation through chain: inner throws → outer catches
        // TR: Zincir boyunca istisna yayma: inner fırlatır → outer yakalar
        auto failing_inner = []() -> Task<int> {
            throw std::logic_error("Inner computation failed");
            co_return 0;
        };

        auto catching_outer = [&failing_inner]() -> Task<std::string> {
            try {
                int val = co_await failing_inner();
                co_return "Got: " + std::to_string(val);
            } catch (const std::logic_error& e) {
                std::cout << "  [outer] Caught from inner: " << e.what() << "\n";
                co_return "Fallback: error handled";
            }
        };

        auto chain_result = catching_outer();
        std::cout << "  Chain result: " << chain_result.result() << "\n\n";
    }
```

```cpp

    // ─── Demo 3: Coroutine Scheduler (Round-Robin) ──────────────────────
    {
        std::cout << "--- Demo 3: Coroutine Scheduler (Round-Robin) ---\n";

        Scheduler scheduler;

        // EN: Create 3 coroutines that cooperatively yield to the scheduler
        // TR: Zamanlayıcıya işbirlikçi olarak veren 3 coroutine oluştur
        auto make_task = [&scheduler](std::string name, int steps) -> ScheduledTask {
            for (int i = 1; i <= steps; ++i) {
                std::cout << "  [" << name << "] Step " << i << "/" << steps << "\n";
                co_await scheduler.suspend();
            }
            std::cout << "  [" << name << "] DONE\n";
        };

        // EN: Create coroutines (they don't start until scheduled)
        // TR: Coroutine'leri oluştur (zamanlanana kadar başlamazlar)
        auto taskA = make_task("TaskA", 3);
        auto taskB = make_task("TaskB", 2);
        auto taskC = make_task("TaskC", 4);

        // EN: Register with the scheduler
        // TR: Zamanlayıcıya kaydet
        scheduler.schedule(taskA.handle());
        scheduler.schedule(taskB.handle());
        scheduler.schedule(taskC.handle());

        // EN: Run the event loop — observe round-robin interleaving
        // TR: Olay döngüsünü çalıştır — round-robin araya girmeyi gözlemle
        std::cout << "  [Scheduler] Starting event loop...\n";
        scheduler.run();
        std::cout << "  [Scheduler] All tasks completed.\n\n";
    }

    // ─── Demo 4: Async Pipeline ─────────────────────────────────────────
    {
        std::cout << "--- Demo 4: Async Pipeline (Producer → Transform → Consumer) ---\n";

        Scheduler scheduler;
        Channel<int> raw_channel;
        Channel<std::string> processed_channel;

        // EN: Producer: generates raw sensor values
        // TR: Üretici: ham sensör değerleri üretir
        auto producer = [&scheduler, &raw_channel]() -> ScheduledTask {
            int values[] = {25, 80, 3500, 42, 95};
            for (int v : values) {
                std::cout << "  [Producer] Sending raw value: " << v << "\n";
                raw_channel.send(v);
                co_await scheduler.suspend();
            }
            raw_channel.close();
            std::cout << "  [Producer] Channel closed.\n";
        };

        // EN: Transformer: reads raw values, processes them, sends to next stage
        // TR: Dönüştürücü: ham değerleri okur, işler, sonraki aşamaya gönderir
        auto transformer = [&scheduler, &raw_channel, &processed_channel]() -> ScheduledTask {
            while (!raw_channel.is_closed()) {
                auto val = raw_channel.try_receive();
                if (val) {
                    std::string result = "Processed(" + std::to_string(*val) + " → "
                                         + std::to_string(*val * 2) + ")";
                    std::cout << "  [Transform] " << result << "\n";
                    processed_channel.send(std::move(result));
                }
                co_await scheduler.suspend();
            }
            processed_channel.close();
            std::cout << "  [Transform] Done.\n";
        };
```

```cpp

        // EN: Consumer: reads processed values and displays them
        // TR: Tüketici: işlenmiş değerleri okur ve gösterir
        auto consumer = [&scheduler, &processed_channel]() -> ScheduledTask {
            while (!processed_channel.is_closed()) {
                auto val = processed_channel.try_receive();
                if (val) {
                    std::cout << "  [Consumer] Received: " << *val << "\n";
                }
                co_await scheduler.suspend();
            }
            std::cout << "  [Consumer] Pipeline complete.\n";
        };

        auto prod = producer();
        auto trans = transformer();
        auto cons = consumer();

        scheduler.schedule(prod.handle());
        scheduler.schedule(trans.handle());
        scheduler.schedule(cons.handle());

        scheduler.run();
        std::cout << "\n";
    }

    // ─── Demo 5: Cancellation Token ──────────────────────────────────
    {
        std::cout << "--- Demo 5: Cancellation Token ---\n";

        CancellationToken token;

        // EN: Coroutine that checks for cancellation at each step
        // TR: Her adımda iptali kontrol eden coroutine
        auto cancellable_task = [&token]() -> Task<std::string> {
            std::string progress;
            for (int i = 1; i <= 10; ++i) {
                // EN: Check cancellation — if cancelled, this throws
                // TR: İptali kontrol et — iptal edildiyse fırlatır
                co_await check_cancellation(token);
                progress += "[Step" + std::to_string(i) + "]";
                std::cout << "  [Task] Completed step " << i << "/10\n";

                // EN: Simulate: cancel after step 3
                // TR: Simülasyon: 3. adımdan sonra iptal et
                if (i == 3) {
                    std::cout << "  [External] Requesting cancellation!\n";
                    token.cancel();
                }
            }
            co_return progress;
        };

        auto task = cancellable_task();
        try {
            auto result = task.result();
            std::cout << "  Task completed: " << result << "\n";
        } catch (const std::runtime_error& e) {
            std::cout << "  Task cancelled: " << e.what() << "\n";
        }
        std::cout << "\n";
    }

    // ─── Demo 6: Sensor Polling Simulation ───────────────────────────
    {
        std::cout << "--- Demo 6: Automotive Sensor Polling (Scheduler) ---\n";

        Scheduler scheduler;
        std::vector<SensorData> collected_data;

        // EN: Simulate reading multiple sensors — each takes different number of steps
        // TR: Birden fazla sensör okumayı simüle et — her biri farklı adım sayısı alır
        auto poll_sensor = [&scheduler, &collected_data](
            std::string name, double base_val, std::string unit, int latency_steps) -> ScheduledTask
{

```

```
            for (int step = 0; step < latency_steps; ++step) {
                std::cout << "  [" << name << "] Polling step " << (step + 1)
                          << "/" << latency_steps << "...\n";
                co_await scheduler.suspend();
            }

            // EN: Simulate reading the sensor value
            // TR: Sensör değerini okumayı simüle et
            double value = base_val + static_cast<double>(latency_steps) * 0.5;
            collected_data.push_back(SensorData{name, value, unit});
            std::cout << "  [" << name << "] READ: " << value << " " << unit << "\n";
        };

        // EN: Create sensor polling tasks with different latencies
        // TR: Farklı gecikmelere sahip sensör sorgulama görevleri oluştur
        auto temp_task   = poll_sensor("Temperature", 85.0, "°C", 2);
        auto press_task  = poll_sensor("Pressure",    1013.0, "hPa", 3);
        auto rpm_task    = poll_sensor("RPM",         3500.0, "rpm", 1);

        scheduler.schedule(temp_task.handle());
        scheduler.schedule(press_task.handle());
        scheduler.schedule(rpm_task.handle());

        std::cout << "  [ECU] Starting cooperative sensor polling...\n";
        scheduler.run();

        std::cout << "\n  [ECU] All sensors polled. Results:\n";
        for (const auto& data : collected_data) {
            std::cout << "    " << data.name << " = " << data.value << " " << data.unit << "\n";
        }
    }

    std::cout << "\n==========================================\n";
    std::cout << "   End of Module 12 — Async Coroutine Patterns\n";
    std::cout << "==========================================\n";

    return 0;
}
```

Modül 13: Networking — Ağ Programlama

Ağ Programlama

2 kaynak dosya · Standart: C++20

C++20

► 01. Socket Fundamentals

module_13_networking/01_socket_fundamentals.cpp

Konu: Network Programming: BSD Sockets, TCP/UDP — Ağ Programlama: BSD Soketleri, TCP/UDP

Teori / Theory

THEORY: What Is a Socket?

A socket is an endpoint for sending/receiving data across a network. Think of it like a phone: you need one on each end to have a conversation.
The socket API (BSD sockets) is a C interface from 1983 that still powers virtually ALL network communication today — from web browsers to game servers.
Key concepts:

```
┌──────────────────┬──────────────────────────────────────────┐
│ Concept          │ Description                              │
├──────────────────┼──────────────────────────────────────────┤
│ IP Address       │ Machine address on network (IPv4: x.x.x.x)    │
│ Port             │ Service identifier (0-65535)                  │
│ Socket           │ IP + Port + Protocol = unique endpoint        │
│ Protocol         │ TCP (reliable stream) or UDP (fast datagram)  │
│ File Descriptor  │ Integer handle returned by socket()           │
└──────────────────┴──────────────────────────────────────────┘
```

TEORİ: Soket Nedir?

Soket, ağ üzerinden veri göndermek/almak için bir uç noktadır. Telefon gibi düşün: konuşmak için her iki uçta da birer tane gerekli.
Soket API'si (BSD soketleri) 1983'ten kalma bir C arayüzüdür ve web tarayıcılarından oyun sunucularına kadar bugün hâlâ TÜM ağ iletişimini güçlendirir. * Temel kavramlar:

```
┌──────────────────┬──────────────────────────────────────────┐
│ Kavram           │ Açıklama                                 │
├──────────────────┼──────────────────────────────────────────┤
│ IP Adresi        │ Ağdaki makine adresi (IPv4: x.x.x.x)          │
│ Port             │ Servis tanımlayıcısı (0-65535)                │
│ Soket            │ IP + Port + Protokol = benzersiz uç nokta     │
│ Protokol         │ TCP (güvenilir akış) veya UDP (hızlı datagram) │
│ Dosya Tanımlayıcı│ socket() tarafından döndürülen tam sayı tutamak│
└──────────────────┴──────────────────────────────────────────┘ *
```

THEORY: TCP vs UDP

```
┌──────────────────┬──────────────────────┬──────────────────────────┐
│ Feature          │ TCP                  │ UDP                      │
├──────────────────┼──────────────────────┼──────────────────────────┤
│ Connection       │ Connection-oriented  │ Connectionless           │
│ Reliability      │ Guaranteed delivery  │ Best-effort (may lose)   │
│ Ordering         │ Ordered              │ No ordering guarantee    │
│ Speed            │ Slower (handshake)   │ Faster (no handshake)    │
│ Use case         │ HTTP, file transfer  │ DNS, video stream, games │
│ Analogy          │ Phone call           │ Sending postcards        │
│ Overhead         │ Higher (ACK, retx)   │ Minimal (just send)      │
│ C++ socket type  │ SOCK_STREAM          │ SOCK_DGRAM               │
└──────────────────┴──────────────────────┴──────────────────────────┘
```

TEORİ: TCP ve UDP KARŞILAŞTIRMASI

```
┌──────────────────┬──────────────────────┬──────────────────────────┐
│ Özellik          │ TCP                  │ UDP                      │
├──────────────────┼──────────────────────┼──────────────────────────┤
│ Bağlantı         │ Bağlantı yönelimli   │ Bağlantısız              │
│ Güvenilirlik     │ Teslim garantili     │ En iyi çaba (kaybolabilir)│
│ Sıralama         │ Sıralı               │ Sıralama garantisi yok   │
│ Hız              │ Yavaş (el sıkışma)   │ Hızlı (el sıkışma yok)   │
│ Kullanım         │ HTTP, dosya transfer │ DNS, video, oyunlar      │
│ Benzetme         │ Telefon görüşmesi    │ Kartpostal göndermek     │
└──────────────────┴──────────────────────┴──────────────────────────┘
```

THEORY: TCP Server Flow

The classic TCP server lifecycle (a.k.a. "the socket dance"):
SERVER CLIENT
────── ──────
1. socket() → create socket
2. bind() → assign address/port
3. listen() → mark as passive (accepting connections)
4. accept() → BLOCK until client connects ←── connect()
5. recv/send → data exchange ←→ recv/send
6. close() → shutdown ←── close()

IMPORTANT RAII note: In C, forgetting close() LEAKS file descriptors. In this file, we wrap the fd in a RAII class for safety.

TEORİ: TCP Sunucu Akışı

Klasik TCP sunucu yaşam döngüsü ("soket dansı"):
1. socket() → soket oluştur
2. bind() → adres/port ata
3. listen() → pasif olarak işaretle (bağlantı kabul edecek)
4. accept() → istemci bağlanana kadar BEKLE ←── connect()
5. recv/send → veri alışverişi ←→ recv/send
6. close() → kapat ←── close()

ÖNEMLİ RAII notu: C'de close() unutmak dosya tanımlayıcılarını SIZDIRIR. Bu dosyada fd'yi güvenlik için RAII sınıfında sarıyoruz.

THEORY: Byte Order

Network byte order is BIG-ENDIAN (most significant byte first). x86 CPUs use LITTLE-ENDIAN (least significant byte first). Conversion functions: htons() / htonl() → Host TO Network (short/long) ntohs() / ntohl() → Network TO Host (short/long) ALWAYS use these for port numbers and IP addresses! Forgetting byte order conversion = silent data corruption.

TEORİ: Bayt Sırası

Ağ bayt sırası BIG-ENDIAN (en anlamlı bayt önce). x86 CPU'lar LITTLE-ENDIAN kullanır (en az anlamlı bayt önce). Dönüşüm fonksiyonları: htons() / htonl() → Host'TAN Ağ'A (kısa/uzun) ntohs() / ntohl() → Ağ'DAN Host'A (kısa/uzun) Port numaraları ve IP adresleri için HER ZAMAN bunları kullan!

→ Ref: https://man7.org/linux/man-pages/man2/socket.2.html
→ Ref: https://man7.org/linux/man-pages/man7/tcp.7.html
→ Ref: https://beej.org/guide/bgnet/ — "Beej's Guide to Network Programming"

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <string>
#include <cstring>       // EN: memset, strerror / TR: memset, strerror
#include <array>
#include <vector>
#include <thread>
#include <chrono>

// ── POSIX headers for socket programming ──
// EN: These are C headers from the POSIX standard — available on Linux/macOS.
//     On Windows, you'd use <winsock2.h> and <ws2tcpip.h> instead.
// TR: Bunlar POSIX standardından C başlıkları — Linux/macOS'ta mevcuttur.
//     Windows'ta <winsock2.h> ve <ws2tcpip.h> kullanılır.
#include <sys/socket.h>  // EN: socket(), bind(), listen(), accept(), connect()
#include <netinet/in.h>  // EN: sockaddr_in, INADDR_LOOPBACK, htons()
#include <arpa/inet.h>   // EN: inet_ntop(), inet_pton() — IP address conversion
#include <unistd.h>      // EN: close() — close file descriptors

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: RAII Socket Wrapper
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: RAII wrapper for a socket file descriptor.
 *     In C, socket() returns an int (file descriptor). If you forget to close()
 *     it, you leak the fd. This wrapper ensures close() is called automatically
 *     when the object goes out of scope — exactly like unique_ptr for memory.
 *
 *     Rule: NEVER use raw `int fd` for sockets in C++ code. Always wrap in RAII.
 *
 * TR: Soket dosya tanımlayıcısı için RAII sarmalayıcı.
 *     C'de socket() bir int (dosya tanımlayıcısı) döndürür. close() unutulursa
 *     fd sızar. Bu sarmalayıcı nesne scope'dan çıkınca close()'un otomatik
 *     çağrılmasını garanti eder — bellek için unique_ptr gibi.
 */
class SocketRAII {
public:
    // EN: Default constructor — no socket yet
    // TR: Varsayılan constructor — henüz soket yok
    SocketRAII() : fd_(-1) {}

    // EN: Construct from an existing file descriptor
    // TR: Mevcut dosya tanımlayıcısından oluştur
    explicit SocketRAII(int fd) : fd_(fd) {}

```

```
46      // EN: Destructor — close the socket if valid
47      // TR: Destructor — geçerliyse soketi kapat
48      ~SocketRAII() {
49          if (fd_ >= 0) {
50              ::close(fd_);
51              // EN: We use ::close (global scope) to avoid naming conflicts
52              // TR: İsimlendirme çakışmasını önlemek için ::close (global scope) kullanıyoruz
53          }
54      }
55
56      // EN: Move only — ownership semantics (like unique_ptr)
57      // TR: Sadece taşınabilir — sahiplik semantiği (unique_ptr gibi)
58      SocketRAII(const SocketRAII&) = delete;
59      SocketRAII& operator=(const SocketRAII&) = delete;
60
61      SocketRAII(SocketRAII&& other) noexcept : fd_(other.fd_) {
62          other.fd_ = -1;  // EN: Source loses ownership / TR: Kaynak sahipliği kaybeder
63      }
64      SocketRAII& operator=(SocketRAII&& other) noexcept {
65          if (this != &other) {
66              if (fd_ >= 0) ::close(fd_);
67              fd_ = other.fd_;
68              other.fd_ = -1;
69          }
70          return *this;
71      }
72
73      // EN: Get the raw file descriptor (for use with C APIs)
74      // TR: Ham dosya tanımlayıcısını al (C API'leri ile kullanmak için)
75      int get() const { return fd_; }
76
77      // EN: Check if the socket is valid
78      // TR: Soketin geçerli olup olmadığını kontrol et
79      bool valid() const { return fd_ >= 0; }
80
81      // EN: Release ownership without closing (rare, but sometimes needed)
82      // TR: Kapatmadan sahipliği bırak (nadir ama bazen gerekli)
83      int release() {
84          int temp = fd_;
85          fd_ = -1;
86          return temp;
87      }
88
89  private:
90      int fd_;
91  };
92
93  // ═══════════════════════════════════════════════════════════════════════
94  // PART 2: TCP Echo Server (single-client, educational)
95  // ═══════════════════════════════════════════════════════════════════════
96
97  /**
98   * EN: A simple TCP echo server that:
99   *     1. Listens on a given port
100  *     2. Accepts ONE client
101  *     3. Reads a message
102  *     4. Sends it back (echo)
103  *     5. Closes
104  *
105  *     This function demonstrates every step of the "socket dance":
106  *     socket() → bind() → listen() → accept() → recv() → send() → close()
107  *
108  * TR: Basit bir TCP echo (yankı) sunucusu:
109  *     1. Verilen portta dinler
110  *     2. BİR istemci kabul eder
111  *     3. Mesaj okur
112  *     4. Geri gönderir (yankı)
113  *     5. Kapatır
114  */
```

```cpp
void run_tcp_echo_server(uint16_t port) {
    std::cout << "  [SERVER] Starting TCP echo server on port " << port << "...\n";

    // ── Step 1: socket() — Create the socket ──
    // EN: AF_INET     = IPv4 address family
    //     SOCK_STREAM = TCP (reliable, ordered, byte-stream)
    //     0           = Let OS choose protocol (TCP for SOCK_STREAM)
    // TR: AF_INET     = IPv4 adres ailesi
    //     SOCK_STREAM = TCP (güvenilir, sıralı, bayt akışı)
    //     0           = İşletim sistemi protokolü seçsin (SOCK_STREAM için TCP)
    SocketRAII server_socket(::socket(AF_INET, SOCK_STREAM, 0));
    if (!server_socket.valid()) {
        std::cerr << "  [SERVER] socket() failed: " << strerror(errno) << "\n";
        return;
    }

    // EN: SO_REUSEADDR allows reusing the port immediately after server stops.
    //     Without this, bind() fails with "Address already in use" for ~60 seconds
    //     after stopping the server (due to TCP TIME_WAIT state).
    // TR: SO_REUSEADDR sunucu durduktan hemen sonra portu tekrar kullanmaya izin verir.
    //     Bu olmadan bind() ~60 saniye "Address already in use" ile başarısız olur
    //     (TCP TIME_WAIT durumu yüzünden).
    int opt = 1;
    setsockopt(server_socket.get(), SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    // ── Step 2: bind() — Assign address and port ──
    // EN: sockaddr_in is the IPv4 address structure:
    //     sin_family = AF_INET (IPv4)
    //     sin_port   = port in NETWORK byte order (htons converts host→network)
    //     sin_addr   = IP address (INADDR_LOOPBACK = 127.0.0.1 = localhost)
    // TR: sockaddr_in IPv4 adres yapısıdır:
    //     sin_family = AF_INET (IPv4)
    //     sin_port   = AĞ bayt sırasında port (htons host→ağ dönüştürür)
    //     sin_addr   = IP adresi (INADDR_LOOPBACK = 127.0.0.1 = yerel makine)
    struct sockaddr_in server_addr{};
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(port);  // EN: Host-to-network byte order! / TR: Host→ağ bayt sırası!
    server_addr.sin_addr.s_addr = htonl(INADDR_LOOPBACK);

    if (::bind(server_socket.get(),
               reinterpret_cast<struct sockaddr*>(&server_addr),
               sizeof(server_addr)) < 0) {
        std::cerr << "  [SERVER] bind() failed: " << strerror(errno) << "\n";
        return;
    }

    // ── Step 3: listen() — Mark socket as passive (ready to accept) ──
    // EN: The backlog (5) is the maximum queue of pending connections.
    //     If 6th client tries to connect while 5 are waiting, it gets ECONNREFUSED.
    // TR: Backlog (5) bekleyen bağlantıların maksimum kuyruğudur.
    //     5 beklerken 6. istemci bağlanmaya çalışırsa ECONNREFUSED alır.
    if (::listen(server_socket.get(), 5) < 0) {
        std::cerr << "  [SERVER] listen() failed: " << strerror(errno) << "\n";
        return;
    }
    std::cout << "  [SERVER] Listening on 127.0.0.1:" << port << " (waiting for client...)\n";

    // ── Step 4: accept() — Wait for a client to connect ──
    // EN: accept() BLOCKS until a client calls connect().
    //     It returns a NEW socket fd for this specific conversation.
    //     The original socket continues to listen for other clients.
    //     client_addr is filled with the client's IP/port info.
    // TR: accept() bir istemci connect() çağırana kadar BLOKE olur.
    //     Bu belirli konuşma için YENİ bir soket fd döndürür.
    //     Orijinal soket diğer istemcileri dinlemeye devam eder.
    struct sockaddr_in client_addr{};
    socklen_t client_len = sizeof(client_addr);
    SocketRAII client_socket(
        ::accept(server_socket.get(),
                 reinterpret_cast<struct sockaddr*>(&client_addr),
                 &client_len)
    );

```

```
188     if (!client_socket.valid()) {
189         std::cerr << "  [SERVER] accept() failed: " << strerror(errno) << "\n";
190         return;
191     }
192 
193     // EN: Convert client IP from binary to human-readable string
194     // TR: İstemci IP'sini ikili formattan okunabilir string'e dönüştür
195     char client_ip[INET_ADDRSTRLEN];
196     inet_ntop(AF_INET, &client_addr.sin_addr, client_ip, sizeof(client_ip));
197     std::cout << "  [SERVER] Client connected from " << client_ip
198               << ":" << ntohs(client_addr.sin_port) << "\n";
199 
200     // ── Step 5: recv() — Read data from client ──
201     // EN: recv() reads up to buffer_size bytes from the client socket.
202     //     Returns the number of bytes actually received, or 0 if client closed, or -1 on error.
203     //     IMPORTANT: recv() may return LESS data than sent — TCP is a BYTE STREAM,
204     //     not a message protocol. In production, loop until you have the full message.
205     // TR: recv() istemci soketinden buffer_size bayta kadar okur.
206     //     Gerçekten alınan bayt sayısını döndürür, istemci kapattıysa 0, hatada -1.
207     //     ÖNEMLİ: recv() gönderilenden AZ veri döndürebilir — TCP BAYT AKIŞIDIR,
208     //     mesaj protokolü değil. Üretimde tam mesaj alana kadar döngü kullan.
209     std::array<char, 1024> buffer{};
210     ssize_t bytes_received = ::recv(client_socket.get(), buffer.data(),
211                                     buffer.size() - 1, 0);  // -1 for null-terminator space
212 
213     if (bytes_received <= 0) {
214         std::cout << "  [SERVER] Client disconnected or error.\n";
215         return;
216     }
217 
218     buffer[static_cast<size_t>(bytes_received)] = '\0';
219     std::cout << "  [SERVER] Received (" << bytes_received << " bytes): " << buffer.data() << "\n";
220 
221     // ── Step 6: send() — Send response back (echo) ──
222     // EN: send() writes data to the socket. Returns bytes sent or -1 on error.
223     //     Like recv(), may send LESS than requested in one call.
224     //     For large messages, loop until all bytes are sent.
225     // TR: send() sokete veri yazar. Gönderilen baytları veya hatada -1 döndürür.
226     //     recv() gibi, tek çağrıda istenenin AZINI gönderebilir.
227     std::string response = "ECHO: " + std::string(buffer.data());
228     ssize_t bytes_sent = ::send(client_socket.get(), response.c_str(),
229                                 response.size(), 0);
230     std::cout << "  [SERVER] Sent " << bytes_sent << " bytes back.\n";
231 
232     // EN: SocketRAII destructors close both sockets automatically here
233     // TR: SocketRAII destructor'ları her iki soketi de burada otomatik kapatır
234     std::cout << "  [SERVER] Connection closed (RAII cleanup).\n";
235 }
236 
237 // ═══════════════════════════════════════════════════════════════════════════
238 // PART 3: TCP Client
239 // ═══════════════════════════════════════════════════════════════════════════
240 
241 /**
242  * EN: TCP client that connects to the echo server, sends a message,
243  *     and receives the echoed response.
244  *
245  *     Client flow is simpler than server:
246  *     socket() → connect() → send() → recv() → close()
247  *
248  * TR: Echo sunucusuna bağlanan, mesaj gönderen ve yankılanan yanıtı alan TCP istemci.
249  *     İstemci akışı sunucudan daha basit:
250  *     socket() → connect() → send() → recv() → close()
251  */
252 void run_tcp_client(uint16_t port, const std::string& message) {
253     std::cout << "  [CLIENT] Connecting to 127.0.0.1:" << port << "...\n";
254 
255     // EN: Create a TCP socket (same as server)
256     // TR: TCP soketi oluştur (sunucuyla aynı)
257     SocketRAII client_socket(::socket(AF_INET, SOCK_STREAM, 0));
258     if (!client_socket.valid()) {
259         std::cerr << "  [CLIENT] socket() failed: " << strerror(errno) << "\n";
260         return;
261     }
```

```cpp

    // EN: Set up server address to connect to
    // TR: Bağlanılacak sunucu adresini ayarla
    struct sockaddr_in server_addr{};
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(port);
    server_addr.sin_addr.s_addr = htonl(INADDR_LOOPBACK);

    // EN: connect() initiates the TCP 3-way handshake:
    //     Client → SYN → Server
    //     Client ← SYN-ACK ← Server
    //     Client → ACK → Server
    //     After this, the connection is established and data can flow.
    // TR: connect() TCP 3'lü el sıkışmasını başlatır:
    //     İstemci → SYN → Sunucu
    //     İstemci ← SYN-ACK ← Sunucu
    //     İstemci → ACK → Sunucu
    //     Bundan sonra bağlantı kurulur ve veri akabilir.
    if (::connect(client_socket.get(),
                  reinterpret_cast<struct sockaddr*>(&server_addr),
                  sizeof(server_addr)) < 0) {
        std::cerr << "  [CLIENT] connect() failed: " << strerror(errno) << "\n";
        return;
    }
    std::cout << "  [CLIENT] Connected!\n";

    // EN: Send the message
    // TR: Mesajı gönder
    ::send(client_socket.get(), message.c_str(), message.size(), 0);
    std::cout << "  [CLIENT] Sent: " << message << "\n";

    // EN: Receive the echo response
    // TR: Yankı yanıtını al
    std::array<char, 1024> buffer{};
    ssize_t bytes_received = ::recv(client_socket.get(), buffer.data(),
                                    buffer.size() - 1, 0);
    if (bytes_received > 0) {
        buffer[static_cast<size_t>(bytes_received)] = '\0';
        std::cout << "  [CLIENT] Received: " << buffer.data() << "\n";
    }

    std::cout << "  [CLIENT] Done.\n";
}

// ═══════════════════════════════════════════════════════════════════════════
// PART 4: UDP Sender / Receiver
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: UDP is connectionless — no connect(), no accept(), no handshake.
 *     Just sendto() and recvfrom() with the destination address each time.
 *     This is faster but unreliable: packets can be lost, duplicated, or reordered.
 *
 *     Use cases: DNS lookups, real-time video, game state updates, sensor telemetry
 *
 * TR: UDP bağlantısızdır — connect() yok, accept() yok, el sıkışma yok.
 *     Her seferinde hedef adresle sendto() ve recvfrom().
 *     Daha hızlı ama güvenilmez: paketler kaybolabilir, çoğaltılabilir, yeniden sıralanabilir.
 *
 *     Kullanım: DNS sorguları, gerçek zamanlı video, oyun durum güncellemeleri, sensör telemetrisi
 */
void run_udp_receiver(uint16_t port) {
    std::cout << "  [UDP-RX] Starting UDP receiver on port " << port << "...\n";

    // EN: SOCK_DGRAM = UDP (datagram-based, connectionless)
    // TR: SOCK_DGRAM = UDP (datagram tabanlı, bağlantısız)
    SocketRAII udp_socket(::socket(AF_INET, SOCK_DGRAM, 0));
    if (!udp_socket.valid()) {
        std::cerr << "  [UDP-RX] socket() failed\n";
        return;
    }

```

```
    int opt = 1;
    setsockopt(udp_socket.get(), SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in recv_addr{};
    recv_addr.sin_family = AF_INET;
    recv_addr.sin_port = htons(port);
    recv_addr.sin_addr.s_addr = htonl(INADDR_LOOPBACK);

    if (::bind(udp_socket.get(),
               reinterpret_cast<struct sockaddr*>(&recv_addr),
               sizeof(recv_addr)) < 0) {
        std::cerr << "  [UDP-RX] bind() failed: " << strerror(errno) << "\n";
        return;
    }

    // EN: recvfrom() blocks until a datagram arrives.
    //     Unlike TCP, each recvfrom() returns ONE complete datagram (message boundary preserved).
    //     sender_addr is filled with the sender's IP/port.
    // TR: recvfrom() bir datagram gelene kadar bloke olur.
    //     TCP'den farklı olarak, her recvfrom() TAM BİR datagram döndürür (mesaj sınırı korunur).
    struct sockaddr_in sender_addr{};
    socklen_t sender_len = sizeof(sender_addr);
    std::array<char, 1024> buffer{};

    std::cout << "  [UDP-RX] Waiting for datagram...\n";
    ssize_t bytes = ::recvfrom(udp_socket.get(), buffer.data(), buffer.size() - 1, 0,
                               reinterpret_cast<struct sockaddr*>(&sender_addr),
                               &sender_len);
    if (bytes > 0) {
        buffer[static_cast<size_t>(bytes)] = '\0';
        char sender_ip[INET_ADDRSTRLEN];
        inet_ntop(AF_INET, &sender_addr.sin_addr, sender_ip, sizeof(sender_ip));
        std::cout << "  [UDP-RX] Received " << bytes << " bytes from "
                  << sender_ip << ":" << ntohs(sender_addr.sin_port)
                  << " → \"" << buffer.data() << "\"\n";
    }
}

/**
 * EN: UDP sender — sends a single datagram to the receiver.
 *     No connection setup needed. Just fire and forget.
 *
 * TR: UDP gönderici — alıcıya tek bir datagram gönderir.
 *     Bağlantı kurulumu gerekmez. Sadece gönder ve unut.
 */
void run_udp_sender(uint16_t port, const std::string& message) {
    std::cout << "  [UDP-TX] Sending UDP datagram to port " << port << "...\n";

    SocketRAII udp_socket(::socket(AF_INET, SOCK_DGRAM, 0));
    if (!udp_socket.valid()) {
        std::cerr << "  [UDP-TX] socket() failed\n";
        return;
    }

    struct sockaddr_in dest_addr{};
    dest_addr.sin_family = AF_INET;
    dest_addr.sin_port = htons(port);
    dest_addr.sin_addr.s_addr = htonl(INADDR_LOOPBACK);

    // EN: sendto() sends the datagram to dest_addr. No connect() needed!
    //     The entire message is sent as ONE datagram (up to 65507 bytes for IPv4).
    //     UDP either sends the WHOLE datagram or NONE — never partial.
    // TR: sendto() datagramı dest_addr'a gönderir. connect() gerekmez!
    //     Tüm mesaj TEK datagram olarak gönderilir (IPv4 için 65507 bayta kadar).
    //     UDP ya TÜMÜNÜ gönderir ya da HİÇBİRİNİ — asla kısmi değil.
    ssize_t bytes_sent = ::sendto(udp_socket.get(), message.c_str(), message.size(), 0,
                                  reinterpret_cast<const struct sockaddr*>(&dest_addr),
                                  sizeof(dest_addr));
    std::cout << "  [UDP-TX] Sent " << bytes_sent << " bytes: \"" << message << "\"\n";
}

```

```cpp
// ═══════════════════════════════════════════════════════════════════════
// PART 5: Byte Order Demonstration
// ═══════════════════════════════════════════════════════════════════════

/**
 * EN: Shows byte order conversion functions and why they matter.
 * TR: Bayt sırası dönüşüm fonksiyonlarını ve neden önemli olduklarını gösterir.
 */
void demo_byte_order() {
    uint16_t host_port = 8080;
    uint16_t net_port = htons(host_port);

    std::cout << "  Host byte order (port 8080):\n";
    std::cout << "    host_port = 0x" << std::hex << host_port << std::dec
              << " (" << host_port << ")\n";
    std::cout << "    net_port  = 0x" << std::hex << net_port << std::dec
              << " (" << net_port << ")\n";

    // EN: Show the individual bytes to make byte order visible
    // TR: Bayt sırasını görünür kılmak için bireysel baytları göster
    auto* host_bytes = reinterpret_cast<const uint8_t*>(&host_port);
    auto* net_bytes = reinterpret_cast<const uint8_t*>(&net_port);
    std::cout << "    Host bytes: [" << static_cast<int>(host_bytes[0]) << "]["
              << static_cast<int>(host_bytes[1]) << "]\n";
    std::cout << "    Net bytes:  [" << static_cast<int>(net_bytes[0]) << "]["
              << static_cast<int>(net_bytes[1]) << "]\n";

    // EN: On x86 (little-endian), host_port bytes are [0x90, 0x1F]
    //     After htons(), net_port bytes are [0x1F, 0x90] (big-endian)
    //     If you forget htons(): port 8080 looks like port 36895 on the network!
    // TR: x86'da (little-endian), host_port baytları [0x90, 0x1F]
    //     htons() sonrası, net_port baytları [0x1F, 0x90] (big-endian)
    //     htons() unutulursa: port 8080 ağda port 36895 gibi görünür!

    uint32_t ip_addr = 0xC0A80001;  // EN: 192.168.0.1 / TR: 192.168.0.1
    uint32_t net_ip = htonl(ip_addr);
    std::cout << "\n  IP address 192.168.0.1 (0xC0A80001):\n";
    std::cout << "    Host order: 0x" << std::hex << ip_addr << std::dec << "\n";
    std::cout << "    Net order:  0x" << std::hex << net_ip << std::dec << "\n";
}

// ═══════════════════════════════════════════════════════════════════════
// PART 6: Address Conversion (inet_pton / inet_ntop)
// ═══════════════════════════════════════════════════════════════════════

/**
 * EN: Demonstrates safe IP address conversion functions:
 *     inet_pton() → "presentation to numeric" (string → binary)
 *     inet_ntop() → "numeric to presentation" (binary → string)
 *     NEVER use inet_addr() or inet_aton() — they can't handle errors properly.
 *
 * TR: Güvenli IP adresi dönüşüm fonksiyonlarını gösterir:
 *     inet_pton() → "gösterimden sayısala" (string → ikili)
 *     inet_ntop() → "sayısaldan gösterime" (ikili → string)
 *     ASLA inet_addr() veya inet_aton() kullanma — hataları düzgün yakalayamaz.
 */
void demo_address_conversion() {
    // EN: Convert human-readable IP string to binary
    // TR: Okunabilir IP string'ini ikiliye dönüştür
    struct in_addr addr{};
    const char* ip_str = "192.168.1.42";

    if (inet_pton(AF_INET, ip_str, &addr) == 1) {
        std::cout << "  inet_pton(\"" << ip_str << "\") → binary 0x"
                  << std::hex << ntohl(addr.s_addr) << std::dec << "\n";
    }

    // EN: Convert binary back to human-readable string
    // TR: İkiliyi tekrar okunabilir string'e dönüştür
    char buffer[INET_ADDRSTRLEN];
    if (inet_ntop(AF_INET, &addr, buffer, sizeof(buffer))) {
        std::cout << "  inet_ntop(binary) → \"" << buffer << "\"\n";
    }
```

```

    // EN: Try an invalid IP — inet_pton returns 0 (not -1!)
    // TR: Geçersiz IP dene — inet_pton 0 döndürür (-1 değil!)
    struct in_addr bad_addr{};
    int result = inet_pton(AF_INET, "999.888.777.666", &bad_addr);
    std::cout << "  inet_pton(\"999.888.777.666\") → " << result << " (0 = invalid)\n";
}

// ═══════════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "═══════════════════════════════════════════\n";
    std::cout << "   Module 13 - Socket Fundamentals\n";
    std::cout << "═══════════════════════════════════════════\n\n";

    // ── Demo 1: Byte Order ──────────────────────────────────────────
    {
        std::cout << "--- Demo 1: Byte Order (Network vs Host) ---\n";
        demo_byte_order();
        std::cout << "\n";
    }

    // ── Demo 2: Address Conversion ──────────────────────────────────
    {
        std::cout << "--- Demo 2: IP Address Conversion ---\n";
        demo_address_conversion();
        std::cout << "\n";
    }

    // ── Demo 3: TCP Echo — Server + Client ──────────────────────────
    {
        std::cout << "--- Demo 3: TCP Echo (Server + Client) ---\n";

        // EN: We run server in a thread, give it a moment to start listening,
        //     then run the client in the main thread.
        //     The server thread receives the message, echoes it, and exits.
        //
        //     Port selection: Using a high port (> 1024) that doesn't need root.
        //     We use a somewhat random port to avoid conflicts with other services.
        //
        // TR: Sunucuyu bir thread'de çalıştırıp dinlemeye başlaması için bir an veriyoruz,
        //     sonra istemciyi ana thread'de çalıştırıyoruz.
        //
        //     Port seçimi: root gerektirmeyen yüksek port (> 1024) kullanıyoruz.
        constexpr uint16_t TCP_PORT = 19850;

        std::thread server_thread(run_tcp_echo_server, TCP_PORT);

        // EN: Small delay so server has time to bind() and listen()
        // TR: Sunucunun bind() ve listen() yapması için küçük gecikme
        std::this_thread::sleep_for(std::chrono::milliseconds(100));

        run_tcp_client(TCP_PORT, "Hello from CAN controller module!");

        server_thread.join();
        std::cout << "\n";
    }

    // ── Demo 4: UDP Datagram ────────────────────────────────────────
    {
        std::cout << "--- Demo 4: UDP Datagram (Send + Receive) ---\n";

        constexpr uint16_t UDP_PORT = 19851;

        // EN: Start receiver in a thread, then send a datagram
        // TR: Alıcıyı thread'de başlat, sonra datagram gönder
        std::thread receiver_thread(run_udp_receiver, UDP_PORT);

        std::this_thread::sleep_for(std::chrono::milliseconds(100));

        run_udp_sender(UDP_PORT, "ECU_TEMP:85.3:NORMAL");

        receiver_thread.join();

```

```
        // EN: Note: No connection was established! Pure fire-and-forget.
        //     In real systems, UDP is used for telemetry where occasional
        //     packet loss is acceptable (better than TCP's latency overhead).
        // TR: Not: Bağlantı kurulmadı! Saf gönder-ve-unut.
        //     Gerçek sistemlerde UDP, arada paket kaybının kabul edilebilir
        //     olduğu telemetri için kullanılır (TCP'nin gecikme yükünden daha iyi).
        std::cout << "\n";
    }

    // ─── Demo 5: Socket Structure Sizes ──────────────────────────────
    {
        std::cout << "--- Demo 5: Socket Structure Sizes ---\n";
        std::cout << "  sizeof(sockaddr_in):  " << sizeof(sockaddr_in) << " bytes (IPv4)\n";
        std::cout << "  sizeof(sockaddr_in6): " << sizeof(sockaddr_in6) << " bytes (IPv6)\n";
        std::cout << "  sizeof(SocketRAII):   " << sizeof(SocketRAII) << " bytes (our wrapper)\n";

        // EN: sockaddr_in is 16 bytes:
        //     sin_family (2) + sin_port (2) + sin_addr (4) + sin_zero (8) = 16
        //     It's padded to match the size of the generic sockaddr structure.
        // TR: sockaddr_in 16 bayttır:
        //     sin_family (2) + sin_port (2) + sin_addr (4) + sin_zero (8) = 16
        //     Genel sockaddr yapısının boyutuyla eşleşmek için doldurulur.
        std::cout << "\n";
    }

    std::cout << "============================================\n";
    std::cout << "   End of Module 13 - Socket Fundamentals\n";
    std::cout << "============================================\n";

    return 0;
}
```

► 02. Nonblocking Io Epoll

module_13_networking/02_nonblocking_io_epoll.cpp

Konu: Non-Blocking I/O, poll, epoll — Bloklamayan I/O, poll, epoll

Teori / Theory

THEORY: Blocking vs Non-Blocking I/O

By default, socket operations (accept, recv, send) are BLOCKING: the calling thread sleeps until data arrives or a client connects. This is fine for simple servers, but doesn't scale:

```
┌──────────────────────────────────────────────────────────┐
│ Problem: Thread-per-Connection                           │
│                                                          │
│ Client 1  ──→  Thread 1  (blocked on recv...)            │
│ Client 2  ──→  Thread 2  (blocked on recv...)            │
│ Client 3  ──→  Thread 3  (blocked on recv...)            │
│ ...                                                      │
│ Client 10000 ──→ Thread 10000  □ OS limit / memory exhaustion      |
└──────────────────────────────────────────────────────────┘
```

Each thread costs ~8MB stack → 10K clients = ~80GB just for stacks!
Non-blocking I/O solves this:
- Set socket to O_NONBLOCK via fcntl()
- recv() returns immediately with EAGAIN/EWOULDBLOCK if no data
- send() returns immediately with EAGAIN if buffer is full
- One thread can handle THOUSANDS of connections

TEORİ: Bloklayan vs Bloklamayan I/O

Varsayılan olarak soket işlemleri (accept, recv, send) BLOKLAYICIDIR: çağıran thread, veri gelene veya istemci bağlanana kadar uyur. Basit sunucular için sorunsuz ama ölçeklenmez:

```
┌──────────────────────────────────────────────────────────────────┐
│ Sorun: Bağlantı-Başına-Thread                                    │
│                                                                  │
│ İstemci 1    ──→  Thread 1  (recv'de bloklandı...)               │
│ İstemci 2    ──→  Thread 2  (recv'de bloklandı...)               │
│ İstemci 3    ──→  Thread 3  (recv'de bloklandı...)               │
│ ...                                                              │
│ İstemci 10000 ──→ Thread 10000  □ OS sınırı / bellek tükenmesi   │
└──────────────────────────────────────────────────────────────────┘
```

Her thread ~8MB yığın → 10K istemci = sadece yığınlar için ~80GB!
Bloklamayan I/O bunu çözer:
- Soketi fcntl() ile O_NONBLOCK yap
- Veri yoksa recv() hemen EAGAIN/EWOULDBLOCK ile döner
- Tampon doluysa send() hemen EAGAIN ile döner
- Tek thread BİNLERCE bağlantıyı yönetebilir

THEORY: I

I/O multiplexing lets a single thread monitor MULTIPLE file descriptors and react when ANY of them is ready for I/O.

Mechanism	Introduced	Scalability	Limitations
select()	1983 (4.2BSD)	O(n), FD_SETSIZE ≤ 1024 (default)	Max 1024 FDs (hard!) Copies fd_set to/from kernel each call
poll()	1986 (SVR3)	O(n), no limit on FD count	Still linear scan of all FDs every call
epoll	2002 (Linux 2.5.44)	O(1) for ready FDs, kernel-managed	Linux-only (BSD has kqueue, Windows has IOCP)

TEORİ: O Multiplexing: select → poll → epoll / I/O Çoklama

I/O çoklama, tek bir thread'in BİRDEN FAZLA dosya tanımlayıcısını izlemesine ve herhangi biri I/O için hazır olduğunda tepki vermesine olanak tanır.

Mekanizma	Tanıtım	Ölçeklenebilirlik	Kısıtlamalar
select()	1983 (4.2BSD)	O(n), FD_SETSIZE	Maks 1024 FD (sabit!)
poll()	1986 (SVR3)	O(n), sınır yok	Her çağrıda doğrusal
epoll	2002 (Linux)	Hazır FD'ler için O(1), çekirdek yönetimli	Sadece Linux

THEORY: epoll Architecture

epoll uses a 3-step API:
1. epoll_create1(0) → Create an epoll instance (returns an fd)
2. epoll_ctl(epfd, op, fd, &event) → Add/modify/remove monitored fds
- EPOLL_CTL_ADD: start monitoring fd
- EPOLL_CTL_MOD: change events of interest
- EPOLL_CTL_DEL: stop monitoring fd
3. epoll_wait(epfd, events[], maxevents, timeout) → Wait for ready fds
- Returns ONLY the ready fds (not all of them like poll!)
- This is why it's O(1) for ready events
Trigger modes:

```
┌──────────────────┬──────────────────────────────────────────────┐
│ Level-Triggered  │ Default. epoll_wait returns if fd STILL has data.│
│ (LT)             │ Easy to use. Can re-read later.                  │
├──────────────────┼──────────────────────────────────────────────┤
│ Edge-Triggered   │ EPOLLET flag. Only notifies on NEW data arrival. │
│ (ET)             │ MUST read ALL available data or you lose events! │
│                  │ More efficient but harder to use correctly.      │
└──────────────────┴──────────────────────────────────────────────┘
```

TEORİ: epoll Mimarisi

epoll 3 adımlı bir API kullanır:
1. epoll_create1(0) → epoll örneği oluştur (bir fd döndürür)
2. epoll_ctl(epfd, op, fd, &event) → İzlenen fd'leri ekle/değiştir/kaldır
- EPOLL_CTL_ADD: fd izlemeyi başlat
- EPOLL_CTL_MOD: ilgilenilen olayları değiştir
- EPOLL_CTL_DEL: fd izlemeyi durdur
3. epoll_wait(epfd, events[], maxevents, timeout) → Hazır fd'leri bekle
- Sadece hazır fd'leri döndürür (poll gibi hepsini değil!)
- Bu yüzden hazır olaylar için O(1)'dir
Tetikleme modları:

```
┌──────────────────┬──────────────────────────────────────────────┐
│ Seviye Tetiklemeli│ Varsayılan. fd'de HÂLÂ veri varsa epoll_wait döner. │
│ (LT)             │ Kullanımı kolay. Sonra tekrar okunabilir.        │
├──────────────────┼──────────────────────────────────────────────┤
│ Kenar Tetiklemeli │ EPOLLET bayrağı. Sadece YENİ veri geldiğinde bildir.│
│ (ET)             │ Tüm mevcut veriyi okuMALISINIZ yoksa olay kaçar!  │
│                  │ Daha verimli ama doğru kullanması zor.           │
└──────────────────┴──────────────────────────────────────────────┘
```

THEORY: Reactor Pattern

The Reactor pattern is an event-driven architecture for handling I/O:

```
┌──────────────────────────────────────────────────────────┐
│                    Reactor Pattern                       │
│                                                          │
│  ┌──────────┐  register  ┌──────────────┐  dispatch  ┌──────┐  │
│  │ Handler1 │ ──────────→ │   Reactor    │ ──────────→ │Handler│  │
│  │ Handler2 │            │ (epoll_wait) │            │  (cb) │  │
│  │ Handler3 │            │              │            └──────┘  │
│  └──────────┘            └──────┬───────┘                      │
│                                 │                              │
│                            event loop                          │
│                                 │                              │
│                      ┌──────────▼──────────┐                  │
│                      │  I/O Ready Event    │                  │
│                      │  (read/write/error)│                  │
│                      └─────────────────────┘                  │
└──────────────────────────────────────────────────────────┘
```

This solves the C10K problem (handling 10,000+ concurrent connections):
- One thread, one epoll instance
- Register interest in events (EPOLLIN, EPOLLOUT)
- When epoll_wait returns, dispatch to the appropriate handler
- No threads wasted on waiting

Used in: nginx, Node.js (libuv), Redis, HAProxy, Boost.Asio

TEORİ: Reaktör Kalıbı

Reaktör kalıbı, I/O yönetimi için olay güdümlü mimaridir:

```
┌──────────────────────────────────────────────────────────┐
│                    Reaktör Kalıbı                        │
│                                                          │
│  ┌──────────┐  kaydet    ┌──────────────┐  dağıt     ┌──────┐  │
│  │ Handler1 │ ──────────→ │   Reaktör    │ ──────────→ │Handler│  │
│  │ Handler2 │            │ (epoll_wait) │            │  (cb) │  │
│  │ Handler3 │            │              │            └──────┘  │
│  └──────────┘            └──────┬───────┘                      │
│                                 │                              │
│                            olay döngüsü                        │
│                                 │                              │
│                      ┌──────────▼──────────┐                  │
│                      │  I/O Hazır Olayı    │                  │
│                      │ (oku/yaz/hata)      │                  │
│                      └─────────────────────┘                  │
└──────────────────────────────────────────────────────────┘
```

C10K problemini çözer (10.000+ eşzamanlı bağlantı):
- Tek thread, tek epoll örneği
- Olaylara ilgi kaydet (EPOLLIN, EPOLLOUT)
- epoll_wait döndüğünde, uygun handler'a dağıt
- Beklemede harcanan thread yok

Kullananlar: nginx, Node.js (libuv), Redis, HAProxy, Boost.Asio

→ Ref: https://man7.org/linux/man-pages/man2/epoll_wait.2.html
→ Ref: https://man7.org/linux/man-pages/man2/poll.2.html
→ Ref: https://man7.org/linux/man-pages/man2/fcntl.2.html
→ Ref: https://en.wikipedia.org/wiki/C10k_problem

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <cstring>
#include <string>
#include <array>
#include <vector>
#include <unordered_map>
#include <functional>
#include <memory>
#include <algorithm>
#include <sstream>
#include <cerrno>
#include <cassert>
#include <iomanip>

// EN: POSIX / Linux headers for socket and epoll
// TR: Soket ve epoll için POSIX / Linux başlıkları
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <fcntl.h>
#include <poll.h>
#include <sys/epoll.h>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: Non-Blocking Socket Setup (RAII Wrapper)
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: RAII socket wrapper with non-blocking support. Builds on the Socket class
 *     from module_13/01, adding:
 *     - set_nonblocking(): sets O_NONBLOCK via fcntl()
 *     - set_reuseaddr(): prevents "Address already in use" error
 *     - Proper error handling without exceptions (returns error codes)
 *
 * TR: Bloklamayan desteğe sahip RAII soket sarmalayıcı. module_13/01'deki Socket
 *     sınıfının üzerine şunları ekler:
 *     - set_nonblocking(): fcntl() ile O_NONBLOCK ayarlar
 *     - set_reuseaddr(): "Address already in use" hatasını önler
 *     - İstisnasız uygun hata yönetimi (hata kodları döndürür)
 */
class Socket {
    int fd_ = -1;

public:
    Socket() = default;

    /// EN: Construct from an existing file descriptor (e.g., from accept())
    /// TR: Mevcut bir dosya tanımlayıcıdan oluştur (ör. accept()'ten)
    explicit Socket(int fd) : fd_(fd) {}

    /// EN: Create a new socket with given domain, type, protocol
    /// TR: Verilen domain, tip, protokol ile yeni soket oluştur
    static Socket create(int domain = AF_INET, int type = SOCK_STREAM, int protocol = 0) {
        int fd = ::socket(domain, type, protocol);
        if (fd < 0) {
            std::cerr << "  [ERROR] socket() failed: " << strerror(errno) << "\n";
        }
        return Socket(fd);
    }

    // EN: RAII — close on destruction / TR: RAII — yıkımda kapat
    ~Socket() { close(); }

    // EN: Move-only (transfer ownership like unique_ptr)
    // TR: Sadece taşınabilir (unique_ptr gibi sahiplik transferi)
    Socket(Socket&& other) noexcept : fd_(other.fd_) { other.fd_ = -1; }
    Socket& operator=(Socket&& other) noexcept {
```

```
71          if (this != &other) {
72              close();
73              fd_ = other.fd_;
74              other.fd_ = -1;
75          }
76          return *this;
77      }
78      Socket(const Socket&) = delete;
79      Socket& operator=(const Socket&) = delete;
80
81      void close() {
82          if (fd_ >= 0) {
83              ::close(fd_);
84              fd_ = -1;
85          }
86      }
87
88      [[nodiscard]] int fd() const { return fd_; }
89      [[nodiscard]] bool valid() const { return fd_ >= 0; }
90
91      /// EN: Set socket to non-blocking mode using fcntl O_NONBLOCK.
92      ///     After this, recv/send return EAGAIN instead of blocking.
93      /// TR: Soketi fcntl O_NONBLOCK ile bloklamayan moda ayarla.
94      ///     Bundan sonra recv/send bloklama yerine EAGAIN döndürür.
95      bool set_nonblocking() {
96          int flags = fcntl(fd_, F_GETFL, 0);
97          if (flags < 0) return false;
98          return fcntl(fd_, F_SETFL, flags | O_NONBLOCK) >= 0;
99      }
100
101     /// EN: Allow address reuse — prevents "Address already in use" after server restart
102     /// TR: Adres yeniden kullanımına izin ver — sunucu yeniden başlatıldıktan sonra
103     ///     "Address already in use" hatasını önler
104     bool set_reuseaddr() {
105         int opt = 1;
106         return setsockopt(fd_, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) >= 0;
107     }
108
109     /// EN: Bind to an address and port / TR: Adres ve porta bağla
110     bool bind(uint16_t port, const char* addr = "0.0.0.0") {
111         struct sockaddr_in sa{};
112         sa.sin_family = AF_INET;
113         sa.sin_port = htons(port);
114         inet_pton(AF_INET, addr, &sa.sin_addr);
115         return ::bind(fd_, reinterpret_cast<struct sockaddr*>(&sa), sizeof(sa)) >= 0;
116     }
117
118     /// EN: Start listening for connections / TR: Bağlantıları dinlemeye başla
119     bool listen(int backlog = SOMAXCONN) {
120         return ::listen(fd_, backlog) >= 0;
121     }
122
123     /// EN: Accept a new connection (returns a new Socket with the client fd)
124     /// TR: Yeni bağlantı kabul et (istemci fd'si ile yeni Socket döndürür)
125     Socket accept() {
126         struct sockaddr_in client_addr{};
127         socklen_t len = sizeof(client_addr);
128         int client_fd = ::accept(fd_, reinterpret_cast<struct sockaddr*>(&client_addr), &len);
129         return Socket(client_fd);
130     }
131
132     /// EN: Release ownership of the fd (caller takes responsibility)
133     /// TR: fd sahipliğini bırak (çağıran sorumluluk alır)
134     int release() {
135         int fd = fd_;
136         fd_ = -1;
137         return fd;
138     }
139 };
140
```

```cpp
// ═══════════════════════════════════════════════════════════════════════════
// PART 2: poll()-Based Multi-Client Server
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: A multi-client TCP echo server using poll(). poll() is more portable than
 *     epoll (works on macOS, BSDs, etc.) and has no FD limit like select().
 *
 *     How poll() works:
 *     - You provide an array of pollfd structs: {fd, events_to_watch, returned_events}
 *     - poll() blocks until at least one fd is ready
 *     - You iterate ALL fds to check which ones have events → O(n)
 *
 *     This is fine for ~100-1000 clients. For 10K+, use epoll.
 *
 * TR: poll() kullanan çoklu istemci TCP echo sunucusu. poll(), epoll'dan daha
 *     taşınabilirdir (macOS, BSD'lerde çalışır) ve select() gibi FD sınırı yoktur.
 *
 *     poll() nasıl çalışır:
 *     - pollfd yapıları dizisi verirsiniz: {fd, izlenecek_olaylar, dönen_olaylar}
 *     - En az bir fd hazır olana kadar poll() bloklar
 *     - Hangi olayların olduğunu kontrol etmek için TÜM fd'leri iterasyonla gezersiniz → O(n)
 *
 *     ~100-1000 istemci için uygundur. 10K+ için epoll kullanın.
 */
class PollServer {
    Socket listen_socket_;
    std::vector<struct pollfd> poll_fds_;
    static constexpr int BUFFER_SIZE = 1024;

public:
    /// EN: Initialize the server socket (bind + listen)
    /// TR: Sunucu soketini başlat (bağla + dinle)
    bool init(uint16_t port) {
        listen_socket_ = Socket::create();
        if (!listen_socket_.valid()) return false;

        listen_socket_.set_reuseaddr();
        listen_socket_.set_nonblocking();

        if (!listen_socket_.bind(port)) {
            std::cerr << "  [PollServer] Bind failed: " << strerror(errno) << "\n";
            return false;
        }
        if (!listen_socket_.listen()) {
            std::cerr << "  [PollServer] Listen failed: " << strerror(errno) << "\n";
            return false;
        }

        // EN: Add the listening socket as the first pollfd entry
        // TR: Dinleme soketini ilk pollfd girdisi olarak ekle
        poll_fds_.push_back({listen_socket_.fd(), POLLIN, 0});
        return true;
    }

    /// EN: Run one iteration of the poll loop (for demonstration).
    ///     In production, this would be an infinite loop.
    /// TR: Poll döngüsünün bir iterasyonunu çalıştır (gösterim için).
    ///     Üretimde bu sonsuz döngü olurdu.
    void poll_once(int timeout_ms = 100) {
        int ready = poll(poll_fds_.data(), static_cast<nfds_t>(poll_fds_.size()), timeout_ms);

        if (ready < 0) {
            if (errno != EINTR) {
                std::cerr << "  [PollServer] poll() error: " << strerror(errno) << "\n";
            }
            return;
        }

        if (ready == 0) return;  // EN: Timeout, no events / TR: Zaman aşımı, olay yok

        // EN: Check the listening socket for new connections
        // TR: Dinleme soketini yeni bağlantılar için kontrol et
        if (poll_fds_[0].revents & POLLIN) {
            accept_new_client();
        }
```

```cpp

        // EN: Check all client sockets for data (iterate backwards for safe removal)
        // TR: Tüm istemci soketlerini veri için kontrol et (güvenli kaldırma için geriye iterasyon)
        for (std::size_t i = poll_fds_.size(); i > 1; --i) {
            std::size_t idx = i - 1;
            if (poll_fds_[idx].revents & (POLLIN | POLLERR | POLLHUP)) {
                handle_client(idx);
            }
        }
    }

    [[nodiscard]] std::size_t client_count() const {
        return poll_fds_.size() > 0 ? poll_fds_.size() - 1 : 0;
    }

private:
    void accept_new_client() {
        auto client = listen_socket_.accept();
        if (!client.valid()) return;

        // EN: Set new client to non-blocking
        // TR: Yeni istemciyi bloklamayan yap
        client.set_nonblocking();
        int fd = client.release();  // EN: Transfer ownership to poll_fds_ / TR: Sahipliği poll_fds_'e aktar

        poll_fds_.push_back({fd, POLLIN, 0});
        std::cout << "  [PollServer] New client connected (fd=" << fd
                  << "), total clients: " << client_count() << "\n";
    }

    void handle_client(std::size_t idx) {
        char buffer[BUFFER_SIZE];
        ssize_t n = recv(poll_fds_[idx].fd, buffer, sizeof(buffer) - 1, 0);

        if (n <= 0) {
            // EN: Client disconnected or error → remove from poll set
            // TR: İstemci bağlantıyı kesti veya hata → poll kümesinden kaldır
            std::cout << "  [PollServer] Client disconnected (fd=" << poll_fds_[idx].fd << ")\n";
            ::close(poll_fds_[idx].fd);
            poll_fds_.erase(poll_fds_.begin() + static_cast<std::ptrdiff_t>(idx));
            return;
        }

        buffer[n] = '\0';
        std::cout << "  [PollServer] Received from fd=" << poll_fds_[idx].fd
                  << ": " << buffer;

        // EN: Echo back to client / TR: İstemciye geri yankıla
        send(poll_fds_[idx].fd, buffer, static_cast<size_t>(n), 0);
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: epoll Event Loop — Multi-Client TCP Server
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: High-performance multi-client TCP server using epoll (Linux).
 *     epoll is the gold standard for I/O multiplexing on Linux:
 *     - O(1) event notification (only returns READY fds)
 *     - Kernel maintains the interest list → no copying fd sets per call
 *     - Scales to millions of connections (used by nginx, Redis)
 *
 *     RAII design: epoll fd is managed automatically, client fds tracked in a map.
 *
 * TR: epoll (Linux) kullanan yüksek performanslı çoklu istemci TCP sunucusu.
 *     epoll, Linux'ta I/O çoklamanın altın standardıdır:
 *     - O(1) olay bildirimi (sadece HAZIR fd'leri döndürür)
 *     - Çekirdek ilgi listesini yönetir → çağrı başına fd seti kopyalama yok
 *     - Milyonlarca bağlantıya ölçeklenir (nginx, Redis tarafından kullanılır)
 *
 *     RAII tasarım: epoll fd otomatik yönetilir, istemci fd'leri haritada izlenir.
 */
```

```
290 class EpollServer {
291     Socket listen_socket_;
292     int epoll_fd_ = -1;
293     std::unordered_map<int, std::string> clients_;  // EN: fd → client info / TR: fd → istemci
    bilgisi
294     static constexpr int MAX_EVENTS = 64;
295     static constexpr int BUFFER_SIZE = 4096;
296
297 public:
298     EpollServer() = default;
299
300     ~EpollServer() {
301         // EN: Close all client connections / TR: Tüm istemci bağlantılarını kapat
302         for (auto& [fd, info] : clients_) {
303             ::close(fd);
304         }
305         // EN: Close epoll fd / TR: epoll fd'yi kapat
306         if (epoll_fd_ >= 0) ::close(epoll_fd_);
307     }
308
309     EpollServer(const EpollServer&) = delete;
310     EpollServer& operator=(const EpollServer&) = delete;
311
312     /// EN: Initialize: create socket, bind, listen, create epoll instance
313     /// TR: Başlat: soket oluştur, bağla, dinle, epoll örneği oluştur
314     bool init(uint16_t port) {
315         // EN: Step 1: Create and configure listening socket
316         // TR: Adım 1: Dinleme soketi oluştur ve yapılandır
317         listen_socket_ = Socket::create();
318         if (!listen_socket_.valid()) return false;
319
320         listen_socket_.set_reuseaddr();
321         listen_socket_.set_nonblocking();
322
323         if (!listen_socket_.bind(port)) {
324             std::cerr << "  [EpollServer] Bind failed: " << strerror(errno) << "\n";
325             return false;
326         }
327         if (!listen_socket_.listen()) {
328             std::cerr << "  [EpollServer] Listen failed: " << strerror(errno) << "\n";
329             return false;
330         }
331
332         // EN: Step 2: Create epoll instance
333         // TR: Adım 2: epoll örneği oluştur
334         epoll_fd_ = epoll_create1(0);
335         if (epoll_fd_ < 0) {
336             std::cerr << "  [EpollServer] epoll_create1 failed: " << strerror(errno) << "\n";
337             return false;
338         }
339
340         // EN: Step 3: Register listening socket with epoll
341         // TR: Adım 3: Dinleme soketini epoll'e kaydet
342         struct epoll_event ev{};
343         ev.events = EPOLLIN;               // EN: Watch for incoming connections / TR: Gelen
    bağlantıları izle
344         ev.data.fd = listen_socket_.fd();
345         if (epoll_ctl(epoll_fd_, EPOLL_CTL_ADD, listen_socket_.fd(), &ev) < 0) {
346             std::cerr << "  [EpollServer] epoll_ctl ADD failed: " << strerror(errno) << "\n";
347             return false;
348         }
349
350         std::cout << "  [EpollServer] Listening on port " << port << "...\n";
351         return true;
352     }
353
```

```
354     /// EN: Run one iteration of the epoll event loop
355     /// TR: epoll olay döngüsünün bir iterasyonunu çalıştır
356     void poll_once(int timeout_ms = 100) {
357         std::array<struct epoll_event, MAX_EVENTS> events{};
358         int n = epoll_wait(epoll_fd_, events.data(), MAX_EVENTS, timeout_ms);
359 
360         if (n < 0) {
361             if (errno != EINTR) {
362                 std::cerr << "  [EpollServer] epoll_wait error: " << strerror(errno) << "\n";
363             }
364             return;
365         }
366 
367         // EN: Process ONLY the ready events (this is why epoll is O(1) per ready fd)
368         // TR: SADECE hazır olayları işle (epoll'un hazır fd başına O(1) olmasının nedeni)
369         for (int i = 0; i < n; ++i) {
370             if (events[static_cast<std::size_t>(i)].data.fd == listen_socket_.fd()) {
371                 accept_new_client();
372             } else {
373                 handle_client_event(events[static_cast<std::size_t>(i)]);
374             }
375         }
376     }
377 
378     [[nodiscard]] std::size_t client_count() const { return clients_.size(); }
379 
380 private:
381     void accept_new_client() {
382         // EN: Accept in a loop — epoll may batch multiple accepts
383         // TR: Döngüde kabul et — epoll birden fazla accept'i gruplayabilir
384         while (true) {
385             auto client = listen_socket_.accept();
386             if (!client.valid()) {
387                 if (errno == EAGAIN || errno == EWOULDBLOCK) break;  // EN: No more pending / TR:
    Bekleyen kalmadı
388                 break;
389             }
390 
391             client.set_nonblocking();
392             int fd = client.release();
393 
394             // EN: Register the new client with epoll (watch for read events)
395             // TR: Yeni istemciyi epoll'e kaydet (okuma olaylarını izle)
396             struct epoll_event ev{};
397             ev.events = EPOLLIN | EPOLLET;  // EN: Edge-triggered for efficiency / TR: Verimlilik
    için kenar tetiklemeli
398             ev.data.fd = fd;
399             if (epoll_ctl(epoll_fd_, EPOLL_CTL_ADD, fd, &ev) < 0) {
400                 ::close(fd);
401                 continue;
402             }
403 
404             struct sockaddr_in peer{};
405             socklen_t peer_len = sizeof(peer);
406             getpeername(fd, reinterpret_cast<struct sockaddr*>(&peer), &peer_len);
407 
408             char addr_str[INET_ADDRSTRLEN];
409             inet_ntop(AF_INET, &peer.sin_addr, addr_str, sizeof(addr_str));
410 
411             clients_[fd] = std::string(addr_str) + ":" + std::to_string(ntohs(peer.sin_port));
412             std::cout << "  [EpollServer] Client connected: " << clients_[fd]
413                       << " (fd=" << fd << "), total: " << clients_.size() << "\n";
414         }
415     }
416 
417     void handle_client_event(const struct epoll_event& event) {
418         int fd = event.data.fd;
419 
420         if (event.events & (EPOLLERR | EPOLLHUP)) {
421             remove_client(fd);
422             return;
423         }
```

```

        if (event.events & EPOLLIN) {
            // EN: Edge-triggered: MUST read ALL available data
            // TR: Kenar tetiklemeli: TÜM mevcut veriyi okuMALISIN
            char buffer[BUFFER_SIZE];
            while (true) {
                ssize_t n = recv(fd, buffer, sizeof(buffer) - 1, 0);

                if (n < 0) {
                    if (errno == EAGAIN || errno == EWOULDBLOCK) break;  // EN: All data read / TR: Tüm veri okundu
                    remove_client(fd);
                    return;
                }

                if (n == 0) {
                    // EN: Client gracefully disconnected
                    // TR: İstemci düzgünce bağlantıyı kesti
                    remove_client(fd);
                    return;
                }

                buffer[n] = '\0';
                std::cout << "  [EpollServer] From " << clients_[fd] << ": " << buffer;

                // EN: Echo back / TR: Geri yankıla
                send(fd, buffer, static_cast<size_t>(n), MSG_NOSIGNAL);
            }
        }
    }

    void remove_client(int fd) {
        // EN: epoll automatically removes closed fds, but we call CTL_DEL for clarity
        // TR: epoll kapatılan fd'leri otomatik kaldırır ama açıklık için CTL_DEL çağırırız
        epoll_ctl(epoll_fd_, EPOLL_CTL_DEL, fd, nullptr);
        std::cout << "  [EpollServer] Client disconnected: " << clients_[fd] << "\n";
        clients_.erase(fd);
        ::close(fd);
    }
};

// ============================================================================
// PART 4: Edge-Triggered epoll with RAII (EpollHandle)
// ============================================================================

/**
 * EN: RAII wrapper for an epoll file descriptor. Ensures the epoll instance is
 *     always properly closed, even on exceptions. Follows the Rule of Five.
 *
 *     This is a reusable building block for any epoll-based server or event loop.
 *     Combined with the Socket RAII wrapper, you get fully deterministic cleanup.
 *
 * TR: epoll dosya tanımlayıcısı için RAII sarmalayıcı. İstisna durumunda bile
 *     epoll örneğinin her zaman düzgün kapatılmasını sağlar. Beşli Kuralını takip eder.
 *
 *     Herhangi bir epoll tabanlı sunucu veya olay döngüsü için yeniden kullanılabilir
 *     yapı taşıdır. Socket RAII sarmalayıcısıyla birlikte tam belirleyici temizlik sağlar.
 */
class EpollHandle {
    int fd_ = -1;

public:
    EpollHandle() : fd_(epoll_create1(0)) {
        if (fd_ < 0) {
            std::cerr << "  [EpollHandle] epoll_create1 failed: " << strerror(errno) << "\n";
        }
    }

    ~EpollHandle() { if (fd_ >= 0) ::close(fd_); }

    // EN: Move-only / TR: Sadece taşınabilir
    EpollHandle(EpollHandle&& o) noexcept : fd_(o.fd_) { o.fd_ = -1; }
    EpollHandle& operator=(EpollHandle&& o) noexcept {
        if (this != &o) { if (fd_ >= 0) ::close(fd_); fd_ = o.fd_; o.fd_ = -1; }
        return *this;
    }
    EpollHandle(const EpollHandle&) = delete;
```

```
    EpollHandle& operator=(const EpollHandle&) = delete;

    [[nodiscard]] int fd() const { return fd_; }
    [[nodiscard]] bool valid() const { return fd_ >= 0; }

    /// EN: Add a file descriptor to the epoll interest list
    /// TR: Dosya tanımlayıcısını epoll ilgi listesine ekle
    bool add(int target_fd, uint32_t events) {
        struct epoll_event ev{};
        ev.events = events;
        ev.data.fd = target_fd;
        return epoll_ctl(fd_, EPOLL_CTL_ADD, target_fd, &ev) >= 0;
    }

    /// EN: Modify events for an already-registered fd
    /// TR: Zaten kayıtlı bir fd için olayları değiştir
    bool modify(int target_fd, uint32_t events) {
        struct epoll_event ev{};
        ev.events = events;
        ev.data.fd = target_fd;
        return epoll_ctl(fd_, EPOLL_CTL_MOD, target_fd, &ev) >= 0;
    }

    /// EN: Remove a file descriptor from epoll
    /// TR: Dosya tanımlayıcısını epoll'den kaldır
    bool remove(int target_fd) {
        return epoll_ctl(fd_, EPOLL_CTL_DEL, target_fd, nullptr) >= 0;
    }

    /// EN: Wait for events with timeout
    /// TR: Zaman aşımı ile olayları bekle
    int wait(struct epoll_event* events, int max_events, int timeout_ms) {
        return epoll_wait(fd_, events, max_events, timeout_ms);
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 5: Real-World — Diagnostic Gateway (Automotive)
// ═══════════════════════════════════════════════════════════════════════════

/**
 * EN: Simulates an automotive diagnostic gateway scenario:
 *     A diagnostic tester (like ADOS, CANoe, or a workshop tool) connects to
 *     the ECU's diagnostic port. The gateway:
 *     1. Listens for incoming diagnostic clients (UDS over DoIP)
 *     2. Routes diagnostic requests to appropriate ECUs
 *     3. Handles multiple concurrent diagnostic sessions
 *
 *     In real automotive systems, this is built on top of epoll/IOCP for performance.
 *     The gateway must handle: diagnostic session timeouts, priority scheduling,
 *     and keep-alive messages — all while maintaining low latency.
 *
 * TR: Otomotiv teşhis ağ geçidi senaryosunu simüle eder:
 *     Teşhis test cihazı (ADOS, CANoe veya atölye aracı gibi) ECU'nun teşhis
 *     portuna bağlanır. Ağ geçidi:
 *     1. Gelen teşhis istemcilerini dinler (DoIP üzerinden UDS)
 *     2. Teşhis isteklerini uygun ECU'lara yönlendirir
 *     3. Birden fazla eşzamanlı teşhis oturumunu yönetir
 *
 *     Gerçek otomotiv sistemlerinde performans için epoll/IOCP üzerine inşa edilir.
 */

/// EN: UDS (Unified Diagnostic Services) service identifiers — ISO 14229
/// TR: UDS (Birleşik Teşhis Servisleri) servis tanımlayıcıları — ISO 14229
enum class UdsService : uint8_t {
    DiagnosticSessionControl = 0x10,
    ECUReset                 = 0x11,
    ReadDataByIdentifier     = 0x22,
    WriteDataByIdentifier    = 0x2E,
    RoutineControl           = 0x31,
    RequestDownload          = 0x34,
    TesterPresent            = 0x3E,
};
```

```cpp

const char* uds_service_name(UdsService svc) {
    switch (svc) {
        case UdsService::DiagnosticSessionControl: return "DiagSessionCtrl(0x10)";
        case UdsService::ECUReset:                 return "ECUReset(0x11)";
        case UdsService::ReadDataByIdentifier:     return "ReadDID(0x22)";
        case UdsService::WriteDataByIdentifier:    return "WriteDID(0x2E)";
        case UdsService::RoutineControl:           return "RoutineCtrl(0x31)";
        case UdsService::RequestDownload:          return "ReqDownload(0x34)";
        case UdsService::TesterPresent:            return "TesterPresent(0x3E)";
        default: return "Unknown";
    }
}

/// EN: Simulated diagnostic request routing
/// TR: Simüle edilmiş teşhis istek yönlendirmesi
struct DiagRequest {
    UdsService service;
    uint16_t target_ecu_id;            // EN: Logical ECU address / TR: Mantıksal ECU adresi
    std::vector<uint8_t> payload;
};

/// EN: Simulated diagnostic response
/// TR: Simüle edilmiş teşhis yanıtı
struct DiagResponse {
    uint8_t service_id;                // EN: service + 0x40 = positive response / TR: servis + 0x40 = olumlu yanıt
    uint16_t source_ecu_id;
    std::vector<uint8_t> data;
    bool positive;
};

DiagResponse simulate_ecu_response(const DiagRequest& req) {
    DiagResponse resp;
    resp.service_id = static_cast<uint8_t>(req.service) + 0x40;  // EN: Positive response / TR: Olumlu yanıt
    resp.source_ecu_id = req.target_ecu_id;
    resp.positive = true;

    switch (req.service) {
        case UdsService::ReadDataByIdentifier:
            resp.data = {0xF1, 0x90, 'V', 'I', 'N', '1', '2', '3'};  // EN: VIN example / TR: VIN örneği
            break;
        case UdsService::TesterPresent:
            resp.data = {};  // EN: Empty positive response / TR: Boş olumlu yanıt
            break;
        default:
            resp.data = {0x00};  // EN: Generic positive / TR: Genel olumlu
            break;
    }
    return resp;
}

// ═══════════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 13 — Non-Blocking I/O & epoll\n";
    std::cout << "============================================\n\n";

    // ─── Demo 1: Non-Blocking Socket Setup ──────────────────────────────
    {
        std::cout << "--- Demo 1: Non-Blocking Socket Setup ---\n";

        // EN: Create a socket and set it to non-blocking mode
        // TR: Bir soket oluştur ve bloklamayan moda ayarla
        Socket sock = Socket::create(AF_INET, SOCK_STREAM, 0);
        if (sock.valid()) {
            std::cout << "  Socket created: fd=" << sock.fd() << "\n";

```

```cpp
            // EN: Set non-blocking — recv will return EAGAIN instead of sleeping
            // TR: Bloklamayan ayarla — recv uyumak yerine EAGAIN döndürecek
            if (sock.set_nonblocking()) {
                std::cout << "  Set to non-blocking mode (O_NONBLOCK)\n";

                // EN: Verify with fcntl
                // TR: fcntl ile doğrula
                int flags = fcntl(sock.fd(), F_GETFL, 0);
                std::cout << "  Socket flags: 0x" << std::hex << flags << std::dec;
                if (flags & O_NONBLOCK) {
                    std::cout << " [O_NONBLOCK is SET]\n";
                } else {
                    std::cout << " [O_NONBLOCK is NOT set]\n";
                }
            }

            // EN: Set address reuse — essential for server sockets
            // TR: Adres yeniden kullanımını ayarla — sunucu soketleri için zorunlu
            if (sock.set_reuseaddr()) {
                std::cout << "  SO_REUSEADDR enabled\n";
            }
        }

        std::cout << "  Socket will be closed automatically (RAII)\n\n";
        // EN: Socket closes here — RAII destructor / TR: Soket burada kapanır — RAII destructor
    }

    // ─── Demo 2: poll()-Based Server Concept ──────────────────────────
    {
        std::cout << "--- Demo 2: poll() Server Architecture ---\n";
        std::cout << "  poll() provides I/O multiplexing without the FD limit of select().\n\n";

        // EN: Demonstrate poll concepts without starting a server
        //     (starting a server would require a separate client process)
        // TR: Sunucu başlatmadan poll kavramlarını göster
        std::cout << "  pollfd structure:\n";
        std::cout << "    struct pollfd {\n";
        std::cout << "      int   fd;       // file descriptor to watch\n";
        std::cout << "      short events;   // events to watch (POLLIN, POLLOUT)\n";
        std::cout << "      short revents;  // events that occurred (filled by poll)\n";
        std::cout << "    };\n\n";

        std::cout << "  Event flags:\n";
        std::cout << "    POLLIN  = 0x" << std::hex << POLLIN  << " → data available to read\n";
        std::cout << "    POLLOUT = 0x" << POLLOUT << " → ready to write\n";
        std::cout << "    POLLERR = 0x" << POLLERR << " → error condition\n";
        std::cout << "    POLLHUP = 0x" << POLLHUP << " → hang up (peer closed)\n";
        std::cout << std::dec;

        // EN: Demonstrate with a self-connected pipe (non-network example of poll)
        // TR: Kendi kendine bağlı pipe ile göster (poll'un ağ dışı örneği)
        int pipefd[2];
        if (pipe(pipefd) == 0) {
            // EN: Write some data to the pipe
            // TR: Pipe'a bazı veriler yaz
            const char* msg = "Hello from pipe!";
            write(pipefd[1], msg, strlen(msg));

            // EN: Poll the read end — should be immediately ready
            // TR: Okuma ucunu poll et — hemen hazır olmalı
            struct pollfd pfd{};
            pfd.fd = pipefd[0];
            pfd.events = POLLIN;

            int ready = poll(&pfd, 1, 0);  // EN: timeout=0 → non-blocking check / TR: timeout=0 → bloklamayan kontrol
            std::cout << "\n  poll() on pipe read-end: ready=" << ready;
            if (pfd.revents & POLLIN) {
                char buf[64];
                ssize_t n = read(pipefd[0], buf, sizeof(buf) - 1);
                if (n > 0) {
                    buf[n] = '\0';
                    std::cout << ", data=\"" << buf << "\"";
                }
            }
            std::cout << "\n";

```

```
            close(pipefd[0]);
            close(pipefd[1]);
        }
        std::cout << "\n";
    }

    // ── Demo 3: epoll Server Architecture ──────────────────────────────
    {
        std::cout << "--- Demo 3: epoll Server Architecture ---\n";
        std::cout << "  epoll is Linux's high-performance I/O multiplexing mechanism.\n\n";

        // EN: Demonstrate epoll concepts with the RAII wrapper
        // TR: RAII sarmalayıcı ile epoll kavramlarını göster
        EpollHandle epoller;
        if (epoller.valid()) {
            std::cout << "  epoll instance created: fd=" << epoller.fd() << "\n";

            // EN: Demonstrate with a pipe (like demo 2, but using epoll)
            // TR: Pipe ile göster (demo 2 gibi ama epoll kullanarak)
            int pipefd[2];
            if (pipe(pipefd) == 0) {
                // EN: Set read end to non-blocking
                // TR: Okuma ucunu bloklamayan yap
                int flags = fcntl(pipefd[0], F_GETFL, 0);
                fcntl(pipefd[0], F_SETFL, flags | O_NONBLOCK);

                // EN: Register read end with epoll
                // TR: Okuma ucunu epoll'e kaydet
                if (epoller.add(pipefd[0], EPOLLIN | EPOLLET)) {
                    std::cout << "  Registered pipe read-end with epoll (EPOLLIN | EPOLLET)\n";
                }

                // EN: Write data to trigger the event
                // TR: Olayı tetiklemek için veri yaz
                const char* msg = "epoll event triggered!";
                write(pipefd[1], msg, strlen(msg));

                // EN: Wait for events (should be immediately ready)
                // TR: Olayları bekle (hemen hazır olmalı)
                struct epoll_event events[4];
                int n = epoller.wait(events, 4, 100);
                std::cout << "  epoll_wait returned: " << n << " event(s)\n";

                for (int i = 0; i < n; ++i) {
                    if (events[i].events & EPOLLIN) {
                        char buf[128];
                        ssize_t bytes = read(events[i].data.fd, buf, sizeof(buf) - 1);
                        if (bytes > 0) {
                            buf[bytes] = '\0';
                            std::cout << "  [EPOLLIN] fd=" << events[i].data.fd
                                      << " data=\"" << buf << "\"\n";
                        }
                    }
                }

                // EN: Edge-triggered: second read should return EAGAIN (no new data)
                // TR: Kenar tetiklemeli: ikinci okuma EAGAIN döndürmeli (yeni veri yok)
                char buf2[128];
                ssize_t n2 = read(pipefd[0], buf2, sizeof(buf2));
                if (n2 < 0 && (errno == EAGAIN || errno == EWOULDBLOCK)) {
                    std::cout << "  [ET Mode] Second read → EAGAIN (no new data, as expected)\n";
                }

                epoller.remove(pipefd[0]);
                close(pipefd[0]);
                close(pipefd[1]);
            }
        }
        std::cout << "\n";
    }
```

```cpp

    // ─── Demo 4: epoll Level-Triggered vs Edge-Triggered ─────────────────
    {
        std::cout << "--- Demo 4: Level-Triggered (LT) vs Edge-Triggered (ET) ---\n\n";

        std::cout << "  Level-Triggered (default):\n";
        std::cout << "    • epoll_wait returns AS LONG AS data is available\n";
        std::cout << "    • You can read partial data and get notified again\n";
        std::cout << "    • Simpler but slightly less efficient\n";
        std::cout << "    • Safe for beginners\n\n";

        std::cout << "  Edge-Triggered (EPOLLET):\n";
        std::cout << "    • epoll_wait returns ONLY when NEW data arrives\n";
        std::cout << "    • You MUST read ALL data in one go (loop until EAGAIN)\n";
        std::cout << "    • More efficient (fewer epoll_wait wakeups)\n";
        std::cout << "    • Used in production servers (nginx, etc.)\n\n";

        // EN: Practical demonstration with pipe
        // TR: Pipe ile pratik gösteri
        int pipefd[2];
        if (pipe(pipefd) == 0) {
            int flags = fcntl(pipefd[0], F_GETFL, 0);
            fcntl(pipefd[0], F_SETFL, flags | O_NONBLOCK);

            // EN: Test Level-Triggered (LT) behavior
            // TR: Seviye Tetiklemeli (LT) davranışını test et
            {
                EpollHandle lt_epoller;
                lt_epoller.add(pipefd[0], EPOLLIN);  // EN: No EPOLLET = Level-Triggered / TR: EPOLLET yok = Seviye Tetiklemeli

                // EN: Write 10 bytes / TR: 10 bayt yaz
                write(pipefd[1], "1234567890", 10);

                // EN: First epoll_wait — should be ready (10 bytes available)
                // TR: İlk epoll_wait — hazır olmalı (10 bayt mevcut)
                struct epoll_event events[4];
                int n = lt_epoller.wait(events, 4, 50);
                std::cout << "  [LT] After writing 10 bytes: epoll_wait returned " << n << " event(s)\n";

                // EN: Read only 5 bytes (partial read)
                // TR: Sadece 5 bayt oku (kısmi okuma)
                char buf[16];
                read(pipefd[0], buf, 5);
                std::cout << "  [LT] Read 5 bytes. 5 bytes remaining in pipe.\n";

                // EN: Second epoll_wait — LT will STILL report ready (5 bytes remain!)
                // TR: İkinci epoll_wait — LT HALİYLE hazır bildirecek (5 bayt kaldı!)
                n = lt_epoller.wait(events, 4, 50);
                std::cout << "  [LT] Second epoll_wait: returned " << n << " event(s) → "
                          << (n > 0 ? "STILL READY (correct for LT)" : "NOT ready") << "\n";

                // EN: Drain remaining / TR: Kalanı boşalt
                read(pipefd[0], buf, sizeof(buf));
                lt_epoller.remove(pipefd[0]);
            }

            // EN: Test Edge-Triggered (ET) behavior
            // TR: Kenar Tetiklemeli (ET) davranışını test et
            {
                EpollHandle et_epoller;
                et_epoller.add(pipefd[0], EPOLLIN | EPOLLET);  // EN: EPOLLET = Edge-Triggered / TR: EPOLLET = Kenar Tetiklemeli

                // EN: Write 10 bytes / TR: 10 bayt yaz
                write(pipefd[1], "ABCDEFGHIJ", 10);

                struct epoll_event events[4];
                int n = et_epoller.wait(events, 4, 50);
                std::cout << "\n  [ET] After writing 10 bytes: epoll_wait returned " << n << " event(s)\n";

```

```
                // EN: Read only 5 bytes (partial read)
                // TR: Sadece 5 bayt oku (kısmi okuma)
                char buf[16];
                read(pipefd[0], buf, 5);
                std::cout << "  [ET] Read 5 bytes. 5 bytes remaining.\n";

                // EN: Second epoll_wait — ET will NOT report ready (no NEW data arrived!)
                // TR: İkinci epoll_wait — ET hazır BİLDİRMEYECEK (YENİ veri gelmedi!)
                n = et_epoller.wait(events, 4, 50);
                std::cout << "  [ET] Second epoll_wait: returned " << n << " event(s) → "
                          << (n == 0 ? "NOT READY (correct for ET — no new data!)" : "READY") << "\n";
                std::cout << "  [ET] ⚠  This is why ET mode MUST drain all data in one read!\n";

                // EN: Drain remaining / TR: Kalanı boşalt
                read(pipefd[0], buf, sizeof(buf));
                et_epoller.remove(pipefd[0]);
            }

            close(pipefd[0]);
            close(pipefd[1]);
        }
        std::cout << "\n";
    }

    // ─── Demo 5: Diagnostic Gateway Simulation ──────────────────────────
    {
        std::cout << "--- Demo 5: Automotive Diagnostic Gateway (UDS/DoIP) ---\n";
        std::cout << "  Simulating diagnostic request/response routing.\n\n";

        // EN: Simulate diagnostic requests from a tester tool
        // TR: Bir test cihazından teşhis isteklerini simüle et
        std::vector<DiagRequest> requests = {
            {UdsService::DiagnosticSessionControl, 0x0010, {0x01}},        // EN: Default session / TR: Varsayılan oturum
            {UdsService::TesterPresent,            0x0010, {0x00}},        // EN: Keep-alive / TR: Canlı tut
            {UdsService::ReadDataByIdentifier,     0x0020, {0xF1, 0x90}},  // EN: Read VIN / TR: VIN oku
            {UdsService::RoutineControl,           0x0030, {0x01, 0xFF, 0x00}}, // EN: Start routine / TR: Rutin başlat
            {UdsService::ECUReset,                 0x0010, {0x01}},        // EN: Hard reset / TR: Sert sıfırlama
        };

        std::cout << "  Tester → Gateway → ECU routing:\n";
        std::cout << "  ┌──────────────────────────┬──────────┬──────────────────────┐\n";
        std::cout << "  │ Service                  │ ECU Addr │ Response             │\n";
        std::cout << "  ├──────────────────────────┼──────────┼──────────────────────┤\n";

        for (const auto& req : requests) {
            auto resp = simulate_ecu_response(req);

            std::cout << "  │ " << std::left << std::setw(26) << uds_service_name(req.service)
                      << "│ 0x" << std::hex << std::setfill('0') << std::setw(4) << req.target_ecu_id
                      << std::dec << std::setfill(' ')
                      << "   │ " << std::setw(21)
                      << (resp.positive ? "Positive (+" + std::to_string(resp.service_id) + "h)" : "Negative")
                      << "│\n";
        }

        std::cout << "  └──────────────────────────┴──────────┴──────────────────────┘\n";

        // EN: Demonstrate timing of diagnostic sessions
        // TR: Teşhis oturumlarının zamanlamasını göster
        std::cout << "\n  Diagnostic session timing (S3 timeout = 5s):\n";
        std::cout << "    Tester → DiagSessionCtrl(0x10, Extended) → ECU: Positive\n";
        std::cout << "    ...3s... Tester → TesterPresent(0x3E) → ECU: Positive (session alive)\n";
        std::cout << "    ...3s... Tester → TesterPresent(0x3E) → ECU: Positive (session alive)\n";
        std::cout << "    ...6s... (no TesterPresent) → ECU: Session timeout → Default session\n";
    }

```

```
    std::cout << "\n==========================================\n";
    std::cout << "   End of Module 13 — Non-Blocking I/O\n";
    std::cout << "==========================================\n";

    return 0;
}
```

Modül 14: Automotive Networks — Otomotiv Ağları

Otomotiv Ağları

6 kaynak dosya · Standart: C++20

C++20

► 01. Can Fd And Xl

module_14_automotive_networks/01_can_fd_and_xl.cpp

Konu: CAN FD & CAN XL — Modern CAN Protocols — Modern CAN Protokolleri

Teori / Theory

THEORY: CAN FD — Flexible Data-rate

CAN FD (ISO 11898-1:2015) extends Classic CAN with two key improvements:
1) Higher data rates: arbitration at 500 kbps, data phase up to 8 Mbps
2) Larger payloads: up to 64 bytes (vs 8 bytes in Classic CAN)

Feature	Classic CAN	CAN FD	CAN XL
Max Data (bytes)	8	64	2048
Max Bitrate (data)	1 Mbps	8 Mbps	20 Mbps
Arbitration	ID-based	ID-based	ID + priority
CRC Field	CRC-15	CRC-17/CRC-21	CRC-32
Error Handling	Error frame	Error frame	Error frame
Backward Compat.	—	Classic CAN	CAN FD+Classic
ISO Standard	11898-1:2003	11898-1:2015	CiA 610-1
Typical Use	Body, Pwtrain	ADAS, GW	Eth bridge

CAN FD Frame format (key fields):

SOF	Arb	IDE	FDF	BRS	ESI	DLC → Data	CRC	EOF
1b	11/29	1b	1b	1b	1b	0-64 bytes	17/21	7b

FDF = FD Format (1=FD, 0=Classic) BRS = Bit Rate Switch (1=higher speed for data phase) ESI = Error State Indicator (0=active, 1=passive)

TEORİ: CAN FD — Esnek Veri Hızı

CAN FD, klasik CAN'ı iki önemli iyileştirme ile genişletir:
1) Daha yüksek veri hızları: arbitrasyon 500 kbps, veri fazı 8 Mbps'ye kadar
2) Daha büyük veri yükü: 64 byte'a kadar (Klasik CAN'da 8 byte)

Özellik	Klasik CAN	CAN FD	CAN XL
Maks Veri (byte)	8	64	2048
Maks Bit Hızı (veri)	1 Mbps	8 Mbps	20 Mbps
Arbitrasyon	ID tabanlı	ID tabanlı	ID + öncelik
CRC Alanı	CRC-15	CRC-17/CRC-21	CRC-32
Geriye Uyumluluk	—	Klasik CAN	CAN FD+Klasik
ISO Standardı	11898-1:2003	11898-1:2015	CiA 610-1
Tipik Kullanım	Gövde, Motor	ADAS, GW	Eth köprüsü

CAN FD Çerçeve formatı (anahtar alanlar):

SOF	Arb	IDE	FDF	BRS	ESI	DLC → Veri	CRC	EOF
1b	11/29	1b	1b	1b	1b	0-64 byte	17/21	7b

FDF = FD Format (1=FD, 0=Klasik) — FD çerçevesi olduğunu belirtir BRS = Bit Rate Switch (1=veri fazında yüksek hız) — hız geçişi kontrolü ESI = Error State Indicator (0=aktif, 1=pasif) — hata durumu bildirimi

THEORY: CAN XL — Extended Length

CAN XL (CiA 610-1, 2024) bridges CAN and Ethernet worlds:
- Up to 2048 bytes payload
- Up to 20 Mbps data rate
- SDU Type field enables Ethernet frame tunneling
- Priority-based arbitration (0=highest)
- Designed for zonal architecture backbone

TEORİ: CAN XL — Genişletilmiş Uzunluk

CAN XL, CAN ve Ethernet dünyalarını birleştirir:
- 2048 byte'a kadar veri yükü (CAN FD'nin 32 katı)
- 20 Mbps'ye kadar veri hızı
- SDU Type alanı ile Ethernet çerçeve tünellemesi (0x05 = Ethernet)
- Öncelik tabanlı arbitrasyon (0 = en yüksek öncelik)
- Zonal mimari omurgası (domain → zone geçişi) için tasarlanmıştır

THEORY: DBC File Format — CAN Database

DBC (DataBase CAN) files define signal layout within CAN messages. Used by Vector CANdb++, PEAK Symbol Editor, Kayak, cantools (Python).
Example DBC entry: BO_ 0x123 EngineData: 8 Engine_ECU SG_ RPM : 0|16@1+ (0.25,0) [0|16383.75] "rpm" Gateway SG_ Temp : 16|8@1+ (1,-40) [-40|215] "degC" Gateway
Format: SG_ <name> : <start_bit>|<length>@<byte_order><sign> (<factor>,<offset>) [<min>|<max>] "<unit>" <receivers> byte_order: 1=little-endian(Intel), 0=big-endian(Motorola)

TEORİ: CAN Veritabanı

DBC dosyaları CAN mesajları içindeki sinyal düzenini tanımlar. Araçlar: Vector CANdb++, PEAK Symbol Editor, Kayak, cantools (Python).
Örnek DBC girişi: BO_ 0x123 EngineData: 8 Engine_ECU SG_ RPM : 0|16@1+ (0.25,0) [0|16383.75] "rpm" Gateway SG_ Temp : 16|8@1+ (1,-40) [-40|215] "degC" Gateway
Format: SG_ <ad> : <başlat_bit>|<uzunluk>@<byte_sırası><işaret> (<faktör>,<offset>) [<min>|<maks>] "<birim>" <alıcılar> byte_sırası: 1=Intel (little-endian), 0=Motorola (big-endian) Fiziksel = ham_değer × faktör + offset

THEORY: SocketCAN — Linux CAN Interface

SocketCAN is the Linux kernel's CAN subsystem. It provides:
- Standard socket API for CAN (PF_CAN address family)
- Virtual CAN interface (vcan) for testing without hardware
- CAN filters (mask + id) for selective reception
- Support for classic CAN, CAN FD, and raw frames
Setup: sudo modprobe vcan sudo ip link add dev vcan0 type vcan sudo ip link set up vcan0

TEORİ: SocketCAN — Linux CAN Arayüzü

SocketCAN, Linux çekirdeğinin CAN alt sistemidir. Sağladıkları:
- CAN için standart soket API'si (PF_CAN adres ailesi)
- Sanal CAN arayüzü (vcan) — donanımsız test için
- CAN filtreleri (mask + id) — seçici alım: (alınan_id & mask) == (filtre_id & mask)
- Klasik CAN, CAN FD ve ham çerçeve desteği
Kurulum: sudo modprobe vcan sudo ip link add dev vcan0 type vcan sudo ip link set up vcan0

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <cstring>
#include <array>
#include <vector>
#include <string>
#include <algorithm>
#include <sstream>
#include <iomanip>
#include <functional>
#include <map>
#include <cassert>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: CAN Frame Structures — Classic CAN, CAN FD, CAN XL
// ═══════════════════════════════════════════════════════════════════════════

// EN: DLC (Data Length Code) to actual byte count for CAN FD
// TR: CAN FD için DLC → gerçek byte sayısı dönüşümü
constexpr uint8_t dlc_to_len(uint8_t dlc) {
    constexpr uint8_t table[] = {0,1,2,3,4,5,6,7,8,12,16,20,24,32,48,64};
    return (dlc < 16) ? table[dlc] : 64;
}

// EN: Reverse: byte count to DLC
// TR: Tersine: byte sayısı → DLC
constexpr uint8_t len_to_dlc(uint8_t len) {
    if (len <= 8) return len;
    if (len <= 12) return 9;
    if (len <= 16) return 10;
    if (len <= 20) return 11;
    if (len <= 24) return 12;
    if (len <= 32) return 13;
    if (len <= 48) return 14;
```

```
    return 15;  // 49-64 bytes
}

// EN: Classic CAN frame (8 bytes max)
// TR: Klasik CAN frame (maks 8 byte)
struct ClassicCANFrame {
    uint32_t id       = 0;     // 11-bit (standart) veya 29-bit (genişletilmiş) arbitrasyon ID'si
    bool     extended = false; // IDE: 0=standart (11-bit), 1=genişletilmiş (29-bit)
    bool     rtr      = false; // Remote Transmission Request — veri yerine uzak istek
    uint8_t  dlc      = 0;    // Data Length Code (0-8) — veri alanı byte sayısı
    std::array<uint8_t, 8> data{};

    void print(const std::string& label) const {
        std::cout << "  [Classic CAN] " << label << "\n";
        std::cout << "    ID: 0x" << std::hex << std::setw(3) << std::setfill('0') << id
                  << std::dec << (extended ? " (EXT 29-bit)" : " (STD 11-bit)")
                  << " | DLC: " << static_cast<int>(dlc)
                  << " | RTR: " << (rtr ? "Yes" : "No") << "\n";
        if (!rtr && dlc > 0) {
            std::cout << "    Data:";
            for (uint8_t i = 0; i < dlc && i < 8; ++i)
                std::cout << " " << std::hex << std::setw(2) << std::setfill('0')
                          << static_cast<int>(data[i]);
            std::cout << std::dec << "\n";
        }
    }
};

// EN: CAN FD frame (up to 64 bytes, dual bit rate)
// TR: CAN FD frame (64 byte'a kadar, çift bit hızı)
struct CANFDFrame {
    uint32_t id       = 0;
    bool     extended = false;
    bool     brs      = false; // Bit Rate Switch: veri fazında yüksek hız (arbitrasyon 500k, veri 2-8M)
    bool     esi      = false; // Error State Indicator: 0=aktif (sağlıklı), 1=pasif (hata sayıcı yüksek)
    uint8_t  dlc      = 0;    // DLC 0-15 (dlc_to_len ile 0-64 byte'a eşlenir)
    std::array<uint8_t, 64> data{};

    uint8_t data_length() const { return dlc_to_len(dlc); }

    void print(const std::string& label) const {
        std::cout << "  [CAN FD] " << label << "\n";
        std::cout << "    ID: 0x" << std::hex << std::setw(3) << std::setfill('0') << id
                  << std::dec << (extended ? " (EXT)" : " (STD)")
                  << " | DLC: " << static_cast<int>(dlc)
                  << " (" << static_cast<int>(data_length()) << " bytes)"
                  << " | BRS: " << (brs ? "ON" : "OFF")
                  << " | ESI: " << (esi ? "Passive" : "Active") << "\n";
        uint8_t len = data_length();
        if (len > 0) {
            std::cout << "    Data:";
            for (uint8_t i = 0; i < len; ++i) {
                if (i > 0 && i % 16 == 0) std::cout << "\n          ";
                std::cout << " " << std::hex << std::setw(2) << std::setfill('0')
                          << static_cast<int>(data[i]);
            }
            std::cout << std::dec << "\n";
        }
    }
};

// EN: CAN XL frame (up to 2048 bytes, priority-based)
// TR: CAN XL frame (2048 byte'a kadar, öncelik tabanlı)
struct CANXLFrame {
    uint8_t  priority    = 0;    // 0 = en yüksek öncelik (CAN ID yerine ayrı alan)
    uint32_t acceptance_field = 0; // Kabul filtresi alanı (CAN ID'nin yerini alır)
    uint8_t  sdu_type    = 0;    // Service Data Unit türü (0x05 = tünellenmiş Ethernet)
    uint16_t data_length = 0;    // Gerçek veri uzunluğu (1-2048 byte)
    std::vector<uint8_t> data;

    static constexpr uint8_t SDU_CAN_FD    = 0x03;
    static constexpr uint8_t SDU_ETHERNET  = 0x05;
    static constexpr uint8_t SDU_RAW       = 0x01;

```

```
    void print(const std::string& label) const {
        std::cout << "  [CAN XL] " << label << "\n";
        std::cout << "    Priority: " << static_cast<int>(priority)
                  << " | AF: 0x" << std::hex << acceptance_field << std::dec
                  << " | SDU Type: 0x" << std::hex
                  << static_cast<int>(sdu_type) << std::dec
                  << " | Length: " << data_length << " bytes\n";
        if (!data.empty()) {
            std::cout << "    Data (first 32):";
            size_t show = std::min(data.size(), size_t{32});
            for (size_t i = 0; i < show; ++i) {
                if (i > 0 && i % 16 == 0) std::cout << "\n                    ";
                std::cout << " " << std::hex << std::setw(2) << std::setfill('0')
                          << static_cast<int>(data[i]);
            }
            if (data.size() > 32) std::cout << " ... (" << std::dec << data.size() << " total)";
            std::cout << std::dec << "\n";
        }
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: CRC Calculation — CAN FD CRC-17 & CRC-21
// ═══════════════════════════════════════════════════════════════════════════

// EN: CAN FD uses CRC-17 (for ≤16 bytes) or CRC-21 (for >16 bytes).
//     Polynomial CRC-17: x^17 + x^16 + x^14 + x^13 + x^11 + x^6 + x^4 + x^3 + x^1 + 1
//                        = 0x3685B
//     Polynomial CRC-21: x^21 + x^20 + x^13 + x^11 + x^7 + x^4 + x^3 + x^1 + 1
//                        = 0x302899
// TR: CAN FD, ≤16 byte için CRC-17, >16 byte için CRC-21 kullanır.

uint32_t crc17_canfd(const uint8_t* data_bytes, size_t len) {
    // EN: Simplified bit-by-bit CRC-17 calculation for educational purposes
    // TR: Eğitim amaçlı basitleştirilmiş bit-bit CRC-17 hesaplama
    uint32_t crc = 0;
    constexpr uint32_t poly = 0x3685B;
    for (size_t i = 0; i < len; ++i) {
        crc ^= (static_cast<uint32_t>(data_bytes[i]) << 9); // align to bit 16
        for (int bit = 0; bit < 8; ++bit) {
            if (crc & 0x10000)  // bit 16 set?
                crc = (crc << 1) ^ poly;
            else
                crc <<= 1;
        }
    }
    return crc & 0x1FFFF;  // mask to 17 bits
}

uint32_t crc21_canfd(const uint8_t* data_bytes, size_t len) {
    uint32_t crc = 0;
    constexpr uint32_t poly = 0x302899;
    for (size_t i = 0; i < len; ++i) {
        crc ^= (static_cast<uint32_t>(data_bytes[i]) << 13); // align to bit 20
        for (int bit = 0; bit < 8; ++bit) {
            if (crc & 0x100000)  // bit 20 set?
                crc = (crc << 1) ^ poly;
            else
                crc <<= 1;
        }
    }
    return crc & 0x1FFFFF;  // mask to 21 bits
}

// EN: Auto-select CRC based on CAN FD data length
// TR: CAN FD veri uzunluğuna göre otomatik CRC seçimi
uint32_t canfd_crc(const CANFDFrame& frame) {
    uint8_t len = frame.data_length();
    if (len <= 16) {
        return crc17_canfd(frame.data.data(), len);
    } else {
        return crc21_canfd(frame.data.data(), len);
    }
}
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: CAN Arbitration Simulation
// ═══════════════════════════════════════════════════════════════════════════

// EN: CAN uses bitwise arbitration — lowest ID wins (dominant='0' wins over recessive='1').
//     This is non-destructive: losing nodes detect their recessive bit was overwritten.
// TR: CAN bit bazlı arbitrasyon kullanır — en düşük ID kazanır (dominant='0' baskındır).

struct ArbitrationEntry {
    uint32_t id;
    std::string ecu_name;
    std::string message_name;
};

// EN: Simulate bitwise arbitration for N ECUs competing simultaneously
// TR: Aynı anda yarışan N ECU için bit bazlı arbitrasyon simülasyonu
ArbitrationEntry simulate_arbitration(std::vector<ArbitrationEntry>& entries) {
    // EN: Sort by ID (lowest wins — dominant)
    // TR: ID'ye göre sırala (en düşük kazanır — dominant '0' baskındır)
    std::sort(entries.begin(), entries.end(),
              [](const ArbitrationEntry& a, const ArbitrationEntry& b) {
                  return a.id < b.id;
              });

    std::cout << "  Arbitration race (bit-by-bit):\n";
    for (const auto& e : entries) {
        std::cout << "    ECU [" << e.ecu_name << "] ID=0x"
                  << std::hex << std::setw(3) << std::setfill('0') << e.id
                  << std::dec << " (" << e.message_name << ")\n";
    }
    std::cout << "  Winner: " << entries[0].ecu_name
              << " (lowest ID 0x" << std::hex << entries[0].id << std::dec << ")\n";
    return entries[0];
}

// ═══════════════════════════════════════════════════════════════════════════
// PART 4: DBC Signal Extraction — CAN Database Parsing
// ═══════════════════════════════════════════════════════════════════════════

// EN: A DBC "signal" describes how to extract a physical value from CAN raw bytes.
//     Physical value = (raw_value * factor) + offset
// TR: DBC "sinyal" tanımı, CAN ham baytlarından fiziksel değerin nasıl çıkarılacağını belirtir.

struct DBCSignal {
    std::string name;
    uint8_t  start_bit   = 0;     // çerçevedeki bit konumu (LSB pozisyonu)
    uint8_t  bit_length  = 0;     // bit sayısı (sinyal genişliği)
    bool     big_endian  = false; // true=Motorola (big), false=Intel (little-endian)
    bool     is_signed   = false;
    double   factor      = 1.0;
    double   offset      = 0.0;
    double   min_val     = 0.0;
    double   max_val     = 0.0;
    std::string unit;

    // EN: Extract raw value from CAN data (Intel / little-endian byte order)
    // TR: CAN verisinden ham değeri çıkar (Intel / little-endian byte sırası)
    uint64_t extract_raw(const uint8_t* frame_data, uint8_t /*frame_len*/) const {
        // EN: For simplicity, implement Intel (little-endian) byte order extraction
        // TR: Basitlik için Intel (little-endian) byte sırası çıkarımı
        // Start bit is the LSB position in the data
        uint64_t raw = 0;
        for (uint8_t i = 0; i < bit_length; ++i) {
            uint8_t bit_pos = start_bit + i;
            uint8_t byte_idx = bit_pos / 8;
            uint8_t bit_in_byte = bit_pos % 8;
            if (frame_data[byte_idx] & (1u << bit_in_byte)) {
                raw |= (uint64_t{1} << i);
            }
        }
        return raw;
    }
```

```

    // EN: Convert raw to physical value
    // TR: Ham değeri fiziksel değere dönüştür
    double to_physical(uint64_t raw) const {
        if (is_signed) {
            // EN: Sign-extend if MSB is set
            // TR: MSB set ise işaret uzatması yap (negatif değerler için)
            if (raw & (uint64_t{1} << (bit_length - 1))) {
                // Set all upper bits to 1 for sign extension
                uint64_t mask = ~((uint64_t{1} << bit_length) - 1);
                raw |= mask;
            }
            return static_cast<double>(static_cast<int64_t>(raw)) * factor + offset;
        }
        return static_cast<double>(raw) * factor + offset;
    }

    // EN: Full extraction pipeline
    // TR: Tam çıkarım hattı (ham → fiziksel tek adımda)
    double decode(const uint8_t* frame_data, uint8_t frame_len) const {
        uint64_t raw = extract_raw(frame_data, frame_len);
        return to_physical(raw);
    }
};

// EN: CAN message definition (from DBC)
// TR: CAN mesaj tanımı (DBC'den)
struct DBCMessage {
    uint32_t id;
    std::string name;
    uint8_t dlc;
    std::string transmitter;
    std::vector<DBCSignal> signals;

    void decode_and_print(const uint8_t* frame_data, uint8_t frame_len) const {
        std::cout << "  Message: " << name << " (0x" << std::hex << id << std::dec
                  << ") from " << transmitter << "\n";
        for (const auto& sig : signals) {
            double phys = sig.decode(frame_data, frame_len);
            std::cout << "    " << sig.name << " = " << phys << " " << sig.unit;
            if (phys < sig.min_val || phys > sig.max_val) {
                std::cout << " [OUT OF RANGE!]";
            }
            std::cout << "\n";
        }
    }
};

// ============================================================================
// PART 5: CAN Filter — ID Mask Filtering
// ============================================================================

// EN: CAN controllers use hardware filters to reduce CPU load.
//     Filter passes when: (received_id & mask) == (filter_id & mask)
// TR: CAN denetleyicileri CPU yükünü azaltmak için donanım filtreleri kullanır.
//     Filtre geçiş koşulu: (alınan_id & mask) == (filtre_id & mask)

struct CANFilter {
    uint32_t filter_id;
    uint32_t mask;  // 1 = eşleşmeli, 0 = önemsiz (don't care)

    bool matches(uint32_t received_id) const {
        return (received_id & mask) == (filter_id & mask);
    }

    void print() const {
        std::cout << "    Filter: ID=0x" << std::hex << std::setw(3) << std::setfill('0')
                  << filter_id << " Mask=0x" << std::setw(3) << mask << std::dec << "\n";
    }
};

```

```cpp
// ═══════════════════════════════════════════════════════════════════════════
// PART 6: CAN Error Handling — Bus-Off Recovery
// ═══════════════════════════════════════════════════════════════════════════

// EN: CAN nodes maintain Transmit Error Counter (TEC) and Receive Error Counter (REC).
//     State machine: Error Active → Error Passive → Bus-Off
//     - Error Active: TEC < 128 AND REC < 128 (normal operation)
//     - Error Passive: TEC ≥ 128 OR REC ≥ 128 (restricted participation)
//     - Bus-Off: TEC ≥ 256 (node disconnects, must recover)
// TR: CAN düğümleri TEC (Gönderim Hata Sayacı) ve REC (Alım Hata Sayacı) tutar.
//     Durum makinesi: Error Active → Error Passive → Bus-Off
//     - Error Active: TEC < 128 VE REC < 128 (normal çalışma)
//     - Error Passive: TEC ≥ 128 VEYA REC ≥ 128 (kısıtlı katılım)
//     - Bus-Off: TEC ≥ 256 (düğüm bağlantıyı keser, kurtarma gerekir)

enum class CANErrorState {
    ErrorActive,   // Normal — aktif hata çerçeveleri gönderebilir
    ErrorPassive,  // Kısıtlı — sadece pasif hata çerçeveleri
    BusOff         // Bağlantı kesildi — bus-off kurtarma gerekli
};

std::string to_string(CANErrorState s) {
    switch (s) {
        case CANErrorState::ErrorActive:  return "Error Active";
        case CANErrorState::ErrorPassive: return "Error Passive";
        case CANErrorState::BusOff:       return "Bus-Off";
    }
    return "Unknown";
}

class CANErrorManager {
    uint16_t tec_ = 0;  // Transmit Error Counter (Gönderim Hata Sayacı)
    uint16_t rec_ = 0;  // Receive Error Counter (Alım Hata Sayacı)
    int busoff_recovery_count_ = 0;

public:
    CANErrorState state() const {
        if (tec_ >= 256) return CANErrorState::BusOff;
        if (tec_ >= 128 || rec_ >= 128) return CANErrorState::ErrorPassive;
        return CANErrorState::ErrorActive;
    }

    // EN: Called when a transmit error occurs (+8 per error, per ISO 11898)
    // TR: Gönderim hatasında çağrılır (ISO 11898: her hatada TEC += 8)
    void transmit_error() {
        if (state() != CANErrorState::BusOff)
            tec_ = std::min<uint16_t>(tec_ + 8, 256);
    }

    // EN: Called when a receive error occurs (+1 per error)
    // TR: Alım hatasında çağrılır (REC += 1)
    void receive_error() {
        rec_ = std::min<uint16_t>(rec_ + 1, 128);
    }

    // EN: Called on successful transmission (-1, min 0)
    // TR: Başarılı gönderimde TEC -= 1 (minimum 0)
    void transmit_success() {
        if (tec_ > 0) --tec_;
    }

    void receive_success() {
        if (rec_ > 0) --rec_;
    }
```

```cpp

    // EN: Bus-off recovery: 128 occurrences of 11 consecutive recessive bits
    // TR: Bus-off kurtarma: 128 kez 11 ardışık recessive bit algılanmalı
    bool attempt_busoff_recovery() {
        if (state() != CANErrorState::BusOff) return false;
        ++busoff_recovery_count_;
        if (busoff_recovery_count_ >= 128) {
            tec_ = 0;
            rec_ = 0;
            busoff_recovery_count_ = 0;
            return true;  // recovered
        }
        return false;
    }

    void print_status() const {
        std::cout << "    TEC=" << tec_ << " REC=" << rec_
                  << " State=" << to_string(state()) << "\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 7: Mini ECU Network Simulation — 4 ECU Car Network
// ═══════════════════════════════════════════════════════════════════════════

// EN: Simulates a simple car CAN network with 4 ECUs:
//     Engine ECU → publishes RPM, coolant temp, throttle position
//     ABS ECU → publishes wheel speeds
//     HVAC ECU → publishes cabin temperature, fan speed
//     Gateway ECU → receives and routes all messages
// TR: 4 ECU'lu basit bir araç CAN ağı simülasyonu:
//     Motor ECU → RPM, soğutma sıcaklığı, gaz kelebeği pozisyonu yayınlar
//     ABS ECU → tekerlek hızları yayınlar
//     HVAC ECU → kabin sıcaklığı, fan hızı yayınlar
//     Gateway ECU → tüm mesajları alır ve yönlendirir

struct ECU {
    std::string name;
    std::vector<CANFDFrame> tx_queue;   // gönderilecek mesajlar
    std::vector<CANFDFrame> rx_buffer;  // alınan mesajlar
    std::vector<CANFilter> filters;     // kabul filtreleri (donanım filtresi simülasyonu)

    void queue_tx(CANFDFrame frame) {
        tx_queue.push_back(std::move(frame));
    }

    bool accept(uint32_t msg_id) const {
        if (filters.empty()) return true;  // filtre yok = hepsini kabul et (gateway modu)
        for (const auto& f : filters) {
            if (f.matches(msg_id)) return true;
        }
        return false;
    }

    void receive(const CANFDFrame& frame) {
        if (accept(frame.id)) {
            rx_buffer.push_back(frame);
        }
    }
};

// EN: CAN bus — connect ECUs and handle transmission
// TR: CAN bus — ECU'ları bağla ve iletişimi yönet
class CANBus {
    std::vector<ECU*> nodes_;
    size_t total_frames_ = 0;

public:
    void connect(ECU& ecu) { nodes_.push_back(&ecu); }

```

```
461     // EN: Transmit all queued frames (arbitration: lowest ID first)
462     // TR: Kuyruklanmış tüm frameleri ilet (arbitrasyon: en düşük ID önce)
463     void transmit_cycle() {
464         // EN: Collect all pending frames with source info
465         // TR: Bekleyen tüm frameleri kaynak bilgisiyle topla
466         struct PendingFrame {
467             CANFDFrame frame;
468             ECU* source;
469         };
470         std::vector<PendingFrame> all_pending;
471
472         for (auto* node : nodes_) {
473             for (auto& f : node->tx_queue) {
474                 all_pending.push_back({f, node});
475             }
476             node->tx_queue.clear();
477         }
478
479         // EN: Arbitration — sort by ID (lowest wins)
480         // TR: Arbitrasyon — ID'ye göre sırala (en düşük kazanır — dominant baskınlığı)
481         std::sort(all_pending.begin(), all_pending.end(),
482                   [](const PendingFrame& a, const PendingFrame& b) {
483                       return a.frame.id < b.frame.id;
484                   });
485
486         // EN: Deliver each frame to all other nodes
487         // TR: Her frame'i gönderen hariç tüm düğümlere ilet (broadcast bus)
488         for (const auto& pf : all_pending) {
489             for (auto* node : nodes_) {
490                 if (node != pf.source) {
491                     node->receive(pf.frame);
492                 }
493             }
494             ++total_frames_;
495         }
496     }
497
498     size_t total() const { return total_frames_; }
499 };
500
501 // ============================================================================
502 // PART 8: SocketCAN Reference (Linux Only)
503 // ============================================================================
504
505 // EN: SocketCAN code reference (compiles only on Linux with linux/can.h)
506 //     Included as commented reference for real-world usage.
507 // TR: SocketCAN kod referansı (sadece Linux'ta linux/can.h ile derlenir)
508
509 /*
510  * ==========  SocketCAN Example (Linux Only) ==========
511  *
512  * #include <linux/can.h>
513  * #include <linux/can/raw.h>
514  * #include <net/if.h>
515  * #include <sys/ioctl.h>
516  * #include <sys/socket.h>
517  * #include <unistd.h>
518  *
519  * // --- Setup vcan0 (run as root before this program): ---
520  * // sudo modprobe vcan
521  * // sudo ip link add dev vcan0 type vcan
522  * // sudo ip link set up vcan0
523  *
524  * // --- CAN Socket (Classic CAN) ---
525  * int sock = socket(PF_CAN, SOCK_RAW, CAN_RAW);
526  * struct ifreq ifr;
527  * std::strcpy(ifr.ifr_name, "vcan0");
528  * ioctl(sock, SIOCGIFINDEX, &ifr);
529  *
530  * struct sockaddr_can addr{};
531  * addr.can_family = AF_CAN;
532  * addr.can_ifindex = ifr.ifr_ifindex;
533  * bind(sock, (struct sockaddr*)&addr, sizeof(addr));
534  *
```

```
 * // --- Send Classic CAN frame ---
 * struct can_frame frame{};
 * frame.can_id = 0x123;
 * frame.can_dlc = 8;
 * std::memset(frame.data, 0xAA, 8);
 * write(sock, &frame, sizeof(frame));
 *
 * // --- Send CAN FD frame ---
 * int enable_canfd = 1;
 * setsockopt(sock, SOL_CAN_RAW, CAN_RAW_FD_FRAMES,
 *            &enable_canfd, sizeof(enable_canfd));
 * struct canfd_frame fdframe{};
 * fdframe.can_id = 0x456;
 * fdframe.len = 24;
 * fdframe.flags = CANFD_BRS;  // Bit Rate Switch
 * write(sock, &fdframe, sizeof(fdframe));
 *
 * // --- CAN Filter (receive only 0x100-0x1FF) ---
 * struct can_filter rfilter;
 * rfilter.can_id   = 0x100;
 * rfilter.can_mask = 0x700;  // match upper 3 bits of 11-bit ID
 * setsockopt(sock, SOL_CAN_RAW, CAN_RAW_FILTER,
 *            &rfilter, sizeof(rfilter));
 *
 * // --- Receive ---
 * struct can_frame rx_frame;
 * read(sock, &rx_frame, sizeof(rx_frame));
 *
 * close(sock);
 */

// ═══════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 14 - CAN FD & CAN XL\n";
    std::cout << "============================================\n\n";

    // ─── Demo 1: Frame Structures ──────────────────────────────────
    {
        std::cout << "--- Demo 1: CAN Frame Structures ---\n";

        // EN: Classic CAN — Engine RPM message
        // TR: Klasik CAN — Motor devir mesajı (8 byte sınırı)
        ClassicCANFrame classic;
        classic.id = 0x0C0;
        classic.dlc = 8;
        classic.data = {0x0B, 0xB8, 0x50, 0x00, 0xFF, 0x7F, 0x00, 0x00};  // RPM=3000
        classic.print("Engine RPM (Classic CAN)");

        // EN: CAN FD — ADAS sensor fusion data (needs >8 bytes)
        // TR: CAN FD — ADAS sensör füzyon verisi (>8 byte gerektirir)
        CANFDFrame fd_frame;
        fd_frame.id = 0x200;
        fd_frame.dlc = 13; // DLC 13 → 32 bytes
        fd_frame.brs = true;
        fd_frame.esi = false;
        for (uint8_t i = 0; i < 32; ++i) fd_frame.data[i] = i;
        fd_frame.print("ADAS Sensor Fusion (CAN FD, 32 bytes)");

        // EN: CAN XL — Tunneled Ethernet frame
        // TR: CAN XL — Tünellenmiş Ethernet frame (SDU_ETHERNET=0x05)
        CANXLFrame xl_frame;
        xl_frame.priority = 1;
        xl_frame.acceptance_field = 0x00AA;
        xl_frame.sdu_type = CANXLFrame::SDU_ETHERNET;
        xl_frame.data_length = 128;
        xl_frame.data.resize(128);
        for (int i = 0; i < 128; ++i) xl_frame.data[static_cast<size_t>(i)] = static_cast<uint8_t>(i
& 0xFF);
        xl_frame.print("Tunneled Ethernet (CAN XL, 128 bytes)");
        std::cout << "\n";
    }
```

```

    // ─── Demo 2: CAN FD CRC Calculation ──────────────────────────────
    {
        std::cout << "--- Demo 2: CAN FD CRC Calculation ---\n";

        // EN: Short message → CRC-17
        // TR: Kısa mesaj (≤8 byte) → CRC-17 kullanılır
        CANFDFrame short_msg;
        short_msg.dlc = 8;  // 8 bytes
        short_msg.data = {};
        for (uint8_t i = 0; i < 8; ++i) short_msg.data[i] = static_cast<uint8_t>(0x10 + i);
        uint32_t crc_short = canfd_crc(short_msg);
        std::cout << "  Short msg (8 bytes) → CRC-17: 0x" << std::hex
                  << std::setw(5) << std::setfill('0') << crc_short << "\n";

        // EN: Long message → CRC-21
        // TR: Uzun mesaj (>16 byte) → CRC-21 kullanılır
        CANFDFrame long_msg;
        long_msg.dlc = 14;  // 48 bytes
        for (uint8_t i = 0; i < 48; ++i) long_msg.data[i] = static_cast<uint8_t>(i);
        uint32_t crc_long = canfd_crc(long_msg);
        std::cout << "  Long msg (48 bytes) → CRC-21: 0x" << std::hex
                  << std::setw(6) << std::setfill('0') << crc_long << "\n";

        // EN: Verify CRC changes with data change
        // TR: Veri değiştiğinde CRC'nin de değiştiğini doğrula (bütünlük kontrolü)
        long_msg.data[0] = 0xFF;  // corrupt one byte
        uint32_t crc_corrupt = canfd_crc(long_msg);
        std::cout << "  Corrupted msg       → CRC-21: 0x" << std::setw(6)
                  << std::setfill('0') << crc_corrupt;
        std::cout << (crc_long != crc_corrupt ? " (DIFFERENT ✓)" : " (SAME — BUG!)") << "\n";
        std::cout << std::dec << "\n";
    }

    // ─── Demo 3: CAN Arbitration ─────────────────────────────────────
    {
        std::cout << "--- Demo 3: CAN Arbitration Simulation ---\n";

        std::vector<ArbitrationEntry> race = {
            {0x200, "ADAS_ECU",    "RadarTarget"},
            {0x0C0, "Engine_ECU",  "EngineRPM"},
            {0x100, "ABS_ECU",     "WheelSpeed"},
            {0x300, "HVAC_ECU",    "CabinTemp"},
            {0x080, "Airbag_ECU",  "CrashSensor"}
        };

        auto winner = simulate_arbitration(race);
        std::cout << "  → " << winner.ecu_name << " transmits first ("
                  << winner.message_name << ")\n";
        std::cout << "  NOTE: Lower ID = Higher priority (safety-critical first)\n\n";
    }

    // ─── Demo 4: DBC Signal Extraction ───────────────────────────────
    {
        std::cout << "--- Demo 4: DBC Signal Extraction ---\n";
        std::cout << "  DBC definition (simulated):\n";
        std::cout << "    BO_ 0x0C0 EngineData: 8 Engine_ECU\n";
        std::cout << "      SG_ RPM  : 0|16@1+ (0.25,0) [0|16383.75] \"rpm\"\n";
        std::cout << "      SG_ Temp : 16|8@1+ (1,-40) [-40|215] \"degC\"\n";
        std::cout << "      SG_ Load : 24|8@1+ (0.392157,0) [0|100] \"%\"\n\n";

        // EN: Define the message with signals
        // TR: DBC sinyal tanımlarıyla mesajı oluştur
        DBCMessage engine_msg;
        engine_msg.id = 0x0C0;
        engine_msg.name = "EngineData";
        engine_msg.dlc = 8;
        engine_msg.transmitter = "Engine_ECU";

```

```
        // EN: RPM signal: start_bit=0, length=16, Intel byte order, unsigned
        // TR: RPM sinyali: başlat_bit=0, uzunluk=16, Intel byte sırası, işaretsiz
        //     physical = raw * 0.25 + 0
        DBCSignal rpm_sig;
        rpm_sig.name = "RPM";
        rpm_sig.start_bit = 0;
        rpm_sig.bit_length = 16;
        rpm_sig.factor = 0.25;
        rpm_sig.offset = 0.0;
        rpm_sig.min_val = 0.0;
        rpm_sig.max_val = 16383.75;
        rpm_sig.unit = "rpm";
        engine_msg.signals.push_back(rpm_sig);

        // EN: Temperature signal: start_bit=16, length=8, offset=-40
        // TR: Sıcaklık sinyali: başlat_bit=16, uzunluk=8, offset=-40
        //     physical = raw * 1 + (-40)
        DBCSignal temp_sig;
        temp_sig.name = "CoolantTemp";
        temp_sig.start_bit = 16;
        temp_sig.bit_length = 8;
        temp_sig.factor = 1.0;
        temp_sig.offset = -40.0;
        temp_sig.min_val = -40.0;
        temp_sig.max_val = 215.0;
        temp_sig.unit = "degC";
        engine_msg.signals.push_back(temp_sig);

        // EN: Engine Load: start_bit=24, length=8
        // TR: Motor yükü: başlat_bit=24, uzunluk=8
        DBCSignal load_sig;
        load_sig.name = "EngineLoad";
        load_sig.start_bit = 24;
        load_sig.bit_length = 8;
        load_sig.factor = 0.392157;
        load_sig.offset = 0.0;
        load_sig.min_val = 0.0;
        load_sig.max_val = 100.0;
        load_sig.unit = "%";
        engine_msg.signals.push_back(load_sig);

        // EN: Simulated CAN data: RPM=3000 → raw=12000 (0x2EE0), Temp=90°C → raw=130 (0x82), Load=75% → raw=191 (0xBF)
        // TR: Simüle CAN verisi: RPM=3000→ham=12000, Sıcak=90°C→ham=130, Yük=75%→ham=191
        uint8_t can_data[] = {0xE0, 0x2E, 0x82, 0xBF, 0x00, 0x00, 0x00, 0x00};
        engine_msg.decode_and_print(can_data, 8);
        std::cout << "\n";
    }

    // ── Demo 5: CAN Filter ──────────────────────────────────────────
    {
        std::cout << "--- Demo 5: CAN Filter Matching ---\n";

        // EN: Filter: accept only 0x0C0, 0x0C1, ..., 0x0CF (engine messages)
        // TR: Filtre: sadece 0x0C0-0x0CF kabul et (motor mesajları)
        CANFilter engine_filter{0x0C0, 0x7F0};  // mask upper 8 bits of 11-bit ID
        engine_filter.print();

        uint32_t test_ids[] = {0x0C0, 0x0C5, 0x0CF, 0x0D0, 0x100, 0x200};
        for (uint32_t tid : test_ids) {
            bool pass = engine_filter.matches(tid);
            std::cout << "    ID 0x" << std::hex << std::setw(3) << std::setfill('0')
                      << tid << std::dec << ": " << (pass ? "PASS ✓" : "REJECT ✗") << "\n";
        }
        std::cout << "\n";
    }
```

```cpp

    // ─── Demo 6: Error State Machine ──────────────────────────────────
    {
        std::cout << "--- Demo 6: CAN Error State Machine ---\n";

        CANErrorManager err_mgr;
        std::cout << "  Initial:";
        err_mgr.print_status();

        // EN: Simulate repeated transmit errors
        // TR: Tekrarlanan gönderim hatalarını simüle et (her hata TEC += 8)
        for (int i = 0; i < 16; ++i) err_mgr.transmit_error();  // 16 * 8 = 128
        std::cout << "  After 16 TX errors (TEC+=128):";
        err_mgr.print_status();

        for (int i = 0; i < 16; ++i) err_mgr.transmit_error();  // 128 more → 256
        std::cout << "  After 16 more TX errors (TEC=256):";
        err_mgr.print_status();

        // EN: Attempt bus-off recovery
        // TR: Bus-off kurtarma girişimi (128 recessive bit dizisi gerekli)
        std::cout << "  Attempting bus-off recovery (128 recessive bit sequences)...\n";
        bool recovered = false;
        for (int i = 0; i < 128 && !recovered; ++i) {
            recovered = err_mgr.attempt_busoff_recovery();
        }
        std::cout << "  After recovery:";
        err_mgr.print_status();
        std::cout << "  Recovered: " << (recovered ? "Yes ✓" : "No") << "\n\n";
    }

    // ─── Demo 7: 4-ECU Car Network Simulation ────────────────────────
    {
        std::cout << "--- Demo 7: 4-ECU Car Network Simulation ---\n";

        ECU engine_ecu{"Engine_ECU", {}, {}, {}};
        ECU abs_ecu{"ABS_ECU", {}, {}, {}};
        ECU hvac_ecu{"HVAC_ECU", {}, {}, {}};
        ECU gateway{"Gateway", {}, {}, {}};  // no filter → accepts all

        // EN: ABS only interested in engine and wheel speed
        // TR: ABS sadece motor ve tekerlek hızı mesajlarıyla ilgilenir
        abs_ecu.filters.push_back({0x0C0, 0x7F0});  // Engine 0x0Cx
        abs_ecu.filters.push_back({0x100, 0x7F0});  // Wheel  0x10x

        // EN: HVAC only interested in cabin temp request
        // TR: HVAC sadece kabin sıcaklık talebiyle ilgilenir (tam eşleşme)
        hvac_ecu.filters.push_back({0x300, 0x7FF});  // exact match

        CANBus bus;
        bus.connect(engine_ecu);
        bus.connect(abs_ecu);
        bus.connect(hvac_ecu);
        bus.connect(gateway);

        // EN: Engine ECU transmits RPM (3000 rpm) and coolant temp (90°C)
        // TR: Motor ECU devir (3000 rpm) ve soğutma sıvısı sıcaklığı (90°C) iletir
        {
            CANFDFrame rpm_msg;
            rpm_msg.id = 0x0C0;
            rpm_msg.dlc = 8;
            rpm_msg.data[0] = 0xE0; rpm_msg.data[1] = 0x2E;  // RPM raw=12000
            rpm_msg.data[2] = 0x82;  // Temp raw=130 → 90°C
            engine_ecu.queue_tx(rpm_msg);
        }

        // EN: ABS ECU transmits wheel speeds
        // TR: ABS ECU tekerlek hızlarını iletir (4 tekerlek × 2 byte)
        {
            CANFDFrame wheel_msg;
            wheel_msg.id = 0x100;
            wheel_msg.dlc = 8;
            wheel_msg.data[0] = 0x40; wheel_msg.data[1] = 0x01;  // FL speed
            wheel_msg.data[2] = 0x40; wheel_msg.data[3] = 0x01;  // FR speed
            wheel_msg.data[4] = 0x38; wheel_msg.data[5] = 0x01;  // RL speed
            wheel_msg.data[6] = 0x38; wheel_msg.data[7] = 0x01;  // RR speed
            abs_ecu.queue_tx(wheel_msg);
```

```
        }

        // EN: HVAC transmits cabin temperature
        // TR: HVAC kabin sıcaklığı ve fan hızı iletir
        {
            CANFDFrame cabin_msg;
            cabin_msg.id = 0x300;
            cabin_msg.dlc = 4;
            cabin_msg.data[0] = 0x16;  // 22°C
            cabin_msg.data[1] = 0x03;  // Fan speed level 3
            hvac_ecu.queue_tx(cabin_msg);
        }

        // EN: Run one bus cycle
        // TR: Bir bus döngüsü çalıştır (arbitrasyon + iletim + filtreleme)
        bus.transmit_cycle();

        std::cout << "  Results after 1 bus cycle (" << bus.total() << " frames transmitted):\n";
        std::cout << "    Engine_ECU rx: " << engine_ecu.rx_buffer.size() << " msgs\n";
        std::cout << "    ABS_ECU rx:    " << abs_ecu.rx_buffer.size() << " msgs (filtered)\n";
        std::cout << "    HVAC_ECU rx:   " << hvac_ecu.rx_buffer.size() << " msgs (filtered)\n";
        std::cout << "    Gateway rx:    " << gateway.rx_buffer.size() << " msgs (accepts all)\n";

        // EN: Verify gateway received all messages
        // TR: Gateway'in tüm mesajları aldığını doğrula (filtresiz = hepsini kabul)
        std::cout << "\n  Gateway received IDs:";
        for (const auto& f : gateway.rx_buffer) {
            std::cout << " 0x" << std::hex << f.id;
        }
        std::cout << std::dec << "\n";

        // EN: Verify ABS filter — should have engine (0x0C0) but NOT HVAC (0x300)
        // TR: ABS filtresi doğrulama — motor (0x0C0) var, HVAC (0x300) yok olmalı
        std::cout << "  ABS received IDs:";
        for (const auto& f : abs_ecu.rx_buffer) {
            std::cout << " 0x" << std::hex << f.id;
        }
        std::cout << std::dec << "\n";

        // EN: Verify HVAC filter — should NOT have engine or wheel speed
        // TR: HVAC filtresi doğrulama — motor veya tekerlek hızı olmaması gerek
        std::cout << "  HVAC received IDs:";
        for (const auto& f : hvac_ecu.rx_buffer) {
            std::cout << " 0x" << std::hex << f.id;
        }
        std::cout << std::dec << " (only gets 0x300 from others, but none sent it back)\n\n";
    }

    // ─── Demo 8: Comparison Table ──────────────────────────────────────
    {
        std::cout << "--- Demo 8: Classic CAN vs CAN FD vs CAN XL ---\n\n";
        std::cout << "  ┌──────────────────┬──────────────┬──────────────┬──────────────┐\n";
        std::cout << "  │ Feature          │ Classic CAN  │ CAN FD       │ CAN XL       │\n";
        std::cout << "  ├──────────────────┼──────────────┼──────────────┼──────────────┤\n";
        std::cout << "  │ Max Payload      │ 8 bytes      │ 64 bytes     │ 2048 bytes   │\n";
        std::cout << "  │ Max Bitrate      │ 1 Mbps       │ 8 Mbps       │ 20 Mbps      │\n";
        std::cout << "  │ Dual Bitrate     │ No           │ Yes (BRS)    │ Yes          │\n";
        std::cout << "  │ CRC              │ CRC-15       │ CRC-17/21    │ CRC-32       │\n";
        std::cout << "  │ ID Bits          │ 11/29        │ 11/29        │ Priority+AF  │\n";
        std::cout << "  │ Ethernet Bridge  │ No           │ No           │ Yes (SDU)    │\n";
        std::cout << "  │ Backward Compat  │ —            │ ← Classic    │ ← FD+Classic │\n";
        std::cout << "  │ Wiring           │ Twisted pair │ Twisted pair │ Twisted pair │\n";
        std::cout << "  │ Typical ECU      │ Body, BCM    │ ADAS, GW     │ Zone Ctrl    │\n";
        std::cout << "  │ Year Adopted     │ 1993         │ 2012/2015    │ 2024         │\n";
        std::cout << "  └──────────────────┴──────────────┴──────────────┴──────────────┘\n\n";
    }

    std::cout << "==========================================\n";
    std::cout << "   End of CAN FD & CAN XL\n";
    std::cout << "==========================================\n";

    return 0;
}
```

► 02. Lin Bus

module_14_automotive_networks/02_lin_bus.cpp

Konu: LIN Bus — Low-Cost Automotive Sub-Network — LIN Bus — Düşük Maliyetli Otomotiv Alt Ağı

Teori / Theory

THEORY: LIN — Local Interconnect Network

LIN (Local Interconnect Network) is a low-cost, single-wire serial bus used for non-safety-critical body electronics. Standardized by LIN Consortium (now part of ISO 17987).
Key characteristics:
- Single-wire + ground (very cheap wiring)
- Master-slave architecture: 1 master, up to 16 slaves
- Max speed: 20 kbps (typical: 9.6 or 19.2 kbps)
- Deterministic: master controls ALL communication via schedule table
- No arbitration needed (unlike CAN) — master decides who talks when
LIN Frame Structure:

```
┌────────┬─────────┬──────────────────────────────────────────┐
│ HEADER │         │  RESPONSE                                │
│(Master)│         │  (Slave or Master)                       │
├────────┴─────────┴──────────┬──────────────┬─────────────────┤
│ Break  │ Sync    │ PID       │ Data         │ Checksum        │
│(≥13bit)│ (0x55)  │ (6bit ID  │ (1-8 bytes)  │ (Classic/       │
│        │         │  +2 parity)│              │  Enhanced)      │
└────────┴─────────┴──────────┴──────────────┴─────────────────┘
```

PID (Protected Identifier):
- 6-bit frame ID (0x00-0x3F) = 64 possible frames
- 2 parity bits: P0 = ID0⊕ID1⊕ID2⊕ID4, P1 = ¬(ID1⊕ID3⊕ID4⊕ID5)
- ID 0x3C-0x3D: Diagnostic frames (master request / slave response)
- ID 0x3E-0x3F: Reserved
Frame Types:

```
┌──────────────────┬──────────────────────────────────────┐
│ Type             │ Description                          │
├──────────────────┼──────────────────────────────────────┤
│ Unconditional    │ Fixed schedule, slave always responds │
│ Event-triggered  │ Multiple slaves, respond only if data changed│
│ Sporadic         │ Master sends own data when slot available  │
│ Diagnostic       │ ID=0x3C (MasterReq), ID=0x3D (SlaveResp)   │
└──────────────────┴──────────────────────────────────────┘
```

TEORİ: LIN — Yerel Bağlantı Ağı

LIN, güvenlik-kritik olmayan gövde elektroniği için kullanılan düşük maliyetli, tek telli seri bus'tır (ISO 17987). Master-slave mimarisinde master tüm iletişimi schedule table ile yönetir.
Temel özellikler:
- Tek tel + toprak (oldukça ucuz kablolama)
- Master-slave mimari: 1 master, 16'ya kadar slave
- Maks hız: 20 kbps (tipik: 9.6 veya 19.2 kbps)
- Belirlenimci: master schedule table ile tüm iletişimi kontrol eder
- Arbitrasyon yok (CAN'dan farklı) — master kimin ne zaman konuşacağına karar verir
LIN Çerçeve Yapısı:

```
┌────────┬──────────┬──────────────────────────────────────┐
│ BAŞLIK │          │  YANIT                               │
│(Master)│          │  (Slave veya Master)                 │
├────────┼──────────┼────────────┬──────────────┬──────────┤
│ Break  │ Sync     │ PID        │ Veri         │ Checksum │
│(≥13bit)│ (0x55)   │ (6bit ID   │ (1-8 bayt)   │ (Classic/ │
│        │          │  +2 parite)│              │  Enhanced) │
└────────┴──────────┴────────────┴──────────────┴──────────┘
```

PID (Korumalı Tanımlayıcı):
- 6-bit frame ID (0x00-0x3F) = 64 olası çerçeve
- 2 parite biti: P0 = ID0⊕ID1⊕ID2⊕ID4, P1 = ¬(ID1⊕ID3⊕ID4⊕ID5)
- ID 0x3C-0x3D: Teşhis çerçeveleri (master isteği / slave yanıtı)
- ID 0x3E-0x3F: Ayrılmış (reserved)
Çerçeve Türleri:
- Unconditional: Sabit zamanlama, slave her zaman yanıt verir
- Event-triggered: Birden fazla slave, sadece veri değişirse yanıt
- Sporadic: Master, slot müsaitse kendi verisini gönderir
- Diagnostic: ID=0x3C (MasterReq), ID=0x3D (SlaveResp)

THEORY: LIN Checksum — Classic vs Enhanced

Two checksum types exist:
- Classic (LIN 1.x): checksum over DATA bytes only
- Enhanced (LIN 2.x): checksum over PID + DATA bytes Both use inverted modulo-256 sum with carry propagation.

TEORİ: LIN Sağlama Toplamı — Klasik ve Gelişmiş

İki checksum türü vardır:
- Classic (LIN 1.x): sadece DATA baytları üzerinden checksum
- Enhanced (LIN 2.x): PID + DATA baytları üzerinden checksum Her ikisi de elde-yayılmalı ters modülü-256 toplamı kullanır.

THEORY: LIN vs CAN — When to Use Which

```
┌──────────────────┬──────────────┬──────────────────────────┐
│ Feature          │ LIN          │ CAN                      │
├──────────────────┼──────────────┼──────────────────────────┤
│ Speed            │ 20 kbps      │ 1 Mbps (Classic), 8M (FD)  │
│ Wiring           │ Single wire  │ Twisted pair (2 wires)     │
│ Cost per node    │ ~$0.5        │ ~$2-5                      │
│ Topology         │ Master-slave │ Multi-master (peer-to-peer) │
│ Arbitration      │ None (sched) │ Bitwise ID-based           │
│ Error detection  │ Checksum     │ CRC, ACK, error frame      │
│ Max nodes        │ 1+16         │ Theoretically unlimited    │
│ Deterministic    │ Yes (table)  │ Not guaranteed (priority)  │
│ Typical use      │ Window, seat │ Engine, ABS, ADAS          │
│ Safety level     │ QM, ASIL-A   │ Up to ASIL-D               │
│ Wake-up          │ Bus wakeup   │ Bus wakeup                 │
└──────────────────┴──────────────┴──────────────────────────┘
```

TEORİ: LIN ve CAN — Hangisi Ne Zaman Kullanılır

LIN vs CAN karşılaştırması:
- Hız: LIN 20 kbps, CAN 1-8 Mbps — LIN düşük hız yeterli olduğunda
- Maliyet: LIN ~$0.5/düğüm, CAN ~$2-5 — LIN kablolama maliyetini düşürür
- Topoloji: LIN master-slave, CAN multi-master — LIN basit kontrol için
- Arbitrasyon: LIN yok (schedule), CAN bit-bazlı ID — LIN çakışmayı önler
- Güvenlik: LIN QM/ASIL-A, CAN ASIL-D'ye kadar — LIN güvenlik-kritik değil
- Tipik kullanım: LIN cam/koltuk/ayna, CAN motor/ABS/ADAS
- Uyandırma: Her ikisi de bus wakeup destekler

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <array>
#include <vector>
#include <string>
#include <functional>
#include <iomanip>
#include <cassert>
#include <map>

// ═══════════════════════════════════════════════════════════════════════
// PART 1: LIN Frame Structures
// ═══════════════════════════════════════════════════════════════════════

// EN: Frame type classification
// TR: Frame tipi sınıflandırması
enum class LINFrameType {
    Unconditional,   // Sabit zamanlama, slave her zaman yanıt verir
    EventTriggered,  // Sadece veri değiştiğinde yanıt (bus yükü azalması)
    Sporadic,        // Master slot müsaitse kendi verisini gönderir
    DiagRequest,     // Master teşhis isteği (ID=0x3C, her zaman 8 bayt)
    DiagResponse     // Slave teşhis yanıtı (ID=0x3D, her zaman 8 bayt)
};

std::string to_string(LINFrameType t) {
    switch (t) {
        case LINFrameType::Unconditional:  return "Unconditional";
        case LINFrameType::EventTriggered: return "Event-Triggered";
        case LINFrameType::Sporadic:       return "Sporadic";
        case LINFrameType::DiagRequest:    return "Diagnostic Request";
        case LINFrameType::DiagResponse:   return "Diagnostic Response";
    }
    return "Unknown";
}
```

```cpp

// EN: Protected Identifier (PID) calculation
// TR: Korumalı Tanımlayıcı (PID) hesaplaması
class LINPID {
public:
    // EN: Calculate PID from 6-bit frame ID (0x00-0x3F)
    // TR: 6-bit frame ID'den PID hesapla (parite bitleri P0, P1 eklenir)
    //     PID = ID[5:0] + P0 + P1
    //     P0 = ID0 ⊕ ID1 ⊕ ID2 ⊕ ID4
    //     P1 = ¬(ID1 ⊕ ID3 ⊕ ID4 ⊕ ID5)
    static uint8_t calculate(uint8_t id) {
        id &= 0x3F;  // mask to 6 bits

        uint8_t p0 = ((id >> 0) ^ (id >> 1) ^ (id >> 2) ^ (id >> 4)) & 1u;
        uint8_t p1 = static_cast<uint8_t>(~((id >> 1) ^ (id >> 3) ^ (id >> 4) ^ (id >> 5)) & 1u);

        return static_cast<uint8_t>(id | (p0 << 6) | (p1 << 7));
    }

    // EN: Extract frame ID from PID
    // TR: PID'den frame ID'yi çıkar (alt 6 bit maskesi)
    static uint8_t extract_id(uint8_t pid) {
        return pid & 0x3F;
    }

    // EN: Verify PID parity
    // TR: PID parite doğrulaması (ıletim hatası tespiti için)
    static bool verify(uint8_t pid) {
        uint8_t id = extract_id(pid);
        return calculate(id) == pid;
    }
};

// EN: LIN Checksum calculation
// TR: LIN Checksum hesaplaması
class LINChecksum {
public:
    // EN: Classic checksum (LIN 1.x): sum over DATA only
    // TR: Klasik checksum (LIN 1.x): sadece DATA baytları üzerine toplam
    static uint8_t classic(const uint8_t* data, uint8_t len) {
        uint16_t sum = 0;
        for (uint8_t i = 0; i < len; ++i) {
            sum += data[i];
            if (sum > 0xFF) sum = (sum & 0xFF) + 1;  // elde yayılması (carry propagation)
        }
        return static_cast<uint8_t>(~sum & 0xFF);
    }

    // EN: Enhanced checksum (LIN 2.x): sum over PID + DATA
    // TR: Gelişmiş checksum (LIN 2.x): PID + DATA baytları üzerine toplam
    static uint8_t enhanced(uint8_t pid, const uint8_t* data, uint8_t len) {
        uint16_t sum = pid;
        for (uint8_t i = 0; i < len; ++i) {
            sum += data[i];
            if (sum > 0xFF) sum = (sum & 0xFF) + 1;
        }
        return static_cast<uint8_t>(~sum & 0xFF);
    }
};

// EN: Complete LIN frame
// TR: Tam LIN frame yapısı
struct LINFrame {
    uint8_t      id      = 0;   // 6-bit frame ID (0x00-0x3F), slot numarasını belirler
    uint8_t      pid     = 0;   // Korumalı ID (ID + P0/P1 parite bitleri)
    uint8_t      dlc     = 0;   // Veri uzunluğu (1-8 bayt)
    std::array<uint8_t, 8> data{};
    uint8_t      checksum = 0;
    bool         enhanced_checksum = true;  // LIN 2.x varsayılan (PID dahil)
    LINFrameType type = LINFrameType::Unconditional;
    std::string  name;

    void compute() {
        pid = LINPID::calculate(id);
```

```
        if (enhanced_checksum) {
            checksum = LINChecksum::enhanced(pid, data.data(), dlc);
        } else {
            checksum = LINChecksum::classic(data.data(), dlc);
        }
    }

    bool verify() const {
        if (!LINPID::verify(pid)) return false;
        uint8_t expected;
        if (enhanced_checksum) {
            expected = LINChecksum::enhanced(pid, data.data(), dlc);
        } else {
            expected = LINChecksum::classic(data.data(), dlc);
        }
        return checksum == expected;
    }

    void print() const {
        std::cout << "    [LIN] " << name << " (ID=0x"
                  << std::hex << std::setw(2) << std::setfill('0')
                  << static_cast<int>(id)
                  << " PID=0x" << static_cast<int>(pid)
                  << ") Type=" << to_string(type)
                  << " DLC=" << std::dec << static_cast<int>(dlc) << "\n";
        std::cout << "      Data:";
        for (uint8_t i = 0; i < dlc; ++i) {
            std::cout << " " << std::hex << std::setw(2) << std::setfill('0')
                      << static_cast<int>(data[i]);
        }
        std::cout << std::dec << " | Chk=0x" << std::hex
                  << static_cast<int>(checksum)
                  << (enhanced_checksum ? " (enhanced)" : " (classic)")
                  << std::dec << " | Valid=" << (verify() ? "YES" : "NO") << "\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: LIN Slave Node Simulation
// ═══════════════════════════════════════════════════════════════════════════

// EN: A LIN slave node responds to specific frame IDs from the master.
//     When the master sends a header with a matching ID, the slave fills the response.
// TR: LIN slave düğümü, master'dan gelen belirli frame ID'lerine yanıt verir.

class LINSlaveNode {
    std::string name_;
    // EN: Map of frame ID → response callback
    // TR: Frame ID → yanıt callback haritası (slave hangi ID'ye yanıt veriyor)
    std::map<uint8_t, std::function<LINFrame()>> response_handlers_;
    bool awake_ = true;

public:
    explicit LINSlaveNode(std::string name) : name_(std::move(name)) {}

    const std::string& name() const { return name_; }

    // EN: Register a response handler for a frame ID
    // TR: Bir frame ID için yanıt işleyicisi kaydet
    void on_frame(uint8_t frame_id, std::function<LINFrame()> handler) {
        response_handlers_[frame_id] = std::move(handler);
    }

    // EN: Master sends header — slave responds if it owns this frame
    // TR: Master header gönderir — bu frame'in sahibi slave ise yanıt verir
    bool responds_to(uint8_t frame_id) const {
        return response_handlers_.count(frame_id) > 0;
    }
```

```cpp

    LINFrame respond(uint8_t frame_id) {
        auto it = response_handlers_.find(frame_id);
        if (it != response_handlers_.end()) {
            LINFrame frame = it->second();
            frame.id = frame_id;
            frame.compute();
            return frame;
        }
        return {};
    }

    // EN: Sleep/wakeup mechanism
    // TR: Uyku/uyanma mekanizması (LIN bus güç tasarrufu için uyutulabilir)
    void sleep() { awake_ = false; }
    void wakeup() { awake_ = true; }
    bool is_awake() const { return awake_; }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: LIN Schedule Table
// ═══════════════════════════════════════════════════════════════════════════

// EN: The schedule table defines WHEN each frame is transmitted.
//     The master cycles through the table, sending headers at fixed intervals.
//     This makes LIN fully deterministic — no collisions possible.
// TR: Schedule table her frame'in NE ZAMAN iletileceğini tanımlar.
//     Master, tabloyu döngüsel olarak takip eder.

struct ScheduleEntry {
    uint8_t frame_id;
    uint16_t delay_ms;    // delay after this frame before next
    LINFrameType type;
    std::string description;
};

class LINScheduleTable {
    std::string name_;
    std::vector<ScheduleEntry> entries_;
    size_t current_index_ = 0;

public:
    explicit LINScheduleTable(std::string name) : name_(std::move(name)) {}

    void add(uint8_t frame_id, uint16_t delay_ms, LINFrameType type,
             const std::string& desc) {
        entries_.push_back({frame_id, delay_ms, type, desc});
    }

    const ScheduleEntry& next() {
        const auto& entry = entries_[current_index_];
        current_index_ = (current_index_ + 1) % entries_.size();
        return entry;
    }

    size_t size() const { return entries_.size(); }
    const std::string& name() const { return name_; }

    void print() const {
        std::cout << "  Schedule Table: " << name_ << " (" << entries_.size() << " slots)\n";
        std::cout << "  ┌──────┬──────────┬──────────────┬──────────────────────────┐\n";
        std::cout << "  │ Slot │ Frame ID │ Delay (ms)   │ Description              │\n";
        std::cout << "  ├──────┼──────────┼──────────────┼──────────────────────────┤\n";
        for (size_t i = 0; i < entries_.size(); ++i) {
            const auto& e = entries_[i];
            std::cout << "  │ " << std::dec << std::setfill(' ') << std::setw(4) << i
                      << " │ 0x" << std::hex << std::setfill('0') << std::setw(2)
                      << static_cast<int>(e.frame_id) << std::dec << std::setfill(' ')
                      << "     │ " << std::setw(16) << e.delay_ms
                      << " │ " << std::left << std::setw(25) << e.description
                      << std::right << " │\n";
        }
        std::cout << "  └──────┴──────────┴──────────────┴──────────────────────────┘\n";
    }
};
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════
// PART 4: LIN Master Node
// ═══════════════════════════════════════════════════════════════════════════

// EN: The master node controls all communication:
//     1) Follows the schedule table
//     2) Sends break + sync + PID header
//     3) Waits for the designated slave to respond with data + checksum
// TR: Master tüm iletişimi kontrol eder: schedule table'ı takip eder,
//     header gönderir, slave'in yanıt vermesini bekler.

class LINMaster {
    std::string name_;
    std::vector<LINSlaveNode*> slaves_;
    LINScheduleTable* active_schedule_ = nullptr;
    std::vector<LINFrame> message_log_;  // all received responses

public:
    explicit LINMaster(std::string name) : name_(std::move(name)) {}

    void add_slave(LINSlaveNode& slave) { slaves_.push_back(&slave); }
    void set_schedule(LINScheduleTable& table) { active_schedule_ = &table; }

    // EN: Execute one schedule cycle (all entries once)
    // TR: Bir schedule döngüsü çalıştır (tüm girişler sırayla)
    void execute_cycle() {
        if (!active_schedule_) return;

        size_t num_slots = active_schedule_->size();
        uint32_t total_time_ms = 0;

        std::cout << "  Master [" << name_ << "] executing schedule '"
                  << active_schedule_->name() << "' (" << num_slots << " slots)\n";

        for (size_t slot = 0; slot < num_slots; ++slot) {
            const auto& entry = active_schedule_->next();
            total_time_ms += entry.delay_ms;

            // EN: Master sends header (break + sync + PID)
            // TR: Master header gönderir (break + sync + PID)
            uint8_t pid = LINPID::calculate(entry.frame_id);

            // EN: Find responding slave
            // TR: Yanıt veren slave'i bul
            bool responded = false;
            for (auto* slave : slaves_) {
                if (slave->is_awake() && slave->responds_to(entry.frame_id)) {
                    LINFrame response = slave->respond(entry.frame_id);
                    response.name = entry.description;
                    response.type = entry.type;
                    message_log_.push_back(response);

                    std::cout << "    t=" << std::setw(5) << total_time_ms << "ms | "
                              << "Header PID=0x" << std::hex << std::setw(2) << std::setfill('0')
                              << static_cast<int>(pid) << std::dec << std::setfill(' ')
                              << " → " << slave->name() << " responds: ";

                    // EN: Print decoded data summary
                    // TR: Çözümlü veri özetini yazdır
                    std::cout << "[";
                    for (uint8_t i = 0; i < response.dlc; ++i) {
                        if (i > 0) std::cout << " ";
                        std::cout << std::hex << std::setw(2) << std::setfill('0')
                                  << static_cast<int>(response.data[i]);
                    }
                    std::cout << std::dec << std::setfill(' ') << "] ✓\n";
                    responded = true;
                    break;
                }
            }
```

```
            if (!responded) {
                std::cout << "    t=" << std::setw(5) << total_time_ms << "ms | "
                          << "Header PID=0x" << std::hex << std::setw(2) << std::setfill('0')
                          << static_cast<int>(pid) << std::dec << std::setfill(' ')
                          << " → No response\n";
            }
        }
        std::cout << "  Cycle complete: " << message_log_.size() << " responses, "
                  << total_time_ms << "ms total\n";
    }

    const std::vector<LINFrame>& log() const { return message_log_; }
    void clear_log() { message_log_.clear(); }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 5: LIN Sleep/Wakeup
// ═══════════════════════════════════════════════════════════════════════════

// EN: LIN supports sleep mode for power saving:
//     - Master sends go-to-sleep command: Diagnostic frame ID=0x3C, data[0]=0x00
//     - Slave wakes up by pulling bus dominant for 250µs-5ms
//     - After wakeup, master resumes schedule table execution
// TR: LIN güç tasarrufu için uyku modunu destekler:
//     - Master uyku komutu gönderir: ID=0x3C, data[0]=0x00
//     - Slave, bus'ı dominant çekerek uyanır

struct LINBusState {
    bool bus_sleeping = false;
    uint32_t sleep_count = 0;
    uint32_t wakeup_count = 0;

    void go_to_sleep() {
        bus_sleeping = true;
        ++sleep_count;
    }

    void wakeup() {
        bus_sleeping = false;
        ++wakeup_count;
    }

    void print() const {
        std::cout << "    Bus state: " << (bus_sleeping ? "SLEEPING" : "ACTIVE")
                  << " | Sleep count: " << sleep_count
                  << " | Wakeup count: " << wakeup_count << "\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 14 - LIN Bus\n";
    std::cout << "============================================\n\n";

    // ─── Demo 1: PID Calculation & Verification ──────────────────────
    {
        std::cout << "--- Demo 1: LIN PID Calculation ---\n";

        std::cout << "  ┌──────────┬──────────┬──────────┬──────────┐\n";
        std::cout << "  │ Frame ID │ PID (hex)│ P0 P1    │ Verify   │\n";
        std::cout << "  ├──────────┼──────────┼──────────┼──────────┤\n";

        uint8_t test_ids[] = {0x00, 0x01, 0x10, 0x20, 0x3C, 0x3D, 0x3F};
```

```cpp
        for (uint8_t fid : test_ids) {
            uint8_t pid = LINPID::calculate(fid);
            uint8_t p0 = (pid >> 6) & 1;
            uint8_t p1 = (pid >> 7) & 1;
            bool valid = LINPID::verify(pid);
            std::cout << "  │ 0x" << std::hex << std::setw(2) << std::setfill('0')
                      << static_cast<int>(fid)
                      << "     │ 0x" << std::setw(2) << static_cast<int>(pid)
                      << "     │ P0=" << static_cast<int>(p0) << " P1=" << static_cast<int>(p1)
                      << " │ " << std::dec << (valid ? "VALID ✓   " : "INVALID ✗ ")
                      << " │\n";
        }
        std::cout << "  └──────────┴──────────┴──────────┴──────────┘\n\n";
    }

    // ─── Demo 2: Checksum Calculation ──────────────────────────────────
    {
        std::cout << "--- Demo 2: LIN Checksum (Classic vs Enhanced) ---\n";

        uint8_t data[] = {0x40, 0x01, 0x20, 0xFF, 0x00, 0x00, 0x00, 0x00};
        uint8_t pid = LINPID::calculate(0x10);

        uint8_t chk_classic  = LINChecksum::classic(data, 4);
        uint8_t chk_enhanced = LINChecksum::enhanced(pid, data, 4);

        std::cout << "  Data: [40 01 20 FF], Frame ID=0x10, PID=0x"
                  << std::hex << static_cast<int>(pid) << "\n";
        std::cout << "  Classic checksum  (data only):     0x"
                  << std::setw(2) << std::setfill('0') << static_cast<int>(chk_classic) << "\n";
        std::cout << "  Enhanced checksum (PID + data):    0x"
                  << std::setw(2) << static_cast<int>(chk_enhanced) << "\n";
        std::cout << std::dec;
        std::cout << "  Classic ≠ Enhanced: " << (chk_classic != chk_enhanced ? "Yes ✓" : "Bug!")
                  << " (PID changes the sum)\n\n";
    }

    // ─── Demo 3: Complete LIN Frame ────────────────────────────────────
    {
        std::cout << "--- Demo 3: Complete LIN Frame ---\n";

        // EN: Window position frame from door module slave
        // TR: Kapı modülü slave'inden cam pozisyonu frame'i
        LINFrame window_frame;
        window_frame.id = 0x10;
        window_frame.dlc = 2;
        window_frame.data[0] = 0x50;  // window position: 80%
        window_frame.data[1] = 0x00;  // status: no error
        window_frame.enhanced_checksum = true;
        window_frame.type = LINFrameType::Unconditional;
        window_frame.name = "Window Position";
        window_frame.compute();
        window_frame.print();

        // EN: Corrupt the checksum and verify
        // TR: Checksum'u boz ve doğrulama hatası göster
        LINFrame corrupt = window_frame;
        corrupt.checksum = 0xFF;
        corrupt.name = "Corrupted Frame";
        corrupt.print();
        std::cout << "\n";
    }

    // ─── Demo 4: Schedule Table ────────────────────────────────────────
    {
        std::cout << "--- Demo 4: LIN Schedule Table ---\n";

```

```
        // EN: BCM (Body Control Module) schedule for door/window/seat control
        // TR: BCM schedule tablosu — kapı/cam/koltuk kontrol döngüsü
        LINScheduleTable bcm_schedule("BCM_Main_Schedule");

        bcm_schedule.add(0x10, 10, LINFrameType::Unconditional, "WindowPos_FL");
        bcm_schedule.add(0x11, 10, LINFrameType::Unconditional, "WindowPos_FR");
        bcm_schedule.add(0x12, 10, LINFrameType::Unconditional, "MirrorPos_L");
        bcm_schedule.add(0x13, 10, LINFrameType::Unconditional, "MirrorPos_R");
        bcm_schedule.add(0x20, 15, LINFrameType::EventTriggered, "SeatPosition");
        bcm_schedule.add(0x30, 20, LINFrameType::Sporadic, "InteriorLight");
        bcm_schedule.add(0x3C, 10, LINFrameType::DiagRequest, "DiagRequest");
        bcm_schedule.add(0x3D, 10, LINFrameType::DiagResponse, "DiagResponse");

        bcm_schedule.print();
        std::cout << "\n";
    }

    // ─── Demo 5: Master-Slave Communication Simulation ──────────────────
    {
        std::cout << "--- Demo 5: Master-Slave LIN Network ---\n";

        // EN: Create master (BCM) and 3 slaves (door module, mirror, seat)
        // TR: Master (BCM) ve 3 slave (kapı, ayna, koltuk) oluştur
        LINMaster bcm("BCM_Master");

        LINSlaveNode door_fl("Door_FL");
        LINSlaveNode mirror_l("Mirror_L");
        LINSlaveNode seat_driver("Seat_Driver");

        // EN: Door slave responds to frame 0x10 with window position
        // TR: Kapı slave'i frame 0x10'a cam pozisyonuyla yanıt verir
        door_fl.on_frame(0x10, []() {
            LINFrame f;
            f.dlc = 3;
            f.data[0] = 0x50;  // window: 80%
            f.data[1] = 0x00;  // motor: stopped
            f.data[2] = 0x01;  // obstacle detected: no
            return f;
        });

        // EN: Door slave also responds to frame 0x11 (lock status)
        // TR: Kapı slave'i frame 0x11'e kilit durumu ile yanıt verir
        door_fl.on_frame(0x11, []() {
            LINFrame f;
            f.dlc = 2;
            f.data[0] = 0x01;  // locked: yes
            f.data[1] = 0x00;  // child lock: off
            return f;
        });

        // EN: Mirror slave responds to frame 0x12
        // TR: Ayna slave'i frame 0x12'ye pozisyon ve ısıtıcı bilgisi ile yanıt verir
        mirror_l.on_frame(0x12, []() {
            LINFrame f;
            f.dlc = 4;
            f.data[0] = 0x32;  // horizontal position: 50
            f.data[1] = 0x28;  // vertical position: 40
            f.data[2] = 0x00;  // fold state: unfolded
            f.data[3] = 0x01;  // heater: on
            return f;
        });

        // EN: Seat slave responds to frame 0x20
        // TR: Koltuk slave'i frame 0x20'ye 6 byte pozisyon verisiyle yanıt verir
        seat_driver.on_frame(0x20, []() {
            LINFrame f;
            f.dlc = 6;
            f.data[0] = 0x64;  // fore-aft: 100mm
            f.data[1] = 0x50;  // height: 80mm
            f.data[2] = 0x28;  // recline: 40 degrees
            f.data[3] = 0x3C;  // lumbar: 60%
            f.data[4] = 0x01;  // heater: level 1
            f.data[5] = 0x00;  // ventilation: off
            return f;
        });

```

```
        bcm.add_slave(door_fl);
        bcm.add_slave(mirror_l);
        bcm.add_slave(seat_driver);

        // EN: Create schedule
        // TR: Schedule tablosu oluştur (sabit zamanlamalı döngü)
        LINScheduleTable schedule("BCM_Schedule");
        schedule.add(0x10, 10, LINFrameType::Unconditional, "WindowPos_FL");
        schedule.add(0x11, 10, LINFrameType::Unconditional, "DoorLock_FL");
        schedule.add(0x12, 10, LINFrameType::Unconditional, "MirrorPos_L");
        schedule.add(0x20, 15, LINFrameType::EventTriggered, "SeatPos_Driver");
        schedule.add(0x25, 10, LINFrameType::Unconditional, "AmbientLight");

        bcm.set_schedule(schedule);
        bcm.execute_cycle();

        // EN: Verify responses
        // TR: Yanıtları doğrula
        std::cout << "\n  Detailed responses:\n";
        for (const auto& frame : bcm.log()) {
            frame.print();
        }
        std::cout << "\n";
    }

    // ─── Demo 6: Sleep/Wakeup ──────────────────────────────────────────
    {
        std::cout << "--- Demo 6: LIN Sleep/Wakeup ---\n";

        LINBusState bus_state;
        bus_state.print();

        // EN: Master sends go-to-sleep command
        // TR: Master uyku komutu gönderir (ID=0x3C, data[0]=0x00)
        std::cout << "  Master sends go-to-sleep (ID=0x3C, data[0]=0x00)\n";
        bus_state.go_to_sleep();

        LINSlaveNode test_slave("TestSlave");
        test_slave.sleep();
        std::cout << "  Slave state: " << (test_slave.is_awake() ? "AWAKE" : "SLEEPING") << "\n";
        bus_state.print();

        // EN: Wakeup signal from slave
        // TR: Slave'den uyanma sinyali (dominant darbe 250µs-5ms)
        std::cout << "  Slave sends wakeup signal (dominant pulse 250us-5ms)\n";
        test_slave.wakeup();
        bus_state.wakeup();
        std::cout << "  Slave state: " << (test_slave.is_awake() ? "AWAKE" : "SLEEPING") << "\n";
        bus_state.print();
        std::cout << "\n";
    }

    // ─── Demo 7: Diagnostic Frames ─────────────────────────────────────
    {
        std::cout << "--- Demo 7: LIN Diagnostic Frames ---\n";
        std::cout << "  LIN diagnostic uses ID=0x3C (MasterRequest) and ID=0x3D (SlaveResponse)\n";
        std::cout << "  Both are always 8 bytes.\n\n";

        // EN: Master diagnostic request: Read Supplier ID
        // TR: Master teşhis isteği: Tedarikçi ID'sini oku (SID=0xB2)
        LINFrame diag_req;
        diag_req.id = 0x3C;
        diag_req.dlc = 8;
        diag_req.data = {0x06, 0xB2, 0x00, 0x00, 0xFF, 0xFF, 0xFF, 0xFF};
        // NAD=0x06, SID=0xB2 (ReadByIdentifier), ID=0x0000 (Supplier ID)
        diag_req.enhanced_checksum = false;  // Diagnostic frames use classic checksum
        diag_req.type = LINFrameType::DiagRequest;
        diag_req.name = "DiagRequest: ReadByID (SupplierID)";
        diag_req.compute();
        diag_req.print();

```

```
        // EN: Slave diagnostic response
        // TR: Slave teşhis yanıtı (RSID=0xF2, SupplierID=0x1234)
        LINFrame diag_resp;
        diag_resp.id = 0x3D;
        diag_resp.dlc = 8;
        diag_resp.data = {0x06, 0xF2, 0x00, 0x12, 0x34, 0x01, 0xFF, 0xFF};
        // NAD=0x06, RSID=0xF2, SupplierID=0x1234, FunctionID=0x01
        diag_resp.enhanced_checksum = false;
        diag_resp.type = LINFrameType::DiagResponse;
        diag_resp.name = "DiagResponse: SupplierID=0x1234";
        diag_resp.compute();
        diag_resp.print();

        std::cout << "\n  LIN Diagnostic Service IDs:\n";
        std::cout << "   ┌──────────┬──────────────────────────────────────┐\n";
        std::cout << "   │ SID      │ Service Name                         │\n";
        std::cout << "   ├──────────┼──────────────────────────────────────┤\n";
        std::cout << "   │ 0xB0     │ AssignNAD (Node Address assignment)  │\n";
        std::cout << "   │ 0xB2     │ ReadByIdentifier                     │\n";
        std::cout << "   │ 0xB3     │ ConditionalChangeNAD                 │\n";
        std::cout << "   │ 0xB4     │ DataDump (vendor-specific)           │\n";
        std::cout << "   │ 0xB5     │ AssignNAD via SNPD                   │\n";
        std::cout << "   │ 0xB6     │ SaveConfiguration                    │\n";
        std::cout << "   │ 0xB7     │ AssignFrameIdRange                   │\n";
        std::cout << "   └──────────┴──────────────────────────────────────┘\n\n";
    }

    // ─── Demo 8: LIN vs CAN Comparison ──────────────────────────────
    {
        std::cout << "--- Demo 8: LIN vs CAN Comparison ---\n\n";
        std::cout << "   ┌──────────────────────┬──────────────┬──────────────────────────┐\n";
        std::cout << "   │ Feature              │ LIN          │ CAN                      │\n";
        std::cout << "   ├──────────────────────┼──────────────┼──────────────────────────┤\n";
        std::cout << "   │ Speed                │ 20 kbps      │ 1 Mbps / 8 Mbps (FD)     │\n";
        std::cout << "   │ Wiring               │ 1 wire + GND │ 2 wires (twisted pair)   │\n";
        std::cout << "   │ Cost/node            │ ~$0.50       │ ~$2-5                    │\n";
        std::cout << "   │ Architecture         │ Master-Slave │ Multi-master             │\n";
        std::cout << "   │ Arbitration          │ Schedule     │ ID-based bitwise         │\n";
        std::cout << "   │ Max payload          │ 8 bytes      │ 8B / 64B (FD)            │\n";
        std::cout << "   │ Max nodes            │ 1+16         │ Unlimited (practical ~30)│\n";
        std::cout << "   │ Deterministic        │ Yes (100%)   │ No (priority-based)      │\n";
        std::cout << "   │ Error detection      │ Checksum     │ CRC + ACK + error frame  │\n";
        std::cout << "   │ Safety level         │ QM / ASIL-A  │ Up to ASIL-D             │\n";
        std::cout << "   │ Typical use          │ Window, seat │ Engine, brakes, ADAS     │\n";
        std::cout << "   │                      │ mirror, light│                          │\n";
        std::cout << "   └──────────────────────┴──────────────┴──────────────────────────┘\n\n";

        std::cout << "  Typical car LIN topology:\n";
        std::cout << "   ┌──────────────────┐\n";
        std::cout << "   │   BCM            │  Body Control Module = LIN Master\n";
        std::cout << "   │  (Master)        │  Also CAN node → bridges LIN↔CAN\n";
        std::cout << "   └──────┬──┬──┬──┬──┘\n";
        std::cout << "          │  │  │  │    Single wire LIN bus\n";
        std::cout << "       ┌──▼─┐┌▼──┐┌▼───┐┌▼────┐\n";
        std::cout << "       │Win ││Mir││Seat││Light│  LIN Slaves\n";
        std::cout << "       │Mot ││Adj││Ctl ││Ctrl │  (sub-$1 MCU each)\n";
        std::cout << "       └────┘└───┘└────┘└─────┘\n\n";
    }

    std::cout << "============================================\n";
    std::cout << "   End of LIN Bus\n";
    std::cout << "============================================\n";

    return 0;
}
```

► 03. Flexray

module_14_automotive_networks/03_flexray.cpp

Konu: FlexRay — High-Reliability Automotive Bus — FlexRay — Yüksek Güvenilirlikli Otomotiv Bus

Teori / Theory

THEORY: FlexRay — Deterministic, Fault-Tolerant Communication

FlexRay (ISO 17458) is a high-speed, deterministic, fault-tolerant bus designed for safety-critical X-by-wire systems (steer-by-wire, brake-by-wire). Developed by FlexRay Consortium (BMW, Bosch, Daimler, Freescale/NXP).
Key characteristics:
- Dual-channel (A + B): redundancy for fault tolerance
- Max speed: 10 Mbps per channel (20 Mbps aggregate)
- Max payload: 254 bytes
- Deterministic: TDMA (static) + FTDMA (dynamic) mixed scheduling
- Time-synchronization: Global clock, macrotick/microtick precision
- Startup protocol: Coldstart nodes synchronize the cluster
Communication Cycle:

```
┌──────────────────────────────────────────────────────────┐
│                   One Communication Cycle                 │
├──────────────────┬──────────────────┬──────────┬──────────────────┤
│  Static Segment  │ Dynamic Segment  │ Symbol   │ NIT (Network     │
│  (TDMA)          │ (FTDMA)          │ Window   │  Idle Time)      │
│                  │                  │          │                  │
│ Slot 1 | 2 | 3.. │ MiniSlot 1|2|3.. │ (opt)    │ Clock sync       │
│ Fixed bandwidth  │ Event-driven     │ Wakeup/  │ correction       │
│ Guaranteed       │ Best-effort      │ CAS      │                  │
└──────────────────┴──────────────────┴──────────┴──────────────────┘
```

FlexRay Frame Format:

```
┌─────────┬──────────────────────────────────────┬──────────┐
│ Header  │ Payload                              │ Trailer  │
│ (5 B)   │ (0-254 bytes)                        │ (3 B)    │
├─────────┼──────────────────────────────────────┼──────────┤
│ Flags   │ Data bytes                           │ CRC-24   │
│ FrameID │                                      │          │
│ PayLen  │                                      │          │
│ HdrCRC  │                                      │          │
│ CycCnt  │                                      │          │
└─────────┴──────────────────────────────────────┴──────────┘
```

Header Flags:
- Reserved, Payload Preamble Indicator, Null Frame Indicator, Sync Frame Indicator, Startup Frame Indicator

TEORİ: FlexRay — Deterministik, Hata Toleranslı İletişim

FlexRay (ISO 17458), steer-by-wire ve brake-by-wire gibi güvenlik-kritik X-by-wire sistemleri için tasarlanmış yüksek hızlı, belirlenimci, hata-toleranslı bus'tır. FlexRay Konsorsiyumu (BMW, Bosch, Daimler, Freescale/NXP) tarafından geliştirilmiştir.

Temel özellikler:
- Çift kanal (A + B): hata toleransı için yedeklilik
- Maks hız: Kanal başına 10 Mbps (toplam 20 Mbps)
- Maks veri yükü: 254 bayt
- Belirlenimci: TDMA (statik) + FTDMA (dinamik) karma zamanlama
- Zaman senkronizasyonu: Global saat, macrotick/microtick hassasiyeti
- Başlatma protokolü: Coldstart düğümleri kümeyi senkronize eder

İletişim Döngüsü:

```
┌──────────────────────────────────────────────────────────────┐
│                   Bir İletişim Döngüsü                       │
├──────────────────┬──────────────────┬──────────┬──────────────┤
│ Statik Segment   │ Dinamik Segment  │ Sembol   │ NIT (Ağ Boşta  │
│ (TDMA)           │ (FTDMA)          │ Penceresi│  Süresi)       │
│                  │                  │          │                │
│ Slot 1 | 2 | 3.. │ MiniSlot 1|2|3.. │ (ops.)   │ Saat düzeltme  │
│ Sabit bant gen.  │ Olay-güdümlü     │ Uyanma/  │ hesaplaması    │
│ Garantili        │ En-iyi-çaba      │ CAS      │                │
└──────────────────┴──────────────────┴──────────┴──────────────┘
```

FlexRay Çerçeve Formatı:

```
┌─────────┬──────────────────────────────────────┬──────────┐
│ Başlık  │ Veri Yükü                            │ Kuyruk   │
│ (5 B)   │ (0-254 bayt)                         │ (3 B)    │
├─────────┼──────────────────────────────────────┼──────────┤
│ Bayrak  │ Veri baytları                        │ CRC-24   │
│ FrameID │                                      │          │
│ PayLen  │                                      │          │
│ HdrCRC  │                                      │          │
│ CycCnt  │                                      │          │
└─────────┴──────────────────────────────────────┴──────────┘
```

Başlık Bayrakları:
- Reserved, Payload Preamble Indicator, Null Frame Indicator, Sync Frame Indicator, Startup Frame Indicator

THEORY: FlexRay vs CAN vs Ethernet

```
┌──────────────────┬──────────────┬──────────────┬──────────────┐
│ Feature          │ CAN FD       │ FlexRay      │ Eth 100BASE-T1│
├──────────────────┼──────────────┼──────────────┼──────────────┤
│ Speed            │ 8 Mbps       │ 2x10 Mbps    │ 100 Mbps     │
│ Payload          │ 64 bytes     │ 254 bytes    │ 1500 bytes   │
│ Determinism      │ Priority     │ TDMA ✓       │ TSN needed   │
│ Redundancy       │ No           │ Dual-channel │ Switch-based │
│ Topology         │ Bus          │ Bus/Star     │ Star/Switch  │
│ Sync precision   │ N/A          │ ~1 µs        │ gPTP ~100ns  │
│ Cost             │ Low          │ Medium-High  │ Medium       │
│ Safety level     │ ASIL-D       │ ASIL-D       │ ASIL-D (TSN) │
│ Future outlook   │ Maintained   │ Legacy→Eth   │ Growing      │
└──────────────────┴──────────────┴──────────────┴──────────────┘
```

TEORİ: FlexRay, CAN ve Ethernet Karşılaştırması

FlexRay vs CAN vs Ethernet karşılaştırması:
- Hız: CAN FD 8 Mbps, FlexRay çift 10 Mbps, Ethernet 100+ Mbps
- Veri yükü: CAN FD 64 B, FlexRay 254 B, Ethernet 1500+ B
- Belirlenimcilik: CAN öncelik tabanlı, FlexRay doğal TDMA, Ethernet TSN gerektirir
- Yedeklilik: CAN yok, FlexRay çift kanal donanımsal, Ethernet HSR/PRP yazılımsal
- Maliyet: CAN düşük, FlexRay orta-yüksek, Ethernet azalan trend
- Gelecek: CAN bakımda, FlexRay eski→Ethernet'e geçiş, Ethernet büyüyen standart
- Neden FlexRay yerine Ethernet: Daha yüksek bant genişliği, SOA desteği, IP tabanlı servisler (SOME/IP), OTA güncelleme, evrensel ekosistem

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <array>
#include <vector>
#include <string>
#include <iomanip>
#include <algorithm>
#include <cassert>
#include <map>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: FlexRay Frame Structure
// ═══════════════════════════════════════════════════════════════════════════

// EN: FlexRay frame header flags
// TR: FlexRay çerçeve başlık bayrakları
struct FlexRayFlags {
    bool startup_frame  = false;  // Küme başlatma sırasında kullanılır (coldstart düğümü tarafından)
    bool sync_frame     = false;  // Saat senkronizasyonu için kullanılır (global zaman hizalama)
    bool null_frame     = false;  // Geçerli veri yok (slot hâlâ dolu — bant genişliği korunur)
    bool payload_preamble = false;  // Veri yükü NM vektörü veya mesaj ID'si ile başlar
};

// EN: Complete FlexRay frame
// TR: Tam FlexRay çerçevesi — başlık, veri ve CRC alanlarını kapsar
struct FlexRayFrame {
    FlexRayFlags flags;
    uint16_t frame_id      = 0;   // 1-2047 arası slot numarası (0 = geçersiz, TDMA zamanlamasını belirler)
    uint8_t  payload_length = 0;  // 16-bit kelime cinsinden (gerçek bayt = ×2, maks 127 → 254 bayt)
    uint16_t header_crc    = 0;   // 11-bit CRC — başlık bütünlüğünü doğrular
    uint8_t  cycle_count   = 0;   // 0-63 döngü sayacı (64 döngüde sıfırlanır)
    std::vector<uint8_t> payload;
    uint32_t frame_crc     = 0;   // CRC-24 — başlık + veri yükü üzerinden hesaplanır

    // EN: Actual data length in bytes
    // TR: Gerçek veri uzunluğu (bayt) — payload_length 16-bit kelime, ×2 ile bayta çevrilir
    size_t data_bytes() const { return static_cast<size_t>(payload_length) * 2; }

    // EN: Channel assignment
    // TR: Kanal ataması — A, B veya AB (çift kanal yedeklilik)
    enum class Channel { A, B, AB };
    Channel channel = Channel::AB;

```

```cpp
    void print(const std::string& label) const {
        std::cout << "  [FlexRay] " << label << "\n";
        std::cout << "    FrameID: " << frame_id
                  << " | Cycle: " << static_cast<int>(cycle_count)
                  << " | PayLen: " << static_cast<int>(payload_length)
                  << " (" << data_bytes() << " bytes)"
                  << " | Ch: " << (channel == Channel::A ? "A" :
                                   channel == Channel::B ? "B" : "A+B") << "\n";
        std::cout << "    Flags: "
                  << (flags.startup_frame ? "[STARTUP] " : "")
                  << (flags.sync_frame ? "[SYNC] " : "")
                  << (flags.null_frame ? "[NULL] " : "")
                  << (flags.payload_preamble ? "[PREAMBLE]" : "")
                  << "\n";
        if (!payload.empty() && !flags.null_frame) {
            std::cout << "    Data:";
            size_t show = std::min(payload.size(), size_t{32});
            for (size_t i = 0; i < show; ++i) {
                if (i > 0 && i % 16 == 0) std::cout << "\n           ";
                std::cout << " " << std::hex << std::setw(2) << std::setfill('0')
                          << static_cast<int>(payload[i]);
            }
            if (payload.size() > 32)
                std::cout << " ... (" << std::dec << payload.size() << " total)";
            std::cout << std::dec << "\n";
        }
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: FlexRay Timing — Macrotick, Microtick, Cycle
// ═══════════════════════════════════════════════════════════════════════════

// EN: FlexRay time hierarchy:
//     Microtick: smallest time unit (based on oscillator, ~12.5-50 ns)
//     Macrotick: cluster-wide tick (N microticks, typically 1-6 µs)
//     Static Slot: fixed duration in macroticks
//     Communication Cycle: all segments combined (typ. 1-5 ms)
// TR: FlexRay zaman hiyerarşisi:
//     Microtick: En küçük zaman birimi (osilatör tabanlı, ~12.5-50 ns)
//     Macrotick: Küme çapında tik (N microtick, tipik 1-6 µs)
//     Statik Slot: Macrotick cinsinden sabit süre
//     İletişim Döngüsü: Tüm segmentler birleşik (tipik 1-5 ms)

struct FlexRayClusterConfig {
    // EN: Timing parameters
    // TR: Zamanlama parametreleri — küme çapında senkron çalışma için
    uint16_t macrotick_ns       = 1000;  // 1 macrotick = 1000 ns = 1 µs
    uint16_t static_slot_mt     = 25;    // static slot = 25 macroticks
    uint16_t num_static_slots   = 20;    // 20 static slots in static segment
    uint16_t dynamic_segment_mt = 100;   // dynamic segment = 100 macroticks
    uint16_t minislot_mt        = 5;     // one minislot = 5 macroticks
    uint16_t symbol_window_mt   = 10;    // symbol window = 10 macroticks
    uint16_t nit_mt             = 30;    // Network Idle Time = 30 macroticks
    uint8_t  max_cycle          = 64;    // cycle counter wraps at 64

    // EN: Calculated cycle length
    // TR: Hesaplanan döngü uzunluğu — statik + dinamik + sembol + NIT toplamı
    uint32_t cycle_length_mt() const {
        return static_cast<uint32_t>(static_slot_mt) * num_static_slots
             + dynamic_segment_mt + symbol_window_mt + nit_mt;
    }

    uint32_t cycle_length_us() const {
        return cycle_length_mt() * macrotick_ns / 1000;
    }

    void print() const {
        std::cout << "  FlexRay Cluster Configuration:\n";
        std::cout << "    Macrotick:        " << macrotick_ns << " ns\n";
        std::cout << "    Static slot:      " << static_slot_mt << " MT ("
                  << static_slot_mt * macrotick_ns / 1000 << " µs)\n";
        std::cout << "    Num static slots: " << num_static_slots << "\n";
        std::cout << "    Dynamic segment:  " << dynamic_segment_mt << " MT\n";
        std::cout << "    Minislot:         " << minislot_mt << " MT\n";
        std::cout << "    Symbol window:    " << symbol_window_mt << " MT\n";
        std::cout << "    NIT:              " << nit_mt << " MT\n";
```

```cpp
        std::cout << "    Cycle length:     " << cycle_length_mt() << " MT ("
                  << cycle_length_us() << " µs)\n";
        std::cout << "    Cycle frequency:  " << (1000000.0 / static_cast<double>(cycle_length_us()))
                  << " Hz\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: FlexRay Static Segment — TDMA Scheduling
// ═══════════════════════════════════════════════════════════════════════════

// EN: In the static segment, slots are assigned to nodes at design time.
//     Each node knows which slot(s) it owns. Slot = guaranteed bandwidth.
//     TDMA = Time Division Multiple Access — no contention, no collision.
// TR: Statik segmentte slotlar tasarım zamanında düğümlere atanır.
//     Her düğüm hangi slot(lar)ın kendisine ait olduğunu bilir. Slot = garantili bant genişliği.
//     TDMA = Zaman Bölümlü Çoklu Erişim — çakışma yok, çarpışma yok.

struct StaticSlotAssignment {
    uint16_t slot_id;       // 1-tabanlı slot numarası (TDMA zamanlama indeksi)
    std::string node_name;  // Bu slota sahip ECU adı
    std::string signal_name; // Slotta taşınan sinyal (ör. SteeringAngle)
    uint8_t payload_words;  // 16-bit kelime cinsinden veri yükü
    FlexRayFrame::Channel channel; // A, B veya AB kanalı
};

class FlexRayStaticSchedule {
    std::vector<StaticSlotAssignment> assignments_;

public:
    void assign(uint16_t slot, const std::string& node, const std::string& signal,
                uint8_t payload_words, FlexRayFrame::Channel ch) {
        assignments_.push_back({slot, node, signal, payload_words, ch});
    }

    const std::vector<StaticSlotAssignment>& slots() const { return assignments_; }

    void print() const {
        std::cout << "  Static Segment Schedule:\n";
        std::cout << "  ┌──────┬──────────────┬────────────────────┬────────┬────┐\n";
        std::cout << "  │ Slot │ Node         │ Signal             │ Bytes  │ Ch │\n";
        std::cout << "  ├──────┼──────────────┼────────────────────┼────────┼────┤\n";
        for (const auto& a : assignments_) {
            std::string ch_str = (a.channel == FlexRayFrame::Channel::A ? "A" :
                                  a.channel == FlexRayFrame::Channel::B ? "B" : "A+B");
            std::cout << "  │ " << std::setw(4) << a.slot_id
                      << " │ " << std::left << std::setw(15) << a.node_name
                      << " │ " << std::setw(20) << a.signal_name
                      << " │ " << std::right << std::setw(6) << static_cast<int>(a.payload_words) * 2
                      << " │ " << std::setw(3) << ch_str << " │\n";
        }
        std::cout << "  └──────┴──────────────┴────────────────────┴────────┴────┘\n";
    }
};

```

```cpp
// ═══════════════════════════════════════════════════════════════════════════
// PART 4: FlexRay Startup Protocol
// ═══════════════════════════════════════════════════════════════════════════

// EN: FlexRay startup requires special nodes:
//     - Coldstart nodes (2-3 minimum): initiate cluster startup
//     - Integration nodes: join after coldstart is complete
//
//     Startup State Machine:
//     DEFAULT_CONFIG → READY → STARTUP → NORMAL_ACTIVE
//          (leading coldstart)        (integrated into cluster)
//
//     A leading coldstart node sends CAS (Collision Avoidance Symbol) on
//     the bus, then sends sync+startup frames. Other coldstart nodes
//     integrate. Once all agree, the cluster enters NORMAL_ACTIVE.
// TR: FlexRay başlatma, özel coldstart düğümleri gerektirir (en az 2-3).
//     Entegrasyon düğümleri: coldstart tamamlandıktan sonra katılır.
//
//     Başlatma Durum Makinesi:
//     DEFAULT_CONFIG → READY → STARTUP → NORMAL_ACTIVE
//          (öncü coldstart)           (kümeye entegre)
//
//     Öncü coldstart düğümü bus'üzerinde CAS (Collision Avoidance Symbol) gönderir,
//     ardından sync+startup çerçeveleri yollar. Diğer coldstart düğümleri entegre olur.
//     Tümü anlaştığında küme NORMAL_ACTIVE durumuna geçer.

enum class FlexRayNodeState {
    DefaultConfig,
    Ready,
    WakeupListen,
    WakeupSend,
    StartupPrepare,
    ColdstartListen,
    ColdstartCollisionResolution,
    ColdstartConsistencyCheck,
    ColdstartGap,
    IntegrationColdstartCheck,
    IntegrationListen,
    IntegrationConsistencyCheck,
    NormalActive,
    NormalPassive,
    Halt
};

std::string to_string(FlexRayNodeState s) {
    switch (s) {
        case FlexRayNodeState::DefaultConfig:     return "DEFAULT_CONFIG";
        case FlexRayNodeState::Ready:             return "READY";
        case FlexRayNodeState::WakeupListen:      return "WAKEUP_LISTEN";
        case FlexRayNodeState::WakeupSend:        return "WAKEUP_SEND";
        case FlexRayNodeState::StartupPrepare:    return "STARTUP_PREPARE";
        case FlexRayNodeState::ColdstartListen:   return "COLDSTART_LISTEN";
        case FlexRayNodeState::ColdstartCollisionResolution: return "COLDSTART_COLLISION_RES";
        case FlexRayNodeState::ColdstartConsistencyCheck:    return "COLDSTART_CONSISTENCY";
        case FlexRayNodeState::ColdstartGap:      return "COLDSTART_GAP";
        case FlexRayNodeState::IntegrationColdstartCheck:    return "INTEGRATION_COLDSTART";
        case FlexRayNodeState::IntegrationListen: return "INTEGRATION_LISTEN";
        case FlexRayNodeState::IntegrationConsistencyCheck:  return "INTEGRATION_CONSISTENCY";
        case FlexRayNodeState::NormalActive:       return "NORMAL_ACTIVE";
        case FlexRayNodeState::NormalPassive:      return "NORMAL_PASSIVE";
        case FlexRayNodeState::Halt:               return "HALT";
    }
    return "UNKNOWN";
}

class FlexRayNode {
    std::string name_;
    bool is_coldstart_;
    FlexRayNodeState state_ = FlexRayNodeState::DefaultConfig;
    bool sync_frame_tx_ = false;
    uint8_t current_cycle_ = 0;

```

```cpp
public:
    FlexRayNode(std::string name, bool coldstart)
        : name_(std::move(name)), is_coldstart_(coldstart) {}

    const std::string& name() const { return name_; }
    bool is_coldstart() const { return is_coldstart_; }
    FlexRayNodeState state() const { return state_; }

    // EN: Simplified startup sequence
    // TR: Basitleştirilmiş başlatma dizisi — düğümü Ready durumuna geçirir
    void configure() { state_ = FlexRayNodeState::Ready; }

    void initiate_startup() {
        if (!is_coldstart_) {
            state_ = FlexRayNodeState::IntegrationListen;
            return;
        }
        state_ = FlexRayNodeState::ColdstartListen;
    }

    void send_cas() {
        if (is_coldstart_ && state_ == FlexRayNodeState::ColdstartListen) {
            state_ = FlexRayNodeState::ColdstartCollisionResolution;
            sync_frame_tx_ = true;
        }
    }

    void coldstart_complete() {
        if (is_coldstart_) {
            state_ = FlexRayNodeState::NormalActive;
        }
    }

    void integrate() {
        if (!is_coldstart_ && state_ == FlexRayNodeState::IntegrationListen) {
            state_ = FlexRayNodeState::IntegrationConsistencyCheck;
        }
    }

    void integration_complete() {
        state_ = FlexRayNodeState::NormalActive;
    }

    void increment_cycle() {
        current_cycle_ = static_cast<uint8_t>((current_cycle_ + 1) % 64);
    }

    void print_status() const {
        std::cout << "    " << std::left << std::setw(20) << name_
                  << std::right
                  << " | State: " << std::left << std::setw(28) << to_string(state_)
                  << std::right
                  << " | " << (is_coldstart_ ? "COLDSTART" : "INTEGRATION")
                  << " | Cycle: " << static_cast<int>(current_cycle_) << "\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 5: FlexRay Dual-Channel Redundancy
// ═══════════════════════════════════════════════════════════════════════════

// EN: FlexRay dual-channel provides:
//     1) Redundancy: Same data on A+B → detect channel failure
//     2) Bandwidth doubling: Different data on A and B → 2x throughput
//     3) Mixed: Some frames on A+B (safety), some on A or B only
// TR: FlexRay çift kanal şunları sağlar:
//     1) Yedeklilik: A+B'üzerinde aynı veri → kanal arızasını tespit et
//     2) Bant genişliği iki katına: A ve B'üzerinde farklı veri → 2x verim
//     3) Karışık: Bazı çerçeveler A+B (güvenlik), bazıları yalnız A veya B

struct DualChannelResult {
    bool channel_a_ok = true;
    bool channel_b_ok = true;
    bool data_consistent = true;  // A ve B eşleşiyor mu (her iki kanalda aynı veri gönderildiğinde)

```

```cpp
    void print() const {
        std::cout << "    Channel A: " << (channel_a_ok ? "OK" : "FAULT")
                  << " | Channel B: " << (channel_b_ok ? "OK" : "FAULT")
                  << " | Consistency: " << (data_consistent ? "MATCH ✓" : "MISMATCH ✗")
                  << "\n";
    }
};

DualChannelResult check_redundancy(const FlexRayFrame& frame_a, const FlexRayFrame& frame_b) {
    DualChannelResult result;
    result.channel_a_ok = !frame_a.flags.null_frame;
    result.channel_b_ok = !frame_b.flags.null_frame;

    if (result.channel_a_ok && result.channel_b_ok) {
        // EN: Compare payloads for consistency
        // TR: Tutarlılık için veri yüklerini karşılaştır — A ve B aynı olmalı
        result.data_consistent = (frame_a.payload == frame_b.payload) &&
                                 (frame_a.frame_id == frame_b.frame_id);
    } else {
        result.data_consistent = false;
    }
    return result;
}

// ═══════════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 14 - FlexRay\n";
    std::cout << "============================================\n\n";

    // ─── Demo 1: FlexRay Frame Structure ──────────────────────────────
    {
        std::cout << "--- Demo 1: FlexRay Frame Structure ---\n";

        // EN: Steering angle frame — safety-critical, dual channel
        // TR: Direksiyon açısı çerçevesi — güvenlik-kritik, çift kanal (ASIL-D)
        FlexRayFrame steering;
        steering.frame_id = 1;
        steering.payload_length = 4;  // 8 bytes
        steering.cycle_count = 0;
        steering.flags.sync_frame = true;
        steering.flags.startup_frame = true;
        steering.channel = FlexRayFrame::Channel::AB;
        steering.payload = {0x03, 0xE8, 0x00, 0x50, 0x01, 0x00, 0x00, 0x00};
        // Direksiyon açısı: 0x03E8 = 1000 (= 100.0°), tork: 0x0050 = 80 Nm
        steering.print("SteeringAngle (ASIL-D, dual-channel)");

        // EN: Brake pressure frame
        // TR: Fren basıncı çerçevesi — ASIL-D, çift kanal yedeklilik
        FlexRayFrame brake;
        brake.frame_id = 2;
        brake.payload_length = 3;  // 6 bytes
        brake.cycle_count = 0;
        brake.flags.sync_frame = true;
        brake.channel = FlexRayFrame::Channel::AB;
        brake.payload = {0x07, 0xD0, 0x00, 0x64, 0x01, 0x01};
        // Fren basıncı: 0x07D0 = 2000 (= 20.0 bar), ABS aktif: 0x01
        brake.print("BrakePressure (ASIL-D, dual-channel)");

        // EN: Non-safety data: infotainment (single channel, larger payload)
        // TR: Güvenlik dışı veri: infotainment (tek kanal, büyük veri yükü)
        FlexRayFrame infotainment;
        infotainment.frame_id = 15;
        infotainment.payload_length = 64;  // 128 bytes
        infotainment.cycle_count = 0;
        infotainment.channel = FlexRayFrame::Channel::A;
        infotainment.payload.resize(128);
        for (int i = 0; i < 128; ++i) infotainment.payload[static_cast<size_t>(i)] = static_cast<uint8_t>(i & 0xFF);
        infotainment.print("MediaStream (QM, channel A only)");
        std::cout << "\n";
    }
```

```cpp

    // ─── Demo 2: Cluster Timing Configuration ──────────────────────────
    {
        std::cout << "--- Demo 2: FlexRay Timing Configuration ---\n";

        FlexRayClusterConfig config;
        config.macrotick_ns = 1000;       // 1 µs per macrotick
        config.static_slot_mt = 25;       // 25 µs per static slot
        config.num_static_slots = 20;     // 20 static slots
        config.dynamic_segment_mt = 100;  // 100 µs dynamic
        config.symbol_window_mt = 10;
        config.nit_mt = 30;

        config.print();

        std::cout << "\n  Communication Cycle Layout:\n";
        uint32_t static_end = static_cast<uint32_t>(config.static_slot_mt) * config.num_static_slots;
        uint32_t dynamic_end = static_end + config.dynamic_segment_mt;
        uint32_t sw_end = dynamic_end + config.symbol_window_mt;
        uint32_t nit_end = sw_end + config.nit_mt;
        std::cout << "  |← Static: 0-" << static_end << " MT →|"
                  << "← Dynamic: " << static_end << "-" << dynamic_end << " MT →|"
                  << "← SW: " << dynamic_end << "-" << sw_end << " →|"
                  << "← NIT: " << sw_end << "-" << nit_end << " →|\n\n";
    }

    // ─── Demo 3: Static Segment TDMA Schedule ──────────────────────────
    {
        std::cout << "--- Demo 3: Static Segment TDMA Schedule ---\n";
        std::cout << "  Scenario: Steer-by-wire system\n\n";

        FlexRayStaticSchedule schedule;

        // EN: Safety-critical signals on A+B (redundant)
        // TR: Güvenlik-kritik sinyaller A+B'üzerinde (yedekli)
        schedule.assign(1, "SteeringECU", "SteeringAngle", 4, FlexRayFrame::Channel::AB);
        schedule.assign(2, "BrakeECU", "BrakePressure", 3, FlexRayFrame::Channel::AB);
        schedule.assign(3, "SteeringECU", "SteeringTorque", 2, FlexRayFrame::Channel::AB);
        schedule.assign(4, "BrakeECU", "WheelSpeed_FL", 2, FlexRayFrame::Channel::AB);
        schedule.assign(5, "BrakeECU", "WheelSpeed_FR", 2, FlexRayFrame::Channel::AB);
        schedule.assign(6, "RackActuator", "RackPosition", 4, FlexRayFrame::Channel::AB);
        schedule.assign(7, "RackActuator", "MotorCurrent", 2, FlexRayFrame::Channel::AB);

        // EN: Medium-priority signals on single channel
        // TR: Orta öncelikli sinyaller tek kanalda — bant genişliği ikiye katlama
        schedule.assign(8, "SuspensionECU", "DamperForce_FL", 2, FlexRayFrame::Channel::A);
        schedule.assign(9, "SuspensionECU", "DamperForce_FR", 2, FlexRayFrame::Channel::A);
        schedule.assign(10, "SuspensionECU", "DamperForce_RL", 2, FlexRayFrame::Channel::B);
        schedule.assign(11, "SuspensionECU", "DamperForce_RR", 2, FlexRayFrame::Channel::B);

        // EN: Lower-priority / diagnostic
        // TR: Düşük öncelikli / teşhis verisi — tek kanal yeterli
        schedule.assign(15, "Gateway", "DiagnosticData", 32, FlexRayFrame::Channel::A);

        schedule.print();
        std::cout << "\n  NOTE: Slots 1-7 on A+B = ASIL-D redundant (steer-by-wire)\n";
        std::cout << "        Slots 8-11 split A/B = bandwidth doubling (suspension)\n";
        std::cout << "        Slot 15 on A only = non-safety diagnostic data\n\n";
    }

    // ─── Demo 4: Startup Protocol Simulation ───────────────────────────
    {
        std::cout << "--- Demo 4: FlexRay Startup Protocol ---\n";

        // EN: Create cluster: 2 coldstart nodes + 2 integration nodes
        // TR: Küme oluştur: 2 coldstart + 2 entegrasyon düğümü
        FlexRayNode steering("SteeringECU", true);   // coldstart
        FlexRayNode brake("BrakeECU", true);          // coldstart
        FlexRayNode suspension("SuspensionECU", false); // integration
        FlexRayNode gateway("Gateway", false);          // integration

        std::vector<FlexRayNode*> cluster = {&steering, &brake, &suspension, &gateway};

        // EN: Step 1: Configure all nodes
        // TR: Adım 1: Tüm düğümleri yapılandır — DEFAULT_CONFIG → READY
        std::cout << "  Step 1: Configure nodes\n";
```

```
472         for (auto* node : cluster) {
473             node->configure();
474             node->print_status();
475         }
476 
477         // EN: Step 2: Initiate startup
478         // TR: Adım 2: Başlatmayı tetikle — coldstart dinlemeye, entegrasyon beklemeye geçer
479         std::cout << "\n  Step 2: Initiate startup\n";
480         for (auto* node : cluster) {
481             node->initiate_startup();
482             node->print_status();
483         }
484 
485         // EN: Step 3: Leading coldstart node sends CAS
486         // TR: Adım 3: Öncü coldstart düğümü CAS (Collision Avoidance Symbol) gönderir
487         std::cout << "\n  Step 3: Coldstart nodes send CAS (Collision Avoidance Symbol)\n";
488         steering.send_cas();  // leading coldstart
489         brake.send_cas();     // following coldstart
490         for (auto* node : cluster) node->print_status();
491 
492         // EN: Step 4: Coldstart complete → NORMAL_ACTIVE
493         // TR: Adım 4: Coldstart tamamlandı → NORMAL_ACTIVE durumuna geçiş
494         std::cout << "\n  Step 4: Coldstart complete\n";
495         steering.coldstart_complete();
496         brake.coldstart_complete();
497         for (auto* node : cluster) node->print_status();
498 
499         // EN: Step 5: Integration nodes join
500         // TR: Adım 5: Entegrasyon düğümleri kümeye katılır
501         std::cout << "\n  Step 5: Integration nodes join cluster\n";
502         suspension.integrate();
503         gateway.integrate();
504         for (auto* node : cluster) node->print_status();
505 
506         // EN: Step 6: Integration complete — all NORMAL_ACTIVE
507         // TR: Adım 6: Entegrasyon tamamlandı — tüm düğümler NORMAL_ACTIVE
508         std::cout << "\n  Step 6: All nodes NORMAL_ACTIVE\n";
509         suspension.integration_complete();
510         gateway.integration_complete();
511         for (auto* node : cluster) node->print_status();
512         std::cout << "\n";
513     }
514 
515     // ─── Demo 5: Dual Channel Redundancy Check ──────────────────────────
516     {
517         std::cout << "--- Demo 5: Dual Channel Redundancy ---\n";
518 
519         // EN: Case 1: Both channels OK, data matches
520         // TR: Durum 1: Her iki kanal da sağlam, veriler eşleşiyor
521         std::cout << "  Case 1: Normal operation (both channels OK)\n";
522         FlexRayFrame frame_a1;
523         frame_a1.frame_id = 1;
524         frame_a1.payload = {0x03, 0xE8, 0x00, 0x50};
525         FlexRayFrame frame_b1 = frame_a1;  // same data on B
526         auto result1 = check_redundancy(frame_a1, frame_b1);
527         result1.print();
528 
529         // EN: Case 2: Channel B failure (null frame)
530         // TR: Durum 2: Kanal B arızası (boş çerçeve alındı)
531         std::cout << "  Case 2: Channel B failure (null frame received)\n";
532         FlexRayFrame frame_b2;
533         frame_b2.frame_id = 1;
534         frame_b2.flags.null_frame = true;
535         auto result2 = check_redundancy(frame_a1, frame_b2);
536         result2.print();
537 
```

```
        // EN: Case 3: Data corruption on Channel B
        // TR: Durum 3: Kanal B'üzerinde veri bozulması — tutarsızlık tespiti
        std::cout << "  Case 3: Data corruption on Channel B\n";
        FlexRayFrame frame_b3;
        frame_b3.frame_id = 1;
        frame_b3.payload = {0x03, 0xE8, 0x00, 0x51};  // 0x50→0x51 corruption
        auto result3 = check_redundancy(frame_a1, frame_b3);
        result3.print();

        std::cout << "\n  Redundancy strategies:\n";
        std::cout << "    1. Both match → Use data normally\n";
        std::cout << "    2. One channel fault → Use other channel (degraded mode)\n";
        std::cout << "    3. Data mismatch → Enter safe state / re-synchronize\n\n";
    }

    // ─── Demo 6: Steer-by-Wire Simulation ──────────────────────────
    {
        std::cout << "--- Demo 6: Steer-by-Wire Simulation ---\n";
        std::cout << "  Simulating 10 communication cycles (steering + rack feedback)\n\n";

        double steering_angle = 0.0;   // degrees
        double rack_position = 0.0;    // mm
        constexpr double steering_ratio = 16.0;  // steering:rack ratio
        constexpr double angle_step = 5.0;       // degrees per cycle (driver turning)

        std::cout << "  ┌────────┬──────────────┬──────────────┬──────────────────┐\n";
        std::cout << "  │ Cycle  │ SteerAngle(°)│ RackPos(mm)  │ Status           │\n";
        std::cout << "  ├────────┼──────────────┼──────────────┼──────────────────┤\n";

        for (int cycle = 0; cycle < 10; ++cycle) {
            steering_angle += angle_step;
            rack_position = steering_angle / steering_ratio;

            // EN: Pack into FlexRay frame
            // TR: FlexRay çerçevesine paketle — 0.1° ve 0.01mm çözünürlükle
            FlexRayFrame steer_frame;
            steer_frame.frame_id = 1;
            steer_frame.cycle_count = static_cast<uint8_t>(cycle);
            auto angle_raw = static_cast<int16_t>(steering_angle * 10);  // 0.1° resolution
            auto rack_raw = static_cast<int16_t>(rack_position * 100);    // 0.01mm resolution
            steer_frame.payload = {
                static_cast<uint8_t>(angle_raw & 0xFF),
                static_cast<uint8_t>((angle_raw >> 8) & 0xFF),
                static_cast<uint8_t>(rack_raw & 0xFF),
                static_cast<uint8_t>((rack_raw >> 8) & 0xFF)
            };

            std::string status = "OK";
            if (steering_angle > 40.0) status = "WARN: fast turn";
            if (steering_angle > 45.0) status = "LIMIT: max assist";

            std::cout << "  │ " << std::setw(5) << cycle
                      << " │ " << std::setw(12) << std::fixed << std::setprecision(1)
                      << steering_angle
                      << " │ " << std::setw(12) << std::setprecision(2) << rack_position
                      << " │ " << std::left << std::setw(17) << status << std::right
                      << " │\n";
        }
        std::cout << "  └────────┴──────────────┴──────────────┴──────────────────┘\n\n";
    }

    // ─── Demo 7: FlexRay Future — Migration to Ethernet ────────────
    {
        std::cout << "--- Demo 7: FlexRay vs Ethernet — Migration Path ---\n\n";
        std::cout << "  FlexRay is being replaced by Automotive Ethernet + TSN:\n\n";
        std::cout << "  ┌──────────────────────┬──────────────────┬──────────────────┐\n";
        std::cout << "  │ Aspect               │ FlexRay          │ Ethernet + TSN   │\n";
        std::cout << "  ├──────────────────────┼──────────────────┼──────────────────┤\n";
        std::cout << "  │ Speed                │ 2x10 Mbps        │ 100M-10G bps     │\n";
        std::cout << "  │ Payload              │ 254 bytes        │ 1500+ bytes      │\n";
        std::cout << "  │ Determinism          │ Native TDMA      │ TSN (802.1Qbv)   │\n";
        std::cout << "  │ Redundancy           │ Dual-channel     │ HSR/PRP (IEC)    │\n";
        std::cout << "  │ Ecosystem            │ Automotive only  │ Universal IT/OT  │\n";
        std::cout << "  │ Cost trend           │ Flat             │ Decreasing       │\n";
        std::cout << "  │ SDV compatible       │ Limited          │ Native SOA       │\n";
        std::cout << "  │ IP-based services    │ No               │ Yes (SOME/IP)    │\n";
        std::cout << "  │ OTA update support   │ Limited          │ Full             │\n";
```

```
        std::cout << "  │ New vehicle designs    │ Rare (2024+)     │ Standard         │\n";
        std::cout << "  └────────────────────────┴──────────────────┴──────────────────┘\n\n";

        std::cout << "  Timeline:\n";
        std::cout << "    2006-2015: FlexRay in BMW 5/7 Series, Mercedes S-Class\n";
        std::cout << "    2015-2020: Automotive Ethernet adoption starts (ADAS cameras)\n";
        std::cout << "    2020-2025: TSN matures, replaces FlexRay in new platforms\n";
        std::cout << "    2025+:     FlexRay → legacy support only, Ethernet dominates\n\n";
    }

    std::cout << "==========================================\n";
    std::cout << "   End of FlexRay\n";
    std::cout << "==========================================\n";

    return 0;
}
```

► 04. Automotive Ethernet

module_14_automotive_networks/04_automotive_ethernet.cpp

Konu: Automotive Ethernet — 100BASE-T1, SOME/IP, DoIP, TSN

Teori / Theory

THEORY: Automotive Ethernet — High-Speed Vehicle Networking

Automotive Ethernet is the backbone of next-generation E/E architectures. Unlike consumer Ethernet (Cat5/6, 8 wires), automotive uses BroadR-Reach (100BASE-T1): single unshielded twisted pair, 100 Mbps, full-duplex.
Physical Layers:

```
┌──────────────────┬──────────┬──────┬──────────────────────┐
│ Standard         │ Speed    │ Pairs │ Use Case             │
├──────────────────┼──────────┼──────┼──────────────────────┤
│ 100BASE-T1       │ 100 Mbps │ 1 UTP │ General (ADAS, body)  │
│ 1000BASE-T1      │ 1 Gbps   │ 1 UTP │ Camera, infotainment  │
│ 10BASE-T1S       │ 10 Mbps  │ 1 UTP │ 10SPE multidrop bus   │
│ MGBASE-T1 (2.5/5)│ 2.5-5G   │ 1 UTP │ High-res sensors      │
│ 10GBASE-T1       │ 10 Gbps  │ 1 UTP │ Central computer      │
└──────────────────┴──────────┴──────┴──────────────────────┘
```

Protocol Stack:

```
┌──────────────────────────────────┐
│ Application Layer                │
│   SOME/IP, DoIP, DDS, HTTP/REST  │
├──────────────────────────────────┤
│ Transport Layer                  │
│   TCP, UDP                       │
├──────────────────────────────────┤
│ Network Layer                    │
│   IPv4, IPv6                     │
├──────────────────────────────────┤
│ Data Link Layer                  │
│   Ethernet II, VLAN (802.1Q), TSN │
├──────────────────────────────────┤
│ Physical Layer                   │
│   100BASE-T1, 1000BASE-T1        │
└──────────────────────────────────┘
```

TEORİ: Otomotiv Ethernet — Yüksek Hızlı Araç Ağı

Otomotiv Ethernet, yeni nesil E/E mimarilerinin omurgasıdır. Tüketici Ethernet'inden (Cat5/6, 8 kablo) farklı olarak otomotiv BroadR-Reach (100BASE-T1) kullanır: tek korunmasız bükümlü çift, 100 Mbps, tam çift yönlü.
Fiziksel Katmanlar:

Standart	Hız	Çift	Kullanım Alanı
100BASE-T1	100 Mbps	1 UTP	Genel (ADAS, gövde)
1000BASE-T1	1 Gbps	1 UTP	Kamera, infotainment
10BASE-T1S	10 Mbps	1 UTP	10SPE multidrop bus
MGBASE-T1 (2.5/5)	2.5-5G	1 UTP	Yüksek çöz. sensör
10GBASE-T1	10 Gbps	1 UTP	Merkezi bilgisayar

Protokol Yığını:

```
┌──────────────────────────────────┐
│ Uygulama Katmanı                 │
│   SOME/IP, DoIP, DDS, HTTP/REST  │
├──────────────────────────────────┤
│ Taşıma Katmanı                   │
│   TCP, UDP                       │
├──────────────────────────────────┤
│ Ağ Katmanı                       │
│   IPv4, IPv6                     │
├──────────────────────────────────┤
│ Veri Bağlantı Katmanı            │
│   Ethernet II, VLAN (802.1Q), TSN │
├──────────────────────────────────┤
│ Fiziksel Katman                  │
│   100BASE-T1, 1000BASE-T1        │
└──────────────────────────────────┘
```

THEORY: SOME/IP — Scalable service-Oriented MiddlewarE over IP

SOME/IP is the de facto service middleware for AUTOSAR Adaptive.
- Request/Response (RPC): Client calls service method
- Fire & Forget: Notification without reply
- Events / Publish-Subscribe: Periodic or on-change notifications
- Service Discovery (SOME/IP-SD): Dynamic find/offer services

SOME/IP Header (16 bytes):

```
┌──────────┬──────────┬──────────┬──────────┐
│ Service ID  │ Method ID│ Length   │ Client ID │
│ (2 bytes)   │ (2 bytes)│ (4 bytes)│ (2 bytes) │
├──────────┼──────────┼──────────┼──────────┤
│ Session ID  │ Proto Ver│ Iface Ver│ Msg Type  │
│ (2 bytes)   │ (1 byte) │ (1 byte) │ (1 byte)  │
├──────────┼──────────┴──────────┴──────────┤
│ Return Code │ Payload...                    │
│ (1 byte)    │                               │
└──────────┴──────────────────────────────┘
```

TEORİ: SOME/IP — IP Üzerinden Ölçeklenebilir Servis Odaklı Katman

SOME/IP, AUTOSAR Adaptive için fiili servis ara katıdır.
- İstek/Yanıt (RPC): İstemci servis metodunu çağırır
- Fire & Forget: Yanıt beklenmeden bildirim
- Olaylar / Pub-Sub: Periyodik veya değişiklik bazlı bildirimler
- Servis Keşfi (SOME/IP-SD): Dinamik servis bulma/sunma

SOME/IP Başlık (16 bayt):

```
┌──────────────┬──────────┬──────────┬──────────┐
│ Service ID   │ Method ID│ Length   │ Client ID │
│ (2 bayt)     │ (2 bayt) │ (4 bayt) │ (2 bayt)  │
├──────────────┼──────────┼──────────┼──────────┤
│ Session ID   │ Proto Ver│ Iface Ver│ Msg Type  │
│ (2 bayt)     │ (1 bayt) │ (1 bayt) │ (1 bayt)  │
├──────────────┼──────────┼──────────┼──────────┤
│ Return Code  │ Payload...                      │
│ (1 bayt)     │                                 │
└──────────────┴─────────────────────────────────┘
```

THEORY: DoIP — Diagnostics over IP

DoIP (ISO 13400) enables UDS diagnostic messages over Ethernet/TCP. Used by diagnostic testers to flash ECUs and read DTCs over Ethernet. Protocol: UDP port 13400 (discovery), TCP port 13400 (data).

TEORİ: DoIP — IP Üzerinden Teşhis

DoIP (ISO 13400), UDS teşhis mesajlarını Ethernet/TCP üzerinden taşır. Teşhis test cihazları tarafından ECU flash'ı ve DTC okuma için kullanılır. Protokol: UDP port 13400 (keşif), TCP port 13400 (veri).

THEORY: TSN — Time-Sensitive Networking

TSN is a set of IEEE 802.1 standards that add determinism to Ethernet:
- 802.1AS (gPTP): Generalized Precision Time Protocol (~100 ns sync)
- 802.1Qbv (TAS): Time-Aware Shaper — TDMA-like gate scheduling
- 802.1Qci (PSFP): Per-Stream Filtering and Policing
- 802.1CB (FRER): Frame Replication and Elimination for Redundancy
- 802.1Qav (CBS): Credit-Based Shaper for AVB streams

TEORİ: TSN — Zamana Duyarlı Ağ İletişimi

TSN, Ethernet'e FlexRay benzeri determinizm ekleyen IEEE 802.1 standartları kümesidir:
- 802.1AS (gPTP): Genelleştirilmiş Hassas Zaman Protokolü (~100 ns senkron)
- 802.1Qbv (TAS): Zamana Duyarlı Şekillendirici — TDMA benzeri kapı zamanlama
- 802.1Qci (PSFP): Akış Başına Filtreleme ve Denetleme
- 802.1CB (FRER): Yedeklilik için Çerçeve Çoğaltma ve Eleme
- 802.1Qav (CBS): AVB akışları için Kredi Tabanlı Şekillendirici

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <vector>
#include <string>
#include <iomanip>
#include <map>
#include <algorithm>
#include <cassert>
#include <array>
#include <sstream>
#include <functional>
#include <chrono>

// ═══════════════════════════════════════════════════════════════════════
// PART 1: Ethernet Frame & VLAN Tagging
// ═══════════════════════════════════════════════════════════════════════

using MacAddress = std::array<uint8_t, 6>;

std::string mac_to_string(const MacAddress& mac) {
    std::ostringstream oss;
    for (size_t i = 0; i < 6; ++i) {
        if (i > 0) oss << ":";
        oss << std::hex << std::setw(2) << std::setfill('0') << static_cast<int>(mac[i]);
    }
    return oss.str();
}

struct EthernetFrame {
    MacAddress dst_mac {};
    MacAddress src_mac {};
    uint16_t ethertype = 0x0800;  // IPv4 varsayılan (0x0800=IPv4, 0x86DD=IPv6, 0x88A8=VLAN)
    bool     has_vlan = false;
    uint16_t vlan_id = 0;         // 12-bit VLAN ID (trafik izolasyonu için)
    uint8_t  vlan_pcp = 0;        // 3-bit Öncelik Kod Noktası (0-7, 7=en yüksek)
    std::vector<uint8_t> payload;

    size_t frame_size() const {
        return 14 + (has_vlan ? 4 : 0) + payload.size() + 4; // başlık(14) + VLAN(4) + veri + FCS(4)
    }

    void print(const std::string& label) const {
        std::cout << "  [ETH] " << label << "\n";
        std::cout << "    DST: " << mac_to_string(dst_mac)
                  << " | SRC: " << mac_to_string(src_mac) << "\n";
        std::cout << "    EtherType: 0x" << std::hex << std::setw(4) << std::setfill('0')
                  << ethertype << std::dec;
        if (has_vlan) {
            std::cout << " | VLAN: " << vlan_id << " (PCP=" << static_cast<int>(vlan_pcp) << ")";
        }
        std::cout << " | Size: " << frame_size() << " bytes\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════
// PART 2: SOME/IP Message Structure
// ═══════════════════════════════════════════════════════════════════════

enum class SomeIpMessageType : uint8_t {
    Request         = 0x00,
    RequestNoReturn = 0x01,  // Fire & Forget
    Notification    = 0x02,
    Response        = 0x80,
    Error           = 0x81,
};

enum class SomeIpReturnCode : uint8_t {
    E_OK                = 0x00,
    E_NOT_OK            = 0x01,
    E_UNKNOWN_SERVICE   = 0x02,
    E_UNKNOWN_METHOD    = 0x03,
    E_NOT_READY         = 0x04,
    E_NOT_REACHABLE     = 0x05,
    E_TIMEOUT           = 0x06,
    E_WRONG_PROTOCOL    = 0x07,
```

```
    E_WRONG_INTERFACE   = 0x08,
    E_MALFORMED_MESSAGE = 0x09,
};

std::string to_string(SomeIpMessageType t) {
    switch (t) {
        case SomeIpMessageType::Request:         return "REQUEST";
        case SomeIpMessageType::RequestNoReturn: return "FIRE_AND_FORGET";
        case SomeIpMessageType::Notification:    return "NOTIFICATION";
        case SomeIpMessageType::Response:        return "RESPONSE";
        case SomeIpMessageType::Error:           return "ERROR";
    }
    return "UNKNOWN";
}

std::string to_string(SomeIpReturnCode c) {
    switch (c) {
        case SomeIpReturnCode::E_OK:                return "E_OK";
        case SomeIpReturnCode::E_NOT_OK:            return "E_NOT_OK";
        case SomeIpReturnCode::E_UNKNOWN_SERVICE:   return "E_UNKNOWN_SERVICE";
        case SomeIpReturnCode::E_UNKNOWN_METHOD:    return "E_UNKNOWN_METHOD";
        case SomeIpReturnCode::E_NOT_READY:         return "E_NOT_READY";
        case SomeIpReturnCode::E_NOT_REACHABLE:     return "E_NOT_REACHABLE";
        case SomeIpReturnCode::E_TIMEOUT:           return "E_TIMEOUT";
        case SomeIpReturnCode::E_WRONG_PROTOCOL:    return "E_WRONG_PROTOCOL";
        case SomeIpReturnCode::E_WRONG_INTERFACE:   return "E_WRONG_INTERFACE";
        case SomeIpReturnCode::E_MALFORMED_MESSAGE: return "E_MALFORMED_MESSAGE";
    }
    return "UNKNOWN";
}

struct SomeIpMessage {
    uint16_t service_id  = 0;
    uint16_t method_id   = 0;
    uint16_t client_id   = 0;
    uint16_t session_id  = 0;
    uint8_t  protocol_version = 1;
    uint8_t  interface_version = 1;
    SomeIpMessageType message_type = SomeIpMessageType::Request;
    SomeIpReturnCode  return_code  = SomeIpReturnCode::E_OK;
    std::vector<uint8_t> payload;

    uint32_t length() const {
        return static_cast<uint32_t>(8 + payload.size()); // header(8) + payload
    }

    void print(const std::string& label) const {
        std::cout << "  [SOME/IP] " << label << "\n";
        std::cout << "    Service: 0x" << std::hex << std::setw(4) << std::setfill('0') << service_id
                  << " Method: 0x" << std::setw(4) << method_id << std::dec
                  << " | Client: " << client_id << " Session: " << session_id << "\n";
        std::cout << "    Type: " << to_string(message_type)
                  << " | Return: " << to_string(return_code)
                  << " | PayLen: " << payload.size() << " bytes\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: SOME/IP Service Discovery (SOME/IP-SD)
// ═══════════════════════════════════════════════════════════════════════════

// EN: Service Discovery allows runtime find/offer of services.
//     - OfferService: Server announces availability
//     - FindService: Client searches for a service
//     - Subscribe: Client subscribes to event group
//     - SubscribeAck: Server confirms subscription
// TR: Service Discovery, servislerin çalışma zamanında bulunmasını sağlar.

enum class SDEntryType {
    FindService,
    OfferService,
    StopOfferService,
    SubscribeEventgroup,
    SubscribeEventgroupAck,
};
```

```cpp

std::string to_string(SDEntryType t) {
    switch (t) {
        case SDEntryType::FindService:              return "FIND_SERVICE";
        case SDEntryType::OfferService:             return "OFFER_SERVICE";
        case SDEntryType::StopOfferService:         return "STOP_OFFER";
        case SDEntryType::SubscribeEventgroup:      return "SUBSCRIBE_EG";
        case SDEntryType::SubscribeEventgroupAck:   return "SUBSCRIBE_ACK";
    }
    return "UNKNOWN";
}

struct SDEntry {
    SDEntryType type;
    uint16_t service_id;
    uint16_t instance_id;
    uint8_t  major_version;
    uint32_t ttl;  // Time-to-live in seconds (0 = stop)
    std::string endpoint_ip;
    uint16_t endpoint_port;
};

class SomeIpServiceDirectory {
    std::map<uint32_t, SDEntry> services_;  // key = (service_id << 16) | instance_id

    static uint32_t make_key(uint16_t sid, uint16_t iid) {
        return (static_cast<uint32_t>(sid) << 16) | iid;
    }

public:
    void offer(const SDEntry& entry) {
        services_[make_key(entry.service_id, entry.instance_id)] = entry;
    }

    void stop_offer(uint16_t service_id, uint16_t instance_id) {
        services_.erase(make_key(service_id, instance_id));
    }

    const SDEntry* find(uint16_t service_id, uint16_t instance_id = 0xFFFF) const {
        if (instance_id != 0xFFFF) {
            auto it = services_.find(make_key(service_id, instance_id));
            return (it != services_.end()) ? &it->second : nullptr;
        }
        // EN: Any instance
        // TR: Herhangi bir örnek — ilk eşleşen servisi döndür
        for (auto& [k, v] : services_) {
            if ((k >> 16) == service_id) return &v;
        }
        return nullptr;
    }

    void print_all() const {
        std::cout << "  ┌──────────┬──────────┬─────┬──────────────────┬─────┐\n";
        std::cout << "  │ Service  │ Instance │ Ver │ Endpoint         │ TTL │\n";
        std::cout << "  ├──────────┼──────────┼─────┼──────────────────┼─────┤\n";
        for (auto& [k, e] : services_) {
            std::cout << "  │ 0x" << std::hex << std::setw(4) << std::setfill('0') << e.service_id
                      << "   │ 0x" << std::setw(4) << e.instance_id << std::dec
                      << "   │ " << std::setw(4) << static_cast<int>(e.major_version)
                      << " │ " << std::left << std::setw(13) << (e.endpoint_ip + ":" + std::to_string(e.endpoint_port))
                      << std::right << " │ " << std::setw(5) << e.ttl << " │\n";
        }
        std::cout << "  └──────────┴──────────┴─────┴──────────────────┴─────┘\n";
    }
};

```

```cpp
// ═══════════════════════════════════════════════════════════════════════════
// PART 4: DoIP — Diagnostics over IP
// ═══════════════════════════════════════════════════════════════════════════

// EN: DoIP is used by diagnostic tools (e.g., OBD tester) to communicate
//     with ECUs over TCP/UDP. ISO 13400-2.
// TR: DoIP, teşhis araçlarının Ethernet üzerinden ECU'larla iletişim kurmasını sağlar.

enum class DoIPPayloadType : uint16_t {
    VehicleIdentRequest            = 0x0001,
    VehicleIdentResponse           = 0x0004,
    RoutingActivationRequest       = 0x0005,
    RoutingActivationResponse      = 0x0006,
    DiagnosticMessage              = 0x8001,
    DiagnosticMessageAck           = 0x8002,
    DiagnosticMessageNack          = 0x8003,
};

std::string to_string(DoIPPayloadType t) {
    switch (t) {
        case DoIPPayloadType::VehicleIdentRequest:       return "VEHICLE_IDENT_REQ";
        case DoIPPayloadType::VehicleIdentResponse:      return "VEHICLE_IDENT_RESP";
        case DoIPPayloadType::RoutingActivationRequest:  return "ROUTING_ACT_REQ";
        case DoIPPayloadType::RoutingActivationResponse: return "ROUTING_ACT_RESP";
        case DoIPPayloadType::DiagnosticMessage:         return "DIAG_MESSAGE";
        case DoIPPayloadType::DiagnosticMessageAck:      return "DIAG_MSG_ACK";
        case DoIPPayloadType::DiagnosticMessageNack:     return "DIAG_MSG_NACK";
    }
    return "UNKNOWN";
}

struct DoIPHeader {
    uint8_t protocol_version = 0x03;  // DoIP ISO 13400-2:2019
    uint8_t inverse_version  = 0xFC;  // ~protocol_version
    DoIPPayloadType payload_type;
    uint32_t payload_length = 0;
    std::vector<uint8_t> payload;

    void print(const std::string& label) const {
        std::cout << "  [DoIP] " << label << "\n";
        std::cout << "    Version: " << static_cast<int>(protocol_version)
                  << " | Type: " << to_string(payload_type)
                  << " | PayLen: " << payload_length << " bytes\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 5: TSN — Time-Aware Shaper (802.1Qbv)
// ═══════════════════════════════════════════════════════════════════════════

// EN: TAS uses gate control lists (GCL) to open/close traffic class queues
//     at precise time intervals. This creates TDMA-like determinism.
// TR: TAS, trafik sınıfı kuyruklarını hassas zaman aralıklarında açıp
//     kapatarak TDMA benzeri determinizm oluşturur.

struct GateControlEntry {
    uint8_t gate_states;      // 8 bits → 8 traffic classes (TC0-TC7), 1=open, 0=closed
    uint32_t time_interval_ns; // Duration in nanoseconds

    void print() const {
        std::cout << "      Gates: ";
        for (int i = 7; i >= 0; --i) {
            std::cout << "TC" << i << "=" << ((gate_states >> i) & 1 ? "O" : "C") << " ";
        }
        std::cout << "| Duration: " << time_interval_ns / 1000 << " µs\n";
    }
};

struct GateControlList {
    std::vector<GateControlEntry> entries;
    uint32_t cycle_time_ns = 0;  // Total cycle = sum of all intervals

    void add(uint8_t gates, uint32_t time_ns) {
        entries.push_back({gates, time_ns});
        cycle_time_ns += time_ns;
    }
```

```cpp

    void print() const {
        std::cout << "  TAS Gate Control List (cycle=" << cycle_time_ns / 1000 << " µs):\n";
        for (size_t i = 0; i < entries.size(); ++i) {
            std::cout << "    Slot " << i << ": ";
            entries[i].print();
        }
    }
};

// EN: Traffic class mapping for automotive
// TR: Otomotiv için trafik sınıfı eşlemesi — PCP önceliğine göre sınıflandırma
struct TrafficClassConfig {
    std::string name;
    uint8_t tc_index;       // 0-7
    uint8_t vlan_pcp;       // VLAN Priority Code Point
    std::string description;
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 6: Ethernet Switch Simulation
// ═══════════════════════════════════════════════════════════════════════════

class EthernetSwitch {
    std::string name_;
    std::map<MacAddress, int> mac_table_;  // MAC → port number
    int num_ports_;
    std::vector<std::string> port_names_;

public:
    EthernetSwitch(std::string name, int ports)
        : name_(std::move(name)), num_ports_(ports), port_names_(static_cast<size_t>(ports)) {}

    void set_port_name(int port, const std::string& name) {
        if (port >= 0 && port < num_ports_)
            port_names_[static_cast<size_t>(port)] = name;
    }

    void learn_mac(const MacAddress& mac, int port) {
        mac_table_[mac] = port;
    }

    int lookup(const MacAddress& mac) const {
        auto it = mac_table_.find(mac);
        return (it != mac_table_.end()) ? it->second : -1;  // -1 = flood
    }

    void forward(const EthernetFrame& frame) const {
        int dst_port = lookup(frame.dst_mac);
        std::cout << "    " << name_ << ": "
                  << mac_to_string(frame.src_mac) << " → " << mac_to_string(frame.dst_mac);
        if (dst_port >= 0) {
            std::cout << " → Port " << dst_port;
            if (!port_names_[static_cast<size_t>(dst_port)].empty())
                std::cout << " (" << port_names_[static_cast<size_t>(dst_port)] << ")";
            std::cout << " [UNICAST]\n";
        } else {
            std::cout << " → ALL ports [FLOOD]\n";
        }
    }

    void print_mac_table() const {
        std::cout << "  " << name_ << " MAC Table:\n";
        std::cout << "  ┌──────────────────┬──────┬──────────────┐\n";
        std::cout << "  │ MAC Address      │ Port │ Device       │\n";
        std::cout << "  ├──────────────────┼──────┼──────────────┤\n";
        for (auto& [mac, port] : mac_table_) {
            std::cout << "  │ " << mac_to_string(mac)
                      << " │ " << std::setw(4) << port
                      << " │ " << std::left << std::setw(15)
                      << (port >= 0 && port < num_ports_ ? port_names_[static_cast<size_t>(port)] : "")
                      << std::right << " │\n";
        }
        std::cout << "  └──────────────────┴──────┴──────────────┘\n";
    }
};
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 14 - Automotive Ethernet\n";
    std::cout << "============================================\n\n";

    // ─── Demo 1: Ethernet Frames & VLAN ──────────────────────────────
    {
        std::cout << "--- Demo 1: Automotive Ethernet Frames ---\n";

        // EN: ADAS camera frame (high priority, VLAN tagged)
        // TR: ADAS kamera çerçevesi (yüksek öncelik, VLAN etiketli)
        EthernetFrame camera_frame;
        camera_frame.dst_mac = {0x01, 0x00, 0x5E, 0x00, 0x00, 0x01};  // multicast
        camera_frame.src_mac = {0x00, 0x1A, 0x2B, 0x3C, 0x4D, 0x01};
        camera_frame.ethertype = 0x0800;
        camera_frame.has_vlan = true;
        camera_frame.vlan_id = 100;
        camera_frame.vlan_pcp = 6;  // high priority
        camera_frame.payload.resize(1400);  // camera image chunk
        camera_frame.print("ADAS Camera (VLAN 100, PCP=6 Critical)");

        // EN: CAN-over-Ethernet gateway frame
        // TR: CAN-üzerinden-Ethernet ağ geçidi çerçevesi
        EthernetFrame gw_frame;
        gw_frame.dst_mac = {0x00, 0x1A, 0x2B, 0x3C, 0x4D, 0x10};
        gw_frame.src_mac = {0x00, 0x1A, 0x2B, 0x3C, 0x4D, 0x02};
        gw_frame.ethertype = 0x0800;
        gw_frame.has_vlan = true;
        gw_frame.vlan_id = 200;
        gw_frame.vlan_pcp = 4;  // medium priority
        gw_frame.payload.resize(64);  // tunneled CAN frame
        gw_frame.print("CAN-to-Ethernet Gateway (VLAN 200, PCP=4)");

        // EN: OTA update frame (best effort)
        // TR: OTA güncelleme çerçevesi (en iyi çaba, düşük öncelik)
        EthernetFrame ota_frame;
        ota_frame.dst_mac = {0x00, 0x1A, 0x2B, 0x3C, 0x4D, 0x20};
        ota_frame.src_mac = {0x00, 0x1A, 0x2B, 0x3C, 0x4D, 0x03};
        ota_frame.ethertype = 0x0800;
        ota_frame.has_vlan = true;
        ota_frame.vlan_id = 300;
        ota_frame.vlan_pcp = 0;  // best effort
        ota_frame.payload.resize(1500);
        ota_frame.print("OTA Update (VLAN 300, PCP=0 Best Effort)");

        std::cout << "\n  VLAN PCP Priority Mapping (automotive):\n";
        std::cout << "    PCP 7: Network Control (switch management)\n";
        std::cout << "    PCP 6: Safety-Critical (ADAS, braking)\n";
        std::cout << "    PCP 5: Control Data (steering, powertrain)\n";
        std::cout << "    PCP 4: Audio/Video streaming\n";
        std::cout << "    PCP 3: Infotainment\n";
        std::cout << "    PCP 2: Medium priority\n";
        std::cout << "    PCP 1: Background\n";
        std::cout << "    PCP 0: Best Effort (OTA, logging)\n\n";
    }

    // ─── Demo 2: SOME/IP Request-Response ────────────────────────────
    {
        std::cout << "--- Demo 2: SOME/IP Request-Response ---\n";
        std::cout << "  Scenario: ADAS requests camera image from Camera ECU\n\n";

        // EN: Request: ADAS → Camera ECU
        // TR: İstek: ADAS → Kamera ECU — GetImage metodu çağrısı
        SomeIpMessage request;
        request.service_id = 0x1001;   // Camera Service
        request.method_id  = 0x0001;   // GetImage method
        request.client_id  = 0x0100;   // ADAS ECU
        request.session_id = 0x0001;
        request.message_type = SomeIpMessageType::Request;
        request.payload = {0x01, 0x00};  // camera_id=1, format=RAW
        request.print("Request: ADAS → Camera (GetImage)");

```

```
        // EN: Response: Camera ECU → ADAS
        // TR: Yanıt: Kamera ECU → ADAS — görüntü verisi döndürülür
        SomeIpMessage response;
        response.service_id = 0x1001;
        response.method_id  = 0x0001;
        response.client_id  = 0x0100;
        response.session_id = 0x0001;
        response.message_type = SomeIpMessageType::Response;
        response.return_code  = SomeIpReturnCode::E_OK;
        response.payload.resize(256);  // image data chunk
        response.print("Response: Camera → ADAS (ImageData)");

        // EN: Error case
        // TR: Hata durumu — bilinmeyen metod çağrısı
        SomeIpMessage error_msg;
        error_msg.service_id = 0x1001;
        error_msg.method_id  = 0x0002;  // non-existent method
        error_msg.client_id  = 0x0100;
        error_msg.session_id = 0x0002;
        error_msg.message_type = SomeIpMessageType::Error;
        error_msg.return_code  = SomeIpReturnCode::E_UNKNOWN_METHOD;
        error_msg.print("Error: Unknown method");
        std::cout << "\n";
    }

    // ─── Demo 3: SOME/IP-SD Service Discovery ──────────────────────────
    {
        std::cout << "--- Demo 3: SOME/IP-SD Service Discovery ---\n";

        SomeIpServiceDirectory directory;

        // EN: ECUs offer their services
        // TR: ECU'lar servislerini sunar — SOME/IP-SD ile keşif
        directory.offer({SDEntryType::OfferService, 0x1001, 0x0001, 1, 3600,
                         "192.168.1.10", 30490});
        directory.offer({SDEntryType::OfferService, 0x2001, 0x0001, 1, 3600,
                         "192.168.1.20", 30491});
        directory.offer({SDEntryType::OfferService, 0x3001, 0x0001, 2, 3600,
                         "192.168.1.30", 30492});

        std::cout << "  Offered services:\n";
        directory.print_all();

        // EN: Client finds a service
        // TR: İstemci bir servis arar — service_id ile sorgulama
        auto* cam = directory.find(0x1001);
        std::cout << "\n  Find 0x1001 (Camera): "
                  << (cam ? "FOUND at " + cam->endpoint_ip + ":" + std::to_string(cam->endpoint_port)
                          : "NOT FOUND") << "\n";

        auto* unknown = directory.find(0x9999);
        std::cout << "  Find 0x9999 (Unknown): "
                  << (unknown ? "FOUND" : "NOT FOUND") << "\n";

        // EN: Stop offer
        // TR: Servis sunumunu durdur — TTL=0 ile StopOffer mesajı
        directory.stop_offer(0x2001, 0x0001);
        std::cout << "\n  After StopOffer(0x2001):\n";
        directory.print_all();
        std::cout << "\n";
    }

    // ─── Demo 4: DoIP — Diagnostic Session ──────────────────────────────
    {
        std::cout << "--- Demo 4: DoIP Diagnostic Session ---\n";
        std::cout << "  Scenario: Tester connects to ECU over Ethernet\n\n";

        // EN: Step 1: Vehicle identification (UDP broadcast)
        // TR: Adım 1: Araç tanımlama (UDP yayını, port 13400)
        DoIPHeader ident_req;
        ident_req.payload_type = DoIPPayloadType::VehicleIdentRequest;
        ident_req.payload_length = 0;
        ident_req.print("Step 1: Vehicle Identification Request (UDP:13400)");

        DoIPHeader ident_resp;
        ident_resp.payload_type = DoIPPayloadType::VehicleIdentResponse;
```

```cpp
        // EN: VIN + logical address + EID + GID
        // TR: VIN + mantıksal adres + EID + GID — araç kimlik bilgileri
        ident_resp.payload = {
            'W', 'V', 'W', 'Z', 'Z', 'Z', '3', 'C', 'Z', 'W', 'E', '0', '0', '0', '0', '0', '1',  // VIN
            0x00, 0x10,  // logical address 0x0010
        };
        ident_resp.payload_length = static_cast<uint32_t>(ident_resp.payload.size());
        ident_resp.print("Step 2: Vehicle Identification Response");
        std::cout << "    VIN: WVWZZZ3CZWE000001, LogAddr: 0x0010\n";

        // EN: Step 3: Routing activation
        // TR: Adım 3: Yönlendirme aktivasyonu — TCP bağlantısı kurulur
        DoIPHeader routing_req;
        routing_req.payload_type = DoIPPayloadType::RoutingActivationRequest;
        routing_req.payload = {0x00, 0x01, 0x00};  // source_address, activation_type
        routing_req.payload_length = static_cast<uint32_t>(routing_req.payload.size());
        routing_req.print("Step 3: Routing Activation Request (TCP:13400)");

        DoIPHeader routing_resp;
        routing_resp.payload_type = DoIPPayloadType::RoutingActivationResponse;
        routing_resp.payload = {0x00, 0x01, 0x00, 0x10, 0x10};
        routing_resp.payload_length = static_cast<uint32_t>(routing_resp.payload.size());
        routing_resp.print("Step 4: Routing Activation Response (success)");

        // EN: Step 5: UDS DiagnosticMessage over DoIP
        // TR: Adım 5: DoIP üzerinden UDS teşhis mesajı (ReadDID, VIN okuma)
        DoIPHeader diag_msg;
        diag_msg.payload_type = DoIPPayloadType::DiagnosticMessage;
        diag_msg.payload = {
            0x00, 0x01,  // source address (tester)
            0x00, 0x10,  // target address (ECU)
            0x22, 0xF1, 0x90  // UDS: ReadDataByIdentifier, DID=0xF190 (VIN)
        };
        diag_msg.payload_length = static_cast<uint32_t>(diag_msg.payload.size());
        diag_msg.print("Step 5: UDS ReadDID(VIN) over DoIP");

        DoIPHeader diag_ack;
        diag_ack.payload_type = DoIPPayloadType::DiagnosticMessageAck;
        diag_ack.payload = {0x00, 0x10, 0x00};
        diag_ack.payload_length = static_cast<uint32_t>(diag_ack.payload.size());
        diag_ack.print("Step 6: Diagnostic Message ACK");
        std::cout << "\n";
    }

    // ─── Demo 5: TSN Time-Aware Shaper ──────────────────────────────
    {
        std::cout << "--- Demo 5: TSN Time-Aware Shaper (802.1Qbv) ---\n";

        // EN: Define traffic classes for automotive
        // TR: Otomotiv için trafik sınıflarını tanımla — öncelik bazlı kuyruklar
        std::vector<TrafficClassConfig> tc_config = {
            {"Safety-Critical", 7, 7, "ADAS control, braking (< 1ms)"},
            {"Control",         6, 6, "Steering, powertrain (< 5ms)"},
            {"Audio/Video",     5, 5, "Camera streams, surround view"},
            {"Diagnostics",     4, 4, "UDS/DoIP diagnostic messages"},
            {"Infotainment",    3, 3, "Navigation, media streaming"},
            {"OTA/Logging",     1, 1, "Software updates, data logging"},
            {"Best-Effort",     0, 0, "Non-critical background traffic"},
        };

        std::cout << "  Traffic Class Configuration:\n";
        std::cout << "  ┌──────┬──────────────────┬─────┬──────────────────────────────┐\n";
        std::cout << "  │ TC   │ Name             │ PCP │ Description                  │\n";
        std::cout << "  ├──────┼──────────────────┼─────┼──────────────────────────────┤\n";
        for (auto& tc : tc_config) {
            std::cout << "  │ TC" << static_cast<int>(tc.tc_index)
                      << "  │ " << std::left << std::setw(17) << tc.name
                      << " │ " << std::right << std::setw(3) << static_cast<int>(tc.vlan_pcp)
                      << " │ " << std::left << std::setw(31) << tc.description
                      << std::right << " │\n";
        }
        std::cout << "  └──────┴──────────────────┴─────┴──────────────────────────────┘\n\n";
```

```

        // EN: Gate Control List — deterministic schedule
        // TR: Kapı Kontrol Listesi — belirlenimci zamanlama, her slot farklı TC açar
        GateControlList gcl;
        // Slot 0: Safety only (TC7)
        gcl.add(0b10000000, 200'000);   // 200 µs — only TC7 open
        // Slot 1: Control (TC6+TC7)
        gcl.add(0b11000000, 300'000);   // 300 µs — TC6+TC7 open
        // Slot 2: AV Streams (TC5+TC6+TC7)
        gcl.add(0b11100000, 200'000);   // 200 µs
        // Slot 3: Everything (all TCs)
        gcl.add(0b11111111, 300'000);   // 300 µs — all open (best effort window)

        gcl.print();

        std::cout << "\n  TAS Timeline (1 cycle = " << gcl.cycle_time_ns / 1000 << " µs):\n";
        std::cout << "  |— Safety —|—— Control ——|—— AV ——|——— Best Effort ——|\n";
        std::cout << "  |  200µs   |   300µs     | 200µs  |     300µs        |\n";
        std::cout << "  Guaranteed: Safety data ALWAYS gets its 200µs window\n\n";
    }

    // ——— Demo 6: Ethernet Switch Forwarding ————————————————————————
    {
        std::cout << "--- Demo 6: Automotive Ethernet Switch ---\n";

        EthernetSwitch sw("ZonalSwitch_Front", 5);
        sw.set_port_name(0, "ADAS_ECU");
        sw.set_port_name(1, "Camera_FL");
        sw.set_port_name(2, "Camera_FR");
        sw.set_port_name(3, "Radar_Front");
        sw.set_port_name(4, "Uplink_Central");

        // EN: Learn MAC addresses
        // TR: MAC adreslerini öğren — anahtarlama tablosu oluştur
        MacAddress mac_adas   = {0x00, 0x1A, 0x2B, 0x3C, 0x4D, 0x01};
        MacAddress mac_cam_fl = {0x00, 0x1A, 0x2B, 0x3C, 0x4D, 0x02};
        MacAddress mac_cam_fr = {0x00, 0x1A, 0x2B, 0x3C, 0x4D, 0x03};
        MacAddress mac_radar  = {0x00, 0x1A, 0x2B, 0x3C, 0x4D, 0x04};
        MacAddress mac_central = {0x00, 0x1A, 0x2B, 0x3C, 0x4D, 0x10};
        MacAddress mac_unknown = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};

        sw.learn_mac(mac_adas, 0);
        sw.learn_mac(mac_cam_fl, 1);
        sw.learn_mac(mac_cam_fr, 2);
        sw.learn_mac(mac_radar, 3);
        sw.learn_mac(mac_central, 4);

        sw.print_mac_table();

        std::cout << "\n  Frame forwarding:\n";
        // Camera→ADAS (known unicast)
        EthernetFrame f1;
        f1.src_mac = mac_cam_fl;
        f1.dst_mac = mac_adas;
        sw.forward(f1);

        // ADAS→Central (uplink)
        EthernetFrame f2;
        f2.src_mac = mac_adas;
        f2.dst_mac = mac_central;
        sw.forward(f2);

        // Broadcast (flood)
        EthernetFrame f3;
        f3.src_mac = mac_radar;
        f3.dst_mac = mac_unknown;
        sw.forward(f3);
        std::cout << "\n";
    }
```

```cpp

    // ─── Demo 7: Network Topology — Zonal Architecture ─────────────────
    {
        std::cout << "--- Demo 7: Automotive Ethernet Topology ---\n\n";
        std::cout << "  Modern Zonal Architecture (Ethernet backbone):\n\n";
        std::cout << "  ┌──────────────────────────────────────────────────────┐\n";
        std::cout << "  │                  Central Compute (HPC)               │\n";
        std::cout << "  │                ┌──────────────────┐                  │\n";
        std::cout << "  │                │ Vehicle Computer │                  │\n";
        std::cout << "  │                │ (AUTOSAR Adaptive │                  │\n";
        std::cout << "  │                │  + Linux + SOME/IP) │                  │\n";
        std::cout << "  │                └──────────────────┘                  │\n";
        std::cout << "  │                        │                             │\n";
        std::cout << "  │                        │ 1Gbps Ethernet Backbone     │\n";
        std::cout << "  │      ┌─────────────────┼──────────────┬──────────┐   │\n";
        std::cout << "  │      │                 │              │          │   │\n";
        std::cout << "  │   ┌──┴──┐          ┌──┴──┐        ┌──┴──┐    ┌──┴──┐ │\n";
        std::cout << "  │   │Zone │          │Zone │        │Zone │    │Zone │ │\n";
        std::cout << "  │   │Front│          │Rear │        │Left │    │Right│ │\n";
        std::cout << "  │   │Switch│         │Switch│       │Switch│    │Switch│ │\n";
        std::cout << "  │   └──┬──┘          └──┬──┘        └──┬──┘    └──┬──┘ │\n";
        std::cout << "  │      │                │              │          │   │\n";
        std::cout << "  │  100BASE-T1       100BASE-T1     CAN/LIN     CAN/LIN  │\n";
        std::cout << "  │  Camera×2         Camera×2      Door ECU    Door ECU  │\n";
        std::cout << "  │  Radar            Ultrasonic     Window      Mirror   │\n";
        std::cout << "  │  Lidar            Parking        Seat        Light    │\n";
        std::cout << "  └──────────────────────────────────────────────────────┘\n\n";

        std::cout << "  Key features:\n";
        std::cout << "    - Central HPC replaces 100+ traditional ECUs\n";
        std::cout << "    - Zonal switches reduce wiring harness weight by 30-40%%\n";
        std::cout << "    - 1Gbps backbone handles ADAS + infotainment + body\n";
        std::cout << "    - CAN/LIN still used for simple sensors/actuators at edges\n";
        std::cout << "    - TSN ensures real-time performance for safety-critical data\n\n";
    }

    // ─── Demo 8: Protocol Comparison ──────────────────────────────────
    {
        std::cout << "--- Demo 8: Automotive Protocol Comparison ---\n\n";
        std::cout << "  ┌──────────────┬──────────┬──────────┬──────────┬──────────┬──────────┐\n";
        std::cout << "  | Property     | CAN      | CAN FD   | LIN      | FlexRay  | Ethernet |\n";
        std::cout << "  ├──────────────┼──────────┼──────────┼──────────┼──────────┼──────────┤\n";
        std::cout << "  | Speed        | 1 Mbps   | 8 Mbps   | 20 kbps  | 10 Mbps  | 100M-10G |\n";
        std::cout << "  | Payload      | 8 B      | 64 B     | 8 B      | 254 B    | 1500+ B  |\n";
        std::cout << "  | Topology     | Bus      | Bus      | Bus      | Bus/Star | Star     |\n";
        std::cout << "  | Determinism  | Priority | Priority | Master   | TDMA     | TSN      |\n";
        std::cout << "  | Redundancy   | No       | No       | No       | Dual-ch  | FRER     |\n";
        std::cout << "  | Wires        | 2        | 2        | 1        | 2/4      | 1 pair   |\n";
        std::cout << "  | Cost/node    | $1-3     | $3-5     | $0.5-1   | $10-20   | $5-15    |\n";
        std::cout << "  | ASIL level   | D        | D        | B        | D        | D (TSN)  |\n";
        std::cout << "  | Use case     | Body,    | Power-   | Comfort  | X-by-wire| ADAS,    |\n";
        std::cout << "  |              | chassis  | train    | sensors  | safety   | central  |\n";
        std::cout << "  └──────────────┴──────────┴──────────┴──────────┴──────────┴──────────┘\n";
    }

    std::cout << "\n==========================================\n";
    std::cout << "   End of Automotive Ethernet\n";
    std::cout << "==========================================\n";

    return 0;
}
```

► 05. Inter Mcu Communication

module_14_automotive_networks/05_inter_mcu_communication.cpp

Konu: Inter-MCU Communication — SPI, I2C, UART, Shared Memory

Teori / Theory

THEORY: Inter-MCU Communication in Automotive

While CAN/LIN/Ethernet handle inter-ECU communication, within a single ECU board, multiple MCUs/processors communicate via hardware peripherals:

```
┌──────────────────────────────────────────────┐
│ ECU Board (e.g., ADAS Domain Controller)     │
│                                              │
│  ┌──────────┐    SPI (50MHz)     ┌──────────┐ │
│  │ Main SoC │──────────────────→│ Safety MCU │ │
│  │(Linux/QNX)│    I2C (400kHz)   │ (ASIL-D)  │ │
│  │          │──────────────────→│           │ │
│  │          │    UART (115200)  │           │ │
│  │          │──────────────────→│           │ │
│  │          │                   └──────────┘ │
│  │          │    Shared Memory              │
│  │          │←──────────────→ SRAM (dual-port) │
│  └──────────┘                               │
└──────────────────────────────────────────────┘
```

Protocol Comparison:

Protocol	Speed	Wires	Topology	Use Case
SPI	50+ MHz	4 (MOSI MISO SCK,CS)	Master-Slave	Fast data (ADC, DAC, Flash, FPGA)
I2C	100k-3.4 MHz	2 (SDA, SCL)	Multi-Master	Low-speed sensors, EEPROMs, PMICs
UART	up to 4 Mbps	2 (TX,RX)	Point-to Point	Debug console, GPS, Bluetooth
Shared Memory	Bus speed (GBps)	N/A (on-chip)	N/A	Multi-core IPC, DMA transfer

TEORİ: Otomotivde MCU'lar Arası İletişim

CAN/LIN/Ethernet ECU'lar arasında haberleşmeyi sağlarken, tek bir ECU kartı üzerindeki birden fazla MCU/işlemci donanım çevre birimleri ile iletişim kurar:

```
┌──────────────────────────────────────────────────────┐
│ ECU Kartı (ör. ADAS Alan Denetleyicisi)               │
│                                                      │
│  ┌──────────┐    SPI (50MHz)     ┌──────────┐        │
│  │ Ana SoC  │────────────────────→│ Güvenlik  │        │
│  │(Linux/QNX)│    I2C (400kHz)    │ MCU(ASIL-D)│        │
│  │          │────────────────────→│           │        │
│  │          │    UART (115200)   │           │        │
│  │          │────────────────────→│           │        │
│  │          │                    └──────────┘        │
│  │          │    Paylaşımlı Bellek                   │
│  │          │←────────────────────→ SRAM (çift-port)  │
│  └──────────┘                                        │
└──────────────────────────────────────────────────────┘
```

Protokol Karşılaştırması:

```
┌──────────┬──────────┬──────────┬──────────┬──────────────────────┐
│ Protokol │ Hız      │ Kablo    │ Topoloji │ Kullanım Alanı       │
├──────────┼──────────┼──────────┼──────────┼──────────────────────┤
│ SPI      │ 50+ MHz  │ 4 (MOSI  │ Master-  │ Hızlı veri (ADC,DAC, │
│          │          │  MISO    │ Slave    │  Flash, FPGA)        │
│          │          │  SCK,CS) │          │                      │
├──────────┼──────────┼──────────┼──────────┼──────────────────────┤
│ I2C      │ 100k-    │ 2 (SDA,  │ Çoklu    │ Düşük hızlı sensör,  │
│          │ 3.4 MHz  │  SCL)    │ Master   │  EEPROM, PMIC        │
├──────────┼──────────┼──────────┼──────────┼──────────────────────┤
│ UART     │ 4 Mbps'e │ 2 (TX,RX)│ Noktadan │ Debug konsol,        │
│          │ kadar    │          │ Noktaya  │  GPS, Bluetooth      │
├──────────┼──────────┼──────────┼──────────┼──────────────────────┤
│ Paylaşım │ Bus hızı │ Yok      │ Yok      │ Çok çekirdekli IPC,  │
│ Bellek   │ (GBps)   │ (çip içi)│          │  DMA transferi       │
└──────────┴──────────┴──────────┴──────────┴──────────────────────┘
```

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <vector>
#include <string>
#include <iomanip>
#include <array>
#include <queue>
#include <algorithm>
#include <cassert>
#include <functional>
#include <map>
#include <cstring>

```

```cpp
// ═══════════════════════════════════════════════════════════════════════════
// PART 1: SPI — Serial Peripheral Interface
// ═══════════════════════════════════════════════════════════════════════════

// EN: SPI uses 4 wires: MOSI (Master Out Slave In), MISO (Master In Slave Out),
//     SCK (Serial Clock), CS/SS (Chip Select, active low).
//     Full-duplex: Master sends on MOSI, receives on MISO simultaneously.
//     Clock modes: CPOL (idle level) + CPHA (sampling edge) → 4 modes (0-3).
// TR: SPI: 4 kablo (MOSI, MISO, SCK, CS), tam çift yönlü, master-slave mimarisi.
//     Master MOSI'üzerinde gönderir, MISO'üzerinde aynı anda alır.
//     Saat modları: CPOL (boşta seviye) + CPHA (örnekleme kenarı) → 4 mod (0-3).

enum class SPIMode : uint8_t {
    Mode0 = 0,  // CPOL=0, CPHA=0 — idle low, sample on leading edge
                // TR: boşta düşük, yükselen kenarda örnekle
    Mode1 = 1,  // CPOL=0, CPHA=1 — idle low, sample on trailing edge
                // TR: boşta düşük, düşen kenarda örnekle
    Mode2 = 2,  // CPOL=1, CPHA=0 — idle high, sample on leading edge
                // TR: boşta yüksek, düşen kenarda örnekle
    Mode3 = 3,  // CPOL=1, CPHA=1 — idle high, sample on trailing edge
                // TR: boşta yüksek, yükselen kenarda örnekle
};

class SPIMaster {
    std::string name_;
    SPIMode mode_;
    uint32_t clock_hz_;
    int num_slaves_;
    int active_cs_ = -1;

public:
    SPIMaster(std::string name, SPIMode mode, uint32_t clock_hz, int num_slaves)
        : name_(std::move(name)), mode_(mode), clock_hz_(clock_hz), num_slaves_(num_slaves) {}

    bool select_slave(int cs) {
        if (cs < 0 || cs >= num_slaves_) return false;
        active_cs_ = cs;
        return true;
    }

    void deselect() { active_cs_ = -1; }

    // EN: SPI is full-duplex: send tx_data, receive rx_data simultaneously
    // TR: SPI tam çift yönlü: tx_data gönderilirken rx_data aynı anda alınır
    std::vector<uint8_t> transfer(const std::vector<uint8_t>& tx_data,
                                  std::function<uint8_t(uint8_t)> slave_handler) {
        std::vector<uint8_t> rx_data;
        rx_data.reserve(tx_data.size());
        for (auto byte : tx_data) {
            rx_data.push_back(slave_handler(byte));
        }
        return rx_data;
    }

    void print_config() const {
        std::cout << "    SPI Master: " << name_ << "\n";
        std::cout << "      Clock: " << clock_hz_ / 1'000'000 << " MHz"
                  << " | Mode: " << static_cast<int>(mode_)
                  << " (CPOL=" << (static_cast<int>(mode_) >> 1)
                  << " CPHA=" << (static_cast<int>(mode_) & 1) << ")"
                  << " | Slaves: " << num_slaves_
                  << " | Active CS: " << active_cs_ << "\n";
    }
};
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: I2C — Inter-Integrated Circuit
// ═══════════════════════════════════════════════════════════════════════════

// EN: I2C uses 2 wires: SDA (data) + SCL (clock). Open-drain with pull-ups.
//     7-bit addressing (128 devices). Multi-master capable.
//     Transaction: START → Address+R/W → ACK → Data → ACK → ... → STOP
// TR: I2C: 2 kablo (SDA + SCL), açık-drain + pull-up, 7-bit adresleme (128 cihaz).
//     Çoklu master desteği. İşlem: START → Adres+R/W → ACK → Veri → ACK → STOP

enum class I2CSpeed : uint32_t {
    Standard   = 100'000,    // 100 kHz   — TR: Standart mod
    Fast       = 400'000,    // 400 kHz   — TR: Hızlı mod
    FastPlus   = 1'000'000,  // 1 MHz     — TR: Hızlı+ mod
    HighSpeed  = 3'400'000,  // 3.4 MHz   — TR: Yüksek Hız modu
};

std::string to_string(I2CSpeed spd) {
    switch (spd) {
        case I2CSpeed::Standard:  return "Standard (100kHz)";
        case I2CSpeed::Fast:      return "Fast (400kHz)";
        case I2CSpeed::FastPlus:  return "Fast+ (1MHz)";
        case I2CSpeed::HighSpeed: return "High-Speed (3.4MHz)";
    }
    return "Unknown";
}

struct I2CDevice {
    uint8_t address;  // 7-bit — TR: 7-bit I2C adresi (0x00-0x7F)
    std::string name;
    std::map<uint8_t, uint8_t> registers;  // register_addr → value
                                           // TR: yazmaç_adresi → değer eşleme tablosu

    void write_register(uint8_t reg, uint8_t value) {
        registers[reg] = value;
    }

    uint8_t read_register(uint8_t reg) const {
        auto it = registers.find(reg);
        return (it != registers.end()) ? it->second : 0xFF;
    }
};

class I2CBus {
    std::string name_;
    I2CSpeed speed_;
    std::map<uint8_t, I2CDevice*> devices_;

public:
    I2CBus(std::string name, I2CSpeed speed) : name_(std::move(name)), speed_(speed) {}

    bool attach(I2CDevice& dev) {
        if (devices_.count(dev.address)) return false;  // address conflict — TR: adres çakışması
        devices_[dev.address] = &dev;
        return true;
    }

    bool write(uint8_t addr, uint8_t reg, uint8_t data) {
        auto it = devices_.find(addr);
        if (it == devices_.end()) return false;  // NACK — TR: cihaz yanıt vermedi
        it->second->write_register(reg, data);
        return true;  // ACK — TR: cihaz onayladı
    }

    bool read(uint8_t addr, uint8_t reg, uint8_t& data) {
        auto it = devices_.find(addr);
        if (it == devices_.end()) return false;
        data = it->second->read_register(reg);
        return true;
    }
```

```cpp

    // EN: Bus scan — detect all devices
    // TR: Bus tarama — bağlı tüm cihazları tespit et (ACK yanıtı kontrolü)
    std::vector<uint8_t> scan() const {
        std::vector<uint8_t> found;
        for (auto& [addr, dev] : devices_) {
            found.push_back(addr);
        }
        return found;
    }

    void print_config() const {
        std::cout << "    I2C Bus: " << name_
                  << " | Speed: " << to_string(speed_)
                  << " | Devices: " << devices_.size() << "\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: UART — Universal Asynchronous Receiver-Transmitter
// ═══════════════════════════════════════════════════════════════════════════

// EN: UART uses 2 wires (TX, RX), asynchronous (no clock wire).
//     Frame: [START(1)] [DATA(5-9)] [PARITY(0-1)] [STOP(1-2)]
//     Common config: 8N1 (8 data, No parity, 1 stop bit)
// TR: UART: 2 kablo (TX, RX), asenkron (saat kablosu yok).
//     Çerçeve: [START(1)] [VERİ(5-9)] [PARİTE(0-1)] [STOP(1-2)]
//     Yaygın konfigürasyon: 8N1 (8 veri, parite yok, 1 stop bit)

enum class UARTParity { None, Even, Odd };

struct UARTConfig {
    uint32_t baud_rate  = 115200;
    uint8_t  data_bits  = 8;
    UARTParity parity   = UARTParity::None;
    uint8_t  stop_bits  = 1;

    std::string config_string() const {
        std::string s = std::to_string(data_bits);
        switch (parity) {
            case UARTParity::None: s += "N"; break;
            case UARTParity::Even: s += "E"; break;
            case UARTParity::Odd:  s += "O"; break;
        }
        s += std::to_string(stop_bits);
        return s;  // e.g. "8N1" — TR: ör. "8N1" (8 veri, parite yok, 1 stop)
    }
};

class UARTPort {
    std::string name_;
    UARTConfig config_;
    std::queue<uint8_t> tx_buffer_;
    std::queue<uint8_t> rx_buffer_;

public:
    UARTPort(std::string name, UARTConfig config)
        : name_(std::move(name)), config_(config) {}

    void send(const std::vector<uint8_t>& data) {
        for (auto b : data) tx_buffer_.push(b);
    }

    void send_string(const std::string& str) {
        for (char c : str) tx_buffer_.push(static_cast<uint8_t>(c));
    }

    // EN: Connect two UARTs (TX of one → RX of other)
    // TR: İki UART bağla (birinin TX'ı diğerinin RX'ına)
    void connect_to(UARTPort& other) {
        while (!tx_buffer_.empty()) {
            other.rx_buffer_.push(tx_buffer_.front());
            tx_buffer_.pop();
        }
    }

```

```
225     std::vector<uint8_t> receive() {
226         std::vector<uint8_t> data;
227         while (!rx_buffer_.empty()) {
228             data.push_back(rx_buffer_.front());
229             rx_buffer_.pop();
230         }
231         return data;
232     }
233 
234     std::string receive_string() {
235         auto data = receive();
236         return std::string(data.begin(), data.end());
237     }
238 
239     size_t rx_available() const { return rx_buffer_.size(); }
240 
241     void print_config() const {
242         std::cout << "    UART: " << name_
243                   << " | Baud: " << config_.baud_rate
244                   << " | Config: " << config_.config_string() << "\n";
245     }
246 };
247 
248 // ═══════════════════════════════════════════════════════════════════════
249 // PART 4: Shared Memory IPC
250 // ═══════════════════════════════════════════════════════════════════════
251 
252 // EN: Multi-core MCUs often share SRAM for inter-processor communication.
253 //     Critical: needs memory barriers, mailbox interrupts, or spinlocks.
254 //     Common pattern: ring buffer with producer/consumer + doorbell interrupt.
255 // TR: Çok çekirdekli MCU'lar paylaşımlı SRAM ile haberleşir.
256 //     Kritik: bellek bariyerleri, mailbox kesmeleri veya spinlock gerekir.
257 //     Yaygın desen: üretici/tüketici + doorbell kesmesi ile halka tampon.
258 
259 struct SharedMemoryMailbox {
260     static constexpr size_t BUFFER_SIZE = 256;
261     static constexpr size_t MSG_MAX_SIZE = 64;
262 
263     struct Message {
264         uint8_t type;
265         uint8_t length;
266         std::array<uint8_t, MSG_MAX_SIZE> data {};
267 
268         void print(const std::string& label) const {
269             std::cout << "      " << label << " Type:0x"
270                       << std::hex << std::setw(2) << std::setfill('0')
271                       << static_cast<int>(type) << std::dec
272                       << " Len:" << static_cast<int>(length) << " Data:";
273             for (int i = 0; i < length && i < 16; ++i)
274                 std::cout << " " << std::hex << std::setw(2) << std::setfill('0')
275                           << static_cast<int>(data[static_cast<size_t>(i)]);
276             std::cout << std::dec << "\n";
277         }
278     };
279 
280 private:
281     std::queue<Message> core0_to_core1_;
282     std::queue<Message> core1_to_core0_;
283     uint32_t msg_count_01_ = 0;
284     uint32_t msg_count_10_ = 0;
285 
286 public:
287     // EN: Core 0 sends to Core 1
288     // TR: Çekirdek 0, Çekirdek 1'e mesaj gönderir
289     bool send_0to1(uint8_t type, const std::vector<uint8_t>& data) {
290         if (data.size() > MSG_MAX_SIZE) return false;
291         Message msg;
292         msg.type = type;
293         msg.length = static_cast<uint8_t>(data.size());
294         std::copy(data.begin(), data.end(), msg.data.begin());
295         core0_to_core1_.push(msg);
296         ++msg_count_01_;
297         return true;
298     }
```

```cpp

    bool send_1to0(uint8_t type, const std::vector<uint8_t>& data) {
        if (data.size() > MSG_MAX_SIZE) return false;
        Message msg;
        msg.type = type;
        msg.length = static_cast<uint8_t>(data.size());
        std::copy(data.begin(), data.end(), msg.data.begin());
        core1_to_core0_.push(msg);
        ++msg_count_10_;
        return true;
    }

    // EN: Core 1 reads from Core 0
    // TR: Çekirdek 1, Çekirdek 0'dan okur
    bool receive_on_core1(Message& msg) {
        if (core0_to_core1_.empty()) return false;
        msg = core0_to_core1_.front();
        core0_to_core1_.pop();
        return true;
    }

    bool receive_on_core0(Message& msg) {
        if (core1_to_core0_.empty()) return false;
        msg = core1_to_core0_.front();
        core1_to_core0_.pop();
        return true;
    }

    void print_stats() const {
        std::cout << "    Mailbox stats: Core0→Core1: " << msg_count_01_
                  << " msgs | Core1→Core0: " << msg_count_10_ << " msgs\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 5: Message Framing Protocol
// ═══════════════════════════════════════════════════════════════════════════

// EN: Raw UART/SPI byte streams need framing to detect message boundaries.
//     Common patterns:
//     1) Length-delimited: [HEADER][LENGTH][PAYLOAD][CHECKSUM]
//     2) SLIP/COBS: Byte stuffing to avoid delimiter collisions
//     3) Fixed-length: All messages same size (padding)
// TR: Ham UART/SPI bayt akışlarında mesaj sınırlarını belirlemek için çerçeveleme gerekir.
//     Yaygın desenler:
//     1) Uzunluk-sınırlı: [BAŞLIK][UZUNLUK][VERİ][SAĞLAMA]
//     2) SLIP/COBS: Sınırlayıcı çakışmasını önlemek için bayt doldurma
//     3) Sabit uzunluk: Tüm mesajlar aynı boyutta (dolgulu)

struct FramedMessage {
    static constexpr uint8_t SYNC_BYTE = 0xAA;
    static constexpr uint8_t VERSION   = 0x01;

    uint8_t  msg_id;
    uint8_t  length;
    std::vector<uint8_t> payload;
    uint8_t  checksum;  // XOR of all bytes (msg_id + length + payload)
                        // TR: tüm baytların XOR'u (msg_id + uzunluk + veri)

    static uint8_t calc_checksum(uint8_t id, uint8_t len, const std::vector<uint8_t>& data) {
        uint8_t cs = id ^ len;
        for (auto b : data) cs ^= b;
        return cs;
    }

    // EN: Build a framed message
    // TR: Çerçeveli mesaj oluştur — ID, veri ve sağlama toplamı hesapla
    static FramedMessage build(uint8_t id, const std::vector<uint8_t>& data) {
        FramedMessage msg;
        msg.msg_id = id;
        msg.length = static_cast<uint8_t>(data.size());
        msg.payload = data;
        msg.checksum = calc_checksum(id, msg.length, data);
        return msg;
    }

```

```cpp
    bool verify() const {
        return checksum == calc_checksum(msg_id, length, payload);
    }

    // EN: Serialize to byte stream
    // TR: Bayt akışına serileştir — SYNC + VER + ID + LEN + veri + checksum
    std::vector<uint8_t> serialize() const {
        std::vector<uint8_t> bytes;
        bytes.push_back(SYNC_BYTE);
        bytes.push_back(VERSION);
        bytes.push_back(msg_id);
        bytes.push_back(length);
        bytes.insert(bytes.end(), payload.begin(), payload.end());
        bytes.push_back(checksum);
        return bytes;
    }

    // EN: Deserialize from byte stream
    // TR: Bayt akışından çözümle — SYNC/VER doğrula, veriyi çıkar, checksum kontrol et
    static bool deserialize(const std::vector<uint8_t>& bytes, FramedMessage& msg) {
        if (bytes.size() < 5) return false;  // minimum: SYNC+VER+ID+LEN+CS
                                             // TR: minimum: SYNC+VER+ID+UZUNLUK+SAĞLAMA
        if (bytes[0] != SYNC_BYTE) return false;
        if (bytes[1] != VERSION) return false;
        msg.msg_id = bytes[2];
        msg.length = bytes[3];
        if (bytes.size() < static_cast<size_t>(4 + msg.length + 1)) return false;
        msg.payload.assign(bytes.begin() + 4, bytes.begin() + 4 + msg.length);
        msg.checksum = bytes[static_cast<size_t>(4 + msg.length)];
        return msg.verify();
    }

    void print(const std::string& label) const {
        std::cout << "    " << label << " ID:0x"
                  << std::hex << std::setw(2) << std::setfill('0')
                  << static_cast<int>(msg_id) << std::dec
                  << " Len:" << static_cast<int>(length)
                  << " CS:0x" << std::hex << std::setw(2) << std::setfill('0')
                  << static_cast<int>(checksum) << std::dec
                  << " Valid:" << (verify() ? "YES ✓" : "NO ✗") << "\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 14 - Inter-MCU Communication\n";
    std::cout << "============================================\n\n";

    // ─── Demo 1: SPI Master-Slave Transfer ──────────────────────────
    {
        std::cout << "--- Demo 1: SPI Master-Slave Transfer ---\n";
        std::cout << "  Scenario: SoC reads accelerometer via SPI\n\n";

        SPIMaster spi("SPI1", SPIMode::Mode0, 10'000'000, 3);
        spi.print_config();

        // EN: Select accelerometer (CS=0) and read WHO_AM_I register
        // TR: İvmeölçerü seç (CS=0) ve WHO_AM_I kaydını oku
        spi.select_slave(0);

        // EN: Simulated accelerometer: responds with register data
        // TR: Simüle ivmeölçer: kayıt verisiyle yanıt verir
        // Read command: 0x80 | reg_addr (bit 7 = read flag)
        // TR: Okuma komutu: 0x80 | yazmaç_adresi (bit 7 = okuma bayrağı)
        std::map<uint8_t, uint8_t> accel_regs = {
            {0x0F, 0x33},  // WHO_AM_I = 0x33 (LIS3DH) — TR: cihaz kimlik kaydı
            {0x28, 0xF0},  // OUT_X_L  — TR: X ekseni alt bayt
            {0x29, 0x01},  // OUT_X_H → X = 0x01F0 = 496 → ~0.12g — TR: X ekseni üst bayt
            {0x2A, 0x20},  // OUT_Y_L  — TR: Y ekseni alt bayt
            {0x2B, 0x00},  // OUT_Y_H → Y = 0x0020 = 32 — TR: Y ekseni üst bayt
            {0x2C, 0x00},  // OUT_Z_L  — TR: Z ekseni alt bayt
            {0x2D, 0x10},  // OUT_Z_H → Z = 0x1000 = 4096 → ~1.0g — TR: Z ekseni üst bayt
        };

```

```
        uint8_t last_addr = 0;
        auto slave_handler = [&](uint8_t tx) -> uint8_t {
            if (tx & 0x80) {
                // Read command — TR: Okuma komutu (bit 7 set)
                last_addr = tx & 0x7F;
                return 0xFF;  // dummy byte during address phase
                              // TR: adres fazında kukla bayt
            }
            // Data phase: return register value
            // TR: Veri fazı: yazmaç değerini döndür
            auto it = accel_regs.find(last_addr);
            uint8_t val = (it != accel_regs.end()) ? it->second : 0x00;
            ++last_addr;
            return val;
        };

        // EN: Read WHO_AM_I (addr 0x0F)
        // TR: WHO_AM_I oku (adres 0x0F) — cihaz kimliği doğrulaması
        auto rx1 = spi.transfer({0x8F, 0x00}, slave_handler);
        std::cout << "    WHO_AM_I: TX={0x8F, 0x00} → RX={0x"
                  << std::hex << std::setw(2) << std::setfill('0') << static_cast<int>(rx1[0])
                  << ", 0x" << std::setw(2) << static_cast<int>(rx1[1])
                  << "} → ID=0x" << std::setw(2) << static_cast<int>(rx1[1])
                  << std::dec << " (LIS3DH ✓)\n";

        // EN: Read 6 bytes of XYZ data (burst read from 0x28)
        // TR: 6 bayt XYZ verisi oku (0x28'den burst okuma)
        auto rx2 = spi.transfer({0xE8, 0, 0, 0, 0, 0, 0}, slave_handler);
        int16_t x = static_cast<int16_t>(static_cast<uint16_t>(rx2[2]) << 8 | rx2[1]);
        int16_t y = static_cast<int16_t>(static_cast<uint16_t>(rx2[4]) << 8 | rx2[3]);
        int16_t z = static_cast<int16_t>(static_cast<uint16_t>(rx2[6]) << 8 | rx2[5]);
        std::cout << "    Accel XYZ: X=" << x << " Y=" << y << " Z=" << z << "\n";
        std::cout << "    (Z ≈ 1g → correctly oriented, face-up)\n\n";

        spi.deselect();
    }

    // ── Demo 2: SPI Clock Modes ──────────────────────────────────────
    {
        std::cout << "--- Demo 2: SPI Clock Modes ---\n\n";
        std::cout << "  ┌──────┬──────┬──────┬──────────────────────┬────────────────────┐\n";
        std::cout << "  │ Mode │ CPOL │ CPHA │ Timing               │ Common Devices     │\n";
        std::cout << "  ├──────┼──────┼──────┼──────────────────────┼────────────────────┤\n";
        std::cout << "  │  0   │  0   │  0   │ Idle=LOW, Sample=Rise │ Most sensors       │\n";
        std::cout << "  │  1   │  0   │  1   │ Idle=LOW, Sample=Fall │ Some ADCs          │\n";
        std::cout << "  │  2   │  1   │  0   │ Idle=HIGH,Sample=Fall │ Some DACs          │\n";
        std::cout << "  │  3   │  1   │  1   │ Idle=HIGH,Sample=Rise │ SD cards, Flash    │\n";
        std::cout << "  └──────┴──────┴──────┴──────────────────────┴────────────────────┘\n\n";

        std::cout << "  SPI Signal Diagram (Mode 0, CPOL=0, CPHA=0):\n";
        std::cout << "                    ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐\n";
        std::cout << "  SCK:      ────┘ └─┘ └─┘ └─┘ └─┘ └─┘ └─┘ └─┘ └────\n";
        std::cout << "  CS:       ──┐                                   ┌──\n";
        std::cout << "                └───────────────────────────────────┘\n";
        std::cout << "  MOSI:     ── D7 ─ D6 ─ D5 ─ D4 ─ D3 ─ D2 ─ D1 ─ D0 ──\n";
        std::cout << "  MISO:     ── D7 ─ D6 ─ D5 ─ D4 ─ D3 ─ D2 ─ D1 ─ D0 ──\n";
        std::cout << "  Sample:        ↑    ↑    ↑    ↑    ↑    ↑    ↑    ↑\n\n";
    }

    // ── Demo 3: I2C Bus Scan & Register Access ───────────────────────
    {
        std::cout << "--- Demo 3: I2C Bus Scan & Register Access ---\n";
        std::cout << "  Scenario: SoC communicates with sensors on I2C bus\n\n";

        I2CBus bus("I2C1", I2CSpeed::Fast);
        bus.print_config();

```

```
        // EN: Devices on the bus
        // TR: Bus üzerindeki cihazlar
        I2CDevice temp_sensor;
        temp_sensor.address = 0x48;  // TMP102 — TR: sıcaklık sensörü I2C adresi
        temp_sensor.name = "TMP102 (Temp)";
        temp_sensor.write_register(0x00, 75);   // Temperature = 75 (raw) — TR: ham sıcaklık değeri
        temp_sensor.write_register(0x01, 0x60);  // Config register — TR: yapılandırma kaydı

        I2CDevice accel;
        accel.address = 0x1D;  // ADXL345 — TR: ivmeölçer I2C adresi
        accel.name = "ADXL345 (Accel)";
        accel.write_register(0x00, 0xE5);  // DEVID = 0xE5 — TR: cihaz kimlik değeri

        I2CDevice eeprom;
        eeprom.address = 0x50;  // AT24C02 — TR: EEPROM I2C adresi
        eeprom.name = "AT24C02 (EEPROM)";
        eeprom.write_register(0x00, 0xCA);
        eeprom.write_register(0x01, 0xFE);

        I2CDevice pmic;
        pmic.address = 0x68;  // PMIC — TR: güç yönetim IC'si I2C adresi
        pmic.name = "PMIC (Power)";
        pmic.write_register(0x00, 0x01);  // Status: OK — TR: durum: normal

        bus.attach(temp_sensor);
        bus.attach(accel);
        bus.attach(eeprom);
        bus.attach(pmic);

        // EN: I2C bus scan
        // TR: I2C bus taraması — tüm adreslere ACK sorgusu
        auto found = bus.scan();
        std::cout << "\n    I2C Bus scan results:\n";
        std::cout << "    ┌──────────┬──────────────────────────┐\n";
        std::cout << "    │ Address  │ Device                   │\n";
        std::cout << "    ├──────────┼──────────────────────────┤\n";

        std::map<uint8_t, std::string> addr_names = {
            {0x48, "TMP102 (Temperature)"}, {0x1D, "ADXL345 (Accelerometer)"},
            {0x50, "AT24C02 (EEPROM)"},     {0x68, "PMIC (Power Mgmt)"}
        };
        for (auto addr : found) {
            std::cout << "    │ 0x" << std::hex << std::setw(2) << std::setfill('0')
                      << static_cast<int>(addr) << std::dec
                      << "     │ " << std::left << std::setw(24) << addr_names[addr]
                      << std::right << " │ ACK ✓\n";
        }
        std::cout << "    └──────────┴──────────────────────────┘\n";

        // EN: Read temperature sensor
        // TR: Sıcaklık sensörü oku — TMP102, ham değer × 0.5 = °C
        uint8_t temp_val = 0;
        bus.read(0x48, 0x00, temp_val);
        std::cout << "\n    Read TMP102 [0x48] reg 0x00: " << static_cast<int>(temp_val)
                  << " → " << (static_cast<double>(temp_val) * 0.5) << " °C\n";

        // EN: Write EEPROM
        // TR: EEPROM yaz — AT24C02, bayt bazlı yazma/okuma
        bus.write(0x50, 0x10, 0xBE);
        uint8_t ee_val = 0;
        bus.read(0x50, 0x10, ee_val);
        std::cout << "    Write/Read EEPROM [0x50] reg 0x10: 0x"
                  << std::hex << std::setw(2) << std::setfill('0')
                  << static_cast<int>(ee_val) << std::dec << " ✓\n\n";
    }
```

```cpp

    // ─── Demo 4: UART Point-to-Point ──────────────────────────────────
    {
        std::cout << "--- Demo 4: UART Point-to-Point Communication ---\n";
        std::cout << "  Scenario: MCU ↔ GPS module via UART\n\n";

        UARTConfig gps_config{9600, 8, UARTParity::None, 1};
        UARTPort mcu_uart("MCU_UART2", gps_config);
        UARTPort gps_uart("GPS_Module", gps_config);

        mcu_uart.print_config();
        gps_uart.print_config();

        // EN: GPS sends NMEA sentence to MCU
        // TR: GPS, MCU'ya NMEA cümlesi gönderir
        gps_uart.send_string("$GPGGA,123519,4807.038,N,01131.000,E,1,08,0.9,545.4,M,47.0,M,,*47\r\n");
        gps_uart.connect_to(mcu_uart);

        std::string nmea = mcu_uart.receive_string();
        std::cout << "\n    GPS → MCU: " << nmea.substr(0, nmea.size() - 2) << "\n";  // strip \r\n
                                                                                          // TR: \r\n kırp

        // EN: MCU sends configuration command to GPS
        // TR: MCU, GPS'çe yapılandırma komutu gönderir
        mcu_uart.send_string("$PMTK314,0,1,0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0*28\r\n");
        mcu_uart.connect_to(gps_uart);

        std::string cmd = gps_uart.receive_string();
        std::cout << "    MCU → GPS: " << cmd.substr(0, cmd.size() - 2) << "\n\n";
    }

    // ─── Demo 5: Shared Memory IPC ────────────────────────────────────
    {
        std::cout << "--- Demo 5: Shared Memory IPC (Multi-Core MCU) ---\n";
        std::cout << "  Scenario: Core0 (Linux) ↔ Core1 (Safety RTOS) via mailbox\n\n";

        SharedMemoryMailbox mailbox;

        // EN: Core0 → Core1: Send control commands
        // TR: Çekirdek0 → Çekirdek1: Kontrol komutları gönder
        mailbox.send_0to1(0x01, {0x10, 0x20, 0x30});  // Start motor — TR: Motor başlat
        mailbox.send_0to1(0x02, {0x00, 0x64});         // Set speed = 100 — TR: Hız = 100 ayarla
        mailbox.send_0to1(0x03, {0x01});               // Enable watchdog — TR: Watchdog etkinleştir

        std::cout << "    Core0 → Core1 (commands sent):\n";

        // EN: Core1 receives and processes
        // TR: Çekirdek1 alır ve işler
        SharedMemoryMailbox::Message msg;
        int count = 0;
        while (mailbox.receive_on_core1(msg)) {
            msg.print("Received on Core1:");
            ++count;
        }
        std::cout << "    Total received: " << count << "\n\n";

        // EN: Core1 → Core0: Send status updates
        // TR: Çekirdek1 → Çekirdek0: Durum güncellemeleri gönder
        mailbox.send_1to0(0x81, {0x00, 0x01, 0x64});  // Status: OK, motor running, speed=100
                                                       // TR: Durum: OK, motor çalışıyor, hız=100
        mailbox.send_1to0(0x82, {0x25, 0x00});         // Temperature: 37°C — TR: Sıcaklık: 37°C

        std::cout << "    Core1 → Core0 (status sent):\n";
        while (mailbox.receive_on_core0(msg)) {
            msg.print("Received on Core0:");
        }

        mailbox.print_stats();
        std::cout << "\n";
    }
```

```cpp

    // ─── Demo 6: Message Framing Protocol ──────────────────────────────
    {
        std::cout << "--- Demo 6: Message Framing Protocol (UART) ---\n\n";

        // EN: Frame format: [SYNC=0xAA][VER=0x01][ID][LEN][PAYLOAD...][CHECKSUM]
        // TR: Çerçeve formatı: [SYNC=0xAA][VER=0x01][ID][UZUNLUK][VERİ...][SAĞLAMA]
        std::cout << "  Frame format:\n";
        std::cout << "  ┌──────┬──────┬──────┬──────┬──────────────┬──────────┐\n";
        std::cout << "  │ SYNC │ VER  │ ID   │ LEN  │ PAYLOAD      │ CHECKSUM │\n";
        std::cout << "  │ 0xAA │ 0x01 │ 1B   │ 1B   │ 0-255 bytes  │ XOR      │\n";
        std::cout << "  └──────┴──────┴──────┴──────┴──────────────┴──────────┘\n\n";

        // EN: Build and serialize messages
        // TR: Mesajları oluştur ve serileştir
        auto msg1 = FramedMessage::build(0x10, {0x01, 0x02, 0x03, 0x04});
        msg1.print("Msg1 (MotorCmd):");

        auto msg2 = FramedMessage::build(0x20, {0x00, 0x64, 0x25});
        msg2.print("Msg2 (SensorData):");

        // EN: Serialize → transmit → deserialize
        // TR: Serileştir → ilet → çözümle
        auto bytes1 = msg1.serialize();
        std::cout << "    Serialized:";
        for (auto b : bytes1)
            std::cout << " " << std::hex << std::setw(2) << std::setfill('0')
                      << static_cast<int>(b);
        std::cout << std::dec << "\n";

        FramedMessage decoded;
        bool ok = FramedMessage::deserialize(bytes1, decoded);
        decoded.print("Decoded:");
        std::cout << "    Decode result: " << (ok ? "SUCCESS ✓" : "FAILED ✗") << "\n";

        // EN: Corrupted message
        // TR: Bozulmuş mesaj — checksum hatası tespit edilmeli
        auto bad_bytes = bytes1;
        bad_bytes[4] = 0xFF;  // corrupt payload — TR: veriyi boz (checksum hatası oluştur)
        FramedMessage bad_decoded;
        bool bad_ok = FramedMessage::deserialize(bad_bytes, bad_decoded);
        std::cout << "    Corrupted decode: " << (bad_ok ? "SUCCESS (BAD!)" : "FAILED ✓ (corruption detected)")
                  << "\n\n";
    }

    // ─── Demo 7: Automotive ECU Internal Architecture ──────────────────
    {
        std::cout << "--- Demo 7: Automotive ECU Internal Architecture ---\n\n";
        std::cout << "  Typical ADAS Domain Controller (internal buses):\n\n";
        std::cout << "  ┌──────────────────────────────────────────────────────────┐\n";
        std::cout << "  │ ADAS ECU Board                                           │\n";
        std::cout << "  │                                                           │\n";
        std::cout << "  │  ┌──────────────┐      SPI (50MHz)      ┌──────────────┐  │\n";
        std::cout << "  │  │ Main SoC     │──────────────────────→│ Safety MCU   │  │\n";
        std::cout << "  │  │ (S32G/TC397) │      I2C (400kHz)     │ (RL78/STM32) │  │\n";
        std::cout << "  │  │              │──────────────────────→│ ASIL-D       │  │\n";
        std::cout << "  │  │              │      UART (debug)     │              │  │\n";
        std::cout << "  │  │              │──────────────────────→│              │  │\n";
        std::cout << "  │  │              │                        └──────────────┘  │\n";
        std::cout << "  │  │              │      SPI (20MHz)                         │\n";
        std::cout << "  │  │              │──────────────────────→ SPI Flash (FW)    │\n";
        std::cout << "  │  │              │      I2C (100kHz)                        │\n";
        std::cout << "  │  │              │──────────────────────→ EEPROM (config)   │\n";
        std::cout << "  │  │              │      I2C (100kHz)                        │\n";
        std::cout << "  │  │              │──────────────────────→ PMIC (power)      │\n";
        std::cout << "  │  │              │      SPI (1MHz)                          │\n";
        std::cout << "  │  │              │──────────────────────→ CAN Transceiver   │\n";
        std::cout << "  │  │              │                        (MCP2515/TJA1145) │\n";
        std::cout << "  │  └──────────────┘                                         │\n";
        std::cout << "  │                                                           │\n";
        std::cout << "  │  External Connectors:                                     │\n";
        std::cout << "  │    CAN FD × 2  |  LIN × 1  |  100BASE-T1 × 4  |  MIPI × 2 │\n";
        std::cout << "  └──────────────────────────────────────────────────────────┘\n\n";
    }

```

```
    std::cout << "==========================================\n";
    std::cout << "   End of Inter-MCU Communication\n";
    std::cout << "==========================================\n";

    return 0;
}
```

► 06. Uds Diagnostics

module_14_automotive_networks/06_uds_diagnostics.cpp

Konu: UDS — Unified Diagnostic Services (ISO 14229) & ISO-TP (ISO 15765)

Teori / Theory

THEORY: UDS — Unified Diagnostic Services

UDS (ISO 14229) is the diagnostic protocol used in all modern vehicles. Diagnostic testers, OBD tools, and factory tools use UDS to:
- Read/clear DTCs (Diagnostic Trouble Codes)
- Flash ECU firmware (reprogramming)
- Read/write ECU data (VIN, calibration, configuration)
- Control actuators (test mode)
- Security access (seed-key authentication)

UDS Service Catalog (key SIDs):

SID	Service Name	Use Case
0x10	DiagnosticSessionControl	Switch session mode
0x11	ECUReset	Hard/soft reset ECU
0x14	ClearDiagnosticInformation	Clear DTCs
0x19	ReadDTCInformation	Read fault codes
0x22	ReadDataByIdentifier	Read DID (VIN, etc.)
0x23	ReadMemoryByAddress	Direct memory read
0x27	SecurityAccess	Seed-key unlock
0x2E	WriteDataByIdentifier	Write DID
0x2F	InputOutputControlByID	Actuator control
0x31	RoutineControl	Run diagnostics
0x34	RequestDownload	Start flash session
0x36	TransferData	Send flash data
0x37	RequestTransferExit	End flash session
0x3E	TesterPresent	Keep session alive
0x85	ControlDTCSetting	Enable/Disable DTCs

UDS Sessions:
- 0x01: Default Session (limited services)
- 0x02: Programming Session (flash/download)
- 0x03: Extended Diagnostic Session (full access)

Negative Response Codes (NRC):

NRC	Meaning
0x10	General Reject
0x11	Service Not Supported
0x12	Sub-Function Not Supported
0x13	Incorrect Message Length
0x14	Response Too Long
0x22	Conditions Not Correct
0x24	Request Sequence Error
0x25	No Response From Sub-Net Component
0x31	Request Out Of Range
0x33	Security Access Denied
0x35	Invalid Key
0x36	Exceeded Number Of Attempts
0x72	General Programming Failure
0x78	Request Correctly Received, Response Pending

TEORİ: UDS — Birleşik Teşhis Hizmetleri

UDS (ISO 14229) tüm modern araçlarda kullanılan teşhis protokolüdür. Teşhis cihazları, OBD araçları ve fabrika araçları UDS ile:
- DTC okuma/silme (Arıza Kodları)
- ECU yazılımı güncelleme (yeniden programlama)
- ECU verisi okuma/yazma (VIN, kalibrasyon, konfigürasyon)
- Aktüatör kontrolü (test modu)
- Güvenlik erişimi (tohum-anahtar doğrulama)

UDS Servis Kataloğu (temel SID'ılar):

SID	Servis Adı	Kullanım Alanı
0x10	DiagnosticSessionControl	Oturum modu değiştir
0x11	ECUReset	ECU sıfırla(sert/yumuşak)
0x14	ClearDiagnosticInformation	DTC'ıları temizle
0x19	ReadDTCInformation	Arıza kodlarını oku
0x22	ReadDataByIdentifier	DID oku (VIN vb.)
0x23	ReadMemoryByAddress	Doğrudan bellek oku
0x27	SecurityAccess	Tohum-anahtar kilit aç
0x2E	WriteDataByIdentifier	DID yaz
0x2F	InputOutputControlByID	Aktüatör kontrolü
0x31	RoutineControl	Teşhis rutini çalıştır
0x34	RequestDownload	Flash oturumu başlat
0x36	TransferData	Flash verisi gönder
0x37	RequestTransferExit	Flash oturumu sonlandır
0x3E	TesterPresent	Oturumu canlı tut
0x85	ControlDTCSetting	DTC etkin/devre dışı

UDS Oturumları:
- 0x01: Varsayılan Oturum (sınırlı hizmetler)
- 0x02: Programlama Oturumu (flash/indirme)
- 0x03: Genişletilmiş Teşhis Oturumu (tam erişim)

Olumsuz Yanıt Kodları (NRC):

NRC	Anlam
0x10	Genel Red
0x11	Servis Desteklenmiyor
0x12	Alt Fonksiyon Desteklenmiyor
0x13	Yanlış Mesaj Uzunluğu
0x14	Yanıt Çok Uzun
0x22	Koşullar Uygun Değil
0x24	İstek Sırası Hatası
0x25	Alt Ağ Bileşeninden Yanıt Yok
0x31	İstek Aralık Dışı
0x33	Güvenlik Erişimi Reddedildi
0x35	Geçersiz Anahtar
0x36	Deneme Sayısı Aşıldı
0x72	Genel Programlama Hatası
0x78	İstek Doğru Alındı, Yanıt Beklemede

THEORY: ISO-TP — Transport Protocol (ISO 15765-2)

UDS messages can exceed 8 bytes (CAN) or 64 bytes (CAN FD). ISO-TP handles segmentation into multiple CAN frames:
Frame Types:
- SF (Single Frame): ≤7 bytes, fits in one CAN frame
- FF (First Frame): First segment of multi-frame message (total length)
- CF (Consecutive): Subsequent segments (sequence number 0-F)
- FC (Flow Control): Receiver tells sender: block size + timing

TEORİ: ISO-TP — Taşıma Protokolü (ISO 15765-2)

UDS mesajları CAN frame boyutunu (8/64 bayt) aşabilir. ISO-TP segmentasyon sağlar: Çerçeve Türleri:
- SF (Tek Çerçeve): ≤7 bayt, tek CAN frame'çe sığar
- FF (İlk Çerçeve): Çok-çerçeveli mesajın ilk segmenti (toplam uzunluk)
- CF (Ardışık): Sonraki segmentler (sıra numarası 0-F)
- FC (Akış Kontrolü): Alıcı gönderene bildirir: blok boyutu + zamanlama

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <vector>
#include <string>
#include <iomanip>
#include <map>
#include <algorithm>
#include <cassert>
#include <functional>
#include <array>
#include <sstream>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: UDS Core — SID, NRC, Sessions
// ═══════════════════════════════════════════════════════════════════════════

enum class UDSSession : uint8_t {
    Default     = 0x01,
    Programming = 0x02,
    Extended    = 0x03,
};

// EN: UDS Service IDs
// TR: UDS Servis Kimlikleri — teşhis işlemlerini tanımlayan sabit kodlar
namespace SID {
    constexpr uint8_t DiagnosticSessionControl = 0x10;
    constexpr uint8_t ECUReset                 = 0x11;
    constexpr uint8_t ClearDTC                 = 0x14;
    constexpr uint8_t ReadDTCInformation       = 0x19;
    constexpr uint8_t ReadDataByIdentifier     = 0x22;
    constexpr uint8_t ReadMemoryByAddress      = 0x23;
    constexpr uint8_t SecurityAccess           = 0x27;
    constexpr uint8_t WriteDataByIdentifier    = 0x2E;
    constexpr uint8_t IOControl                = 0x2F;
    constexpr uint8_t RoutineControl           = 0x31;
    constexpr uint8_t RequestDownload          = 0x34;
    constexpr uint8_t TransferData             = 0x36;
    constexpr uint8_t RequestTransferExit      = 0x37;
    constexpr uint8_t TesterPresent            = 0x3E;
    constexpr uint8_t ControlDTCSetting        = 0x85;
    constexpr uint8_t NegativeResponse         = 0x7F;
    constexpr uint8_t PositiveOffset           = 0x40; // positive response = SID + 0x40
                                                       // TR: olumlu yanıt = SID + 0x40
}
```

```cpp

// EN: Negative Response Codes
// TR: Olumsuz Yanıt Kodları — hata nedenini belirten NRC değerleri
namespace NRC {
    constexpr uint8_t GeneralReject           = 0x10;
    constexpr uint8_t ServiceNotSupported     = 0x11;
    constexpr uint8_t SubFunctionNotSupported = 0x12;
    constexpr uint8_t IncorrectMessageLength  = 0x13;
    constexpr uint8_t ConditionsNotCorrect    = 0x22;
    constexpr uint8_t RequestSequenceError    = 0x24;
    constexpr uint8_t RequestOutOfRange       = 0x31;
    constexpr uint8_t SecurityAccessDenied    = 0x33;
    constexpr uint8_t InvalidKey              = 0x35;
    constexpr uint8_t ExceededAttempts        = 0x36;
    constexpr uint8_t ResponsePending         = 0x78;
}

std::string nrc_to_string(uint8_t nrc) {
    switch (nrc) {
        case NRC::GeneralReject:           return "General Reject";
        case NRC::ServiceNotSupported:     return "Service Not Supported";
        case NRC::SubFunctionNotSupported: return "Sub-Function Not Supported";
        case NRC::IncorrectMessageLength:  return "Incorrect Message Length";
        case NRC::ConditionsNotCorrect:    return "Conditions Not Correct";
        case NRC::RequestSequenceError:    return "Request Sequence Error";
        case NRC::RequestOutOfRange:       return "Request Out Of Range";
        case NRC::SecurityAccessDenied:    return "Security Access Denied";
        case NRC::InvalidKey:              return "Invalid Key";
        case NRC::ExceededAttempts:        return "Exceeded Attempts";
        case NRC::ResponsePending:         return "Response Pending";
        default: return "Unknown NRC (0x" +
            ([](uint8_t v) {
                std::ostringstream o;
                o << std::hex << std::setw(2) << std::setfill('0') << static_cast<int>(v);
                return o.str();
            })(nrc) + ")";
    }
}

std::string sid_to_string(uint8_t sid) {
    switch (sid) {
        case SID::DiagnosticSessionControl: return "DiagnosticSessionControl";
        case SID::ECUReset:                 return "ECUReset";
        case SID::ClearDTC:                 return "ClearDTC";
        case SID::ReadDTCInformation:       return "ReadDTCInformation";
        case SID::ReadDataByIdentifier:     return "ReadDataByIdentifier";
        case SID::SecurityAccess:           return "SecurityAccess";
        case SID::WriteDataByIdentifier:    return "WriteDataByIdentifier";
        case SID::IOControl:                return "IOControl";
        case SID::RoutineControl:           return "RoutineControl";
        case SID::RequestDownload:          return "RequestDownload";
        case SID::TransferData:             return "TransferData";
        case SID::RequestTransferExit:      return "RequestTransferExit";
        case SID::TesterPresent:            return "TesterPresent";
        case SID::ControlDTCSetting:        return "ControlDTCSetting";
        default: return "Unknown";
    }
}

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: UDS ECU Server Simulation
// ═══════════════════════════════════════════════════════════════════════════

struct DTC {
    uint32_t code;        // 3-byte DTC (e.g., 0x012345) — TR: 3 baytlık arıza kodu
    uint8_t  status;      // DTC status mask — TR: DTC durum maskesi (aktif, onaylı vb.)
    std::string description;
};
```

```cpp

class UDSServer {
    std::string name_;
    UDSSession session_ = UDSSession::Default;
    bool security_unlocked_ = false;
    uint16_t seed_ = 0;

    // EN: Data Identifiers (DIDs)
    // TR: Veri Tanımlayıcıları (DID) — VIN, parça no, yazılım sürümü vb.
    std::map<uint16_t, std::vector<uint8_t>> dids_;
    // EN: DTC storage
    // TR: DTC deposu — aktif hata kodlarını tutar
    std::vector<DTC> dtc_list_;

    // EN: Build negative response
    // TR: Olumsuz yanıt oluştur — [0x7F, SID, NRC] formatında
    static std::vector<uint8_t> negative(uint8_t sid, uint8_t nrc) {
        return {SID::NegativeResponse, sid, nrc};
    }

    // EN: Build positive response
    // TR: Olumlu yanıt oluştur — SID + 0x40 ile başlar
    static std::vector<uint8_t> positive(uint8_t sid, const std::vector<uint8_t>& data = {}) {
        std::vector<uint8_t> resp;
        resp.push_back(static_cast<uint8_t>(sid + SID::PositiveOffset));
        resp.insert(resp.end(), data.begin(), data.end());
        return resp;
    }

public:
    UDSServer(std::string name) : name_(std::move(name)) {
        // EN: Pre-populate DIDs
        // TR: DID'ıları önceden doldur — VIN, parça no, donanım/yazılım sürümü
        // DID 0xF190 = VIN
        // TR: DID 0xF190 = Araç Kimlik Numarası (VIN)
        std::string vin = "WVWZZZ3CZWE000001";
        dids_[0xF190] = std::vector<uint8_t>(vin.begin(), vin.end());
        // DID 0xF187 = Part Number
        // TR: DID 0xF187 = Parça Numarası
        std::string part = "8V0907115A";
        dids_[0xF187] = std::vector<uint8_t>(part.begin(), part.end());
        // DID 0xF191 = ECU Hardware Version
        // TR: DID 0xF191 = ECU Donanım Sürümü
        dids_[0xF191] = {0x01, 0x03};
        // DID 0xF195 = ECU Software Version
        // TR: DID 0xF195 = ECU Yazılım Sürümü
        dids_[0xF195] = {0x02, 0x01, 0x00};
    }

    void add_dtc(uint32_t code, uint8_t status, const std::string& desc) {
        dtc_list_.push_back({code, status, desc});
    }

    std::vector<uint8_t> handle_request(const std::vector<uint8_t>& request) {
        if (request.empty()) return negative(0x00, NRC::IncorrectMessageLength);

        uint8_t sid = request[0];

        switch (sid) {
            case SID::DiagnosticSessionControl: {
                if (request.size() < 2) return negative(sid, NRC::IncorrectMessageLength);
                uint8_t sub = request[1];
                if (sub >= 0x01 && sub <= 0x03) {
                    session_ = static_cast<UDSSession>(sub);
                    security_unlocked_ = false;
                    return positive(sid, {sub});
                }
                return negative(sid, NRC::SubFunctionNotSupported);
            }

            case SID::TesterPresent: {
                return positive(sid, {0x00});
            }

```

```cpp
            case SID::ReadDataByIdentifier: {
                if (request.size() < 3) return negative(sid, NRC::IncorrectMessageLength);
                uint16_t did = static_cast<uint16_t>((static_cast<uint16_t>(request[1]) << 8) | request[2]);
                auto it = dids_.find(did);
                if (it == dids_.end()) return negative(sid, NRC::RequestOutOfRange);
                std::vector<uint8_t> data = {request[1], request[2]};
                data.insert(data.end(), it->second.begin(), it->second.end());
                return positive(sid, data);
            }

            case SID::WriteDataByIdentifier: {
                if (session_ != UDSSession::Extended)
                    return negative(sid, NRC::ConditionsNotCorrect);
                if (!security_unlocked_)
                    return negative(sid, NRC::SecurityAccessDenied);
                if (request.size() < 4) return negative(sid, NRC::IncorrectMessageLength);
                uint16_t did = static_cast<uint16_t>((static_cast<uint16_t>(request[1]) << 8) | request[2]);
                dids_[did] = std::vector<uint8_t>(request.begin() + 3, request.end());
                return positive(sid, {request[1], request[2]});
            }

            case SID::SecurityAccess: {
                if (request.size() < 2) return negative(sid, NRC::IncorrectMessageLength);
                uint8_t sub = request[1];
                if (sub == 0x01) {
                    // EN: Request Seed
                    // TR: Tohum iste — güvenlik erişimi için ilk adım
                    seed_ = 0xA5B7;  // simple fixed seed for demo
                                     // TR: demo için sabit tohum değeri
                    return positive(sid, {0x01,
                        static_cast<uint8_t>(seed_ >> 8),
                        static_cast<uint8_t>(seed_ & 0xFF)});
                }
                if (sub == 0x02) {
                    // EN: Send Key (expected: seed XOR 0x1234)
                    // TR: Anahtar gönder (beklenen: seed XOR 0x1234)
                    if (request.size() < 4) return negative(sid, NRC::IncorrectMessageLength);
                    uint16_t key = static_cast<uint16_t>((static_cast<uint16_t>(request[2]) << 8) | request[3]);
                    uint16_t expected = seed_ ^ 0x1234;
                    if (key == expected) {
                        security_unlocked_ = true;
                        return positive(sid, {0x02});
                    }
                    return negative(sid, NRC::InvalidKey);
                }
                return negative(sid, NRC::SubFunctionNotSupported);
            }

            case SID::ReadDTCInformation: {
                if (request.size() < 2) return negative(sid, NRC::IncorrectMessageLength);
                uint8_t sub = request[1];
                if (sub == 0x01) {
                    // EN: Report number of DTCs by status mask
                    // TR: Durum maskesine göre DTC sayısını raporla
                    uint8_t count = static_cast<uint8_t>(dtc_list_.size());
                    return positive(sid, {0x01, 0xFF, 0x00, count});
                }
                if (sub == 0x02) {
                    // EN: Report DTC by status mask
                    // TR: Durum maskesine göre DTC'ıları listele
                    std::vector<uint8_t> data = {0x02, 0xFF};
                    for (auto& dtc : dtc_list_) {
                        data.push_back(static_cast<uint8_t>((dtc.code >> 16) & 0xFF));
                        data.push_back(static_cast<uint8_t>((dtc.code >> 8) & 0xFF));
                        data.push_back(static_cast<uint8_t>(dtc.code & 0xFF));
                        data.push_back(dtc.status);
                    }
                    return positive(sid, data);
                }
                return negative(sid, NRC::SubFunctionNotSupported);
            }

```

```
            case SID::ClearDTC: {
                dtc_list_.clear();
                return positive(sid);
            }

            case SID::ECUReset: {
                if (request.size() < 2) return negative(sid, NRC::IncorrectMessageLength);
                session_ = UDSSession::Default;
                security_unlocked_ = false;
                return positive(sid, {request[1]});
            }

            default:
                return negative(sid, NRC::ServiceNotSupported);
        }
    }

    const std::string& name() const { return name_; }
};

// ============================================================================
// PART 3: ISO-TP Segmentation (ISO 15765-2)
// ============================================================================

enum class ISOTPFrameType : uint8_t {
    SingleFrame     = 0x0,
    FirstFrame      = 0x1,
    ConsecutiveFrame = 0x2,
    FlowControl     = 0x3,
};

struct ISOTPFrame {
    ISOTPFrameType type;
    uint8_t        sequence_number = 0;   // for CF — TR: Ardışık Çerçeve sıra numarası
    uint16_t       total_length = 0;      // for FF — TR: İlk Çerçeve toplam mesaj uzunluğu
    uint8_t        block_size = 0;        // for FC — TR: Akış Kontrolü blok boyutu
    uint8_t        st_min = 0;            // for FC (separation time min, ms)
                                          // TR: Akış Kontrolü minimum ayırma süresi (ms)
    std::vector<uint8_t> data;

    void print(const std::string& label) const {
        std::cout << "    " << label << " [";
        switch (type) {
            case ISOTPFrameType::SingleFrame:     std::cout << "SF"; break;
            case ISOTPFrameType::FirstFrame:      std::cout << "FF"; break;
            case ISOTPFrameType::ConsecutiveFrame: std::cout << "CF#" << static_cast<int>(sequence_number); break;
            case ISOTPFrameType::FlowControl:     std::cout << "FC"; break;
        }
        std::cout << "] ";
        if (type == ISOTPFrameType::FirstFrame)
            std::cout << "TotalLen=" << total_length << " ";
        if (type == ISOTPFrameType::FlowControl)
            std::cout << "BS=" << static_cast<int>(block_size) << " STmin=" << static_cast<int>(st_min) << "ms ";
        std::cout << "Data:";
        for (auto b : data)
            std::cout << " " << std::hex << std::setw(2) << std::setfill('0') << static_cast<int>(b);
        std::cout << std::dec << "\n";
    }
};

class ISOTPCodec {
public:
    // EN: Segment a message into ISO-TP frames (CAN: max 8 bytes per frame)
    // TR: Mesajı ISO-TP çerçevelerine böl (CAN: çerçeve başına maks 8 bayt)
    static std::vector<ISOTPFrame> segment(const std::vector<uint8_t>& message) {
        std::vector<ISOTPFrame> frames;

```

```cpp
        if (message.size() <= 7) {
            // EN: Single Frame
            // TR: Tek Çerçeve — ≤7 bayt, dolgu ile 8 bayta tamamlanır
            ISOTPFrame sf;
            sf.type = ISOTPFrameType::SingleFrame;
            sf.data.push_back(static_cast<uint8_t>(message.size()));  // PCI byte: 0x0N
                                                                       // TR: PCI baytı: 0x0N (uzunluk)
            sf.data.insert(sf.data.end(), message.begin(), message.end());
            // Pad to 8 bytes — TR: 8 bayta doldur
            while (sf.data.size() < 8) sf.data.push_back(0xCC);
            frames.push_back(sf);
        } else {
            // EN: First Frame
            // TR: İlk Çerçeve — toplam uzunluk + ilk 6 veri baytı
            ISOTPFrame ff;
            ff.type = ISOTPFrameType::FirstFrame;
            ff.total_length = static_cast<uint16_t>(message.size());
            // PCI: 0x1NNN (N = total length, 12-bit)
            // TR: PCI: 0x1NNN (N = toplam uzunluk, 12-bit)
            ff.data.push_back(static_cast<uint8_t>(0x10 | ((message.size() >> 8) & 0x0F)));
            ff.data.push_back(static_cast<uint8_t>(message.size() & 0xFF));
            // First 6 data bytes — TR: İlk 6 veri baytı
            size_t offset = 0;
            while (ff.data.size() < 8 && offset < message.size()) {
                ff.data.push_back(message[offset++]);
            }
            frames.push_back(ff);

            // EN: Consecutive Frames
            // TR: Ardışık Çerçeveler — sıra numarası ile kalan veriyi taşır
            uint8_t sn = 1;
            while (offset < message.size()) {
                ISOTPFrame cf;
                cf.type = ISOTPFrameType::ConsecutiveFrame;
                cf.sequence_number = sn;
                cf.data.push_back(static_cast<uint8_t>(0x20 | (sn & 0x0F)));
                while (cf.data.size() < 8 && offset < message.size()) {
                    cf.data.push_back(message[offset++]);
                }
                while (cf.data.size() < 8) cf.data.push_back(0xCC);
                frames.push_back(cf);
                sn = (sn + 1) & 0x0F;
            }
        }
        return frames;
    }

```

```cpp
    // EN: Reassemble ISO-TP frames back into a message
    // TR: ISO-TP çerçevelerini tekrar mesaja birleştir
    static std::vector<uint8_t> reassemble(const std::vector<ISOTPFrame>& frames) {
        if (frames.empty()) return {};

        auto& first = frames[0];
        if (first.type == ISOTPFrameType::SingleFrame) {
            uint8_t len = first.data[0] & 0x0F;
            return std::vector<uint8_t>(first.data.begin() + 1, first.data.begin() + 1 + len);
        }

        // First Frame
        // TR: İlk Çerçeve
        uint16_t total = static_cast<uint16_t>(
            ((static_cast<uint16_t>(first.data[0]) & 0x0F) << 8) | first.data[1]);
        std::vector<uint8_t> result;
        // Data from FF (bytes 2-7)
        // TR: FF'den veri (bayt 2-7)
        for (size_t i = 2; i < first.data.size() && result.size() < total; ++i)
            result.push_back(first.data[i]);

        // Consecutive Frames
        // TR: Ardışık Çerçeveler
        for (size_t f = 1; f < frames.size() && result.size() < total; ++f) {
            auto& cf = frames[f];
            if (cf.type != ISOTPFrameType::ConsecutiveFrame) continue;
            for (size_t i = 1; i < cf.data.size() && result.size() < total; ++i)
                result.push_back(cf.data[i]);
        }
        return result;
    }
};

// ═══════════════════════════════════════════════════════════════════════
// Helper: print UDS request/response
// ═══════════════════════════════════════════════════════════════════════
void print_uds(const std::string& dir, const std::vector<uint8_t>& msg) {
    std::cout << "    " << dir << ": [";
    for (size_t i = 0; i < msg.size(); ++i) {
        if (i > 0) std::cout << " ";
        std::cout << std::hex << std::setw(2) << std::setfill('0') << static_cast<int>(msg[i]);
    }
    std::cout << std::dec << "]";

    if (!msg.empty()) {
        if (msg[0] == SID::NegativeResponse && msg.size() >= 3) {
            std::cout << " → NRC: " << nrc_to_string(msg[2]);
        } else if (msg[0] >= 0x50) {
            std::cout << " → Positive: " << sid_to_string(static_cast<uint8_t>(msg[0] - SID::PositiveOffset));
        }
    }
    std::cout << "\n";
}

// ═══════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 14 - UDS Diagnostics & ISO-TP\n";
    std::cout << "============================================\n\n";

    UDSServer ecu("EngineECU");
    ecu.add_dtc(0x012345, 0x2F, "P0123 - Throttle Position Sensor A - High");
    ecu.add_dtc(0x050067, 0x01, "P0500 - Vehicle Speed Sensor A");
    ecu.add_dtc(0x030089, 0x08, "P0300 - Random Misfire Detected");

    // ─── Demo 1: Session Control ──────────────────────────────────
    {
        std::cout << "--- Demo 1: DiagnosticSessionControl ---\n";

```

```
        // EN: Switch to Extended Session
        // TR: Genişletilmiş Oturuma geç — gelişmiş teşhis hizmetlerini açar
        auto req = std::vector<uint8_t>{SID::DiagnosticSessionControl, 0x03};
        print_uds("REQ", req);
        auto resp = ecu.handle_request(req);
        print_uds("RSP", resp);

        // EN: Invalid session
        // TR: Geçersiz oturum — desteklenmeyen alt fonksiyon reddedilir
        req = {SID::DiagnosticSessionControl, 0x99};
        print_uds("REQ", req);
        resp = ecu.handle_request(req);
        print_uds("RSP", resp);
        std::cout << "\n";
    }

    // ── Demo 2: Read Data by Identifier ──────────────────────────────
    {
        std::cout << "--- Demo 2: ReadDataByIdentifier ---\n";

        // EN: Read VIN (DID 0xF190)
        // TR: VIN oku (DID 0xF190) — araç kimlik numarası
        auto req = std::vector<uint8_t>{SID::ReadDataByIdentifier, 0xF1, 0x90};
        print_uds("REQ", req);
        auto resp = ecu.handle_request(req);
        print_uds("RSP", resp);
        if (resp[0] == 0x62 && resp.size() > 3) {
            std::string vin(resp.begin() + 3, resp.end());
            std::cout << "    → VIN: " << vin << "\n";
        }

        // EN: Read ECU SW version (DID 0xF195)
        // TR: ECU yazılım sürümü oku (DID 0xF195)
        req = {SID::ReadDataByIdentifier, 0xF1, 0x95};
        print_uds("REQ", req);
        resp = ecu.handle_request(req);
        print_uds("RSP", resp);

        // EN: Read non-existent DID
        // TR: Var olmayan DID oku — RequestOutOfRange hatasi beklenir
        req = {SID::ReadDataByIdentifier, 0xFF, 0xFF};
        print_uds("REQ", req);
        resp = ecu.handle_request(req);
        print_uds("RSP", resp);
        std::cout << "\n";
    }

    // ── Demo 3: Security Access (Seed-Key) ───────────────────────────
    {
        std::cout << "--- Demo 3: SecurityAccess (Seed-Key) ---\n";

        // EN: Step 1: Request seed
        // TR: Adım 1: Tohum iste — güvenlik erişimi başlat
        auto req = std::vector<uint8_t>{SID::SecurityAccess, 0x01};
        print_uds("REQ", req);
        auto resp = ecu.handle_request(req);
        print_uds("RSP", resp);

        uint16_t seed = static_cast<uint16_t>(
            (static_cast<uint16_t>(resp[2]) << 8) | resp[3]);
        std::cout << "    Seed: 0x" << std::hex << std::setw(4) << std::setfill('0')
                  << seed << std::dec << "\n";

        // EN: Step 2a: Wrong key → should fail
        // TR: Adım 2a: Yanlış anahtar → başarısız olmalı
        auto bad_req = std::vector<uint8_t>{SID::SecurityAccess, 0x02, 0x00, 0x00};
        print_uds("REQ", bad_req);
        auto bad_resp = ecu.handle_request(bad_req);
        print_uds("RSP", bad_resp);

```

```
514         // EN: Step 2b: Correct key (seed XOR 0x1234)
515         // TR: Adım 2b: Doğru anahtar (seed XOR 0x1234) → güvenlik kilit açılır
516         uint16_t key = seed ^ 0x1234;
517         auto good_req = std::vector<uint8_t>{SID::SecurityAccess, 0x02,
518             static_cast<uint8_t>(key >> 8), static_cast<uint8_t>(key & 0xFF)};
519         print_uds("REQ", good_req);
520         auto good_resp = ecu.handle_request(good_req);
521         print_uds("RSP", good_resp);
522         std::cout << "    Key: 0x" << std::hex << std::setw(4) << std::setfill('0')
523                   << key << std::dec << " → Security UNLOCKED ✓\n\n";
524     }
525
526     // ─── Demo 4: Read / Clear DTCs ──────────────────────────────────────
527     {
528         std::cout << "--- Demo 4: DTC Read & Clear ---\n";
529
530         // EN: Read DTC count
531         // TR: DTC sayısını oku
532         auto req = std::vector<uint8_t>{SID::ReadDTCInformation, 0x01};
533         print_uds("REQ", req);
534         auto resp = ecu.handle_request(req);
535         print_uds("RSP", resp);
536         std::cout << "    DTC count: " << static_cast<int>(resp[4]) << "\n";
537
538         // EN: Read all DTCs
539         // TR: Tüm DTC'ıları oku — durum maskesi ile filtreleme
540         req = {SID::ReadDTCInformation, 0x02};
541         print_uds("REQ", req);
542         resp = ecu.handle_request(req);
543         print_uds("RSP", resp);
544
545         // EN: Parse DTCs from response
546         // TR: Yanıttan DTC'ıları ayrıştır — her DTC: 3 bayt kod + 1 bayt durum
547         std::cout << "    ┌──────────────┬────────┬──────────────────────────────────┐\n";
548         std::cout << "    │ DTC Code     │ Status │ Description                      │\n";
549         std::cout << "    ├──────────────┼────────┼──────────────────────────────────┤\n";
550         // Response: [59 02 FF] [DTC_HI DTC_MI DTC_LO STATUS] × N
551         // TR: Yanıt: [59 02 FF] [DTC_ÜST DTC_ORTA DTC_ALT DURUM] × N
552         std::map<uint32_t, std::string> dtc_names = {
553             {0x012345, "P0123 - Throttle Pos Sensor A - High"},
554             {0x050067, "P0500 - Vehicle Speed Sensor A"},
555             {0x030089, "P0300 - Random Misfire Detected"},
556         };
557         for (size_t i = 3; i + 3 < resp.size(); i += 4) {
558             uint32_t code = (static_cast<uint32_t>(resp[i]) << 16) |
559                             (static_cast<uint32_t>(resp[i + 1]) << 8) |
560                             static_cast<uint32_t>(resp[i + 2]);
561             uint8_t status = resp[i + 3];
562             std::string desc = dtc_names.count(code) ? dtc_names[code] : "Unknown";
563             std::cout << "    │ 0x" << std::hex << std::setw(6) << std::setfill('0') << code
564                       << std::dec << "    │ 0x" << std::hex << std::setw(2)
565                       << static_cast<int>(status) << std::dec
566                       << "   │ " << std::left << std::setw(37) << desc << std::right << " │\n";
567         }
568         std::cout << "    └──────────────┴────────┴──────────────────────────────────┘\n";
569
570         // EN: Clear DTCs
571         // TR: DTC'ıları temizle
572         req = {SID::ClearDTC};
573         print_uds("REQ", req);
574         resp = ecu.handle_request(req);
575         print_uds("RSP", resp);
576
577         // EN: Verify cleared
578         // TR: Temizlenmiş olduğunu doğrula — sayı 0 olmalı
579         req = {SID::ReadDTCInformation, 0x01};
580         resp = ecu.handle_request(req);
581         std::cout << "    After clear → DTC count: " << static_cast<int>(resp[4]) << "\n\n";
582     }
```

```cpp

    // ─── Demo 5: Write DID (requires security) ──────────────────────
    {
        std::cout << "--- Demo 5: WriteDataByIdentifier ---\n";

        // EN: Write without security → should fail
        // TR: Güvenlik kilidi açılmadan yazma → SecurityAccessDenied beklenir
        auto req = std::vector<uint8_t>{SID::WriteDataByIdentifier, 0xF1, 0x99, 0x42};
        print_uds("REQ", req);

        // EN: Need to be in extended session with security unlocked
        // TR: Genişletilmiş oturumda ve güvenlik açık olmalı
        // Session was set to Extended in Demo 1, security unlocked in Demo 3
        // TR: Oturum Demo 1'de Genişletilmiş'e geçti, güvenlik Demo 3'te açıldı
        auto resp = ecu.handle_request(req);
        print_uds("RSP", resp);

        // EN: Read it back
        // TR: Yazılanı geri oku — doğrulama
        req = {SID::ReadDataByIdentifier, 0xF1, 0x99};
        print_uds("REQ", req);
        resp = ecu.handle_request(req);
        print_uds("RSP", resp);
        std::cout << "\n";
    }

    // ─── Demo 6: ISO-TP Segmentation ────────────────────────────────
    {
        std::cout << "--- Demo 6: ISO-TP Segmentation ---\n";

        // EN: Short message → Single Frame
        // TR: Kısa mesaj → Tek Çerçeve (SF) — ≤7 bayt
        std::cout << "  Case 1: Short UDS message (fits in SF)\n";
        std::vector<uint8_t> short_msg = {0x22, 0xF1, 0x90};  // ReadDID(VIN)
                                                                // TR: VIN oku isteği
        auto sf_frames = ISOTPCodec::segment(short_msg);
        for (auto& f : sf_frames) f.print("SF");

        // EN: Long message → FF + CFs
        // TR: Uzun mesaj → FF + CF'ılar — çoklu CAN çerçevesine bölünür
        std::cout << "\n  Case 2: Long UDS response (requires FF + CFs)\n";
        // Simulating a ReadDID response with 25 bytes
        // TR: 25 baytlık ReadDID yanıtı simülasyonu
        std::vector<uint8_t> long_msg = {0x62, 0xF1, 0x90};
        std::string vin_str = "WVWZZZ3CZWE000001";
        long_msg.insert(long_msg.end(), vin_str.begin(), vin_str.end());
        // append more data to force multiple CFs
        // TR: çoklu CF zorlamak için ek veri ekle
        for (int i = 0; i < 10; ++i)
            long_msg.push_back(static_cast<uint8_t>(0x30 + i));

        std::cout << "    Original message: " << long_msg.size() << " bytes\n";
        auto multi_frames = ISOTPCodec::segment(long_msg);
        for (auto& f : multi_frames) f.print("");

        // EN: Reassemble
        // TR: Yeniden birleştir — orijinal mesajla eşleşmeli
        auto reassembled = ISOTPCodec::reassemble(multi_frames);
        std::cout << "    Reassembled: " << reassembled.size() << " bytes"
                  << " | Match: " << (reassembled == long_msg ? "YES ✓" : "NO ✗") << "\n\n";
    }

    // ─── Demo 7: Full Diagnostic Sequence ───────────────────────────
    {
        std::cout << "--- Demo 7: Full Diagnostic Sequence (Flash Prep) ---\n";
        std::cout << "  Typical ECU reprogramming preparation sequence:\n\n";

        UDSServer flash_ecu("ADAS_ECU");
        flash_ecu.add_dtc(0xC10000, 0x01, "U0100 - Lost Comm with ECM");

        struct Step {
            std::string description;
            std::vector<uint8_t> request;
        };

```

```
        std::vector<Step> sequence = {
            {"1. Switch to Extended Session",
             {SID::DiagnosticSessionControl, 0x03}},
            {"2. Tester Present (keep-alive)",
             {SID::TesterPresent, 0x00}},
            {"3. Read ECU SW Version",
             {SID::ReadDataByIdentifier, 0xF1, 0x95}},
            {"4. Security Access: Request Seed",
             {SID::SecurityAccess, 0x01}},
            {"5. Read DTCs before flash",
             {SID::ReadDTCInformation, 0x01}},
            {"6. Clear DTCs",
             {SID::ClearDTC}},
            {"7. Verify DTCs cleared",
             {SID::ReadDTCInformation, 0x01}},
            {"8. Switch to Programming Session",
             {SID::DiagnosticSessionControl, 0x02}},
            {"9. ECU Reset (post-flash)",
             {SID::ECUReset, 0x01}},
        };

        for (auto& step : sequence) {
            std::cout << "  " << step.description << "\n";
            print_uds("REQ", step.request);
            auto resp = flash_ecu.handle_request(step.request);
            print_uds("RSP", resp);

            // EN: If security seed received, send the key
            // TR: Güvenlik tohumu alındıysa, anahtarı gönder
            if (step.request[0] == SID::SecurityAccess && step.request[1] == 0x01 &&
                resp[0] == 0x67 && resp.size() >= 4) {
                uint16_t seed = static_cast<uint16_t>(
                    (static_cast<uint16_t>(resp[2]) << 8) | resp[3]);
                uint16_t key = seed ^ 0x1234;
                std::cout << "  4b. Security Access: Send Key\n";
                std::vector<uint8_t> key_req = {SID::SecurityAccess, 0x02,
                    static_cast<uint8_t>(key >> 8), static_cast<uint8_t>(key & 0xFF)};
                print_uds("REQ", key_req);
                auto key_resp = flash_ecu.handle_request(key_req);
                print_uds("RSP", key_resp);
            }
            std::cout << "\n";
        }
    }

    std::cout << "============================================\n";
    std::cout << "   End of UDS Diagnostics & ISO-TP\n";
    std::cout << "============================================\n";

    return 0;
}
```

Modül 15: HW Interfaces — Donanım Arayüzleri

Donanım Arayüzleri

5 kaynak dosya · Standart: C++20

C++20

► 01. Gpio And Registers

module_15_hw_interfaces/01_gpio_and_registers.cpp

Konu: GPIO & Register-Level Programming — Memory-Mapped I/O, Bitfields

Teori / Theory

THEORY: Memory-Mapped I/O and Registers

In embedded systems, hardware peripherals (GPIO, UART, SPI, etc.) are controlled through memory-mapped registers. The CPU accesses these registers as if they were normal memory addresses.
Memory Map Example (ARM Cortex-M / STM32):

```
┌──────────────────┬──────────────────────────────────────┐
│ Address Range    │ Peripheral                           │
├──────────────────┼──────────────────────────────────────┤
│ 0x0000_0000      │ Flash memory (code)                  │
│ 0x2000_0000      │ SRAM                                 │
│ 0x4000_0000      │ APB1 peripherals (UART, I2C, SPI)    │
│ 0x4001_0000      │ APB2 peripherals (GPIO, ADC, TIM)    │
│ 0x4002_0000      │ AHB1 peripherals (DMA, RCC)          │
│ 0xE000_0000      │ Cortex-M system (NVIC, SysTick)      │
└──────────────────┴──────────────────────────────────────┘
```

Register Access Rules:
1. Use volatile — compiler must not optimize away reads/writes
2. Use bit manipulation — set/clear individual bits
3. Use read-modify-write (RMW) for shared registers
4. Some registers are write-only or read-only
GPIO Register Set (typical STM32):

```
┌──────────┬─────────┬──────────────────────────────────────┐
│ Offset   │ Name    │ Description                          │
├──────────┼─────────┼──────────────────────────────────────┤
│ 0x00     │ MODER   │ Mode Register (input/output/AF/analog)  │
│ 0x04     │ OTYPER  │ Output Type (push-pull / open-drain)     │
│ 0x08     │ OSPEEDR│ Output Speed (low/med/high/very-high)    │
│ 0x0C     │ PUPDR   │ Pull-up/Pull-down                        │
│ 0x10     │ IDR     │ Input Data Register (read-only)          │
│ 0x14     │ ODR     │ Output Data Register                     │
│ 0x18     │ BSRR    │ Bit Set/Reset Register (atomic)          │
│ 0x1C     │ LCKR    │ Lock Register                            │
│ 0x20     │ AFRL    │ Alternate Function Low (pins 0-7)        │
│ 0x24     │ AFRH    │ Alternate Function High (pins 8-15)      │
└──────────┴─────────┴──────────────────────────────────────┘
```

TEORİ: Bellek Eşlemeli G/Ç ve Yazmaçlar

Gömülü sistemlerde donanım çevre birimleri (GPIO, UART, SPI vb.) bellek-eşlemeli yazmaçlar ile kontrol edilir. CPU bu yazmaçlara normal bellek adresleri gibi erişir.
Bellek Haritası (ARM Cortex-M / STM32):

```
┌──────────────┬──────────────────────────────────┐
│ Adres Aralığı │ Çevre Birimi                     │
├──────────────┼──────────────────────────────────┤
│ 0x0000_0000  │ Flash bellek (kod)               │
│ 0x2000_0000  │ SRAM                             │
│ 0x4000_0000  │ APB1 çevre birimleri (UART, I2C, SPI)  │
│ 0x4001_0000  │ APB2 çevre birimleri (GPIO, ADC, TIM)  │
│ 0x4002_0000  │ AHB1 çevre birimleri (DMA, RCC)  │
│ 0xE000_0000  │ Cortex-M sistem (NVIC, SysTick)  │
└──────────────┴──────────────────────────────────┘
```

Yazmaç Erişim Kuralları:
1. volatile kullan — derleyici okuma/yazmaları optimize etmesin
2. Bit manipülasyonu — tek tek bitleri ayarla/temizle
3. Paylaşılan yazmaçlarda oku-değiştir-yaz (RMW) kullan
4. Bazı yazmaçlar salt-okunur veya salt-yazılır
GPIO Yazmaç Seti (STM32 tipik):

```
┌────────┬──────┬──────────────────────────────────┐
│ Ofset  │ Ad   │ Açıklama                         │
├────────┼──────┼──────────────────────────────────┤
│ 0x00   │ MODER  │ Mod Yazmacı (giriş/çıkış/AF/analog)  │
│ 0x04   │ OTYPER │ Çıkış Tipi (push-pull / open-drain)  │
│ 0x08   │ OSPEEDR│ Çıkış Hızı (düşük/orta/yüksek/çok-yük)  │
│ 0x0C   │ PUPDR  │ Pull-up / Pull-down              │
│ 0x10   │ IDR    │ Giriş Veri Yazmacı (salt-okunur) │
│ 0x14   │ ODR    │ Çıkış Veri Yazmacı               │
│ 0x18   │ BSRR   │ Bit Set/Reset Yazmacı (atomik)   │
│ 0x1C   │ LCKR   │ Kilit Yazmacı                    │
│ 0x20   │ AFRL   │ Alternatif Fonksiyon Düşük (pin 0-7)  │
│ 0x24   │ AFRH   │ Alternatif Fonksiyon Yüksek (pin 8-15)  │
└────────┴──────┴──────────────────────────────────┘
```

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <array>
#include <string>
#include <iomanip>
#include <bitset>
#include <cassert>
#include <functional>
#include <map>

```

```cpp
// ═══════════════════════════════════════════════════════════════════════════
// PART 1: Simulated Register — volatile Memory-Mapped I/O
// ═══════════════════════════════════════════════════════════════════════════

// EN: On real hardware: volatile uint32_t* REG = (volatile uint32_t*)0x40020000;
//     We simulate this with a register class that tracks reads/writes.
// TR: Gerçek donanımda volatile pointer kullanılır: volatile uint32_t* REG = ...;
//     Burada donanımı simüle eden bir sınıf kullanıyoruz; okuma/yazma davranışını
//     taklit eder (salt-okunur, salt-yazılır, yazma maskesi desteği ile).

class SimRegister {
    uint32_t value_;
    std::string name_;
    bool read_only_;
    bool write_only_;
    uint32_t write_mask_;  // bits that can be written — TR: yazılabilir bitler

public:
    SimRegister(std::string name, uint32_t initial = 0,
                bool ro = false, bool wo = false, uint32_t mask = 0xFFFFFFFF)
        : value_(initial), name_(std::move(name)),
          read_only_(ro), write_only_(wo), write_mask_(mask) {}

    uint32_t read() const {
        if (write_only_) return 0;  // write-only registers read as 0
                                    // TR: salt-yazılır yazmaçlar 0 döndürür
        return value_;
    }

    void write(uint32_t val) {
        if (read_only_) return;  // read-only registers ignore writes
                                 // TR: salt-okunur yazmaçlar yazmayı yok sayar
        value_ = (value_ & ~write_mask_) | (val & write_mask_);
    }

    // EN: Force-write for simulation (bypasses read-only flag)
    // TR: Simülasyon için zorla yazma (salt-okunur bayrağını atlar)
    void force_write(uint32_t val) { value_ = val; }

    // EN: Atomic bit set/reset (like BSRR)
    // TR: Atomik bit set/reset (BSRR gibi — oku-değiştir-yaz gerektirmez)
    void bit_set(uint32_t mask) { value_ |= mask; }
    void bit_clear(uint32_t mask) { value_ &= ~mask; }

    // EN: Read-Modify-Write
    // TR: Oku-Değiştir-Yaz — paylaşılan yazmaçlarda diğer bitlere dokunmadan değişiklik
    void rmw(uint32_t clear_mask, uint32_t set_mask) {
        uint32_t tmp = value_;
        tmp &= ~clear_mask;
        tmp |= set_mask;
        value_ = tmp;
    }

    const std::string& name() const { return name_; }

    void print() const {
        std::cout << "    " << std::left << std::setw(8) << name_ << std::right
                  << ": 0x" << std::hex << std::setw(8) << std::setfill('0') << value_
                  << std::dec << std::setfill(' ')
                  << " (" << std::bitset<32>(value_) << ")\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: GPIO Port Simulation (STM32-like)
// ═══════════════════════════════════════════════════════════════════════════

enum class GPIOMode : uint8_t {
    Input    = 0b00,
    Output   = 0b01,
    AltFunc  = 0b10,
    Analog   = 0b11,
};
```

```cpp

enum class GPIOSpeed : uint8_t {
    Low      = 0b00,
    Medium   = 0b01,
    High     = 0b10,
    VeryHigh = 0b11,
};

enum class GPIOPull : uint8_t {
    None     = 0b00,
    PullUp   = 0b01,
    PullDown = 0b10,
};

enum class GPIOOutputType : uint8_t {
    PushPull  = 0,
    OpenDrain = 1,
};

class GPIOPort {
    std::string port_name_;
    SimRegister MODER;    // offset 0x00 — TR: Mod Yazmaçı
    SimRegister OTYPER;   // offset 0x04 — TR: Çıkış Tipi Yazmaçı
    SimRegister OSPEEDR;  // offset 0x08 — TR: Çıkış Hızı Yazmaçı
    SimRegister PUPDR;    // offset 0x0C — TR: Pull-up/Pull-down Yazmaçı
    SimRegister IDR;      // offset 0x10 (read-only) — TR: Giriş Veri Yazmaçı (salt-okunur)
    SimRegister ODR;      // offset 0x14 — TR: Çıkış Veri Yazmaçı
    SimRegister BSRR;     // offset 0x18 (write-only) — TR: Bit Set/Reset (salt-yazılır)

    uint32_t external_input_ = 0;  // simulated external voltage levels
                                   // TR: simüle harici gerilim seviyeleri

public:
    GPIOPort(std::string name)
        : port_name_(std::move(name)),
          MODER("MODER", 0),
          OTYPER("OTYPER", 0),
          OSPEEDR("OSPEEDR", 0),
          PUPDR("PUPDR", 0),
          IDR("IDR", 0, true),    // read-only — TR: salt-okunur
          ODR("ODR", 0),
          BSRR("BSRR", 0, false, true)  // write-only — TR: salt-yazılır
    {}

    // EN: Configure pin mode (2 bits per pin in MODER)
    // TR: Pin modunu yapılandır (MODER'de her pin için 2 bit)
    void set_mode(int pin, GPIOMode mode) {
        uint32_t shift = static_cast<uint32_t>(pin) * 2;
        MODER.rmw(0x3U << shift, static_cast<uint32_t>(mode) << shift);
    }

    void set_output_type(int pin, GPIOOutputType type) {
        if (type == GPIOOutputType::OpenDrain)
            OTYPER.bit_set(1U << static_cast<uint32_t>(pin));
        else
            OTYPER.bit_clear(1U << static_cast<uint32_t>(pin));
    }

    void set_speed(int pin, GPIOSpeed speed) {
        uint32_t shift = static_cast<uint32_t>(pin) * 2;
        OSPEEDR.rmw(0x3U << shift, static_cast<uint32_t>(speed) << shift);
    }

    void set_pull(int pin, GPIOPull pull) {
        uint32_t shift = static_cast<uint32_t>(pin) * 2;
        PUPDR.rmw(0x3U << shift, static_cast<uint32_t>(pull) << shift);
    }

    // EN: Write output pin (via ODR)
    // TR: Çıkış pinine yaz (ODR üzerinden — oku-değiştir-yaz gerektirir)
    void write_pin(int pin, bool high) {
        if (high)
            ODR.bit_set(1U << static_cast<uint32_t>(pin));
        else
            ODR.bit_clear(1U << static_cast<uint32_t>(pin));
        // Update IDR to reflect output (for output pins)
```

```
        // TR: Çıkış pinleri için IDR'ı güncelle
        update_idr();
    }

    // EN: Atomic write via BSRR (no read-modify-write needed)
    // TR: BSRR ile atomik yazma (RMW gerektirmez — ISR-güvenli)
    void atomic_set(int pin) {
        ODR.bit_set(1U << static_cast<uint32_t>(pin));
        update_idr();
    }

    void atomic_reset(int pin) {
        ODR.bit_clear(1U << static_cast<uint32_t>(pin));
        update_idr();
    }

    // EN: Read input pin
    // TR: Giriş pinini oku (IDR'den — donanım tarafından güncellenir)
    bool read_pin(int pin) const {
        return (IDR.read() >> static_cast<uint32_t>(pin)) & 1;
    }

    // EN: Simulate external input (for input pins)
    // TR: Harici girişi simüle et (giriş pinleri için — buton/sensör yerine geçer)
    void set_external_input(int pin, bool high) {
        if (high)
            external_input_ |= (1U << static_cast<uint32_t>(pin));
        else
            external_input_ &= ~(1U << static_cast<uint32_t>(pin));
        update_idr();
    }

    void print_registers() const {
        std::cout << "  " << port_name_ << " Registers:\n";
        MODER.print();
        OTYPER.print();
        OSPEEDR.print();
        PUPDR.print();
        IDR.print();
        ODR.print();
    }

    void print_pin_config(int pin) const {
        uint32_t shift = static_cast<uint32_t>(pin) * 2;
        auto mode = static_cast<GPIOMode>((MODER.read() >> shift) & 0x3);
        auto speed = static_cast<GPIOSpeed>((OSPEEDR.read() >> shift) & 0x3);
        auto pull = static_cast<GPIOPull>((PUPDR.read() >> shift) & 0x3);
        auto otype = static_cast<GPIOOutputType>(
            (OTYPER.read() >> static_cast<uint32_t>(pin)) & 0x1);

        std::string mode_str, speed_str, pull_str, otype_str;
        switch (mode) {
            case GPIOMode::Input:   mode_str = "INPUT";  break;
            case GPIOMode::Output:  mode_str = "OUTPUT"; break;
            case GPIOMode::AltFunc: mode_str = "AF";     break;
            case GPIOMode::Analog:  mode_str = "ANALOG"; break;
        }
        switch (speed) {
            case GPIOSpeed::Low:      speed_str = "LOW";       break;
            case GPIOSpeed::Medium:   speed_str = "MEDIUM";    break;
            case GPIOSpeed::High:     speed_str = "HIGH";      break;
            case GPIOSpeed::VeryHigh: speed_str = "VERY_HIGH"; break;
        }
        switch (pull) {
            case GPIOPull::None:     pull_str = "NONE";      break;
            case GPIOPull::PullUp:   pull_str = "PULL_UP";   break;
            case GPIOPull::PullDown: pull_str = "PULL_DOWN"; break;
        }
        otype_str = (otype == GPIOOutputType::PushPull) ? "PP" : "OD";

```

```
        std::cout << "    Pin " << std::setw(2) << pin << ": " << std::left
                  << std::setw(6) << mode_str << " " << std::setw(9) << speed_str
                  << " " << std::setw(9) << pull_str << " " << otype_str
                  << std::right;
        if (mode == GPIOMode::Output) {
            std::cout << " → " << (read_pin(pin) ? "HIGH" : "LOW");
        } else if (mode == GPIOMode::Input) {
            std::cout << " ← " << (read_pin(pin) ? "HIGH" : "LOW");
        }
        std::cout << "\n";
    }

private:
    void update_idr() {
        // EN: IDR reflects output for output pins, external for input pins
        // TR: IDR çıkış pinlerinde ODR'yi, giriş pinlerinde harici durumu yansıtır
        uint32_t idr_val = 0;
        for (int i = 0; i < 16; ++i) {
            auto shift = static_cast<uint32_t>(i) * 2;
            auto mode = static_cast<GPIOMode>((MODER.read() >> shift) & 0x3);
            if (mode == GPIOMode::Output || mode == GPIOMode::AltFunc) {
                idr_val |= (ODR.read() & (1U << static_cast<uint32_t>(i)));
            } else {
                idr_val |= (external_input_ & (1U << static_cast<uint32_t>(i)));
            }
        }
        // EN: Force-update IDR in simulation (on real HW, hardware drives IDR)
        // TR: Simülasyonda IDR'yi zorla güncelle (gerçek donanımda IDR'yi donanım sürer)
        IDR.force_write(idr_val);
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: Bit Manipulation Utilities
// ═══════════════════════════════════════════════════════════════════════════

// EN: Common bit manipulation patterns used in register programming
// TR: Yazmaç programlamada kullanılan temel bit manipülasyonu kalıpları
namespace BitOps {
    // Set bit at position — TR: Belirtilen pozisyonda biti ayarla
    constexpr uint32_t set_bit(uint32_t reg, int bit) {
        return reg | (1U << static_cast<uint32_t>(bit));
    }
    // Clear bit at position — TR: Belirtilen pozisyonda biti temizle
    constexpr uint32_t clear_bit(uint32_t reg, int bit) {
        return reg & ~(1U << static_cast<uint32_t>(bit));
    }
    // Toggle bit — TR: Bit değiştir (XOR)
    constexpr uint32_t toggle_bit(uint32_t reg, int bit) {
        return reg ^ (1U << static_cast<uint32_t>(bit));
    }
    // Read bit — TR: Bit oku
    constexpr bool read_bit(uint32_t reg, int bit) {
        return (reg >> static_cast<uint32_t>(bit)) & 1U;
    }
    // Set bitfield: clear [pos+width-1:pos], then set value
    // TR: Bit alanı ayarla: önce temizle, sonra değer yaz
    constexpr uint32_t set_field(uint32_t reg, int pos, int width, uint32_t value) {
        uint32_t mask = ((1U << static_cast<uint32_t>(width)) - 1) << static_cast<uint32_t>(pos);
        return (reg & ~mask) | ((value << static_cast<uint32_t>(pos)) & mask);
    }
    // Read bitfield — TR: Bit alanı oku
    constexpr uint32_t read_field(uint32_t reg, int pos, int width) {
        uint32_t mask = (1U << static_cast<uint32_t>(width)) - 1;
        return (reg >> static_cast<uint32_t>(pos)) & mask;
    }
}

```

```cpp
// ═══════════════════════════════════════════════════════════════════════════
// PART 4: Clock Enable (RCC) Simulation
// ═══════════════════════════════════════════════════════════════════════════

// EN: Before using any peripheral, its clock must be enabled in the RCC
//     (Reset and Clock Control) register. Forgetting this is a classic bug!
// TR: Herhangi bir çevre birimini kullanmadan önce RCC'de (Reset ve Saat
//     Kontrolü) saat sinyali etkinleştirilmelidir. Bunu unutmak klasik bir hatadır
//     — yazımışç 0 okur ve nedenini anlayamaz!

class RCCSimulator {
    SimRegister AHB1ENR;  // AHB1 Enable Register (GPIO clocks) — TR: GPIO saat sinyali
    SimRegister APB1ENR;  // APB1 Enable (UART, I2C, SPI) — TR: seri iletişim saatleri
    SimRegister APB2ENR;  // APB2 Enable (ADC, TIM) — TR: ADC/Zamanlayıcı saatleri

    // EN: Clock enable bit positions
    // TR: Saat etkinleştirme bit pozisyonları
    static constexpr int GPIOA_EN = 0;
    static constexpr int GPIOB_EN = 1;
    static constexpr int GPIOC_EN = 2;
    static constexpr int DMA1_EN  = 21;

public:
    RCCSimulator()
        : AHB1ENR("AHB1ENR", 0),
          APB1ENR("APB1ENR", 0),
          APB2ENR("APB2ENR", 0) {}

    void enable_gpio(char port) {
        int bit = port - 'A';
        AHB1ENR.bit_set(1U << static_cast<uint32_t>(bit));
    }

    bool is_gpio_enabled(char port) const {
        int bit = port - 'A';
        return (AHB1ENR.read() >> static_cast<uint32_t>(bit)) & 1;
    }

    void print() const {
        std::cout << "  RCC Registers:\n";
        AHB1ENR.print();
        APB1ENR.print();
        APB2ENR.print();
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 15 - GPIO & Register Programming\n";
    std::cout << "============================================\n\n";

    // ─── Demo 1: Register Bit Manipulation ──────────────────────────────
    {
        std::cout << "--- Demo 1: Bit Manipulation Operations ---\n";
        uint32_t reg = 0x00000000;

        reg = BitOps::set_bit(reg, 5);
        std::cout << "    After set_bit(5):     0x" << std::hex << std::setw(8)
                  << std::setfill('0') << reg << std::dec << "\n";

        reg = BitOps::set_bit(reg, 13);
        std::cout << "    After set_bit(13):    0x" << std::hex << std::setw(8)
                  << std::setfill('0') << reg << std::dec << "\n";

        reg = BitOps::clear_bit(reg, 5);
        std::cout << "    After clear_bit(5):   0x" << std::hex << std::setw(8)
                  << std::setfill('0') << reg << std::dec << "\n";

        reg = BitOps::toggle_bit(reg, 13);
        std::cout << "    After toggle_bit(13): 0x" << std::hex << std::setw(8)
                  << std::setfill('0') << reg << std::dec << "\n";

```

```cpp
        // EN: Bitfield operations
        // TR: Çoklu-bit alanı işlemleri (yazmaçtaki belirli bir alanı ayarla/oku)
        reg = 0;
        reg = BitOps::set_field(reg, 4, 3, 0b101);  // set bits [6:4] = 5
        std::cout << "    After set_field(4,3,0b101): 0x" << std::hex << std::setw(8)
                  << std::setfill('0') << reg << std::dec << "\n";
        uint32_t field = BitOps::read_field(reg, 4, 3);
        std::cout << "    read_field(4,3): " << field << " (expected: 5)\n\n";
    }

    // ─── Demo 2: RCC Clock Enable ──────────────────────────────────────
    {
        std::cout << "--- Demo 2: RCC Clock Enable ---\n";
        std::cout << "  IMPORTANT: Always enable peripheral clock before access!\n\n";

        RCCSimulator rcc;
        std::cout << "  Before enabling clocks:\n";
        rcc.print();

        rcc.enable_gpio('A');
        rcc.enable_gpio('B');
        rcc.enable_gpio('C');

        std::cout << "\n  After enabling GPIOA, GPIOB, GPIOC clocks:\n";
        rcc.print();

        std::cout << "\n  Clock status: GPIOA=" << rcc.is_gpio_enabled('A')
                  << " GPIOB=" << rcc.is_gpio_enabled('B')
                  << " GPIOC=" << rcc.is_gpio_enabled('C')
                  << " GPIOD=" << rcc.is_gpio_enabled('D') << "\n\n";
    }

    // ─── Demo 3: GPIO Output Configuration ─────────────────────────────
    {
        std::cout << "--- Demo 3: GPIO Output (LED Control) ---\n";
        std::cout << "  Scenario: LED on PA5 (STM32 Nucleo built-in LED)\n\n";

        GPIOPort gpioa("GPIOA");

        // EN: Configure PA5 as output, push-pull, medium speed, no pull
        // TR: PA5'i çıkış olarak yapılandır: push-pull, orta hız, pull yok
        //     (STM32 Nucleo kartındaki dahili LED pini)
        gpioa.set_mode(5, GPIOMode::Output);
        gpioa.set_output_type(5, GPIOOutputType::PushPull);
        gpioa.set_speed(5, GPIOSpeed::Medium);
        gpioa.set_pull(5, GPIOPull::None);

        gpioa.print_pin_config(5);

        // EN: Turn LED ON
        // TR: LED'i yak (ODR'de ilgili biti set et)
        gpioa.write_pin(5, true);
        std::cout << "    LED ON  → Pin 5 = " << gpioa.read_pin(5) << "\n";

        // EN: Turn LED OFF
        // TR: LED'i söndür (ODR'de ilgili biti temizle)
        gpioa.write_pin(5, false);
        std::cout << "    LED OFF → Pin 5 = " << gpioa.read_pin(5) << "\n";

        // EN: Atomic operations (BSRR-style)
        // TR: Atomik işlemler (BSRR tarzı — kesme güvenli, RMW gerektirmez)
        gpioa.atomic_set(5);
        std::cout << "    atomic_set(5)   → Pin 5 = " << gpioa.read_pin(5) << "\n";
        gpioa.atomic_reset(5);
        std::cout << "    atomic_reset(5) → Pin 5 = " << gpioa.read_pin(5) << "\n\n";
    }
```

```

    // ─── Demo 4: GPIO Input (Button) ──────────────────────────────────
    {
        std::cout << "--- Demo 4: GPIO Input (Button Read) ---\n";
        std::cout << "  Scenario: Button on PC13 (active-low with pull-up)\n\n";

        GPIOPort gpioc("GPIOC");

        // EN: Configure PC13 as input with pull-up
        // TR: PC13'ü giriş olarak yapılandır, dahili pull-up ile
        //     (buton başına basılınca LOW seviyesine çeker — active-low)
        gpioc.set_mode(13, GPIOMode::Input);
        gpioc.set_pull(13, GPIOPull::PullUp);

        gpioc.print_pin_config(13);

        // EN: Simulate button states
        // TR: Buton durumlarını simüle et (pull-up ile serbest=HIGH, basılı=LOW)
        gpioc.set_external_input(13, true);  // not pressed (pull-up) — TR: basılmamış (pull-up=HIGH)
        std::cout << "    Button released (HIGH): " << gpioc.read_pin(13) << "\n";

        gpioc.set_external_input(13, false);  // pressed (active-low) — TR: basılı (active-low=LOW)
        std::cout << "    Button pressed (LOW):   " << gpioc.read_pin(13) << "\n\n";
    }

    // ─── Demo 5: Multiple Pin Configuration ──────────────────────────
    {
        std::cout << "--- Demo 5: Multi-Pin Automotive ECU GPIO Layout ---\n\n";

        GPIOPort gpiob("GPIOB");

        // EN: Typical ECU GPIO assignment
        // TR: Tipik ECU GPIO pin ataması (otomotiv uygulaması: ADC, SPI, I2C,
        //     UART, CAN, LED — tümü aynı GPIO portu üzerinde yapılandırılır)
        struct PinConfig {
            int pin;
            GPIOMode mode;
            GPIOSpeed speed;
            GPIOPull pull;
            GPIOOutputType otype;
            std::string function;
        };

        std::vector<PinConfig> pins = {
            {0,  GPIOMode::Analog,  GPIOSpeed::Low,      GPIOPull::None,
GPIOOutputType::PushPull, "ADC_IN8 (Temp Sensor)"},
            {1,  GPIOMode::Analog,  GPIOSpeed::Low,      GPIOPull::None,
GPIOOutputType::PushPull, "ADC_IN9 (Batt Voltage)"},
            {3,  GPIOMode::AltFunc, GPIOSpeed::VeryHigh,  GPIOPull::None,
GPIOOutputType::PushPull, "SPI1_SCK"},
            {4,  GPIOMode::AltFunc, GPIOSpeed::VeryHigh,  GPIOPull::None,
GPIOOutputType::PushPull, "SPI1_MISO"},
            {5,  GPIOMode::AltFunc, GPIOSpeed::VeryHigh,  GPIOPull::None,
GPIOOutputType::PushPull, "SPI1_MOSI"},
            {6,  GPIOMode::AltFunc, GPIOSpeed::High,      GPIOPull::PullUp,
GPIOOutputType::OpenDrain, "I2C1_SCL"},
            {7,  GPIOMode::AltFunc, GPIOSpeed::High,      GPIOPull::PullUp,
GPIOOutputType::OpenDrain, "I2C1_SDA"},
            {8,  GPIOMode::Output,  GPIOSpeed::Low,       GPIOPull::None,
GPIOOutputType::PushPull, "LED_Status (Green)"},
            {9,  GPIOMode::Output,  GPIOSpeed::Low,       GPIOPull::None,
GPIOOutputType::PushPull, "LED_Error (Red)"},
            {10, GPIOMode::AltFunc, GPIOSpeed::High,      GPIOPull::PullUp,
GPIOOutputType::PushPull, "USART3_TX (Debug)"},
            {11, GPIOMode::AltFunc, GPIOSpeed::High,      GPIOPull::PullUp,
GPIOOutputType::PushPull, "USART3_RX (Debug)"},
            {12, GPIOMode::Input,   GPIOSpeed::Low,       GPIOPull::PullDown,
GPIOOutputType::PushPull, "CAN_STB (Standby)"},
            {13, GPIOMode::AltFunc, GPIOSpeed::VeryHigh,  GPIOPull::None,
GPIOOutputType::PushPull, "CAN2_TX"},
            {14, GPIOMode::AltFunc, GPIOSpeed::VeryHigh,  GPIOPull::None,
GPIOOutputType::PushPull, "CAN2_RX"},
        };
```

```cpp

        for (auto& p : pins) {
            gpiob.set_mode(p.pin, p.mode);
            gpiob.set_speed(p.pin, p.speed);
            gpiob.set_pull(p.pin, p.pull);
            gpiob.set_output_type(p.pin, p.otype);
        }

        std::cout << "  GPIOB Pin Assignments (Automotive ECU):\n";
        std::cout << "  ┌──────┬──────────┬──────────┬──────────┬────┬──────────────────────┐\n";
        std::cout << "  │ Pin  │ Mode     │ Speed    │ Pull     │ OT │ Function             │\n";
        std::cout << "  ├──────┼──────────┼──────────┼──────────┼────┼──────────────────────┤\n";
        for (auto& p : pins) {
            std::string mode_s, speed_s, pull_s, ot_s;
            switch (p.mode) {
                case GPIOMode::Input:   mode_s = "INPUT";  break;
                case GPIOMode::Output:  mode_s = "OUTPUT"; break;
                case GPIOMode::AltFunc: mode_s = "AF";     break;
                case GPIOMode::Analog:  mode_s = "ANALOG"; break;
            }
            switch (p.speed) {
                case GPIOSpeed::Low:      speed_s = "LOW";       break;
                case GPIOSpeed::Medium:   speed_s = "MEDIUM";    break;
                case GPIOSpeed::High:     speed_s = "HIGH";      break;
                case GPIOSpeed::VeryHigh: speed_s = "VERY_HIGH"; break;
            }
            switch(p.pull) {
                case GPIOPull::None:     pull_s = "NONE";      break;
                case GPIOPull::PullUp:   pull_s = "PULL_UP";   break;
                case GPIOPull::PullDown: pull_s = "PULL_DOWN"; break;
            }
            ot_s = (p.otype == GPIOOutputType::PushPull) ? "PP" : "OD";

            std::cout << "  │ " << std::setw(4) << ("PB" + std::to_string(p.pin))
                      << " │ " << std::left << std::setw(6) << mode_s
                      << " │ " << std::setw(9) << speed_s
                      << " │ " << std::setw(9) << pull_s
                      << " │ " << std::setw(2) << ot_s
                      << " │ " << std::setw(23) << p.function
                      << std::right << " │\n";
        }
        std::cout << "  └──────┴──────────┴──────────┴──────────┴────┴──────────────────────┘\n";

        std::cout << "\n  Register state after configuration:\n";
        gpiob.print_registers();
        std::cout << "\n";
    }

    // ─── Demo 6: volatile Keyword Importance ──────────────────────────
    {
        std::cout << "--- Demo 6: volatile Keyword in Register Access ---\n\n";
        std::cout << "  Why volatile is ESSENTIAL for hardware registers:\n\n";
        std::cout << "  WITHOUT volatile (WRONG):\n";
        std::cout << "    uint32_t* REG = (uint32_t*)0x40020014;\n";
        std::cout << "    *REG = 0x01;  // compiler may optimize this away!\n";
        std::cout << "    while (*REG & 0x02) {}  // infinite loop: compiler\n";
        std::cout << "                            // caches value, never re-reads\n\n";

        std::cout << "  WITH volatile (CORRECT):\n";
        std::cout << "    volatile uint32_t* REG = (volatile uint32_t*)0x40020014;\n";
        std::cout << "    *REG = 0x01;  // always written to hardware\n";
        std::cout << "    while (*REG & 0x02) {}  // re-reads every iteration\n\n";

        std::cout << "  C++ best practice — use std::atomic or volatile:\n";
        std::cout << "    template<typename T>\n";
        std::cout << "    struct HWRegister {\n";
        std::cout << "        volatile T value;\n";
        std::cout << "        void write(T v) { value = v; }\n";
        std::cout << "        T read() const  { return value; }\n";
        std::cout << "    };\n\n";

```

```
569         // EN: Demonstrate the concept (without real hardware)
570         // TR: Kavramı göster (gerçek donanım olmadan — volatile okuma/yazma
571         //     derleyicinin değeri önbelleğe almasını önler)
572         volatile uint32_t sim_reg = 0;
573         sim_reg = 0xDEADBEEF;
574         uint32_t read_back = sim_reg;
575         std::cout << "  Simulated volatile register:\n";
576         std::cout << "    Write: 0xDEADBEEF → Read: 0x"
577                   << std::hex << std::setw(8) << std::setfill('0')
578                   << read_back << std::dec << " ✓\n\n";
579     }
580
581     // ─── Demo 7: Common Register Patterns ──────────────────────────────
582     {
583         std::cout << "--- Demo 7: Common Register Programming Patterns ---\n\n";
584
585         std::cout << "  ┌──────────────────────────┬──────────────────────────────────────┐\n";
586         std::cout << "  │ Pattern                  │ Code                                 │\n";
587         std::cout << "  ├──────────────────────────┼──────────────────────────────────────┤\n";
588         std::cout << "  │ Set single bit           │ REG |= (1 << bit)                    │\n";
589         std::cout << "  │ Clear single bit         │ REG &= ~(1 << bit)                   │\n";
590         std::cout << "  │ Toggle bit               │ REG ^= (1 << bit)                    │\n";
591         std::cout << "  │ Check bit                │ if (REG & (1 << bit))                │\n";
592         std::cout << "  │ Set multi-bit field      │ REG = (REG & ~mask) | (val<<pos)     │\n";
593         std::cout << "  │ Atomic set (BSRR)        │ BSRR = (1 << bit)                    │\n";
594         std::cout << "  │ Atomic reset (BSRR)      │ BSRR = (1 << (bit+16))               │\n";
595         std::cout << "  │ Read-Modify-Write        │ tmp=REG; tmp|=val; REG=tmp;          │\n";
596         std::cout << "  │ Wait for flag            │ while(!(REG & FLAG)) {}              │\n";
597         std::cout << "  │ Write-only register      │ REG = value; (no read first!)        │\n";
598         std::cout << "  └──────────────────────────┴──────────────────────────────────────┘\n\n";
599
600         std::cout << "  Common mistakes:\n";
601         std::cout << "    1. Forgetting volatile → compiler optimizes away HW access\n";
602         std::cout << "    2. Forgetting RCC clock enable → registers read as 0\n";
603         std::cout << "    3. Wrong bit position → corrupts adjacent peripheral config\n";
604         std::cout << "    4. RMW on write-only registers → read returns 0, destroys data\n";
605         std::cout << "    5. Not waiting for ready flag → using peripheral before it's ready\n";
606     }
607
608     std::cout << "\n============================================\n";
609     std::cout << "   End of GPIO & Register Programming\n";
610     std::cout << "============================================\n";
611
612     return 0;
613 }
```

► 02. Adc Dac

module_15_hw_interfaces/02_adc_dac.cpp

Konu: ADC & DAC — Analog-to-Digital / Digital-to-Analog Conversion

Teori / Theory

THEORY: ADC (Analog-to-Digital Converter)

ADC converts continuous analog voltages to discrete digital values. Key parameters:
- Resolution: number of bits (8, 10, 12, 16, 24-bit)
- Sampling Rate: conversions per second (Samples/s)
- Reference Voltage (Vref): defines full-scale range
- Input Range: 0V to Vref (single-ended) or ±Vref (differential)

ADC Types:

Architecture	Speed	Resolution	Use Case
SAR	1-5 MSPS	8-18 bit	MCU general purpose
Sigma-Delta (ΣΔ)	<100 SPS	16-24 bit	Precision measurement
Pipeline	>100 MSPS	8-16 bit	Video, radar
Flash	>1 GSPS	4-8 bit	Oscilloscope, telecom

Conversion Formula: Digital_Value = (V_in / V_ref) × (2^n - 1) V_in = Digital_Value × V_ref / (2^n - 1)

TEORİ: ADC (Analog-Dijital Dönüştürücü)

ADC sürekli analog gerilimleri ayrık dijital değerlere çevirir. Temel parametreler: Çözünürlük (bit sayısı), Örnekleme Hızı, Referans Gerilim (Vref), Giriş Aralığı (0V-Vref tek uçlu veya ±Vref farksal).
ADC Türleri:

Mimari	Hız	Çözünürlük	Kullanım Alanı
SAR	1-5 MSPS	8-18 bit	MCU genel amaçlı
Sigma-Delta (ΣΔ)	<100 SPS	16-24 bit	Hassas ölçüm
Pipeline	>100 MSPS	8-16 bit	Video, radar
Flash	>1 GSPS	4-8 bit	Osiloskop, telekom

Dönüşüm Formülü: Digital = (V_in / V_ref) × (2^n - 1) V_in = Digital × V_ref / (2^n - 1)

THEORY: DAC (Digital-to-Analog Converter)

DAC converts digital values back to analog voltages. V_out = Digital_Value × V_ref / (2^n - 1)
Common uses: audio output, waveform generation, control loops

TEORİ: DAC (Dijital-Analog Dönüştürücü)

DAC ise tersini yapar: dijital değerleri analog gerilime çevirir. V_out = Digital_Value × V_ref / (2^n - 1) Yaygın kullanım: ses çıkışı, dalga formu üretimi, kontrol döngüleri.

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <cmath>
#include <array>
#include <vector>
#include <string>
#include <iomanip>
#include <numeric>
#include <algorithm>
#include <functional>
#include <random>
#include <cassert>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: ADC Simulator
// ═══════════════════════════════════════════════════════════════════════════

// EN: Simulates a SAR ADC with configurable resolution and Vref
// TR: Yapılandırılabilir çözünürlük ve Vref ile SAR ADC simülasyonu

enum class ADCResolution : uint8_t {
    Bit8  = 8,
    Bit10 = 10,
    Bit12 = 12,
    Bit16 = 16,
};

struct ADCConfig {
    ADCResolution resolution = ADCResolution::Bit12;
    double vref = 3.3;              // Reference voltage — TR: Referans gerilim
    uint32_t sample_rate_hz = 1000000;  // 1 MSPS — TR: 1 milyon örnekleme/saniye
    bool differential = false;       // single-ended vs differential
                                     // TR: tek uçlu / farksal giriş seçimi

    uint32_t max_value() const {
        return (1U << static_cast<uint32_t>(resolution)) - 1;
    }

    double lsb_voltage() const {
        return vref / static_cast<double>(max_value());
    }
};

class ADCSimulator {
    ADCConfig config_;
    std::mt19937 rng_{42};  // fixed seed for reproducibility — TR: tekrarlanabilirlik için sabit tohum
    std::normal_distribution<double> noise_{0.0, 0.5};  // ±0.5 LSB noise — TR: kuantalama gürültüsü

    // EN: Simulated analog input voltages (up to 16 channels)
    // TR: Simüle edilmiş analog giriş gerilimleri (16 kanala kadar)
    std::array<double, 16> channel_voltages_{};

public:
    explicit ADCSimulator(ADCConfig cfg = {}) : config_(std::move(cfg)) {}

    void set_channel_voltage(int ch, double voltage) {
        if (ch >= 0 && ch < 16) {
            channel_voltages_[static_cast<size_t>(ch)] = voltage;
        }
    }

    // EN: Convert analog voltage to digital value
    // TR: Analog gerilimi dijital değere çevir (SAR dönüşüm simülasyonu)
    uint32_t convert(int channel) {
        double v_in = channel_voltages_[static_cast<size_t>(channel)];
        // Clamp to valid range — TR: Geçerli aralığa sınırla
        v_in = std::clamp(v_in, 0.0, config_.vref);

        double ideal = (v_in / config_.vref) * static_cast<double>(config_.max_value());
        // Add quantization noise — TR: Kuantalama gürültüsü ekle
        double noisy = ideal + noise_(rng_);
        noisy = std::clamp(noisy, 0.0, static_cast<double>(config_.max_value()));

        return static_cast<uint32_t>(std::round(noisy));
```

```
75      }
76
77      // EN: Convert digital value back to voltage
78      // TR: Dijital değeri tekrar gerilime çevir (ADC okumasını yorumlamak için)
79      double to_voltage(uint32_t digital) const {
80          return static_cast<double>(digital) * config_.vref
81                 / static_cast<double>(config_.max_value());
82      }
83
84      // EN: Multi-sample averaging (oversampling for noise reduction)
85      // TR: Çoklu örnekleme ortalaması (gürültü azaltma için aşırı örnekleme)
86      uint32_t convert_averaged(int channel, int samples) {
87          uint64_t sum = 0;
88          for (int i = 0; i < samples; ++i) {
89              sum += convert(channel);
90          }
91          return static_cast<uint32_t>(sum / static_cast<uint64_t>(samples));
92      }
93
94      const ADCConfig& config() const { return config_; }
95  };
96
97  // ============================================================================
98  // PART 2: DAC Simulator
99  // ============================================================================
100
101 struct DACConfig {
102     uint8_t resolution_bits = 12;
103     double vref = 3.3;
104
105     uint32_t max_value() const {
106         return (1U << resolution_bits) - 1;
107     }
108 };
109
110 class DACSimulator {
111     DACConfig config_;
112     uint32_t current_value_ = 0;
113
114 public:
115     explicit DACSimulator(DACConfig cfg = {}) : config_(std::move(cfg)) {}
116
117     void write(uint32_t value) {
118         current_value_ = std::min(value, config_.max_value());
119     }
120
121     double output_voltage() const {
122         return static_cast<double>(current_value_) * config_.vref
123                / static_cast<double>(config_.max_value());
124     }
125
126     uint32_t current_value() const { return current_value_; }
127     const DACConfig& config() const { return config_; }
128 };
129
130 // ============================================================================
131 // PART 3: Automotive Sensor Models
132 // ============================================================================
133
134 // EN: Common automotive sensor transfer functions
135 // TR: Yaygın otomotiv sensör transfer fonksiyonları
136
137 struct SensorCalibration {
138     std::string name;
139     std::string unit;
140     double v_min;       // voltage at physical minimum — TR: fiziksel minimumdaki gerilim
141     double v_max;       // voltage at physical maximum — TR: fiziksel maksimumdaki gerilim
142     double phys_min;    // physical minimum — TR: fiziksel minimum değer
143     double phys_max;    // physical maximum — TR: fiziksel maksimum değer
144
145     // EN: Linear transfer function: V → Physical
146     // TR: Doğrusal transfer fonksiyonu: gerilim → fiziksel değer
147     double voltage_to_physical(double voltage) const {
148         if (std::abs(v_max - v_min) < 1e-9) return phys_min;
149         return phys_min + (voltage - v_min) / (v_max - v_min) * (phys_max - phys_min);
150     }
```

```cpp

    // EN: Physical → Voltage
    // TR: Fiziksel değer → gerilim (DAC çıkışı hesaplaması için)
    double physical_to_voltage(double physical) const {
        if (std::abs(phys_max - phys_min) < 1e-9) return v_min;
        return v_min + (physical - phys_min) / (phys_max - phys_min) * (v_max - v_min);
    }

    // EN: Check if voltage is in valid range (open/short circuit detection)
    // TR: Gerilimin geçerli aralıkta olup olmadığını kontrol et
    //     (açık/kısa devre tespiti — otomotivde sensör hatası teşhisi için kritik)
    bool is_valid(double voltage) const {
        constexpr double margin = 0.1;  // 100 mV margin — TR: 100 mV tolerans marjı
        return voltage >= (v_min - margin) && voltage <= (v_max + margin);
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 4: Waveform Generator (DAC Application)
// ═══════════════════════════════════════════════════════════════════════════

class WaveformGenerator {
    DACSimulator& dac_;

public:
    explicit WaveformGenerator(DACSimulator& dac) : dac_(dac) {}

    // EN: Generate one period of waveform, return voltage samples
    // TR: Bir periyotluk dalga formu üret, gerilim örneklerini döndür
    std::vector<double> generate_sine(int samples_per_period, double amplitude_v,
                                      double offset_v) {
        std::vector<double> result;
        result.reserve(static_cast<size_t>(samples_per_period));
        for (int i = 0; i < samples_per_period; ++i) {
            double t = static_cast<double>(i) / static_cast<double>(samples_per_period);
            double v = offset_v + amplitude_v * std::sin(2.0 * M_PI * t);
            v = std::clamp(v, 0.0, dac_.config().vref);
            uint32_t digital = static_cast<uint32_t>(
                std::round(v / dac_.config().vref * static_cast<double>(dac_.config().max_value())));
            dac_.write(digital);
            result.push_back(dac_.output_voltage());
        }
        return result;
    }

    std::vector<double> generate_sawtooth(int samples_per_period, double max_v) {
        std::vector<double> result;
        result.reserve(static_cast<size_t>(samples_per_period));
        for (int i = 0; i < samples_per_period; ++i) {
            double v = max_v * static_cast<double>(i) / static_cast<double>(samples_per_period);
            uint32_t digital = static_cast<uint32_t>(
                std::round(v / dac_.config().vref * static_cast<double>(dac_.config().max_value())));
            dac_.write(digital);
            result.push_back(dac_.output_voltage());
        }
        return result;
    }

    std::vector<double> generate_triangle(int samples_per_period, double max_v) {
        std::vector<double> result;
        result.reserve(static_cast<size_t>(samples_per_period));
        int half = samples_per_period / 2;
        for (int i = 0; i < samples_per_period; ++i) {
            double v;
            if (i < half)
                v = max_v * static_cast<double>(i) / static_cast<double>(half);
            else
                v = max_v * static_cast<double>(samples_per_period - i) / static_cast<double>(half);
            uint32_t digital = static_cast<uint32_t>(
                std::round(v / dac_.config().vref * static_cast<double>(dac_.config().max_value())));
            dac_.write(digital);
            result.push_back(dac_.output_voltage());
        }
        return result;
    }
};
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════
// PART 5: Oversampling & Averaging Filter
// ═══════════════════════════════════════════════════════════════════════════

// EN: Oversampling effectively increases ADC resolution.
//     Extra bits = log2(oversampling_ratio) / 2
//     4x oversampling = +1 bit, 16x = +2 bits, 64x = +3 bits, 256x = +4 bits
// TR: Aşırı örnekleme ADC çözünürlüğünü etkin olarak artırır.
//     Ekstra bit = log2(örnekleme_oranı) / 2
//     4x = +1 bit, 16x = +2 bit, 64x = +3 bit, 256x = +4 bit
//     12-bit ADC + 256x örnekleme = etkin 16-bit çözünürlük (yavaş ama hassas)

struct OversamplingResult {
    uint32_t raw_avg;
    double effective_resolution_bits;
    double voltage;
    int num_samples;
};

OversamplingResult oversample(ADCSimulator& adc, int channel, int ratio) {
    uint64_t sum = 0;
    for (int i = 0; i < ratio; ++i) {
        sum += adc.convert(channel);
    }
    uint32_t avg = static_cast<uint32_t>(sum / static_cast<uint64_t>(ratio));
    double extra_bits = std::log2(static_cast<double>(ratio)) / 2.0;
    double eff_res = static_cast<double>(static_cast<uint32_t>(adc.config().resolution)) +
 extra_bits;

    return {avg, eff_res, adc.to_voltage(avg), ratio};
}

// ═══════════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 15 - ADC & DAC Programming\n";
    std::cout << "============================================\n\n";

    // ─── Demo 1: ADC Resolution Comparison ──────────────────────────────
    {
        std::cout << "--- Demo 1: ADC Resolution Comparison ---\n\n";
        std::cout << "  Input voltage: 1.65V (Vref=3.3V, mid-scale)\n\n";

        std::cout << "  ┌──────────────┬──────────────┬──────────────────┬────────────────┐\n";
        std::cout << "  │ Resolution   │ Max Value    │ Digital Output   │ LSB Voltage    │\n";
        std::cout << "  ├──────────────┼──────────────┼──────────────────┼────────────────┤\n";

        for (auto res : {ADCResolution::Bit8, ADCResolution::Bit10,
                         ADCResolution::Bit12, ADCResolution::Bit16}) {
            ADCConfig cfg{res, 3.3, 1000000, false};
            ADCSimulator adc(cfg);
            adc.set_channel_voltage(0, 1.65);
            uint32_t raw = adc.convert(0);
            double lsb = cfg.lsb_voltage();

            std::cout << "  │ " << std::setw(6) << static_cast<int>(res) << "-bit"
                      << " │ " << std::setw(10) << cfg.max_value()
                      << " │ " << std::setw(14) << raw
                      << " │ " << std::fixed << std::setprecision(6)
                      << std::setw(9) << (lsb * 1000.0) << " mV │\n";
        }
        std::cout << "  └──────────────┴──────────────┴──────────────────┴────────────────┘\n\n";
    }
```

```cpp

    // ─── Demo 2: Automotive Sensor Reading ──────────────────────────────
    {
        std::cout << "--- Demo 2: Automotive Sensor Reading ---\n\n";

        ADCSimulator adc(ADCConfig{ADCResolution::Bit12, 3.3});

        // EN: Define automotive sensors
        // TR: Otomotiv sensörlerini tanımla (her biri farklı gerilim-fiziksel aralığa sahip)
        std::vector<SensorCalibration> sensors = {
            {"Coolant Temp (NTC)",  "°C",   0.5, 2.8,  -40.0, 150.0},
            {"Throttle Position",   "%",    0.5, 4.5,    0.0, 100.0},
            {"MAP Sensor",          "kPa",  0.4, 4.65,  20.0, 250.0},
            {"O2 Sensor (Lambda)",  "V",    0.1, 0.9,    0.1,   0.9},
            {"Battery Voltage",     "V",    0.0, 3.3,    0.0,  16.5},
            {"Fuel Level",          "%",    0.3, 3.0,    0.0, 100.0},
        };

        // EN: Set simulated sensor voltages
        // TR: Simüle edilmiş sensör gerilimlerini ayarla
        double test_voltages[] = {1.8, 2.5, 2.0, 0.45, 2.8, 1.5};

        std::cout << "  ┌──────────────────────┬──────────┬──────────┬──────────┬────────┐\n";
        std::cout << "  │ Sensor               │ V_in     │ Raw ADC  │ V_read   │ Phys   │\n";
        std::cout << "  ├──────────────────────┼──────────┼──────────┼──────────┼────────┤\n";

        for (size_t i = 0; i < sensors.size(); ++i) {
            int ch = static_cast<int>(i);
            adc.set_channel_voltage(ch, test_voltages[i]);
            uint32_t raw = adc.convert_averaged(ch, 16);
            double v_read = adc.to_voltage(raw);
            double phys = sensors[i].voltage_to_physical(v_read);
            bool valid = sensors[i].is_valid(v_read);

            std::cout << "  │ " << std::left << std::setw(21) << sensors[i].name
                      << std::right
                      << " │ " << std::fixed << std::setprecision(3)
                      << std::setw(5) << test_voltages[i] << " V"
                      << " │ " << std::setw(7) << raw
                      << " │ " << std::setprecision(3) << std::setw(7) << v_read << " V"
                      << " │ " << std::setprecision(1) << std::setw(5) << phys
                      << (valid ? " " : "!") << " │\n";
        }
        std::cout << "  └──────────────────────┴──────────┴──────────┴──────────┴────────┘\n";
        std::cout << "  (! = out-of-range / sensor fault)\n\n";
    }

    // ─── Demo 3: Oversampling for Resolution ────────────────────────────
    {
        std::cout << "--- Demo 3: Oversampling for Higher Resolution ---\n\n";

        ADCSimulator adc(ADCConfig{ADCResolution::Bit12, 3.3});
        adc.set_channel_voltage(0, 1.234);  // precise voltage

        std::cout << "  Target voltage: 1.234V (12-bit ADC, Vref=3.3V)\n\n";
        std::cout << "  ┌──────────────────┬────────────────┬──────────────┬────────────┐\n";
        std::cout << "  │ Oversampling     │ Eff. Res.      │ Raw Avg      │ Voltage    │\n";
        std::cout << "  ├──────────────────┼────────────────┼──────────────┼────────────┤\n";

        for (int ratio : {1, 4, 16, 64, 256}) {
            auto result = oversample(adc, 0, ratio);
            std::cout << "  │ " << std::setw(7) << ratio << "× samples"
                      << " │ " << std::fixed << std::setprecision(1)
                      << std::setw(8) << result.effective_resolution_bits << " bits"
                      << " │ " << std::setw(10) << result.raw_avg
                      << " │ " << std::setprecision(4)
                      << std::setw(7) << result.voltage << " V │\n";
        }
        std::cout << "  └──────────────────┴────────────────┴──────────────┴────────────┘\n\n";
    }
```

```

    // ── Demo 4: DAC Basic Operation ──────────────────────────────────
    {
        std::cout << "--- Demo 4: DAC Basic Operation ---\n\n";

        DACSimulator dac(DACConfig{12, 3.3});

        std::cout << "  12-bit DAC, Vref=3.3V:\n\n";

        struct TestCase { uint32_t value; std::string label; };
        std::vector<TestCase> tests = {
            {0,    "Zero"},
            {2048, "Mid-scale"},
            {4095, "Full-scale"},
            {1000, "~0.806V"},
            {3000, "~2.418V"},
        };

        std::cout << "  ┌──────────────┬──────────────┬──────────────┐\n";
        std::cout << "  │ Digital      │ Voltage      │ Label        │\n";
        std::cout << "  ├──────────────┼──────────────┼──────────────┤\n";
        for (auto& t : tests) {
            dac.write(t.value);
            std::cout << "  │ " << std::setw(5) << t.value << " / 4095"
                      << " │ " << std::fixed << std::setprecision(3)
                      << std::setw(7) << dac.output_voltage() << " V"
                      << "   │ " << std::left << std::setw(12) << t.label
                      << std::right << " │\n";
        }
        std::cout << "  └──────────────┴──────────────┴──────────────┘\n\n";
    }

    // ── Demo 5: DAC Waveform Generation ──────────────────────────────
    {
        std::cout << "--- Demo 5: DAC Waveform Generation ---\n\n";

        DACSimulator dac(DACConfig{12, 3.3});
        WaveformGenerator wg(dac);

        // EN: Generate sine wave (1 period, 20 samples)
        // TR: Sinüs dalgası üret (1 periyot, 20 örnek — DAC çıkışı simülasyonu)
        auto sine = wg.generate_sine(20, 1.5, 1.65);

        std::cout << "  Sine wave (Amp=1.5V, Offset=1.65V):\n  ";
        // EN: ASCII art waveform
        // TR: ASCII dalga formu görselleştirmesi
        for (size_t i = 0; i < sine.size(); ++i) {
            int col = static_cast<int>(sine[i] / 3.3 * 30.0);
            for (int j = 0; j < col; ++j) std::cout << ' ';
            std::cout << '*' << "\n  ";
        }
        std::cout << "\n";

        // EN: Triangle wave
        // TR: Üçgen dalga formu
        auto tri = wg.generate_triangle(20, 3.0);
        std::cout << "  Triangle wave (max=3.0V):\n  ";
        for (size_t i = 0; i < tri.size(); ++i) {
            int col = static_cast<int>(tri[i] / 3.3 * 30.0);
            for (int j = 0; j < col; ++j) std::cout << ' ';
            std::cout << '*' << "\n  ";
        }
        std::cout << "\n";
    }

    // ── Demo 6: Sensor Fault Detection ───────────────────────────────
    {
        std::cout << "--- Demo 6: Sensor Fault Detection (Open/Short Circuit) ---\n\n";

        SensorCalibration throttle{"Throttle Position", "%", 0.5, 4.5, 0.0, 100.0};

        struct FaultTest {
            double voltage;
            std::string scenario;
        };

```

```
438         std::vector<FaultTest> tests = {
439             {2.5,  "Normal (50% throttle)"},
440             {0.0,  "SHORT to GND"},
441             {3.3,  "SHORT to Vcc (3.3V)"},
442             {0.3,  "Below valid range (wire damage?)"},
443             {0.5,  "At min boundary"},
444         };
445 
446         std::cout << "  ┌─────────┬──────────────────────────────┬────────────┬────────┐\n";
447         std::cout << "  │ V_in    │ Scenario                     │ Physical   │ Status │\n";
448         std::cout << "  ├─────────┼──────────────────────────────┼────────────┼────────┤\n";
449         for (auto& t : tests) {
450             double phys = throttle.voltage_to_physical(t.voltage);
451             bool valid = throttle.is_valid(t.voltage);
452             std::cout << "  │ " << std::fixed << std::setprecision(1)
453                       << std::setw(5) << t.voltage << " V"
454                       << " │ " << std::left << std::setw(33) << t.scenario << std::right
455                       << " │ " << std::setprecision(1) << std::setw(7) << phys << " %"
456                       << "  │ " << std::setw(6) << (valid ? "OK" : "FAULT") << " │\n";
457         }
458         std::cout << "  └─────────┴──────────────────────────────┴────────────┴────────┘\n\n";
459     }
460 
```

```cpp
    // ── Demo 7: ADC+DAC Loopback ──────────────────────────────────────
    {
        std::cout << "--- Demo 7: ADC+DAC Loopback (Closed-Loop Control) ---\n\n";

        std::cout << "  Scenario: PID-like control using ADC to read sensor,\n";
        std::cout << "            DAC to drive actuator.\n\n";

        ADCSimulator adc(ADCConfig{ADCResolution::Bit12, 3.3});
        DACSimulator dac(DACConfig{12, 3.3});

        double target_temp = 80.0;  // °C target
        SensorCalibration temp_sensor{"Coolant Temp", "°C", 0.5, 2.8, -40.0, 150.0};

        // EN: Simulate simple proportional control
        // TR: Basit oransal kontrol simülasyonu (P kontrol — hata × Kp = çıkış)
        double current_temp = 60.0;  // starting temperature
        double kp = 0.05;            // proportional gain — TR: oransal kazanç

        std::cout << "  Target: " << target_temp << " °C\n\n";
        std::cout << "  ┌──────┬──────────┬─────────┬─────────┬─────────┬─────────┐\n";
        std::cout << "  │ Step │ Temp(°C) │ ADC Raw │ Error   │ DAC Out │ Actuator │\n";
        std::cout << "  ├──────┼──────────┼─────────┼─────────┼─────────┼─────────┤\n";

        for (int step = 0; step < 8; ++step) {
            // Sensor voltage from current temperature
            double v_sensor = temp_sensor.physical_to_voltage(current_temp);
            adc.set_channel_voltage(0, v_sensor);
            uint32_t raw = adc.convert_averaged(0, 4);
            double v_read = adc.to_voltage(raw);
            double measured_temp = temp_sensor.voltage_to_physical(v_read);

            double error = target_temp - measured_temp;
            // DAC output = proportional to error (0V=off, 3.3V=full power)
            double dac_v = std::clamp(error * kp, 0.0, 3.3);
            uint32_t dac_val = static_cast<uint32_t>(
                std::round(dac_v / 3.3 * 4095.0));
            dac.write(dac_val);

            std::cout << "  │ " << std::setw(4) << step
                      << " │ " << std::fixed << std::setprecision(1)
                      << std::setw(6) << current_temp << " °"
                      << " │ " << std::setw(7) << raw
                      << " │ " << std::setprecision(1)
                      << std::setw(6) << error << " °"
                      << " │ " << std::setprecision(3)
                      << std::setw(7) << dac.output_voltage() << " V"
                      << " │ " << std::setprecision(0)
                      << std::setw(5) << (dac.output_voltage()/3.3*100.0) << " %"
                      << "  │\n";

            // EN: Simulate temperature rising toward target
            // TR: Sıcaklığın hedefe doğru yükseldiğini simüle et
            current_temp += error * 0.3;
        }
        std::cout << "  └──────┴──────────┴─────────┴─────────┴─────────┴─────────┘\n\n";
    }

    // ── Demo 8: ADC Architecture Comparison ──────────────────────────
    {
        std::cout << "--- Demo 8: ADC Architecture Comparison ---\n\n";

        std::cout << "  ┌──────────────────┬──────────┬──────────┬──────────┬──────────────────────┐\n";
        std::cout << "  │ Architecture     │ Speed    │ Res.     │ Power    │ Typical Application  │\n";
        std::cout << "  ├──────────────────┼──────────┼──────────┼──────────┼──────────────────────┤\n";
        std::cout << "  │ SAR              │ 1M SPS   │ 12-18b   │ Low      │ MCU (STM32, NXP)     │\n";
        std::cout << "  │ Sigma-Delta (ΣΔ) │ 10 SPS   │ 24b      │ Low      │ Weigh scales, temp   │\n";
        std::cout << "  │ Pipeline         │ 100M SPS │ 12-16b   │ High     │ Radar, video, SDR    │\n";
        std::cout << "  │ Flash            │ 1G SPS   │ 4-8b     │ V.High   │ Oscilloscope, telecom │\n";
        std::cout << "  │ Dual-slope       │ <10 SPS  │ 14-20b   │ V.Low    │ DMM, precision meters │\n";
```

```
        std::cout << "  └────────────────────┴──────────┴──────────┴──────────┴──────────────────────┘\n\n";

        std::cout << "  Automotive ADC Selection:\n";
        std::cout << "    - Engine control (MAP, TPS): 12-bit SAR, ~1 MSPS\n";
        std::cout << "    - Battery management (cell V): 16-bit ΣΔ, precise\n";
        std::cout << "    - Radar front-end: 12-14 bit pipeline, 50+ MSPS\n";
        std::cout << "    - Knock sensor: 10-bit SAR, fast sampling for vibration\n";
        std::cout << "    - Interior temp/humidity: 10-bit SAR, low-power\n";
    }

    std::cout << "\n==========================================\n";
    std::cout << "   End of ADC & DAC Programming\n";
    std::cout << "==========================================\n";

    return 0;
}
```

► 03. Serial Protocols Deep

module_15_hw_interfaces/03_serial_protocols_deep.cpp

Konu: SPI/I2C/UART — Register-Level Deep Dive & Advanced Patterns

Teori / Theory

THEORY: Serial Protocol Register-Level Programming

This file goes deeper than module_14's overview. Here we focus on:
- Register-level peripheral initialization
- Error detection and recovery
- DMA integration patterns
- Multi-master/multi-slave topologies
- Real hardware register maps (STM32-like)

SPI Register Map (STM32):

```
┌─────────┬────────┬──────────────────────────────────┐
│ Offset  │ Name   │ Key Fields                       │
├─────────┼────────┼──────────────────────────────────┤
│ 0x00    │ CR1    │ BIDIMODE, BIDIOE, CRCEN, CRCNEXT,      │
│         │        │ DFF, RXONLY, SSM, SSI, LSBFIRST,       │
│         │        │ SPE, BR[2:0], MSTR, CPOL, CPHA         │
│ 0x04    │ CR2    │ TXEIE, RXNEIE, ERRIE, SSOE,            │
│         │        │ TXDMAEN, RXDMAEN                       │
│ 0x08    │ SR     │ BSY, OVR, MODF, CRCERR, UDR,           │
│         │        │ TXE, RXNE                              │
│ 0x0C    │ DR     │ Data register (8/16-bit)               │
│ 0x14    │ CRCPR  │ CRC polynomial register                │
└─────────┴────────┴──────────────────────────────────┘
```

I2C Register Map (STM32):

```
┌─────────┬────────┬──────────────────────────────────┐
│ Offset  │ Name   │ Key Fields                       │
├─────────┼────────┼──────────────────────────────────┤
│ 0x00    │ CR1    │ SWRST, ALERT, PEC, POS, ACK, STOP,     │
│         │        │ START, NOSTRETCH, ENGC, PE             │
│ 0x04    │ CR2    │ LAST, DMAEN, ITBUFEN, ITEVTEN,         │
│         │        │ ITERREN, FREQ[5:0]                     │
│ 0x08    │ OAR1   │ ADDMODE, ADD[9:0]                      │
│ 0x14    │ SR1    │ SB, ADDR, BTF, ADD10, STOPF,           │
│         │        │ RXNE, TXE, BERR, ARLO, AF, OVR         │
│ 0x18    │ SR2    │ DUALF, SMBHOST, SMBDEFAULT, GENCALL,   │
│         │        │ TRA, BUSY, MSL                         │
│ 0x1C    │ CCR    │ F/S, DUTY, CCR[11:0]                   │
└─────────┴────────┴──────────────────────────────────┘
```

UART Register Map (STM32):

```
┌─────────┬────────┬──────────────────────────────────┐
│ Offset  │ Name   │ Key Fields                       │
├─────────┼────────┼──────────────────────────────────┤
│ 0x00    │ SR     │ CTS, LBD, TXE, TC, RXNE, IDLE, ORE,    │
│         │        │ NF, FE, PE                             │
│ 0x04    │ DR     │ Data register (9-bit)                  │
│ 0x08    │ BRR    │ DIV_Mantissa[11:0], DIV_Fraction[3:0]  │
│ 0x0C    │ CR1    │ UE, M, WAKE, PCE, PS, PEIE, TXEIE,     │
│         │        │ TCIE, RXNEIE, IDLEIE, TE, RE           │
│ 0x10    │ CR2    │ STOP[1:0], CLKEN, CPOL, CPHA, LBCL     │
│ 0x14    │ CR3    │ DMAT, DMAR, SCEN, NACK, HDSEL,         │
│         │        │ IRLP, IREN, EIE                        │
└─────────┴────────┴──────────────────────────────────┘
```

TEORİ: Seri Protokol Yazmaç Düzeyi Programlama

Bu dosya modül 14'teki genel bakıştan daha derine iner. Yazmaç seviyesinde çevre birimi başlatma, hata tespiti, kurtarma ve DMA entegrasyonu ele alınır.
SPI Yazmaç Haritası (STM32):

```
┌─────────┬────────┬──────────────────────────────────┐
│ Ofset   │ Ad     │ Temel Alanlar                    │
├─────────┼────────┼──────────────────────────────────┤
│ 0x00    │ CR1    │ MSTR, CPOL, CPHA, BR[2:0], SPE, DFF,  │
│         │        │ SSM, SSI, LSBFIRST, CRCEN, BIDIMODE   │
│ 0x04    │ CR2    │ TXEIE, RXNEIE, ERRIE, TXDMAEN,RXDMAEN │
│ 0x08    │ SR     │ TXE, RXNE, BSY, OVR, MODF, CRCERR     │
│ 0x0C    │ DR     │ Veri yazmacı (8/16-bit)               │
│ 0x14    │ CRCPR  │ CRC polinom yazmacı                   │
└─────────┴────────┴──────────────────────────────────┘
```

I2C Yazmaç Haritası (STM32):

```
┌─────────┬────────┬──────────────────────────────────┐
│ Ofset   │ Ad     │ Temel Alanlar                    │
├─────────┼────────┼──────────────────────────────────┤
│ 0x00    │ CR1    │ START, STOP, ACK, PE, SWRST           │
│ 0x04    │ CR2    │ FREQ[5:0], DMAEN, ITEVTEN, ITERREN    │
│ 0x08    │ OAR1   │ ADDMODE, ADD[9:0]                     │
│ 0x14    │ SR1    │ SB, ADDR, BTF, TXE, RXNE, AF, ARLO    │
│ 0x18    │ SR2    │ BUSY, MSL, TRA                        │
│ 0x1C    │ CCR    │ F/S, CCR[11:0] — f_PCLK/(2×f_SCL)     │
└─────────┴────────┴──────────────────────────────────┘
```

UART Yazmaç Haritası (STM32):

```
┌─────────┬────────┬──────────────────────────────────┐
│ Ofset   │ Ad     │ Temel Alanlar                    │
├─────────┼────────┼──────────────────────────────────┤
│ 0x00    │ SR     │ TXE, TC, RXNE, ORE, FE, PE, NF        │
│ 0x04    │ DR     │ Veri yazmacı (9-bit)                  │
│ 0x08    │ BRR    │ DIV_Mantissa[11:0], Fraction[3:0]     │
│ 0x0C    │ CR1    │ UE, TE, RE, RXNEIE, TXEIE, PCE, M     │
│ 0x10    │ CR2    │ STOP[1:0], CLKEN, CPOL, CPHA          │
│ 0x14    │ CR3    │ DMAT, DMAR, HDSEL, SCEN, EIE          │
└─────────┴────────┴──────────────────────────────────┘
```

DMA Entegrasyonu: Çevre birimi ↔ bellek transferini CPU müdahalesi olmadan gerçekleştirir — yüksek hızlı SPI Flash okumaları için gerekli.

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <array>
#include <vector>
#include <string>
#include <iomanip>
#include <bitset>
#include <queue>
#include <functional>
#include <optional>
#include <map>
#include <cassert>

```

```cpp
// ═══════════════════════════════════════════════════════════════════════════
// PART 1: SPI Register-Level Simulation
// ═══════════════════════════════════════════════════════════════════════════

namespace spi_regs {
    // CR1 bit positions — TR: CR1 bit pozisyonları
    constexpr uint32_t CPHA      = 0;
    constexpr uint32_t CPOL      = 1;
    constexpr uint32_t MSTR      = 2;
    constexpr uint32_t BR_SHIFT  = 3;  // BR[2:0] — TR: Baud hızı böleni (3 bit)
    constexpr uint32_t BR_MASK   = 0x7;
    constexpr uint32_t SPE       = 6;
    constexpr uint32_t LSBFIRST  = 7;
    constexpr uint32_t SSI       = 8;
    constexpr uint32_t SSM       = 9;
    constexpr uint32_t DFF       = 11; // 0=8-bit, 1=16-bit — TR: veri çerçeve formatı

    // SR bit positions — TR: SR (Durum Yazmaçı) bit pozisyonları
    constexpr uint32_t RXNE      = 0;
    constexpr uint32_t TXE       = 1;
    constexpr uint32_t BSY       = 7;
    constexpr uint32_t OVR       = 6;
    constexpr uint32_t MODF      = 5;

    // CR2 bit positions — TR: CR2 bit pozisyonları
    constexpr uint32_t RXDMAEN   = 0;
    constexpr uint32_t TXDMAEN   = 1;
    constexpr uint32_t RXNEIE    = 6;
    constexpr uint32_t TXEIE     = 7;
}

class SPIPeripheral {
    uint32_t CR1_ = 0;
    uint32_t CR2_ = 0;
    uint32_t SR_  = (1U << spi_regs::TXE);  // TXE = 1 at reset — TR: sıfırlamada TXE=1
    uint32_t DR_  = 0;

    std::queue<uint8_t> rx_fifo_;
    std::function<uint8_t(uint8_t)> slave_handler_;

public:
    // EN: Register write
    // TR: Yazmaç yazma — doğrudan CR1'e değer atar (HAL katmanı yerine)
    void write_CR1(uint32_t val) { CR1_ = val; }
    void write_CR2(uint32_t val) { CR2_ = val; }

    uint32_t read_CR1() const { return CR1_; }
    uint32_t read_SR()  const { return SR_; }

    void set_slave_handler(std::function<uint8_t(uint8_t)> h) {
        slave_handler_ = std::move(h);
    }

    // EN: Write data register → initiates transfer
    // TR: Veri yazmacına yaz → aktarımı başlatır (full-duplex: aynı anda gönder/al)
    void write_DR(uint8_t data) {
        if (!(CR1_ & (1U << spi_regs::SPE))) return;  // SPI not enabled — TR: SPI etkin değil

        SR_ &= ~(1U << spi_regs::TXE);  // TXE = 0 during transfer — TR: aktarım sırasında
        SR_ |= (1U << spi_regs::BSY);     // BSY = 1 — TR: meşgul

        // EN: Simulate full-duplex transfer
        // TR: Tam çift yönlü aktarımı simüle et — MOSI'dan gönderirken MISO'dan alır
        uint8_t rx = slave_handler_ ? slave_handler_(data) : 0xFF;

        rx_fifo_.push(rx);
        SR_ |= (1U << spi_regs::RXNE);   // RXNE = 1 — TR: alınan veri var
        SR_ |= (1U << spi_regs::TXE);     // TXE = 1 (ready for next) — TR: sonraki veriye hazır
        SR_ &= ~(1U << spi_regs::BSY);    // BSY = 0 — TR: aktarım bitti
    }
```

```cpp

    // EN: Read data register
    // TR: Veri yazmacını oku — RXNE sıfırlanır, FIFO'dan veri çekilir
    uint8_t read_DR() {
        if (rx_fifo_.empty()) return 0;
        uint8_t data = rx_fifo_.front();
        rx_fifo_.pop();
        if (rx_fifo_.empty())
            SR_ &= ~(1U << spi_regs::RXNE);  // RXNE = 0 — TR: alınan veri okundu
        return data;
    }

    bool is_tx_empty() const { return SR_ & (1U << spi_regs::TXE); }
    bool is_rx_ready() const { return SR_ & (1U << spi_regs::RXNE); }
    bool is_busy() const { return SR_ & (1U << spi_regs::BSY); }

    void print_config() const {
        bool cpol = CR1_ & (1U << spi_regs::CPOL);
        bool cpha = CR1_ & (1U << spi_regs::CPHA);
        bool master = CR1_ & (1U << spi_regs::MSTR);
        bool enabled = CR1_ & (1U << spi_regs::SPE);
        uint32_t br = (CR1_ >> spi_regs::BR_SHIFT) & spi_regs::BR_MASK;
        bool lsb = CR1_ & (1U << spi_regs::LSBFIRST);
        bool dff16 = CR1_ & (1U << spi_regs::DFF);

        int divisors[] = {2, 4, 8, 16, 32, 64, 128, 256};

        std::cout << "    SPI Configuration:\n"
                  << "      Mode:     " << (master ? "MASTER" : "SLAVE") << "\n"
                  << "      CPOL:     " << cpol << " (idle " << (cpol ? "HIGH" : "LOW") << ")\n"
                  << "      CPHA:     " << cpha << " (sample on " << (cpha ? "2nd" : "1st") << " edge)\n"
                  << "      SPI Mode: " << (cpol * 2 + cpha) << "\n"
                  << "      Baud div: f_PCLK/" << divisors[br] << "\n"
                  << "      Bit order:" << (lsb ? " LSB first" : " MSB first") << "\n"
                  << "      Frame:    " << (dff16 ? "16" : "8") << "-bit\n"
                  << "      Enabled:  " << (enabled ? "YES" : "NO") << "\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: I2C Register-Level Simulation
// ═══════════════════════════════════════════════════════════════════════════

namespace i2c_regs {
    // CR1 — EN: Control Register 1 — TR: Kontrol Yazmacı 1
    constexpr uint32_t PE       = 0;
    constexpr uint32_t START    = 8;
    constexpr uint32_t STOP     = 9;
    constexpr uint32_t ACK      = 10;
    // SR1 — EN: Status Register 1 — TR: Durum Yazmacı 1
    constexpr uint32_t SB       = 0;   // Start bit generated — TR: Start biti üretildi
    constexpr uint32_t ADDR     = 1;   // Address sent — TR: Adres gönderildi
    constexpr uint32_t BTF      = 2;   // Byte transfer finished — TR: Bayt aktarımı bitti
    constexpr uint32_t TXE      = 7;   // TX empty — TR: Gönder tamponu boş
    constexpr uint32_t RXNE     = 6;   // RX not empty — TR: Alınan veri var
    constexpr uint32_t AF       = 10;  // Acknowledge failure — TR: Onay hatası (NACK)
    constexpr uint32_t ARLO     = 9;   // Arbitration lost — TR: Arbitrasyon kaybı
    constexpr uint32_t BERR     = 8;   // Bus error — TR: Bus hatası
}

class I2CPeripheral {
    uint32_t CR1_ = 0;
    uint32_t CR2_ = 0;
    uint32_t SR1_ = 0;
    uint32_t SR2_ = 0;
    uint32_t CCR_ = 0;

    // EN: Simulated slave devices on the bus
    // TR: Bus üzerindeki simüle edilmiş slave cihazlar (her biri 7-bit adres + yazmaç haritası)
    struct I2CSlave {
        uint8_t address;   // 7-bit — TR: 7-bit I2C adresi
        std::map<uint8_t, uint8_t> registers;
    };
    std::vector<I2CSlave> slaves_;

```

```cpp
    uint8_t current_addr_ = 0;
    bool write_mode_ = true;
    int current_slave_ = -1;
    uint8_t current_reg_ = 0;
    int byte_count_ = 0;

public:
    void add_slave(uint8_t addr, std::map<uint8_t, uint8_t> regs) {
        slaves_.push_back({addr, std::move(regs)});
    }

    // EN: Enable peripheral
    // TR: Çevre birimini etkinleştir (PE=1 — bus erişimi başlar)
    void enable() { CR1_ |= (1U << i2c_regs::PE); }

    // EN: Generate START condition
    // TR: START koşulu üret — SDA, SCL yüksekken düşer (bus sahipliği alınır)
    bool start() {
        if (!(CR1_ & (1U << i2c_regs::PE))) return false;
        CR1_ |= (1U << i2c_regs::START);
        SR1_ |= (1U << i2c_regs::SB);
        byte_count_ = 0;
        return true;
    }

    // EN: Send address byte (7-bit addr + R/W)
    // TR: Adres baytı gönder (7-bit adres + R/W biti — 0=yaz, 1=oku)
    bool send_address(uint8_t addr_rw) {
        current_addr_ = static_cast<uint8_t>(addr_rw >> 1);
        write_mode_ = !(addr_rw & 1);

        current_slave_ = -1;
        for (size_t i = 0; i < slaves_.size(); ++i) {
            if (slaves_[i].address == current_addr_) {
                current_slave_ = static_cast<int>(i);
                break;
            }
        }

        SR1_ &= ~(1U << i2c_regs::SB);  // clear SB
        if (current_slave_ >= 0) {
            SR1_ |= (1U << i2c_regs::ADDR);  // address matched — TR: adres eşleşti
            SR1_ |= (1U << i2c_regs::TXE);
            return true;
        } else {
            SR1_ |= (1U << i2c_regs::AF);  // NACK — TR: cihaz bulunamadı
            return false;
        }
    }

    // EN: Write data byte
    // TR: Veri baytı yaz — ilk bayt yazmaç adresi, sonrakiler veri
    bool write_data(uint8_t data) {
        if (current_slave_ < 0) return false;

        if (byte_count_ == 0) {
            current_reg_ = data;  // first byte = register address — TR: ilk bayt = yazmaç adresi
        } else {
            slaves_[static_cast<size_t>(current_slave_)].registers[current_reg_] = data;
            current_reg_++;
        }
        byte_count_++;
        SR1_ |= (1U << i2c_regs::BTF);
        SR1_ |= (1U << i2c_regs::TXE);
        return true;
    }

    // EN: Read data byte
    // TR: Veri baytı oku — slave'in yazmaç adresinden sıralı okuma yapar
    std::optional<uint8_t> read_data() {
        if (current_slave_ < 0) return std::nullopt;

        auto& slave = slaves_[static_cast<size_t>(current_slave_)];
        auto it = slave.registers.find(current_reg_);
        uint8_t val = (it != slave.registers.end()) ? it->second : 0xFF;
        current_reg_++;
        SR1_ |= (1U << i2c_regs::RXNE);
```

```cpp
        SR1_ |= (1U << i2c_regs::BTF);
        return val;
    }

    // EN: Generate STOP condition
    // TR: STOP koşulu üret — SDA, SCL yüksekken yükselir (bus serbest bırakılır)
    void stop() {
        CR1_ |= (1U << i2c_regs::STOP);
        SR1_ = 0;
        current_slave_ = -1;
        byte_count_ = 0;
    }

    // EN: Error detection
    // TR: Hata tespiti — NACK/arbitrasyon/bus hatası bayrakları SR1'den okunur
    struct I2CError {
        bool nack = false;
        bool arbitration_lost = false;
        bool bus_error = false;
    };

    I2CError check_errors() const {
        return {
            (SR1_ & (1U << i2c_regs::AF)) != 0,
            (SR1_ & (1U << i2c_regs::ARLO)) != 0,
            (SR1_ & (1U << i2c_regs::BERR)) != 0,
        };
    }

    void set_clock(uint32_t pclk_mhz, uint32_t target_khz) {
        CR2_ = pclk_mhz & 0x3F;
        // EN: CCR calculation for standard mode
        // TR: CCR hesabı (standart mod) — saat bölücü değer
        // CCR = f_PCLK / (2 × f_SCL) — TR: saat bölücü formülü
        uint32_t ccr = (pclk_mhz * 1000) / (2 * target_khz);
        CCR_ = ccr & 0xFFF;
    }

    void print_config() const {
        std::cout << "    I2C Configuration:\n"
                  << "      Enabled: " << ((CR1_ & (1U << i2c_regs::PE)) ? "YES" : "NO") << "\n"
                  << "      PCLK:    " << (CR2_ & 0x3F) << " MHz\n"
                  << "      CCR:     " << (CCR_ & 0xFFF) << "\n"
                  << "      Slaves:  " << slaves_.size() << " on bus\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: UART Register-Level Simulation
// ═══════════════════════════════════════════════════════════════════════════

namespace uart_regs {
    // SR — EN: Status Register — TR: Durum Yazmacı
    constexpr uint32_t PE_FLAG  = 0;   // Parity error — TR: Parite hatası
    constexpr uint32_t FE       = 1;   // Framing error — TR: Çerçeve hatası
    constexpr uint32_t NF       = 2;   // Noise flag — TR: Gürültü bayrağı
    constexpr uint32_t ORE      = 3;   // Overrun error — TR: Taşma hatası
    constexpr uint32_t IDLE     = 4;
    constexpr uint32_t RXNE     = 5;
    constexpr uint32_t TC       = 6;   // Transmission complete — TR: Gönderim tamamlandı
    constexpr uint32_t TXE      = 7;   // Transmit data register empty — TR: Gönder yazmacı boş
    // CR1 — EN: Control Register 1 — TR: Kontrol Yazmacı 1
    constexpr uint32_t UE       = 13;  // USART enable — TR: USART etkinleştir
    constexpr uint32_t TE       = 3;   // Transmitter enable — TR: Verici aç
    constexpr uint32_t RE       = 2;   // Receiver enable — TR: Alıcı aç
    constexpr uint32_t RXNEIE   = 5;   // RXNE interrupt enable — TR: RXNE kesme etkin
    constexpr uint32_t TXEIE    = 7;   // TXE interrupt enable — TR: TXE kesme etkin
    constexpr uint32_t PCE      = 10;  // Parity control enable — TR: Parite kontrolü etkin
    constexpr uint32_t M        = 12;  // Word length (0=8bit, 1=9bit) — TR: Kelime uzunluğu
}
```

```cpp

class UARTPeripheral {
    uint32_t SR_  = (1U << uart_regs::TXE) | (1U << uart_regs::TC);
    uint32_t DR_  = 0;
    uint32_t BRR_ = 0;
    uint32_t CR1_ = 0;
    uint32_t CR2_ = 0;
    uint32_t CR3_ = 0;

    std::queue<uint8_t> tx_buffer_;
    std::queue<uint8_t> rx_buffer_;

    UARTPeripheral* connected_ = nullptr;

public:
    void connect(UARTPeripheral& other) {
        connected_ = &other;
        other.connected_ = this;
    }

    // EN: Configure baud rate
    // TR: Baud hızını yapılandır — BRR = f_PCLK / (16 × baud)
    // BRR = f_PCLK / (16 × baud) — TR: baud bölücü formülü
    void set_baud(uint32_t pclk_hz, uint32_t baud) {
        uint32_t div = pclk_hz / baud;
        uint32_t mantissa = div / 16;
        uint32_t fraction = div % 16;
        BRR_ = (mantissa << 4) | (fraction & 0xF);
    }

    void enable(bool tx, bool rx) {
        CR1_ |= (1U << uart_regs::UE);
        if (tx) CR1_ |= (1U << uart_regs::TE);
        if (rx) CR1_ |= (1U << uart_regs::RE);
    }

    // EN: Transmit byte
    // TR: Bayt gönder — TXE kontrolü yap, DR'ye yaz, bağlı UART'a ilet
    bool transmit(uint8_t data) {
        if (!(CR1_ & (1U << uart_regs::UE))) return false;
        if (!(CR1_ & (1U << uart_regs::TE))) return false;

        // Wait for TXE — TR: TXE bekle (gönder tamponu boş olmalı)
        if (!(SR_ & (1U << uart_regs::TXE))) return false;

        DR_ = data;
        SR_ &= ~(1U << uart_regs::TXE);
        SR_ &= ~(1U << uart_regs::TC);

        // EN: Simulate transmission → arrives at connected UART
        // TR: İletimi simüle et → bağlı UART'a ulaşır (TX → RX çapraz bağlantı)
        if (connected_) {
            connected_->receive_byte(data);
        }

        SR_ |= (1U << uart_regs::TXE);
        SR_ |= (1U << uart_regs::TC);
        return true;
    }

    // EN: Receive byte (called internally)
    // TR: Bayt al (dahili çağrı — karşı UART'tan gelen veri)
    void receive_byte(uint8_t data) {
        rx_buffer_.push(data);
        SR_ |= (1U << uart_regs::RXNE);
    }

    // EN: Read received byte
    // TR: Alınan baytı oku — RXNE temizlenir, kuyruktan çekilir
    std::optional<uint8_t> read() {
        if (rx_buffer_.empty()) return std::nullopt;
        uint8_t data = rx_buffer_.front();
        rx_buffer_.pop();
        if (rx_buffer_.empty())
            SR_ &= ~(1U << uart_regs::RXNE);
        return data;
    }
```

```cpp

    bool is_rx_ready() const { return SR_ & (1U << uart_regs::RXNE); }
    bool is_tx_empty() const { return SR_ & (1U << uart_regs::TXE); }

    struct UARTError {
        bool parity  = false;
        bool framing = false;
        bool noise   = false;
        bool overrun = false;
    };

    UARTError check_errors() const {
        return {
            (SR_ & (1U << uart_regs::PE_FLAG)) != 0,
            (SR_ & (1U << uart_regs::FE)) != 0,
            (SR_ & (1U << uart_regs::NF)) != 0,
            (SR_ & (1U << uart_regs::ORE)) != 0,
        };
    }

    // EN: Simulate overrun error (new data when RXNE still set)
    // TR: Taşma hatası simüle et (RXNE hala 1 iken yeni veri gelirse veri kaybolur)
    void simulate_overrun() {
        SR_ |= (1U << uart_regs::ORE);
    }

    void clear_errors() {
        SR_ &= ~((1U << uart_regs::PE_FLAG) | (1U << uart_regs::FE) |
                 (1U << uart_regs::NF) | (1U << uart_regs::ORE));
    }

    void print_config() const {
        uint32_t mantissa = BRR_ >> 4;
        uint32_t fraction = BRR_ & 0xF;
        bool enabled = CR1_ & (1U << uart_regs::UE);
        bool tx_en = CR1_ & (1U << uart_regs::TE);
        bool rx_en = CR1_ & (1U << uart_regs::RE);

        std::cout << "    UART Configuration:\n"
                  << "      Enabled: " << (enabled ? "YES" : "NO") << "\n"
                  << "      TX:      " << (tx_en ? "ON" : "OFF") << "\n"
                  << "      RX:      " << (rx_en ? "ON" : "OFF") << "\n"
                  << "      BRR:     " << mantissa << "." << fraction
                  << " (mantissa.fraction)\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 4: DMA Channel Simulation
// ═══════════════════════════════════════════════════════════════════════════

// EN: Simulates DMA transfer between memory and peripheral
// TR: DMA transferini simüle eder — çevre birimi ↔ bellek arası CPU müdahalesi olmadan veri taşır
struct DMAChannel {
    enum class Direction { PeriphToMem, MemToPeriph };
    enum class Mode { Normal, Circular };

    Direction direction = Direction::PeriphToMem;
    Mode mode = Mode::Normal;
    uint32_t data_count = 0;
    uint32_t transferred = 0;
    bool enabled = false;
    bool complete = false;

    std::vector<uint8_t> buffer;

    void configure(Direction dir, Mode m, uint32_t count) {
        direction = dir;
        mode = m;
        data_count = count;
        buffer.resize(count, 0);
        transferred = 0;
        complete = false;
    }

```

```cpp
    void enable_channel() { enabled = true; complete = false; transferred = 0; }

    // EN: Simulate one DMA transfer step
    // TR: Bir DMA transfer adımını simüle et — Normal modda bittiğinde durur, Circular modda başa döner
    bool transfer_step(uint8_t data_in, uint8_t& data_out) {
        if (!enabled || complete) return false;

        if (direction == Direction::PeriphToMem) {
            buffer[transferred] = data_in;
            data_out = data_in;
        } else {
            data_out = buffer[transferred];
        }

        transferred++;
        if (transferred >= data_count) {
            if (mode == Mode::Normal) {
                complete = true;
                enabled = false;
            } else {
                transferred = 0;  // circular — TR: döngüsel mod, başa sar
            }
        }
        return true;
    }

    void print_status() const {
        std::cout << "    DMA: " << transferred << "/" << data_count
                  << " [" << (enabled ? "RUNNING" : "STOPPED")
                  << (complete ? " COMPLETE" : "")
                  << "] " << (mode == Mode::Circular ? "CIRCULAR" : "NORMAL") << "\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 15 - Serial Protocols Deep Dive\n";
    std::cout << "============================================\n\n";

    // ─── Demo 1: SPI Register-Level Init ──────────────────────────────
    {
        std::cout << "--- Demo 1: SPI Register-Level Initialization ---\n\n";

        SPIPeripheral spi;

        // EN: Step-by-step register configuration (as in STM32 HAL)
        // TR: Adım adım yazmaç yapılandırması (STM32 HAL kalıbı — önce CR1, sonra SPE)
        uint32_t cr1 = 0;
        cr1 |= (1U << spi_regs::MSTR);      // Master mode — TR: Master modu
        cr1 |= (1U << spi_regs::CPOL);       // CPOL=1 — TR: boşta yüksek
        cr1 |= (0U << spi_regs::CPHA);       // CPHA=0 → SPI Mode 2 — TR: düşen kenarda örnekle
        cr1 |= (0b010U << spi_regs::BR_SHIFT); // f_PCLK/8 — TR: saat bölücü
        cr1 |= (1U << spi_regs::SSM);        // Software slave management — TR: yazılımsal CS
        cr1 |= (1U << spi_regs::SSI);        // Internal slave select — TR: dahili slave seçimi
        // Do NOT set SPE yet — TR: SPE'yi henüz AYARLAMA (yapılandırma bitmeden)

        spi.write_CR1(cr1);

        std::cout << "  Step 1: CR1 = 0x" << std::hex << std::setw(4)
                  << std::setfill('0') << cr1 << std::dec << std::setfill(' ') << "\n";
        spi.print_config();

        // EN: Enable SPI (set SPE bit)
        // TR: SPI'yı etkinleştir (SPE biti — tüm yapılandırma bittikten sonra set edilir)
        cr1 |= (1U << spi_regs::SPE);
        spi.write_CR1(cr1);
        std::cout << "\n  Step 2: SPE=1 → SPI enabled\n\n";

```

```cpp
        // EN: Attach a simulated slave (ADXL345 accelerometer)
        // TR: Simüle edilmiş bir slave bağla (ADXL345 ivmeölçer — DEVID=0xE5)
        spi.set_slave_handler([](uint8_t tx) -> uint8_t {
            static uint8_t reg_addr = 0;
            static bool first = true;
            if (first) { reg_addr = tx; first = false; return 0; }
            first = true;
            if (reg_addr == 0x00) return 0xE5;  // DEVID — TR: cihaz kimliği
            if (reg_addr == 0x32) return 0x12;  // DATAX0 — TR: X ekseni alt bayt
            if (reg_addr == 0x33) return 0x00;  // DATAX1 — TR: X ekseni üst bayt
            return 0xFF;
        });

        // EN: Read DEVID register (0x00 | 0x80 for read)
        // TR: DEVID yazmacını oku (0x80 = okuma komutu | 0x00 = yazmaç adresi)
        spi.write_DR(0x80);  // read command + reg 0x00 — TR: okuma komutu + yazmaç adresi
        spi.read_DR();        // discard first byte — TR: ilk bayt at (çöp)
        spi.write_DR(0x00);   // clock out data — TR: veri için saat gönder
        uint8_t devid = spi.read_DR();
        std::cout << "  ADXL345 DEVID: 0x" << std::hex << static_cast<int>(devid)
                  << std::dec << " (expected: 0xe5)\n\n";
    }

    // ─── Demo 2: SPI Mode Comparison ──────────────────────────────────
    {
        std::cout << "--- Demo 2: All Four SPI Modes ---\n\n";

        std::cout << "  ┌──────────┬──────┬──────┬──────────────┬──────────────┐\n";
        std::cout << "  │ SPI Mode │ CPOL │ CPHA │ Clock Idle   │ Data Sample  │\n";
        std::cout << "  ├──────────┼──────┼──────┼──────────────┼──────────────┤\n";
        std::cout << "  │ Mode 0   │  0   │  0   │ LOW          │ Rising edge  │\n";
        std::cout << "  │ Mode 1   │  0   │  1   │ LOW          │ Falling edge │\n";
        std::cout << "  │ Mode 2   │  1   │  0   │ HIGH         │ Falling edge │\n";
        std::cout << "  │ Mode 3   │  1   │  1   │ HIGH         │ Rising edge  │\n";
        std::cout << "  └──────────┴──────┴──────┴──────────────┴──────────────┘\n\n";

        std::cout << "  Timing Diagram (Mode 0: CPOL=0, CPHA=0):\n";
        std::cout << "        ┌─┐   ┌─┐   ┌─┐   ┌─┐   ┌─┐   ┌─┐   ┌─┐   ┌─┐\n";
        std::cout << "  SCK ─┘ └───┘ └───┘ └───┘ └───┘ └───┘ └───┘ └───┘ └─\n";
        std::cout << "  MOSI ╳ D7  ╳ D6  ╳ D5  ╳ D4  ╳ D3  ╳ D2  ╳ D1  ╳ D0\n";
        std::cout << "       ↑    ↑    ↑    ↑    ↑    ↑    ↑    ↑\n";
        std::cout << "     sample points (rising edge)\n\n";

        // EN: Common device SPI modes
        // TR: Yaygın cihazların SPI modları ve maksimum saat hızları
        std::cout << "  Common device SPI modes:\n";
        std::cout << "  ┌──────────────────────┬──────────┬──────────┐\n";
        std::cout << "  │ Device               │ SPI Mode │ Max Clock │\n";
        std::cout << "  ├──────────────────────┼──────────┼──────────┤\n";
        std::cout << "  │ ADXL345 (Accel)      │ 0 or 3   │ 5 MHz    │\n";
        std::cout << "  │ BME280 (Press/Temp/Hum)│ 0 or 3  │ 10 MHz   │\n";
        std::cout << "  │ MCP2515 (CAN)        │ 0 or 3   │ 10 MHz   │\n";
        std::cout << "  │ W25Q128 (Flash)      │ 0 or 3   │ 133 MHz  │\n";
        std::cout << "  │ MAX31856 (Thermo)    │ 1 or 3   │ 5 MHz    │\n";
        std::cout << "  │ ADS1256 (ADC)        │ 1        │ 7.68 MHz │\n";
        std::cout << "  └──────────────────────┴──────────┴──────────┘\n\n";
    }

    // ─── Demo 3: I2C Register-Level Transaction ──────────────────────
    {
        std::cout << "--- Demo 3: I2C Register-Level Read/Write ---\n\n";

        I2CPeripheral i2c;
        i2c.set_clock(42, 400);  // 42 MHz PCLK, 400 kHz Fast Mode

        // EN: Add devices on the bus
        // TR: Bus üzerine cihazlar ekle (TMP102 sıcaklık sensörü + MPU6050 IMU)
        i2c.add_slave(0x48, {{0x00, 0x1A}, {0x01, 0x60}});  // TMP102 temp sensor — TR: TMP102 sıcaklık sensörü
        i2c.add_slave(0x68, {{0x75, 0x68}, {0x3B, 0x12}, {0x3C, 0x34}});  // MPU6050 — TR: 6 eksenli IMU

        i2c.enable();
        i2c.print_config();
        std::cout << "\n";

```

```cpp
        // EN: Read WHO_AM_I from MPU6050 (addr=0x68, reg=0x75)
        // TR: MPU6050'den WHO_AM_I oku (cihaz kimliği doğrulama — 0x68 beklenir)
        std::cout << "  I2C Read: MPU6050 WHO_AM_I (0x68, reg 0x75):\n";
        std::cout << "    1. START\n";
        i2c.start();

        std::cout << "    2. Send addr 0x68 + WRITE\n";
        i2c.send_address(0x68 << 1 | 0);  // write mode — TR: yazma modu (R/W=0)

        std::cout << "    3. Send register addr 0x75\n";
        i2c.write_data(0x75);

        std::cout << "    4. Repeated START\n";
        i2c.start();

        std::cout << "    5. Send addr 0x68 + READ\n";
        i2c.send_address(0x68 << 1 | 1);  // read mode — TR: okuma modu (R/W=1)

        auto who_am_i = i2c.read_data();
        std::cout << "    6. Read data: 0x" << std::hex
                  << static_cast<int>(who_am_i.value_or(0xFF))
                  << std::dec << " (expected: 0x68)\n";

        std::cout << "    7. NACK + STOP\n";
        i2c.stop();

        // EN: Write to TMP102 config register
        // TR: TMP102 yapılandırma yazmacına yaz (adres + veri sıralı gönderilir)
        std::cout << "\n  I2C Write: TMP102 config (0x48, reg 0x01 = 0xA0):\n";
        i2c.start();
        i2c.send_address(0x48 << 1 | 0);
        i2c.write_data(0x01);
        i2c.write_data(0xA0);
        i2c.stop();
        std::cout << "    Written 0xA0 to register 0x01 ✓\n";

        // EN: Test NACK on invalid address
        // TR: Geçersiz adrese NACK testi (bus'ta cihaz yoksa AF bayrağı set edilir)
        std::cout << "\n  I2C NACK test (address 0x50, no device):\n";
        i2c.start();
        bool ack = i2c.send_address(0x50 << 1 | 0);
        auto errors = i2c.check_errors();
        std::cout << "    ACK received: " << ack
                  << " | NACK flag: " << errors.nack << "\n";
        i2c.stop();
        std::cout << "\n";
    }

    // ─── Demo 4: UART Register-Level Comms ──────────────────────────────
    {
        std::cout << "--- Demo 4: UART Register-Level Communication ---\n\n";

        UARTPeripheral uart1, uart2;
        uart1.connect(uart2);

        // EN: Configure UART1: 115200 baud, 42 MHz PCLK
        // TR: UART1 yapılandır: 115200 baud, 42 MHz PCLK (BRR otomatik hesaplanır)
        uart1.set_baud(42000000, 115200);
        uart1.enable(true, true);
        uart1.print_config();

        uart2.set_baud(42000000, 115200);
        uart2.enable(true, true);
        std::cout << "\n";

        // EN: Send "HELLO" from UART1 → UART2
        // TR: UART1'den UART2'ye "HELLO" gönder (TX → RX çapraz bağlantı)
        std::string msg = "HELLO";
        std::cout << "  UART1 → UART2: \"" << msg << "\"\n";
        for (char c : msg) {
            uart1.transmit(static_cast<uint8_t>(c));
        }
```

```cpp

        // EN: Read on UART2
        // TR: UART2'den oku (RXNE bayrağı kontrolü ile)
        std::cout << "  UART2 received: \"";
        while (uart2.is_rx_ready()) {
            auto byte = uart2.read();
            if (byte) std::cout << static_cast<char>(*byte);
        }
        std::cout << "\"\n";

        // EN: Error handling demo
        // TR: Hata yönetimi demosu — taşma (overrun) simülasyonu ve temizleme
        uart2.simulate_overrun();
        auto err = uart2.check_errors();
        std::cout << "\n  Error flags after overrun:\n"
                  << "    Parity:  " << err.parity << "\n"
                  << "    Framing: " << err.framing << "\n"
                  << "    Noise:   " << err.noise << "\n"
                  << "    Overrun: " << err.overrun << "  ← data lost!\n";
        uart2.clear_errors();
        std::cout << "    After clear: " << uart2.check_errors().overrun << "\n\n";
    }

    // ─── Demo 5: DMA + SPI Transfer ──────────────────────────────────────
    {
        std::cout << "--- Demo 5: DMA-Assisted SPI Transfer ---\n\n";

        std::cout << "  Scenario: Read 8 bytes from SPI Flash via DMA\n\n";

        SPIPeripheral spi;
        uint32_t cr1 = (1U << spi_regs::MSTR) | (1U << spi_regs::SSM) |
                       (1U << spi_regs::SSI)  | (1U << spi_regs::SPE);
        spi.write_CR1(cr1);

        // EN: Flash returns sequential bytes
        // TR: Flash ardışık baytlar döndürür (0xA0, 0xA1, ... gerçek okuma simülasyonu)
        uint8_t flash_counter = 0xA0;
        spi.set_slave_handler([&flash_counter](uint8_t) -> uint8_t {
            return flash_counter++;
        });

        DMAChannel dma_rx;
        dma_rx.configure(DMAChannel::Direction::PeriphToMem, DMAChannel::Mode::Normal, 8);
        dma_rx.enable_channel();

        std::cout << "  DMA transfer progress:\n";
        for (uint32_t i = 0; i < 8; ++i) {
            spi.write_DR(0xFF);  // clock out dummy bytes — TR: saat üretmek için kukla bayt gönder
            uint8_t rx = spi.read_DR();
            uint8_t dma_out = 0;
            dma_rx.transfer_step(rx, dma_out);
            std::cout << "    [" << i << "] SPI_DR=0x" << std::hex
                      << static_cast<int>(rx) << " → DMA buffer\n";
        }
        std::cout << std::dec;
        dma_rx.print_status();

        std::cout << "  DMA Buffer: ";
        for (size_t i = 0; i < 8; ++i) {
            std::cout << "0x" << std::hex << static_cast<int>(dma_rx.buffer[i])
                      << (i < 7 ? " " : "");
        }
        std::cout << std::dec << "\n\n";
    }
```

```cpp

    // ── Demo 6: Multi-Slave SPI Bus ──────────────────────────────────
    {
        std::cout << "--- Demo 6: Multi-Slave SPI Bus Topology ---\n\n";

        std::cout << "   ┌───────┐    SCK   ┌──────────────┐\n";
        std::cout << "   │       │──────────→│ Slave 0      │ (Accel)  CS=PA4\n";
        std::cout << "   │       │    MOSI  ├──────────────┤\n";
        std::cout << "   │ MCU   │──────────→│ Slave 1      │ (Flash)  CS=PA5\n";
        std::cout << "   │(SPI1) │    MISO  ├──────────────┤\n";
        std::cout << "   │       │←──────────│ Slave 2      │ (CAN)    CS=PA6\n";
        std::cout << "   │       │           └──────────────┘\n";
        std::cout << "   │       │    CS0   CS1   CS2\n";
        std::cout << "   │       │────┤     ┤     ┤  (active LOW)\n";
        std::cout << "   └───────┘\n\n";

        std::cout << "  Chip Select Management:\n";
        std::cout << "    1. Pull ALL CS pins HIGH (deselect all)\n";
        std::cout << "    2. Pull target CS LOW (select one slave)\n";
        std::cout << "    3. Perform SPI transfer\n";
        std::cout << "    4. Pull CS HIGH (deselect)\n\n";

        std::cout << "  Code pattern:\n";
        std::cout << "    GPIO_SetBits(GPIOA, CS0|CS1|CS2);   // all HIGH\n";
        std::cout << "    GPIO_ResetBits(GPIOA, CS1);          // select Flash\n";
        std::cout << "    SPI_Transmit(&spi1, tx_buf, 4);      // send command\n";
        std::cout << "    SPI_Receive(&spi1, rx_buf, 256);     // read data\n";
        std::cout << "    GPIO_SetBits(GPIOA, CS1);            // deselect\n\n";

        std::cout << "  CAUTION: Never select multiple slaves simultaneously!\n";
        std::cout << "  CAUTION: Ensure CS timing requirements (setup/hold) are met!\n\n";
    }

    // ── Demo 7: I2C Error Recovery ───────────────────────────────────
    {
        std::cout << "--- Demo 7: I2C Error Handling & Recovery ---\n\n";

        std::cout << "   ┌──────────────────┬──────────────────────────────────┐\n";
        std::cout << "   │ Error            │ Recovery Action                  │\n";
        std::cout << "   ├──────────────────┼──────────────────────────────────┤\n";
        std::cout << "   │ NACK (AF)        │ Send STOP, retry or report error │\n";
        std::cout << "   │ Arbitration Lost │ Release bus, retry after delay   │\n";
        std::cout << "   │ Bus Error        │ Reset I2C peripheral (SWRST)     │\n";
        std::cout << "   │ Clock Stretching │ Timeout → toggle SCL manually (9×)│\n";
        std::cout << "   │ SDA stuck LOW    │ Generate 9 SCL clocks + STOP     │\n";
        std::cout << "   │ Bus Locked       │ GPIO bit-bang SCL to unlock      │\n";
        std::cout << "   └──────────────────┴──────────────────────────────────┘\n\n";

        std::cout << "  I2C Bus Recovery sequence (SDA stuck LOW):\n";
        std::cout << "    1. Configure SCL as GPIO output\n";
        std::cout << "    2. Toggle SCL 9 times (clock out stuck byte)\n";
        std::cout << "    3. Generate STOP condition (SDA rises while SCL high)\n";
        std::cout << "    4. Re-configure SCL/SDA as I2C alternate function\n";
        std::cout << "    5. Re-initialize I2C peripheral\n\n";

        std::cout << "  Pseudo-code:\n";
        std::cout << "    void i2c_bus_recovery() {\n";
        std::cout << "        GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_OD;\n";
        std::cout << "        HAL_GPIO_Init(GPIOB, &GPIO_InitStruct); // SCL=PB6\n";
        std::cout << "        for (int i = 0; i < 9; i++) {\n";
        std::cout << "            HAL_GPIO_WritePin(GPIOB, SCL, GPIO_PIN_SET);\n";
        std::cout << "            delay_us(5);\n";
        std::cout << "            HAL_GPIO_WritePin(GPIOB, SCL, GPIO_PIN_RESET);\n";
        std::cout << "            delay_us(5);\n";
        std::cout << "        }\n";
        std::cout << "        // Generate STOP\n";
        std::cout << "        HAL_GPIO_WritePin(GPIOB, SDA, GPIO_PIN_RESET);\n";
        std::cout << "        HAL_GPIO_WritePin(GPIOB, SCL, GPIO_PIN_SET);\n";
        std::cout << "        delay_us(5);\n";
        std::cout << "        HAL_GPIO_WritePin(GPIOB, SDA, GPIO_PIN_SET);\n";
        std::cout << "        MX_I2C1_Init(); // re-init\n";
        std::cout << "    }\n\n";
    }
```

```

    // ─── Demo 8: Protocol Comparison ──────────────────────────────────
    {
        std::cout << "--- Demo 8: SPI vs I2C vs UART Comparison ---\n\n";

        std::cout << "  ┌──────────────────┬──────────────────┬──────────────────┬──────────────────┐\n";
        std::cout << "  │ Feature          │ SPI              │ I2C              │ UART             │\n";
        std::cout << "  ├──────────────────┼──────────────────┼──────────────────┼──────────────────┤\n";
        std::cout << "  │ Wires            │ 4+ (MOSI,        │ 2 (SDA,SCL)      │ 2 (TX, RX)       │\n";
        std::cout << "  │                  │ MISO,SCK,CS)     │                  │                  │\n";
        std::cout << "  │ Speed            │ Up to 100MHz     │ 100k-3.4MHz      │ Up to 4.5Mbps    │\n";
        std::cout << "  │ Duplex           │ Full             │ Half             │ Full             │\n";
        std::cout << "  │ Multi-device     │ CS per slave     │ Address          │ Point-to-pt      │\n";
        std::cout << "  │ Flow control     │ Hardware (CS)    │ ACK/NACK         │ RTS/CTS          │\n";
        std::cout << "  │ Max distance     │ ~30 cm PCB       │ ~1 m             │ ~15 m (RS232)    │\n";
        std::cout << "  │ Overhead         │ None             │ ACK per byte     │ Start/Stop       │\n";
        std::cout << "  │ Error detection  │ None (CRC opt)   │ ACK/NACK         │ Parity           │\n";
        std::cout << "  │ Clock            │ Master-driven    │ Master-driven    │ Async            │\n";
        std::cout << "  │ HW complexity    │ Low              │ Medium           │ Medium           │\n";
        std::cout << "  │ Pin count (3dev) │ 7                │ 2                │ 6                │\n";
        std::cout << "  ├──────────────────┼──────────────────┼──────────────────┼──────────────────┤\n";
        std::cout << "  │ Best for         │ High-speed       │ Multi-sensor     │ Debug/logging    │\n";
        std::cout << "  │                  │ flash, ADC,      │ config, EEPROM   │ GPS, BT,         │\n";
        std::cout << "  │                  │ display          │                  │ console          │\n";
        std::cout << "  └──────────────────┴──────────────────┴──────────────────┴──────────────────┘\n";
    }

    std::cout << "\n==========================================\n";
    std::cout << "   End of Serial Protocols Deep Dive\n";
    std::cout << "==========================================\n";

    return 0;
}
```

► 04. Dma And Interrupts

module_15_hw_interfaces/04_dma_and_interrupts.cpp

Konu: DMA Engines & Interrupt Handling — NVIC, ISR, DMA Streams

Teori / Theory

THEORY: Interrupt System (ARM Cortex-M NVIC)

The Nested Vectored Interrupt Controller (NVIC) manages all interrupts:
- Up to 240 external interrupts + 16 system exceptions
- Programmable priority (0-255, lower = higher priority)
- Priority grouping: preemption priority + sub-priority
- Tail-chaining: back-to-back ISR without full context switch
- Late-arriving: higher-priority preempts during stacking

Interrupt Lifecycle:

```
┌─────────┐   ┌─────────┐   ┌─────────┐   ┌─────────┐
│ Trigger │ → │ Pending │ → │ Active  │ → │ Return  │
└─────────┘   └─────────┘   └─────────┘   └─────────┘
```

HW event NVIC queues ISR runs Context restore

Vector Table (Cortex-M):

```
┌─────────┬──────────────────────────────┐
│ Offset  │ Exception / IRQ              │
├─────────┼──────────────────────────────┤
│ 0x00    │ Initial SP value             │
│ 0x04    │ Reset handler                │
│ 0x08    │ NMI handler                  │
│ 0x0C    │ HardFault handler            │
│ 0x10    │ MemManage handler            │
│ 0x14    │ BusFault handler             │
│ 0x18    │ UsageFault handler           │
│ 0x2C    │ SVCall handler               │
│ 0x38    │ PendSV handler               │
│ 0x3C    │ SysTick handler              │
│ 0x40+   │ External IRQ 0, 1, 2, ...    │
└─────────┴──────────────────────────────┘
```

TEORİ: Kesme Sistemi (ARM Cortex-M NVIC)

NVIC (Nested Vectored Interrupt Controller), ARM Cortex-M işlemcilerde tüm kesmeleri öncelik tabanlı yönetir:
- 240'a kadar dış kesme + 16 sistem istisnası
- Programlanabilir öncelik (0-255, düşük sayı = yüksek öncelik)
- Öncelik gruplama: preemption önceliği + alt-öncelik
- Tail-chaining: ardışık ISR'lar tam bağlam geri yüklemesi olmadan çalışır
- Late-arriving: yığma (stacking) sırasında gelen yüksek öncelikli kesme önce alınır

Kesme Yaşam Döngüsü:

```
┌────────┐   ┌──────────┐   ┌────────┐   ┌────────┐
│ Tetikle │ → │ Beklemede │ → │ Aktif  │ → │ Dönüş  │
└────────┘   └──────────┘   └────────┘   └────────┘
```

HW olay NVIC kuyruğa ISR çalışır Bağlam geri yükle

Vektör Tablosu (Cortex-M):

```
┌─────────┬──────────────────────────────┐
│ Offset  │ İstisna / IRQ                │
├─────────┼──────────────────────────────┤
│ 0x00    │ Başlangıç SP değeri          │
│ 0x04    │ Reset handler                │
│ 0x08    │ NMI handler                  │
│ 0x0C    │ HardFault handler            │
│ 0x10    │ MemManage handler            │
│ 0x14    │ BusFault handler             │
│ 0x18    │ UsageFault handler           │
│ 0x2C    │ SVCall handler               │
│ 0x38    │ PendSV handler               │
│ 0x3C    │ SysTick handler              │
│ 0x40+   │ Dış IRQ 0, 1, 2, ...         │
└─────────┴──────────────────────────────┘
```

THEORY: DMA (Direct Memory Access)

DMA transfers data between memory and peripherals without CPU:
CPU workload comparison:

```
┌──────────────────────┬──────────┬──────────┐
│ Transfer Method      │ CPU Usage │ Latency  │
├──────────────────────┼──────────┼──────────┤
│ Polling              │ 100%     │ Lowest   │
│ Interrupt per byte   │ ~30-50%  │ Medium   │
│ DMA                  │ ~0-5%    │ Background │
│ DMA + Double buffer  │ ~0%      │ Zero-copy │
└──────────────────────┴──────────┴──────────┘
```

DMA Modes:
- Normal: transfer N bytes, then stop
- Circular: automatically restart after N bytes
- Double-buffer: ping-pong between two memory regions

TEORİ: DMA (Doğrudan Bellek Erişimi)

DMA (Direct Memory Access), CPU müdahalesi olmadan veri aktarır:
CPU yükü karşılaştırması:

```
┌──────────────────────┬──────────┬──────────┐
│ Aktarım Yöntemi      │ CPU Kullan.│ Gecikme │
├──────────────────────┼──────────┼──────────┤
│ Polling              │ %100     │ En düşük │
│ Kesme/bayt           │ ~%30-50  │ Orta     │
│ DMA                  │ ~%0-5    │ Arka plan │
│ DMA + Çift buffer    │ ~%0      │ Sıfır kopya │
└──────────────────────┴──────────┴──────────┘
```

DMA Modları:
- Normal: N bayt transfer et, sonra dur
- Circular: N bayt sonra otomatik başa dön
- Double-buffer: iki bellek bölgesi arasında ping-pong

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <array>
#include <vector>
#include <string>
#include <iomanip>
#include <functional>
#include <queue>
#include <algorithm>
#include <cassert>
#include <bitset>

// ═══════════════════════════════════════════════════════════════════════
// PART 1: NVIC Simulator
// ═══════════════════════════════════════════════════════════════════════

class NVICSimulator {
public:
    static constexpr int MAX_IRQ = 64;

```

```
21      struct IRQConfig {
22          std::string name;
23          uint8_t priority = 15;     // 0 = highest, 15 = lowest (4-bit)
24          bool enabled = false;
25          bool pending = false;
26          bool active = false;
27          std::function<void()> handler;
28      };
29
30  private:
31      std::array<IRQConfig, MAX_IRQ> irqs_{};
32      int active_irq_ = -1;
33      int preemption_count_ = 0;
34      std::vector<std::string> execution_log_;
35
36  public:
37      void configure_irq(int irq_num, std::string name, uint8_t priority,
38                         std::function<void()> handler) {
39          if (irq_num < 0 || irq_num >= MAX_IRQ) return;
40          auto& irq = irqs_[static_cast<size_t>(irq_num)];
41          irq.name = std::move(name);
42          irq.priority = priority;
43          irq.handler = std::move(handler);
44      }
45
46      void enable_irq(int irq_num) {
47          if (irq_num >= 0 && irq_num < MAX_IRQ)
48              irqs_[static_cast<size_t>(irq_num)].enabled = true;
49      }
50
51      void disable_irq(int irq_num) {
52          if (irq_num >= 0 && irq_num < MAX_IRQ)
53              irqs_[static_cast<size_t>(irq_num)].enabled = false;
54      }
55
56      // EN: Set interrupt pending (hardware trigger)
57      // TR: Kesmeyi beklemede olarak işaretle (donanım tetiklemesi simülasyonu)
58      void set_pending(int irq_num) {
59          if (irq_num >= 0 && irq_num < MAX_IRQ)
60              irqs_[static_cast<size_t>(irq_num)].pending = true;
61      }
62
63      // EN: Process all pending interrupts by priority
64      // TR: Bekleyen tüm kesmeleri önceliğe göre işle (düşük sayı = yüksek öncelik)
65      void process_pending() {
66          // EN: Find highest priority pending+enabled interrupt
67          // TR: En yüksek öncelikli bekleyen+etkin kesmeyi bul
68          int best = -1;
69          uint8_t best_pri = 255;
70
71          for (int i = 0; i < MAX_IRQ; ++i) {
72              auto& irq = irqs_[static_cast<size_t>(i)];
73              if (irq.pending && irq.enabled && !irq.active && irq.priority < best_pri) {
74                  // EN: Check preemption: can only preempt if higher priority than active
75                  // TR: Preemption kontrolü: yalnızca aktif kesmeden yüksek öncelikliyse kesebilir
76                  if (active_irq_ >= 0 &&
77                      irq.priority >= irqs_[static_cast<size_t>(active_irq_)].priority) {
78                      continue;  // cannot preempt — TR: kesme yapamaz (düşük öncelik)
79                  }
80                  best = i;
81                  best_pri = irq.priority;
82              }
83          }
84
85          if (best < 0) return;
86
87          auto& irq = irqs_[static_cast<size_t>(best)];
88          int prev_active = active_irq_;
89
90          if (active_irq_ >= 0) {
91              preemption_count_++;
92              execution_log_.push_back("  [PREEMPT] " + irqs_[static_cast<size_t>(active_irq_)].name +
93                                       " → " + irq.name);
94          }
95
```

```
        irq.pending = false;
        irq.active = true;
        active_irq_ = best;

        execution_log_.push_back("  [ENTER] IRQ " + std::to_string(best) +
                                 " (" + irq.name + ") pri=" +
                                 std::to_string(irq.priority));

        if (irq.handler) irq.handler();

        irq.active = false;
        execution_log_.push_back("  [EXIT]  IRQ " + std::to_string(best) +
                                 " (" + irq.name + ")");
        active_irq_ = prev_active;

        // EN: Tail-chaining: check for more pending
        // TR: Tail-chaining: daha fazla bekleyen kesme var mı kontrol et
        //     (bağlam geri yüklemesi atlanarak hız kazanılır)
        process_pending();
    }

    void print_config() const {
        std::cout << "  NVIC Configuration:\n";
        std::cout << "  ┌──────┬──────────────────────┬──────────┬─────────┐\n";
        std::cout << "  │ IRQ  │ Name                 │ Priority │ Enabled │\n";
        std::cout << "  ├──────┼──────────────────────┼──────────┼─────────┤\n";
        for (int i = 0; i < MAX_IRQ; ++i) {
            auto& irq = irqs_[static_cast<size_t>(i)];
            if (!irq.name.empty()) {
                std::cout << "  │ " << std::setw(3) << i
                          << " │ " << std::left << std::setw(19) << irq.name << std::right
                          << " │ " << std::setw(8) << static_cast<int>(irq.priority)
                          << " │ " << std::setw(7) << (irq.enabled ? "YES" : "NO")
                          << " │\n";
            }
        }
        std::cout << "  └──────┴──────────────────────┴──────────┴─────────┘\n";
    }

    void print_log() const {
        std::cout << "  Execution Log:\n";
        for (auto& entry : execution_log_)
            std::cout << entry << "\n";
    }

    int preemption_count() const { return preemption_count_; }

    void clear_log() {
        execution_log_.clear();
        preemption_count_ = 0;
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: DMA Stream Simulator
// ═══════════════════════════════════════════════════════════════════════════

enum class DMADirection : uint8_t {
    PeriphToMemory = 0,
    MemoryToPerph  = 1,
    MemoryToMemory = 2,
};

enum class DMADataWidth : uint8_t {
    Byte     = 1,
    HalfWord = 2,
    Word     = 4,
};

enum class DMAMode : uint8_t {
    Normal   = 0,
    Circular = 1,
};
```

```cpp

enum class DMAPriority : uint8_t {
    Low    = 0,
    Medium = 1,
    High   = 2,
    VeryHigh = 3,
};

struct DMAStreamConfig {
    int stream_num = 0;
    int channel = 0;
    DMADirection direction = DMADirection::PeriphToMemory;
    DMADataWidth periph_size = DMADataWidth::Byte;
    DMADataWidth mem_size = DMADataWidth::Byte;
    DMAMode mode = DMAMode::Normal;
    DMAPriority priority = DMAPriority::Medium;
    bool mem_increment = true;
    bool periph_increment = false;
    uint32_t data_count = 0;
};

class DMAStream {
    DMAStreamConfig config_;
    std::vector<uint8_t> source_;
    std::vector<uint8_t> destination_;
    uint32_t transfer_index_ = 0;
    bool enabled_ = false;
    bool complete_ = false;
    bool half_complete_ = false;

    // EN: Callbacks for interrupts
    // TR: Kesme callback'leri (yarı-transfer, tamamlanma, hata)
    std::function<void()> on_complete_;
    std::function<void()> on_half_complete_;
    std::function<void()> on_error_;

public:
    void configure(DMAStreamConfig cfg) {
        config_ = std::move(cfg);
        destination_.resize(config_.data_count, 0);
        transfer_index_ = 0;
        complete_ = false;
        half_complete_ = false;
    }

    void set_source(std::vector<uint8_t> data) { source_ = std::move(data); }

    void set_callbacks(std::function<void()> on_complete,
                       std::function<void()> on_half = nullptr,
                       std::function<void()> on_error = nullptr) {
        on_complete_ = std::move(on_complete);
        on_half_complete_ = std::move(on_half);
        on_error_ = std::move(on_error);
    }

    void enable() { enabled_ = true; complete_ = false; transfer_index_ = 0; }
    void disable() { enabled_ = false; }

    // EN: Simulate one DMA transfer beat
    // TR: Bir DMA transfer adımını simüle et (her beat bir veri birimi taşır)
    bool transfer_beat() {
        if (!enabled_ || complete_) return false;
        if (transfer_index_ >= config_.data_count) return false;

        if (transfer_index_ < source_.size()) {
            destination_[transfer_index_] = source_[transfer_index_];
        }
        transfer_index_++;

        // EN: Half-transfer interrupt
        // TR: Yarı-transfer kesmesi — çift buffer senaryolarında CPU'ya işleme başla sinyali
        if (!half_complete_ && transfer_index_ >= config_.data_count / 2) {
            half_complete_ = true;
            if (on_half_complete_) on_half_complete_();
        }
```

```cpp

        // EN: Transfer complete
        // TR: Transfer tamamlandı — Normal modda durur, Circular modda başa döner
        if (transfer_index_ >= config_.data_count) {
            if (config_.mode == DMAMode::Circular) {
                transfer_index_ = 0;
                half_complete_ = false;
            } else {
                complete_ = true;
                enabled_ = false;
            }
            if (on_complete_) on_complete_();
        }
        return true;
    }

    // EN: Run entire transfer
    // TR: Tüm transferi çalıştır (tamamlanana kadar döngü)
    void run_all() {
        while (enabled_ && !complete_) {
            transfer_beat();
        }
    }

    const std::vector<uint8_t>& destination() const { return destination_; }
    bool is_complete() const { return complete_; }
    uint32_t remaining() const { return config_.data_count - transfer_index_; }

    void print_status() const {
        std::string dir_str;
        switch (config_.direction) {
            case DMADirection::PeriphToMemory: dir_str = "P→M"; break;
            case DMADirection::MemoryToPerph:  dir_str = "M→P"; break;
            case DMADirection::MemoryToMemory: dir_str = "M→M"; break;
        }
        std::cout << "    DMA Stream " << config_.stream_num
                  << " [Ch" << config_.channel << "] "
                  << dir_str << " "
                  << transfer_index_ << "/" << config_.data_count
                  << (enabled_ ? " RUNNING" : " STOPPED")
                  << (complete_ ? " COMPLETE" : "")
                  << (config_.mode == DMAMode::Circular ? " CIRCULAR" : "") << "\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: Double-Buffer DMA
// ═══════════════════════════════════════════════════════════════════════════

// EN: Double-buffering allows zero-copy streaming: while DMA fills buffer A,
//     CPU processes buffer B, then they swap.
// TR: Çift buffer: DMA buffer A'yı doldururken CPU buffer B'yi işler,
//     sonra takas edilir — sıfır kopya (zero-copy) akış sağlar.
class DoubleBufferDMA {
    std::vector<uint8_t> buffer_a_;
    std::vector<uint8_t> buffer_b_;
    int active_buffer_ = 0;  // 0=A, 1=B — TR: aktif buffer indeksi
    uint32_t buffer_size_;
    int swap_count_ = 0;

public:
    explicit DoubleBufferDMA(uint32_t size)
        : buffer_a_(size, 0), buffer_b_(size, 0), buffer_size_(size) {}

    // EN: Get the buffer DMA is currently filling
    // TR: DMA'nın şu anda doldurduğu buffer'a eriş
    std::vector<uint8_t>& dma_buffer() {
        return active_buffer_ == 0 ? buffer_a_ : buffer_b_;
    }

    // EN: Get the buffer CPU can safely process
    // TR: CPU'nun güvenle işleyebileceği buffer'a eriş (DMA bu buffer'a dokunmaz)
    const std::vector<uint8_t>& cpu_buffer() const {
        return active_buffer_ == 0 ? buffer_b_ : buffer_a_;
    }
```

```cpp

    // EN: Swap buffers (called on DMA TC interrupt)
    // TR: Buffer'ları takas et (DMA transfer-complete kesmesinde çağrılır)
    void swap() {
        active_buffer_ = 1 - active_buffer_;
        swap_count_++;
    }

    int swap_count() const { return swap_count_; }
    uint32_t size() const { return buffer_size_; }

    void print_status() const {
        std::cout << "    Double-Buffer: DMA→Buf" << (active_buffer_ == 0 ? 'A' : 'B')
                  << " | CPU←Buf" << (active_buffer_ == 0 ? 'B' : 'A')
                  << " | Swaps: " << swap_count_ << "\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 4: ISR Best Practices Model
// ═══════════════════════════════════════════════════════════════════════════

// EN: Demonstrates the pattern of keeping ISR short and deferring work
// TR: ISR'yı kısa tutup işi ana döngüye erteleme kalıbını gösterir
class ISRWorkQueue {
    struct WorkItem {
        std::string description;
        std::function<void()> handler;
    };

    std::queue<WorkItem> deferred_queue_;
    int isr_calls_ = 0;
    int deferred_calls_ = 0;

public:
    // EN: Called from ISR context — MUST be short!
    // TR: ISR bağlamından çağrılır — MÜTLAKA kısa olmalı! (<10 µs hedef)
    void isr_handler(const std::string& source, uint8_t data,
                     std::function<void(uint8_t)> process) {
        isr_calls_++;
        // EN: Only critical work in ISR:
        // TR: ISR'da yalnızca kritik işler:
        // 1. Clear interrupt flag (simulated)
        // 2. Read hardware data
        // 3. Defer processing

        deferred_queue_.push({
            source + " data=0x" + std::to_string(data),
            [process, data]() { process(data); }
        });
    }

    // EN: Called from main loop — processes deferred work
    // TR: Ana döngüden çağrılır — ertelenmiş işleri işler (ağır hesaplama burada yapılır)
    void process_deferred() {
        while (!deferred_queue_.empty()) {
            auto item = std::move(deferred_queue_.front());
            deferred_queue_.pop();
            deferred_calls_++;
            item.handler();
        }
    }

    bool has_work() const { return !deferred_queue_.empty(); }
    int isr_count() const { return isr_calls_; }
    int deferred_count() const { return deferred_calls_; }
};

// ═══════════════════════════════════════════════════════════════════════════
// MAIN
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "============================================\n";
    std::cout << "   Module 15 - DMA & Interrupt Handling\n";
    std::cout << "============================================\n\n";
```

```cpp

    // ── Demo 1: NVIC Priority & Execution Order ──────────────────────────
    {
        std::cout << "--- Demo 1: NVIC Priority & Execution Order ---\n\n";

        NVICSimulator nvic;

        nvic.configure_irq(0, "TIM2_IRQHandler", 5,  []{ std::cout << "    → TIM2: 1ms tick\n"; });
        nvic.configure_irq(1, "EXTI0_IRQHandler", 2, []{ std::cout << "    → EXTI0: button press\n"; });
        nvic.configure_irq(2, "DMA1_S0_IRQHandler", 1, []{ std::cout << "    → DMA1: transfer complete\n"; });
        nvic.configure_irq(3, "USART2_IRQHandler", 8, []{ std::cout << "    → USART2: byte received\n"; });
        nvic.configure_irq(4, "CAN1_RX0_IRQHandler", 0, []{ std::cout << "    → CAN1: frame received\n"; });

        for (int i = 0; i < 5; ++i) nvic.enable_irq(i);

        nvic.print_config();
        std::cout << "\n  Triggering all interrupts simultaneously:\n\n";

        // EN: All interrupt at once — NVIC processes by priority
        // TR: Tüm kesmeler aynı anda — NVIC önceliğe göre işler (0=en önce)
        for (int i = 0; i < 5; ++i) nvic.set_pending(i);
        nvic.process_pending();

        std::cout << "\n";
        nvic.print_log();
        std::cout << "\n";
    }

    // ── Demo 2: Interrupt Preemption ─────────────────────────────────────
    {
        std::cout << "--- Demo 2: Interrupt Preemption ---\n\n";

        NVICSimulator nvic;

        // EN: Low-priority ISR triggers high-priority interrupt
        // TR: Düşük öncelikli ISR, yüksek öncelikli kesmeyi tetikler (preemption gösterimi)
        nvic.configure_irq(10, "SPI1_IRQHandler", 6,
            [&nvic]{
                std::cout << "    → SPI1: processing (pri=6)...\n";
                // EN: While processing SPI, CAN interrupt arrives
                // TR: SPI işlenirken CAN kesmesi gelir → preemption
                nvic.set_pending(11);
                nvic.process_pending();
                std::cout << "    → SPI1: resumed after preemption\n";
            });
        nvic.configure_irq(11, "CAN1_RX_IRQHandler", 2,
            []{ std::cout << "    → CAN1: URGENT frame (pri=2) □\n"; });

        nvic.enable_irq(10);
        nvic.enable_irq(11);

        std::cout << "  SPI (pri=6) starts, then CAN (pri=2) preempts:\n\n";
        nvic.set_pending(10);
        nvic.process_pending();

        std::cout << "\n";
        nvic.print_log();
        std::cout << "  Preemptions: " << nvic.preemption_count() << "\n\n";
    }
```

```cpp

    // ─── Demo 3: DMA Normal Transfer ──────────────────────────────────
    {
        std::cout << "--- Demo 3: DMA Normal Transfer (UART RX) ---\n\n";

        DMAStream dma;
        DMAStreamConfig cfg;
        cfg.stream_num = 5;
        cfg.channel = 4;
        cfg.direction = DMADirection::PeriphToMemory;
        cfg.mode = DMAMode::Normal;
        cfg.data_count = 12;
        cfg.priority = DMAPriority::High;
        dma.configure(cfg);

        // EN: Simulate UART data arriving
        // TR: UART verisinin gelişini simüle et (12 bayt "HELLO WORLD!")
        std::vector<uint8_t> uart_data = {'H','E','L','L','O',' ','W','O','R','L','D','!'};
        dma.set_source(uart_data);

        bool got_half = false, got_complete = false;
        dma.set_callbacks(
            [&got_complete]{ got_complete = true; },
            [&got_half]{ got_half = true; }
        );

        dma.enable();
        dma.print_status();

        dma.run_all();
        dma.print_status();

        std::cout << "    Half-transfer IRQ: " << (got_half ? "YES" : "NO") << "\n";
        std::cout << "    Complete IRQ:      " << (got_complete ? "YES" : "NO") << "\n";

        std::cout << "    Received: \"";
        for (auto b : dma.destination()) std::cout << static_cast<char>(b);
        std::cout << "\"\n\n";
    }

    // ─── Demo 4: DMA Circular Mode (ADC Continuous) ──────────────────
    {
        std::cout << "--- Demo 4: DMA Circular Mode (ADC Continuous) ---\n\n";

        std::cout << "  Scenario: ADC reads 4 channels continuously via DMA\n";
        std::cout << "  DMA buffer is continuously overwritten (ring buffer)\n\n";

        DMAStream dma;
        DMAStreamConfig cfg;
        cfg.stream_num = 0;
        cfg.channel = 0;
        cfg.direction = DMADirection::PeriphToMemory;
        cfg.mode = DMAMode::Circular;
        cfg.data_count = 4;
        cfg.priority = DMAPriority::High;
        dma.configure(cfg);

        int tc_count = 0;
        dma.set_callbacks([&tc_count]{ tc_count++; });

        // EN: Simulate 3 complete rounds of ADC conversion
        // TR: 3 tam ADC dönüşüm turunu simüle et (Circular mod: otomatik başa döner)
        std::vector<std::vector<uint8_t>> rounds = {
            {0x12, 0x34, 0x56, 0x78},  // round 1 — TR: 1. tur
            {0xA0, 0xB0, 0xC0, 0xD0},  // round 2 — TR: 2. tur
            {0x11, 0x22, 0x33, 0x44},  // round 3 — TR: 3. tur
        };

        dma.enable();
        for (int round = 0; round < 3; ++round) {
            dma.set_source(rounds[static_cast<size_t>(round)]);
            for (int i = 0; i < 4; ++i) dma.transfer_beat();

            std::cout << "    Round " << round + 1 << " buffer: ";
            for (auto b : dma.destination()) {
                std::cout << "0x" << std::hex << std::setw(2) << std::setfill('0')
                          << static_cast<int>(b) << " ";
```

```
            }
            std::cout << std::dec << std::setfill(' ') << "\n";
        }
        std::cout << "    TC interrupts fired: " << tc_count << "\n\n";
    }

    // ── Demo 5: Double-Buffer DMA ──────────────────────────────────────
    {
        std::cout << "--- Demo 5: Double-Buffer DMA (Zero-Copy Audio) ---\n\n";

        std::cout << "  Scenario: Audio DAC playing from memory via DMA\n";
        std::cout << "  While DMA sends buffer A, CPU fills buffer B\n\n";

        DoubleBufferDMA db(8);

        std::cout << "  ┌────────┬──────────────────┬──────────────────┐\n";
        std::cout << "  │ Cycle  │ DMA sending      │ CPU filling      │\n";
        std::cout << "  ├────────┼──────────────────┼──────────────────┤\n";

        for (int cycle = 0; cycle < 4; ++cycle) {
            // EN: CPU fills its accessible buffer while DMA sends the other
            // TR: DMA bir buffer'dan gönderirken CPU diğerini doldurur (zero-copy)
            // cpu_buffer() is safe for CPU, dma_buffer() is for DMA
            // TR: cpu_buffer() CPU için güvenli, dma_buffer() DMA kullanımında

            std::cout << "  │ " << std::setw(3) << cycle << "    │    Buf "
                      << (db.swap_count() % 2 == 0 ? 'A' : 'B')
                      << "        │    Buf "
                      << (db.swap_count() % 2 == 0 ? 'B' : 'A')
                      << "        │\n";

            // EN: DMA finishes → swap
            // TR: DMA bitti → buffer takası (TC kesmesinde gerçekleşir)
            db.swap();
        }
        std::cout << "  └────────┴──────────────────┴──────────────────┘\n";
        db.print_status();
        std::cout << "\n";
    }

    // ── Demo 6: ISR Best Practices ─────────────────────────────────────
    {
        std::cout << "--- Demo 6: ISR Best Practices (Deferred Processing) ---\n\n";

        ISRWorkQueue wq;

        std::cout << "  Pattern: ISR does minimal work, defers to main loop.\n\n";

        // EN: Simulate interrupts arriving
        // TR: Gelen kesmeleri simüle et (UART, CAN, ADC — ISR'da minimal iş yapılır)
        wq.isr_handler("UART_RX", 0x41, [](uint8_t d){
            std::cout << "    [Main] Process UART byte: '" << static_cast<char>(d) << "'\n";
        });
        wq.isr_handler("CAN_RX", 0x55, [](uint8_t d){
            std::cout << "    [Main] Process CAN frame ID: 0x" << std::hex
                      << static_cast<int>(d) << std::dec << "\n";
        });
        wq.isr_handler("ADC_EOC", 0xAB, [](uint8_t d){
            std::cout << "    [Main] Process ADC value: " << static_cast<int>(d) << "\n";
        });

        std::cout << "  ISR calls: " << wq.isr_count() << " | Pending work: "
                  << wq.has_work() << "\n\n";

```

```
        // EN: Main loop processes deferred work
        // TR: Ana döngü ertelenmiş işleri işler (ağır hesaplama burada güvenle yapılır)
        std::cout << "  Main loop processing:\n";
        wq.process_deferred();

        std::cout << "\n  Deferred calls processed: " << wq.deferred_count() << "\n\n";

        std::cout << "  ISR Rules:\n";
        std::cout << "    ✓ Clear interrupt flag first\n";
        std::cout << "    ✓ Read hardware registers only\n";
        std::cout << "    ✓ Set a flag or enqueue data\n";
        std::cout << "    ✓ Keep execution < 10 µs\n";
        std::cout << "    ✗ No printf/cout in real ISR!\n";
        std::cout << "    ✗ No malloc/new in ISR!\n";
        std::cout << "    ✗ No mutex/semaphore wait in ISR!\n";
        std::cout << "    ✗ No floating-point in ISR (unless FPU context saved)!\n\n";
    }

    // ─── Demo 7: DMA Channel Mapping ──────────────────────────────────
    {
        std::cout << "--- Demo 7: STM32 DMA Channel Mapping ---\n\n";

        std::cout << "  DMA1 Stream Assignments (STM32F4):\n";
        std::cout << "  ┌──────────┬────────┬──────────────────────────────┐\n";
        std::cout << "  │ Stream   │ Ch     │ Peripheral                   │\n";
        std::cout << "  ├──────────┼────────┼──────────────────────────────┤\n";
        std::cout << "  │ S0       │ Ch0    │ SPI3_RX                      │\n";
        std::cout << "  │ S1       │ Ch3    │ TIM2_UP / TIM2_CH3           │\n";
        std::cout << "  │ S2       │ Ch3    │ I2C3_RX                      │\n";
        std::cout << "  │ S3       │ Ch0    │ SPI2_RX                      │\n";
        std::cout << "  │ S4       │ Ch0    │ SPI2_TX                      │\n";
        std::cout << "  │ S5       │ Ch4    │ USART2_RX                    │\n";
        std::cout << "  │ S6       │ Ch4    │ USART2_TX                    │\n";
        std::cout << "  │ S7       │ Ch0    │ SPI3_TX                      │\n";
        std::cout << "  └──────────┴────────┴──────────────────────────────┘\n\n";

        std::cout << "  DMA2 Stream Assignments:\n";
        std::cout << "  ┌──────────┬────────┬──────────────────────────────┐\n";
        std::cout << "  │ Stream   │ Ch     │ Peripheral                   │\n";
        std::cout << "  ├──────────┼────────┼──────────────────────────────┤\n";
        std::cout << "  │ S0       │ Ch0    │ ADC1                         │\n";
        std::cout << "  │ S2       │ Ch3    │ SPI1_RX                      │\n";
        std::cout << "  │ S3       │ Ch3    │ SPI1_TX                      │\n";
        std::cout << "  │ S5       │ Ch4    │ USART1_RX                    │\n";
        std::cout << "  │ S7       │ Ch4    │ USART1_TX                    │\n";
        std::cout << "  └──────────┴────────┴──────────────────────────────┘\n";
    }

    std::cout << "\n============================================\n";
    std::cout << "   End of DMA & Interrupt Handling\n";
    std::cout << "============================================\n";

    return 0;
}
```

► 05. Memory Types

module_15_hw_interfaces/05_memory_types.cpp

Konu: Memory Technologies — EEPROM, FLASH, FRAM, RAM, DDR, NAND, NVMe

Teori / Theory

THEORY: Embedded Memory Technologies

Modern embedded / automotive systems use multiple memory types, each with different speed, endurance, power, and retention characteristics.
Memory Taxonomy:

```
┌──────────────────────────────────────────────────────┐
│                       MEMORY                         │
├────────────────────┬─────────────────────────────────┤
│     VOLATILE       │           NON-VOLATILE          │
├────────────────────┼─────────────────────────────────┤
│ SRAM  — MCU cache  │ NOR Flash  — code execution (XIP)  │
│ DRAM  — main RAM   │ NAND Flash — mass storage          │
│ DDR4/5             │ EEPROM     — byte-level config storage │
│                    │ FRAM (FeRAM) — fast NV, high endurance │
│                    │ MRAM       — emerging NV, fast     │
│                    │ OTP ROM    — one-time programmable │
└────────────────────┴─────────────────────────────────┘
```

Key Metrics:

```
┌──────────────┬──────────┬──────────┬──────────────┬──────────┐
│ Memory       │ Read     │ Write    │ Endurance    │ Retention │
├──────────────┼──────────┼──────────┼──────────────┼──────────┤
│ SRAM         │ ~1 ns    │ ~1 ns    │ Unlimited    │ Volatile  │
│ DRAM/DDR     │ ~10 ns   │ ~10 ns   │ Unlimited    │ Volatile  │
│ NOR Flash    │ ~70 ns   │ ~200 µs  │ 100K cycles  │ 20+ years │
│ NAND Flash   │ ~25 µs   │ ~200 µs  │ 10K-100K     │ 10+ years │
│ EEPROM       │ ~200 ns  │ ~5 ms    │ 1M cycles    │ 40+ years │
│ FRAM         │ ~100 ns  │ ~100 ns  │ 10^14 cycles │ 10+ years │
│ MRAM         │ ~35 ns   │ ~35 ns   │ 10^15 cycles │ 20+ years │
│ NVMe SSD     │ ~10 µs   │ ~20 µs   │ 3K-100K TBW  │ Power-off │
└──────────────┴──────────┴──────────┴──────────────┴──────────┘
```

TEORİ: Gömülü Bellek Teknolojileri

Gömülü/otomotiv sistemlerinde farklı hız, dayanıklılık, güç ve veri koruma özelliklerine sahip birden fazla bellek türü kullanılır.
Bellek Taksonomisi:

```
┌──────────────────────────────────────────────────────────┐
│                        BELLEK                            │
├──────────────────────┬───────────────────────────────────┤
│   UÇUCU (Volatile)   │      KALICI (Non-Volatile)        │
├──────────────────────┼───────────────────────────────────┤
│ SRAM  — MCU önbelk.  │ NOR Flash  — kod çalıştırma (XIP) │
│ DRAM  — ana bellek   │ NAND Flash — toplu depolama       │
│ DDR4/5               │ EEPROM     — bayt düzeyinde yapılandırma │
│                      │ FRAM (FeRAM) — hızlı NV, yüksek dayanık. │
│                      │ MRAM       — yeni nesil NV, hızlı │
│                      │ OTP ROM  — tek seferlik programlanabilir │
└──────────────────────┴───────────────────────────────────┘
```

Temel Metrikler:

Bellek	Okuma	Yazma	Dayanıklılık	Saklama
SRAM	~1 ns	~1 ns	Sınırsız	Uçucu
DRAM/DDR	~10 ns	~10 ns	Sınırsız	Uçucu
NOR Flash	~70 ns	~200 µs	100K döngü	20+ yıl
NAND Flash	~25 µs	~200 µs	10K-100K	10+ yıl
EEPROM	~200 ns	~5 ms	1M döngü	40+ yıl
FRAM	~100 ns	~100 ns	10^14 döngü	10+ yıl
MRAM	~35 ns	~35 ns	10^15 döngü	20+ yıl
NVMe SSD	~10 µs	~20 µs	3K-100K TBW	Güç kes.

Flash'ta "sadece bit temizlenebilir" kuralı geçerlidir (1→0): yeniden yazmadan önce sektör/blok silme zorunludur.

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <array>
#include <vector>
#include <string>
#include <iomanip>
#include <algorithm>
#include <numeric>
#include <cassert>
#include <cstring>
#include <map>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: EEPROM Simulator (I2C AT24C256-like)
// ═══════════════════════════════════════════════════════════════════════════

// EN: EEPROM characteristics: byte-addressable, slow write, high endurance
// TR: EEPROM: bayt adreslenebilir, yavaş yazma, yüksek dayanıklılık

class EEPROM {
    static constexpr uint32_t SIZE = 32768;  // 256 Kbit = 32 KB
    static constexpr int PAGE_SIZE = 64;     // page write size — TR: sayfa yazma boyutu
    static constexpr int MAX_CYCLES = 1000000;

    std::array<uint8_t, SIZE> data_;
    std::array<uint32_t, SIZE / PAGE_SIZE> page_wear_;  // write cycles per page — TR: sayfa başına yazma döngüsü
    uint32_t total_writes_ = 0;

```

```
public:
    EEPROM() {
        data_.fill(0xFF);  // erased state — TR: silinmiş durum (tüm bitler 1)
        page_wear_.fill(0);
    }

    // EN: Byte read — fast (~200ns on real HW)
    // TR: Bayt okuma — hızlı (~200ns gerçek donanımda)
    uint8_t read_byte(uint16_t addr) const {
        return data_[addr % SIZE];
    }

    // EN: Sequential read
    // TR: Sıralı okuma (adres otomatik artar)
    std::vector<uint8_t> read(uint16_t addr, uint16_t len) const {
        std::vector<uint8_t> result(len);
        for (uint16_t i = 0; i < len; ++i)
            result[i] = data_[static_cast<uint16_t>((addr + i) % SIZE)];
        return result;
    }

    // EN: Byte write — slow (~5ms on real HW)
    // TR: Bayt yazma — yavaş (~5ms gerçek donanımda, aşınmış sayfa kontrolü yapılır)
    bool write_byte(uint16_t addr, uint8_t val) {
        uint32_t page = (addr % SIZE) / PAGE_SIZE;
        if (page_wear_[page] >= MAX_CYCLES) return false;  // worn out — TR: sayfa aşınmış
        data_[addr % SIZE] = val;
        page_wear_[page]++;
        total_writes_++;
        return true;
    }

    // EN: Page write (up to PAGE_SIZE bytes at once)
    // TR: Sayfa yazma (tek seferde PAGE_SIZE bayta kadar — sınırı aşmak hatalı sarmaya neden olur)
    bool write_page(uint16_t addr, const uint8_t* buf, int len) {
        if (len > PAGE_SIZE) return false;
        uint32_t page = (addr % SIZE) / PAGE_SIZE;
        if (page_wear_[page] >= MAX_CYCLES) return false;
        for (int i = 0; i < len; ++i)
            data_[static_cast<uint16_t>((addr + i) % SIZE)] = buf[i];
        page_wear_[page]++;
        total_writes_++;
        return true;
    }

    uint32_t page_cycles(uint16_t addr) const {
        return page_wear_[(addr % SIZE) / PAGE_SIZE];
    }

    uint32_t total_writes() const { return total_writes_; }
    static constexpr uint32_t size() { return SIZE; }
    static constexpr int page_size() { return PAGE_SIZE; }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: NOR Flash Simulator (STM32 Internal Flash-like)
// ═══════════════════════════════════════════════════════════════════════════

class NORFlash {
    static constexpr uint32_t SECTOR_SIZE = 4096;  // 4 KB sector
    static constexpr uint32_t NUM_SECTORS = 64;    // 256 KB total
    static constexpr uint32_t TOTAL_SIZE = SECTOR_SIZE * NUM_SECTORS;
    static constexpr uint32_t MAX_ERASE_CYCLES = 100000;

    std::vector<uint8_t> data_;
    std::array<uint32_t, NUM_SECTORS> erase_count_;

public:
    NORFlash() : data_(TOTAL_SIZE, 0xFF) { erase_count_.fill(0); }

    // EN: Read — fast, supports XIP (execute in place)
    // TR: Okuma — hızlı, XIP destekler (kod doğrudan Flash'tan çalışır)
    uint8_t read(uint32_t addr) const { return data_[addr % TOTAL_SIZE]; }

```

```
103     // EN: Program (can only change 1→0, not 0→1!)
104     // TR: Programla (sadece 1→0 yapılabilir, 0→1 için silme zorunlu!)
105     bool program(uint32_t addr, uint8_t val) {
106         addr %= TOTAL_SIZE;
107         // EN: Flash physics: can only clear bits, not set them
108         // TR: Flash fiziği: sadece bitleri temizleyebilir (1→0), set edemez (0→1)
109         data_[addr] &= val;
110         return true;
111     }
112
113     // EN: Program word (4 bytes)
114     // TR: 32-bit kelime programla (4 bayt sırayla yazılır)
115     bool program_word(uint32_t addr, uint32_t val) {
116         for (int i = 0; i < 4; ++i) {
117             program(addr + static_cast<uint32_t>(i),
118                     static_cast<uint8_t>((val >> (i * 8)) & 0xFF));
119         }
120         return true;
121     }
122
123     // EN: Sector erase (must erase before re-programming)
124     // TR: Sektör silme (yeniden programlamadan önce zorunlu — tüm sektör 0xFF olur)
125     bool erase_sector(uint32_t sector) {
126         if (sector >= NUM_SECTORS) return false;
127         if (erase_count_[sector] >= MAX_ERASE_CYCLES) return false;
128
129         uint32_t base = sector * SECTOR_SIZE;
130         std::fill(data_.begin() + base,
131                   data_.begin() + base + SECTOR_SIZE, static_cast<uint8_t>(0xFF));
132         erase_count_[sector]++;
133         return true;
134     }
135
136     uint32_t sector_erase_count(uint32_t sector) const {
137         return (sector < NUM_SECTORS) ? erase_count_[sector] : 0;
138     }
139
140     static constexpr uint32_t sector_size() { return SECTOR_SIZE; }
141     static constexpr uint32_t total_size() { return TOTAL_SIZE; }
142 };
143
144 // ═══════════════════════════════════════════════════════════════════════
145 // PART 3: FRAM Simulator (Ferroelectric RAM)
146 // ═══════════════════════════════════════════════════════════════════════
147
148 // EN: FRAM: fast as SRAM, non-volatile, extremely high endurance (10^14)
149 // TR: FRAM: SRAM hızında, uçucu olmayan, çok yüksek dayanıklılık
150
151 class FRAM {
152     static constexpr uint32_t SIZE = 8192;  // 64 Kbit = 8 KB
153
154     std::array<uint8_t, SIZE> data_;
155     uint64_t write_cycles_ = 0;
156
157 public:
158     FRAM() { data_.fill(0x00); }
159
160     // EN: Read — fast (~100ns, same as SRAM)
161     // TR: Okuma — hızlı (~100ns, SRAM ile aynı hızda)
162     uint8_t read(uint16_t addr) const { return data_[addr % SIZE]; }
163
164     // EN: Write — fast (~100ns, no page/sector erase needed!)
165     // TR: Yazma — hızlı (~100ns, sayfa/sektör silmeye GEREK YOK!)
166     void write(uint16_t addr, uint8_t val) {
167         data_[addr % SIZE] = val;
168         write_cycles_++;
169     }
170
171     // EN: Bulk write — no page boundary limitation
172     // TR: Toplu yazma — sayfa sınırı kısıtlaması yok (EEPROM'dan farkı)
173     void write_block(uint16_t addr, const uint8_t* buf, uint16_t len) {
174         for (uint16_t i = 0; i < len; ++i) {
175             data_[static_cast<uint16_t>((addr + i) % SIZE)] = buf[i];
176         }
177         write_cycles_ += len;
178     }
179
```

```cpp
    uint64_t write_cycles() const { return write_cycles_; }
    static constexpr uint32_t size() { return SIZE; }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 4: NAND Flash + Wear Leveling
// ═══════════════════════════════════════════════════════════════════════════

class NANDFlash {
    static constexpr uint32_t PAGE_SIZE = 2048;
    static constexpr uint32_t PAGES_PER_BLOCK = 64;
    static constexpr uint32_t BLOCK_SIZE = PAGE_SIZE * PAGES_PER_BLOCK;
    static constexpr uint32_t NUM_BLOCKS = 16;
    static constexpr uint32_t MAX_ERASE = 10000;

    struct Block {
        std::vector<uint8_t> data;
        uint32_t erase_count = 0;
        bool bad = false;
        Block() : data(BLOCK_SIZE, 0xFF) {}
    };

    std::array<Block, NUM_BLOCKS> blocks_;

public:
    NANDFlash() = default;

    // EN: Read page
    // TR: Sayfa oku (blok + sayfa numarası ile adreslenir)
    std::vector<uint8_t> read_page(uint32_t block, uint32_t page) const {
        if (block >= NUM_BLOCKS || page >= PAGES_PER_BLOCK)
            return {};
        auto& blk = blocks_[block];
        if (blk.bad) return {};
        uint32_t offset = page * PAGE_SIZE;
        return {blk.data.begin() + offset,
                blk.data.begin() + offset + PAGE_SIZE};
    }

    // EN: Program page (must be erased first)
    // TR: Sayfa programla (önce blok silinmiş olmalı — Flash kuralı: 1→0)
    bool program_page(uint32_t block, uint32_t page,
                      const std::vector<uint8_t>& data) {
        if (block >= NUM_BLOCKS || page >= PAGES_PER_BLOCK) return false;
        auto& blk = blocks_[block];
        if (blk.bad) return false;

        uint32_t offset = page * PAGE_SIZE;
        for (size_t i = 0; i < std::min(data.size(), static_cast<size_t>(PAGE_SIZE)); ++i) {
            blk.data[offset + i] &= data[i];  // can only clear bits — TR: sadece bit temizlenebilir (1→0)
        }
        return true;
    }

    // EN: Block erase
    // TR: Blok silme (NAND'da minimum silme birimi bloktur — 64 sayfa = 128KB)
    bool erase_block(uint32_t block) {
        if (block >= NUM_BLOCKS) return false;
        auto& blk = blocks_[block];
        if (blk.erase_count >= MAX_ERASE) {
            blk.bad = true;
            return false;
        }
        std::fill(blk.data.begin(), blk.data.end(), static_cast<uint8_t>(0xFF));
        blk.erase_count++;
        return true;
    }

    uint32_t block_erase_count(uint32_t block) const {
        return (block < NUM_BLOCKS) ? blocks_[block].erase_count : 0;
    }

    bool is_bad_block(uint32_t block) const {
        return (block < NUM_BLOCKS) && blocks_[block].bad;
    }

```

```
256     static constexpr uint32_t page_size() { return PAGE_SIZE; }
257     static constexpr uint32_t block_size() { return BLOCK_SIZE; }
258     static constexpr uint32_t num_blocks() { return NUM_BLOCKS; }
259 };
260
261 // EN: Simple wear-leveling: always write to least-erased block
262 // TR: Basit aşınma dengeleme: her zaman en az silinmiş bloğa yaz
263 //     (bloklar arası eşit aşınma sağlanır → ömür uzar)
264 class WearLeveler {
265     NANDFlash& flash_;
266
267 public:
268     explicit WearLeveler(NANDFlash& f) : flash_(f) {}
269
270     // EN: Find block with lowest erase count (not bad)
271     // TR: En düşük silme sayısına sahip geçerli bloğu bul
272     uint32_t find_best_block() const {
273         uint32_t best = 0;
274         uint32_t min_wear = UINT32_MAX;
275         for (uint32_t b = 0; b < NANDFlash::num_blocks(); ++b) {
276             if (!flash_.is_bad_block(b) && flash_.block_erase_count(b) < min_wear) {
277                 min_wear = flash_.block_erase_count(b);
278                 best = b;
279             }
280         }
281         return best;
282     }
283
284     void print_wear_map() const {
285         std::cout << "    Wear Map [";
286         for (uint32_t b = 0; b < NANDFlash::num_blocks(); ++b) {
287             if (flash_.is_bad_block(b))
288                 std::cout << " XX";
289             else
290                 std::cout << " " << std::setw(2) << flash_.block_erase_count(b);
291         }
292         std::cout << " ]\n";
293     }
294 };
295
296 // ═══════════════════════════════════════════════════════════════════════════
297 // MAIN
298 // ═══════════════════════════════════════════════════════════════════════════
299 int main() {
300     std::cout << "============================================\n";
301     std::cout << "   Module 15 - Memory Types & Technologies\n";
302     std::cout << "============================================\n\n";
303
304     // ─── Demo 1: EEPROM Read/Write ──────────────────────────────────────
305     {
306         std::cout << "--- Demo 1: EEPROM (AT24C256) Simulation ---\n\n";
307
308         EEPROM eeprom;
309
310         // EN: Store calibration data
311         // TR: Kalibrasyon verisini depola (EEPROM'un tipik kullanımı)
312         struct CalibData {
313             uint16_t magic = 0xCAFE;
314             float gain = 1.234f;
315             int16_t offset = -42;
316         };
317
318         CalibData cal;
319         const auto* bytes = reinterpret_cast<const uint8_t*>(&cal);
320         for (size_t i = 0; i < sizeof(cal); ++i) {
321             eeprom.write_byte(static_cast<uint16_t>(i), bytes[i]);
322         }
323
324         // EN: Read back
325         // TR: Geri oku ve doğrula (EEPROM verisi kalıcıdır)
326         CalibData read_cal{};
327         auto data = eeprom.read(0, static_cast<uint16_t>(sizeof(CalibData)));
328         std::memcpy(&read_cal, data.data(), sizeof(CalibData));
329
330         std::cout << "  EEPROM: " << EEPROM::size() << " bytes, "
331                   << EEPROM::page_size() << " byte pages\n";
332         std::cout << "  Written calibration:\n";
```

```
        std::cout << "    magic:  0x" << std::hex << read_cal.magic << std::dec << "\n";
        std::cout << "    gain:   " << read_cal.gain << "\n";
        std::cout << "    offset: " << read_cal.offset << "\n";
        std::cout << "    page wear (addr 0): " << eeprom.page_cycles(0) << " cycles\n";
        std::cout << "    total writes: " << eeprom.total_writes() << "\n\n";
    }

    // ── Demo 2: Flash Program & Erase ──────────────────────────────
    {
        std::cout << "--- Demo 2: NOR Flash Program & Erase ---\n\n";

        NORFlash flash;

        std::cout << "  NOR Flash: " << NORFlash::total_size() / 1024 << " KB, "
                  << NORFlash::sector_size() << " byte sectors\n\n";

        // EN: Program some data
        // TR: Veri programla (NOR Flash'a 32-bit kelime yazma)
        flash.program_word(0x1000, 0xDEADBEEF);

        uint32_t readback = 0;
        for (int i = 0; i < 4; ++i)
            readback |= static_cast<uint32_t>(flash.read(0x1000 + static_cast<uint32_t>(i))) << (i * 8);

        std::cout << "  Program 0xDEADBEEF at 0x1000:\n";
        std::cout << "    Read back: 0x" << std::hex << readback << std::dec << " ✓\n\n";

        // EN: Try to overwrite without erasing — can only clear bits!
        // TR: Silmeden üzerine yaz — sadece bit temizlenebilir (1→0, 0→1 olmaz!)
        std::cout << "  Overwrite without erase (0x12345678 → 0x1000):\n";
        flash.program_word(0x1000, 0x12345678);
        readback = 0;
        for (int i = 0; i < 4; ++i)
            readback |= static_cast<uint32_t>(flash.read(0x1000 + static_cast<uint32_t>(i))) << (i * 8);
        std::cout << "    Read back: 0x" << std::hex << readback << std::dec << "\n";
        std::cout << "    Expected:  0x" << std::hex
                  << (0xDEADBEEFu & 0x12345678u) << std::dec
                  << " (bits can only go 1→0!)\n\n";

        // EN: Proper sequence: erase then program
        // TR: Doğru sıra: önce sil sonra programla (Flash kullanmanın temel kuralı)
        flash.erase_sector(0x1000 / NORFlash::sector_size());
        flash.program_word(0x1000, 0x12345678);
        readback = 0;
        for (int i = 0; i < 4; ++i)
            readback |= static_cast<uint32_t>(flash.read(0x1000 + static_cast<uint32_t>(i))) << (i * 8);
        std::cout << "  After erase + re-program:\n";
        std::cout << "    Read back: 0x" << std::hex << readback << std::dec << " ✓\n";
        std::cout << "    Sector erase count: " << flash.sector_erase_count(1) << "\n\n";
    }

    // ── Demo 3: FRAM Fast NV Storage ────────────────────────────────
    {
        std::cout << "--- Demo 3: FRAM — Fast Non-Volatile Memory ---\n\n";

        FRAM fram;

        std::cout << "  FRAM: " << FRAM::size() << " bytes\n";
        std::cout << "  Key advantages over EEPROM:\n";
        std::cout << "    - Write speed: ~100 ns (vs ~5 ms for EEPROM)\n";
        std::cout << "    - No page boundaries for writes\n";
        std::cout << "    - Endurance: 10^14 cycles (vs 10^6 for EEPROM)\n";
        std::cout << "    - No erase needed before write\n\n";

        // EN: Write crash counter (simulating power-loss safe storage)
        // TR: Çökme sayaçı yaz (güç kesintisine dayanıklı depolama simülasyonu)
        uint32_t crash_count = 42;
        auto* p = reinterpret_cast<const uint8_t*>(&crash_count);
        fram.write_block(0x100, p, sizeof(crash_count));

```

```
        // EN: Read back
        // TR: Geri oku (FRAM'da yazma sonrası bekleme gerekmez)
        uint32_t read_val = 0;
        for (int i = 0; i < 4; ++i)
            read_val |= static_cast<uint32_t>(fram.read(static_cast<uint16_t>(0x100 + i))) << (i * 8);

        std::cout << "  Written crash_count=42 at 0x100\n";
        std::cout << "  Read back: " << read_val << " ✓\n";
        std::cout << "  Write cycles: " << fram.write_cycles() << "\n\n";

        std::cout << "  Automotive FRAM uses:\n";
        std::cout << "    - Odometer / trip counter (frequent updates)\n";
        std::cout << "    - DTC freeze-frame data (must survive power-loss)\n";
        std::cout << "    - CAN message counters (rolling counters)\n";
        std::cout << "    - Battery management cell history\n\n";
    }

    // ─── Demo 4: NAND Wear Leveling ──────────────────────────────────
    {
        std::cout << "--- Demo 4: NAND Flash Wear Leveling ---\n\n";

        NANDFlash nand;
        WearLeveler wl(nand);

        std::cout << "  NAND: " << NANDFlash::num_blocks() << " blocks × "
                  << NANDFlash::block_size() / 1024 << " KB = "
                  << NANDFlash::num_blocks() * NANDFlash::block_size() / 1024 << " KB\n\n";

        // EN: Without wear leveling: hammer block 0
        // TR: Aşınma dengeleme olmadan: hep blok 0'a yaz (erken bozulur)
        std::cout << "  Without wear leveling (always use block 0):\n";
        for (int i = 0; i < 5; ++i) nand.erase_block(0);
        wl.print_wear_map();

        // EN: With wear leveling: spread writes
        // TR: Aşınma dengeleme ile: yazmaları dağıt (tüm bloklar eşit aşınır)
        std::cout << "\n  With wear leveling (least-erased block):\n";
        for (int i = 0; i < 20; ++i) {
            uint32_t best = wl.find_best_block();
            nand.erase_block(best);
        }
        wl.print_wear_map();
        std::cout << "\n";
    }

    // ─── Demo 5: Memory Map — Automotive ECU ─────────────────────────
    {
        std::cout << "--- Demo 5: Automotive ECU Memory Map ---\n\n";

        std::cout << "  ┌──────────────────┬──────────────┬──────────────────────────┐\n";
        std::cout << "  │ Address Range    │ Size         │ Memory / Purpose         │\n";
        std::cout << "  ├──────────────────┼──────────────┼──────────────────────────┤\n";
        std::cout << "  │ 0x0000_0000      │ 2 MB         │ Internal Flash (Code)    │\n";
        std::cout << "  │ 0x0800_0000      │ 256 KB       │ Internal Flash (Calibration) │\n";
        std::cout << "  │ 0x1000_0000      │ 64 KB        │ OTP Memory (keys, serial#) │\n";
        std::cout << "  │ 0x2000_0000      │ 512 KB       │ SRAM (stack, heap, globals) │\n";
        std::cout << "  │ 0x2008_0000      │ 128 KB       │ SRAM2 (DMA buffers)      │\n";
        std::cout << "  │ 0x4000_0000      │ -            │ Peripheral registers     │\n";
        std::cout << "  │ 0x6000_0000      │ 4 MB         │ External QSPI Flash (maps) │\n";
        std::cout << "  │ 0x8000_0000      │ 32 KB        │ External EEPROM (I2C)    │\n";
        std::cout << "  │ 0x9000_0000      │ 8 KB         │ External FRAM (SPI)      │\n";
        std::cout << "  │ 0xE000_0000      │ -            │ Cortex-M System (NVIC, etc.) │\n";
        std::cout << "  └──────────────────┴──────────────┴──────────────────────────┘\n\n";

        std::cout << "  Data Placement Strategy:\n";
        std::cout << "    ┌──────────────────────────────┬──────────────┐\n";
        std::cout << "    │ Data Type                    │ Storage      │\n";
        std::cout << "    ├──────────────────────────────┼──────────────┤\n";
        std::cout << "    │ Firmware code                │ NOR Flash    │\n";
        std::cout << "    │ Bootloader                   │ NOR Flash    │\n";
        std::cout << "    │ Calibration parameters       │ NOR Flash    │\n";
        std::cout << "    │ DTC / Freeze frame           │ EEPROM/FRAM  │\n";
        std::cout << "    │ Odometer / trip meter        │ FRAM         │\n";
        std::cout << "    │ Security keys                │ OTP / HSM    │\n";
        std::cout << "    │ Navigation maps              │ QSPI Flash   │\n";
        std::cout << "    │ Runtime variables            │ SRAM         │\n";
```

```
        std::cout << "    │ DMA transfer buffers          │ SRAM2       │\n";
        std::cout << "    │ OTA update staging            │ Ext. Flash  │\n";
        std::cout << "    └───────────────────────────────┴─────────────┘\n\n";
    }

    // ─── Demo 6: ECC & Data Integrity ──────────────────────────────────
    {
        std::cout << "--- Demo 6: ECC & Data Integrity ---\n\n";

        // EN: Simple parity-based ECC simulation
        // TR: Basit parite tabanlı ECC simülasyonu (tek bit hatası tespiti)
        auto calc_parity = [](const std::vector<uint8_t>& data) -> uint8_t {
            uint8_t parity = 0;
            for (auto b : data) parity ^= b;
            return parity;
        };

        std::vector<uint8_t> orig_data = {0x12, 0x34, 0x56, 0x78, 0x9A};
        uint8_t ecc = calc_parity(orig_data);

        std::cout << "  Original data: ";
        for (auto b : orig_data) std::cout << std::hex << "0x" << static_cast<int>(b) << " ";
        std::cout << "\n  ECC (XOR parity): 0x" << static_cast<int>(ecc) << std::dec << "\n\n";

        // EN: Simulate single-bit corruption
        // TR: Tek bit bozulmasını simüle et (ECC ile tespit edilebilir)
        auto corrupted = orig_data;
        corrupted[2] ^= 0x04;  // flip bit 2 of byte 2 — TR: 2. baytın 2. bitini çevir

        uint8_t ecc_check = calc_parity(corrupted);
        std::cout << "  Corrupted data: ";
        for (auto b : corrupted) std::cout << std::hex << "0x" << static_cast<int>(b) << " ";
        std::cout << "\n  ECC check: 0x" << static_cast<int>(ecc_check)
                  << " (expected: 0x" << static_cast<int>(ecc) << ")\n"
                  << std::dec;
        std::cout << "  " << ((ecc_check != ecc) ? "CORRUPTION DETECTED ✓" : "OK") << "\n\n";

        std::cout << "  Memory ECC in Automotive:\n";
        std::cout << "  ┌──────────────────┬──────────┬──────────────────────┐\n";
        std::cout << "  │ Memory           │ ECC Type │ ASIL Requirement     │\n";
        std::cout << "  ├──────────────────┼──────────┼──────────────────────┤\n";
        std::cout << "  │ Internal SRAM    │ SECDED   │ ASIL-B and above     │\n";
        std::cout << "  │ Flash (code)     │ SECDED/BCH │ ASIL-B and above     │\n";
        std::cout << "  │ External DDR     │ ECC-DIMM │ ASIL-D (autonomous)  │\n";
        std::cout << "  │ External NAND    │ BCH/LDPC │ Managed by FTL       │\n";
        std::cout << "  │ Cache            │ Parity/ECC │ ASIL-D               │\n";
        std::cout << "  └──────────────────┴──────────┴──────────────────────┘\n";
        std::cout << "  SECDED = Single Error Correct, Double Error Detect\n\n";
    }

    // ─── Demo 7: Memory Comparison Table ──────────────────────────────
    {
        std::cout << "--- Demo 7: Complete Memory Comparison ---\n\n";

        std::cout << "  ┌──────────┬────────┬──────────┬──────────────┬──────────┬──────────────┐\n";
        std::cout << "  │ Type     │ Vola.  │ Speed    │ Endurance    │ Erase Unit │ Best For     │\n";
        std::cout << "  ├──────────┼────────┼──────────┼──────────────┼──────────┼──────────────┤\n";
        std::cout << "  │ SRAM     │ Yes    │ ~1 ns    │ Unlimited    │ N/A      │ Cache, stack │\n";
        std::cout << "  │ DRAM/DDR │ Yes    │ ~10 ns   │ Unlimited    │ N/A      │ Main memory  │\n";
        std::cout << "  │ NOR Flash│ No     │ 70 ns R  │ 100K cycles  │ Sector   │ Code, XIP    │\n";
        std::cout << "  │ NAND Flash│ No    │ 25 µs R  │ 10K-100K     │ Block    │ Mass storage │\n";
        std::cout << "  │ EEPROM   │ No     │ 200 ns R │ 1M cycles    │ Byte     │ Config data  │\n";
        std::cout << "  │ FRAM     │ No     │ 100 ns   │ 10^14 cycles │ Byte     │ Counters, DTC│\n";
        std::cout << "  │ MRAM     │ No     │ 35 ns    │ 10^15+ cycles │ Byte     │ Next-gen NVM │\n";
        std::cout << "  │ OTP ROM  │ No     │ Fast     │ 1 write      │ N/A      │ Keys, serial │\n";
```

```
        std::cout << "  | NVMe SSD  | No     | 10 µs    | 3K-100K TBW   | Block/page | ADAS logging |\n";
        std::cout << "  └──────────┴────────┴──────────┴────────────────┴────────────┴──────────────┘\n";
    }

    std::cout << "\n==========================================\n";
    std::cout << "   End of Memory Types & Technologies\n";
    std::cout << "==========================================\n";

    return 0;
}
```

Modül 16: AUTOSAR — Otomotiv Yazılım Mimarisi

Otomotiv Yazılım Mimarisi

5 kaynak dosya · Standart: C++20

C++20

► 01. Classic Platform

module_16_autosar/01_classic_platform.cpp

Konu: AUTOSAR Classic Platform — Layered Architecture / Katmanlı Mimari

Teori / Theory

THEORY: AUTOSAR Classic Platform — Overview

AUTOSAR (AUTomotive Open System ARchitecture) Classic Platform is the de-facto standard software architecture for microcontroller-based ECUs. It defines a layered architecture separating application logic from hardware, enabling portability, reusability, and standardized interfaces.
AUTOSAR Classic Architecture:

```
┌──────────────────────────────────────────────────────────┐
│                  APPLICATION LAYER                       │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐ │
│   │  SWC_1   │  │  SWC_2   │  │  SWC_3   │  │  SWC_N   │ │
│   │(Runnable)│  │(Runnable)│  │(Runnable)│  │(Runnable)│ │
│   └────┬─────┘  └────┬─────┘  └────┬─────┘  └────┬─────┘ │
├────────┴─────────────┴─────────────┴─────────────┴───────┤
│             RTE (Runtime Environment)                    │
│  Virtual Function Bus — abstracts all SWC communication  │
├──────────────────────────────────────────────────────────┤
│             BSW (Basic Software)                         │
│  ┌──────────────────────────────────────────────────┐   │
│  │ Services Layer: OS, COM, NvM, Dcm, Dem, BswM, EcuM │   │
│  ├──────────────────────────────────────────────────┤   │
│  │ ECU Abstraction: CanIf, SpiIf, DioIf, AdcIf, PwmIf │   │
│  ├──────────────────────────────────────────────────┤   │
│  │ MCAL: Can, Spi, Dio, Adc, Pwm, Gpt, Mcu, Port, Icu │   │
│  └──────────────────────────────────────────────────┘   │
├──────────────────────────────────────────────────────────┤
│             HARDWARE (MCU + Peripherals)                 │
└──────────────────────────────────────────────────────────┘
```

TEORİ: Genel Bakış

AUTOSAR Classic Platform, mikrodenetleyici tabanlı ECU'lar için fiili standart yazılım mimarisidir. Uygulama mantığını donanımdan ayıran katmanlı bir mimari tanımlar; taşınabilirlik, yeniden kullanılabilirlik ve standart arayüzler sağlar.
AUTOSAR Classic Mimarisi:

```
┌──────────────────────────────────────────────────────────┐
│                  UYGULAMA KATMANI                        │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  │
│   │  SWC_1   │  │  SWC_2   │  │  SWC_3   │  │  SWC_N   │  │
│   │(Runnable)│  │(Runnable)│  │(Runnable)│  │(Runnable)│  │
│   └────┬─────┘  └────┬─────┘  └────┬─────┘  └────┬─────┘  │
├────────┴─────────────┴─────────────┴─────────────┴────────┤
│          RTE (Çalışma Zamanı Ortamı)                     │
│  Sanal Fonksiyon Veri Yolu — tüm SWC iletişimini soyutlar  │
├──────────────────────────────────────────────────────────┤
│          BSW (Temel Yazılım)                             │
│  ┌──────────────────────────────────────────────────┐    │
│  │ Servis Katmanı: OS, COM, NvM, Dcm, Dem, BswM, EcuM │    │
│  ├──────────────────────────────────────────────────┤    │
│  │ ECU Soyutlama: CanIf, SpiIf, DioIf, AdcIf, PwmIf   │    │
│  ├──────────────────────────────────────────────────┤    │
│  │ MCAL: Can, Spi, Dio, Adc, Pwm, Gpt, Mcu, Port, Icu │    │
│  └──────────────────────────────────────────────────┘    │
├──────────────────────────────────────────────────────────┤
│          DONANIM (MCU + Çevresel Birimler)               │
└──────────────────────────────────────────────────────────┘
```

THEORY: SWC — Software Component

SWC is the atomic unit of application software. Each SWC:
- Contains one or more Runnables (executable entities)
- Communicates only via Ports (Sender/Receiver or Client/Server)
- Is hardware-independent (runs on any ECU with matching RTE)

Port Types:

```
┌──────────────────┬──────────────────────────────────────┐
│ Port Type        │ Description                          │
├──────────────────┼──────────────────────────────────────┤
│ SenderReceiver   │ Async data passing (signal-level)    │
│ ClientServer     │ Sync function call (request/response)  │
│ ModeSwitchPort   │ Mode change notification (EcuM, BswM)  │
│ NvDataPort       │ Non-volatile data access (NvM interface) │
└──────────────────┴──────────────────────────────────────┘
```

TEORİ: Yazılım Bileşeni

SWC, uygulama yazılımının atomik birimidir. Her SWC:
- Bir veya daha fazla Runnable (çalıştırılabilir varlık) içerir
- Yalnızca Port'lar aracılığıyla iletişim kurar (Sender/Receiver veya Client/Server)
- Donanımdan bağımsızdır (eşleşen RTE'ye sahip herhangi bir ECU'da çalışır)

Port Türleri:

```
┌──────────────────┬──────────────────────────────────────┐
│ Port Türü        │ Açıklama                             │
├──────────────────┼──────────────────────────────────────┤
│ SenderReceiver   │ Asenkron veri geçişi (sinyal-seviyesi)  │
│ ClientServer     │ Senkron fonksiyon çağrısı (istek/yanıt)  │
│ ModeSwitchPort   │ Mod değişikliği bildirimi (EcuM, BswM)  │
│ NvDataPort       │ Kalıcı veri erişimi (NvM arayüzü)    │
└──────────────────┴──────────────────────────────────────┘
```

THEORY: COM Stack — PDU Routing

The communication stack routes Protocol Data Units (PDUs) from application to hardware:
COM (Signal ↔ PDU group) ↓ PduR (PDU Router — routing table) ↓ CanIf / LinIf / FrIf / SoAd (Interface layer) ↓ CanDrv / LinDrv / FrDrv / TcpIp (Driver — MCAL) ↓ CAN Controller / LIN Transceiver / FlexRay CC / Ethernet PHY

TEORİ: COM Yığını — PDU Yönlendirme

İletişim yığını, Protocol Data Unit'leri (PDU) uygulamadan donanıma yönlendirir:
COM (Sinyal ↔ PDU grubu) ↓ PduR (PDU Yönlendirici — yönlendirme tablosu) ↓ CanIf / LinIf / FrIf / SoAd (Arayüz katmanı) ↓ CanDrv / LinDrv / FrDrv / TcpIp (Sürücü — MCAL) ↓ CAN Denetleyici / LIN Transceiver / FlexRay CC / Ethernet PHY

THEORY: AUTOSAR OS — OSEK/VDX Based

AUTOSAR OS extends OSEK/VDX OS with:
- Scalability Classes (SC1-SC4): from basic tasks to memory protection
- Tasks: Basic (non-preemptive) and Extended (can wait on events)
- ISR Category 1 (no OS calls) and Category 2 (OS calls allowed)
- Alarms: periodic activation of tasks/callbacks
- Schedule Tables: deterministic activation sequence
- Counters: hardware or software tick sources

SC Class	Features
SC1	Basic tasks, ISR Cat2, Alarms, Counters
SC2	SC1 + Timing Protection
SC3	SC1 + Memory Protection
SC4	SC2 + SC3 (full protection)

TEORİ: AUTOSAR OS — OSEK/VDX Tabanlı

AUTOSAR OS, OSEK/VDX OS'u genisletir:
- Ölçeklenebilirlik Sınıfları (SC1-SC4): temel görevlerden bellek korumaya
- Görevler: Temel (preemptive olmayan) ve Genişletilmiş (olay bekleyebilir)
- ISR Kategori 1 (OS çağrısı yok) ve Kategori 2 (OS çağrısına izin var)
- Alarm: Görevlerin/geri çağrıların periyodik etkinleştirmesi
- Schedule Table: Deterministik etkinleştirme dizisi
- Counter: Donanım veya yazılım tik kaynakları

SC Sınıfı	Özellikler
SC1	Temel görevler, ISR Kat2, Alarm, Counter
SC2	SC1 + Zamanlandırma Koruması
SC3	SC1 + Bellek Koruması
SC4	SC2 + SC3 (tam koruma)

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <array>
#include <vector>
#include <string>
#include <string_view>
#include <functional>
#include <map>
#include <queue>
#include <iomanip>
#include <algorithm>
#include <cassert>
#include <variant>
#include <optional>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: AUTOSAR-style SWC and RTE Simulation
// ═══════════════════════════════════════════════════════════════════════════

// EN: Port interfaces — Sender/Receiver and Client/Server abstraction
// TR: Port arayüzleri — Sender/Receiver ve Client/Server soyutlama

namespace autosar_classic {

// --- Sender/Receiver Port Data Types ---
struct EngineData {
    uint16_t rpm = 0;
    int8_t   coolant_temp = -40;  // °C, offset -40
    uint8_t  throttle_pct = 0;    // 0-100%
};

struct VehicleSpeed {
    uint16_t speed_kph_x10 = 0;  // 0.1 km/h resolution
    bool     valid = false;
};

// ---------------------------------------------------------------------------
// RTE — Runtime Environment
// ---------------------------------------------------------------------------
// EN: RTE acts as the "Virtual Function Bus" — all SWC communication
//     passes through RTE. No direct SWC-to-SWC calls allowed.
// TR: RTE, "Virtual Function Bus" işlevi görür. SWC'ler arası doğrudan
//     çağrı yasaktır; tüm iletişim RTE üzerinden geçer.

class RTE {
    EngineData    engine_data_{};
    VehicleSpeed  vehicle_speed_{};

    // EN: Client/Server function registry
    // TR: Client/Server fonksiyon kaydı
    using ServerFunction = std::function<uint8_t(const std::vector<uint8_t>&, std::vector<uint8_t>&)>;
    std::map<std::string, ServerFunction> server_registry_;

public:
    // --- Sender/Receiver API ---
    // EN: Write signal to RTE (sender side)
    // TR: RTE'ye sinyal yaz (gönderici tarafı)
    void Rte_Write_EngineData(const EngineData& data) { engine_data_ = data; }
    [[nodiscard]] EngineData Rte_Read_EngineData() const { return engine_data_; }

    void Rte_Write_VehicleSpeed(const VehicleSpeed& data) { vehicle_speed_ = data; }
    [[nodiscard]] VehicleSpeed Rte_Read_VehicleSpeed() const { return vehicle_speed_; }

    // --- Client/Server API ---
    void RegisterServer(const std::string& service, ServerFunction func) {
        server_registry_[service] = std::move(func);
    }

```

```
69      uint8_t Rte_Call(const std::string& service,
70                       const std::vector<uint8_t>& request,
71                       std::vector<uint8_t>& response) {
72          auto it = server_registry_.find(service);
73          if (it == server_registry_.end()) return 0xFF;  // E_NOT_OK
74          return it->second(request, response);
75      }
76  };
77
78  // ---------------------------------------------------------------------------
79  // SWC: Engine Management (Sensor Reading + Data Publishing)
80  // ---------------------------------------------------------------------------
81
82  class SWC_EngineManagement {
83      RTE& rte_;
84      uint32_t cycle_count_ = 0;
85
86  public:
87      explicit SWC_EngineManagement(RTE& rte) : rte_(rte) {}
88
89      // EN: Runnable — called every 10ms by OS task
90      // TR: Runnable — OS tarafından her 10ms'de çağrılır
91      void Runnable_ReadSensors() {
92          EngineData data{};
93          // EN: Simulate sensor reading (in real system: ADC → scaling → engineering value)
94          // TR: Sensör okuma simülasyonu
95          data.rpm = static_cast<uint16_t>(800 + (cycle_count_ % 50) * 100);
96          data.coolant_temp = static_cast<int8_t>(20 + static_cast<int8_t>(cycle_count_ % 30));
97          data.throttle_pct = static_cast<uint8_t>(10 + cycle_count_ % 80);
98          rte_.Rte_Write_EngineData(data);
99          ++cycle_count_;
100     }
101 };
102
103 // ---------------------------------------------------------------------------
104 // SWC: Vehicle Speed Calculator
105 // ---------------------------------------------------------------------------
106
107 class SWC_SpeedCalculator {
108     RTE& rte_;
109
110 public:
111     explicit SWC_SpeedCalculator(RTE& rte) : rte_(rte) {}
112
113     // EN: Runnable — derives vehicle speed from engine data
114     // TR: Runnable — motor verisinden araç hızını türetir
115     void Runnable_CalcSpeed() {
116         auto eng = rte_.Rte_Read_EngineData();
117         // EN: Simplified: speed ~proportional to RPM * throttle / gear_ratio
118         // TR: Basitleştirilmiş: hız ~ RPM * gaz / dişli oranı
119         uint16_t speed = static_cast<uint16_t>(
120             (static_cast<uint32_t>(eng.rpm) * eng.throttle_pct) / 500u);
121         VehicleSpeed vs{};
122         vs.speed_kph_x10 = speed;
123         vs.valid = (eng.rpm > 0);
124         rte_.Rte_Write_VehicleSpeed(vs);
125     }
126 };
127
128 // ---------------------------------------------------------------------------
129 // SWC: Dashboard Display
130 // ---------------------------------------------------------------------------
131
132 class SWC_Dashboard {
133     RTE& rte_;
134
135 public:
136     explicit SWC_Dashboard(RTE& rte) : rte_(rte) {}
137
```

```
138     void Runnable_UpdateDisplay() {
139         auto eng = rte_.Rte_Read_EngineData();
140         auto spd = rte_.Rte_Read_VehicleSpeed();
141         std::cout << "  [Dashboard] RPM=" << eng.rpm
142                   << " Temp=" << static_cast<int>(eng.coolant_temp) << "C"
143                   << " Throttle=" << static_cast<int>(eng.throttle_pct) << "%"
144                   << " Speed=" << (spd.speed_kph_x10 / 10) << "."
145                   << (spd.speed_kph_x10 % 10) << " km/h"
146                   << (spd.valid ? " [OK]" : " [INVALID]") << "\n";
147     }
148 };
149
150 }  // namespace autosar_classic
151
152 // ============================================================================
153 // PART 2: AUTOSAR OS Simulation (Task, Alarm, Schedule Table)
154 // ============================================================================
155
156 namespace autosar_os {
157
158 // EN: Task priority (higher value = higher priority in AUTOSAR OS)
159 // TR: Task önceliği (yüksek değer = yüksek öncelik)
160
161 enum class TaskState : uint8_t { SUSPENDED, READY, RUNNING, WAITING };
162
163 struct TaskConfig {
164     std::string         name;
165     uint8_t             priority = 0;
166     bool                preemptive = true;
167     std::function<void()> entry;  // task body
168     uint32_t            stack_size = 1024;
169 };
170
171 struct Task {
172     TaskConfig config;
173     TaskState  state = TaskState::SUSPENDED;
174     uint32_t   activation_count = 0;
175 };
176
177 // EN: Alarm — periodic or one-shot task activation
178 // TR: Alarm — periyodik veya tek seferlik task aktivasyonu
179 struct Alarm {
180     std::string name;
181     std::string task_name;
182     uint32_t    cycle_ticks = 0;   // 0 = one-shot
183     uint32_t    offset_ticks = 0;
184     uint32_t    remaining = 0;
185     bool        active = false;
186 };
187
188 // EN: Minimal AUTOSAR OS scheduler simulation
189 // TR: Minimal AUTOSAR OS zamanlayıcı simülasyonu
190 class OS {
191     std::vector<Task>  tasks_;
192     std::vector<Alarm> alarms_;
193     uint32_t           tick_counter_ = 0;
194
195 public:
196     void DeclareTask(const TaskConfig& cfg) {
197         tasks_.push_back({cfg, TaskState::SUSPENDED, 0});
198     }
199
200     void SetAlarm(const std::string& alarm_name, const std::string& task_name,
201                   uint32_t offset, uint32_t cycle) {
202         alarms_.push_back({alarm_name, task_name, cycle, offset, offset, true});
203     }
204
205     // EN: Advance system tick and check alarms
206     // TR: Sistem tick'ini ilerlet ve alarmları kontrol et
207     void Tick() {
208         ++tick_counter_;
209         for (auto& alarm : alarms_) {
210             if (!alarm.active) continue;
211             if (alarm.remaining > 0) {
212                 --alarm.remaining;
213             }
```

```
            if (alarm.remaining == 0) {
                ActivateTask(alarm.task_name);
                if (alarm.cycle_ticks > 0) {
                    alarm.remaining = alarm.cycle_ticks;
                } else {
                    alarm.active = false;  // one-shot
                }
            }
        }
        Schedule();
    }

    void ActivateTask(const std::string& name) {
        for (auto& t : tasks_) {
            if (t.config.name == name && t.state == TaskState::SUSPENDED) {
                t.state = TaskState::READY;
                ++t.activation_count;
            }
        }
    }

    // EN: Priority-based scheduling (run all READY tasks, highest priority first)
    // TR: Öncelik tabanlı zamanlama (tüm READY task'lar öncelik sırasıyla çalışır)
    void Schedule() {
        for (;;) {
            Task* best = nullptr;
            for (auto& t : tasks_) {
                if (t.state == TaskState::READY) {
                    if (!best || t.config.priority > best->config.priority)
                        best = &t;
                }
            }
            if (!best) break;
            best->state = TaskState::RUNNING;
            if (best->config.entry) best->config.entry();
            best->state = TaskState::SUSPENDED;  // TerminateTask implicit
        }
    }

    void PrintStatus() const {
        std::cout << "  OS Tick=" << tick_counter_ << " | Tasks: ";
        for (const auto& t : tasks_) {
            std::cout << t.config.name << "(act=" << t.activation_count << ") ";
        }
        std::cout << "\n";
    }
};

}  // namespace autosar_os

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: COM Stack — PDU Router Simulation
// ═══════════════════════════════════════════════════════════════════════════

namespace autosar_com {

// EN: PDU (Protocol Data Unit) — the fundamental data exchange unit
// TR: PDU — temel veri alışveriş birimi

struct PDU {
    uint32_t               id = 0;
    std::vector<uint8_t>   data;
    std::string            source;
    std::string            destination;
};

// EN: PDU Router — routes PDUs between COM, CanIf, LinIf, SoAd
// TR: PDU Router — PDU'ları COM, CanIf, LinIf, SoAd arasında yönlendirir

struct RoutingEntry {
    uint32_t    src_pdu_id;
    std::string src_module;
    uint32_t    dst_pdu_id;
    std::string dst_module;
};
```

```

class PduRouter {
    std::vector<RoutingEntry> routing_table_;
    uint32_t routed_count_ = 0;

public:
    void AddRoute(uint32_t src_id, const std::string& src_mod,
                  uint32_t dst_id, const std::string& dst_mod) {
        routing_table_.push_back({src_id, src_mod, dst_id, dst_mod});
    }

    std::optional<PDU> Route(const PDU& pdu) {
        for (const auto& entry : routing_table_) {
            if (pdu.id == entry.src_pdu_id && pdu.source == entry.src_module) {
                PDU routed = pdu;
                routed.id = entry.dst_pdu_id;
                routed.source = entry.src_module;
                routed.destination = entry.dst_module;
                ++routed_count_;
                return routed;
            }
        }
        return std::nullopt;  // no route found
    }

    void PrintTable() const {
        std::cout << "  PDU Routing Table (" << routing_table_.size() << " entries):\n";
        std::cout << "  ┌──────────┬──────────┬──────────┬──────────┐\n";
        std::cout << "  │ Src PDU  │ Src Module │ Dst PDU  │ Dst Module │\n";
        std::cout << "  ├──────────┼──────────┼──────────┼──────────┤\n";
        for (const auto& e : routing_table_) {
            std::cout << "  │ 0x" << std::hex << std::setw(4) << std::setfill('0') << e.src_pdu_id
                      << "   │ " << std::setw(10) << std::setfill(' ') << std::left << e.src_module
                      << " │ 0x" << std::setw(4) << std::setfill('0') << std::right << e.dst_pdu_id
                      << "   │ " << std::setw(10) << std::setfill(' ') << std::left << e.dst_module
                      << " │\n" << std::dec;
        }
        std::cout << std::right << std::setfill(' ');
        std::cout << "  └──────────┴──────────┴──────────┴──────────┘\n";
        std::cout << "  Total routed: " << routed_count_ << "\n";
    }
};

// ---------------------------------------------------------------------
// CanIf — CAN Interface Layer (between PduR and CanDrv)
// ---------------------------------------------------------------------

class CanIf {
    uint32_t tx_count_ = 0;
    uint32_t rx_count_ = 0;

public:
    void Transmit(const PDU& pdu) {
        ++tx_count_;
        std::cout << "  [CanIf] TX → CAN ID=0x" << std::hex << pdu.id
                  << " len=" << std::dec << pdu.data.size() << " bytes\n";
    }

    PDU Receive(uint32_t can_id, const std::vector<uint8_t>& data) {
        ++rx_count_;
        PDU pdu{};
        pdu.id = can_id;
        pdu.data = data;
        pdu.source = "CanIf";
        return pdu;
    }

    void PrintStats() const {
        std::cout << "  [CanIf] TX=" << tx_count_ << " RX=" << rx_count_ << "\n";
    }
};

}  // namespace autosar_com
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════
// PART 4: Memory Stack — NvM Simulation
// ═══════════════════════════════════════════════════════════════════════════

namespace autosar_nvm {

// EN: NvM (Non-volatile Memory Manager) — manages read/write of NV blocks
//     Architecture: NvM → MemIf → Fee(Flash) or Ea(EEPROM) → Fls/Eep(MCAL)
// TR: NvM — kalıcı bellek blok yöneticisi
//     Mimari: NvM → MemIf → Fee(Flash) veya Ea(EEPROM) → Fls/Eep(MCAL)

enum class NvMRequestResult : uint8_t {
    NVM_REQ_OK = 0,
    NVM_REQ_NOT_OK,
    NVM_REQ_PENDING,
    NVM_REQ_INTEGRITY_FAILED,
    NVM_REQ_BLOCK_SKIPPED
};

struct NvMBlock {
    uint16_t              block_id;
    std::string           name;
    std::vector<uint8_t>  ram_mirror;
    std::vector<uint8_t>  nv_data;    // simulated NV storage
    bool                  dirty = false;
    NvMRequestResult      status = NvMRequestResult::NVM_REQ_OK;
    uint16_t              crc = 0;
};

class NvM {
    std::vector<NvMBlock> blocks_;

    static uint16_t CalculateCRC(const std::vector<uint8_t>& data) {
        uint16_t crc = 0xFFFF;
        for (auto byte : data) {
            crc ^= static_cast<uint16_t>(byte);
            for (int i = 0; i < 8; ++i)
                crc = (crc & 1) ? ((crc >> 1) ^ 0xA001u) : (crc >> 1);
        }
        return crc;
    }

public:
    void ConfigureBlock(uint16_t id, const std::string& name, size_t size) {
        NvMBlock block{};
        block.block_id = id;
        block.name = name;
        block.ram_mirror.resize(size, 0xFF);
        block.nv_data.resize(size, 0xFF);
        blocks_.push_back(std::move(block));
    }

    NvMRequestResult ReadBlock(uint16_t id) {
        auto* block = FindBlock(id);
        if (!block) return NvMRequestResult::NVM_REQ_NOT_OK;
        // EN: Copy NV → RAM mirror, verify CRC
        // TR: NV'den RAM aynasına kopyala, CRC doğrula
        block->ram_mirror = block->nv_data;
        uint16_t calc_crc = CalculateCRC(block->nv_data);
        if (block->crc != 0 && calc_crc != block->crc) {
            block->status = NvMRequestResult::NVM_REQ_INTEGRITY_FAILED;
            return NvMRequestResult::NVM_REQ_INTEGRITY_FAILED;
        }
        block->status = NvMRequestResult::NVM_REQ_OK;
        return NvMRequestResult::NVM_REQ_OK;
    }

    NvMRequestResult WriteBlock(uint16_t id, const std::vector<uint8_t>& data) {
        auto* block = FindBlock(id);
        if (!block) return NvMRequestResult::NVM_REQ_NOT_OK;
        if (data.size() > block->ram_mirror.size())
            return NvMRequestResult::NVM_REQ_NOT_OK;
        std::copy(data.begin(), data.end(), block->ram_mirror.begin());
        block->nv_data = block->ram_mirror;
        block->crc = CalculateCRC(block->nv_data);
        block->dirty = false;
```

```
439         block->status = NvMRequestResult::NVM_REQ_OK;
440         return NvMRequestResult::NVM_REQ_OK;
441     }
442 
443     void PrintBlocks() const {
444         std::cout << "  NvM Block Status:\n";
445         for (const auto& b : blocks_) {
446             std::cout << "    Block " << b.block_id << " [" << b.name
447                       << "] size=" << b.nv_data.size()
448                       << " CRC=0x" << std::hex << b.crc << std::dec
449                       << " status=" << static_cast<int>(b.status) << "\n";
450         }
451     }
452 
453 private:
454     NvMBlock* FindBlock(uint16_t id) {
455         for (auto& b : blocks_)
456             if (b.block_id == id) return &b;
457         return nullptr;
458     }
459 };
460 
461 }  // namespace autosar_nvm
462 
463 // ═══════════════════════════════════════════════════════════════════════
464 // PART 5: Diagnostic — DCM / DEM Simulation
465 // ═══════════════════════════════════════════════════════════════════════
466 
467 namespace autosar_diag {
468 
469 // EN: DEM — Diagnostic Event Manager: stores DTCs (Diagnostic Trouble Codes)
470 // TR: DEM — Teşhis Olay Yöneticisi: DTC'leri depolar
471 
472 enum class DTCStatus : uint8_t {
473     PASSED     = 0x00,
474     FAILED     = 0x01,
475     PENDING    = 0x04,
476     CONFIRMED  = 0x08,
477     TEST_NOT_COMPLETED = 0x50
478 };
479 
480 struct DTC {
481     uint32_t   dtc_number;        // e.g. 0xC07300
482     std::string description;
483     DTCStatus  status = DTCStatus::PASSED;
484     uint16_t   occurrence_counter = 0;
485     uint16_t   aging_counter = 0;
486 };
487 
488 class DEM {
489     std::vector<DTC> dtc_list_;
490 
491 public:
492     void RegisterDTC(uint32_t number, const std::string& desc) {
493         dtc_list_.push_back({number, desc, DTCStatus::PASSED, 0, 0});
494     }
495 
496     void ReportEvent(uint32_t dtc_number, bool test_failed) {
497         for (auto& dtc : dtc_list_) {
498             if (dtc.dtc_number == dtc_number) {
499                 if (test_failed) {
500                     dtc.status = DTCStatus::FAILED;
501                     ++dtc.occurrence_counter;
502                     dtc.aging_counter = 0;
503                     if (dtc.occurrence_counter >= 3)
504                         dtc.status = DTCStatus::CONFIRMED;
505                 } else {
506                     if (dtc.status == DTCStatus::FAILED)
507                         dtc.status = DTCStatus::PENDING;
508                     ++dtc.aging_counter;
509                     if (dtc.aging_counter >= 40)
510                         dtc.status = DTCStatus::PASSED;
511                 }
512                 return;
513             }
514         }
515     }
```

```cpp

    void ClearAll() {
        for (auto& dtc : dtc_list_) {
            dtc.status = DTCStatus::PASSED;
            dtc.occurrence_counter = 0;
            dtc.aging_counter = 0;
        }
    }

    void PrintAll() const {
        std::cout << "  DEM — DTC Status:\n";
        std::cout << "  ┌──────────┬──────────────────────────────┬──────────┬───────┐\n";
        std::cout << "  │ DTC      │ Description                  │ Status   │ Count │\n";
        std::cout << "  ├──────────┼──────────────────────────────┼──────────┼───────┤\n";
        for (const auto& d : dtc_list_) {
            std::cout << "  │ 0x" << std::hex << std::setw(6) << std::setfill('0')
                      << d.dtc_number << " │ " << std::dec << std::setfill(' ')
                      << std::setw(28) << std::left << d.description << " │ "
                      << std::setw(8) << StatusStr(d.status) << " │ "
                      << std::right << std::setw(5) << d.occurrence_counter << " │\n";
        }
        std::cout << std::right << std::setfill(' ');
        std::cout << "  └──────────┴──────────────────────────────┴──────────┴───────┘\n";
    }

private:
    static std::string_view StatusStr(DTCStatus s) {
        switch (s) {
            case DTCStatus::PASSED:     return "PASSED";
            case DTCStatus::FAILED:     return "FAILED";
            case DTCStatus::PENDING:    return "PENDING";
            case DTCStatus::CONFIRMED:  return "CONFIRM";
            case DTCStatus::TEST_NOT_COMPLETED: return "NOT_CMP";
        }
        return "UNKNOWN";
    }
};

}  // namespace autosar_diag

// ═══════════════════════════════════════════════════════════════════════════
// PART 6: EcuM / BswM — ECU State Management
// ═══════════════════════════════════════════════════════════════════════════

namespace autosar_ecum {

// EN: EcuM phases: STARTUP → UP (RUN/POST_RUN) → SHUTDOWN → SLEEP/OFF
// TR: EcuM fazları: STARTUP → UP (RUN/POST_RUN) → SHUTDOWN → SLEEP/OFF

enum class EcuMState : uint8_t {
    STARTUP, RUN, POST_RUN, PREP_SHUTDOWN, GO_SLEEP, SLEEP, GO_OFF, OFF
};

class EcuM {
    EcuMState state_ = EcuMState::STARTUP;
    uint32_t  uptime_ticks_ = 0;

public:
    void Init() {
        state_ = EcuMState::STARTUP;
        std::cout << "  [EcuM] Init → STARTUP\n";
        // EN: Init sequence: Mcu_Init, Port_Init, Dio_Init, SchM_Init, BswM_Init, ...
        // TR: Init sırası: Mcu_Init, Port_Init, Dio_Init, SchM_Init, BswM_Init, ...
        state_ = EcuMState::RUN;
        std::cout << "  [EcuM] Startup complete → RUN\n";
    }

    void MainFunction() {
        ++uptime_ticks_;
        // EN: Evaluate wakeup sources, manage run requests
        // TR: Uyandırma kaynaklarını değerlendir, çalışma isteklerini yönet
    }
```

```cpp

    void RequestShutdown() {
        std::cout << "  [EcuM] Shutdown requested → PREP_SHUTDOWN\n";
        state_ = EcuMState::PREP_SHUTDOWN;
        // EN: NvM_WriteAll, ComM_DeInit, ...
        // TR: NvM_WriteAll, ComM_DeInit, ...
        state_ = EcuMState::GO_OFF;
        std::cout << "  [EcuM] GO_OFF → Uptime was " << uptime_ticks_ << " ticks\n";
        state_ = EcuMState::OFF;
    }

    [[nodiscard]] EcuMState GetState() const { return state_; }

    static std::string_view StateStr(EcuMState s) {
        switch (s) {
            case EcuMState::STARTUP:       return "STARTUP";
            case EcuMState::RUN:           return "RUN";
            case EcuMState::POST_RUN:      return "POST_RUN";
            case EcuMState::PREP_SHUTDOWN: return "PREP_SHUTDOWN";
            case EcuMState::GO_SLEEP:      return "GO_SLEEP";
            case EcuMState::SLEEP:         return "SLEEP";
            case EcuMState::GO_OFF:        return "GO_OFF";
            case EcuMState::OFF:           return "OFF";
        }
        return "UNKNOWN";
    }
};

}  // namespace autosar_ecum

// ═══════════════════════════════════════════════════════════════════════
// MAIN — Demos
// ═══════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "╔═══════════════════════════════════════════════════════╗\n"
              << "║ Module 16: AUTOSAR Classic Platform                   ║\n"
              << "╚═══════════════════════════════════════════════════════╝\n\n";

    // --- Demo 1: SWC + RTE Communication ---
    std::cout << "--- Demo 1: SWC + RTE Communication (Sender/Receiver) ---\n";
    {
        autosar_classic::RTE rte;
        autosar_classic::SWC_EngineManagement engine_swc(rte);
        autosar_classic::SWC_SpeedCalculator  speed_swc(rte);
        autosar_classic::SWC_Dashboard        dash_swc(rte);

        for (int cycle = 0; cycle < 5; ++cycle) {
            engine_swc.Runnable_ReadSensors();
            speed_swc.Runnable_CalcSpeed();
            dash_swc.Runnable_UpdateDisplay();
        }
    }

    // --- Demo 2: Client/Server via RTE ---
    std::cout << "\n--- Demo 2: Client/Server via RTE ---\n";
    {
        autosar_classic::RTE rte;
        // EN: Register a diagnostic server (SWC_Diag provides ClearDTC service)
        // TR: Teşhis sunucusu kaydet (SWC_Diag ClearDTC servisi)
        rte.RegisterServer("ClearDTC",
            [](const std::vector<uint8_t>& req, std::vector<uint8_t>& resp) -> uint8_t {
                if (req.size() >= 3) {
                    uint32_t dtc = (static_cast<uint32_t>(req[0]) << 16) |
                                   (static_cast<uint32_t>(req[1]) << 8)  |
                                   static_cast<uint32_t>(req[2]);
                    std::cout << "  [Server] ClearDTC(0x" << std::hex << dtc << std::dec << ") → OK\n";
                    resp = {0x00};  // positive response
                    return 0x00;   // E_OK
                }
                resp = {0x13};  // incorrectMessageLength
                return 0x01;
            });

```

```
        // Client call
        std::vector<uint8_t> req = {0xC0, 0x73, 0x00};  // DTC 0xC07300
        std::vector<uint8_t> resp;
        auto result = rte.Rte_Call("ClearDTC", req, resp);
        std::cout << "  Client result: 0x" << std::hex << static_cast<int>(result)
                  << std::dec << "\n";
    }

    // --- Demo 3: AUTOSAR OS — Tasks and Alarms ---
    std::cout << "\n--- Demo 3: AUTOSAR OS — Tasks and Alarms ---\n";
    {
        autosar_os::OS os;

        int task_10ms_count = 0, task_100ms_count = 0;

        os.DeclareTask({"Task_10ms", 10, true, [&]() {
            ++task_10ms_count;
        }, 512});
        os.DeclareTask({"Task_100ms", 5, true, [&]() {
            ++task_100ms_count;
        }, 512});

        os.SetAlarm("Alarm_10ms",  "Task_10ms",  1, 1);   // every tick
        os.SetAlarm("Alarm_100ms", "Task_100ms", 10, 10);  // every 10 ticks

        for (int i = 0; i < 100; ++i) os.Tick();

        os.PrintStatus();
        std::cout << "  Task_10ms executed: " << task_10ms_count << " times\n";
        std::cout << "  Task_100ms executed: " << task_100ms_count << " times\n";
    }

    // --- Demo 4: COM Stack — PDU Routing ---
    std::cout << "\n--- Demo 4: COM Stack — PDU Routing ---\n";
    {
        autosar_com::PduRouter pdur;
        autosar_com::CanIf    can_if;

        // EN: Configure routes: COM → CanIf, LinIf → COM
        // TR: Rotaları yapılandır
        pdur.AddRoute(0x0100, "COM",   0x0200, "CanIf");
        pdur.AddRoute(0x0150, "COM",   0x0250, "CanIf");
        pdur.AddRoute(0x0300, "CanIf", 0x0400, "COM");
        pdur.PrintTable();

        // EN: Route a TX PDU from COM to CanIf
        // TR: COM'dan CanIf'e PDU yönlendir
        autosar_com::PDU tx_pdu{0x0100, {0x11, 0x22, 0x33, 0x44}, "COM", ""};
        auto routed = pdur.Route(tx_pdu);
        if (routed) can_if.Transmit(*routed);

        // EN: Receive from CAN bus and route to COM
        // TR: CAN bus'tan al ve COM'a yönlendir
        auto rx_pdu = can_if.Receive(0x0300, {0xAA, 0xBB});
        auto routed_rx = pdur.Route(rx_pdu);
        if (routed_rx)
            std::cout << "  [PduR] Routed RX PDU 0x" << std::hex << routed_rx->id
                      << " → " << routed_rx->destination << std::dec << "\n";
        can_if.PrintStats();
    }

    // --- Demo 5: NvM — Non-Volatile Memory Manager ---
    std::cout << "\n--- Demo 5: NvM — Non-Volatile Memory Manager ---\n";
    {
        autosar_nvm::NvM nvm;
        nvm.ConfigureBlock(1, "CalibData", 16);
        nvm.ConfigureBlock(2, "LearnValues", 8);
        nvm.ConfigureBlock(3, "DTCSnapshot", 32);

        // EN: Write calibration data
        // TR: Kalibrasyon verisi yaz
        nvm.WriteBlock(1, {0x10, 0x20, 0x30, 0x40, 0x50, 0x60});
        nvm.WriteBlock(2, {0xAB, 0xCD, 0xEF, 0x01});

```

```
        // EN: Read back and verify
        // TR: Geri oku ve doğrula
        auto result = nvm.ReadBlock(1);
        std::cout << "  NvM ReadBlock(1) result: "
                  << (result == autosar_nvm::NvMRequestResult::NVM_REQ_OK ? "OK" : "FAIL")
                  << "\n";
        nvm.PrintBlocks();
    }

    // --- Demo 6: DEM — Diagnostic Event Manager ---
    std::cout << "\n--- Demo 6: DEM — Diagnostic Event Manager (DTC lifecycle) ---\n";
    {
        autosar_diag::DEM dem;
        dem.RegisterDTC(0xC07300, "Lost CAN communication");
        dem.RegisterDTC(0xC10000, "Sensor plausibility");
        dem.RegisterDTC(0xC41400, "Battery voltage low");

        // EN: Simulate events: C07300 fails 3x → confirmed, then heals
        // TR: Olay simülasyonu: C07300 3 kez fail → onaylandı, sonra iyileşir
        dem.ReportEvent(0xC07300, true);
        dem.ReportEvent(0xC07300, true);
        dem.ReportEvent(0xC07300, true);   // → CONFIRMED
        dem.ReportEvent(0xC10000, true);
        dem.ReportEvent(0xC10000, false);  // → PENDING
        dem.PrintAll();

        std::cout << "  Clearing all DTCs...\n";
        dem.ClearAll();
        dem.PrintAll();
    }

    // --- Demo 7: EcuM — State Management ---
    std::cout << "\n--- Demo 7: EcuM — ECU State Management ---\n";
    {
        autosar_ecum::EcuM ecum;
        ecum.Init();
        for (int i = 0; i < 50; ++i) ecum.MainFunction();
        std::cout << "  Current state: "
                  << autosar_ecum::EcuM::StateStr(ecum.GetState()) << "\n";
        ecum.RequestShutdown();
        std::cout << "  Final state: "
                  << autosar_ecum::EcuM::StateStr(ecum.GetState()) << "\n";
    }

    // --- Demo 8: AUTOSAR vs Bare-metal vs Linux ---
    std::cout << "\n--- Demo 8: AUTOSAR vs Bare-metal vs Linux Comparison ---\n";
    {
        std::cout << "  ┌──────────────────────┬────────────┬──────────────┬──────────────┐\n";
        std::cout << "  | Feature              | Bare-Metal | AUTOSAR Classic | Linux/POSIX |\n";
        std::cout << "  ├──────────────────────┼────────────┼──────────────┼──────────────┤\n";
        std::cout << "  | OS                   | None/custom | AUTOSAR OS(OSEK) | Linux kernel |\n";
        std::cout << "  | Language             | C / ASM    | C (SWS), C++ SWC | C, C++, Rust |\n";
        std::cout << "  | Communication        | Direct reg | COM/PduR/CanIf | Socket/IPC |\n";
        std::cout << "  | Diagnostics          | Custom     | DCM/DEM/NRC | Custom/SOME-IP |\n";
        std::cout << "  | NV Storage           | Direct EEPROM | NvM/Fee/Ea | Filesystem |\n";
        std::cout << "  | Configuration        | #define    | ARXML/tools | Config files |\n";
        std::cout << "  | Code Reuse           | Low        | High (SWC port.) | Medium |\n";
        std::cout << "  | Boot Time            | ~1ms       | ~50ms-300ms | ~500ms-5s |\n";
        std::cout << "  | Safety (ISO 26262)   | Possible   | ASIL-D ready | ASIL-B (QNX-D) |\n";
        std::cout << "  | Typical MCU          | ATmega/PIC | AURIX/RH850/S32K | i.MX8/SA8155P |\n";
        std::cout << "  └──────────────────────┴────────────┴──────────────┴──────────────┘\n";
    }

```

```
    std::cout << "\n[All demos completed successfully.]\n";
    return 0;
}
```

► 02. Adaptive Platform

module_16_autosar/02_adaptive_platform.cpp

Konu: AUTOSAR Adaptive Platform — Modern C++ in Automotive / Otomotivde Modern C++

Teori / Theory

THEORY: AUTOSAR Adaptive Platform — Overview

AUTOSAR Adaptive Platform (AP) is designed for high-performance ECUs running POSIX-based OS (Linux, QNX). Unlike Classic (C-based, static), Adaptive uses C++14/17, dynamic communication, and SOA.
Adaptive vs Classic:

```
┌──────────────────┬──────────────────┬──────────────────────┐
│ Aspect           │ Classic Platform │ Adaptive Platform    │
├──────────────────┼──────────────────┼──────────────────────┤
│ Language         │ C (MISRA C)      │ C++14/17 (MISRA C++ '23) │
│ OS               │ AUTOSAR OS(OSEK) │ POSIX PSE51 (Linux/QNX)  │
│ Communication    │ Signal-based     │ Service-oriented (SOA)   │
│ Update           │ Reflash ECU      │ OTA, dynamic deploy      │
│ Scheduling       │ Static (offline) │ Dynamic (OS scheduler)   │
│ Memory           │ Static alloc     │ Dynamic alloc possible   │
│ Binding          │ Compile-time     │ Runtime service binding  │
│ MCU/SoC          │ AURIX, RH850     │ SA8155P, i.MX8, Orin     │
│ Typical Use      │ Powertrain, body │ ADAS, cockpit, gateway   │
│ Safety           │ ASIL-D           │ ASIL-B+ (QNX → ASIL-D)   │
└──────────────────┴──────────────────┴──────────────────────┘
```

TEORİ: AUTOSAR Adaptive Platform — Genel Bakış

AUTOSAR Adaptive Platform (AP), POSIX tabanlı OS üzerinde çalışan yüksek performanslı ECU'lar için tasarlanmıştır. Classic'in aksine (C tabanlı, statik), Adaptive C++14/17, dinamik iletişim ve SOA kullanır.
Adaptive vs Classic:

```
┌──────────────────┬──────────────────┬──────────────────────┐
│ Yön              │ Classic Platform │ Adaptive Platform    │
├──────────────────┼──────────────────┼──────────────────────┤
│ Dil              │ C (MISRA C)      │ C++14/17 (MISRA C++ '23)  │
│ İşletim Sistemi  │ AUTOSAR OS(OSEK) │ POSIX PSE51 (Linux/QNX)   │
│ İletişim         │ Sinyal tabanlı   │ Servis odaklı (SOA)       │
│ Güncelleme       │ ECU yeniden yaz  │ OTA, dinamik dağıtım      │
│ Zamanlama        │ Statik(çevrimdışı)│ Dinamik (OS zamanlayıcı)  │
│ Bellek           │ Statik tahsis    │ Dinamik tahsis mümkün     │
│ Bağlama          │ Derleme zamanı   │ Çalışma zamanı servis bağ.│
│ MCU/SoC          │ AURIX, RH850     │ SA8155P, i.MX8, Orin      │
│ Tipik Kullanım   │ Güç aktar., gövde│ ADAS, kokpit, gateway     │
│ Güvenlik         │ ASIL-D           │ ASIL-B+ (QNX → ASIL-D)    │
└──────────────────┴──────────────────┴──────────────────────┘
```

THEORY: ARA — AUTOSAR Runtime for Adaptive Applications

ARA is the middleware API layer exposing platform services:

```
┌──────────────────────────────────────────────────────┐
│          Adaptive Application (C++14/17)              │
├──────────┬──────────┬──────────┬──────────┬──────────┬──────────┤
│ara::com  │ara::exec│ara::diag│ara::per│ara::log│ara::crypto│
├──────────┴──────────┴──────────┴──────────┴──────────┴──────────┤
│     Operating System (POSIX PSE51 – Linux/QNX)            │
├──────────────────────────────────────────────────────┤
│     Hardware (SoC: ARM Cortex-A, GPU, NPU)                │
└──────────────────────────────────────────────────────┘
```

ARA Functional Clusters:

```
┌──────────────┬──────────────────────────────────────────┐
│ Cluster      │ Purpose                                  │
├──────────────┼──────────────────────────────────────────┤
│ ara::com     │ Service-oriented communication (SOME/IP,DDS) │
│ ara::exec    │ Execution & state management, process life   │
│ ara::diag    │ UDS-over-DoIP diagnostic communication       │
│ ara::per     │ Persistency (key-value store, file storage)  │
│ ara::log     │ Logging (DLT – Diagnostic Log and Trace)     │
│ ara::crypto  │ Crypto services (HSM abstraction)            │
│ ara::iam     │ Identity & Access Mgmt (security policy)     │
│ ara::ucm     │ Update & Config Mgmt (OTA deployment)        │
│ ara::phm     │ Platform Health Mgmt (watchdog, supervision) │
│ ara::tsync   │ Time Synchronization (gPTP, StbM)            │
└──────────────┴──────────────────────────────────────────┘
```

TEORİ: ARA — Uyarlanabilir Uygulamalar İçin AUTOSAR Çalışma Zamanı

ARA, platform servislerini açığa çıkaran middleware API katmanıdır:

```
┌──────────────────────────────────────────────────────┐
│          Adaptive Uygulama (C++14/17)                 │
├──────────┬──────────┬──────────┬──────────┬──────────┬──────────┤
│ara::com  │ara::exec│ara::diag│ara::per│ara::log│ara::crypto│
├──────────┴──────────┴──────────┴──────────┴──────────┴──────────┤
│   İşletim Sistemi (POSIX PSE51 – Linux/QNX)               │
├──────────────────────────────────────────────────────┤
│   Donanım (SoC: ARM Cortex-A, GPU, NPU)                   │
└──────────────────────────────────────────────────────┘
```

ARA Fonksiyonel Kümeleri:

```
┌──────────────┬──────────────────────────────────────────┐
│ Küme         │ Amaç                                     │
├──────────────┼──────────────────────────────────────────┤
│ ara::com     │ Servis odaklı iletişim (SOME/IP, DDS)        │
│ ara::exec    │ Yürütme ve durum yönetimi, işlem yaşam döngüsü │
│ ara::diag    │ UDS-over-DoIP tanılama iletişimi             │
│ ara::per     │ Kalıcılık (key-value deposu, dosya depolama) │
│ ara::log     │ Günlükleme (DLT)                             │
│ ara::crypto  │ Kripto servisleri (HSM soyutlaması)          │
│ ara::iam     │ Kimlik ve Erişim Yönetimi (güvenlik politikası)│
│ ara::ucm     │ Güncelleme ve Yapılandırma Yönetimi (OTA)    │
│ ara::phm     │ Platform Sağlık Yönetimi (watchdog, denetim) │
│ ara::tsync   │ Zaman Senkronizasyonu (gPTP, StbM)           │
└──────────────┴──────────────────────────────────────────┘
```

THEORY: ara::com — Proxy/Skeleton Pattern

ara::com uses the Proxy/Skeleton pattern for service communication:
- Skeleton: Service provider (server-side implementation)
- Proxy: Service consumer (client-side stub)
- Communication elements: Methods, Events, Fields

```
┌──────────────┐            ┌────────────────┐
│  Consumer    │            │   Provider     │
│ Application  │            │  Application   │
├──────────────┤            ├────────────────┤
│ Proxy        │◄──────────►│ Skeleton       │
│ (generated)  │  SOME/IP   │ (generated)    │
│  FindService│  or DDS    │  OfferService  │
│  .Method()   │            │  .Method()     │
│  .Event      │            │  .Event.Send()│
│  .Field.Get  │            │  .Field.Update│
└──────────────┘            └────────────────┘
```

Discovery:
1. Skeleton calls OfferService()
2. Proxy calls FindService() → gets ServiceHandle
3. Proxy creates instance from handle → communicates

TEORİ: ara::com — Proxy/Skeleton Deseni

ara::com, Proxy/Skeleton kalıbını kullanır:
- Skeleton: Servis sağlayıcı (sunucu tarafı uygulaması)
- Proxy: Servis tüketici (istemci tarafı stub)
- İletişim öğeleri: Method, Event, Field

```
┌──────────────┐            ┌────────────────┐
│  Tüketici    │            │   Sağlayıcı    │
│  Uygulama    │            │   Uygulama     │
├──────────────┤            ├────────────────┤
│ Proxy        │◄──────────►│ Skeleton       │
│ (üretilmiş)  │  SOME/IP   │ (üretilmiş)    │
│ FindService  │  veya DDS  │ OfferService   │
└──────────────┘            └────────────────┘
```

Keşif:
1. Skeleton OfferService() çağrır
2. Proxy FindService() çağrır → ServiceHandle alır
3. Proxy handle'dan örnek oluşturur → iletişim başlar

THEORY: POSIX PSE51 Restrictions

Adaptive Platform mandates POSIX PSE51 profile (minimal realtime): ALLOWED: threads, mutexes, condition variables, timers, file I/O (basic) FORBIDDEN: fork(), exec(), signals (SIGKILL etc.), shared memory (shm_open), System V IPC, dynamic library loading (dlopen)
→ Purpose: Ensure deterministic behavior on safety-certified RTOS (QNX)

TEORİ: POSIX PSE51 Kısıtlamaları

PSE51 profili minimum gerçek zamanlı POSIX alt kümesidir. İZİN VERİLEN: thread'ler, mutex'ler, condition variable, timer, dosya G/Ç YASAKLANAN: fork(), exec(), sinyaller (SIGKILL vb.), shm_open, System V IPC, dlopen (dinamik kütüphane yükleme)
→ Amaç: Güvenlik sertifikalı RTOS'üzerinde (QNX) deterministik davranış sağlama

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <string>
#include <string_view>
#include <vector>
#include <map>
#include <unordered_map>
#include <functional>
#include <optional>
#include <variant>
#include <memory>
#include <algorithm>
#include <iomanip>
#include <cassert>
#include <sstream>
#include <chrono>
#include <any>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: ara::com — Service-Oriented Communication (Proxy/Skeleton)
// ═══════════════════════════════════════════════════════════════════════════

namespace ara::com {

// EN: Result type (ara::core::Result analog)
// TR: Sonuç tipi (ara::core::Result benzeri)
template <typename T, typename E = int>
class Result {
    std::variant<T, E> storage_;
public:
    explicit Result(T value) : storage_(std::move(value)) {}
    explicit Result(E error) : storage_(std::move(error)) {}
    [[nodiscard]] bool HasValue() const { return std::holds_alternative<T>(storage_); }
    [[nodiscard]] const T& Value() const { return std::get<T>(storage_); }
    [[nodiscard]] const E& Error() const { return std::get<E>(storage_); }
};

// EN: Future — simplified async result (ara::core::Future analog)
// TR: Future — basitleştirilmiş asenkron sonuç
template <typename T>
class Future {
    std::optional<T> value_;
    bool ready_ = false;
public:
    void SetValue(T val) { value_ = std::move(val); ready_ = true; }
    [[nodiscard]] bool is_ready() const { return ready_; }
    T get() { return *value_; }
};

// EN: ServiceHandle — returned by FindService, used to create Proxy
// TR: ServiceHandle — FindService'den döner, Proxy oluşturmak için kullanılır
struct ServiceHandle {
    uint16_t service_id = 0;
    uint16_t instance_id = 0;
    std::string instance_name;
};

// EN: Service discovery registry — simulates SOME/IP-SD
// TR: Servis keşif kaydı — SOME/IP-SD simülasyonu
class ServiceRegistry {
    std::vector<ServiceHandle> offered_services_;
    static ServiceRegistry* instance_;
public:
    static ServiceRegistry& Instance() {
        static ServiceRegistry reg;
        return reg;
    }
    void OfferService(uint16_t sid, uint16_t iid, const std::string& name) {
        offered_services_.push_back({sid, iid, name});
    }
```

```
    void StopOfferService(uint16_t sid, uint16_t iid) {
        std::erase_if(offered_services_, [&](const ServiceHandle& h) {
            return h.service_id == sid && h.instance_id == iid;
        });
    }
    std::vector<ServiceHandle> FindService(uint16_t sid) const {
        std::vector<ServiceHandle> result;
        for (const auto& h : offered_services_)
            if (h.service_id == sid) result.push_back(h);
        return result;
    }
    void Clear() { offered_services_.clear(); }
};

// ---------------------------------------------------------------------------
// Radar Service — Skeleton (Provider)
// ---------------------------------------------------------------------------

// EN: Service interface definition: RadarService
//     - Methods: Calibrate(mode) → CalibResult
//     - Events:  ObjectDetected (range, angle, velocity)
//     - Fields:  UpdateRate (get/set)
// TR: Servis arayüz tanımı: RadarService

struct RadarObject {
    float range_m = 0.0f;
    float angle_deg = 0.0f;
    float velocity_mps = 0.0f;
    uint32_t object_id = 0;
};

struct CalibResult {
    bool success = false;
    std::string message;
};

// EN: Skeleton — service provider (server implementation)
// TR: Skeleton — servis sağlayıcı (sunucu implementasyonu)
class RadarServiceSkeleton {
    uint16_t service_id_;
    uint16_t instance_id_;
    uint32_t update_rate_hz_ = 20;

    // EN: Event subscribers
    // TR: Olay aboneleri
    std::vector<std::function<void(const RadarObject&)>> object_subscribers_;

public:
    RadarServiceSkeleton(uint16_t sid, uint16_t iid)
        : service_id_(sid), instance_id_(iid) {}

    void OfferService() {
        ServiceRegistry::Instance().OfferService(service_id_, instance_id_,
            "RadarService_" + std::to_string(instance_id_));
        std::cout << "  [Skeleton] OfferService(sid=0x"
                  << std::hex << service_id_ << ", iid=" << instance_id_
                  << std::dec << ")\n";
    }

    void StopOfferService() {
        ServiceRegistry::Instance().StopOfferService(service_id_, instance_id_);
    }

    // --- Method: Calibrate ---
    CalibResult Calibrate(uint8_t mode) {
        std::cout << "  [Skeleton] Calibrate(mode=" << static_cast<int>(mode) << ")\n";
        if (mode <= 3) return {true, "Calibration OK, mode=" + std::to_string(mode)};
        return {false, "Invalid calibration mode"};
    }

    // --- Event: ObjectDetected (send to all subscribers) ---
    void RegisterSubscriber(std::function<void(const RadarObject&)> cb) {
        object_subscribers_.push_back(std::move(cb));
    }
```

```cpp

    void SendObjectEvent(const RadarObject& obj) {
        for (const auto& sub : object_subscribers_)
            sub(obj);
    }

    // --- Field: UpdateRate ---
    [[nodiscard]] uint32_t GetUpdateRate() const { return update_rate_hz_; }
    void SetUpdateRate(uint32_t hz) { update_rate_hz_ = hz; }
};

// ---------------------------------------------------------------------------
// Radar Service — Proxy (Consumer)
// ---------------------------------------------------------------------------

// EN: Proxy — service consumer (client stub)
// TR: Proxy — servis tüketici (istemci stub'ı)
class RadarServiceProxy {
    ServiceHandle handle_;
    RadarServiceSkeleton* skeleton_ = nullptr;  // direct binding for simulation

public:
    explicit RadarServiceProxy(const ServiceHandle& h) : handle_(h) {}

    // EN: In real AP, FindService returns Future<ServiceHandleContainer>
    // TR: Gerçek AP'de FindService, Future<ServiceHandleContainer> döner
    static std::vector<ServiceHandle> FindService(uint16_t sid) {
        return ServiceRegistry::Instance().FindService(sid);
    }

    void BindToSkeleton(RadarServiceSkeleton* skel) { skeleton_ = skel; }

    // --- Method call ---
    Future<CalibResult> Calibrate(uint8_t mode) {
        Future<CalibResult> fut;
        if (skeleton_) {
            auto result = skeleton_->Calibrate(mode);
            fut.SetValue(result);
        }
        return fut;
    }

    // --- Event subscription ---
    void SubscribeObjectDetected(std::function<void(const RadarObject&)> cb) {
        if (skeleton_) skeleton_->RegisterSubscriber(std::move(cb));
    }

    // --- Field access ---
    Future<uint32_t> GetUpdateRate() {
        Future<uint32_t> fut;
        if (skeleton_) fut.SetValue(skeleton_->GetUpdateRate());
        return fut;
    }

    void SetUpdateRate(uint32_t hz) {
        if (skeleton_) skeleton_->SetUpdateRate(hz);
    }
};

}  // namespace ara::com

// ===========================================================================
// PART 2: ara::exec — Execution Management
// ===========================================================================

namespace ara::exec {

// EN: Application states in Adaptive Platform
// TR: Adaptive Platform'da uygulama durumları
//      ┌─────────┐    Init    ┌──────────┐   Run    ┌─────────┐   Term   ┌──────────┐
//      │kInit    │──────────► │kRunning  │──────►   │kTermin. │──────►   │kTerminated │
//      └─────────┘            └──────────┘          └─────────┘          └──────────┘

enum class ExecutionState : uint8_t {
    kInit, kRunning, kTerminating, kTerminated
};
```

```cpp

// EN: Execution Client — reports state to Execution Management
// TR: Execution Client — durumu Execution Management'a raporlar
class ExecutionClient {
    std::string app_name_;
    ExecutionState state_ = ExecutionState::kInit;

public:
    explicit ExecutionClient(std::string name) : app_name_(std::move(name)) {}

    void ReportExecutionState(ExecutionState state) {
        state_ = state;
        std::cout << "  [ara::exec] " << app_name_ << " → "
                  << StateStr(state) << "\n";
    }

    [[nodiscard]] ExecutionState GetState() const { return state_; }

    static std::string_view StateStr(ExecutionState s) {
        switch (s) {
            case ExecutionState::kInit:        return "kInit";
            case ExecutionState::kRunning:     return "kRunning";
            case ExecutionState::kTerminating: return "kTerminating";
            case ExecutionState::kTerminated:  return "kTerminated";
        }
        return "Unknown";
    }
};

// EN: Deterministic Client — for deterministic execution (ADAS pipelines)
// TR: Deterministik İstemci — deterministik yürütme (ADAS pipeline'ları)
class DeterministicClient {
    uint64_t cycle_counter_ = 0;
    uint32_t cycle_time_us_ = 0;

public:
    explicit DeterministicClient(uint32_t cycle_us) : cycle_time_us_(cycle_us) {}

    // EN: WaitForNextActivation() blocks until next deterministic cycle
    // TR: Sonraki deterministik döngüye kadar bekler
    uint64_t WaitForNextActivation() {
        ++cycle_counter_;
        // EN: In real AP, this synchronizes with OS timer
        // TR: Gerçek AP'de OS zamanlayıcı ile senkronize olur
        return cycle_counter_;
    }

    [[nodiscard]] uint32_t GetCycleTimeUs() const { return cycle_time_us_; }
};

}  // namespace ara::exec

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: ara::diag — Diagnostic Communication (UDS-over-DoIP)
// ═══════════════════════════════════════════════════════════════════════════

namespace ara::diag {

// EN: Diagnostic conversation — UDS session over DoIP
// TR: Teşhis görüşmesi — DoIP üzerinden UDS oturumu

enum class SessionType : uint8_t {
    kDefault = 0x01, kProgramming = 0x02, kExtended = 0x03
};

enum class NRC : uint8_t {
    kPositiveResponse           = 0x00,
    kSubFunctionNotSupported    = 0x12,
    kIncorrectMessageLength     = 0x13,
    kConditionsNotCorrect       = 0x22,
    kRequestOutOfRange          = 0x31,
    kSecurityAccessDenied       = 0x33,
    kRequestSequenceError       = 0x24
};

```

```
struct DiagRequest {
    uint8_t              sid;
    std::vector<uint8_t> sub_data;
};

struct DiagResponse {
    uint8_t              sid;          // SID + 0x40 for positive
    NRC                  nrc;
    std::vector<uint8_t> data;
};

class DiagServer {
    SessionType session_ = SessionType::kDefault;
    bool security_unlocked_ = false;

    // EN: DID storage (simplified)
    // TR: DID deposu (basitleştirilmiş)
    std::map<uint16_t, std::vector<uint8_t>> did_data_;

public:
    DiagServer() {
        // EN: Pre-populate some DIDs
        // TR: Bazı DID'leri önceden doldur
        did_data_[0xF190] = {'V', 'I', 'N', '1', '2', '3', '4', '5',
                             '6', '7', '8', '9', 'A', 'B', 'C', 'D', 'E'};
        did_data_[0xF187] = {'S', 'W', '_', 'v', '2', '.', '1'};
        did_data_[0x0100] = {0x03, 0xE8};  // engine RPM
    }

    DiagResponse HandleRequest(const DiagRequest& req) {
        DiagResponse resp{};
        switch (req.sid) {
            case 0x10: return HandleSessionControl(req);
            case 0x22: return HandleReadDID(req);
            case 0x27: return HandleSecurityAccess(req);
            case 0x3E: return HandleTesterPresent(req);
            default:
                resp.sid = 0x7F;
                resp.nrc = NRC::kSubFunctionNotSupported;
                return resp;
        }
    }

private:
    DiagResponse HandleSessionControl(const DiagRequest& req) {
        DiagResponse resp{};
        if (req.sub_data.empty()) {
            resp.sid = 0x7F; resp.nrc = NRC::kIncorrectMessageLength;
            return resp;
        }
        session_ = static_cast<SessionType>(req.sub_data[0]);
        resp.sid = 0x50;  // positive
        resp.nrc = NRC::kPositiveResponse;
        resp.data = {req.sub_data[0]};
        std::cout << "  [ara::diag] Session → 0x"
                  << std::hex << static_cast<int>(req.sub_data[0]) << std::dec << "\n";
        return resp;
    }

    DiagResponse HandleReadDID(const DiagRequest& req) {
        DiagResponse resp{};
        if (req.sub_data.size() < 2) {
            resp.sid = 0x7F; resp.nrc = NRC::kIncorrectMessageLength;
            return resp;
        }
        uint16_t did = static_cast<uint16_t>(
            (static_cast<uint16_t>(req.sub_data[0]) << 8) | req.sub_data[1]);
        auto it = did_data_.find(did);
        if (it == did_data_.end()) {
            resp.sid = 0x7F; resp.nrc = NRC::kRequestOutOfRange;
            return resp;
        }
        resp.sid = 0x62;  // positive ReadDID
        resp.nrc = NRC::kPositiveResponse;
        resp.data = {req.sub_data[0], req.sub_data[1]};
        resp.data.insert(resp.data.end(), it->second.begin(), it->second.end());
        return resp;
```

```
373     }
374
375     DiagResponse HandleSecurityAccess(const DiagRequest& req) {
376         DiagResponse resp{};
377         if (req.sub_data.empty()) {
378             resp.sid = 0x7F; resp.nrc = NRC::kIncorrectMessageLength;
379             return resp;
380         }
381         if (req.sub_data[0] == 0x01) {
382             // EN: Seed request
383             // TR: Seed isteği
384             resp.sid = 0x67;
385             resp.nrc = NRC::kPositiveResponse;
386             resp.data = {0x01, 0xDE, 0xAD, 0xBE, 0xEF};  // seed
387         } else if (req.sub_data[0] == 0x02) {
388             // EN: Key response — accept any key for demo
389             // TR: Anahtar yanıtı — demo için herhangi bir anahtarı kabul et
390             security_unlocked_ = true;
391             resp.sid = 0x67;
392             resp.nrc = NRC::kPositiveResponse;
393             resp.data = {0x02};
394         } else {
395             resp.sid = 0x7F;
396             resp.nrc = NRC::kSubFunctionNotSupported;
397         }
398         return resp;
399     }
400
401     DiagResponse HandleTesterPresent(const DiagRequest& /*req*/) {
402         return {0x7E, NRC::kPositiveResponse, {}};
403     }
404 };
405
406 }  // namespace ara::diag
407
408 // ═══════════════════════════════════════════════════════════════════════
409 // PART 4: ara::per — Persistency (Key-Value Store)
410 // ═══════════════════════════════════════════════════════════════════════
411
412 namespace ara::per {
413
414 // EN: Key-Value Storage — simplified ara::per::KeyValueStorage
415 // TR: Anahtar-Değer Deposu — basitleştirilmiş ara::per::KeyValueStorage
416
417 class KeyValueStorage {
418     std::string storage_id_;
419     std::map<std::string, std::vector<uint8_t>> store_;
420
421 public:
422     explicit KeyValueStorage(std::string id) : storage_id_(std::move(id)) {}
423
424     bool SetValue(const std::string& key, const std::vector<uint8_t>& value) {
425         store_[key] = value;
426         return true;
427     }
428
429     std::optional<std::vector<uint8_t>> GetValue(const std::string& key) const {
430         auto it = store_.find(key);
431         if (it == store_.end()) return std::nullopt;
432         return it->second;
433     }
434
435     bool RemoveKey(const std::string& key) {
436         return store_.erase(key) > 0;
437     }
438
439     // EN: Persist data (in real AP: write to file system)
440     // TR: Verileri kalıcı hale getir (gerçek AP'de dosya sistemine yaz)
441     bool SyncToStorage() {
442         std::cout << "  [ara::per] Syncing '" << storage_id_
443                   << "' (" << store_.size() << " keys) to NV\n";
444         return true;
445     }
```

```

    void PrintContents() const {
        std::cout << "  [ara::per] Storage '" << storage_id_ << "':\n";
        for (const auto& [key, val] : store_) {
            std::cout << "    " << key << " = [";
            for (size_t i = 0; i < val.size(); ++i) {
                if (i > 0) std::cout << ",";
                std::cout << "0x" << std::hex << static_cast<int>(val[i]);
            }
            std::cout << std::dec << "]\n";
        }
    }
};

}  // namespace ara::per

// ═══════════════════════════════════════════════════════════════════════════
// PART 5: ara::log — Logging (DLT Compatible)
// ═══════════════════════════════════════════════════════════════════════════

namespace ara::log {

enum class LogLevel : uint8_t {
    kFatal = 1, kError, kWarn, kInfo, kDebug, kVerbose
};

// EN: Logger — DLT-compatible logging (app_id + context_id)
// TR: Logger — DLT uyumlu loglama (app_id + context_id)
class Logger {
    std::string app_id_;
    std::string ctx_id_;
    LogLevel    threshold_ = LogLevel::kInfo;

public:
    Logger(std::string app, std::string ctx)
        : app_id_(std::move(app)), ctx_id_(std::move(ctx)) {}

    void SetThreshold(LogLevel lvl) { threshold_ = lvl; }

    void Log(LogLevel lvl, std::string_view msg) const {
        if (static_cast<uint8_t>(lvl) > static_cast<uint8_t>(threshold_)) return;
        std::cout << "  [DLT:" << app_id_ << "/" << ctx_id_ << "] "
                  << LevelStr(lvl) << ": " << msg << "\n";
    }

    void Fatal(std::string_view msg) const { Log(LogLevel::kFatal, msg); }
    void Error(std::string_view msg) const { Log(LogLevel::kError, msg); }
    void Warn(std::string_view msg)  const { Log(LogLevel::kWarn, msg); }
    void Info(std::string_view msg)  const { Log(LogLevel::kInfo, msg); }
    void Debug(std::string_view msg) const { Log(LogLevel::kDebug, msg); }

private:
    static std::string_view LevelStr(LogLevel l) {
        switch (l) {
            case LogLevel::kFatal:   return "FATAL";
            case LogLevel::kError:   return "ERROR";
            case LogLevel::kWarn:    return " WARN";
            case LogLevel::kInfo:    return " INFO";
            case LogLevel::kDebug:   return "DEBUG";
            case LogLevel::kVerbose: return " VERB";
        }
        return "?????";
    }
};

}  // namespace ara::log
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════
// PART 6: ara::phm — Platform Health Management
// ═══════════════════════════════════════════════════════════════════════════

namespace ara::phm {

// EN: Supervision modes: Alive, Deadline, Logical
// TR: Denetim modları: Yaşam, Son tarih, Mantıksal

enum class SupervisionResult : uint8_t {
    kCorrect, kFailed, kExpired
};

struct AliveSupervision {
    std::string checkpoint_name;
    uint32_t    min_margin_ms = 0;
    uint32_t    max_margin_ms = 0;
    uint32_t    expected_alive_indications = 0;
    uint32_t    actual_indications = 0;
    uint32_t    failed_count = 0;
};

class HealthManager {
    std::vector<AliveSupervision> supervisions_;

public:
    void AddAliveSupervision(const std::string& name, uint32_t expected,
                             uint32_t min_ms, uint32_t max_ms) {
        supervisions_.push_back({name, min_ms, max_ms, expected, 0, 0});
    }

    void ReportAlive(const std::string& checkpoint) {
        for (auto& s : supervisions_) {
            if (s.checkpoint_name == checkpoint) {
                ++s.actual_indications;
                return;
            }
        }
    }

    // EN: Called periodically to evaluate supervision
    // TR: Denetimi değerlendirmek için periyodik olarak çağrılır
    void Evaluate() {
        for (auto& s : supervisions_) {
            if (s.actual_indications < s.expected_alive_indications) {
                ++s.failed_count;
                std::cout << "  [ara::phm] " << s.checkpoint_name
                          << " FAILED (got " << s.actual_indications
                          << ", expected " << s.expected_alive_indications << ")\n";
            } else {
                std::cout << "  [ara::phm] " << s.checkpoint_name << " OK\n";
            }
            s.actual_indications = 0;  // reset for next cycle
        }
    }

    void PrintStatus() const {
        for (const auto& s : supervisions_) {
            std::cout << "    " << s.checkpoint_name
                      << " — failures=" << s.failed_count
                      << " expected=" << s.expected_alive_indications << "/cycle\n";
        }
    }
};

}  // namespace ara::phm

```

```cpp
// ═══════════════════════════════════════════════════════════════════════
// MAIN — Demos
// ═══════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "╔═══════════════════════════════════════════════════════════╗\n"
              << "║ Module 16: AUTOSAR Adaptive Platform — ara::* Simulation  ║\n"
              << "╚═══════════════════════════════════════════════════════════╝\n\n";

    // --- Demo 1: ara::com — Proxy/Skeleton Pattern ---
    std::cout << "--- Demo 1: ara::com — Proxy/Skeleton (Radar Service) ---\n";
    {
        ara::com::ServiceRegistry::Instance().Clear();

        // EN: Provider side — create skeleton and offer service
        // TR: Sağlayıcı tarafı — skeleton oluştur ve servis sun
        ara::com::RadarServiceSkeleton skeleton(0x1234, 1);
        skeleton.OfferService();

        // EN: Consumer side — find service and create proxy
        // TR: Tüketici tarafı — servisi bul ve proxy oluştur
        auto handles = ara::com::RadarServiceProxy::FindService(0x1234);
        assert(!handles.empty());
        std::cout << "  [Proxy] Found " << handles.size() << " instance(s)\n";

        ara::com::RadarServiceProxy proxy(handles[0]);
        proxy.BindToSkeleton(&skeleton);  // simulation binding

        // --- Method call ---
        auto calib_result = proxy.Calibrate(2);
        if (calib_result.is_ready()) {
            auto cr = calib_result.get();
            std::cout << "  [Proxy] Calibrate result: "
                      << (cr.success ? "OK" : "FAIL") << " — " << cr.message << "\n";
        }

        // --- Event ---
        proxy.SubscribeObjectDetected([](const ara::com::RadarObject& obj) {
            std::cout << "  [Proxy] Object detected: id=" << obj.object_id
                      << " range=" << obj.range_m << "m"
                      << " angle=" << obj.angle_deg << "deg"
                      << " vel=" << obj.velocity_mps << "m/s\n";
        });
        skeleton.SendObjectEvent({25.5f, -3.2f, 12.0f, 42});
        skeleton.SendObjectEvent({8.1f, 15.7f, -5.0f, 43});

        // --- Field ---
        auto rate = proxy.GetUpdateRate();
        std::cout << "  [Proxy] UpdateRate = " << rate.get() << " Hz\n";
        proxy.SetUpdateRate(50);
        auto rate2 = proxy.GetUpdateRate();
        std::cout << "  [Proxy] UpdateRate (after set) = " << rate2.get() << " Hz\n";
    }

    // --- Demo 2: ara::exec — Execution State Management ---
    std::cout << "\n--- Demo 2: ara::exec — Execution Client ---\n";
    {
        ara::exec::ExecutionClient app("RadarApp");
        app.ReportExecutionState(ara::exec::ExecutionState::kRunning);

        ara::exec::DeterministicClient det(20000);  // 20ms cycle
        for (int i = 0; i < 5; ++i) {
            auto cycle = det.WaitForNextActivation();
            std::cout << "  [DeterministicClient] Cycle " << cycle
                      << " (every " << det.GetCycleTimeUs() << " us)\n";
        }

        app.ReportExecutionState(ara::exec::ExecutionState::kTerminating);
        app.ReportExecutionState(ara::exec::ExecutionState::kTerminated);
    }
```

```
    // --- Demo 3: ara::diag — UDS over DoIP ---
    std::cout << "\n--- Demo 3: ara::diag — UDS Diagnostic Server ---\n";
    {
        ara::diag::DiagServer server;

        // EN: Switch to extended session
        // TR: Extended session'a geç
        auto resp = server.HandleRequest({0x10, {0x03}});
        std::cout << "  DiagSession resp SID=0x" << std::hex
                  << static_cast<int>(resp.sid) << std::dec << "\n";

        // EN: Read VIN (DID 0xF190)
        // TR: VIN oku (DID 0xF190)
        resp = server.HandleRequest({0x22, {0xF1, 0x90}});
        std::cout << "  ReadDID(F190) SID=0x" << std::hex
                  << static_cast<int>(resp.sid) << std::dec << " data='";
        for (size_t i = 2; i < resp.data.size(); ++i)
            std::cout << static_cast<char>(resp.data[i]);
        std::cout << "'\n";

        // EN: Security Access — seed request
        // TR: Güvenlik Erişimi — seed isteği
        resp = server.HandleRequest({0x27, {0x01}});
        std::cout << "  SecurityAccess seed: ";
        for (auto b : resp.data)
            std::cout << "0x" << std::hex << static_cast<int>(b) << " ";
        std::cout << std::dec << "\n";

        // EN: Tester Present
        // TR: Test Cihazı Mevcutluğu
        resp = server.HandleRequest({0x3E, {0x00}});
        std::cout << "  TesterPresent resp SID=0x" << std::hex
                  << static_cast<int>(resp.sid) << std::dec << "\n";
    }

    // --- Demo 4: ara::per — Persistency ---
    std::cout << "\n--- Demo 4: ara::per — Key-Value Storage ---\n";
    {
        ara::per::KeyValueStorage store("radar_config");
        store.SetValue("detection_threshold", {0x00, 0x64});  // 100
        store.SetValue("max_range_m", {0x01, 0xF4});          // 500
        store.SetValue("firmware_ver", {'2', '.', '0', '.', '1'});

        auto val = store.GetValue("detection_threshold");
        if (val) {
            std::cout << "  detection_threshold = ";
            for (auto b : *val) std::cout << std::hex << static_cast<int>(b) << " ";
            std::cout << std::dec << "\n";
        }
        store.PrintContents();
        store.SyncToStorage();
    }

    // --- Demo 5: ara::log — DLT Logging ---
    std::cout << "\n--- Demo 5: ara::log — DLT-Compatible Logging ---\n";
    {
        ara::log::Logger logger("RADR", "MAIN");
        logger.SetThreshold(ara::log::LogLevel::kDebug);

        logger.Info("Radar application starting");
        logger.Debug("Initializing antenna array");
        logger.Warn("Signal strength below threshold");
        logger.Error("Lost tracking on object #42");
        logger.Fatal("Hardware fault detected on channel 2");
    }

    // --- Demo 6: ara::phm — Health Management ---
    std::cout << "\n--- Demo 6: ara::phm — Platform Health Management ---\n";
    {
        ara::phm::HealthManager hm;
        hm.AddAliveSupervision("RadarCycle", 3, 0, 25);
        hm.AddAliveSupervision("FusionCycle", 2, 0, 50);

```

```
        // EN: Cycle 1 — radar OK, fusion missed
        // TR: Döngü 1 — radar OK, fusion atlanmış
        hm.ReportAlive("RadarCycle");
        hm.ReportAlive("RadarCycle");
        hm.ReportAlive("RadarCycle");
        hm.ReportAlive("FusionCycle");  // only 1 of 2
        std::cout << "  === Evaluate Cycle 1 ===\n";
        hm.Evaluate();

        // EN: Cycle 2 — both OK
        // TR: Döngü 2 — ikisi de OK
        for (int i = 0; i < 3; ++i) hm.ReportAlive("RadarCycle");
        for (int i = 0; i < 2; ++i) hm.ReportAlive("FusionCycle");
        std::cout << "  === Evaluate Cycle 2 ===\n";
        hm.Evaluate();

        std::cout << "  --- Final Status ---\n";
        hm.PrintStatus();
    }

    // --- Demo 7: Manifest Summary ---
    std::cout << "\n--- Demo 7: Adaptive Platform Manifest Types ---\n";
    {
        std::cout << "  ┌──────────────────────────────┬──────────────────────────────────────┐\n";
        std::cout << "  │ Manifest                     │ Content                              │\n";
        std::cout << "  ├──────────────────────────────┼──────────────────────────────────────┤\n";
        std::cout << "  │ Execution Manifest           │ Processes, startup config, deps      │\n";
        std::cout << "  │ Service Instance Manifest    │ Port bindings (SOME/IP, DDS)         │\n";
        std::cout << "  │ Machine Manifest             │ Network, HW capabilities, services   │\n";
        std::cout << "  │ Diagnostic Extract           │ DID/DTC definitions, sessions        │\n";
        std::cout << "  │ Persistency Deployment       │ Key-value/file storage layout        │\n";
        std::cout << "  └──────────────────────────────┴──────────────────────────────────────┘\n";
    }

    std::cout << "\n[All demos completed successfully.]\n";
    return 0;
}
```

► 03. Functional Safety

module_16_autosar/03_functional_safety.cpp

Konu: Functional Safety — ISO 26262 & ASIL / Fonksiyonel Güvenlik

Teori / Theory

THEORY: ISO 26262 — Functional Safety for Road Vehicles

ISO 26262 is the international standard for functional safety of electrical/electronic systems in road vehicles. It defines:
V-Model (ISO 26262 Parts):

```
┌──────────────────────────────────────────────────────────┐
│ Part 3: Concept Phase     ←──────────→  Part 4: System Level   │
│   Item Definition                        System Design        │
│   HARA (Hazard Analysis)                 Safety Validation    │
│   Safety Goals (ASIL)                                         │
│                                                               │
│ Part 5: HW Level          ←──────────→  Part 6: SW Level       │
│   HW Safety Requirements                 SW Safety Req.       │
│   HW Design                              SW Architecture      │
│   HW Verification                        Unit Design/Test     │
│                                          Integration Test     │
│                                                               │
│ Part 7: Production  │  Part 8: Supporting  │  Part 9: ASIL-D  │
└────────────────────┴──────────────────────┴──────────────────┘
```

TEORİ: ISO 26262 — Karayolu Araçları İçin Fonksiyonel Güvenlik

ISO 26262, karayolu araçlarındaki elektriksel/elektronik sistemler için uluslararası fonksiyonel güvenlik standardıdır.
V-Modeli (ISO 26262 Bölümleri):

```
┌──────────────────────────────────────────────────────────┐
│ Bölüm 3: Konsept Fazı   ←──────────→ Bölüm 4: Sistem Seviyesi  │
│   Öğe Tanımı                          Sistem Tasarımı         │
│   HARA (Tehlike Analizi)              Güvenlik Doğrulama      │
│   Güvenlik Hedefleri (ASIL)                                   │
│                                                               │
│ Bölüm 5: HW Seviyesi   ←──────────→ Bölüm 6: SW Seviyesi       │
│   HW Güvenlik Gereks.                SW Güvenlik Gereks.      │
│   HW Tasarımı                         SW Mimarisi             │
│   HW Doğrulama                        Birim Tasarım/Test      │
│                                       Entegrasyon Testi       │
│                                                               │
│ Bölüm 7: Üretim     │  Bölüm 8: Destek   │  Bölüm 9: ASIL-D   │
└────────────────────┴────────────────────┴────────────────────┘
```

THEORY: ASIL — Automotive Safety Integrity Level

ASIL is determined by HARA (Hazard Analysis & Risk Assessment): Three factors: Severity (S), Exposure (E), Controllability (C)

ASIL	Description
QM	Quality Management — no safety requirements
A	Lowest safety level (e.g., rear light failure)
B	Medium-low (e.g., headlight failure at night)
C	Medium-high (e.g., airbag deployment timing)
D	Highest safety level (e.g., steering, braking)

ASIL Determination Matrix (simplified):

Severity	Exposure	Controllability	ASIL
S3	E4	C3	D
S3	E4	C2	C
S3	E3	C3	C
S2	E4	C3	C
S2	E3	C3	B
S1	E4	C3	B
S1	E3	C3	A
S1	E2	C2	QM

ASIL Decomposition: ASIL-D = ASIL-B(D) + ASIL-B(D) (independent dual path) ASIL-D = ASIL-C(D) + ASIL-A(D) (asymmetric decomposition)
→ Allows simpler implementation per channel if independence proven.

TEORİ: ASIL — Otomotiv Güvenlik Bütünlük Seviyesi

ASIL, HARA (Tehlike Analizi ve Risk Değerlendirmesi) ile belirlenir: Üç faktör: Şiddet (S), Maruziyet (E), Kontrol Edilebilirlik (C)

ASIL	Açıklama
QM	Kalite Yönetimi — güvenlik gereksinimi yok
A	En düşük güvenlik (örn. arka lamba arızası)
B	Orta-düşük (örn. gece far arızası)
C	Orta-yüksek (örn. hava yastığı zamanl.)
D	En yüksek güvenlik (örn. direksiyon, fren)

ASIL Belirleme Matrisi (basitleştirilmiş):

Şiddet	Maruziyet	Kontrol Edileb.	ASIL
S3	E4	C3	D
S3	E4	C2	C
S3	E3	C3	C
S2	E4	C3	C
S2	E3	C3	B
S1	E4	C3	B
S1	E3	C3	A
S1	E2	C2	QM

ASIL Ayrıştırma: ASIL-D = ASIL-B(D) + ASIL-B(D) (bağımsız çift yol) ASIL-D = ASIL-C(D) + ASIL-A(D) (asimetrik ayrıştırma)
→ Bağımsızlık kanıtlanırsa kanal başına daha basit uygulama sağlar.

THEORY: Defensive Programming Patterns

Safety-critical software must employ:
1) Range check: validate all inputs against physical limits
2) Plausibility check: cross-validate redundant sensors
3) Alive counter: detect frozen/stuck signal values
4) CRC/checksum: detect data corruption in communication
5) Program flow monitoring: ensure correct execution sequence
6) Safe state: define and transition to safe state on failure

TEORİ: Savunmacı Programlama Kalıpları

Güvenlik kritik yazılımda savunmacı programlama kalıpları:
1) Aralık kontrolü: tüm girdileri fiziksel sınırlara karşı doğrula
2) Makullük kontrolü: yedekli sensörleri çapraz doğrula
3) Canlılık sayacı: donmuş/takılı sinyal değerlerini tespit et
4) CRC/sağlama: iletişimde veri bozulmasını tespit et
5) Program akış izleme: doğru yürütme sırasını sağla
6) Güvenli durum: arızada güvenli duruma geçişi tanımla

THEORY: Code Coverage Requirements per ASIL

ASIL	Statement	Branch	MC/DC	Method
A	Recommended	–	–	Required
B	Required	Recommended	–	Required
C	Required	Required	Recommended	Required
D	Required	Required	Required	Required

MC/DC = Modified Condition/Decision Coverage
→ Every condition independently affects the decision outcome.

TEORİ: ASIL Seviyesine Göre Kod Kapsama Gereksinimleri

ASIL seviyesine göre kod kapsama gereksinimleri:

ASIL	Deyim	Dal	MC/DC	Metot
A	Önerilen	–	–	Zorunlu
B	Zorunlu	Önerilen	–	Zorunlu
C	Zorunlu	Zorunlu	Önerilen	Zorunlu
D	Zorunlu	Zorunlu	Zorunlu	Zorunlu

MC/DC = Değiştirilmiş Koşul/Karar Kapsaması
→ Her koşul karar sonucunu bağımsız olarak etkiler.

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <array>
#include <vector>
#include <string>
#include <string_view>
#include <functional>
#include <cmath>
#include <iomanip>
#include <algorithm>
#include <numeric>
#include <cassert>
#include <optional>
#include <variant>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: ASIL Determination — HARA Simulation
// ═══════════════════════════════════════════════════════════════════════════

namespace safety {

// EN: Severity classes (S0-S3)
// TR: Şiddet sınıfları (S0-S3)
enum class Severity : uint8_t { S0 = 0, S1 = 1, S2 = 2, S3 = 3 };

// EN: Exposure classes (E0-E4)
// TR: Maruz kalma sınıfları (E0-E4)
enum class Exposure : uint8_t { E0 = 0, E1 = 1, E2 = 2, E3 = 3, E4 = 4 };

// EN: Controllability classes (C0-C3)
// TR: Kontrol edilebilirlik sınıfları (C0-C3)
enum class Controllability : uint8_t { C0 = 0, C1 = 1, C2 = 2, C3 = 3 };

enum class ASIL : uint8_t { QM = 0, A = 1, B = 2, C = 3, D = 4 };

```

```cpp
struct HazardEntry {
    std::string            hazard;
    Severity               severity;
    Exposure               exposure;
    Controllability        controllability;
    ASIL                   determined_asil = ASIL::QM;
};

// EN: ASIL determination lookup (simplified)
// TR: ASIL belirleme tablosu (basitleştirilmiş)
inline ASIL DetermineASIL(Severity s, Exposure e, Controllability c) {
    if (static_cast<uint8_t>(s) == 0) return ASIL::QM;
    if (static_cast<uint8_t>(e) == 0) return ASIL::QM;
    if (static_cast<uint8_t>(c) == 0) return ASIL::QM;

    int score = static_cast<int>(s) + static_cast<int>(e) + static_cast<int>(c);
    if (score >= 10) return ASIL::D;
    if (score >= 8)  return ASIL::C;
    if (score >= 6)  return ASIL::B;
    if (score >= 4)  return ASIL::A;
    return ASIL::QM;
}

inline std::string_view ASILStr(ASIL a) {
    switch (a) {
        case ASIL::QM: return "QM";
        case ASIL::A:  return "ASIL-A";
        case ASIL::B:  return "ASIL-B";
        case ASIL::C:  return "ASIL-C";
        case ASIL::D:  return "ASIL-D";
    }
    return "???";
}

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: Defensive Programming — Range & Plausibility Check
// ═══════════════════════════════════════════════════════════════════════════

// EN: Range check: is value within physical limits?
// TR: Aralık kontrolü: değer fiziksel sınırlar içinde mi?
template <typename T>
struct RangeChecker {
    T min_val;
    T max_val;
    std::string signal_name;

    [[nodiscard]] bool Check(T value) const {
        return value >= min_val && value <= max_val;
    }

    void Report(T value) const {
        if (!Check(value)) {
            std::cout << "  [RANGE_FAIL] " << signal_name << "=" << value
                      << " not in [" << min_val << ", " << max_val << "]\n";
        }
    }
};

// EN: Plausibility check: cross-validate two redundant sensors
// TR: Makullük kontrolü: iki yedekli sensörü çapraz doğrula
struct PlausibilityChecker {
    double max_deviation;
    std::string name;

    [[nodiscard]] bool Check(double sensor_a, double sensor_b) const {
        return std::abs(sensor_a - sensor_b) <= max_deviation;
    }

    void Report(double a, double b) const {
        bool ok = Check(a, b);
        std::cout << "  [PLAUSIBILITY] " << name
                  << " A=" << a << " B=" << b
                  << " diff=" << std::abs(a - b)
                  << " max=" << max_deviation
                  << (ok ? " → OK" : " → FAIL") << "\n";
    }
};
```

```cpp

// EN: Alive counter: detect frozen signal
// TR: Canlılık sayacı: donmuş sinyal tespiti
class AliveCounter {
    uint8_t  expected_counter_ = 0;
    uint32_t fail_count_ = 0;
    uint32_t pass_count_ = 0;
    std::string name_;

public:
    explicit AliveCounter(std::string name) : name_(std::move(name)) {}

    bool Validate(uint8_t received_counter) {
        // EN: Counter should increment by 1 each cycle (mod 16)
        // TR: Sayaç her döngüde 1 artmalı (mod 16)
        bool ok = (received_counter == expected_counter_);
        expected_counter_ = static_cast<uint8_t>((received_counter + 1u) & 0x0Fu);
        if (ok) ++pass_count_; else ++fail_count_;
        return ok;
    }

    [[nodiscard]] uint32_t FailCount() const { return fail_count_; }
    [[nodiscard]] uint32_t PassCount() const { return pass_count_; }

    void Report() const {
        std::cout << "  [ALIVE] " << name_
                  << " pass=" << pass_count_ << " fail=" << fail_count_
                  << (fail_count_ > 0 ? " → DEGRADED" : " → OK") << "\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════
// PART 3: Throttle Position Plausibility (ASIL-D Example)
// ═══════════════════════════════════════════════════════════════════════

// EN: Real ASIL-D scenario: Electronic Throttle Control (ETC)
//     Two independent TPS sensors (TPS1 + TPS2) for redundancy
//     If plausibility fails → limp-home mode (limited torque)
// TR: Gerçek ASIL-D senaryosu: Elektronik Gaz Kontrolü
//     İki bağımsız TPS sensörü (TPS1 + TPS2) yedeklilik için

enum class ThrottleState : uint8_t {
    NORMAL, DEGRADED, LIMP_HOME, SAFE_STOP
};

struct ThrottleReading {
    double tps1_percent;       // 0-100%
    double tps2_percent;       // 0-100% (redundant)
    uint8_t alive_counter;     // 4-bit rolling counter
};

class ThrottlePlausibility {
    RangeChecker<double> range_{0.0, 100.0, "TPS"};
    PlausibilityChecker  plaus_{5.0, "TPS1_vs_TPS2"};
    AliveCounter         alive_{"TPS_Alive"};
    ThrottleState        state_ = ThrottleState::NORMAL;
    uint32_t             consecutive_fails_ = 0;
    static constexpr uint32_t FAIL_THRESHOLD = 3;  // debounce
    static constexpr uint32_t FTTI_MS = 50;  // Fault Tolerant Time Interval

public:
    ThrottleState Evaluate(const ThrottleReading& reading) {
        bool range_ok = range_.Check(reading.tps1_percent) &&
                        range_.Check(reading.tps2_percent);
        bool plaus_ok = plaus_.Check(reading.tps1_percent, reading.tps2_percent);
        bool alive_ok = alive_.Validate(reading.alive_counter);

        if (range_ok && plaus_ok && alive_ok) {
            consecutive_fails_ = 0;
            if (state_ == ThrottleState::DEGRADED)
                state_ = ThrottleState::NORMAL;  // heal
        } else {
            ++consecutive_fails_;
```

```
186             if (consecutive_fails_ >= FAIL_THRESHOLD) {
187                 if (!range_ok || !alive_ok)
188                     state_ = ThrottleState::SAFE_STOP;
189                 else
190                     state_ = ThrottleState::LIMP_HOME;
191             } else {
192                 state_ = ThrottleState::DEGRADED;
193             }
194         }
195         return state_;
196     }
197 
198     [[nodiscard]] double GetSafeThrottle(const ThrottleReading& r) const {
199         switch (state_) {
200             case ThrottleState::NORMAL:
201                 return (r.tps1_percent + r.tps2_percent) / 2.0;
202             case ThrottleState::DEGRADED:
203                 return (r.tps1_percent + r.tps2_percent) / 2.0;
204             case ThrottleState::LIMP_HOME:
205                 return std::min(r.tps1_percent, 20.0);  // max 20%
206             case ThrottleState::SAFE_STOP:
207                 return 0.0;
208         }
209         return 0.0;
210     }
211 
212     static std::string_view StateStr(ThrottleState s) {
213         switch (s) {
214             case ThrottleState::NORMAL:    return "NORMAL";
215             case ThrottleState::DEGRADED:  return "DEGRADED";
216             case ThrottleState::LIMP_HOME: return "LIMP_HOME";
217             case ThrottleState::SAFE_STOP: return "SAFE_STOP";
218         }
219         return "???";
220     }
221 
222     void PrintDiag() const {
223         std::cout << "  [Throttle] State=" << StateStr(state_)
224                   << " ConsecFails=" << consecutive_fails_
225                   << " FTTI=" << FTTI_MS << "ms\n";
226         alive_.Report();
227     }
228 };
229 
230 // ═══════════════════════════════════════════════════════════════════════
231 // PART 4: Redundancy Patterns
232 // ═══════════════════════════════════════════════════════════════════════
233 
234 // EN: Dual-execution with majority voting (TMR — Triple Modular Redundancy)
235 // TR: Çift yürütme ile çoğunluk oylama
236 
237 template <typename T>
238 struct RedundantValue {
239     T channel_a;
240     T channel_b;
241     T channel_c;
242 
243     // EN: 2-out-of-3 majority voting
244     // TR: 3'ten 2'si çoğunluk oylama
245     [[nodiscard]] T Vote() const {
246         if (channel_a == channel_b) return channel_a;
247         if (channel_a == channel_c) return channel_a;
248         if (channel_b == channel_c) return channel_b;
249         return channel_a;  // no majority — use channel A as default
250     }
251 
252     [[nodiscard]] bool IsConsistent() const {
253         return (channel_a == channel_b) || (channel_a == channel_c) ||
254                (channel_b == channel_c);
255     }
256 };
```

```cpp

// EN: Diverse Redundancy — different algorithms for same computation
// TR: Çeşitli Yedeklilik — aynı hesaplama için farklı algoritmalar

struct DiverseChecksum {
    // EN: Method A: simple XOR checksum
    // TR: Yöntem A: basit XOR sağlama
    static uint8_t ComputeXOR(const std::vector<uint8_t>& data) {
        uint8_t cs = 0;
        for (auto b : data) cs ^= b;
        return cs;
    }

    // EN: Method B: arithmetic sum checksum (complemented)
    // TR: Yöntem B: aritmetik toplam sağlama (tümleyen)
    static uint8_t ComputeSum(const std::vector<uint8_t>& data) {
        uint8_t sum = 0;
        for (auto b : data) sum = static_cast<uint8_t>(sum + b);
        return static_cast<uint8_t>(~sum + 1u);  // two's complement
    }

    // EN: Verify using both methods — diverse redundancy
    // TR: Her iki yöntemle doğrula — çeşitli yedeklilik
    static bool Verify(const std::vector<uint8_t>& data,
                       uint8_t expected_xor, uint8_t expected_sum) {
        return (ComputeXOR(data) == expected_xor) &&
               (ComputeSum(data) == expected_sum);
    }
};

// ═══════════════════════════════════════════════════════════════════════
// PART 5: Program Flow Monitoring
// ═══════════════════════════════════════════════════════════════════════

// EN: Ensures correct execution sequence: Init → Read → Process → Output
// TR: Doğru yürütme sırasını garanti eder: Init → Read → Process → Output

class ProgramFlowMonitor {
    using CheckpointID = uint16_t;
    std::vector<CheckpointID> expected_sequence_;
    std::vector<CheckpointID> actual_sequence_;
    uint32_t violation_count_ = 0;

public:
    void DefineSequence(std::initializer_list<CheckpointID> seq) {
        expected_sequence_.assign(seq);
    }

    void ReportCheckpoint(CheckpointID cp) {
        actual_sequence_.push_back(cp);
    }

    bool Evaluate() {
        bool ok = (actual_sequence_ == expected_sequence_);
        if (!ok) {
            ++violation_count_;
            std::cout << "  [FLOW_MON] VIOLATION! Expected: ";
            for (auto c : expected_sequence_) std::cout << c << " ";
            std::cout << "| Actual: ";
            for (auto c : actual_sequence_) std::cout << c << " ";
            std::cout << "\n";
        }
        actual_sequence_.clear();
        return ok;
    }

    [[nodiscard]] uint32_t ViolationCount() const { return violation_count_; }
};

```

```cpp
// ═══════════════════════════════════════════════════════════════════════════
// PART 6: ASIL Decomposition Calculator
// ═══════════════════════════════════════════════════════════════════════════

struct ASILDecomposition {
    ASIL original;
    ASIL channel_a;
    ASIL channel_b;
    bool valid;
};

// EN: Valid ASIL decompositions per ISO 26262-9
// TR: ISO 26262-9'a göre geçerli ASIL ayrıştırmaları
inline std::vector<ASILDecomposition> GetDecompositions(ASIL target) {
    std::vector<ASILDecomposition> results;
    switch (target) {
        case ASIL::D:
            results.push_back({ASIL::D, ASIL::D, ASIL::QM, true});
            results.push_back({ASIL::D, ASIL::C, ASIL::A, true});
            results.push_back({ASIL::D, ASIL::B, ASIL::B, true});
            break;
        case ASIL::C:
            results.push_back({ASIL::C, ASIL::C, ASIL::QM, true});
            results.push_back({ASIL::C, ASIL::B, ASIL::A, true});
            break;
        case ASIL::B:
            results.push_back({ASIL::B, ASIL::B, ASIL::QM, true});
            results.push_back({ASIL::B, ASIL::A, ASIL::A, true});
            break;
        case ASIL::A:
            results.push_back({ASIL::A, ASIL::A, ASIL::QM, true});
            break;
        default:
            break;
    }
    return results;
}

}  // namespace safety

// ═══════════════════════════════════════════════════════════════════════════
// MAIN — Demos
// ═══════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "╔═══════════════════════════════════════════════════════════╗\n"
              << "║ Module 16: Functional Safety — ISO 26262 & ASIL           ║\n"
              << "╚═══════════════════════════════════════════════════════════╝\n\n";

    // --- Demo 1: HARA — ASIL Determination ---
    std::cout << "--- Demo 1: HARA — ASIL Determination ---\n";
    {
        using namespace safety;
        std::vector<HazardEntry> hazards = {
            {"Unintended steering torque",       Severity::S3, Exposure::E4, Controllability::C3, ASIL::QM},
            {"Headlight sudden off (night)",      Severity::S2, Exposure::E3, Controllability::C3, ASIL::QM},
            {"Rear wiper stuck on",               Severity::S0, Exposure::E4, Controllability::C1, ASIL::QM},
            {"Brake force reduction >50%",        Severity::S3, Exposure::E4, Controllability::C2, ASIL::QM},
            {"Airbag early deployment (no crash)", Severity::S3, Exposure::E3, Controllability::C3, ASIL::QM},
            {"Seat heater overheat",              Severity::S1, Exposure::E3, Controllability::C2, ASIL::QM},
        };

        for (auto& h : hazards)
            h.determined_asil = DetermineASIL(h.severity, h.exposure, h.controllability);

        std::cout << "  ┌──────────────────────────────────┬───┬───┬───┬─────────┐\n";
        std::cout << "  │ Hazard                           │ S │ E │ C │ ASIL    │\n";
        std::cout << "  ├──────────────────────────────────┼───┼───┼───┼─────────┤\n";
        for (const auto& h : hazards) {
            std::cout << "  │ " << std::setw(34) << std::left << h.hazard << " │ S"
                      << static_cast<int>(h.severity) << " │ E"
```

```
                        << static_cast<int>(h.exposure) << " │ C"
                        << static_cast<int>(h.controllability) << " │ "
                        << std::setw(6) << ASILStr(h.determined_asil) << " │\n";
        }
        std::cout << std::right;
        std::cout << "  └──────────────────────────────────────┴────┴────┴────┴──────────┘\n";
    }

    // --- Demo 2: Throttle Plausibility (ASIL-D) ---
    std::cout << "\n--- Demo 2: Throttle Plausibility Check (ASIL-D) ---\n";
    {
        safety::ThrottlePlausibility checker;

        struct TestCase { safety::ThrottleReading r; std::string desc; };
        std::vector<TestCase> tests = {
            {{45.0,  44.5, 0}, "Normal — sensors agree"},
            {{45.5,  45.2, 1}, "Normal — sensors agree"},
            {{50.0,  42.0, 2}, "Plausibility fail — >5% diff"},
            {{50.0,  42.0, 3}, "Plausibility fail — >5% diff"},
            {{50.0,  42.0, 4}, "Plausibility fail x3 → LIMP_HOME"},
            {{30.0,  29.8, 5}, "Healed — sensors agree again"},
            {{110.0, 30.0, 6}, "Range fail — TPS1 >100%"},
            {{110.0, 30.0, 7}, "Range fail again "},
            {{110.0, 30.0, 8}, "Range fail x3 → SAFE_STOP"},
        };

        for (auto& tc : tests) {
            auto state = checker.Evaluate(tc.r);
            double safe = checker.GetSafeThrottle(tc.r);
            std::cout << "  [" << std::setw(40) << std::left << tc.desc << "] "
                      << "TPS1=" << std::setw(5) << std::right << tc.r.tps1_percent
                      << " TPS2=" << std::setw(5) << tc.r.tps2_percent
                      << " → " << std::setw(10) << std::left
                      << safety::ThrottlePlausibility::StateStr(state)
                      << " safe_throttle=" << std::right << safe << "%\n";
        }
        std::cout << std::right;
        checker.PrintDiag();
    }

    // --- Demo 3: Range Check ---
    std::cout << "\n--- Demo 3: Range Check (Defensive Programming) ---\n";
    {
        safety::RangeChecker<double> rpm_check{0.0, 8000.0, "RPM"};
        safety::RangeChecker<double> temp_check{-40.0, 150.0, "Coolant_Temp"};
        safety::RangeChecker<int>    volt_check{6, 18, "Battery_Voltage"};

        double test_rpm[] = {3500.0, 8500.0, -100.0, 0.0, 7999.0};
        for (auto v : test_rpm) {
            std::cout << "  RPM=" << std::setw(8) << v << " → "
                      << (rpm_check.Check(v) ? "OK" : "FAIL") << "\n";
        }
        rpm_check.Report(8500.0);
        temp_check.Report(200.0);
        volt_check.Report(5);
    }

    // --- Demo 4: Alive Counter ---
    std::cout << "\n--- Demo 4: Alive Counter (Frozen Signal Detection) ---\n";
    {
        safety::AliveCounter alive("CAN_0x123_Alive");

        // EN: Correct sequence: 0, 1, 2, ..., 15, 0, 1, ...
        // TR: Doğru sıra: 0, 1, 2, ..., 15, 0, 1, ...
        for (uint8_t i = 0; i < 20; ++i) {
            uint8_t counter = static_cast<uint8_t>(i & 0x0Fu);
            alive.Validate(counter);
        }
        std::cout << "  After 20 correct: ";
        alive.Report();
```

```cpp

        // EN: Simulate frozen counter (stays at 5)
        // TR: Donmuş sayaç simülasyonu (5'te kalır)
        for (int i = 0; i < 5; ++i)
            alive.Validate(5);
        std::cout << "  After 5 frozen (value=5): ";
        alive.Report();
    }

    // --- Demo 5: Redundancy — TMR Voting ---
    std::cout << "\n--- Demo 5: Triple Modular Redundancy (TMR Voting) ---\n";
    {
        safety::RedundantValue<int> sensor1{100, 100, 100};
        std::cout << "  Channels [100, 100, 100] → Vote=" << sensor1.Vote()
                  << " consistent=" << sensor1.IsConsistent() << "\n";

        safety::RedundantValue<int> sensor2{100, 100, 105};
        std::cout << "  Channels [100, 100, 105] → Vote=" << sensor2.Vote()
                  << " consistent=" << sensor2.IsConsistent() << "\n";

        safety::RedundantValue<int> sensor3{100, 105, 110};
        std::cout << "  Channels [100, 105, 110] → Vote=" << sensor3.Vote()
                  << " consistent=" << sensor3.IsConsistent() << "\n";
    }

    // --- Demo 6: Diverse Checksum ---
    std::cout << "\n--- Demo 6: Diverse Redundancy (Dual Checksum) ---\n";
    {
        std::vector<uint8_t> data = {0x10, 0x22, 0x33, 0x44, 0x55};
        auto cs_xor = safety::DiverseChecksum::ComputeXOR(data);
        auto cs_sum = safety::DiverseChecksum::ComputeSum(data);
        std::cout << "  Data: [10 22 33 44 55]\n";
        std::cout << "  XOR checksum = 0x" << std::hex << static_cast<int>(cs_xor) << "\n";
        std::cout << "  SUM checksum = 0x" << static_cast<int>(cs_sum) << std::dec << "\n";
        bool ok = safety::DiverseChecksum::Verify(data, cs_xor, cs_sum);
        std::cout << "  Diverse verify: " << (ok ? "PASS" : "FAIL") << "\n";

        // EN: Corrupt one byte and re-verify
        // TR: Bir byte boz ve tekrar doğrula
        data[2] = 0xFF;
        ok = safety::DiverseChecksum::Verify(data, cs_xor, cs_sum);
        std::cout << "  After corruption: " << (ok ? "PASS" : "FAIL") << "\n";
    }

    // --- Demo 7: Program Flow Monitoring ---
    std::cout << "\n--- Demo 7: Program Flow Monitoring ---\n";
    {
        safety::ProgramFlowMonitor pfm;
        pfm.DefineSequence({10, 20, 30, 40});  // Init=10, Read=20, Process=30, Output=40

        // EN: Correct sequence
        // TR: Doğru sıra
        pfm.ReportCheckpoint(10);
        pfm.ReportCheckpoint(20);
        pfm.ReportCheckpoint(30);
        pfm.ReportCheckpoint(40);
        std::cout << "  Correct sequence: " << (pfm.Evaluate() ? "PASS" : "FAIL") << "\n";

        // EN: Wrong sequence (skipped 30)
        // TR: Yanlış sıra (30 atlanmış)
        pfm.ReportCheckpoint(10);
        pfm.ReportCheckpoint(20);
        pfm.ReportCheckpoint(40);  // skipped 30!
        std::cout << "  Wrong sequence:   " << (pfm.Evaluate() ? "PASS" : "FAIL") << "\n";

        std::cout << "  Total violations: " << pfm.ViolationCount() << "\n";
    }

    // --- Demo 8: ASIL Decomposition ---
    std::cout << "\n--- Demo 8: ASIL Decomposition Options ---\n";
    {
        for (auto target : {safety::ASIL::D, safety::ASIL::C, safety::ASIL::B}) {
            auto decomps = safety::GetDecompositions(target);
            std::cout << "  " << safety::ASILStr(target) << " decompositions:\n";
            for (const auto& d : decomps) {
                std::cout << "    " << safety::ASILStr(d.original) << " = "
                          << safety::ASILStr(d.channel_a) << " + "
```

```
                                << safety::ASILStr(d.channel_b) << "\n";
                }
            }
        }

        std::cout << "\n[All demos completed successfully.]\n";
        return 0;
    }
```

► 04. Automotive Cybersecurity

module_16_autosar/04_automotive_cybersecurity.cpp

Konu: Automotive Cybersecurity — SecOC, HSM, ISO 21434 / Otomotiv Siber Güvenlik

Teori / Theory

THEORY: Automotive Attack Vectors

Modern vehicles expose multiple attack surfaces:

```
┌──────────────────────────────────────────────────────┐
│                 ATTACK SURFACE MAP                   │
├────────────────────┼─────────────────────────────────┤
│ Physical Access    │ OBD-II port, CAN bus, debug (JTAG)  │
│ Wireless           │ Wi-Fi, Bluetooth, cellular (4G/5G)  │
│ V2X                │ V2V (DSRC/C-V2X), V2I, V2C (cloud)  │
│ Infotainment       │ USB, CD/DVD, OTA payload            │
│ Sensor Spoofing    │ GPS, radar, LiDAR, camera           │
└────────────────────┴─────────────────────────────────┘
```

Common CAN Attacks:

```
┌────────────────────┬──────────────────────────────────┐
│ Attack             │ Description                      │
├────────────────────┼──────────────────────────────────┤
│ CAN Injection      │ Send forged frames (no auth on CAN)   │
│ Bus-Off Attack     │ Force ECU into error-passive/bus-off  │
│ Replay Attack      │ Record and replay valid CAN frames    │
│ Fuzzing            │ Send random CAN data to find bugs     │
│ DoS (flooding)     │ High-priority frame flood             │
└────────────────────┴──────────────────────────────────┘
```

TEORİ: Otomotiv Saldırı Vektörleri

Modern araçlar birden fazla saldırı yüzeyine sahiptir:

```
┌──────────────────┬──────────────────────────────────┐
│ Fiziksel Erişim  │ OBD-II portu, CAN bus, JTAG debug  │
│ Kablosuz         │ Wi-Fi, Bluetooth, hücresel (4G/5G) │
│ V2X              │ V2V (DSRC/C-V2X), V2I, V2C (bulut) │
│ Infotainment     │ USB, CD/DVD, OTA yükü              │
│ Sensör Sahteleme │ GPS, radar, LiDAR, kamera          │
└──────────────────┴──────────────────────────────────┘
```

Yaygın CAN Saldırıları:

```
┌──────────────────┬──────────────────────────────────┐
│ Saldırı          │ Açıklama                         │
├──────────────────┼──────────────────────────────────┤
│ CAN Injection    │ Sahte frame gönder (CAN'de auth yok)  │
│ Bus-Off Saldırısı │ ECU'yu error-passive/bus-off'a zorla  │
│ Replay Saldırısı │ Geçerli CAN frame'leri kaydet ve tekrarla│
│ Fuzzing          │ Hata bulmak için rastgele CAN verisi  │
│ DoS (flooding)   │ Yüksek öncelikli frame selı           │
└──────────────────┴──────────────────────────────────┘
```

THEORY: SecOC — Secure Onboard Communication (AUTOSAR)

SecOC adds authentication to CAN/FlexRay/Ethernet frames:
- Appends MAC (Message Authentication Code) to PDU
- Uses Freshness Value (counter/timestamp) against replay attacks
- CMAC (Cipher-based MAC) using AES-128 is typical

SecOC PDU format:

```
┌──────────────────┬──────────────────┬──────────────────┐
│ Authentic PDU Data │ Freshness Value  │ MAC (truncated)  │
│ (original payload) │ (counter/time)   │ (e.g. 28 bits)   │
└──────────────────┴──────────────────┴──────────────────┘
```

SecOC TX Flow: PDU → get FreshnessValue → compute MAC → append → send SecOC RX Flow: receive → extract MAC → recompute → compare → accept/reject

TEORİ: SecOC — Güvenli Araç İçi İletişim (AUTOSAR)

SecOC, CAN/FlexRay/Ethernet frame'lerine kimlik doğrulama ekler:
- PDU'ya MAC (Message Authentication Code) ekler
- Replay saldırılarına karşı Freshness Value (sayaç/zaman damgası) kullanır
- CMAC (AES-128 tabanlı şifre MAC) tipiktir

SecOC PDU formatı:

```
┌──────────────────┬──────────────────┬──────────────────┐
│ Otantik PDU Verisi │ Freshness Value  │ MAC (kesik)      │
│ (orijinal veri)    │ (sayaç/zaman)    │ (örn. 28 bit)    │
└──────────────────┴──────────────────┴──────────────────┘
```

SecOC TX Akışı: PDU → FreshnessValue al → MAC hesapla → ekle → gönder SecOC RX Akışı: al → MAC çıkar → yeniden hesapla → karşılaştır → kabul/ret

THEORY: HSM — Hardware Security Module

HSM provides hardware-level security for ECUs:
- Secure key storage (cannot be read out)
- Hardware crypto acceleration (AES, SHA, RSA, ECDSA)
- Secure boot verification
- Random Number Generation (TRNG) Examples: Infineon AURIX HSM, NXP SHE/CSE, Renesas ICU-S

TEORİ: HSM — Donanım Güvenlik Modülü

HSM, ECU'lar için donanım seviyesinde güvenlik sağlar:
- Güvenli anahtar depolama (dışarı okunamaz)
- Donanım kripto hızlandırma (AES, SHA, RSA, ECDSA)
- Güvenli önyükleme doğrulaması
- Gerçek Rastgele Sayı Üretici (TRNG) Örnekler: Infineon AURIX HSM, NXP SHE/CSE, Renesas ICU-S

THEORY: UN R155

Since July 2024, UN R155 mandates a Cybersecurity Management System (CSMS) for all new vehicle type approvals. ISO/SAE 21434 provides the engineering standard covering the vehicle lifecycle.

ISO 21434 — Cybersecurity Management / TEORİ: UN R155 / ISO 21434 — Siber Güvenlik Yönetimi

Temmuz 2024'ten itibaren UN R155, tüm yeni araç tip onayları için Siber Güvenlik Yönetim Sistemi (CSMS) zorunlu kılmaktadır.

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <array>
#include <vector>
#include <string>
#include <string_view>
#include <iomanip>
#include <algorithm>
#include <numeric>
#include <cassert>
#include <functional>
#include <map>
#include <random>
#include <optional>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: AES-128 CMAC Simulation (Simplified)
// ═══════════════════════════════════════════════════════════════════════════

namespace crypto {

// EN: Simplified AES-128 CMAC for educational purposes.
//     Real implementations use hardware AES (HSM) or crypto libraries.
// TR: Eğitim amaçlı basitleştirilmiş AES-128 CMAC.
//     Gerçek implementasyonlar HSM veya kripto kütüphaneleri kullanır.

using Key128 = std::array<uint8_t, 16>;
using Block  = std::array<uint8_t, 16>;

```

```cpp
// EN: Simplified block cipher (XOR-based, NOT cryptographically secure)
//     Used only to demonstrate the SecOC MAC computation flow
// TR: Basitleştirilmiş blok şifre (sadece SecOC akışını göstermek için)
class SimplifiedAES {
    Key128 key_;

public:
    explicit SimplifiedAES(const Key128& key) : key_(key) {}

    Block Encrypt(const Block& plaintext) const {
        Block cipher{};
        for (size_t i = 0; i < 16; ++i) {
            // EN: Multi-round XOR substitution (educational, NOT secure)
            // TR: Çok turlu XOR ikamesi (eğitim amaçlı, güvenli DEĞİL)
            uint8_t val = plaintext[i] ^ key_[i];
            val = static_cast<uint8_t>((val << 3) | (val >> 5));  // rotate
            val ^= key_[(i + 7) % 16];
            cipher[i] = val;
        }
        return cipher;
    }
};

// EN: CMAC computation (simplified version of NIST SP 800-38B)
// TR: CMAC hesaplama (NIST SP 800-38B basitleştirilmiş versiyonu)
class CMAC {
    SimplifiedAES aes_;
    Block         subkey1_{};
    Block         subkey2_{};

    static void LeftShift(Block& block) {
        uint8_t carry = 0;
        for (int i = 15; i >= 0; --i) {
            uint8_t new_carry = static_cast<uint8_t>((block[static_cast<size_t>(i)] >> 7) & 1u);
            block[static_cast<size_t>(i)] = static_cast<uint8_t>(
                (block[static_cast<size_t>(i)] << 1) | carry);
            carry = new_carry;
        }
    }

    void GenerateSubkeys() {
        Block zero{};
        zero.fill(0);
        Block l = aes_.Encrypt(zero);
        subkey1_ = l;
        LeftShift(subkey1_);
        if ((l[0] & 0x80u) != 0) subkey1_[15] ^= 0x87u;  // Rb
        subkey2_ = subkey1_;
        LeftShift(subkey2_);
        if ((subkey1_[0] & 0x80u) != 0) subkey2_[15] ^= 0x87u;
    }

public:
    explicit CMAC(const Key128& key) : aes_(key) { GenerateSubkeys(); }

    // EN: Compute CMAC over arbitrary-length message
    // TR: Rastgele uzunluktaki mesaj üzerinde CMAC hesapla
    Block Compute(const std::vector<uint8_t>& message) {
        size_t n_blocks = (message.empty()) ? 1 : (message.size() + 15) / 16;
        bool complete_block = (!message.empty()) && (message.size() % 16 == 0);

        Block x{};
        x.fill(0);

        for (size_t i = 0; i < n_blocks - 1; ++i) {
            for (size_t j = 0; j < 16; ++j)
                x[j] ^= message[i * 16 + j];
            x = aes_.Encrypt(x);
        }

        // EN: Last block - apply subkey
        // TR: Son blok - alt anahtar uygula
        Block last_block{};
        last_block.fill(0);
        size_t last_start = (n_blocks - 1) * 16;
        size_t remaining = message.size() - last_start;
        for (size_t j = 0; j < remaining; ++j)
```

```
            last_block[j] = message[last_start + j];

        if (complete_block) {
            for (size_t j = 0; j < 16; ++j)
                last_block[j] ^= subkey1_[j];
        } else {
            last_block[remaining] = 0x80u;  // padding
            for (size_t j = 0; j < 16; ++j)
                last_block[j] ^= subkey2_[j];
        }

        for (size_t j = 0; j < 16; ++j)
            x[j] ^= last_block[j];
        return aes_.Encrypt(x);
    }

    // EN: Truncate MAC to specified number of bits
    // TR: MAC'i belirtilen bit sayısına kırp
    static std::vector<uint8_t> Truncate(const Block& mac, uint32_t bits) {
        uint32_t bytes = (bits + 7) / 8;
        std::vector<uint8_t> result(mac.begin(),
            mac.begin() + static_cast<ptrdiff_t>(std::min(static_cast<size_t>(bytes), mac.size())));
        if (bits % 8 != 0) {
            uint8_t mask = static_cast<uint8_t>(0xFF << (8 - (bits % 8)));
            result.back() &= mask;
        }
        return result;
    }
};

}  // namespace crypto

// ============================================================================
// PART 2: SecOC — Secure Onboard Communication
// ============================================================================

namespace secoc {

// EN: Freshness Value Manager — counter-based
// TR: Tazelik Değeri Yöneticisi — sayaç tabanlı
class FreshnessManager {
    std::map<uint16_t, uint64_t> counters_;  // PDU_ID → counter

public:
    uint64_t GetTxFreshness(uint16_t pdu_id) {
        return counters_[pdu_id]++;
    }

    bool VerifyRxFreshness(uint16_t pdu_id, uint64_t received_fv) {
        auto& expected = counters_[pdu_id];
        // EN: Accept if received >= expected (allows for small gaps)
        // TR: Alınan >= beklenen ise kabul et
        if (received_fv >= expected) {
            expected = received_fv + 1;
            return true;
        }
        return false;  // replay or stale
    }
};

struct SecOC_PDU {
    uint16_t               pdu_id;
    std::vector<uint8_t>   payload;       // authentic data
    uint64_t               freshness_value;
    std::vector<uint8_t>   mac;           // truncated MAC
    bool                   authenticated = false;
};

// EN: SecOC module — handles TX authentication and RX verification
// TR: SecOC modülü — TX kimlik doğrulama ve RX doğrulama
class SecOCModule {
    crypto::CMAC      cmac_;
    FreshnessManager  fm_;
    uint32_t          mac_bits_;
    uint32_t          tx_count_ = 0;
    uint32_t          rx_pass_ = 0;
    uint32_t          rx_fail_ = 0;
```

```cpp

    std::vector<uint8_t> BuildAuthData(const std::vector<uint8_t>& payload,
                                       uint64_t fv) {
        std::vector<uint8_t> data = payload;
        // EN: Append freshness value (big-endian, 8 bytes)
        // TR: Tazelik değerini ekle (big-endian, 8 byte)
        for (int i = 7; i >= 0; --i)
            data.push_back(static_cast<uint8_t>((fv >> (i * 8)) & 0xFF));
        return data;
    }

public:
    SecOCModule(const crypto::Key128& key, uint32_t mac_bits = 28)
        : cmac_(key), mac_bits_(mac_bits) {}

    // EN: Authenticate PDU for transmission
    // TR: İletim için PDU'yu kimlik doğrulama
    SecOC_PDU AuthenticateTx(uint16_t pdu_id, const std::vector<uint8_t>& payload) {
        SecOC_PDU pdu{};
        pdu.pdu_id = pdu_id;
        pdu.payload = payload;
        pdu.freshness_value = fm_.GetTxFreshness(pdu_id);

        auto auth_data = BuildAuthData(payload, pdu.freshness_value);
        auto full_mac = cmac_.Compute(auth_data);
        pdu.mac = crypto::CMAC::Truncate(full_mac, mac_bits_);
        pdu.authenticated = true;
        ++tx_count_;
        return pdu;
    }

    // EN: Verify received PDU
    // TR: Alınan PDU'yu doğrula
    bool VerifyRx(const SecOC_PDU& pdu) {
        // EN: Step 1: Check freshness
        // TR: Adım 1: Tazelik kontrolü
        if (!fm_.VerifyRxFreshness(pdu.pdu_id, pdu.freshness_value)) {
            ++rx_fail_;
            return false;
        }

        // EN: Step 2: Recompute MAC and compare
        // TR: Adım 2: MAC'i yeniden hesapla ve karşılaştır
        auto auth_data = BuildAuthData(pdu.payload, pdu.freshness_value);
        auto full_mac = cmac_.Compute(auth_data);
        auto expected_mac = crypto::CMAC::Truncate(full_mac, mac_bits_);

        if (expected_mac != pdu.mac) {
            ++rx_fail_;
            return false;
        }

        ++rx_pass_;
        return true;
    }

    void PrintStats() const {
        std::cout << "  [SecOC] TX=" << tx_count_
                  << " RX_PASS=" << rx_pass_
                  << " RX_FAIL=" << rx_fail_ << "\n";
    }
};

}  // namespace secoc
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: HSM Simulation — Key Management
// ═══════════════════════════════════════════════════════════════════════════

namespace hsm {

// EN: Simulated Hardware Security Module
// TR: Simüle edilmiş Donanım Güvenlik Modülü

enum class KeySlot : uint8_t {
    MASTER_ECU_KEY = 0,
    SECOC_KEY_1    = 1,
    SECOC_KEY_2    = 2,
    SECURE_BOOT_KEY= 3,
    OTA_VERIFY_KEY = 4,
    MAX_SLOTS      = 8
};

class HSM {
    std::array<crypto::Key128, 8> key_store_{};
    std::array<bool, 8>           key_valid_{};
    bool                          locked_ = false;
    uint32_t                      crypto_ops_ = 0;

public:
    HSM() { key_valid_.fill(false); }

    // EN: Store key in HSM (only before lock)
    // TR: HSM'e anahtar depola (sadece kilitlenmeden önce)
    bool StoreKey(KeySlot slot, const crypto::Key128& key) {
        if (locked_) {
            std::cout << "  [HSM] ERROR: Cannot store key — HSM is locked\n";
            return false;
        }
        auto idx = static_cast<size_t>(slot);
        if (idx >= key_store_.size()) return false;
        key_store_[idx] = key;
        key_valid_[idx] = true;
        std::cout << "  [HSM] Key stored in slot " << idx << "\n";
        return true;
    }

    // EN: Lock HSM — no more key provisioning
    // TR: HSM'i kilitle — artık anahtar yüklenemez
    void Lock() {
        locked_ = true;
        std::cout << "  [HSM] Locked — key provisioning disabled\n";
    }

    // EN: Compute MAC using key from slot (key never leaves HSM)
    // TR: Slot'taki anahtarı kullanarak MAC hesapla (anahtar HSM'den çıkmaz)
    std::optional<crypto::Block> ComputeMAC(KeySlot slot,
                                            const std::vector<uint8_t>& data) {
        auto idx = static_cast<size_t>(slot);
        if (idx >= key_store_.size() || !key_valid_[idx]) return std::nullopt;
        crypto::CMAC cmac(key_store_[idx]);
        ++crypto_ops_;
        return cmac.Compute(data);
    }

    // EN: Generate random number (TRNG)
    // TR: Rastgele sayı üret (TRNG)
    std::array<uint8_t, 16> GenerateRandom() {
        std::random_device rd;
        std::mt19937 gen(rd());
        std::uniform_int_distribution<uint16_t> dist(0, 255);
        std::array<uint8_t, 16> random{};
        for (auto& b : random) b = static_cast<uint8_t>(dist(gen));
        ++crypto_ops_;
        return random;
    }
```

```
    void PrintStatus() const {
        std::cout << "  [HSM] Locked=" << locked_
                  << " CryptoOps=" << crypto_ops_ << "\n";
        for (size_t i = 0; i < key_valid_.size(); ++i) {
            if (key_valid_[i])
                std::cout << "    Slot " << i << ": KEY_PRESENT\n";
        }
    }
};

}  // namespace hsm

// ═══════════════════════════════════════════════════════════════════════════
// PART 4: Secure Boot Chain
// ═══════════════════════════════════════════════════════════════════════════

namespace secure_boot {

// EN: Simple hash for educational secure boot simulation
// TR: Eğitim amaçlı güvenli başlatma simülasyonu için basit hash

uint32_t SimpleHash(const std::vector<uint8_t>& data) {
    uint32_t hash = 0x811C9DC5u;  // FNV-1a offset basis
    for (auto b : data) {
        hash ^= b;
        hash *= 0x01000193u;      // FNV-1a prime
    }
    return hash;
}

struct BootStage {
    std::string        name;
    std::vector<uint8_t> image;
    uint32_t           expected_hash;
    bool               verified = false;
};

class SecureBootChain {
    std::vector<BootStage> stages_;

public:
    void AddStage(const std::string& name, const std::vector<uint8_t>& image) {
        uint32_t hash = SimpleHash(image);
        stages_.push_back({name, image, hash, false});
    }

    bool ExecuteBootChain() {
        std::cout << "  Secure Boot Chain:\n";
        for (auto& stage : stages_) {
            uint32_t computed = SimpleHash(stage.image);
            stage.verified = (computed == stage.expected_hash);
            std::cout << "    " << stage.name << " — hash=0x"
                      << std::hex << computed << " expected=0x"
                      << stage.expected_hash << std::dec
                      << (stage.verified ? " → PASS" : " → FAIL") << "\n";
            if (!stage.verified) {
                std::cout << "    BOOT HALTED at " << stage.name << "!\n";
                return false;
            }
        }
        std::cout << "    All stages verified → Boot complete\n";
        return true;
    }

    // EN: Simulate tampered image
    // TR: Değiştirilmiş image simülasyonu
    void TamperStage(size_t index) {
        if (index < stages_.size() && !stages_[index].image.empty()) {
            stages_[index].image[0] ^= 0xFF;  // flip bits
        }
    }
};

}  // namespace secure_boot
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════
// PART 5: X.509 Certificate Simplified & TLS Handshake Overview
// ═══════════════════════════════════════════════════════════════════════════

namespace pki {

struct Certificate {
    std::string subject;
    std::string issuer;
    uint32_t    serial_number;
    std::string valid_from;
    std::string valid_to;
    std::string public_key_algo;
    uint32_t    fingerprint;  // simplified
    bool        is_ca = false;
};

class CertificateChain {
    std::vector<Certificate> chain_;

public:
    void AddCert(const Certificate& cert) { chain_.push_back(cert); }

    bool Verify() const {
        if (chain_.empty()) return false;
        std::cout << "  Certificate Chain Verification:\n";
        for (size_t i = 0; i < chain_.size(); ++i) {
            const auto& cert = chain_[i];
            bool issuer_ok = (i + 1 < chain_.size()) ?
                (cert.issuer == chain_[i + 1].subject) : cert.is_ca;
            std::cout << "    [" << i << "] Subject: " << cert.subject
                      << " | Issuer: " << cert.issuer
                      << " | " << (issuer_ok ? "OK" : "BROKEN") << "\n";
            if (!issuer_ok) return false;
        }
        return true;
    }
};

}  // namespace pki

// ═══════════════════════════════════════════════════════════════════════════
// PART 6: CAN Intrusion Detection System (IDS)
// ═══════════════════════════════════════════════════════════════════════════

namespace ids {

struct CANFrame {
    uint32_t  id;
    uint8_t   dlc;
    std::array<uint8_t, 8> data;
    uint64_t  timestamp_us;
};

// EN: Simple IDS — detects frequency anomalies and unknown IDs
// TR: Basit IDS — frekans anomalileri ve bilinmeyen ID'leri tespit eder
class AutomotiveIDS {
    std::map<uint32_t, uint32_t> known_ids_;    // id → expected period_us
    std::map<uint32_t, uint64_t> last_seen_;     // id → last timestamp
    std::map<uint32_t, uint32_t> violation_cnt_; // id → violation count
    uint32_t unknown_id_count_ = 0;
    static constexpr double FREQ_TOLERANCE = 0.5;  // 50% tolerance

public:
    void RegisterExpectedID(uint32_t id, uint32_t period_us) {
        known_ids_[id] = period_us;
    }

    enum class Verdict { OK, FREQUENCY_ANOMALY, UNKNOWN_ID };

    Verdict Analyze(const CANFrame& frame) {
        auto it = known_ids_.find(frame.id);
        if (it == known_ids_.end()) {
            ++unknown_id_count_;
            return Verdict::UNKNOWN_ID;
        }
```

```
        auto last_it = last_seen_.find(frame.id);
        if (last_it != last_seen_.end()) {
            uint64_t delta = frame.timestamp_us - last_it->second;
            double expected = static_cast<double>(it->second);
            double ratio = static_cast<double>(delta) / expected;
            if (ratio < (1.0 - FREQ_TOLERANCE) || ratio > (1.0 + FREQ_TOLERANCE)) {
                violation_cnt_[frame.id]++;
                last_seen_[frame.id] = frame.timestamp_us;
                return Verdict::FREQUENCY_ANOMALY;
            }
        }
        last_seen_[frame.id] = frame.timestamp_us;
        return Verdict::OK;
    }

    void PrintReport() const {
        std::cout << "  [IDS] Unknown IDs detected: " << unknown_id_count_ << "\n";
        for (const auto& [id, cnt] : violation_cnt_) {
            std::cout << "  [IDS] ID 0x" << std::hex << id << std::dec
                      << " frequency violations: " << cnt << "\n";
        }
    }
};

}  // namespace ids

// ═══════════════════════════════════════════════════════════════════════════
// MAIN — Demos
// ═══════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "╔═══════════════════════════════════════════════════════╗\n"
              << "║ Module 16: Automotive Cybersecurity — SecOC, HSM, IDS ║\n"
              << "╚═══════════════════════════════════════════════════════╝\n\n";

    // --- Demo 1: SecOC — Authenticate and Verify ---
    std::cout << "--- Demo 1: SecOC — Secure Onboard Communication ---\n";
    {
        crypto::Key128 key = {0x2B, 0x7E, 0x15, 0x16, 0x28, 0xAE, 0xD2, 0xA6,
                              0xAB, 0xF7, 0x15, 0x88, 0x09, 0xCF, 0x4F, 0x3C};
        secoc::SecOCModule tx_secoc(key);
        secoc::SecOCModule rx_secoc(key);

        // EN: TX side authenticates, RX side verifies
        // TR: TX tarafı kimlik doğrular, RX tarafı doğrular
        auto pdu1 = tx_secoc.AuthenticateTx(0x100, {0x11, 0x22, 0x33, 0x44});
        auto pdu2 = tx_secoc.AuthenticateTx(0x100, {0xAA, 0xBB, 0xCC, 0xDD});

        std::cout << "  PDU1 MAC: ";
        for (auto b : pdu1.mac) std::cout << std::hex << static_cast<int>(b) << " ";
        std::cout << std::dec << " FV=" << pdu1.freshness_value << "\n";

        bool v1 = rx_secoc.VerifyRx(pdu1);
        bool v2 = rx_secoc.VerifyRx(pdu2);
        std::cout << "  PDU1 verify: " << (v1 ? "PASS" : "FAIL") << "\n";
        std::cout << "  PDU2 verify: " << (v2 ? "PASS" : "FAIL") << "\n";

        // EN: Replay attack — send PDU1 again (stale freshness)
        // TR: Tekrar saldırısı — PDU1'i tekrar gönder (eski tazelik)
        bool v3 = rx_secoc.VerifyRx(pdu1);
        std::cout << "  PDU1 replay: " << (v3 ? "PASS" : "FAIL (replay blocked)") << "\n";

        // EN: Tampered payload
        // TR: Değiştirilmiş veri
        auto pdu_tampered = tx_secoc.AuthenticateTx(0x100, {0xFF, 0xEE, 0xDD, 0xCC});
        pdu_tampered.payload[0] = 0x00;  // tamper
        bool v4 = rx_secoc.VerifyRx(pdu_tampered);
        std::cout << "  Tampered PDU: " << (v4 ? "PASS" : "FAIL (tamper detected)") << "\n";

        tx_secoc.PrintStats();
        rx_secoc.PrintStats();
    }
```

```cpp

    // --- Demo 2: HSM — Key Management ---
    std::cout << "\n--- Demo 2: HSM — Hardware Security Module ---\n";
    {
        hsm::HSM module;
        crypto::Key128 secoc_key = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08,
                                    0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F, 0x10};
        module.StoreKey(hsm::KeySlot::SECOC_KEY_1, secoc_key);
        module.StoreKey(hsm::KeySlot::SECURE_BOOT_KEY, {0xAA, 0xBB, 0xCC, 0xDD,
            0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77, 0x88, 0x99, 0xAA, 0xBB, 0xCC});
        module.Lock();

        // EN: Try to store key after lock — should fail
        // TR: Kilitlenmeden sonra anahtar depolamayı dene — başarısız olmalı
        module.StoreKey(hsm::KeySlot::SECOC_KEY_2, secoc_key);

        // EN: Compute MAC using HSM (key stays inside)
        // TR: HSM kullanarak MAC hesapla (anahtar içeride kalır)
        auto mac = module.ComputeMAC(hsm::KeySlot::SECOC_KEY_1,
                                     {0x10, 0x20, 0x30, 0x40});
        if (mac) {
            std::cout << "  MAC computed: ";
            for (size_t i = 0; i < 4; ++i)
                std::cout << std::hex << static_cast<int>((*mac)[i]) << " ";
            std::cout << "...\n" << std::dec;
        }

        auto rng = module.GenerateRandom();
        std::cout << "  TRNG: ";
        for (size_t i = 0; i < 4; ++i)
            std::cout << std::hex << static_cast<int>(rng[i]) << " ";
        std::cout << "...\n" << std::dec;

        module.PrintStatus();
    }

    // --- Demo 3: Secure Boot Chain ---
    std::cout << "\n--- Demo 3: Secure Boot Chain ---\n";
    {
        secure_boot::SecureBootChain chain;
        chain.AddStage("ROM_Bootloader",  {0x10, 0x20, 0x30, 0x40, 0x50});
        chain.AddStage("1st_Stage_Boot",  {0xAA, 0xBB, 0xCC, 0xDD, 0xEE, 0xFF});
        chain.AddStage("2nd_Stage_Boot",  {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07});
        chain.AddStage("Application_FW",  {0x55, 0x66, 0x77, 0x88, 0x99});

        std::cout << "  === Normal Boot ===\n";
        chain.ExecuteBootChain();

        std::cout << "\n  === Tampered Boot (stage 2) ===\n";
        secure_boot::SecureBootChain chain2;
        chain2.AddStage("ROM_Bootloader", {0x10, 0x20, 0x30, 0x40, 0x50});
        chain2.AddStage("1st_Stage_Boot", {0xAA, 0xBB, 0xCC, 0xDD, 0xEE, 0xFF});
        chain2.AddStage("2nd_Stage_TAMPERED", {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07});
        chain2.TamperStage(2);
        chain2.ExecuteBootChain();
    }

    // --- Demo 4: Certificate Chain ---
    std::cout << "\n--- Demo 4: X.509 Certificate Chain (Simplified) ---\n";
    {
        pki::CertificateChain chain;
        chain.AddCert({"ECU_RadarFrontLeft", "OEM_SubCA", 1001, "2025-01-01",
                       "2030-01-01", "ECDSA-P256", 0xABCD, false});
        chain.AddCert({"OEM_SubCA", "OEM_RootCA", 500, "2024-01-01",
                       "2034-01-01", "ECDSA-P384", 0x1234, true});
        chain.AddCert({"OEM_RootCA", "OEM_RootCA", 1, "2020-01-01",
                       "2040-01-01", "RSA-4096", 0x0001, true});
        bool valid = chain.Verify();
        std::cout << "  Chain verification: " << (valid ? "VALID" : "INVALID") << "\n";
    }
```

```

    // --- Demo 5: CAN Intrusion Detection ---
    std::cout << "\n--- Demo 5: CAN Intrusion Detection System (IDS) ---\n";
    {
        ids::AutomotiveIDS detector;
        detector.RegisterExpectedID(0x100, 10000);   // 10ms period
        detector.RegisterExpectedID(0x200, 20000);   // 20ms period
        detector.RegisterExpectedID(0x300, 100000);  // 100ms period

        // EN: Normal traffic
        // TR: Normal trafik
        ids::CANFrame frames[] = {
            {0x100, 8, {}, 0},
            {0x100, 8, {}, 10000},
            {0x100, 8, {}, 20000},
            {0x200, 8, {}, 5000},
            {0x200, 8, {}, 25000},
            // EN: Anomaly: 0x100 too fast (injection attempt)
            // TR: Anomali: 0x100 çok hızlı (enjeksiyon denemesi)
            {0x100, 8, {}, 21000},  // 1ms after last → too fast
            // EN: Unknown ID (possible injection)
            // TR: Bilinmeyen ID (olası enjeksiyon)
            {0x666, 8, {}, 22000},
            {0x100, 8, {}, 31000},  // normal again
        };

        const char* verdict_str[] = {"OK", "FREQ_ANOMALY", "UNKNOWN_ID"};
        for (const auto& f : frames) {
            auto v = detector.Analyze(f);
            std::cout << "  ID=0x" << std::hex << f.id << std::dec
                      << " @" << f.timestamp_us << "us → "
                      << verdict_str[static_cast<int>(v)] << "\n";
        }
        detector.PrintReport();
    }

    // --- Demo 6: Attack Vector Summary ---
    std::cout << "\n--- Demo 6: Cybersecurity Standards Summary ---\n";
    {
        std::cout << "  ┌──────────────────────┬──────────────────────────────────────┐\n";
        std::cout << "  │ Standard             │ Scope                                │\n";
        std::cout << "  ├──────────────────────┼──────────────────────────────────────┤\n";
        std::cout << "  │ UN R155              │ CSMS mandatory for type approval (2024) │\n";
        std::cout << "  │ ISO/SAE 21434        │ Cybersecurity engineering lifecycle  │\n";
        std::cout << "  │ ISO 11452 / CISPR 25 │ EMC, but related to signal integrity │\n";
        std::cout << "  │ AUTOSAR SecOC        │ Onboard CAN/FR/ETH message auth      │\n";
        std::cout << "  │ AUTOSAR IdsM         │ Intrusion Detection System Manager   │\n";
        std::cout << "  │ IEEE 1609.2          │ V2X security (PKI, certificates)     │\n";
        std::cout << "  │ Uptane (TUF)         │ Secure OTA framework for automotive  │\n";
        std::cout << "  └──────────────────────┴──────────────────────────────────────┘\n";
    }

    std::cout << "\n[All demos completed successfully.]\n";
    return 0;
}
```

► 05. Misra Cert Coding Standard

module_16_autosar/05_misra_cert_coding_standard.cpp

Konu: MISRA C++ 2023 & CERT C++ — Automotive Coding Standards

Teori / Theory

THEORY: MISRA C++ 2023 — Overview

MISRA C++ 2023 is the automotive industry coding standard for C++17. It merges and replaces:
- MISRA C++ 2008 (C++03-based)
- AUTOSAR C++ Coding Guidelines 2019 (C++14-based)

Rule Categories:

Category	Description
Mandatory	Must be followed — no deviations allowed
Required	Must be followed — deviations need justification
Advisory	Should be followed — best practice

Key MISRA C++ 2023 Restrictions:

Feature	MISRA C++ 2023 Stance
Exceptions (throw/catch)	FORBIDDEN (no dynamic path)
RTTI (dynamic_cast, typeid)	FORBIDDEN (runtime overhead)
Dynamic allocation (new/delete)	RESTRICTED (init phase only)
goto	FORBIDDEN
C-style casts	FORBIDDEN
Implicit narrowing conversions	FORBIDDEN
Unions with non-trivial members	FORBIDDEN
setjmp/longjmp	FORBIDDEN
constexpr / consteval	ENCOURAGED
auto type deduction	ALLOWED (with care)
Range-based for	ENCOURAGED
Smart pointers (unique_ptr)	ALLOWED
std::optional, std::variant	ALLOWED
Templates / CRTP	ALLOWED
Lambda expressions	ALLOWED (stateless preferred)
Structured bindings	ALLOWED
if constexpr	ENCOURAGED

TEORİ: Genel Bakış

MISRA C++ 2023, C++17 için otomotiv endüstrisi kodlama standardıdır. Önceki MISRA C++ 2008 ve AUTOSAR C++ 2019'u birleştirir ve günceller.
Kural Kategorileri:

Kategori	Açıklama
Mandatory	Mutlaka uyulmalı — sapma izni yok
Required	Mutlaka uyulmalı — sapma gerekçelendirilmeli
Advisory	Uyulması önerilir — en iyi uygulama

Temel MISRA C++ 2023 Kısıtlamaları:

Özellik	MISRA C++ 2023 Tutumu
Exceptions (throw/catch)	YASAK (dinamik yol yok)
RTTI (dynamic_cast, typeid)	YASAK (çalışma zamanı yükü)
Dinamik allokasyon (new/delete)	KISITLI (yalnızca init'te)
goto	YASAK
C-stili dönüşümler	YASAK
Örtük daraltma dönüşümleri	YASAK
Trivial olmayan union üyeleri	YASAK
setjmp/longjmp	YASAK
constexpr / consteval	TEŞVİK EDİLİR
auto tip çıkarımı	İZİNLİ (dikkatli kullanım)
Range-based for	TEŞVİK EDİLİR
Akıllı işaretçiler (unique_ptr)	İZİNLİ
std::optional, std::variant	İZİNLİ
Templates / CRTP	İZİNLİ
Lambda ifadeleri	İZİNLİ (durumsuz tercih)
Structured bindings	İZİNLİ
if constexpr	TEŞVİK EDİLİR

THEORY: CERT C++ Secure Coding Standard

CERT C++ focuses on security vulnerabilities:
- Buffer overflow prevention (bounds checking)
- Integer overflow/underflow detection
- Resource management (RAII, no leaks)
- Null pointer dereference prevention
- Thread safety (data race prevention)

TEORİ: CERT C++ Güvenli Kodlama Standardı

CERT C++, güvenlik açıklarına odaklanır:
- Buffer overflow önleme (sınır kontrolü)
- Integer overflow/underflow tespit
- Kaynak yönetimi (RAII, sızıntı yok)
- Null pointer dereference önleme
- Thread güvenliği (veri yarışı önleme)

THEORY: Static Analysis Tools

Tool	Key Features
cppcheck	Open-source, MISRA addon available
clang-tidy	LLVM-based, CERT/MISRA checks
Parasoft C++test	Commercial, MISRA/CERT/AUTOSAR
Polyspace (MathWorks)	Proves absence of runtime errors
QA·C++ (Perforce)	MISRA C++ reference checker
Coverity (Synopsys)	Large-scale defect detection
PVS-Studio	Static analysis, MISRA support

TEORİ: Statik Analiz Araçları

Statik analiz araçları, kodu çalıştırmadan hataları tespit eder:

Araç	Temel Özellikler
cppcheck	Açık kaynak, MISRA eklentisi mevcut
clang-tidy	LLVM tabanlı, CERT/MISRA kontrolleri
Parasoft C++test	Ticari, MISRA/CERT/AUTOSAR desteği
Polyspace (MathWorks)	Çalışma zamanı hatasız olduğunu kanıtlar
QA·C++ (Perforce)	MISRA C++ referans denetleyicisi
Coverity (Synopsys)	Büyük ölçekli hata tespiti
PVS-Studio	Statik analiz, MISRA desteği

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <array>
#include <vector>
#include <string>
#include <string_view>
#include <iomanip>
#include <algorithm>
#include <cassert>
#include <optional>
#include <variant>
#include <memory>
#include <limits>
#include <type_traits>
#include <numeric>

// ============================================================================
// PART 1: MISRA C++ 2023 — Compliant vs Non-Compliant Examples
// ============================================================================

namespace misra_examples {

// ----------------------------------------------------------------------------
// Rule: "No implicit narrowing conversions" (Required)
// ----------------------------------------------------------------------------

// EN: NON-COMPLIANT — implicit narrowing from int to uint8_t
// TR: UYUMSUZ — int'ten uint8_t'ye örtülü daraltma
/*
    void bad_narrowing() {
        int value = 300;
        uint8_t byte = value;  // MISRA VIOLATION: narrowing conversion
    }
*/
```

```cpp

// EN: COMPLIANT — explicit cast with range check
// TR: UYUMLU — aralık kontrolü ile açık dönüşüm
inline std::optional<uint8_t> safe_narrow(int value) {
    if (value < 0 || value > 255) return std::nullopt;
    return static_cast<uint8_t>(value);
}

// ---------------------------------------------------------------------------
// Rule: "No C-style casts" (Required)
// ---------------------------------------------------------------------------

// EN: NON-COMPLIANT: int x = (int)ptr;
// EN: COMPLIANT: use static_cast, const_cast, reinterpret_cast explicitly
// TR: UYUMSUZ: C-tarzı cast kullanımı | UYUMLU: static_cast/const_cast/reinterpret_cast kullan

// ---------------------------------------------------------------------------
// Rule: "No exceptions (throw/catch)" (Required in safety code)
// ---------------------------------------------------------------------------

// EN: Error handling WITHOUT exceptions — return codes + std::optional
// TR: İstisnasız hata yönetimi — dönüş kodları + std::optional

enum class ErrorCode : uint8_t {
    OK = 0, INVALID_PARAM, OUT_OF_RANGE, TIMEOUT, HW_FAULT
};

template <typename T>
struct Result {
    T          value;
    ErrorCode  error;

    [[nodiscard]] bool IsOk() const { return error == ErrorCode::OK; }
    static Result Ok(T val) { return {std::move(val), ErrorCode::OK}; }
    static Result Fail(ErrorCode e) { return {T{}, e}; }
};

// EN: MISRA-compliant sensor read (no exceptions)
// TR: MISRA uyumlu sensör okuma (istisna yok)
inline Result<uint16_t> ReadSensorValue(uint8_t channel) {
    if (channel > 7)
        return Result<uint16_t>::Fail(ErrorCode::INVALID_PARAM);
    // EN: Simulate ADC read
    // TR: ADC okuma simülasyonu
    uint16_t raw = static_cast<uint16_t>(1024u + channel * 100u);
    if (raw > 4095)
        return Result<uint16_t>::Fail(ErrorCode::OUT_OF_RANGE);
    return Result<uint16_t>::Ok(raw);
}

// ---------------------------------------------------------------------------
// Rule: "All switch cases shall have break or return" (Required)
// ---------------------------------------------------------------------------

enum class GearPosition : uint8_t { PARK, REVERSE, NEUTRAL, DRIVE };

inline std::string_view GearToString(GearPosition gear) {
    switch (gear) {
        case GearPosition::PARK:    return "P";
        case GearPosition::REVERSE: return "R";
        case GearPosition::NEUTRAL: return "N";
        case GearPosition::DRIVE:   return "D";
        // EN: MISRA: ALL enum values must be handled; default optional for enums
        // TR: MISRA: TÜM enum değerleri ele alınmalı
    }
    return "?";  // unreachable if all enums covered — TR: tüm enum değerleri kapsandıysa ulaşılamaz
}

```

```cpp
// ---------------------------------------------------------------------------
// Rule: "Use constexpr for compile-time constants" (Advisory)
// ---------------------------------------------------------------------------

// EN: NON-COMPLIANT: #define MAX_RPM 8000
// EN: COMPLIANT: constexpr
// TR: UYUMSUZ: #define makro sabiti | UYUMLU: constexpr ile derleme zamanı sabiti
constexpr uint16_t MAX_RPM = 8000;
constexpr double   THROTTLE_MIN = 0.0;
constexpr double   THROTTLE_MAX = 100.0;

// EN: constexpr function — computed at compile time
// TR: constexpr fonksiyon — derleme zamanında hesapla
constexpr uint32_t CRC32_POLYNOMIAL = 0xEDB88320u;

constexpr std::array<uint32_t, 256> GenerateCRC32Table() {
    std::array<uint32_t, 256> table{};
    for (uint32_t i = 0; i < 256; ++i) {
        uint32_t crc = i;
        for (int j = 0; j < 8; ++j) {
            if ((crc & 1u) != 0)
                crc = (crc >> 1) ^ CRC32_POLYNOMIAL;
            else
                crc >>= 1;
        }
        table[i] = crc;
    }
    return table;
}

constexpr auto CRC32_TABLE = GenerateCRC32Table();

inline uint32_t ComputeCRC32(const uint8_t* data, size_t length) {
    uint32_t crc = 0xFFFFFFFFu;
    for (size_t i = 0; i < length; ++i) {
        uint8_t index = static_cast<uint8_t>((crc ^ data[i]) & 0xFFu);
        crc = (crc >> 8) ^ CRC32_TABLE[index];
    }
    return crc ^ 0xFFFFFFFFu;
}

// ---------------------------------------------------------------------------
// Rule: "No dynamic_cast or typeid" (Required for safety)
// ---------------------------------------------------------------------------

// EN: Use CRTP instead of virtual + dynamic_cast
// TR: virtual + dynamic_cast yerine CRTP kullan

template <typename Derived>
class SensorBase {
public:
    uint16_t Read() {
        return static_cast<Derived*>(this)->ReadImpl();
    }
    std::string_view Name() {
        return static_cast<Derived*>(this)->NameImpl();
    }
};

class TemperatureSensor : public SensorBase<TemperatureSensor> {
public:
    uint16_t ReadImpl() { return 2048; }  // ~25°C
    static std::string_view NameImpl() { return "TempSensor"; }
};

class PressureSensor : public SensorBase<PressureSensor> {
public:
    uint16_t ReadImpl() { return 3500; }  // ~101.3 kPa
    static std::string_view NameImpl() { return "PressSensor"; }
};

}  // namespace misra_examples
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: CERT C++ — Secure Coding Patterns
// ═══════════════════════════════════════════════════════════════════════════

namespace cert_examples {

// ---------------------------------------------------------------------------
// INT32-C: Ensure integer operations don't overflow
// ---------------------------------------------------------------------------

// EN: Safe integer arithmetic with overflow detection
// TR: Taşma tespitli güvenli tamsayı aritmetiği
inline std::optional<int32_t> SafeAdd(int32_t a, int32_t b) {
    if (b > 0 && a > std::numeric_limits<int32_t>::max() - b) return std::nullopt;
    if (b < 0 && a < std::numeric_limits<int32_t>::min() - b) return std::nullopt;
    return a + b;
}

inline std::optional<int32_t> SafeMultiply(int32_t a, int32_t b) {
    if (a == 0 || b == 0) return 0;
    if (a > 0 && b > 0 && a > std::numeric_limits<int32_t>::max() / b) return std::nullopt;
    if (a > 0 && b < 0 && b < std::numeric_limits<int32_t>::min() / a) return std::nullopt;
    if (a < 0 && b > 0 && a < std::numeric_limits<int32_t>::min() / b) return std::nullopt;
    if (a < 0 && b < 0 && a < std::numeric_limits<int32_t>::max() / b) return std::nullopt;
    return a * b;
}

// ---------------------------------------------------------------------------
// ARR30-C: Do not form or use out-of-bounds references
// ---------------------------------------------------------------------------

// EN: Bounds-checked array access
// TR: Sınır kontrollü dizi erişimi
template <typename T, size_t N>
class SafeArray {
    std::array<T, N> data_{};

public:
    std::optional<T> At(size_t index) const {
        if (index >= N) return std::nullopt;
        return data_[index];
    }

    bool Set(size_t index, T value) {
        if (index >= N) return false;
        data_[index] = std::move(value);
        return true;
    }

    [[nodiscard]] constexpr size_t Size() const { return N; }
};

// ---------------------------------------------------------------------------
// MEM50-CPP: Use RAII for resource management
// ---------------------------------------------------------------------------

// EN: RAII-based peripheral lock (no manual unlock needed)
// TR: RAII tabanlı çevre birimi kilidi (manuel kilit açma gerekmez)
class PeripheralLock {
    uint32_t peripheral_id_;
    bool     locked_ = false;

    static void HW_Lock(uint32_t id) {
        std::cout << "    [HW] Peripheral 0x" << std::hex << id
                  << std::dec << " LOCKED\n";
    }
    static void HW_Unlock(uint32_t id) {
        std::cout << "    [HW] Peripheral 0x" << std::hex << id
                  << std::dec << " UNLOCKED\n";
    }
```

```cpp

public:
    explicit PeripheralLock(uint32_t id) : peripheral_id_(id), locked_(true) {
        HW_Lock(peripheral_id_);
    }
    ~PeripheralLock() {
        if (locked_) HW_Unlock(peripheral_id_);
    }
    PeripheralLock(const PeripheralLock&) = delete;
    PeripheralLock& operator=(const PeripheralLock&) = delete;
    PeripheralLock(PeripheralLock&& other) noexcept
        : peripheral_id_(other.peripheral_id_), locked_(other.locked_) {
        other.locked_ = false;
    }
    PeripheralLock& operator=(PeripheralLock&&) = delete;
};

// ---------------------------------------------------------------------------
// EXP34-C: Do not dereference null pointers
// ---------------------------------------------------------------------------

// EN: Null-safe pointer wrapper
// TR: Null-güvenli pointer sarmalayıcı
template <typename T>
class NonNull {
    T* ptr_;
public:
    explicit NonNull(T* p) : ptr_(p) { assert(p != nullptr); }
    T& operator*() const { return *ptr_; }
    T* operator->() const { return ptr_; }
    [[nodiscard]] T* Get() const { return ptr_; }
};

}  // namespace cert_examples

// ===========================================================================
// PART 3: Compliant vs Non-Compliant Code Comparison
// ===========================================================================

namespace comparison {

// EN: Example: Motor speed controller — showing compliant vs non-compliant
// TR: Örnek: Motor hız denetleyicisi — uyumlu ve uyumsuz karşılaştırma

// --- NON-COMPLIANT version (shown as comments for reference) ---
/*
class BadMotorController {
    int speed;              // MISRA: missing initializer
    int* target;            // MISRA: raw pointer, no RAII
public:
    void setSpeed(int s) {  // MISRA: no range check
        speed = s;          // MISRA: possible overflow
        *target = s;        // CERT: possible null dereference
    }
    int getSpeed() {        // MISRA: should be const, [[nodiscard]]
        return speed;
    }
};
*/

// --- COMPLIANT version ---
class MotorController {
    uint16_t speed_rpm_ = 0;
    static constexpr uint16_t MAX_SPEED = 6000;

public:
    [[nodiscard]] misra_examples::ErrorCode SetSpeed(uint16_t target_rpm) {
        if (target_rpm > MAX_SPEED)
            return misra_examples::ErrorCode::OUT_OF_RANGE;
        speed_rpm_ = target_rpm;
        return misra_examples::ErrorCode::OK;
    }

    [[nodiscard]] uint16_t GetSpeed() const { return speed_rpm_; }

    [[nodiscard]] static constexpr uint16_t GetMaxSpeed() { return MAX_SPEED; }
};
```

```cpp

// EN: Type-safe register access (MISRA compliant)
// TR: Tür-güvenli register erişimi (MISRA uyumlu)
template <uint32_t Address, uint8_t Offset, uint8_t Width>
struct RegisterField {
    static constexpr uint32_t MASK = ((1u << Width) - 1u) << Offset;

    static uint32_t Read(const uint32_t reg_value) {
        return (reg_value & MASK) >> Offset;
    }

    static uint32_t Write(uint32_t reg_value, uint32_t field_value) {
        reg_value &= ~MASK;
        reg_value |= (field_value << Offset) & MASK;
        return reg_value;
    }
};

}  // namespace comparison

// ═══════════════════════════════════════════════════════════════════════════
// PART 4: MISRA Critical Rules Reference
// ═══════════════════════════════════════════════════════════════════════════

namespace misra_rules {

struct MISRARule {
    std::string rule_id;
    std::string category;   // Mandatory/Required/Advisory — TR: Zorunlu/Gerekli/Tavsiye
    std::string description;
    std::string rationale;
};

inline std::vector<MISRARule> GetCriticalRules() {
    return {
        {"Rule 0.1.1", "Required",  "All code shall be reachable",
         "Dead code indicates logic error"},
        {"Rule 0.1.2", "Required",  "All values shall be used",
         "Unused values indicate logic error"},
        {"Rule 4.1.3", "Required",  "No undefined/unspecified behavior",
         "Deterministic behavior required"},
        {"Rule 6.0.1", "Mandatory", "No implicit narrowing conversions",
         "Data loss prevention"},
        {"Rule 6.2.1", "Required",  "Signed/unsigned mixed arithmetic forbidden",
         "Unexpected results prevention"},
        {"Rule 6.4.1", "Required",  "No C-style casts",
         "Type safety enforcement"},
        {"Rule 6.8.1", "Required",  "No goto statements",
         "Structured control flow"},
        {"Rule 7.0.1", "Required",  "No dynamic memory after init",
         "Deterministic memory usage"},
        {"Rule 8.2.5", "Required",  "Functions shall have [[nodiscard]] if return matters",
         "Prevent ignoring error codes"},
        {"Rule 11.6.1","Required",  "No #define for constants (use constexpr)",
         "Type safety enforcement"},
        {"Rule 13.3.1","Required",  "No throw/catch in safety code",
         "Deterministic control flow"},
        {"Rule 14.1.1","Required",  "No RTTI (dynamic_cast/typeid)",
         "No runtime overhead"},
        {"Rule 15.0.1","Required",  "All switch cases shall break/return",
         "Prevent fallthrough bugs"},
        {"Rule 18.3.1","Required",  "No setjmp/longjmp",
         "No non-local jumps"},
        {"Rule 21.6.1","Required",  "No C standard I/O (printf/scanf)",
         "Type-unsafe functions"},
    };
}

}  // namespace misra_rules

```

```cpp
// ═══════════════════════════════════════════════════════════════════════
// MAIN — Demos
// ═══════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "╔═══════════════════════════════════════════════════════════╗\n"
              << "║ Module 16: MISRA C++ 2023 & CERT C++ Coding Standards     ║\n"
              << "╚═══════════════════════════════════════════════════════════╝\n\n";

    // --- Demo 1: Safe Narrowing Conversion ---
    std::cout << "--- Demo 1: Safe Narrowing Conversion (MISRA Rule 6.0.1) ---\n";
    {
        int test_values[] = {100, 255, 256, -1, 0};
        for (int v : test_values) {
            auto result = misra_examples::safe_narrow(v);
            std::cout << "  safe_narrow(" << std::setw(4) << v << ") → "
                      << (result ? std::to_string(*result) : "NONE (overflow)") << "\n";
        }
    }

    // --- Demo 2: Error Handling Without Exceptions ---
    std::cout << "\n--- Demo 2: Error Handling Without Exceptions (MISRA Rule 13.3.1) ---\n";
    {
        for (uint8_t ch = 0; ch < 10; ++ch) {
            auto result = misra_examples::ReadSensorValue(ch);
            std::cout << "  Channel " << static_cast<int>(ch) << ": "
                      << (result.IsOk() ? "value=" + std::to_string(result.value)
                                        : "ERROR(" + std::to_string(static_cast<int>(result.error)) + ")")
                      << "\n";
        }
    }

    // --- Demo 3: constexpr CRC32 Table ---
    std::cout << "\n--- Demo 3: constexpr CRC32 (MISRA Rule 11.6.1) ---\n";
    {
        std::cout << "  CRC32 table (first 8 entries, compile-time generated):\n  ";
        for (int i = 0; i < 8; ++i)
            std::cout << "0x" << std::hex << misra_examples::CRC32_TABLE[static_cast<size_t>(i)] << " ";
        std::cout << std::dec << "\n";

        std::array<uint8_t, 5> data = {0x48, 0x65, 0x6C, 0x6C, 0x6F};  // "Hello"
        auto crc = misra_examples::ComputeCRC32(data.data(), data.size());
        std::cout << "  CRC32(\"Hello\") = 0x" << std::hex << crc << std::dec << "\n";
    }

    // --- Demo 4: CRTP Instead of dynamic_cast ---
    std::cout << "\n--- Demo 4: CRTP Sensors — No RTTI (MISRA Rule 14.1.1) ---\n";
    {
        misra_examples::TemperatureSensor temp;
        misra_examples::PressureSensor    press;
        std::cout << "  " << temp.Name() << " = " << temp.Read() << "\n";
        std::cout << "  " << press.Name() << " = " << press.Read() << "\n";
    }

    // --- Demo 5: CERT — Safe Integer Arithmetic ---
    std::cout << "\n--- Demo 5: CERT INT32-C — Safe Integer Arithmetic ---\n";
    {
        auto r1 = cert_examples::SafeAdd(std::numeric_limits<int32_t>::max(), 1);
        std::cout << "  MAX_INT + 1 = " << (r1 ? std::to_string(*r1) : "OVERFLOW") << "\n";

        auto r2 = cert_examples::SafeAdd(100, 200);
        std::cout << "  100 + 200 = " << (r2 ? std::to_string(*r2) : "OVERFLOW") << "\n";

        auto r3 = cert_examples::SafeMultiply(100000, 100000);
        std::cout << "  100000 * 100000 = " << (r3 ? std::to_string(*r3) : "OVERFLOW") << "\n";

        auto r4 = cert_examples::SafeMultiply(1000, 2000);
        std::cout << "  1000 * 2000 = " << (r4 ? std::to_string(*r4) : "OVERFLOW") << "\n";
    }
```

```cpp

    // --- Demo 6: CERT — Bounds-Checked Array ---
    std::cout << "\n--- Demo 6: CERT ARR30-C — Bounds-Checked Array ---\n";
    {
        cert_examples::SafeArray<int, 5> arr;
        arr.Set(0, 10);
        arr.Set(4, 50);
        bool oob = arr.Set(5, 60);  // out of bounds
        std::cout << "  arr[0] = " << (arr.At(0) ? std::to_string(*arr.At(0)) : "NONE") << "\n";
        std::cout << "  arr[4] = " << (arr.At(4) ? std::to_string(*arr.At(4)) : "NONE") << "\n";
        std::cout << "  arr[5] = " << (arr.At(5) ? "???" : "NONE (out of bounds)") << "\n";
        std::cout << "  Set(5) = " << (oob ? "OK" : "REJECTED") << "\n";
    }

    // --- Demo 7: CERT — RAII Resource Management ---
    std::cout << "\n--- Demo 7: CERT MEM50-CPP — RAII Peripheral Lock ---\n";
    {
        std::cout << "  Entering scope:\n";
        {
            cert_examples::PeripheralLock lock(0x40020000);
            std::cout << "    ... doing SPI transfer ...\n";
            // lock auto-released at scope exit
        }
        std::cout << "  Scope exited — lock released automatically\n";
    }

    // --- Demo 8: Motor Controller (Compliant Example) ---
    std::cout << "\n--- Demo 8: MISRA-Compliant Motor Controller ---\n";
    {
        comparison::MotorController motor;
        auto err1 = motor.SetSpeed(3000);
        std::cout << "  SetSpeed(3000) → "
                  << (err1 == misra_examples::ErrorCode::OK ? "OK" : "ERROR")
                  << "  speed=" << motor.GetSpeed() << "\n";

        auto err2 = motor.SetSpeed(7000);
        std::cout << "  SetSpeed(7000) → "
                  << (err2 == misra_examples::ErrorCode::OK ? "OK" : "OUT_OF_RANGE")
                  << "  speed=" << motor.GetSpeed() << "\n";

        std::cout << "  Max speed (constexpr) = "
                  << comparison::MotorController::GetMaxSpeed() << "\n";
    }

    // --- Demo 9: Type-Safe Register Field ---
    std::cout << "\n--- Demo 9: Type-Safe Register Access ---\n";
    {
        using ModeField = comparison::RegisterField<0x40020000, 0, 4>;
        using SpeedField = comparison::RegisterField<0x40020000, 4, 2>;

        uint32_t reg = 0x00000000;
        reg = ModeField::Write(reg, 0x05);
        reg = SpeedField::Write(reg, 0x02);
        std::cout << "  Register = 0x" << std::hex << reg << std::dec << "\n";
        std::cout << "  Mode  field = " << ModeField::Read(reg) << "\n";
        std::cout << "  Speed field = " << SpeedField::Read(reg) << "\n";
    }

    // --- Demo 10: MISRA Critical Rules Reference Table ---
    std::cout << "\n--- Demo 10: MISRA C++ 2023 Critical Rules ---\n";
```

```
    {
        auto rules = misra_rules::GetCriticalRules();
        std::cout << "  ┌──────────────┬───────────┬──────────────
```

Modül 17: SDV — Yazılım Tanımlı Araç

Yazılım Tanımlı Araç

3 kaynak dosya · Standart: C++20

C++20

► 01. Sdv Architecture

module_17_sdv/01_sdv_architecture.cpp

Konu: Software-Defined Vehicle (SDV) Architecture & C++ Role

Teori / Theory

THEORY: SDV — Software-Defined Vehicle

An SDV treats the vehicle as a software platform where features are defined, updated, and enhanced through software — not hardware changes.
Traditional ECU Architecture SDV Architecture

```
┌──────────────────────┐          ┌──────────────────────────┐
│  ECU1 (Engine)       │          │  HPC (Central Computer)  │
│  ECU2 (Brakes/ABS)   │          │  ┌────────────────────┐  │
│  ECU3 (ADAS)         │          │  │ ADAS  │ Cockpit    │  │
│  ECU4 (Body/BCM)     │   →→→    │  │ Body  │ Powertrain │  │
│  ECU5 (Infotainment) │          │  └────────────────────┘  │
│  ECU6 (Gateway)      │          │  + Zone Controllers (ZC) │
│  ... (70-150 ECUs)   │          │    ZC1  ZC2  ZC3  ZC4    │
└──────────────────────┘          └──────────────────────────┘
```

TEORİ: Yazılım Tanımlı Araç

SDV, aracı bir yazılım platformu olarak ele alır. Özellikler donanım değişikliği olmadan yazılım ile tanımlanır ve güncellenir.
Geleneksel ECU Mimarisi: SDV Mimarisi:

```
┌──────────────────────┐      ┌──────────────────────────┐
│ ECU1 (Motor)         │      │ HPC (Merkezi Bilgisayar) │
│ ECU2 (Fren/ABS)      │      │ ┌────────────────────┐   │
│ ECU3 (ADAS)          │ →→→  │ │ ADAS│Kokpit│Gövde   │   │
│ ECU4 (Gövde/BCM)     │      │ └────────────────────┘   │
│ ECU5 (Bilgi-Eğlence) │      │ + Bölge Kontrolörleri    │
│ ... (70-150 ECU)     │      │   ZC1  ZC2  ZC3  ZC4     │
└──────────────────────┘      └──────────────────────────┘
```

THEORY: Domain → Zonal Architecture Transition

DOMAIN-based (traditional):

```
┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐
│Powertrain│ │  Chassis │ │   Body   │ │   ADAS   │
│ Domain   │ │  Domain  │ │  Domain  │ │  Domain  │
│Controller│ │Controller│ │Controller│ │Controller│
└────┬─────┘ └────┬─────┘ └────┬─────┘ └────┬─────┘
     │            │            │            │
─────┴────────────┴────────────┴────────────┴──── CAN/LIN
```

ZONAL (SDV):

```
         ┌──────────────────┐
         │   HPC / Vehicle  │
         │   Computer       │
         └──┬────┬────┬────┬┘
           │    │    │    │  Ethernet backbone
┌──────────┴┐  ┌┴────┴┐  ┌┴┐ ┌┴──────────┐
│Zone Front │  │Zone  │ │ZR │Zone Rear  │
│Left  (ZFL)│  │FR    │ │L  │Right (ZRR)│
└────┬──────┘  └────┬─┘  └┬┘ └────┬──────┘
Sensors Actuators ... Sensors Actuators
```

TEORİ: Alan → Bölgesel Mimari Geçişi

Domain (alan) tabanlı mimari: her fonksiyon grubunun ayrı kontrolörü vardır (Powertrain, Şasi, Gövde, ADAS) — CAN/LIN ile bağlanır. Zonal mimari: araç fiziksel bölgelere ayrılır (ön-sol, arka-sağ vb.), her bölgede bir Zone Controller bulunur. Tüm mantık merkezi HPC'de çalışır, ZC'ler sensör/aktüatör I/O birimi olarak görev yapar. Avantajlar: kablo ağırlığı ↓, yazılım güncellenebilirliği ↑, maliyet ↓.

THEORY: SOA — Service-Oriented Architecture in Automotive

```
┌──────────────────────────────────────────────────┐
│                 Service Consumer                 │
│  Dashboard App    Navigation    Autonomous Driving  │
└────────┬──────────────┬──────────────┬──────────┘
         │              │              │
┌────────┴──────────────┴──────────────┴──────────┐
│            Service Middleware (SOA)              │
│           SOME/IP  |  DDS  |  iceoryx            │
└────────┬──────────────┬──────────────┬──────────┘
         │              │              │
┌────────┴─────┐   ┌────┴─────┐   ┌────┴─────┐
│VehicleSpeed│   │  Radar   │   │  Camera  │
│  Service   │   │  Service │   │  Service │
└──────────────┘   └──────────┘   └──────────┘
```

TEORİ: SOA — Otomotivde Servis Odaklı Mimari

SOA (Servis Yönelimli Mimari), araç içi yazılımı bağımsız servislere böler. Servisler SOME/IP, DDS veya iceoryx middleware üzerinden birbirini keşfeder ve iletişim kurar. Tüketiciler (Dashboard, ADAS) servis sağlayıcılardan (VehicleSpeed, Radar) veri alır. Avantaj: gevşek bağlama, bağımsız güncelleme, yeniden kullanılabilirlik.

THEORY: Vehicle Signal Specification (VSS)

COVESA VSS is a standardized tree-based vehicle data model: Vehicle.Speed → km/h (uint16) Vehicle.Chassis.SteeringWheel.Angle → degrees (float) Vehicle.Powertrain.FuelSystem.Level → percent (uint8) Vehicle.ADAS.AEB.IsEnabled → bool

TEORİ: Araç Sinyal Spesifikasyonu (VSS)

COVESA VSS, araç sinyallerini standart ağaç yapısında tanımlar. Örnekler: Vehicle.Speed (km/h), Vehicle.Chassis.SteeringWheel.Angle, Vehicle.Powertrain.FuelSystem.Level, Vehicle.ADAS.AEB.IsEnabled. Tüm OEM'ler ortak sinyal isimlendirmesi kullanarak birlikte çalışabilirlik sağlar.

THEORY: C++ Role in SDV

```
┌──────────────────────────────────────────────────┐
│ Layer              │ Language    │ Why             │
├────────────────────┼─────────────┼─────────────────┤
│ ML Training        │ Python      │ Framework ecosystem  │
│ ML Inference       │ C++ / CUDA  │ Real-time latency    │
│ Sensor Fusion      │ C++         │ Deterministic timing │
│ Middleware         │ C++         │ Performance + SOA    │
│ AUTOSAR Adaptive   │ C++14/17    │ ara::* APIs          │
│ Real-time Services │ C++         │ RTOS integration     │
│ HMI / Qt           │ C++ / QML   │ GPU rendering        │
│ Android Automotive │ Java/Kotlin │ App framework        │
│ Safety-critical new│ Rust        │ Memory safety        │
│ Driver/HAL         │ C           │ Kernel requirement   │
└────────────────────┴─────────────┴─────────────────┘
```

TEORİ: SDV'de C++'ın Rolü

SDV'de C++ rolleri:

```
┌────────────────────┬─────────────┬────────────────────┐
│ Katman             │ Dil         │ Neden              │
├────────────────────┼─────────────┼────────────────────┤
│ ML Çıkarım         │ C++ / CUDA  │ Gerçek zamanlı gecikmesi│
│ Sensör Füzyonu     │ C++         │ Deterministik zamanlama │
│ Middleware         │ C++         │ Performans + SOA        │
│ AUTOSAR Adaptive   │ C++14/17    │ ara::* API'leri         │
│ Gerçek zamanlı serv.│ C++         │ RTOS entegrasyonu       │
│ HMI / Qt           │ C++ / QML   │ GPU render              │
│ Sürücü/HAL         │ C           │ Çekirdek gereksinimi    │
└────────────────────┴─────────────┴────────────────────┘
```

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <string>
#include <string_view>
#include <vector>
#include <map>
#include <unordered_map>
#include <functional>
#include <memory>
#include <optional>
#include <variant>
#include <iomanip>
#include <algorithm>
```

```cpp
#include <chrono>
#include <array>
#include <queue>
#include <sstream>
#include <cassert>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: SDV Architecture Model — Domain vs Zonal
// ═══════════════════════════════════════════════════════════════════════════

namespace sdv_arch {

// EN: ECU types in traditional vs SDV architecture
// TR: Geleneksel ve SDV mimarisindeki ECU türleri
enum class ECUType : uint8_t {
    TRADITIONAL_DOMAIN, HPC, ZONE_CONTROLLER
};

struct ECUNode {
    std::string         name;
    ECUType             type;
    std::vector<std::string> functions;
    uint32_t            compute_gflops;  // processing power — TR: işlem gücü (GFLOPS)
};

// EN: Demonstrate domain→zonal consolidation
// TR: Domain→zonal konsolidasyonunu göster
struct VehicleArchitecture {
    std::string               arch_name;
    std::vector<ECUNode>      nodes;
    uint32_t                  total_ecus;
    std::string               backbone;

    void Print() const {
        std::cout << "  Architecture: " << arch_name << "\n"
                  << "  Backbone: " << backbone << "\n"
                  << "  Total nodes: " << total_ecus << "\n";
        for (const auto& n : nodes) {
            std::cout << "    [" << n.name << "] ("
                      << n.compute_gflops << " GFLOPS) — ";
            for (size_t i = 0; i < n.functions.size(); ++i) {
                if (i > 0) std::cout << ", ";
                std::cout << n.functions[i];
            }
            std::cout << "\n";
        }
    }
};

inline VehicleArchitecture CreateTraditional() {
    return {"Domain-Based (Traditional)", {
        {"Engine ECU",       ECUType::TRADITIONAL_DOMAIN, {"Fuel injection", "Ignition"}, 1},
        {"ABS/ESP ECU",      ECUType::TRADITIONAL_DOMAIN, {"Brake control", "Stability"}, 2},
        {"BCM",              ECUType::TRADITIONAL_DOMAIN, {"Lights", "Wipers", "Doors"}, 1},
        {"ADAS ECU",         ECUType::TRADITIONAL_DOMAIN, {"AEB", "LKA", "ACC"}, 20},
        {"Infotainment",     ECUType::TRADITIONAL_DOMAIN, {"Nav", "Media", "Phone"}, 10},
        {"Gateway",          ECUType::TRADITIONAL_DOMAIN, {"CAN routing"}, 1},
    }, 80, "CAN + LIN"};
}

inline VehicleArchitecture CreateZonal() {
    return {"Zonal SDV", {
        {"Vehicle HPC",      ECUType::HPC, {"ADAS", "Cockpit", "Body logic", "OTA"}, 200},
        {"Zone Front-Left",  ECUType::ZONE_CONTROLLER, {"FL sensors", "FL actuators"}, 5},
        {"Zone Front-Right", ECUType::ZONE_CONTROLLER, {"FR sensors", "FR actuators"}, 5},
        {"Zone Rear-Left",   ECUType::ZONE_CONTROLLER, {"RL sensors", "RL actuators"}, 5},
        {"Zone Rear-Right",  ECUType::ZONE_CONTROLLER, {"RR sensors", "RR actuators"}, 5},
    }, 12, "Automotive Ethernet (10GBASE-T1)"};
}

}  // namespace sdv_arch

```

```cpp
// ═══════════════════════════════════════════════════════════════════════════
// PART 2: Vehicle Signal Specification (VSS) — Signal Tree
// ═══════════════════════════════════════════════════════════════════════════

namespace vss {

// EN: VSS data types — mirrors COVESA/GENIVI VSS standard
// TR: VSS veri türleri — COVESA VSS standardını yansıtır
using SignalValue = std::variant<bool, int32_t, uint32_t, float, std::string>;

struct VSSNode {
    std::string  path;         // e.g. "Vehicle.Speed" — TR: örn. "Vehicle.Speed" sinyal yolu
    std::string  description;
    std::string  unit;
    SignalValue  value;
    std::string  data_type;
};

class VSSTree {
    std::map<std::string, VSSNode> signals_;

public:
    void AddSignal(VSSNode node) {
        auto path = node.path;
        signals_[path] = std::move(node);
    }

    std::optional<SignalValue> Get(const std::string& path) const {
        auto it = signals_.find(path);
        if (it == signals_.end()) return std::nullopt;
        return it->second.value;
    }

    bool Set(const std::string& path, SignalValue val) {
        auto it = signals_.find(path);
        if (it == signals_.end()) return false;
        it->second.value = std::move(val);
        return true;
    }

    void PrintAll() const {
        for (const auto& [path, node] : signals_) {
            std::cout << "  " << std::setw(45) << std::left << path;
            std::visit([&](const auto& v) {
                using T = std::decay_t<decltype(v)>;
                if constexpr (std::is_same_v<T, bool>)
                    std::cout << (v ? "true" : "false");
                else if constexpr (std::is_same_v<T, std::string>)
                    std::cout << v;
                else
                    std::cout << v;
            }, node.value);
            std::cout << " " << node.unit << "\n";
        }
    }
};

inline VSSTree CreateDefaultVSS() {
    VSSTree tree;
    tree.AddSignal({"Vehicle.Speed",                        "Current speed",       "km/h",
 uint32_t{0},    "uint16"});
    tree.AddSignal({"Vehicle.Powertrain.FuelSystem.Level",  "Fuel level",          "%",
 float{75.0f},   "float"});
    tree.AddSignal({"Vehicle.Chassis.SteeringWheel.Angle",  "Steering angle",      "deg",
 float{0.0f},    "float"});
    tree.AddSignal({"Vehicle.ADAS.AEB.IsEnabled",           "AEB status",          "",       true,
          "bool"});
    tree.AddSignal({"Vehicle.Body.Lights.Beam.Low.IsOn",    "Low beam status",     "",       false,
          "bool"});
    tree.AddSignal({"Vehicle.Cabin.HVAC.Temperature",       "Cabin temperature",   "°C",
 float{22.0f},   "float"});
    tree.AddSignal({"Vehicle.OBD.EngineLoad",               "Engine load",         "%",
 float{35.0f},   "float"});
    tree.AddSignal({"Vehicle.CurrentLocation.Latitude",     "GPS latitude",        "deg",
 float{39.92f},  "float"});
    tree.AddSignal({"Vehicle.CurrentLocation.Longitude",    "GPS longitude",       "deg",
 float{32.85f},  "float"});
```

```
    tree.AddSignal({"Vehicle.VehicleIdentification.VIN",    "VIN",                  "",
 std::string{"WVWZZZ3CZWE123456"}, "string"});
    return tree;
}

}  // namespace vss

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: SOA — Mini Service-Oriented Architecture Framework
// ═══════════════════════════════════════════════════════════════════════════

namespace soa {

// EN: Service message types
// TR: Servis mesaj türleri
enum class MessageType : uint8_t {
    REQUEST, RESPONSE, EVENT, SUBSCRIBE, UNSUBSCRIBE
};

struct ServiceMessage {
    std::string  service_name;
    std::string  method_or_event;
    MessageType  type;
    std::string  payload;
    uint32_t     client_id;
};

// EN: Service interface — all services implement this
// TR: Servis arayüzü — tüm servisler bunu uygular
class IService {
public:
    virtual ~IService() = default;
    virtual std::string_view GetName() const = 0;
    virtual std::string HandleRequest(const std::string& method,
                                      const std::string& payload) = 0;
};

// EN: SOA Service Registry — central service directory
// TR: SOA Servis Kaydı — merkezi servis dizini
class ServiceRegistry {
    std::unordered_map<std::string, IService*> services_;
    std::unordered_map<std::string,
        std::vector<std::function<void(const std::string&, const std::string&)>>> subscribers_;

public:
    bool Register(IService* svc) {
        auto name = std::string(svc->GetName());
        if (services_.count(name) > 0) return false;
        services_[name] = svc;
        std::cout << "  [Registry] Service registered: " << name << "\n";
        return true;
    }

    std::optional<std::string> Call(const std::string& service,
                                    const std::string& method,
                                    const std::string& payload) {
        auto it = services_.find(service);
        if (it == services_.end()) return std::nullopt;
        return it->second->HandleRequest(method, payload);
    }

    void Subscribe(const std::string& event_topic,
                   std::function<void(const std::string&, const std::string&)> handler) {
        subscribers_[event_topic].push_back(std::move(handler));
    }

    void Publish(const std::string& event_topic, const std::string& data) {
        auto it = subscribers_.find(event_topic);
        if (it != subscribers_.end()) {
            for (auto& handler : it->second) {
                handler(event_topic, data);
            }
        }
    }

    [[nodiscard]] size_t ServiceCount() const { return services_.size(); }
};
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════
// PART 4: Concrete Services — VehicleSpeed, Dashboard, Logger
// ═══════════════════════════════════════════════════════════════════════════

// EN: Vehicle Speed Service (provider)
// TR: Araç Hız Servisi (sağlayıcı)
class VehicleSpeedService : public IService {
    uint32_t speed_kmh_ = 0;
    ServiceRegistry& registry_;

public:
    explicit VehicleSpeedService(ServiceRegistry& reg) : registry_(reg) {}

    std::string_view GetName() const override { return "VehicleSpeedService"; }

    std::string HandleRequest(const std::string& method,
                              const std::string& /*payload*/) override {
        if (method == "GetSpeed") {
            return std::to_string(speed_kmh_);
        }
        return "ERROR:UNKNOWN_METHOD";
    }

    // EN: Simulate speed change and publish event
    // TR: Hız değişikliğini simüle et ve event yayınla
    void UpdateSpeed(uint32_t new_speed) {
        speed_kmh_ = new_speed;
        registry_.Publish("VehicleSpeed.Changed", std::to_string(speed_kmh_));
    }
};

// EN: Dashboard Service (consumer / observer)
// TR: Gösterge Paneli Servisi (tüketici / gözlemci)
class DashboardService : public IService {
    uint32_t displayed_speed_ = 0;

public:
    std::string_view GetName() const override { return "DashboardService"; }

    std::string HandleRequest(const std::string& method,
                              const std::string& /*payload*/) override {
        if (method == "GetDisplayedSpeed")
            return std::to_string(displayed_speed_);
        return "ERROR:UNKNOWN_METHOD";
    }

    void OnSpeedChanged(const std::string& /*topic*/, const std::string& data) {
        displayed_speed_ = static_cast<uint32_t>(std::stoul(data));
        std::cout << "    [Dashboard] Speed updated: " << displayed_speed_ << " km/h\n";
    }
};

// EN: Data Logger Service (observer)
// TR: Veri Kaydedici Servisi (gözlemci)
class DataLoggerService : public IService {
    std::vector<std::string> log_entries_;

public:
    std::string_view GetName() const override { return "DataLoggerService"; }

    std::string HandleRequest(const std::string& method,
                              const std::string& /*payload*/) override {
        if (method == "GetLogCount")
            return std::to_string(log_entries_.size());
        if (method == "GetLogs") {
            std::string result;
            for (const auto& e : log_entries_) {
                result += e + "; ";
            }
            return result;
        }
        return "ERROR:UNKNOWN_METHOD";
    }
```

```cpp

    void OnEvent(const std::string& topic, const std::string& data) {
        auto entry = "[LOG] " + topic + "=" + data;
        log_entries_.push_back(entry);
        std::cout << "    [Logger] " << entry << "\n";
    }

    [[nodiscard]] size_t EntryCount() const { return log_entries_.size(); }
};

}  // namespace soa

// ==========================================================================
// PART 5: HPC Software Stack Model
// ==========================================================================

namespace hpc_stack {

struct SoftwareLayer {
    std::string name;
    std::string language;
    std::string description;
};

inline std::vector<SoftwareLayer> GetHPCStack() {
    return {
        {"Vehicle Apps",      "C++/Python/Java", "ADAS, Cockpit HMI, Body logic"},
        {"ara::* / AOS API",  "C++17",           "AUTOSAR Adaptive / Android Automotive"},
        {"Middleware",         "C++",             "SOME/IP, DDS, iceoryx, gRPC"},
        {"OS Framework",      "C/C++",           "POSIX, QNX/Linux, hypervisor"},
        {"Hypervisor",        "C/ASM",           "QNX Hypervisor, Xen, COQOS"},
        {"BSP / Drivers",     "C",               "Board support, MCAL, kernel modules"},
        {"Hardware SoC",      "RTL",             "Qualcomm SA8295P / NVIDIA Orin / NXP S32G"},
    };
}

}  // namespace hpc_stack

// ==========================================================================
// PART 6: Industry SDV Platform Comparison
// ==========================================================================

namespace industry {

struct SDVPlatform {
    std::string oem;
    std::string platform_name;
    std::string soc;
    std::string os;
    std::string middleware;
    std::string note;
};

inline std::vector<SDVPlatform> GetPlatforms() {
    return {
        {"Volkswagen",  "vw.os",          "Qualcomm SA8295P",  "VW.OS (Linux-based)", "SOME/IP + DDS",  "Cariad development"},
        {"Mercedes",    "MB.OS",          "NVIDIA Orin",       "QNX + Linux",         "SOME/IP",  "Full stack in-house"},
        {"BMW",         "Neue Klasse OS", "Qualcomm SA8295P",  "QNX + Android",       "SOME/IP + DDS",  "2025 launch with iX"},
        {"Tesla",       "Tesla OS",       "HW4.0 (custom)",    "Linux (custom)",      "Custom IPC",  "Full vertical integ."},
        {"Rivian",      "Rivian SW",      "Qualcomm SA8295P",  "Linux + QNX",         "DDS",  "Startup approach"},
        {"Hyundai",     "ccOS",           "Qualcomm SA8155P",  "Linux + RTOS",        "SOME/IP",  "Connected Car OS"},
    };
}

}  // namespace industry

```

```cpp
// ═══════════════════════════════════════════════════════════════════════
// MAIN — Demos
// ═══════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "╔═══════════════════════════════════════════════════════╗\n"
              << "║ Module 17: SDV Architecture & C++ Role                ║\n"
              << "╚═══════════════════════════════════════════════════════╝\n\n";

    // --- Demo 1: Domain → Zonal Transition ---
    std::cout << "--- Demo 1: Domain → Zonal Architecture Transition ---\n";
    {
        auto trad = sdv_arch::CreateTraditional();
        auto zonal = sdv_arch::CreateZonal();

        std::cout << "\n  [TRADITIONAL]\n";
        trad.Print();
        std::cout << "\n  [ZONAL SDV]\n";
        zonal.Print();

        std::cout << "\n  Consolidation: " << trad.total_ecus << " ECUs → "
                  << zonal.total_ecus << " nodes ("
                  << (100 - (zonal.total_ecus * 100 / trad.total_ecus)) << "% reduction)\n";
    }

    // --- Demo 2: VSS Signal Tree ---
    std::cout << "\n--- Demo 2: Vehicle Signal Specification (VSS) Tree ---\n";
    {
        auto vss_tree = vss::CreateDefaultVSS();
        vss_tree.PrintAll();

        // EN: Update signals
        // TR: Sinyalleri güncelle — VSS ağacındaki değerleri variant ile ayarla
        vss_tree.Set("Vehicle.Speed", uint32_t{120});
        vss_tree.Set("Vehicle.Body.Lights.Beam.Low.IsOn", true);

        std::cout << "\n  After update:\n";
        auto speed = vss_tree.Get("Vehicle.Speed");
        if (speed) {
            std::cout << "  Vehicle.Speed = "
                      << std::get<uint32_t>(*speed) << " km/h\n";
        }
        auto lights = vss_tree.Get("Vehicle.Body.Lights.Beam.Low.IsOn");
        if (lights) {
            std::cout << "  Vehicle.Body.Lights.Beam.Low.IsOn = "
                      << (std::get<bool>(*lights) ? "true" : "false") << "\n";
        }
    }

    // --- Demo 3: Mini SOA — Service Provider + Consumer + Observer ---
    std::cout << "\n--- Demo 3: SOA — VehicleSpeed + Dashboard + Logger ---\n";
    {
        soa::ServiceRegistry registry;

        soa::VehicleSpeedService speed_svc(registry);
        soa::DashboardService    dashboard;
        soa::DataLoggerService   logger;

        registry.Register(&speed_svc);
        registry.Register(&dashboard);
        registry.Register(&logger);

        // EN: Subscribe to speed events
        // TR: Hız olaylarına abone ol — Observer deseni ile hız değiştiğinde bildirim alınır
        registry.Subscribe("VehicleSpeed.Changed",
            [&](const std::string& t, const std::string& d) { dashboard.OnSpeedChanged(t, d); });
        registry.Subscribe("VehicleSpeed.Changed",
            [&](const std::string& t, const std::string& d) { logger.OnEvent(t, d); });

        std::cout << "\n  Simulating speed changes:\n";
        speed_svc.UpdateSpeed(60);
        speed_svc.UpdateSpeed(100);
        speed_svc.UpdateSpeed(0);

```

```cpp
        // EN: Query services via registry
        // TR: Kayıt defteri üzerinden servisleri sorgula — istek/yanıt şeklinde metot çağrısı
        auto resp = registry.Call("VehicleSpeedService", "GetSpeed", "");
        std::cout << "\n  Registry.Call(VehicleSpeedService, GetSpeed) = "
                  << resp.value_or("N/A") << "\n";

        auto dash_resp = registry.Call("DashboardService", "GetDisplayedSpeed", "");
        std::cout << "  Registry.Call(DashboardService, GetDisplayedSpeed) = "
                  << dash_resp.value_or("N/A") << "\n";

        auto log_resp = registry.Call("DataLoggerService", "GetLogCount", "");
        std::cout << "  Logger entries: " << log_resp.value_or("0") << "\n";
    }

    // --- Demo 4: HPC Software Stack ---
    std::cout << "\n--- Demo 4: HPC Software Stack Layers ---\n";
    {
        auto stack = hpc_stack::GetHPCStack();
        std::cout << "  ┌─────┬──────────────────┬─────────────────┬──────────────────────────────────┐\n";
        std::cout << "  │ Lyr │ Name             │ Language        │ Description                      │\n";
        std::cout << "  ├─────┼──────────────────┼─────────────────┼──────────────────────────────────┤\n";
        int layer = static_cast<int>(stack.size());
        for (const auto& s : stack) {
            std::cout << "  │ L" << layer-- << "  │ "
                      << std::setw(16) << std::left << s.name << " │ "
                      << std::setw(15) << s.language << " │ "
                      << std::setw(41) << s.description << " │\n";
        }
        std::cout << std::right;
        std::cout << "  └─────┴──────────────────┴─────────────────┴──────────────────────────────────┘\n";
    }

    // --- Demo 5: Industry SDV Platforms ---
    std::cout << "\n--- Demo 5: Industry SDV Platforms ---\n";
    {
        auto platforms = industry::GetPlatforms();
        std::cout << "  ┌──────────────┬──────────────────┬─────────────────┬──────────────────────┬──────────────┐\n";
        std::cout << "  │ OEM          │ Platform         │ SoC             │ OS                   │ Middleware   │\n";
        std::cout << "  ├──────────────┼──────────────────┼─────────────────┼──────────────────────┼──────────────┤\n";
        for (const auto& p : platforms) {
            std::cout << "  │ " << std::setw(11) << std::left << p.oem
                      << " │ " << std::setw(16) << p.platform_name
                      << " │ " << std::setw(16) << p.soc
                      << " │ " << std::setw(20) << p.os
                      << " │ " << std::setw(13) << p.middleware << " │\n";
        }
        std::cout << std::right;
        std::cout << "  └──────────────┴──────────────────┴─────────────────┴──────────────────────┴──────────────┘\n";
    }

    // --- Demo 6: C++ Role Summary ---
    std::cout << "\n--- Demo 6: C++ Role in SDV Ecosystem ---\n";
    {
        struct LangRole { std::string_view lang; std::string_view domain; std::string_view pct; };
        LangRole roles[] = {
            {"C++",     "Middleware, ADAS, Sensor fusion, RT services",  "~45%"},
            {"C",       "Drivers, MCAL, RTOS kernel, bootloader",        "~25%"},
            {"Python",  "ML training, test automation, tooling",         "~15%"},
            {"Rust",    "Safety-critical new components (emerging)",     "~3%"},
            {"Java/Kt", "Android Automotive apps, infotainment",         "~8%"},
            {"Other",   "Shell, YAML, ARXML, build scripts",             "~4%"},
        };
```

```
        std::cout << "  Estimated language distribution in SDV projects:\n";
        for (const auto& r : roles) {
            std::cout << "  " << std::setw(8) << std::left << r.lang
                      << " " << std::setw(5) << r.pct << "  "
                      << r.domain << "\n";
        }
        std::cout << std::right;
    }

    std::cout << "\n[All demos completed successfully.]\n";
    return 0;
}
```

► 02. Ota Updates

module_17_sdv/02_ota_updates.cpp

Konu: OTA (Over-The-Air) Update Mechanism — Automotive Firmware Updates

Teori / Theory

THEORY: OTA Update Lifecycle

OTA allows updating vehicle software without physical access.
Campaign Download Verify Install Activate Confirm Creation Firmware Integrity to Slot B Boot B Success
| | | | | |
▼▼▼▼▼▼

↓ FAIL ↓ FAIL

TEORİ: OTA Güncelleme Yaşam Döngüsü

OTA, araç yazılımını fiziksel erişim olmadan güncellemeye olanak tanır.
Kampanya İndirme Doğrulama Kurulum Aktive et Onay Oluşturma Firmware Bütünlük Slot B'ye B'yi başlat Başarı
| | | | | |
▼▼▼▼▼▼

↓ BAŞARISIZ ↓ BAŞARISIZ

THEORY: A/B Partitioning — Atomic Slot Swap

Flash Memory Layout:

```
┌──────────────────┬──────────────────────────┐
│ Bootloader (ROM) │ Protected, immutable     │
├──────────────────┼──────────────────────────┤
│ Slot A (Active)  │ Currently running FW     │
├──────────────────┼──────────────────────────┤
│ Slot B (Inactive) │ Target for OTA update   │
├──────────────────┼──────────────────────────┤
│ NvM / Config     │ Persistent data          │
└──────────────────────────────────────────────┘
```

Update flow:
1. Download firmware → write to Slot B
2. Verify Slot B (hash + signature)
3. Set boot flag: next_boot = B
4. Reboot → bootloader reads flag → boots Slot B
5. Application confirms success → commit B as active
6. If no confirm within timeout → rollback to Slot A

TEORİ: A/B Bölümleme — Atomik Slot Değişimi

A/B bölümleme — atomik slot takası: Flash bellek iki eşit "slot" içerir (A ve B). Güncelleme her zaman pasif slot'a (B) yazılır; aktif slot (A) çalışmaya devam eder. Akış: B'ye indir → hash+imza doğrula → önyükleyici bayrağını B'ye ayarla
→ yeniden başlat → B başarılıysa onayla → başarısızsa A'ya rollback. Avantaj: güncelleme sırasında araç kullanılabilir, brick riski sıfır.

THEORY: Secure Boot Chain

ROM BL (HW root of trust)
```
│ verify signature
```
▼ 1st Stage BL (SPL)
```
│ verify signature
```
▼ 2nd Stage BL (U-Boot)
```
│ verify signature
```
▼ Application FW

TEORİ: Güvenli Önyükleme Zinciri

Güvenli önyükleme zinciri (Secure Boot Chain): Her aşama bir öncekini doğrular (ROM BL → 1. Aşama BL → 2. Aşama BL → Uygulama). Kök güven (root of trust) donanımda ROM BL'de saklanır; değiştirilemez. İmza doğrulaması başarısız olursa önyükleme durur — manipüle edilmiş firmware çalıştırılmaz.

THEORY: UDS Flash Programming Sequence

```
Step │ UDS Service                │ Description
─────┼────────────────────────────┼──────────────────────
 1   │ 0x10 (DiagSession)         │ Enter Programming session
 2   │ 0x27 (SecurityAccess)      │ Authenticate tester
 3   │ 0x34 (RequestDownload)     │ Negotiate transfer params
 4   │ 0x36 (TransferData)        │ Send firmware blocks
 5   │ 0x37 (RequestTransferExit)│ Finalize transfer
 6   │ 0x31 (RoutineControl)      │ Verify checksum / activate
```

TEORİ: UDS Flash Programlama Sırası

UDS Flash programlama sırası:
1. DiagSession (0x10): Programlama oturumuna gir
2. SecurityAccess (0x27): Test cihazını doğrula
3. RequestDownload (0x34): Transfer parametrelerini müzakere et
4. TransferData (0x36): Firmware bloklarını gönder
5. RequestTransferExit (0x37): Transferi sonlandır
6. RoutineControl (0x31): Checksum doğrula / aktive et

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <vector>
#include <string>
#include <string_view>
#include <array>
#include <iomanip>
#include <algorithm>
#include <optional>
#include <numeric>
#include <cassert>
#include <cstring>
#include <functional>
#include <memory>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: Firmware Image Format
// ═══════════════════════════════════════════════════════════════════════════

namespace ota {

// EN: Firmware image header — precedes the actual binary payload
// TR: Firmware image başlığı — gerçek ikili yükün önünde yer alır
struct FirmwareHeader {
    uint32_t magic;             // 0x4F544131 = "OTA1" — TR: sihirli sayı (geçerlilik kontrolü)
    uint32_t version_major;
    uint32_t version_minor;
    uint32_t version_patch;
    uint32_t payload_size;
    uint32_t payload_crc32;
    uint32_t header_crc32;      // CRC of header fields (excluding this field) — TR: başlık alanlarının CRC'si
    uint64_t timestamp;         // build timestamp (epoch) — TR: derleme zaman damgası
    char     target_ecu[16];    // target ECU identifier — TR: hedef ECU kimliği

    void Print() const {
        std::cout << "    Magic:    0x" << std::hex << magic << std::dec << "\n"
                  << "    Version:  " << version_major << "." << version_minor
                  << "." << version_patch << "\n"
                  << "    Payload:  " << payload_size << " bytes\n"
                  << "    CRC32:    0x" << std::hex << payload_crc32 << std::dec << "\n"
                  << "    Target:   " << std::string(target_ecu, strnlen(target_ecu, 16)) << "\n";
    }
};

// EN: Simple CRC32 for verification (same as MISRA file for consistency)
// TR: Doğrulama için basit CRC32
constexpr uint32_t CRC32_POLY = 0xEDB88320u;

inline uint32_t ComputeCRC32(const uint8_t* data, size_t length) {
    uint32_t crc = 0xFFFFFFFFu;
```

```cpp
    for (size_t i = 0; i < length; ++i) {
        crc ^= data[i];
        for (int j = 0; j < 8; ++j) {
            if ((crc & 1u) != 0)
                crc = (crc >> 1) ^ CRC32_POLY;
            else
                crc >>= 1;
        }
    }
    return crc ^ 0xFFFFFFFFu;
}

// EN: Build a firmware image (header + payload)
// TR: Firmware image oluştur (başlık + yük)
struct FirmwareImage {
    FirmwareHeader          header;
    std::vector<uint8_t>    payload;

    static FirmwareImage Build(uint32_t major, uint32_t minor, uint32_t patch,
                               const std::string& target,
                               const std::vector<uint8_t>& fw_data) {
        FirmwareImage img;
        img.payload = fw_data;

        auto& h = img.header;
        h.magic = 0x4F544131;
        h.version_major = major;
        h.version_minor = minor;
        h.version_patch = patch;
        h.payload_size = static_cast<uint32_t>(fw_data.size());
        h.payload_crc32 = ComputeCRC32(fw_data.data(), fw_data.size());
        h.timestamp = 1717200000;  // 2024-06-01
        std::memset(h.target_ecu, 0, sizeof(h.target_ecu));
        auto copy_len = std::min(target.size(), size_t{15});
        std::memcpy(h.target_ecu, target.c_str(), copy_len);

        // EN: Header CRC covers all fields except header_crc32 itself
        // TR: Header CRC, header_crc32 alanı hariç tüm başlık alanlarını kapsar
        const auto* hdr_bytes = reinterpret_cast<const uint8_t*>(&h);
        h.header_crc32 = ComputeCRC32(hdr_bytes, offsetof(FirmwareHeader, header_crc32));

        return img;
    }

    [[nodiscard]] bool VerifyPayload() const {
        auto actual = ComputeCRC32(payload.data(), payload.size());
        return actual == header.payload_crc32;
    }

    [[nodiscard]] bool VerifyHeader() const {
        const auto* hdr_bytes = reinterpret_cast<const uint8_t*>(&header);
        auto actual = ComputeCRC32(hdr_bytes, offsetof(FirmwareHeader, header_crc32));
        return actual == header.header_crc32;
    }
};

// ============================================================================
// PART 2: A/B Partition Manager
// ============================================================================

enum class Slot : uint8_t { A, B };

inline std::string_view SlotName(Slot s) {
    return s == Slot::A ? "Slot_A" : "Slot_B";
}

struct PartitionInfo {
    Slot     slot;
    uint32_t version_major = 0;
    uint32_t version_minor = 0;
    uint32_t version_patch = 0;
    uint32_t fw_crc32 = 0;
    bool     valid = false;
    bool     confirmed = false;    // EN: set after successful boot / TR: başarılı boot sonrası set edilir
    uint8_t  boot_attempts = 0;
};
```

```cpp

class ABPartitionManager {
    PartitionInfo slot_a_;
    PartitionInfo slot_b_;
    Slot          active_slot_ = Slot::A;
    Slot          next_boot_  = Slot::A;
    static constexpr uint8_t MAX_BOOT_ATTEMPTS = 3;

public:
    ABPartitionManager() {
        slot_a_.slot = Slot::A;
        slot_b_.slot = Slot::B;
    }

    // EN: Install firmware to inactive slot
    // TR: Firmware'i aktif olmayan slot'a yükle
    bool InstallToInactive(const FirmwareImage& img) {
        auto& target = (active_slot_ == Slot::A) ? slot_b_ : slot_a_;
        if (!img.VerifyPayload() || !img.VerifyHeader()) {
            std::cout << "    [AB] Image verification FAILED\n";
            return false;
        }
        target.version_major = img.header.version_major;
        target.version_minor = img.header.version_minor;
        target.version_patch = img.header.version_patch;
        target.fw_crc32 = img.header.payload_crc32;
        target.valid = true;
        target.confirmed = false;
        target.boot_attempts = 0;

        std::cout << "    [AB] Installed v" << target.version_major
                  << "." << target.version_minor << "." << target.version_patch
                  << " to " << SlotName(target.slot) << "\n";
        return true;
    }

    // EN: Swap active slot for next boot
    // TR: Sonraki boot için aktif slot'u değiştir
    void SetNextBoot(Slot s) {
        next_boot_ = s;
        std::cout << "    [AB] Next boot set to " << SlotName(s) << "\n";
    }

    // EN: Simulate bootloader selecting slot
    // TR: Bootloader'ın slot seçimini simüle et
    bool Boot() {
        auto& target = (next_boot_ == Slot::A) ? slot_a_ : slot_b_;
        if (!target.valid) {
            std::cout << "    [AB] " << SlotName(next_boot_) << " invalid, falling back\n";
            next_boot_ = (next_boot_ == Slot::A) ? Slot::B : Slot::A;
            active_slot_ = next_boot_;
            return false;
        }
        ++target.boot_attempts;
        if (target.boot_attempts > MAX_BOOT_ATTEMPTS && !target.confirmed) {
            std::cout << "    [AB] " << SlotName(next_boot_)
                      << " exceeded max attempts, ROLLBACK\n";
            next_boot_ = (next_boot_ == Slot::A) ? Slot::B : Slot::A;
            active_slot_ = next_boot_;
            return false;
        }
        active_slot_ = next_boot_;
        std::cout << "    [AB] Booted from " << SlotName(active_slot_)
                  << " (attempt " << static_cast<int>(target.boot_attempts) << ")\n";
        return true;
    }

    // EN: Application confirms successful boot
    // TR: Uygulama başarılı boot'u onaylar
    void ConfirmBoot() {
        auto& active = (active_slot_ == Slot::A) ? slot_a_ : slot_b_;
        active.confirmed = true;
        std::cout << "    [AB] " << SlotName(active_slot_) << " CONFIRMED\n";
    }
```

```cpp

    void InitSlotA(uint32_t maj, uint32_t min, uint32_t pat) {
        slot_a_.version_major = maj;
        slot_a_.version_minor = min;
        slot_a_.version_patch = pat;
        slot_a_.valid = true;
        slot_a_.confirmed = true;
        active_slot_ = Slot::A;
        next_boot_ = Slot::A;
    }

    void PrintStatus() const {
        auto print_slot = [](const PartitionInfo& p) {
            std::cout << "    " << SlotName(p.slot) << ": v"
                      << p.version_major << "." << p.version_minor
                      << "." << p.version_patch
                      << " valid=" << p.valid
                      << " confirmed=" << p.confirmed
                      << " attempts=" << static_cast<int>(p.boot_attempts) << "\n";
        };
        print_slot(slot_a_);
        print_slot(slot_b_);
        std::cout << "    Active: " << SlotName(active_slot_)
                  << "  Next boot: " << SlotName(next_boot_) << "\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: OTA State Machine
// ═══════════════════════════════════════════════════════════════════════════

enum class OTAState : uint8_t {
    IDLE, DOWNLOADING, VERIFYING, INSTALLING, ACTIVATING,
    MONITORING, CONFIRMED, ROLLBACK, ERROR
};

inline std::string_view OTAStateStr(OTAState s) {
    switch (s) {
        case OTAState::IDLE:        return "IDLE";
        case OTAState::DOWNLOADING: return "DOWNLOADING";
        case OTAState::VERIFYING:   return "VERIFYING";
        case OTAState::INSTALLING:  return "INSTALLING";
        case OTAState::ACTIVATING:  return "ACTIVATING";
        case OTAState::MONITORING:  return "MONITORING";
        case OTAState::CONFIRMED:   return "CONFIRMED";
        case OTAState::ROLLBACK:    return "ROLLBACK";
        case OTAState::ERROR:       return "ERROR";
    }
    return "UNKNOWN";
}

class OTAManager {
    OTAState           state_ = OTAState::IDLE;
    ABPartitionManager ab_mgr_;
    FirmwareImage      pending_image_;
    bool               image_ready_ = false;

    void Transition(OTAState next) {
        std::cout << "  [OTA] " << OTAStateStr(state_) << " → " << OTAStateStr(next) << "\n";
        state_ = next;
    }

public:
    void InitCurrentFirmware(uint32_t maj, uint32_t min, uint32_t pat) {
        ab_mgr_.InitSlotA(maj, min, pat);
    }

    // EN: Simulate full OTA flow
    // TR: Tam OTA akışını simüle et
    bool StartUpdate(const FirmwareImage& img) {
        Transition(OTAState::DOWNLOADING);
        pending_image_ = img;
        image_ready_ = true;
        std::cout << "  [OTA] Downloaded " << img.header.payload_size << " bytes\n";

        // Verify
        Transition(OTAState::VERIFYING);
```

```
278         if (!pending_image_.VerifyPayload() || !pending_image_.VerifyHeader()) {
279             Transition(OTAState::ERROR);
280             return false;
281         }
282         std::cout << "  [OTA] Verification PASSED\n";
283
284         // Install
285         Transition(OTAState::INSTALLING);
286         if (!ab_mgr_.InstallToInactive(pending_image_)) {
287             Transition(OTAState::ERROR);
288             return false;
289         }
290
291         // Activate
292         Transition(OTAState::ACTIVATING);
293         ab_mgr_.SetNextBoot(Slot::B);
294         ab_mgr_.Boot();
295
296         // Monitor
297         Transition(OTAState::MONITORING);
298         // EN: Simulate application health check
299         // TR: Uygulama sağlık kontrolünü simüle et — başarısız olursa rollback tetiklenir
300         bool app_healthy = true;
301         if (app_healthy) {
302             ab_mgr_.ConfirmBoot();
303             Transition(OTAState::CONFIRMED);
304         } else {
305             Transition(OTAState::ROLLBACK);
306             ab_mgr_.SetNextBoot(Slot::A);
307             ab_mgr_.Boot();
308         }
309
310         return state_ == OTAState::CONFIRMED;
311     }
312
313     // EN: Simulate a failed update (corrupted image)
314     // TR: Başarısız güncelleme simülasyonu (bozuk image)
315     bool StartCorruptUpdate(FirmwareImage img) {
316         Transition(OTAState::DOWNLOADING);
317         // EN: Corrupt the payload
318         // TR: Yük verisini boz — CRC uyuşmazlığı tespit mekanizmasını test eder
319         if (!img.payload.empty()) {
320             img.payload[0] ^= 0xFF;
321         }
322         Transition(OTAState::VERIFYING);
323         if (!img.VerifyPayload()) {
324             std::cout << "  [OTA] CRC MISMATCH — image corrupted!\n";
325             Transition(OTAState::ERROR);
326             return false;
327         }
328         return false;
329     }
330
331     void PrintStatus() const {
332         std::cout << "  State: " << OTAStateStr(state_) << "\n";
333         ab_mgr_.PrintStatus();
334     }
335 };
336
337 // ═══════════════════════════════════════════════════════════════════════
338 // PART 4: Delta Update Simulation
339 // ═══════════════════════════════════════════════════════════════════════
340
341 namespace delta {
342
343 // EN: Simple delta patch (byte-level diff)
344 // TR: Basit delta yama (byte seviyesi fark)
345 struct DeltaPatch {
346     struct PatchEntry {
347         uint32_t offset;
348         uint8_t  old_value;
349         uint8_t  new_value;
350     };
351
```

```
    std::vector<PatchEntry> entries;
    uint32_t target_crc32;

    [[nodiscard]] size_t PatchSize() const {
        return entries.size() * sizeof(PatchEntry);
    }
};

inline DeltaPatch CreateDelta(const std::vector<uint8_t>& old_fw,
                              const std::vector<uint8_t>& new_fw) {
    DeltaPatch patch;
    auto min_len = std::min(old_fw.size(), new_fw.size());
    for (size_t i = 0; i < min_len; ++i) {
        if (old_fw[i] != new_fw[i]) {
            patch.entries.push_back({static_cast<uint32_t>(i), old_fw[i], new_fw[i]});
        }
    }
    // EN: Appended bytes
    // TR: Eklenmiş byte'lar — yeni firmware eski firmware'den uzunsa, fazla kısım ekleme olarak kaydedilir
    for (size_t i = min_len; i < new_fw.size(); ++i) {
        patch.entries.push_back({static_cast<uint32_t>(i), 0, new_fw[i]});
    }
    patch.target_crc32 = ComputeCRC32(new_fw.data(), new_fw.size());
    return patch;
}

inline bool ApplyDelta(std::vector<uint8_t>& fw, const DeltaPatch& patch) {
    for (const auto& e : patch.entries) {
        if (e.offset >= fw.size()) fw.resize(e.offset + 1, 0);
        fw[e.offset] = e.new_value;
    }
    auto crc = ComputeCRC32(fw.data(), fw.size());
    return crc == patch.target_crc32;
}

}  // namespace delta

// ═══════════════════════════════════════════════════════════════════════════
// PART 5: UDS Flash Programming Sequence
// ═══════════════════════════════════════════════════════════════════════════

namespace uds_flash {

enum class FlashState : uint8_t {
    IDLE, SESSION_OPEN, SECURITY_UNLOCKED, DOWNLOAD_STARTED,
    TRANSFERRING, TRANSFER_DONE, VERIFIED, ERROR
};

inline std::string_view FlashStateStr(FlashState s) {
    switch (s) {
        case FlashState::IDLE:              return "IDLE";
        case FlashState::SESSION_OPEN:       return "SESSION_OPEN";
        case FlashState::SECURITY_UNLOCKED:  return "SECURITY_UNLOCKED";
        case FlashState::DOWNLOAD_STARTED:   return "DOWNLOAD_STARTED";
        case FlashState::TRANSFERRING:       return "TRANSFERRING";
        case FlashState::TRANSFER_DONE:      return "TRANSFER_DONE";
        case FlashState::VERIFIED:           return "VERIFIED";
        case FlashState::ERROR:              return "ERROR";
    }
    return "UNKNOWN";
}

class FlashProgrammer {
    FlashState state_ = FlashState::IDLE;
    uint32_t   total_size_ = 0;
    uint32_t   received_ = 0;
    uint8_t    block_seq_ = 0;

    void Log(std::string_view service, std::string_view detail) {
        std::cout << "    [UDS] " << service << " — " << detail
                  << " [state=" << FlashStateStr(state_) << "]\n";
    }
```

```

public:
    // EN: 0x10 — DiagnosticSessionControl (Programming session)
    // TR: 0x10 — Tanı Oturum Kontrolü (Programlama oturumuna geçiş)
    bool DiagSessionControl(uint8_t session_type) {
        if (session_type == 0x02) {  // Programming session
            state_ = FlashState::SESSION_OPEN;
            Log("0x10 DiagSession", "Programming session opened");
            return true;
        }
        Log("0x10 DiagSession", "Invalid session type");
        return false;
    }

    // EN: 0x27 — SecurityAccess (seed/key)
    // TR: 0x27 — Güvenlik Erişimi (seed/key ile test cihazı kimlik doğrulaması)
    bool SecurityAccess(uint32_t key) {
        if (state_ != FlashState::SESSION_OPEN) {
            Log("0x27 SecurityAccess", "Wrong state");
            return false;
        }
        // EN: Simplified key validation
        // TR: Basitleştirilmiş anahtar doğrulaması — gerçek sistemde HSM kullanılır
        if (key == 0xDEADBEEF) {
            state_ = FlashState::SECURITY_UNLOCKED;
            Log("0x27 SecurityAccess", "Security UNLOCKED");
            return true;
        }
        Log("0x27 SecurityAccess", "Invalid key");
        return false;
    }

    // EN: 0x34 — RequestDownload
    // TR: 0x34 — İndirme Talebi (hedef adres ve boyut bilgisi ile transfer başlatılır)
    bool RequestDownload(uint32_t address, uint32_t size) {
        if (state_ != FlashState::SECURITY_UNLOCKED) {
            Log("0x34 RequestDownload", "Wrong state");
            return false;
        }
        total_size_ = size;
        received_ = 0;
        block_seq_ = 1;
        state_ = FlashState::DOWNLOAD_STARTED;
        Log("0x34 RequestDownload",
            "addr=0x" + ([&]{ std::ostringstream s; s << std::hex << address; return s.str(); })()
            + " size=" + std::to_string(size));
        return true;
    }

    // EN: 0x36 — TransferData (one block)
    // TR: 0x36 — Veri Transferi (tek blok) — sıra numarası kontrolü ile blok blok gönderilir
    bool TransferData(uint8_t seq, const std::vector<uint8_t>& block) {
        if (state_ != FlashState::DOWNLOAD_STARTED && state_ != FlashState::TRANSFERRING) {
            Log("0x36 TransferData", "Wrong state");
            return false;
        }
        if (seq != block_seq_) {
            Log("0x36 TransferData", "Sequence mismatch");
            state_ = FlashState::ERROR;
            return false;
        }
        state_ = FlashState::TRANSFERRING;
        received_ += static_cast<uint32_t>(block.size());
        Log("0x36 TransferData",
            "block#" + std::to_string(seq) + " (" + std::to_string(block.size())
            + "B) total=" + std::to_string(received_) + "/" + std::to_string(total_size_));
        ++block_seq_;
        return true;
    }
```

```cpp

    // EN: 0x37 — RequestTransferExit
    // TR: 0x37 — Transfer Sonlandırma (tüm bloklar alındıktan sonra transferi kapatır)
    bool RequestTransferExit() {
        if (state_ != FlashState::TRANSFERRING || received_ < total_size_) {
            Log("0x37 TransferExit", "Incomplete transfer");
            return false;
        }
        state_ = FlashState::TRANSFER_DONE;
        Log("0x37 TransferExit", "Transfer complete");
        return true;
    }

    // EN: 0x31 — RoutineControl (verify + activate)
    // TR: 0x31 — Rutin Kontrol (checksum doğrulama + aktivasyon hazırlığı)
    bool RoutineControl_Verify() {
        if (state_ != FlashState::TRANSFER_DONE) {
            Log("0x31 RoutineControl", "Wrong state");
            return false;
        }
        state_ = FlashState::VERIFIED;
        Log("0x31 RoutineControl", "Checksum VERIFIED, activation ready");
        return true;
    }
};

}  // namespace uds_flash

// ═══════════════════════════════════════════════════════════════════════════
// PART 6: Uptane Framework Overview
// ═══════════════════════════════════════════════════════════════════════════

namespace uptane {

// EN: Uptane role simulation (simplified TUF-based)
// TR: Uptane rol simülasyonu (basitleştirilmiş TUF tabanlı)
struct UptaneRole {
    std::string name;
    std::string description;
    uint32_t    key_id;
};

inline std::vector<UptaneRole> GetRoles() {
    return {
        {"Root",      "Top-level trust anchor, signs other keys", 0x0001},
        {"Targets",   "Signs firmware metadata (hash, version)",  0x0002},
        {"Snapshot",  "Signs consistent set of target metadata",  0x0003},
        {"Timestamp", "Prevents replay attacks (freshness)",      0x0004},
        {"Director",  "Vehicle-specific update instructions",     0x0005},
    };
}

}  // namespace uptane

}  // namespace ota

// ═══════════════════════════════════════════════════════════════════════════
// MAIN — Demos
// ═══════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "╔═══════════════════════════════════════════════════════════╗\n"
              << "║ Module 17: OTA Updates — Automotive Firmware Update       ║\n"
              << "╚═══════════════════════════════════════════════════════════╝\n\n";

    // --- Demo 1: Firmware Image Build & Verify ---
    std::cout << "--- Demo 1: Firmware Image Build & Verify ---\n";
    {
```

```
        // EN: Simulated firmware payload
        // TR: Simüle edilmiş firmware yükü — 512 byte'lık test verisi oluşturulur
        std::vector<uint8_t> fw_data(512, 0);
        for (size_t i = 0; i < fw_data.size(); ++i)
            fw_data[i] = static_cast<uint8_t>(i & 0xFF);

        auto img = ota::FirmwareImage::Build(2, 1, 0, "BCM_ECU", fw_data);
        std::cout << "  Built firmware image:\n";
        img.header.Print();
        std::cout << "  Header valid: " << img.VerifyHeader() << "\n";
        std::cout << "  Payload valid: " << img.VerifyPayload() << "\n";

        // EN: Corrupt and re-verify
        // TR: Boz ve tekrar doğrula — CRC uyuşmazlığının tespitini test eder
        img.payload[0] ^= 0xFF;
        std::cout << "  After corruption: payload valid = " << img.VerifyPayload() << "\n";
    }

    // --- Demo 2: A/B Partition OTA Flow ---
    std::cout << "\n--- Demo 2: A/B Partition — Normal Update ---\n";
    {
        ota::OTAManager mgr;
        mgr.InitCurrentFirmware(1, 0, 0);
        std::cout << "  Initial state:\n";
        mgr.PrintStatus();

        std::vector<uint8_t> fw(256, 0xAB);
        auto img = ota::FirmwareImage::Build(2, 0, 0, "BCM_ECU", fw);

        std::cout << "\n  Starting OTA update to v2.0.0:\n";
        bool ok = mgr.StartUpdate(img);
        std::cout << "\n  Result: " << (ok ? "SUCCESS" : "FAILED") << "\n";
        mgr.PrintStatus();
    }

    // --- Demo 3: Corrupted Image Detection ---
    std::cout << "\n--- Demo 3: Corrupted Image Detection ---\n";
    {
        ota::OTAManager mgr;
        mgr.InitCurrentFirmware(1, 0, 0);

        std::vector<uint8_t> fw(256, 0xCD);
        auto img = ota::FirmwareImage::Build(2, 1, 0, "BCM_ECU", fw);

        std::cout << "  Attempting corrupted update:\n";
        bool ok = mgr.StartCorruptUpdate(img);
        std::cout << "  Result: " << (ok ? "SUCCESS" : "BLOCKED") << "\n";
    }

    // --- Demo 4: Delta Update ---
    std::cout << "\n--- Demo 4: Delta Update (Binary Diff) ---\n";
    {
        std::vector<uint8_t> old_fw = {0x10, 0x20, 0x30, 0x40, 0x50, 0x60, 0x70, 0x80};
        std::vector<uint8_t> new_fw = {0x10, 0x21, 0x30, 0x40, 0x55, 0x60, 0x70, 0x80, 0x90};

        auto patch = ota::delta::CreateDelta(old_fw, new_fw);
        std::cout << "  Old FW: " << old_fw.size() << " bytes\n"
                  << "  New FW: " << new_fw.size() << " bytes\n"
                  << "  Delta:  " << patch.entries.size() << " entries ("
                  << patch.PatchSize() << " bytes)\n"
                  << "  Savings: " << (100 - static_cast<int>(patch.PatchSize() * 100 /
new_fw.size())) << "%\n";

        std::cout << "  Patch entries:\n";
        for (const auto& e : patch.entries) {
            std::cout << "    offset=" << e.offset
                      << " old=0x" << std::hex << static_cast<int>(e.old_value)
                      << " new=0x" << static_cast<int>(e.new_value) << std::dec << "\n";
        }

        auto patched = old_fw;
        bool ok = ota::delta::ApplyDelta(patched, patch);
        std::cout << "  Apply result: " << (ok ? "CRC MATCH" : "CRC MISMATCH") << "\n";
        std::cout << "  Patched == New: " << (patched == new_fw ? "YES" : "NO") << "\n";
    }
```

```
    // --- Demo 5: UDS Flash Sequence ---
    std::cout << "\n--- Demo 5: UDS Flash Programming Sequence ---\n";
    {
        ota::uds_flash::FlashProgrammer prog;

        prog.DiagSessionControl(0x02);  // Programming session
        prog.SecurityAccess(0xDEADBEEF);  // Authenticate
        prog.RequestDownload(0x08000000, 768);  // Start download

        // EN: Transfer 3 blocks of 256 bytes
        // TR: 256 byte'lık 3 blok transfer et — toplam 768 byte = RequestDownload boyutu
        std::vector<uint8_t> block(256, 0xAA);
        prog.TransferData(1, block);
        prog.TransferData(2, block);
        prog.TransferData(3, block);

        prog.RequestTransferExit();
        prog.RoutineControl_Verify();
    }

    // --- Demo 6: Uptane Framework Roles ---
    std::cout << "\n--- Demo 6: Uptane Framework (TUF-based OTA Security) ---\n";
    {
        auto roles = ota::uptane::GetRoles();
        std::cout << "  ┌──────────────┬──────────────────────────────────────────┬──────────┐\n";
        std::cout << "  │ Role         │ Description                              │ Key ID   │\n";
        std::cout << "  ├──────────────┼──────────────────────────────────────────┼──────────┤\n";
        for (const auto& r : roles) {
            std::cout << "  │ " << std::setw(12) << std::left << r.name
                      << " │ " << std::setw(44) << r.description
                      << " │ 0x" << std::hex << std::setw(4) << std::setfill('0')
                      << r.key_id << std::dec << std::setfill(' ') << " │\n";
        }
        std::cout << std::right;
        std::cout << "  └──────────────┴──────────────────────────────────────────┴──────────┘\n";

        std::cout << "\n  Uptane update flow:\n"
                  << "    1. OEM Backend → signs image (Targets role)\n"
                  << "    2. Snapshot role → consistent metadata set\n"
                  << "    3. Timestamp role → freshness (anti-replay)\n"
                  << "    4. Director → vehicle-specific instructions\n"
                  << "    5. Primary ECU → verifies all signatures\n"
                  << "    6. Primary ECU → distributes to secondaries\n";
    }

    std::cout << "\n[All demos completed successfully.]\n";
    return 0;
}
```

► 03. Middleware Someip Dds

module_17_sdv/03_middleware_someip_dds.cpp

Konu: Middleware: SOME/IP, DDS, iceoryx — Automotive Communication

Teori / Theory

THEORY: Why Middleware?

Middleware abstracts low-level communication (sockets, shared memory) and provides service-oriented communication for distributed systems.

Without Middleware With Middleware

```
┌──────────┐              ┌──────────┐
│  App A   │              │  App A   │
│ socket() │              │ proxy->  │
│ connect()│              │  call()  │
│ send()   │              └────┬─────┘
│ recv()   │                   │ (transparent)
└────┬─────┘              ┌────┴─────┐
     │ raw TCP/UDP        │Middleware│
┌────┴─────┐              │ SOME/IP  │
│  App B   │              │  or DDS  │
└──────────┘              └────┬─────┘
                           ┌────┴─────┐
                           │  App B   │
                           │ skeleton │
                           └──────────┘
```

TEORİ: Neden Middleware?

Middleware, düşük seviye iletişimi (soket, paylaşımlı bellek) soyutlar ve dağıtık sistemler için servis tabanlı iletişim sağlar.

Middleware Olmadan Middleware İle

```
┌──────────┐              ┌──────────┐
│  Uygulama│              │  Uygulama│
│ socket() │              │ proxy->  │
│ connect()│              │  call()  │
│ send()   │              └────┬─────┘
│ recv()   │                   │ (saydam)
└────┬─────┘              ┌────┴─────┐
     │ ham TCP/UDP        │Middleware│
┌────┴─────┐              │ SOME/IP  │
│  Uygulama│              │  veya DDS│
└──────────┘              └────┬─────┘
                           ┌────┴─────┐
                           │  Uygulama│
                           │ skeleton │
                           └──────────┘
```

Ham soket kullanımı hata eğilimli ve taşınamaz, middleware soyutlar.

THEORY: SOME/IP Header Format

Byte: 0 4 8 12 16

```
┌──────────────┬──────────────┐
│ Message ID   │ Length       │
│ (Service+Meth)│ (payload+8)  │
├──────────────┼──────┬───────┤
│Request ID    │Proto │ Iface │
│(Client+Sess) │Version│Version│
├──────┬───────┼──────┴───────┤
│MsgType│RetCode│  Payload...  │
└──────┴───────┴──────────────┘
```

Message Types: REQUEST(0x00), REQUEST_NO_RETURN(0x01), NOTIFICATION(0x02), RESPONSE(0x80), ERROR(0x81)

TEORİ: SOME/IP Başlık Formatı

SOME/IP başlık formatı:
- Message ID (4 bayt): Service ID (16-bit) + Method ID (16-bit)
- Length (4 bayt): payload + 8 bayt
- Request ID (4 bayt): Client ID + Session ID
- Protocol Version, Interface Version, Message Type, Return Code Mesaj tipleri: REQUEST (istek), NOTIFICATION (bildirim), RESPONSE (yanıt), ERROR (hata).

THEORY: DDS — Data Distribution Service

```
┌──────────────────────────────────────┐
│              DDS Domain              │
│  ┌──────────┐        ┌──────────┐    │
│  │ DataWriter │──Topic──│ DataReader │    │
│  │ (Publisher)│  name  │(Subscriber)│    │
│  └──────────┘        └──────────┘    │
│        ↓                   ↑         │
│   [QoS Policies: Reliability, Durability,  │
│    Deadline, History, Lifespan]      │
└──────────────────────────────────────┘
```

TEORİ: DDS — Veri Dağıtım Servisi

DDS (Data Distribution Service), OMG standardı publish/subscribe middleware'idir. DataWriter konuya (topic) yazar, DataReader okur. QoS politikaları: Reliability (güvenilirlik), Durability (kalıcılık), Deadline, History, Lifespan — her topic bağımsız ayarlanabilir. ADAS ve robotik uygulamalarda yaygın (Fast-DDS, Cyclone DDS).

THEORY: Middleware Comparison

```
┌──────────┬──────────────┬──────────────┬──────────────┬──────────┐
│ Feature  │ SOME/IP      │ DDS          │ iceoryx      │ gRPC     │
├──────────┼──────────────┼──────────────┼──────────────┼──────────┤
│ Pattern  │ Req/Resp+Event│ Pub/Sub      │ Pub/Sub      │ Req/Resp │
│ Transport│ TCP/UDP      │ UDP multicast │ Shared memory│ HTTP/2   │
│ Discovery│ SOME/IP-SD   │ Built-in (DDP)│ RouDi daemon │ Manual   │
│ Latency  │ ~100µs       │ ~50µs        │ <1µs (0-copy)│ ~500µs   │
│ Standard │ AUTOSAR      │ OMG          │ Eclipse      │ Google   │
│ C++ API  │ vsomeip      │ Fast-DDS     │ iceoryx2     │ grpc++   │
│ Use case │ Classic auto │ ADAS, robotics│ Intra-SoC IPC│ Cloud    │
└──────────┴──────────────┴──────────────┴──────────────┴──────────┘
```

TEORİ: Ara Katman Karşılaştırması

Middleware karşılaştırması:

```
┌──────────┬──────────────┬──────────────┬──────────────┬──────────┐
│ Özellik  │ SOME/IP      │ DDS          │ iceoryx      │ gRPC     │
├──────────┼──────────────┼──────────────┼──────────────┼──────────┤
│ Desen    │ İstek/Yan+Olay│ Pub/Sub      │ Pub/Sub      │ İst/Yan  │
│ Transfer │ TCP/UDP      │ UDP multicast │ Paylaş. bellek│ HTTP/2   │
│ Keşif    │ SOME/IP-SD   │ Dahili (DDP) │ RouDi daemon │ El ile   │
│ Gecikme  │ ~100µs       │ ~50µs        │ <1µs (0kopya)│ ~500µs   │
│ Standart │ AUTOSAR      │ OMG          │ Eclipse      │ Google   │
│ C++ API  │ vsomeip      │ Fast-DDS     │ iceoryx2     │ grpc++   │
│ Kullanım │ Klasik oto   │ ADAS, robotik │ SoC içi IPC  │ Bulut    │
└──────────┴──────────────┴──────────────┴──────────────┴──────────┘
```

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <vector>
#include <string>
#include <string_view>
#include <map>
#include <unordered_map>
#include <functional>
#include <memory>
#include <optional>
#include <iomanip>
#include <algorithm>
#include <array>
#include <queue>
#include <cassert>
#include <cstring>
#include <variant>
#include <numeric>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: SOME/IP Protocol Simulation
// ═══════════════════════════════════════════════════════════════════════════

namespace someip {

// EN: SOME/IP message types (AUTOSAR PRS_SOMEIP_00055)
// TR: SOME/IP mesaj türleri
enum class MessageType : uint8_t {
    REQUEST           = 0x00,
    REQUEST_NO_RETURN = 0x01,
    NOTIFICATION      = 0x02,
    RESPONSE          = 0x80,
    ERROR             = 0x81
};
```

```cpp

enum class ReturnCode : uint8_t {
    E_OK               = 0x00,
    E_NOT_OK           = 0x01,
    E_UNKNOWN_SERVICE  = 0x02,
    E_UNKNOWN_METHOD   = 0x03,
    E_NOT_READY        = 0x04,
    E_NOT_REACHABLE    = 0x05
};

inline std::string_view MsgTypeStr(MessageType t) {
    switch (t) {
        case MessageType::REQUEST:           return "REQUEST";
        case MessageType::REQUEST_NO_RETURN: return "REQUEST_NO_RETURN";
        case MessageType::NOTIFICATION:      return "NOTIFICATION";
        case MessageType::RESPONSE:          return "RESPONSE";
        case MessageType::ERROR:             return "ERROR";
    }
    return "UNKNOWN";
}

// EN: SOME/IP header (16 bytes)
// TR: SOME/IP başlığı (16 byte)
struct SomeIpHeader {
    uint16_t    service_id;
    uint16_t    method_id;
    uint32_t    length;         // payload + 8 bytes — TR: yük + 8 bayt
    uint16_t    client_id;
    uint16_t    session_id;
    uint8_t     protocol_version;
    uint8_t     interface_version;
    MessageType message_type;
    ReturnCode  return_code;

    void Print() const {
        std::cout << "    SOME/IP Header:\n"
                  << "      Service: 0x" << std::hex << service_id
                  << "  Method: 0x" << method_id << std::dec << "\n"
                  << "      Client: " << client_id << "  Session: " << session_id << "\n"
                  << "      Type: " << MsgTypeStr(message_type)
                  << "  Return: " << static_cast<int>(return_code) << "\n";
    }
};

struct SomeIpMessage {
    SomeIpHeader         header;
    std::vector<uint8_t> payload;

    static SomeIpMessage CreateRequest(uint16_t svc, uint16_t mth,
                                       uint16_t client, uint16_t session,
                                       const std::vector<uint8_t>& data) {
        SomeIpMessage msg;
        msg.header.service_id = svc;
        msg.header.method_id = mth;
        msg.header.length = static_cast<uint32_t>(data.size() + 8);
        msg.header.client_id = client;
        msg.header.session_id = session;
        msg.header.protocol_version = 1;
        msg.header.interface_version = 1;
        msg.header.message_type = MessageType::REQUEST;
        msg.header.return_code = ReturnCode::E_OK;
        msg.payload = data;
        return msg;
    }

    static SomeIpMessage CreateResponse(const SomeIpMessage& request,
                                        const std::vector<uint8_t>& data) {
        SomeIpMessage msg;
        msg.header = request.header;
        msg.header.message_type = MessageType::RESPONSE;
        msg.header.return_code = ReturnCode::E_OK;
        msg.header.length = static_cast<uint32_t>(data.size() + 8);
        msg.payload = data;
        return msg;
    }

```

```
    static SomeIpMessage CreateNotification(uint16_t svc, uint16_t event_id,
                                            const std::vector<uint8_t>& data) {
        SomeIpMessage msg;
        msg.header.service_id = svc;
        msg.header.method_id = event_id;
        msg.header.length = static_cast<uint32_t>(data.size() + 8);
        msg.header.client_id = 0;
        msg.header.session_id = 0;
        msg.header.protocol_version = 1;
        msg.header.interface_version = 1;
        msg.header.message_type = MessageType::NOTIFICATION;
        msg.header.return_code = ReturnCode::E_OK;
        msg.payload = data;
        return msg;
    }
};

// EN: SOME/IP Service Discovery (simplified)
// TR: SOME/IP Servis Keşfi (basitleştirilmiş)
struct ServiceEntry {
    uint16_t service_id;
    uint16_t instance_id;
    std::string provider_endpoint;
};

class ServiceDiscovery {
    std::vector<ServiceEntry> offered_;

public:
    void OfferService(uint16_t svc, uint16_t inst, const std::string& endpoint) {
        offered_.push_back({svc, inst, endpoint});
        std::cout << "    [SD] OfferService: 0x" << std::hex << svc
                  << " instance=" << inst << std::dec
                  << " at " << endpoint << "\n";
    }

    std::optional<ServiceEntry> FindService(uint16_t svc) const {
        for (const auto& e : offered_) {
            if (e.service_id == svc) return e;
        }
        return std::nullopt;
    }

    [[nodiscard]] size_t Count() const { return offered_.size(); }
};

}  // namespace someip

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: DDS Simulation — Topic-Based Pub/Sub
// ═══════════════════════════════════════════════════════════════════════════

namespace dds {

// EN: QoS (Quality of Service) policies
// TR: QoS (Hizmet Kalitesi) politikaları
enum class Reliability : uint8_t { BEST_EFFORT, RELIABLE };
enum class Durability : uint8_t { VOLATILE, TRANSIENT_LOCAL, TRANSIENT, PERSISTENT };

struct QoSPolicy {
    Reliability reliability = Reliability::RELIABLE;
    Durability  durability  = Durability::VOLATILE;
    uint32_t    deadline_ms = 0;   // 0 = no deadline — TR: 0 = son tarih yok
    uint32_t    history_depth = 1;

    void Print() const {
        std::cout << "    QoS: reliability="
                  << (reliability == Reliability::RELIABLE ? "RELIABLE" : "BEST_EFFORT")
                  << " durability="
                  << static_cast<int>(durability)
                  << " deadline=" << deadline_ms << "ms"
                  << " history=" << history_depth << "\n";
    }
};
```

```cpp

// EN: DDS Sample (data + metadata)
// TR: DDS Örneği (veri + metadata)
struct Sample {
    std::string topic;
    std::vector<uint8_t> data;
    uint64_t    sequence_number;
    uint64_t    timestamp;
};

// EN: DataWriter — publishes samples on a topic
// TR: DataWriter — bir topic üzerinde örnek yayınlar
class DataWriter {
    std::string                                 topic_;
    QoSPolicy                                  qos_;
    uint64_t                                   seq_ = 0;
    std::function<void(const Sample&)>         on_write_;

public:
    DataWriter(std::string topic, QoSPolicy qos)
        : topic_(std::move(topic)), qos_(std::move(qos)) {}

    void SetCallback(std::function<void(const Sample&)> cb) {
        on_write_ = std::move(cb);
    }

    void Write(const std::vector<uint8_t>& data) {
        Sample s;
        s.topic = topic_;
        s.data = data;
        s.sequence_number = ++seq_;
        s.timestamp = 1717200000 + seq_;
        if (on_write_) on_write_(s);
    }

    [[nodiscard]] const std::string& TopicName() const { return topic_; }
    [[nodiscard]] const QoSPolicy& GetQoS() const { return qos_; }
};

// EN: DataReader — subscribes to a topic
// TR: DataReader — bir topic'e abone olur
class DataReader {
    std::string                                 topic_;
    QoSPolicy                                  qos_;
    std::vector<Sample>                        history_;

public:
    DataReader(std::string topic, QoSPolicy qos)
        : topic_(std::move(topic)), qos_(std::move(qos)) {}

    void OnData(const Sample& s) {
        if (s.topic == topic_) {
            history_.push_back(s);
            if (history_.size() > qos_.history_depth)
                history_.erase(history_.begin());
        }
    }

    [[nodiscard]] std::optional<Sample> TakeLast() const {
        if (history_.empty()) return std::nullopt;
        return history_.back();
    }

    [[nodiscard]] size_t SampleCount() const { return history_.size(); }
    [[nodiscard]] const std::string& TopicName() const { return topic_; }
};

// EN: DDS Participant — manages writers and readers
// TR: DDS Participant — writer ve reader'ları yönetir
class DomainParticipant {
    std::string name_;
    std::vector<DataWriter*>                writers_;
    std::vector<DataReader*>                readers_;
    // EN: Topic → readers mapping for routing
    // TR: Konu → okuyucu eşlemesi — yayınlanan veriyi ilgili DataReader'lara yönlendirir
    std::unordered_map<std::string, std::vector<DataReader*>> topic_readers_;

```

```
public:
    explicit DomainParticipant(std::string name) : name_(std::move(name)) {}

    void RegisterWriter(DataWriter* w) {
        writers_.push_back(w);
        w->SetCallback([this](const Sample& s) { Route(s); });
        std::cout << "  [DDS:" << name_ << "] Writer registered: " << w->TopicName() << "\n";
    }

    void RegisterReader(DataReader* r) {
        readers_.push_back(r);
        topic_readers_[r->TopicName()].push_back(r);
        std::cout << "  [DDS:" << name_ << "] Reader registered: " << r->TopicName() << "\n";
    }

    void Route(const Sample& s) {
        auto it = topic_readers_.find(s.topic);
        if (it != topic_readers_.end()) {
            for (auto* reader : it->second) {
                reader->OnData(s);
            }
        }
    }
};

}  // namespace dds

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: iceoryx — Zero-Copy Shared Memory IPC
// ═══════════════════════════════════════════════════════════════════════════

namespace iceoryx_sim {

// EN: Simulated shared memory segment (no real shm, just a buffer)
// TR: Simüle edilmiş paylaşımlı bellek segmenti
class SharedMemorySegment {
    static constexpr size_t SEGMENT_SIZE = 4096;
    std::array<uint8_t, SEGMENT_SIZE> memory_{};
    size_t write_pos_ = 0;

public:
    // EN: "Loan" a chunk — publisher gets pointer, writes, then publishes
    // TR: Bir chunk "ödünç al" — yayıncı pointer alır, yazar, yayınlar
    struct Chunk {
        uint8_t* data;
        size_t   size;
        size_t   offset;
    };

    std::optional<Chunk> Loan(size_t size) {
        if (write_pos_ + size > SEGMENT_SIZE) return std::nullopt;
        Chunk c;
        c.data = &memory_[write_pos_];
        c.size = size;
        c.offset = write_pos_;
        write_pos_ += size;
        return c;
    }

    const uint8_t* Read(size_t offset) const {
        if (offset >= SEGMENT_SIZE) return nullptr;
        return &memory_[offset];
    }
};

// EN: iceoryx Publisher (zero-copy)
// TR: iceoryx Yayıncı (sıfır kopyalama)
class Publisher {
    std::string         topic_;
    SharedMemorySegment& shm_;
    std::function<void(const std::string&, size_t, size_t)> notify_;

```

```
public:
    Publisher(std::string topic, SharedMemorySegment& shm)
        : topic_(std::move(topic)), shm_(shm) {}

    void SetNotifyCallback(std::function<void(const std::string&, size_t, size_t)> cb) {
        notify_ = std::move(cb);
    }

    bool Publish(const void* data, size_t size) {
        auto chunk = shm_.Loan(size);
        if (!chunk) return false;
        std::memcpy(chunk->data, data, size);
        std::cout << "    [iceoryx] Published " << size << " bytes on '" << topic_
                  << "' (zero-copy @ offset " << chunk->offset << ")\n";
        if (notify_) notify_(topic_, chunk->offset, chunk->size);
        return true;
    }
};

// EN: iceoryx Subscriber (zero-copy read)
// TR: iceoryx Abone (sıfır kopyalama okuma)
class Subscriber {
    std::string          topic_;
    SharedMemorySegment& shm_;
    size_t               last_offset_ = 0;
    size_t               last_size_ = 0;
    bool                 has_data_ = false;

public:
    Subscriber(std::string topic, SharedMemorySegment& shm)
        : topic_(std::move(topic)), shm_(shm) {}

    void OnNotify(const std::string& topic, size_t offset, size_t size) {
        if (topic == topic_) {
            last_offset_ = offset;
            last_size_ = size;
            has_data_ = true;
        }
    }

    const uint8_t* Take(size_t& out_size) {
        if (!has_data_) { out_size = 0; return nullptr; }
        has_data_ = false;
        out_size = last_size_;
        return shm_.Read(last_offset_);
    }
};

// EN: RouDi (Routing and Discovery daemon) — simplified
// TR: RouDi (Yönlendirme ve Keşif daemon'u) — basitleştirilmiş
class RouDi {
    SharedMemorySegment shm_;
    std::vector<Publisher*>  publishers_;
    std::vector<Subscriber*> subscribers_;

public:
    SharedMemorySegment& GetShm() { return shm_; }

    void RegisterPublisher(Publisher* pub) {
        pub->SetNotifyCallback([this](const std::string& topic, size_t off, size_t sz) {
            for (auto* sub : subscribers_) {
                sub->OnNotify(topic, off, sz);
            }
        });
        publishers_.push_back(pub);
    }

    void RegisterSubscriber(Subscriber* sub) {
        subscribers_.push_back(sub);
    }
};

}  // namespace iceoryx_sim
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════
// PART 4: Mini Pub/Sub Middleware with Topic Routing + QoS
// ═══════════════════════════════════════════════════════════════════════════

namespace mini_middleware {

enum class QoSReliability : uint8_t { BEST_EFFORT, RELIABLE };

struct TopicMessage {
    std::string              topic;
    std::vector<uint8_t>     data;
    uint64_t                 seq = 0;
};

class MiddlewareBroker {
    struct SubscriberEntry {
        std::string topic;
        QoSReliability qos;
        std::function<void(const TopicMessage&)> callback;
    };

    std::vector<SubscriberEntry>  subscribers_;
    uint64_t global_seq_ = 0;

public:
    void Subscribe(const std::string& topic, QoSReliability qos,
                   std::function<void(const TopicMessage&)> cb) {
        subscribers_.push_back({topic, qos, std::move(cb)});
        std::cout << "  [Broker] Subscribed to '" << topic << "'\n";
    }

    void Publish(const std::string& topic, const std::vector<uint8_t>& data) {
        ++global_seq_;
        TopicMessage msg{topic, data, global_seq_};
        for (auto& sub : subscribers_) {
            if (sub.topic == topic) {
                // EN: Simulate BEST_EFFORT having 20% drop chance
                // TR: BEST_EFFORT için %20 kayıp simülasyonu — gerçek ağda UDP paket kaybını yansıtır
                if (sub.qos == QoSReliability::BEST_EFFORT && (global_seq_ % 5 == 0)) {
                    std::cout << "    [Broker] DROPPED (best-effort) seq=" << global_seq_ << "\n";
                    continue;
                }
                sub.callback(msg);
            }
        }
    }
};

}  // namespace mini_middleware

// ═══════════════════════════════════════════════════════════════════════════
// MAIN — Demos
// ═══════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "╔═══════════════════════════════════════════════════════════╗\n"
              << "║ Module 17: Middleware — SOME/IP, DDS, iceoryx             ║\n"
              << "╚═══════════════════════════════════════════════════════════╝\n\n";

    // --- Demo 1: SOME/IP Message Exchange ---
    std::cout << "--- Demo 1: SOME/IP — Request/Response + Notification ---\n";
    {
        // EN: Create a SOME/IP request for GetVehicleSpeed (service=0x1234, method=0x0001)
        // TR: GetVehicleSpeed için SOME/IP isteği oluştur (service=0x1234, method=0x0001)
        std::vector<uint8_t> req_payload = {0x00};  // empty request
        auto request = someip::SomeIpMessage::CreateRequest(
            0x1234, 0x0001, 100, 1, req_payload);
        std::cout << "  Request:\n";
        request.header.Print();

```

```cpp
        // EN: Server processes and responds with speed=120 km/h
        // TR: Sunucu işler ve hız=120 km/h ile yanıt verir (pozitif yanıt, MsgType=0x80)
        std::vector<uint8_t> resp_payload = {0x00, 0x78};  // 120 in big-endian 16-bit
        auto response = someip::SomeIpMessage::CreateResponse(request, resp_payload);
        std::cout << "  Response:\n";
        response.header.Print();
        uint16_t speed = static_cast<uint16_t>((resp_payload[0] << 8) | resp_payload[1]);
        std::cout << "    Payload: speed = " << speed << " km/h\n";

        // EN: Event notification (speed changed)
        // TR: Olay bildirimi (hız değişti) — MsgType=NOTIFICATION ile abone olan istemcilere gönderilir
        std::vector<uint8_t> event_data = {0x00, 0x82};  // 130 km/h
        auto notif = someip::SomeIpMessage::CreateNotification(0x1234, 0x8001, event_data);
        std::cout << "  Notification:\n";
        notif.header.Print();
    }

    // --- Demo 2: SOME/IP Service Discovery ---
    std::cout << "\n--- Demo 2: SOME/IP Service Discovery ---\n";
    {
        someip::ServiceDiscovery sd;
        sd.OfferService(0x1234, 1, "192.168.1.10:30490");
        sd.OfferService(0x5678, 1, "192.168.1.20:30491");
        sd.OfferService(0xABCD, 2, "192.168.1.30:30492");

        auto found = sd.FindService(0x5678);
        if (found) {
            std::cout << "    Found: service=0x" << std::hex << found->service_id
                      << " at " << found->provider_endpoint << std::dec << "\n";
        }

        auto not_found = sd.FindService(0x9999);
        std::cout << "    FindService(0x9999): "
                  << (not_found ? "found" : "NOT_FOUND") << "\n";
    }

    // --- Demo 3: DDS Topic-Based Pub/Sub ---
    std::cout << "\n--- Demo 3: DDS — Topic Pub/Sub with QoS ---\n";
    {
        dds::DomainParticipant participant("Vehicle");

        dds::QoSPolicy writer_qos{dds::Reliability::RELIABLE, dds::Durability::VOLATILE, 100, 1};
        dds::DataWriter writer("VehicleSpeed", writer_qos);

        dds::QoSPolicy reader_qos{dds::Reliability::RELIABLE, dds::Durability::VOLATILE, 100, 5};
        dds::DataReader reader1("VehicleSpeed", reader_qos);
        dds::DataReader reader2("VehicleSpeed", reader_qos);

        participant.RegisterWriter(&writer);
        participant.RegisterReader(&reader1);
        participant.RegisterReader(&reader2);

        writer.GetQoS().Print();

        // EN: Publish 3 speed values
        // TR: 3 hız değeri yayınla — DataWriter aracılığıyla topic'e veri yazar
        for (uint32_t spd : {60u, 80u, 120u}) {
            std::vector<uint8_t> data(4);
            std::memcpy(data.data(), &spd, 4);
            writer.Write(data);
        }

        // EN: Check reader state
        // TR: Okuyucu durumunu kontrol et — son alınan örneği tampondan çıkar
        auto sample = reader1.TakeLast();
        if (sample) {
            uint32_t received_speed = 0;
            std::memcpy(&received_speed, sample->data.data(), 4);
            std::cout << "  Reader1 last sample: speed=" << received_speed
                      << " km/h, seq=" << sample->sequence_number << "\n";
        }
        std::cout << "  Reader1 samples buffered: " << reader1.SampleCount() << "\n";
        std::cout << "  Reader2 samples buffered: " << reader2.SampleCount() << "\n";
    }
```

```cpp

    // --- Demo 4: iceoryx Zero-Copy ---
    std::cout << "\n--- Demo 4: iceoryx — Zero-Copy Shared Memory IPC ---\n";
    {
        iceoryx_sim::RouDi roudi;

        iceoryx_sim::Publisher  pub("LidarPointCloud", roudi.GetShm());
        iceoryx_sim::Subscriber sub("LidarPointCloud", roudi.GetShm());

        roudi.RegisterPublisher(&pub);
        roudi.RegisterSubscriber(&sub);

        // EN: Simulate publishing a pointcloud (1KB)
        // TR: 1KB'lık nokta bulutu yayınlama simülasyonu — paylaşımlı belleğe kopyalanır
        struct PointCloud {
            uint32_t num_points;
            float    min_range;
            float    max_range;
        };
        PointCloud cloud{1024, 0.5f, 100.0f};
        pub.Publish(&cloud, sizeof(cloud));

        // EN: Subscriber reads — zero copy (pointer into shared memory)
        // TR: Abone okur — sıfır kopya (paylaşımlı bellek işaretçisi ile doğrudan erişim)
        size_t sz = 0;
        const auto* ptr = sub.Take(sz);
        if (ptr != nullptr) {
            PointCloud received{};
            std::memcpy(&received, ptr, sizeof(received));
            std::cout << "    [Sub] Received: points=" << received.num_points
                      << " range=[" << received.min_range << ", " << received.max_range << "]\n";
            std::cout << "    Zero-copy: data read directly from shared memory "
                      << "(no memcpy between processes in real iceoryx)\n";
        }
    }

    // --- Demo 5: Mini Middleware Broker with QoS ---
    std::cout << "\n--- Demo 5: Mini Middleware Broker (Topic + QoS) ---\n";
    {
        mini_middleware::MiddlewareBroker broker;

        int reliable_count = 0;
        int best_effort_count = 0;

        broker.Subscribe("radar/objects", mini_middleware::QoSReliability::RELIABLE,
            [&](const mini_middleware::TopicMessage& m) {
                ++reliable_count;
                std::cout << "    [RELIABLE] Received seq=" << m.seq << "\n";
            });

        broker.Subscribe("radar/objects", mini_middleware::QoSReliability::BEST_EFFORT,
            [&](const mini_middleware::TopicMessage& m) {
                ++best_effort_count;
                std::cout << "    [BEST_EFFORT] Received seq=" << m.seq << "\n";
            });

        std::cout << "  Publishing 10 messages:\n";
        for (int i = 0; i < 10; ++i) {
            broker.Publish("radar/objects", {static_cast<uint8_t>(i)});
        }

        std::cout << "  Reliable received: " << reliable_count << "/10\n";
        std::cout << "  Best-effort received: " << best_effort_count << "/10\n";
    }

    // --- Demo 6: Middleware Comparison Matrix ---
    std::cout << "\n--- Demo 6: Middleware Selection Guide ---\n";
    {
        struct MWEntry { std::string_view name; std::string_view best_for; std::string_view latency;
};
        MWEntry entries[] = {
            {"SOME/IP (vsomeip)",   "AUTOSAR Classic/Adaptive, standard automotive", "~100us"},
            {"DDS (Fast-DDS)",      "ADAS, robotics, pub/sub heavy workloads",       "~50us"},
            {"iceoryx",             "Intra-SoC zero-copy (lidar/camera data)",       "<1us"},
            {"gRPC",                "Cloud backend, REST-like services",              "~500us"},
            {"ROS 2 (rmw_dds)",     "ADAS prototyping, ROS ecosystem",               "~50us"},
        };
```

```
        std::cout << "  ┌──────────────────┬──────────────────────────────────────────┬──────────┐\n";
        std::cout << "  │ Middleware       │ Best For                                 │ Latency │\n";
        std::cout << "  ├──────────────────┼──────────────────────────────────────────┼──────────┤\n";
        for (const auto& e : entries) {
            std::cout << "  │ " << std::setw(18) << std::left << e.name
                      << " │ " << std::setw(46) << e.best_for
                      << " │ " << std::setw(7) << e.latency << " │\n";
        }
        std::cout << std::right;
        std::cout << "  └──────────────────┴──────────────────────────────────────────┴──────────┘\n";
    }

    std::cout << "\n[All demos completed successfully.]\n";
    return 0;
}
```

Modül 18: Driver Dev — Sürücü Geliştirme

Sürücü Geliştirme

4 kaynak dosya · Standart: C++20

C++20

► 01. Driver Hal Bsp

module_18_driver_development/01_driver_hal_bsp.cpp

Konu: Driver / HAL / BSP Layers — C vs C++ in Driver Development

Teori / Theory

THEORY: Driver Layer Hierarchy

```
┌──────────────────────────────────────────────────┐
│                Application Layer                 │
│  (C++ classes, business logic, AUTOSAR SWC)      │
├──────────────────────────────────────────────────┤
│                Framework / Middleware            │
│  (C++: SOME/IP, DDS, Qt, Android HAL)            │
├──────────────────────────────────────────────────┤
│                HAL (C++ or C)                    │
│  Hardware Abstraction Layer                      │
│  Platform-independent API                        │
├──────────────────────────────────────────────────┤
│                Driver (C)                        │
│  Register-level access, interrupt handlers       │
├──────────────────────────────────────────────────┤
│                BSP (Board Support Package)       │
│  Clock config, pin mux, memory map, startup code │
├──────────────────────────────────────────────────┤
│                Hardware (MCU / SoC)              │
└──────────────────────────────────────────────────┘
```

TEORİ: Sürücü Katman Hiyerarşisi

Sürücü katman hiyerarşisi (alttan üste):

```
┌──────────────────────────────────────────────┐
│                Uygulama Katmanı              │
│  (C++ sınıfları, iş mantığı, AUTOSAR SWC)    │
├──────────────────────────────────────────────┤
│              Framework / Middleware          │
│  (C++: SOME/IP, DDS, Qt, Android HAL)        │
├──────────────────────────────────────────────┤
│              HAL (C++ veya C)                │
│  Donanım Soyutlama Katmanı — Platform bağımsız API│
├──────────────────────────────────────────────┤
│              Sürücü (C)                      │
│  Yazmaç seviyesi erişim, kesme işleyicileri  │
├──────────────────────────────────────────────┤
│              BSP (Board Support Package)     │
│  Saat yapılandırma, pin mux, bellek haritası │
├──────────────────────────────────────────────┤
│              Donanım (MCU / SoC)             │
└──────────────────────────────────────────────┘
```

Alt katmanlar genelde C, üst katmanlar C++ ile yazılır.

THEORY: C vs C++ in Driver Development

```
┌──────────────────┬──────────────────┬──────────────────┐
│ Aspect           │ C                │ C++              │
├──────────────────┼──────────────────┼──────────────────┤
│ Kernel driver    │ □ Required       │ □ Not allowed    │
│ MCAL driver      │ □ Standard       │ ⚠□ Wrapper only  │
│ HAL abstraction  │ □ Common         │ □ CRTP, templates│
│ BSP init         │ □ Standard       │ ⚠□ constexpr conf │
│ Framework        │ □ Limited        │ □ Preferred      │
│ Application SWC  │ ⚠□ Possible      │ □ Standard       │
├──────────────────┼──────────────────┼──────────────────┤
│ ABI stability    │ □ Guaranteed     │ □ Name mangling  │
│ No exceptions    │ □ Default        │ ⚠□ -fno-exception │
│ No RTTI          │ □ Default        │ ⚠□ -fno-rtti     │
│ No hidden alloc  │ □ Explicit       │ ⚠□ Must be aware │
│ MISRA compliant  │ □ MISRA C 2012   │ □ MISRA C++ 2023 │
└──────────────────┴──────────────────┴──────────────────┘
```

TEORİ: Sürücü Geliştirmede C ve C++

C vs C++ sürücü geliştirmede:

Yön	C	C++
Kernel sürücü	□ Zorunlu	□ İzin verilmez
MCAL sürücü	□ Standart	⚠□ Yalnızca sarmal
HAL soyutlama	□ Yaygın	□ CRTP, template
BSP init	□ Standart	⚠□ constexpr conf
Framework	□ Sınırlı	□ Tercih edilir
Uygulama SWC	⚠□ Mümkün	□ Standart
ABI kararlılığı	□ Garantili	□ İsim karışt.
İstisna yok	□ Varsayılan	⚠□ -fno-exception
RTTI yok	□ Varsayılan	⚠□ -fno-rtti
Gizli allok yok	□ Açık	⚠□ Dikkat gerekli
MISRA uyumlu	□ MISRA C 2012	□ MISRA C++ 2023

THEORY: extern "C" Boundary — C Driver + C++ Wrapper

When C++ code calls a C driver, the boundary is marked with extern "C" to prevent C++ name mangling and ensure ABI compatibility.
// C header (gpio_driver.h) #ifdef __cplusplus extern "C" { #endif void GPIO_Init(uint32_t port, uint32_t pin, uint32_t mode); void GPIO_Write(uint32_t port, uint32_t pin, uint8_t value); uint8_t GPIO_Read(uint32_t port, uint32_t pin); #ifdef __cplusplus } #endif

TEORİ: extern "C" Sınırı — C Sürücü + C++ Sarmalayıcı

C++ kodu C sürücüsünü çağırdığında, sınır extern "C" ile işaretlenir. Bu sayede C++ isim karıştırması (name mangling) engellenir ve ABI uyumluluğu sağlanır.
// C başlık dosyası (gpio_driver.h) #ifdef __cplusplus extern "C" { #endif void GPIO_Init(uint32_t port, uint32_t pin, uint32_t mode); void GPIO_Write(uint32_t port, uint32_t pin, uint8_t value); uint8_t GPIO_Read(uint32_t port, uint32_t pin); #ifdef __cplusplus } #endif

THEORY: Vendor SDK Patterns

Vendor / SDK	Language	Pattern
STM32 HAL	C	HAL_xxx_Init(), callbacks
Zephyr RTOS	C	Device tree + driver model
Android HAL	C++	HIDL → AIDL interface
QNX Resource Mgr	C	open/read/write/ioctl
AUTOSAR MCAL	C	xxx_Init(), xxx_Read()
Nordic nRF SDK	C	nrfx_xxx + app_xxx
Linux Kernel	C	file_operations, platform_drv

TEORİ: Üretici SDK Kalıpları

Satıcı SDK kalıpları:

Satıcı / SDK	Dil	Desen
STM32 HAL	C	HAL_xxx_Init(), callback'ler
Zephyr RTOS	C	Device tree + sürücü modeli
Android HAL	C++	HIDL → AIDL arayüzü
QNX Resource Mgr	C	open/read/write/ioctl
AUTOSAR MCAL	C	xxx_Init(), xxx_Read()
Nordic nRF SDK	C	nrfx_xxx + app_xxx
Linux Kernel	C	file_operations, platform_drv

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <string>
#include <string_view>
#include <array>
#include <vector>
#include <iomanip>
#include <functional>
#include <optional>
#include <cassert>
#include <cstring>
#include <memory>

// ═══════════════════════════════════════════════════════════════════════
// PART 1: C-Style Register-Level Driver (simulated)
// ═══════════════════════════════════════════════════════════════════════

// EN: Simulated hardware registers (in real HW these are memory-mapped)
// TR: Simüle edilmiş donanım register'ları (gerçekte memory-mapped)

namespace c_driver {

// EN: Simulated GPIO register block
// TR: Simüle edilmiş GPIO register bloğu
struct GPIO_RegBlock {
    uint32_t MODER;    // Mode register (input/output/AF/analog) — TR: Mod yazmaçı
    uint32_t ODR;      // Output data register — TR: Çıkış veri yazmaçı
    uint32_t IDR;      // Input data register — TR: Giriş veri yazmaçı
    uint32_t BSRR;     // Bit set/reset register — TR: Bit set/reset yazmaçı
};

// EN: Simulated GPIO peripheral instances
// TR: Simüle edilmiş GPIO çevre birimi örnekleri — gerçek HW'de sabit adreslerdeki register bloklardır
static GPIO_RegBlock GPIOA_REGS = {0, 0, 0, 0};
static GPIO_RegBlock GPIOB_REGS = {0, 0, 0, 0};

// EN: C-style driver API (extern "C" compatible)
// TR: C tarzı sürücü API'si (extern "C" uyumlu)

enum GPIO_Mode : uint32_t { GPIO_MODE_INPUT = 0, GPIO_MODE_OUTPUT = 1, GPIO_MODE_AF = 2, GPIO_MODE_ANALOG = 3 };

inline GPIO_RegBlock* GPIO_GetPort(uint32_t port) {
    switch (port) {
        case 0: return &GPIOA_REGS;
        case 1: return &GPIOB_REGS;
        default: return nullptr;
    }
}
```

```
49
50  inline int GPIO_Init(uint32_t port, uint32_t pin, GPIO_Mode mode) {
51      if (pin > 15) return -1;
52      auto* regs = GPIO_GetPort(port);
53      if (!regs) return -1;
54      uint32_t shift = pin * 2;
55      regs->MODER &= ~(0x3u << shift);
56      regs->MODER |= (static_cast<uint32_t>(mode) & 0x3u) << shift;
57      return 0;
58  }
59
60  inline int GPIO_Write(uint32_t port, uint32_t pin, uint8_t value) {
61      if (pin > 15) return -1;
62      auto* regs = GPIO_GetPort(port);
63      if (!regs) return -1;
64      if (value != 0)
65          regs->ODR |= (1u << pin);
66      else
67          regs->ODR &= ~(1u << pin);
68      return 0;
69  }
70
71  inline uint8_t GPIO_Read(uint32_t port, uint32_t pin) {
72      if (pin > 15) return 0;
73      auto* regs = GPIO_GetPort(port);
74      if (!regs) return 0;
75      return ((regs->IDR >> pin) & 1u) != 0 ? 1 : 0;
76  }
77
78  inline void GPIO_Toggle(uint32_t port, uint32_t pin) {
79      if (pin > 15) return;
80      auto* regs = GPIO_GetPort(port);
81      if (!regs) return;
82      regs->ODR ^= (1u << pin);
83  }
84
85  }  // namespace c_driver
86
87  // ═══════════════════════════════════════════════════════════════════════
88  // PART 2: C++ CRTP HAL Wrapper
89  // ═══════════════════════════════════════════════════════════════════════
90
91  namespace cpp_hal {
92
93  // EN: Pin mode abstraction
94  // TR: Pin modu soyutlaması
95  enum class PinMode : uint8_t { INPUT, OUTPUT, AF, ANALOG };
96
97  // EN: HAL result type (no exceptions)
98  // TR: HAL sonuç türü (istisna yok)
99  enum class HAL_Status : uint8_t { OK, ERROR, INVALID_PARAM };
100
101 // EN: CRTP base — provides common interface without virtual dispatch
102 // TR: CRTP tabanı — virtual dispatch olmadan ortak arayüz sağlar
103 template <typename Derived>
104 class GPIOBase {
105 public:
106     HAL_Status Init(uint32_t port, uint32_t pin, PinMode mode) {
107         return static_cast<Derived*>(this)->InitImpl(port, pin, mode);
108     }
109     HAL_Status Write(uint32_t port, uint32_t pin, bool value) {
110         return static_cast<Derived*>(this)->WriteImpl(port, pin, value);
111     }
112     bool Read(uint32_t port, uint32_t pin) {
113         return static_cast<Derived*>(this)->ReadImpl(port, pin);
114     }
115     void Toggle(uint32_t port, uint32_t pin) {
116         static_cast<Derived*>(this)->ToggleImpl(port, pin);
117     }
118 };
```

```cpp

// EN: STM32-like GPIO implementation (wraps C driver)
// TR: STM32 benzeri GPIO implementasyonu (C sürücüsünü sarar)
class STM32_GPIO : public GPIOBase<STM32_GPIO> {
public:
    HAL_Status InitImpl(uint32_t port, uint32_t pin, PinMode mode) {
        auto c_mode = static_cast<c_driver::GPIO_Mode>(static_cast<uint8_t>(mode));
        int ret = c_driver::GPIO_Init(port, pin, c_mode);
        return ret == 0 ? HAL_Status::OK : HAL_Status::INVALID_PARAM;
    }

    HAL_Status WriteImpl(uint32_t port, uint32_t pin, bool value) {
        int ret = c_driver::GPIO_Write(port, pin, value ? 1 : 0);
        return ret == 0 ? HAL_Status::OK : HAL_Status::INVALID_PARAM;
    }

    bool ReadImpl(uint32_t port, uint32_t pin) {
        return c_driver::GPIO_Read(port, pin) != 0;
    }

    void ToggleImpl(uint32_t port, uint32_t pin) {
        c_driver::GPIO_Toggle(port, pin);
    }
};

// EN: NXP-like GPIO implementation (different register layout, same HAL interface)
// TR: NXP benzeri GPIO implementasyonu (farklı register düzeni, aynı HAL arayüzü)
class NXP_GPIO : public GPIOBase<NXP_GPIO> {
    // EN: NXP uses different register names but same concept
    // TR: NXP farklı register isimleri kullanır ama aynı konsept (PDDR=yön, PDOR=çıkış, PDIR=giriş)
    struct NXP_Regs {
        uint32_t PDDR;  // Port Data Direction Register — TR: Port yön yazmaçı
        uint32_t PDOR;  // Port Data Output Register — TR: Port çıkış yazmaçı
        uint32_t PDIR;  // Port Data Input Register — TR: Port giriş yazmaçı
    };
    std::array<NXP_Regs, 2> ports_{};

public:
    HAL_Status InitImpl(uint32_t port, uint32_t pin, PinMode mode) {
        if (port > 1 || pin > 31) return HAL_Status::INVALID_PARAM;
        if (mode == PinMode::OUTPUT)
            ports_[port].PDDR |= (1u << pin);
        else
            ports_[port].PDDR &= ~(1u << pin);
        return HAL_Status::OK;
    }

    HAL_Status WriteImpl(uint32_t port, uint32_t pin, bool value) {
        if (port > 1 || pin > 31) return HAL_Status::INVALID_PARAM;
        if (value)
            ports_[port].PDOR |= (1u << pin);
        else
            ports_[port].PDOR &= ~(1u << pin);
        return HAL_Status::OK;
    }

    bool ReadImpl(uint32_t port, uint32_t pin) {
        if (port > 1 || pin > 31) return false;
        // EN: For output pins, PDIR reflects PDOR (simulated loopback)
        // TR: Çıkış pinlerinde PDIR, PDOR'u yansıtır (simüle edilmiş geri dönüş)
        ports_[port].PDIR = ports_[port].PDOR;
        return (ports_[port].PDIR >> pin) & 1u;
    }

    void ToggleImpl(uint32_t port, uint32_t pin) {
        if (port > 1 || pin > 31) return;
        ports_[port].PDOR ^= (1u << pin);
    }
};

// EN: Generic function that works with any GPIO implementation (compile-time polymorphism)
// TR: Herhangi bir GPIO implementasyonuyla çalışan genel fonksiyon (derleme zamanı polimorfizm)
template <typename GPIODriver>
void BlinkLED(GPIOBase<GPIODriver>& gpio, uint32_t port, uint32_t pin, int count) {
    gpio.Init(port, pin, PinMode::OUTPUT);
    for (int i = 0; i < count; ++i) {
        gpio.Toggle(port, pin);
```

```cpp
        auto state = gpio.Read(port, pin);
        std::cout << "    Blink " << (i + 1) << "/" << count << " → LED "
                  << (state ? "ON" : "OFF") << "\n";
    }
}

}  // namespace cpp_hal

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: BSP — Board Support Package Pattern
// ═══════════════════════════════════════════════════════════════════════════

namespace bsp {

// EN: Board configuration (compile-time via constexpr)
// TR: Board konfigürasyonu (constexpr ile derleme zamanı)
struct PinConfig {
    uint32_t port;
    uint32_t pin;
    cpp_hal::PinMode mode;
    std::string_view label;
};

// EN: Board: "STM32F4 Discovery" pin mapping
// TR: "STM32F4 Discovery" board pin haritası
constexpr std::array<PinConfig, 6> DISCOVERY_BOARD_PINS = {{
    {3, 12, cpp_hal::PinMode::OUTPUT, "LED_GREEN"},
    {3, 13, cpp_hal::PinMode::OUTPUT, "LED_ORANGE"},
    {3, 14, cpp_hal::PinMode::OUTPUT, "LED_RED"},
    {3, 15, cpp_hal::PinMode::OUTPUT, "LED_BLUE"},
    {0,  0, cpp_hal::PinMode::INPUT,  "USER_BUTTON"},
    {0,  5, cpp_hal::PinMode::AF,     "SPI1_SCK"},
}};

// EN: BSP init function — configures all board pins
// TR: BSP başlatma fonksiyonu — tüm board pinlerini yapılandırır
template <typename GPIODriver, size_t N>
void BSP_Init(cpp_hal::GPIOBase<GPIODriver>& gpio,
              const std::array<PinConfig, N>& config) {
    std::cout << "  [BSP] Initializing " << N << " pins:\n";
    for (const auto& pin : config) {
        auto status = gpio.Init(pin.port, pin.pin, pin.mode);
        std::cout << "    " << pin.label << " (P" << pin.port
                  << "." << pin.pin << ") → "
                  << (status == cpp_hal::HAL_Status::OK ? "OK" : "ERROR") << "\n";
    }
}

}  // namespace bsp

// ═══════════════════════════════════════════════════════════════════════════
// PART 4: Linux Device Driver Model (Conceptual)
// ═══════════════════════════════════════════════════════════════════════════

namespace linux_driver {

// EN: Simulated Linux char device file_operations
// TR: Simüle edilmiş Linux karakter cihaz dosya_operasyonları
struct FileOperations {
    std::function<int()>                                open;
    std::function<int()>                                release;
    std::function<int(uint8_t* buf, size_t count)>      read;
    std::function<int(const uint8_t* buf, size_t count)> write;
    std::function<int(uint32_t cmd, uint64_t arg)>      ioctl;
};

// EN: Example: Temperature sensor char device
// TR: Örnek: Sıcaklık sensörü karakter cihazı
class TempSensorDevice {
    bool    opened_ = false;
    int32_t temperature_ = 2500;  // 25.00 °C in centi-degrees — TR: santi-derece cinsinden

public:
    FileOperations GetOps() {
        FileOperations ops;
        ops.open = [this]() {
            opened_ = true;
```

```
            std::cout << "    [drv] /dev/temp0 opened\n";
            return 0;
        };
        ops.release = [this]() {
            opened_ = false;
            std::cout << "    [drv] /dev/temp0 closed\n";
            return 0;
        };
        ops.read = [this](uint8_t* buf, size_t count) -> int {
            if (!opened_) return -1;
            if (count < sizeof(temperature_)) return -1;
            std::memcpy(buf, &temperature_, sizeof(temperature_));
            return static_cast<int>(sizeof(temperature_));
        };
        ops.write = [](const uint8_t* /*buf*/, size_t /*count*/) -> int {
            return -1;  // read-only device
        };
        ops.ioctl = [this](uint32_t cmd, uint64_t arg) -> int {
            if (cmd == 0x01) {  // SET_CALIBRATION
                temperature_ += static_cast<int32_t>(arg);
                std::cout << "    [drv] Calibration applied: temp="
                          << temperature_ << " centi-°C\n";
                return 0;
            }
            return -1;
        };
        return ops;
    }
};

// EN: Userspace C++ wrapper (RAII for open/close)
// TR: Kullanıcı alanı C++ sarmalayıcı (open/close için RAII)
class DeviceFile {
    FileOperations ops_;
    bool opened_ = false;

public:
    explicit DeviceFile(FileOperations ops) : ops_(std::move(ops)) {}
    ~DeviceFile() { if (opened_) Close(); }

    DeviceFile(const DeviceFile&) = delete;
    DeviceFile& operator=(const DeviceFile&) = delete;

    int Open() {
        int ret = ops_.open();
        if (ret == 0) opened_ = true;
        return ret;
    }
    int Close() {
        if (!opened_) return -1;
        int ret = ops_.release();
        opened_ = false;
        return ret;
    }
    int Read(uint8_t* buf, size_t count) {
        return ops_.read(buf, count);
    }
    int Ioctl(uint32_t cmd, uint64_t arg) {
        return ops_.ioctl(cmd, arg);
    }
};

}  // namespace linux_driver
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════
// PART 5: extern "C" Boundary Demonstration
// ═══════════════════════════════════════════════════════════════════════════

namespace extern_c_demo {

// EN: Simulated C API (in real code, these would be in a .h with extern "C")
// TR: Simüle edilmiş C API'si (gerçekte .h dosyasında extern "C" ile olur)

// Functions declared as if they were C functions
extern "C" {
    // EN: These represent the C driver functions
    // TR: Bunlar C sürücü fonksiyonlarını temsil eder — extern "C" ile C bağlantı uyumu sağlanır
    inline int UART_Init(uint32_t baudrate) {
        std::cout << "    [C-DRV] UART initialized at " << baudrate << " baud\n";
        return 0;
    }
    inline int UART_Transmit(const uint8_t* data, uint32_t len) {
        std::cout << "    [C-DRV] UART TX " << len << " bytes: ";
        for (uint32_t i = 0; i < len && i < 16; ++i)
            std::cout << std::hex << static_cast<int>(data[i]) << " ";
        std::cout << std::dec << "\n";
        return static_cast<int>(len);
    }
    inline int UART_Receive(uint8_t* data, uint32_t max_len) {
        // EN: Simulate receiving "OK\r\n"
        // TR: "OK\r\n" alma simülasyonu — gerçek HW'de UART RX FIFO'dan okunur
        const uint8_t mock[] = {'O', 'K', '\r', '\n'};
        auto copy_len = std::min(max_len, static_cast<uint32_t>(sizeof(mock)));
        std::memcpy(data, mock, copy_len);
        return static_cast<int>(copy_len);
    }
}

// EN: C++ RAII wrapper for the C UART driver
// TR: C UART sürücüsü için C++ RAII sarmalayıcı
class UARTPort {
    uint32_t baudrate_;
    bool     initialized_ = false;

public:
    explicit UARTPort(uint32_t baud) : baudrate_(baud) {
        if (UART_Init(baudrate_) == 0)
            initialized_ = true;
    }

    [[nodiscard]] bool IsReady() const { return initialized_; }

    int Send(std::string_view msg) {
        if (!initialized_) return -1;
        return UART_Transmit(reinterpret_cast<const uint8_t*>(msg.data()),
                             static_cast<uint32_t>(msg.size()));
    }

    std::optional<std::string> Receive(uint32_t max_len = 64) {
        if (!initialized_) return std::nullopt;
        std::vector<uint8_t> buf(max_len);
        int ret = UART_Receive(buf.data(), max_len);
        if (ret <= 0) return std::nullopt;
        return std::string(buf.begin(), buf.begin() + ret);
    }
};

}  // namespace extern_c_demo
```

```

// ═══════════════════════════════════════════════════════════════════════
// MAIN — Demos
// ═══════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "╔═══════════════════════════════════════════════════════════╗\n"
              << "║ Module 18: Driver / HAL / BSP — C vs C++ Layers           ║\n"
              << "╚═══════════════════════════════════════════════════════════╝\n\n";

    // --- Demo 1: C-Style Register Driver ---
    std::cout << "--- Demo 1: C-Style GPIO Register Driver ---\n";
    {
        c_driver::GPIO_Init(0, 5, c_driver::GPIO_MODE_OUTPUT);
        c_driver::GPIO_Write(0, 5, 1);
        auto val = c_driver::GPIO_Read(0, 5);
        std::cout << "  GPIOA.5: mode=OUTPUT, write=1, read=" << static_cast<int>(val) << "\n";
        // EN: Read back from ODR (simulated IDR = ODR in our simulation)
        // TR: ODR'den geri oku (simülasyonda IDR = ODR) — gerçek HW'de pin durumu IDR'den okunur
        c_driver::GPIOA_REGS.IDR = c_driver::GPIOA_REGS.ODR;
        val = c_driver::GPIO_Read(0, 5);
        std::cout << "  After IDR sync: read=" << static_cast<int>(val) << "\n";

        c_driver::GPIO_Toggle(0, 5);
        c_driver::GPIOA_REGS.IDR = c_driver::GPIOA_REGS.ODR;
        val = c_driver::GPIO_Read(0, 5);
        std::cout << "  After toggle: read=" << static_cast<int>(val) << "\n";

        std::cout << "  GPIOA.MODER = 0x" << std::hex << c_driver::GPIOA_REGS.MODER
                  << "  ODR = 0x" << c_driver::GPIOA_REGS.ODR << std::dec << "\n";
    }

    // --- Demo 2: C++ CRTP HAL Wrapper (STM32) ---
    std::cout << "\n--- Demo 2: C++ CRTP HAL — STM32 GPIO ---\n";
    {
        cpp_hal::STM32_GPIO stm_gpio;
        c_driver::GPIOA_REGS = {0, 0, 0, 0};  // reset
        // EN: Sync IDR with ODR for toggling to be readable
        // TR: Toggle okuması için IDR'yi ODR ile senkronize et — simülasyon gereği
        auto sync = [](){ c_driver::GPIOA_REGS.IDR = c_driver::GPIOA_REGS.ODR; };
        stm_gpio.Init(0, 0, cpp_hal::PinMode::OUTPUT);
        stm_gpio.Write(0, 0, true);
        sync();
        std::cout << "  STM32 PA0 = " << stm_gpio.Read(0, 0) << "\n";
        stm_gpio.Toggle(0, 0);
        sync();
        std::cout << "  After toggle = " << stm_gpio.Read(0, 0) << "\n";
    }

    // --- Demo 3: C++ CRTP HAL — NXP + Generic BlinkLED ---
    std::cout << "\n--- Demo 3: C++ CRTP HAL — NXP GPIO + Generic BlinkLED ---\n";
    {
        cpp_hal::NXP_GPIO nxp_gpio;
        std::cout << "  NXP BlinkLED:\n";
        cpp_hal::BlinkLED(nxp_gpio, 0, 3, 4);
    }

    // --- Demo 4: BSP Board Init ---
    std::cout << "\n--- Demo 4: BSP (Board Support Package) Init ---\n";
    {
        // EN: Using STM32 GPIO with Discovery board config
        // TR: STM32 GPIO'yu Discovery board yapılandırması ile kullan
        // Note: pins with port > 1 are beyond our 2-port simulation, shown for concept
        cpp_hal::STM32_GPIO gpio;
        struct SimplePinConfig {
            uint32_t port; uint32_t pin; cpp_hal::PinMode mode; std::string_view label;
        };
        SimplePinConfig board_pins[] = {
            {0, 12, cpp_hal::PinMode::OUTPUT, "LED_GREEN"},
            {0, 13, cpp_hal::PinMode::OUTPUT, "LED_ORANGE"},
            {0, 14, cpp_hal::PinMode::OUTPUT, "LED_RED"},
            {0,  0, cpp_hal::PinMode::INPUT,  "USER_BUTTON"},
        };
```

```
        std::cout << "  [BSP] Board pin initialization:\n";
        for (const auto& pin : board_pins) {
            auto status = gpio.Init(pin.port, pin.pin, pin.mode);
            std::cout << "    " << pin.label << " (P" << pin.port << "." << pin.pin
                      << ") → " << (status == cpp_hal::HAL_Status::OK ? "OK" : "ERROR") << "\n";
        }
    }

    // --- Demo 5: Linux Device Driver Model ---
    std::cout << "\n--- Demo 5: Linux Char Device Model (Simulated) ---\n";
    {
        linux_driver::TempSensorDevice sensor;
        {
            linux_driver::DeviceFile dev(sensor.GetOps());
            dev.Open();

            int32_t temp = 0;
            int ret = dev.Read(reinterpret_cast<uint8_t*>(&temp), sizeof(temp));
            std::cout << "    Read: " << ret << " bytes, temp = "
                      << (temp / 100) << "." << (temp % 100) << " °C\n";

            dev.Ioctl(0x01, 150);  // calibrate +1.50°C
            ret = dev.Read(reinterpret_cast<uint8_t*>(&temp), sizeof(temp));
            std::cout << "    After calibration: temp = "
                      << (temp / 100) << "." << (temp % 100) << " °C\n";
            // RAII: ~DeviceFile closes the device
        }
        std::cout << "    (device auto-closed via RAII)\n";
    }

    // --- Demo 6: extern "C" — C Driver + C++ Wrapper ---
    std::cout << "\n--- Demo 6: extern \"C\" — UART C Driver + C++ Wrapper ---\n";
    {
        extern_c_demo::UARTPort uart(115200);
        std::cout << "  Ready: " << uart.IsReady() << "\n";

        uart.Send("AT+GMR\r\n");

        auto response = uart.Receive();
        if (response)
            std::cout << "    Received: \"" << *response << "\"\n";
    }

    // --- Demo 7: Layer Comparison Summary ---
    std::cout << "\n--- Demo 7: Driver Layer Summary ---\n";
    {
        struct LayerInfo { std::string_view layer; std::string_view lang; std::string_view example;
};
        LayerInfo layers[] = {
            {"Application",   "C++",     "AUTOSAR SWC, Qt HMI, business logic"},
            {"Framework",     "C++",     "SOME/IP middleware, Android AIDL"},
            {"HAL",           "C/C++",   "CRTP GPIO, STM32 HAL wrapper"},
            {"Driver",        "C",       "Register-level GPIO, UART, SPI"},
            {"BSP",           "C/ASM",   "Clock config, pin mux, startup.s"},
            {"Hardware",      "Verilog", "MCU silicon, PCB, sensors"},
        };
        std::cout << "  ┌──────────────┬─────────┬──────────────────────────────────────┐\n";
        std::cout << "  │ Layer        │ Lang    │ Example                              │\n";
        std::cout << "  ├──────────────┼─────────┼──────────────────────────────────────┤\n";
        for (const auto& l : layers) {
            std::cout << "  │ " << std::setw(13) << std::left << l.layer
                      << " │ " << std::setw(7) << l.lang
                      << " │ " << std::setw(39) << l.example << " │\n";
        }
        std::cout << std::right;
        std::cout << "  └──────────────┴─────────┴──────────────────────────────────────┘\n";
    }

    std::cout << "\n[All demos completed successfully.]\n";
    return 0;
}
```

► 02. Zero Cost Abstraction

module_18_driver_development/02_zero_cost_abstraction.cpp

Konu: CRTP & Policy-Based Design — Zero-Cost Abstraction in C++

Teori / Theory

THEORY: Virtual Dispatch vs CRTP — Cost Analysis

Virtual dispatch induces runtime overhead:
- vtable pointer per object (+8 bytes on 64-bit)
- Indirect function call (pipeline stall, cache miss)
- Cannot be inlined by compiler

CRTP eliminates all three by resolving calls at compile time.
Virtual dispatch CRTP (static polymorphism)

```
┌─────────┐                 ┌─────────┐
│ obj     │                 │ obj     │
│ vptr    │──→ vtable       │ (no ptr)│
└─────────┘    ┌──────┐      └─────────┘
               │func1 │      Compiler resolves:
               │func2 │      Derived::func() inlined
               └──────┘
```

Overhead comparison:

```
┌──────────────────┬──────────────────┬──────────────────┐
│ Metric           │ Virtual          │ CRTP             │
├──────────────────┼──────────────────┼──────────────────┤
│ Object size      │ +8 bytes(vptr)   │ No overhead      │
│ Call overhead    │ ~2-5 cycles      │ 0 (inlined)      │
│ Inlineable       │ No               │ Yes              │
│ Code size (ROM)  │ vtable + code    │ Code per type    │
│ Runtime flexible │ □ Yes            │ □ Compile-time   │
└──────────────────┴──────────────────┴──────────────────┘
```

TEORİ: Sanal Dağıtım ve CRTP — Maliyet Analizi

Virtual dispatch çalışma zamanı ek maliyeti oluşturur:
- vtable işaretçisi: nesne başına +8 bayt (64-bit)
- Dolaylı çağrı: pipeline durması, önbellek ıskası
- Derleyici satır içi yapamaz

CRTP derleme zamanında çözerek tüm ek maliyeti sıfırlar.
Virtual dispatch CRTP (statik polimorfizm)

```
┌─────────┐                 ┌─────────┐
│ obj     │                 │ obj     │
│ vptr    │──→ vtable       │ (ptr yok)│
└─────────┘    ┌──────┐      └─────────┘
               │func1 │      Derleyici çözer:
               │func2 │      Derived::func() satır içi
               └──────┘
```

Ek maliyet karşılaştırması:

```
┌──────────────────┬──────────────────┬──────────────────┐
│ Metrik           │ Virtual          │ CRTP             │
├──────────────────┼──────────────────┼──────────────────┤
│ Nesne boyutu     │ +8 bayt(vptr)    │ Ek yük yok       │
│ Çağrı maliyeti   │ ~2-5 döngü       │ 0 (satır içi)    │
│ Satır içi yapılır│ Hayır            │ Evet             │
│ Çalışma zamanı   │ □ Esnek          │ □ Derleme zamanı│
└──────────────────┴──────────────────┴──────────────────┘
```

THEORY: Policy-Based Design

Policy-based design separates concerns into independent policy classes:
template<typename CommPolicy, typename ErrorPolicy> class Sensor : public CommPolicy, public ErrorPolicy { ... };
Sensor<SPI_Comm, LogError> sensor_spi; Sensor<I2C_Comm, ResetError> sensor_i2c;
Each combination generates optimal code with no runtime overhead.

TEORİ: Politika Tabanlı Tasarım

Policy-based design, kaygıları bağımsız politika sınıflarına ayırır:
template<typename CommPolicy, typename ErrorPolicy> class Sensor : public CommPolicy, public ErrorPolicy { ... };
Sensor<SPI_Comm, LogError> sensor_spi; Sensor<I2C_Comm, ResetError> sensor_i2c;
Her kombinasyon derlemede ayrı optimum kod üretir — çalışma zamanı ek maliyeti sıfır.

THEORY: Embedded C++ Compile Flags

Common embedded-safe flags:

Flag	Effect
-fno-exceptions	Disable C++ exceptions
-fno-rtti	Disable RTTI (dynamic_cast/typeid)
-fno-threadsafe-statics	No thread-safe static init guard
-fno-unwind-tables	Remove stack unwinding tables
-Os / -Oz	Optimize for size
-ffunction-sections	Place each function in own section
-fdata-sections	Place each data item in own section
-Wl,--gc-sections	Linker removes unused sections

TEORİ: Gömülü C++ Derleme Bayrakları

Gömülü C++ için derleme bayrakları:

Bayrak	Etki
-fno-exceptions	C++ istisnalarını devre dışı bırak
-fno-rtti	RTTI'yi devre dışı bırak
-fno-threadsafe-statics	Statik init korum. devre dışı bırak
-fno-unwind-tables	Yığın geri sarım tablolarını kaldır
-Os / -Oz	Boyut için optimize et
-ffunction-sections	Her fonksiyonu kendi bölümüne koy
-fdata-sections	Her veriyi kendi bölümüne koy
-Wl,--gc-sections	Bağlayıcı kullanılmayanı kaldırır

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <string>
#include <string_view>
#include <array>
#include <vector>
#include <iomanip>
#include <type_traits>
```

```cpp
#include <optional>
#include <cassert>
#include <chrono>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: Virtual Dispatch vs CRTP — Side-by-Side
// ═══════════════════════════════════════════════════════════════════════════

namespace virtual_vs_crtp {

// EN: --- Virtual version (runtime polymorphism) ---
// TR: --- Virtual versiyon (çalışma zamanı polimorfizmi) ---
class ISensor_Virtual {
public:
    virtual ~ISensor_Virtual() = default;
    virtual uint16_t Read() = 0;
    virtual std::string_view Name() const = 0;
};

class TempSensor_V : public ISensor_Virtual {
public:
    uint16_t Read() override { return 2048; }
    std::string_view Name() const override { return "TempSensor(V)"; }
};

class PressureSensor_V : public ISensor_Virtual {
public:
    uint16_t Read() override { return 3500; }
    std::string_view Name() const override { return "PressureSensor(V)"; }
};

// EN: --- CRTP version (compile-time polymorphism) ---
// TR: --- CRTP versiyon (derleme zamanı polimorfizmi) ---
template <typename Derived>
class ISensor_CRTP {
public:
    uint16_t Read() { return static_cast<Derived*>(this)->ReadImpl(); }
    std::string_view Name() const {
        return static_cast<const Derived*>(this)->NameImpl();
    }
};

class TempSensor_C : public ISensor_CRTP<TempSensor_C> {
public:
    uint16_t ReadImpl() { return 2048; }
    static std::string_view NameImpl() { return "TempSensor(CRTP)"; }
};

class PressureSensor_C : public ISensor_CRTP<PressureSensor_C> {
public:
    uint16_t ReadImpl() { return 3500; }
    static std::string_view NameImpl() { return "PressureSensor(CRTP)"; }
};

// EN: Size comparison helper
// TR: Boyut karşılaştırma yardımcısı
inline void PrintSizes() {
    std::cout << "  Object sizes:\n";
    std::cout << "    TempSensor_V (virtual):   " << sizeof(TempSensor_V) << " bytes\n";
    std::cout << "    TempSensor_C (CRTP):      " << sizeof(TempSensor_C) << " bytes\n";
    std::cout << "    PressureSensor_V (virtual):" << sizeof(PressureSensor_V) << " bytes\n";
    std::cout << "    PressureSensor_C (CRTP):   " << sizeof(PressureSensor_C) << " bytes\n";
    std::cout << "    vtable pointer overhead:   " << (sizeof(TempSensor_V) - sizeof(TempSensor_C)) << " bytes\n";
}

// EN: Generic function — works with any CRTP sensor
// TR: Jenerik fonksiyon — herhangi bir CRTP sensörüyle çalışır (derleme zamanı polimorfizm)
template <typename SensorType>
uint16_t ReadAndLog(ISensor_CRTP<SensorType>& sensor) {
    auto val = sensor.Read();
    std::cout << "    " << sensor.Name() << " = " << val << "\n";
    return val;
}

}  // namespace virtual_vs_crtp
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: CRTP Driver Pattern — SPI/I2C Backend Selection
// ═══════════════════════════════════════════════════════════════════════════

namespace crtp_driver {

// EN: SPI communication backend
// TR: SPI iletişim arka ucu
class SPI_Backend {
protected:
    static uint16_t TransferRaw(uint8_t reg, [[maybe_unused]] uint8_t dummy) {
        // EN: Simulated SPI register read
        // TR: Simüle edilmiş SPI register okuma — gerçek donanımda SPI MISO hattından veri gelir
        return static_cast<uint16_t>(0x1000 + reg);
    }
    static std::string_view BackendName() { return "SPI"; }
};

// EN: I2C communication backend
// TR: I2C iletişim arka ucu
class I2C_Backend {
protected:
    static uint16_t TransferRaw(uint8_t reg, [[maybe_unused]] uint8_t dummy) {
        // EN: Simulated I2C register read
        // TR: Simüle edilmiş I2C register okuma — gerçek donanımda I2C SDA hattından veri gelir
        return static_cast<uint16_t>(0x2000 + reg);
    }
    static std::string_view BackendName() { return "I2C"; }
};

// EN: CRTP sensor driver — backend selected at compile time
// TR: CRTP sensör sürücüsü — arka uç derleme zamanında seçilir
template <typename Derived, typename CommBackend>
class SensorDriver : public CommBackend {
public:
    uint16_t ReadRegister(uint8_t reg) {
        return CommBackend::TransferRaw(reg, 0);
    }

    float ReadTemperature() {
        uint16_t raw = ReadRegister(0x10);
        return static_cast<Derived*>(this)->ConvertRaw(raw);
    }

    void PrintInfo() {
        std::cout << "    Sensor: " << static_cast<Derived*>(this)->SensorName()
                  << " via " << CommBackend::BackendName() << "\n";
    }
};

// EN: LM75 temperature sensor
// TR: LM75 sıcaklık sensörü
template <typename CommBackend>
class LM75 : public SensorDriver<LM75<CommBackend>, CommBackend> {
public:
    float ConvertRaw(uint16_t raw) { return static_cast<float>(raw) * 0.00625f; }
    static std::string_view SensorName() { return "LM75"; }
};

// EN: BMP280 pressure/temp sensor
// TR: BMP280 basınç/sıcaklık sensörü — Bosch üretimi, I2C/SPI destekli barometrik sensör
template <typename CommBackend>
class BMP280 : public SensorDriver<BMP280<CommBackend>, CommBackend> {
public:
    float ConvertRaw(uint16_t raw) { return static_cast<float>(raw) * 0.01f; }
    static std::string_view SensorName() { return "BMP280"; }
};

}  // namespace crtp_driver

```

```cpp
// ═══════════════════════════════════════════════════════════════════════════
// PART 3: Policy-Based Design — Sensor with Comm + Error Policies
// ═══════════════════════════════════════════════════════════════════════════

namespace policy_design {

// EN: Communication policies
// TR: İletişim politikaları
struct SPI_Policy {
    static uint16_t Read(uint8_t reg) {
        return static_cast<uint16_t>(0x1000 + reg);
    }
    static std::string_view CommName() { return "SPI"; }
};

struct I2C_Policy {
    static uint16_t Read(uint8_t reg) {
        return static_cast<uint16_t>(0x2000 + reg);
    }
    static std::string_view CommName() { return "I2C"; }
};

// EN: Error handling policies
// TR: Hata yönetimi politikaları
struct LogErrorPolicy {
    static void HandleError(std::string_view msg) {
        std::cout << "      [ERROR-LOG] " << msg << "\n";
    }
    static std::string_view ErrorPolicyName() { return "LogError"; }
};

struct ResetErrorPolicy {
    static void HandleError(std::string_view msg) {
        std::cout << "      [ERROR-RESET] " << msg << " → RESETTING\n";
    }
    static std::string_view ErrorPolicyName() { return "ResetError"; }
};

struct IgnoreErrorPolicy {
    static void HandleError([[maybe_unused]] std::string_view msg) {
        // EN: Silently ignore
        // TR: Sessizce yoksay — hata mesajı üretilmez, üretimde dikkatli kullanılmalı
    }
    static std::string_view ErrorPolicyName() { return "IgnoreError"; }
};

// EN: Policy-based sensor class
// TR: Politika tabanlı sensör sınıfı
template <typename CommPolicy, typename ErrorPolicy>
class Sensor {
    uint16_t last_value_ = 0;

public:
    uint16_t Read(uint8_t reg) {
        auto val = CommPolicy::Read(reg);
        if (val == 0) {
            ErrorPolicy::HandleError("Read returned zero — possible HW fault");
            return last_value_;
        }
        last_value_ = val;
        return val;
    }

    void PrintConfig() const {
        std::cout << "    Comm=" << CommPolicy::CommName()
                  << " Error=" << ErrorPolicy::ErrorPolicyName() << "\n";
    }
};

}  // namespace policy_design

```

```
// ═══════════════════════════════════════════════════════════════════════════
// PART 4: if constexpr — Platform-Specific Code Elimination
// ═══════════════════════════════════════════════════════════════════════════

namespace constexpr_dispatch {

// EN: Platform tags
// TR: Platform etiketleri
struct STM32_Platform {};
struct NXP_Platform {};
struct RiscV_Platform {};

// EN: Compile-time clock configuration
// TR: Derleme zamanı saat yapılandırması
template <typename Platform>
constexpr uint32_t GetSystemClock() {
    if constexpr (std::is_same_v<Platform, STM32_Platform>)
        return 168'000'000;  // 168 MHz
    else if constexpr (std::is_same_v<Platform, NXP_Platform>)
        return 120'000'000;  // 120 MHz
    else if constexpr (std::is_same_v<Platform, RiscV_Platform>)
        return 32'000'000;   // 32 MHz
    else
        return 0;
}

template <typename Platform>
constexpr uint32_t GetUARTBaudDivider(uint32_t baud) {
    return GetSystemClock<Platform>() / baud;
}

// EN: Platform-specific initialization (dead code eliminated by compiler)
// TR: Platform-specific başlatma (ölü kod derleyici tarafından elenir)
template <typename Platform>
void InitPeripherals() {
    std::cout << "    System clock: " << GetSystemClock<Platform>() / 1'000'000 << " MHz\n";

    if constexpr (std::is_same_v<Platform, STM32_Platform>) {
        std::cout << "    STM32: Enabling AHB1 bus clocks\n";
        std::cout << "    STM32: Configuring flash wait states = 5\n";
    } else if constexpr (std::is_same_v<Platform, NXP_Platform>) {
        std::cout << "    NXP: Enabling PCC (Peripheral Clock Control)\n";
        std::cout << "    NXP: Configuring flash speculation\n";
    } else if constexpr (std::is_same_v<Platform, RiscV_Platform>) {
        std::cout << "    RISC-V: Configuring PLL from XTAL\n";
        std::cout << "    RISC-V: Setting MSTATUS.MIE = 1\n";
    }

    constexpr auto uart_div = GetUARTBaudDivider<Platform>(115200);
    std::cout << "    UART divider for 115200 baud: " << uart_div << "\n";
}

}  // namespace constexpr_dispatch

// ═══════════════════════════════════════════════════════════════════════════
// PART 5: Tag Dispatch — Communication Type Selection
// ═══════════════════════════════════════════════════════════════════════════

namespace tag_dispatch {

// EN: Tag types for dispatch
// TR: Dağıtım için etiket türleri
struct SPI_tag {};
struct I2C_tag {};
struct UART_tag {};

// EN: Overloaded init functions
// TR: Aşırı yüklenmiş init fonksiyonları
inline void InitComm(SPI_tag, uint32_t speed) {
    std::cout << "    SPI init: speed=" << speed / 1000 << " kHz, CPOL=0, CPHA=0\n";
}

inline void InitComm(I2C_tag, uint32_t speed) {
    std::cout << "    I2C init: speed=" << speed / 1000 << " kHz, 7-bit addressing\n";
}
```

```cpp

inline void InitComm(UART_tag, uint32_t speed) {
    std::cout << "    UART init: baud=" << speed << ", 8N1\n";
}

// EN: Sensor that uses tag dispatch for communication init
// TR: İletişim başlatma için etiket dağıtımı kullanan sensör — CommTag türüne göre doğru InitComm seçilir
template <typename CommTag>
class TaggedSensor {
    std::string_view name_;
    uint32_t comm_speed_;

public:
    TaggedSensor(std::string_view name, uint32_t speed)
        : name_(name), comm_speed_(speed) {}

    void Init() {
        std::cout << "    Initializing " << name_ << ":\n  ";
        InitComm(CommTag{}, comm_speed_);
    }
};

}  // namespace tag_dispatch

// ═══════════════════════════════════════════════════════════════════════════
// PART 6: constexpr Configuration — Compile-Time Register Layout
// ═══════════════════════════════════════════════════════════════════════════

namespace constexpr_config {

// EN: Register address definitions (compile-time)
// TR: Register adresi tanımları (derleme zamanı)
struct PeripheralConfig {
    uint32_t base_address;
    uint32_t clock_enable_bit;
    uint32_t irq_number;
    uint32_t dma_channel;
};

// EN: Compile-time peripheral table
// TR: Derleme zamanı çevre birimi tablosu — tüm adresler ve IRQ'lar constexpr ile doğrulanır
constexpr std::array<PeripheralConfig, 4> PERIPHERALS = {{
    {0x40004400, 17, 37, 4},  // USART2
    {0x40004800, 18, 38, 3},  // USART3
    {0x40013000, 14, 35, 2},  // SPI1
    {0x40003800, 15, 36, 0},  // SPI2
}};

constexpr size_t FindPeripheral(uint32_t base) {
    for (size_t i = 0; i < PERIPHERALS.size(); ++i) {
        if (PERIPHERALS[i].base_address == base) return i;
    }
    return PERIPHERALS.size();  // not found
}

// EN: Verify at compile time
// TR: Derleme zamanında doğrula — yanlış index kullanımı derleme hatasına dönüşür
static_assert(FindPeripheral(0x40013000) == 2, "SPI1 should be at index 2");
static_assert(FindPeripheral(0x40004400) == 0, "USART2 should be at index 0");

// EN: Buffer sizes — compile-time
// TR: Tampon boyutları — derleme zamanı sabitleri, RAM kullanımını önceden belirler
constexpr size_t UART_TX_BUF_SIZE = 256;
constexpr size_t UART_RX_BUF_SIZE = 512;
constexpr size_t SPI_BUF_SIZE = 128;

// EN: Compile-time CRC polynomial check
// TR: Derleme zamanı CRC polinomu kontrolü — sıfır polinom veri bütünlüğünü bozar
constexpr uint32_t CRC_POLY = 0x04C11DB7;
static_assert(CRC_POLY != 0, "CRC polynomial must not be zero");

}  // namespace constexpr_config
```

```

// ═══════════════════════════════════════════════════════════════════════════
// MAIN — Demos
// ═══════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "╔═══════════════════════════════════════════════════════════╗\n"
              << "║ Module 18: Zero-Cost Abstraction — CRTP & Policy Design   ║\n"
              << "╚═══════════════════════════════════════════════════════════╝\n\n";

    // --- Demo 1: Virtual vs CRTP — Size + Call ---
    std::cout << "--- Demo 1: Virtual vs CRTP — Size Comparison ---\n";
    {
        virtual_vs_crtp::PrintSizes();

        std::cout << "\n  Virtual dispatch:\n";
        virtual_vs_crtp::TempSensor_V temp_v;
        virtual_vs_crtp::PressureSensor_V press_v;
        virtual_vs_crtp::ISensor_Virtual* sensors[] = {&temp_v, &press_v};
        for (auto* s : sensors)
            std::cout << "    " << s->Name() << " = " << s->Read() << "\n";

        std::cout << "  CRTP dispatch:\n";
        virtual_vs_crtp::TempSensor_C temp_c;
        virtual_vs_crtp::PressureSensor_C press_c;
        virtual_vs_crtp::ReadAndLog(temp_c);
        virtual_vs_crtp::ReadAndLog(press_c);
    }

    // --- Demo 2: CRTP Driver — SPI vs I2C Backend ---
    std::cout << "\n--- Demo 2: CRTP Driver — SPI vs I2C Backend ---\n";
    {
        crtp_driver::LM75<crtp_driver::SPI_Backend> lm75_spi;
        crtp_driver::LM75<crtp_driver::I2C_Backend> lm75_i2c;
        crtp_driver::BMP280<crtp_driver::SPI_Backend> bmp_spi;

        lm75_spi.PrintInfo();
        std::cout << "    Temp(SPI) = " << std::fixed << std::setprecision(2)
                  << lm75_spi.ReadTemperature() << " °C\n";

        lm75_i2c.PrintInfo();
        std::cout << "    Temp(I2C) = " << lm75_i2c.ReadTemperature() << " °C\n";

        bmp_spi.PrintInfo();
        std::cout << "    Temp(SPI) = " << bmp_spi.ReadTemperature() << " °C\n";
        std::cout << std::defaultfloat;
    }

    // --- Demo 3: Policy-Based Design ---
    std::cout << "\n--- Demo 3: Policy-Based Design — Comm + Error Policies ---\n";
    {
        policy_design::Sensor<policy_design::SPI_Policy, policy_design::LogErrorPolicy> sensor1;
        policy_design::Sensor<policy_design::I2C_Policy, policy_design::ResetErrorPolicy> sensor2;
        policy_design::Sensor<policy_design::SPI_Policy, policy_design::IgnoreErrorPolicy> sensor3;

        sensor1.PrintConfig();
        auto v1 = sensor1.Read(0x10);
        std::cout << "    Read(0x10) = 0x" << std::hex << v1 << std::dec << "\n";

        sensor2.PrintConfig();
        auto v2 = sensor2.Read(0x20);
        std::cout << "    Read(0x20) = 0x" << std::hex << v2 << std::dec << "\n";

        sensor3.PrintConfig();
        auto v3 = sensor3.Read(0x30);
        std::cout << "    Read(0x30) = 0x" << std::hex << v3 << std::dec << "\n";

        std::cout << "\n  Object sizes (all policies generate separate types):\n";
        std::cout << "    sensor1: " << sizeof(sensor1) << " bytes\n";
        std::cout << "    sensor2: " << sizeof(sensor2) << " bytes\n";
        std::cout << "    sensor3: " << sizeof(sensor3) << " bytes\n";
    }
```

```
    // --- Demo 4: if constexpr — Platform-Specific Code ---
    std::cout << "\n--- Demo 4: if constexpr — Platform Code Elimination ---\n";
    {
        std::cout << "  [STM32]\n";
        constexpr_dispatch::InitPeripherals<constexpr_dispatch::STM32_Platform>();
        std::cout << "  [NXP]\n";
        constexpr_dispatch::InitPeripherals<constexpr_dispatch::NXP_Platform>();
        std::cout << "  [RISC-V]\n";
        constexpr_dispatch::InitPeripherals<constexpr_dispatch::RiscV_Platform>();
    }

    // --- Demo 5: Tag Dispatch ---
    std::cout << "\n--- Demo 5: Tag Dispatch — Communication Type Selection ---\n";
    {
        tag_dispatch::TaggedSensor<tag_dispatch::SPI_tag>  accel("Accelerometer", 10'000'000);
        tag_dispatch::TaggedSensor<tag_dispatch::I2C_tag>  temp("TempSensor", 400'000);
        tag_dispatch::TaggedSensor<tag_dispatch::UART_tag> gps("GPS Module", 9600);

        accel.Init();
        temp.Init();
        gps.Init();
    }

    // --- Demo 6: constexpr Configuration ---
    std::cout << "\n--- Demo 6: constexpr Compile-Time Configuration ---\n";
    {
        std::cout << "  Peripheral table (resolved at compile time):\n";
        std::cout << "  ┌──────────────┬────────────────┬─────┬─────┐\n";
        std::cout << "  │ Base Addr    │ ClkEn Bit      │ IRQ │ DMA │\n";
        std::cout << "  ├──────────────┼────────────────┼─────┼─────┤\n";
        for (const auto& p : constexpr_config::PERIPHERALS) {
            std::cout << "  │ 0x" << std::hex << std::setw(8) << std::setfill('0')
                      << p.base_address << std::dec << std::setfill(' ')
                      << " │ bit " << std::setw(8) << p.clock_enable_bit
                      << " │ " << std::setw(3) << p.irq_number
                      << " │ " << std::setw(3) << p.dma_channel << " │\n";
        }
        std::cout << "  └──────────────┴────────────────┴─────┴─────┘\n";

        constexpr auto spi1_idx = constexpr_config::FindPeripheral(0x40013000);
        std::cout << "  SPI1 found at index " << spi1_idx
                  << " (IRQ=" << constexpr_config::PERIPHERALS[spi1_idx].irq_number << ")\n";

        std::cout << "  Buffer sizes: UART_TX=" << constexpr_config::UART_TX_BUF_SIZE
                  << " UART_RX=" << constexpr_config::UART_RX_BUF_SIZE
                  << " SPI=" << constexpr_config::SPI_BUF_SIZE << "\n";
    }

    // --- Demo 7: Zero-Cost Summary ---
    std::cout << "\n--- Demo 7: Zero-Cost Abstraction Summary ---\n";
```

```
    {
        struct Technique { std::string_view name; std::string_view overhead; std::string_view use_case; };
        Technique techs[] = {
            {"CRTP",                "0 cycles", "Static polymorphism (sensors, drivers)"},
            {"Policy-based design", "0 cycles", "Configurable behavior (comm, error)"},
            {"if constexpr",        "0 cycles", "Platform-specific code elimination"},
            {"Tag dispatch",        "0 cycles", "Overload resolution by type tag"},
            {"constexpr config",    "0 cycles", "Compile-time peripheral configuration"},
            {"templates",           "0 cycles", "Generic algorithms, type-safe APIs"},
        };
        std::cout << "  ┌──────────────────────┬────────────┬──────────────────────────────────────┐\n";
        std::cout << "  │ Technique            │ Overhead   │ Use Case                             │\n";
        std::cout << "  ├──────────────────────┼────────────┼──────────────────────────────────────┤\n";
        for (const auto& t : techs) {
            std::cout << "  │ " << std::setw(20) << std::left << t.name
                      << " │ " << std::setw(9) << t.overhead
                      << " │ " << std::setw(38) << t.use_case << " │\n";
        }
        std::cout << std::right;
        std::cout << "  └──────────────────────┴────────────┴──────────────────────────────────────┘\n";
    }

    std::cout << "\n[All demos completed successfully.]\n";
    return 0;
}
```

► 03. Peripheral Driver

module_18_driver_development/03_peripheral_driver.cpp

Konu: Peripheral Driver Development — GPIO, UART, SPI, ISR, State Machine

Teori / Theory

THEORY: Register-Level Driver Architecture

Bare-metal drivers manipulate memory-mapped registers via volatile pointers. Every peripheral has a register block at a fixed base address.
Register Bit Operations:

```
┌──────────────────────────────────────────┐
│ SET bit n:    reg |=  (1u << n)          │
│ CLEAR bit n:  reg &= ~(1u << n)          │
│ TOGGLE bit n: reg ^=  (1u << n)          │
│ READ bit n:   (reg >> n) & 1u            │
│ MODIFY field: reg = (reg & ~mask) | (val << pos) │
└──────────────────────────────────────────┘
```

TEORİ: Yazmaç Düzeyi Sürücü Mimarisi

Bare-metal sürücüler, bellek eşlemeli (memory-mapped) register'lara volatile işaretçiler aracılığıyla erişir. Her çevre birimi sabit bir taban adresinde bir register bloğuna sahiptir.
- SET: İlgili biti 1 yapar → çevre birimini etkinleştirir
- CLEAR: İlgili biti 0 yapar → özelliği devre dışı bırakır
- MODIFY: Çok bitli alanı maskeleyerek güvenli değiştirir Neden önemli: Yanlış bit manipülasyonu donanımı kilitleyebilir. Ne zaman: Her register erişiminde bu temel işlemler kullanılır.

THEORY: Type-Safe Register Access

Raw bit manipulation is error-prone. A type-safe RegisterField template encapsulates base address, bit offset and width:
template <uint32_t Base, uint32_t Offset, uint32_t Width> struct RegisterField { static void Set(uint32_t val); static uint32_t Get(); };
This prevents mixing registers from different peripherals.

TEORİ: Tip Güvenli Yazmaç Erişimi

Ham bit manipülasyonu hataya açıktır — yanlış register'a yanlış bit yazılması donanım hatasına yol açar. Tip-güvenli RegisterField şablonu taban adresi, bit konumu ve genişliği derleme zamanında kapsüller.
- Ne: Şablon parametreleri ile register alanını statik olarak tanımlar
- Neden: Farklı çevre birimlerinin register'larını karıştırmayı engeller
- Ne zaman: Birden fazla register'a erişen karmaşık sürücülerde kullanılır

THEORY: Driver State Machine

```
UNINITIALIZED ──Init()──► INITIALIZED ──Start()──► RUNNING
      ▲                    |                    |
|                  DeInit()             error/Stop()
      |                    |                    |
      └──── DeInit() ◄── STOPPED ◄──────── ERROR ◄┘
```

Every driver follows this state machine. Invalid transitions are rejected.

TEORİ: Sürücü Durum Makinesi

Her sürücü bu durum makinesini izler. Geçersiz geçişler reddedilir. Durum akışı: UNINITIALIZED →Init→ INITIALIZED →Start→ RUNNING Hata/Durdurma durumunda ERROR/STOPPED'a geçilir. DeInit() ile başlangıç durumuna dönülür.

THEORY: ISR Callback Mechanism

In embedded, peripherals trigger hardware interrupts. The ISR (Interrupt Service Routine) must be fast — typically sets a flag or pushes data to a ring buffer, then signals the main loop.
ISR callback pattern:
1. Driver registers a callback function pointer
2. Simulated IRQ handler calls the callback
3. Callback stores data / sets flag (minimal work)
4. Main loop processes the flag / data

TEORİ: ISR Geri Çağırma Mekanizması

Gömülü sistemlerde çevre birimleri donanım kesmelerine (interrupt) neden olur. ISR (Kesme Servis Rutini) hızlı olmalıdır — genellikle bir bayrak ayarlar veya halka tamponuna veri yazar, ardından ana döngüye sinyal gönderir. ISR geri çağrı deseni:
1. Sürücü bir callback fonksiyon işaretçisi kaydeder
2. Simüle edilmiş IRQ handler callback'i çağırır
3. Callback veri depolar / bayrak ayarlar (minimum iş)
4. Ana döngü bayrak / veriyi işler Neden: ISR içinde uzun işlem yapmak kesme gecikmesini artırır. Ne zaman: UART RX, ADC dönüşüm tamamlandı gibi asenkron donanım olaylarında.

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <string>
#include <string_view>
#include <array>
#include <vector>
#include <functional>
#include <iomanip>
#include <optional>
#include <cassert>

// ============================================================================
// PART 1: Register Bit Manipulation Utilities
// ============================================================================

namespace reg_ops {

// EN: Basic register bit operations on a simulated register variable
// TR: Simüle edilmiş register değişkeni üzerinde temel bit işlemleri
struct RegOps {
    static void SetBit(uint32_t& reg, uint32_t bit) { reg |= (1u << bit); }
    static void ClearBit(uint32_t& reg, uint32_t bit) { reg &= ~(1u << bit); }
    static void ToggleBit(uint32_t& reg, uint32_t bit) { reg ^= (1u << bit); }
    static bool ReadBit(uint32_t reg, uint32_t bit) { return (reg >> bit) & 1u; }

    // EN: Modify a multi-bit field within a register
    // TR: Register içinde çok bitli bir alanı değiştir
    static void ModifyField(uint32_t& reg, uint32_t pos, uint32_t width, uint32_t value) {
        uint32_t mask = ((1u << width) - 1u) << pos;
        reg = (reg & ~mask) | ((value & ((1u << width) - 1u)) << pos);
    }

    static uint32_t ReadField(uint32_t reg, uint32_t pos, uint32_t width) {
        uint32_t mask = (1u << width) - 1u;
        return (reg >> pos) & mask;
    }
};

// EN: Type-safe register field template
// TR: Tip-güvenli register alanı şablonu
template <uint32_t BitOffset, uint32_t BitWidth>
struct RegisterField {
    static_assert(BitOffset + BitWidth <= 32, "Field exceeds register width");

    static constexpr uint32_t MASK = ((1u << BitWidth) - 1u) << BitOffset;

    static void Set(uint32_t& reg, uint32_t value) {
        reg = (reg & ~MASK) | ((value & ((1u << BitWidth) - 1u)) << BitOffset);
    }

    static uint32_t Get(uint32_t reg) {
        return (reg >> BitOffset) & ((1u << BitWidth) - 1u);
    }
};

}  // namespace reg_ops

// ============================================================================
// PART 2: Complete GPIO Driver
// ============================================================================

namespace gpio_driver {

enum class PinMode : uint8_t { INPUT = 0, OUTPUT = 1, ALT_FUNC = 2, ANALOG = 3 };
enum class PullType : uint8_t { NONE = 0, PULL_UP = 1, PULL_DOWN = 2 };

```

```cpp
// EN: Simulated GPIO port register block (mirrors STM32 layout)
// TR: Simüle edilmiş GPIO port register bloğu (STM32 düzenini yansıtır)
struct GPIO_Regs {
    uint32_t MODER   = 0;  // Mode register (2 bits per pin) — TR: Mod yazmacı (pin başına 2 bit)
    uint32_t OTYPER  = 0;  // Output type — TR: Çıkış tipi (push-pull/open-drain)
    uint32_t OSPEEDR = 0;  // Output speed — TR: Çıkış hızı
    uint32_t PUPDR   = 0;  // Pull-up/pull-down — TR: Yukarı/aşağı çekme direnci
    uint32_t IDR     = 0;  // Input data — TR: Giriş verisi (salt okunur)
    uint32_t ODR     = 0;  // Output data — TR: Çıkış verisi
    uint32_t BSRR    = 0;  // Bit set/reset (write-only) — TR: Bit set/reset (salt yazılır)
};

class GPIODriver {
    GPIO_Regs regs_{};
    std::string_view port_name_;

public:
    explicit GPIODriver(std::string_view name) : port_name_(name) {}

    // EN: Configure pin mode
    // TR: Pin modunu yapılandır — her pin için MODER (giriş/çıkış/alternatif) ve PUPDR (pull-up/down) ayarlanır
    void ConfigurePin(uint32_t pin, PinMode mode, PullType pull = PullType::NONE) {
        if (pin >= 16) return;
        // EN: MODER: 2 bits per pin
        // TR: MODER: pin başına 2 bit — 00=giriş, 01=çıkış, 10=alt fonksiyon, 11=analog
        uint32_t pos = pin * 2;
        reg_ops::RegOps::ModifyField(regs_.MODER, pos, 2, static_cast<uint32_t>(mode));
        reg_ops::RegOps::ModifyField(regs_.PUPDR, pos, 2, static_cast<uint32_t>(pull));
    }

    void WritePin(uint32_t pin, bool state) {
        if (pin >= 16) return;
        if (state)
            regs_.BSRR = (1u << pin);          // Set
        else
            regs_.BSRR = (1u << (pin + 16));   // Reset
        // EN: Apply BSRR to ODR (simulating hardware behavior)
        // TR: BSRR'yi ODR'ye uygula (donanım davranışını simüle eder) — atomik set/reset
        ApplyBSRR();
    }

    bool ReadPin(uint32_t pin) const {
        if (pin >= 16) return false;
        return (regs_.IDR >> pin) & 1u;
    }

    void TogglePin(uint32_t pin) {
        if (pin >= 16) return;
        regs_.ODR ^= (1u << pin);
        regs_.IDR = regs_.ODR;  // EN: Simulate loopback
        // TR: Geri döngü simülasyonu — çıkış pinlerinin değeri IDR'den okunabilir
    }

    void PrintRegisters() const {
        std::cout << "    " << port_name_ << " registers:\n";
        std::cout << "      MODER  = 0x" << std::hex << std::setw(8) << std::setfill('0')
                  << regs_.MODER << "\n";
        std::cout << "      PUPDR  = 0x" << std::setw(8) << regs_.PUPDR << "\n";
        std::cout << "      ODR    = 0x" << std::setw(8) << regs_.ODR << "\n";
        std::cout << "      IDR    = 0x" << std::setw(8) << regs_.IDR << std::dec
                  << std::setfill(' ') << "\n";
    }

private:
    void ApplyBSRR() {
        // EN: Low 16 bits = set, high 16 bits = reset
        // TR: Alt 16 bit = set (1 yap), üst 16 bit = reset (0 yap) — BSRR atomik pin kontrolü sağlar
        uint32_t set_mask = regs_.BSRR & 0xFFFF;
        uint32_t reset_mask = (regs_.BSRR >> 16) & 0xFFFF;
        regs_.ODR |= set_mask;
        regs_.ODR &= ~reset_mask;
        regs_.IDR = regs_.ODR;  // EN: Simulate loopback for output pins
        // TR: Çıkış pinleri için geri döngü — ODR değeri IDR'ye kopyalanır
        regs_.BSRR = 0;
    }
};
```

```cpp

}  // namespace gpio_driver

// ═══════════════════════════════════════════════════════════════════════════
// PART 3: UART Driver with TX Polling, RX Ring Buffer + ISR Callback
// ═══════════════════════════════════════════════════════════════════════════

namespace uart_driver {

// EN: Simple ring buffer for RX FIFO
// TR: RX FIFO için basit halka tampon
template <size_t N>
class RingBuffer {
    std::array<uint8_t, N> buf_{};
    size_t head_ = 0;
    size_t tail_ = 0;
    size_t count_ = 0;

public:
    bool Push(uint8_t byte) {
        if (count_ >= N) return false;  // Full
        buf_[head_] = byte;
        head_ = (head_ + 1) % N;
        ++count_;
        return true;
    }

    std::optional<uint8_t> Pop() {
        if (count_ == 0) return std::nullopt;
        uint8_t byte = buf_[tail_];
        tail_ = (tail_ + 1) % N;
        --count_;
        return byte;
    }

    [[nodiscard]] size_t Size() const { return count_; }
    [[nodiscard]] bool Empty() const { return count_ == 0; }
    [[nodiscard]] bool Full() const { return count_ >= N; }
};

struct UART_Regs {
    uint32_t CR1 = 0;   // Control register 1 — TR: Kontrol yazmaçı 1
    uint32_t BRR = 0;   // Baud rate register — TR: Baud hızı yazmaçı
    uint32_t SR  = 0;   // Status register — TR: Durum yazmaçı
    uint32_t DR  = 0;   // Data register — TR: Veri yazmaçı
};

// EN: Status register bits
// TR: Durum register bitleri — TXE: gönderim tamponu boş, RXNE: alım tamponu dolu
constexpr uint32_t UART_SR_TXE  = (1u << 7);  // TX empty — TR: TX boş
constexpr uint32_t UART_SR_RXNE = (1u << 5);  // RX not empty — TR: RX boş değil
constexpr uint32_t UART_SR_TC   = (1u << 6);  // Transmission complete — TR: İletim tamamlandı

// EN: Control register bits
// TR: Kontrol register bitleri — UE: UART etkin, TE: verici etkin, RE: alıcı etkin
constexpr uint32_t UART_CR1_UE   = (1u << 0); // UART enable — TR: UART etkinleştir
constexpr uint32_t UART_CR1_TE   = (1u << 3); // Transmitter enable — TR: Verici etkinleştir
constexpr uint32_t UART_CR1_RE   = (1u << 2); // Receiver enable — TR: Alıcı etkinleştir
constexpr uint32_t UART_CR1_RXNEIE = (1u << 5); // RX interrupt enable — TR: RX kesmesi etkinleştir

// EN: Callback type for RX interrupt
// TR: RX kesme geri çağrı tipi — alınan her byte için çağrılacak fonksiyon imzası
using RxCallback = std::function<void(uint8_t)>;

class UARTDriver {
    UART_Regs regs_{};
    RingBuffer<64> rx_fifo_;
    RxCallback rx_callback_;
    bool initialized_ = false;

```

```
public:
    bool Init(uint32_t sys_clock, uint32_t baud) {
        regs_.BRR = sys_clock / baud;
        regs_.CR1 = UART_CR1_UE | UART_CR1_TE | UART_CR1_RE;
        regs_.SR = UART_SR_TXE | UART_SR_TC;  // Ready to transmit
        initialized_ = true;
        std::cout << "    UART init: BRR=" << regs_.BRR << " (baud=" << baud << ")\n";
        return true;
    }

    void RegisterRxCallback(RxCallback cb) {
        rx_callback_ = std::move(cb);
        regs_.CR1 |= UART_CR1_RXNEIE;
    }

    // EN: Polling TX — wait for TXE, write byte
    // TR: Yoklamalı TX — TXE bayrağını bekle, byte yaz (gerçek HW'de çevrim döngüsü gerekir)
    bool TransmitByte(uint8_t byte) {
        if (!initialized_) return false;
        // EN: In real HW, we'd spin on !(regs_.SR & UART_SR_TXE)
        // TR: Gerçek HW'de TXE=0 iken döngüde beklenir — burada simülasyon olduğu için atlıyoruz
        regs_.DR = byte;
        regs_.SR |= UART_SR_TC;
        return true;
    }

    bool TransmitString(std::string_view msg) {
        for (char c : msg) {
            if (!TransmitByte(static_cast<uint8_t>(c))) return false;
        }
        return true;
    }

    // EN: Simulate receiving a byte (as if hardware put data in DR)
    // TR: Byte alma simülasyonu — donanım DR'ye veri koymuş gibi davranır ve ISR tetikler
    void SimulateRxIRQ(uint8_t byte) {
        regs_.DR = byte;
        regs_.SR |= UART_SR_RXNE;
        // EN: ISR handler
        // TR: ISR işleyici — RXNE bayrağı kontrol edilir, veri FIFO'ya yazılır, callback çağrılır
        HandleRxIRQ();
    }

    std::optional<uint8_t> ReadByte() { return rx_fifo_.Pop(); }
    [[nodiscard]] size_t RxAvailable() const { return rx_fifo_.Size(); }

private:
    void HandleRxIRQ() {
        if (regs_.SR & UART_SR_RXNE) {
            auto byte = static_cast<uint8_t>(regs_.DR & 0xFF);
            rx_fifo_.Push(byte);
            regs_.SR &= ~UART_SR_RXNE;
            if (rx_callback_) rx_callback_(byte);
        }
    }
};

}  // namespace uart_driver

// ═══════════════════════════════════════════════════════════════════════════
// PART 4: SPI Driver — Full Duplex + Chip Select
// ═══════════════════════════════════════════════════════════════════════════

namespace spi_driver {

struct SPI_Regs {
    uint32_t CR1 = 0;
    uint32_t CR2 = 0;
    uint32_t SR  = 0;
    uint32_t DR  = 0;
};

```

```cpp
constexpr uint32_t SPI_SR_TXE  = (1u << 1);
constexpr uint32_t SPI_SR_RXNE = (1u << 0);
constexpr uint32_t SPI_SR_BSY  = (1u << 7);

enum class SPI_Mode : uint8_t { MODE_0 = 0, MODE_1, MODE_2, MODE_3 };

struct SPI_Config {
    uint32_t clock_divider = 8;
    SPI_Mode mode = SPI_Mode::MODE_0;
    bool lsb_first = false;
};

class SPIDriver {
    SPI_Regs regs_{};
    SPI_Config config_{};
    bool cs_active_ = false;
    bool initialized_ = false;

    // EN: Simulated shift register (loopback: MOSI → MISO)
    // TR: Simüle edilmiş kaydırma register'ı (geri döngü: MOSI → MISO) — test amaçlı
    uint8_t shift_reg_ = 0;

public:
    bool Init(const SPI_Config& cfg) {
        config_ = cfg;
        uint32_t cpol = (static_cast<uint8_t>(cfg.mode) >> 1) & 1u;
        uint32_t cpha = static_cast<uint8_t>(cfg.mode) & 1u;
        regs_.CR1 = (cpol << 1) | cpha | (1u << 6);  // SPE=1
        regs_.SR = SPI_SR_TXE;
        initialized_ = true;
        std::cout << "    SPI init: div=" << cfg.clock_divider
                  << " mode=" << static_cast<int>(cfg.mode)
                  << " LSB=" << (cfg.lsb_first ? "yes" : "no") << "\n";
        return true;
    }

    void ChipSelect(bool active) {
        cs_active_ = active;
        std::cout << "    CS " << (active ? "LOW (active)" : "HIGH (inactive)") << "\n";
    }

    // EN: Full-duplex transfer: send tx_byte, return rx_byte
    // TR: Tam çift yönlü transfer: tx_byte gönder, rx_byte döndür
    uint8_t Transfer(uint8_t tx_byte) {
        if (!initialized_ || !cs_active_) return 0xFF;

        regs_.DR = tx_byte;
        regs_.SR |= SPI_SR_BSY;

        // EN: Simulate loopback (MOSI → MISO with bit inversion for variety)
        // TR: Geri döngü simülasyonu (MOSI → MISO, çeşitlilik için bit terslemesi) — gerçek HW'de slave cevap verir
        shift_reg_ = static_cast<uint8_t>(~tx_byte);

        regs_.SR &= ~SPI_SR_BSY;
        regs_.SR |= SPI_SR_RXNE;

        return shift_reg_;
    }

    // EN: Multi-byte transfer
    // TR: Çok byte'lık transfer — blok halinde veri gönderip alır, DMA ile hızlandırılabilir
    std::vector<uint8_t> TransferBlock(const std::vector<uint8_t>& tx_data) {
        std::vector<uint8_t> rx_data;
        rx_data.reserve(tx_data.size());
        for (auto b : tx_data) {
            rx_data.push_back(Transfer(b));
        }
        return rx_data;
    }
};

}  // namespace spi_driver
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════
// PART 5: Driver State Machine
// ═══════════════════════════════════════════════════════════════════════════

namespace driver_fsm {

enum class DriverState : uint8_t {
    UNINITIALIZED,
    INITIALIZED,
    RUNNING,
    ERROR,
    STOPPED
};

constexpr std::string_view StateToString(DriverState s) {
    switch (s) {
        case DriverState::UNINITIALIZED: return "UNINITIALIZED";
        case DriverState::INITIALIZED:   return "INITIALIZED";
        case DriverState::RUNNING:       return "RUNNING";
        case DriverState::ERROR:         return "ERROR";
        case DriverState::STOPPED:       return "STOPPED";
    }
    return "UNKNOWN";
}

// EN: Return code for driver operations (instead of exceptions)
// TR: Sürücü işlemleri için dönüş kodu (istisnalar yerine)
enum class DriverResult : uint8_t {
    OK = 0,
    ERROR_INVALID_STATE,
    ERROR_HW_FAULT,
    ERROR_TIMEOUT,
    ERROR_PARAM
};

constexpr std::string_view ResultToString(DriverResult r) {
    switch (r) {
        case DriverResult::OK:                  return "OK";
        case DriverResult::ERROR_INVALID_STATE: return "INVALID_STATE";
        case DriverResult::ERROR_HW_FAULT:      return "HW_FAULT";
        case DriverResult::ERROR_TIMEOUT:       return "TIMEOUT";
        case DriverResult::ERROR_PARAM:         return "BAD_PARAM";
    }
    return "UNKNOWN";
}

// EN: Generic peripheral driver with state machine
// TR: Durum makineli genel çevre birimi sürücüsü
class ManagedDriver {
    DriverState state_ = DriverState::UNINITIALIZED;
    std::string_view name_;

    bool IsValidTransition(DriverState to) const {
        switch (state_) {
            case DriverState::UNINITIALIZED:
                return to == DriverState::INITIALIZED;
            case DriverState::INITIALIZED:
                return to == DriverState::RUNNING || to == DriverState::UNINITIALIZED;
            case DriverState::RUNNING:
                return to == DriverState::STOPPED || to == DriverState::ERROR;
            case DriverState::ERROR:
                return to == DriverState::STOPPED;
            case DriverState::STOPPED:
                return to == DriverState::UNINITIALIZED;
        }
        return false;
    }
```

```

    DriverResult Transition(DriverState to) {
        if (!IsValidTransition(to)) {
            std::cout << "    [" << name_ << "] REJECTED: "
                      << StateToString(state_) << " → " << StateToString(to) << "\n";
            return DriverResult::ERROR_INVALID_STATE;
        }
        std::cout << "    [" << name_ << "] " << StateToString(state_)
                  << " → " << StateToString(to) << "\n";
        state_ = to;
        return DriverResult::OK;
    }

public:
    explicit ManagedDriver(std::string_view name) : name_(name) {}

    DriverResult Init()   { return Transition(DriverState::INITIALIZED); }
    DriverResult Start()  { return Transition(DriverState::RUNNING); }
    DriverResult Stop()   { return Transition(DriverState::STOPPED); }
    DriverResult DeInit() { return Transition(DriverState::UNINITIALIZED); }
    DriverResult ReportError() { return Transition(DriverState::ERROR); }

    [[nodiscard]] DriverState GetState() const { return state_; }
};

}  // namespace driver_fsm

// ═══════════════════════════════════════════════════════════════════════════
// PART 6: Error Handling with Return Codes and std::expected-like
// ═══════════════════════════════════════════════════════════════════════════

namespace error_handling {

// EN: Lightweight Expected type (C++23 std::expected not always available)
// TR: Hafif Expected tipi (C++23 std::expected her yerde mevcut değildir)
template <typename T, typename E>
class Expected {
    bool has_value_;
    union { T value_; E error_; };

public:
    // EN: Success constructor
    // TR: Başarı yapıcısı — geçerli değer taşıyan Expected nesnesi oluşturur
    static Expected Success(const T& val) {
        Expected e;
        e.has_value_ = true;
        e.value_ = val;
        return e;
    }

    // EN: Error constructor
    // TR: Hata yapıcısı — hata bilgisi taşıyan Expected nesnesi oluşturur (istisna yerine)
    static Expected Failure(const E& err) {
        Expected e;
        e.has_value_ = false;
        e.error_ = err;
        return e;
    }

    [[nodiscard]] bool HasValue() const { return has_value_; }
    [[nodiscard]] const T& Value() const { return value_; }
    [[nodiscard]] const E& Error() const { return error_; }

private:
    Expected() : has_value_(false), error_{} {}
};
```

```cpp

// EN: Example: ADC read with error handling
// TR: Örnek: Hata yönetimi ile ADC okuma
enum class ADCError : uint8_t { TIMEOUT, NOT_CALIBRATED, CHANNEL_INVALID };

constexpr std::string_view ADCErrorStr(ADCError e) {
    switch (e) {
        case ADCError::TIMEOUT:         return "TIMEOUT";
        case ADCError::NOT_CALIBRATED:  return "NOT_CALIBRATED";
        case ADCError::CHANNEL_INVALID: return "CHANNEL_INVALID";
    }
    return "UNKNOWN";
}

class ADCDriver {
    bool calibrated_ = false;
    static constexpr uint32_t MAX_CHANNELS = 16;

public:
    void Calibrate() {
        calibrated_ = true;
        std::cout << "    ADC calibrated\n";
    }

    Expected<uint16_t, ADCError> Read(uint32_t channel) {
        if (!calibrated_)
            return Expected<uint16_t, ADCError>::Failure(ADCError::NOT_CALIBRATED);
        if (channel >= MAX_CHANNELS)
            return Expected<uint16_t, ADCError>::Failure(ADCError::CHANNEL_INVALID);

        // EN: Simulated conversion result
        // TR: Simüle edilmiş dönüşüm sonucu — gerçek HW'de ADC kanal değerine göre analog okuma yapar
        auto value = static_cast<uint16_t>(1000 + channel * 100);
        return Expected<uint16_t, ADCError>::Success(value);
    }
};

}  // namespace error_handling

// ═══════════════════════════════════════════════════════════════════════
// MAIN — Demos
// ═══════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "╔═══════════════════════════════════════════════════════════╗\n"
              << "║ Module 18: Peripheral Driver — GPIO, UART, SPI, FSM       ║\n"
              << "╚═══════════════════════════════════════════════════════════╝\n\n";

    // --- Demo 1: Register Bit Manipulation ---
    std::cout << "--- Demo 1: Register Bit Manipulation ---\n";
    {
        uint32_t reg = 0;
        reg_ops::RegOps::SetBit(reg, 3);
        reg_ops::RegOps::SetBit(reg, 7);
        std::cout << "    SET bits 3,7:   0x" << std::hex << reg << std::dec << "\n";

        reg_ops::RegOps::ClearBit(reg, 3);
        std::cout << "    CLEAR bit 3:    0x" << std::hex << reg << std::dec << "\n";

        reg_ops::RegOps::ToggleBit(reg, 7);
        std::cout << "    TOGGLE bit 7:   0x" << std::hex << reg << std::dec << "\n";

        // EN: Multi-bit field: set bits [5:4] = 0b11
        // TR: Çok bitli alan: [5:4] bitlerini 0b11 yap — ModifyField maskeleme ile güvenli değiştirir
        reg = 0;
        reg_ops::RegOps::ModifyField(reg, 4, 2, 3);
        std::cout << "    ModifyField[5:4]=3: 0x" << std::hex << reg << std::dec << "\n";

```

```cpp
        // EN: Type-safe RegisterField
        // TR: Tip-güvenli RegisterField — derleme zamanı bit konumu/genişliği doğrulaması sağlar
        using MODER_PIN5 = reg_ops::RegisterField<10, 2>;  // Pin 5: bits [11:10]
        uint32_t moder = 0;
        MODER_PIN5::Set(moder, 0b01);  // Output
        std::cout << "    RegisterField<10,2> set to 01: MODER=0x"
                  << std::hex << moder << std::dec
                  << " read=" << MODER_PIN5::Get(moder) << "\n";
    }

    // --- Demo 2: GPIO Driver ---
    std::cout << "\n--- Demo 2: GPIO Driver ---\n";
    {
        gpio_driver::GPIODriver gpioa("GPIOA");

        gpioa.ConfigurePin(5, gpio_driver::PinMode::OUTPUT, gpio_driver::PullType::NONE);
        gpioa.ConfigurePin(13, gpio_driver::PinMode::INPUT, gpio_driver::PullType::PULL_UP);

        gpioa.WritePin(5, true);
        std::cout << "    Pin 5 written HIGH, read=" << gpioa.ReadPin(5) << "\n";

        gpioa.TogglePin(5);
        std::cout << "    Pin 5 toggled,      read=" << gpioa.ReadPin(5) << "\n";

        gpioa.TogglePin(5);
        std::cout << "    Pin 5 toggled,      read=" << gpioa.ReadPin(5) << "\n";

        gpioa.PrintRegisters();
    }

    // --- Demo 3: UART Driver — TX Polling + RX ISR Callback ---
    std::cout << "\n--- Demo 3: UART Driver — TX Polling + RX ISR ---\n";
    {
        uart_driver::UARTDriver uart;
        uart.Init(168'000'000, 115200);

        // EN: Register RX callback
        // TR: RX geri çağrı fonksiyonunu kaydet — her alınan byte'ta ISR içinden çağrılır
        int rx_count = 0;
        uart.RegisterRxCallback([&rx_count](uint8_t byte) {
            std::cout << "    [ISR] RX byte: 0x" << std::hex
                      << static_cast<int>(byte) << std::dec << "\n";
            ++rx_count;
        });

        // EN: TX (polling)
        // TR: TX (yoklamalı) — TXE bayrağı kontrol edilerek byte byte gönderilir
        std::cout << "    TX: ";
        uart.TransmitString("Hi");
        std::cout << "sent 'Hi'\n";

        // EN: Simulate RX interrupts
        // TR: RX kesmelerini simüle et — donanım DR'ye veri koymuş gibi SimulateRxIRQ çağrılır
        std::cout << "    Simulating 3 RX interrupts:\n";
        uart.SimulateRxIRQ(0x41);  // 'A'
        uart.SimulateRxIRQ(0x42);  // 'B'
        uart.SimulateRxIRQ(0x43);  // 'C'

        std::cout << "    RX FIFO size: " << uart.RxAvailable()
                  << ", callback invoked " << rx_count << " times\n";

        // EN: Read from FIFO
        // TR: FIFO'dan oku — halka tamponundaki verileri sırayla çıkar (FIFO prensibi)
        while (auto byte = uart.ReadByte()) {
            std::cout << "    FIFO pop: '" << static_cast<char>(*byte) << "'\n";
        }
    }

    // --- Demo 4: SPI Driver — Full Duplex ---
    std::cout << "\n--- Demo 4: SPI Driver — Full Duplex ---\n";
    {
        spi_driver::SPIDriver spi;
        spi_driver::SPI_Config cfg{.clock_divider = 4, .mode = spi_driver::SPI_Mode::MODE_0,
.lsb_first = false};
        spi.Init(cfg);

        spi.ChipSelect(true);
```

```

        // EN: Single byte
        // TR: Tek byte transferi — tam çift yönlü: TX gönderilirken eşzamanlı RX alınır
        uint8_t rx = spi.Transfer(0xAA);
        std::cout << "    TX=0xAA → RX=0x" << std::hex << static_cast<int>(rx) << std::dec << "\n";

        // EN: Block transfer
        // TR: Blok transfer — birden çok byte'ı sırayla gönderip alır
        std::vector<uint8_t> tx_data = {0x10, 0x20, 0x30, 0x40};
        auto rx_data = spi.TransferBlock(tx_data);
        std::cout << "    Block TX: ";
        for (auto b : tx_data) std::cout << "0x" << std::hex << static_cast<int>(b) << " ";
        std::cout << "\n    Block RX: ";
        for (auto b : rx_data) std::cout << "0x" << std::hex << static_cast<int>(b) << " ";
        std::cout << std::dec << "\n";

        spi.ChipSelect(false);
    }

    // --- Demo 5: Driver State Machine ---
    std::cout << "\n--- Demo 5: Driver State Machine ---\n";
    {
        driver_fsm::ManagedDriver drv("ADC_Driver");

        std::cout << "  Valid lifecycle:\n";
        auto r1 = drv.Init();
        auto r2 = drv.Start();
        auto r3 = drv.Stop();
        auto r4 = drv.DeInit();
        std::cout << "    Results: " << driver_fsm::ResultToString(r1) << ", "
                  << driver_fsm::ResultToString(r2) << ", "
                  << driver_fsm::ResultToString(r3) << ", "
                  << driver_fsm::ResultToString(r4) << "\n";

        std::cout << "  Invalid transitions:\n";
        drv.Init();
        auto bad1 = drv.Stop();   // Init → Stop = invalid
        auto bad2 = drv.DeInit(); // Init → Uninit (but tested as Stop first)
        std::cout << "    Stop from INITIALIZED: " << driver_fsm::ResultToString(bad1) << "\n";
        std::cout << "    DeInit from INITIALIZED: " << driver_fsm::ResultToString(bad2) << "\n";

        std::cout << "  Error recovery:\n";
        driver_fsm::ManagedDriver drv2("SPI_Driver");
        drv2.Init();
        drv2.Start();
        drv2.ReportError();
        drv2.Stop();
        drv2.DeInit();
    }

    // --- Demo 6: Error Handling with Expected ---
    std::cout << "\n--- Demo 6: Error Handling with Expected ---\n";
    {
        error_handling::ADCDriver adc;

        // EN: Read before calibration
        // TR: Kalibrasyon öncesi okuma — kalibre edilmemiş ADC hata döndürmeli
        auto r1 = adc.Read(0);
        if (!r1.HasValue())
            std::cout << "    Read ch0 (uncalibrated): ERROR="
                      << error_handling::ADCErrorStr(r1.Error()) << "\n";

        adc.Calibrate();

        // EN: Valid read
        // TR: Geçerli okuma — kalibre edilmiş ADC'den kanal 3 değeri başarıyla okunur
        auto r2 = adc.Read(3);
        if (r2.HasValue())
            std::cout << "    Read ch3: value=" << r2.Value() << "\n";

```

```
705         // EN: Invalid channel
706         // TR: Geçersiz kanal — MAX_CHANNELS üstü kanal numarası CHANNEL_INVALID hatası döndürür
707         auto r3 = adc.Read(20);
708         if (!r3.HasValue())
709             std::cout << "    Read ch20: ERROR="
710                       << error_handling::ADCErrorStr(r3.Error()) << "\n";
711
712         // EN: Read all valid channels
713         // TR: Tüm geçerli kanalları oku — döngüde Expected<T,E> ile hata kontrolü yapılır
714         std::cout << "    All channels: ";
715         for (uint32_t ch = 0; ch < 4; ++ch) {
716             auto r = adc.Read(ch);
717             if (r.HasValue()) std::cout << "ch" << ch << "=" << r.Value() << " ";
718         }
719         std::cout << "\n";
720     }
721
722     // --- Demo 7: Driver Layer Summary ---
723     std::cout << "\n--- Demo 7: Peripheral Driver Architecture Summary ---\n";
724     {
725         struct Layer { std::string_view name; std::string_view description; };
726         Layer layers[] = {
727             {"Register Ops",    "Raw bit set/clear/toggle/modify"},
728             {"RegisterField<>", "Type-safe compile-time field access"},
729             {"GPIO Driver",     "Pin config, write, read, toggle via BSRR"},
730             {"UART Driver",     "Polling TX, interrupt RX with ring buffer"},
731             {"SPI Driver",      "Full-duplex transfer, chip select control"},
732             {"State Machine",   "UNINIT→INIT→RUN→STOP→UNINIT lifecycle    "},
733             {"Error Handling",  "Return codes + Expected<T,E> pattern"},
734         };
735         std::cout << "  ┌──────────────────┬──────────────────────────────────────┐\n";
736         std::cout << "  │ Component        │ Description                          │\n";
737         std::cout << "  ├──────────────────┼──────────────────────────────────────┤\n";
738         for (const auto& l : layers) {
739             std::cout << "  │ " << std::setw(17) << std::left << l.name
740                       << " │ " << std::setw(41) << l.description << " │\n";
741         }
742         std::cout << std::right;
743         std::cout << "  └──────────────────┴──────────────────────────────────────┘\n";
744     }
745
746     std::cout << "\n[All demos completed successfully.]\n";
747     return 0;
748 }
```

► 04. Mini Ecu Project

module_18_driver_development/04_mini_ecu_project.cpp

Konu: Capstone Project — Body Control Module (BCM) Simulation

Teori / Theory

THEORY: BCM (Body Control Module) Architecture

A BCM manages body electronics in a vehicle: lights, wipers, windows, door locks, interior lighting, etc. It communicates via CAN and LIN buses, stores diagnostic trouble codes (DTCs), responds to UDS diagnostic requests, and runs cyclic tasks at different periods.
BCM Software Architecture:

```
┌──────────────────────────────────────────────┐
│                  APPLICATION                 │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐  │
│  │Headlamp │ │Door Lock │ │Wiper    │ │Interior  │  │
│  │Control  │ │Control   │ │Control  │ │Light     │  │
│  └────┬────┘ └────┬─────┘ └────┬────┘ └────┬─────┘  │
│       │           │            │           │        │
├───────┴───────────┴────────────┴───────────┴────────┤
│                SERVICES LAYER                 │
│  ┌────┐ ┌────┐ ┌────┐ ┌────┐ ┌────┐ ┌────┐  │
│  │ CAN │ │ LIN │ │ NvM │ │ DTC │ │  UDS │ │ WDG  │  │
│  └────┘ └────┘ └────┘ └────┘ └────┘ └────┘  │
├──────────────────────────────────────────────┤
│                 HAL / DRIVERS                │
│      GPIO  |  UART  |  SPI  |  Timer  |  Flash     |
└──────────────────────────────────────────────┘
```

THEORY: Cyclic Task Scheduling

```
┌──────────┬──────────────────────────────┐
│ Period    │ Tasks                        │
├──────────┼──────────────────────────────┤
│ 10 ms     │ CAN RX, Debounce, Watchdog feed  │
│ 50 ms     │ Lamp control, Wiper control      │
│ 100 ms    │ LIN schedule, DTC check, NvM     │
│ 1000 ms   │ Diagnostics, Health monitoring   │
└──────────┴──────────────────────────────┘
```

TEORİ: Döngüsel Görev Zamanlama

BCM, araç gövde elektroniğini yönetir: farlar, silecekler, camlar, kapı kilitleri, iç aydınlatma vb. CAN ve LIN bus'ları üzerinden haberleşir, DTC (arıza kodu) saklar, UDS tanı taleplerine cevap verir.
BCM Yazılım Mimarisi:

```
┌──────────────────────────────────────────────┐
│                  UYGULAMA                    │
│  ┌─────────┐ ┌──────────┐ ┌─────────┐ ┌──────────┐  │
│  │Far      │ │Kapı Kilit│ │Silecek  │ │İç Aydın. │  │
│  │Kontrol  │ │Kontrol   │ │Kontrol  │ │Kontrol   │  │
│  └────┬────┘ └────┬─────┘ └────┬────┘ └────┬─────┘  │
├───────┴──────────┴──────────┴──────────┴────────┤
│              SERVİS KATMANI                  │
│  ┌────┐ ┌────┐ ┌────┐ ┌────┐ ┌────┐ ┌────┐  │
│  │ CAN │ │ LIN │ │ NvM │ │ DTC │ │  UDS │ │ WDG  │  │
│  └────┘ └────┘ └────┘ └────┘ └────┘ └────┘  │
├──────────────────────────────────────────────┤
│               HAL / SÜRÜCÜLER                │
│    GPIO  |  UART  |  SPI  |  Timer  |  Flash      |
└──────────────────────────────────────────────┘
```

Periyodik görev zamanlası:

```
┌──────────┬──────────────────────────────┐
│ Periyot  │ Görevler                     │
├──────────┼──────────────────────────────┤
│ 10 ms    │ CAN RX, debounce, WDG besleme    │
│ 50 ms    │ Lamba kontrolü, silecek kontrolü │
│ 100 ms   │ LIN çizelgesi, DTC kontrol, NvM  │
│ 1000 ms  │ Tanı, sağlık izleme              │
└──────────┴──────────────────────────────┘
```

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <cstdint>
#include <string>
#include <string_view>
#include <array>
#include <vector>
#include <functional>
#include <iomanip>
#include <optional>
#include <cassert>
#include <map>
#include <algorithm>

// ═══════════════════════════════════════════════════════════════════════════
// PART 1: GPIO Module — Digital I/O for BCM
// ═══════════════════════════════════════════════════════════════════════════

namespace bcm {

enum class PinState : uint8_t { LOW = 0, HIGH = 1 };

```

```cpp
// EN: Named BCM GPIO pins
// TR: İsimlendirilmiş BCM GPIO pinleri
enum class BCM_Pin : uint8_t {
    HEADLAMP_LEFT = 0,
    HEADLAMP_RIGHT,
    TAIL_LAMP_LEFT,
    TAIL_LAMP_RIGHT,
    INTERIOR_LIGHT,
    DOOR_LOCK_MOTOR,
    WIPER_MOTOR,
    BRAKE_LIGHT,
    DOOR_SWITCH,      // Input
    LIGHT_SENSOR,     // Input (simulated)
    PIN_COUNT
};

constexpr std::string_view PinName(BCM_Pin pin) {
    switch (pin) {
        case BCM_Pin::HEADLAMP_LEFT:   return "HeadlampL";
        case BCM_Pin::HEADLAMP_RIGHT:  return "HeadlampR";
        case BCM_Pin::TAIL_LAMP_LEFT:  return "TailLampL";
        case BCM_Pin::TAIL_LAMP_RIGHT: return "TailLampR";
        case BCM_Pin::INTERIOR_LIGHT:  return "InteriorLt";
        case BCM_Pin::DOOR_LOCK_MOTOR: return "DoorLock";
        case BCM_Pin::WIPER_MOTOR:     return "WiperMotor";
        case BCM_Pin::BRAKE_LIGHT:     return "BrakeLamp";
        case BCM_Pin::DOOR_SWITCH:     return "DoorSwitch";
        case BCM_Pin::LIGHT_SENSOR:    return "LightSens";
        case BCM_Pin::PIN_COUNT:       break;
    }
    return "Unknown";
}

class GPIOModule {
    std::array<PinState, static_cast<size_t>(BCM_Pin::PIN_COUNT)> pins_{};

public:
    GPIOModule() { pins_.fill(PinState::LOW); }

    void Write(BCM_Pin pin, PinState state) {
        pins_[static_cast<size_t>(pin)] = state;
    }

    PinState Read(BCM_Pin pin) const {
        return pins_[static_cast<size_t>(pin)];
    }

    void Toggle(BCM_Pin pin) {
        auto& p = pins_[static_cast<size_t>(pin)];
        p = (p == PinState::HIGH) ? PinState::LOW : PinState::HIGH;
    }

    void DumpOutputs() const {
        std::cout << "    GPIO: ";
        for (size_t i = 0; i < static_cast<size_t>(BCM_Pin::PIN_COUNT); ++i) {
            auto pin = static_cast<BCM_Pin>(i);
            if (pins_[i] == PinState::HIGH)
                std::cout << PinName(pin) << "=ON ";
        }
        std::cout << "\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 2: CAN Module — Message Routing
// ═══════════════════════════════════════════════════════════════════════════

struct CAN_Message {
    uint32_t id = 0;
    uint8_t dlc = 0;
    std::array<uint8_t, 8> data{};
};
```

```

// EN: Known CAN message IDs for BCM
// TR: BCM için bilinen CAN mesaj ID'leri — her komut sabit bir ID ile tanımlanır
constexpr uint32_t CAN_ID_LIGHT_CMD    = 0x120;
constexpr uint32_t CAN_ID_DOOR_CMD     = 0x130;
constexpr uint32_t CAN_ID_WIPER_CMD    = 0x140;
constexpr uint32_t CAN_ID_BRAKE_STATUS = 0x150;
constexpr uint32_t CAN_ID_BCM_STATUS   = 0x200;

class CANModule {
    std::vector<CAN_Message> rx_queue_;
    std::vector<CAN_Message> tx_queue_;

public:
    // EN: Simulate receiving a CAN message
    // TR: CAN mesajı alımını simüle et — test amaçlı RX kuyruğuna mesaj ekler
    void SimulateRx(const CAN_Message& msg) { rx_queue_.push_back(msg); }

    std::optional<CAN_Message> GetNextRx() {
        if (rx_queue_.empty()) return std::nullopt;
        auto msg = rx_queue_.front();
        rx_queue_.erase(rx_queue_.begin());
        return msg;
    }

    void Transmit(const CAN_Message& msg) {
        tx_queue_.push_back(msg);
    }

    void PrintTxQueue() const {
        for (const auto& msg : tx_queue_) {
            std::cout << "    CAN TX [0x" << std::hex << msg.id << std::dec
                      << "] DLC=" << static_cast<int>(msg.dlc) << " data:";
            for (uint8_t i = 0; i < msg.dlc && i < 8; ++i)
                std::cout << " " << std::hex << std::setw(2) << std::setfill('0')
                          << static_cast<int>(msg.data[i]);
            std::cout << std::dec << std::setfill(' ') << "\n";
        }
    }

    void ClearTx() { tx_queue_.clear(); }
    [[nodiscard]] size_t RxPending() const { return rx_queue_.size(); }
};

// ═══════════════════════════════════════════════════════════════════════
// PART 3: LIN Module — Schedule Table Execution
// ═══════════════════════════════════════════════════════════════════════

struct LIN_Frame {
    uint8_t pid = 0;       // Protected identifier — TR: Korumalı tanımlayıcı (PID)
    uint8_t length = 0;
    std::array<uint8_t, 8> data{};
    std::string_view description;
};

class LINModule {
    struct ScheduleEntry { uint8_t pid; uint16_t slot_ms; };

    std::vector<ScheduleEntry> schedule_;
    size_t schedule_idx_ = 0;
    std::vector<LIN_Frame> responses_;

public:
    void AddScheduleEntry(uint8_t pid, uint16_t slot_ms) {
        schedule_.push_back({pid, slot_ms});
    }

    void AddResponse(const LIN_Frame& frame) {
        responses_.push_back(frame);
    }
```

```cpp

    // EN: Execute one slot of the schedule table
    // TR: Çizelge tablosunun bir slotunu çalıştır — sıradaki PID'ye ait yanıt aranır
    std::optional<LIN_Frame> ExecuteSlot() {
        if (schedule_.empty()) return std::nullopt;

        auto& entry = schedule_[schedule_idx_];
        schedule_idx_ = (schedule_idx_ + 1) % schedule_.size();

        // EN: Find matching response
        // TR: Eşleşen yanıtı bul — PID'ye göre önceden kaydedilmiş LIN frame döndürür
        for (const auto& resp : responses_) {
            if (resp.pid == entry.pid) return resp;
        }
        return std::nullopt;
    }

    void PrintSchedule() const {
        std::cout << "    LIN Schedule:\n";
        for (size_t i = 0; i < schedule_.size(); ++i) {
            std::cout << "      Slot " << i << ": PID=0x" << std::hex
                      << static_cast<int>(schedule_[i].pid) << std::dec
                      << " period=" << schedule_[i].slot_ms << "ms\n";
        }
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 4: DTC Storage + NvM (Non-Volatile Memory)
// ═══════════════════════════════════════════════════════════════════════════

enum class DTC_Status : uint8_t {
    PASSIVE = 0,
    ACTIVE  = 1,
    CONFIRMED = 2
};

struct DTC_Entry {
    uint32_t code = 0;
    DTC_Status status = DTC_Status::PASSIVE;
    uint16_t occurrence_count = 0;
    uint32_t first_occurrence_tick = 0;
    std::string_view description;
};

constexpr std::string_view DTCStatusStr(DTC_Status s) {
    switch (s) {
        case DTC_Status::PASSIVE:   return "PASSIVE";
        case DTC_Status::ACTIVE:    return "ACTIVE";
        case DTC_Status::CONFIRMED: return "CONFIRMED";
    }
    return "UNKNOWN";
}

class DTCStore {
    static constexpr size_t MAX_DTCS = 16;
    std::array<DTC_Entry, MAX_DTCS> dtcs_{};
    size_t count_ = 0;

public:
    bool Report(uint32_t code, std::string_view desc, uint32_t tick) {
        // EN: Find existing or add new
        // TR: Mevcut DTC'yi bul veya yeni ekle — aynı kod 3 kez tekrarlanırsa CONFIRMED olur
        for (size_t i = 0; i < count_; ++i) {
            if (dtcs_[i].code == code) {
                dtcs_[i].status = DTC_Status::ACTIVE;
                dtcs_[i].occurrence_count++;
                if (dtcs_[i].occurrence_count >= 3)
                    dtcs_[i].status = DTC_Status::CONFIRMED;
                return true;
            }
        }
        if (count_ >= MAX_DTCS) return false;
        dtcs_[count_++] = {code, DTC_Status::ACTIVE, 1, tick, desc};
        return true;
    }
```

```cpp

    void ClearAll() {
        for (size_t i = 0; i < count_; ++i)
            dtcs_[i].status = DTC_Status::PASSIVE;
        count_ = 0;
    }

    void Print() const {
        std::cout << "    DTC Store (" << count_ << " entries):\n";
        for (size_t i = 0; i < count_; ++i) {
            std::cout << "      [0x" << std::hex << std::setw(6) << std::setfill('0')
                      << dtcs_[i].code << std::dec << std::setfill(' ')
                      << "] " << std::setw(10) << DTCStatusStr(dtcs_[i].status)
                      << " cnt=" << dtcs_[i].occurrence_count
                      << " " << dtcs_[i].description << "\n";
        }
    }

    [[nodiscard]] size_t Count() const { return count_; }

    std::optional<DTC_Entry> Get(size_t idx) const {
        if (idx >= count_) return std::nullopt;
        return dtcs_[idx];
    }
};

// EN: Simple NvM simulation (learn values / calibration data)
// TR: Basit NvM simülasyonu (öğrenme değerleri / kalibrasyon verisi)
class NvMStore {
    static constexpr size_t MAX_BLOCKS = 8;
    struct NvMBlock { uint16_t id; uint32_t data; bool valid; };
    std::array<NvMBlock, MAX_BLOCKS> blocks_{};
    size_t count_ = 0;

public:
    bool Write(uint16_t id, uint32_t data) {
        for (size_t i = 0; i < count_; ++i) {
            if (blocks_[i].id == id) {
                blocks_[i].data = data;
                blocks_[i].valid = true;
                return true;
            }
        }
        if (count_ >= MAX_BLOCKS) return false;
        blocks_[count_++] = {id, data, true};
        return true;
    }

    std::optional<uint32_t> Read(uint16_t id) const {
        for (size_t i = 0; i < count_; ++i) {
            if (blocks_[i].id == id && blocks_[i].valid)
                return blocks_[i].data;
        }
        return std::nullopt;
    }

    void Print() const {
        std::cout << "    NvM Store (" << count_ << " blocks):\n";
        for (size_t i = 0; i < count_; ++i) {
            std::cout << "      Block 0x" << std::hex << blocks_[i].id << std::dec
                      << " = " << blocks_[i].data
                      << (blocks_[i].valid ? " [valid]" : " [invalid]") << "\n";
        }
    }
};

```

```cpp
// ═══════════════════════════════════════════════════════════════════════
// PART 5: UDS Diagnostic Handler
// ═══════════════════════════════════════════════════════════════════════

class UDSHandler {
public:
    // EN: UDS Service IDs
    // TR: UDS Servis ID'leri — ISO 14229 standardına göre tanı hizmet kodları
    static constexpr uint8_t SID_READ_DID       = 0x22;
    static constexpr uint8_t SID_WRITE_DID      = 0x2E;
    static constexpr uint8_t SID_CLEAR_DTC      = 0x14;
    static constexpr uint8_t SID_READ_DTC_INFO  = 0x19;
    static constexpr uint8_t SID_ROUTINE_CTRL   = 0x31;

    // EN: DID identifiers
    // TR: DID tanımlayıcıları — okuma/yazma yapılabilecek veri noktalarının adresleri
    static constexpr uint16_t DID_SW_VERSION    = 0xF189;
    static constexpr uint16_t DID_HW_VERSION    = 0xF191;
    static constexpr uint16_t DID_SERIAL_NUMBER = 0xF18C;
    static constexpr uint16_t DID_LAMP_STATUS   = 0x0100;

    struct UDS_Request  { uint8_t sid; uint16_t param; std::vector<uint8_t> data; };
    struct UDS_Response { uint8_t sid; bool positive; uint8_t nrc; std::vector<uint8_t> data; };

    UDS_Response Process(const UDS_Request& req, DTCStore& dtcs, NvMStore& nvm,
                         const GPIOModule& gpio) {
        switch (req.sid) {
            case SID_READ_DID:    return HandleReadDID(req, nvm, gpio);
            case SID_WRITE_DID:   return HandleWriteDID(req, nvm);
            case SID_CLEAR_DTC:   return HandleClearDTC(dtcs);
            case SID_READ_DTC_INFO: return HandleReadDTCInfo(dtcs);
            case SID_ROUTINE_CTRL:  return HandleRoutineCtrl(req);
            default:
                return {req.sid, false, 0x11, {}};  // serviceNotSupported — TR: servis desteklenmiyor
        }
    }

private:
    UDS_Response HandleReadDID(const UDS_Request& req, NvMStore& nvm,
                               const GPIOModule& gpio) {
        switch (req.param) {
            case DID_SW_VERSION:
                return {static_cast<uint8_t>(req.sid + 0x40), true, 0, {'1', '.', '0', '.', '3'}};
            case DID_HW_VERSION:
                return {static_cast<uint8_t>(req.sid + 0x40), true, 0, {'H', 'W', '2', '.', '1'}};
            case DID_SERIAL_NUMBER: {
                auto sn = nvm.Read(0x0001);
                uint32_t val = sn.value_or(0);
                return {static_cast<uint8_t>(req.sid + 0x40), true, 0,
                    {static_cast<uint8_t>((val >> 24) & 0xFF),
                     static_cast<uint8_t>((val >> 16) & 0xFF),
                     static_cast<uint8_t>((val >> 8) & 0xFF),
                     static_cast<uint8_t>(val & 0xFF)}};
            }
            case DID_LAMP_STATUS: {
                uint8_t status = 0;
                if (gpio.Read(BCM_Pin::HEADLAMP_LEFT) == PinState::HIGH)  status |= 0x01;
                if (gpio.Read(BCM_Pin::HEADLAMP_RIGHT) == PinState::HIGH) status |= 0x02;
                if (gpio.Read(BCM_Pin::TAIL_LAMP_LEFT) == PinState::HIGH) status |= 0x04;
                if (gpio.Read(BCM_Pin::TAIL_LAMP_RIGHT) == PinState::HIGH) status |= 0x08;
                return {static_cast<uint8_t>(req.sid + 0x40), true, 0, {status}};
            }
            default:
                return {req.sid, false, 0x31, {}};  // requestOutOfRange — TR: istek aralık dışı
        }
    }
```

```cpp

    static UDS_Response HandleWriteDID(const UDS_Request& req, NvMStore& nvm) {
        if (req.param == DID_SERIAL_NUMBER && req.data.size() >= 4) {
            uint32_t sn = (static_cast<uint32_t>(req.data[0]) << 24) |
                          (static_cast<uint32_t>(req.data[1]) << 16) |
                          (static_cast<uint32_t>(req.data[2]) << 8)  |
                          static_cast<uint32_t>(req.data[3]);
            nvm.Write(0x0001, sn);
            return {static_cast<uint8_t>(req.sid + 0x40), true, 0, {}};
        }
        return {req.sid, false, 0x31, {}};
    }

    static UDS_Response HandleClearDTC(DTCStore& dtcs) {
        dtcs.ClearAll();
        return {static_cast<uint8_t>(SID_CLEAR_DTC + 0x40), true, 0, {}};
    }

    static UDS_Response HandleReadDTCInfo(DTCStore& dtcs) {
        std::vector<uint8_t> data;
        for (size_t i = 0; i < dtcs.Count(); ++i) {
            auto dtc = dtcs.Get(i);
            if (dtc) {
                data.push_back(static_cast<uint8_t>((dtc->code >> 16) & 0xFF));
                data.push_back(static_cast<uint8_t>((dtc->code >> 8) & 0xFF));
                data.push_back(static_cast<uint8_t>(dtc->code & 0xFF));
                data.push_back(static_cast<uint8_t>(dtc->status));
            }
        }
        return {static_cast<uint8_t>(SID_READ_DTC_INFO + 0x40), true, 0, data};
    }

    static UDS_Response HandleRoutineCtrl(const UDS_Request& req) {
        std::cout << "      Routine 0x" << std::hex << req.param << std::dec << " executed\n";
        return {static_cast<uint8_t>(SID_ROUTINE_CTRL + 0x40), true, 0, {0x00}};
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// PART 6: Watchdog Module
// ═══════════════════════════════════════════════════════════════════════════

class Watchdog {
    uint32_t timeout_ticks_;
    uint32_t counter_;
    bool expired_ = false;

public:
    explicit Watchdog(uint32_t timeout) : timeout_ticks_(timeout), counter_(timeout) {}

    void Feed() { counter_ = timeout_ticks_; expired_ = false; }

    void Tick() {
        if (counter_ > 0) --counter_;
        if (counter_ == 0 && !expired_) {
            expired_ = true;
            std::cout << "    [WDG] *** WATCHDOG EXPIRED — SYSTEM RESET ***\n";
        }
    }

    [[nodiscard]] bool IsExpired() const { return expired_; }
    [[nodiscard]] uint32_t Remaining() const { return counter_; }
};
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════
// PART 7: BCM State Manager + Cyclic Task Scheduler
// ═══════════════════════════════════════════════════════════════════════════

enum class BCM_State : uint8_t { STARTUP, NORMAL, SLEEP, WAKEUP };

constexpr std::string_view BCMStateStr(BCM_State s) {
    switch (s) {
        case BCM_State::STARTUP: return "STARTUP";
        case BCM_State::NORMAL:  return "NORMAL";
        case BCM_State::SLEEP:   return "SLEEP";
        case BCM_State::WAKEUP:  return "WAKEUP";
    }
    return "UNKNOWN";
}

// EN: Cyclic task entry
// TR: Periyodik görev girişi — isim, periyot, son çalıştırma zamanı ve çalıştırma fonksiyonunu taşır
struct CyclicTask {
    std::string_view name;
    uint32_t period_ms;
    uint32_t last_run;
    std::function<void(uint32_t)> execute;
};

class BCMController {
    GPIOModule gpio_;
    CANModule can_;
    LINModule lin_;
    DTCStore dtcs_;
    NvMStore nvm_;
    UDSHandler uds_;
    Watchdog wdg_{100};  // 100 tick timeout
    BCM_State state_ = BCM_State::STARTUP;
    uint32_t tick_ = 0;
    std::vector<CyclicTask> tasks_;

public:
    BCMController() { SetupTasks(); SetupLIN(); }

    void SetupTasks() {
        // EN: 10ms task group
        // TR: 10ms görev grubu — CAN RX ve watchdog besleme (en yüksek öncelik)
        tasks_.push_back({"CAN_RX", 10, 0, [this](uint32_t t) { Task_CAN_RX(t); }});
        tasks_.push_back({"WDG_Feed", 10, 0, [this](uint32_t) { wdg_.Feed(); }});

        // EN: 50ms task group
        // TR: 50ms görev grubu — lamba ve silecek kontrolü (orta öncelik)
        tasks_.push_back({"LampControl", 50, 0, [this](uint32_t t) { Task_LampControl(t); }});

        // EN: 100ms task group
        // TR: 100ms görev grubu — LIN çizelgesi, DTC kontrolü ve NvM işlemleri
        tasks_.push_back({"LIN_Schedule", 100, 0, [this](uint32_t t) { Task_LIN(t); }});
        tasks_.push_back({"DTC_Check", 100, 0, [this](uint32_t t) { Task_DTCCheck(t); }});

        // EN: 1000ms task group
        // TR: 1000ms görev grubu — durum TX ve sağlık izleme (düşük öncelik)
        tasks_.push_back({"Status_TX", 1000, 0, [this](uint32_t t) { Task_StatusTX(t); }});
    }

    void SetupLIN() {
        lin_.AddScheduleEntry(0x20, 10);   // Rain sensor
        lin_.AddScheduleEntry(0x21, 10);   // Ambient light
        lin_.AddResponse({0x20, 2, {0x00, 0x50, 0,0,0,0,0,0}, "RainSensor"});
        lin_.AddResponse({0x21, 2, {0x03, 0xE8, 0,0,0,0,0,0}, "AmbientLight"});
    }

    // EN: Run scheduler for N ticks (each tick = 10ms)
    // TR: Zamanlayıcıyı N tick çalıştır (her tick = 10ms) — durum makinesini günceller, vadesi gelen görevleri yürütür
    void Run(uint32_t num_ticks) {
        for (uint32_t i = 0; i < num_ticks; ++i) {
            tick_ += 10;  // 10ms per tick
            wdg_.Tick();

```

```
            // EN: State machine
            // TR: Durum makinesini güncelle — STARTUP→NORMAL geçişi zamana bağlı
            UpdateState();

            if (state_ == BCM_State::SLEEP) continue;

            // EN: Execute due tasks
            // TR: Vadesi gelen görevleri çalıştır — geçen süre periyodu aştıysa tetiklenir
            for (auto& task : tasks_) {
                if ((tick_ - task.last_run) >= task.period_ms) {
                    task.execute(tick_);
                    task.last_run = tick_;
                }
            }
        }
    }

    // EN: Inject CAN message for processing
    // TR: İşlenmek üzere CAN mesajı enjekte et — test amaçlı RX kuyruğuna ekler
    void InjectCAN(const CAN_Message& msg) { can_.SimulateRx(msg); }

    // EN: Process UDS request
    // TR: UDS talebini işle — DTC, NvM ve GPIO modüllerine erişerek tanı hizmeti sunar
    UDSHandler::UDS_Response ProcessUDS(const UDSHandler::UDS_Request& req) {
        return uds_.Process(req, dtcs_, nvm_, gpio_);
    }

    void PrintStatus() const {
        std::cout << "    BCM State: " << BCMStateStr(state_)
                  << " | Tick: " << tick_ << "ms"
                  << " | WDG: " << wdg_.Remaining() << "\n";
        gpio_.DumpOutputs();
        dtcs_.Print();
        nvm_.Print();
    }

    void PrintCANTx() const { can_.PrintTxQueue(); }
    void ClearCANTx() { can_.ClearTx(); }

    // EN: Report DTC externally (for testing)
    // TR: Dışarıdan DTC bildir (test amaçlı) — arıza kodunu DTC deposuna kaydeder
    void ReportDTC(uint32_t code, std::string_view desc) {
        dtcs_.Report(code, desc, tick_);
    }

    // EN: Write NvM (for testing)
    // TR: NvM'ye yaz (test amaçlı) — seri numarası, kalibrasyon gibi kalıcı verileri saklar
    void WriteNvM(uint16_t id, uint32_t data) { nvm_.Write(id, data); }

private:
    void UpdateState() {
        switch (state_) {
            case BCM_State::STARTUP:
                // EN: Transition to NORMAL after initialization time
                // TR: Başlatma süresi sonrası NORMAL'e geç — tick >= 100 (1 saniye) olunca
                if (tick_ >= 100) {
                    state_ = BCM_State::NORMAL;
                    std::cout << "    [STATE] STARTUP → NORMAL (tick=" << tick_ << ")\n";
                }
                break;
            case BCM_State::NORMAL:
                // EN: Could transition to SLEEP on bus silence or explicit command
                // TR: Bus sessizliğinde veya açık komutla SLEEP'e geçebilir
                break;
            case BCM_State::SLEEP:
                break;
            case BCM_State::WAKEUP:
                state_ = BCM_State::NORMAL;
                break;
        }
    }

    void Task_CAN_RX([[maybe_unused]] uint32_t tick) {
        while (auto msg = can_.GetNextRx()) {
            ProcessCANMessage(*msg, tick);
        }
    }
```

```cpp

    void ProcessCANMessage(const CAN_Message& msg, uint32_t tick) {
        switch (msg.id) {
            case CAN_ID_LIGHT_CMD:
                if (msg.dlc >= 1) {
                    bool on = (msg.data[0] & 0x01) != 0;
                    auto s = on ? PinState::HIGH : PinState::LOW;
                    gpio_.Write(BCM_Pin::HEADLAMP_LEFT, s);
                    gpio_.Write(BCM_Pin::HEADLAMP_RIGHT, s);
                    gpio_.Write(BCM_Pin::TAIL_LAMP_LEFT, s);
                    gpio_.Write(BCM_Pin::TAIL_LAMP_RIGHT, s);
                    std::cout << "    [CAN] Light cmd: " << (on ? "ON" : "OFF")
                              << " (tick=" << tick << ")\n";
                }
                break;
            case CAN_ID_DOOR_CMD:
                if (msg.dlc >= 1) {
                    bool lock = (msg.data[0] & 0x01) != 0;
                    gpio_.Write(BCM_Pin::DOOR_LOCK_MOTOR,
                                lock ? PinState::HIGH : PinState::LOW);
                    std::cout << "    [CAN] Door " << (lock ? "LOCK" : "UNLOCK")
                              << " (tick=" << tick << ")\n";
                }
                break;
            case CAN_ID_WIPER_CMD:
                if (msg.dlc >= 1) {
                    bool on = (msg.data[0] & 0x01) != 0;
                    gpio_.Write(BCM_Pin::WIPER_MOTOR,
                                on ? PinState::HIGH : PinState::LOW);
                    std::cout << "    [CAN] Wiper " << (on ? "ON" : "OFF")
                              << " (tick=" << tick << ")\n";
                }
                break;
            case CAN_ID_BRAKE_STATUS:
                if (msg.dlc >= 1) {
                    bool brake = (msg.data[0] & 0x01) != 0;
                    gpio_.Write(BCM_Pin::BRAKE_LIGHT,
                                brake ? PinState::HIGH : PinState::LOW);
                }
                break;
            default:
                break;
        }
    }

    void Task_LampControl([[maybe_unused]] uint32_t tick) {
        // EN: Simple auto-light logic based on light sensor
        // TR: Işık sensörüne dayalı basit otomatik lamba mantığı (gerçek BCM'de ADC ile okunur)
        // (In real BCM, this reads ADC value from ambient light sensor)
    }

    void Task_LIN([[maybe_unused]] uint32_t tick) {
        auto frame = lin_.ExecuteSlot();
        if (frame) {
            // EN: Process LIN response silently in background
            // TR: LIN yanıtını arka planda sessizce işle — sensör verisi alınır
            (void)frame;
        }
    }

    void Task_DTCCheck(uint32_t tick) {
        // EN: Check for faults (simulated: headlamp both must be same state)
        // TR: Arıza kontrolü (simülasyon: her iki far aynı durumda olmalı, asimetri = arıza)
        auto left = gpio_.Read(BCM_Pin::HEADLAMP_LEFT);
        auto right = gpio_.Read(BCM_Pin::HEADLAMP_RIGHT);
        if (left != right) {
            dtcs_.Report(0xC10100, "Headlamp asymmetry", tick);
        }
    }

    void Task_StatusTX(uint32_t tick) {
        CAN_Message status{};
        status.id = CAN_ID_BCM_STATUS;
        status.dlc = 4;
        status.data[0] = static_cast<uint8_t>(state_);
        status.data[1] = static_cast<uint8_t>(dtcs_.Count());
        status.data[2] = static_cast<uint8_t>((tick >> 8) & 0xFF);
```

```
        status.data[3] = static_cast<uint8_t>(tick & 0xFF);
        can_.Transmit(status);
    }
};

}  // namespace bcm

// ═══════════════════════════════════════════════════════════════════════
// MAIN — BCM Integration Demos
// ═══════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "╔═══════════════════════════════════════════════════════╗\n"
              << "║ Module 18: Mini ECU Project — BCM (Body Control Module)  ║\n"
              << "╚═══════════════════════════════════════════════════════╝\n\n";

    bcm::BCMController ecu;

    // --- Demo 1: Startup + State Transition ---
    std::cout << "--- Demo 1: BCM Startup Sequence ---\n";
    {
        ecu.WriteNvM(0x0001, 0x12345678);  // Serial number
        ecu.Run(12);  // 120ms → should transition STARTUP → NORMAL
        ecu.PrintStatus();
    }

    // --- Demo 2: CAN Message Routing ---
    std::cout << "\n--- Demo 2: CAN Message Routing ---\n";
    {
        // EN: Turn on headlamps via CAN
        // TR: Farları CAN üzerinden aç — 0x120 ID'li mesajın data[0]=0x01 gönderilir
        bcm::CAN_Message light_on{bcm::CAN_ID_LIGHT_CMD, 1, {0x01}};
        ecu.InjectCAN(light_on);
        ecu.Run(1);  // Process CAN

        // EN: Lock doors via CAN
        // TR: Kapıları CAN üzerinden kilitle — 0x130 ID'li mesaj ile kilit motoru etkinleşir
        bcm::CAN_Message door_lock{bcm::CAN_ID_DOOR_CMD, 1, {0x01}};
        ecu.InjectCAN(door_lock);
        ecu.Run(1);

        // EN: Enable wiper
        // TR: Sileceği etkinleştir — 0x140 ID'li CAN mesajı ile silecek motoru çalışır
        bcm::CAN_Message wiper_on{bcm::CAN_ID_WIPER_CMD, 1, {0x01}};
        ecu.InjectCAN(wiper_on);
        ecu.Run(1);

        ecu.PrintStatus();
    }

    // --- Demo 3: DTC Storage ---
    std::cout << "\n--- Demo 3: DTC Reporting + Confirmation ---\n";
    {
        //  EN: Report same DTC 3 times → should become CONFIRMED
        //  TR: Aynı DTC'yi 3 kez bildir → CONFIRMED durumuna geçmeli (otomotiv standardı)
        ecu.ReportDTC(0xB10000, "Interior light short circuit");
        ecu.ReportDTC(0xB10000, "Interior light short circuit");
        ecu.ReportDTC(0xB10000, "Interior light short circuit");
        ecu.ReportDTC(0xC20100, "LIN communication timeout");
        ecu.PrintStatus();
    }

    // --- Demo 4: UDS Diagnostics ---
    std::cout << "\n--- Demo 4: UDS Diagnostic Services ---\n";
    {
        // EN: ReadDataByIdentifier — SW Version
        // TR: ReadDataByIdentifier — Yazılım Sürümü (DID=0xF189 ile okunur)
        auto resp1 = ecu.ProcessUDS({bcm::UDSHandler::SID_READ_DID, bcm::UDSHandler::DID_SW_VERSION, {}});
        std::cout << "    ReadDID(SWVer): positive=" << resp1.positive << " data='";
        for (auto b : resp1.data) std::cout << static_cast<char>(b);
        std::cout << "'\n";

```

```cpp
        // EN: ReadDID — Lamp Status
        // TR: ReadDID — Lamba Durumu (GPIO pinlerinin bit maskesi olarak döndürür)
        auto resp2 = ecu.ProcessUDS({bcm::UDSHandler::SID_READ_DID, bcm::UDSHandler::DID_LAMP_STATUS, {}});
        std::cout << "    ReadDID(LampStatus): 0x" << std::hex
                  << static_cast<int>(resp2.data.empty() ? 0 : resp2.data[0])
                  << std::dec << "\n";

        // EN: ReadDTCInformation
        // TR: DTC bilgisi oku — tüm kayıtlı arıza kodları ve durumları listelenir
        auto resp3 = ecu.ProcessUDS({bcm::UDSHandler::SID_READ_DTC_INFO, 0, {}});
        std::cout << "    ReadDTCInfo: " << resp3.data.size() / 4 << " DTCs reported\n";

        // EN: WriteDID — Serial Number
        // TR: WriteDID — Seri Numarası (NvM'ye 4 byte veri yazılır)
        auto resp4 = ecu.ProcessUDS({bcm::UDSHandler::SID_WRITE_DID, bcm::UDSHandler::DID_SERIAL_NUMBER, {0xAB, 0xCD, 0xEF, 0x01}});
        std::cout << "    WriteDID(SerialNum): positive=" << resp4.positive << "\n";

        // EN: ReadDID — Serial Number (verify write)
        // TR: ReadDID — Seri Numarası (yazılan değerin doğrulaması)
        auto resp5 = ecu.ProcessUDS({bcm::UDSHandler::SID_READ_DID, bcm::UDSHandler::DID_SERIAL_NUMBER, {}});
        std::cout << "    ReadDID(SerialNum): ";
        for (auto b : resp5.data) std::cout << std::hex << std::setw(2) << std::setfill('0') << static_cast<int>(b);
        std::cout << std::dec << std::setfill(' ') << "\n";

        // EN: RoutineControl
        // TR: Rutin Kontrol — test veya kalibrasyon rutini başlatır (SID=0x31)
        auto resp6 = ecu.ProcessUDS({bcm::UDSHandler::SID_ROUTINE_CTRL, 0xFF00, {}});
        std::cout << "    RoutineCtrl(0xFF00): positive=" << resp6.positive << "\n";

        // EN: ClearDTC
        // TR: DTC Temizle — tüm kayıtlı arıza kodlarını sıfırlar (SID=0x14)
        auto resp7 = ecu.ProcessUDS({bcm::UDSHandler::SID_CLEAR_DTC, 0, {}});
        std::cout << "    ClearDTC: positive=" << resp7.positive << "\n";
    }

    // --- Demo 5: Cyclic Scheduling ---
    std::cout << "\n--- Demo 5: Cyclic Task Execution (100 ticks) ---\n";
    {
        ecu.ClearCANTx();
        ecu.Run(100);  // 1000ms
        std::cout << "    BCM status TX messages sent:\n";
        ecu.PrintCANTx();
    }

    // --- Demo 6: Turn Off All + Final Status ---
    std::cout << "\n--- Demo 6: Turn Off All Outputs + Final Status ---\n";
    {
        bcm::CAN_Message light_off{bcm::CAN_ID_LIGHT_CMD, 1, {0x00}};
        bcm::CAN_Message door_unlock{bcm::CAN_ID_DOOR_CMD, 1, {0x00}};
        bcm::CAN_Message wiper_off{bcm::CAN_ID_WIPER_CMD, 1, {0x00}};
        ecu.InjectCAN(light_off);
        ecu.InjectCAN(door_unlock);
        ecu.InjectCAN(wiper_off);
        ecu.Run(1);
        ecu.PrintStatus();
    }

    // --- Demo 7: Architecture Summary ---
    std::cout << "\n--- Demo 7: BCM Architecture Summary ---\n";
    {
        struct Module { std::string_view name; std::string_view description; };
        Module modules[] = {
            {"GPIO",         "Digital I/O: lamps, motors, switches"},
            {"CAN",          "Message RX/TX: light, door, wiper, brake, status"},
            {"LIN",          "Schedule table: rain sensor, ambient light"},
            {"DTC Store",    "Fault logging: active→confirmed, occurrence count "},
            {"NvM",          "Non-volatile: serial number, learn values"},
            {"UDS Handler",  "Diagnostics: ReadDID, WriteDID, ClearDTC, Routine"},
            {"Watchdog",     "Supervision: 100-tick timeout, feed in 10ms task"},
            {"State Machine","STARTUP→NORMAL→SLEEP→WAKEUP lifecycle             "},
            {"Scheduler",    "Cyclic tasks: 10ms/50ms/100ms/1000ms groups"},
        };
        std::cout << "  ┌──────────────┬──────────────────────────────────────────────────────────┐\n";
```

```
809         std::cout << "  │ Module         │ Description                                                │\n";
810         std::cout << "  ├────────────────┼────────────────────────────────────────────────────────────┤\n";
811         for (const auto& m : modules) {
812             std::cout << "  │ " << std::setw(14) << std::left << m.name
813                       << " │ " << std::setw(50) << m.description << " │\n";
814         }
815         std::cout << std::right;
816         std::cout << "  └────────────────┴────────────────────────────────────────────────────────────┘\n";
817     }
818 
819     std::cout << "\n[All demos completed successfully.]\n";
820     return 0;
821 }
```

Ek: Derleme ve Referans Kılavuzu

A. Proje Özeti / Project Overview

Öğe / Item	Detay / Detail
Modüller	18 (Temeller → Sürücü Geliştirme)
Kaynak dosyalar	114 .cpp + 2 .txt + 2 .md + main.cpp
Standartlar	C++17, C++20, C++23
Derleyici	g++ 13+ / clang++ 17+ önerilir
Platform	Linux (birincil), macOS, WSL2

Modül Dağılımı / Module Breakdown

#	Modül / Module	Dosya	Standart
1	Basics — Temeller ve Hafıza	10	C++17
2	OOP — Nesne Yönelimli Programlama	8	C++17
3	Modern C++ — Güvenlik ve Performans	6	C++17/23
4	Templates & STL — Şablonlar ve STL	9	C++17
5	Certification Prep — Sertifikasyon Hazırlığı	7	C++17
6	Industry Standards — Endüstri Standartları	10	C++17
7	Under the Hood — OS Derinlikleri	11	C++17
8	C++ Core Guidelines — Üst Düzey Kurallar	8	C++17
9	Advanced Mechanics — İleri Mekanikler	8	C++20
10	Design Patterns — Tasarım Kalıpları	7	C++20
11	Embedded & Real-Time — Gömülü Sistemler	5	C++20
12	Coroutines — Eşyordamlar	2	C++20
13	Networking — Ağ Programlama	2	C++20
14	Automotive Networks — Otomotiv Ağları	6	C++20
15	HW Interfaces — Donanım Arayüzleri	5	C++20
16	AUTOSAR — Otomotiv Yazılım Mimarisi	5	C++20
17	SDV — Yazılım Tanımlı Araç	3	C++20
18	Driver Dev — Sürücü Geliştirme	4	C++20

B. Derleme Talimatları / Compilation Instructions

Tüm kaynak dosyalar sıkı uyarı flag'leriyle hatasız derlenir. Dosya kategorisine göre aşağıdaki komutları kullanın.

```
# ── Standart derleme (C++20 — çoğu dosya) ──────────────────
g++ -std=c++20 -Wall -Wextra -Wpedantic -Wshadow \
    -Wconversion -Wsign-conversion -Wnull-dereference \
    -Wformat=2 <dosya_adi>.cpp -o output

# ── C++23 gerektiren dosyalar ──────────────────────────────
# module_03/06_cpp23_features.cpp
# module_04/03_stl_containers_deepdive.cpp  (std::flat_map)
# module_05/07_error_handling_advanced.cpp   (std::expected)
# module_07/06_abi_and_name_mangling.cpp
# module_18/03_peripheral_driver.cpp
g++ -std=c++23 -Wall -Wextra -Wpedantic -Wshadow \
    -Wconversion -Wsign-conversion -Wnull-dereference \
    -Wformat=2 <dosya_adi>.cpp -o output

# ── Eşzamanlılık / threading (-pthread gerekli) ────────────
# module_03/02  module_05/05  module_06/01  module_06/05
# module_06/09  module_06/10  module_07/03  module_07/04
# module_08/04  module_10/07  module_11/01  module_11/04
# module_11/05  module_12/02  module_13/01  module_15/01
g++ -std=c++20 ... -pthread <dosya_adi>.cpp -o output

# ── Coroutine'ler (C++20 coroutine desteği) ────────────────
# module_12/01_coroutine_fundamentals.cpp
# module_12/02_async_coroutine_patterns.cpp
g++ -std=c++20 ... -fcoroutines <dosya_adi>.cpp -o output
# Not: GCC < 13 için -fcoroutines gerekir; GCC 13+
# -std=c++20 ile otomatik aktiftir.

# ── POSIX / Sadece Linux (fork, soket, epoll) ──────────────
# module_07/03_fork_vs_thread_pool.cpp     (fork/waitpid)
# module_13/01_socket_fundamentals.cpp     (POSIX soketler)
# module_13/02_nonblocking_io_epoll.cpp    (epoll)
# module_14/01_can_fd_and_xl.cpp           (SocketCAN)
g++ -std=c++20 ... -pthread <dosya_adi>.cpp -o output
# Bu dosyalar POSIX API kullanır, sadece Linux/macOS'ta derlenir.

# ── Filesystem (eski derleyicilerde -lstdc++fs) ────────────
# module_01/10_filesystem_and_binary_io.cpp
# module_06/10_advanced_logging.cpp
g++ -std=c++20 ... <dosya_adi>.cpp -o output -lstdc++fs
# Not: GCC 9+ <filesystem>'i otomatik bağlar.

# ── Google Test (libgtest-dev gerekli) ─────────────────────
# module_06/03_unit_testing_gtest.cpp
# module_06/11_gtest_advanced.cpp
g++ -std=c++20 ... <dosya_adi>.cpp -o output \
    -lgtest -lgtest_main -pthread
```

Not: Tüm 119 kaynak dosya yukarıdaki sıkı flag'lerle GCC 13 / Clang 17 üzerinde **0 uyarı, 0 hata** ile derlenir.

C. Gerekli Araçlar / Required Tools

Araç / Tool	Amaç / Purpose	Kurulum / Install (Ubuntu)
g++ 13+	C++20/23 derleyici	apt install g++-13
CMake 3.20+	Derleme sistemi / Build system	apt install cmake
GDB	Hata ayıklayıcı / Debugger	apt install gdb
Valgrind	Bellek denetleyici / Memory checker	apt install valgrind
Google Test	Birim test / Unit testing	apt install libgtest-dev
AddressSanitizer	Bellek hatası dedektörü	GCC/Clang (-fsanitize=address)
ThreadSanitizer	Veri yarışı dedektörü	GCC/Clang (-fsanitize=thread)
perf	Performans profiler (Linux)	apt install linux-tools-common

Sanitizer ve Profiling Komutları

```
# AddressSanitizer (ASan) — use-after-free, buffer overflow tespiti:
g++ -std=c++20 -fsanitize=address -fno-omit-frame-pointer \
    -g <dosya_adi>.cpp -o output

# ThreadSanitizer (TSan) — veri yarışı tespiti:
g++ -std=c++20 -fsanitize=thread -g <dosya_adi>.cpp -o output

# UndefinedBehaviorSanitizer (UBSan):
g++ -std=c++20 -fsanitize=undefined -g <dosya_adi>.cpp -o output

# Valgrind bellek kontrolü:
valgrind --leak-check=full --show-leak-kinds=all ./output
```

D. CMake Hızlı Başlangıç / Quick Start

Çoklu dosya derlemesi için proje kök dizininde bir CMakeLists.txt oluşturun:

```
cmake_minimum_required(VERSION 3.20)
project(CppMastery LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

# Sıkı uyarılar / Strict warnings
add_compile_options(-Wall -Wextra -Wpedantic -Wshadow
                    -Wconversion -Wsign-conversion
                    -Wnull-dereference -Wformat=2)

# Örnek: tek modül dosyası derle
add_executable(variables
    module_01_basics/01_variables_and_memory.cpp)

# Threading örneği
add_executable(concurrency
    module_06_industry_standards/01_concurrency_threads.cpp)
target_link_libraries(concurrency PRIVATE pthread)

# Google Test örneği
find_package(GTest REQUIRED)
add_executable(test_gtest
    module_06_industry_standards/03_unit_testing_gtest.cpp)
target_link_libraries(test_gtest PRIVATE GTest::GTest
                                         GTest::Main pthread)
```

E. Referanslar ve İleri Okuma / References

Dil ve Standart Kütüphane / Language & Standard Library

- cppreference.com
- ISO C++ (isocpp.org)
- C++ Core Guidelines
- C++20 Standard Draft (N4861)
- C++23 Standard Draft (N4950)

Araçlar ve Framework'ler / Tools & Frameworks

- Google Test / Google Mock
- CMake Dokümantasyonu
- Valgrind Kılavuzu
- GDB Dokümantasyonu
- Clang Sanitizer'lar

Otomotiv ve Gömülü / Automotive & Embedded

- AUTOSAR Classic Platform
- AUTOSAR Adaptive Platform
- MISRA C++ 2023
- ISO 26262 (Fonksiyonel Güvenlik)
- SOME/IP Protokol Spesifikasyonu

Önerilen Kitaplar / Recommended Books

- Effective Modern C++ — Scott Meyers
- C++ Concurrency in Action — Anthony Williams
- The C++ Programming Language — Bjarne Stroustrup
- Design Patterns — Gang of Four
- Programming Embedded Systems in C and C++ — Michael Barr